Graph Theory in Lean

Richard M. Hill and John Talbot

First edition · Release v4.36.0-rc2 · November 2026

Lean v4.36.0-rc2 · Mathlib 3d48751 (2026-11-13) · Verso 71a2035 (2026-10-27)

Contents

Contents i

1 Introduction 1
1.1 Installation . . . 2
1.2 How to Read This Guide . . . 2
1.3 Finding Lemmas . . . 3
1.4 How This Guide is Structured . . . 3
1.5 Acknowledgements . . . 6
1.6 Use of AI . . . 6

2 SimpleGraph 7
2.1 SimpleGraph . . . 7
2.2 Edges . . . 8
2.3 Vertices . . . 8
2.4 Darts . . . 10
2.5 Naming Conventions . . . 11
2.6 Order . . . 11
2.7 Constructing a SimpleGraph . . . 14
2.8 Computations . . . 20
2.9 Exercises . . . 20

3 Constructions 23
3.1 Basic Families of Graphs . . . 23
3.2 Algebraic Constructions . . . 28
3.3 Operations on Graphs . . . 31
3.4 Hypercube . . . 35
3.5 Mycielskian . . . 38
3.6 Complete Multipartite Graphs and Blow-ups . . . 42
3.7 Kneser Graphs* . . . 46
3.8 Exercises . . . 47

4 Decidability, Finiteness and Degrees 49
4.1 Decidability . . . 49
4.2 Finite Types . . . 60
4.3 Degrees . . . 69
4.4 Computations with Hypercubes . . . 78
4.5 Computations with Mycielskians . . . 83
4.6 Degree Sequences* . . . 86

4.7 Exercises . . . 93

5 Maps 98
5.1 Functions, Embeddings and Equivalences . . . 98
5.2 Comaps . . . 99
5.3 Morphisms . . . 101
5.4 Maps . . . 107
5.5 Hypercubes and Mycielskians . . . 109
5.6 Automorphisms . . . 111
5.7 Tensor Products* . . . 115
5.8 Claw-Free Graphs* . . . 119
5.9 Exercises . . . 123

6 Cliques 127
6.1 Cliques . . . 127
6.2 Finite Cliques . . . 128
6.3 Clique-Free . . . 130
6.4 The Finset of n-Cliques . . . 131
6.5 Clique Number . . . 132
6.6 Independent Sets . . . 135
6.7 Clique and Independence Number of the Mycielskian . . . 141
6.8 Exercises . . . 150

7 Colorings 156
7.1 Colourings . . . 156
7.2 Chromatic Number . . . 157
7.3 Colourings, Cliques and Independent Sets . . . 158
7.4 Bipartite Graphs . . . 161
7.5 Colouring Concrete Graphs . . . 163
7.6 Colouring Mycielskians . . . 166
7.7 Edge Colourings . . . 169
7.8 Greedy Colouring* . . . 170
7.9 Exercises . . . 184

8 Walks 188
8.1 Constructing Walks . . . 188
8.2 Vertices and Edges of a Walk . . . 191
8.3 New Walks from Old . . . 193
8.4 Trails, Paths, Circuits and Cycles . . . 194
8.5 Fixing Type Mismatches . . . 203
8.6 Mapping Walks . . . 209
8.7 Walks in the Hypercube . . . 211
8.8 Walks in the Mycielskian . . . 216
8.9 Hamiltonian Graphs: Ore, Dirac and Bondy–Chvátal* . . . 222
8.10 Exercises . . . 239

9 Subgraphs 246
9.1 Subgraphs . . . 246

9.2 Constructing Subgraphs . . . 249
9.3 Subgraphs as Simple Graphs . . . 252
9.4 Maps and Comaps . . . 258
9.5 Matching Number . . . 259
9.6 Partial Colourings* . . . 268
9.7 Exercises . . . 272

10 Connectivity 278
10.1 Connectivity . . . 278
10.2 Trees . . . 285
10.3 Eulerian Circuits and Trails . . . 288
10.4 Edge and Vertex Connectivity . . . 296
10.5 Brooks' Theorem* . . . 304
10.6 Cut Vertices and Blocks* . . . 317
10.7 Exercises . . . 326

11 Distance 332
11.1 Graph Distance . . . 332
11.2 Eccentricity, Diameter and Radius . . . 336
11.3 Albertson's Precolouring Extension Theorem . . . 339
11.4 Girth . . . 341
11.5 Moore and Cage Bounds* . . . 346
11.6 Exercises . . . 357

12 Algebraic Graph Theory 360
12.1 Automorphism Groups . . . 360
12.2 The Adjacency Matrix . . . 365
12.3 The Laplacian . . . 372
12.4 Strongly Regular Graphs . . . 373
12.5 The Hypercube and the Hamming Cube . . . 377
12.6 The Automorphism Group of the Hypercube* . . . 379
12.7 Counting Copies* . . . 386
12.8 Subcubes of the Hypercube* . . . 392
12.9 Exercises . . . 400

13 Theorems 405
13.1 Matchings . . . 406
13.2 Turán's Theorem, Stability and Asymptotics . . . 413
13.3 Zykov's Theorem* . . . 426
13.4 Regularity and Removal* . . . 438
13.5 Infinite Graphs* . . . 443
13.6 Exercises . . . 447

Solutions 452
Chapter 2: SimpleGraph . . . 452
Chapter 3: Constructions . . . 454
Chapter 4: Decidability, Finiteness and Degrees . . . 456
Chapter 5: Maps . . . 462

Chapter 6: Cliques 467
Chapter 7: Colorings 477
Chapter 8: Walks 482
Chapter 9: Subgraphs 489
Chapter 10: Connectivity 498
Chapter 11: Distance 508
Chapter 12: Algebraic Graph Theory 511
Chapter 13: Theorems 518

Appendix 527
Notation 527
Definitions 528
Results 539

Bibliography 554

1 Introduction

Our goal in writing this guide is to introduce graph theorists to the interactive proof assistant Lean[1] and its associated library Mathlib[2].

We don't aim to give a comprehensive introduction to Lean for mathematicians: there are many excellent tutorials and texts for this, including:

- Mathematics in Lean[3] (the definitive introduction to Lean for mathematicians)
- Theorem Proving in Lean 4[4] (introduction to interactive theorem proving with dependent types using Lean)
- A Glimpse of Lean[5] ("an introduction to theorem proving in Lean for the impatient")
- The Hitchhiker's Guide to Logical Verification[6] (companion text to a graduate-level course on interactive theorem proving)
- How To Prove It With Lean[7] (an introduction to proofs in Lean based on the classic textbook)
- Logic and Proof[8] (a course in logic and proof using Lean)
- Lean Game Portal[9] (interactive games such as the Natural Number Game)
- Visual Lean[10] (an experimental graphical user interface for writing Lean code where you can play the Natural Number Game)

A first course in graph theory is assumed, as is a basic level of Lean proficiency (for example chapters 1 to 8 of Mathematics in Lean[11]).

Our aim is to give graph theorists an idea of how formalizing their work might look in Lean, making use of Mathlib. In particular we will try to point out the differences that we encounter in Lean, compared to informal definitions and proofs.

[1] https://lean-lang.org/
[2] https://leanprover-community.github.io/
[3] https://leanprover-community.github.io/mathematics_in_lean/
[4] https://lean-lang.org/theorem_proving_in_lean4/
[5] https://github.com/PatrickMassot/GlimpseOfLean/
[6] https://github.com/lean-forward/logical_verification_2025
[7] https://djvelleman.github.io/HTPIwL/
[8] https://leanprover-community.github.io/logic_and_proof/
[9] https://adam.math.hhu.de/
[10] https://leangame.io
[11] https://leanprover-community.github.io/mathematics_in_lean/

1.1 Installation

If you haven't done so already please install Lean[12] and clone a copy of the repository https://github.com/jt496/GraphTheoryInLean[13].

1.2 How to Read This Guide

If you are reading the `.lean` files directly in VS Code or similar then see the `README.md` file for build instructions (`lake exe cache get` followed by `lake build`). Each chapter is in a separate folder that also contains any associated exercises. In VS Code the InfoView panel shows the output of each `#command` and the proof state at the cursor. Hovering over a name shows its type and any doc-string, and Ctrl+click (Cmd+click on a Mac) or F12 takes you to its definition, in Mathlib's source or in the guide.

If you are reading the web version then please note that this is published using Verso[14]. You should be able to see most of the interactive features of Lean such as InfoView messages and `#command` outputs.

To see the actual output of a `#command` you need to hover over the `#command` itself with your cursor (the `#command` will have wavy underlining). Hovering over a name or term shows its type, and also its doc-string if it has one. Clicking a name takes you to its definition: a name from Mathlib opens its page in the Mathlib documentation, while a name defined in this guide takes you to where we define it. In tactic proofs (i.e. proofs that start with `:= by`) hover over the small lozenge after a tactic to see the proof state at that point, and click it to keep the proof state open (click again to close it).

For example, below you should be able to hover over the `#source` command to see the verbatim source code of the definition of `SimpleGraph` in Mathlib. Hovering over `SimpleGraph` itself shows its type and doc-string; clicking on it takes you to its page in the Mathlib documentation. Hovering over the `#check` tells you the type of `G.Adj u v` (it is a `Prop`), while hovering over `u` will confirm that it has type `V`, which in turn you can hover over to see its type `Type u_1`.

```
variable {V : Type*} {G : SimpleGraph V} (u v : V)
#source SimpleGraph
#check G.Adj u v -- Prop
#check sum_degrees_eq_twice_card_edges -- ∑ v, G.degree v = 2 * G.edgeFinset.card
```

Throughout this guide we work in the `namespace SimpleGraph`, so for example `Adj` means `SimpleGraph.Adj`, and our definitions get names like `SimpleGraph.pathNat`. When we want to put results or definitions in other namespaces we will do this explicitly.

In the interest of making this guide as readable as possible by graph theorists we introduce quite a lot of "standard" graph theory notation that is not in Mathlib (for example `K[n]`, `C[n]`, `K[s, t]`, `Q[n]`, `ω G`, `χ G`, etc.). For reference, see Notation (p. 527). We also provide lists of the Definitions (p. 528) introduced in the guide and of the main Results (p. 539) proved in it.

[12] https://lean-lang.org/install/
[13] https://github.com/jt496/GraphTheoryInLean
[14] https://verso.lean-lang.org/

1.3 Finding Lemmas

Much of the work of formalizing is finding the right lemma in Mathlib. Mathlib's naming conventions (see Naming Conventions (p. 11)) help, since a name should describe the statement (for example `card_neighborFinset_eq_degree`), but they only get you so far: the degree of a vertex in a complete graph is `complete_graph_degree`, not `degree_top`. So it is worth knowing how to search.

- The tactic `exact?` searches Mathlib for a lemma that closes the current goal, and `apply?` and `rw?` do the same for `apply` and `rw`.
- Loogle[15] searches by constants and expressions, for example `SimpleGraph.degree` and `SimpleGraph.completeGraph`.
- LeanSearch[16] searches using natural language, for example "degree of a vertex in the complete graph".
- Mathlib's documentation pages, and its source files (reached in VS Code with Ctrl+click or F12), list the declarations of each file in order, so the lemmas about a definition can usually be found near it.

1.4 How This Guide is Structured

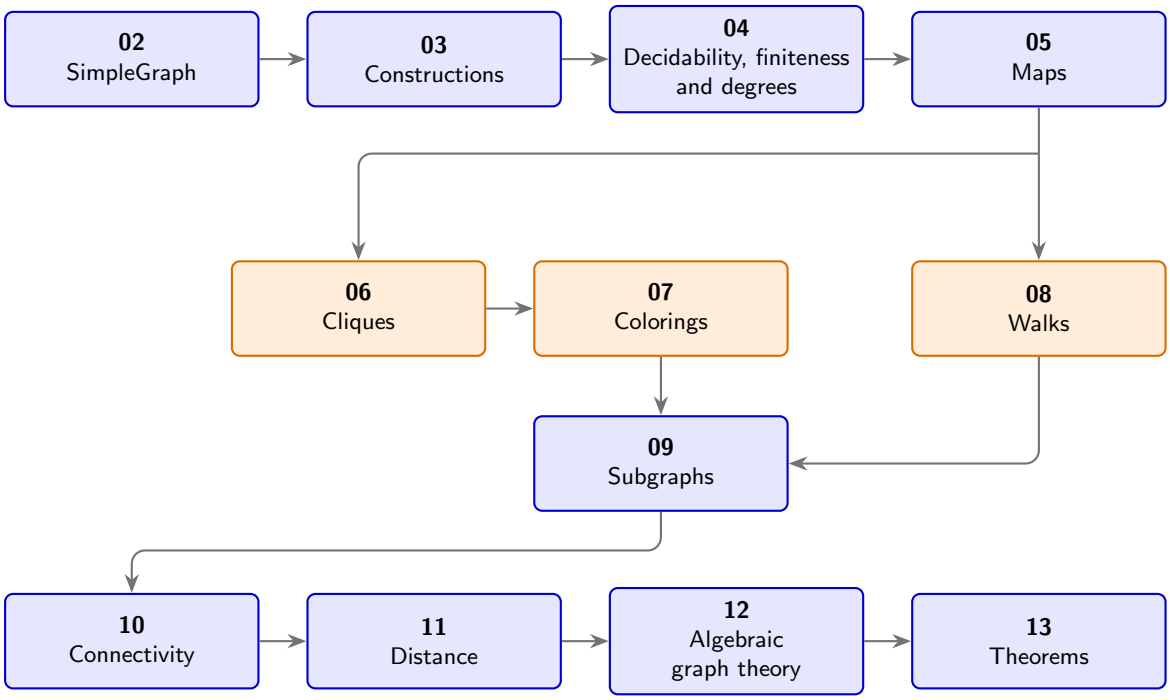
This guide is focused entirely on `SimpleGraph`. We start with the basic definitions in the next chapter before moving on to give many different concrete examples in Constructions (p. 23). The following chapter, Decidability, Finiteness and Degrees (p. 49), shows how to compute with graphs and introduces vertex degrees.

The rest of the guide introduces topics mainly according to dependencies in Mathlib: Maps (p. 98), Cliques (p. 127), Colorings (p. 156), Walks (p. 188), Subgraphs (p. 246), Connectivity (p. 278), Distance (p. 332), Algebraic Graph Theory (p. 360), before looking at some of the classical Theorems (p. 405) about graphs in Mathlib.

After Maps (p. 98) the guide splits into two strands that can be read in either order: Cliques (p. 127) and Colorings (p. 156) on the one hand, and Walks (p. 188) on the other. The strands meet in Subgraphs (p. 246), and from there each chapter builds on the previous ones.

Most chapters begin with a few basic sections before moving on to more involved material. Sections or subsections whose titles are marked with a star are more advanced. Later sections sometimes use their results, and recall what they need when they do.

[15] `https://loogle.lean-lang.org/`
[16] `https://leansearch.net/`



A route through the guide

You certainly don't have to read the guide in order. If you prefer to start from something familiar, pick a theorem you already know in Theorems (p. 405), see how it is stated in Lean, and work backwards to the definitions it uses.

We use three running examples throughout:

- `pathNat`: the infinite path on $\mathbb{N}$ (p. 14)
- `Q[n]`: the n-dimensional hypercube (p. 35)
- `mμ(G)`: the Mycielskian (p. 38)

Examples of more advanced topics that we cover in the guide include:

- Graphic Sequences (p. 86)
- Independence Number of the Kneser Graph (p. 137)
- Colouring Mycielskians (p. 166)
- Greedy Colouring (p. 170)
- Pósa Rotations (p. 223)
- Bondy–Chvátal Theorem (p. 229)
- De Bruijn–Erdős Theorem (p. 256)
- Vertex Connectivity (p. 297)

- Brooks' Theorem (p. 304)
- Cut Vertices and Blocks (p. 317)
- Moore and Cage Bounds (p. 346)
- Albertson's Precolouring Extension Theorem (p. 339)
- The Laplacian (p. 372)
- Strongly Regular Graphs (p. 373)
- Counting Copies (p. 386)
- The Automorphism Group of the Hypercube (p. 379)
- Kőnig's Theorem (p. 409)
- Füredi's Stability Theorem (p. 419)
- Zykov's Theorem (p. 426)
- Ruzsa–Szemerédi Theorem (p. 440)
- Ends (p. 444)

Theorems from Mathlib that we survey, mostly in the final chapter Theorems (p. 405), include:

- Hall's Marriage Theorem (p. 406)
- Tutte's Theorem (p. 413)
- Turán's Theorem (p. 415) and Mantel's Theorem (p. 415)
- Andrásfai–Erdős–Sós Theorem (p. 423)
- Erdős–Stone Theorem (p. 425)
- Szemerédi Regularity Lemma (p. 439)
- Triangle Removal Lemma (p. 439) and Roth's Theorem (p. 440)
- Kőnig's Lemma (p. 443)

Each chapter (except this one) has some exercises with worked solutions. Each set of exercises is given first with `sorry`, and the solutions follow.

Since we want to develop results for our running examples some of these exercises are named lemmas or theorems rather than anonymous examples. To avoid naming clashes we add a prime (`foo'`) to the versions with `sorry`.

1.5 Acknowledgements

This guide could not exist without the work of all the contributors to Lean and Mathlib.

We gratefully acknowledge support from the AI for Math Fund, a program of Renaissance Philanthropy and founding donor XTX Markets.

Access to AI tools was provided through Anthropic's Claude for Scientists Team Plan and OpenAI's ChatGPT for Academic Researchers.

1.6 Use of AI

We used AI tools (Claude and ChatGPT) in the later stages of this project for:

- Lean code: drafting some definitions and proofs, which we then reviewed and revised.
- Writing: copy-editing, editorial review (suggesting cuts, reorganization and naming), preparing the bibliography.
- Figures: creating most of the TikZ diagrams.
- Tooling: the Lean and Verso customization, together with Python scripts to simplify our conversion process.

The choice of topics and the mathematical exposition are our own.

All the proofs in this guide are checked by Lean, but it can't check that our definitions and statements mean what we claim they mean, or that the prose is correct. We have reviewed everything and take full responsibility for all of it, especially the errors.

2 SimpleGraph

In this chapter we introduce the basics of `SimpleGraph`, Mathlib's definition of a simple, loopless, undirected graph. We will explore how adjacency, edges, darts, and neighbourhoods are defined. We will also look at the process of constructing a graph (the infinite path on $\mathbb{N}$) using a few different methods. Finally we will look at how to perform basic computations on graphs.

2.1 SimpleGraph

Mathlib already contains a few different definitions of graphs (for example directed graphs as `Digraph` and multigraphs as `Graph`), but we will only look at `SimpleGraph`. [1]

The definition of `SimpleGraph` in Mathlib says that, for a fixed vertex type `V`, a term of `SimpleGraph V` is a structure consisting of a binary relation `Adj` on `V` that tells us which pairs of vertices are adjacent, and two proofs `symm` and `loopless` that this relation is symmetric and irreflexive. [2] Note that `V` is a parameter of the structure rather than one of its fields: graphs on different vertex types have different types.

```
section basics
variable {V : Type*} (G : SimpleGraph V)

#source SimpleGraph -- structure SimpleGraph (V : Type u) where
                    --  Adj : V → V → Prop
                    --  symm : Std.Symm Adj := by aesop_graph
                    --  loopless : Std.Irrefl Adj := by aesop_graph

#check G.Adj        -- a relation on V describing adjacency in G
#check G.symm       -- adjacency is symmetric (undirected edges)
#check G.loopless   -- adjacency is irreflexive (no loops)
```

Both `Std.Symm Adj` and `Std.Irrefl Adj` are structures with a single field, namely `symm : ∀ a b, Adj a b → Adj b a` and `irrefl : ∀ a, ¬Adj a a`. So to give these proofs by hand we write `symm := ⟨fun a b h => …⟩` (or give a tactic proof with `symm := by constructor; …`).

We will meet the formal definition of a graph's edges in a moment, but usually we reason directly with the adjacency relation `G.Adj`. Informally we write $u - v$ for an edge of G joining u and v; in Lean we instead work with a proof h of the proposition `G.Adj u v : Prop`. (Strictly speaking, `h : G.Adj u v` records the *directed* edge $u \to v$, but by symmetry it also gives `G.Adj v u`, namely `h.symm`, so we can treat it as the undirected edge $u - v$.)

[1] Although `SimpleGraph` is currently the most commonly used graph structure in Mathlib, it has some real limitations: for example any non-spanning subgraph of a `SimpleGraph V` is **not** a `SimpleGraph V`. However, it has a well-developed API and many results can be formalized with relative ease.

[2] We can omit proofs of `symm` and `loopless` and Lean will try to fill them in using the automated `aesop_graph` tactic.

```
example {u v : V} (h : G.Adj u v) : G.Adj v u := h.symm
```

The fact that `G` does not have a loop at a vertex `u` is the proposition `¬G.Adj u u`.

```
example {u : V} : ¬G.Adj u u := G.irrefl
```

2.2 Edges

Since `SimpleGraph` is a loopless undirected graph, its edges are *unordered* pairs of distinct vertices. In Lean unordered pairs have type `Sym2 V`, the symmetric square of `V`. Formally this is the quotient of `V × V` modulo swaps (`Sym2.Rel`). Elements (or more precisely terms) of `Sym2 V` are denoted `s(u, v)`. Note that although `s(v, v)` cannot be an edge of a `SimpleGraph` (which is loopless), the type `Sym2 V` still contains such pairs.

```
example {u v : V} : s(u, v) = s(v, u) := Sym2.eq_swap
```

The *edge set* of `G` is `G.edgeSet` and is a `Set (Sym2 V)`. Membership is definitionally equivalent to adjacency. (This means that the proof is `rfl`.)

```
example {u v : V} : s(u, v) ∈ G.edgeSet ↔ G.Adj u v := by rfl
```

However using `G.Adj u v` has a couple of advantages over `s(u, v) ∈ G.edgeSet`: it avoids the need for `Sym2 V` and, via dot notation, we have easy access to convenient lemmas about adjacency.

```
#check Adj.ne -- G.Adj a b → a ≠ b
example {u v : V} (h : G.Adj u v) : u ≠ v := h.ne
example {u v : V} (h : G.Adj u v) : v ≠ u := h.ne'
```

Generally we prefer to use `G.Adj u v` rather than `s(u, v) ∈ G.edgeSet` when talking about individual edges, but when we want to talk about a *set* of (possible) edges of a graph then `s : Set (Sym2 V)` is what we need.

When we do need to prove something about an arbitrary `e : Sym2 V`, the usual first step is `induction e using Sym2.ind`. Every term of `Sym2 V` is `s(x, y)` for some `x y : V`, and `Sym2.ind` reduces the goal to that case. For example, no edge of `G` is a loop. Mathlib calls a pair of the form `s(v, v)` *diagonal* (`Sym2.IsDiag`).

```
example (e : Sym2 V) (he : e ∈ G.edgeSet) : ¬e.IsDiag := by
  induction e using Sym2.ind with
  | _ x y =>                  -- he : s(x, y) ∈ G.edgeSet ⊢ ¬s(x, y).IsDiag
    rw [mem_edgeSet] at he
    rw [Sym2.mk_isDiag_iff]
    exact he.ne
```

Notice that `induction` replaced `e` by `s(x, y)` in the hypothesis `he` as well as in the goal. This result is `G.not_isDiag_of_mem_edgeSet` in Mathlib.

2.3 Vertices

The vertex set of a `SimpleGraph V` is `V` (although strictly speaking `V` is a type, not a set). These can be infinite (`ℕ`), or empty (`Fin 0`). Nonemptiness and finiteness are extra hypotheses that need to be

stated: for example as [Nonempty V] and [Fintype V]. See Decidability, Finiteness and Degrees (p. 49) for more details.

Mathlib defines a number of useful subsets of V (terms s : Set V), in particular the support of the graph and the neighborSet of a vertex. (We can refer to the entire vertex set of a SimpleGraph V as a *set* by Set.univ : Set V.)

Support

The *support* of a graph is the set of vertices that belong to at least one edge.

```
#check mem_support -- (G : SimpleGraph V) {v : V} : v ∈ G.support ↔ ∃ w, G.Adj v w
```

If G.Adj u v then u is in the support of G.

```
example {u v : V} (h : G.Adj u v) : u ∈ G.support := h.left_mem_support
```

Notice that in the previous example we could use h.left_mem_support where h : G.Adj u v since the theorem we applied is in the SimpleGraph.Adj namespace. Also note the use of "left" in the name of the lemma. You should be able to guess how to prove v ∈ G.support from h : G.Adj u v.

```
#check SimpleGraph.Adj.left_mem_support
```

Neighbours

The *neighbours* of a vertex v in a graph G are the vertices adjacent to v.

```
#source neighborSet -- def neighborSet (v : V) : Set V := {w | G.Adj v w}

example {u v : V} : G.Adj u v ↔ v ∈ G.neighborSet u := by rfl

example {u : V} : G.neighborSet u ⊆ G.support := by
  intro v hv
  exact ⟨u, hv.symm⟩
```

This result is also in Mathlib.

```
example {u : V} : G.neighborSet u ⊆ G.support := G.neighborSet_subset_support u
```

The *incidence set* of v is the set of edges that contain v. (For e : Sym2 V, v ∈ e means that v is one of the two endpoints of e.) A vertex is *isolated* if it has no neighbours; it is *universal* if it is adjacent to every other vertex.

```
#source incidenceSet -- {e ∈ G.edgeSet | v ∈ e}
#check mem_incidenceSet -- s(v, w) ∈ G.incidenceSet v ↔ G.Adj v w
#source IsIsolated -- ∀ w, ¬G.Adj v w
#source IsUniversal -- ∀ ⦃w⦄, v ≠ w → G.Adj v w
```

The braces ∀ ⦃w⦄ in the definition of IsUniversal mark w as a *strict-implicit* argument: Lean fills it in like {w}, but only once a later explicit argument (here the proof of v ≠ w) is supplied.

Sets as Types

Although `G.neighborSet u` is a `Set V`, Lean also lets us use it as a *type*. A term `w : G.neighborSet u` is a vertex together with a proof that this vertex is a neighbour of `u`.

Lean displays this type as `↑(G.neighborSet u)`: it is the *subtype* `{w : V // w ∈ G.neighborSet u}`. We build such terms with the anonymous constructor `⟨v, h⟩` and take them apart with `.val` (the vertex) and `.property` (the proof). Lean inserts `.val` automatically wherever a term of `V` is expected, and displays it as `↑w`.

```
example {u v : V} (h : G.Adj u v) : G.neighborSet u := ⟨v, h⟩

example {u : V} (w : G.neighborSet u) : G.Adj u w.val := by
  exact w.property

example {u : V} (w : G.neighborSet u) : G.Adj u w := by
  exact w.property
```

2.4 Darts

Since edges in `SimpleGraph` are undirected it might seem strange to introduce the notion of a dart (an oriented edge). However we already noted that `h : G.Adj u v` has a direction to it; darts simply give us a way to talk about the ordered pairs of adjacent vertices in a graph. These can be very useful, for example in double counting arguments such as proving the *handshake lemma*.

A *dart* `d : G.Dart` is an ordered pair of vertices together with a proof that they are adjacent in `G`. Since edges are unoriented, darts come in symmetric pairs: if `d : G.Dart` then `d.symm` is the opposite dart and `d.edge` is the edge corresponding to `d`.

```
section darts

#source Dart

variable (d : G.Dart)
#check d.fst  -- the first vertex of the dart
#check d.snd  -- the second vertex of the dart
#check d.adj  -- the adjacency proof of the dart
#check d.edge -- the edge corresponding to the dart
#check d.symm -- the opposite dart from d.snd to d.fst
```

Edges are unoriented so `d` and `d.symm` correspond to the same edge.

```
example : d.symm.edge = d.edge :=
  Dart.edge_symm d

example : d.fst = d.symm.snd := rfl

end darts
```

2.5 Naming Conventions

We have now met a few Mathlib names, and they follow fixed patterns. Knowing the rules will help us guess a lemma's name. The full rules can be found in Mathlib's naming conventions[3]. The key points are:

- Theorems are named in `snake_case`, predicates and structures in `UpperCamelCase` (`IsIsolated`, `Dart`) and other definitions in `lowerCamelCase` (`neighborSet`).
- A theorem's name describes its statement, with the conclusion first and any hypotheses after `of`. Symbols have standard names: `¬` is `not`, `∈` is `mem`, `⊆` is `subset`, `≠` is `ne`, `∉` is `notMem` and `⊤` is `top`. So `not_isDiag_of_mem_edgeSet` says that an edge `e ∈ G.edgeSet` is not a loop, and `neighborSet_subset_support` says that `G.neighborSet u ⊆ G.support`.
- An equivalence (`↔`) that characterises membership of a set, such as `mem_edgeSet` or `mem_support`, is named after its left-hand side alone. Other equivalences often end in `_iff`, such as `Sym2.mk_isDiag_iff`.
- A theorem about `h : G.Adj u v` lives in the namespace `SimpleGraph.Adj`, so we can use it with dot notation, as with `h.ne` and `h.left_mem_support`. A prime marks a minor variant, so `h.ne` proves `u ≠ v` while `h.ne'` proves `v ≠ u`.

When the rules don't allow us to guess the name, we can search for it as described in Finding Lemmas (p. 3).

2.6 Order

There is a partial order on `SimpleGraph V` given by `G ≤ H` iff `G` is a subgraph of `H`. [4]

This gives `SimpleGraph V` the structure of a *complete atomic Boolean algebra*, with operations `sup` (`⊔`), `inf` (`⊓`), `compl` (`ᶜ`) and `sdiff` (`\`), and elements `top` (`⊤`) and `bot` (`⊥`). This terminology comes from order theory rather than graph theory, but it is very natural in the context of formalization. Here *complete* means that every set of graphs in `SimpleGraph V` has a supremum (the graph whose edges are those present in at least one graph in the set) and an infimum (the graph whose edges are those present in every graph in the set). The *atoms* of `SimpleGraph V` are the graphs with exactly one edge, and *atomic* means that every graph is the supremum of the atoms below it, i.e. the union of its single-edge subgraphs. Mathlib registers this structure as an *instance*, which Lean supplies automatically wherever it is needed. The command `#synth` asks Lean to find (synthesize) an instance and shows it; we will say more about instances in Decidability, Finiteness and Degrees (p. 49).

```
section order

#synth CompleteAtomicBooleanAlgebra (SimpleGraph V)

#source SimpleGraph.instLE -- le x y := ∀ ⦃v w : V⦄, x.Adj v w → y.Adj v w
```

[3] `https://leanprover-community.github.io/contribute/naming.html`

[4] Note that these are necessarily spanning subgraphs since all `SimpleGraph V` have the same vertex set. We will consider general subgraphs later, see Subgraphs (p. 246).

```
variable (H : SimpleGraph V)
#check G ≤ H
```

The definition of *proper* subgraph (`G < H`) is phrased differently to how we might expect. Rather than `G < H ↔ G ≤ H ∧ G ≠ H` we have:

```
example : G < H ↔ G ≤ H ∧ ¬H ≤ G := by rfl
```

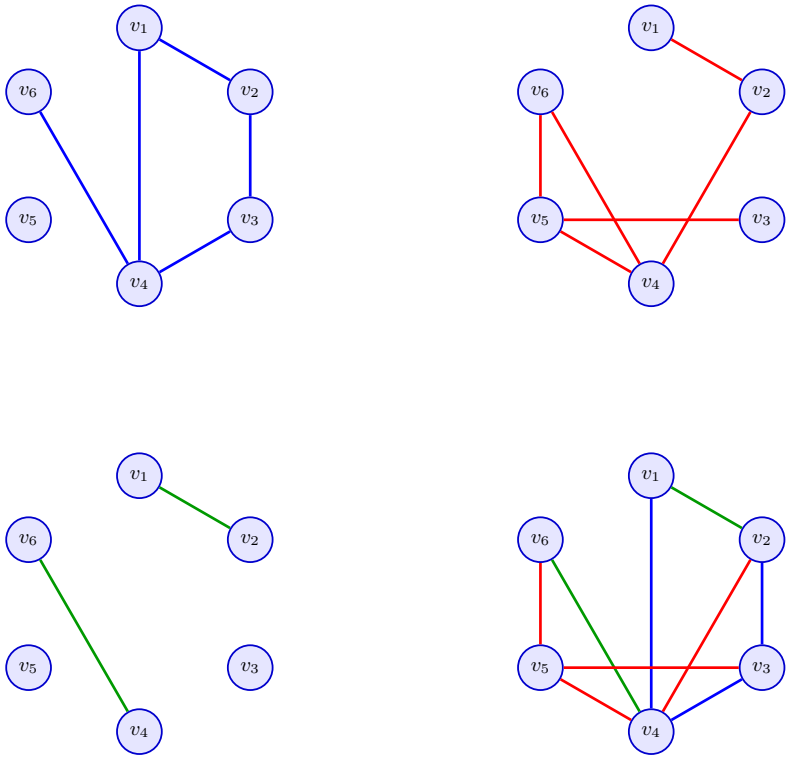
Of course they are equivalent.

```
example : G < H ↔ G ≤ H ∧ G ≠ H := lt_iff_le_and_ne
```

The *sup* of `G` and `H`, denoted `G ⊔ H`, is the `SimpleGraph V` in which any pair of vertices from `V` are adjacent iff they are adjacent in `G` or `H` (i.e. it is the union of `G` and `H`).

```
#check sup_adj
```

The *inf* of `G` and `H`, denoted `G ⊓ H`, is the `SimpleGraph V` in which any pair of vertices from `V` are adjacent iff they are adjacent in both `G` and `H` (i.e. it is the intersection of `G` and `H`).



Top: graphs `G` (blue) and `H` (red) on the same vertices.
Bottom: their inf `G ⊓ H` (left) and sup `G ⊔ H` (right), with the edges in both graphs in green

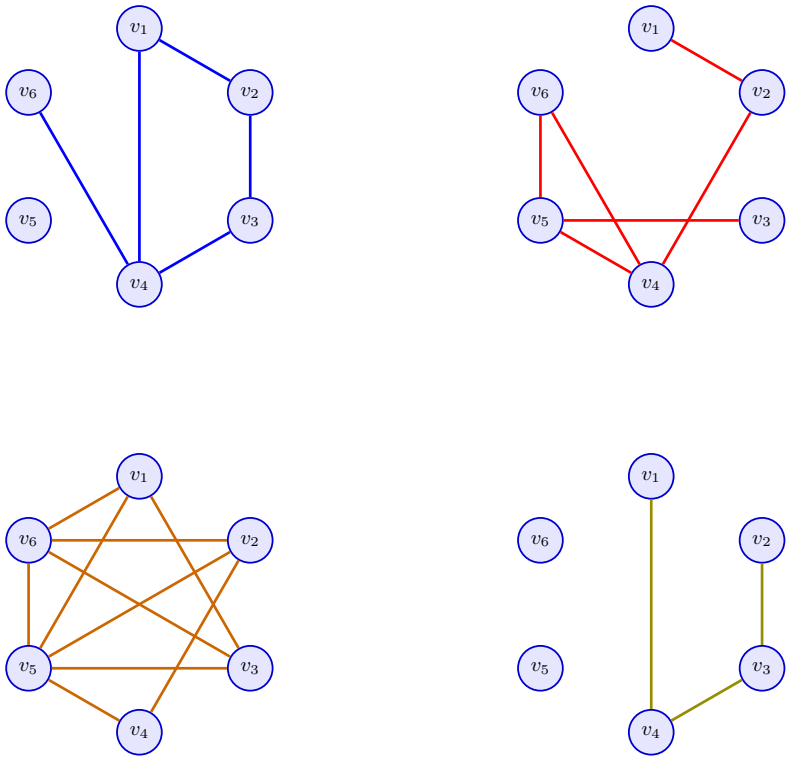
```
#check inf_adj
```

The *complement* of `G`, denoted `Gᶜ`, is the `SimpleGraph V` in which the edges are all pairs of distinct vertices that are not adjacent in `G`.

```
#check compl_adj
```

The *set difference* of two graphs, denoted `G \ H`, has the edges of `G` with the edges of `H` removed. (Since the resulting graph is a `SimpleGraph V` the vertices are necessarily unchanged.)

```
#check sdiff_adj
```



Top: graphs `G` and `H`. Bottom: the complement `Gᶜ` (left) and the set difference `G \ H` (right)

We can remove a set of edges `s : Set (Sym2 V)` from a graph `G` to form `G.deleteEdges s`. If `s = H.edgeSet` then this is the same as the set difference `G \ H`.

```
#source deleteEdges
#check deleteEdges_adj      -- (G.deleteEdges s).Adj v w ↔ G.Adj v w ∧ s(v, w) ∉ s
#check deleteEdges_edgeSet  -- G.deleteEdges H.edgeSet = G \ H
```

Top and Bottom

The *complete graph* `⊤` has all possible edges (every pair of distinct vertices is adjacent), while the *empty graph* `⊥` has no edges. These are the greatest and least elements of the subgraph order. Although `completeGraph V` and `emptyGraph V` are abbreviations for `⊤` and `⊥`, we generally use `top` and `bot` in naming results about these graphs.

```
#check top_adj  -- (⊤ : SimpleGraph V).Adj v w ↔ v ≠ w
#check bot_adj  -- (⊥ : SimpleGraph V).Adj v w ↔ False
```

Every graph is a subgraph of the complete graph.

```
example : G ≤ ⊤ := le_top
```

The empty graph is a subgraph of every graph.

```
example : ⊥ ≤ G := bot_le
```

If G is a proper subgraph of H then G is a proper subgraph of the complete graph.

```
example {G H : SimpleGraph V} (h : G < H) : G < ⊤ := lt_top_of_lt h

end order
end basics
```

Notice that the last few proofs all made use of results about orders rather than anything specific to graphs. They all follow easily from Mathlib's instance of `CompleteAtomicBooleanAlgebra (SimpleGraph V)`, which we displayed with `#synth` at the start of this section.

2.7 Constructing a SimpleGraph

The simplest way to build a `SimpleGraph` is with its default constructor: `SimpleGraph.mk`, supplying an adjacency relation together with proofs of symmetry and irreflexivity. In many cases (such as below) the tactic `aesop_graph` can discharge one or both proofs so they can be omitted. In this section we will look at three different ways of constructing the same graph: the path graph on $\mathbb{N}$.

The path graph on $\mathbb{N}$ defined using the default constructor, with `x` and `y` adjacent iff `x + 1 = y` or `y + 1 = x`.

```
@[simps]
abbrev pathNat : SimpleGraph ℕ where
  Adj x y := x + 1 = y ∨ y + 1 = x
```

The `@[simps]` attribute automatically generates the `simp` lemma `pathNat_adj`. An *attribute*, written `@[…]` before a declaration, tells Lean to do something extra with the declaration once it is defined: `@[simp]` registers a lemma for use by `simp`, and `@[simps]` generates `simp` lemmas from a definition. (If you want to see which lemmas are generated by a `@[simps]` attribute you can change it to `@[simps?]` and the InfoView will tell you.)

```
#check pathNat_adj -- pathNat.Adj x y = (x + 1 = y ∨ y + 1 = x)
```

Another way to construct the same graph is to use the `SimpleGraph.fromRel` constructor which symmetrizes any given relation and removes loops. [5]

```
#source SimpleGraph.fromRel
```

The path graph on $\mathbb{N}$ defined using `SimpleGraph.fromRel`.

```
@[simps!]
def pathNatFromRel : SimpleGraph ℕ := SimpleGraph.fromRel (· + 1 = ·)
```

This time we used `@[simps!]` rather than `@[simps]`. Plain `@[simps]` reads its lemmas straight off a definition written as a structure, as `pathNat` was (using `where`), since the value of each field is written out explicitly. The body of `pathNatFromRel` is instead an application of `fromRel`, so to find `Adj` it must first unfold `fromRel` and then tidy up the result with `simp`. The `!` asks for both. (If we wrote `@[simps]` here, a linter warning would suggest `@[simps!]`.)

[5] The expression `(· + 1 = ·)` below is shorthand for the relation `fun x y => x + 1 = y`.

```
#check pathNatFromRel_adj -- pathNatFromRel.Adj a b =
                          --   (¬a = b ∧ (a + 1 = b ∨ b + 1 = a))
```

Also notice that `pathNatFromRel` is a **`def`**, whereas `pathNat` (and `pathNatFromEdgeSet` below) are **`abbrev`**s. The difference is that an **`abbrev`** is a definition that Lean unfolds automatically when it needs to. This is deliberate: we will see in Decidability, Finiteness and Degrees (p. 49) how this choice affects Lean's ability to compute with these graphs.

The two graphs `pathNat` and `pathNatFromRel` are equal, but not *definitionally* equal: `rfl` does not prove the equality. We explain this important distinction in Definitional and Propositional Equalities (p. 16) below.

In order to prove this we need to show that they have the same adjacency relation (i.e. the same edges). We *don't* need to check that they have the same vertex sets since this is done automatically by the type system: both `pathNat` and `pathNatFromRel` are `SimpleGraph ℕ`.

Lean's type checker ensures that we can't even state a theorem saying that two graphs with different vertex sets are equal. For example if `G : SimpleGraph V` then the expression `G = pathNat` is not well-typed since `pathNat` has vertex type `ℕ` and `G` has vertex type `V`.

```
/- error: Type mismatch
  pathNat
has type
  SimpleGraph.{0} ℕ
-/
example {V : Type*} (G : SimpleGraph V) : G = pathNat := by
  sorry
```

Any two graphs on the same vertex type with the same edges are equal by *extensionality*, and we can use the **`ext`** tactic to prove that `pathNat` and `pathNatFromRel` are equal. The definition of `SimpleGraph` is tagged with the **`@[ext]`** attribute which gives us the following **`ext`** lemmas for free.

```
#check SimpleGraph.ext
#check SimpleGraph.ext_iff
```

The **`ext`** tactic reduces the goal to showing that for all `a` and `b`, `pathNat.Adj a b ↔ pathNatFromRel.Adj a b`. After rewriting with the two adjacency lemmas, what remains is a statement of logic and arithmetic about natural numbers, which **`grind`** proves. **`grind`** is an automated tactic that combines case splitting, congruence closure and linear arithmetic; we use it throughout this guide to finish off goals of this kind, and say more about it in Grind (p. 19).

```
example : pathNat = pathNatFromRel := by
  ext a b
  -- now rewrite using the definition of adjacency in both graphs
  rw [pathNat_adj, pathNatFromRel_adj]
  -- the goal is now to show that the two relations are equivalent
  -- ⊢ a + 1 = b ∨ b + 1 = a ↔ ¬a = b ∧ (a + 1 = b ∨ b + 1 = a)
  grind
```

A third way to construct a `SimpleGraph` is from a set of edges `s : Set (Sym2 V)`. (Recall that edges are unordered pairs of vertices, which have type `Sym2 V` in Lean.)

```
#source SimpleGraph.fromEdgeSet
```

The path graph on $\mathbb{N}$ defined using `SimpleGraph.fromEdgeSet`.

```
@[simps!]
abbrev pathNatFromEdgeSet : SimpleGraph ℕ :=
  SimpleGraph.fromEdgeSet {e | ∃ n, e = s(n, n + 1)}
```

Again we can show that `pathNatFromEdgeSet` is equal to `pathNat` using `ext` (hence all three definitions agree).

```
lemma pathNat_eq_pathNatFromEdgeSet : pathNat = pathNatFromEdgeSet := by
  ext a b
  rw [pathNat_adj, pathNatFromEdgeSet_adj]
  grind
```

We can also state this equality in terms of adjacency, which is usually more convenient.

```
lemma pathNat_adj_iff_pathNatFromEdgeSet {x y : ℕ} :
    pathNat.Adj x y ↔ pathNatFromEdgeSet.Adj x y := by
  rw [pathNat_eq_pathNatFromEdgeSet]
```

Definitional and Propositional Equalities

All three of our path graphs are equal, but Lean distinguishes two kinds of equality. An equality `a = b` is *definitional* if both sides *unfold* to the same term, so that `rfl` proves it; we then say that `a` and `b` are *definitionally equal*, or *defeq* for short. Otherwise the equality is merely *propositional*: it may still be true, but it needs a genuine proof. (Every definitional equality is also propositional, with proof `rfl`.) The equality `pathNat = pathNatFromRel` is propositional but not definitional: we proved it with `ext` and a short argument about natural numbers, while `rfl` fails.

```
/- error: Tactic rfl failed: The left-hand side
  pathNat
is not definitionally equal to the right-hand side
  pathNatFromRel
-/
example : pathNat = pathNatFromRel := by rfl
```

Here *unfolding* means more than replacing names by their definitions: Lean also *computes* (or *reduces*), running any recursion or pattern match whose argument it can see is a constructor. For example `+` on $\mathbb{N}$ is defined by recursion on its second argument, so `n + 0` reduces to `n` even for a variable `n`, and `rfl` proves `n + 0 = n`. But in `0 + n` the recursion is on the variable `n`, which is neither `0` nor a successor, so the computation is *stuck*: `0 + n = n` is true, but only propositionally, and its proof needs induction on `n`.

```
example (n : ℕ) : n + 0 = n := by rfl

/- error: Tactic rfl failed: The left-hand side
  0 + n
is not definitionally equal to the right-hand side
  n
-/
example (n : ℕ) : 0 + n = n := by rfl
```

The distinction matters more in Lean than on paper. Lean's type checker, and its matching of terms when we apply a lemma, only see definitional equalities, so two things that are merely propositionally equal cannot be used interchangeably without an explicit step.

Definitionally equal statements, on the other hand, are interchangeable, and the `change` tactic lets us restate one as another: `change t` replaces the goal by `t`, provided the two are definitionally equal (and `change t at h` does the same for a hypothesis `h`). One use is to unfold a definition that automation cannot see through. Below, `grind` fails on the goal `pathNatFromRel.Adj x (x + 1)`, but once `change` has unfolded `pathNatFromRel` and `fromRel` the goal is about natural numbers, and `grind` proves it. By contrast, `change pathNat.Adj x (x + 1)` would fail, since `pathNat` and `pathNatFromRel` are not definitionally equal.

```
example (x : ℕ) : pathNatFromRel.Adj x (x + 1) := by
  change x ≠ x + 1 ∧ (x + 1 = x + 1 ∨ x + 1 + 1 = x)
  grind
```

Relying on how a definition happens to unfold is sometimes called *defeq abuse*. The proof above depends on the implementation of Mathlib's `fromRel` and might break if that changed, whereas `rw [pathNatFromRel_adj]` (or just `simp`) relies only on a stated lemma. So when a definition comes with lemmas describing it, prefer them, and keep `change` for goals where no such lemma is available.

The Simplifier

The `simp` tactic, Lean's *simplifier*, will be one of our most frequently used tactics, so it is worth knowing how it works. It rewrites the goal using the lemmas in the ***simp** set*, which consists of all the lemmas tagged `@[simp]` (including those generated by `@[simps]`) in Mathlib and in the files we import. Each of these lemmas is an equation or an `↔`, which `simp` uses to rewrite from *left* to *right*. It simplifies the parts of a term before the term itself, and keeps going until no lemma applies. If the goal has become `True` (for example an equation whose two sides are the same) then `simp` closes it. The most common variants are:

- `simp at h` simplifies the hypothesis `h` instead of the goal;
- `simp [foo, h]` also rewrites with the lemma `foo` and the hypothesis `h`, while `simp [-foo]` stops it from using the `simp` lemma `foo`;
- `simp_all` simplifies the goal and all the hypotheses, using each hypothesis to simplify the others and the goal, and is applied until nothing changes;
- `simp only [foo, h]` rewrites with just the listed lemmas and hypotheses.

`simp?` works exactly like `simp`, but also suggests a `simp only` call listing the lemmas that were actually used (clicking on the suggestion in the InfoView replaces `simp?` by it). Running it on the adjacency relations of our three path graphs gives three different answers.

```
example (a b : ℕ) : pathNatFromRel.Adj a b ↔ a ≠ b ∧ (a + 1 = b ∨ b + 1 = a) := by
  simp? -- Try this: simp only [pathNatFromRel_adj, ne_eq]

example (a b : ℕ) : pathNat.Adj a b ↔ a + 1 = b ∨ b + 1 = a := by
  simp? -- Try this: simp only
```

```
example (a b : ℕ) : pathNatFromEdgeSet.Adj a b ↔
    (∃ n, a = n ∧ b = n + 1 ∨ a = n + 1 ∧ b = n) ∧ a ≠ b := by
  simp? -- Try this: simp only [fromEdgeSet_adj, Set.mem_ofPred_eq, Sym2.eq, ...]
```

Two things determine what `simp` does here: whether it can see inside the definition, and what it finds there. `simp` does not unfold a `def` (unless we ask it to, with `simp [pathNatFromRel]`), so all it can do with `pathNatFromRel.Adj a b` is rewrite it using a lemma. It uses `pathNatFromRel_adj`, which `@[simps!]` generated for exactly this purpose. An `abbrev`, on the other hand, is unfolded by `simp`, and what happens next depends on its body. The body of `pathNat` is a structure with its `Adj` field written out, so `simp` reads `pathNat.Adj a b` straight off it, without using any lemma at all. The body of `pathNatFromEdgeSet` is `fromEdgeSet s` for a set of edges `s`, so `simp` sees `(fromEdgeSet s).Adj a b` and rewrites it with Mathlib's lemma `fromEdgeSet_adj` (and then tidies up the result).

So `simp` never uses `pathNat_adj` or `pathNatFromEdgeSet_adj`. They are still useful as named lemmas, for example with `rw` in the proofs above: `pathNatFromEdgeSet_adj` states adjacency in a tidied-up form, whereas rewriting with `fromEdgeSet_adj` would leave a goal about `Sym2`.

In fact Mathlib would not accept `pathNatFromEdgeSet_adj` as a `simp` lemma. If another `simp` lemma also applies to the left-hand side of a `simp` lemma, then `simp` may use that one instead, and the lemma never gets used. So the left-hand side of a `simp` lemma should be in *simp normal form*: no other `simp` lemma should apply to it. The left-hand side `pathNatFromEdgeSet.Adj a b` fails this test, since the `simp` lemma `fromEdgeSet_adj` applies to it. Mathlib checks every `simp` lemma for this with its `simpNF` linter, which you can run on your own file with `#lint only simpNF`. We will meet simp normal forms again in Copy (p. 205).

`simp?` has a second use. The `simp` set grows as Mathlib does, so a plain `simp` may do something different in a future version of Mathlib. That's fine if `simp` closes the goal, but if later steps depend on exactly what `simp` produced then the proof may break. So the convention in Mathlib is that a *non-terminal* `simp`, one that does not close the goal, should be a `simp only`. Mathlib's `flexible` linter, which is enabled for this guide, warns about a non-terminal `simp` (its warning calls `simp` a *flexible* tactic) and suggests using `simp?`. For example, if `a + 1` and `b + 1` are adjacent in `pathNat` then so are `a` and `b`, but the following proof of this triggers the warning.

```
example (a b : ℕ) (h : pathNat.Adj (a + 1) (b + 1)) : pathNat.Adj a b := by
  simp at h
  exact h
```

Following its advice and running `simp?` leads to a proof that does not depend on the rest of the `simp` set.

```
example (a b : ℕ) (h : pathNat.Adj (a + 1) (b + 1)) : pathNat.Adj a b := by
  simp only [Nat.add_right_cancel_iff] at h
  exact h
```

Often a neater fix is to let `simp` finish the proof. `simpa using e` simplifies both the goal and the type of `e`, then closes the goal with `e` (and `simpa [foo] using e` also rewrites with the lemma `foo`). Without the simplification `exact h` would fail, since `h` and the goal only agree once both have been simplified.

```
example (a b : ℕ) (h : pathNat.Adj (a + 1) (b + 1)) : pathNat.Adj a b := by
  simpa using h
```

Grind

`grind` is the other automated tactic we use throughout this guide, and it works quite differently from `simp`. Rather than rewriting the goal, it assumes that the goal is false and looks for a contradiction. Along the way it collects facts, keeps track of which terms are equal, splits into cases, applies lemmas whenever it meets terms that match them, and uses built-in procedures for linear arithmetic over $\mathbb{N}$ and $\mathbb{Z}$ and for algebra in commutative rings. It only ever finishes a goal: if it cannot, it fails and lists the facts it had collected, which often shows what it was missing.

`grind` does not use the `simp` set. Unlike `simp`, it always uses all the hypotheses in the local context (so passing one, as in `grind [h]`, is an error). Apart from these and its built-in reasoning, it only uses the lemmas tagged with a `grind` attribute and those we pass to it, as in `grind [foo, bar]`. Very few of Mathlib's lemmas about graphs are tagged for `grind`, which is why our `grind` calls often list lemmas.

Like `simp`, `grind` unfolds an `abbrev` but not a `def`.

```
example (x : ℕ) : pathNat.Adj x (x + 1) := by grind

example (x : ℕ) : pathNatFromRel.Adj x (x + 1) := by grind [pathNatFromRel_adj]

example (x : ℕ) : pathNatFromEdgeSet.Adj x (x + 1) := by
  grind [pathNatFromEdgeSet_adj]
```

Plain `grind` fails on the last two goals: it cannot see inside the `def` `pathNatFromRel`. It does unfold the `abbrev` `pathNatFromEdgeSet`, but Mathlib's `fromEdgeSet_adj` is not a `grind` lemma, so it is left with a set of edges in `Sym2 ℕ` that it cannot work with. For the last goal `grind [fromEdgeSet_adj]` would also work, but the tidied-up statement of `pathNatFromEdgeSet_adj` (the lemma that `simp` never uses) is more useful in general. With it, `grind` proves that `pathNat` and `pathNatFromEdgeSet` are equal without rewriting first, whereas `grind [fromEdgeSet_adj]` fails here.

```
example : pathNat = pathNatFromEdgeSet := by
  ext a b
  grind [pathNatFromEdgeSet_adj]
```

Like `simp?`, `grind?` suggests a call `grind only [...]` that uses just the lemmas that were needed (it may also offer other forms of the call).

```
example (x : ℕ) : pathNatFromRel.Adj x (x + 1) := by
  grind? [pathNatFromRel_adj] -- Try these: grind only [pathNatFromRel_adj], ...
```

Instead of passing a lemma to each `grind` call, we can tag it with a `grind` attribute, so that every later `grind` call can use it. Most such tags in Mathlib are `@[grind =]`, on equations or ↔s characterizing a definition (like `pathNatFromRel_adj`): `grind` then uses the lemma whenever it meets a term matching its left-hand side. (Similarly `@[grind →]` uses an implication whenever it meets terms matching its hypotheses.) Here we add the attribute for a single example only, with `attribute [local grind =] … in`.

```
attribute [local grind =] pathNatFromRel_adj in
example (x : ℕ) : pathNatFromRel.Adj x (x + 1) := by grind
```

Tags should be added sparingly, since they affect every later `grind` call, and a lemma that matches too many terms can make `grind` slow. The command `#grind_lint inspect foo` warns if the

lemma `foo` leads `grind` into long chains of lemma applications. In this guide we do not tag any lemmas for `grind`. Instead each call lists the lemmas it needs, which also shows the reader which facts the proof uses.

Finally, `grind` is not designed for problems that need a huge number of case splits, such as checking by brute force that a particular graph has no 3-colouring. For finite checks like this we use `decide`, introduced in Decidability, Finiteness and Degrees (p. 49).

2.8 Computations

As well as reasoning about graphs in Lean we can also do computations with them. For example we can check that `0` and `3` are adjacent in the complete graph on `Fin 5` (the type consisting of `{0, 1, 2, 3, 4}`).

```
#eval (⊤ : SimpleGraph (Fin 5)).Adj 0 3   -- true
```

We can count edges in finite graphs. For example the `completeGraph` on `Fin 6` has `15` edges.

```
#eval (⊤ : SimpleGraph (Fin 6)).edgeFinset.card -- 15
```

Even when our goal is to prove theorems, computations such as these can be very useful. Firstly, they allow us to perform quick sanity checks of our definitions and secondly they can be used inside proofs with the `decide` tactic.

```
example : (⊤ : SimpleGraph (Fin 6)).Adj 2 3 := by decide
```

Both `#eval` and `decide` rely on adjacency being *decidable*: Lean needs a concrete method for deciding whether or not any given pair of vertices are adjacent. We will return to this, and to computations with graphs more generally, in Decidability, Finiteness and Degrees (p. 49).

2.9 Exercises

1. Show that the edge set of `pathNat` is `{e | ∃ n, e = s(n, n + 1)}`. Hint: use `induction e using Sym2.ind`.

```
variable {V : Type*}
example : pathNat.edgeSet = {e | ∃ n, e = s(n, n + 1)} := by
  sorry
```

2. Show that `pathNat` has support `ℕ`. (Remember `ℕ` is a type not a set so we need to use `Set.univ` to talk about `ℕ` as a `Set ℕ`.)

```
example : pathNat.support = Set.univ := by
  sorry
```

3. Show that the neighbours of `n + 1` in `pathNat` are exactly `n` and `n + 2`.

```
example (n : ℕ) : pathNat.neighborSet (n + 1) = {n, n + 2} := by
  sorry
```

4. Show that the darts whose first vertex is `v` are exactly the darts obtained from the neighbours of `v` by `dartOfNeighborSet`.

```
#source dartOfNeighborSet

example {G : SimpleGraph V} {v : V} :
    {d : G.Dart | d.fst = v} = Set.range (G.dartOfNeighborSet v) := by
  sorry
```

5. Show that $G \le H$ if and only if $H^c \le G^c$.

```
example (G H : SimpleGraph V) : G ≤ H ↔ Hᶜ ≤ Gᶜ := by
  sorry
```

6. Show that the complement of the complete graph is the empty graph.

```
example : (⊤ : SimpleGraph V)ᶜ = ⊥ := by
  sorry
```

In the next few exercises we follow a convention that we will use throughout this guide. We use a prime, as in `shiftedPathNat'`, to denote a version of a definition or result that contains a `sorry`. This is so that we can use the same names (without a prime) in the solutions below, thus ensuring that when we refer to them in later chapters they resolve to the correct definition.

7. For $i : \mathbb{N}$, define the graph `shiftedPathNat' i` to be a `SimpleGraph ℕ` with edges from `n` to `n + 1` iff $i \le$ `n`. (If `aesop_graph` can prove the `symm` and `loopless` fields for your definition then you can simply delete those two lines.)

```
def shiftedPathNat' (i : ℕ) : SimpleGraph ℕ where
  Adj n m := sorry
  symm := sorry
  loopless := sorry
```

8. Prove that `shiftedPathNat' 0 = pathNat`.

```
example : shiftedPathNat' 0 = pathNat := by
  sorry
```

9. For $i : \mathbb{N}$, define the graph `shortPathNat' i` to be a `SimpleGraph ℕ` with edges from `n` to `n + 1` iff `n` $<$ i.

```
def shortPathNat' (i : ℕ) : SimpleGraph ℕ where
  Adj n m := sorry
  symm := sorry
  loopless := sorry
```

10. Prove that `shortPathNat' i ⊔ shiftedPathNat' i = pathNat`.

```
example (i : ℕ) : shortPathNat' i ⊔ shiftedPathNat' i = pathNat := by
  sorry
```

11. Show that the graph `shiftedPathNat' j` is a subgraph of `shiftedPathNat' i` if and only if `i ≤ j`.

```
example (i j : ℕ) : shiftedPathNat' j ≤ shiftedPathNat' i ↔ i ≤ j := by
  sorry
```

12. Prove that `shortPathNat' i ⊓ shiftedPathNat' i` is the empty graph (on $\mathbb{N}$).

```
example (i : ℕ) : shortPathNat' i ⊓ shiftedPathNat' i = ⊥ := by
  sorry
```

13. Prove that `shortPathNat' i ⊔ shortPathNat' j = shortPathNat' (max i j)`.

```
example (i j : ℕ) : shortPathNat' i ⊔ shortPathNat' j = shortPathNat' (max i j) := by
  sorry
```

14. Show that `pathNat \ shortPathNat' i` is `shiftedPathNat' i`.

```
example (i : ℕ) : pathNat \ shortPathNat' i = shiftedPathNat' i := by
  sorry
```

15. `SimpleGraph` has an *indexed supremum* operation `⨆ i, G i` which is the graph with edges between `u` and `v` if and only if there exists an index `i` such that `u` and `v` are adjacent in `G i`. Prove that `pathNat = ⨆ i : ℕ, shortPathNat' i`.

```
example : pathNat = ⨆ i : ℕ, shortPathNat' i := by
  sorry
```

3 Constructions

In this chapter we will introduce some of the concrete examples of simple graphs that can be found in Mathlib together with a few new ones. The idea is to give a flavour of the kinds of constructions that are possible in Mathlib. It also offers a good excuse to introduce many of the fundamental types that may be unfamiliar to graph theorists but are used throughout Mathlib. These include `Bool`, `Fin`, `Option`, `Prod`, `Sigma`, `Subtype`, `Sum`, `Unit` and `ZMod`.

Two of the constructions we introduce below, the hypercube (p. 35) and Mycielskian (p. 38), will serve as running examples throughout the remaining chapters.

Beware that in this chapter we are looking at *constructions*. We will see examples of complete bipartite graphs and cycle graphs, but we won't yet discuss what it means for a graph to *be* bipartite or to *contain* a cycle.

To sanity-check our definitions, we will want to perform computations on small examples using `#eval` and `decide`. We will look at such computations in much more detail in the next chapter, but for them to work we need to provide decidability instances for adjacency in the graphs we define. Feel free to ignore these and revisit them once you have read the next chapter.

3.1 Basic Families of Graphs

We start with the most basic families of graphs: complete and empty graphs, single edges, paths, cycles and complete bipartite graphs.

Complete and Empty Graphs

Given a vertex set `V` the *complete* graph on `V` has all possible edges while the *empty* graph on `V` has no edges.

In Mathlib these are usually referred to by their roles as the top `⊤` and bottom `⊥` elements of the complete atomic Boolean algebra formed by `SimpleGraph V` (see Order (p. 11) for more details).

```
#source completeGraph
#source emptyGraph
```

As we saw in Top and Bottom (p. 13), results about these graphs use the `top` and `bot` names, as in `top_adj` and `bot_adj`.

Given `n : ℕ`, `Fin n` is the type of natural numbers less than `n` with addition mod `n`. (In particular `Fin 0` is the empty type.)

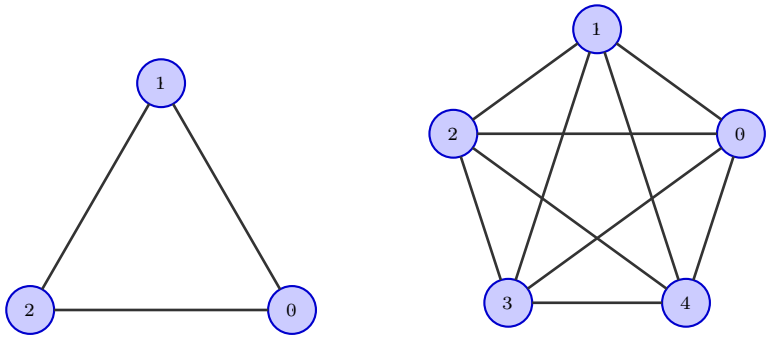
We can construct the complete graph on `Fin n` as follows.

```
example (n : ℕ) : SimpleGraph (Fin n) := completeGraph (Fin n)
```

We introduce notation for the complete graph on `Fin n`. (The keyword `scoped` makes the notation available only inside `namespace SimpleGraph`, or wherever `SimpleGraph` is opened.)

`K[n]` is the complete graph on `Fin n`.

```
scoped notation "K[" n "]" => completeGraph (Fin n)
```



The complete graphs `K[0]`, `K[3]` and `K[5]`. `K[0]` has no vertices, so nothing is drawn on the left.

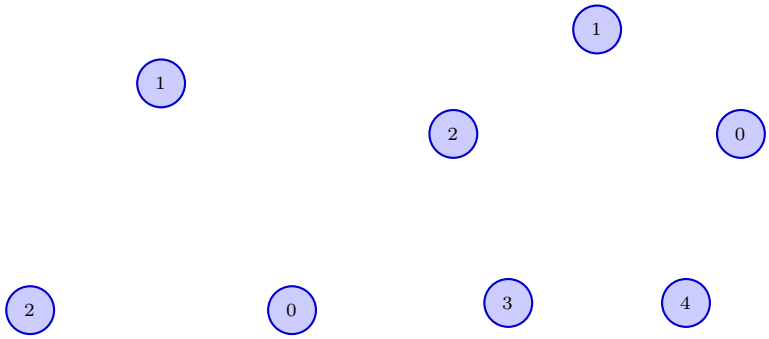
Since `Fin n` has decidable equality, `K[n]` has decidable adjacency: if we have a procedure for deciding whether two vertices are equal then we can decide whether they are adjacent in the complete graph, since adjacency is precisely not-equal-to `≠`. For example, `10` and `111` are adjacent in `K[417]`.

```
example : K[417].Adj 10 111 := by decide
```

We also introduce notation for the empty graph on `Fin n`.

`E[n]` is the empty graph on `Fin n`.

```
scoped notation "E[" n "]" => emptyGraph (Fin n)
```



The empty graphs `E[0]`, `E[3]` and `E[5]`. `E[0]` has no vertices, so nothing is drawn on the left.

```
#check E[7]        -- E[7] : SimpleGraph (Fin 7)
#eval E[4].Adj 1 2 -- false
```

A Single Edge

We can construct a `SimpleGraph V` with just one edge between two vertices `s : V` and `t : V` using `edge s t`. If `s = t` then this is just the empty graph on `V`. (Recall that `SimpleGraph V` is a `CompleteAtomicBooleanAlgebra`: for `s ≠ t` these graphs are the atoms of the algebra.)

```
#source edge  -- def edge : SimpleGraph V := fromEdgeSet {s(s, t)}
```

You may wonder why we care about the graph with a single edge, but in fact it can be very useful. Arguments about adding a single edge to, or removing one from, a graph can be formulated using it.

For example, if we add `edge s t` to `G`, where `s` and `t` are distinct vertices that do not form an edge in `G`, then we get a strictly larger graph.

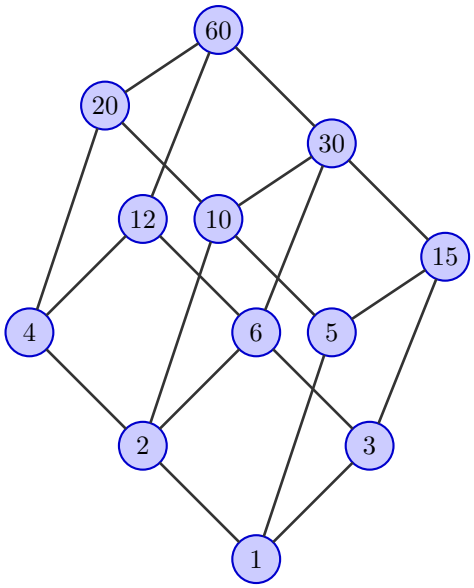
```
#check lt_sup_edge -- .. (hne : s ≠ t) (hn : ¬G.Adj s t) : G < G ⊔ edge s t
```

Path Graphs

In Mathlib the canonical path graph on `n` vertices is constructed as the Hasse diagram of the linear order on `Fin n`, so we first describe Hasse diagrams.

Given a poset `(P, ≤)` we say that `a ∈ P` is *covered by* `b ∈ P` if `a < b` and `∀ c ∈ P, a < c → ¬c < b`. In Mathlib this is denoted `a ⋖ b`.

The *Hasse diagram* of a poset `(P, ≤)` is the graph with vertex set `P` and edges between pairs of vertices such that one is covered by the other.



The Hasse diagram of the divisors poset of `60`

```
example {P : Type*} {u v : P} [PartialOrder P] :
    (hasse P).Adj u v ↔ u ⋖ v ∨ v ⋖ u := by rfl
```

The path graph on `n` vertices is then the Hasse diagram of `Fin n`. Recall that addition in `Fin n` is defined mod `n`.

```
example : (2 : Fin 4) + 2 = 0 := rfl
```

So when describing adjacency in this graph we need to use the underlying $\mathbb{N}$ values.

```
#source pathGraph
```

```
example (n : ℕ) (a b : Fin n) :
    (pathGraph n).Adj a b ↔ a.val + 1 = b.val ∨ b.val + 1 = a.val := pathGraph_adj
```

We introduce notation for the path graph on `Fin n`.

`P[n]` is the path graph `pathGraph n` on `Fin n`.

```
scoped notation "P[" n "]" => pathGraph n
```



The path graphs `P[2]`, `P[4]` and `P[7]`

Cycle Graphs

For any `n : ℕ` we can define the *cycle graph* on `n` vertices as the graph with vertex set `Fin n` and edges between vertices that differ by `1` modulo `n`. Although we usually think of cycles as always containing at least `3` vertices, in Mathlib we also have cycle graphs on `0`, `1`, and `2` vertices. This is a common pattern in Mathlib: rather than add side conditions to the definition, we will instead need to check when using `cycleGraph` whether or not it is in fact a cycle!

For `3 ≤ n` the definition of adjacency is obvious: `Adj a b := a - b = 1 ∨ b - a = 1`. This also makes sense for `n = 2` but gives us the graph on `Fin 2` with a single edge.

The cases `n = 0` and `n = 1` need a bit more care: for `a b : Fin 0` we can't even write `a - b = 1 ∨ b - a = 1` since there is no `1 : Fin 0`. While for `a b : Fin 1` we can write `a - b = 1 ∨ b - a = 1` but `1 = 0` in `Fin 1` so this would define a graph with loops, which is not allowed.

```
example : (1 : Fin 1) = 0 := rfl
```

The solution is to define `cycleGraph n` to be the empty graph for `n = 0` and `n = 1`.

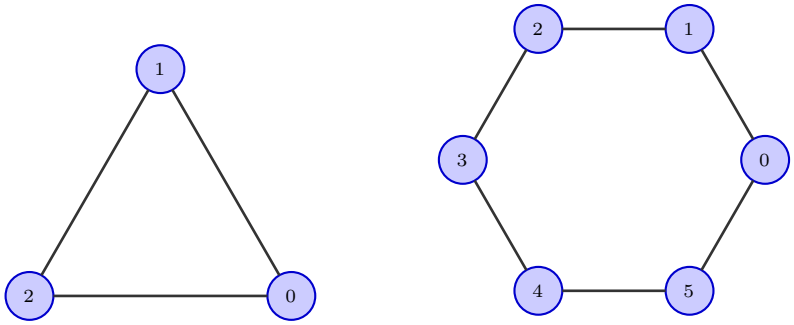
```
#source cycleGraph --  def cycleGraph : (n : ℕ) → SimpleGraph (Fin n)
                   --    | 0 | 1 => ⊥
                   --    | _ + 2 => {
                   --      Adj a b := a - b = 1 ∨ b - a = 1
                   --    }
```

We introduce notation for cycle graphs.

`C[n]` is the cycle graph `cycleGraph n` on `Fin n`.

```
scoped notation "C[" n "]" => cycleGraph n

#check C[5]   -- C[5] : SimpleGraph (Fin 5)
example : C[5].Adj 0 1 := by decide
example : ¬C[5].Adj 4 2 := by decide
```



The cycle graphs `C[3]` and `C[6]`

As we saw above, for `n = 0` and `n = 1`, `C[n]` is the empty graph on `Fin n`, while for `n = 2` it is the complete graph `K[2]`. The first two of these are *definitional* equalities (in the sense of Definitional and Propositional Equalities (p. 16)), while the last is not. The adjacency relation `C[2].Adj a b` is `a - b = 1 ∨ b - a = 1`, while `K[2].Adj a b` is `a ≠ b`. We can prove that these are equivalent, but they are not definitionally equal.

```
example : C[0] = E[0] := rfl
example : C[1] = E[1] := rfl
example : C[2] = K[2] := cycleGraph_two_eq_top
```

Complete Bipartite Graphs

Given two disjoint vertex sets `V` and `W` we can form the *complete bipartite* graph with classes `V` and `W`. This seems to involve the idea that `V` and `W` are disjoint, but in Lean we are working with types not sets and so have no way of talking about *disjointness* of two arbitrary types.

Lean does however have the very useful `Sum` type, written `V ⊕ W`, which plays the role of the disjoint union of types `V` and `W`. Note that this is not the same as the set-theoretic union: any `x : V ⊕ W` "knows" which of the two sides of the sum it came from. It is either `Sum.inl v` for some `v : V`, or `Sum.inr w` for some `w : W`. In particular we can form the disjoint union of a type `V` with itself as `V ⊕ V`.

```
#source Sum
example {V : Type*} (v : V) : V ⊕ V := Sum.inl v
```

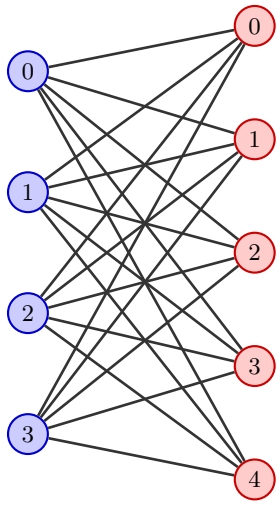
This gives a natural way to define *complete bipartite* graphs.

```
#source completeBipartiteGraph
```

We introduce notation for the canonical complete bipartite graph with parts `Fin s` and `Fin t`.

`K[s, t]` is the complete bipartite graph on `Fin s ⊕ Fin t`.

```
scoped notation "K[" s ", " t "]" => completeBipartiteGraph (Fin s) (Fin t)
```



The complete bipartite graph `K[4, 5]`

```
#check K[4, 5]  -- K[4, 5] : SimpleGraph (Fin 4 ⊕ Fin 5)
```

If we try to check adjacency of two vertices in `K[4, 5]` we get an error.

```
/- error: failed to synthesize
 Decidable (K[4, 5].Adj (Sum.inl 0) (Sum.inr 3))
-/
#eval K[4, 5].Adj (Sum.inl 0) (Sum.inr 3)
```

This is because there is currently no instance of decidable adjacency for `completeBipartiteGraph` in Mathlib. In this case it is easy to give. (See Decidability, Finiteness and Degrees (p. 49) for more on decidability.)

```
instance (V W : Type*) : DecidableRel (completeBipartiteGraph V W).Adj := by
  intro v w
  unfold completeBipartiteGraph
  infer_instance

example : K[4, 5].Adj (Sum.inl 0) (Sum.inr 3) := by decide
```

3.2 Algebraic Constructions

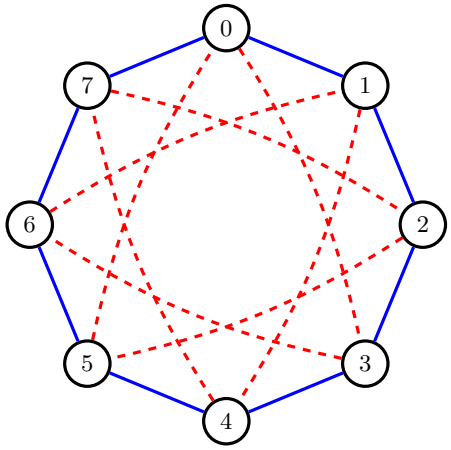
Next we look at two families of graphs whose vertices are the elements of a group, with adjacency defined using the group operation.

Circulant Graphs

If `G` is an additive group and `s ⊆ G`, the associated *circulant graph* has vertex set `G` with edges between vertices whose difference belongs to `s` (suitably symmetrized so that the resulting graph is undirected and loopless).

```
#source circulantGraph -- def circulantGraph {G : Type*} [AddGroup G] (s : Set G) :
                       --        SimpleGraph G := fromRel (· - · ∈ s)
```

```
example : SimpleGraph (Fin 8) := circulantGraph ({1, 3} : Set (Fin 8))
```



The circulant graph on `Fin 8` with differences `1` and `3`

Paley Graphs

A special case of a circulant graph is the Paley graph. If *F* is a finite field, the *Paley graph* [Paley 1933] of *F* is the circulant graph on *F* whose differences are the nonzero squares in *F*. Usually one also requires `|F| ≡ 1 mod 4`: then `-1` is a square, so `u - v` is a square if and only if `v - u` is, and each vertex is adjacent to exactly half of the others. (For any other finite field the definition below gives the complete graph: if `|F| ≡ 3 mod 4` then exactly one of `d` and `-d` is a square for each `d ≠ 0`, while if `|F|` is even then every element of *F* is a square.) The `circulantGraph` calls the constructor `fromRel (· - · ∈ s)`, which automatically removes loops and symmetrizes the relation, so we can simply take all the squares, including `0`.

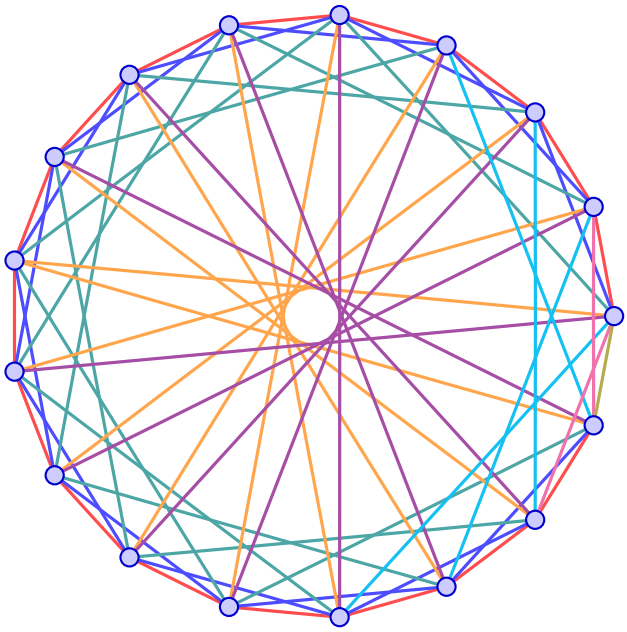
The Paley graph of a field *F*: distinct elements are adjacent if their difference is a square.

```
abbrev paleyGraph (F : Type*) [Field F] : SimpleGraph F :=
  circulantGraph {y | IsSquare y}
```

Mathlib's type of integers modulo `n` is `ZMod n`. (For `n ≠ 0` it is defined to be `Fin n`.) If `p` is prime then `ZMod p` is a field, so `paleyGraph (ZMod p)` is the Paley graph on `p` vertices. However, Lean will only find the `Field (ZMod p)` instance if it can find a proof that `p` is prime, in the form of an instance of `Fact (Nat.Prime p)`. Mathlib provides these for `p = 2` and `p = 3`, but for `p = 17` we need to supply one ourselves.

```
instance fact_prime_seventeen : Fact (Nat.Prime 17) := ⟨by decide⟩

#source ZMod
example : SimpleGraph (ZMod 17) := paleyGraph (ZMod 17)
```



The Paley graph `paleyGraph (ZMod 17)`

The vertex type here is `ZMod 17` rather than `Fin 17` because `paleyGraph` needs a field. Although `ZMod 17` is defined to be `Fin 17`, instance search does not unfold this definition, so the two types can carry different instances. Mathlib puts the `Field` instance on `ZMod p` and has none on `Fin n`. The cycle graph `C[n]`, on the other hand, only needs addition mod `n`, and Mathlib does make `Fin n` an additive group for `n ≠ 0`. Since `cycleGraph n` has to make sense for every `n`, and `ZMod 0` is `ℤ`, `Fin n` is the better vertex type there.

Cayley Graphs

Cayley graphs [Cayley 1878] can also be found in Mathlib. They are defined for any vertex type with a multiplication or addition operation, not just for groups.

If you look at the definition you will see the use of the `@[to_additive]` attribute to give both multiplicative and additive versions of the same construction. (In particular if you look for the source code for `addCayley`, you will be taken to `mulCayley`.)

```
#source mulCayley
```

The symmetric group on four points is `Equiv.Perm (Fin 4)` in Mathlib, and its elements can be written in cycle notation using `c[...]`. Taking a 3-cycle and a 4-cycle as generators gives the graph drawn below.

The Cayley graph of the symmetric group on `Fin 4` generated by a 3-cycle and a 4-cycle.

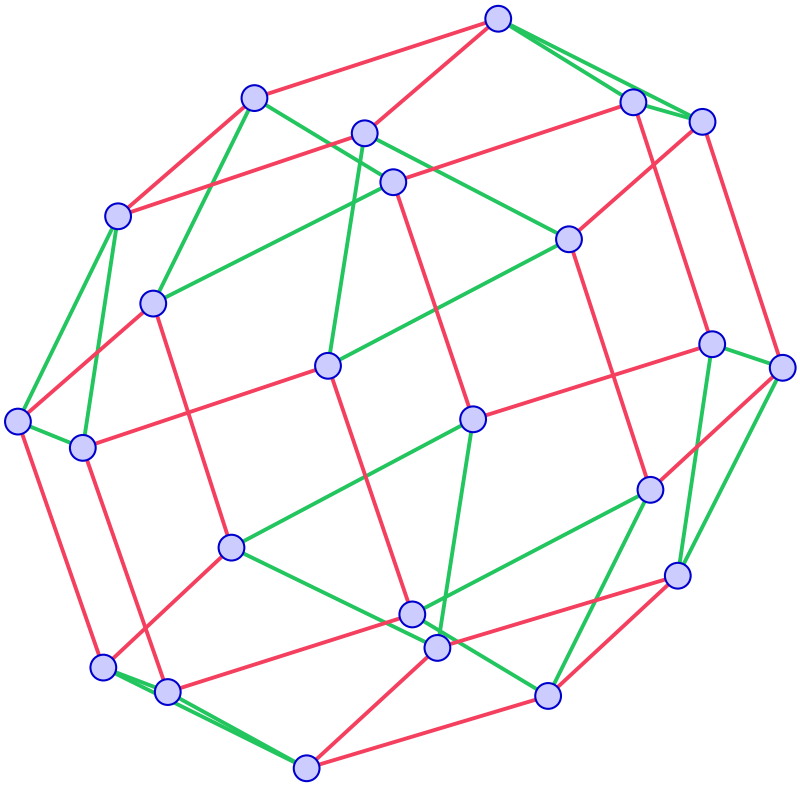
```
abbrev cayleyS4 : SimpleGraph (Equiv.Perm (Fin 4)) :=
  mulCayley {c[0, 1, 2], c[0, 1, 2, 3]}
```

It has one vertex for each of the 24 permutations and is 4-regular, since each generator accounts for two of the edges at every vertex: one to the neighbour it multiplies to, and one back from its inverse.

```
#eval Fintype.card (Equiv.Perm (Fin 4)) -- 24
#eval cayleyS4.degree 1 -- 4
#eval cayleyS4.edgeFinset.card -- 48
```

The two edge colours in the figure are the two generators: the triangles are the edges contributed by the 3-cycle and the squares those contributed by the 4-cycle. Splitting the graph up like this is exactly `mulCayley_union`, which says that a union of generating sets gives the union of the corresponding Cayley graphs.

```
example : cayleyS4 = mulCayley {c[0, 1, 2]} ⊔ mulCayley {c[0, 1, 2, 3]} := by
  rw [← mulCayley_union, Set.singleton_union]
```



The Cayley graph `cayleyS4` of `Equiv.Perm (Fin 4)`, generated by a 3-cycle and a 4-cycle

3.3 Operations on Graphs

So far each construction has built a graph from scratch. We now look at some ways of building new graphs from old ones.

Line Graphs

Given `G : SimpleGraph V`, its *line graph* is the graph whose vertices are the edges of `G`, in which two distinct edges are adjacent iff they share a vertex. In Mathlib this is defined as follows.

```
#source lineGraph -- def lineGraph : SimpleGraph G.edgeSet where
                  --     Adj e₁ e₂ := e₁ ≠ e₂ ∧ (e₁ ∩ e₂ : Set V).Nonempty
                  --     symm.symm e₁ e₂ hadj := by rwa [ne_comm, Set.inter_comm]
```

We introduce notation for the line graph of `G`.

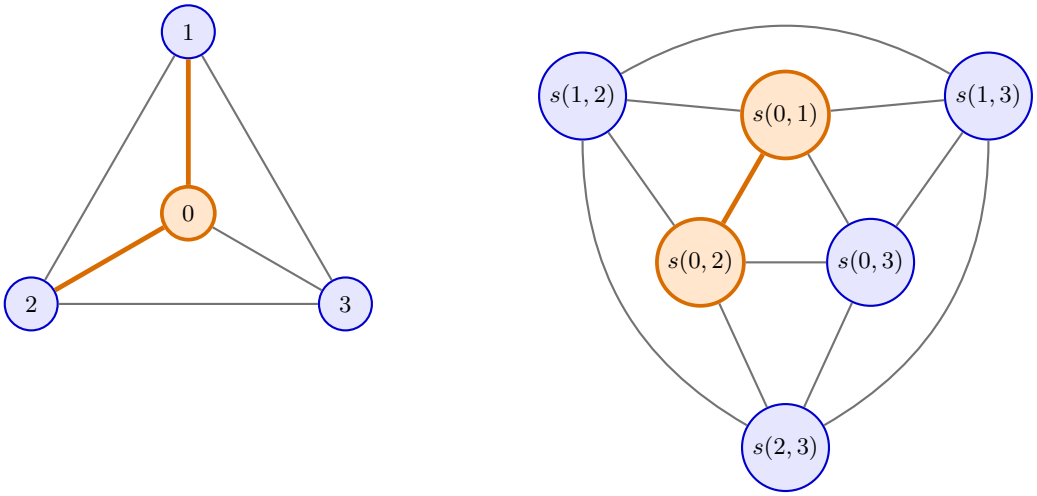
`L[G]` is the line graph of `G`: its vertices are the edges of `G`, and two are adjacent if they share a vertex.

```
scoped notation "L[" G "]" => lineGraph G
```

The edge set `G.edgeSet` is a `Set (Sym2 V)`, so to use it as the vertex type of `L[G]`, Lean coerces it to the subtype `{e : Sym2 V // e ∈ G.edgeSet}`, written `↑G.edgeSet`. A vertex of `L[G]` is therefore an edge bundled with a proof that it belongs to `G`: a term `⟨s(u, v), h⟩` with `h : s(u, v) ∈ G.edgeSet`. This membership is the same as `G.Adj u v` (`mem_edgeSet`). For a concrete graph, **`decide`** can supply `h`.

```
#check L[K[4]] -- SimpleGraph ↑K[4].edgeSet
#check (⟨s(0, 1), by decide⟩ : K[4].edgeSet)
```

Adjacency in `L[G]` has two parts. The condition `e₁ ≠ e₂` is needed because every edge shares a vertex with itself, and simple graphs have no loops. The condition `(e₁ ∩ e₂ : Set V).Nonempty` says that the edges share a vertex. To state it, each edge is coerced twice: first from `↑G.edgeSet` to `Sym2 V`, and then from `Sym2 V` to `Set V`, the set of its endpoints.



Left: `K[4]`. Right: its line graph `L[K[4]]`, the octahedron

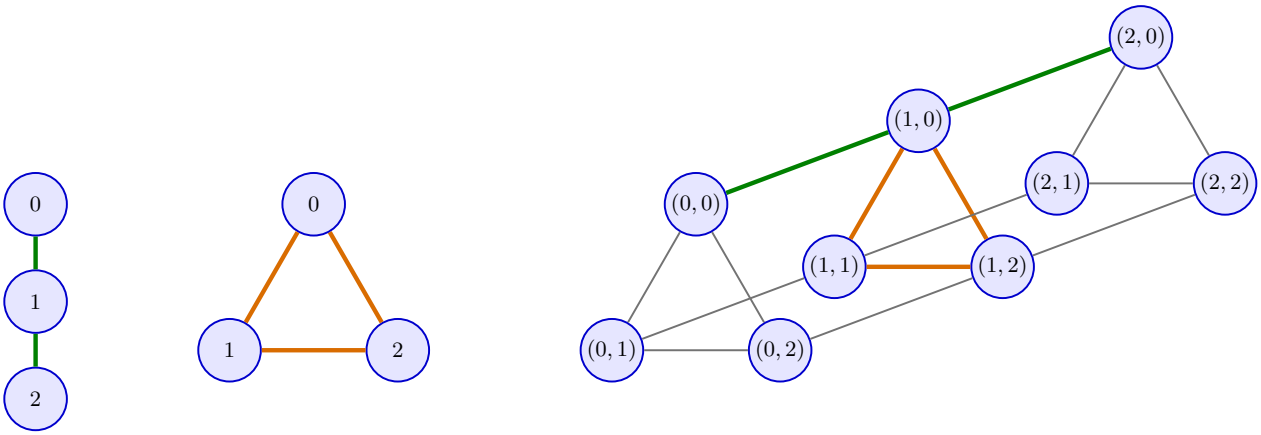
For example, `s(0, 1)` and `s(0, 2)` are adjacent in `L[K[4]]`. In the proof below, the first goal is that they are distinct and the second that they share a vertex, for which `0` is a witness.

```
example : L[K[4]].Adj ⟨s(0, 1), by decide⟩ ⟨s(0, 2), by decide⟩ := by
  constructor
  · -- ⊢ ⟨s(0, 1), ⋯⟩ ≠ ⟨s(0, 2), ⋯⟩
    grind
  · -- After the double coercion we need to check that the Set (Fin 4)
    -- {0, 1} and {0, 2} have nonempty intersection.
    -- ⊢ (↑↑⟨s(0, 1), ⋯⟩ ∩ ↑↑⟨s(0, 2), ⋯⟩).Nonempty
    use 0; simp
```

Box Products

The *box product* of two graphs `G` and `H` is the graph on the Cartesian product of their vertex sets, where a pair of vertices `(g₁, h₁)` and `(g₂, h₂)` are adjacent if either `g₁ = g₂` and `H.Adj h₁ h₂` or

`h₁ = h₂` and `G.Adj g₁ g₂`. This is written `G □ H` in Mathlib. Its vertex type is the product type `V × W`, whose elements are pairs `(g, h)`; for `x : V × W` the two components are `x.1` and `x.2` (also written `x.fst` and `x.snd`). For each vertex `h` of `H`, the vertices `(g, h)` span a copy of `G` in `G □ H`, the *G-fibre* at `h`; similarly each vertex `g` of `G` gives a copy of `H`, the *H-fibre* at `g`.



The box product `P[3] □ K[3]`, with the `K[3]`-fibre at `1` (orange) and the `P[3]`-fibre at `0` (green).

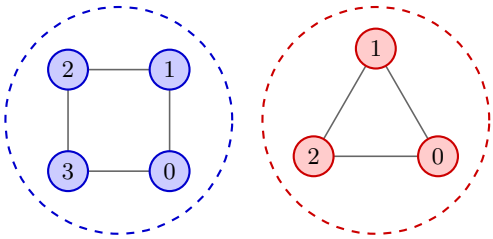
```
#source boxProd
```

Adjacency in the box product of the path graph `P[m]` with the complete graph `K[n]`.

```
example {m n : ℕ} {a b : Fin m} {i j : Fin n} : (P[m] □ K[n]).Adj (a, i) (b, j) ↔
    ((a.val + 1 = b.val ∨ b.val + 1 = a.val) ∧ i = j) ∨ (i ≠ j ∧ a = b) := by
  simp [pathGraph_adj]
```

Sum and Sigma Graphs*

Given graphs `G : SimpleGraph V` and `H : SimpleGraph W`, their *(disjoint) sum* `G ⊕g H` is the graph formed by placing a copy of `G` and a copy of `H` side by side, with no edges between them. Its vertex type is the sum type `V ⊕ W`, whose elements are either `Sum.inl v` with `v : V` or `Sum.inr w` with `w : W`. Two vertices `Sum.inl v₁` and `Sum.inl v₂` are adjacent iff `G.Adj v₁ v₂`, two vertices `Sum.inr w₁` and `Sum.inr w₂` are adjacent iff `H.Adj w₁ w₂`, and no vertex `Sum.inl v` is adjacent to any vertex `Sum.inr w`. Mathlib defines `G ⊕g H` by pattern matching on these cases.



The sum `C[4] ⊕g K[3]`

```
#source SimpleGraph.sum
#check sum_adj_inl
#check sum_adj_inr
```

More generally, we can take the disjoint union of an indexed family of graphs, which we call a *sigma graph*. This is not currently in Mathlib, so we define it here. If the graph indexed by `i : ι` has vertex type `V i`, then the vertex type of the disjoint union is the *dependent pair* (or `Sigma`) type `Σ i, V i`.

```
#source Sigma
```

A vertex of `Σ i, V i` is a pair `⟨i, u⟩` consisting of an index `i : ι` and an element `u : V i`. The pair is called *dependent* because the type of its second component depends on the value of its first. Lean prints `Σ i, V i` as `(i : ι) × V i`, so expect to see that form in goals and `#check` output.

Two vertices `⟨i, u⟩` and `⟨j, v⟩` should be adjacent iff `i = j` and `u` and `v` are adjacent in `G i`. Stating this takes some care: `u : V i` but `v : V j`, so `(G i).Adj u v` does not type-check. A proof `h : i = j` does not fix this on its own, because Lean only identifies types that are equal by definition, and `V i` and `V j` are not. Instead we use `h` to cast `v` along the equation, giving `h ▸ v : V i`.

We mark the definition `protected`, so that even inside `namespace SimpleGraph` it must be called `SimpleGraph.sigma`. This stops the short name `sigma` from clashing with other functions of that name, such as `Finset.sigma`.

The disjoint union of an indexed family of graphs. Given an index type `ι` and, for each `i : ι`, a graph `G i` on the vertex type `V i`, the disjoint union `Σg G` has vertex type `Σ i, V i`, and two vertices are adjacent iff they lie in the same graph `G i` and are adjacent there.

```
@[simps!]
protected def sigma {ι : Type*} {V : ι → Type*} (G : (i : ι) → SimpleGraph (V i)) :
    SimpleGraph (Σ i, V i) where
  Adj := fun ⟨i, u⟩ ⟨j, v⟩ => ∃ h : i = j, (G i).Adj u (h ▸ v)
  symm := by
    constructor
    rintro ⟨i, u⟩ ⟨j, v⟩ ⟨rfl, hadj⟩
    exact ⟨rfl, hadj.symm⟩
```

In the proof of `symm` above, `rintro ⟨i, u⟩ ⟨j, v⟩ ⟨rfl, hadj⟩` introduces the two vertices and splits the adjacency proof into its two parts. The pattern `rfl` substitutes `i = j` everywhere, after which `h ▸ v` reduces to `v`.

The substitution is needed because the cast `h ▸ v` only reduces to `v` (in the sense of Definitional and Propositional Equalities (p. 16)) when both sides of `h` are definitionally equal. While `i` and `j` are distinct variables the cast is stuck, just like `0 + n`, and no amount of unfolding gets past it to `v`. We will meet this issue again in Decidability and Kernel Reduction (p. 57) and when we need to change the endpoints of walks in Copy (p. 205).

The `@[simps!]` attribute generates `sigma_adj`, in which the ⋯ hides the proof `h` that the two first components agree.

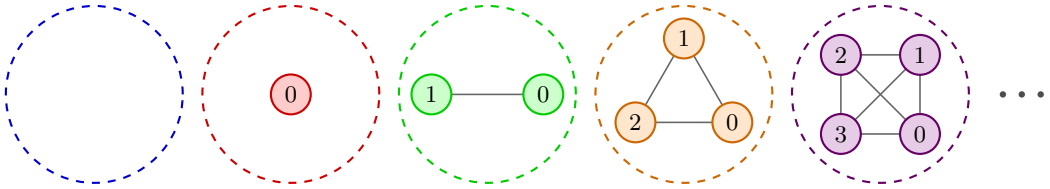
```
#check SimpleGraph.sigma_adj
```

We write `Σg G` for `SimpleGraph.sigma G`.

```
@[inherit_doc] scoped prefix:60 "Σg " => SimpleGraph.sigma
```

The disjoint union of the complete graphs `K[0]`, `K[1]`, `K[2]`, ...

```
abbrev sigmaCompleteNat : SimpleGraph (Σ i, Fin i) :=
  Σg fun i : ℕ => K[i]
```



Part of `sigmaCompleteNat`

In our next example we show that two vertices $\langle i, a\rangle$ and $\langle j, b\rangle$ of `sigmaCompleteNat` are adjacent iff they lie in the same complete graph ($i = j$) and are distinct. We cannot write $a \neq b$ here, since a : `Fin` i and b : `Fin` j have different types; this is the same problem as above. Comparing the underlying natural numbers (a : $\mathbb{N}$) and (b : $\mathbb{N}$) avoids it.

In the proof, `t₁ <;> t₂` runs the tactic `t₂` on every goal produced by `t₁`: here `constructor` splits the $\leftrightarrow$ into two implications, and `rintro ⟨rfl, h⟩` then introduces the hypotheses in both, substituting as in the `sigma` proof above.

```
example {i j : ℕ} {a : Fin i} {b : Fin j} :
    sigmaCompleteNat.Adj ⟨i, a⟩ ⟨j, b⟩ ↔ i = j ∧ (a : ℕ) ≠ b := by
  constructor <;> rintro ⟨rfl, h⟩
  · exact ⟨rfl, Fin.val_ne_of_ne h⟩
  · exact ⟨rfl, fun hab => h (congrArg Fin.val hab)⟩
```

3.4 Hypercube

Often there is more than one way to construct a particular graph. A good example of this is the *n-dimensional hypercube*, which can be constructed in many different ways: as the Hasse diagram of the Boolean algebra of subsets of an n-element set; as the graph on `Fin n → Bool` with edges between vertices that differ in exactly one coordinate (i.e. at Hamming distance one [Hamming 1950]); or as the box product of n copies of `K₂`.

The Hasse diagram construction is very easy to give, but then is a pain to work with.

The Hasse diagram construction of the n-dimensional hypercube.

```
abbrev hypercubeHasse (n : ℕ) : SimpleGraph (Finset (Fin n)) :=
  hasse _
```

The Hamming cube construction is also easy to give, and easier to work with than the Hasse diagram: adjacency is a single symmetric condition, and all coordinates are treated alike.

The Hamming cube definition of the n-dimensional hypercube.

```
def hypercubeHamming (n : ℕ) : SimpleGraph (Fin n → Bool) where
  Adj x y := hammingDist x y = 1
  symm := ⟨by simp [hammingDist_comm]⟩
```

But for our purposes the most useful construction of the hypercube is as the box product of n copies of `K₂`.

This construction requires a bit more setup, but will be worth it. The reason we choose it is that it is *recursive* in n: its vertex type is built so that `Q[n + 1] = Q[n] □ K₂` holds definitionally, where `Q[n]` denotes the n-dimensional hypercube. Almost every result we prove about `Q[n]` in the later

chapters is by induction on n, and this recursive structure is exactly what allows these to work. It also lets us reuse Mathlib's box product lemmas directly. (The price is that the last coordinate is singled out, so for results about the symmetries of the hypercube, such as its automorphism group, we will pass to the Hamming cube in The Hypercube and the Hamming Cube (p. 377).)

To give the box product construction we first need to define its vertex type. There are two things to decide. Firstly, what should the vertex type of `Q[0]` be? Since `Q[0]` has a single vertex we will use the canonical type with exactly one element, `Unit`.

```
#check Unit
```

Secondly, to form `Q[n + 1] = Q[n] □ K₂` we need to choose a vertex type for `K₂`. We will use `Bool`, whose two elements are `false` and `true`.

```
#source Bool
```

The vertex type of the n-dimensional hypercube (box product construction).

```
@[implicit_reducible]
def HypercubeVert : ℕ → Type
  | 0 => Unit                        -- The canonical type with exactly one element
  | n + 1 => HypercubeVert n × Bool -- The previous vertex type times Bool
```

(Note that `HypercubeVert n` is really just `Fin n → Bool` but defined so that the recursion holds definitionally. We can still easily take the Hamming distance viewpoint and we will equip it with the Hamming distance when needed in Hamming Paths (p. 211).)

The `@[implicit_reducible]` attribute lets Lean unfold `HypercubeVert (n + 1)` to `HypercubeVert n × Bool` when it checks that implicit arguments have the expected type, as `simp` does before rewriting. Instance search still treats `HypercubeVert` as opaque, so the instances we provide for `HypercubeVert n` in Computations with Hypercubes (p. 78) are still found.

```
@[simp]
lemma hypercubeVert_zero : HypercubeVert 0 = Unit := rfl

lemma hypercubeVert_succ (n : ℕ) :
    HypercubeVert (n + 1) = (HypercubeVert n × Bool) := rfl
```

The n-dimensional hypercube can now be defined recursively as the box product of the previous hypercube with `K₂`, the complete graph on two vertices (where `Bool` is the vertex type of `K₂`).

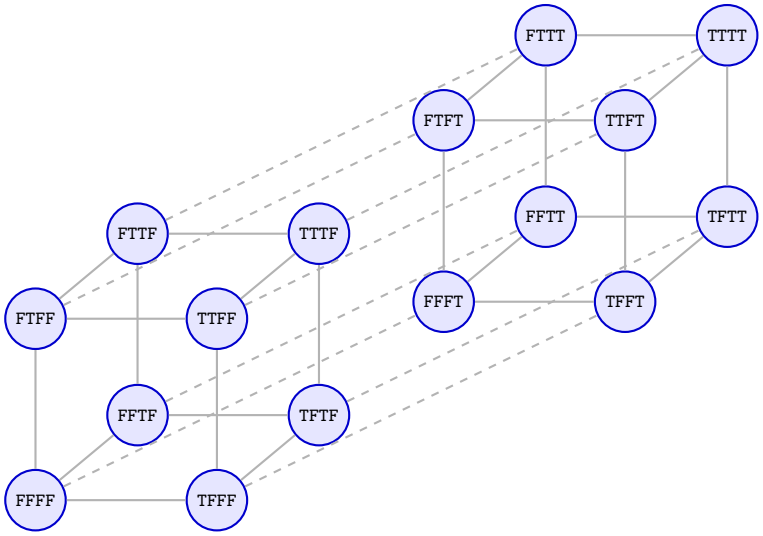
The n-dimensional hypercube defined as the box product of n copies of `K₂`.

```
def hypercube : (n : ℕ) → SimpleGraph (HypercubeVert n)
    -- The empty graph on one vertex
  | 0 => ⊥
    -- The box product of the n-hypercube with K₂ on Bool
  | n + 1 => (hypercube n).boxProd ⊤
```

We will use `Q[n]` as notation for the n-dimensional hypercube.

`Q[n]` is the n-dimensional hypercube `hypercube n`, the box product of n copies of `K₂`.

```
scoped notation "Q[" n "]" => hypercube n
```



The 4-dimensional hypercube `Q[4]`

```
@[simp]
lemma hypercube_zero : Q[0] = ⊥ := rfl

lemma hypercube_succ (n : ℕ) : Q[n + 1] = Q[n].boxProd ⊤ := rfl

@[simp]
lemma hypercube_succ_adj {n : ℕ} {u v : HypercubeVert (n + 1)} :
    Q[n + 1].Adj u v ↔ (Q[n].Adj u.1 v.1 ∧ u.2 = v.2) ∨ (u.2 ≠ v.2 ∧ u.1 = v.1) :=
  Iff.rfl
```

Both `HypercubeVert` and `hypercube` are defined by recursion on *n*, so above we recorded their defining equations as the lemmas `hypercubeVert_zero`, `hypercubeVert_succ`, `hypercube_zero` and `hypercube_succ`, together with `hypercube_succ_adj`, which spells out adjacency in `Q[n + 1]`. All five hold by `rfl`, but only the two `_zero` lemmas and `hypercube_succ_adj` are marked as `@[simp]`.

The `_zero` lemmas replace `HypercubeVert 0` and `Q[0]` by `Unit` and `⊥`, which `simp` already knows a lot about, so they can only make a goal simpler. For example, `simp` can prove `¬Q[0].Adj u v` by chaining `hypercube_zero` with `bot_adj`.

```
example (u v : HypercubeVert 0) : ¬Q[0].Adj u v := by simp
```

The `_succ` lemmas just unfold the definition by one step, and we leave them out of the `simp` set, as Mathlib does for recursive definitions (e.g. `pow_zero` is a `simp` lemma but `pow_succ` is not). Recall that `simp` simplifies the parts of a term before the term itself (The Simplifier (p. 17)), so if `hypercube_succ` were a `simp` lemma it would turn `Q[n + 1]` into `Q[n] □ ⊤` before any lemma about the hypercube got a chance to apply.

The exception is `hypercube_succ_adj`. It also unfolds a step, but only inside the proposition `Q[n + 1].Adj u v`, which `simp` can rewrite wherever it occurs, and it leaves the graph `Q[n + 1]` and its vertex type alone. Its right-hand side is what Mathlib's `simp` lemmas `boxProd_adj` and `top_adj` would give if `simp` could see that `Q[n + 1]` is a box product. With it, `simp` can check adjacency between explicit vertices of `Q[n]` one coordinate at a time.

3.5 Mycielskian

The *Mycielskian* of a graph `G : SimpleGraph V` is constructed as follows. Its vertex set consists of two disjoint copies V^l and V^r of `V` together with a new apex vertex which we call `none`.

The edges inside V^l are identical to those of `G`; there are no edges inside V^r; there is an edge between vertices $a \in V^l$ and $b \in V^r$ iff there is an edge in `G` between the vertices of which `a` and `b` are copies. Finally `none` is adjacent to every vertex in V^r but to no vertices in V^l.

This construction is due to Jan Mycielski [Mycielski 1955], who showed that:

1. if `G` is triangle-free, then `mμ(G)` is also triangle-free.
2. if `G` has chromatic number `k`, then `mμ(G)` has chromatic number `k + 1`.

Thus starting with any triangle-free graph we can use it to construct triangle-free graphs with arbitrarily large chromatic number.

In order to define the Mycielskian of a graph `G` in Lean we need to first describe its vertex type. Given that it is built from two disjoint copies of the vertex type of `G` together with a new vertex, we can use `Option (V ⊕ V)`. (We already saw the `Sum` type `⊕` for `completeBipartiteGraph` and `Option` is the natural way to extend a type with a new element called `none`.)

```
section mycielskian_section
variable {V : Type*}

#source Option
```

The *Mycielskian* of a graph *G*.

```
@[simps]
def mycielskian (G : SimpleGraph V) : SimpleGraph (Option (V ⊕ V)) where
  Adj x y := match x, y with
    | some (Sum.inl v), some (Sum.inl w) => G.Adj v w
    | some (Sum.inl v), some (Sum.inr w) => G.Adj v w
    | some (Sum.inr v), some (Sum.inl w) => G.Adj v w
    | none, some (Sum.inr w) => True
    | some (Sum.inr v), none => True
    | _, _ => False
  symm := ⟨by rintro (_ | _ | _) (_ | _ | _) h <;> simp_all [Adj.symm]⟩
  loopless := ⟨by rintro (_ | _ | _) <;> simp⟩
```

We introduce the notation `mμ(G)` for the Mycielskian of a graph *G*.

`mμ(G)` is the Mycielskian `mycielskian G` of *G*.

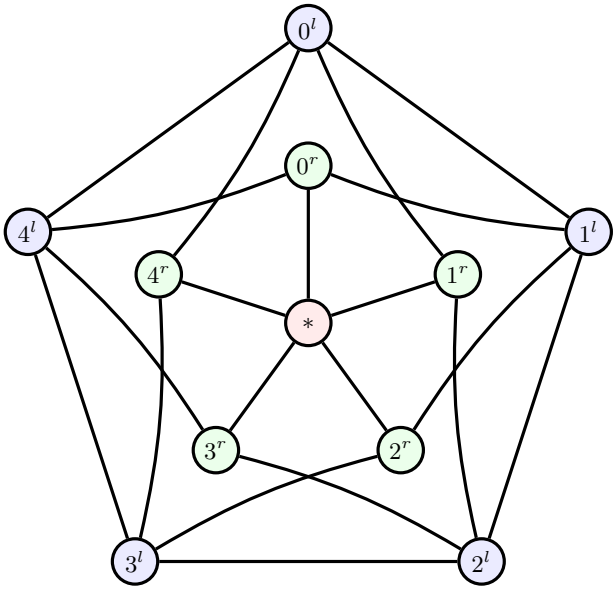
```
scoped notation "mμ(" G ")" => mycielskian G
```

As mentioned above if we iterate the Mycielskian construction starting with a triangle-free graph then we obtain a sequence of triangle-free graphs with increasing chromatic number.

In particular we can start with `K[2]`. Its Mycielskian is isomorphic to the 5-cycle `C[5]` (see Vertex-Transitivity of Mycielskians (p. 114)), and the Mycielskian of `C[5]` is called the *Grötzsch graph* [Grötzsch 1959].

The *Grötzsch graph* (i.e. the Mycielskian of `C[5]`).

```
abbrev grotzschGraph : SimpleGraph (Option (Fin 5 ⊕ Fin 5)) := mμ(C[5])
```



The grotzschGraph (i.e. the Mycielskian mμ(C[5]))

In order to talk about the vertices of mμ(G) it will be useful to introduce some terminology. We name the two copies of V in a mycielskian namespace, since these names only make sense for this construction. The @[match_pattern] attribute lets us use them (and the notation below) as patterns when defining a function by pattern matching, just like the constructors none, some, Sum.inl and Sum.inr.

```
namespace mycielskian
```

The left copy some (Sum.inl v) of v in Option (V ⊕ V), the vertex type of the Mycielskian.

```
@[match_pattern]
abbrev left (v : V) : Option (V ⊕ V) := some (Sum.inl v)
```

The right copy some (Sum.inr v) of v in Option (V ⊕ V), the vertex type of the Mycielskian.

```
@[match_pattern]
abbrev right (v : V) : Option (V ⊕ V) := some (Sum.inr v)

end mycielskian
```

We can introduce notation for these as follows. The left copy of $v : V$ is denoted v^l and the right copy is denoted v^r.

v^l is the left copy some (Sum.inl v) of v in Option (V ⊕ V), the vertex type of the Mycielskian.

```
scoped postfix:max "ˡ" => mycielskian.left
```

v^r is the right copy some (Sum.inr v) of v in Option (V ⊕ V), the vertex type of the Mycielskian.

```
scoped postfix:max "ʳ" => mycielskian.right
```

Lean prints `v✝¹` only for a term of the form `mycielskian.left v`. Tactics that split a vertex into cases, such as `rintro (_ | v | v)`, use the constructors themselves and produce `some (Sum.inl v)`, so without help the same vertex can appear in both forms in a single goal. The following *delaborator* (an extension of Lean's pretty-printer) prints `some (Sum.inl v)` as `v✝¹` and `some (Sum.inr v)` as `v✝`. Like the notation it is `scoped`, so it applies only when the `SimpleGraph` namespace is open.

Print `some (Sum.inl v)` as `v✝¹` and `some (Sum.inr v)` as `v✝`, provided the two summands of the `Sum` type are the same.

```
open Lean PrettyPrinter.Delaborator SubExpr in
@[scoped delab app.Option.some]
private def delabMycielskianVert : Delab := whenPPOption getPPNotation do
  -- the argument of some, which should be @Sum.inl V V v or @Sum.inr V V v
  let .app _ x ← getExpr | failure
  guard ◁ x.getAppNumArgs == 3 && x.getArg! 0 == x.getArg! 1
  let v ← withAppArg ◁ withAppArg delab
  if x.isAppOf ``Sum.inl then `($v✝¹)
  else if x.isAppOf ``Sum.inr then `($v✝)
  else failure
```

The `@[simps]` attribute on our definition of `mycielskian` generated a `@[simp]` lemma `mycielskian_adj`, which rewrites `mμ(G).Adj x y` to the `match` in its definition. Once both vertices are named, as in v^{l}, w^{r} or none, `simp` reduces the `match`, so it (and tactics such as `aesop` that call it) can decide any adjacency between named vertices.

```
example {G : SimpleGraph V} (v w : V) : mμ(G).Adj v✝¹ w✝ ↔ G.Adj v w := by simp

example {G : SimpleGraph V} (v w : V) : ¬mμ(G).Adj v✝ w✝ := by simp
```

We now prove a few lemmas about the Mycielskian construction that will be useful later on.

The maps $v \mapsto v^{\mathrm{l}}$ and $v \mapsto v^{\mathrm{r}}$ are injective. (In the proofs, `e |>.comp f` means `(e).comp f`: the `|>.` lets us use dot notation on the result of `Option.some_injective _` without extra brackets.)

```
lemma mycielskian.left_injective : Function.Injective fun v : V => v✝¹ :=
  Option.some_injective _ |>.comp Sum.inl_injective

lemma mycielskian.right_injective : Function.Injective fun v : V => v✝ :=
  Option.some_injective _ |>.comp Sum.inr_injective
```

The neighbourhoods of the vertices in `mμ(G)` are as follows.

A left copy v^{l} is adjacent to the left and right copies of the neighbours of v in G.

```
@[simp]
lemma neighborSet_mycielskian_left {G : SimpleGraph V} (v : V) :
    mμ(G).neighborSet v✝¹ = (G.neighborSet v).image (·✝¹) ∪
      (G.neighborSet v).image (·✝) := by
  ext x
  grind [mem_neighborSet, mycielskian_adj]
```

A right copy v^{r} is adjacent to the left copies of the neighbours of v in G and to none.

```
@[simp]
lemma neighborSet_mycielskian_right {G : SimpleGraph V} (v : V) :
```

```
    mμ(G).neighborSet v^r = insert none ((G.neighborSet v).image (·^l)) := by
  ext x
  grind [mem_neighborSet, mycielskian_adj]
```

The apex vertex none is adjacent to the right copies of all vertices in G.

```
@[simp]
lemma neighborSet_mycielskian_none {G : SimpleGraph V} :
    mμ(G).neighborSet none = Set.range (·^r) := by
  ext x
  grind [mem_neighborSet, mycielskian_adj]
```

The left and right copies are disjoint.

```
@[simp]
lemma mycielskian.disjoint_image_left_image_right (s t : Set V) :
    Disjoint (s.image (·^l)) (t.image (·^r)) := by
  grind

end mycielskian_section
```

As mentioned above we will be interested in iterating the Mycielskian construction, but in order to do this we first need to define the iterated vertex type. [1]

As with `HypercubeVert`, the `@[implicit_reducible]` attribute lets Lean unfold this definition when checking implicit and instance-implicit arguments.

The vertex type of the n-th Mycielskian of a `SimpleGraph V` (denoted `mμᵥ^n(V)`).

```
@[implicit_reducible]
def MycielskianVert (V : Type*) : ℕ → Type _
  | 0     => V
  | n + 1 => Option (MycielskianVert V n ⊕ MycielskianVert V n)

@[simp]
lemma mycielskianVert_zero {V : Type*} : MycielskianVert V 0 = V := rfl

lemma mycielskianVert_succ {V : Type*} (n : ℕ) :
    MycielskianVert V (n + 1) =
      Option (MycielskianVert V n ⊕ MycielskianVert V n) := rfl
```

We write `mμᵥ^n(V)` for `MycielskianVert V n`.

`mμᵥ^n(V)` is the vertex type of the n-th iterated Mycielskian of a `SimpleGraph V`.

```
scoped notation "mμᵥ^" n:max "(" V ")" => MycielskianVert V n
```

The n-th iterated Mycielskian of a graph G (denoted `mμ^n(G)`).

```
def iterMycielskian {V : Type*} (G : SimpleGraph V) : (n : ℕ) → SimpleGraph mμᵥ^n(V)
  | 0     => G
  | n + 1 => mμ(iterMycielskian G n)
```

[1] Below we use `Type _` which means that the universe level is inferred (it will be the same as the universe level of V).

```
@[simp]
lemma iterMycielskian_zero {V : Type*} (G : SimpleGraph V) :
    iterMycielskian G 0 = G := rfl

lemma iterMycielskian_succ {V : Type*} (G : SimpleGraph V) (n : ℕ) :
    iterMycielskian G (n + 1) = mμ(iterMycielskian G n) := rfl
```

Similarly, we write `mμ^n(G)` for `iterMycielskian G n`.

`mμ^n(G)` is the n-th iterated Mycielskian of G.

```
scoped notation "mμ^" n:max "(" G ")" => iterMycielskian G n

#check mμ^3(C[5])
```

The postfix operators ^{l} and ^{r} can be stacked: `(0 : Fin 5)`^{lrr} means $((0^{l})^{r})^{r}$, the right copy of the right copy of the left copy of `0`, which is a vertex of the 3rd iterated Mycielskian of `C[5]`. Written out in full this would be the unreadable term `some (Sum.inr (some (Sum.inr (some (Sum.inl 0)))))`. The adjacency lemmas above peel off one layer at a time: left–left and right–left pairs are adjacent exactly when the underlying vertices are, so `(0 : Fin 5)`^{lrr} is adjacent to `(1 : Fin 5)`^{lll} because `0` and `1` are adjacent in `C[5]`.

```
example : mμ^3(C[5]).Adj (0 : Fin 5)ˡʳʳ (1 : Fin 5)ˡˡˡ := by
  simpa [iterMycielskian_succ] using (by decide : C[5].Adj 0 1)
```

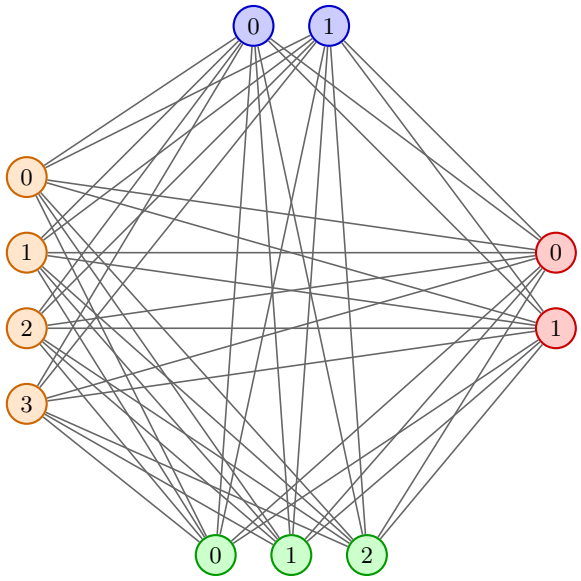
3.6 Complete Multipartite Graphs and Blow-ups

Having earlier introduced complete bipartite graphs we would now like to discuss complete multipartite graphs and their generalization: blow-up graphs. However these constructions make use of the `SimpleGraph.comap` construction which we won't formally introduce until Comaps (p. 99). In order to properly understand these constructions you may wish to read that short section now.

Complete Multipartite Graphs

A *complete multipartite* graph is a graph in which the vertices are partitioned into classes so that two vertices are adjacent if and only if they are in different classes. The definition in Mathlib looks rather forbidding at first glance:

```
#source completeMultipartiteGraph  -- (V : ι → Type*) : SimpleGraph (Σ i, V i)
                                   --             := SimpleGraph.comap Sigma.fst ⊤
```



A complete 4-partite graph

We start with an index type `ι` (the labels of the vertex classes) and a function `V : ι → Type*`. The vertex type of the graph is `Σ i, V i`, the dependent pair or `Sigma` type that we met in the definition of `SimpleGraph.sigma`.

```
#source Sigma
```

Recall that a vertex `v : Σ i, V i` is a pair `⟨i, w⟩` consisting of an index `i : ι` and an element `w : V i`; the class labelled by `i` is then the set of vertices whose first component is `i`.

The `Sigma` type plays the same role here that the `Sum` type played in `completeBipartiteGraph`. Just as a vertex of a sum type knows which of the two sides it came from, since it is either a `Sum.inl` or a `Sum.inr`, a vertex of `Σ i, V i` knows which class it lies in, since it carries its index. The two alternatives `Sum.inl` and `Sum.inr` have simply been replaced by one alternative for each `i : ι`.

```
#check Sigma.fst -- The first component of a dependent pair ⟨i, w⟩.
```

The `SimpleGraph.comap` then defines edges in the graph by pulling back the adjacency relation from `⊤ : SimpleGraph ι`, the complete graph on `ι`, using `Sigma.fst`. So vertices are adjacent if and only if their first components are different.

```
#source SimpleGraph.comap
```

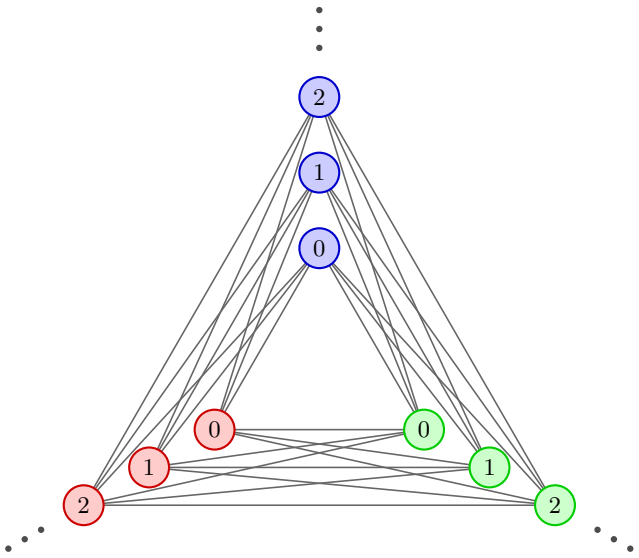
Adjacency in this graph is then exactly as we would expect: two vertices `⟨i, u⟩` and `⟨j, v⟩` are adjacent iff they belong to different classes, i.e. `i ≠ j`.

```
lemma completeMultipartiteGraph_adj {ι : Type*} (V : ι → Type*) (u v : Σ i, V i) :
    (completeMultipartiteGraph V).Adj u v ↔ u.fst ≠ v.fst := Iff.rfl
```

We give two concrete examples.

The complete multipartite graph with n classes, where each class is a copy of $\mathbb{N}$.

```
abbrev completeMultipartiteFinNat (n : ℕ) : SimpleGraph (Σ _ : Fin n, ℕ) :=
  completeMultipartiteGraph fun (_ : Fin n) => ℕ
```

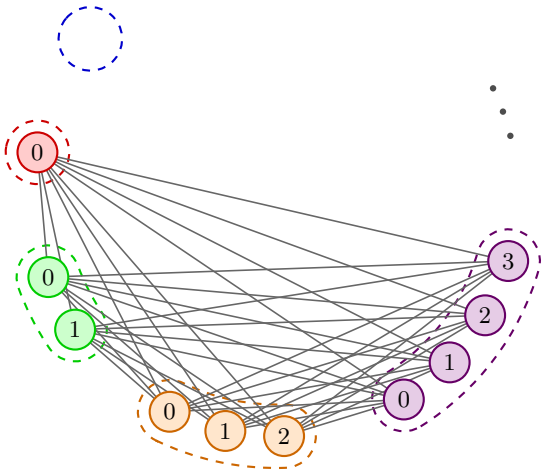


Part of `completeMultipartiteFinNat 3`

We can also define the complete multipartite graph with classes indexed by $\mathbb{N}$, where the n-th class is `Fin n`. (The names of these two graphs spell out their vertex types, `Σ _ : Fin n, ℕ` and `Σ n : ℕ, Fin n`.)

The complete multipartite graph with classes indexed by $\mathbb{N}$, where the n-th class is `Fin n`.

```
abbrev completeMultipartiteNatFin : SimpleGraph (Σ n : ℕ, Fin n) :=
  completeMultipartiteGraph fun n => Fin n
```



Part of `completeMultipartiteNatFin`

Deciding whether two vertices are adjacent in a complete multipartite graph is easy: they are adjacent if and only if their first components are different. So decidability of adjacency follows from decidability of equality on their index type. Since `Fin n` and $\mathbb{N}$ have decidable equality, `completeMultipartiteFinNat` and `completeMultipartiteNatFin` have decidable adjacency.

```
example : (completeMultipartiteFinNat 3).Adj ⟨0, 12⟩ ⟨2, 10⟩ := by decide
```

```
example : ¬completeMultipartiteNatFin.Adj ⟨3, 0⟩ ⟨3, 1⟩ := by decide
```

Blow-ups

While not currently in Mathlib we can define the *blow-up* of a graph `G`. Informally, this is the graph obtained from `G` by replacing each vertex with a class of vertices and defining edges between vertices in different classes according to the edges in `G`. (This is a trivial generalization of the `completeMultipartiteGraph` construction, which is the special case where `G` is a complete graph.)

The blow-up of a graph `G` given by replacing each vertex `i` with a class of vertices `V i`.

```
abbrev blowUpGraph {ι : Type*} (G : SimpleGraph ι) (V : ι → Type*) :
    SimpleGraph (Σ i, V i) := SimpleGraph.comap Sigma.fst G
```

The `n`-uniform blow-up of a graph `G` is the blow-up of `G` where each vertex is replaced by a class of size `n`.

The `n`-uniform blow-up of a graph `G` given by replacing each vertex `i` with `n` vertices (`Fin n`), denoted `G[n]`.

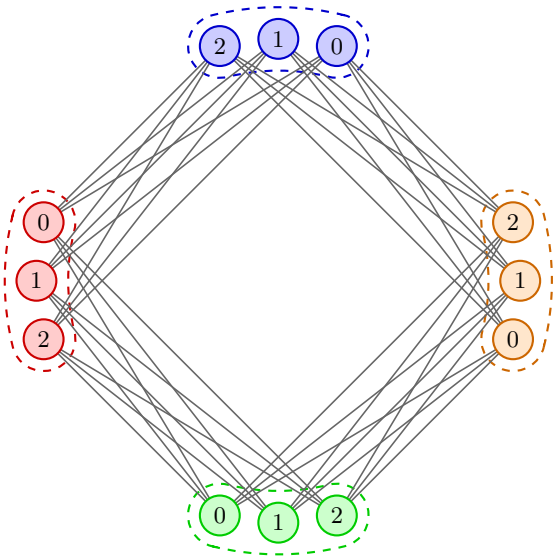
```
abbrev blowUpN {ι : Type*} (G : SimpleGraph ι) (n : ℕ) :
    SimpleGraph (Σ _ : ι, Fin n) := G.blowUpGraph fun _ => Fin n
```

We introduce the notation `G[n]` for the `n`-uniform blow-up of `G`.

`G[n]` is the `n`-uniform blow-up of `G`, which replaces each vertex of `G` with `n` vertices (`Fin n`).

```
scoped notation:max G "[" n "]" => blowUpN G n
```

```
#check C[4][3] -- the blow-up of C[4] replacing each vertex by a class of size 3
example : C[4][3].Adj ⟨1, 0⟩ ⟨2, 0⟩ := by decide
example : ¬C[4][3].Adj ⟨0, 0⟩ ⟨2, 1⟩ := by decide
```



`C[4][3]`: the 3-uniform blow-up of `C[4]`

3.7 Kneser Graphs*

The k-th Kneser graph [Kneser 1955] on a type β has as vertices the k-element subsets of β, with edges between any distinct disjoint subsets. Before we can define this as a `SimpleGraph` we need to decide what its vertex type should be. If we are happy to work with finite types (i.e. to assume `Fintype β`) then we could use `(Finset.univ : Finset β).powersetCard k` which is the finite family of k-element subsets of a finite type β.

```
#check Finset.powersetCard
```

Using `Finset.powersetCard` gives us easy access to its API consisting of counting results about families of finite sets, which would be useful.

```
@[simps]
private def kneserGraphFintype (β : Type*) [Fintype β] (k : ℕ) :
    SimpleGraph ((Finset.univ : Finset β).powersetCard k) where
  Adj u v := Disjoint u.val v.val ∧ u ≠ v
  symm := ⟨fun _ _ h => ⟨h.1.symm, h.2.symm⟩⟩
```

But there are two drawbacks with this definition. Firstly, the finiteness assumption is a bit restrictive. Secondly, this vertex type mentions `Finset.univ`, and hence the `Fintype β` instance: two different instances give vertex types that are equal but not definitionally equal, so a lemma proved for one does not apply to the other. (Since we won't actually use this definition we marked it `private`.)

Instead we take the vertex type to be the subtype of `Finset β` consisting of k-element subsets, i.e. `{s : Finset β // s.card = k}`. This means we no longer need to assume `Fintype β` but we lose direct access to the `Finset.powersetCard` API (even when β is finite). However we can solve this issue with a simple translation lemma saying that, for any predicate `p`, counting the k-element subsets satisfying `p` as terms of the subtype `{s : Finset β // s.card = k}` gives the same answer as counting them as members of the family `(Finset.univ : Finset β).powersetCard k`. (See Independence Number of the Kneser Graph (p. 137) for details.)

The k-th Kneser graph on β: vertices are k-element subsets of β, and edges join disjoint, distinct subsets.

```
@[simps]
def kneserGraph (β : Type*) (k : ℕ) :
    SimpleGraph {s : Finset β // s.card = k} where
  Adj u v := Disjoint u.val v.val ∧ u ≠ v
  symm := ⟨fun _ _ h => ⟨h.1.symm, h.2.symm⟩⟩

#check kneserGraph_adj -- (kneserGraph β k).Adj u v = (Disjoint ↑u ↑v ∧ u ≠ v)

instance (β : Type*) (k : ℕ) [DecidableEq β] :
    DecidableRel (kneserGraph β k).Adj := fun u v => by
  unfold kneserGraph; infer_instance
```

We introduce the notation `Kn[n, k]` for the k-th Kneser graph on `Fin n`.

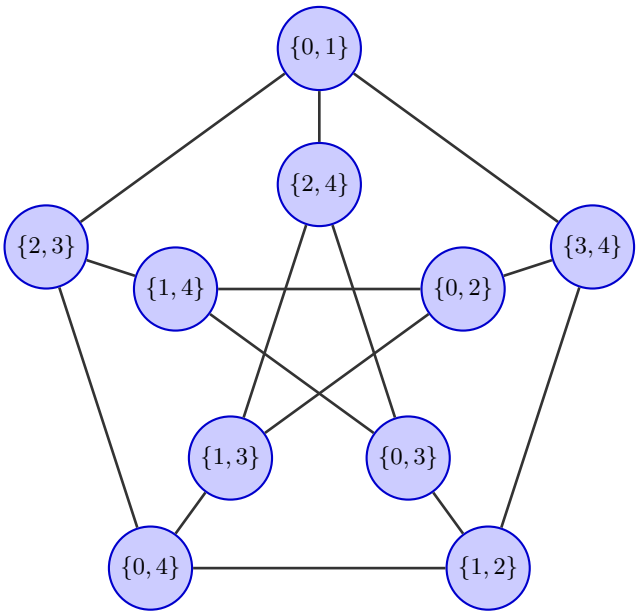
`Kn[n, k]` is the Kneser graph `kneserGraph (Fin n) k`: its vertices are the k-element subsets of `Fin n`, and edges join disjoint, distinct subsets.

```
scoped notation "Kn[" n ", " k "]" => kneserGraph (Fin n) k
```

A special case of the Kneser graph is the *Petersen* graph [Petersen 1898], which is `Kn[5, 2]`.

The *Petersen* graph which is the Kneser graph `Kn[5, 2]`.

```
abbrev petersenGraph : SimpleGraph {s : Finset (Fin 5) // s.card = 2} := Kn[5, 2]
```



The `petersenGraph` as the Kneser graph `Kn[5, 2]`

We will look at the independence number of `Kn[n, k]` in Cliques (p. 127) and colourings of `Kn[n, k]` in Exercises Colorings (p. 184). Computing its independence number is really just a case of translating the Erdős–Ko–Rado theorem (which is in Mathlib) into the language of `SimpleGraph`.

3.8 Exercises

1. Prove that the cycle graph `C[3]` is the complete graph on `Fin 3`. (Mathlib has this as `cycleGraph_three_eq_top`, but try to prove it directly.)

```
example : C[3] = K[3] := by
  sorry
```

2. Prove that the cycle graph `C[n + 2]` is `circulantGraph {1}`. (Mathlib has this, for `C[n + 1]`, as `cycleGraph_eq_circulantGraph`, but prove it directly.)

```
example (n : ℕ) : C[n + 2] = circulantGraph {1} := by
  sorry
```

3. Show that the Paley graph on `17` vertices is a circulant graph with the following differences. Hint: first show that the set of squares in `ZMod 17` is `{0, 1, 2, 4, 8, 9, 13, 15, 16}`. After `ext` the goal mentions a new variable of type `ZMod 17`, but `decide` only works on statements with no free variables. Instead use `decide +revert`, which first reverts the variable and then checks the resulting statement `∀ y : ZMod 17, …` case by case. Finally, `circulantGraph_eq_erase_zero` lets you drop `0` from the set of squares.

```
lemma paleyGraph₁₇_eq_circulant' :
    paleyGraph (ZMod 17) = circulantGraph {1, 2, 4, 8, 9, 13, 15, 16} := by
  sorry
```

4. Show that a Cayley graph defined on a commutative additive group is simply a circulant graph. Hint: `addCayley_adj'`, `circulantGraph_adj` and `add_sub_cancel_left` could be useful.

```
example (G : Type*) [AddCommGroup G] (s : Set G) :
    addCayley s = circulantGraph s := by
  sorry
```

5. Prove that the union (i.e. sup ⊔) of the complete bipartite graph between V and W with the disjoint union ⊕g (i.e. `sum`) of complete graphs on V and W is the complete graph on $V \oplus W$.

```
example {V W : Type*} : completeBipartiteGraph V W ⊔ (⊤ ⊕g ⊤) = ⊤ := by
  sorry
```

6. Prove that the disjoint union of empty graphs is the empty graph.

```
example {ι : Type*} {V : ι → Type*} : Σg (fun i => (⊥ : SimpleGraph (V i))) = ⊥ := by
  sorry
```

7. Prove that every left copy of a vertex is isolated in the Mycielskian of the empty graph. (So `mμ(⊥)` is a star with centre `none` whose leaves are the right copies, together with the isolated left copies.)

```
example {V : Type*} (u : V) : mμ(⊥).IsIsolated u.l := by
  sorry
```

8. Prove that the complete 1-partite graph with a single class $\mathbb{N}$ is the empty graph. Hint: if i : `Fin 1` then `fin_cases i` splits the goal into one case for each possible value of i.

```
example : completeMultipartiteFinNat 1 = ⊥ := by
  sorry
```

9. Prove that `sigmaCompleteNat` is the complement of the complete multipartite graph `completeMultipartiteNatFin`. Hint: after `ext ⟨i, a⟩ ⟨j, b⟩`, consider the cases `i = j` and `i ≠ j` separately. In the first case `subst` the equation, so that `a` and `b` have the same type.

```
example : sigmaCompleteNat = completeMultipartiteNatFinᶜ := by
  sorry
```

4 Decidability, Finiteness and Degrees

This chapter is about *computing* with graphs in Lean, and about the type class machinery that makes computation possible. We begin with decidability: the contrast between `Bool` and `Prop`, the `Decidable` class, the commands and tactics that use it (`#eval`, `decide` and `native_decide`), and the `DecidableRel G.Adj` instances required to decide adjacency. We then turn to finiteness, starting from the hierarchy `List` → `Multiset` → `Finset` and arriving at `Fintype`, which is *data*, and `Finite`, which is a proposition. Along the way we compare the ways Mathlib has of counting the elements of a finite type or set. A point we return to more than once is that because `Fintype` instances are data, the same type can receive two different instances by two different routes of type class inference. This is known as a *diamond*, after the shape of the two routes meeting at one type. Such a diamond is a nuisance rather than a feature: the two instances are always equal, but not necessarily *by definition*, so a theorem stated using one instance will not apply to a goal that uses the other. We will see how to deal with such a mismatch when it arises, and how to state theorems so that it does not arise in the first place.

The second half applies all this to graphs. We study finite neighbourhoods and vertex degrees, including minimum degree (`δ(G)`) and maximum degree (`Δ(G)`), the handshake lemma (`sum_degrees_eq_twice_card_edges`) and regular graphs. We then do computations with two families of graphs, hypercubes and Mycielskians, building the `DecidableRel` and `Fintype` instances needed both to evaluate and to prove facts about their degrees and edge counts.

Finally, an advanced section on degree sequences defines the degree multiset of a graph and asks which multisets are *graphic* (`Graphic`). It states the Erdős–Gallai (`erdos_gallai`) and Havel–Hakimi (`havel_hakimi`) characterizations without proof, and uses Havel–Hakimi to give a function (`havelHakimiEdges`) that computes the edges of a graph with a given graphic degree sequence.

4.1 Decidability

In classical logic the *law of the excluded middle* says that every proposition is either true or false. For proofs this is very useful: it allows us to reason by cases. In one branch of a proof we assume `p` holds and in the other we assume `¬p` holds. But for computations this isn't very helpful: we want to know *which* branch to take.

Bool and Prop

Lean has two different types that can play the role of truth-values. The first is `Bool`, the ordinary inductive type with two elements `true` and `false`.

```
#source Bool
```

The second is `Prop`, the type of propositions: a term `p : Prop` is a statement, and a term `h : p` is a proof of that statement.

```
#check Prop -- Prop : Type
```

Although `Prop` has many terms, such as `1 = 1`, `2 + 2 = 5` and Goldbach's conjecture, Lean treats logically equivalent propositions as equal (`propext`), and classically every `p : Prop` is equal to either `True` or `False` (`Classical.propComplete`). So, up to equality, `Prop` also has just two values. The difference is that a `Bool` is a value that a program can compute and branch on, while a `Prop` is a statement that we may or may not be able to prove.

For `Bool`, Lean provides the usual Boolean operations `&&` (and), `||` (or) and `!` (not), each defined simply by matching on `true` and `false`, and `bif b then x else y` returns `x` or `y` according to the value of `b`. Combining these gives expressions that `#eval` can evaluate.

```
#eval bif !false && (false || true) then "apples" else "pears" -- "apples"
```

Lean has other `Bool`-valued functions such as `Nat.beq m n`, which checks whether `m = n` by recursion on $\mathbb{N}$. We can use it to write a parity test.

Return `true` if `n` is even, `false` otherwise.

```
private def myEven (n : ℕ) : Bool := Nat.beq (n % 2) 0

#eval myEven 42 -- true
#eval myEven 123 -- false
```

Compare this with Mathlib's `Even`, which is a predicate: `Even n` is the statement that `n = r + r` for some `r`. Nothing in this definition tells Lean how to find `r`, or how to show that no such `r` exists.

```
#print Even -- def Even : {α : Type u_2} → [Add α] → α → Prop :=
            --   fun {α} [Add α] a ↦ ∃ r, a = r + r
```

Mathlib states almost everything using `Prop`, since propositions can be quantified over, rewritten with, and proved by any argument at all, constructive or not.

Going from `Bool` to `Prop` is easy: Lean coerces `b : Bool` to the proposition `b = true`.

```
#check fun b : Bool => (b : Prop) -- fun b ↦ b = true : Bool → Prop
```

Going from `Prop` to `Bool` is the hard direction, and this is exactly what the decidability type class provides.

The Decidable Class

To compute with a proposition we need to turn it into a `Bool`, that is, to *decide* whether it is true or false. In computability theory, decidability is a property of a *family* of propositions, such as "`n` is prime" for `n : ℕ`. The family is *decidable* if there is a single algorithm that, given `n`, determines whether the proposition holds. For one fixed proposition this notion is vacuous, since one of the two algorithms "answer true" and "answer false" is always correct, even if we do not know which.

In Lean, decidability is expressed by the type class `Decidable`. An instance of `Decidable p` is not the claim that `p` is decidable but a piece of *data*: a Boolean `decide p` giving the answer, packaged with a proof that this answer is correct.

```
#source Decidable -- class Decidable (p : Prop) where
                  --   intro ::
                  --   decide : Bool
                  --   reflects_decide (p) : decide.Reflects p
```

The proof field uses `Bool.Reflects`, which is defined so that `true.Reflects p` is p and `false.Reflects p` is $\neg p$. So an instance of `Decidable p` is either $\langle$`true, h`$\rangle$ with $h : p$, or $\langle$`false, h`$\rangle$ with $h : \neg p$, and Lean names these two as `Decidable.isTrue h` and `Decidable.isFalse h`. Proofs are erased when code is compiled, so evaluating an instance amounts to computing the Boolean `decide p`, which tells us whether p is true or false.

So how do we provide such instances? Given a proof of p or $\neg p$ we can construct a `Decidable p` instance directly:

```
example {p : Prop} (h : p) : Decidable p := ⟨true, h⟩
example {p : Prop} (h : ¬p) : Decidable p := ⟨false, h⟩
example {p : Prop} (h : p) : Decidable p := isTrue h
```

Instances like these do no computation of their own: they simply record an answer we already know. Useful `Decidable` instances instead contain an algorithm that computes the answer. Lean can find such an instance automatically for many propositions, such as $a \neq b$ for $a\ b : \mathbb{N}$, but not for all: it has none for Goldbach's conjecture, and for propositions involving our own definitions it may need some help.

`Decidable` is a *class*: a structure whose values, called *instances*, Lean finds for us. Each **`instance`** declaration, in Mathlib or in our own files, registers a way of building one, possibly from other instances. When a definition or theorem has an argument in square brackets, an *instance argument* such as the `[Add α]` in `Even` above, the caller does not supply it. Lean finds it by *type class inference*: it searches the hypotheses in the local context first and then the registered instances, combining instances as needed. (We also say that Lean *synthesizes* the instance, and call this process *instance search*.) The term `inferInstance` and the tactic **`infer_instance`** both ask for an instance in the same way.

```
private def natNeDec (a b : ℕ) : Decidable (a ≠ b) := inferInstance

private def natEvenDec (m : ℕ) : Decidable (Even m) := by infer_instance
```

Using **`#print`** with `set_option pp.explicit true` below we can see exactly which instance Lean has synthesized. For `Decidable (a ≠ b)`, Lean uses the instance `instDecidableNot` to infer `Decidable (a ≠ b)` from the instance of decidable equality on $\mathbb{N}$, `DecidableEq ℕ`, provided by `instDecidableEqNat`.

```
#check instDecidableNot -- {p : Prop} [dp : Decidable p] : Decidable ¬p
#check instDecidableEqNat -- DecidableEq ℕ

set_option pp.explicit true in
#print natNeDec -- @inferInstance (Decidable (@Ne Nat a b))
                --      (@instDecidableNot (@Eq Nat a b) (instDecidableEqNat a b))
```

For `Decidable (Even m)` it uses the instance `Nat.instDecidablePredEven`, which says that `Even` $: \mathbb{N} \to$ `Prop` is a decidable predicate (i.e. `Even m` is decidable for every $m : \mathbb{N}$).

```
#check Nat.instDecidablePredEven -- DecidablePred Even
```

Both examples are really families of instances: the arguments `(a b : ℕ)` and `(m : ℕ)` make `natNeDec` and `natEvenDec` functions that produce a `Decidable` instance for every choice of these arguments.

Once the arguments are supplied, the Boolean `decide` in each of these instances evaluates to `true` or `false`. This allows us to use `Decidable` instances to perform computations that depend on the truth-value of a proposition.

For example, the following function takes a decidable proposition *p*, computes the Boolean `decide p` from its `Decidable p` instance, and branches on it with **`bif`** to return a string telling us the truth-value of *p*.

Given a decidable proposition *p*, return a string indicating its truth-value.

```
def myIfThenElse (p : Prop) [Decidable p] : String :=
  bif decide p then "This is true" else "This is false"


#eval myIfThenElse (Even 42) -- "This is true"
#eval myIfThenElse (42 ≠ 42) -- "This is false"
example : myIfThenElse (12 + 12 ≠ 24) = "This is false" := by decide
```

In fact `if-then-else` conditionals are defined in almost exactly this way in Lean, using the `ite` function: type class inference synthesizes the `Decidable c` instance, and `ite` branches on its Boolean `decide c`.

```
#check ite -- {α : Sort u} → (c : Prop) → [h : Decidable c] → α → α → α
```

For comparison, the law of the excluded middle gives *every* proposition a `Decidable` instance, `Classical.propDecidable`. It is built from `Classical.em` using the axiom of choice, so it asserts that one of `isTrue h` or `isFalse h` exists without saying which: it contains no algorithm and cannot be evaluated.

```
#check Classical.propDecidable -- (a : Prop) : Decidable a
```

The **`classical`** tactic makes this instance available for the rest of a proof or definition, but only as a fallback: if type class inference finds a genuine `Decidable p` instance, that is used instead. In `myClassicalIfThenElse` below, *p* is an arbitrary proposition with no `Decidable p` instance, so the `if-then-else` uses `Classical.propDecidable`, and Lean refuses to compile the definition unless we mark it **`noncomputable`**.

Given a proposition *p*, return a string indicating its truth-value.

```
noncomputable def myClassicalIfThenElse (p : Prop) : String := by
  classical
  exact if p then "This is true" else "This is false"
```

Printing the definition with explicit arguments shows an ordinary call to `ite`, in which **`classical`** has supplied `Classical.propDecidable p` as the `Decidable p` instance.

```
set_option pp.explicit true in
#print myClassicalIfThenElse -- fun p ↦ @ite String p (Classical.propDecidable p) ..
```

Classical instances are not always avoidable: sometimes no algorithm exists. Equality of real numbers is an obvious example. No algorithm can decide, for an arbitrary pair of reals `x` and `y`, whether `x = y`, since it can only examine finitely many rational approximations to each, and these may agree to every precision checked even though `x ≠ y`.

Mathlib still provides an instance of `DecidableEq ℝ`, but it is built using classical reasoning and so is noncomputable. It can be used in proofs but not in computations.

```
#synth DecidableEq ℝ -- Real.decidableEq

/- error: failed to compile definition, consider marking it as 'noncomputable'
because it depends on 'Real.decidableEq', which is 'noncomputable'
-/
#eval decide ((1 : ℝ) ≠ 2)
```

The families of instances we saw for `natNeDec` and `natEvenDec` are typical: an algorithm in the computability sense corresponds to a family of `Decidable` instances, one for each input. Lean has names for the two most common ones. For a predicate `p : α → Prop`, `DecidablePred p` provides a `Decidable (p x)` instance for every `x : α`.

```
#source DecidablePred -- abbrev DecidablePred {α : Sort u} (r : α → Prop) :=
                      --   (a : α) → Decidable (r a)
```

For example, the predicate `Nat.Prime : ℕ → Prop`, which expresses whether a number is prime, is decidable.

```
#synth DecidablePred (fun n : ℕ => n.Prime) -- Nat.decidablePrime
#eval Nat.Prime 337031327       -- true
#eval Nat.Prime (337031327 + 1) -- false
```

We also have `DecidableRel r` for a relation `r : α → β → Prop`. This says that `r a b` is decidable for every `a : α` and `b : β`, or more precisely, it provides an instance of `Decidable (r a b)` for every `a : α` and `b : β`.

```
#source DecidableRel  -- DecidableRel {α : Sort u} {β : Sort v} (r : α → β → Prop) :=
                      --            (a : α) → (b : β) → Decidable (r a b)
```

This last example is particularly relevant for graphs, since the adjacency relation of a graph is a relation on the vertex set. For a graph `G : SimpleGraph V`, ensuring that we have an instance of `DecidableRel G.Adj` allows us to perform computations that depend on the truth-value of adjacency.

```
#synth DecidableRel (fun a b : ℕ => a = b)  -- instDecidableEqNat
#synth DecidableRel (⊤ : SimpleGraph ℕ).Adj -- fun a b ↦ Top.adjDecidable ℕ a b
```

Evaluating and Deciding

We have already used `#eval` and `decide` informally above and in earlier chapters. So what do they do and how are they related to decidability?

Of the two `#eval` is the more powerful. It *runs* a program: the expression is compiled and then executed, and the resulting value is printed. Nothing is proved (the result never reaches Lean's

kernel), but any value can be computed, and usually quickly. So long as the output of the program is a value that can be printed, `#eval` will work.

For example, `#eval` can decide whether a pair of vertices is adjacent in the 5-cycle. To do this it uses `instDecidableRelFinAdjCycleGraph`, the instance of `DecidableRel C[n].Adj` from Mathlib. The line `set_option trace.Meta.synthInstance true in` below asks Lean to report how it searched for the instances it needed: expand the trace attached to the `#eval` to see this. Tracing can be useful when instance search fails, or finds an instance you did not expect.

```
set_option trace.Meta.synthInstance true in
#eval C[5].Adj 0 2            -- false
#check instDecidableRelFinAdjCycleGraph -- (n : ℕ) : DecidableRel C[n].Adj
```

The `decide` tactic instead *proves* a decidable proposition `p`: it asks the kernel to reduce the Boolean `decide p` to `true`, and if this succeeds the proof is `of_decide_eq_true rfl`. The result is a genuine proof, checked by the same small trusted kernel as every other proof in Lean. The price is that kernel reduction is much slower than running compiled code, so `decide` is only suitable for small computations. Since `decide` is *proving* a proposition we can only use it to prove true statements, so our previous example using `#eval` has to be negated to be used with `decide`. Note that the `decide` tactic still uses the same instance of `DecidableRel C[n].Adj` as `#eval` to decide adjacency.

```
set_option trace.Meta.synthInstance true in
example : ¬C[5].Adj 0 2 := by decide
```

We will introduce `Fintype` instances formally later in this chapter. For now it is enough to know that `K[n].edgeFinset` is a computable finite set of the edges of the complete graph `K[n]`, and `K[n].edgeFinset.card` is its cardinality, the number of edges in `K[n]`.

Again `#eval` can compute this quickly.

```
#eval K[1000].edgeFinset.card -- 499500
```

We can also use `decide` to prove that the number of edges in `K[10]` is `45`, but it is much slower and if we try to use `decide` to prove that the number of edges in `K[1000]` is `499500` then it will complain.

```
example : K[10].edgeFinset.card = 45 := by decide


/- maximum recursion depth has been reached use set_option maxRecDepth <num> to
increase limit use set_option diagnostics true to get diagnostic information
-/
example : K[1000].edgeFinset.card = 499500 := by decide
```

In between `#eval` and `decide` sits the `native_decide` tactic, which proves a proposition by *running* the compiled decision procedure and trusting the answer. It should be as fast as `#eval`, but any proof using it then depends on the correctness of the entire Lean compiler, not just the kernel, so it is banned in Mathlib. (This project enables Mathlib's standard set of linters, which warns if you use it; a project without those linters gives no warning.) For a full account of what is trusted when checking a proof see Validating a Lean Proof[1] in the Lean Language Reference.

```
example : K[1000].edgeFinset.card = 499500 := by
  native_decide
```

[1] https://lean-lang.org/doc/reference/latest/ValidatingProofs/

The following table summarizes the differences. (The final row previews a subtlety that we will meet in Decidability and Kernel Reduction (p. 57) below.)

Command/tactic	`#eval`	`decide`	`native_decide`
What it is	command that runs code	tactic that builds a proof	tactic that builds a proof
Evaluated by	compiled code	kernel reduction	compiled code
Result	a printed value	a kernel-checked proof	a proof trusted as an axiom
Trusts	the entire compiler	only the kernel	the entire compiler
Speed	fast	slow (small cases only)	fast
Stuck by `Eq.rec` casts	no (proofs are erased)	yes	no

All three of `#eval`, `decide`, and `native_decide` rely on having computable `Decidable` instances. For graphs this means having an instance of `DecidableRel G.Adj`, allowing us to compute adjacencies in `G`.

Deciding Adjacency

Where does an instance of `DecidableRel G.Adj` used by `#eval` and `decide` come from? Usually we would like Lean to synthesize these automatically by type class inference.

To see which instance is synthesized for a particular graph we can use the `#synth` command:

```
#synth DecidableRel K[5].Adj -- fun a b ↦ Top.adjDecidable (Fin 5) a b
#check SimpleGraph.Top.adjDecidable -- the instance requires [DecidableEq V]
```

So the instance of decidable adjacency in the complete graph requires an instance of `DecidableEq` for its vertex type. This is natural: any two vertices in a complete graph are adjacent if and only if they are distinct. So deciding adjacency in a complete graph is the same as deciding equality of vertices.

Note that for the empty graph (on any type) we always have decidability of the adjacency relation, since no vertices are adjacent. This holds even on $\mathbb{R}$, where equality itself is undecidable.

```
#synth DecidableRel (⊥ : SimpleGraph ℝ).Adj
#check SimpleGraph.Bot.adjDecidable -- no assumption on V is required
```

If we revisit the three definitions `pathNat`, `pathNatFromRel` and `pathNatFromEdgeSet` of the path graph on $\mathbb{N}$ we can ask two obvious questions: do they have decidable adjacency and if so will Lean automatically synthesize the required instance in each case?

The answer to the first question should be independent of which definition we use: as we already proved they all define the same graph! So what would a method for deciding adjacency in this graph look like? Given `a : ℕ` and `b : ℕ` we need to check if `a + 1 = b` or `b + 1 = a` holds. Since $\mathbb{N}$ has decidable equality this is clearly decidable.

Turning to the second question of whether or not Lean will automatically synthesize the required instance we see that the exact form of our definition matters. It turns out that Lean can only infer the required instance for `pathNat`.

```
#source pathNat -- abbrev pathNat : SimpleGraph ℕ where
                --   Adj x y := x + 1 = y ∨ y + 1 = x
#synth DecidableRel pathNat.Adj -- fun a b ↦ instDecidableOr
```

Since pathNat is an **abbrev** Lean can see through its definition to find pathNat.Adj x y := x + 1 = y ∨ y + 1 = x. Since Or is decidable and each of the propositions x + 1 = y and y + 1 = x is decidable (using decidable equality on ℕ) Lean can infer decidability of adjacency in pathNat.

For pathNatFromRel our definition is a **def** rather than an **abbrev**.

```
#source pathNatFromRel -- def pathNatFromRel : SimpleGraph ℕ :=
                       --   SimpleGraph.fromRel (· + 1 = ·)
```

Lean can't see through the **def** so fails to infer the required instance.

```
/- error: failed to synthesize
 DecidableRel pathNatFromRel.Adj
-/
#synth DecidableRel pathNatFromRel.Adj -- fails
```

If we change the **def** to an **abbrev** then it works!

```
abbrev pathNatFromRelAbbrev : SimpleGraph ℕ := SimpleGraph.fromRel (· + 1 = ·)
#synth DecidableRel pathNatFromRelAbbrev.Adj
  -- .. instDecidableRelAdjFromRelOfDecidableEq
```

Lean sees through the **abbrev** and finds pathNatFromRelAbbrev := SimpleGraph.fromRel (· + 1 = ·). It then finds the instance instDecidableRelAdjFromRelOfDecidableEq which is the instance of decidable adjacency for a graph defined from a decidable relation using the SimpleGraph.fromRel constructor. In our case the relation is (· + 1 = ·), i.e. **fun** *x y* => *x* + 1 = *y* whose decidability can be inferred from the decidable equality of ℕ.

Rather than changing the definition of pathNatFromRel to an **abbrev** we can also provide a decidability instance for pathNatFromRel.Adj directly, in three different ways. One way is by **unfold**ing its definition and then calling **infer_instance**. The second way is to use inferInstanceAs to specify the type of the instance we want to find. In the latter case we know that Mathlib contains an instance of DecidableRel for the adjacency relation of a graph defined using SimpleGraph.fromRel so we tell it to use this. We write both of these as **example**s, since we only want one instance for pathNatFromRel.Adj.

```
example : DecidableRel pathNatFromRel.Adj := by
  unfold pathNatFromRel
  -- goal is now ⊢ DecidableRel (fromRel fun x1 x2 ↦ x1 + 1 = x2).Adj
  -- so Lean can use the instance instDecidableRelAdjFromRelOfDecidableEq
  infer_instance


example : DecidableRel pathNatFromRel.Adj :=
  inferInstanceAs (DecidableRel (SimpleGraph.fromRel _).Adj)
```

The third way, and the one we use, is to let Lean write the instance for us with the deriving instance command. This unfolds pathNatFromRel.Adj until type class inference finds an instance, much as we did by hand above. The parentheses are needed because Lean reads pathNatFromRel.Adj as a single name, and when the target of deriving instance is just a name it looks for a declaration with that name; there is no declaration called pathNatFromRel.Adj. In parentheses it is read as an ordinary expression, meaning SimpleGraph.Adj pathNatFromRel.

```
deriving instance DecidableRel for (pathNatFromRel.Adj)
```

```
example : pathNatFromRel.Adj 3 4 := by decide
```

When the value of a `def` is a `Prop`, we can instead write `deriving Decidable` at the end of the definition itself, and Lean builds the instance in the same way. This only works if the definition already takes every instance needed to decide its value; otherwise we can use `deriving instance` as above, supplying the missing instances as section variables.

Finally we have `pathNatFromEdgeSet` which was defined using `SimpleGraph.fromEdgeSet`. Lean again fails to infer the required instance (despite this being an `abbrev`).

```
#source pathNatFromEdgeSet -- abbrev pathNatFromEdgeSet : SimpleGraph ℕ :=
                           --   SimpleGraph.fromEdgeSet {e | ∃ n, e = s(n, n + 1)}


/- error: failed to synthesize
 DecidableRel pathNatFromEdgeSet.Adj
-/
#synth DecidableRel pathNatFromEdgeSet.Adj
```

The problem here is that although Mathlib contains an instance for the decidability of adjacency in any graph built by the `SimpleGraph.fromEdgeSet` constructor, this instance requires that membership of the defining edge set be decidable. But Lean doesn't know how to decide whether or not a potential edge `s(a, b)` is in the edge set `{e | ∃ n, e = s(n, n + 1)}`.

```
#check instDecidableRelAdjFromEdgeSetOfDecidablePredSym2MemSetOfDecidableEq
          -- .. (s : Set (Sym2 V)) → [DecidablePred fun x ↦ x ∈ s]
          --     → [DecidableEq V] → DecidableRel (fromEdgeSet s).Adj
```

One solution in this case is to use the fact, which we already proved, that adjacency in `pathNatFromEdgeSet` is the same as in `pathNat` and that Lean can infer decidable adjacency of `pathNat`.

```
#check pathNat_adj_iff_pathNatFromEdgeSet
```

Here we transfer the decidability instance from `pathNat` to `pathNatFromEdgeSet` using `decidable_of_iff`. [2]

```
instance : DecidableRel pathNatFromEdgeSet.Adj := fun u v =>
  decidable_of_iff (pathNat.Adj u v) pathNat_adj_iff_pathNatFromEdgeSet


example : pathNatFromEdgeSet.Adj 42 43 := by decide
```

Decidability and Kernel Reduction*

We already provided decidable adjacency instances for several of the graphs constructed in Constructions (p. 23). We now provide two more, for the box product `G □ H` and for the single-edge graph `edge s t` (replacing Mathlib's instance), and use them to explain why `rw` can be harmless or harmful inside a `Decidable` instance.

[2] Defining it by rewriting directly with `pathNat_adj_iff_pathNatFromEdgeSet` would also type-check and work with `#eval`, but it would leave `decide` permanently stuck: see Decidability and Kernel Reduction (p. 57) for an explanation.

In order to decide if `(g₁, h₁)` is adjacent to `(g₂, h₂)` in `G □ H` we need to check if either `g₁ = g₂` and `H.Adj h₁ h₂`, or `h₁ = h₂` and `G.Adj g₁ g₂`. So we need the obvious decidability assumptions on the vertex types of `G` and `H` as well as their adjacency relations.

Decidability of the adjacency relation for the box product of two graphs.

```
instance boxProdDecidableRelAdj {V W : Type*} {G : SimpleGraph V} {H : SimpleGraph W}
    [DecidableEq V] [DecidableEq W] [DecidableRel G.Adj] [DecidableRel H.Adj] :
    DecidableRel (G □ H).Adj := by
  intro a b; rw [boxProd_adj]; infer_instance
```

This instance uses `rw` to reduce adjacency in `G □ H` to a decidable proposition before calling `infer_instance`. This looks harmless, but can in some cases cause problems.

A proof by `rw` inserts a *cast* (an application of the recursor `Eq.rec`) into the term it builds. Whether this matters depends on how the instance is used: `#eval` runs compiled code, from which all proofs, including the cast, have been erased, so evaluation is unaffected. But `decide` proves a proposition `p` by asking the kernel to reduce the Boolean `decide p` to `true`, which means first reducing the `Decidable p` instance far enough to read off its `decide` field. The kernel can only reduce an `Eq.rec` cast when the equation being cast along holds by definition (so that, by proof irrelevance, the proof might as well be `rfl`). Otherwise the cast is stuck, exactly as `h ▸ v` was in the disjoint union `Σg` (`SimpleGraph.sigma`) in Sum and Sigma Graphs (p. 33) before we substituted `i = j`.

For the box product instance, we got away with it: `boxProd_adj` is definitional (proved by `Iff.rfl`), so the cast vanishes during kernel reduction.

```
example {V W : Type*} {G : SimpleGraph V} {H : SimpleGraph W} {x y : V × W} :
    (G □ H).Adj x y ↔ (G.Adj x.1 y.1 ∧ x.2 = y.2) ∨ (H.Adj x.2 y.2 ∧ x.1 = y.1) :=
  Iff.rfl

example : (K[2] □ K[3]).Adj (0, 0) (0, 1) := by decide
```

Mathlib's instance for the single-edge graph `edge s t` is not so lucky. Since `edge` is defined using `fromEdgeSet`, the rewrite lemma `edge_adj` is a genuine theorem, not provable by `Iff.rfl`, yet Mathlib's instance uses the same `rw` recipe.

```
#source instDecidableRelAdjEdge -- instance : DecidableRel (edge s t).Adj :=
                                --   fun _ _ ↦ by rw [edge_adj]; infer_instance
```

With this instance, `#eval` works fine.

```
#eval (edge 0 2).Adj 0 2 -- true
```

But the tactic `decide` gets stuck on the cast. Note the `⋯ ▸ inferInstance` in the error message below: `▸` is the cast inserted by `rw`.[3]

```
/- error: Tactic decide failed to reduce
  @decide ((edge 0 2).Adj 0 2) (instDecidableRelAdjEdge 0 2 0 2)
to true or false.

After unfolding the instance instDecidableRelAdjEdge, reduction got stuck at
  ⋯ ▸ inferInstance
```

[3] This is the instance in the version of Mathlib used by this guide, and may well be fixed in later versions!

```
Hint: Reduction got stuck on ▸ (Eq.rec), which suggests that one of the Decidable
instances is defined using tactics such as rw or simp. To avoid tactics, make use
of functions such as «inferInstanceAs» or decidable_of_decidable_of_iff to alter
a proposition.
-/
example : (edge 0 2).Adj 0 2 := by decide -- fails
```

So how should we fix this? The error message tells us what will work here: use `decidable_of_decidable_of_iff`.

```
#source decidable_of_decidable_of_iff -- .. (h : p ↔ q) : Decidable q where
                                      --      decide := decide p
                                      --      reflects_decide := ..
```

Note that this isn't a theorem, it is a function that provides a `Decidable` instance for `q` using a `Decidable` instance for `p` and a proof of `p ↔ q`. Its Boolean is simply `decide p`: it asks the instance for `p` for its verdict and returns the same verdict for `q`. The proof `h : p ↔ q` is used only in the `reflects_decide` field, to turn the proof that this verdict is correct for `p` into one for `q`.

The crucial difference from the `rw` version is where `h` ends up. The `rw` version casts the *whole* instance along `h`, so the kernel has to get past the cast before it can read off the `decide` field. Here `h` appears only inside `reflects_decide`, which is a proof that `decide` never needs to reduce. The `decide` field itself contains no `Eq.rec` cast, so nothing blocks kernel reduction.

Lean tries the most recently declared instances first, so from now on our instance given below takes priority over Mathlib's.

Decidability of the adjacency relation for the single-edge graph `edge s t`.

```
instance edgeDecidableRelAdj {V : Type*} [DecidableEq V] (s t : V) :
    DecidableRel (edge s t).Adj := fun v w =>
  decidable_of_decidable_of_iff (edge_adj s t v w).symm
```

Now both `#eval` and `decide` work fine!

```
#eval (edge (0 : Fin 5) 2).Adj 0 2 -- true
example : (edge 0  2).Adj 0 2 := by decide
example : ¬(edge 0 2).Adj 1 2 := by decide
```

In practice we rarely call `decidable_of_decidable_of_iff` directly: `decidable_of_iff a h` is the same function with the proposition a given explicitly, and `decidable_of_iff' b h` takes `h : a ↔ b` the other way round. We used `decidable_of_iff` for `pathNatFromEdgeSet` above.

```
#check decidable_of_iff  -- (a : Prop) (h : a ↔ b) [Decidable a] : Decidable b
#check decidable_of_iff' -- (b : Prop) (h : a ↔ b) [Decidable b] : Decidable a
```

More Decidable Adjacency Instances*

We give two more instances of decidable adjacency, for the disjoint unions `SimpleGraph.sigma` and `SimpleGraph.sum`.

Recall `Σg G`, the `SimpleGraph.sigma` disjoint union of an indexed family of graphs.

```
#source SimpleGraph.sigma -- protected def sigma (G : (i : ι) → SimpleGraph (V i)) :
                          --     SimpleGraph (Σ i, V i) where
```

```
--   Adj := fun ⟨i, u⟩ ⟨j, v⟩ =>
--            ∃ h : i = j, (G i).Adj u (h ▸ v)
```

We can give an instance of `DecidableRel (Σg G).Adj` for `SimpleGraph.sigma`, assuming the index type has decidable equality and that each of the graphs in the family has decidable adjacency. Here we have to do a case split on `i = j` (using the decidable equality of the index type).

```
instance {ι : Type*} {V : ι → Type*} (G : (i : ι) → SimpleGraph (V i))
    [DecidableEq ι] [∀ i, DecidableRel (G i).Adj] : DecidableRel (Σg G).Adj :=
  fun ⟨i, u⟩ ⟨j, v⟩ =>
    if h : i = j then
  -- when the indices agree, adjacency reduces to adjacency in the single graph G i
      decidable_of_iff ((G i).Adj u (h ▸ v)) (by subst h; simp)
    else
  -- vertices in different parts are never adjacent
      isFalse (by simp [h])
```

Recall `SimpleGraph.sum`, the disjoint union of two graphs `G ⊕g H`. Mathlib is missing an instance of `DecidableRel (G ⊕g H).Adj`.

```
instance {V W : Type*} (G : SimpleGraph V) (H : SimpleGraph W) [DecidableRel G.Adj]
    [DecidableRel H.Adj] : DecidableRel (G ⊕g H).Adj := fun u v =>
  match u, v with
  | Sum.inl u', Sum.inl v' => inferInstanceAs (Decidable (G.Adj u' v'))
  | Sum.inr u', Sum.inr v' => inferInstanceAs (Decidable (H.Adj u' v'))
  -- There is something slightly odd about the definition of SimpleGraph.sum
  -- Non-adjacency of vertices from different parts is defined using false : Bool,
  -- rather than False : Prop, hence the need for Decidable false below.
  | Sum.inl _, Sum.inr _ => inferInstanceAs (Decidable false)
  | Sum.inr _, Sum.inl _ => inferInstanceAs (Decidable false)
```

4.2 Finite Types

The main constructive way of expressing that a type is finite in Mathlib is via the `Fintype` class. Before we discuss `Fintype` we introduce the three closely related data structures that underlie almost all finite reasoning in Mathlib: `List`, `Multiset` and `Finset`.

Lists, Multisets and Finsets

`List`, `Multiset` and `Finset` form a hierarchy: `List` (ordered, duplicates allowed) → `Multiset` (unordered, duplicates allowed) → `Finset` (unordered, no duplicates).

A `List α` is an ordered, finite sequence of elements of `α`, with duplicates allowed. Both the order and the multiplicity of elements matter, so `[1, 1, 2]` and `[2, 1, 1]` are *different* lists both containing 1 twice and 2 once.

```
#check ([1, 1, 2] : List ℕ)              --  [1, 1, 2] : List ℕ
#eval ([2, 1, 1] : List ℕ) = [1, 1, 2]   -- false
#eval [2, 1, 1].count 1                  -- 2
```

A `Multiset α` is a `List α` in which we no longer care about the *order* of the elements, but we *do* still record how many times each element occurs. Formally it is the quotient of `List α` by permutation.

```
#source Multiset -- def Multiset.{u} (α : Type u) : Type u :=
                 --   Quotient (List.isSetoid α)
```

Because order is forgotten the two lists above become *equal* as multisets, but the repeated `1` is still counted twice.

```
#eval ({2, 1, 1} : Multiset ℕ) = {1, 1, 2}   -- true
#eval ({2, 1, 1} : Multiset ℕ).count 1       -- 2
```

Any `List α` can be coerced to a `Multiset α` by forgetting the order of its elements.

```
#eval ([1, 3, 1, 2] : Multiset ℕ)            -- {1, 3, 1, 2}
```

Finally, a `Finset α` is a `Multiset α` that contains no duplicates. It is implemented as a `Multiset` (its `val`) bundled with a proof (`nodup`) that this multiset has no repeated elements.

```
#source Finset -- structure Finset (α : Type*) where
               --   /- The underlying multiset -/
               --   val : Multiset α
               --   /- val contains no duplicates -/
               --   nodup : Nodup val

#check ({1, 2, 4} : Finset ℕ)         -- {1, 2, 4} : Finset ℕ
#eval ({3, 1, 1, 2} : Finset ℕ)       -- {3, 1, 2}
```

So a `Finset` is Mathlib's basic notion of a "finite set", and the one that computations work with. Building one using `{...}` notation silently discards any duplicates.

```
example : ({1, 1, 2} : Finset ℕ) = {1, 2} := rfl
```

Set-Builder Notation

Both `Set` and `Finset` can be written with the same *set-builder* notation `{x | p x}`, but the two behave differently.

For a `Set α` the set-builder `{x | p x}` stands for `Set.ofPred (fun x => p x)`, the set of *all* `x : α` satisfying `p`. No finiteness or decidability is required, because a `Set α` is just a predicate `α → Prop` wrapped in a one-field structure, and membership unwraps it again: `x ∈ Set.ofPred p` is definitionally `p x`.

```
example : {x : ℕ | x < 3} = Set.ofPred (fun x => x < 3) := rfl

example {α : Type*} (p : α → Prop) (x : α) : x ∈ Set.ofPred p ↔ p x := Iff.rfl
```

The neighbourhoods and edge set of a graph are sets of exactly this form. For instance `G.neighborSet v` is *definitionally* the set-builder `{w | G.Adj v w}`.

```
example {V : Type*} (G : SimpleGraph V) (v : V) :
    G.neighborSet v = {w | G.Adj v w} := rfl
```

A `Finset α`, by contrast, must actually be finite. The fundamental way to build one using a predicate is the form `{x ∈ s | p x}`, which filters an existing `s : Finset α`, retaining only those el-

ements that satisfy p. This is notation for Finset.filter, and because Lean has to test p on each element it needs a DecidablePred p instance.

```
example (s : Finset ℕ) : {x ∈ s | x < 3} = s.filter (fun x => x < 3) := rfl
#eval ({1, 2, 4, 5} : Finset ℕ).filter (fun x => x > 3) -- {4, 5}
```

The plain set-builder {x | p x} can also produce a Finset: it is the special case of {x ∈ s | p x} in which s is Finset.univ, the Finset of *all* elements of the ambient type. This only makes sense when there is a [Fintype α] instance. (Fintype instances and Finset.univ are introduced properly in Fintypes (p. 62).)

Left to its own devices {x | p x} produces a Set α. But suppose the expected type is Finset α, and Lean can find a Fintype α instance (so that Finset.univ exists) and a DecidablePred p. Then the same notation elaborates to the Finset α {x ∈ Finset.univ | p x}, that is, Finset.univ.filter p.

```
example {α : Type*} [Fintype α] (p : α → Prop) [DecidablePred p] :
    ({x | p x} : Finset α) = Finset.univ.filter (fun x => p x) := rfl

#eval ({x | x < 3} : Finset (Fin 5))              -- {0, 1, 2}
#eval {x ∈ ({0, 1, 2, 3, 4} : Finset ℕ) | x < 3}   -- {0, 1, 2}
```

Finally the {x ∈ s | p x} notation also works for Set α where it abbreviates Set.ofPred (fun x => x ∈ s ∧ p x). This does not require decidability.

Fintypes

A significant portion of work in graph theory is done with *finite* graphs. Nothing is finite by default, so we have to explicitly add finiteness assumptions when we need them.

There are different ways of expressing finiteness in Mathlib but for SimpleGraph the most common is to provide Fintype instances (for the vertex type or edge set etc).

A Fintype V instance is not just the proposition that V is finite: it is *data*. A Fintype V instance is a Finset V, called elems, together with a proof that elems contains every element of V. Since Finset already guarantees a lack of duplicates, this ensures that elems contains each element of V *exactly* once.

Later in this chapter (see Cardinality (p. 66)) we will see examples of other finiteness assumptions: in particular Finite, but for now we only consider Fintype since it is the most common and also the most problematic!

```
#source Fintype -- class Fintype (α : Type*) where
                --    /- The Finset containing all elements of a Fintype -/
                --  elems : Finset α
                --    /- A proof that elems contains every element of the type -/
                --  complete : ∀ x : α, x ∈ elems
```

While the underlying Finset of a Fintype instance is called elems we refer to it as Finset.univ (it is after all the Finset version of Set.univ).

```
#source Finset.univ -- def univ : Finset α := @Fintype.elems α _
#source Set.univ    -- def univ : Set α := {_a | True}
```

As with `Decidable` instances, `Fintype` instances are usually constructed automatically by type class inference.

For example, for any `n : ℕ`, there is a `Fintype` instance for `Fin n`.

```
#synth Fintype (Fin 5) -- Fin.fintype 5
#eval (Fin.fintype 5).elems     -- {0, 1, 2, 3, 4}
#check (Fin.fintype 5).complete --  ∀ (x : Fin 5), x ∈ Fintype.elems
```

This `Fintype` instance for `Fin n` is defined using `List.finRange n`, which is `[0, 1, ⋯, n - 1]`, as a `List (Fin n)`, and a proof, `List.nodup_finRange n`, that this list contains no duplicates. Together these define the `Finset elems`. The proof `complete` that this contains every element of `Fin n` is given by `List.mem_finRange`.

```
#source Fin.fintype  -- instance Fin.fintype (n : ℕ) : Fintype (Fin n) :=
                     -- ⟨⟨List.finRange n, List.nodup_finRange n⟩, List.mem_finRange⟩
#eval List.finRange 5        -- [0, 1, 2, 3, 4]
#check List.nodup_finRange   -- List.nodup_finRange (n : ℕ) : (List.finRange n).Nodup
#check List.mem_finRange     -- {n : ℕ} (x : Fin n) : x ∈ List.finRange n
```

A set `s : Set α` can also be treated as a type, the subtype `↥s` of elements of `α` that lie in `s`, so it too can have a `Fintype` instance. Such an instance is exactly what we need to turn a `Set` into a `Finset`: `Set.toFinset s` requires `[Fintype ↥s]` and has the same elements as `s`. In the other direction every `Finset` coerces to a `Set`, written `(t : Set α)`.

```
#check Set.toFinset     -- (s : Set α) [Fintype ↥s] : Finset α
#check Set.mem_toFinset -- a ∈ s.toFinset ↔ a ∈ s
example (t : Finset ℕ) : (t : Set ℕ).toFinset = t := Finset.toFinset_coe t
```

Mathlib's `Finset` versions of the sets attached to a graph are built in this way. For instance, the `Finset` of edges `G.edgeFinset` is `G.edgeSet.toFinset`, so it exists only when Lean can find a `Fintype G.edgeSet` instance, and `coe_edgeFinset` turns it back into the edge set.

```
#source edgeFinset    -- def edgeFinset : Finset (Sym2 V) := Set.toFinset G.edgeSet
#check coe_edgeFinset -- (G.edgeFinset : Set (Sym2 V)) = G.edgeSet
```

Instance Arguments in Statements

If `G : SimpleGraph V` then a `Fintype V` instance is generally not enough to do computations with the graph. For example, given just a `Fintype V` instance Lean won't even let us talk about the number of edges in `G`.

```
/- failed to synthesize instance of type class
  Fintype ↥G.edgeSet
-/
lemma bad_card_edgeFinset {V : Type*} (G : SimpleGraph V) [Fintype V] :
    ∃ n, G.edgeFinset.card = n := by
  sorry
```

This is because to form the `edgeFinset` of `G`, Lean needs to infer a `Fintype` instance for the `edgeSet` of `G`. But it can't do this with just `Fintype V`.

This can be very confusing: surely a graph on a finite vertex set has finitely many edges! But recall that a `Fintype` instance is data: an explicit `Finset` of the elements of the type.

The `edgeSet` of a graph `G` is a set of unordered pairs, i.e. a `Set (Sym2 V)`. Mathlib's instance `fintypeEdgeSet` applies to an arbitrary graph. It builds `Fintype G.edgeSet` as a subtype of `Sym2 V`, so it needs a `Fintype` instance for `Sym2 V` *and* a way to decide whether any given pair of vertices is adjacent in `G` (and so whether the corresponding edge belongs to the edge set). That is, it needs an instance of `DecidableRel G.Adj`!

```
#source fintypeEdgeSet
-- instance fintypeEdgeSet [Fintype (Sym2 V)] [DecidableRel G.Adj] :
--   Fintype G.edgeSet := Subtype.fintype _
```

Assuming `Fintype V`, Lean can easily infer `Fintype (Sym2 V)`.

```
example {V : Type*} [Fintype V] : Fintype (Sym2 V) := inferInstance
#check Sym2.instFintype -- the instance for Fintype (Sym2 V)
```

If we also provide a `DecidableRel G.Adj` instance then Lean can infer a `Fintype` instance for `G.edgeSet`.

```
example {V : Type*} (G : SimpleGraph V) [Fintype V] [DecidableRel G.Adj] :
    Fintype G.edgeSet := inferInstance
```

We can now complete the broken example from above.

```
lemma bad_card_edgeFinset {V : Type*} (G : SimpleGraph V) [Fintype V]
    [DecidableRel G.Adj] : ∃ n, G.edgeFinset.card = n := by
  simp
```

Although this works, it isn't great style and we will take a moment to explain why.

While it is very common to use the `Fintype V` and `DecidableRel G.Adj` assumptions to *infer* a `Fintype G.edgeSet` instance, this shouldn't be baked into a theorem in this way. By using these instance assumptions we have proved that:

> There exists `n : ℕ` such that the number of edges of `G` is `n`, provided that the finiteness of its edge set is inferred from the finiteness of `V` and the decidability of adjacency in `G`.

So if you have a graph `G`, which for some *other* reason you know to have finitely many edges, then this result does not apply! A simple example is the empty graph on `ℕ`. It has no edges, and Mathlib's instance `fintypeEdgeSetBot` makes its edge set a `Fintype` (for any vertex type), but `ℕ` is infinite, so `bad_card_edgeFinset` cannot be applied to it.

```
#check fintypeEdgeSetBot -- Fintype ↑⊥.edgeSet

/- failed to synthesize instance of type class
  Fintype ℕ
-/
#check bad_card_edgeFinset (⊥ : SimpleGraph ℕ)
```

This is a lesson in how not to structure instance assumptions: a theorem should ask for exactly the instances that its *statement* actually uses. Anything stronger silently restricts the theorem to those callers who happen to obtain the required instances by that particular route.

Here the conclusion mentions only `G.edgeFinset`, and `G.edgeFinset` is by definition the `Finset` underlying a `[Fintype G.edgeSet]` instance. That single instance is the *only* finiteness the statement needs, so it is the only one we should assume:

```
theorem good_card_edgeFinset {V : Type*} (G : SimpleGraph V) [Fintype G.edgeSet] :
    ∃ n, G.edgeFinset.card = n := by simp
```

Stated this way the theorem is reusable, because Lean will discharge the `[Fintype G.edgeSet]` argument from *whatever* is in scope at the call site. A caller that has `[Fintype V] [DecidableRel G.Adj]` can get the instance by inference, exactly the chain we traced from `Fintype (Sym2 V)` to `Fintype G.edgeSet`:

```
example {V : Type*} (G : SimpleGraph V) [Fintype V] [DecidableRel G.Adj] :
    ∃ n, G.edgeFinset.card = n := G.good_card_edgeFinset
```

While a caller that knows the edge set is finite for some entirely different reason, such as the empty graph on $\mathbb{N}$, can supply the instance directly, with no mention of `Fintype V` or decidable adjacency:

```
example : ∃ n, (⊥ : SimpleGraph ℕ).edgeFinset.card = n := good_card_edgeFinset ⊥
```

When you don't need to *compute* at all, the cleanest choice is often to drop `Fintype` data entirely in favour of `Finite`/`Nat.card`; see Cardinality (p. 66).

Building `Fintype` Instances

Having told this cautionary tale, we now return to the basic question of how to infer other common `Fintype` instances for graphs. Our next example is finiteness of the neighbourhood of a vertex `v`: this is `Fintype (G.neighborSet v)`.

This would again follow from `Fintype V` and `DecidableRel G.Adj`, but we can get away with less. We don't need `G.Adj a b` to be decidable for *all* pairs of vertices, we only need to be able to decide whether or not any given vertex is adjacent to `v`, i.e. we only need `DecidablePred (· ∈ G.neighborSet v)`. With this Lean can filter the `Fintype V` instance down to the neighbours of `v` to produce a `Fintype (G.neighborSet v)` instance.

```
example {V : Type*} (G : SimpleGraph V) (v : V) [Fintype V]
    [DecidablePred (· ∈ G.neighborSet v)] : Fintype (G.neighborSet v) :=
  inferInstance

#check Subtype.fintype -- the instance used above
```

Obviously there are plenty of graphs with finite neighbourhoods that don't have finite vertex sets, for example the infinite path graph `pathNat`. In such cases we can provide our own `Fintype` instance for the neighbourhood of a vertex.

When we need to build a `Fintype` instance by hand like this, the following constructors are often the most convenient starting points.

```
#check Fintype.ofEquiv      -- given Fintype α and a bijection α ≃ β
                            -- we can get Fintype β
#check Fintype.ofSurjective -- given Fintype α and f : α → β surjective
                            -- we can get Fintype β
#check Fintype.ofInjective  -- given Fintype β and f : α → β injective
                            -- we can get Fintype α
```

```
#check Subtype.fintype      -- given Fintype α and p : α → Prop with
                            -- DecidablePred p we can get Fintype {x // p x}
#check Fintype.ofFinset     -- given p : Set α, s : Finset α and a proof
                            -- ∀ (x : α), x ∈ s ↔ x ∈ p we can get Fintype ↑p
```

For the neighbourhoods of `pathNat`, `Fintype.ofFinset` is the natural choice: we simply hand over the `Finset` of neighbours of each vertex and let `simp` do the rest.

```
instance (n : ℕ) : Fintype (pathNat.neighborSet n) :=
  match n with
  | 0 =>  Fintype.ofFinset {1} (by intro; simp [Eq.comm])
  | n + 1 =>  Fintype.ofFinset {n, n + 2} (by intro; simp [Eq.comm, Or.comm])
```

We will explore finite neighbourhoods in more detail in Finite Neighbourhoods (p. 69).

Cardinality

Counting crops up everywhere in graph theory: the number of vertices, the number of edges, the degree of a vertex (the number of its neighbours), the size of a clique, and so on. Mathlib offers several different notions of *cardinality*, and which one to use depends on what kind of object we are counting and what finiteness information we have available. We describe the main ones below.

Counting a `Finset`: `Finset.card`

The most basic notion is the cardinality of a `Finset`. A `Finset α` is by construction a finite collection of elements of `α`, so it always has a well-defined size: a natural number. This is `Finset.card`, written either `s.card` or (after `open Finset`) with the prefix notation `#s`.

```
section finset
open Finset

#check Finset.card -- {α : Type u_1} → Finset α → ℕ
#eval ({0, 1, 2} : Finset ℕ).card -- 3
#eval #K[4].edgeFinset            -- 6
#eval #K[4, 3].edgeFinset         -- 12

end finset
```

In particular, the degree of a vertex (see Finite Neighbourhoods (p. 69) below) is precisely `#(G.neighborFinset v)`, and `#G.edgeFinset` is the number of edges of `G`.

Counting a Type: `Fintype.card`

When a type `α` carries a `Fintype α` instance it comes bundled with a canonical `Finset` of *all* its elements, `Finset.univ`. The cardinality of the type is then just the cardinality of that `Finset`:

```
#check Fintype.card -- (α : Type u_3) → [Fintype α] → ℕ
#check Finset.card_univ  -- Finset.univ.card = Fintype.card α

#eval Fintype.card (Fin 5) -- 5
#eval Fintype.card (Sum (Option (Fin 5 × Bool)) (Fin 3 → Bool))
                                          -- (1 + 5 * 2) + 2 ^ 3 = 19
```

Because it is built from `Finset.univ`, `Fintype.card` is *computable* whenever the `Fintype` instance is: it is the notion we use with `#eval`, `decide` and `native_decide`. For example the degree of a vertex `v` in a graph `G` is defined as a `Finset.card`, but it equals `Fintype.card ↑(G.neighborSet v)`.

```
#check card_neighborSet_eq_degree -- Fintype.card ↑(G.neighborSet v) = G.degree v
```

Finiteness Without Data: the `Finite` Type Class

Sometimes we know a type is finite but have no canonical enumeration of it, or we simply don't want to drag `Fintype` data, and the `Decidable` instances it forces, through a proof. For this Mathlib provides the `Finite` type class. It tells us that there exist a natural number `n` and a bijection between the type and `Fin n`, the canonical type of cardinality `n`.

```
#source Finite -- class inductive Finite (α : Sort*) : Prop
               --     | intro {n : ℕ} : α ≃ Fin n → Finite _

example {α : Type*} : Finite α ↔ ∃ n, Nonempty (α ≃ Fin n) :=
  finite_iff_exists_equiv_fin
```

The crucial difference is that `Finite V` is a *proposition* whereas `Fintype V` is *data*.

Aspect	`Fintype V`	`Finite V`
Nature	*data* (a bundled structure)	a *proposition* (`Prop`-valued)
Carries	a `Finset V` of all elements	only `∃ n, Nonempty (V ≃ Fin n)`
Says	*how* to enumerate `V`	*that* `V` can be enumerated
Two instances	equal, but not always definitionally equal	definitionally equal (proof irrelevance)
Computable?	if the instance is (`Fintype.card`, `decide`)	no (use `Nat.card`)

We always have `Fintype V → Finite V`; the reverse recovers data from a proof, so it needs the axiom of choice and is noncomputable (but is still very useful in proofs).

```
#check Finite.of_fintype   -- [Fintype α] : Finite α
#source Fintype.ofFinite   -- noncomputable def Fintype.ofFinite
                           --       (α : Type*) [Finite α] : Fintype α
```

So being data is not the same as being computable. `Fintype.ofFinite`, like any instance built with the help of the `classical` tactic, is a perfectly good `Fintype` instance for use in proofs, but `#eval` cannot run it.

One final remark about `Fintype` instances is that because two instances for the same type need not be *definitionally equal*, they can be the source of *diamonds* in type class inference. This can make proofs fail in mysterious ways. We will see a simple example of one below in The Handshake Lemma (p. 73).

Counting with Only `Finite`: `Nat.card`

If all we have is `Finite α` (or nothing at all) we cannot use `Fintype.card`. Instead we can use `Nat.card α`, the cardinality of a type as a natural number, defined for *every* type by falling back to `0` when the type is infinite.

```
#check Nat.card -- Type u_3 → ℕ
#check Nat.card_eq_zero_of_infinite -- [Infinite α] : Nat.card α = 0
```

`Nat.card` agrees with `Fintype.card` whenever a `Fintype` instance happens to be available, so the two never give conflicting answers on a finite type.

```
#check Nat.card_eq_fintype_card -- [Fintype α] : Nat.card α = Fintype.card α
```

The trade-off is that `Nat.card` is noncomputable: it cannot be `#eval`-uated. Its advantage is that it needs no `Fintype` data.

Beware of the `0` convention, though: `Nat.card α = 0` can mean either "α is empty" *or* "α is infinite".

Counting in $\mathbb{N}\infty$: `ENat.card`

The `0`-on-infinite convention of `Nat.card` is convenient but lossy. When we want to actually distinguish *infinite* from *empty* we use `ENat.card`, which takes values in the extended naturals $\mathbb{N}\infty$ = `WithTop ℕ` and returns `⊤` for infinite types.

```
#check ENat.card         -- Type u_3 → ℕ∞
#check ENat.card_eq_top  -- ENat.card α = ⊤ ↔ Infinite α
#check ENat.card_lt_top  -- ENat.card α < ⊤ ↔ Finite α
```

On a finite type it agrees with `Fintype.card` (cast into $\mathbb{N}\infty$), and `Nat.card` is recovered from it as `(ENat.card α).toNat`. Like `Nat.card`, `ENat.card` is noncomputable.

```
#check ENat.card_eq_coe_fintype_card -- [Fintype α] : ENat.card α = ↑(Fintype.card α)
example {V : Type*} : (ENat.card V).toNat = Nat.card V := rfl
```

Counting a `Set`: `Set.ncard` and `Set.encard`

Finally these two ways of counting types extend to sets.

- `Set.ncard s : ℕ` is the size as a natural number, `0` if `s` is infinite. It is `Nat.card` of the corresponding subtype.
- `Set.encard s : ℕ∞` is the size in $\mathbb{N}\infty$, `⊤` if `s` is infinite (the `Set` analogue of `ENat.card`).

```
#check Set.ncard           -- {α : Type u_1} → Set α → ℕ
#check Set.encard          -- {α : Type u_1} → Set α → ℕ∞
#check Nat.card_coe_set_eq -- (s : Set α) : Nat.card ↑s = s.ncard
```

When the set is the coercion of a `Finset`, `ncard` is just that `Finset`'s `card`.

```
example {V : Type*} (s : Finset V) : (s : Set V).ncard = s.card :=
  Set.ncard_coe_finset _
```

So for a finite graph the (noncomputable) `G.edgeSet.ncard` and the (computable) `G.edgeFinset.card` agree.

```
example {V : Type*} (G : SimpleGraph V) [Fintype G.edgeSet] :
    G.edgeSet.ncard = G.edgeFinset.card := by
  rw [← coe_edgeFinset, Set.ncard_coe_finset]
```

Which One Should I Use?

To count …	given …	use	computable?[4]
a `Finset s`	(nothing)	`#s` / `s.card`	yes
a type `α`	`[Fintype α]`	`Fintype.card α`	yes
a type `α`	`[Finite α]` or nothing	`Nat.card α` (`0` if infinite)	no
a type `α`	nothing	`ENat.card α` (`⊤` if infinite)	no
a `Set s`	`[Fintype ↥s]`	`Fintype.card ↥s`	yes
a `Set s`	(nothing)	`s.ncard` / `s.encard`	no

As a rule of thumb: use `Finset.card` / `Fintype.card` when you want to *compute* (with `#eval`, `decide`), and `Nat.card` / `ENat.card` (or `Set.ncard` / `Set.encard`) when you only want to *reason*. The bridging lemmas above let you switch between them whenever both are available. Sets also have a `Prop`-valued finiteness predicate, `Set.Finite s`, which is the `Set` analogue of `Finite` and from which `Set.Finite.toFinset` produces a (noncomputable) `Finset`. (Having said all this, the current `SimpleGraph` API is heavily biased towards the use of `Fintype` and `Finset`.)

4.3 Degrees

The degree of a vertex is the size of its neighbourhood. For this to be a natural number the neighbourhood must be finite, so we begin with finite neighbourhoods.

Finite Neighbourhoods

We already saw finite neighbourhoods briefly above, using `Fintype ↥(G.neighborSet v)` to form the `Finset` of neighbours of a vertex `v`.

```
#source neighborFinset -- : Finset V := (G.neighborSet v).toFinset
```

So the *degree* of a vertex is the cardinality of its neighbourhood, assuming that this neighbourhood is finite in the sense of `Fintype ↥(G.neighborSet v)`.

```
#source degree -- def degree : ℕ := #(G.neighborFinset v)
```

```
#eval K[6].degree 3                     -- 5
#eval K[5].neighborFinset 3             -- {0, 1, 2, 4}
#eval K[3, 2].neighborFinset (Sum.inl 1) -- {Sum.inr 0, Sum.inr 1}
```

Whenever we have `Fintype V` and `DecidableRel G.Adj` we automatically get an instance of `Fintype ↥(G.neighborSet v)` for all vertices `v`.

```
example {V : Type*} {v : V} {G : SimpleGraph V} [Fintype V] [DecidableRel G.Adj] :
    Fintype (G.neighborSet v) := inferInstance
```

However, a vertex can have a finite neighbourhood even when `V` is infinite, so `[Fintype V] [DecidableRel G.Adj]` is more than we need. A result about `G.degree v` only needs the instance `Fintype ↥(G.neighborSet v)` for that one vertex.

```
example {V : Type*} (G : SimpleGraph V) (v : V) [Fintype (G.neighborSet v)] :
    G.degree v = 0 ↔ G.neighborFinset v = ∅ := by
```

[4] Here "yes" means that the counting function is computable; an actual `#eval` also needs the `Finset` or `Fintype` instance to have been built computably (not via `Fintype.ofFinite`, `Set.Finite.toFinset` or `classical`).

```
  rw [← card_neighborFinset_eq_degree, Finset.card_eq_zero]
```

If we want to assume that all vertices have finite neighbourhoods, we can do this by assuming [G.LocallyFinite].

```
#source LocallyFinite -- abbrev LocallyFinite := ∀ v : V, Fintype (G.neighborSet v)
```

In a locally finite graph, with no isolated vertices, all vertices have positive degree. (Recall G.support is the set of non-isolated vertices.)

```
example {V : Type*} (G : SimpleGraph V) [G.LocallyFinite] (h : ∀ v, v ∈ G.support) :
    ∀ v, 0 < G.degree v := by
  intro v
  obtain ⟨w, hadj⟩ := h v
  exact hadj.degree_pos_left

example {V : Type*} {G : SimpleGraph V} [Fintype V] [DecidableRel G.Adj] :
    G.LocallyFinite := inferInstance
```

If G has finitely many edges then we can obtain an instance of G.LocallyFinite without requiring Fintype V: every neighbour of v is the other endpoint of an edge at v. Mathlib's equivalence incidenceSetEquivNeighborSet between the edges at v and the neighbours of v turns a Fintype G.edgeSet instance into a Fintype (G.neighborSet v) instance. Notice that, unlike the instance built from Fintype V, this does not require DecidableRel G.Adj.

We make this a def rather than an instance. As an instance it would give a second route to Fintype (G.neighborSet v), alongside the one through Fintype V and DecidableRel G.Adj: a *diamond* of the kind we will meet in The Handshake Lemma (p. 73).

A graph with finitely many edges is locally finite.

```
@[instance_reducible]
def locallyFiniteOfFintypeEdgeSet {V : Type*}
    {G : SimpleGraph V} [DecidableEq V] [Fintype G.edgeSet] : LocallyFinite G :=
  fun v => Fintype.ofEquiv {e ∈ G.edgeSet | v ∈ e} (G.incidenceSetEquivNeighborSet v)
```

The same equivalence shows that the degree of v is also the number of edges at v, i.e. the size of G.incidenceFinset v, the Finset version of incidenceSet.

```
#check card_incidenceFinset_eq_degree -- #(G.incidenceFinset v) = G.degree v
```

Degrees in the n-Uniform Blow-up of a Graph

Recall the n-uniform blow-up G[n] of a graph G (see blowUpN).

```
#source blowUpGraph -- abbrev blowUpGraph (G : SimpleGraph ι) (V : ι → Type*) :
                    --   SimpleGraph (Σ i, V i) := SimpleGraph.comap Sigma.fst G
#source blowUpN -- abbrev blowUpN (G : SimpleGraph ι) (n : ℕ) :
                --   SimpleGraph (Σ _ : ι, Fin n) := G.blowUpGraph (fun _ => Fin n)
```

If v : V is a vertex with a finite neighbourhood in G then, in the n-uniform blow-up G[n], every vertex ⟨v, i⟩, where i : Fin n, has a finite neighbourhood. The neighbourhood of ⟨v, i⟩ in G[n] is {⟨w, j⟩ | G.Adj v w}, which is in bijection with G.neighborSet v × Fin n, which in turn is finite since both factors are finite.

When `V` is finite and adjacency in `G` is decidable, Lean already infers a `Fintype` instance for this neighbourhood by filtering the vertices of `G[n]`. When `G` is only locally finite it can't, so we build the instance ourselves. For the same reason as `locallyFiniteOfFintypeEdgeSet`, we make it a `def` rather than an `instance`.

To build it we give the bijection (or more precisely the `Equiv`) to `Fintype.ofEquiv` which then returns the instance. An `Equiv α ≃ β` is a structure bundling a function `toFun : α → β`, its inverse `invFun : β → α`, and proofs `left_inv` and `right_inv` that the two are mutually inverse (see Maps (p. 98)).

A vertex `w` in the `n`-uniform blow-up `G[n]` has a finite neighbourhood in `G[n]` assuming `w.fst` has a finite neighbourhood in `G`.

```
@[instance_reducible]
def blowUpNFintypeNeighborSet {V : Type*} (G : SimpleGraph V) (n : ℕ)
    (w : Σ _ : V, Fin n) [Fintype (G.neighborSet w.fst)] :
    Fintype (G[n].neighborSet w) :=
  -- Given any vertex w = ⟨v, i⟩ in the blow-up, its neighbourhood is equivalent to
  -- the product of the neighbourhood of v in G and Fin n.
  -- Lean can infer that the product G.neighborSet v × Fin n is finite since
  -- both factors are finite, so we just need to give the equivalence.
  Fintype.ofEquiv (G.neighborSet w.fst × Fin n)
  -- We now need to give the Equiv from
  -- G.neighborSet v × Fin n to G[n].neighborSet ⟨v, i⟩
    { toFun := fun ⟨⟨x, hx⟩, j⟩ => ⟨⟨x, j⟩, hx⟩
      invFun := fun ⟨⟨x, j⟩, hadj⟩ => ⟨⟨x, hadj⟩, j⟩
     -- since the proofs of left_inv and right_inv are rfl we can omit them
    }
```

This lets us talk about the degrees of vertices in the `n`-uniform blow-up of any locally finite graph, and relate them to the degrees in the original graph.

If `v` has a finite neighbourhood in `G` then each vertex `⟨v, i⟩` of `G[n]` has degree `G.degree v * n`. (The statement takes both neighbourhood instances as arguments, as recommended in Instance Arguments in Statements (p. 63), so it applies with either the filtering instance or `blowUpNFintypeNeighborSet`.)

```
lemma degree_blowUpN {V : Type*} (G : SimpleGraph V) (n : ℕ) (v : V) (i : Fin n)
    [Fintype (G.neighborSet v)] [Fintype (G[n].neighborSet ⟨v, i⟩)] :
    G[n].degree ⟨v, i⟩ = G.degree v * n := by
  have h : G[n].neighborFinset ⟨v, i⟩ =
        (G.neighborFinset v).sigma (fun _ => Finset.univ) := by
    ext ⟨w, j⟩
    simp
  simp [← card_neighborFinset_eq_degree, h]
```

For a graph with infinitely many vertices, such as `pathNat`, there is no filtering instance, so we switch `blowUpNFintypeNeighborSet` on ourselves. The command `attribute [local instance] blowUpNFintypeNeighborSet` in makes it an instance for the declaration that follows, and nowhere else.

```
attribute [local instance] blowUpNFintypeNeighborSet in
example (v n : ℕ) (i : Fin n) : pathNat[n].degree ⟨v, i⟩ = pathNat.degree v * n :=
  degree_blowUpN pathNat _ v i
```

Minimum and Maximum Degree

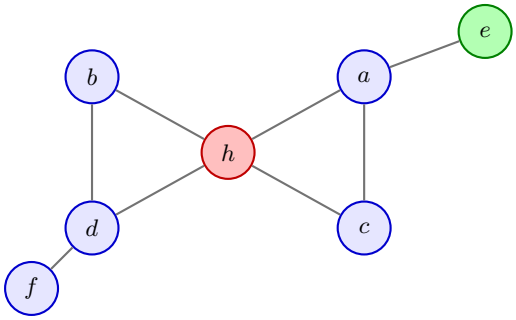
For a finite graph we can talk about the minimum degree $\delta(G)$ and maximum degree $\Delta(G)$. In Mathlib these are `G.minDegree` and `G.maxDegree`. Both require `Fintype V` and `DecidableRel G.Adj`, and (very reasonably) default to `0` if `G` has no vertices.

```
#source minDegree  -- def minDegree [DecidableRel G.Adj] : ℕ :=
                   --   WithTop.untopD 0 (univ.image fun v => G.degree v).min

#source maxDegree  -- def maxDegree [DecidableRel G.Adj] : ℕ :=
                   --   WithBot.unbotD 0 (univ.image fun v => G.degree v).max
```

We introduce the standard notation $\delta(G)$ for the minimum degree and $\Delta(G)$ for the maximum degree of the graph `G`.

```
@[inherit_doc minDegree] scoped notation "δ(" G ")" => SimpleGraph.minDegree G
@[inherit_doc maxDegree] scoped notation "Δ(" G ")" => SimpleGraph.maxDegree G
```



A graph `G` with `δ(G) = 1` and `Δ(G) = 4`

Every vertex has degree at least the minimum degree and at most the maximum degree.

```
#check minDegree_le_degree  -- δ(G) ≤ G.degree v
#check degree_le_maxDegree  -- G.degree v ≤ Δ(G)
```

If `G` is a subgraph of `H` then `δ(G) ≤ δ(H)`.

```
#check minDegree_le_minDegree -- G ≤ H → δ(G) ≤ δ(H)
```

The junk value `0` does come with a cost. If every vertex has degree at least `k` then `k ≤ δ(G)`, but only when `V` is nonempty. Likewise a vertex of minimum (or maximum) degree exists only when `V` is nonempty. The upper bound for `Δ(G)` needs no such hypothesis, since the junk value `0` satisfies it anyway.

```
#check le_minDegree_of_forall_le_degree -- [Nonempty V] (k : ℕ) :
                                        --   (∀ v, k ≤ G.degree v) → k ≤ δ(G)
#check maxDegree_le_of_forall_degree_le -- (k : ℕ) : (∀ v, G.degree v ≤ k) → Δ(G) ≤ k
#check exists_minimal_degree_vertex     -- [Nonempty V] : ∃ v, δ(G) = G.degree v
#check exists_maximal_degree_vertex     -- [Nonempty V] : ∃ v, Δ(G) = G.degree v
```

One problem with the definitions of `minDegree` and `maxDegree` is that they require `Fintype V`, but plenty of graphs have finite minimum and maximum degree without having finite vertex sets. For example the infinite path graph `pathNat` has minimum degree `1` and maximum degree `2` but

we can't state these facts using `minDegree` and `maxDegree` (because `pathNat` has infinitely many vertices).

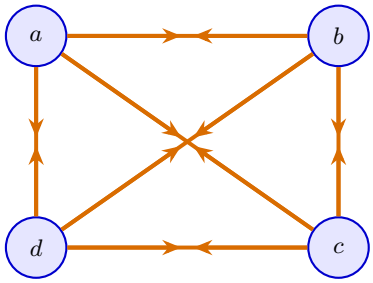
We can however characterize the neighbourhoods and degrees of vertices in `pathNat`.

```
@[simp]
lemma neighborFinset_pathNat (n : ℕ) :
    pathNat.neighborFinset n = if n = 0 then {1} else {n - 1, n + 1} := by
  ext x; cases n <;> simp [eq_comm, or_comm]

@[simp]
lemma degree_pathNat (n : ℕ) :
    pathNat.degree n = if n = 0 then 1 else 2 := by
  rw [← card_neighborFinset_eq_degree, neighborFinset_pathNat]
  -- the split_ifs tactic is a convenient way to handle if statements
  split_ifs <;> grind
```

The Handshake Lemma

In a finite graph the sum of all vertex degrees equals twice the number of edges.



Each edge contributes two darts, one from each endpoint

```
#check sum_degrees_eq_twice_card_edges -- ∑ v, G.degree v = 2 * G.edgeFinset.card
```

The proof counts the darts of `G` (`Dart`) in two ways. Each vertex `v` is the first entry of exactly `G.degree v` darts, so the number of darts is the sum of the degrees. Each edge gives exactly two darts, one in each direction, so the number of darts is also twice the number of edges.

```
#check dart_card_eq_sum_degrees      -- Fintype.card G.Dart = ∑ v, G.degree v
#check dart_card_eq_twice_card_edges -- Fintype.card G.Dart = 2 * #G.edgeFinset
```

The key to the first count is the bijection from the neighbourhood of `v` to the darts of `G` whose first element is `v`.

```
#check dartOfNeighborSet_injective -- Function.Injective (G.dartOfNeighborSet v)
#check dart_fst_fiber -- {d | d.toProd.1 = v} = image (G.dartOfNeighborSet v) univ
```

As noted in Mathlib's documentation, the handshake lemma sometimes refers to the following consequence of this lemma: the number of vertices of odd degree is even.

```
#check even_card_odd_degree_vertices -- Even #{v | Odd (G.degree v)}
```

Since we spent some time looking at instance arguments earlier, we note that the signature of `sum_degrees_eq_twice_card_edges` follows the pattern of `bad_card_edgeFinset`. It takes `[Fintype`

`V`] and [`DecidableRel G.Adj`], and its statement derives two further instances from these: the `Fintype` instance for each neighbourhood (used by `degree`) and the one for the edge set (used by `edgeFinset`). So it only directly applies to graphs whose instances are synthesized by that route. The instances its statement actually requires are [`Fintype V`] to sum over all vertices, [`G.LocallyFinite`] to talk about degrees of vertices, and [`Fintype G.edgeSet`] to count edges.

We can easily derive a more general version with only these arguments. But we first show how the obvious proof fails because the failure can seem rather mysterious.

```
/- error: Type mismatch
  sum_degrees_eq_twice_card_edges G
has type
  ∑ v, @degree V G v (Subtype.fintype (Membership.mem (G.neighborSet v))) = 2 *
  ↪  G.edgeFinset.card
but is expected to have type
  ∑ v, @degree V G v (inst✝ v) = 2 * G.edgeFinset.card
-/
example {V : Type*} (G : SimpleGraph V) [Fintype V] [Fintype G.edgeSet]
    [G.LocallyFinite] : ∑ v, G.degree v = 2 * G.edgeFinset.card := by
  classical
  -- This makes DecidableRel G.Adj available,
  -- so we can use the original lemma.
  -- But the obvious proof fails due to a Fintype instance mismatch.
  exact G.sum_degrees_eq_twice_card_edges
```

The error is caused by `Fintype` instance *diamonds*. The `degree` function takes a `Fintype (G.neighborSet v)` instance argument. We provided this via the `G.LocallyFinite` instance, but the original lemma `sum_degrees_eq_twice_card_edges` uses a different route: namely `Fintype V` + `DecidableRel G.Adj` to synthesize a `Fintype (G.neighborSet v)` instance. Since these are not definitionally equal (see Definitional and Propositional Equalities (p. 16)), Lean refuses to unify them, and the proof fails. (The same happens with the edge set instance used by `edgeFinset`.)

However, any two `Fintype` instances $\texttt{inst}_1$ and $\texttt{inst}_2$ for the same type are propositionally equal. Each instance consists of a `Finset` together with a proof that the `Finset` contains every element of the type. So the two `Finsets` have the same members, and hence are equal by extensionality, while the two proofs of completeness are equal by proof irrelevance. So $\texttt{inst}_1 = \texttt{inst}_2$.

This general argument means that, for every type `α`, `Fintype α` is a `Subsingleton` (a type in which any two elements are equal). `Subsingleton` is itself a type class, and Mathlib provides the instance `Fintype.subsingleton`, so `Subsingleton.elim` can produce a proof of $\texttt{inst}_1 = \texttt{inst}_2$ for us whenever we need one. This is exactly what **`convert`** does in our proof below.

```
#synth (α : Type*) → Subsingleton (Fintype α) -- Fintype.subsingleton
#check Subsingleton.elim -- {α : Sort u} [h : Subsingleton α] (a b : α) : a = b
```

The **handshake lemma** the degrees of a finite graph sum to twice its number of edges (assuming only the instances its statement needs)

```
lemma sum_degrees_eq_twice_card_edges' {V : Type*} (G : SimpleGraph V)
    [Fintype V] [Fintype G.edgeSet] [G.LocallyFinite] :
    ∑ v, G.degree v = 2 * G.edgeFinset.card := by
  classical
  convert G.sum_degrees_eq_twice_card_edges
```

Regular Graphs

If `G` is locally finite then we can talk about `G` being *regular of degree* `d` for some `d : ℕ`.

```
#source IsRegularOfDegree -- .. (d : ℕ) : Prop := ∀ v : V, G.degree v = d
```

If `G` is regular of degree `d` then its `n`-uniform blow-up is regular of degree `d * n`.

```
lemma IsRegularOfDegree.blowUpN {V : Type*} (G : SimpleGraph V) [G.LocallyFinite]
    (n d : ℕ) [G[n].LocallyFinite] (h : G.IsRegularOfDegree d) :
    G[n].IsRegularOfDegree (d * n) := by
  intro ⟨v, i⟩
  rw [degree_blowUpN, h v]
```

So the `n`-uniform blow-up of the complete graph on `m` vertices is regular of degree `(m - 1) * n`.

```
example (m n : ℕ) : K[m][n].IsRegularOfDegree ((m - 1) * n) :=
  IsRegularOfDegree.blowUpN K[m] n (m - 1)
    (by simpa using IsRegularOfDegree.top (V := Fin m))
```

Mathlib knows that the complete graph on a finite vertex type is regular, but it doesn't state the converse: any graph that is regular of degree `n - 1`, where `Fintype.card V = n`, is complete. It does, however, have the two ingredients. A vertex is *universal* (`G.IsUniversal v`) iff it has degree `Fintype.card V - 1` and a graph is complete iff every vertex is universal.

```
#check degree_eq_card_sub_one -- G.degree v = Fintype.card V - 1 ↔ G.IsUniversal v
#check eq_top_iff_forall_isUniversal -- G = ⊤ ↔ ∀ v, G.IsUniversal v
```

A graph is regular of degree `Fintype.card V - 1` iff it is complete. (Mathlib's `degree_eq_card_sub_one` uses the `Fintype` instance for neighbourhoods coming from `DecidableRel G.Adj`, while ours comes from `G.LocallyFinite`; `convert` bridges the two, just as it did in the proof of `sum_degrees_eq_twice_card_edges'`.)

```
theorem isRegularOfDegree_card_sub_one_iff_eq_top {V : Type*} (G : SimpleGraph V)
    [Fintype V] [G.LocallyFinite] :
    G.IsRegularOfDegree (Fintype.card V - 1) ↔ G = ⊤ := by
  classical
  rw [eq_top_iff_forall_isUniversal]
  apply forall_congr'
  intro v
  convert G.degree_eq_card_sub_one v
```

Complements of regular graphs are also regular. In G^c a vertex is adjacent to exactly those other vertices that it is not adjacent to in `G`, so its degree is given by `degree_compl`. Hence the complement of a graph on `n` vertices that is regular of degree `k` is regular of degree `n - 1 - k`.

```
#check degree_compl -- Gᶜ.degree v = Fintype.card V - 1 - G.degree v
#check IsRegularOfDegree.compl
  -- G.IsRegularOfDegree k → Gᶜ.IsRegularOfDegree (Fintype.card V - 1 - k)
```

For a regular graph the handshake lemma counts the edges: if `G` is regular of degree `d` on `n` vertices and has `e` edges then `2 * e = n * d`. Here we need our new version of the handshake lemma, since our instances come from `G.LocallyFinite` and `[Fintype G.edgeSet]` rather than from `DecidableRel G.Adj`.

```
lemma IsRegularOfDegree.two_mul_card_edgeFinset {V : Type*} {G : SimpleGraph V}
    {d : ℕ} [Fintype V] [Fintype G.edgeSet] [G.LocallyFinite]
    (h : G.IsRegularOfDegree d) : 2 * G.edgeFinset.card = Fintype.card V * d := by
  rw [← sum_degrees_eq_twice_card_edges' G]
  simp [h.degree_eq]
```

Dividing by 2, the number of edges is `Fintype.card V * d / 2`. Division on ℕ rounds down, but here nothing is lost, since the lemma above shows that `Fintype.card V * d` is even.

```
lemma IsRegularOfDegree.card_edgeFinset {V : Type*} {G : SimpleGraph V} {d : ℕ}
    [Fintype V] [Fintype G.edgeSet] [G.LocallyFinite] (h : G.IsRegularOfDegree d) :
    G.edgeFinset.card = Fintype.card V * d / 2 := by
  grind [h.two_mul_card_edgeFinset]
```

Neighbours in a Set

It is often useful to count only the neighbours of a vertex `v` that lie in a given finite set `s`. We call this number `G.degIn s v`. We define it as the size of `s.bipartiteAbove G.Adj v`, which is Mathlib's name for `{w ∈ s | G.Adj v w}`. It comes from Mathlib's double-counting library, so facts about `degIn` can be fed straight into Mathlib's double-counting lemmas. We prove a few simple lemmas about `degIn`, which we will use in Brooks' Theorem (p. 304), for the Moore and Cage Bounds (p. 346), and in the proof of Füredi's Stability Theorem (p. 419).

```
section degIn
open Finset
variable {V : Type*} {G : SimpleGraph V} [DecidableRel G.Adj]
```

The number of neighbours of `v` lying in `s`.

```
def degIn (G : SimpleGraph V) [DecidableRel G.Adj] (s : Finset V) (v : V) : ℕ :=
    #(s.bipartiteAbove G.Adj v)
```

`G.degIn s v` counts the vertices `w ∈ s` adjacent to `v`.

```
lemma degIn_eq_card_filter (s : Finset V) (v : V) :
    G.degIn s v = #{w ∈ s | G.Adj v w} := rfl
```

The set of neighbours of `x` lying in `s` is exactly `G.neighborFinset x ∩ s`.

```
lemma filter_adj_eq_inter [DecidableEq V] (x : V) [Fintype (G.neighborSet x)]
    (s : Finset V) : {v ∈ s | G.Adj x v} = G.neighborFinset x ∩ s := by
  ext v; simp [mem_filter, mem_inter, mem_neighborFinset, and_comm]
```

Since `G.Adj` is symmetric, `bipartiteBelow` also counts the neighbours of `v` in `s`.

```
lemma degIn_eq_card_bipartiteBelow (s : Finset V) (v : V) :
    G.degIn s v = #(s.bipartiteBelow G.Adj v) := by
  rw [degIn_eq_card_filter]
  congr 1
  ext; simp [adj_comm]
```

The number of neighbours of `x` in `s` is at most its degree.

```
lemma degIn_le_degree (s : Finset V) (x : V) [Fintype (G.neighborSet x)] :
    G.degIn s x ≤ G.degree x := by
```

```
  classical
  rw [degIn_eq_card_filter, filter_adj_eq_inter, ← card_neighborFinset_eq_degree]
  exact card_le_card inter_subset_left
```

If x has at least as many neighbours in s as its degree, then every neighbour of x lies in s.

```
lemma neighborFinset_subset_of_degree_le_degIn (x : V) [Fintype (G.neighborSet x)]
    (s : Finset V) (hle : G.degree x ≤ G.degIn s x) : G.neighborFinset x ⊆ s := by
  classical
  have : G.degIn s x = G.degree x := (degIn_le_degree s x).antisymm hle
  rw [degIn_eq_card_filter, filter_adj_eq_inter,
    ← card_neighborFinset_eq_degree] at this
  have heq := eq_of_subset_of_card_le inter_subset_left (le_of_eq this.symm)
  grind
```

If every neighbour of x lies in s, then x has `G.degree x` neighbours in s.

```
lemma degIn_eq_degree {x : V} [Fintype (G.neighborSet x)] {s : Finset V}
    (h : G.neighborFinset x ⊆ s) : G.degIn s x = G.degree x := by
  classical
  rw [degIn_eq_card_filter, filter_adj_eq_inter, inter_eq_left.mpr h,
    card_neighborFinset_eq_degree]
```

The `degIn` of a vertex is monotone in the set.

```
lemma degIn_mono {s t : Finset V} (h : s ⊆ t) (x : V) : G.degIn s x ≤ G.degIn t x :=
  card_le_card (filter_subset_filter _ h)
```

The `degIn` of a vertex is additive over a disjoint union of sets.

```
lemma degIn_union [DecidableEq V] (s t : Finset V) (x : V) (h : Disjoint s t) :
    G.degIn (s ∪ t) x = G.degIn s x + G.degIn t x := by
  simp only [degIn_eq_card_filter]
  rw [filter_union,
    card_union_of_disjoint (h.mono (filter_subset _ _) (filter_subset _ _))]
```

If some neighbour of x lies outside s, then x has strictly less than `G.degree x` neighbours in s.

```
lemma degIn_lt_degree_of_adj_notMem {x y : V} [Fintype (G.neighborSet x)]
    (s : Finset V) (hadj : G.Adj x y) (hy : y ∉ s) : G.degIn s x < G.degree x := by
  classical
  rw [degIn_eq_card_filter, filter_adj_eq_inter, ← card_neighborFinset_eq_degree]
  apply card_lt_card
  rw [ssubset_iff_of_subset inter_subset_left]
  exact ⟨y, (mem_neighborFinset _ _ _).mpr hadj, fun h => hy (mem_inter.mp h).2⟩
```

If every neighbour of x except possibly w lies in B, then x has at most one neighbour outside B.

```
lemma degree_le_degIn_add_one (B : Finset V) {x : V} [Fintype (G.neighborSet x)]
    (w : V) (h : ∀ y, G.Adj x y → y ≠ w → y ∈ B) : G.degree x ≤ G.degIn B x + 1 := by
  rw [← card_neighborFinset_eq_degree]
  classical
  calc #(G.neighborFinset x) ≤ #(insert w {v ∈ B | G.Adj x v}) :=
        card_le_card fun y hy => by
          rw [mem_neighborFinset] at hy
```

```
          by_cases hyw : y = w
          · simp [hyw]
          · simp [h y hy hyw, hy]
    _ ≤ G.degIn B x + 1 := card_insert_le _ _

end degIn
```

4.4 Computations with Hypercubes

In order to compute with the hypercube `Q[n]`, and to prove results about the degrees of its vertices and the size of its edge set, we need decidable equality of its vertices, decidable adjacency, and a `Fintype` instance for its vertices. As with many proofs for the hypercube, each of these drops out easily by induction on *n*.

Vertices of the hypercube have decidable equality.

```
instance hypercubeDecidableEq (n : ℕ) : DecidableEq (HypercubeVert n) := by
  induction n <;> unfold HypercubeVert <;> infer_instance
```

Since we used the box product to define the hypercube, decidability of adjacency in `Q[n]` follows from the box product instance `boxProdDecidableRelAdj` (see Decidability and Kernel Reduction (p. 57)).

Decidability of the adjacency relation for the hypercube.

```
instance hypercubeDecidableRelAdj (n : ℕ) : DecidableRel Q[n].Adj := by
  induction n <;> unfold hypercube <;> infer_instance

example : Q[2].Adj (((), false), false) (((), false), true) := by decide
```

The vertex type `HypercubeVert n` is finite.

```
instance hypercubeFintype (n : ℕ) : Fintype (HypercubeVert n) := by
  induction n <;> unfold HypercubeVert <;> infer_instance
```

The *n*-dimensional hypercube has `2 ^ n` vertices.

```
@[simp] lemma card_hypercubeVert (n : ℕ) :
    Fintype.card (HypercubeVert n) = 2 ^ n := by
  induction n with
  | zero => simp
  | succ n ih => rw [pow_succ, ← ih]; exact Fintype.card_prod _ _

example : Fintype.card (HypercubeVert 3) = 8 := by simp
```

We also give a `Nonempty` instance for `HypercubeVert n` since this will be useful in later chapters.

```
instance hypercubeNonempty {n : ℕ} : Nonempty (HypercubeVert n) := by
  induction n <;> unfold HypercubeVert <;> infer_instance
```

There is only one vertex in `Q[0]`, so we can give a `Subsingleton` instance for it.

```
instance hypercubeZeroSubsingleton : Subsingleton (HypercubeVert 0) :=
  inferInstanceAs (Subsingleton Unit)
```

Before we try some more interesting computations with hypercubes, we give a convenient way to describe and print vertices of the hypercube.

Hypercube vertex as a binary string.

```
def hypercubeVertToBin : (n : ℕ) → HypercubeVert n → String
  | 0, _ => ""
  | n + 1, (v, b) => hypercubeVertToBin n v ++ if b then "1" else "0"
```

Lean has the `Repr` type class for types that can be pretty printed. We can now use `hypercubeVertToBin` to give a `Repr` instance for `HypercubeVert n`.

A `Repr` instance for hypercube vertices to allow pretty printing.

```
instance hypercubeRepr {n : ℕ} : Repr (HypercubeVert n) where
  reprPrec v _ := hypercubeVertToBin n v
```

We can also define a convenient macro `v[...]` for writing out vertices of the hypercube. Since Mathlib makes `Bool` a Boolean ring, with `0 = false` and `1 = true`, we can write the coordinates as `0` and `1`.

```
namespace HypercubeVert
```

`v[b₀, …, bₙ₋₁]` is the vertex of `Q[n]` with coordinates `b₀, …, bₙ₋₁`, where `0` and `1` stand for `false` and `true`.

```
scoped syntax "v[" term,* "]" : term
scoped macro_rules
  | `(v[ $[$xs],* ]) => do
    let mut acc ← `(())
    for x in xs do
      acc ← `(($acc, $x))
    return acc

#eval Q[3].neighborFinset v[0,1,1] -- {111, 010, 001}

example : Q[3].Adj v[1, 1, 1] v[1, 0, 1] := by simp

end HypercubeVert
```

Returning now to computations involving hypercubes, we note that Lean can infer that every vertex in `Q[n]` has a "computable" `Fintype` instance for its neighbourhood using `DecidableRel Q[n].Adj` and `Fintype (HypercubeVert n)`.

We can use this to compute the degree of a vertex in `Q[15]`. However this is a bad idea since this is basically looking at each of the `2 ^ 15` vertices and checking adjacency. (In the case of a `Q[30]` vertex this would simply time out.)

```
#synth Q[15].LocallyFinite
-- fun v ↦ Subtype.fintype (Membership.mem (Q[15].neighborSet v))

open HypercubeVert -- for the v[...] vertex notation

#eval Q[15].degree v[0,1,0,0,1,1,1,1,1,1,0,1,1,0,1] -- 15
```

We can however give a more efficient `Fintype` instance for the neighbourhoods of vertices in `Q[n]` using Mathlib's custom `boxProdFintypeNeighborSet` instance for the box product, which is exactly how `Q[n]` is defined (for `n ≠ 0`).

```
#check boxProdFintypeNeighborSet
```

We use `inferInstanceAs` to tell Lean to build the neighbourhood through the box product. (Here `f ◁ x` means `f x`; it saves writing brackets around a long argument.) As with `locallyFiniteOfFintypeEdgeSet` and `blowUpNFintypeNeighborSet`, we make this a **`def`** rather than an **`instance`**.

An efficient `Fintype` instance for the neighbourhood of a vertex in `Q[n]`, built through the box product rather than by filtering all `2 ^ n` vertices.

```
@[instance_reducible]
def hypercubeFintypeNeighborSet (n : ℕ) (v : HypercubeVert n) :
    Fintype (Q[n].neighborSet v) := by
  induction n with
  | zero => infer_instance
  -- the induction hypothesis ih is in the local context, and local hypotheses of
  -- class type are candidates for instance synthesis, so this finds it
  | succ n ih => exact inferInstanceAs ◁ Fintype ((_ □ _).neighborSet _)
```

To compute with it we switch it on with `attribute [local instance]`, as we did for `blowUpNFintypeNeighborSet`. This time we want it for two commands, so we drop the `in` and put both commands in a **`section`**: the attribute then lasts until the end of the section. The trace output confirms that Lean now uses the efficient instance, so this is no longer simply filtering a set of `2 ^ 60` vertices!

```
section efficient_hypercube

attribute [local instance] hypercubeFintypeNeighborSet

set_option trace.Meta.synthInstance true in
#synth Q[60].LocallyFinite -- hypercubeFintypeNeighborSet 60

#eval Q[60].degree v[0,1,1,0,1,0,1,0,1,0,0,1,1,0,1,0,1,0,1,0,0,0,1,1,0,
  0,1,1,0,1,0,1,0,1,0,0,1,1,1,1,1,1,1,1,1,1,1,1,1,0,1,0,1,0,1,0,0,0,1,1,0] -- 60

end efficient_hypercube
```

The efficient instance also gives a short proof that `Q[n]` is regular of degree `n`. We give it in the next section, together with a look at what happens when a type *does* have two `Fintype` instances.

Stating Results Across a Diamond*

Because `hypercubeFintypeNeighborSet` is not an instance, `Q[n].neighborSet v` has a single `Fintype` instance, the filtering one, and the statement that `Q[n]` is n-regular simply uses it. The proof, however, needs the efficient instance: because it is built through the box product, **`change`** can restate a degree in `Q[n + 1]` as a degree in `Q[n] □ ⊤`, ready for `degree_boxProd`. So the proof switches instances, and this is what its first two lines do.

The **`let`** puts the efficient instance in the local context, where type class inference looks first. (It must be a **`let`**, not a **`have`**: an instance is data, and **`change`** needs to see its value.) It covers every

dimension `m`, not just `n`, since the induction step needs the efficient instance for `Q[n]` as well as for `Q[n + 1]`. Then `convert_to` restates the goal, so that its instance is synthesized as the local one, and closes the gap between the two instances using the `Subsingleton` argument from the proof of `sum_degrees_eq_twice_card_edges'`. In the InfoView the goal looks the same before and after, since instance arguments are not displayed; hovering over `IsRegularOfDegree` in the goal shows that the filtering instance has become `efficient n`. Because the switch happens inside the proof, no caller ever needs to know about it.

```
lemma IsRegularOfDegree.hypercube (n : ℕ) : Q[n].IsRegularOfDegree n := by
  -- Hovering over IsRegularOfDegree in the InfoView you will see that initially
  -- the goal is @IsRegularOfDegree _ Q[n] (fun v ↦ Subtype.fintype _) n
  -- which uses the filtering instance.
  let efficient: ∀ m (v : HypercubeVert m), Fintype (Q[m].neighborSet v) :=
    hypercubeFintypeNeighborSet
  -- Swap out the filtering instance for the efficient one, in every dimension
  convert_to Q[n].IsRegularOfDegree n
  -- goal is now @IsRegularOfDegree _ Q[n] (efficient n) n, with the efficient one
  induction n with
  | zero => exact IsRegularOfDegree.bot
  | succ n ih =>
    intro v
    -- Q[n + 1] is Q[n] □ ⊤ by definition, and this is the box product instance
    change (Q[n] □ ⊤).degree v = n + 1
    simp [degree_boxProd, ih v.1]
```

Since the statement uses the only instance there is, the lemma combines directly with Mathlib's results stated using `[Fintype V]` and `[DecidableRel G.Adj]`. For example, `IsRegularOfDegree.minDegree_eq` says that a graph that is regular of degree `d` has minimum degree `d`.

```
example (n : ℕ) : δ(Q[n]) = n := (IsRegularOfDegree.hypercube n).minDegree_eq
```

Mathlib does not take this route for the box product. Its `boxProdFintypeNeighborSet` is a global instance. Mathlib has no decidability instance for adjacency in a box product, but we gave one, `boxProdDecidableRelAdj`, in Decidability and Kernel Reduction (p. 57). So when `G` and `H` are finite with decidable adjacency, the filtering instance applies to `G □ H` too, and `(G □ H).neighborSet x` has two `Fintype` instances, the filtering one and the box product one. This is a `Fintype` *diamond*: the situation we warned about in Instance Arguments in Statements (p. 63) and Cardinality (p. 66), and explored in the proof of `sum_degrees_eq_twice_card_edges'`. Inference prefers the box product instance. Whatever decidability instance we gave, the filtering instance would be built differently from the box product one, so the only way to avoid the diamond would be to give up decidable adjacency for box products.

```
#synth (C[5] □ K[2]).LocallyFinite -- fun v ↦ boxProdFintypeNeighborSet v
```

The same happens for the disjoint union `G ⊕g H`: Mathlib has a `LocallyFinite` instance for it, and our decidability instance from More Decidable Adjacency Instances (p. 59) makes the filtering one available too.

```
#synth (C[4] ⊕g K[1]).LocallyFinite -- fun v ↦ instLocallyFiniteSumSumOfDecidableEq v
```

How much trouble does this cause? Nothing in Mathlib breaks because of it, and the reason is visible in `degree_boxProd`: its third instance argument is an *arbitrary* `Fintype` for the box product

neighbourhood [Fintype ↑((G □ H).neighborSet x)], not boxProdFintypeNeighborSet.

```
#check degree_boxProd
```

So degree_boxProd works with whichever instance is in scope where it is used. Two instances only cause difficulty when a theorem *statement* names one of them.

We therefore state that a box product of regular graphs is regular with its LocallyFinite instance as an argument. Without this, Lean would have written the box product instance into the statement, and the lemma could then only be used where that is the instance in scope. The proof needs no particular instance, since degree_boxProd doesn't.

```
lemma IsRegularOfDegree.boxProd {V W : Type*} {G : SimpleGraph V} {H : SimpleGraph W}
    [G.LocallyFinite] [H.LocallyFinite] [(G □ H).LocallyFinite] {d e : ℕ}
    (hG : G.IsRegularOfDegree d) (hH : H.IsRegularOfDegree e) :
    (G □ H).IsRegularOfDegree (d + e) := fun x => by
  rw [degree_boxProd, hG, hH]
```

Note, however, that an instance argument is filled in by *inference*, which for a box product finds the box product instance, not by matching the type that the caller expects. IsRegularOfDegree.minDegree_eq takes [DecidableRel G.Adj] like minDegree itself, so it asks for regularity with respect to the filtering instance, and the obvious proof fails.

```
/- error: Application type mismatch: The argument
  IsRegularOfDegree.boxProd hG hH
has type
  @IsRegularOfDegree (V × W) (G □ H) (fun v ↦ boxProdFintypeNeighborSet v) (d + e)
but is expected to have type
-/
example {V W : Type*} [Fintype V] [Fintype W] [Nonempty V] [Nonempty W]
    [DecidableEq V] [DecidableEq W] {G : SimpleGraph V} {H : SimpleGraph W}
    [DecidableRel G.Adj] [DecidableRel H.Adj] {d e : ℕ}
    (hG : G.IsRegularOfDegree d) (hH : H.IsRegularOfDegree e) : δ(G □ H) = d + e :=
  (hG.boxProd hH).minDegree_eq
```

Because our lemma takes its instance as an argument, putting the filtering instance in scope with let fixes this. (Had the box product instance been written into the statement, we would need convert instead.) This let is the price of the diamond: we need it whenever we combine the lemma with a result stated using [Fintype V] and [DecidableRel G.Adj].

```
example {V W : Type*} [Fintype V] [Fintype W] [Nonempty V] [Nonempty W]
    [DecidableEq V] [DecidableEq W] {G : SimpleGraph V} {H : SimpleGraph W}
    [DecidableRel G.Adj] [DecidableRel H.Adj] {d e : ℕ}
    (hG : G.IsRegularOfDegree d) (hH : H.IsRegularOfDegree e) :
    δ(G □ H) = d + e := by
  let : (G □ H).LocallyFinite := fun x => Subtype.fintype _
  exact (hG.boxProd hH).minDegree_eq
```

Our own IsRegularOfDegree.card_edgeFinset takes its instances as arguments, so it also combines with IsRegularOfDegree.hypercube directly.

The n-dimensional hypercube has $n * 2 ^ (n - 1)$ edges.

```
lemma card_edgeFinset_hypercube (n : ℕ) [Fintype Q[n].edgeSet] :
    Q[n].edgeFinset.card = n * 2 ^ (n - 1) := by
```

```
  rw [(IsRegularOfDegree.hypercube n).card_edgeFinset, card_hypercubeVert]
  cases n with
  | zero => rfl
  | succ n => rw [pow_succ, Nat.add_sub_cancel]; grind
```

All of this leads to two rules that we follow in the rest of the guide. Firstly, we avoid giving a type a second instance of the same class: a second `Fintype` instance, such as `hypercubeFintypeNeighborSet`, is a **`def`**, switched on locally where it is needed. Secondly, a statement that depends on a `Fintype` (or `LocallyFinite`) instance on a type with more than one, for example through `degree`, `neighborFinset` or `edgeFinset`, takes that instance as an argument. If the proof needs a particular instance, it switches to it with **`let`** and **`convert_to`**, as above. For a type with a single instance, such as `HypercubeVert n`, a statement about it simply uses that instance.

The second rule can only be applied to instances that a definition takes as arguments. Mathlib's `minDegree` and `maxDegree` take no neighbourhood instance: they take `[Fintype V]` and `[DecidableRel G.Adj]` and build the neighbourhood instances from these, by filtering. A statement about them therefore takes `[Fintype V]` and `[DecidableRel G.Adj]` as arguments, and its neighbourhood instance is always the filtering one. This is why `minDegree_eq` needed the **`let`** above.

4.5 Computations with Mycielskians

Recall the iterated Mycielskian graphs (`iterMycielskian`). In order to do computations with these graphs we need to provide computable instances of `Fintype` for their vertices and `DecidableRel` for adjacency. We do this first for the Mycielskian of a graph and then for the iterated Mycielskian.

```
variable {V : Type*} {G : SimpleGraph V}

#source mycielskian -- def mycielskian (G : SimpleGraph V) :
                    --     SimpleGraph (Option (V ⊕ V)) where
                    --   Adj := fun x y => match x, y with
                    --     | some (Sum.inl v), some (Sum.inl w) => G.Adj v w
                    --     | some (Sum.inl v), some (Sum.inr w) => G.Adj v w
                    --     | some (Sum.inr v), some (Sum.inl w) => G.Adj v w
                    --     | none, some (Sum.inr w) => True
                    --     | some (Sum.inr v), none => True
                    --     | _, _ => False
```

The Mycielskian of a graph with decidable adjacency also has decidable adjacency.

```
instance mycielskianDecidableRelAdj [DecidableRel G.Adj] : DecidableRel mμ(G).Adj :=
  fun x y => by
    rcases x with _ | (_ | _) <;> rcases y with _ | (_ | _)
       <;> unfold mycielskian <;> infer_instance

#source MycielskianVert -- notation: mμᵥ^n(V)
```

The vertex type of the n-th iterated Mycielskian of a graph with decidable vertex equality also has decidable vertex equality.

```
instance iterMycielskianDecidableEq (n : ℕ) [DecidableEq V] :
    DecidableEq mμᵥ^n(V) := by
  induction n <;> unfold MycielskianVert <;> infer_instance
```

The vertex type of the n-th iterated Mycielskian of a graph with finitely many vertices also has finitely many vertices.

```
instance iterMycielskianFintype (n : ℕ) [Fintype V] : Fintype mμᵥ^n(V) := by
  induction n <;> unfold MycielskianVert <;> infer_instance

#source iterMycielskian -- notation: mμ^n(G)
```

If G has decidable adjacency then so does the n-th iterated Mycielskian of G.

```
instance iterMycielskianDecidableRelAdj (n : ℕ) [DecidableRel G.Adj] :
    DecidableRel mμ^n(G).Adj := by
  induction n <;> unfold iterMycielskian <;> infer_instance
```

These instances combine with those from earlier in the chapter. For example, Lean can find a `Fintype` instance for the edge set of the 4th iterated Mycielskian of `C[5][2]`, the 2-uniform blow-up of `C[5]` (see `blowUpN`).

```
#synth Fintype mμ^4(C[5][2]).edgeSet -- mμ^4(C[5][2]).fintypeEdgeSet
```

The 6th iterated Mycielskian of `C[5]` has 383 vertices and 7271 edges; it has minimum degree 8 and maximum degree 191.

```
#eval Fintype.card mμᵥ^6(Fin 5)  -- 383
#eval mμ^6(C[5]).edgeFinset.card -- 7271
#eval δ(mμ^6(C[5]))              -- 8
#eval Δ(mμ^6(C[5]))              -- 191
```

Recall the notation we introduced for the vertices of the iterated Mycielskian. The vertex $\mathtt{v}^{\mathtt{l}}$ is the left copy of `v` in the Mycielskian of a `SimpleGraph V`, and $\mathtt{v}^{\mathtt{r}}$ is its right copy.

Recall also that the Grötzsch graph is the Mycielskian of `C[5]`. This has decidable adjacency so we can reason about it in proofs using **`decide`**.

```
example : grotzschGraph.Adj 2ˡ 3ʳ := by decide

example : mμ^7(C[5]).Adj (0 : Fin 5)ˡˡʳʳʳˡˡ (1 : Fin 5)ʳʳˡˡˡʳˡ := by decide
#eval mμ^10(C[5]).degree noneʳˡʳˡʳˡˡ -- 396
```

Mycielskian Degrees

As well as doing computations with Mycielskian graphs, we can also prove some general results about their vertex degrees and the size of their edge sets. We prove one such result below and leave the rest as exercises.

Note that we give no efficient `Fintype` construction for the neighbourhoods of vertices in the Mycielskian, as we did with `hypercubeFintypeNeighborSet` for the hypercube. There is no need: unlike the hypercube, the filtering instance is fast enough, since it filters only the `2 * Fintype.card V + 1` vertices of `Option (V ⊕ V)`, so the **`#eval`**s above run without difficulty.

The statement below mentions `G.degree v` and `mμ(G).degree v.l`, so, following the rule from Stating Results Across a Diamond (p. 80), it takes the `Fintype` instances for these two neighbourhoods as arguments. It needs nothing else; in particular it holds whether or not `V` is finite. The proof begins with the `classical` tactic, which supplies the `DecidableEq V` needed to form the images and the union of finsets below.

```
open Finset in
@[simp]
lemma degree_mycielskian_left (v : V) [Fintype (G.neighborSet v)]
    [Fintype (mμ(G).neighborSet v.l)] : mμ(G).degree v.l = 2 * G.degree v := by
  classical
  have hnb : mμ(G).neighborFinset v.l
      = (G.neighborFinset v).image (·.l) ∪ (G.neighborFinset v).image (·.r) := by
    ext x
    simp [← mem_neighborSet]
  rw [← card_neighborFinset_eq_degree, ← card_neighborFinset_eq_degree, hnb,
    card_union_of_disjoint (by simp [← Finset.disjoint_coe]),
    card_image_of_injective _ mycielskian.left_injective,
    card_image_of_injective _ mycielskian.right_injective, two_mul]
```

If `V` is finite then the vertex type `Option (V ⊕ V)` of the Mycielskian has `2 * |V| + 1` elements. (Counting the vertices of the n-th iterated Mycielskian is left as an exercise.)

This is not a `simp` lemma, since its left-hand side is not in simp normal form (The Simplifier (p. 17)): `simp` rewrites `Fintype.card (Option (V ⊕ V))` to `Fintype.card V + Fintype.card V + 1` using Mathlib's `simp` lemmas `Fintype.card_option` and `Fintype.card_sum`, so it would never use our lemma. That is also how the proof works, with `two_mul` rewriting the right-hand side to match.

```
lemma card_mycielskianVert_one [Fintype V] :
    Fintype.card (Option (V ⊕ V)) = 2 * Fintype.card V + 1 := by
  simp [two_mul]
```

We can list the edges of the Mycielskian of `C[5]` as follows but the output isn't very readable.

```
#eval mμ(C[5]).edgeFinset -- {s(none, some (Sum.inr 0)),...
```

The solution is to provide a custom `Repr` instance for the vertices of the Mycielskian. We make it a `scoped instance`: it is active inside `namespace SimpleGraph` (and wherever `SimpleGraph` is opened), so later chapters print Mycielskian vertices this way, but other code using `Option (V ⊕ V)` is unaffected.

```
scoped instance mycielskianRepr [Repr V] :
    Repr (Option (V ⊕ V)) where
  reprPrec x _ := match x with
    | none => "*"
    | v.l => repr v ++ "ˡ"
    | v.r => repr v ++ "ʳ"

#eval mμ(C[5]).edgeFinset -- {s(*, 0ʳ), s(*, 1ʳ),..., s(4ˡ, 3ʳ)}
```

We can extend this to the vertices of the iterated Mycielskian.

```
scoped instance iterMycielskianRepr [Repr V] (n : ℕ) :
    Repr mμᵥ^n(V) := by
```

```
  induction n <;> unfold MycielskianVert <;> infer_instance

#eval mμ^2(C[5]).edgeFinset -- {s(*, *ʳ), s(*, 0ˡʳ),..., s(4ʳˡ, 3ˡʳ)}
```

4.6 Degree Sequences*

There isn't a definition of degree sequence in Mathlib, so we provide one here. We will take this to be the non-increasing list of degrees of the vertices of a finite graph.

We start with the `Finset.univ` of all vertices of a `SimpleGraph V` and then take its underlying `Multiset` (`univ.val`) and map it to the `Multiset` of degrees. We will call this the `degreeMultiset` of the graph. (In The Laplacian (p. 372) we meet the *degree matrix*, the diagonal matrix whose entries are these degrees.)

The multiset of degrees of a finite graph.

```
def degreeMultiset [Fintype V] (G : SimpleGraph V) [DecidableRel G.Adj] :
    Multiset ℕ := Finset.univ.val.map (G.degree ·)
```

The *degree sequence* is then the sorted non-increasing list of degrees of the vertices of a finite graph as a `List ℕ`.

```
#check Multiset.sort -- returns a List
```

The degree sequence of a finite graph, sorted in non-increasing order.

```
def degreeSequence [Fintype V] (G : SimpleGraph V) [DecidableRel G.Adj] :
    List ℕ := G.degreeMultiset.sort (· ≥ ·)

#eval K[4, 5].degreeSequence     -- [5, 5, 5, 5, 4, 4, 4, 4, 4]

#eval mμ^6(C[5]).degreeSequence  -- [191, 190, 188, 184, 176, 160, 128, 128, ..., 8]
```

Graphic Sequences

The graph realization problem asks: which non-increasing sequences of non-negative integers occur as the degree sequence of a finite simple graph?

Rather than as a predicate on sorted lists, this is more naturally defined in Lean as a predicate on `Multiset ℕ` (i.e. a list in which order doesn't matter). (Later when we want to talk about graphic degree sequences we can simply use the fact that any `List ℕ` can be coerced to a `Multiset ℕ`.)

We will call a `Multiset ℕ` *graphic* if it is the multiset of degrees of some finite graph.

Since `G.degreeMultiset` requires `Fintype V` and `DecidableRel G.Adj` instances, we will quantify over an arbitrary finite vertex type `V : Type` and carry the `Fintype` and `DecidableRel` instances as existential witnesses that `degreeMultiset` requires. [5]

A multiset of degrees is *graphic* if it is the degree multiset of some finite simple graph.

[5] Every finite type, in any universe, is equivalent to a type in `Type` (indeed to `Fin n` for some `n`), and transporting a graph along such an equivalence leaves its degree multiset unchanged, so restricting to `Type` here loses no generality.

```
def Graphic (m : Multiset ℕ) : Prop :=
  ∃ (V : Type) (_ : Fintype V) (G : SimpleGraph V) (_ : DecidableRel G.Adj),
    m = G.degreeMultiset
```

Notice that `Graphic` doesn't actually mention the number of vertices in the graph even though it must be the same as the cardinality of the multiset `m`. We can easily deduce this as a theorem and leaving it out of the definition of `Graphic` makes it easier to work with.

```
theorem Graphic.exists_card_eq {m : Multiset ℕ} (h : Graphic m) :
    ∃ (V : Type) (_ : Fintype V) (G : SimpleGraph V) (_ : DecidableRel G.Adj),
      m = G.degreeMultiset ∧ Fintype.card V = m.card := by
  obtain ⟨V, hFV, G, hDG, rfl⟩ := h
  exact ⟨V, hFV, G, hDG, rfl, (Multiset.card_map _ _).symm⟩
```

With `Graphic` quantifying over an arbitrary finite vertex type, the fact that every finite simple graph has a graphic degree multiset is immediate: the graph itself is the witness.

```
theorem graphic_degreeMultiset {V : Type} [Fintype V] (G : SimpleGraph V)
    [DecidableRel G.Adj] : Graphic G.degreeMultiset :=
  ⟨V, inferInstance, G, inferInstance, rfl⟩
```

Since a `Multiset` is just a `List` with its order forgotten (i.e. quotiented by the permutation relation), there is a natural coercion from `List` to `Multiset`.

```
#synth Coe (List ℕ) (Multiset ℕ) -- Multiset.instCoeList
```

Every finite simple graph has a graphic degree sequence.

```
theorem graphic_degreeSequence {V : Type} [Fintype V] (G : SimpleGraph V)
    [DecidableRel G.Adj] : Graphic G.degreeSequence :=
  ⟨V, inferInstance, G, inferInstance, by simp [degreeSequence]⟩
```

The multiset consisting of n copies of k is defined by `Multiset.replicate n k`.

```
#check Multiset.replicate    -- Multiset.replicate (n : ℕ) (a : α) : Multiset α
#eval Multiset.replicate 5 3 -- {3, 3, 3, 3, 3}
```

The empty graph on `Fin n` has the all-zeros degree multiset.

```
theorem degreeMultiset_bot (n : ℕ) :
    E[n].degreeMultiset = Multiset.replicate n 0 := by
  simp [degreeMultiset]
```

A multiset of n zeros is graphic, realized by the empty graph.

```
theorem graphic_zeros (n : ℕ) : Graphic (Multiset.replicate n 0) :=
  degreeMultiset_bot n ▸ graphic_degreeMultiset E[n]
```

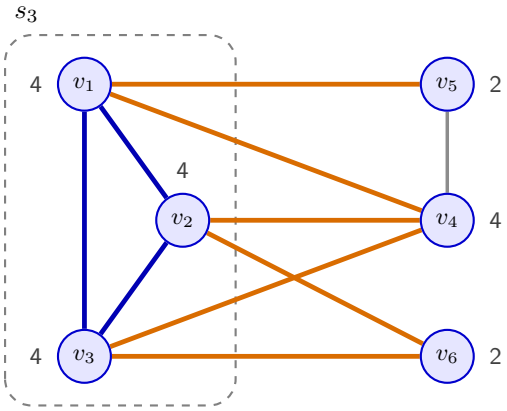
In particular the empty multiset is graphic (notice that the empty multiset is denoted by `0`).

```
lemma graphic_zero : Graphic 0 := graphic_zeros 0
```

The Erdős–Gallai Theorem

There are two major results in graph theory that characterize graphic sequences. One is the *Erdős–Gallai* theorem [Erdős–Gallai 1960]: given a non-increasing sequence `[d₁, d₂, …, dₙ]` this says that the sequence is graphic iff its sum is even and for each `k ≤ n` we have `(d₁ + d₂ + … + dₖ) ≤ k * (k - 1) + (min(k, dₖ₊₁) + min(k, dₖ₊₂) + … + min(k, dₙ))`.

The necessity of this is easy informally: suppose we have a graph with this degree sequence and `deg(vᵢ) = dᵢ`. Summing the first `k` terms we count the edges in the subgraph induced by `sₖ = {v₁, …, vₖ}` twice and the edges in the bipartite graph between `sₖ` and its complement once. No subgraph has more edges than a clique, so the induced subgraph on `sₖ` has at most `k * (k - 1) / 2` edges and contributes at most `k * (k - 1)` to the sum. Each vertex `vₖ₊ᵢ` sends at most `min(k, dₖ₊ᵢ)` edges to `sₖ`, and summing these bounds the number of edges in the bipartite graph.



The case `k = 3` of the Erdős–Gallai conditon: `d₁ + d₂ + d₃ = 12` counts each edge inside `s₃` twice and each edge leaving `s₃` once, and is at most `3 * 2 + min(3, 4) + min(3, 2) + min(3, 2) = 13`

We can use `Multiset.sort`, `List.sum`, `List.take` and `List.drop` to express this predicate in Lean. `Multiset.sort` sorts a multiset and returns the sorted `List` while `List.sum` sums the elements of a list. The `List.take k` function takes the first `k` elements of a list (or the whole list if it has less than `k` elements) and `List.drop k` drops the first `k` elements of a list (or returns the empty list if it has less than `k` elements).

```
#check Multiset.sort
#check List.sum
#check List.take
#check List.drop
```

The predicate `ErdosGallaiLE m k` is true if the k-th Erdős–Gallai inequality holds for m : `Multiset ℕ` and k : $\mathbb{N}$.

```
abbrev ErdosGallaiLE (m : Multiset ℕ) (k : ℕ) : Prop :=
    ((m.sort (· ≥ ·)).take k).sum ≤
        k * (k - 1) + (((m.sort (· ≥ ·)).drop k).map (min k)).sum
```

Now the Erdős–Gallai theorem can be stated as follows.

The **Erdős–Gallai theorem**: a multiset m of natural numbers is graphic iff its sum is even and `Erdos-GallaiLE m k` holds for every k ≤ `m.card`.

```
theorem erdos_gallai (m : Multiset ℕ) :
    (Even m.sum ∧ ∀ (k : ℕ), k ≤ m.card → ErdosGallaiLE m k) ↔ Graphic m := by
```

```
  sorry
```

The Erdős–Gallai conditions are easy to check by computer, so (modulo its proof) the theorem gives an efficient decision procedure for graphic sequences. It is a beautiful and very useful result, but it doesn't actually give us a construction of a graph from a given graphic sequence.

The Havel–Hakimi Theorem: a Computable Constructor for Graphic Sequences

The *Havel–Hakimi* theorem [Havel 1955] [Hakimi 1962] also characterizes graphic sequences and in the process provides a constructive method for producing a graph with a given degree sequence.

The Havel–Hakimi theorem says that a non-increasing sequence `[d₁, d₂, …, dₙ]` is graphic iff the sequence reduces to the empty sequence using the following recursive procedure:

1. If the list is empty then accept.
2. Otherwise the list is `d :: tail` for some `d : ℕ`. If `tail` has at least `d` entries and none of its first `d` entries is zero, then remove the first entry from the list, reduce each of the next `d` entries by `1`, and recurse; otherwise reject.

The recursive call must again be given a non-increasing list. Rather than sorting from scratch we can *merge*: the `d` decremented entries are still in non-increasing order (subtracting `1` preserves the order, and since each was at least `1` no entry is truncated at `0`), and so are the remaining entries, being a suffix of a sorted list. Merging two sorted lists with `List.merge` takes linear time and gives exactly the same list that sorting would have produced.

The check that none of the first `d` entries is zero matters even more in Lean than on paper. Subtraction on `ℕ` is truncated, so `0 - 1 = 0`, and without the check `[1, 0]` would reduce to `[0]` and then to `[]`, and so be accepted, although no graph has one vertex of degree `1` and one of degree `0`.

Havel–Hakimi, assuming the input list is sorted in non-increasing order.

```
def havelHakimiAux : List ℕ → Bool
  | [] => true -- the empty graph on no vertices
  | d :: tail =>
    let s := tail.take d -- s is the next d entries
    if (s.length == d) && s.all (· != 0) then
      -- if s has at least d entries and all are non-zero then
      -- reduce each of the next d entries by 1 and recurse
      havelHakimiAux ((s.map (· - 1)).merge (tail.drop d) (· ≥ ·))
    else
      false
  -- The recursive call is not on a sub-list of the input, so Lean cannot see for
  -- itself that havelHakimiAux terminates.
  -- termination_by tells it to check that the length of the list decreases
  --  at each call, and decreasing_by proves this.
termination_by l => l.length
decreasing_by
  grind [List.length_merge]
```

So we can decide whether a `Multiset ℕ` or `List ℕ` is graphic using Havel–Hakimi.

The input `m` is first sorted into non-increasing order, so this is correct for an arbitrarily ordered list of degrees.

Decide whether a `Multiset ℕ` or `List ℕ` is graphic using Havel–Hakimi.

```
def havelHakimi (m : Multiset ℕ) : Bool := havelHakimiAux (m.sort (· ≥ ·))

#eval havelHakimi [6, 5, 4, 3, 2, 1, 1] -- false: it reduces to [4, 3, 2, 1, 0, 0]
                                        -- which has too many zeros so is rejected.
```

The **Havel–Hakimi theorem**: the procedure `havelHakimi` accepts *m* iff *m* is graphic.

```
theorem havel_hakimi (m : Multiset ℕ) : havelHakimi m ↔ Graphic m := by
  sorry
```

Modulo a proof of the Havel–Hakimi theorem, we have an efficient decision procedure. We make it a `local instance`, so that an instance based on a `sorry` is not available outside of this chapter.

```
local instance : DecidablePred Graphic := fun m =>
  decidable_of_iff _ (havel_hakimi m)
```

Since this instance relies on the `sorry` in the proof of `havel_hakimi`, `#eval` will refuse to evaluate it, but we can still use it with `#eval!`, and the answers it gives are still meaningful. As we saw in Decidability and Kernel Reduction (p. 57), the verdict of `decidable_of_iff` comes entirely from the `Decidable` instance for its left-hand side, here `havelHakimi m = true`. The proof `havel_hakimi` of the equivalence is used only to build the accompanying proof terms, and compiled code erases those. So `#eval!` genuinely runs `havelHakimi`. What the `sorry` leaves unverified is *only* the claim that its verdict agrees with `Graphic`.

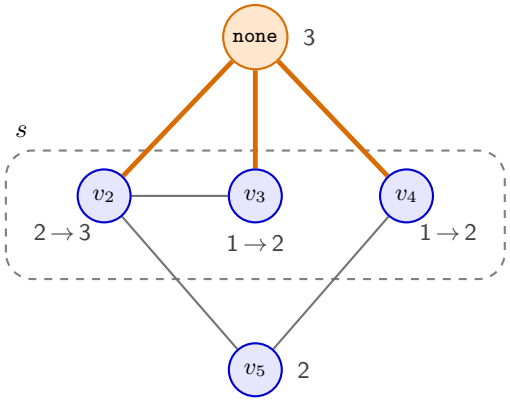
```
#eval! Graphic mμ^3(C[5]).degreeMultiset -- true
#eval! Graphic [6, 5, 4, 3, 2, 1, 1]     -- false
```

As with the Erdős–Gallai theorem we won't attempt a proof, but one of the crucial ingredients is the following `coneGraph` construction. Given `H : SimpleGraph V` and `s : Set V` this is the graph formed by adding a new vertex adjacent to exactly the vertices in `s`. We build this on the vertex type `Option V` in the obvious way.

The cone graph appears naturally in the proof of the Havel–Hakimi theorem. We need to show that a sequence $[d_1, d_2, \ldots, d_n]$ is graphic iff the sequence $[d_2 - 1, \ldots, d_{k+1} - 1, d_{k+2}, \ldots, d_n]$ is graphic where $k = d_1$. The reverse direction is easy: if the second sequence is graphic and it is realized by the graph *H*, and *s* is the set of d_1 vertices of *H* with degrees $d_2 - 1, \ldots, d_{k+1} - 1$ then `coneGraph H s` is a graph that realizes the original degree sequence.

`coneGraph H s` adds a new vertex `none` to *H*, adjacent to precisely the vertices in the set *s*.

```
@[simps]
def coneGraph (H : SimpleGraph V) (s : Set V) : SimpleGraph (Option V) where
  Adj x y := match x, y with
    | none, some w => w ∈ s
    | some w, none => w ∈ s
    | some a, some b => H.Adj a b
    | none, none => False
  symm := ⟨by rintro (_ | a) (_ | b) h <;> simp_all [SimpleGraph.adj_comm]⟩
  loopless := ⟨by rintro (_ | a) h <;> simp_all⟩
```



A Havel–Hakimi step: `[3, 3, 2, 2, 2]` reduces to `[2, 2, 1, 1]`, realized by the path `H`. `coneGraph H s` joins none to the $d_1 = 3$ vertices of `s` (orange edges). Degrees in `H` and `coneGraph H s` are shown

If membership of the set `s` is decidable then the cone graph has a decidable adjacency relation (assuming `H` does).

```
instance (H : SimpleGraph V) (s : Set V) [DecidablePred (· ∈ s)]
    [DecidableRel H.Adj] : DecidableRel (coneGraph H s).Adj := by
  rintro (_ | a) (_ | b) <;> unfold coneGraph <;> infer_instance
```

If the set `s` is finite then the new vertex has degree `s.card`.

The neighbourhood of `none` is the image of `s` under `some : V → Option V`, which we form with `Finset.map`. This maps a `Finset` along an *embedding* `f : α ↪ β`, that is, an injective function bundled with a proof that it is injective (we study embeddings properly in Maps (p. 98)); here `f` is `Function.Embedding.some`. Since `f` is injective there are no duplicates to remove, so unlike `Finset.image` it needs no `DecidableEq β`, and `Finset.card_map` says that it preserves cardinality.

```
#check Finset.card_map -- (s.map f).card = s.card

@[simp]
theorem degree_coneGraph_none (H : SimpleGraph V) (s : Finset V)
    [Fintype ((coneGraph H s).neighborSet none)] :
    (coneGraph H s).degree none = s.card := by
  have h : (coneGraph H s).neighborFinset none = s.map Function.Embedding.some := by
    ext x; cases x <;> simp
  rw [← card_neighborFinset_eq_degree, h, Finset.card_map]
```

This is still a long way from the proof of Havel–Hakimi but we have at least one crucial ingredient!

We can adapt the decision procedure `havelHakimiAux` above to construct the edges of a graph with any given graphic degree sequence based on the `coneGraph` construction.

The following function takes a `List (ℕ × ℕ)`, i.e. a list of pairs `(d, v)`, where the labels `v : ℕ` should be distinct and `d` is the required degree of the vertex `v`. If the sequence is graphic it returns `some` list of the edges of a graph realizing it, each edge given once as a pair `(v, w)`, in which each vertex `v` has degree `d`; if not, it returns `none`. (The input list should be sorted in non-increasing order of degrees.)

Each recursive step is exactly one `coneGraph` construction: it realizes the reduced sequence by some graph `H`, then joins a new vertex `v` onto the `d` highest degree vertices (no edges at all when `d`

= 0), so the returned edge list is built up "cone by cone". The inline comments below relate each piece to `coneGraph H s`.

Constructive Havel–Hakimi: given a labelled degree sequence `(d, v)`, sorted by non-increasing `d`, return the edge list of a graph realizing it, or none if the sequence is not graphic.

```
def havelHakimiEdges : List (ℕ × ℕ) → Option (List (ℕ × ℕ))
  | [] => some []      -- empty list returns no edges
  | (d, v) :: tail =>  -- vertex v has degree d (possibly 0)
    let s := tail.take d
    if (s.length == d) && s.all (·.1 != 0) then
      -- this is the coneGraph reverse direction above
      let next := (s.map (fun (p : ℕ × ℕ) => (p.1 - 1, p.2))).merge (tail.drop d)
                  (fun a b => a.1 ≥ b.1)
      -- recurse to realize the sequence next by a graph H with edge list E
      match havelHakimiEdges next with
      | some E =>
        -- the edges coneGraph H s adds are v joined to each vertex of s.
        let new := s.map fun (p : ℕ × ℕ) => (v, p.2)
        some (new ++ E)
      | none => none -- next is not graphic, so neither is the original sequence
    else none
termination_by l => l.length
decreasing_by
  grind [List.length_merge]

#eval havelHakimiEdges ◁ [5, 4, 4, 4, 4, 3, 2].zip (.range 7)
    -- some [(0, 1), (0, 2), (0, 3), (0, 4), (0, 5), (1, 2), (1, 3), (1, 4),
    --       (2, 3), (2, 4), (5, 6), (5, 3), (6, 4)]
```

We can check this answer by building the graph. Any relation `r` on `V` gives a graph `SimpleGraph.fromRel r`, which makes `r` symmetric and removes any loops. As we saw for `pathNatFromRelAbbrev`, Lean synthesizes decidable adjacency for such a graph when `r` is decidable, provided the graph is an **abbrev**, so we can compute its degree sequence.

The graph on `Fin n` whose edges are the pairs in `E`.

```
abbrev graphOfEdgeList (n : ℕ) (E : List (ℕ × ℕ)) : SimpleGraph (Fin n) :=
  SimpleGraph.fromRel fun v w => (v.val, w.val) ∈ E

#eval (havelHakimiEdges ◁ [5, 4, 4, 4, 4, 3, 2].zip (.range 7)).map
  fun E => (graphOfEdgeList 7 E).degreeSequence -- some [5, 4, 4, 4, 4, 3, 2]

#eval (havelHakimiEdges ◁ [6, 5, 4, 3, 2, 1, 1].zip (.range 7)).map
  fun E => (graphOfEdgeList 7 E).degreeMultiset -- none
```

If we give the `havelHakimiEdges` function a graphic degree sequence for a particular graph there is absolutely no reason it should return the edges of the same graph (or even an isomorphic copy of it).

For example if we give it the degree sequence of `C[6]` it returns the edges of two disjoint copies of `C[3]`, i.e. `C[3] ⊕g C[3]`.

```
#eval C[6].degreeSequence                    -- [2, 2, 2, 2, 2, 2]
#eval havelHakimiEdges                       -- some [(0, 1), (0, 2), (3, 4),
```

```
          ◁ C[6].degreeSequence.zip (.range 6)     --       (3, 5), (4, 5), (1, 2)]

#eval (C[3] ⊕g C[3]).degreeSequence = C[6].degreeSequence -- true
```

4.7 Exercises

1. Give a decidable adjacency instance for the line graph `L[G]` of a graph on a vertex type with decidable equality. Recall that the vertices of `L[G]` are the edges of `G`, and that two of them are adjacent iff they are distinct and share a vertex (`lineGraph_adj_iff_exists`).

Hint: `∃ v, P v` is not decidable in general, but the bounded `∃ v ∈ s, P v` is when `s` is a `Finset` and `P` is decidable. Turn e_1 `: Sym2 V` into a `Finset V` with `Sym2.toFinset`, note that `v ∈` e_2 is decidable (`Sym2.Mem.decidable`), and finish with `decidable_of_iff`.

```
variable {V : Type*} {G : SimpleGraph V}

example [DecidableEq V] : DecidableRel L[G].Adj :=
  sorry
```

The next few exercises are on finiteness and counting.

2. In the chapter we saw that `bad_card_edgeFinset` demanded more instances than its statement used. Prove that a graph has at most `Fintype.card (Sym2 V)` edges, assuming only the two instances that the statement itself mentions.

```
example [Fintype G.edgeSet] [Fintype (Sym2 V)] :
    G.edgeFinset.card ≤ Fintype.card (Sym2 V) := by
  sorry
```

3. Show that the edge set of a graph on a finite vertex type is finite, using `Finite` rather than `Fintype`. Note that no `DecidableRel G.Adj` is needed, unlike for `Fintype G.edgeSet`: why not?

```
example [Finite V] : Finite G.edgeSet := by
  sorry
```

4. The chapter showed `G.edgeSet.ncard = G.edgeFinset.card`. Prove the $\mathbb{N}\infty$ version.

```
example [Fintype G.edgeSet] : G.edgeSet.encard = G.edgeFinset.card := by
  sorry
```

5. The infinite path `pathNat` has infinitely many edges. Show that the $\mathbb{N}\infty$-valued `encard` of its edge set is therefore `⊤`, while the $\mathbb{N}$-valued `ncard` returns the junk value `0`. Hint: the edges `s(n, n + 1)` are distinct for distinct `n`, so `Set.infinite_of_injective_forall_mem` shows that the edge set is infinite.

```
example : pathNat.edgeSet.encard = ⊤ ∧ pathNat.edgeSet.ncard = 0 := by
  sorry
```

6. The degree of a vertex is the `Nat.card` of its neighbourhood.

```
example (v : V) [Fintype (G.neighborSet v)] :
    Nat.card (G.neighborSet v) = G.degree v := by
  sorry
```

7. In the complete graph every vertex is adjacent to every other vertex: its `neighborFinset` is `Finset.univ` with the vertex itself removed.

```
example (n : ℕ) (v : Fin n) : K[n].neighborFinset v = Finset.univ.erase v := by
  sorry
```

8. If *G* is locally finite there is a vertex of degree at least 3 iff `∃ a b c d, G.Adj a b ∧ G.Adj a c ∧ G.Adj a d ∧ b ≠ c ∧ b ≠ d ∧ c ≠ d`.

```
lemma exists_three_le_degree_iff' [G.LocallyFinite] : (∃ v, 3 ≤ G.degree v) ↔
    ∃ a b c d, G.Adj a b ∧ G.Adj a c ∧ G.Adj a d ∧ b ≠ c ∧ b ≠ d ∧ c ≠ d := by
  sorry
```

9. We saw that a vertex *w* of the n-uniform blow-up *G*`[n]` has a finite neighbourhood whenever *w*`.fst` has a finite neighbourhood in *G*. In this exercise we generalize this to blow-ups whose classes may have different sizes and need not all be finite: it is enough that the classes of the neighbours of *w*`.fst` are finite. As with `blowUpNFintypeNeighborSet`, we make this a **`def`** rather than an **`instance`**.

```
@[instance_reducible]
def blowUpGraphFintypeNeighborSet' {ι : Type*} (H : SimpleGraph ι) (V : ι → Type*)
    (w : Σ i, V i) [Fintype (H.neighborSet w.fst)]
    [∀ k : H.neighborSet w.fst, Fintype (V k)] :
    Fintype ((H.blowUpGraph V).neighborSet w) :=
  Fintype.ofEquiv (Σ k : H.neighborSet w.fst, V k)
    -- the goal is to show the obvious Equiv between neighbourhoods
    sorry
```

10. The infinite path `pathNat` has no minimum or maximum degree in the sense of `minDegree` and `maxDegree`, since its vertex type is infinite. Show instead that `1` is the least, and `2` the greatest, degree of a vertex of `pathNat`.

```
example : IsLeast {d | ∃ v, pathNat.degree v = d} 1 ∧
    IsGreatest {d | ∃ v, pathNat.degree v = d} 2 := by
  sorry
```

11. Show that *k* times the number of vertices of degree at least *k* is at most twice the number of edges. (The set of such vertices is a `Finset` written with the set-builder notation of Set-Builder Notation (p. 61). Compare it with the handshake lemma `sum_degrees_eq_twice_card_edges'`.)

```
example [Fintype V] [Fintype G.edgeSet] [G.LocallyFinite] (k : ℕ) :
    k * ({v | k ≤ G.degree v} : Finset V).card ≤ 2 * G.edgeFinset.card := by
  sorry
```

12. A graph on an odd number of vertices that is regular of degree d must have d even. Hint: use `IsRegularOfDegree.two_mul_card_edgeFinset`. The statement does not mention the edge set, so get the `Fintype G.edgeSet` instance that the proof needs from `classical`.

```
example [Fintype V] [G.LocallyFinite] {d : ℕ} (h : G.IsRegularOfDegree d)
    (hV : Odd (Fintype.card V)) : Even d := by
  sorry
```

13. Prove that the complete graph on `Fin (n + 1)` has `n * (n + 1) / 2` edges.

```
example (n : ℕ) : K[n + 1].edgeFinset.card = n * (n + 1) / 2 := by
  sorry
```

14. Show that in any finite graph `G.edgeFinset.card ≤ Fintype.card V * (Fintype.card V - 1) / 2`.

```
example [Fintype V] [Fintype G.edgeSet] :
    G.edgeFinset.card ≤ Fintype.card V * (Fintype.card V - 1) / 2 := by
  sorry
```

15. The complement of the hypercube `Q[n]` is regular of degree `2 ^ n - 1 - n`. Hint: use `IsRegularOfDegree.compl`, `IsRegularOfDegree.hypercube` and `card_hypercubeVert`.

```
example (n : ℕ) : Q[n]ᶜ.IsRegularOfDegree (2 ^ n - 1 - n) := by
  sorry
```

16. Prove that if V is finite then the n-th iterated Mycielskian has `2 ^ n * Fintype.card V + 2 ^ n - 1` vertices.

```
lemma card_mycielskianVert' (n : ℕ) [Fintype V] :
    Fintype.card mμᵥ^n(V) = 2 ^ n * Fintype.card V + 2 ^ n - 1 := by
  sorry
```

The next few exercises are on the degrees of vertices in the Mycielskian.

As in `degree_mycielskian_left`, statements about degrees and edge sets take the `Fintype` instances they mention as arguments. Those about `δ`, `Δ` and `degreeMultiset` take `[Fintype V]` and `[DecidableRel G.Adj]` instead, since these definitions build their own instances from them. Use the `classical` tactic for any further instances a proof needs. (We also `open Finset` since we will need quite a few lemmas about `Finset`.)

17. The degree of the new vertex `none` in the Mycielskian is the number of vertices in the original graph.

```
open Finset

lemma degree_mycielskian_none' [Fintype V] [Fintype (mμ(G).neighborSet none)] :
    mμ(G).degree none = Fintype.card V := by
  sorry
```

18. The degree of the right copy of a vertex in the Mycielskian is one more than the degree of the vertex in the original graph.

```
lemma degree_mycielskian_right' (v : V) [Fintype (G.neighborSet v)]
    [Fintype (mμ(G).neighborSet vʳ)] :
    mμ(G).degree vʳ = G.degree v + 1 := by
  sorry
```

19. The number of edges in `mμ(G)` is `3 * G.edgeFinset.card + Fintype.card V`. Hint: use `sum_degrees_eq_twice_card_edges'`.

```
lemma card_edgeFinset_mycielskian' [Fintype V] [Fintype G.edgeSet]
    [Fintype mμ(G).edgeSet] :
    #mμ(G).edgeFinset = 3 * #G.edgeFinset + Fintype.card V := by
  sorry
```

20. The maximum degree of `mμ(G)` is the maximum of `2 * Δ(G)` and the number of vertices in `G`. Hint: `exists_maximal_degree_vertex`, after treating `IsEmpty V` separately.

```
lemma maxDegree_mycielskian' [Fintype V] [DecidableRel G.Adj] :
    Δ(mμ(G)) = max (2 * Δ(G)) (Fintype.card V) := by
  sorry
```

21. The minimum degree of `mμ(G)` is the minimum of `2 * δ(G)` and `δ(G) + 1`. Deduce that `δ(mμ(G)) = δ(G) + 1` whenever `δ(G) ≥ 1`. Hint: `exists_minimal_degree_vertex`, after treating `IsEmpty V` separately.

```
lemma minDegree_mycielskian' [Fintype V] [DecidableRel G.Adj] :
    δ(mμ(G)) = min (2 * δ(G)) (δ(G) + 1) := by
  sorry

lemma minDegree_mycielskian_of_one_le' [Fintype V] [DecidableRel G.Adj]
    (h : 1 ≤ δ(G)) : δ(mμ(G)) = δ(G) + 1 := by
  sorry
```

22. Every graphic multiset has an even sum. (This shows that the parity condition in the Erdős–Gallai theorem is necessary.)

```
theorem Graphic.even_sum' {m : Multiset ℕ} (h : Graphic m) : Even m.sum := by
  sorry
```

23. In the chapter we computed the degree of the new vertex of `coneGraph H s`. Show that an old vertex's degree goes up by one exactly when it lies in `s`.

```
lemma degree_coneGraph_some' [DecidableEq V] (H : SimpleGraph V) (s : Finset V)
    (w : V) [Fintype (H.neighborSet w)]
    [Fintype ((coneGraph H s).neighborSet (some w))] :
    (coneGraph H s).degree (some w) = H.degree w + (if w ∈ s then 1 else 0) := by
  sorry
```

24. The `degreeMultiset` of the cone: the new vertex contributes `#s`, and every old vertex has its old degree plus one when it lies in `s`. Hint: the stub below records that `Finset.univ` on `Option V` is none followed by the image of `Finset.univ` under `some`. This holds by `rfl`.

```
theorem degreeMultiset_coneGraph' [Fintype V] [DecidableEq V] (H : SimpleGraph V)
    [DecidableRel H.Adj] (s : Finset V) :
    (coneGraph H s).degreeMultiset = #s ::ₘ (Finset.univ : Finset V).val.map
      (fun w => H.degree w + if w ∈ s then 1 else 0) := by
  have h : (Finset.univ : Finset (Option V)).val =
      none ::ₘ (Finset.univ : Finset V).val.map some := rfl
  sorry
```

25. The `degreeMultiset` of `mμ(G)` is the multiset of degrees of `G` with each degree doubled, together with the degrees of the new vertices. Hint: the stub below records how `Finset.univ` unfolds on `Option (V ⊕ V)` and on `V ⊕ V`. Both facts hold by `rfl`, as in Exercise 24.

```
lemma degreeMultiset_mycielskian' [Fintype V] [DecidableRel G.Adj] :
    mμ(G).degreeMultiset = {Fintype.card V} + G.degreeMultiset.map (2 * ·)
                         + G.degreeMultiset.map (· + 1) := by
  simp only [degreeMultiset]
  have h1 : (Finset.univ : Finset (Option (V ⊕ V))).val =
        none ::ₘ Multiset.map Option.some (Finset.univ : Finset (V ⊕ V)).val := rfl
  have h2 : (Finset.univ : Finset (V ⊕ V)).val = Multiset.map Sum.inl
              Finset.univ.val + Multiset.map Sum.inr Finset.univ.val := rfl
  sorry
```

5 Maps

Maps, in particular graph homomorphisms, are a fundamental concept in graph theory but they are often glossed over in informal treatments. In Lean they play a much more central role. For example, to say that a graph contains an induced 5-cycle we need to talk about embeddings of `C[5]` into the graph. Underlying this is the fact that any `SimpleGraph V` has a fixed vertex type `V`. So even to say that a graph `G` contains a copy of `H`, when the two have different vertex types, we need a map.

In this chapter we will study mappings between graphs. We first review functions, embeddings, and equivalences of types, and then the construction `SimpleGraph.comap`, which pulls a graph back along a function (it is used to define induced subgraphs). Next we introduce graph homomorphisms (`→g`), graph embeddings (`↪g`), and graph isomorphisms (`≃g`), together with `SimpleGraph.Copy`, the type of injective graph homomorphisms. We then look at `SimpleGraph.map`, which pushes a graph forward along a function, and at the natural maps associated with our running examples, the hypercube and the Mycielskian. Next we use graph automorphisms to define vertex-transitivity, a notion that recurs throughout the guide.

The chapter ends with two more advanced sections, on the tensor product of graphs and on claw-free graphs, where we characterize when the line graph of a graph contains a triangle (`top_three_isContained_lineGraph_iff`).

5.1 Functions, Embeddings and Equivalences

The basic function type in Mathlib is `V → W`, but we also have the embedding type `V ↪ W` for injective functions and the equivalence type `V ≃ W` for equivalences (bijections). Both are *bundled*: a term of each type is a structure packaging a function (or in the case of an equivalence, a function and its inverse) together with proofs of the properties it is required to have.

An *embedding* consists of a function together with a proof that it is injective.

```
section functions
variable {V W X : Type*}

#source Function.Embedding

example (f : V ↪ W) : Function.Injective f := f.injective
```

The identity function as an embedding.

```
example : V ↪ V := Function.Embedding.refl V
```

We can compose embeddings using `.trans`.

```
example (f : V ↪ W) (g : W ↪ X) : V ↪ X := f.trans g
```

An *equivalence* is not simply a bijective function: it is a pair of functions together with proofs that they are left/right inverses of each other.

```
#source Equiv
```

Any equivalence is a bijection.

```
example (e : V ≃ W) : Function.Bijective e := e.bijective
```

Conversely, `Equiv.ofBijective` turns a bijective function into an equivalence, but it is `noncomputable`: for each `w : W` the inverse has to *choose* some `v` with `f v = w`, and a proof of surjectivity only tells us that such a `v` exists.

```
noncomputable example (f : V → W) (hf : Function.Bijective f) : V ≃ W :=
  Equiv.ofBijective f hf
```

The *inverse* of an equivalence `e` is `e.symm`.

```
example (e : V ≃ W) : W ≃ V := e.symm
```

Examples of equivalences include the identity equivalence and the equivalence that swaps two given elements of a type.

```
example : V ≃ V := Equiv.refl V
example (u v : V) [DecidableEq V] : V ≃ V := Equiv.swap u v
```

Equivalences can be composed using `.trans`.

```
example (e : V ≃ W) (f : W ≃ X) : V ≃ X := e.trans f
```

Given embeddings `V ↪ W` and `W ↪ V` the Cantor–Schröder–Bernstein theorem says that there exists an equivalence `V ≃ W`. Its proof is non-constructive, so Mathlib states it as `Nonempty (V ≃ W)`, a proposition saying that an equivalence exists, rather than as a definition producing the equivalence itself, which would be data. (Compare `Finite` and `Fintype` in Finiteness Without Data: the Finite Type Class (p. 67).)

```
example (f : V ↪ W) (g : W ↪ V) : Nonempty (V ≃ W) :=
  Function.Embedding.antisymm f g

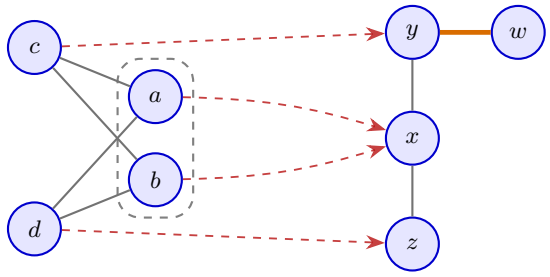
end functions
```

5.2 Comaps

Given a function `f : V → W`, we can push a graph `G : SimpleGraph V` forwards along `f` or pull a graph `H : SimpleGraph W` back along `f`. The former is called the *map* of `G` along `f` and the latter is called the *comap* of `H` along `f`.

Of these, `SimpleGraph.comap` is used frequently in constructions of concrete graphs, so we start with it. (The `SimpleGraph.map` construction is less frequently used for constructing concrete graphs, but often appears in proofs; we return to it in Maps (p. 107), once we have met graph homomorphisms.)

Given `f : V → W` and `G : SimpleGraph W`, `G.comap f` is the `SimpleGraph V` where two vertices are adjacent iff their images under `f` are adjacent in `G`.



`G` (right) pulled back along `f : V → W` (dashed arrows), to give `G.comap f` (left)

```
#source SimpleGraph.comap
example {V W : Type*} (f : V → W) {v₁ v₂ : V} (G : SimpleGraph W) :
  (G.comap f).Adj v₁ v₂ ↔ G.Adj (f v₁) (f v₂) := comap_adj ..
```

Two examples of graphs defined using `SimpleGraph.comap` are complete multipartite (`completeMultipartiteGrap` and blow-up graphs (`blowUpGraph`). (If you read the section Complete Multipartite Graphs and Blow-ups (p. 42), you have already seen them.) Two vertices are adjacent in a complete multipartite graph iff they are in different parts.

```
#source completeMultipartiteGraph    -- SimpleGraph (Σ i, V i) := .comap Sigma.fst ᵀ
example {ι : Type*} (V : ι → Type*) {u v : Σ i, V i} :
    (completeMultipartiteGraph V).Adj u v ↔ u.fst ≠ v.fst := Iff.rfl
```

Two vertices are adjacent in the blow-up of a graph `G` iff they are in parts that are adjacent in `G`.

```
#source blowUpGraph -- abbrev blowUpGraph {ι : Type*} (G : SimpleGraph ι)
                    --    (V : ι → Type*) : SimpleGraph (Σ i, V i) :=
                    --                          SimpleGraph.comap Sigma.fst G


example {ι : Type*} (V : ι → Type*) {u v : Σ i, V i} (G : SimpleGraph ι) :
    (G.blowUpGraph V).Adj u v ↔ G.Adj u.fst v.fst := Iff.rfl
```

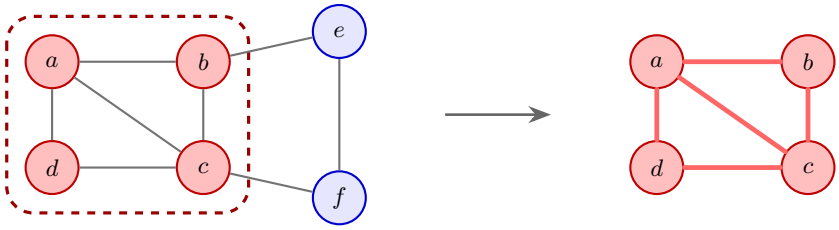
In both cases the adjacency relation is defined by pulling back the adjacency relation of a graph using `SimpleGraph.comap` and the map `Sigma.fst : Σ i, V i → ι` that sends a vertex to its part.

Induced Subgraphs via Comap

The `SimpleGraph.comap` gives a natural way to define induced subgraphs (`SimpleGraph.induce`). Given a set of vertices `s : Set V`, the embedding of `s` into `V` as a `Subtype` is given by `Function.Embedding.subtype (· ∈ s)`.

```
section induced
variable {V : Type*} {s : Set V} {G : SimpleGraph V}
#check Function.Embedding.subtype (· ∈ s) -- {x // x ∈ s} ↪ V
```

The induced subgraph of `G` on the vertex set `s` is then given by the `SimpleGraph.comap` of `G` by this embedding.



Left: a graph `G` with a subset `s` of its vertices. Right: the induced subgraph `G.induce s`

```
#source induce -- abbrev induce (s : Set V) (G : SimpleGraph V) : SimpleGraph s :=
               --   G.comap (Function.Embedding.subtype _)

#check SimpleGraph.induce_adj -- (induce s G).Adj u v ↔ G.Adj ↑u ↑v
#check SimpleGraph.induce s   -- induce s : SimpleGraph V → SimpleGraph ↑s
```

Notice that `G.induce s` gives a `SimpleGraph ↑s` whose vertices have a different type to those of the original graph. So for example in the subgraph of `C[5]` induced by the set `{0, 1, 2}` the vertices are of type `↑{0, 1, 2}`. So when talking about vertices in this graph we need to provide a pair `⟨v, hv⟩` where `v : Fin 5` and `hv : v ∈ {0, 1, 2}`. Often these proof obligations can be discharged automatically using **`simp`** or **`decide`**. (Either will work in the example below.)

```
example : (C[5].induce {0, 1, 2}).Adj ⟨0, by simp⟩ ⟨1, by decide⟩ := by decide
#check petersenGraph
```

In the subgraph of the path on $\mathbb{N}$ induced by even natural numbers, no two (necessarily even) vertices are adjacent.

```
example {a b : {n : ℕ // Even n}} : ¬(pathNat.induce {n | Even n}).Adj a b := by
  rw [induce_adj, pathNat_adj]
  grind

example {a b : ℕ} :
    ¬(pathNat.induce {n | Even n}).Adj ⟨2 * a, by simp⟩ ⟨2 * b, by simp⟩ := by
  rw [induce_adj, pathNat_adj]
  -- ⊢ ¬(↑⟨2 * a, ⋯⟩ + 1 = ↑⟨2 * b, ⋯⟩ ∨ ↑⟨2 * b, ⋯⟩ + 1 = ↑⟨2 * a, ⋯⟩)
  grind
```

If you want a subgraph whose vertices have type `V` but containing only the vertices in `s` (which is often a more useful definition of an induced subgraph) then you need to use the `Subgraph` type (see Subgraphs (p. 246) for details).

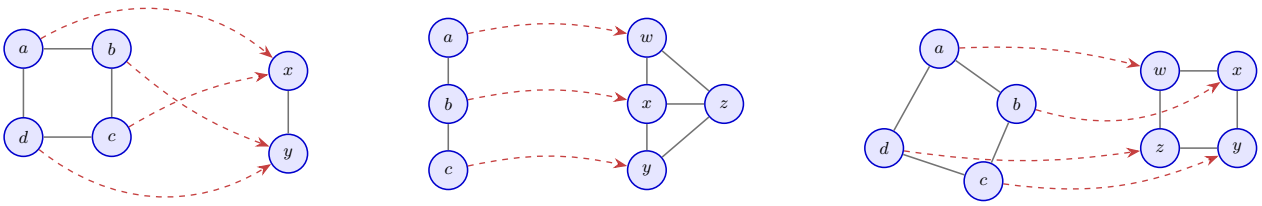
```
#check (⊤ : G.Subgraph).induce s

end induced
```

5.3 Morphisms

Graph homomorphisms, embeddings, and isomorphisms are the structure-preserving maps between graphs. Unlike `SimpleGraph.comap`, which takes a plain function and builds a new graph, a morphism is *data* relating two given graphs: a function between their vertex types together with a proof that it respects adjacency. We will see below how the two ideas are related.

Before looking at morphisms in detail we note that they are all defined as `abbrev`s for the corresponding `Rel*` types in Mathlib. This means that occasionally when working with these maps we will use results that refer to `RelHom`, `RelEmbedding` or `RelIso`.



Left: a homomorphism from `C[4]` onto `K[2]`, identifying opposite vertices.
Centre: an embedding of a path. Right: an isomorphism

Graph Homomorphisms

```
section hom
variable {V W X : Type*} {G : SimpleGraph V} {H : SimpleGraph W} {K : SimpleGraph X}
```

A *graph homomorphism* from `G` to `H` is a map between vertex types sending edges to edges. In Mathlib this is called `SimpleGraph.Hom G H` and written `G →g H`. Homomorphisms preserve adjacency.

```
example (f : G →g H) (u v : V) (h : G.Adj u v) : H.Adj (f u) (f v) := f.map_adj h
```

The identity as a graph homomorphism:

```
#check Hom.id   -- G →g G
```

We can compose homomorphisms.

```
example (g : H →g K) (f : G →g H) : G →g K := g.comp f
```

If `G₁ ≤ G₂` then there is an associated homomorphism `G₁ →g G₂` given by the identity function on vertices.

```
#check Hom.ofLE --  {G₁ G₂ : SimpleGraph V} (h : G₁ ≤ G₂) : G₁ →g G₂
```

A homomorphism maps neighbourhoods into neighbourhoods.

```
example (f : G →g H) (v : V) : (G.neighborSet v).image f ⊆ H.neighborSet (f v) := by
  intro w hw
  obtain ⟨u, hu, rfl⟩ := hw
  exact f.map_adj hu
```

We can now relate homomorphisms to `SimpleGraph.comap`. A function `f : V → W` underlies a homomorphism `G →g H` iff `G` is a subgraph of `H.comap f`: both say that `f` sends every edge of `G` to an edge of `H`.

```
example (f : G →g H) : G ≤ H.comap f := f.le_comap
example (f : V → W) (h : G ≤ H.comap f) : G →g H := ⟨f, fun huv ↦ h huv⟩
```

In particular, taking `G` to be `H.comap f` itself, `f` is always a homomorphism `H.comap f →g H`. Mathlib calls this `Hom.comap`.

```
#check Hom.comap -- (f : V → W) (G : SimpleGraph W) : G.comap f →g G
```

Any graph homomorphism from the complete graph `⊤` is injective.

```
#check Hom.injective_of_top_hom
```

Homomorphisms into a complete graph are also important. A homomorphism `G →g (⊤ : SimpleGraph γ)` sends adjacent vertices of `G` to distinct elements of `γ`, so it is exactly a proper colouring of `G` with colours from `γ`. Indeed this is how Mathlib defines `G.Coloring γ` (see Colorings (p. 156)). Composing with a homomorphism `G →g H` turns colourings of `H` into colourings of `G`, so if `H` is n-colourable then so is `G`.

```
example (γ : Type*) : G.Coloring γ = (G →g (⊤ : SimpleGraph γ)) := rfl
#check Colorable.of_hom -- (f : G →g G') : G'.Colorable n → G.Colorable n

end hom
```

Graph Embeddings

```
section embed
variable {V W X : Type*} {G : SimpleGraph V} {H : SimpleGraph W} {K : SimpleGraph X}
         (f : G ↪g H) (g : H ↪g K)
```

A `SimpleGraph.Embedding G H`, written `G ↪g H`, is an embedding `V ↪ W` (i.e. an injective map) such that `∀ u v, G.Adj u v ↔ H.Adj (f u) (f v)`. In other words, `f` is an injective homomorphism that also *reflects* edges: `H.Adj (f u) (f v) → G.Adj u v`. This means that `G` is isomorphic to an induced subgraph of `H`.

```
example {u v : V} : H.Adj (f u) (f v) ↔ G.Adj u v := f.map_adj_iff
```

In terms of `SimpleGraph.comap`: an injective function `f` underlies an embedding `G ↪g H` iff pulling back `H` along `f` gives exactly `G`. Conversely, `SimpleGraph.comap` along any injective function is an embedding (`Embedding.comap`), and a special case of this is the inclusion of an induced subgraph (`Embedding.induce`).

```
example : H.comap f = G := f.comap_eq
#check Embedding.comap  -- (f : V ↪ W) (G : SimpleGraph W) : G.comap f ↪g G
#check Embedding.induce -- (s : Set V) : G.induce s ↪g G
```

Graph embeddings compose. (We can write this in two superficially different ways; the second makes use of the fact that `SimpleGraph.Embedding` is an **abbrev** for `RelEmbedding`.)

```
#check g.comp f  -- G ↪g K
#check f.trans g -- RelEmbedding.trans f g : G.Adj ↪r K.Adj
example : g.comp f = f.trans g := rfl
```

Every graph embedding gives rise to a homomorphism and an underlying embedding of vertex types.

```
#check Embedding.toHom  -- .. (f : G ↪g G') : G →g G'
#check f.toEmbedding    -- V ↪ W
```

```
end embed
```

Recall `pathNat`, the path graph on $\mathbb{N}$. We can embed this in itself by sending `i` to `i + j`, for any fixed $j : \mathbb{N}$.

```
#source pathNat
```

The graph embedding `pathNat ↪g pathNat`, with `i ↦ i + j`.

```
@[simps]
def pathNat.translate (j : ℕ) : pathNat ↪g pathNat where
  toFun i := i + j
  inj' := add_left_injective j
  map_rel_iff' := by grind [Function.Embedding.coeFn_mk]
```

We can embed G into any blow-up `G.blowUpGraph W` so long as the parts of the blow-up are nonempty (the argument f below picks an element of each part).

Embedding of G into `G.blowUpGraph W` with nonempty parts.

```
@[simps]
def Embedding.blowUp {ι : Type*} (G : SimpleGraph ι) (W : ι → Type*)
    (f : (i : ι) → W i) : G ↪g G.blowUpGraph W where
  toFun i := ⟨i, f i⟩
  inj' _ _ h := congrArg Sigma.fst h
  map_rel_iff' := Iff.rfl
```

For example, here is the embedding of `C[4]` into its blow-up with all parts equal to `Fin 3`, sending each vertex `i : Fin 4` of `C[4]` to the vertex `⟨i, 0⟩` of the blow-up.

```
#check Embedding.blowUp C[4] (fun _ => Fin 3) (fun _ => 0)
```

Copies

We already met graph homomorphisms `A →g B` and graph embeddings `A ↪g B`. `Copy A B` falls between the two: it is the type of injective graph homomorphisms from `A` to `B`.

While `A ↪g B` captures the type of *induced* copies of a graph `A` in a graph `B`, `Copy A B` captures the type of *not-necessarily-induced* copies of `A` in `B`.

```
#source Copy
```

Note that the copies are *labelled*: an element of `Copy A B` records where each vertex of `A` goes. So if `A` has nontrivial automorphisms then a single subgraph of `B` isomorphic to `A` is the image of several elements of `Copy A B`, one for each automorphism of `A`. We return to this distinction in Counting Copies (p. 386).

An induced copy of `A` in `B` (i.e. `f : A ↪g B`) gives rise to a copy of `A` in `B`.

```
#source Embedding.toCopy -- .. (f : A ↪g B) : Copy A B := f.toHom.toCopy f.injective
```

Note that any copy of a complete graph is necessarily induced, so we can always upgrade a `Copy ⊤ G` to a graph embedding `⊤ ↪g G`.

```
example {V W : Type*} {G : SimpleGraph V} (c : Copy (T : SimpleGraph W) G) :
    (T : SimpleGraph W) ↪g G := c.topEmbedding
```

We say that A is *contained* in B, written A.IsContained B or A ⊑ B, iff there exists a Copy A B.

```
#source IsContained -- (A : SimpleGraph α) (B : SimpleGraph β) := Nonempty (Copy A B)
```

If B contains no copy of A then we say that B is A-*free*, written A.Free B. This follows the convention of extremal graph theory, where "A-free" means containing no copy of A at all (as in "triangle-free"). Beware that structural graph theory often uses "A-free" to mean containing no *induced* copy of A (as in "claw-free"). For that notion we need a separate definition, which we give in Claw-Free Graphs (p. 119).

```
#source Free -- abbrev Free (A : SimpleGraph α) (B : SimpleGraph β) := ¬A ⊑ B
```

Graph Isomorphisms

```
section iso
variable {V W X : Type*} {G : SimpleGraph V} {H : SimpleGraph W} {K : SimpleGraph X}
         (f : G ≃g H) (g : H ≃g K)
```

A SimpleGraph.Iso G H, written G ≃g H, is a graph isomorphism, i.e. it is an equivalence of vertex types that respects adjacency.

```
#check Iso
```

The identity isomorphism:

```
#check Iso.refl  -- G ≃g G
```

The inverse of an isomorphism is an isomorphism.

```
#check Iso.symm  -- G ≃g H → H ≃g G
```

Isomorphisms compose:

```
#check g.comp f -- G ≃g K
```

The underlying equivalence of vertex types:

```
#check f.toEquiv  -- V ≃ W
```

Any graph isomorphism gives rise to a graph embedding and graph homomorphism.

```
#check Iso.toEmbedding  -- G ≃g G' → G ↪g G'
#check Iso.toHom        -- G ≃g G' → G →g G'
```

Isomorphic graphs have the same graph-theoretic invariants. For example, an isomorphism preserves the degree of each vertex, the minimum and maximum degree, and the number of edges.

```
#check Iso.degree_eq          -- (f : G ≃g G') (x : V) : G'.degree (f x) = G.degree x
#check Iso.minDegree_eq       -- (f : G ≃g G') : δ(G) = δ(G')
#check Iso.maxDegree_eq       -- (f : G ≃g G') : Δ(G) = Δ(G')
#check Iso.card_edgeFinset_eq -- (f : G ≃g G') : #G.edgeFinset = #G'.edgeFinset
```

Informally, "isomorphic graphs have the same properties" is a single meta-theorem that we use without comment. In Lean there is no such meta-theorem: every invariant needs its own transfer lemma, which is why Mathlib contains many lemmas of the form `Iso.*_eq` and `Iso.*_iff`. Here are some for notions we meet later: connectedness (`Connected`), acyclicity (`IsAcyclic`) and the chromatic number (`chromaticNumber`).

```
#check Iso.connected_iff  -- G ≃g H → (G.Connected ↔ H.Connected)

#check Iso.isAcyclic_iff  -- G ≃g H → (G.IsAcyclic ↔ H.IsAcyclic)

example (e : G ≃g H) : G.chromaticNumber = H.chromaticNumber :=
  chromaticNumber_congr e
```

Equivalences of types induce isomorphisms of complete graphs on those types.

```
#check Iso.completeGraph
```

The analogous fact for empty graphs is also true, but not supplied by Mathlib, so we give it here.

Equivalences of types induce isomorphisms of empty graphs on those types.

```
protected def Iso.emptyGraph {α β : Type*} (e : α ≃ β) :
    emptyGraph α ≃g emptyGraph β where
  __ := e
  map_rel_iff' := by simp

end iso
```

Coercions to Functions

Graph homomorphisms, embeddings and isomorphisms are not simply functions. A homomorphism `f : G →g H` is a structure bundling a function `f.toFun : V → W` with the proof `f.map_rel'` that it preserves adjacency, and embeddings and isomorphisms are structures in the same way (as are `V ↪ W` and `V ≃ W` from the first section). Nevertheless we have been writing `f v` for the image of a vertex throughout. This works because Mathlib registers a `FunLike` instance for each of `G →g H`, `G ↪g H` and `G ≃g H`, which lets Lean insert a *coercion* to a function whenever a map is applied to an argument. The InfoView marks a coerced map with an arrow, `↑f`.

```
section coe
variable {V W X : Type*} {G : SimpleGraph V} {H : SimpleGraph W} {K : SimpleGraph X}
         (f : G →g H) (g : H →g K) (e : G ↪g H) (φ : G ≃g H)

#check fun v => f v   -- fun v ↦ f v : V → W
set_option pp.coercions false in
#check fun v => f v   -- fun v ↦ DFunLike.coe f v : V → W
```

The coercion is definitionally the underlying function, and composing maps composes the coerced functions (`Hom.coe_comp` and its analogues). Note that `g.comp f` is not literally `g ∘ f`: in the latter Lean coerces the two homomorphisms to functions and composes those. The two are equal as functions, but only `g.comp f` is a homomorphism.

```
example : (f : V → W) = f.toFun := rfl
example : ↑(g.comp f) = g ∘ f := rfl
```

Since each kind of map is determined by its function, two maps of the same kind are equal if they agree on every vertex (by *extensionality*).

```
example (f₁ f₂ : G →g H) (h : ∀ v, f₁ v = f₂ v) : f₁ = f₂ := DFunLike.ext _ _ h
example (φ₁ φ₂ : G ≃g H) (h : ∀ v, φ₁ v = φ₂ v) : φ₁ = φ₂ := DFunLike.ext _ _ h
```

Likewise, properties of functions are properties of the coerced function: `Function.Injective e` means `Function.Injective ⇑e`, and Mathlib supplies the expected facts (via `RelEmbedding.injective`, `RelIso.bijective`, and so on).

```
example : Function.Injective e := e.injective
example : Function.Bijective φ := φ.bijective

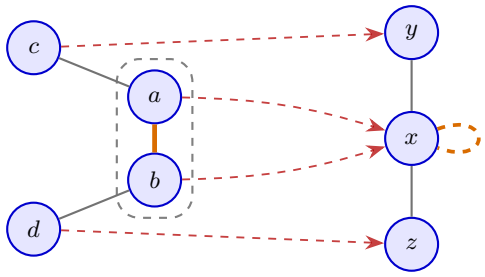
end coe
```

5.4 Maps

```
section maps
variable {V W X : Type*} {G : SimpleGraph V} {H : SimpleGraph W} {K : SimpleGraph X}

#source SimpleGraph.map
```

Given any function `f : V → W` we can form the *map* of a graph `G : SimpleGraph V` along `f`. This gives a graph `G.map f : SimpleGraph W` on the vertex set `W` where two vertices are adjacent iff they are distinct and are the images of adjacent vertices in `G`. The distinctness condition is needed because `f` need not be injective: if `f` sends two adjacent vertices to the same vertex then their edge would otherwise become a loop.



`G` (left) pushed forward along `f : V → W` (dashed arrows), to give `G.map f` (right)

```
example (f : V → W) {w₁ w₂ : W} :
    (G.map f).Adj w₁ w₂ ↔ w₁ ≠ w₂ ∧ ∃ v₁ v₂, G.Adj v₁ v₂ ∧ f v₁ = w₁ ∧ f v₂ = w₂ :=
  Iff.rfl
```

When `f : V ↪ W` is an embedding the distinctness condition is automatic, and Mathlib's `map_adj` omits it.

```
example (f : V ↪ W) {w₁ w₂ : W} :
  (G.map f).Adj w₁ w₂ ↔ ∃ v₁ v₂, G.Adj v₁ v₂ ∧ f v₁ = w₁ ∧ f v₂ = w₂ := map_adj ..
```

Either way, the adjacency relation of `G.map f` contains an existential quantifier: two vertices are adjacent if there *exist* adjacent preimages. This makes graphs defined using `SimpleGraph.map` harder to work with, particularly in computations.

How does SimpleGraph.map relate to homomorphisms? The image of G under a homomorphism f : G →g H is a subgraph of H (Hom.map_le). For an embedding of vertex types f : V ↪ W the converse also holds, so in that case G.map f ≤ H and G ≤ H.comap f both say that f is a homomorphism G →g H.

```
example (f : G →g H) : G.map f ≤ H := f.map_le
example (f : V ↪ W) : G.map f ≤ H ↔ G ≤ H.comap f := map_le_iff_le_comap
```

For a function that is not injective the converse fails: SimpleGraph.map may send two adjacent vertices to the same vertex, so G.map f ≤ H does not ensure that f sends every edge of G to an edge of H.

In the language of order theory, map_le_iff_le_comap says that SimpleGraph.map f and SimpleGraph.comap f form a *Galois connection* when f is an embedding. It is in fact a *Galois coinsertion*: mapping forwards and then pulling back recovers the original graph, while pulling back and then mapping forwards only gives a subgraph. (See the exercises for an example where this inequality is strict.)

```
example (f : V ↪ W) : (G.map f).comap f = G := comap_map_eq ..
example (f : V ↪ W) : (H.comap f).map f ≤ H := map_comap_le ..
```

Mathlib also packages SimpleGraph.map as morphisms: an embedding f : V ↪ W gives a graph embedding G ↪g G.map f, and an equivalence e : V ≃ W gives a graph isomorphism G ≃g G.map e.

```
#check Embedding.map -- (f : V ↪ W) (G : SimpleGraph V) : G ↪g G.map f
#check Iso.map       -- (f : V ≃ W) (G : SimpleGraph V) : G ≃g G.map f

end maps
```

SpanningCoe

Having defined induced subgraphs via SimpleGraph.comap we can also go in the other direction: starting with G : SimpleGraph ↑s, where s : Set V, we can use SimpleGraph.map and Subtype s ↪ V to get a SimpleGraph V whose edges are exactly those of G. Since this graph contains all vertices of V but is otherwise the same as G, it is called the spanningCoe of G.

```
section spanningcoe
variable {V : Type*}

#source spanningCoe -- spanningCoe {s : Set V} (G : SimpleGraph s) :
                    --    SimpleGraph V := G.map (Function.Embedding.subtype _)
```

Since SimpleGraph.map and SimpleGraph.comap along the embedding Subtype s ↪ V form a Galois coinsertion, we have two very useful results relating induce and spanningCoe.

Firstly, if we start with G : SimpleGraph ↑s (where s : Set V), take its spanningCoe and form the induced subgraph on s, then we recover G.

```
example {s : Set V} {G : SimpleGraph ↑s} :
  G.spanningCoe.induce s = G := induce_spanningCoe
```

Secondly, if we take the spanningCoe of an induced subgraph we get a graph on V that is a subgraph of the original graph.

```
example {s : Set V} {G : SimpleGraph V} :
  (G.induce s).spanningCoe ≤ G := spanningCoe_induce_le ..
```

Moreover, recall that the `support` of a `G : SimpleGraph V` is the set of vertices that are incident to at least one edge. The `spanningCoe` of an induced subgraph of `G` is equal to `G` iff the support of `G` is contained in the set of vertices we are inducing on.

```
example {s : Set V} {G : SimpleGraph V} :
  (G.induce s).spanningCoe = G ↔ G.support ⊆ s := spanningCoe_induce_eq_self ..

end spanningcoe
```

We have now met three ways of describing a subgraph of `G : SimpleGraph V`: a graph `H ≤ G` with the same vertex type `V` (Order (p. 11)), such as `(G.induce s).spanningCoe`; an induced subgraph `G.induce s : SimpleGraph ↥s`, whose vertex type is the subtype `↥s` of `V`; and a copy `Copy A G` of a graph `A` whose vertices may have any type (Copies (p. 104)). Mathlib has a fourth, `G.Subgraph`, in which the vertex set can vary inside `V`. We compare all four later in Which Notion of Subgraph? (p. 255).

5.5 Hypercubes and Mycielskians

We now introduce the natural maps associated with hypercubes and Mycielskians.

Embeddings of Hypercubes

There are two natural embeddings of `Q[n]` into `Q[n + 1]`, given by sending a vertex `v` to `(v, false)` or `(v, true)`. These will be very useful when we start looking at walks in the hypercube. We can define these in terms of the `boxProdLeft` graph embedding.

```
#check boxProdLeft --  G ↪g G □ H
```

The embedding `Q[n] ↪g Q[n + 1]`, with `v ↦ (v, b)` (where `b : Bool`).

```
@[simps!]
def hypercube.embedBool (n : ℕ) (b : Bool) : Q[n] ↪g Q[n + 1] :=
  Q[n].boxProdLeft _ b
```

The embedding `Q[n] ↪g Q[n + 1]`, with `v ↦ (v, false)`.

```
abbrev hypercube.embedFalse (n : ℕ) : Q[n] ↪g Q[n + 1] :=
  hypercube.embedBool n false
```

The embedding `Q[n] ↪g Q[n + 1]`, with `v ↦ (v, true)`.

```
abbrev hypercube.embedTrue (n : ℕ) : Q[n] ↪g Q[n + 1] :=
  hypercube.embedBool n true
```

Embeddings and Maps of Mycielskians

There is a natural embedding of a graph `G` into its Mycielskian `mμ(G)`, given by sending a vertex `v` to the left copy of `v` in the Mycielskian.

The graph embedding `G ↪g mμ(G)` of a graph into its Mycielskian.

```
@[simps]
def Embedding.mycielskianLeft {V : Type*} (G : SimpleGraph V) : G ↪g mμ(G) where
  toFun := (·ˡ) -- V → Option (V ⊕ V); v ↦ some (Sum.inl v)
  inj' := Option.some_injective _ |>.comp Sum.inl_injective
  map_rel_iff' := Iff.rfl
```

Any graph homomorphism `f : G →g H` induces a homomorphism `mμ(G) →g mμ(H)`. Notice that below we use some useful shorthand for defining a function on `Option (V ⊕ V)`.

```
#check Option.elim'
#check Sum.elim
```

A graph homomorphism `f : G →g H` induces a homomorphism `mμ(G) →g mμ(H)`, showing that the Mycielskian is functorial on `SimpleGraph.Hom`.

```
@[simps]
def Hom.mapMycielskian {V W : Type*} {G : SimpleGraph V} {H : SimpleGraph W}
    (f : G →g H) : mμ(G) →g mμ(H) where
  toFun := Option.elim' none ◁ Sum.elim (fun v => (f v)ˡ) (fun v => (f v)ʳ)
  map_rel' := by
    intro u v h
    rcases u with _ | (_|_) <;> rcases v with _ | (_|_) <;>
      simp_all [f.map_adj]
```

`Hom.mapMycielskian` sends the identity homomorphism on `G` to the identity on `mμ(G)`.

```
theorem Hom.mapMycielskian_id {V : Type*} (G : SimpleGraph V) :
    (Hom.id (G := G)).mapMycielskian = Hom.id := by
  ext x; rcases x with _ | (_|_) <;> rfl
```

`Hom.mapMycielskian` distributes over composition: the map induced by `g.comp f` is the composite of the maps induced by `f` and `g`. (Together with `Hom.mapMycielskian_id` this says that the Mycielskian is a *functor* on the category of simple graphs and homomorphisms: it acts on homomorphisms as well as on graphs, compatibly with identities and composition.)

```
lemma Hom.mapMycielskian_comp {V W X : Type*} {G : SimpleGraph V}
    {H : SimpleGraph W} {K : SimpleGraph X} (f : G →g H) (g : H →g K) :
    (g.comp f).mapMycielskian = g.mapMycielskian.comp f.mapMycielskian := by
  ext x; rcases x with _ | (_|_) <;> rfl
```

The same construction turns embeddings into embeddings and isomorphisms into isomorphisms.

A graph embedding `f : G ↪g H` induces an embedding `mμ(G) ↪g mμ(H)`.

```
@[simps]
def Embedding.mapMycielskian {V W : Type*} {G : SimpleGraph V} {H : SimpleGraph W}
    (f : G ↪g H) : mμ(G) ↪g mμ(H) where
  toFun := f.toHom.mapMycielskian
  inj' u v h := by
    rcases u with _ | (_|_) <;> rcases v with _ | (_|_) <;> simp_all
  map_rel_iff' {u v} := by
    rcases u with _ | (_|_) <;> rcases v with _ | (_|_) <;> simp [Hom.mapMycielskian]
```

A graph isomorphism `f : G ≃g H` induces an isomorphism `mμ(G) ≃g mμ(H)`.

```
@[simps]
def Iso.mapMycielskian {V W : Type*} {G : SimpleGraph V} {H : SimpleGraph W}
    (f : G ≃g H) : Mμ(G) ≃g Mμ(H) where
  toFun  := f.toHom.mapMycielskian
  invFun := f.symm.toHom.mapMycielskian
  left_inv x := by rcases x with _ | (_|_) <;> simp
  right_inv x := by rcases x with _ | (_|_) <;> simp
  map_rel_iff' := f.toEmbedding.mapMycielskian.map_rel_iff
```

5.6 Automorphisms

An *automorphism* of a graph `G` is an isomorphism `G ≃g G` from `G` to itself. The automorphisms of `G` form a group under composition, the *automorphism group* of `G`, which we study in Algebraic Graph Theory (p. 360). In this section we use automorphisms to define vertex-transitivity, and show that complete graphs, cycles and hypercubes are vertex-transitive (empty graphs are left to the exercises), while the Mycielskian of a finite nonempty graph `G` is vertex-transitive only when `G` is a single edge.

Vertex-Transitivity

A graph is *vertex-transitive* if its automorphism group acts transitively on the vertices: any vertex can be sent to any other by some automorphism `G ≃g G`.

This isn't currently in Mathlib, so we introduce it here. [1]

`G` is *vertex-transitive* if for every pair of vertices `u` and `v` there is an automorphism of `G` mapping `u` to `v`.

```
abbrev IsVertexTransitive {V : Type*} (G : SimpleGraph V) : Prop :=
  ∀ u v : V, ∃ f : G ≃g G, f u = v
```

Any vertex-transitive graph is regular.

```
theorem IsVertexTransitive.exists_isRegularOfDegree {V : Type*} {G : SimpleGraph V}
    [G.LocallyFinite] (h : G.IsVertexTransitive) : ∃ d, G.IsRegularOfDegree d := by
  rcases isEmpty_or_nonempty V with hV | ⟨⟨v₀⟩⟩
  · exact ⟨0, isEmptyElim⟩
  · exact ⟨G.degree v₀, fun v => by obtain ⟨f, rfl⟩ := h v₀ v; simp⟩
```

Isomorphisms preserve vertex-transitivity.

```
lemma Iso.isVertexTransitive {V W : Type*} {G : SimpleGraph V} {H : SimpleGraph W}
    (f : G ≃g H) (h : G.IsVertexTransitive) : H.IsVertexTransitive := by
  intro u v
  obtain ⟨g, hg⟩ := h (f.symm u) (f.symm v)
  exact ⟨(f.comp g).comp f.symm, by simp [hg]⟩
```

Any equivalence of the underlying type gives an automorphism of the complete graph: this is Mathlib's `Iso.completeGraph`.

[1] We use ∃ here rather than `Nonempty`. `Nonempty T` says that the type `T` has an element, while `∃ x : T, p x` says that some element of `T` satisfies an extra condition `p`. So `IsContained` is `Nonempty (Copy A B)`, because the conditions on a copy are built into the type `Copy A B`. There is no type of automorphisms sending `u` to `v`, so here we use ∃. The two are equivalent: `nonempty_subtype` says `Nonempty {x // p x} ↔ ∃ x, p x`, and simp uses it to rewrite to the ∃ form.

```
#check Iso.completeGraph -- (f : α ≃ β) : completeGraph α ≃g completeGraph β
```

In particular, swapping any pair of vertices gives an automorphism, so the complete graph is vertex-transitive.

```
lemma isVertexTransitive_top {V : Type*} :
    IsVertexTransitive (⊤ : SimpleGraph V) := by
  intro u v
  classical
  exact ⟨Iso.completeGraph (Equiv.swap u v), Equiv.swap_apply_left u v⟩
```

The empty graph is vertex-transitive too: the exercises deduce this from the complete graph, by showing that the complement of a vertex-transitive graph is vertex-transitive.

The cycle graph `C[n]` is vertex-transitive. To prove this we define an automorphism of `C[n]` by rotating the vertices modulo `n`: the rotation `cycleGraph.rotate d` is the one sending the vertex `d` to `0`, i.e. `x ↦ x - d`. (Rotating in this direction rather than `x ↦ x + d` makes the rotations match Mathlib's conventions for dihedral groups, as we shall see in Automorphisms of Cycle Graphs (p. 362).) The definition makes sense for every `n`, with the cases `n = 0, 1` trivial since `C[n]` is `⊥` there; only the proofs need to treat the degenerate cases separately.

The automorphism of the cycle graph `C[n]` given by rotating the vertices modulo `n`, sending the vertex `d : Fin n` to `0`.

```
@[simps]
def cycleGraph.rotate {n : ℕ} (d : Fin n) : C[n] ≃g C[n] where
  toFun x := x - d
  invFun x := x + d
  left_inv x := by cases n <;> grind
  right_inv x := by cases n <;> grind
  map_rel_iff' := by
    match n with
    | 0 | 1 | n + 2 => simp [cycleGraph_adj]
```

The cycle `C[n]` is vertex-transitive for all `n`. The rotation sending `u - v` to `0` sends `u` to `v`. (For `n = 0` there is nothing to prove since there are no vertices.)

```
lemma isVertexTransitive_cycleGraph (n : ℕ) : C[n].IsVertexTransitive :=
  match n with
  | 0 => fun u _ => u.elim0
  | _ + 1 => fun u v => ⟨cycleGraph.rotate (u - v), sub_sub_cancel u v⟩
```

Vertex-Transitivity of Hypercubes

The hypercube is vertex-transitive: "translation" by a fixed vertex `t`, i.e. componentwise `xor` with `t`, is an automorphism of `Q[n]`, and translating by `u.xor v` sends `u` to `v`.

Componentwise `xor` (symmetric difference) of two hypercube vertices.

```
def HypercubeVert.xor : {n : ℕ} → HypercubeVert n → HypercubeVert n → HypercubeVert n
  | 0,     _,      _      => ()
  | _ + 1, (v, a), (w, b) => (v.xor w, a ^^ b)
```

Notation `v ⊕ₕ w` for the componentwise `xor` of hypercube vertices `v w : HypercubeVert n`, i.e. addition in $(\mathbb{Z}/2)^n$. (We use `⊕ₕ` rather than `⊕` to avoid clashing with the `Sum` notation `α ⊕ β`.)

```
@[inherit_doc HypercubeVert.xor]
scoped infixl:65 " ⊕ₕ " => HypercubeVert.xor
```

`xor`-ing twice by the same vertex is the identity, so translation is an involution.

```
@[simp]
lemma HypercubeVert.xor_xor {n : ℕ} (v t : HypercubeVert n) : (v ⊕ₕ t) ⊕ₕ t = v := by
  induction n with
  | zero => rfl
  | succ n ih =>
    obtain ⟨v, a⟩ := v; obtain ⟨t, s⟩ := t
    exact Prod.ext (ih v t) (by cases a <;> cases s <;> rfl)
```

We record this using Mathlib's `Function.Involutive`, which gives us injectivity and the inverse equivalence for free.

```
lemma HypercubeVert.xor_involutive {n : ℕ} (t : HypercubeVert n) :
    Function.Involutive (· ⊕ₕ t) :=
  (HypercubeVert.xor_xor · t)
```

`xor`-ing u into $u \oplus_h v$ recovers v.

```
@[simp]
lemma HypercubeVert.xor_xor_cancel {n : ℕ} (u v : HypercubeVert n) :
    u ⊕ₕ (u ⊕ₕ v) = v := by
  induction n with
  | zero => rfl
  | succ n ih =>
    obtain ⟨u, a⟩ := u
    obtain ⟨v, b⟩ := v
    exact Prod.ext (ih u v) (by cases a <;> cases b <;> rfl)
```

Translation by t is injective.

```
@[simp]
lemma HypercubeVert.xor_left_inj {n : ℕ} {v w t : HypercubeVert n} :
    v ⊕ₕ t = w ⊕ₕ t ↔ v = w :=
  (HypercubeVert.xor_involutive t).injective.eq_iff
```

Translating both vertices of a potential edge by t preserves adjacency in `Q[n]`.

```
lemma HypercubeVert.xor_adj {n : ℕ} (v w t : HypercubeVert n) :
    Q[n].Adj (v ⊕ₕ t) (w ⊕ₕ t) ↔ Q[n].Adj v w := by
  induction n with
  | zero => simp
  | succ n ih =>
    cases v; cases w; cases t
    simp [ih, HypercubeVert.xor]
```

Translation of `Q[n]` by a fixed vertex t, via componentwise `xor`, as a graph automorphism.

```
def hypercube.translate {n : ℕ} (t : HypercubeVert n) : Q[n] ≃g Q[n] where
  toEquiv := (HypercubeVert.xor_involutive t).toPerm
```

```
  map_rel_iff' := HypercubeVert.xor_adj _ _ t
```

The hypercube `Q[n]` is vertex-transitive.

```
lemma isVertexTransitive_hypercube (n : ℕ) : Q[n].IsVertexTransitive :=
  fun u v => ⟨hypercube.translate (u ⊕ₕ v), HypercubeVert.xor_xor_cancel u v⟩
```

We do not describe the full automorphism group of `Q[n]` here: it is the wreath product `ℤ/2 ≀ Sₙ`, generated by the translations above together with permutations of the coordinates. While our construction of the hypercube using the vertices `HypercubeVert n` is very convenient for inductive proofs, to talk about the "coordinates" of a vertex it is better to use `Fin n → Bool`. We return to this in The Hypercube and the Hamming Cube (p. 377), where we give a graph isomorphism `Q[n] ≃g hypercubeHamming n` between our box product model of the hypercube and the Hamming cube on `Fin n → Bool`, and then use it to identify the automorphism group of `Q[n]`.

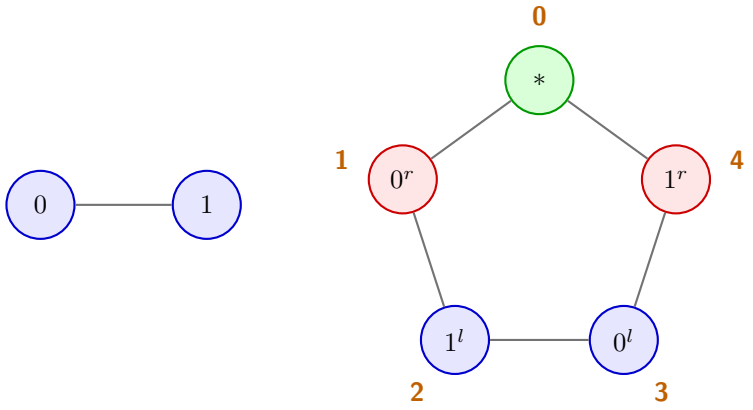
Vertex-Transitivity of Mycielskians

The Mycielskian of a graph is rarely vertex-transitive. In fact we can see this easily by looking at degrees, which are equal in any vertex-transitive graph (`IsVertexTransitive.exists_isRegularOfDegree`). If `V` is finite and nonempty then `mμ(G)` is vertex-transitive iff `G` is isomorphic to `K[2]`.

To help in the proof we first construct a graph isomorphism from `mμ(K[2])` to `C[5]`, which we already know to be vertex-transitive. We define both maps using Mathlib's *vector notation*: `![a, b, c]` is the function `Fin 3 → α` sending `0`, `1` and `2` to `a`, `b` and `c`. So the forward map sends the apex `none` to `0`, 0^l and 1^l to `3` and `2`, and 0^r and 1^r to `1` and `4`.

A graph isomorphism from `mμ(K[2])` to `C[5]`.

```
def mycielskianCompleteTwoIsoCycleFive : mμ(K[2]) ≃g C[5] where
  toFun := Option.elim' 0 (Sum.elim ![3, 2] ![1, 4])
  invFun := ![none, 0ʳ, 1ˡ, 0ˡ, 1ʳ]
  left_inv := by decide +revert
  right_inv := by decide +revert
  map_rel_iff' := by decide +revert
```



Left: `K[2]`. Right: its Mycielskian `mμ(K[2])` drawn as a 5-cycle. The orange numbers are the vertices of `C[5]` to which `mycielskianCompleteTwoIsoCycleFive` sends them

The forward direction only needs `mμ(G)` to be regular, so we isolate it as a lemma: comparing the degrees of u^l, u^r and none shows that if `V` is finite and nonempty and `mμ(G)` is regular then `G` is a single edge.

```
lemma nonempty_iso_completeGraph_two_of_regular_mycielskian {V : Type*}
    {G : SimpleGraph V} [Finite V] [Nonempty V] [mμ(G).LocallyFinite] {d : ℕ}
    (hd : mμ(G).IsRegularOfDegree d) : Nonempty (G ≃g K[2]) := by
  classical
  obtain ⟨v : V⟩ := ‹Nonempty V›
  cases nonempty_fintype V
  -- From d = 2 * G.degree u and d = G.degree u + 1 we get G.degree u = 1.
  have h1 : G.IsRegularOfDegree 1 := fun u => by
    have hₗ : mμ(G).degree u^l = 2 * G.degree u := by simp
    have hᵣ : mμ(G).degree u^r = G.degree u + 1 := by simp
    grind [hd u^l, hd u^r]
  -- Then d = Fintype.card V forces Fintype.card V = 2, as required.
  have hcard : Fintype.card V = 2 := by
    have h₀ : mμ(G).degree none = Fintype.card V := by simp
    have hᵣ : mμ(G).degree v^r = G.degree v + 1 := by simp
    grind [hd none, hd v^r, h1 v]
  -- we now use the fact that if G is regular of degree Fintype.card V - 1
  -- then G is complete.
  obtain rfl : G = ⊤ := G.isRegularOfDegree_card_sub_one_iff_eq_top.1 (by grind)
  exact ⟨Iso.completeGraph (Fintype.equivFinOfCardEq hcard)⟩
```

The Mycielskian of a graph on a finite nonempty vertex type is vertex-transitive iff the original graph is isomorphic to `K[2]`.

```
theorem isVertexTransitive_mycielskian_iff {V : Type*} (G : SimpleGraph V) [Finite V]
    [Nonempty V] : mμ(G).IsVertexTransitive ↔ Nonempty (G ≃g K[2]) := by
  classical
  constructor
  · intro h
    cases nonempty_fintype V
    obtain ⟨d, hd⟩ := IsVertexTransitive.exists_isRegularOfDegree h
    exact nonempty_iso_completeGraph_two_of_regular_mycielskian hd
  · intro ⟨e⟩
    exact e.mapMycielskian.symm.comp mycielskianCompleteTwoIsoCycleFive.symm
      |>.isVertexTransitive (isVertexTransitive_cycleGraph _)
```

5.7 Tensor Products*

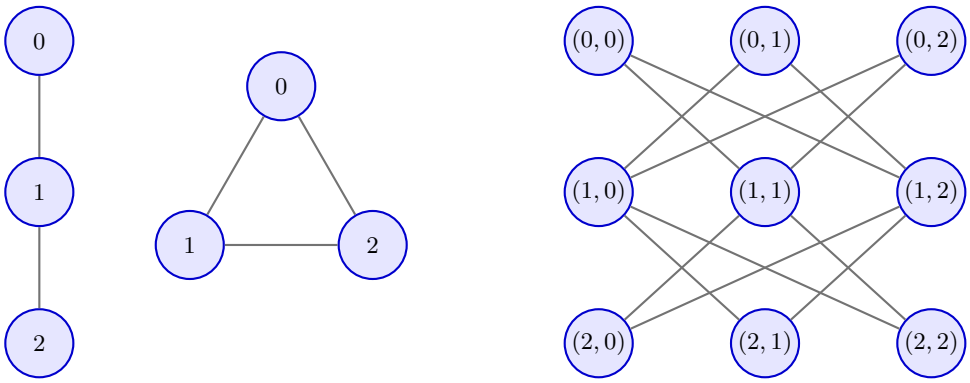
As a further example of a construction using `SimpleGraph.comap`, we define the *tensor product* of two graphs, and then the tensor product of an indexed family of graphs.

The tensor product `G ⊗g H` of graphs on `V` and `W` is the `SimpleGraph (V × W)` where two vertices `(v₁, w₁)` and `(v₂, w₂)` are adjacent iff `G.Adj v₁ v₂` and `H.Adj w₁ w₂`.

```
def tensorProd {V W : Type*} (G : SimpleGraph V) (H : SimpleGraph W) :
    SimpleGraph (V × W) :=
  G.comap Prod.fst ⊓ H.comap Prod.snd
```

We introduce the notation `G ⊗g H` for the tensor product of two graphs.

```
@[inherit_doc]
scoped infixl:70 " ⊠g " => SimpleGraph.tensorProd
```



The tensor product `P[3] ⊠g K[3]`

Adjacency in the tensor product is given by the adjacency relations of the two graphs.

```
@[simp]
lemma tensorProd_adj {V W : Type*} {G : SimpleGraph V} {H : SimpleGraph W}
    {x y : V × W} : (G ⊠g H).Adj x y ↔ G.Adj x.1 y.1 ∧ H.Adj x.2 y.2 := Iff.rfl
```

Decidable adjacency for tensor products follows from decidable adjacency for the two graphs.

```
instance {V W : Type*} (G : SimpleGraph V) (H : SimpleGraph W) [DecidableRel G.Adj]
    [DecidableRel H.Adj] : DecidableRel (G ⊠g H).Adj :=
  fun _ _ ↦ decidable_of_iff' _ tensorProd_adj
```

If `x.1` and `x.2` have finite neighbourhoods in `G` and `H` respectively, then so does `x` in `G ⊠g H`, since its neighbourhood is the product `s ×ˢ t` of the two neighbourhoods. As with `blowUpNFintypeNeighborSet` (see Degrees in the n-Uniform Blow-up of a Graph (p. 70)), we make this a `def` rather than an `instance`, to avoid a second route to the `Fintype` instance that Lean finds by filtering. (We state it for `x : V × W` rather than for a pair `(v, w)`, so that it also applies to a general `x : V × W` when we use it as an instance.)

If `G` and `H` have finite neighbourhoods at `x.fst` and `x.snd` respectively, then their tensor product has a finite neighbourhood at `x`.

```
@[instance_reducible]
def tensorProdFintypeNeighborSet {V W : Type*} {x : V × W} {G : SimpleGraph V}
    {H : SimpleGraph W} [Fintype (G.neighborSet x.fst)]
    [Fintype (H.neighborSet x.snd)] : Fintype ((G ⊠g H).neighborSet x) :=
  Fintype.ofFinset (G.neighborFinset x.fst ×ˢ H.neighborFinset x.snd) (by simp)
```

The degree of a vertex in the tensor product is the product of the degrees of its components.

```
lemma degree_tensorProd {V W : Type*} {x : V × W} {G : SimpleGraph V}
    {H : SimpleGraph W} [Fintype (G.neighborSet x.fst)]
    [Fintype (H.neighborSet x.snd)] [Fintype ((G ⊠g H).neighborSet x)] :
    (G ⊠g H).degree x = G.degree x.fst * H.degree x.snd := by
  simp_rw [← card_neighborFinset_eq_degree, ← Finset.card_product]
  congr
  ext ⟨x, y⟩
```

```
  simp
```

Maps of Tensor Products

The two projections `Prod.fst` and `Prod.snd` are graph homomorphisms out of `G ⊠g H`. Conversely, a pair of homomorphisms from a common graph `K` into `G` and `H` combines into a single homomorphism `K →g G ⊠g H`. So homomorphisms into `G ⊠g H` are exactly pairs of homomorphisms into `G` and `H`. (In the language of category theory: the tensor product is the *product* in the category of simple graphs and homomorphisms.)

The first projection is a graph homomorphism `G ⊠g H →g G`.

```
def tensorProd.fstHom {V W : Type*} {G : SimpleGraph V} {H : SimpleGraph W} :
    G ⊠g H →g G where
  toFun := Prod.fst
  map_rel' h := h.1
```

The second projection is a graph homomorphism `G ⊠g H →g H`.

```
def tensorProd.sndHom {V W : Type*} {G : SimpleGraph V} {H : SimpleGraph W} :
    G ⊠g H →g H where
  toFun := Prod.snd
  map_rel' h := h.2
```

Pairing two homomorphisms `f : K →g G` and `g : K →g H` gives a homomorphism into the tensor product. This is the universal property of the product.

```
def tensorProd.prod {V W X : Type*} {G : SimpleGraph V} {H : SimpleGraph W}
    {K : SimpleGraph X} (f : K →g G) (g : K →g H) : K →g G ⊠g H where
  toFun x := (f x, g x)
  map_rel' h := ⟨f.map_adj h, g.map_adj h⟩
```

Composing a colouring of `G` with the projection `tensorProd.fstHom` gives a colouring of `G ⊠g H`, and similarly for `H`. So the chromatic number of `G ⊠g H` is at most the minimum of the chromatic numbers of `G` and `H`.

```
example {V W : Type*} {G : SimpleGraph V} {H : SimpleGraph W} {n : ℕ}
    (h : G.Colorable n) : (G ⊠g H).Colorable n :=
  Colorable.of_hom tensorProd.fstHom h
```

Indexed Tensor Products

The tensor product generalizes, using the *indexed infimum* `⨅`, to the product of an arbitrary indexed family of graphs.

The indexed tensor product of a family of graphs, denoted `⊠g i, G i`.

```
def piTensorProd {ι : Type*} {V : ι → Type*} (G : (i : ι) → SimpleGraph (V i)) :
    SimpleGraph ((i : ι) → V i) := ⨅ i, (G i).comap (fun f ↦ f i)

@[inherit_doc piTensorProd]
scoped notation3 "⊠g "(...)", "r:67:(scoped G => SimpleGraph.piTensorProd G) => r
```

Adjacency in the indexed tensor product is given by adjacency in each component graph *and* the vertices being distinct. The distinctness condition is inherited from `SimpleGraph.iInf_adj`. When ι is nonempty it follows from adjacency in any one component, so it only matters when ι is empty. Then $(i : \iota) \to V\ i$ has a single element, and without the distinctness condition this vertex would be adjacent to itself.

```
@[simp]
lemma piTensorProd_adj {ι : Type*} {V : ι → Type*} {G : (i : ι) → SimpleGraph (V i)}
    {f g : (i : ι) → V i} :
    (⊗g i, G i).Adj f g ↔ (∀ i, (G i).Adj (f i) (g i)) ∧ f ≠ g :=
  iInf_adj
```

So when ι is nonempty, adjacency in the indexed tensor product is just adjacency in every component. (We give this higher simp priority than `piTensorProd_adj`, so that it fires first when ι is nonempty.)

```
@[simp high]
lemma piTensorProd_adj_of_nonempty {ι : Type*} [Nonempty ι] {V : ι → Type*}
    {G : (i : ι) → SimpleGraph (V i)} {f g : (i : ι) → V i} :
    (⊗g i, G i).Adj f g ↔ ∀ i, (G i).Adj (f i) (g i) := by
  rw [piTensorProd_adj]
  refine ⟨And.left, fun h => ⟨h, ?_⟩⟩
  obtain ⟨i⟩ := ‹Nonempty ι›
  exact fun e => (h i).ne (congrFun e i)
```

The proof above uses **`refine`**, which we will use often. Like **`exact`** it takes a term that proves the goal, but the term may contain holes `?_`, and each hole becomes a new goal. Here the term is an anonymous constructor for the $\leftrightarrow$: `And.left` proves the forward direction outright, and the reverse direction is `fun h => ⟨h, ?_⟩`. So after **`refine`** the only goal is the hole, $f \neq g$, with `h` in context.

Compare this with **`apply`**, which chooses the new goals itself: it makes a goal of every argument that it cannot fill in. Here `apply Iff.intro` leaves both directions as goals, including the one that `And.left` proves. Moreover **`apply`** elaborates its term before comparing it with the goal, so `apply ⟨And.left, fun h => ⟨h, ?_⟩⟩` fails: an anonymous constructor must know which type it is constructing.

We can only expect to decide adjacency in the indexed tensor product by checking every factor when the index type is finite: you can't check infinitely many conditions! The tensor product of a finite family of graphs has decidable adjacency if each graph in the family has decidable adjacency and each vertex type has decidable equality. (Decidable equality is needed to decide the distinctness condition $f \neq g$.)

```
instance {ι : Type*} {V : ι → Type*} [Fintype ι] [∀ i, DecidableEq (V i)]
    (G : (i : ι) → SimpleGraph (V i)) [∀ i, DecidableRel (G i).Adj] :
    DecidableRel (⊗g i, G i).Adj :=
  fun _ _ ↦ decidable_of_iff' _ piTensorProd_adj
```

If each `G i` has a finite neighbourhood at `f i` then the tensor product has a finite neighbourhood at f, provided the index type is finite, nonempty and has decidable equality. Again we make this a **`def`**, for the same reason as `tensorProdFintypeNeighborSet`.

If each `G i` has a finite neighbourhood at `f i`, then the indexed tensor product has a finite neighbourhood at f.

```
@[instance_reducible]
def piTensorProdFintypeNeighborSet {ι : Type*} {V : ι → Type*}
    {G : (i : ι) → SimpleGraph (V i)} {f : (i : ι) → V i} [Fintype ι] [DecidableEq ι]
    [Nonempty ι] [∀ i, Fintype ((G i).neighborSet (f i))] :
    Fintype ((⊗g i, G i).neighborSet f) :=
  Fintype.ofFinset (Fintype.piFinset fun i ↦ (G i).neighborFinset (f i)) fun y ↦ by
    simp
```

The analogues for the indexed tensor product of the maps in Maps of Tensor Products (p. 117) are in the exercises (Exercises Maps (p. 123)).

5.8 Claw-Free Graphs*

We say that `H` contains an induced copy of `G` if there is an embedding `G ↪g H`. This is written `G ≼ H`.

```
#source IsIndContained -- def IsIndContained (G : SimpleGraph V)
                       --       (H : SimpleGraph W) : Prop := Nonempty (G ↪g H)
```

There isn't currently a standard name for the property of not containing an induced copy of a graph in Mathlib, so we introduce one here (following the pattern of `Free`).

`A.IndFree B` means that `B` does not contain an induced copy of `A`.

```
abbrev IndFree {V W : Type*} (A : SimpleGraph V) (B : SimpleGraph W) := ¬A ≼ B
```

The *claw* is the complete bipartite graph `K[1, 3]`.

A graph is *claw-free* if it does not contain an *induced* copy of `K[1, 3]`.

```
abbrev ClawFree {V : Type*} (G : SimpleGraph V) : Prop := K[1, 3].IndFree G
```

Note that claw-free refers to *induced* copies of the claw. A copy of the claw that need not be induced is just a vertex with three distinct neighbours, so being claw-free in that sense would simply mean that every vertex has at most two neighbours. The two notions really are different: `K[4]` is claw-free, since any four of its vertices induce a copy of `K[4]`, yet `K[1, 3] ⊑ K[4]`, since every vertex of `K[4]` has three neighbours (see the exercises).

Line graphs are always claw-free.

```
theorem clawFree_lineGraph {V : Type*} (G : SimpleGraph V) : L[G].ClawFree := by
  classical
  -- Suppose not: then we have an embedding f : K[1, 3] ↪g L[G].
  rintro ⟨f⟩
  -- The centre of the claw is adjacent to each leaf, so its edge f (.inl 0) shares
  -- a vertex w i with each leaf edge f (.inr i).
  have key : ∀ i, ∃ v, v ∈ (f (.inl 0) : Sym2 V) ∧ v ∈ (f (.inr i) : Sym2 V) := by
    intro i
    have hadj : K[1, 3].Adj (.inl 0) (.inr i) := by simp
    rw [← f.map_adj_iff, lineGraph_adj_iff_exists] at hadj
    exact hadj.2
  -- The choose tactic turns key into a function w : Fin 3 → V together with
  -- hwc : ∀ i, w i ∈ f (.inl 0) and hwl : ∀ i, w i ∈ f (.inr i).
  -- This uses the axiom of choice.
  choose w hwc hwl using key
```

```
  -- The centre edge is s(a, b) with a ≠ b.
  obtain ⟨b, hb⟩ := Sym2.mem_iff_exists.1 (hwc 0)
  let a := w 0
  have hne : a ≠ b := by
    have hmem : (f (.inl 0) : Sym2 V) ∈ G.edgeSet := (f (.inl 0)).2
    rw [hb, mem_edgeSet] at hmem
    exact hmem.ne
  -- Each leaf's shared vertex is one of the two endpoints a or b.
  have hsides : ∀ i, w i = a ∨ w i = b := by grind
  -- Three leaves, two endpoints, so two leaves i ≠ j must share the same endpoint!
  -- We do this by the pigeonhole principle, applied to the map Fin 3 → Prop
  -- sending i to the proposition w i = a. This gives i ≠ j with w i = a and
  -- w j = a taking the same truth value and hence w i = w j.
  -- (Since either both are a or both are ≠ a and hence equal b.)
  obtain ⟨i, j, hij, hsame⟩ :=
    Fintype.exists_ne_map_eq_of_card_lt (w · = a) (by decide)
  have hwij : w i = w j := by grind
  -- But then the two leaf edges share w i, hence are adjacent in the line graph,
  -- contradicting that leaves of the claw are non-adjacent.
  have hfne : f (.inr i) ≠ f (.inr j) := fun h => hij (by simpa using f.injective h)
  have hadj : L[G].Adj (f (.inr i)) (f (.inr j)) :=
    lineGraph_adj_iff_exists.2 ⟨hfne, w j, hwij ▸ hwl i, hwl j⟩
  simp [f.map_adj_iff] at hadj
```

Triangles in Line Graphs

Our next goal is to show that the line graph of a graph `G` contains a copy of `K[3]` iff `G` contains either a copy of `K[3]` or a copy of the claw `K[1, 3]`.

First, two simple helper lemmas, which build the required copies directly from their vertices.

If `G` contains three mutually adjacent vertices then it contains a copy of the triangle `K[3]`.

```
lemma top_three_isContained {V : Type*} {G : SimpleGraph V} {a b c : V}
    (hab : G.Adj a b) (hac : G.Adj a c) (hbc : G.Adj b c) : K[3] ⊑ G := by
  -- Map the vertices 0, 1, 2 of K[3] to a, b, c using vector notation ![ ].
  -- Distinct vertices of K[3] are adjacent, and go to adjacent vertices of G.
  -- fin_cases i splits the goal into one case for each value of i : Fin 3.
  let f : K[3] →g G := ⟨![a, b, c], by
    intro i j h
    fin_cases i <;> fin_cases j <;> simp_all [hab.symm, hac.symm, hbc.symm]⟩
  -- A homomorphism from a complete graph is injective, so f gives a copy.
  exact ⟨f.toCopy f.injective_of_top_hom⟩
```

A vertex `a` with three distinct neighbours `b`, `c`, `d` gives a copy of the claw `K[1, 3]`.

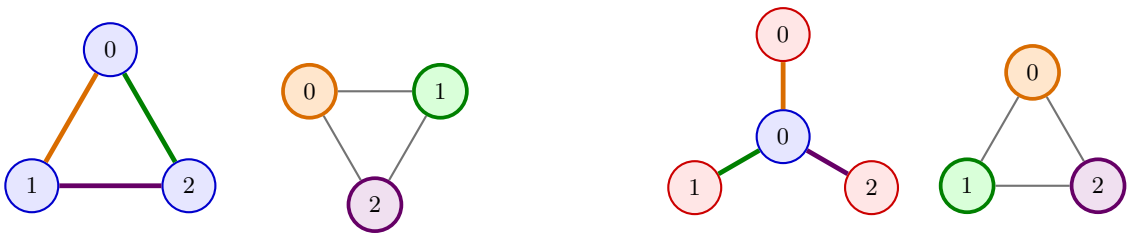
```
lemma claw_isContained {V : Type*} {G : SimpleGraph V} {a b c d : V} (hb : G.Adj a b)
    (hc : G.Adj a c) (hd : G.Adj a d) (hbc : b ≠ c) (hbd : b ≠ d) (hcd : c ≠ d) :
    K[1, 3] ⊑ G := by
  -- Map the centre of the claw K[1, 3] to a and the three leaves to b, c, d.
  -- We use Sum.elim to map the two parts of the sum type, and the
  -- vector notation ![ ], to map Fin 3 as 0 ↦ b, 1 ↦ c, 2 ↦ d.
  let φ : Fin 1 ⊕ Fin 3 → V := Sum.elim (fun _ => a) ![b, c, d]
  -- φ preserves adjacency: every edge of the claw maps to an edge of G.
  have hom : ∀ {x y : Fin 1 ⊕ Fin 3}, K[1, 3].Adj x y → G.Adj (φ x) (φ y) := by
```

```
    rintro (i | i) (j | j) h <;>
      fin_cases i <;> fin_cases j <;> -- 16 cases
        simp_all [φ, Adj.symm]
  -- φ is injective, since a, b, c, d are pairwise distinct.
  have inj : Function.Injective φ := by
    rintro (i | i) (j | j) h <;>
      fin_cases i <;> fin_cases j <;> simp_all [φ]
  -- Package the homomorphism ⟨φ, hom⟩ together with injectivity into a Copy.
  exact ⟨⟨φ, hom⟩, inj⟩
```

A locally finite graph `G` contains a copy of the claw `K[1, 3]` iff it has a vertex of degree at least 3.

```
theorem claw_isContained_iff {V : Type*} (G : SimpleGraph V) [G.LocallyFinite] :
    K[1, 3] ⊑ G ↔ ∃ v, 3 ≤ G.degree v := by
  rw [exists_three_le_degree_iff]
  constructor
  · intro ⟨f⟩
    refine ⟨f (.inl 0), f (.inr 0), f (.inr 1), f (.inr 2), ?_, ?_, ?_, ?_, ?_, ?_⟩
      <;> first
        | exact f.toHom.map_adj (by decide)
        | exact f.injective.ne (by decide)
  · rintro ⟨_, _, _, _, hb, hc, hd, hbc, hbd, hcd⟩
    exact claw_isContained hb hc hd hbc hbd hcd
```

Next we look at the line graphs of `K[3]` and `K[1, 3]` themselves: both are isomorphic to `K[3]`.



Left: `K[3]` and its line graph. Right: the claw `K[1, 3]` and its line graph.

In defining the two isomorphisms we make heavy use of `decide`. Indeed it is in situations like this that we see the value of making things computable. We really don't want to perform tedious small case analysis by hand, so we should always aim to use `decide` or similar automation (such as `fin_cases`) where possible.

The line graph of `K[3]` is isomorphic to a triangle `K[3]`.

```
def lineGraphCompleteThreeIsoCompleteThree : L[K[3]] ≃g K[3] where
  toFun e := if (e : Sym2 (Fin 3)) = s(0, 1) then 0
    else if (e : Sym2 (Fin 3)) = s(0, 2) then 1 else 2
  invFun i := if i = 0 then ⟨s(0, 1), by decide⟩ else
              if i = 1 then ⟨s(0, 2), by decide⟩ else ⟨s(1, 2), by decide⟩
  left_inv := by decide
  right_inv := by decide
  map_rel_iff' := by decide
```

The line graph of the claw graph `K[1, 3]` is isomorphic to a triangle `K[3]`.

```
def lineGraphClawIsoCompleteThree : L[K[1, 3]] ≃g K[3] where
  toFun e := if (e : Sym2 (Fin 1 ⊕ Fin 3)) = s(.inl 0, .inr 0) then 0
```

```
      else if (e : Sym2 (Fin 1 ⊕ Fin 3)) = s(.inl 0, .inr 1) then 1 else 2
  invFun i := if i = 0 then ⟨s(.inl 0, .inr 0), by decide⟩ else
    if i = 1 then ⟨s(.inl 0, .inr 1), by decide⟩ else ⟨s(.inl 0, .inr 2), by decide⟩
  left_inv := by decide
  right_inv := by decide
  map_rel_iff' := by decide
```

Since containment passes to line graphs (`IsContained.lineGraph`), a copy of `K[3]` in *G* gives a copy of `L[K[3]]`, and hence of `K[3]`, in the line graph of *G*. Likewise a copy of the claw in *G* gives a copy of `L[K[1, 3]]`, which is again a triangle.

If a graph *G* contains a triangle, then its line graph contains a triangle.

```
lemma isContained_lineGraph_of_triangle {V : Type*} {G : SimpleGraph V}
    (h : K[3] ⊑ G) : K[3] ⊑ L[G] :=
  lineGraphCompleteThreeIsoCompleteThree.isContained'.trans h.lineGraph
```

If a graph *G* contains a claw, then its line graph contains a triangle.

```
lemma isContained_lineGraph_of_claw {V : Type*} {G : SimpleGraph V}
    (h : K[1, 3] ⊑ G) : K[3] ⊑ L[G] :=
  lineGraphClawIsoCompleteThree.isContained'.trans h.lineGraph
```

Conversely, a triangle in the line graph comes from either a triangle or a copy of the claw in *G*: three pairwise distinct, pairwise intersecting edges either share a common vertex (giving a copy of the claw) or form a triangle.

If the line graph of a graph *G* contains a triangle, then *G* contains a triangle or a copy of the claw.

```
lemma top_three_or_claw_isContained_of_isContained_lineGraph {V : Type*}
    {G : SimpleGraph V} (h : K[3] ⊑ L[G]) : K[3] ⊑ G ∨ K[1, 3] ⊑ G := by
  obtain ⟨f⟩ := h
  -- the copy f of K[3] in the line graph maps pairs of distinct vertices
  -- of K[3] to pairs of intersecting edges in G.
  have meet : ∀ i j : Fin 3, i ≠ j →
      f i ≠ f j ∧ ∃ v, v ∈ (f i : Sym2 V) ∧ v ∈ (f j : Sym2 V) := by
    intro i j hij
    have hadj : L[G].Adj (f i) (f j) := f.topEmbedding.map_adj_iff.2 hij
    rwa [lineGraph_adj_iff_exists] at hadj
  -- get the three meeting points of the three pairs of edges in G corresponding
  -- to the three pairs of vertices of K[3] in the line graph.
  obtain ⟨h01, x, hx0, hx1⟩ := meet 0 1 (by decide)
  obtain ⟨h12, y, hy1, hy2⟩ := meet 1 2 (by decide)
  obtain ⟨h02, z, hz0, hz2⟩ := meet 0 2 (by decide)
  by_cases hxy : x = y
  · -- The shared vertex x = y lies on all three edges: a copy of the claw centred
    -- at x.
    obtain ⟨a, ha⟩ := Sym2.mem_iff_exists.1 hx0
    obtain ⟨b, hb⟩ := Sym2.mem_iff_exists.1 hx1
    obtain ⟨c, hc⟩ := Sym2.mem_iff_exists.1 (hxy.symm ▸ hy2)
    -- (f i).prop : ↑(f i) ∈ G.edgeSet, so rewriting with ha : ↑(f 0) = s(x, a)
    -- gives s(x, a) ∈ G.edgeSet, which is defeq to G.Adj x a (similarly for hb
    -- and hc).
    exact Or.inr ◁ claw_isContained (ha ▸ (f 0).prop) (hb ▸ (f 1).prop)
                    (hc ▸ (f 2).prop) (by grind) (by grind) (by grind)
```

```
  · -- Otherwise the three meeting points x, y, z are distinct: a triangle.
    have hf1 : (f 1 : Sym2 V) = s(x, y) := (Sym2.mem_and_mem_iff hxy).1 ⟨hx1, hy1⟩
    obtain ⟨hzx, hzy⟩ : z ≠ x ∧ z ≠ y := by grind [Sym2.mem_and_mem_iff]
    have : f 0 = s(x, z) := (Sym2.mem_and_mem_iff hzx.symm).1 ⟨hx0, hz0⟩
    have : f 2 = s(y, z) := (Sym2.mem_and_mem_iff hzy.symm).1 ⟨hy2, hz2⟩
    obtain ⟨axy, axz, ayz⟩ : G.Adj x y ∧ G.Adj x z ∧ G.Adj y z := by
      grind [mem_edgeSet]
    exact Or.inl ◁ top_three_isContained axy axz ayz
```

Putting everything together we have a complete characterization.

The line graph of G contains a triangle iff G contains a triangle or a copy of the claw.

```
theorem top_three_isContained_lineGraph_iff {V : Type*}
    {G : SimpleGraph V} : K[3] ⊑ L[G] ↔ K[3] ⊑ G ∨ K[1, 3] ⊑ G :=
  ⟨top_three_or_claw_isContained_of_isContained_lineGraph,
    fun h => h.elim isContained_lineGraph_of_triangle isContained_lineGraph_of_claw⟩
```

5.9 Exercises

1. Show that there is no graph homomorphism from `K[n + 1]` to `K[n]` (i.e. `K[n + 1]` is not n-colourable).

```
variable {V W X : Type*}
example (n : ℕ) : IsEmpty (K[n + 1] →g K[n]) := by
  sorry
```

2. Define an embedding of `P[n]` into `P[n + k]` so that the image is exactly the last n vertices.

```
def pathGraph.embedding' (n k : ℕ) : P[n] ↪g P[n + k] := sorry
```

3. Show that any $G \hookrightarrow g\ H$ gives rise to an embedding of G into any blow-up of H with nonempty parts.

```
def Embedding.toBlowUp' {G : SimpleGraph V} {ι : Type*} {H : SimpleGraph ι}
    (e : G ↪g H) (U : ι → Type*) (f : ∀ (i : ι), U i) :
    G ↪g H.blowUpGraph U := by
  sorry
```

4. Show that if $G \leq H$ are graphs on the same vertex type then $G \sqsubseteq H$.

```
example {G H : SimpleGraph V} (h : G ≤ H) : G ⊑ H := by
  sorry
```

5. Show that if $G \simeq g\ H$ then $G^c \simeq g\ H^c$.

```
example (G : SimpleGraph V) (H : SimpleGraph W) (f : G ≃g H) : Gᶜ ≃g Hᶜ := by
  sorry
```

6. Show that a graph isomorphic to `⊥` is itself `⊥`.

```
example (G : SimpleGraph V) (f : G ≃g (⊥ : SimpleGraph W)) : G = ⊥ := by
  sorry
```

7. Show that the path graph `P[4]` is *self-complementary*, i.e. isomorphic to its own complement. Hint: find a suitable permutation of the vertices; you can define it and its inverse using `Matrix` vector notation `![_, _, _, _]`.

```
example : P[4] ≃g P[4]ᶜ := by
  sorry
```

8. Show that the cycle `C[5]` is also self-complementary. Hint: try `i ↦ 2 * i`.

```
example : C[5] ≃g C[5]ᶜ := by
  sorry
```

9. We saw that for an embedding `f : V ↪ W` we have `(G.map f).comap f = G`, but only `(H.comap f).map f ≤ H`. Show that this inequality can be strict.

```
example : ∃ (f : Fin 1 ↪ Fin 2) (H : SimpleGraph (Fin 2)),
    (H.comap f).map f ≠ H := by
  sorry
```

10. Define an embedding of `Q[n]` into `Q[n + k]` by iterating `hypercube.embedFalse`.

```
def hypercube.embedding' (n : ℕ) : (k : ℕ) → Q[n] ↪g Q[n + k] := sorry
```

The next question asks for an isomorphism `Q[2] ≃g C[4]`. In Algebraic Graph Theory (p. 360) we use it to identify the automorphism group of the 2-dimensional hypercube.

11. Show that `Q[2]` is isomorphic to the cycle `C[4]`. Both graphs are concrete finite graphs with decidable adjacency (`DecidableRel Q[n].Adj` is `hypercubeDecidableRelAdj`, and Mathlib provides `instDecidableRelFinAdjCycleGraph` for `C[n]`), so every proof obligation can be checked by `decide`. Recall also that `v[i, j]` where `i, j ∈ {0, 1}` is scoped notation for the vertices of `Q[2]`. [2]

```
open HypercubeVert

noncomputable def hypercubeTwoIsoCycleFour' : Q[2] ≃g C[4] := sorry
```

The next two exercises are about vertex-transitive graphs.

12. Show that if `G` is vertex-transitive then so is its complement `Gᶜ`, and deduce that the empty graph `⊥` is vertex-transitive. Hint: as in Exercise 5, an automorphism of `G` is also an automorphism of `Gᶜ`; and `compl_top` says `⊤ᶜ = ⊥`.

[2] The stub is marked `noncomputable` only so that its `sorry` does not cause problems when this guide is built; you can remove the keyword once you have filled in the solution.

```
lemma IsVertexTransitive.compl' {G : SimpleGraph V} (h : G.IsVertexTransitive) :
    Gᶜ.IsVertexTransitive := by
  sorry

lemma isVertexTransitive_bot' : (⊥ : SimpleGraph V).IsVertexTransitive := by
  sorry
```

13. Our characterization `isVertexTransitive_mycielskian_iff` of the vertex-transitive Mycielskians assumes that `V` is nonempty. Show that this assumption is needed: if `V` is empty then the Mycielskian of any graph on `V` is vertex-transitive, but the graph is not isomorphic to `K[2]`.

Hint: if `V` is empty then the Mycielskian has the single vertex `none`.

```
lemma isVertexTransitive_mycielskian_of_isEmpty' [IsEmpty V] (G : SimpleGraph V) :
    mμ(G).IsVertexTransitive :=
  sorry

lemma isEmpty_iso_completeGraph_two_of_isEmpty' [IsEmpty V] (G : SimpleGraph V) :
    IsEmpty (G ≃g K[2]) :=
  sorry
```

Exercises 14–18 are about the indexed tensor product `⊗g i, G i` (`piTensorProd`) from Indexed Tensor Products (p. 117), in the starred section on tensor products.

14. Show that for each index `j` the projection out of the indexed tensor product `⊗g i, G i` is a graph homomorphism to `G j`.

```
example {ι : Type*} {U : ι → Type*} (G : ∀ i, SimpleGraph (U i)) (j : ι) :
    ⊗g i, G i →g G j := sorry
```

15. The universal property of the indexed tensor product. Given graph homomorphisms from `K` to each `G i`, there is a homomorphism from `K` to the indexed tensor product `⊗g i, G i` (provided `ι` is nonempty, which here follows from the existence of `i₀`; the next exercise looks at the case when `ι` is empty).

```
example {ι : Type*} {U : ι → Type*} (i₀ : ι) {K : SimpleGraph X}
    (G : ∀ i, SimpleGraph (U i)) (f : ∀ i, K →g G i) : K →g ⊗g i, G i := sorry
```

16. Prove that if `ι` is empty then there is a graph homomorphism from a graph `K` to the indexed tensor product `⊗g i, G i` iff `K` is the empty graph `⊥`.

```
lemma nonempty_hom_piTensorProd_iff_of_isEmpty' {ι : Type*} [IsEmpty ι]
    {U : ι → Type*} {K : SimpleGraph X} (G : ∀ i, SimpleGraph (U i)) :
    Nonempty (K →g ⊗g i, G i) ↔ K = ⊥ := by
  sorry
```

17. The degree of `f` in the indexed tensor product is the product of the degrees of its components. Hint: the neighbour set is the `Fintype.piFinset` of the componentwise neighbour sets, and (using `[Nonempty ι]`) the `≠` condition in the adjacency is automatic.

```
example {ι : Type*} {U : ι → Type*} {G : (i : ι) → SimpleGraph (U i)}
    {f : (i : ι) → U i} [Fintype ι] [Nonempty ι]
    [∀ i, Fintype ((G i).neighborSet (f i))] [Fintype ((⊗g i, G i).neighborSet f)] :
    (⊗g i, G i).degree f = ∏ i, (G i).degree (f i) := by
  sorry
```

18. Given graph homomorphisms from K to each G `i`, one of which, $f\ i_0$, is a graph embedding, we have a graph embedding of K into the indexed tensor product. (We say that $f\ i_0$ is a graph embedding by giving both additional properties that allow the original homomorphism to be upgraded: injectivity *inj* and adjacency reflection *reflect*.)

```
example {ι : Type*} {U : ι → Type*} (i₀ : ι) {K : SimpleGraph X}
    (G : ∀ i, SimpleGraph (U i)) (f : ∀ i, K →g G i)
    (inj : Function.Injective (f i₀))
    (reflect : ∀ {x y}, (G i₀).Adj (f i₀ x) (f i₀ y) → K.Adj x y) :
    K ↪g ⊗g i, G i where
  toFun x i := sorry
  inj' := sorry
  map_rel_iff' := sorry
```

The last exercise is about `ClawFree`.

19. Show that `K[4]` is claw-free, but contains a non-induced copy of the claw `K[1, 3]`.

```
example : K[4].ClawFree ∧ K[1, 3] ⊑ K[4] := by
  sorry
```

6 Cliques

Cliques (sets of pairwise adjacent vertices) and independent sets (sets of pairwise non-adjacent vertices) are among the most basic substructures of a graph, and the sizes of the largest ones, the *clique number* and the *independence number*, are two of the most studied graph parameters. The archetypal question in extremal graph theory asks how many edges a graph can have without containing a large clique, and, as we will see in the next chapter, the clique number is a natural lower bound for the chromatic number. The Mycielskian is important precisely because it shows how bad this bound can be: it allows us to construct triangle-free graphs with arbitrarily large chromatic number.

In this chapter we introduce Mathlib's cliques (`IsClique`, `IsNClique`) and clique-free graphs (`CliqueFree`), and show that graph homomorphisms map cliques to cliques. Mathlib has no `Decidable` instance for `CliqueFree`, so we build one from the finset of all n-cliques (`cliqueFinset`). We then introduce the clique number (`cliqueNum`), independent sets (`IsIndepSet`) and the independence number (`indepNum`), and compute the independence number of the Kneser graph (`indepNum_kneserGraph`) using the Erdős–Ko–Rado theorem (`Finset.erdos_ko_rado`). Finally we prove the well-known result that the Mycielskian preserves the clique number (`cliqueNum_mycielskian_of_ne_bot`), and give a formula for the independence number of the Mycielskian (`indepNum_mycielskian`).

6.1 Cliques

A *clique* in a graph is a (not necessarily finite) set of vertices that are pairwise adjacent.

```
section cliques
variable {V : Type*} {G : SimpleGraph V}

#source IsClique -- abbrev IsClique (s : Set α) : Prop := s.Pairwise G.Adj


#source Set.Pairwise -- def Set.Pairwise (s : Set α) (r : α → α → Prop) :=
                     --   ∀ ⦃x⦄, x ∈ s → ∀ ⦃y⦄, y ∈ s → x ≠ y → r x y
```

If $G \le H$ and `s` is a clique in G then `s` is also a clique in `H`.

```
#check IsClique.mono
```

If s is a clique in G then s is also a clique in the graph $G \sqcup H$ (where H is any other graph on the same vertex set).

```
example {s : Set V} (h : G.IsClique s) {H : SimpleGraph V} : (G ⊔ H).IsClique s :=
  h.mono le_sup_left
```

If `s` is a clique in `G` then any subset of `s` is also a clique in `G`.

```
#check IsClique.subset
```

```
example {s t : Set V} (h : G.IsClique (s ∪ t)) : G.IsClique t := by
  apply h.subset
  exact Set.subset_union_right
```

If `s : Set V` and `a : V` then `insert a s` is the set `{a} ∪ s`.

```
example (s : Set V) (a : V) : insert a s = {a} ∪ s := rfl
```

If `s` is a clique in `G` and `a` is a vertex adjacent to every vertex in `s` that is not equal to `a` then `insert a s` is also a clique in `G`.

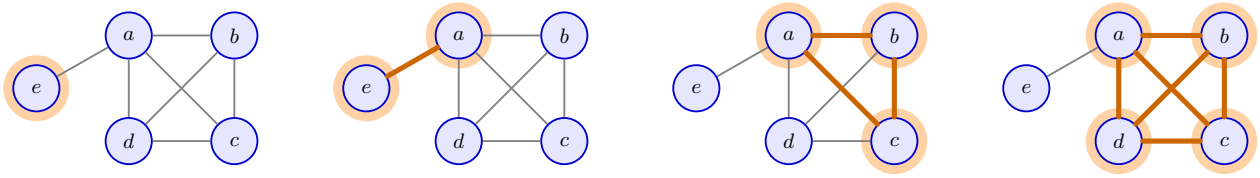
```
#check IsClique.insert
```

Recall that if `u ≠ v` then `edge u v` is the `SimpleGraph V` with the single edge `s(u, v)` and so `G ⊔ edge u v` is the graph obtained by adding this edge to `G`.

If `u ≠ v` then any set of vertices `s` is a clique in `G ⊔ edge u v` if and only if `s \ {u}` and `s \ {v}` are both cliques in `G`.

```
#check isClique_sup_edge_of_ne_iff
```

6.2 Finite Cliques



Examples of cliques in a graph

If `s : Finset V` is a clique with `n` vertices then we call it an `n`-clique. This is `IsNClique` in Lean, which is a structure that bundles a proof that `s` is a clique together with a proof that `s` has cardinality `n`.

Notice that `IsClique` takes a `Set V` while `IsNClique` takes a `Finset V`. A clique may be infinite, so `IsClique` needs to work with general sets. But an `n`-clique needs a size in $\mathbb{N}$, and `#s` is a natural number for a finset (and computable, so `decide` and `#eval` can use it). The field `isClique : G.IsClique s` silently coerces the finset `s` to a set. This coercion is written `↑s` (or `(s : Set V)`).

```
#source IsNClique -- structure IsNClique (n : ℕ) (s : Finset α) : Prop where
                  --   isClique : G.IsClique s
                  --   card_eq : #s = n
```

The `Finset {a, b}` is a 2-clique if and only if `G.Adj a b` holds.

```
lemma is2Clique_pair_iff {a b : V} [DecidableEq V] :
    G.IsNClique 2 {a, b} ↔ G.Adj a b := by
  constructor <;> intro h
  · apply h.isClique (by grind) (by grind)
```

```
    rintro rfl
    grind [h.card_eq]
  · constructor <;> simp [h, h.ne]
```

The `Finset {a, b, c}` is a 3-clique if and only if a, b, and c are pairwise adjacent. Notice in this and the previous lemma we need to give Lean a `DecidableEq V` instance to be able to form the `Finset {a, b}` and `{a, b, c}`. Also we don't need to assume that the vertices in the cliques are distinct because pairwise adjacency trivially implies that they are distinct.

```
example {a b c : V} [DecidableEq V] :
    G.IsNClique 3 {a, b, c} ↔ G.Adj a b ∧ G.Adj a c ∧ G.Adj b c :=
  is3Clique_triple_iff
```

If we insert a vertex a into an `n`-clique `s` and a is adjacent to every vertex in `s` then `insert a s` is an `(n + 1)`-clique.

```
#check IsNClique.insert
```

For concrete finite graphs `decide` can check small examples directly.

```
example : K[9].IsNClique 5 {0, 1, 3, 5, 7} := by decide
```

Famously the Mycielskian of any triangle-free graph is also triangle-free, so in particular any iterated Mycielskian of `K[2]` is triangle-free. (We will prove this and a more general result later.)

For now we just check that a particular set of 3 vertices in the 3rd iterated Mycielskian of `K[2]` (`mμ^3(K[2])`) is not a 3-clique.

```
example :
    ¬mμ^3(K[2]).IsNClique 3 {(0 : Fin 2)ʳʳʳ, (1 : Fin 2)ˡˡˡ, (1 : Fin 2)ˡʳˡ} := by
  decide
```

Any map $f : V \to W$ that sends adjacent vertices of a clique s of G to adjacent vertices of a graph H is injective on s: distinct vertices of s are adjacent, so their images are adjacent, hence distinct (since H is loopless).

```
lemma IsClique.injOn_of_adj {W : Type*} {H : SimpleGraph W} {s : Set V}
    (hs : G.IsClique s) {f : V → W}
    (hadj : ∀ ⦃x⦄, x ∈ s → ∀ ⦃y⦄, y ∈ s → G.Adj x y → H.Adj (f x) (f y)) :
    Set.InjOn f s := by
  intro x hx y hy hxy
  by_contra hne
  exact (hadj hx hy (hs hx hy hne)).ne hxy
```

In particular a graph homomorphism `G →g H` is injective on any clique. This generalizes `Hom.injective_of_top_hom` (the case where the graph is a clique).

```
lemma Hom.injOn_of_isClique {W : Type*} {H : SimpleGraph W} (f : G →g H)
    {s : Set V} (hs : G.IsClique s) : Set.InjOn f s :=
  hs.injOn_of_adj fun _ _ _ _ h => f.map_adj h
```

Hence the image of an n-clique s under such a map is an n-clique of H. We will use this several times, with f a graph homomorphism or a projection.

If a map `f : V → W` sends adjacent vertices of an `n`-clique `s` of `G` to adjacent vertices of `H` then the image of `s` is an `n`-clique of `H`.

```
lemma IsNClique.image_of_adj {W : Type*} {H : SimpleGraph W} [DecidableEq W]
    {n : ℕ} {s : Finset V} (hs : G.IsNClique n s) {f : V → W}
    (hadj : ∀ ⦃x⦄, x ∈ s → ∀ ⦃y⦄, y ∈ s → G.Adj x y → H.Adj (f x) (f y)) :
    H.IsNClique n (s.image f) := by
  have hinj := hs.isClique.injOn_of_adj hadj
  refine ⟨?_, by rw [Finset.card_image_of_injOn hinj, hs.card_eq]⟩
  rintro _ ha _ hb hne
  obtain ⟨x, hx, rfl⟩ := Finset.mem_image.mp ha
  obtain ⟨y, hy, rfl⟩ := Finset.mem_image.mp hb
  exact hadj hx hy (hs.isClique hx hy fun h => hne (by rw [h]))
```

The image of an `n`-clique under a graph homomorphism is an `n`-clique.

```
protected lemma IsNClique.image {W : Type*} {H : SimpleGraph W} [DecidableEq W]
    {n : ℕ} {s : Finset V} (hs : G.IsNClique n s) (f : G →g H) :
    H.IsNClique n (s.image f) :=
  hs.image_of_adj fun _ _ _ _ h => f.map_adj h
```

6.3 Clique-Free

Much of extremal graph theory is concerned with properties of graphs that do not contain cliques of a certain size. For example, Mantel's theorem says that a triangle-free graph on n vertices has at most `n ^ 2 / 4` edges.

We say that a graph `G` is *n-clique-free* if it contains no n-cliques.

```
#source CliqueFree -- def CliqueFree (n : ℕ) : Prop := ∀ t, ¬G.IsNClique n t
```

So `¬G.CliqueFree n` holds if and only if `G` contains an `n`-clique.

```
example (n : ℕ) : ¬G.CliqueFree n ↔ ∃ t, G.IsNClique n t := by
  simp [CliqueFree]
```

If `G ≤ H` and `H` is `n`-clique-free then so is `G`, so the property of being `n`-clique-free is monotone downwards (in Mathlib, `Antitone`).

```
#check CliqueFree.anti      -- G ≤ H → CliqueFree n H → CliqueFree n G
```

More generally, Mathlib's `CliqueFree.comap` says that if `G` contains a copy of `H` (`H ⊑ G`, see `IsContained`) and `G` is n-clique-free then so is `H`. Since a copy is just an injective homomorphism, we can ask whether injectivity is needed. It is not: a homomorphism `G →g H` is enough, since by `IsNClique.image` it maps any n-clique of `G` to an n-clique of `H`.

```
#check CliqueFree.comap     -- G ⊑ H → CliqueFree n H → CliqueFree n G
```

```
lemma CliqueFree.of_hom {W : Type*} {H : SimpleGraph W} {n : ℕ} (f : G →g H)
    (h : H.CliqueFree n) : G.CliqueFree n :=
  fun t ht => by classical exact h _ (ht.image f)
```

An `n`-clique is the same thing as a copy of the complete graph `K[n]`, so `G` is `n`-clique-free exactly when it contains no copy of `K[n]`, i.e. when `G` is `K[n]`-free in the sense of `Free`.

```
#check not_cliqueFree_iff_top_isContained -- ¬G.CliqueFree n ↔ K[n] ⊑ G
#check cliqueFree_iff_free_top_fin        -- G.CliqueFree n ↔ K[n].Free G

end cliques
```

If we take any complete multipartite graph with nonempty parts indexed by a type `ι` then it will contain an `n`-clique if and only if the index type has cardinality at least `n`. Notice that this result makes sense for `ι` an infinite type, so we need a way to say that `n ≤ #ι` without assuming that `ι` is finite.

In this situation we can use `ENat.card` which is the cardinality of a type as an extended natural number `ENat` or $\mathbb{N}\infty$. (See Cardinality (p. 66) for more details on the different ways to talk about cardinality.)

```
#check completeMultipartiteGraph.not_cliqueFree_of_le_enatCard
```

Notice how this lemma requires the parts to be nonempty. Instead of a hypothesis `∀ i, Nonempty (W i)` it takes a function `f : ∀ i, W i` choosing a vertex in each part. Having such a choice function is equivalent to assuming that all its parts are nonempty, but is often easier to work with. We do the same in `cliqueFree_blowUpGraph_iff` below.

Conversely, if `ι` is finite with less than `n` elements then the graph is `n`-clique-free. (Mathlib states this direction for a `Fintype` index.)

```
#check cliqueFree_completeMultipartiteGraph
```

The blow-up of a graph *G* with nonempty parts is *n*-clique-free if and only if *G* is *n*-clique-free.

```
lemma cliqueFree_blowUpGraph_iff {n : ℕ} {ι : Type*} (G : SimpleGraph ι)
    (W : ι → Type*) (f : ∀ i, W i) :
    (G.blowUpGraph W).CliqueFree n ↔ G.CliqueFree n := by
  constructor <;> intro h
  · exact h.comap ⟨(Embedding.blowUp _ _ f).toCopy⟩
  · -- the projection Sigma.fst to the index of the part is a homomorphism
    exact h.of_hom ⟨Sigma.fst, id⟩
```

Note that `CliqueFree` does not have a `Decidable` instance in Mathlib, so we cannot use **`decide`** or **`#eval`** to check if a graph is *n*-clique-free. However, there is a computable way to enumerate all *n*-cliques in a finite graph, and this will allow us to provide such an instance.

6.4 The Finset of n-Cliques

If `G` is a finite graph with a `Fintype V` instance then we can form the `Finset` of all `n`-cliques in `G`.

```
section more_cliques
variable {V : Type*} {G : SimpleGraph V}

#source cliqueFinset -- def cliqueFinset (n : ℕ) : Finset (Finset α) :=
                     --   {s | G.IsNClique n s}
```

Recall our discussion of Set-Builder Notation (p. 61). The finset `G.cliqueFinset n` is `univ : Finset (Finset V)`, i.e. the powerset of *V*, filtered for those `Finset`s that are *n*-cliques in *G*.

```
example [Fintype V] (n : ℕ) [DecidableRel G.Adj] [DecidableEq V] :
  G.cliqueFinset n =
     (Finset.univ : Finset (Finset V)).filter (G.IsNClique n · ) := rfl
```

This is computable so we can use `#eval` to compute the `Finset (Finset V)` of all n-cliques in a finite graph `G : SimpleGraph V`.

```
#eval C[5].cliqueFinset 2     -- {{0, 1}, {1, 2}, {2, 3}, {0, 4}, {3, 4}}
#eval mμ(K[4]).cliqueFinset 3 -- {{0ˡ, 1ˡ, 2ˡ}, {0ˡ, 1ˡ, 3ˡ},..., {1ˡ, 2ˡ, 3ʳ}}
#eval mμ(C[5]).cliqueFinset 5 -- ∅
```

The `cliqueFinset` of n-cliques in a finite graph is empty if and only if the graph is n-clique-free.

```
#check cliqueFinset_eq_empty_iff -- G.cliqueFinset n = ∅ ↔ G.CliqueFree n
```

Hence we can decide if a finite graph is n-clique-free by checking if `G.cliqueFinset n = ∅`.

```
instance decidableCliqueFree [Fintype V] (n : ℕ) [DecidableRel G.Adj]
    [DecidableEq V] : Decidable (G.CliqueFree n) :=
  decidable_of_iff _ cliqueFinset_eq_empty_iff

#eval C[5].CliqueFree 3     -- true
#eval mμ(C[5]).CliqueFree 3 -- true
#eval K[4].CliqueFree 3     -- false
```

6.5 Clique Number

The *clique number* of a graph is the size of the largest clique in the graph. In Mathlib this is defined as the `sSup` of the set of n such that there exists an n-clique in the graph. We write `ω G` for the clique number of `G`.

```
#source cliqueNum -- noncomputable def cliqueNum (G : SimpleGraph α) : ℕ :=
                  --   sSup {n | ∃ s, G.IsNClique n s}

@[inherit_doc]
scoped notation "ω" => SimpleGraph.cliqueNum
```

Since this is our first time seeing `sSup` we will take a moment to explain what it is. The name `sSup` stands for *set supremum*; for `ℕ` this is the largest element of a nonempty bounded `Set ℕ`, or `0` if the set is empty or unbounded.

```
#check Nat.sSup_def
#check Nat.sSup_of_not_bddAbove
```

What this means for our definition of `cliqueNum` is that:

1. If there is no bound on the size of cliques in `G` then `G.cliqueNum = 0`. This is the same lossy `0`-convention we met with `Nat.card` in Cardinality (p. 66): `sSup` on `ℕ` returns the genuine largest element only when its argument is nonempty and bounded above, and falls back to `0` otherwise.

2. If `G` has no vertices then `G.cliqueNum = 0`, but this time for a genuine reason: the empty set is a `0`-clique, so `0` really is the largest clique size. (So in particular `G.cliqueNum = 0` can hold for two very different reasons.)

3. It is noncomputable. This is the trade-off for being able to define it for arbitrary graphs. (Although since computing clique numbers is NP-hard, a computable version would only go so far.)

4. Because it doesn't rely on `Fintype V`, when we prove theorems about it for finite graphs we can use `Finite V` as our finiteness assumption.

For example, the complete graph on $\mathbb{N}$ has cliques of every size, so its clique number is the junk value `0`, while the complete graph on `Fin 3` has clique number `3`. (In the next chapter we will see that Mathlib makes a different choice for the chromatic number, which takes values in $\mathbb{N}\infty$, so that an infinite chromatic number is recorded as `⊤` rather than `0`.)

```
example : ω (⊤ : SimpleGraph ℕ) = 0 := by simp
example : ω (⊤ : SimpleGraph (Fin 3)) = 3 := by simp
```

In a finite graph any clique has size at most the `cliqueNum`:

```
#check IsClique.card_le_cliqueNum
```

An n-clique in a finite graph provides a lower bound on the clique number of the graph.

```
lemma IsNClique.le_cliqueNum [Finite V] {n : ℕ} {s : Finset V}
    (h : G.IsNClique n s) : n ≤ ω G :=
  h.card_eq ▸ h.isClique.card_le_cliqueNum
```

In a finite graph the clique number is at most the number of vertices, since any clique is a subset of the vertex set.

```
#check cliqueNum_le_natCard -- ω G ≤ Nat.card V
```

The clique number of the complete graph on a type is its cardinality measured using `Nat.card`. It needs no finiteness assumption: on an infinite type both sides are the junk value `0`.

```
#check cliqueNum_top -- (⊤ : SimpleGraph α).cliqueNum = Nat.card α
```

Any nonempty finite graph has clique number at least 1 (a single vertex is a `1`-clique). Mathlib states this as `cliqueNum_ne_zero_of_finite`.

```
lemma one_le_cliqueNum [Finite V] [Nonempty V] : 1 ≤ ω G :=
  (G.cliqueNum_ne_zero_of_finite).bot_lt
```

Every graph, finite or not, has a clique whose size is the clique number: if the clique sizes are unbounded then `ω G = 0`, and $\varnothing$ is a `0`-clique!

```
#check exists_isNClique_cliqueNum -- ∃ s, G.IsNClique (ω G) s
```

A graph homomorphism into a finite graph cannot decrease the clique number: by `exists_isNClique_cliqueNum` there is a clique of size `ω G`, and it maps to a clique of the same size in `H`. The hypothesis `[Finite W]` cannot be dropped. For example, `K[3]` maps into the complete graph on $\mathbb{N}$, but the complete graph on $\mathbb{N}$ has clique number `0 < 3`.

```
lemma Hom.cliqueNum_le {W : Type*} [Finite W] {H : SimpleGraph W} (f : G →g H) :
    ω G ≤ ω H := by
  classical
  obtain ⟨s, hs⟩ := G.exists_isNClique_cliqueNum
  exact (hs.image f).le_cliqueNum
```

By contrast, the clique number is invariant under any graph isomorphism. An isomorphism matches up the cliques of the two graphs, so the two sets of clique sizes are equal, and hence so are their sSups (junk values included).

```
lemma Iso.cliqueNum_eq {W : Type*} {H : SimpleGraph W} (e : G ≃g H) : ω G = ω H := by
  classical
  simp only [cliqueNum]
  congr 1
  ext n
  exact ⟨fun ⟨_, hs⟩ => ⟨_, hs.image e.toHom⟩,
         fun ⟨_, hs⟩ => ⟨_, hs.image e.symm.toHom⟩⟩
```

If G is $(n + 1)$-clique-free then $\omega(G) \le n$: otherwise the clique of size $\omega(G)$ given by `exists_isNClique_cliqueNum` would contain an $(n + 1)$-clique.

```
lemma CliqueFree.cliqueNum_le {n : ℕ} (h : G.CliqueFree (n + 1)) : ω G ≤ n := by
  obtain ⟨s, hs⟩ := G.exists_isNClique_cliqueNum
  -- by_contra! is by_contra with the negation pushed inwards: hlt : n < ω G
  by_contra! hlt
  exact h.mono hlt _ hs
```

So to compute the clique number it is enough to find an n-clique and show that the graph is $(n + 1)$-clique-free. The clique-free hypothesis bounds the size of every clique, so n is the largest element of the set of clique sizes, and `le_csSup` applies.

```
theorem IsNClique.cliqueNum_eq_of_cliqueFree {s : Finset V}
    {n : ℕ} (hs : G.IsNClique n s) (h : G.CliqueFree (n + 1)) :
    ω G = n := by
  refine h.cliqueNum_le.antisymm ?_
  refine le_csSup (s := {k | ∃ t, G.IsNClique k t}) ?_ ⟨s, hs⟩
  refine ⟨n, ?_⟩
  rintro k ⟨t, ht⟩
  by_contra! hk
  exact h.mono hk _ ht
```

For example, the 5-cycle has clique number 2: `{0, 1}` is an edge and the cycle is triangle-free.

```
example : ω C[5] = 2 := by
  have h2 : C[5].IsNClique 2 {0, 1} := by decide
  exact h2.cliqueNum_eq_of_cliqueFree (by decide)
```

Graph theorists distinguish a *maximum* clique (one of the largest possible size) from a *maximal* clique (one that is not contained in any larger clique). Every maximum clique is maximal but not conversely: an edge hanging off a triangle is a maximal clique of size 2. Mathlib expresses maximality with the general order-theoretic predicate `Maximal G.IsClique s`, while `IsMaximumClique` (for finite graphs) says that a finset is a clique of size $\omega(G)$.

```
#check IsMaximumClique.isMaximalClique -- G.IsMaximumClique s → Maximal G.IsClique ↑s
#check maximumClique_exists            -- ∃ s, G.IsMaximumClique s

end more_cliques
```

6.6 Independent Sets

An *independent set* in a graph is a set of vertices that are pairwise non-adjacent. This is defined analogously to `IsClique`.

```
section indset
variable {V : Type*} {G : SimpleGraph V}
#source IsIndepSet -- abbrev IsIndepSet (s : Set α) : Prop :=
                   --   s.Pairwise (fun v w ↦ ¬G.Adj v w)
```

Since a `SimpleGraph` is loopless, `s : Set V` is an independent set iff every pair of vertices in `s` is non-adjacent (irrespective of whether or not they are distinct).

```
lemma isIndepSet_iff_forall_not_adj {s : Set V} :
    G.IsIndepSet s ↔ ∀ ⦃v w⦄, v ∈ s → w ∈ s → ¬G.Adj v w := by
  rw [isIndepSet_iff, Set.Pairwise]
  grind [SimpleGraph.irrefl]
```

If `s` is an independent set in `G` then its complement meets every edge of `G`. (In other words, the complement of an independent set is a *vertex cover*.)

```
#check IsIndepSet.nonempty_mem_compl_mem_edge
```

The independent sets of `G` are exactly the cliques of its complement G^c, so each notion for cliques has an independent-set counterpart. The analogues of `IsNClique` and `CliqueFree` are `IsNIndepSet` and `IndepSetFree`.

```
#source IsNIndepSet  -- structure IsNIndepSet (n : ℕ) (s : Finset α) : Prop where
                     --   isIndepSet : G.IsIndepSet s
                     --   card_eq : s.card = n

#source IndepSetFree -- def IndepSetFree (n : ℕ) : Prop := ∀ t, ¬G.IsNIndepSet n t
```

Mathlib's simp lemmas `isNClique_compl` and `cliqueFree_compl` translate between the two. Mathlib has a `Decidable` instance for `IsNIndepSet` but not for `IndepSetFree`, so to check the latter with `decide` we first rewrite it as a statement about cliques in G^c, where our `decidableCliqueFree` instance applies.

```
#check isNClique_compl  -- Gᶜ.IsNClique n s ↔ G.IsNIndepSet n s
#check cliqueFree_compl -- Gᶜ.CliqueFree n ↔ G.IndepSetFree n
```

There is an analogous definition of the *independence number* of a graph, as the size of the largest independent set.

```
#source indepNum -- noncomputable def indepNum (G : SimpleGraph α) : ℕ :=
                 --   sSup {n | ∃ s, G.IsNIndepSet n s}
```

We introduce notation for the independence number that looks familiar, but *beware*: `α` is such a common name for a type variable that using it as notation would clash, so we use the almost indistinguishable APL symbol `α` instead. This guide's VS Code settings let you type it as `\apla`; elsewhere you can always write `G.indepNum`.

The maximum number of vertices of an independent set in a graph `G`. Denoted `α G`. (The symbol is the APL alpha `α`: Unicode `U+237A`, typed as `\apla` rather than the Greek `α`, to avoid clashes with generic type variables.)

```
scoped notation "α" => SimpleGraph.indepNum
```

The independence number of `G` is the clique number of `Gᶜ`, and the analogues of the clique-number lemmas above hold, such as `IsIndepSet.card_le_indepNum` and `exists_isNIndepSet_indepNum`.

```
#check cliqueNum_compl -- ω Gᶜ = α G
#check IsIndepSet.card_le_indepNum
#check exists_isNIndepSet_indepNum
```

The analogue of `IsNClique.cliqueNum_eq_of_cliqueFree` for independent sets: if `s` is an `n`-independent set in `G` and `G` has no `(n + 1)`-independent set then `α G = n`. Rather than repeating the argument we pass to the complement.

```
theorem IsNIndepSet.indepNum_eq_of_indepSetFree {s : Finset V}
    {n : ℕ} (hs : G.IsNIndepSet n s) (h : G.IndepSetFree (n + 1)) :
    α G = n := by
  rw [← cliqueNum_compl]
  exact (G.isNClique_compl.mpr hs).cliqueNum_eq_of_cliqueFree
    fun t ht => h t (G.isNClique_compl.mp ht)
```

Any finite edgeless graph has independence number equal to the number of vertices. This is the complement of `cliqueNum_top`, since `⊥ᶜ = ⊤`. (The proof still works for `V` infinite, so we omit the finiteness assumption.)

```
@[simp]
lemma indepNum_bot : α (⊥ : SimpleGraph V) = Nat.card V := by
  simp [← cliqueNum_compl]
```

Triangles and Independent Sets

In a triangle-free graph the neighbourhood of any vertex `v` is an independent set, since two adjacent neighbours of `v` would form a triangle with `v`.

```
#check isIndepSet_neighborSet_of_triangleFree
```

In a finite triangle-free graph every vertex has degree at most `α G`.

```
lemma degree_le_indepNum_of_triangleFree [Finite V] [G.LocallyFinite]
    (h : G.CliqueFree 3) (v : V) : G.degree v ≤ α G := by
  rw [← card_neighborFinset_eq_degree]
  exact IsIndepSet.card_le_indepNum
    (by simpa using G.isIndepSet_neighborSet_of_triangleFree h v)
```

So in a triangle-free graph with no large independent sets every vertex has small degree. Applying this to both a graph and its complement is the idea behind the simplest case of Ramsey's theorem

[Ramsey 1930]. The *Ramsey number* `R(3, 3)` is the least `n` such that every graph on `n` vertices contains a triangle or an independent set of size `3`. Mathlib has no graph Ramsey numbers, but we can still prove the classical value `R(3, 3) = 6`.

The 5-cycle shows that `R(3, 3) > 5`: it is triangle-free and has no independent set of size `3`. As explained above, we rewrite `IndepSetFree` in terms of the complement before calling `decide`.

```
example : C[5].CliqueFree 3 ∧ C[5].IndepSetFree 3 :=
  ⟨by decide, by rw [← cliqueFree_compl]; decide⟩

example : α C[5] = 2 := by
  have h2 : C[5].IsNIndepSet 2 {0, 2} := by decide
  exact h2.indepNum_eq_of_indepSetFree (by rw [← cliqueFree_compl]; decide)
```

The upper bound `R(3, 3) ≤ 6` is left as an exercise: applying `degree_le_indepNum_of_triangleFree` to both G and G^{c} does most of the work.

The 5-cycle also shows that the product of the clique and independence numbers can be smaller than the number of vertices: `ω C[5] * α C[5] = 4 < 5`. Contrast this with the bound $|V| \le \chi\, G * \alpha\, G$ that we will prove in the next chapter. On the other hand the sum satisfies `ω G + α G ≤ |V| + 1` for every finite graph, since a clique and an independent set share at most one vertex (see the exercises).

Independence Number of the Infinite Path on $\mathbb{N}$

The infinite path graph on $\mathbb{N}$ (`pathNat`) has unbounded independent sets, so what is its independence number? Due to the `sSup` definition, it is `0`!

Adjacent vertices of `pathNat` differ by one, so they have different parities. Hence any set of natural numbers that all have the same parity is independent.

```
lemma isIndepSet_pathNat_of_mod_two_eq {s : Set ℕ}
    (h : ∀ ⦃a b⦄, a ∈ s → b ∈ s → a % 2 = b % 2) : pathNat.IsIndepSet s := by
  rw [isIndepSet_iff_forall_not_adj]
  rintro a b ha hb (rfl | rfl) <;> grind
```

In particular, for every `n` the first `n` even numbers form an `n`-independent set. So the set of sizes of independent sets is not bounded above, and `Nat.sSup_of_not_bddAbove` gives `α pathNat = 0`.

```
lemma indepNum_pathNat : α pathNat = 0 := by
  apply Nat.sSup_of_not_bddAbove
  rintro ⟨N, hN⟩
  have hs : pathNat.IsNIndepSet (N + 1)
      ((Finset.range (N + 1)).map ⟨(2 * ·), mul_right_injective₀ two_ne_zero⟩) := by
    refine ⟨isIndepSet_pathNat_of_mod_two_eq ?_, by simp⟩
    grind [Function.Embedding.coeFn_mk]
  exact N.not_succ_le_self (hN ⟨_, hs⟩)

end indset
```

Independence Number of the Kneser Graph*

Recall that the Kneser graph `kneserGraph β k` is the graph whose vertices are the `k`-element subsets of a type `β`, and in which two vertices are adjacent iff the subsets are disjoint and distinct.

When β is `Fin n`, we have the notation `Kn[n, k]`.

So an independent set in `Kn[n, k]` is a family of `k`-element subsets of `Fin n` in which no two members are disjoint: an *intersecting family*. The Erdős–Ko–Rado theorem [Erdős–Ko–Rado 1961] says that if `2 * k ≤ n` then an intersecting family of `k`-element subsets of an `n`-element set has at most `(n - 1).choose (k - 1)` members. This is in Mathlib, and it is essentially the computation of the independence number of `Kn[n, k]`.

```
#check Finset.erdos_ko_rado
```

Mathlib states the theorem for a finite family of finsets `𝒜 : Finset (Finset (Fin n))` with three hypotheses:

- `Set.Intersecting`: no two members of `𝒜` are disjoint. Note that this quantifies over *all* pairs `a, b ∈ 𝒜`, including `a = b`. So a family containing `∅` is never intersecting, since `∅` is disjoint from itself.
- `Set.Sized r`: every member of `𝒜` has exactly `r` elements.
- `r ≤ n / 2`: here `/` is natural-number (floor) division, so this is equivalent to `2 * r ≤ n`.

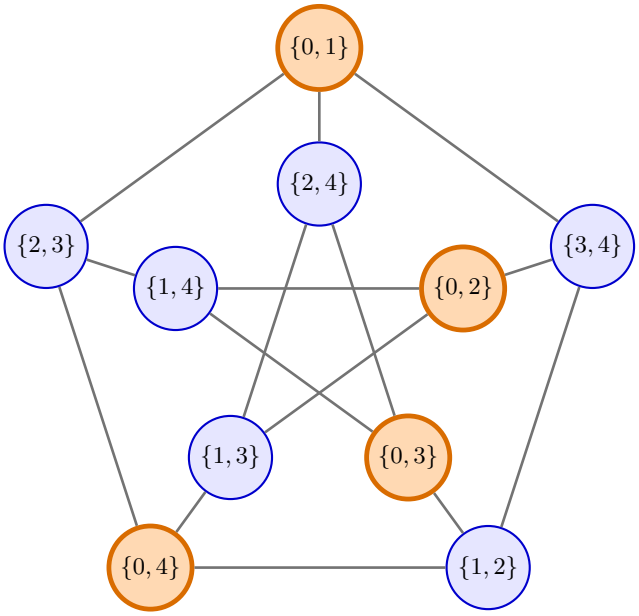
```
section kneser
open Finset
variable {k : ℕ} {β : Type*}
```

The vertices of `kneserGraph β k` have type `{s : Finset β // #s = k}` which is a bit of a pain to write, so we introduce a local notation for it.

The type of `k`-element subsets of `β`: `{s : Finset β // #s = k}`, the vertices of `kneserGraph β k`.

```
local notation "β_k" => {s : Finset β // #s = k}
```

For any `i : β`, the set of all vertices containing `i` (the *star* at `i`) is an independent set in `kneserGraph β k`: any two such vertices share `i`, so they are not disjoint. When `1 ≤ k` and `2 * k ≤ #β` this is a largest independent set: we will see below that the Erdős–Ko–Rado theorem gives the matching upper bound. (For larger `k` the graph has no edges at all.)



The star at `0`, a largest independent set in the `petersenGraph` (`Kn[5, 2]`)

```
lemma isIndepSet_kneserGraph_of_mem (i : β) :
    (kneserGraph β k).IsIndepSet {b | i ∈ b.val} := by
  intro a ha b hb hne hf
  rw [kneserGraph_adj, ne_eq] at hf
  exact Finset.disjoint_left.mp hf.1 ha hb
```

We now want to start using the `Finset.powersetCard` API to compute sizes of families of k-element subsets of a fintype β. Due to our choice of vertex type for the Kneser graph we need the following bookkeeping lemma which has zero mathematical content.

Given any predicate `p : Finset β → Prop`, the number of k-element subsets of β satisfying `p` can be counted in two different but equivalent ways: as vertices of the Kneser graph, or as members of `Finset.powersetCard k univ`. (The bijection between the two is `Subtype.val`.)

```
lemma card_filter_val_eq_card_filter_powersetCard [Fintype β] (p : Finset β → Prop)
    [DecidablePred p] : #{b ∈ (Finset.univ : Finset βₖ) | p b.val} =
    #(((Finset.univ : Finset β).powersetCard k).filter p) := by
  classical
  refine card_bij (fun b _ => b.val) (by simp_all)
    (by simp [Subtype.ext_iff]) ?_
  simp only [mem_filter, mem_powersetCard]
  exact fun t ht => ⟨⟨t, ht.1.2⟩, by simpa using ht.2, rfl⟩
```

To compute the independence number we need the size of the *star* at i: the number of k-element subsets of a finite type β containing i is `(Fintype.card β - 1).choose (k - 1)`. We get this from Mathlib's `card_filter_powersetCard_subset`, which counts the k-element subsets containing a given finset `s` (here `s = {i}`).

The hypothesis `[NeZero k]` is needed. For `k = 0` the only `0`-element subset of β is $\varnothing$, which does not contain i, so the count is `0`. But natural-number subtraction gives `k - 1 = 0`, so the formula evaluates to `1`. (`NeZero k` is a type class recording that $k \neq 0$; as an instance argument it can be found automatically, for example when k is `n + 1`.)

```
lemma card_filter_mem_eq_choose_sub_one [Fintype β] [DecidableEq β]
    (i : β) [NeZero k] : #{b ∈ (Finset.univ : Finset βₖ) | i ∈ b.val} =
    (Fintype.card β - 1).choose (k - 1) := by
  rw [card_filter_val_eq_card_filter_powersetCard (i ∈ ·)]
  simpa [singleton_subset_iff] using card_filter_powersetCard_subset {i} univ k
    (subset_univ _) (by simpa using NeZero.one_le)
```

As well as stars, the Kneser graph also contains independent sets determined by size: if we take less than `2 * k` elements of β then the set of all k-element subsets of this set is an independent set in `kneserGraph β k`.

```
lemma isIndepSet_kneserGraph_of_card_lt {s : Finset β} (h : #s < 2 * k) :
    (kneserGraph β k).IsIndepSet {b | b.val ⊆ s} := by
  classical
  intro a ha b hb hne hf
  rw [kneserGraph_adj, ne_eq] at hf
  have hcard : (a.val ∪ b.val).card = 2 * k := by
    rw [card_union_of_disjoint hf.1, a.2, b.2, two_mul]
  grind [card_le_card (union_subset ha hb)]
```

Taking `s = univ` in `isIndepSet_kneserGraph_of_card_lt`: if β has less than `2 * k` elements then `kneserGraph β k` is the edgeless graph, since any distinct pair of k-element subsets of β must intersect.

```
lemma kneserGraph_eq_bot_of_card_lt [Finite β] (h : Nat.card β < 2 * k) :
    kneserGraph β k = ⊥ := by
  have := Fintype.ofFinite β
  rw [Nat.card_eq_fintype_card] at h
  have := isIndepSet_kneserGraph_of_card_lt (k := k) (s := Finset.univ)
    (by simpa using h)
  ext u v
  simpa using fun huv => this (by simp) (by simp) huv.2 huv
```

To apply the Erdős–Ko–Rado theorem we must transport an independent set of `kneserGraph β k`, which is a finset of subtypes, to a family of `Finset β`s. The image under `Subtype.val` of an independent set is an intersecting family, provided $k \neq 0$. Since Mathlib's `Intersecting` also requires each member to meet itself, this is where `[NeZero k]` is needed: the only `0`-element subset is $\varnothing$.

```
lemma kneserGraph_isIndepSet_image_intersecting [DecidableEq β] [NeZero k]
    {t : Finset βₖ} (ht : (kneserGraph β k).IsIndepSet (t : Set _)) :
    (t.image Subtype.val : Set (Finset β)).Intersecting := by
  simp only [Finset.coe_image]
  rintro _ ⟨x, hx, rfl⟩ _ ⟨y, hy, rfl⟩ hdisj
  obtain rfl | hne := eq_or_ne x y
  · -- we use the fact that k ≠ 0 to prove that x is not disjoint from itself
    exact NeZero.ne k (by simpa [disjoint_self.mp hdisj] using x.prop.symm)
  · exact ht hx hy hne ⟨hdisj, hne⟩
```

The image under `Subtype.val` of any finset of k-element subsets of β is a family of k-element subsets of β.

```
lemma sized_image_val [DecidableEq β] (t : Finset βₖ) :
    (t.image Subtype.val : Set (Finset β)).Sized k := by
```

```
  simp_all [Set.Sized]
```

Since the version of the Erdős–Ko–Rado theorem in Mathlib is stated in terms of intersecting families of k-element subsets of `Fin n` we now take `β := Fin n` and work in `Kn[n, k]`.

If `k ≤ n / 2` and `k ≠ 0` then the independence number of `Kn[n, k]` is achieved by the star at `0`, with the upper bound given by the Erdős–Ko–Rado theorem. If instead `n / 2 < k` then the Kneser graph is the edgeless graph and its independence number is the size of its vertex set.

```
theorem indepNum_kneserGraph {n : ℕ} [NeZero k] :
    α Kn[n, k] = if k ≤ n / 2 then (n - 1).choose (k - 1) else n.choose k := by
  by_cases hk : k ≤ n / 2
  · rw [ite_eq_left hk]
    have : NeZero n := NeZero.of_pos (by grind [NeZero.pos k])
    classical
    apply IsNIndepSet.indepNum_eq_of_indepSetFree
      (s := {b ∈ (Finset.univ : Finset {s : Finset (Fin n) // #s = k}) |
        (0 : Fin n) ∈ b.val})
    · constructor
      · simp [isIndepSet_kneserGraph_of_mem (0 : Fin n)]
      · simp [card_filter_mem_eq_choose_sub_one]
    · intro t ht
      have hAcard : (t.image Subtype.val).card = t.card :=
        Finset.card_image_of_injective _ Subtype.val_injective
      have hInt := kneserGraph_isIndepSet_image_intersecting ht.isIndepSet
      have hSized := sized_image_val t
      have hbound := Finset.erdos_ko_rado hInt hSized hk
      grind [ht.card_eq]
  · have hn : Nat.card (Fin n) < 2 * k := by grind [Nat.card_fin]
    simp [hk, kneserGraph_eq_bot_of_card_lt hn]
```

For example, the Petersen graph `Kn[5, 2]` has independence number `(5 - 1).choose (2 - 1) = 4`. Combined with the bound `|V| ≤ χ G * α G` from the next chapter, this shows that the Petersen graph is not 2-colourable, since it has `10 > 2 * 4` vertices.

```
example : α Kn[5, 2] = 4 := by simp [indepNum_kneserGraph]

end kneser
```

6.7 Clique and Independence Number of the Mycielskian

Recall from Mycielskian (p. 38) Mycielski's two facts: if `G` is triangle-free then so is `mμ(G)`, and if `G` has chromatic number `k` then `mμ(G)` has chromatic number `k + 1`. In this section we prove the first fact, and more generally that the Mycielskian preserves the clique number. The second fact is `chromaticNumber_mycielskian` in the next chapter. We finish by computing the independence number of `mμ(G)`.

Triangle-Free Mycielskians

We start with the promised proof that the Mycielskian of a triangle-free graph is triangle-free, before moving on to the more general result that the Mycielskian preserves the clique number (of any finite graph with at least one edge).

```
section mycielskian_cliquenum
variable {V : Type*} {G : SimpleGraph V}

theorem CliqueFree.mycielskian (h : G.CliqueFree 3) : mμ(G).CliqueFree 3 := by
  intro t ht
  -- classical provides the DecidableEq V instance required in the proof
  classical
  rw [is3Clique_iff] at ht
  obtain ⟨a, b, c, hab, hac, hbc, _⟩ := ht
  rcases a with _ | (a₁ | a₂) <;>
  rcases b with _ | (b₁ | b₂) <;>
  rcases c with _ | (c₁ | c₂) <;>
  -- 27 goals, but 23 are easily dispatched by simp only since they either assert
  -- the existence of loops; edges inside the "right" copy of V; or edges from the
  -- "left" copy of V to none, all of which do not exist in mμ(G).
  simp only [mycielskian_adj] at hab hac hbc <;>
    -- only 4 goals remain!
    -- In each goal the local context contains three adjacency hypotheses
    -- that assert the existence of a triangle in G, contradicting our
    -- assumption that G is triangle-free.
    exact h _ ◁ is3Clique_triple_iff.mpr ⟨hab, hac, hbc⟩
```

Hence the iterated Mycielskian of any triangle-free graph is also triangle-free.

```
theorem CliqueFree.iterMycielskian (n : ℕ) (h : G.CliqueFree 3) :
    mμ^n(G).CliqueFree 3 := by
  induction n with
  | zero => exact h
  | succ _ ih => exact ih.mycielskian
```

In particular every iterated Mycielskian of the single edge `K[2]` is triangle-free. Up to isomorphism this sequence begins `K[2]`, `C[5]`, `grotzschGraph`.

```
theorem cliqueFree_three_iterMycielskian_completeGraph_two (n : ℕ) :
    mμ^n(K[2]).CliqueFree 3 :=
  CliqueFree.iterMycielskian n (by decide)
```

Clique Number of the Mycielskian

We now turn to the clique number of the Mycielskian.

On paper this is easy: any clique in `G` also appears in `mμ(G)` via the obvious embedding. Conversely any clique of size at least 3 in `mμ(G)` cannot use the vertex none since none is only adjacent to the vertices in the right copy of `V`, and the right copy of `V` is an independent set. Moreover no clique in `mμ(G)` can contain v^l and v^r for any $v \in V$. (This would imply `G.Adj v v`, which is impossible.) Hence any clique of size at least 3 in `mμ(G)` can be mapped to a clique of the same size in `G` using the projection we will define below.

We first prove that the Mycielskian embedding mapping `G` into its Mycielskian sends `k`-cliques in `G` to `k`-cliques in `mμ(G)`.

```
lemma IsNClique.mycielskian {k : ℕ} {s : Finset V} (h : G.IsNClique k s) :
    mμ(G).IsNClique k (s.map (Embedding.mycielskianLeft G).toEmbedding) := by
  classical
```

```
  simpa [Finset.map_eq_image] using h.image (Embedding.mycielskianLeft G).toHom
```

Next we want to show that the vertex none cannot belong to any k-clique in $\mathrm{M}\mu(G)$ when $3 \le k$, because none is only adjacent to right vertices, and no two right vertices are adjacent.

```
lemma mycielskian_none_notMem_clique {k : ℕ} (h3 : 3 ≤ k)
    {t : Finset (Option (V ⊕ V))} (ht : Mμ(G).IsNClique k t) : none ∉ t := by
  classical -- needed to use Finset.erase below since it requires DecidableEq V
  intro hmem
  -- since t is a k-clique with 3 ≤ k, t.erase none has at least 2 elements.
  have hcard : 2 ≤ (t.erase none).card := by
    grind [Finset.card_erase_of_mem hmem, ht.card_eq]
  -- so we can find two distinct vertices a, b in t.erase none, and they are
  -- adjacent since t is a clique.
  obtain ⟨a, ha, b, hb, hab⟩ := Finset.one_lt_card.mp hcard
  -- both a and b are adjacent to none in mμ(G)
  have h_na := ht.isClique hmem (Finset.mem_of_mem_erase ha) (by grind)
  have h_nb := ht.isClique hmem (Finset.mem_of_mem_erase hb) (by grind)
  -- so they must be "right" vertices
  rcases a with _ | ⟨va | va⟩ <;> rcases b with _ | ⟨vb | vb⟩
    <;> simp only [mycielskian_adj] at h_na h_nb
  -- but no two "right" vertices are adjacent, so we have a contradiction.
  exact ht.isClique (Finset.mem_of_mem_erase ha) (Finset.mem_of_mem_erase hb) hab
```

If s is a clique in the Mycielskian then s cannot contain both v^{l} and v^{r} for any $\mathrm{v} \in V$ (since `Mμ(G).Adj` v^{l} v^{r} would imply `G.Adj v v`, which is impossible). Moreover we just proved that any k-clique with $3 \le k$ cannot contain none. So consider the projection map from the vertices of $\mathrm{M}\mu(G)$ to the vertices of G that "forgets" which side (left or right) a vertex came from, and sends none to an arbitrary vertex of V. This map preserves adjacency, and it is injective on any clique in $\mathrm{M}\mu(G)$ of size at least 3.

Projection from the vertex set of $\mathrm{M}\mu(G)$ to that of G (preserves k-cliques for $k \ge 3$).

```
@[simp]
def mycielskianProj (v₀ : V) : Option (V ⊕ V) → V
  | none => v₀
  | v^l => v
  | v^r => v
```

Any k-clique in $\mathrm{M}\mu(G)$ (with $3 \le k$) maps to a k-clique in G under `mycielskianProj`. Such a clique doesn't contain none, and every edge of $\mathrm{M}\mu(G)$ not involving none projects to an edge of G.

```
lemma IsNClique.of_mycielskian [DecidableEq V] (v₀ : V) {k : ℕ} (h3 : 3 ≤ k)
    {t : Finset (Option (V ⊕ V))} (ht : Mμ(G).IsNClique k t) :
    G.IsNClique k (t.image (mycielskianProj v₀)) := by
  -- any clique of size at least 3 in mμ(G) cannot contain none
  have hnone := mycielskian_none_notMem_clique h3 ht
  -- so it suffices to show that the projection preserves adjacency on t
  refine ht.image_of_adj fun x hx y hy hadj => ?_
  -- first | t₁ | t₂ | … tries the tactics in turn, until one succeeds
  rcases x with _ | ⟨_ | _⟩ <;> rcases y with _ | ⟨_ | _⟩ <;> -- 9 goals
    first | grind | exact hadj | contradiction
```

This generalizes `CliqueFree.mycielskian`: for every $r \ge 3$, the Mycielskian of an r-clique-free

graph is r-clique-free. To apply `IsNClique.of_mycielskian` we need a vertex $v_0 : V$, and the clique itself provides one: it has a vertex, which is not `none`, so it is v^l or v^r for some $v \in V$. The bound $3 \le r$ cannot be dropped. The empty graph on one vertex is 2-clique-free, but its Mycielskian has an edge from `none` to the right copy of the vertex.

```
theorem CliqueFree.mycielskian_of_three_le {r : ℕ} (hr : 3 ≤ r)
    (h : G.CliqueFree r) : mμ(G).CliqueFree r := by
  classical -- to infer DecidableEq V
  intro t ht
  -- an r-clique with 3 ≤ r has a vertex x
  obtain ⟨x, hx⟩ : t.Nonempty := by grind [Finset.card_pos, ht.card_eq]
  rcases x with _ | ⟨v | v⟩
  · exact mycielskian_none_notMem_clique hr ht hx
  -- if x is v^l or v^r then t projects to an r-clique in G
  all_goals exact h _ (ht.of_mycielskian v hr)
```

Any finite graph with at least one edge has the same clique number as its Mycielskian. (For the edgeless graph $\bot$ the answer depends on whether V is empty; see the exercises.)

```
theorem cliqueNum_mycielskian_of_ne_bot [Finite V] (h : G ≠ ⊥) :
    ω mμ(G) = ω G := by
  classical -- to infer DecidableEq V
  -- G ≠ ⊥ gives an edge, i.e. a 2-clique, so 2 ≤ ω G
  obtain ⟨a, b, hab⟩ := ne_bot_iff_exists_adj.mp h
  have h2 : 2 ≤ ω G := (G.is2Clique_pair_iff.mpr hab).le_cliqueNum
  -- G is (ω G + 1)-clique-free, so mμ(G) is too since 3 ≤ ω G + 1
  have hfree : G.CliqueFree (ω G + 1) := fun _ hs => by grind [hs.le_cliqueNum]
  -- and a maximum clique of G maps to an ω G-clique in mμ(G)
  obtain ⟨s, hs⟩ := G.exists_isNClique_cliqueNum
  exact hs.mycielskian.cliqueNum_eq_of_cliqueFree
    (hfree.mycielskian_of_three_le (by grind))
```

Applying this repeatedly to `K[2]`, which has clique number 2 by `cliqueNum_top`, we see that every iterated Mycielskian of `K[2]` has clique number 2. The Mycielskian step needs `mμ^n(K[2]) ≠ ⊥`, which follows from the induction hypothesis, since a graph with clique number 2 has an edge.

```
theorem cliqueNum_iterMycielskian_completeGraph_two (n : ℕ) :
    ω mμ^n(K[2]) = 2 := by
  induction n with
  | zero => simp
  | succ n ih =>
    -- ω = 2 gives a 2-clique, i.e. an edge, so mμ^n(K[2]) ≠ ⊥
    have hne : mμ^n(K[2]) ≠ ⊥ := fun h => by
      grind [(cliqueFree_two.2 h).cliqueNum_le]
    rwa [iterMycielskian_succ, cliqueNum_mycielskian_of_ne_bot hne]

end mycielskian_cliquenum
```

In Colouring Mycielskians (p. 166) we show that `mμ^n(K[2])` has chromatic number $n + 2$. So these graphs all have clique number 2 but have arbitrarily large chromatic number.

Independence Number of the Mycielskian*

We now derive a formula for the independence number of the Mycielskian of a finite graph. To state it we need the *open neighbourhood* of a set of vertices `s` in `G`: the set of vertices adjacent to at least one vertex of `s`. It is called *open* to distinguish it from the *closed neighbourhood*, which also contains `s` itself. Mathlib has no definition of the neighbourhood of a set of vertices, so we make our own.

```
section open_neighborhoods
variable {V : Type*}
```

The open neighbourhood of a set of vertices in a graph `G` is the union of the neighbourhoods of each vertex in the set.

```
def openNeighborhoodSet (G : SimpleGraph V) (s : Set V) : Set V :=
  ⋃ v ∈ s, G.neighborSet v
```

The bounded union `⋃ v ∈ s, _` is notation for a nested `Set.iUnion`, indexed first by `v : V` and then by proofs of `v ∈ s`.

```
#check Set.iUnion
```

Membership of the open neighbourhood and of its complement (the vertices with no neighbour in `s`) unfold as expected.

```
@[simp]
lemma mem_openNeighborhoodSet {G : SimpleGraph V} {s : Set V} {x : V} :
    x ∈ G.openNeighborhoodSet s ↔ ∃ a ∈ s, G.Adj a x := by
  simp [openNeighborhoodSet]

lemma mem_compl_openNeighborhoodSet {G : SimpleGraph V} {s : Set V} {x : V} :
    x ∈ (G.openNeighborhoodSet s)ᶜ ↔ ∀ a ∈ s, ¬G.Adj a x := by
  simp
```

We also need a `Finset` version of the open neighbourhood of a finset `s`. This only needs each vertex of `s` to have a `neighborFinset`, but for simplicity we assume the graph is locally finite. The bounded union `⋃ v ∈ s, _` that we used to define `openNeighborhoodSet` has a direct `Finset` counterpart: `s.biUnion f` is the union of the finsets `f v` over `v ∈ s`. Note that it requires `DecidableEq` on the target type, since building a `Finset` union means deciding which elements are duplicates.

```
#check Finset.biUnion -- [DecidableEq β] (s : Finset α) (t : α → Finset β) : Finset β
#check Finset.coe_biUnion -- ↑(s.biUnion t) = ⋃ x ∈ s, ↑(t x)
```

The open neighbourhood of a finset `s` in a graph `G` is the finset of vertices that are adjacent to at least one vertex in `s`.

```
def openNeighborhoodFinset (G : SimpleGraph V) [G.LocallyFinite] [DecidableEq V]
    (s : Finset V) : Finset V := s.biUnion (fun v ↦ G.neighborFinset v)
```

It will be convenient to write `N(s)` for the open neighbourhood of a finset `s`, as is standard in graph theory, so that (in a finite graph) the vertices with no neighbours in `s` form the finset `N(s)ᶜ`. We want to do this with a fixed `G` here so we use `local notation` whose expansion mentions the ambient section variable `G` explicitly, so `N(s)` always refers to the current graph `G`.

```
variable (G : SimpleGraph V)
@[inherit_doc]
local notation:max "N(" s ")" => openNeighborhoodFinset G s
```

Membership of `N(s)` and its complement, and the coercion from `N(s)` to a `Set`, unfold as expected. Here `variable {G}`, with no type, does not declare a new variable: it changes the existing section variable `G` from explicit to implicit, so the lemmas below infer `G` from their other arguments. (A `variable (G)` would change it back.)

```
variable {G}
@[simp]
lemma mem_openNeighborhoodFinset [G.LocallyFinite] [DecidableEq V] {s : Finset V}
    {x : V} : x ∈ N(s) ↔ ∃ a ∈ s, G.Adj a x := by
  simp [openNeighborhoodFinset]

@[simp]
lemma coe_openNeighborhoodFinset [G.LocallyFinite] [DecidableEq V] (s : Finset V) :
    (N(s) : Set V) = G.openNeighborhoodSet ↑s := by
  ext x
  simp

lemma mem_compl_openNeighborhoodFinset [Fintype V] [G.LocallyFinite] [DecidableEq V]
    {s : Finset V} {x : V} : x ∈ N(s)ᶜ ↔ ∀ a ∈ s, ¬G.Adj a x := by
  simp
```

Now for independent sets in the Mycielskian. Any independent set `s` in `G` gives rise to an independent set in `mμ(G)` consisting of the left copy of `s` together with the right copy of every vertex having no neighbour in `s`, i.e. `s^l ∪ (N(s)ᶜ)^r`.

```
lemma IsIndepSet.mycielskian {s : Set V} (h : G.IsIndepSet s) :
    mμ(G).IsIndepSet (s.image (·^l) ∪ (G.openNeighborhoodSet s)ᶜ.image (·^r)) := by
  rintro a ha b hb hne hf
  -- a and b are each either a left copy of a vertex of s or a right copy
  -- of a vertex with no neighbour in s; we split on both. The right-right case
  -- has no edge, so simp closes it, leaving the three cases below.
  obtain (⟨x, hx, rfl⟩ | ⟨x, hx, rfl⟩) := ha
    <;> obtain (⟨y, hy, rfl⟩ | ⟨y, hy, rfl⟩) := hb
      <;> simp only [mycielskian_adj] at hf
  -- Now we have G.Adj x y and at least one of x or y is in s
  · exact h hx hy hf.ne hf -- both in s^l: contradicts independence of s
  -- y has no neighbour in s, yet x ∈ s and G.Adj x y
  · exact mem_compl_openNeighborhoodSet.mp hy _ hx hf
  -- symmetric to the previous case
  · exact mem_compl_openNeighborhoodSet.mp hx _ hy hf.symm
```

The following simple lemma will help us to compute the size of independent sets in `mμ(G)`. The left and right images of any two finsets are disjoint (they lie in different summands) and both maps are injective, so the union of the images has size `#s + #t`.

```
open Finset mycielskian

lemma card_image_left_union_image_right [DecidableEq V] (s t : Finset V) :
    #(s.image (·^l) ∪ t.image (·^r)) = #s + #t := by
```

```
  rw [Finset.card_union_of_disjoint,
    Finset.card_image_of_injective _ left_injective,
    Finset.card_image_of_injective _ right_injective]
  exact Finset.disjoint_left.mpr fun a ha hb => by grind
```

We will show that the independence number of $m\mu(G)$ is the maximum, over all independent sets s of G, of $\#s + \#N(s)^c$. The lower bound comes straight from `IsIndepSet.mycielskian`.

```
lemma le_indepNum_mycielskian [Fintype V] [DecidableEq V] [G.LocallyFinite]
    {s : Finset V} (h : G.IsIndepSet ↑s) : #s + #N(s)ᶜ ≤ α mμ(G) := by
  have hSindep : mμ(G).IsIndepSet ↑(s.image (·ˡ) ∪ N(s)ᶜ.image (·ʳ)) := by
    simpa using h.mycielskian
  rw [← card_image_left_union_image_right s N(s)ᶜ]
  exact hSindep.card_le_indepNum
```

The left preimage of an independent set of $m\mu(G)$ is an independent set of G (the left copy is a copy of G).

```
lemma isIndepSet_preimage_left {t : Set (Option (V ⊕ V))}
    (h : mμ(G).IsIndepSet t) : G.IsIndepSet ((·ˡ) ⁻¹' t) := by
  intro x hx y hy hne
  simpa using h hx hy (by simp [hne])
```

The `Finset` version of the previous lemma uses `Finset.preimage`. Unlike `Set.preimage`, it takes an extra argument: a proof that the map is injective on the preimage. This guarantees that the preimage of a finset is finite. Here the map $(\cdot^l)$ is injective, so we can always supply `left_injective.injOn`.

```
#check Finset.preimage -- (s : Finset β) (f : α → β) (hf : Set.InjOn f (f ⁻¹' ↑s))

lemma isIndepSet_preimage_left_finset {T : Finset (Option (V ⊕ V))}
    (hT : mμ(G).IsIndepSet ↑T) :
    G.IsIndepSet ↑(T.preimage (·ˡ) left_injective.injOn) := by
  simpa using G.isIndepSet_preimage_left hT
```

An independent set s has no neighbours in itself, i.e. $s \subseteq N(s)^c$: adjacent vertices are distinct, so if `a, z ∈ s` were adjacent this would contradict independence.

```
lemma IsIndepSet.subset_compl_openNeighborhoodFinset [Fintype V] [DecidableEq V]
    [G.LocallyFinite] {s : Finset V}
    (h : G.IsIndepSet ↑s) : s ⊆ N(s)ᶜ :=
  fun z hz => by simpa using fun a ha hadj => h ha hz hadj.ne hadj
```

In a nonempty finite graph the finset of non-neighbours of an independent set s is nonempty: it contains s itself, unless s is empty in which case it is all of V.

```
lemma IsIndepSet.compl_openNeighborhoodFinset_nonempty [Fintype V] [Nonempty V]
    [DecidableEq V] [G.LocallyFinite] {s : Finset V}
    (h : G.IsIndepSet ↑s) : N(s)ᶜ.Nonempty := by
  rcases s.eq_empty_or_nonempty with rfl | ⟨z, hz⟩
  · simp [openNeighborhoodFinset]
  · exact ⟨z, h.subset_compl_openNeighborhoodFinset hz⟩
```

If an independent set T of $\mathrm{m}\mu(G)$ contains none then, since none is adjacent to every right vertex, T contains no right vertices. Hence T is contained in none together with the left copies of its left part.

```
lemma mycielskian_indepSet_subset_insert_none [DecidableEq V]
    {T : Finset (Option (V ⊕ V))} (hT : mμ(G).IsIndepSet ↑T) (hnone : none ∈ T) :
    T ⊆ insert none ((T.preimage (·ˡ) left_injective.injOn).image (·ˡ)) := by
  intro v hv
  rcases v with _ | (x | x)
  · exact mem_insert_self _ _
  · exact mem_insert_of_mem (mem_image.mpr ⟨x, Finset.mem_preimage.mpr hv, rfl⟩)
  -- xʳ cannot lie in T since it is adjacent to none ∈ T
  · exact (hT (mem_coe.mpr hnone) (mem_coe.mpr hv) (by simp) (by simp)).elim
```

No vertex in the right part of an independent finset T of $\mathrm{m}\mu(G)$ has a neighbour in its left part (there are no `left–right` edges within T).

```
lemma mycielskian_indepSet_preimage_right_subset [Fintype V] [DecidableEq V]
    [G.LocallyFinite]
    {T : Finset (Option (V ⊕ V))} (hT : mμ(G).IsIndepSet ↑T) :
    T.preimage (·ʳ) right_injective.injOn ⊆
      N(T.preimage (·ˡ) left_injective.injOn)ᶜ := by
  intro y hy
  refine mem_compl_openNeighborhoodFinset.mpr fun a ha => ?_
  simpa using hT (Finset.mem_preimage.mp ha) (Finset.mem_preimage.mp hy) (by simp)
```

Any finset T of vertices of $\mathrm{m}\mu(G)$ not containing none is covered by the left copies of its left part together with the right copies of its right part. (This is just bookkeeping, no graph is involved: every vertex of $\mathrm{m}\mu(G)$ is none, a left copy, or a right copy.)

```
lemma subset_image_preimage_left_union_right [DecidableEq V]
    {T : Finset (Option (V ⊕ V))} (hnone : none ∉ T) :
    T ⊆ (T.preimage (·ˡ) left_injective.injOn).image (·ˡ) ∪
      (T.preimage (·ʳ) right_injective.injOn).image (·ʳ) := by
  intro v hv
  rcases v with _ | (x | x)
  · exact absurd hv hnone
  · exact mem_union_left _ (mem_image.mpr ⟨x, Finset.mem_preimage.mpr hv, rfl⟩)
  · exact mem_union_right _ (mem_image.mpr ⟨x, Finset.mem_preimage.mpr hv, rfl⟩)
```

Key calculation for the upper bound: an independent set T of $\mathrm{m}\mu(G)$ is no larger than `#A + #N(A)ᶜ`, where `A` is the set of vertices of G whose left copy lies in T.

```
lemma mycielskian_indepSet_card_le [Fintype V] [Nonempty V] [DecidableEq V]
    [G.LocallyFinite] {T : Finset (Option (V ⊕ V))} (hT : mμ(G).IsIndepSet ↑T) :
    #T ≤ #(T.preimage (·ˡ) left_injective.injOn) +
      #N(T.preimage (·ˡ) left_injective.injOn)ᶜ := by
  -- A (resp. B) is the set of vertices of G whose
  -- left (resp. right) copy lies in T.
  let A : Finset V := T.preimage (·ˡ) left_injective.injOn
  let B : Finset V := T.preimage (·ʳ) right_injective.injOn
  -- A is an independent set of G, so N(A)ᶜ is nonempty (previous lemmas).
  have hNne : N(A)ᶜ.Nonempty :=
    (isIndepSet_preimage_left_finset hT).compl_openNeighborhoodFinset_nonempty
  by_cases hnone : none ∈ T
```

```
  · -- none ∈ T : T is covered by none and the left copies of A, and
    -- N(A)ᶜ contributes at least 1 since it is nonempty.
    calc #T ≤ #(insert none (A.image (·ˡ))) :=
          Finset.card_le_card (mycielskian_indepSet_subset_insert_none hT hnone)
      _ ≤ #(A.image (·ˡ)) + 1 := Finset.card_insert_le _ _
      _ = #A + 1 := by
            rw [Finset.card_image_of_injective _ left_injective]
      _ ≤ #A + #N(A)ᶜ := Nat.add_le_add_left hNne.card_pos _
  · -- none ∉ T : T is covered by the left and right images of A and B,
    -- and B ⊆ N(A)ᶜ since there are no left–right edges in T.
    calc #T ≤ #(A.image (·ˡ) ∪ B.image (·ʳ)) :=
          Finset.card_le_card (subset_image_preimage_left_union_right hnone)
      _ = #A + #B := card_image_left_union_image_right A B
      _ ≤ #A + #N(A)ᶜ := Nat.add_le_add_left
          (Finset.card_le_card (mycielskian_indepSet_preimage_right_subset hT)) _
```

Putting the two bounds together we obtain a formula for the independence number of the Mycielskian: `α mμ(G)` is the maximum of `#s + #N(s)ᶜ` over all independent sets `s` of `G`, and this maximum is realized by the independent set `sˡ ∪ (N(s)ᶜ)ʳ`. In particular there is always a maximum independent set of `mμ(G)` avoiding `none` (although other maximum independent sets may contain `none`, as happens for `G = ⊥` on a single vertex).

We write the maximum using `Finset.sup`: for an $\mathbb{N}$-valued function `f`, `t.sup f` is the largest value of `f` on the finset `t` (and `0` if `t` is empty). The hypothesis `[Nonempty V]` is needed: if `V` is empty then `mμ(G)` still has the vertex `none`, so `α mμ(G) = 1`, but the only independent set of `G` is `∅` and the formula gives `0`.

```
theorem indepNum_mycielskian [Fintype V] [Nonempty V] [DecidableEq V]
    [DecidableRel G.Adj] :
    α mμ(G) = ({s | G.IsIndepSet ↑s} : Finset (Finset V)).sup
    (fun s => #s + #N(s)ᶜ) := by
  apply le_antisymm
  · -- ≤ : a maximum independent set T of mμ(G) is at most the value of the
    -- sup at its left part A.
    obtain ⟨T, hT⟩ := mμ(G).exists_isNIndepSet_indepNum
    rw [← hT.card_eq]
    have hAF : T.preimage (·ˡ) left_injective.injOn ∈
        univ.filter fun s : Finset V => G.IsIndepSet ↑s :=
      mem_filter.mpr ⟨mem_univ _, isIndepSet_preimage_left_finset hT.isIndepSet⟩
    exact (G.mycielskian_indepSet_card_le hT.isIndepSet).trans
      (Finset.le_sup (f := fun s => #s + #N(s)ᶜ) hAF)
  · -- ≥ : each independent s achieves its value in the sup.
    exact Finset.sup_le fun s hs => le_indepNum_mycielskian (mem_filter.mp hs).2
```

Taking `s = ∅` in the formula gives `α mμ(G) ≥ Nat.card V`, and taking `s` to be a maximum independent set gives `α mμ(G) ≥ 2 * α G` (since `s ⊆ N(s)ᶜ` for an independent set `s`). But there are examples where neither is sharp: add an isolated vertex `w` to `C[5]`; the resulting graph has `6` vertices and independence number `3`, so both bounds give `6`, but `s = {w}` gives `1 + 6 = 7`.

For the `5`-cycle the only maximizer is `s = ∅`, with `#∅ + #N(∅)ᶜ = 0 + 5 = 5`, while every nonempty independent `s` gives at most `4 = 2 * α C[5]`.

The Mycielskian of the `5`-cycle is the *Grötzsch graph*, so we can use the previous result to compute its independence number. After rewriting with `indepNum_mycielskian`, `decide` evaluates the maxi-

mum over the 11 independent sets of C[5] (by checking all 2 ^ 5 = 32 subsets of Fin 5).

```
example : α grotzschGraph = 5 := by
  rw [indepNum_mycielskian]
  decide

end open_neighborhoods
```

6.8 Exercises

1. Show that any finite subset t of a clique s is a #t-clique.

```
open Finset
variable {V : Type*} {G : SimpleGraph V}

example {s : Set V} {t : Finset V} (hc : G.IsClique s) (h : ↑t ⊆ s) :
    G.IsNClique #t t := by
  sorry
```

2. Show that {0, 1, 2} is a triangle in the Paley graph on 17 vertices.

```
example : (paleyGraph (ZMod 17)).IsNClique 3 {0, 1, 2} := by
  sorry
```

3. Show that the Paley graph on 17 vertices is 4-clique-free. Hint: `decide` will probably time out, so use `native_decide` instead.

```
lemma cliqueFree_paleyGraph_seventeen_four' :
    (paleyGraph (ZMod 17)).CliqueFree 4 := by
  sorry
```

4. Show that the clique number of finite graphs is monotone: if G ≤ H then ω G ≤ ω H. (There is a one-line proof using a lemma from the chapter, and a direct proof using exists_isNClique_cliqueNum.)

```
lemma cliqueNum_mono' [Finite V] {H : SimpleGraph V} (h : G ≤ H) : ω G ≤ ω H := by
  sorry
```

5. Show that the clique number of the Paley graph on 17 vertices is 3. (Recall that we have IsNClique.cliqueNum_eq_of_cliqueFree, so we just need to give an example of a 3-clique and show that the graph is 4-clique-free.)

```
example : ω (paleyGraph (ZMod 17)) = 3 := by
  sorry
```

In the next few exercises we look at the clique number of graph products starting with the tensor product G ⊗g H. Recall that two vertices (v₁, w₁), (v₂, w₂) of the tensor product are adjacent if and only if G.Adj v₁ v₂ and H.Adj w₁ w₂.

6. The first projection of a clique in the tensor product `G ⊗g H` is a clique in `G`.

```
example {W : Type*} {H : SimpleGraph W} {s : Set (V × W)}
    (h : (G ⊗g H).IsClique s) : G.IsClique (Prod.fst '' s) := by
  sorry
```

7. If `G` is `n`-clique-free then so is its tensor product `G ⊗g H` with any graph `H`.

```
lemma CliqueFree.tensorProd_left' {W : Type*} {H : SimpleGraph W} {n : ℕ}
    (h : G.CliqueFree n) : (G ⊗g H).CliqueFree n := by
  sorry
```

8. The `j`-th projection of a clique in the indexed tensor product `⊗g i, G i` is a clique in `G j`.

```
example {ι : Type*} {U : ι → Type*} {G : (i : ι) → SimpleGraph (U i)} (j : ι)
    {s : Set ((i : ι) → U i)} (h : (⊗g i, G i).IsClique s) :
    (G j).IsClique ((· j) '' s) := by
  sorry
```

9. The clique number of `G ⊗g H` is `min (ω G) (ω H)` (for finite graphs). Hint: for the lower bound, let `m = min (ω G) (ω H)` and use `not_cliqueFree_iff_top_isContained` to find copies of `K[m]` in both `G` and `H`. Pair them with `tensorProd.prod` to get a homomorphism `K[m] →g G ⊗g H`.

```
lemma cliqueNum_tensorProd' {W : Type*} [Finite V] [Finite W] {H : SimpleGraph W} :
    ω (G ⊗g H) = min (ω G) (ω H) := by
  sorry
```

We now turn to the box product `G □ H`. Recall that two vertices `(v₁, w₁), (v₂, w₂)` of the box product are adjacent if and only if either `G.Adj v₁ v₂` and `w₁ = w₂`, or `v₁ = v₂` and `H.Adj w₁ w₂`. This implies that cliques are always constant in one coordinate.

10. A clique `s` in `G` gives a "row" clique in the box product `G □ H` (so long as there is a vertex `w₀` in `H` to fix the second coordinate).

```
example {W : Type*} {H : SimpleGraph W} {s : Set V} (w₀ : W) (h : G.IsClique s) :
    (G □ H).IsClique ((·, w₀) '' s) := by
  sorry
```

11. In any finite box product `G □ H` with nonempty second factor we have `ω G ≤ ω (G □ H)`: the embedding `boxProdLeft` can transport a maximum clique of `G` to a clique of the same size in `G □ H`.

```
lemma cliqueNum_le_cliqueNum_boxProd_left' {W : Type*} [Finite V] [Finite W]
    [Nonempty W] {H : SimpleGraph W} : ω G ≤ ω (G □ H) := by
  sorry
```

12. A clique of `G □ H` containing a "horizontal" edge (two vertices sharing their second coordinate) is constant in the second coordinate.

```
lemma IsClique.boxProd_snd_eq' {W : Type*} {H : SimpleGraph W} {s : Set (V × W)}
    {x y : V × W} (hs : (G □ H).IsClique s) (hx : x ∈ s) (hy : y ∈ s)
    (hG : G.Adj x.1 y.1) (hsnd : x.2 = y.2) : ∀ z ∈ s, z.2 = x.2 := by
  sorry
```

13. `Prod.fst` is injective on a clique of $G \square H$ whose vertices all share the same second coordinate, so such a clique projects to a clique of G of the same size.

```
lemma IsNClique.boxProd_image_fst' {W : Type*} [DecidableEq V] {H : SimpleGraph W}
    {m : ℕ} {t : Finset (V × W)} {w : W} (ht : (G □ H).IsNClique m t)
    (hconst : ∀ z ∈ t, z.2 = w) : G.IsNClique m (t.image Prod.fst) := by
  sorry
```

14. If G is finite then a clique of $G \square H$ containing a "horizontal" edge has size at most $\omega(G)$.

```
lemma IsNClique.boxProd_le_cliqueNum_left' {W : Type*} [Finite V]
    {H : SimpleGraph W} {m : ℕ} {t : Finset (V × W)} {x y : V × W}
    (ht : (G □ H).IsNClique m t) (hx : x ∈ t) (hy : y ∈ t) (hG : G.Adj x.1 y.1)
    (hsnd : x.2 = y.2) : m ≤ ω G := by
  sorry
```

15. The clique number of $G \square H$ is $\max(\omega(G))(\omega(H))$ (for finite nonempty graphs).

```
lemma cliqueNum_boxProd' {W : Type*} [Finite V] [Finite W] [Nonempty V]
    [Nonempty W] {H : SimpleGraph W} :
    ω (G □ H) = max (ω G) (ω H) := by
  sorry
```

16. The clique number of the hypercube $Q[n]$ is 1 for $n = 0$ and 2 otherwise. (There are easier ways to prove this: for instance, once we have colourings we could use a two-colouring to prove $\omega(Q[n]) \le 2$, but we can also use the result of the previous exercise.)

```
lemma cliqueNum_hypercube' {n : ℕ} : ω Q[n] = if n = 0 then 1 else 2 := by
  sorry
```

17. Show that a set of vertices is a clique in G if and only if it is an independent set in the complement of G. (Mathlib has this as `isIndepSet_compl`, but prove it directly from the definitions, using `compl_adj`.)

```
example {s : Set V} : G.IsClique s ↔ Gᶜ.IsIndepSet s := by
  sorry
```

18. Show that the Paley graph on 17 vertices has independence number 3. Together with Exercise 3 this shows that there is a graph on 17 vertices with no 4-clique and no independent set of size 4, so the Ramsey number $R(4,4)$ is at least 18. (In fact $R(4,4) = 18$.) Hint: as in the chapter, rewrite with `cliqueFree_compl`, then use `native_decide` as in Exercise 3.

```
example : α (paleyGraph (ZMod 17)) = 3 := by
  sorry
```

19. Show that a clique and an independent set share at most one vertex, and deduce that `ω G + α G ≤ Nat.card V + 1` in any finite graph.

```
lemma IsClique.card_inter_le_one_of_isIndepSet' [DecidableEq V] {s t : Finset V}
    (hs : G.IsClique ↑s) (ht : G.IsIndepSet ↑t) : #(s ∩ t) ≤ 1 := by
  sorry

lemma cliqueNum_add_indepNum_le' [Finite V] : ω G + α G ≤ Nat.card V + 1 := by
  sorry
```

20. Show that any graph on at least 6 vertices contains a triangle or an independent set of size 3, so that `R(3, 3) = 6`. Hint: suppose not, and apply `degree_le_indepNum_of_triangleFree` to both `G` and G^c, bounding both independence numbers with `CliqueFree.cliqueNum_le`. Mathlib's `degree_compl` relates the degrees of a vertex in `G` and G^c.

```
theorem not_cliqueFree_three_and_indepSetFree_three' [Fintype V]
    (h : 6 ≤ Fintype.card V) : ¬(G.CliqueFree 3 ∧ G.IndepSetFree 3) := by
  sorry
```

21. Unlike the junk value `α pathNat = 0` of the independence number, the clique number of the infinite path on $\mathbb{N}$ is what you expect. Show that `ω pathNat = 2`. Hint: `pathNat` is triangle-free, since adjacent vertices have different parity. The lemma `IsNClique.le_cliqueNum` needs `[Finite V]`, but `IsNClique.cliqueNum_eq_of_cliqueFree` needs no finiteness.

```
example : ω pathNat = 2 := by
  sorry
```

22. Show that the Kneser graph `Kn[n, 1]` is isomorphic to the complete graph `K[n]`. (Its vertices are the singletons `{a}`, and distinct singletons are disjoint.)

```
example (n : ℕ) : Nonempty (Kn[n, 1] ≃g K[n]) := by
  sorry
```

23. Compute the clique number of the Mycielskian of the edgeless graph `⊥`. The answer depends on whether `V` is empty. If `V` is empty then `mμ(⊥)` has only the vertex `none`, so its clique number is 1. If `V` is nonempty then `none` and `v ᵀ` are adjacent and `mμ(⊥)` is triangle-free, so its clique number is 2.

```
lemma cliqueNum_mycielskian_bot_of_isEmpty' [IsEmpty V] :
    ω mμ((⊥ : SimpleGraph V)) = 1 := by
  sorry
```

```
lemma cliqueNum_mycielskian_bot' [Nonempty V] :
    ω mμ((⊥ : SimpleGraph V)) = 2 := by
  sorry
```

The last few exercises are on independent sets in complete bipartite graphs (and their Mycielskians).

24. In a complete bipartite graph the non-neighbours of a left vertex are precisely the left part, and similarly for right vertices.

```
lemma completeBipartiteGraph_not_adj_inl_iff' {W : Type*} {v : V} {x : V ⊕ W} :
    ¬(completeBipartiteGraph V W).Adj (.inl v) x ↔
      x ∈ Set.range .inl := by
  sorry

lemma completeBipartiteGraph_not_adj_inr_iff' {W : Type*} {w : W} {x : V ⊕ W} :
    ¬(completeBipartiteGraph V W).Adj (.inr w) x ↔
      x ∈ Set.range .inr := by
  sorry
```

25. An independent set of a complete bipartite graph is contained entirely in the left part or entirely in the right part: if it contains a left vertex then all its vertices are non-neighbours of that vertex, so all lie in the left part.

```
lemma completeBipartiteGraph_isIndepSet_subset_left_or_right' {W : Type*}
    {s : Set (V ⊕ W)} (hs : (completeBipartiteGraph V W).IsIndepSet s) :
    s ⊆ Set.range .inl ∨ s ⊆ Set.range .inr := by
  sorry
```

26. Any `Finset` contained in the range of a function `f : α → β` with finite domain has at most `Nat.card α` elements. In particular a `Finset (V ⊕ W)` contained in the left part has at most `Nat.card V` elements (and similarly on the right), since the left part is the range of `Sum.inl`.

```
lemma _root_.Finset.card_le_of_subset_range' {α β : Type*} [Finite α] {f : α → β}
    {s : Finset β} (h : ↑s ⊆ Set.range f) : #s ≤ Nat.card α := by
  sorry
```

27. Any set of vertices contained in one part of a complete bipartite graph is independent, since a part contains no edges (we only need this for the right part).

```
lemma isIndepSet_completeBipartiteGraph_of_subset_range_inr' {W : Type*}
    {s : Set (V ⊕ W)} (h : s ⊆ Set.range .inr) :
    (completeBipartiteGraph V W).IsIndepSet s := by
  sorry
```

28. The independence number of a complete bipartite graph is the size of the larger part; we state this as `α K[a, b] = b` when `a ≤ b`. The right part is a `b`-element independent set, and by Exercises 25 and 26 no independent set is larger.

```
lemma indepNum_completeBipartiteGraph' {a b : ℕ} (h : a ≤ b) :
    α K[a, b] = b := by
  sorry
```

If we want to bound the independence number of the Mycielskian of a complete bipartite graph we need a bound on the size of any independent set `s` of `K[a, b]` plus the size of the complement of its open neighbourhood `N(s)ᶜ`. We give this bound in the next exercise. The last exercise uses `le_indepNum_mycielskian` and `mycielskian_indepSet_card_le` from the starred section Independence Number of the Mycielskian (p. 145).

29. Any independent set `s` of `K[a, b]` (with `a ≤ b`) satisfies `#s + #N(s)ᶜ ≤ 2 * b`. We state this with an abstract finset `t` in place of `N(s)ᶜ`, characterized by its membership property, so that the lemma applies whatever decidability instances `N(s)ᶜ` is written with. If `s` is empty then `t` is all `a + b` vertices; otherwise `s` lies in a single part (Exercise 25) and `t` lies in the same part (every vertex of the other part is a neighbour of any vertex of `s`), so each has size at most that of the part.

```
lemma completeBipartiteGraph_isIndepSet_card_add_le' {a b : ℕ} (hab : a ≤ b)
    {s t : Finset (Fin a ⊕ Fin b)} (hs : K[a, b].IsIndepSet ↑s)
    (ht : ∀ x, x ∈ t ↔ ∀ y ∈ s, ¬K[a, b].Adj y x) :
    #s + #t ≤ 2 * b := by
  sorry
```

30. The independence number of the Mycielskian of `K[a, b]` is `2 * b` whenever `a ≤ b` and `b ≠ 0`: the right part `B` is an independent set with `B ⊆ N(B)ᶜ`, so it contributes at least `2 * b`, and no independent set does better by Exercise 29. (Note that `b ≠ 0` is needed: `mμ(K[0, 0])` still has the vertex none, so its independence number is `1`.)

```
lemma indepNum_mycielskian_completeBipartiteGraph' {a b : ℕ} [NeZero b]
    (h : a ≤ b) : α mμ(K[a, b]) = 2 * b := by
  sorry
```

7 Colorings

A *proper colouring* of a graph assigns a colour to each vertex so that adjacent vertices receive different colours, and the *chromatic number* is the least number of colours that suffice. Colourings are among the most studied topics in graph theory. Any clique requires as many colours as it has vertices, so the clique number is a lower bound for the chromatic number.

In this chapter we study graph colourings (`Coloring`), colourability (`Colorable`), the chromatic number (`chromaticNumber`) and bipartite graphs (`IsBipartite`). We relate colourings to cliques and independent sets, showing that a finite graph has at most `χ G * α G` vertices (`natCard_le_chromaticNumber_mul_indepNum`) and that colourings of two graphs whose supports meet in a clique can be amalgamated (`Colorable.sup_of_isClique_inter`). We then colour some concrete graphs: blow-ups, box products and hypercubes. We show that the chromatic number of the Mycielskian is exactly one more than that of the original graph (`chromaticNumber_mycielskian`), which is what makes the clique bound arbitrarily bad. We also look briefly at edge colourings.

Finally we turn to the *greedy* colouring algorithm. We first define it in a form that is easy to reason about, using `Nat.find` and well-founded recursion (`greedy`), and use this to prove that any locally finite graph on a countable vertex type with all degrees at most `Δ` is `(Δ + 1)`-colourable (`colorable_of_degree_le_of_countable`). We look at the crown graph to see how badly the greedy colouring can perform. We end by looking at how to make it more efficient.

7.1 Colourings

In Mathlib, a `γ`-colouring of `G` is a *graph homomorphism* from `G` to the complete graph `⊤` on `γ`. Since all distinct pairs of vertices are adjacent in `⊤`, this corresponds exactly to a proper `γ`-colouring. (We call the type of colours `γ`, but Mathlib's own statements, such as the definition below, call it `α`.)

```
open Finset
section colorings
variable {V W : Type*} {G : SimpleGraph V} {H : SimpleGraph W}

#source Coloring -- abbrev Coloring (α : Type v) := G →g completeGraph α
```

Adjacent vertices get different colours.

```
#check Coloring.valid  -- C.valid : G.Adj u v → C u ≠ C v
```

The colour class of a colour `c` is the set of vertices coloured `c`.

```
#check Coloring.colorClass -- C.colorClass c : Set V
```

Colour classes are independent sets.

```
#check Coloring.isIndepSet_colorClass -- G.IsIndepSet (C.colorClass c)
```

So a colouring is the same thing as a partition of the vertices into independent sets, one for each colour. Mathlib makes this precise with `Coloring.toPartition` and `partitionable_iff_colorable`.

```
#check Coloring.toPartition -- (C : G.Coloring α) : G.Partition
#check partitionable_iff_colorable -- G.Partitionable n ↔ G.Colorable n
```

Any cycle graph can be 3-coloured, and any even cycle can be 2-coloured using `Bool` as the set of colours. Colourings of concrete graphs can also be evaluated.

```
#eval cycleGraph.tricoloring 7 -- {0 ↦ 0, 1 ↦ 1, 2 ↦ 0, 3 ↦ 1, 4 ↦ 0, 5 ↦ 1, 6 ↦ 2}

#eval cycleGraph.bicoloring_of_even 4 (by decide) -- {0 ↦ true, 1 ↦ false,
                                                  --      2 ↦ true, 3 ↦ false}
```

If `n : ℕ` then `G.Colorable n` says that `G` has a proper colouring with `n` colours (using `Fin n`). Note the difference between the two notions: a `G.Coloring γ` is *data*, a specific colouring, whereas `G.Colorable n` is a `Prop` that only records that some colouring exists (`Nonempty`). Inside a proof, `obtain ⟨C⟩ := h` recovers a colouring `C` from `h : G.Colorable n`.

```
example (n : ℕ) : G.Colorable n ↔ Nonempty (G.Coloring (Fin n)) := Iff.rfl
```

In the other direction, a colouring `C : G.Coloring γ` with a finite type of colours `γ` shows that `G` is `Fintype.card γ`-colourable.

```
#check Coloring.colorable
-- [Fintype α] (C : G.Coloring α) : G.Colorable (Fintype.card α)
```

Note that `G.Colorable 0` holds iff `G` can be coloured with the colours `Fin 0`. Since `Fin 0` has no elements, this is possible iff `G` has no vertices.

```
#check colorable_zero_iff -- G.Colorable 0 ↔ IsEmpty V
```

If `G` is `n`-colourable and `n ≤ m`, then `G` is `m`-colourable.

```
#check Colorable.mono
```

Since colourings are homomorphisms to `⊤`, we can compose them with any other homomorphism: if `G →g G'` and `G'` is `n`-colourable then so is `G`.

```
#check Colorable.of_hom -- (f : G →g G') (h : G'.Colorable n) : G.Colorable n
```

7.2 Chromatic Number

The *chromatic number* of a graph `G` is the minimum number of colours needed to colour `G`. In Mathlib, this is defined using `ℕ∞` which is `ℕ` together with `⊤` (`∞`). So the chromatic number is the minimum `n` such that `G.Colorable n` holds, or `⊤` if there is no such `n`.

```
#source SimpleGraph.chromaticNumber  -- .. := ⨅ n ∈ Set.ofPred G.Colorable, (n : ℕ∞)
```

We introduce the standard `χ G` notation for the chromatic number of `G`.

```
@[inherit_doc]
scoped notation "χ" => SimpleGraph.chromaticNumber
```

The basic tool for working with `χ G` is that, for `n : ℕ`, we have `χ G ≤ n` iff `G` is `n`-colourable. Many proofs about the chromatic number start by rewriting with this.

```
#check chromaticNumber_le_iff_colorable -- χ G ≤ n ↔ G.Colorable n
```

The chromatic number is `n + 1` iff `G` is `(n + 1)`-colourable but not `n`-colourable.

```
#check chromaticNumber_eq_iff_colorable_not_colorable
```

If there is a homomorphism `G →g G'` then `χ G ≤ χ G'`. In particular a subgraph has chromatic number at most that of the whole graph.

```
#check chromaticNumber_mono_of_hom -- (f : G →g G') : χ G ≤ χ G'
#check chromaticNumber_mono -- (G' : SimpleGraph V) (h : G ≤ G') : χ G ≤ χ G'
```

Any graph on a finite set of vertices can be coloured with at most as many colours as there are vertices.

```
#check SimpleGraph.chromaticNumber_le_card  -- χ G ≤ Fintype.card V
```

Any graph on a finite vertex type has finite chromatic number.

```
theorem chromaticNumber_ne_top_of_finite (G : SimpleGraph V) [Finite V] :
    χ G ≠ ⊤ :=
  chromaticNumber_ne_top_iff_exists.mpr ⟨_, G.colorable_chromaticNumber_of_fintype⟩
```

On an infinite vertex type the chromatic number can be `⊤`: the complete graph on `ℕ` cannot be coloured with finitely many colours. Compare this with the `ℕ`-valued clique number, for which `ω (⊤ : SimpleGraph ℕ) = 0` is a junk value.

```
example : χ (⊤ : SimpleGraph ℕ) = ⊤ := chromaticNumber_top_eq_top_of_infinite ℕ
```

The complete graph on `Fin n` has chromatic number `n`.

```
example (n : ℕ) : χ K[n] = n := by
  simp
```

7.3 Colourings, Cliques and Independent Sets

Any `k`-clique in a graph requires at least `k` colours to colour it, so the clique number of a graph is at most its chromatic number.

```
example {s : Finset V} (h : G.IsClique s) : #s ≤ χ G := h.card_le_chromaticNumber
```

So if `χ G < n` then `G` is `n`-clique-free.

```
#check cliqueFree_of_chromaticNumber_lt
```

If a clique has at least as many vertices as there are colours, then every colour appears on the clique.

```
#check Coloring.surjOn_of_card_le_isClique
```

A proper colouring is injective on any clique: two distinct vertices of a clique are adjacent, so they receive different colours. Since a colouring is a homomorphism, this is a special case of `Hom.injOn_of_isClique` from Finite Cliques (p. 128), which dot notation finds for us.

```
example {γ : Type*} (c : G.Coloring γ) {s : Set V} (hs : G.IsClique s) :
    Set.InjOn c s :=
  c.injOn_of_isClique hs
```

If H is not n-colourable and G is n-colourable then G cannot contain a copy of H, e.g. a bipartite graph cannot contain a triangle.

```
#check free_of_colorable
```

A graph is *complete multipartite* (`IsCompleteMultipartite`) if its vertices split into parts so that two vertices are adjacent iff they lie in different parts. Mathlib defines this as transitivity of non-adjacency in G. Non-adjacency is always reflexive (there are no loops) and symmetric, so transitivity makes it an equivalence relation, and its equivalence classes are the parts.

```
#source IsCompleteMultipartite
```

What can we say about maximally clique-free graphs? Suppose that G is `(r + 1)`-clique-free but adding any edge creates an `(r + 1)`-clique. Then G is `r`-colourable iff it is complete multipartite.

```
#check colorable_iff_isCompleteMultipartite_of_maximal_cliqueFree
```

Colour classes give a bound in the other direction, in terms of independent sets. If G is k-colourable then its vertices are covered by k colour classes, each an independent set of at most `α G` vertices, so G has at most `k * α G` vertices. No finiteness assumption is needed: if V is infinite then `Nat.card V` is the junk value `0`, so the inequality holds trivially.

```
theorem Colorable.natCard_le_mul_indepNum {k : ℕ} (h : G.Colorable k) :
    Nat.card V ≤ k * α G := by
  classical
  cases finite_or_infinite V with
  | inr => simp -- Nat.card V is the junk value 0
  | inl =>
  have := Fintype.ofFinite V
  obtain ⟨C⟩ := h
  calc Nat.card V = ∑ i, #{v | C v = i} := by
        rw [Nat.card_eq_fintype_card, ← card_univ,
          card_eq_sum_card_fiberwise fun v _ => mem_univ (C v)]
    _ ≤ ∑ _ : Fin k, α G := by
        refine sum_le_sum fun i _ => IsIndepSet.card_le_indepNum ?_
        simpa [Coloring.colorClass] using C.isIndepSet_colorClass i
    _ = k * α G := by simp
```

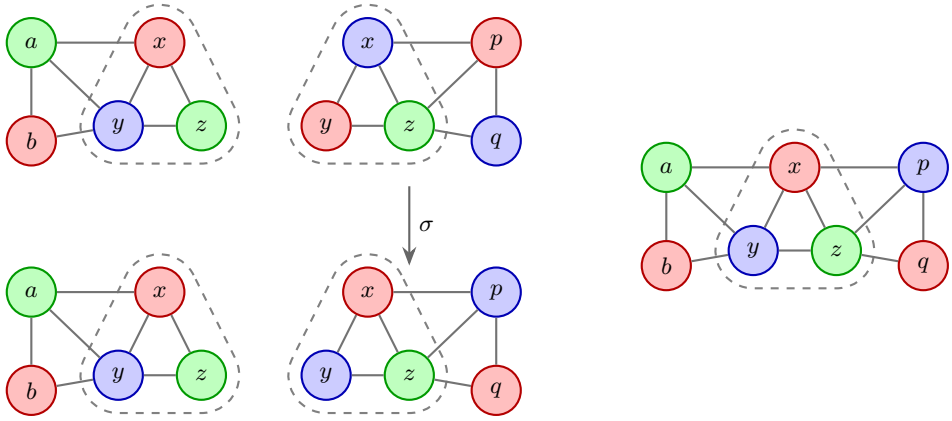
Taking `k = χ G` shows that a finite graph has at most `χ G * α G` vertices. As before, the Lean statement holds trivially when V is infinite. Since `χ G` takes values in $\mathbb{N}\infty$, the statement lives in $\mathbb{N}\infty$: the natural numbers `Nat.card V` and `α G` are silently cast to $\mathbb{N}\infty$ (compare Counting in $\mathbb{N}\infty$: ENat.card (p. 68)).

For finite V we cannot apply `Colorable.natCard_le_mul_indepNum` with $k = \chi\ G$ directly, since k must be a natural number. This is a job for the **`lift`** tactic, which moves a value from $\mathbb{N}_\infty$ down to $\mathbb{N}$ once we know that it is finite. Given a proof that $\chi\ G \neq \top$ (`chromaticNumber_ne_top_of_finite`), `lift χ G to ℕ using … with k hk` introduces a natural number k with $hk : \uparrow k = \chi\ G$ and replaces $\chi\ G$ by $\uparrow$k in the goal. (We must name k here, since $\chi\ G$ is an expression rather than a variable; when lifting a variable `x`, **`lift`** reuses the name `x`.) Now G is k-colourable by `chromaticNumber_le_iff_colorable`, and **`exact_mod_cast`** transfers the inequality from $\mathbb{N}$ to $\mathbb{N}_\infty$.

```
theorem natCard_le_chromaticNumber_mul_indepNum : Nat.card V ≤ χ G * α G := by
  cases finite_or_infinite V with
  | inr => simp -- Nat.card V is the junk value 0
  | inl =>
  lift χ G to ℕ using G.chromaticNumber_ne_top_of_finite with k hk
  exact_mod_cast (chromaticNumber_le_iff_colorable.mp hk.ge).natCard_le_mul_indepNum
```

Amalgamating Colourings over a Clique*

Recall that the `support` of a graph is the set of vertices that are not isolated. Suppose G_1 and G_2 share a vertex type and their supports meet in a set that is a clique in both graphs. Then any colourings of G_1 and G_2 drawn from a common palette can be amalgamated into a colouring of the union $G_1 \sqcup G_2$ — a *clique-sum*. On a clique both colourings are injective, so `Equiv.Perm.exists_extending_pair` gives a permutation σ of the palette with $\sigma\ (c_2\ x) = c_1\ x$ for every `x` in the overlap. Then c_1 on G_1 and the recoloured c_2 elsewhere agree on the overlap.



Top left: G_1 coloured by c_1 and G_2 coloured by c_2; supports meet on `x`, `y`, `z`, and c_1 and c_2 disagree at `x` and `y`.
Bottom left: G_1 again, and G_2 recoloured by $\sigma \circ c_2$, where σ swaps red and blue. Right: the glued colouring of $G_1 \sqcup G_2$

```
theorem Colorable.sup_of_isClique_inter {G₁ G₂ : SimpleGraph V}
    (hs₁ : G₁.IsClique (G₁.support ∩ G₂.support))
    (hs₂ : G₂.IsClique (G₁.support ∩ G₂.support))
    {n : ℕ} (h₁ : G₁.Colorable n) (h₂ : G₂.Colorable n) :
    (G₁ ⊔ G₂).Colorable n := by
  classical
  obtain ⟨c₁⟩ := h₁
  obtain ⟨c₂⟩ := h₂
  let s := G₁.support ∩ G₂.support
  -- Both colourings are injective on the shared clique s
  have hinj₁ : Function.Injective (fun x : s => c₁ x) :=
```

```
    Set.injOn_iff_injective.1 (c₁.injOn_of_isClique hs₁)
  have hinj₂ : Function.Injective (fun x : s => c₂ x) :=
    Set.injOn_iff_injective.1 (c₂.injOn_of_isClique hs₂)
  -- which is therefore finite
  have : Finite s := Finite.of_injective _ hinj₁
  -- so a permutation σ of the palette carries c₂ to c₁ on s.
  obtain ⟨σ, hσ⟩ := Equiv.Perm.exists_extending_pair
    (fun x : s => c₂ x) (fun x : s => c₁ x) hinj₂ hinj₁
  have hσkey : ∀ x ∈ s, σ (c₂ x) = c₁ x := fun x hx => hσ ⟨x, hx⟩
  -- The glued colouring: c₁ on G₁'s support, the recoloured c₂ elsewhere.
  let f : V → Fin n := fun x => if x ∈ G₁.support then c₁ x else σ (c₂ x)
  -- On G₂'s support, f coincides with the recoloured c₂ (they agree on the
  -- shared clique).
  have key : ∀ {x}, x ∈ G₂.support → f x = σ (c₂ x) := fun {x} hx => by
    simp only [f]; split_ifs with hx1
    -- we use the exacts tactic to close the two goals
    exacts [(hσkey x ⟨hx1, hx⟩).symm, rfl]
  refine ⟨Coloring.mk f ?_⟩
  rintro a b (hab | hab)
  · -- An edge of G₁: both endpoints use c₁.
    simpa [f, hab.mem_support_left, hab.mem_support_right] using c₁.valid hab
  · -- An edge of G₂: both endpoints use the recoloured c₂.
    rw [key hab.mem_support_left, key hab.mem_support_right]
    exact fun h => c₂.valid hab (σ.injective h)
```

In particular if the supports of the two graphs share at most the single vertex v, then any colourings of G_1 and G_2 with the same palette merge into a colouring of $G_1 \sqcup G_2$. This is the special case of `Colorable.sup_of_isClique_inter` where the overlap is a subsingleton, hence vacuously a clique in each part.

```
theorem Colorable.sup_of_subsingleton_inter {G₁ G₂ : SimpleGraph V}
    {v : V} (hmeet : G₁.support ∩ G₂.support ⊆ {v})
    {n : ℕ} (h₁ : G₁.Colorable n) (h₂ : G₂.Colorable n) :
    (G₁ ⊔ G₂).Colorable n := by
  have hsub := Set.subsingleton_of_subset_singleton hmeet
  exact Colorable.sup_of_isClique_inter (hsub.pairwise _) (hsub.pairwise _) h₁ h₂
```

This last result is the key step in colouring a graph block by block: in Colouring a Graph from its Blocks (p. 322) we use it to show that if $n \geq 1$ and every block of a finite graph G is n-colourable, then so is G.

7.4 Bipartite Graphs

A graph is *bipartite* iff its vertices can be partitioned into two parts with no edges within parts. In Mathlib this is defined in the obvious way via 2-colourability, so `G.IsBipartite` is exactly `G.Colorable 2`.

```
#source IsBipartite -- abbrev IsBipartite (G : SimpleGraph V) : Prop := G.Colorable 2
```

Any complete bipartite graph is 2-colourable (bipartite). Mathlib gives the natural colouring, `CompleteBipartiteGraph.bicoloring`, which uses `Bool` as the set of colours (`true` on the right part). By Mathlib's naming conventions (p. 11) it should be called `completeBipartiteGraph`.

`bicoloring` (compare `pathGraph.bicoloring`), so we introduce that alias and use it for the rest of the guide.

```
alias completeBipartiteGraph.bicoloring := CompleteBipartiteGraph.bicoloring

#check completeBipartiteGraph.bicoloring
  -- (V W : Type*) : (completeBipartiteGraph V W).Coloring Bool
```

By `Coloring.colorable`, this colouring shows that the complete bipartite graph is bipartite. (Mathlib also has this as `IsBipartite.completeBipartiteGraph`.)

```
example : (completeBipartiteGraph V W).IsBipartite :=
  (completeBipartiteGraph.bicoloring V W).colorable
```

When both parts are nonempty there is an edge, so the chromatic number is exactly `2`. In general, a graph has chromatic number `2` iff it is bipartite and has an edge.

```
#check chromaticNumber_eq_two_iff -- χ G = 2 ↔ G.IsBipartite ∧ G ≠ ⊥
```

For example, colouring each vertex by its parity shows that the infinite path `pathNat` has chromatic number 2: an infinite graph can have finite chromatic number.

The natural `Bool`-colouring of `pathNat`, by parity.

```
def pathNat.bicoloring : pathNat.Coloring Bool where
  toFun n := n % 2 = 0
  map_rel' := by rintro a b (rfl | rfl) <;> grind [top_adj]

example : χ pathNat = 2 :=
  chromaticNumber_eq_two_iff.2 ⟨pathNat.bicoloring.colorable,
    ne_bot_iff_exists_adj.2 ⟨0, 1, Or.inl rfl⟩⟩
```

We also have `IsBipartiteWith` which says that *G* is bipartite with all edges going between the disjoint sets `s` and `t`. (Note that any isolated vertex can be in either or neither of the parts.)

```
#source IsBipartiteWith
```

For example, the complete bipartite graph is bipartite with parts the left and right copies. (Mathlib also has this as `IsBipartiteWith.completeBipartiteGraph`.)

```
example : (completeBipartiteGraph V W).IsBipartiteWith
    (Set.range .inl) (Set.range .inr) where
  disjoint := by grind
  mem_of_adj {u v} h := by grind
```

More generally, any `Bool`-colouring *c* of *G* makes *G* bipartite with parts the vertices coloured `true` and the vertices coloured `false`.

```
lemma Coloring.isBipartiteWith (c : G.Coloring Bool) :
    G.IsBipartiteWith {v | c v} {v | ¬c v} where
  disjoint := Set.disjoint_left.2 fun _ h h' => h' h
  mem_of_adj _ _ h := by grind [c.valid h]
```

Mathlib also shows that acyclic graphs (`IsAcyclic`), and in particular trees (`IsTree`), are bipartite. The classical characterization, that a graph is bipartite iff it has no odd cycles, is the subject of Odd Cycles and Bipartite Graphs (p. 199).

```
#check IsAcyclic.colorable_two -- (hG : G.IsAcyclic) : G.Colorable 2
#check IsTree.colorable_two -- (hG : G.IsTree) : G.Colorable 2
```

7.5 Colouring Concrete Graphs

Colouring Blow-ups

A graph G is n-colourable iff every blow-up of G with nonempty parts is n-colourable. In one direction we pull a colouring back along the embedding `Embedding.blowUp` of G into its blow-up (this is where we need the parts to be nonempty); in the other we compose with the projection `Sigma.fst`, which is a homomorphism from the blow-up to G.

```
lemma colorable_blowUpGraph_iff {n : ℕ} {ι : Type*} {W : ι → Type*}
    (G : SimpleGraph ι) (f : ∀ i, W i) :
    (G.blowUpGraph W).Colorable n ↔ G.Colorable n :=
  ⟨fun h => h.of_hom (Embedding.blowUp G W f).toHom,
   fun h => h.of_hom ⟨Sigma.fst, fun h => by simpa using h⟩⟩
```

For example, Mathlib's complete multipartite graph `completeMultipartiteGraph W`, with parts `W i`, is by definition the blow-up of the complete graph on the index type. So when there are finitely many parts, all nonempty, its chromatic number is the number of parts.

```
example {ι : Type*} (W : ι → Type*) :
    completeMultipartiteGraph W = (⊤ : SimpleGraph ι).blowUpGraph W := rfl


#check completeMultipartiteGraph.chromaticNumber
  -- [Fintype ι] (f : ∀ i, W i) : χ (completeMultipartiteGraph W) = Fintype.card ι
```

Colouring Box Products

We can restrict a colouring C of $G \square H$, along any fixed vertex of one factor, to a colouring of the other factor with the same colours: `v ↦ C (v, w₀)` colours G and `w ↦ C (v₀, w)` colours H.

The colouring of G induced by a colouring of $G \square H$.

```
def Coloring.ofBoxProdLeft {γ : Type*} (C : (G □ H).Coloring γ) (w : W) :
    G.Coloring γ :=
  C.comp ◁ boxProdLeft G H w
```

The colouring of H induced by a colouring of $G \square H$.

```
def Coloring.ofBoxProdRight {γ : Type*} (C : (G □ H).Coloring γ) (v : V) :
    H.Coloring γ :=
  C.comp ◁ boxProdRight G H v
```

Conversely, given k-colourings of G and H we can k-colour $G \square H$ as follows. If `C₁ : G.Coloring (Fin k)` and `C₂ : H.Coloring (Fin k)` then `C : V × W → Fin k` defined by `C (v, w) = C₁ v +`

`C₂ w`, with addition modulo `k` in `Fin k`, is a valid colouring of `G □ H`. Along an edge one coordinate is fixed, so the colours differ because the colouring of the other factor is valid.

The only property of addition this uses is cancellation: `a + b = a + c` implies `b = c`, and `b + a = c + a` implies `b = c` (`IsCancelAdd`). So we define the colouring for any colour type `γ` with a cancellative addition. Mathlib has `IsCancelAdd (Fin k)` for every `k`, so this covers `Fin k`. [1]

The colouring `(v, w) ↦ C₁ v + C₂ w` of `G □ H` induced by colourings `C₁` of `G` and `C₂` of `H`, for colours with a cancellative addition.

```
def Coloring.boxProd {γ : Type*} [Add γ] [IsCancelAdd γ] (C₁ : G.Coloring γ)
    (C₂ : H.Coloring γ) : (G □ H).Coloring γ :=
  Coloring.mk (fun x => C₁ x.1 + C₂ x.2) ◁ by
    rintro ⟨v, w⟩ ⟨v', w'⟩ (⟨hadj, rfl⟩ | ⟨hadj, rfl⟩)
    · simpa using C₁.valid hadj
    · simpa using C₂.valid hadj
```

If `G` and `H` are both `k`-colourable then so is `G □ H`.

```
theorem Colorable.boxProd {k : ℕ} (h₁ : G.Colorable k) (h₂ : H.Colorable k) :
    (G □ H).Colorable k :=
  ⟨Coloring.boxProd h₁.some h₂.some⟩
```

Combining this with the restricted colourings above, if `G` and `H` are both nonempty then `G □ H` is `k`-colourable iff both `G` and `H` are. (Nonemptiness is needed here: if `H` has no vertices then neither does `G □ H`, so `G □ H` is colourable even when `G` is not.)

```
theorem colorable_boxProd_iff [Nonempty V] [Nonempty W] {k : ℕ} :
    (G □ H).Colorable k ↔ G.Colorable k ∧ H.Colorable k :=
  ⟨fun ⟨C⟩ => ⟨⟨Coloring.ofBoxProdLeft C (Classical.arbitrary W)⟩,
      ⟨Coloring.ofBoxProdRight C (Classical.arbitrary V)⟩⟩,
   fun h => h.1.boxProd h.2⟩
```

Hence if `G` and `H` are both nonempty then the chromatic number of `G □ H` is the maximum of the chromatic numbers of `G` and `H`.

To prove an equality `a = b` in `ℕ∞` it suffices to show that `b ≤ m ↔ a ≤ m` for all `m` (`eq_of_forall_ge_iff`). We then split into the cases `m = ⊤`, which is trivial, and `m` a natural number, where `chromaticNumber_le_iff_colorable` turns both sides into colourability statements.

```
theorem chromaticNumber_boxProd [Nonempty V] [Nonempty W] :
    χ (G □ H) = max (χ G) (χ H) := by
  refine eq_of_forall_ge_iff fun m => ?_
  cases m with
  | top => simp
  | coe m => simp [chromaticNumber_le_iff_colorable, colorable_boxProd_iff]
```

Mathlib has the same result for the disjoint union `G ⊕g H`, where no nonemptiness assumption is needed.

```
#check chromaticNumber_sum -- χ (G ⊕g H) = max (χ G) (χ H)
```

[1] Asking for an additive group instead would be too much: `Fin k` is an additive group only when `k ≠ 0`.

Colouring Hypercubes

Recall that `Q[n + 1]` is defined to be `Q[n] □ T`, where `T` is the complete graph on `Bool`. So `Colorable.boxProd` and induction on n show at once that every hypercube is bipartite: `Q[0]` has no edges, and `T` on `Bool` is 2-colourable.

```
lemma IsBipartite.hypercube (n : ℕ) : Q[n].IsBipartite := by
  induction n with
  | zero => exact ⟨Coloring.mk (fun _ => 0) (by simp)⟩
  | succ n ih => exact ih.boxProd (T : SimpleGraph Bool).colorable_of_fintype
```

Behind this proof is an explicit colouring, but we cannot compute it from the proof: `IsBipartite` is a `Prop`, and `Colorable.boxProd` picks its colourings with `Nonempty.some`. However `Coloring.boxProd` is data, and it works for any colours with a cancellative addition, so we can use the same induction with `Bool` as the colours. Mathlib's addition (`+`) on `Bool` is `Bool.xor` (addition modulo 2), and we colour `T` on `Bool` by the identity (`selfColoring`).

The natural `Bool`-colouring of the hypercube `Q[n]`.

```
def hypercube.bicoloring : (n : ℕ) → Q[n].Coloring Bool
  | 0 => Coloring.mk (fun _ => false) (by simp)
  | n + 1 => (hypercube.bicoloring n).boxProd (T : SimpleGraph Bool).selfColoring
```

So the vertex `(v, b)` of `Q[n + 1]` gets the colour of `v` xor `b`. Unwinding the recursion, each vertex is coloured by the parity of its coordinates.

The parity of a hypercube vertex (the `xor` of its coordinates).

```
def HypercubeVert.parity : {n : ℕ} → HypercubeVert n → Bool
  | 0, _ => false
  | _ + 1, (v, b) => v.parity ^^ b


lemma hypercube.bicoloring_apply {n : ℕ} (v : HypercubeVert n) :
    hypercube.bicoloring n v = v.parity := by
  induction n with
  | zero => rfl
  | succ n ih =>
    rcases v with ⟨v, b⟩
    exact congrArg (· ^^ b) (ih v)
```

By `Coloring.isBipartiteWith`, the hypercube is bipartite with the two parts being the sets of vertices with parity `true` and `false`.

```
example (n : ℕ) : Q[n].IsBipartiteWith {v | v.parity} {v | ¬v.parity} := by
  simpa only [hypercube.bicoloring_apply] using
    (hypercube.bicoloring n).isBipartiteWith
```

Bipartite graphs are 3-clique-free, hence `Q[n]` is 3-clique-free.

```
lemma cliqueFree_hypercube_three (n : ℕ) : Q[n].CliqueFree 3 :=
  (IsBipartite.hypercube _).cliqueFree (by decide)
```

The `Bool`-colouring of `Q[4]`, by parity.

```
#eval hypercube.bicoloring 4 -- {1111 ↦ false, 1110 ↦ true, ..., 0000 ↦ false}
```

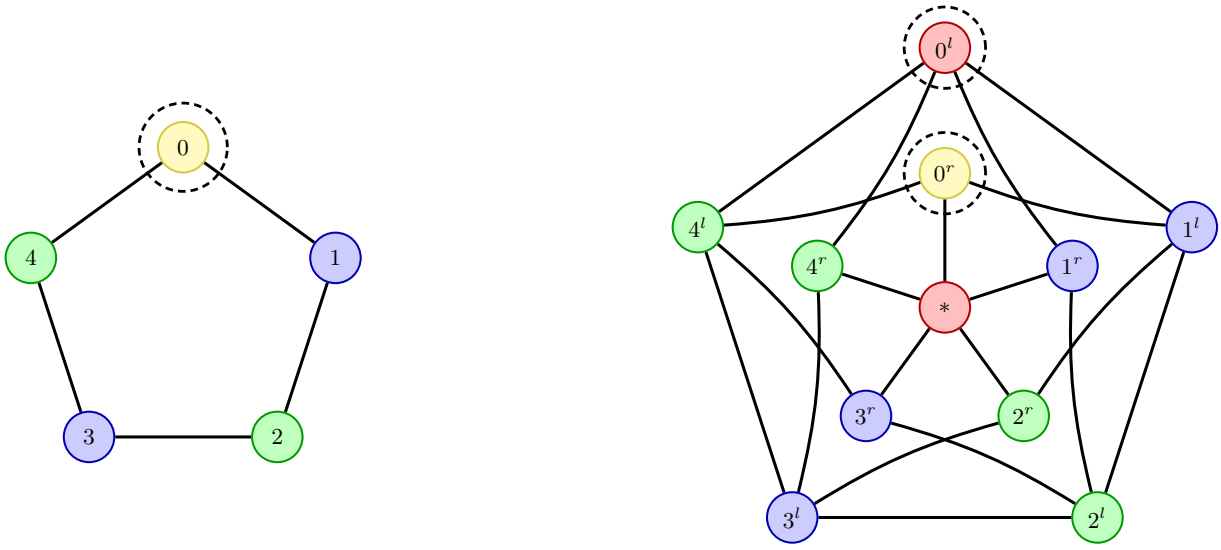
```
end colorings
```

7.6 Colouring Mycielskians

We now come to the main reason for defining the Mycielskian `mμ(G)` (`mycielskian`): its chromatic number is exactly one more than that of `G`. Since `mμ(G)` is also triangle-free whenever `G` is (`CliqueFree.mycielskian`), iterating the construction gives triangle-free graphs of arbitrarily large chromatic number.

To show that `χ mμ(G) = χ G + 1` we will need to construct a `k`-colouring of `G` from a `(k + 1)`-colouring of `mμ(G)` and also construct a `(k + 1)`-colouring of `mμ(G)` from a `k`-colouring of `G`.

The basic idea is that if `mμ(G)` is `(k + 1)`-colourable then we can colour it with the `k + 1` colours of `Fin (k + 1)` (i.e. `{0, 1, ..., k}`) in such a way that the vertex `none` is coloured with colour `k`. Since `none` is adjacent to every vertex in `Vʳ` (the "right" copy of `V`), this ensures that no vertex in `Vʳ` is coloured `k`. We can then construct a `k`-colouring of `G` by colouring each vertex `v : V` with the colour of `vˡ`, if this doesn't equal `k`, and otherwise with the colour of `vʳ` (which by construction is not `k`).



The 3-colouring of `C[5]` induced from a 4-colouring of `mμ(C[5])`

Conversely given a `k`-colouring of `G` we can get a `(k + 1)`-colouring of `mμ(G)` by colouring the "left" and "right" copies of `V` with the same colours as used by their preimages in `G` and colouring `none` with the new colour `k`.

In `Fin (k + 1)` the top colour `k` is `Fin.last k`, and `Fin.castSucc` is the inclusion of the colours `Fin k = {0, ..., k - 1}` into `Fin (k + 1)`. To move the colour of `none` to `k` we use `recolorOfEquiv`, which recolours a colouring along a bijection of the colours, in this case the transposition swapping `k` with the colour of `none`: `Equiv.swap (Fin.last k) (c none)`.

```
section mycielskian_and_edges
variable {V : Type*} {G : SimpleGraph V}
```

Given any `(k + 1)`-colouring of `mμ(G)` this is the equivalent `(k + 1)`-colouring in which the vertex none receives colour `k`.

```
def Coloring.mycielskianLast {k : ℕ} (c : mμ(G).Coloring (Fin (k + 1))) :
    mμ(G).Coloring (Fin (k + 1)) :=
  mμ(G).recolorOfEquiv (Equiv.swap (Fin.last k) (c none)) c
```

No vertex in the "right" copy of V is assigned the colour k in the `Coloring.mycielskianLast`.

```
lemma Coloring.mycielskianLast_ne_last {k : ℕ} (c : mμ(G).Coloring (Fin (k + 1)))
    {v : V} : c.mycielskianLast v.r ≠ Fin.last k :=
  fun hf => c.mycielskianLast.valid (v := v.r) (w := none)
            (by simp) (by rw [hf, Coloring.mycielskianLast]; simp)
```

The natural k-colouring of G induced by a `(k + 1)`-colouring of `mμ(G)`.

```
def Coloring.ofMycielskian {k : ℕ} (c : mμ(G).Coloring (Fin (k + 1))) :
    G.Coloring (Fin k) where
  toFun v := if h : c.mycielskianLast v.l = Fin.last k
             then
               ⟨c.mycielskianLast v.r, Fin.val_lt_last c.mycielskianLast_ne_last⟩
             else
               ⟨c.mycielskianLast v.l, Fin.val_lt_last h⟩
  map_rel' := by
    intro a b hab hf
    -- the edge a – b of G gives three edges of mμ(G)
    have hll : mμ(G).Adj a.l b.l := by simpa using hab
    have hlr : mμ(G).Adj a.l b.r := by simpa using hab
    have hrl : mμ(G).Adj a.r b.l := by simpa using hab
    split_ifs at hf with ha hb hb
    · -- a.l and b.l both have colour k, but are adjacent
      exact c.mycielskianLast.valid hll (by grind)
    · -- hf says that the adjacent vertices a.r and b.l have the same colour
      exact c.mycielskianLast.valid hrl (by grind)
    · -- hf says that the adjacent vertices a.l and b.r have the same colour
      exact c.mycielskianLast.valid hlr (by grind)
    · -- hf says that the adjacent vertices a.l and b.l have the same colour
      exact c.mycielskianLast.valid hll (by grind)
```

`Coloring.ofMycielskian` gives the harder direction; the easier direction is the following colouring.

The `(k + 1)`-colouring of `mμ(G)` induced by a k-colouring of G.

```
def Coloring.mycielskian {k : ℕ} (c : G.Coloring (Fin k)) :
    mμ(G).Coloring (Fin (k + 1)) where
  toFun v :=
    match v with
    | none => Fin.last k
    | v.l | v.r => (c v).castSucc
  map_rel' := by
    -- check all nine kinds of pair: an edge none – v.r has the distinct colours
    -- k and (c v).castSucc; an edge between copies has the colours of two
    -- adjacent vertices of G, which differ by c.valid; the other pairs are
    -- non-edges
    rintro (_ | v | v) (_ | w | w) hab <;>
```

```
      simp_all [c.valid, (Fin.castSucc_ne_last _).symm]
```

G is k-colourable iff $\mu(G)$ is $(k+1)$-colourable.

```
lemma colorable_mycielskian_succ_iff (k : ℕ) :
    μ(G).Colorable (k + 1) ↔ G.Colorable k := by
  constructor <;> intro ⟨c⟩
  · exact ⟨c.ofMycielskian⟩
  · exact ⟨c.mycielskian⟩
```

And so the chromatic number of $\mu(G)$ is $\chi\ G + 1$. We use the same strategy as for `chromaticNumber_boxProd`: compare both sides with an arbitrary $m : \mathbb{N}\infty$. When m is a natural number we split further into the cases $m = 0$ (neither side is at most 0, since $\mu(G)$ has the vertex `none`) and $m = k + 1$, which is `colorable_mycielskian_succ_iff`.

The chromatic number of the Mycielskian of G.

```
theorem chromaticNumber_mycielskian (G : SimpleGraph V) : χ μ(G) = χ G + 1 := by
  refine eq_of_forall_ge_iff fun m => ?_
  cases m with
  | top => simp
  | coe m =>
    rcases m with _ | k
    · simp
    · rw [chromaticNumber_le_iff_colorable, colorable_mycielskian_succ_iff,
        Nat.cast_succ, ENat.add_le_add_iff_right ENat.one_ne_top,
        chromaticNumber_le_iff_colorable]
```

Notice that `chromaticNumber_mycielskian` needs no hypotheses at all. If G has no finite colouring then both sides are $\top$ (in $\mathbb{N}\infty$ we have $\top + 1 = \top$), and if V is empty then $\chi\ G = 0$ while $\mu(G)$ is the single vertex `none`, so $\chi\ \mu(G) = 1$. Compare this with `cliqueNum_mycielskian_of_ne_bot`, which needs both `[Finite V]` and $G \neq \bot$: the $\mathbb{N}$-valued clique number has the junk value 0 when there are arbitrarily large cliques, and if V is nonempty then $\omega\ \bot = 1$ but $\omega\ \mu(\bot) = 2$.

Since the Mycielskian is functorial (`Hom.mapMycielskian`), any homomorphism $G \to g\ H$ lets us turn colourings of $\mu(H)$ into colourings of $\mu(G)$, so $\chi\ \mu(G) \le \chi\ \mu(H)$.

```
#check Hom.mapMycielskian

example {W : Type*} {H : SimpleGraph W} (f : G →g H) (c : μ(H).Coloring V) :
    μ(G).Coloring V :=
  c.comp f.mapMycielskian

example {W : Type*} {H : SimpleGraph W} (f : G →g H) : χ μ(G) ≤ χ μ(H) :=
  chromaticNumber_mono_of_hom f.mapMycielskian
```

The n-th iterated Mycielskian of G has chromatic number $\chi\ G + n$.

```
theorem chromaticNumber_iterMycielskian (n : ℕ) : χ μ^n(G) = χ G + n := by
  induction n with
  | zero => simp
  | succ n ih =>
    rw [iterMycielskian_succ, chromaticNumber_mycielskian, ih, add_assoc]
    -- norm_cast moves the casts ↑ : ℕ → ℕ∞ outwards and cancels them, here
```

```
    -- turning ↑n + 1 into ↑(n + 1); it is the tactic behind exact_mod_cast
    norm_cast
```

Since `K[2]` has chromatic number 2 (`chromaticNumber_top`), `mμ^n(K[2])` has chromatic number `n + 2`. (In particular `C[5] ≃g mμ(K[2])` is 3-chromatic and the Grötzsch graph `mμ(C[5])` is 4-chromatic.)

```
theorem chromaticNumber_iterMycielskian_completeGraph_two (n : ℕ) :
    χ mμ^n(K[2]) = n + 2 := by
  simp [chromaticNumber_iterMycielskian, add_comm]
```

Combining this with `cliqueNum_iterMycielskian_completeGraph_two` gives what we promised, Mycielski's theorem: the graphs `mμ^n(K[2])` all have clique number 2, but their chromatic numbers are unbounded.

Mycielski's theorem: the nth-iterated Mycielskian of `K[2]` has clique number 2 and chromatic number `n + 2`.

```
theorem mycielski (n : ℕ) : ω mμ^n(K[2]) = 2 ∧ χ mμ^n(K[2]) = n + 2 :=
  ⟨cliqueNum_iterMycielskian_completeGraph_two n,
    chromaticNumber_iterMycielskian_completeGraph_two n⟩
```

7.7 Edge Colourings

So far we have coloured vertices. A *proper edge colouring* of `G` colours the edges of `G` so that edges sharing an endpoint receive different colours. Mathlib has no separate notion of this (its `EdgeLabeling` assigns labels to edges with no properness condition), and we don't need one. Two edges share an endpoint exactly when they are adjacent vertices of the line graph `L[G]` (`lineGraph`). So a proper edge colouring of `G` with colours `γ` is just a colouring `L[G].Coloring γ`. The *chromatic index* of `G` is the least number of colours in a proper edge colouring, so it is `χ L[G]`. Mathlib does not have it yet, so we define it as an `abbrev`, which Lean unfolds automatically when it needs to, and introduce the standard notation `χ' G`.

The *chromatic index* of `G`: the least number of colours in a proper edge colouring of `G`, that is, the chromatic number `χ L[G]` of its line graph.

```
noncomputable abbrev chromaticIndex (G : SimpleGraph V) : ℕ∞ := χ L[G]

@[inherit_doc]
scoped notation "χ'" => SimpleGraph.chromaticIndex
```

The edges at a vertex `v` pairwise share the endpoint `v`, so they form a clique in `L[G]`. Hence the chromatic index is at least the degree of any vertex.

```
theorem degree_le_chromaticIndex (v : V) [Fintype (G.neighborSet v)] :
    G.degree v ≤ χ' G := by
  -- the edges s(v, w) at v, as vertices of L[G]
  let f : G.neighborSet v ↪ G.edgeSet :=
    ⟨fun w => ⟨s(v, w), w.2⟩, fun _ _ h => Subtype.ext (Sym2.congr_right.mp
      (congrArg Subtype.val h))⟩
  have hc : L[G].IsClique (univ.map f : Finset G.edgeSet) := by
    simp only [coe_map, coe_univ, Set.image_univ]
```

```
    rintro _ ⟨w, rfl⟩ _ ⟨w', rfl⟩ hne
    exact ⟨hne, v, Sym2.mem_mk_left _ _, Sym2.mem_mk_left _ _⟩
  have := hc.card_le_chromaticNumber
  rwa [card_map, card_univ, card_neighborSet_eq_degree] at this
```

Taking v of maximum degree, the chromatic index of a finite graph is at least its maximum degree $\Delta(G)$.

```
theorem maxDegree_le_chromaticIndex [Fintype V] [DecidableRel G.Adj] :
    Δ(G) ≤ χ' G := by
  cases isEmpty_or_nonempty V
  · simp
  · obtain ⟨v, hv⟩ := G.exists_maximal_degree_vertex
    exact hv ▸ G.degree_le_chromaticIndex v

end mycielskian_and_edges
```

Vizing's theorem [Vizing 1964] says that this lower bound is almost tight: `χ' G ≤ Δ(G) + 1` for every finite graph `G`, and Kőnig's line colouring theorem [Kőnig 1916] says that `χ' G = Δ(G)` for every finite bipartite graph `G`. Neither is in Mathlib yet. The greedy bound of the next section, applied to `L[G]`, whose vertices have degree at most `2 * Δ(G) - 2`, gives the much weaker `χ' G ≤ 2 * Δ(G) - 1`. (Mathlib is also missing many other standard topics of colouring theory such as chromatic polynomials and list colourings.)

7.8 Greedy Colouring*

Greedy colouring means constructing a colouring one vertex at a time, choosing the smallest valid colour at each step, i.e. for the current vertex `v`, the smallest colour not used by any of the neighbours of `v` that we've already coloured. How many colours this uses depends not on the degrees of the vertices but on the number of neighbours that *precede* each vertex in the order in which we colour them, so we start by defining this.

Degree-less-than

In order to define the set of neighbours of a vertex `v` that are less than `v`, we need some kind of ordering on the vertices. The simplest form this can take is an instance of `LT` on the vertex type.

```
section degreeLT
variable {V : Type*} (G : SimpleGraph V) [LT V]
```

The set of neighbours of `v` that are less than `v`.

```
def neighborLTSet (v : V) : Set V := {u | u < v ∧ G.Adj v u}

@[simp]
lemma mem_neighborLTSet (v : V) (u : V) :
    u ∈ G.neighborLTSet v ↔ u < v ∧ G.Adj v u := Iff.rfl
```

The assumption that this set is finite is `Fintype (G.neighborLTSet v)`. With this finiteness assumption we can define the `Finset` version of this set.

The neighbours of v that are less than v as a `Finset`.

```
def neighborLTFinset (v : V) [Fintype (G.neighborLTSet v)] : Finset V :=
  (G.neighborLTSet v).toFinset

@[simp]
lemma mem_neighborLTFinset (v : V) [Fintype (G.neighborLTSet v)] (u : V) :
    u ∈ G.neighborLTFinset v ↔ u < v ∧ G.Adj v u := Set.mem_toFinset
```

Finally the `degreeLT` is just the size of this finite set.

The number of neighbours of v that are less than v.

```
def degreeLT (v : V) [Fintype (G.neighborLTSet v)] : ℕ :=
  (G.neighborLTFinset v).card

end degreeLT
```

Recall from Finite Neighbourhoods (p. 69) that `G.LocallyFinite` is an abbreviation for `∀ v, Fintype (G.neighborSet v)`: the assumption that every vertex in a graph G has finitely many neighbours. We now need its `LT` analogue: every vertex in G has finitely many *earlier* neighbours.

```
abbrev LocallyLTFinite {V : Type*} [LT V] (G : SimpleGraph V) :=
  ∀ n, Fintype (G.neighborLTSet n)
```

Simple Graphs on $\mathbb{N}$

If `G` is a graph with vertex type $\mathbb{N}$ and decidable adjacency, then we get `LocallyLTFinite` for free, since `G.neighborLTSet v` is a subset of the finite set `{0, 1, …, v - 1}`: any vertex `v : ℕ` has at most `v` earlier neighbours! So for now we will restrict our attention to such graphs, and to keep things simple we will also use $\mathbb{N}$ as our set of colours. (Later we will show how to easily extend our greedy colourings to any constructively countable vertex type.)

Any `SimpleGraph ℕ` with decidable adjacency is `LocallyLTFinite`.

```
section greedy
variable (G : SimpleGraph ℕ) [DecidableRel G.Adj]

instance : G.LocallyLTFinite :=
  fun v => Fintype.ofFinset {u < v | G.Adj v u} (by simp)
```

Given an assignment c of colours to `{0, 1, ..., v - 1}`, we want to define the colour of `v` as the smallest natural number that is not used by any of its earlier neighbours. Rather than actually taking c to be a function defined only on `{0, 1, ..., v - 1}`, we will instead take `c : ℕ → ℕ` but then ignore the values of c at vertices `u ≥ v`.

So given `c : ℕ → ℕ` and a vertex `v : ℕ`, how should we define the greedy choice of colour at v? The most direct route uses `Nat.find`: given a decidable predicate `p : ℕ → Prop` and a proof that there exists an `n` such that `p n`, `Nat.find` returns the least such `n`.

```
#check Nat.find      -- Nat.find {p : ℕ → Prop} [DecidablePred p] (H : ∃ n, p n) : ℕ
#check Nat.find_spec -- .. (H : ∃ n, p n) : p (Nat.find H)
#check Nat.find_min  -- .. (H : ∃ n, p n) {m : ℕ} : m < Nat.find H → ¬p m
#check Nat.find_min' -- .. (H : ∃ n, p n) {m : ℕ} (h : p m) : Nat.find H ≤ m
```

To use this for the choice of colour at vertex `v`, we need a proof that at least one natural number is not used by any of the earlier neighbours of `v`. The colours they use form the finite set `(G.neighborLTFinset v).image c`, and since $\mathbb{N}$ is infinite, Mathlib's `Finset.exists_notMem` tells us that some natural number lies outside it.

```
#check Finset.exists_notMem -- .. [Infinite α] (s : Finset α) : ∃ a, a ∉ s
```

In this definition we give the set of used colours a name with `let`. Without this, the set of used colours sits inside the predicate that `Nat.find` tests, and is recomputed for every candidate colour.

Given any `c : ℕ → ℕ` and a vertex `v : ℕ`, `G.greedyChoice c v` is the least colour that is valid at `v` (in the sense that no earlier neighbour of `v` uses this colour).

```
def greedyChoice (c : ℕ → ℕ) (v : ℕ) : ℕ :=
  let used := (G.neighborLTFinset v).image c
  Nat.find used.exists_notMem
```

The colour chosen at vertex `v` is at most the number of *distinct* colours used by its earlier neighbours.

```
lemma greedyChoice_le_card_image (c : ℕ → ℕ) (v : ℕ) :
    G.greedyChoice c v ≤ #((G.neighborLTFinset v).image c) := by
  have hsub : range (G.greedyChoice c v) ⊆ (G.neighborLTFinset v).image c := by
    intro u hu
    exact not_not.mp ◁ Nat.find_min (Finset.exists_notMem _) (mem_range.mp hu)
  simpa using card_le_card hsub
```

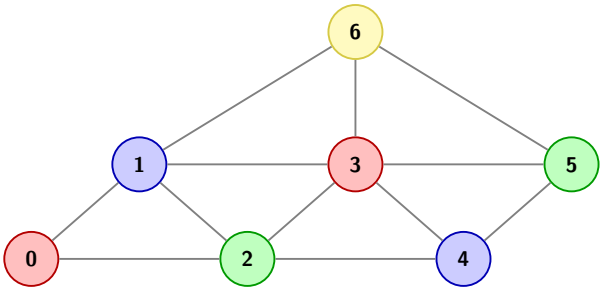
The colour chosen at vertex `v` is never used by any earlier neighbour.

```
lemma greedyChoice_ne_of_mem_neighborLTFinset (c : ℕ → ℕ) {v u : ℕ}
    (hu : u ∈ G.neighborLTFinset v) : G.greedyChoice c v ≠ c u := by
  intro hf
  exact Nat.find_spec (Finset.exists_notMem _) (mem_image.mpr ⟨u, hu, hf.symm⟩)
```

Finally, `greedyChoice c n` only inspects the partial colouring `c` on the earlier neighbours of `n`, so two partial colourings that agree below `n` make the same greedy choice at `n`.

`greedyChoice c n` only depends on the values of `c` below `n`.

```
lemma greedyChoice_congr {c₁ c₂ : ℕ → ℕ} {n : ℕ} (h : ∀ m < n, c₁ m = c₂ m) :
    G.greedyChoice c₁ n = G.greedyChoice c₂ n := by
  simp_all [greedyChoice]
```



n	earlier nbrs	forbidden	chosen
0	$\varnothing$		
1	$\{0\}$		
2	$\{0,1\}$		
3	$\{1,2\}$		
4	$\{2,3\}$		
5	$\{3,4\}$		
6	$\{1,3,5\}$		

Left: a greedy colouring, with colours `0`, `1`, `2` and `3` drawn as red, blue, green and yellow.
Right: for each vertex *n*, its earlier neighbours, the colours they forbid, and the least colour not forbidden

We use `greedyChoice` to define the greedy colouring `greedy` of `G` by *well-founded recursion* on ℕ. To colour a vertex *v* we need to know the colours of its *earlier* vertices `u < v`, so the value of `greedy` at *v* is defined in terms of its values at strictly smaller arguments.

Concretely, we feed `greedyChoice` the "partial colouring" function `fun u ↦ if u < v then greedy u else 42`. The `if u < v` guard makes the recursion well-founded. The recursive call `greedy u` occurs only in the `then` branch, where `u < v` holds, and this is what Lean's termination checker needs to prove that the call is at a strictly smaller argument. The `42` in the `else` branch is just filler; its value is irrelevant, since `greedyChoice` ignores the values of the partial colouring at `u ≥ v`.

The greedy ℕ-colouring of `G` as a function, defined using well-founded recursion on ℕ.

```
def greedy (v : ℕ) : ℕ :=
  G.greedyChoice (fun u ↦ if u < v then greedy u else 42) v
```

Notice that we didn't need to prove that `greedy` is well-defined: by working over ℕ Lean can automatically discharge the termination proof for us. (Under the hood this is using `WellFounded.Nat.fix`.)

```
#print greedy -- .. WellFounded.Nat.fix ..
```

We can unfold `greedy` with `rw [greedy]`. Since the partial colouring used at *v* agrees with `greedy` below *v*, `greedyChoice_congr` lets us drop the `if` altogether. Viewing `G.greedyChoice` as a map from colourings `ℕ → ℕ` to colourings `ℕ → ℕ`, this says that `greedy` is a fixed point of it: `greedy = G.greedyChoice greedy`.

```
lemma greedy_eq_greedyChoice (v : ℕ) : G.greedy v = G.greedyChoice G.greedy v := by
  rw [greedy]
  exact G.greedyChoice_congr fun u hu => ite_eq_left hu
```

The `greedy` colour of *v* is at most the number of distinct colours used by `greedy` on the earlier neighbours of *v*.

```
lemma greedy_le_card_image (v : ℕ) :
    G.greedy v ≤ #((G.neighborLTFinset v).image G.greedy) :=
```

```
  G.greedy_eq_greedyChoice v ▸ G.greedyChoice_le_card_image _ v
```

In particular the `greedy` colour of v is at most the number of earlier neighbours of v, and hence is less than `degreeLT v + 1`.

```
lemma greedy_lt (v : ℕ) : G.greedy v < G.degreeLT v + 1 :=
  Nat.lt_succ_iff.2 ◁ (G.greedy_le_card_image v).trans card_image_le
```

If $u < v$ and v is adjacent to u then the `greedy` colour of v is not equal to the `greedy` colour of u. (The line `variable {G} in` makes G an implicit argument of this lemma only, since it can be read off from *hadj*.)

```
variable {G} in
lemma Adj.greedy_ne_of_gt {u v : ℕ} (hadj : G.Adj v u) (h : u < v) :
    G.greedy v ≠ G.greedy u :=
  G.greedy_eq_greedyChoice v ▸
    G.greedyChoice_ne_of_mem_neighborLTFinset _ (by simp_all)
```

So any pair of adjacent vertices u and v in G receive different colours in the greedy colouring.

```
variable {G} in
lemma Adj.greedy_ne {u v : ℕ} (hadj : G.Adj u v) :
    G.greedy u ≠ G.greedy v := by
  rcases hadj.ne.lt_or_gt with h | h
  · exact (hadj.symm.greedy_ne_of_gt h).symm
  · exact hadj.greedy_ne_of_gt h
```

Together these results pin down the greedy colour exactly when the earlier neighbours of v use precisely the colours `0, 1, ..., k - 1`: the colour of v is then k. (This very special lemma will be useful when we look at the crown graph later.)

```
lemma greedy_eq_of_image_eq_range {v k : ℕ}
    (h : (G.neighborLTFinset v).image G.greedy = range k) : G.greedy v = k := by
  refine le_antisymm (by simpa [h] using G.greedy_le_card_image v) ?_
  by_contra! hlt
  obtain ⟨m, hm, hgm⟩ := mem_image.mp (h ▸ mem_range.2 hlt)
  exact ((G.mem_neighborLTFinset v m).1 hm).2.greedy_ne hgm.symm
```

The Greedy Bound for Graphs on $\mathbb{N}$

With `greedy` in hand we can now formally define the greedy colouring of a graph `G : SimpleGraph ℕ` as a `G.Coloring ℕ`.

The natural greedy $\mathbb{N}$-colouring of a graph `G : SimpleGraph ℕ`.

```
@[simps]
def greedyColoringNat : G.Coloring ℕ where
  toFun := G.greedy
  map_rel' := fun hadj hf => hadj.greedy_ne hf
```

The **greedy colouring bound**: any graph on $\mathbb{N}$ in which every vertex has at most Δ earlier neighbours is $(\Delta + 1)$-colourable.

```
theorem colorable_of_degreeLT_le {Δ : ℕ} (hdeg : ∀ v, G.degreeLT v ≤ Δ) :
    G.Colorable (Δ + 1) :=
```

```
  (colorable_iff_exists_bdd_nat_coloring _).2
    ⟨G.greedyColoringNat, fun v => (G.greedy_lt v).trans_le (by grind [hdeg v])⟩
```

This is stronger than the familiar bound `χ G ≤ Δ(G) + 1` in terms of the maximum degree, since only *earlier* neighbours count. A finite graph whose vertices can be ordered so that each has at most `d` earlier neighbours is called `d`-*degenerate*, so this is the *degeneracy bound* `χ G ≤ d + 1`. As for the maximum degree bound, Brooks' theorem (Brooks' Theorem (p. 304)) shows that a connected graph needs all `Δ(G) + 1` colours only if it is a complete graph or an odd cycle.

Our greedy colouring construction allows us to colour graphs on `ℕ`, for example `pathNat`, the infinite path graph on `ℕ`, and `⊤`, the complete graph on `ℕ`.

```
#eval pathNat.greedyColoringNat 100 -- 0

#eval (⊤ : SimpleGraph ℕ).greedyColoringNat 15 -- 15

end greedy
```

Note that evaluating vertex `30` of `⊤` would take several minutes. We will see why, and fix it, below.

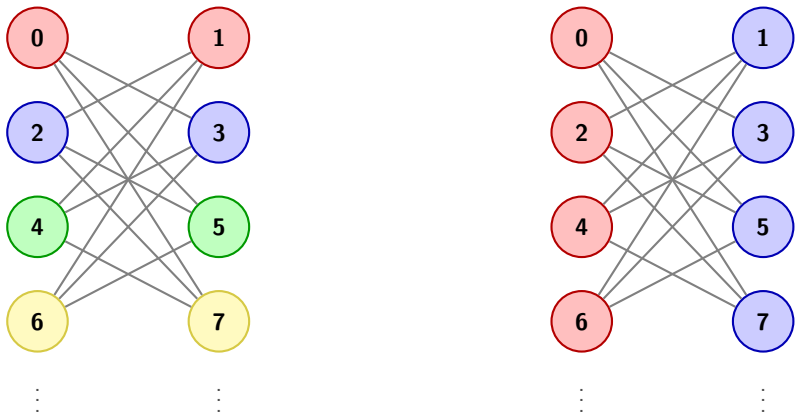
Greedy Colouring vs. the Crown Graph

The classic example of a graph on which greedy colouring performs badly is the balanced complete bipartite graph with the obvious perfect matching removed, known as the *crown graph*, with its vertices ordered left, right, left, right, ...

We will look at its countably infinite incarnation and show that the greedy colouring will use arbitrarily large colours, despite the graph being 2-colourable.

The *crown graph* is the complete bipartite graph on even and odd naturals, minus the perfect matching `2 * n ↦ 2 * n + 1`.

```
@[simps]
def crownGraph : SimpleGraph ℕ where
  Adj a b := a % 2 ≠ b % 2 ∧ a / 2 ≠ b / 2
deriving instance DecidableRel for (crownGraph.Adj)
```



Left: the greedy colouring of `crownGraph` gives vertex `n` colour `n / 2`. Right: an optimal 2-colouring, by parity

The earlier neighbours of n in crownGraph are {1, 3, ..., n - 1}, if n is even, and {0, 2, ..., n - 3} if n is odd. We can see this in examples via #eval and will prove it below.

```
#eval crownGraph.neighborLTFinset 6 -- {1, 3, 5}
#eval crownGraph.neighborLTFinset 11 -- {0, 2, 4, 6, 8}
```

```
lemma neighborLTFinset_crownGraph (n : ℕ) :
    crownGraph.neighborLTFinset n = (range (n / 2)).image
        (fun k => if n % 2 = 0 then 2 * k + 1 else 2 * k) := by
  ext m
  rw [mem_neighborLTFinset, crownGraph_adj, mem_image]
  constructor
  · intro ⟨hlt, hadj⟩
    exact ⟨m / 2, by grind⟩
  · grind
```

The natural Bool-colouring of the crownGraph, by parity.

```
def crownGraph.bicoloring : crownGraph.Coloring Bool where
  toFun n := n % 2 = 0
  map_rel' h := by grind [crownGraph_adj, top_adj]
```

Colouring each vertex by its parity shows that crownGraph is bipartite.

```
example : crownGraph.IsBipartite := crownGraph.bicoloring.colorable
```

The greedy colouring of crownGraph colours vertex n with colour n / 2. Given that degreeLT n = n / 2, this is as bad as it could be! We can see an example via #eval, and prove it below.

```
#eval crownGraph.greedy 20 -- 10
```

```
theorem greedy_crownGraph_eq (n : ℕ) :
    crownGraph.greedy n = n / 2 := by
  -- we use strong induction on n
  induction n using Nat.strong_induction_on with
  | _ n ih =>
  -- by induction the earlier neighbours of n use exactly the colours < n / 2
  apply crownGraph.greedy_eq_of_image_eq_range
  ext c
  simp only [neighborLTFinset_crownGraph, mem_image, mem_range]
  grind
```

Encodable Vertex Types

So far we have computed greedy colourings of graphs on ℕ. Suppose instead that we have a graph G : SimpleGraph V, where V is countable. Informally we might say "let the vertices of G be v_0, v_1, v_2,…", implicitly thinking of G as a SimpleGraph ℕ. How can we formalize this in Lean?

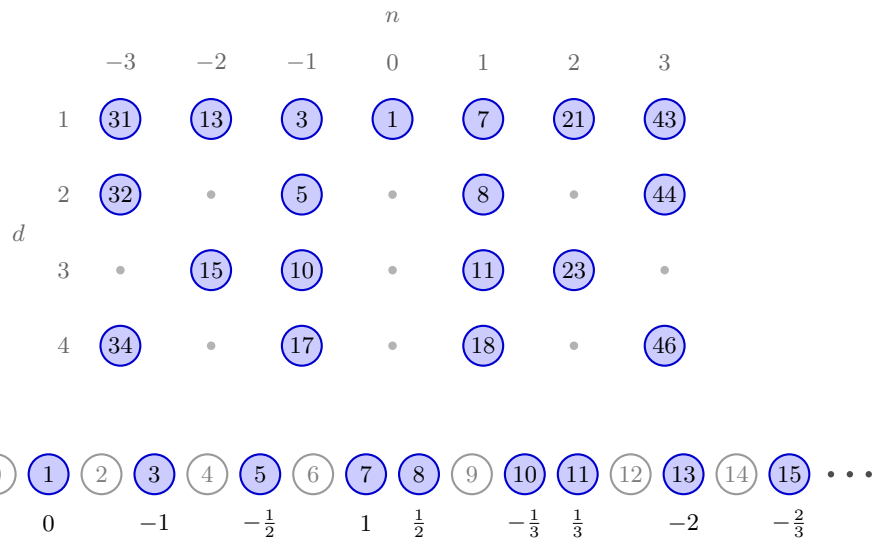
Mathlib has the Encodable type class for constructively countable types (both finite and infinite). It provides an Encodable.encode function that is an injection from V to ℕ together with a partial inverse Encodable.decode : ℕ → Option V, which satisfies decode (encode v) = some v for all v : V.

```
#source Encodable
```

For example, the rationals are encodable.

```
#synth Encodable Rat -- Rat.instEncodable
```

This instance writes a rational in lowest form as `n / d`, with `0 < d`, and encodes it as `Nat.pair (encode n) d`. Here `encode` on ℤ sends `0, -1, 1, -2, 2, …` to `0, 1, 2, 3, 4, …`, and `Nat.pair a b` is `b * b + a` if `a < b` and `a * a + a + b` otherwise, which gives a bijection from ℕ × ℕ to ℕ. So `1 / 2` is encoded as `Nat.pair 2 2 = 8`. Not every natural number is a code: `0 = Nat.pair 0 0` has denominator `0`, and `4 = Nat.pair 0 2` would be `0 / 2`, which is not in lowest terms, so `decode` returns none for both.



Top: the code of `n / d` in column `n`, row `d`. Bottom: the rationals with codes `0` to `15`; hollow numbers are not codes.

```
#eval Encodable.encode (1 / 2 : ℚ)    -- 8
#eval (Encodable.decode 8 : Option ℚ) -- some (1 / 2)
#eval (Encodable.decode 4 : Option ℚ) -- none
#check Nat.pair
```

If *G* has an encodable vertex type then we can construct a greedy colouring of *G* by first encoding its vertices in ℕ and then applying the greedy colouring algorithm to *G* as a graph on ℕ. To build the encoded graph we push *G* forward along `Encodable.encode` using `SimpleGraph.map` (Maps (p. 107)).

The `SimpleGraph ℕ` obtained from a graph `G : SimpleGraph V` with encodable vertex type *V* by pushing it forward along `Encodable.encode`.

```
@[simps!]
protected def encode {V : Type*} [Encodable V] (G : SimpleGraph V) :
    SimpleGraph ℕ := G.map Encodable.encode
```

This captures our informal construction above. [2] However, the `SimpleGraph.map` construction leaves us with existential quantifiers in the adjacency relation.

[2] To be precise we are not claiming that `G.encode` is isomorphic to *G*: in general `encode` need not be surjective, in which case `G.encode` will be a copy of *G* with some additional isolated vertices.

```
section encode
open Encodable
variable {V : Type*} [Encodable V]

example {a b : ℕ} {G : SimpleGraph V} : G.encode.Adj a b ↔
    a ≠ b ∧ ∃ u v, G.Adj u v ∧ encode u = a ∧ encode v = b :=
  Iff.rfl
```

Lean cannot find a `DecidableRel` instance for this relation, even when `G.Adj` is decidable, since it has no way to decide the existential over a possibly infinite `V`. We need decidable adjacency to compute colourings, so we supply an instance ourselves.

Given `[Encodable V]` and `a : ℕ`, the function `Encodable.decode₂ V a` returns `some u` such that `Encodable.encode u = a`, if such a `u : V` exists, and none otherwise. So to decide whether `a b : ℕ` are adjacent in `G.encode` we can **`match`** on `Encodable.decode₂ V a` and `Encodable.decode₂ V b`. If these are `some u` and `some v` then `a` and `b` are adjacent iff `u` and `v` are adjacent in `G`; otherwise one of `a` and `b` is not the encoding of any vertex, so it is isolated. (Plain `Encodable.decode` would not work here: `decode a = some u` does not imply `encode u = a`.)

```
#check decode₂
```

If `G` is a graph on an encodable vertex type with decidable adjacency then its encoded graph also has decidable adjacency.

```
instance {G : SimpleGraph V} [DecidableRel G.Adj] :
    DecidableRel G.encode.Adj := fun a b =>
  match ha : decode₂ V a, hb : decode₂ V b with
  | some u, some v => decidable_of_iff (G.Adj u v) (by
      rw [decode₂_eq_some] at ha hb
      subst ha hb
      simpa [Relation.Map, encode_injective.eq_iff] using G.ne_of_adj)
  | none, _ => isFalse fun ⟨_, u, _, _, hu, _⟩ => by simp [← hu] at ha
  | _, none => isFalse fun ⟨_, _, v, _, _, hv⟩ => by simp [← hv] at hb
```

We made `SimpleGraph.encode` a **`def`** rather than an **`abbrev`** so that this is the only instance. Mathlib already has a `DecidableRel` instance for `G.map f` when `V` is finite, which decides the existential by checking every vertex. Instance search sees through an **`abbrev`**, so for finite `V` that instance would compete with ours, but it does not unfold a **`def`**.

To transfer a greedy colouring of the graph `G.encode` back to a colouring of `G` we use an embedding of `G` into `G.encode`. Mathlib already provides one: for any embedding `f : V ↪ W`, `Embedding.map f G` embeds `G` into `G.map f`, and `Encodable.encode' V` is `Encodable.encode` bundled as an embedding `V ↪ ℕ`.

```
#check Embedding.map
#check encode'
```

The graph embedding of `G` into `G.encode` via `Encodable.encode`.

```
def Embedding.encode (G : SimpleGraph V) : G ↪g G.encode :=
  Embedding.map (encode' V) G
```

We would like to prove the greedy colouring bound for any graph of bounded degree on an `Encodable`, or even a plain `Countable`, vertex type.

We need a couple of simple lemmas. Firstly the earlier neighbourhood of an encoded vertex in the encoded graph is contained in the image of the neighbourhood of that vertex in the original graph under the encoding.

```
lemma neighborLTFinset_encode_subset {G : SimpleGraph V} [DecidableRel G.Adj]
    [G.LocallyFinite] (v : V) : G.encode.neighborLTFinset (encode v)
      ⊆ (G.neighborFinset v).image encode := by
  intro m hm
  obtain ⟨_, u, w, h, hu, rfl⟩ := ((mem_neighborLTFinset ..).mp hm).2
  exact mem_image_of_mem _ (by simpa [encode_injective hu] using h)
```

Hence the `degreeLT` of an encoded vertex is at most the degree of the original vertex.

```
lemma degreeLT_encode_le {G : SimpleGraph V} [DecidableRel G.Adj] [G.LocallyFinite]
    (v : V) : G.encode.degreeLT (encode v) ≤ G.degree v := by
  rw [degreeLT, degree]
  exact (card_le_card (neighborLTFinset_encode_subset v)).trans card_image_le
```

Using these lemmas we can now give the greedy `(Δ + 1)`-colouring of a graph with maximum degree `Δ` on an encodable vertex type with decidable adjacency. (Note that we don't actually refer to `maxDegree` below, since this would require a mathematically unnecessary `Fintype V` instance. Instead we assume that all vertices have finite neighbourhoods using `G.LocallyFinite` and a bound `Δ : ℕ` such that `∀ v, G.degree v ≤ Δ`.)

The greedy `Fin (Δ + 1)` colouring of a graph with bounded degree `Δ`.

```
def greedyColoringFin {Δ : ℕ} {G : SimpleGraph V} [DecidableRel G.Adj]
    [G.LocallyFinite] (hdeg : ∀ v, G.degree v ≤ Δ) : G.Coloring (Fin (Δ + 1)) where
  toFun v := ⟨G.encode.greedy (encode v), by
    grind [G.degreeLT_encode_le v, G.encode.greedy_lt (encode v)]⟩
  map_rel' h := Fin.ne_of_val_ne ((Embedding.encode G).map_rel_iff.mpr h).greedy_ne

end encode
```

We can also prove (classically) that such a colouring exists without any assumptions about encodability or decidability. For this a `Countable` instance, together with the degree bound, suffices.

```
theorem colorable_of_degree_le_of_countable {V : Type*} {G : SimpleGraph V} {Δ : ℕ}
    [Countable V] [G.LocallyFinite] (hdeg : ∀ v, G.degree v ≤ Δ) :
    G.Colorable (Δ + 1) := by
  classical
  rcases nonempty_encodable V with ⟨_⟩
  exact ⟨greedyColoringFin hdeg⟩
```

Later we will use the **De Bruijn–Erdős theorem** (p. 256) to show that the countable assumption is also superfluous, but for this we need to understand `Subgraphs`.

Making `greedy` Efficient*

Our definition of `greedy` has allowed us to prove the greedy colouring bound, but how good is `greedy` as an *algorithm*? As promised, we now investigate why evaluating it on dense graphs is slow, and what to do about this. If you have no interest in efficient algorithms or debugging, skip to Computing Colourings of Encodable Graphs (p. 182): all it needs from this section is that, by the end of it, `#eval` runs a faster implementation of `greedy`.

Debugging `greedy`

Lean provides the debugging tool `dbg_trace`: the expression `dbg_trace "msg"; e` evaluates to `e`, printing `msg` as a side effect each time it is evaluated. We make a version of `greedy` that prints a `*` each time it is called.

```
section evaluating
variable (G : SimpleGraph ℕ) [DecidableRel G.Adj]
```

A copy of `greedy` that prints one `*` per call.

```
def greedyTraced (v : ℕ) : ℕ :=
  dbg_trace "*";
  G.greedyChoice (fun u ↦ if u < v then greedyTraced u else 42) v
```

Rather than printing thousands of stars and asking you to count them, we can capture the trace inside Lean: `IO.FS.withIsolatedStreams` runs an `IO` action with its output redirected into a `String`. To count the `*`s in it we turn it into a list of characters with `String.toList` and use `List.count`. We pass the computation as a function `f : Unit → α` rather than as a value: Lean evaluates arguments before calling a function, so a value would already have printed its stars before the capture starts.

Evaluate `f ()` and return the number of `*`s it prints with `dbg_trace` (here: the number of calls).

```
def withTraceCount {α : Type} (f : Unit → α) : IO ℕ := do
  let (out, _) ← IO.FS.withIsolatedStreams (IO.lazyPure f)
  return out.toList.count '*'
```

We can now check the number of calls made to `greedyTraced` when colouring each vertex `0, 1, ..., 8` of the complete graph on `ℕ`. Before looking at the trace counts below, can you work out how bad it is?

```
#eval (List.range 9).mapM fun n =>
  withTraceCount fun _ => (⊤ : SimpleGraph ℕ).greedyTraced n
```

The number of calls doubles with each vertex: colouring vertex `8` of the complete graph takes `256` calls, and colouring vertex `30` would take more than a billion. The reason for this is that we don't store anything: each time the algorithm needs the colour of an earlier vertex `m`, it recomputes `greedy m` completely from scratch. Writing `T(n)` for the number of calls made when colouring `n`, we have `T(n) = 1 + ∑ T(m)`, where the sum runs over all earlier neighbours `m` of `n`. On the complete graph `⊤` every earlier vertex is a neighbour, so `T(n) = 1 + T(0) + T(1) + ⋯ + T(n - 1) = 2 ^ n`. Note that it would be even worse without the `let` in `greedyChoice`: each of the `n + 1` colours that `Nat.find` tests at vertex `n` would recompute the colours of all the earlier neighbours, and colouring vertex `8` would take `2,345,851` rather than `256` calls.

Memoization

To speed things up we build a list of colours `[c 0, c 1, …, c (n - 1)]` by ordinary structural recursion on `n`, so that each colour is computed exactly once, and then look up the entry we need. This is a simple form of *memoization*.

To read entries off a list we use `List.getD`: `l.getD m d` is the `m`-th entry of `l` (counting from `0`) if `m < l.length`, and the default value `d` otherwise.

```
#check List.getD
```

To compute the next colour we pass `greedyChoice` the partial colouring `fun m => l.getD m 42`, where `l` is the list built so far. As in `greedy`, the default value `42` is just filler: `greedyChoice` only looks at the colours of earlier vertices, and these are all entries of `l`.

The list of the first `n` greedy colours `[c 0, c 1, …, c (n - 1)]`, built by structural recursion.

```
def greedyList : ℕ → List ℕ
  | 0 => []
  | n + 1 =>
    let l := greedyList n
    l ++ [G.greedyChoice (fun m => l.getD m 42) n]
```

The list `greedyList n` has length `n`.

```
@[simp]
lemma length_greedyList (n : ℕ) : (G.greedyList n).length = n := by
  induction n <;> simp_all [greedyList]
```

To colour vertex `n` we build `G.greedyList (n + 1)` and use `List.getD` again to read off its last entry.

The memoized greedy colouring of `G`: build the list of the first `n + 1` colours and look up the last entry.

```
def greedyMemo (n : ℕ) : ℕ :=
  (G.greedyList (n + 1)).getD n 42
```

We now want to prove that this new version computes the same colours as `greedy`. The list is correct: its `m`-th entry is `greedy m`. We use induction on the length of the list: extending the list by one entry preserves the earlier entries (`List.getD_append`), and the new entry is the correct greedy choice by `greedyChoice_congr`.

```
lemma getD_greedyList {n m : ℕ} (hm : m < n) :
    (G.greedyList n).getD m 42 = G.greedy m := by
  induction n generalizing m with
  | zero => grind
  | succ n ih =>
    rcases Nat.lt_succ_iff_lt_or_eq.mp hm with h | rfl
    · rw [greedyList, List.getD_append _ _ _ _ (by simpa using h)]
      exact ih h
    · rw [greedyList, List.getD_append_right _ _ _ _ (by simp), length_greedyList,
        Nat.sub_self, List.getD_cons_zero, greedy_eq_greedyChoice]
      exact G.greedyChoice_congr fun k hk => ih hk

lemma greedyMemo_eq_greedy (n : ℕ) : G.greedyMemo n = G.greedy n :=
  G.getD_greedyList (Nat.lt_succ_self n)
```

Now each colour is computed exactly once, so colouring vertex `n` makes only `n + 1` calls to `greedyChoice`, whatever the graph. We can check this by tracing `greedyList` as we traced `greedy`. Only the `n + 1` branch calls `greedyChoice`, so that is where we print the `*`.

A copy of `greedyList` that prints one `*` per call to `greedyChoice`.

```
def greedyListTraced : ℕ → List ℕ
  | 0 => []
  | n + 1 =>
    dbg_trace "*";
    let l := greedyListTraced n
    l ++ [G.greedyChoice (fun m => l.getD m 42) n]
```

To colour vertex n we look up entry n of `greedyListTraced (n + 1)`, just as `greedyMemo` does. On the complete graph, colouring the vertices `0, 1, ..., 8` now takes `1, 2, ..., 9` calls, rather than `1, 2, 4, ..., 256`.

```
#eval (List.range 9).mapM fun n =>
  withTraceCount fun _ => ((⊤ : SimpleGraph ℕ).greedyListTraced (n + 1)).getD n 42
```

The number of calls is linear, although the running time is not, since appending to a list and looking up its entries take time proportional to its length.

Correct and Efficient: `csimp`

In `greedyMemo_eq_greedy` above we proved that both definitions compute the same colouring: `greedy` has the cleanest definition and proofs due to Lean's support for well-founded recursion on ℕ and the simplicity of `Nat.find`, but `greedyMemo` is much more efficient. So which should we have chosen to implement?

It turns out that we don't have to choose. Lean's `csimp` attribute ("compiler simp") registers a *proved* equation between two functions and makes the compiler use the right-hand side whenever it compiles code that mentions the left-hand side. So by marking the next result with `@[csimp]`, every definition and proof below refers to `greedy`, while every `#eval` compiled from now on runs the efficient memoized implementation.

```
@[csimp]
theorem greedy_eq_greedyMemo : greedy = greedyMemo := by
  funext G _ n
  exact (G.greedyMemo_eq_greedy n).symm
```

Suddenly `greedy` is much faster!

```
#eval (⊤ : SimpleGraph ℕ).greedy 200 -- 200
```

```
end evaluating
```

To be clear: `@[csimp]` does not change `greedy` itself; it changes how code that mentions `greedy` is compiled from now on. Code compiled earlier keeps calling the exponential implementation, so `greedyColoringNat` and `greedyColoringFin`, which were defined before the attribute was registered, are still slow. This is why we define the computable colourings of the next section only now: their definitions refer to `greedy` exactly as before, but their compiled code runs `greedyMemo`.

Computing Colourings of Encodable Graphs

With `greedy` now faster, the same construction as `greedyColoringFin` gives computable greedy colourings of graphs on any encodable vertex type with decidable adjacency.

The natural greedy $\mathbb{N}$-colouring of a graph `G : SimpleGraph V` on an `Encodable` vertex type via its `Encodable` instance.

```
@[simps]
def greedyColoring {V : Type*} [Encodable V] (G : SimpleGraph V)
    [DecidableRel G.Adj] : G.Coloring ℕ where
  toFun v := G.encode.greedy (Encodable.encode v)
  map_rel' h := ((Embedding.encode G).map_rel_iff.mpr h).greedy_ne
```

Most finite and countably infinite types have `Encodable` instances in Mathlib, but for `ZMod n` and our Mycielskian and hypercube vertex types we need to provide them ourselves. Each of these types is defined by cases or recursion on `n` (`ZMod 0` is $\mathbb{Z}$ and `ZMod (n + 1)` is `Fin (n + 1)`), and instance search does not unfold such definitions. So we unfold them by hand and let `infer_instance` handle each case: the same pattern as the instances `hypercubeDecidableEq` and `hypercubeFintype` for `HypercubeVert n`.

```
instance (n : ℕ) : Encodable (ZMod n) := by
  cases n <;> unfold ZMod <;> infer_instance

instance iterMycielskianEncodable {V : Type*} [Encodable V] (n : ℕ) :
    Encodable (μᵥ^n(V)) := by
  induction n <;> unfold MycielskianVert <;> infer_instance

instance hypercubeEncodable (n : ℕ) : Encodable (HypercubeVert n) := by
  induction n <;> unfold HypercubeVert <;> infer_instance
```

We can now compute greedy colourings of many concrete graphs: `K₇`, `C₇`, `K₅,₅`, `paleyGraph₁₇`, `Q₄`, `μ(C₅)`, `μ⁴(C₅)`.

```
#eval K[7].greedyColoring       -- {0 ↦ 0, 1 ↦ 1, 2 ↦ 2, 3 ↦ 3, 4 ↦ 4, 5 ↦ 5, 6 ↦ 6}
#eval C[7].greedyColoring       -- {0 ↦ 0, 1 ↦ 1, 2 ↦ 0, 3 ↦ 1, 4 ↦ 0, 5 ↦ 1, 6 ↦ 2}
#eval K[5, 5].greedyColoring    -- {Sum.inl 0 ↦ 0, Sum.inl 1 ↦ 0, ..., Sum.inr 4 ↦ 1}
#eval Q[4].greedyColoring       -- {1111 ↦ 0, 1110 ↦ 1, ..., 0001 ↦ 1, 0000 ↦ 0}
#eval μ(C[5]).greedyColoring    -- {* ↦ 0, 0ˡ ↦ 0, ..., 4ˡ ↦ 3, ..., 4ʳ ↦ 1}
#eval μ^4(C[5]).greedyColoring -- {* ↦ 0, ..., 4ˡˡˡˡ ↦ 6,..., 4ʳʳʳʳ ↦ 1}
#eval (paleyGraph (ZMod 17)).greedyColoring  -- {0 ↦ 0, ..., 11 ↦ 5, ..., 16 ↦ 4}
#eval (⊤ : SimpleGraph ℕ).greedyColoring 200 -- 200
```

Unlike on the crown graph, `greedy` does well on all of these examples: each colouring above is optimal. For instance the colouring of `μ^4(C[5])` uses the colours `0, ..., 6`, and since `χ C[5] = 3`, `chromaticNumber_iterMycielskian` shows that 7 colours are needed. Similarly the 6 colours used for `paleyGraph (ZMod 17)` are optimal: its independence number is 3 (see the exercises of Cliques (p. 127)), so `natCard_le_chromaticNumber_mul_indepNum` gives `17 ≤ χ (paleyGraph (ZMod 17)) * 3`, forcing at least 6 colours.

Now that `greedy` is faster we can also evaluate it much further along the crown graph.

```
#eval crownGraph.greedy 101 -- 50
```

7.9 Exercises

1. Show that the cycle graph `C[5]` is not 2-colourable. Hint: `chromaticNumber_cycleGraph_of_odd`.

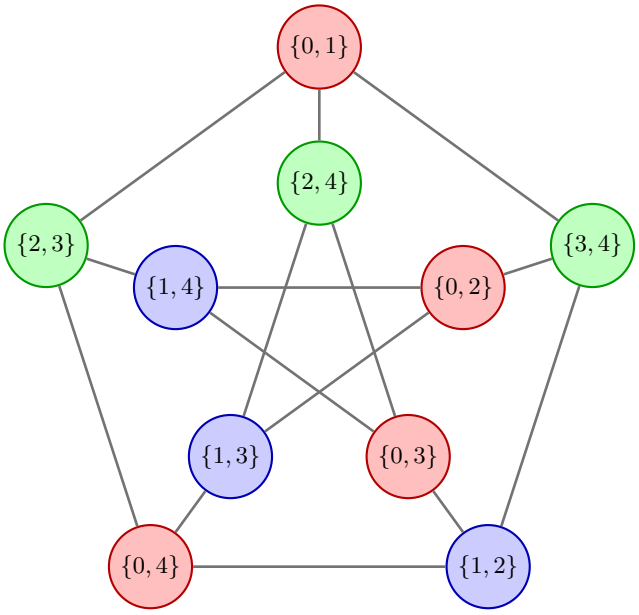
```
open Finset
variable {V : Type*} {G : SimpleGraph V}

example : ¬C[5].Colorable 2 := by
  sorry
```

2. Show that a graph `G` on `n` vertices satisfies `n ≤ χ G * χ Gᶜ`. Hint: combine `natCard_le_chromaticNumber_mul_indepNum` with `cliqueNum_compl` and `cliqueNum_le_chromaticNumber`.

```
example : Nat.card V ≤ χ G * χ Gᶜ := by
  sorry
```

3. Recall the Kneser graph `Kn[n, k]`: the graph whose vertices are the `k`-element subsets of `Fin n = {0, ..., n - 1}` with edges between disjoint subsets. Show that it has an `(n - 2 * k + 2)`-colouring for `2 * k ≤ n` and `0 < k`. The colouring is given below: colour a vertex by its least element, merging all the large colours into one. This is the hardest exercise in this chapter: the case where both vertices receive the merged colour needs a pigeonhole argument, which is `isIndepSet_kneserGraph_of_card_lt` from Cliques (p. 127). Here `s.min' h` is the least element of a nonempty finset `s`, and its basic properties are `Finset.min'_mem` and `Finset.min'_le`.



The colouring below for the `petersenGraph` (`Kn[5, 2]`), with `5 - 2 * 2 + 2 = 3` colours: a vertex is red if its least element is `0`, blue if its least element is `1`, and green (the merged colour) otherwise

The `(n - 2 * k + 2)`-colouring of the Kneser graph `Kn[n, k]`.

```
def kneserGraph.coloring' (n k : ℕ) (hn : 2 * k ≤ n) (hk : 0 < k) :
    Kn[n, k].Coloring (Fin (n - 2 * k + 2)) where
  toFun s :=
    -- A vertex is s : Finset (Fin n) of size k. Since 0 < k, s is
    -- nonempty. We colour s by its minimum element m.
    let m : Fin n := s.val.min' (card_pos.mp (by rw [s.property]; exact hk))
    -- Minima below n - 2 * k + 1 keep their own colour; every larger minimum
    -- shares the last colour n - 2 * k + 1.
    if h : m.val < n - 2 * k + 1 then
      ⟨m.val, by grind⟩
    else
      ⟨n - 2 * k + 1, by grind⟩
  map_rel' := by
    sorry
```

4. Show that if `2 * k ≤ n`, `0 < k` and `Kn[n, k]` is m-colourable then $n \le k * m$, so that $n \le k * \chi$ `Kn[n, k]`. Hint: use `Colorable.natCard_le_mul_indepNum` and the independence number of the Kneser graph, `indepNum_kneserGraph`, from the previous chapter; `Nat.add_one_mul_choose_eq` will help with the binomial coefficients.

```
theorem le_mul_of_colorable_kneserGraph' {n k m : ℕ} (hn : 2 * k ≤ n) (hk : 0 < k)
    (h : Kn[n, k].Colorable m) : n ≤ k * m := by
  sorry
```

5. Use the previous two exercises to show that the Petersen graph `Kn[5, 2]` has chromatic number 3.

```
example : χ Kn[5, 2] = 3 := by
  sorry
```

6. Show that the chromatic number of the hypercube `Q[n + 1]` is 2. Hint: `chromaticNumber_eq_two_iff` reduces this to facts proved in this chapter.

```
example (n : ℕ) : χ Q[n + 1] = 2 := by
  sorry
```

7. Prove the claim from the chapter that for `G : SimpleGraph ℕ` a vertex v has at most v earlier neighbours.

```
lemma degreeLT_le_self' (G : SimpleGraph ℕ) (v : ℕ) [Fintype (G.neighborLTSet v)] :
    G.degreeLT v ≤ v := by
  sorry
```

8. Show that the greedy colouring really is greedy: if k is less than the greedy colour of v, then some earlier neighbour of v has greedy colour k. Hint: rewrite with `greedy_eq_greedyChoice` and use `Nat.find_min`.

```
lemma exists_greedy_eq_of_lt_greedy' (G : SimpleGraph ℕ) [DecidableRel G.Adj]
    {v k : ℕ} (hk : k < G.greedy v) :
    ∃ u ∈ G.neighborLTFinset v, G.greedy u = k := by
  sorry
```

9. An ordering of the vertices of `G` is a *perfect elimination ordering* if the earlier neighbours of every vertex form a clique. (For example, ordering the vertices of an interval graph by the left endpoints of their intervals gives such an ordering.) Show that if the natural ordering of `G` is a perfect elimination ordering and `G` has no `(k + 1)`-clique then `G` is `k`-colourable, so that the greedy colouring uses no more colours than the largest clique needs. Hint: `v` together with its earlier neighbours forms a `(degreeLT v + 1)`-clique, so `CliqueFree.mono` bounds `degreeLT v`; now use `colorable_of_degreeLT_le`.

```
example (G : SimpleGraph ℕ) {k : ℕ}
    (hG : G.CliqueFree (k + 1)) (h : ∀ v, G.IsClique (G.neighborLTSet v)) :
    G.Colorable k := by
  sorry
```

10. Show that if the greedy colour of `v` is `k` then `G` contains an increasing path `f 0 < f 1 < ⋯ < f k = v`, in which consecutive vertices are adjacent. (We describe the path by a function `f : ℕ → ℕ`, since walks only appear in the next chapter.) Hint: prove the statement for all `v` with `G.greedy v = k` by induction on `k`. Exercise 8 gives the vertex `u` before `v`, and `Function.update f (k + 1) v` extends the path `f` ending at `u`.

```
lemma exists_increasing_path_greedy' (G : SimpleGraph ℕ) [DecidableRel G.Adj]
    (v : ℕ) : ∃ f : ℕ → ℕ, f (G.greedy v) = v ∧
      ∀ i < G.greedy v, f i < f (i + 1) ∧ G.Adj (f i) (f (i + 1)) := by
  sorry
```

11. Deduce that if `G` has no increasing path on `k + 1` vertices then `G` is `k`-colourable. In particular, a graph in which no path has `k + 1` vertices is `k`-colourable. Hint: as in `colorable_of_degreeLT_le`, use `colorable_iff_exists_bdd_nat_coloring` with `greedyColoringNat`.

```
example (G : SimpleGraph ℕ) {k : ℕ}
    (h : ¬∃ f : ℕ → ℕ, ∀ i < k, f i < f (i + 1) ∧ G.Adj (f i) (f (i + 1))) :
    G.Colorable k := by
  sorry
```

12. Show that the greedy colouring of the complete graph on `ℕ` colours vertex `n` with colour `n`. (See `greedy_crownGraph_eq` for a similar proof using `Nat.strong_induction_on`.)

```
example (n : ℕ) : (⊤ : SimpleGraph ℕ).greedy n = n := by
  sorry
```

13. The *infinite star* `starNat` joins `0` to every other natural number. Show that `0` has infinitely many neighbours, so that `starNat` is not locally finite and the bound `colorable_of_degree_le_of_countable` does not apply, but that `starNat` is nevertheless 2-colourable, using `colorable_of_degreeLT_le`: every vertex has at most one earlier neighbour.

The infinite star on $\mathbb{N}$: the vertex `0` is adjacent to every other vertex.

```
abbrev starNat : SimpleGraph ℕ where
  Adj a b := (a = 0 ∧ b ≠ 0) ∨ (b = 0 ∧ a ≠ 0)

example : (starNat.neighborSet 0).Infinite ∧ starNat.Colorable 2 := by
  sorry
```

14. The crown graph shows that a bad ordering of the vertices can make the greedy colouring use arbitrarily many colours. Conversely, show that a good ordering makes it optimal: if *C* is a colouring of *G* in which every vertex comes before all neighbours of larger colour, then the greedy colour of each vertex *v* is at most *C v*. Hint: use strong induction on *v* and `greedy_le_card_image`, showing that the greedy colours of the earlier neighbours of *v* all lie in `range (C v)`.

```
example (G : SimpleGraph ℕ) [DecidableRel G.Adj] (C : G.Coloring ℕ)
    (hC : ∀ u v, G.Adj u v → C u < C v → u < v) (v : ℕ) : G.greedy v ≤ C v := by
  sorry
```

8 Walks

Walks are a fundamental concept in graph theory. In this chapter we look at how they are defined in Mathlib. We consider operations on walks such as concatenation (`Walk.append`) and reversal (`Walk.reverse`), as well as the `Walk.length` (number of edges) and `Walk.support` (list of vertices visited) of walks. We then meet the kinds of walks that arise most frequently in graph theory: trails, paths, circuits and cycles, together with Eulerian trails and Hamiltonian paths and cycles. Along the way we show that a graph is bipartite iff it has no odd cycle (`isBipartite_iff_forall_isCycle_even`).

The type of a walk records its endpoints, which in many situations is very useful, but it is also where dependent types show their less appealing side. In order to deal with type mismatches we need to introduce some "plumbing" that has no informal analogue (`Walk.Nil`, `Walk.copy` and `Walk.transfer`).

We then look at maps of walks along graph homomorphisms, and return to our running examples: Hamming paths and the Gray code in the hypercube, and walks in the Mycielskian. Finally we look at Hamiltonian graphs: we define Pósa rotations and prove the Bondy–Chvátal theorem (`isHamiltonian_closure_iff`), with Ore's theorem (`IsOreGraph.isHamiltonian`) and Dirac's theorem (`isHamiltonian_of_card_le_two_mul_minDegree`) as corollaries.

8.1 Constructing Walks

Informally a *walk* in a graph `G` is a sequence of adjacent vertices $v_0, v_1, \ldots, v_k$, so the obvious way to formalize a walk in a `SimpleGraph V` seems to be as a `List V` together with a proof that consecutive vertices are adjacent. You may like to try this and see how far you get: defining functions on such walks, for example concatenation, is possible but painful. As we will see, Mathlib takes a slightly different approach.

In Mathlib walks are defined as an inductive type that encodes the endpoints directly in the type. So a walk is always a walk from `u` to `v` for some explicit `u` and `v`. Such a walk in the graph `G` has type `G.Walk u v`. So `G.Walk` is not a single type but a *family* of types, one for each pair of endpoints, and walks with different endpoints live in different types. In many respects this is an advantage, but it does mean that we sometimes need to deal with type mismatches that have no counterpart in informal graph theory. We look at this in detail in Fixing Type Mismatches (p. 203).

Notice also that `G.Walk u v` is a `Type`, not a `Prop`: two walks with the same endpoints can be different, and we can perform computations with them. The proposition that there is *some* walk from `u` to `v` is `G.Reachable u v`, which we meet in Reachability (p. 196).

```
open Walk
section basics
variable {V : Type*} {G : SimpleGraph V} {u v w : V}
```

```
#source Walk  -- inductive Walk : V → V → Type u
              --   | nil {u : V} : Walk u u
              --   | cons {u v w : V} (h : G.Adj u v) (p : Walk v w) : Walk u w
```

So SimpleGraph.Walk has two constructors: Walk.nil and Walk.cons (in this respect it is very similar to List, which may be worth reviewing if it is unfamiliar).

```
#source List
```

The Walk.nil constructor returns a walk consisting of a single vertex *u* with no edges.

```
#check Walk.nil  -- {G : SimpleGraph V} {u : V} : G.Walk u u
example : G.Walk u u := Walk.nil
```

Notice that the constructor Walk.nil takes no explicit arguments. Its implicit arguments (namely *V*, *G* and *u*) are all determined by the type *G*.Walk *u u* of the walk we are building.

The Walk.cons constructor takes an edge *h* : *G*.Adj *u v* and an existing walk *p* : *G*.Walk *v w* and creates the walk *G*.Walk *u w* by adding the edge to the start of the walk *p*.

```
#check Walk.cons -- .. (h : G.Adj u v) (p : G.Walk v w) : G.Walk u w
```

Recall the path graph we defined on $\mathbb{N}$.

```
#check pathNat
```

We give some examples of walks in this graph below. Notice that we have to specify very little: Lean can infer many of the details from the type of walk we are building in each case.

First we have the nil walk from 7 to 7. (We have opened the namespace Walk in this chapter so we can use nil rather than Walk.nil.)

```
example : pathNat.Walk 7 7 := nil
```

Here the vertex 7 was determined by the type of walk so is not an explicit argument to the constructor nil.

In our next example we are building a walk from 2 to 3, as a cons h nil walk. So the implicit *u* input to nil is 3 and the single edge h must be *G*.Adj 2 3, which Lean can infer (and simp can prove).

```
example : pathNat.Walk 2 3 := cons (by simp) nil
```

In the next example we are building a cons h_1 (cons h_2 nil) walk from 3 to 5, so the first edge must be h_1 : *G*.Adj 3 *v* and the second edge must be h_2 : *G*.Adj *v* 5, for some vertex *v*. We need to tell Lean what *v* is in at least one case, but once we do this for the first edge the second edge is determined (again the actual existence of both edges is easily proved by simp). There are a few ways to do this; we show two below.

First we use type ascription notation (by simp : ...) which tells Lean what we want to prove and then gets simp to prove it.

```
example : pathNat.Walk 3 5 :=
  cons (by simp : (pathNat).Adj 3 4) ◁ cons (by simp) nil
```

Alternatively we can use a variant of the `cons` constructor called `cons'` that takes the three vertices as explicit arguments. Notice the difference in the signatures of `cons` and `cons'` below.

```
#check cons   -- .. {u v w : V} (h : G.Adj u v) (p : G.Walk v w) : G.Walk u w
#check cons'  -- .. (u v w : V) (h : G.Adj u v) (p : G.Walk v w) : G.Walk u w

example : pathNat.Walk 3 5 := cons' _ 4 _ (by simp) ◁ cons (by simp) nil
```

Even though `cons'` takes 3 explicit vertex arguments, we actually only need to give one of them explicitly, since the other two can be inferred from the type of walk we are building and the edge we are using.

Of course a walk may have repeated edges or vertices. So we can build the following walk from 3 to 5, which we write informally as `3 – 2 – 3 – 4 – 5 – 6 – 5`.

```
private def meander35 : pathNat.Walk 3 5 :=
  cons' _ 2 _ (by simp) ◁ cons' _ 3 _  (by simp) ◁ cons' _ 4 _ (by simp)
    ◁ cons' _ 5 _ (by simp) ◁ cons' _ 6 _ (by simp) ◁ cons (by simp) nil
```

The key advantage of defining walks as an inductive type (as opposed to as a `List V` with proofs of adjacency of consecutive vertices) is that many things are simply *correct by construction*. The inductive nature of the type allows for simple inductive definitions and proofs.

For example, the concatenation of two walks p_1 and p_2 is only defined when the walks are compatible: i.e. if p_1 is a walk from u to v then p_2 must be a walk from v to some w, and the result is then automatically a walk from u to w.

```
#check Walk.append  -- ... : G.Walk u v → G.Walk v w → G.Walk u w
#source Walk.append -- def append {u v w : V} : G.Walk u v → G.Walk v w → G.Walk u w
                    --   | nil, q => q
                    --   | cons h p, q => cons h (p.append q)
```

The *length* of a walk is the number of edges it contains. Again this has a simple inductive definition.

```
#source Walk.length
```

The length of the meandering walk `meander35` defined above is 6.

```
#eval meander35.length  -- 6
```

Since `Walk.append` is defined inductively on its first argument, proofs involving `append` will often follow by induction on this argument. For example: the length of the concatenation of two walks is the sum of their lengths. (This is also in Mathlib as `Walk.length_append`.)

```
example (p : G.Walk u v) (q : G.Walk v w) :
    (p.append q).length = p.length + q.length := by
  induction p with
  | nil =>
    rw [add_comm]
    rfl
  | cons h p ih =>
    rw [cons_append, length_cons, ih, length_cons, add_right_comm]
```

If we want to add an edge at the end of a walk rather than at the start, we can use `concat` instead of `cons`.

```
#source Walk.concat
```

To build a walk from i to $i + j$ in `pathNat` we can use `concat` to define it inductively on j.

The walk from i to $i + j$ in `pathNat`.

```
def pathNat.path (i j : ℕ) : pathNat.Walk i (i + j) :=
  match j with
  | 0 => nil
  | j + 1 => (pathNat.path i j).concat (by grind)
```

The walk `pathNat.path i j` has length j since it is built by concatenating j edges.

```
@[simp]
lemma pathNat.length_path (i j : ℕ) : (pathNat.path i j).length = j := by
  induction j with
  | zero => rfl
  | succ j ih => rw [pathNat.path, length_concat, ih]
```

8.2 Vertices and Edges of a Walk

A walk visits a sequence of vertices and traverses a sequence of edges, and Mathlib records both as lists.

Support and getVert

The *support* of a walk is the sequence of vertices it visits in order, as a `List V`.

```
#source Walk.support
```

A `nil` walk has length `0` and support `[u]`. So in particular every walk has at least one vertex in its support.

```
example : (Walk.nil : G.Walk u u).support = [u] := rfl
```

The support of a `cons` walk consists of the first vertex of the new edge followed by the support of the rest of the walk. (Note the use of `List` notation here: `a :: l` is the list with head `a` and tail `l`).

```
example (h : G.Adj u v) (p : G.Walk v w) :
    (Walk.cons h p).support = u :: p.support := Walk.support_cons h p
```

A walk is determined by its support.

```
#check Walk.ext_support -- (h : p.support = q.support) : p = q
```

The start and end vertices of a walk belong to its support. Results involving the start and end of a walk use ”start” and ”end” in their names, but `Walk.start` and `Walk.end` are not defined. (By design, if we have a walk `p : G.Walk u v` then u and v are already available as part of the type of p so we don't need special names for them.)

```
#check Walk.start_mem_support  -- (p : G.Walk u v) : u ∈ p.support
#check Walk.end_mem_support    -- (p : G.Walk u v) : v ∈ p.support
```

The support of the walk `pathNat.path i j` from `i` to `i + j` in `pathNat` is `[i, i + 1, ..., i + j]`, which can be expressed as `List.range' i (j + 1)`.

```
#check List.range'
```

```
@[simp]
lemma pathNat.support_path (i j : ℕ) :
    (pathNat.path i j).support = List.range' i (j + 1) := by
  induction j with
  | zero => simp [pathNat.path]
  | succ n ih => simp [pathNat.path, support_concat, ih, List.range'_1_concat]
```

The i-th vertex of a walk can be obtained with `Walk.getVert`.

```
#source Walk.getVert
```

If `p : G.Walk u v` and `i ≤ p.length`, then `p.getVert i` is the `i`-th vertex in the support of the walk.

```
example (p : G.Walk u v) : p.getVert 0 = u := p.getVert_zero
```

```
example (i : ℕ) : (Walk.nil : G.Walk u u).getVert i = u := Walk.getVert_nil u
```

```
example (h : G.Adj u v) (p : G.Walk v w) : (Walk.cons h p).getVert 1 = v := by
  rw [getVert_cons_succ, getVert_zero]
```

Notice that `getVert i` is well-defined for any `i : ℕ`, rather than requiring us to check that `i ≤ p.length`, it simply returns the end vertex if `p.length ≤ i`.

```
example (p : G.Walk v w) : p.getVert (p.length + 100) = w :=
  p.getVert_of_length_le (by simp)
```

If `i < p.length` then `p.getVert i` and `p.getVert (i + 1)` are adjacent.

```
example {i} {p : G.Walk u v} (hi : i < p.length) :
    G.Adj (p.getVert i) (p.getVert (i + 1)) := p.adj_getVert_succ hi
```

We have `snd` and `penultimate` as abbreviations for `p.getVert 1` and `p.getVert (p.length - 1)` respectively.

```
#source Walk.snd         -- p.getVert 1
#source Walk.penultimate -- p.getVert (p.length - 1)
```

Darts and Edges

Although edges in a `SimpleGraph` are undirected, walks have a direction: from start to end, so darts crop up very naturally when working with walks.

The *darts* of a walk refers to the `List (Dart G)` of darts the walk uses in order.

```
#source Walk.darts
example (h : G.Adj u v) (p : G.Walk v w) :
    (Walk.cons h p).darts = ⟨(u, v), h⟩ :: p.darts := Walk.darts_cons h p
```

The second vertex of a dart in a walk is a vertex in the support of that walk.

```
#check dart_snd_mem_support_of_mem_darts
```

The *edges* of a walk is the `List (Sym2 V)` of edges it uses in order. So the `edges` of a walk `cons h p` is `h` as an edge followed by `p.edges`.

```
#source Walk.edges
example (h : G.Adj u v) (p : G.Walk v w) :
  (Walk.cons h p).edges = s(u, v) :: p.edges := Walk.edges_cons h p
```

If we append two walks then the edges of the resulting walk are the edges of the first walk followed by the edges of the second walk. (Note `++` below is `List.append`.)

```
example (p₁ : G.Walk u v) (p₂ : G.Walk v w) :
    (p₁.append p₂).edges = p₁.edges ++ p₂.edges := Walk.edges_append p₁ p₂
```

We can also view the edges of a walk as a set of edges via `Walk.edgeSet`.

```
#source Walk.edgeSet -- {e | e ∈ p.edges}
```

8.3 New Walks from Old

We already saw how to concatenate two walks with `append` and how to use `cons` and `concat` to add edges at the start and end of a walk. There are also a number of other ways to build new walks from old.

Since edges in a `SimpleGraph` are undirected, we can reverse a walk `p : G.Walk u v` to get a walk `p' : G.Walk v u`.

```
#check Walk.reverse --  (p : G.Walk u v) : G.Walk v u
```

The reversed walk has the same length as the original walk.

```
example (p : G.Walk u v) : p.reverse.length = p.length := Walk.length_reverse p
```

Given a vertex `u` that appears in a walk `p : G.Walk v w`, we can split the walk at the first occurrence of `u` to get walks from `v` to `u` (`takeUntil`) and from `u` to `w` (`dropUntil`).

```
#check Walk.takeUntil  -- (p : G.Walk v w) (u : V) : u ∈ p.support → G.Walk v u

#check Walk.dropUntil  -- (p : G.Walk v w) (u : V) : u ∈ p.support → G.Walk u w
```

These two walks can be concatenated to give back the original walk. (Note `++` is not actually defined for walks but is just useful informal notation for `Walk.append`.)

```
#check Walk.take_spec -- p.takeUntil u h ++ p.dropUntil u h = p
```

Given a closed walk `c₁ : G.Walk v v` and a vertex `u` in its support, we can rotate the walk to get a new closed walk `c₂ : G.Walk u u` (`rotate`) (built as `dropUntil ++ takeUntil`).

```
#check Walk.rotate  -- (c : G.Walk v v) (u : V) (h : u ∈ c.support) : G.Walk u u
```

```
example [DecidableEq V] (c : G.Walk v v) (h : u ∈ c.support) :
  c.rotate u h = (c.dropUntil u h).append (c.takeUntil u h) := rfl
```

Using `getVert` we can also decompose walks by taking or dropping the first `n` edges, so that `p.take n` ends at the vertex `p.getVert n`.

```
#check Walk.take --  (p : G.Walk u v) (n : ℕ) : G.Walk u (p.getVert n)
#check Walk.drop --  (p : G.Walk u v) (n : ℕ) : G.Walk (p.getVert n) v
#check Walk.append_take_drop_eq  -- p.take n ++ p.drop n = p
```

Special cases of these include `tail` and `dropLast`.

```
#check Walk.tail -- (p : G.Walk u v) : G.Walk p.snd v
#check Walk.dropLast -- (p : G.Walk u v) : G.Walk u p.penultimate
```

Notice that all of the walks defined using `getVert` have `p.getVert n` or similar in their type so, when there is a choice, the `takeUntil`, `dropUntil` or `rotate` walks may be easier to work with. A vertex such as `p.getVert n` is often only *propositionally* (not definitionally) equal to the vertex we want, and the resulting type mismatches are the subject of Fixing Type Mismatches (p. 203).

8.4 Trails, Paths, Circuits and Cycles

There are a number of important special kinds of walks, such as paths, cycles, trails and circuits, that crop up frequently in graph theory. Most of the common ones are defined in Mathlib. We gather a summary of these below.

Name	Property	Mathlib
Trail	no repeated *edges*	`Walk.IsTrail`
Path	no repeated *vertices*	`Walk.IsPath`
Circuit	non-nil closed trail	`Walk.IsCircuit`
Cycle	circuit with no repeated vertex in its support's tail	`Walk.IsCycle`
Eulerian	trail using every edge of the graph	`Walk.IsEulerian`
Hamiltonian	path visiting every vertex of the graph	`Walk.IsHamiltonian`
Hamiltonian cycle	cycle visiting every vertex	`Walk.IsHamiltonianCycle`

Trails and Paths

Recall that `List.Nodup` means that a list has no duplicated elements i.e. its elements are pairwise distinct.

```
#source List.Nodup -- def Nodup : List α → Prop := Pairwise (· ≠ ·)
```

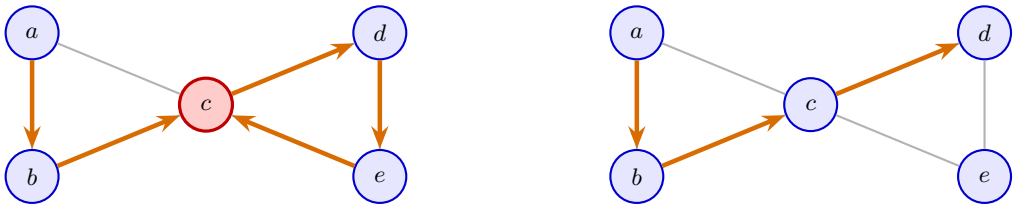
A trail is a walk without duplicated edges.

```
example (p : G.Walk u v) : p.IsTrail ↔ p.edges.Nodup := isTrail_def p
```

A path is a walk without duplicated vertices. Although the proof below is `isPath_def`, if you look at the definition of `IsPath` you will see it `extends IsTrail` so its default constructor asks for proofs of both `edges.Nodup` and `support.Nodup`. But since a proof that there are no duplicate vertices in the support trivially implies there are no duplicate edges, the `edges.Nodup` part of the definition is redundant. For this reason we use the `IsPath.mk'` constructor when proving that a walk is a path.

```
example (p : G.Walk u v) : p.IsPath ↔ p.support.Nodup := isPath_def p

#check IsPath.mk' -- easier to use than the default constructor IsPath.mk
```



A trail that is not a path, and a path, in the same graph

Any path is a trail.

```
#check IsPath.isTrail   -- Walk.IsPath.isTrail : p.IsPath → p.IsTrail
```

If *p* is a path then so is *p*.reverse.

```
example (p : G.Walk u v) (hp : p.IsPath) : p.reverse.IsPath := hp.reverse
```

If *p* is a path then the following subwalks of *p* are also paths.

```
#check IsPath.takeUntil -- (p.takeUntil u h).IsPath
#check IsPath.dropUntil -- (p.dropUntil u h).IsPath
#check IsPath.take      -- (p.take n).IsPath
#check IsPath.drop      -- (p.drop n).IsPath
```

If *h* : *G*.Adj *u* *v* then a walk cons *h* p is a path iff p is a path that does not visit *u*.

```
#check cons_isPath_iff  --  (cons h p).IsPath ↔ p.IsPath ∧ u ∉ p.support
```

If p is a path from *u* to *v* and it contains an edge s(*u*, *w*) then *w* is the second vertex of p.

```
#check IsPath.eq_snd_of_mem_edges
```

If we join two vertex-disjoint paths using an edge then we get a path. (We will need this result a couple of times later in this chapter.)

```
lemma Walk.IsPath.append_cons {u v w z} {h : G.Adj v w} {p : G.Walk u v}
    {q : G.Walk w z} (hp : p.IsPath) (hq : q.IsPath)
    (hd : (p.support).Disjoint q.support) : (p.append ◁ q.cons h).IsPath := by
  apply IsPath.mk'
  rw [support_append, support_cons, List.tail_cons, List.nodup_append']
  exact ⟨hp.support_nodup, hq.support_nodup, hd⟩
```

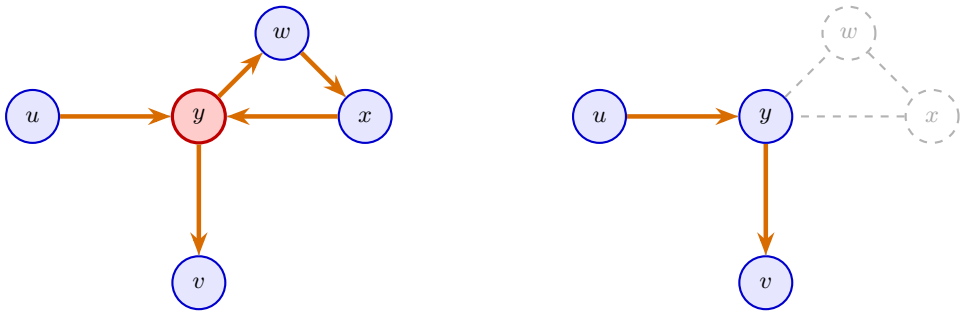
The walk from *i* to *i* + *j* in pathNat is a path.

```
lemma pathNat.isPath_path (i j : ℕ) : (pathNat.path i j).IsPath := by
  induction j with
  | zero => exact IsPath.nil
  | succ n ih => exact (isPath_concat _).2 ⟨ih, by simp⟩
```

A walk is a path iff getVert is injective on the set {*i* | *i* ≤ p.length}.

```
#check IsPath.getVert_injOn_iff
```

If there is a walk from u to v in a graph G, then there is a path from u to v. Informally we might prove this by considering a shortest walk and proving that it is a path. In Mathlib given any walk `p : G.Walk u v` we can get a path from u to v using `Walk.bypass`. This need not be a shortest path; it is simply the walk constructed from p by removing closed subwalks between repeated vertices.



`Walk.bypass` removes closed subwalks to produce a path

```
#source Walk.bypass
```

This returns a walk from u to v that is a path.

```
#check Walk.bypass_isPath
```

Recall our meandering walk `meander35` from `3` to `5` in `pathNat`.

```
#eval meander35.support        -- [3, 2, 3, 4, 5, 6, 5]
#eval meander35.bypass.support -- [3, 4, 5]
```

Reachability

As we noted at the start of the chapter, `G.Walk u v` is a type rather than a proposition. The proposition that there is *some* walk from u to v is `G.Reachable u v`.

```
#source Reachable -- def Reachable (u v : V) : Prop := Nonempty (G.Walk u v)
```

Reachability is an equivalence relation.

```
example : G.Reachable u u := Reachable.refl u
example (h : G.Reachable u v) : G.Reachable v u := h.symm
example (h : G.Reachable u v) (h' : G.Reachable v w) : G.Reachable u w := h.trans h'
```

Adjacent vertices are reachable from each other.

```
example (h : G.Adj u v) : G.Reachable u v := h.reachable
```

Since `Walk.bypass` turns any walk into a path, there is a path between any two reachable vertices.

```
#check Reachable.exists_isPath
```

We study reachability and connectivity in detail in Connectivity (p. 278).

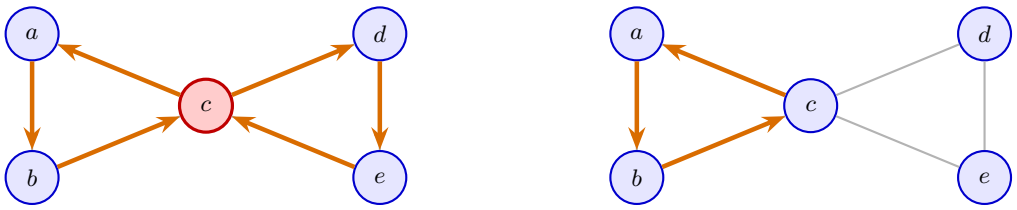
Circuits and Cycles

A *circuit* is a closed trail `p` (i.e. a walk from a vertex to itself without repeated edges) that is not `nil`.

```
#source Walk.IsCircuit -- .. Prop extends isTrail : IsTrail p where
                       --      ne_nil : p ≠ nil
```

A cycle is defined in Mathlib as a circuit in which the `tail` of its `support` has no repeated vertices.

```
#source Walk.IsCycle  -- .. Prop extends isCircuit : IsCircuit p where
                      --      support_nodup : p.support.tail.Nodup
```



A circuit that is not a cycle, and a cycle, in the same graph

Any cycle is a circuit.

```
example (c : G.Walk u u) (hc : c.IsCycle) : c.IsCircuit := hc.isCircuit
```

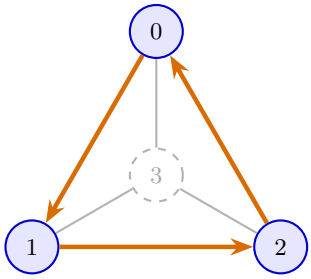
Since a cycle is a circuit, it is not equal to `nil`.

```
example (c : G.Walk u u) (hc : c.IsCycle) : c ≠ nil := hc.isCircuit.ne_nil
```

There is a 3-cycle in the complete graph `K[4]`.

```
example : ∃ (c₃ : K[4].Walk 0 0), c₃.IsCycle ∧ c₃.length = 3 := by
  let c₃: K[4].Walk 0 0 := cons' 0 1 _ (by decide) ◁
            cons (by decide) ◁ cons' 2 0 _ (by decide) nil
  exact ⟨c₃, by simp [c₃, isCycle_def, isTrail_def]⟩
```

The cycle `c₃` defined above and pictured below is `cons` applied three times: it follows the edges `0 – 1`, `1 – 2` and `2 – 0`, so it is the cycle `0 1 2 0`. The remaining vertex 3 of `K[4]` is not used.



The 3-cycle `0 1 2 0` in the complete graph `K[4]`

A closed walk is a cycle if and only if it has length at least 3 and its tail is a path.

```
#check isCycle_iff_isPath_tail_and_le_length
```

If `c` is a cycle then `getVert` is injective on `{i | 1 ≤ i ∧ i ≤ c.length}`.

```
#check IsCycle.getVert_injOn
```

So when is a `cons h p` walk a cycle? Clearly we need `p` to be a path, and also `h` must be the adjacency from the end of `p` to the start of `p` (otherwise the new walk is not closed). Given these two hypotheses, how could `cons h p` fail to be a cycle? Well, we certainly have `(cons h p).support.tail.Nodup` since this is just `p.support.Nodup` and `p` is a path. Also a `cons` walk is never `nil`. So the failure must be that while `p` is a trail, `cons h p` is not a trail. This can only happen if the edge corresponding to `h` is already present. Mathlib has this as `cons_isCycle_iff`.

```
#check cons_isCycle_iff --  (cons h p).IsCycle ↔ p.IsPath ∧ s(u, v) ∉ p.edges
```

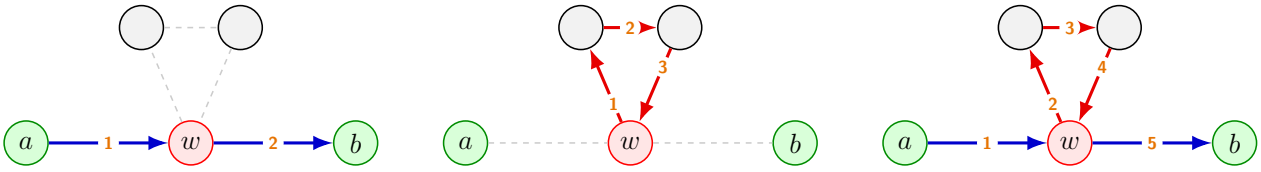
In fact the only way that the path *p* could already contain this edge is if *p* is the walk consisting of the single edge *h*`.symm`. This gives the following variant of `cons_isCycle_iff`, which we will need later: `cons h p` is a cycle iff *p* is a path of length not equal to 1. (The `simp_rw` call in the proof uses `tail_cons` and `isPath_copy`, which both involve `Walk.copy`: see Copy (p. 205).)

```
lemma Walk.cons_isCycle_iff_length_ne_one {u v : V} (p : G.Walk v u)
    (h : G.Adj u v) : (cons h p).IsCycle ↔ p.IsPath ∧ p.length ≠ 1 := by
  have h0 : p.length ≠ 0 := fun h0 => h.ne (p.eq_of_length_eq_zero h0).symm
  simp_rw [isCycle_iff_isPath_tail_and_le_length, getVert_cons_succ, tail_cons,
    isPath_copy, length_cons]
  grind
```

We will also need the `concat` analogue of `cons_isCycle_iff_length_ne_one` which follows by reversing the walk.

```
lemma Walk.concat_isCycle_iff_length_ne_one (p : G.Walk u v) (h : G.Adj v u) :
    (p.concat h).IsCycle ↔ p.IsPath ∧ p.length ≠ 1 := by
  rw [← isCycle_reverse, reverse_concat, cons_isCycle_iff_length_ne_one,
    isPath_reverse_iff, length_reverse]
```

To *splice* a closed walk *c* at *w* into a walk *p* from *a* to *b* that visits *w*, we can split *p* as `(p.takeUntil w) ++ (p.dropUntil w)` and insert *c* to get `(p.takeUntil w) ++ c ++ (p.dropUntil w)`. The result has length `p.length + c.length`, and if *p* and *c* are trails with no common edges then it is again a trail. We will use this to prove Euler's theorem in Eulerian Circuits and Trails (p. 288).



Splicing a closed trail *c* (red) at *w* into a trail *p* (blue) from *a* to *b* gives a longer trail from *a* to *b* (right)

```
lemma Walk.IsTrail.takeUntil_append_append_dropUntil [DecidableEq V] {a b : V}
    {p : G.Walk a b} (hp : p.IsTrail) (hw : w ∈ p.support) {c : G.Walk w w}
    (hc : c.IsTrail) (hdisj : List.Disjoint c.edges p.edges) :
    ((p.takeUntil w hw).append (c.append (p.dropUntil w hw))).IsTrail := by
  simp only [isTrail_append, edges_append]
  refine ⟨hp.takeUntil hw, ⟨hc, hp.dropUntil hw, ?_⟩, ?_⟩
  · -- the edges of c are disjoint from the w → b part of p
    exact fun e he hf => hdisj he (edges_dropUntil_subset_edges p hw hf)
  · -- the a → w part of p is disjoint from both c and the w → b part
    intro e het he'
    rcases List.mem_append.mp he' with hec | hed
    · exact hdisj hec (edges_takeUntil_subset_edges p hw het)
```

```
    · exact hp.disjoint_edges_takeUntil_dropUntil hw het hed

lemma Walk.length_takeUntil_append_append_dropUntil [DecidableEq V] {a b : V}
    (p : G.Walk a b) (hw : w ∈ p.support) (c : G.Walk w w) :
    ((p.takeUntil w hw).append (c.append (p.dropUntil w hw))).length =
      p.length + c.length := by
  have hsp := congrArg Walk.length (p.take_spec hw)
  grind [length_append]
```

Splicing is the reverse of the step in `Walk.bypass` that cuts a closed walk out of a walk. In the next section we will find an odd cycle in a closed walk of odd length by looking at the closed walks cut out in this way.

Odd Cycles and Bipartite Graphs

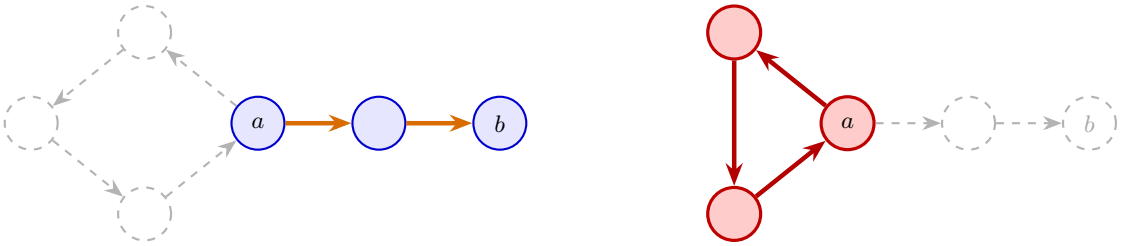
Mathlib contains the following result saying that a graph is bipartite iff every closed walk has even length.

```
#check isBipartite_iff_forall_walk_even_length
  -- G.IsBipartite ↔ ∀ v (p : G.Walk v v), Even p.length
```

However we would really like the stronger result about cycles rather than closed walks: a graph is bipartite iff it contains no odd cycle. One direction is immediate, since a cycle is a closed walk. For the other we need to show that a closed walk of odd length contains a cycle of odd length. This is not currently in Mathlib.

A key step is `Walk.cons_isCycle_iff_length_ne_one`, which we proved in Circuits and Cycles (p. 197).

We show first that any walk can be replaced by a path between the same endpoints whose length has the same parity, unless the graph has an odd cycle. The proof is by induction on the walk, mimicking `Walk.bypass` (Trails and Paths (p. 194)) while keeping track of parity. Given an edge `h : G.Adj a d` and a path `q` from `d` to `b` with the same parity as the rest of the walk, either `a` is not on `q` and `cons h q` is a path of the correct parity, or we split `q` at `a`. In this second case `Walk.bypass` would discard the closed walk `cons h (q.takeUntil a)` and keep `q.dropUntil a`. This preserves parity only if the closed walk has even length; if it has odd length then it is an odd cycle.



Left: skipping an even detour leaves a path from a to b of the same parity. Right: an odd detour is an odd cycle

```
#source Walk.bypass

lemma Walk.exists_isPath_or_odd_isCycle {a b : V} (p : G.Walk a b) :
    (∃ q : G.Walk a b, q.IsPath ∧ (Even q.length ↔ Even p.length)) ∨
      ∃ (v : V) (c : G.Walk v v), c.IsCycle ∧ Odd c.length := by
  classical
```

```
  induction p with
  | nil => exact .inl ⟨nil, .nil, Iff.rfl⟩
  | @cons a d b h p ih =>
    rcases ih with ⟨q, hq, hpar⟩ | hodd
    · by_cases ha : a ∈ q.support
      · have hlen := length_append _ _  ▸ congrArg length (q.take_spec ha)
        rcases Nat.even_or_odd (q.takeUntil a ha).length with he | ho
        · exact .inr ⟨a, cons h (q.takeUntil a ha),
            (cons_isCycle_iff_length_ne_one _ h).mpr ⟨hq.takeUntil ha, by grind⟩,
            by simp_all⟩
        · exact .inl ⟨q.dropUntil a ha, hq.dropUntil ha, by grind [length_cons]⟩
      · exact .inl ⟨cons h q, hq.cons ha, by grind [length_cons]⟩
    · exact .inr hodd
```

Now an odd closed walk `cons h p` contains an odd cycle: either the previous lemma finds one directly, or it replaces `p` by a path `q` of even length, and then `cons h q` is an odd cycle.

```
theorem Walk.exists_odd_isCycle {u : V} (w : G.Walk u u) (hw : Odd w.length) :
    ∃ (v : V) (c : G.Walk v v), c.IsCycle ∧ Odd c.length := by
  cases w with
  | nil => simp at hw
  | cons h p =>
    refine p.exists_isPath_or_odd_isCycle.elim (fun ⟨q, hq, hpar⟩ => ?_) id
    exact ⟨_, _, (cons_isCycle_iff_length_ne_one _ h).mpr
      ⟨hq, by grind [length_cons]⟩, by grind [length_cons]⟩
```

Hence a graph is bipartite iff it has no odd cycle.

```
theorem isBipartite_iff_forall_isCycle_even :
    G.IsBipartite ↔ ∀ u (c : G.Walk u u), c.IsCycle → Even c.length := by
  rw [isBipartite_iff_forall_walk_even_length]
  refine ⟨fun h u c _ => h u c, fun h u w => ?_⟩
  by_contra hw
  obtain ⟨v, c, hc, hodd⟩ := w.exists_odd_isCycle (Nat.not_even_iff_odd.mp hw)
  grind
```

Eulerian Trails

An *Eulerian trail* is a walk using every edge of the graph exactly once.

```
#source Walk.IsEulerian
```

An Eulerian trail is a trail.

```
#check Walk.IsEulerian.isTrail
```

A walk is Eulerian iff it is a trail that uses every edge of the graph.

```
#check Walk.isEulerian_iff -- p.IsEulerian ↔ p.IsTrail ∧ ∀ e ∈ G.edgeSet, e ∈ p.edges
```

Whether a given walk is Eulerian is decidable. We do not need `Fintype V`: the naive `∀ e, e ∈ G.edgeSet → …` quantifies over all of `Sym2 V`, but `isEulerian_iff` lets us restrict it to `G.edgeFinset`, so only `Fintype G.edgeSet` is required (an infinite graph can still have finitely many edges).

`Walk.IsEulerian` is decidable for walks in graphs with finitely many edges.

```
instance [DecidableEq V] [Fintype G.edgeSet] {p : G.Walk u v} :
    Decidable p.IsEulerian :=
  decidable_of_iff (p.edges.Nodup ∧ ∀ e ∈ G.edgeFinset, e ∈ p.edges) ◁ by
    simp [isEulerian_iff, isTrail_def]
```

For a given Eulerian trail in a graph with finitely many edges, we can now use `decide` to prove that it `IsEulerian`. For example the walk `0 1 2 3 4 0 2 4 1 3 0` traverses each of the ten edges of `K[5]` exactly once.

```
private def eulerK5 : K[5].Walk 0 0 :=
  cons' _ 1 _ (by decide) ◁ cons (by decide) ◁
  cons' 2 3 _ (by decide) ◁ cons (by decide) ◁
  cons' 4 0 _ (by decide) ◁ cons (by decide) ◁
  cons' 2 4 _ (by decide) ◁ cons (by decide) ◁
  cons' 1 3 _ (by decide) ◁ cons (by decide) nil
```

And `decide` confirms that this walk is indeed Eulerian.

```
example : eulerK5.IsEulerian := by decide
```

Beware that `IsEulerian` is a property of walks, not of graphs, and that it constrains only the edges. An Eulerian trail never reaches an isolated vertex, so a graph with an Eulerian trail need not be connected: a non-nil Eulerian trail visits exactly the vertices that are not isolated.

```
#check Walk.IsEulerian.mem_support_iff -- w ∈ p.support ↔ ¬G.IsIsolated w
```

At the other extreme, the `nil` walk at a vertex is Eulerian iff *G* has no non-isolated vertices, that is, when *G* is edgeless.

```
lemma Walk.isEulerian_nil_iff [DecidableEq V] (u : V) :
    (nil : G.Walk u u).IsEulerian ↔ G.support = ∅ := by
  rw [support_eq_bot_iff]
  refine ⟨fun h => h.nil_iff.mp Nil.nil, ?_⟩
  rintro rfl
  exact isEulerian_of_bot _
```

Currently Mathlib does not contain a characterization of graphs that admit Eulerian trails, but it does have the necessary condition for graphs on a finite vertex set: if `p` is an Eulerian trail in *G*, then every vertex of *G* has even degree, except possibly the endpoints, which have even degree iff they are equal.

```
#check Walk.IsEulerian.even_degree_iff
```

In a form more familiar to graph theorists: a graph with an Eulerian trail has either no vertices of odd degree or exactly two.

```
#check Walk.IsEulerian.card_odd_degree
```

The `Fintype V` assumption in this last result is overkill. Any Eulerian trail comes with a proof that the edge set of the graph is finite: the walk itself is that proof!

```
#check Walk.IsEulerian.fintypeEdgeSet
```

Using this we can replace the `Fintype V` assumption with `Fintype (G.neighborSet x)` which is needed in order to state the result in terms of degrees.

```
theorem Walk.IsEulerian.even_degree_iff' {x u v : V} {p : G.Walk u v} [DecidableEq V]
    [Fintype (G.neighborSet x)] (h : p.IsEulerian) :
    Even (G.degree x) ↔ u ≠ v → x ≠ u ∧ x ≠ v := by
  have := h.fintypeEdgeSet
  simp [← h.isTrail.even_countP_edges_iff, ← Multiset.coe_countP,
    Multiset.countP_eq_card_filter, ← card_incidenceFinset_eq_degree,
    incidenceFinset_eq_filter, ← h.edgesFinset_eq, Finset.card_def]
```

So every vertex of a graph with an Eulerian circuit has even degree.

```
example {x} {p : G.Walk u u} [DecidableEq V] [Fintype (G.neighborSet x)]
    (h : p.IsEulerian) : Even (G.degree x) := by
  simp [h.even_degree_iff']
```

We will revisit the more interesting question of which graphs admit Eulerian trails in Eulerian Circuits and Trails (p. 288).

Hamiltonian Paths and Cycles

A *Hamiltonian path* is a walk that visits every vertex of the graph exactly once.

```
#source Walk.IsHamiltonian
```

Notice that `IsHamiltonian` is defined using `List.count`, and counting the occurrences of a vertex in a list needs to decide whether two vertices are equal. This is why `[DecidableEq V]` appears in almost every statement about Hamiltonian walks below.

A Hamiltonian walk is a path. (Notice that this is a theorem, not part of the definition.)

```
#check Walk.IsHamiltonian.isPath
```

A path is Hamiltonian iff it contains every vertex of the graph.

```
#check Walk.IsPath.isHamiltonian_iff
```

The length of a Hamiltonian path is exactly one less than the number of vertices in the graph.

```
example [Fintype V] [DecidableEq V] (p : G.Walk u v) (hp : p.IsHamiltonian) :
    p.length = Fintype.card V - 1 := hp.length_eq
```

Mathlib also states this as `p.length + 1 = Fintype.card V`, which avoids the truncated subtraction of $\mathbb{N}$. This is the form we use in proofs later in the chapter.

```
#check Walk.IsHamiltonian.length_add_one_eq -- p.length + 1 = Fintype.card V
```

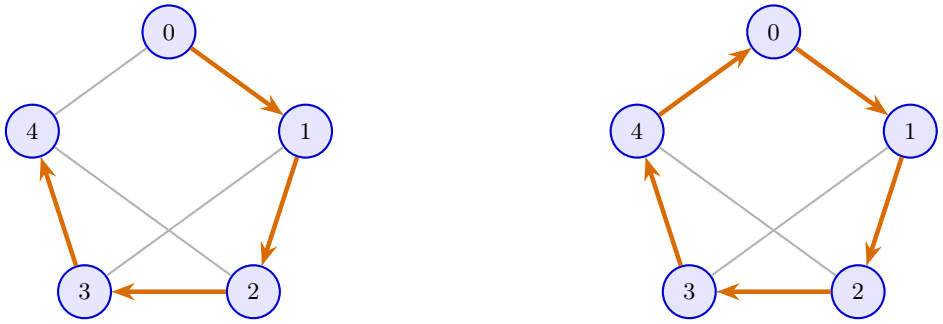
A walk is a finite object: its support is a list. So in the same way that an Eulerian trail in a graph provides a `Fintype` instance for its edge set, a Hamiltonian path provides a `Fintype` instance for the vertex set (provided we also have `DecidableEq V`).

```
#source Walk.IsHamiltonian.fintype
```

A Hamiltonian *cycle* is a cycle containing every vertex of the graph. In Mathlib this is defined as a cycle whose tail is a Hamiltonian path.

```
#source Walk.IsHamiltonianCycle -- structure IsHamiltonianCycle (p : G.Walk a a) :
                                --   Prop extends p.IsCycle where
                                --   isHamiltonian_tail : p.tail.IsHamiltonian
```

The figure below shows a Hamiltonian path and a Hamiltonian cycle on the same five-vertex graph. Both visit every vertex, but, just as for any path or cycle, they need not use every edge: the grey chords are left untraversed (contrast this with an Eulerian trail, which must use every edge).



A Hamiltonian path and a Hamiltonian cycle on the same graph

A rotation of a walk is a Hamiltonian cycle iff the walk is a Hamiltonian cycle.

```
#check Walk.isHamiltonianCycle_rotate
```

Mathlib also has the `SimpleGraph.IsHamiltonian` predicate, which is a property of finite graphs rather than walks. A graph on a finite vertex set is *Hamiltonian* if it contains a Hamiltonian cycle or is a single vertex.

```
#source SimpleGraph.IsHamiltonian -- def IsHamiltonian (G : SimpleGraph α) : Prop :=
                                  --   Fintype.card α ≠ 1 →
                                  --     ∃ a, ∃ p : G.Walk a a, p.IsHamiltonianCycle
```

8.5 Fixing Type Mismatches

The topics we will cover in this section don't appear in informal graph theory, but they are vital when working with walks in Lean. The common theme they share is that they allow us to fix type mismatches.

All of these mismatches come from the difference between definitional and propositional equality, which we met in Definitional and Propositional Equalities (p. 16): for a variable $n : \mathbb{N}$, `n + 0` reduces to `n`, so the two are defeq, while `0 + n` is stuck on the variable `n` and is only propositionally equal to `n`. Lean identifies two types when they are defeq, but not when they are merely propositionally equal. So a walk of type `G.Walk u (n + 0)` is also a walk of type `G.Walk u n`, but a walk of type `G.Walk u (0 + n)` is not.

```
example {G : SimpleGraph ℕ} {u n : ℕ} (p : G.Walk u (n + 0)) : G.Walk u n := p
```

```
/- error: Type mismatch
  p
has type
  G.Walk u (0 + n)
but is expected to have type
```

```
  G.Walk u n
-/
example {G : SimpleGraph ℕ} {u n : ℕ} (p : G.Walk u (0 + n)) : G.Walk u n := p
```

Nil

As we noted at the start of this chapter, the type of a walk encodes its endpoints. We have already seen examples of how this allows for "correct by construction" definitions (such as `Walk.append`), but it also has a cost. Simply stating the proposition that two walks `p : G.Walk u v` and `q : G.Walk x y` are equal is only possible if they have precisely the same endpoints. This kind of problem arises frequently when working with *dependent types* such as `G.Walk u v`, which depend on values (here the endpoints).

We first deal with the special case when one of the walks is `nil`, before turning to the more general problem of endpoint mismatches with `Walk.copy`.

There are two natural ways to state a result about non-nil walks, i.e. walks that have at least one edge. The first is to make the `cons` constructor explicit in the statement of the result.

```
example (h : G.Adj u v) (p : G.Walk v w) : 0 < (p.cons h).length := by
  simp
```

But this is not always convenient: it forces any caller of this result to have an explicit `cons h p` walk, whereas often we have some opaque walk `q` which we know is not `nil`, but whose first edge we don't know.

The second is to hypothesize that the walk is not `nil`. This is potentially problematic: if the result is about a walk `p : G.Walk v w` then `p ≠ nil` does not type-check since `nil` has type `G.Walk u u` for some `u`.

```
/- error: Type mismatch
  nil
has type
  Walk ?m.3 ?m.4 ?m.4
but is expected to have type
  G.Walk v w
-/
example (p : G.Walk v w) (h : p ≠ nil) : 0 < p.length := by sorry
```

The solution is to use the predicate `¬p.Nil` instead.

```
#source Walk.Nil -- inductive Nil : {v w : V} → G.Walk v w → Prop
                 --   | nil {u : V} : Nil (nil : G.Walk u u)
```

To understand how this all works, read on. Otherwise feel free to skip to the next subsection Copy (p. 205).

Recall that `Walk` is an inductive type with two constructors: `nil` (whose type forces its two endpoints to coincide) and `cons` (which does not). Now suppose we have a walk `p : G.Walk v w` and we try the naive test `p = nil`. For this equation to type-check, Lean must unify the type of `nil`, namely `G.Walk u u`, with the type of `p`, namely `G.Walk v w`, which forces `v` and `w` to be *defeq*. So `p = nil` cannot even be *stated* unless `v` and `w` are already defeq.

`Nil` sidesteps this by being a `Prop`-valued predicate *indexed by the walk*. The vertices `{v w : V}` appearing in its definition are indices, free to vary from one constructor to the next, and its single

constructor `| nil {u : V} : Nil (nil : G.Walk u u)` fixes both of them to be the *same* vertex `u`.

Two things follow immediately from this. Firstly, `Nil p` is a well-typed proposition for *every* `p : G.Walk v w`, whatever its endpoints: so we have replaced an equation that may not type-check with a predicate that always does. Secondly, the only way to build a proof of `Nil p` is via this constructor, which applies precisely when `p` is `nil`. There is no constructor mentioning `cons`, so `Nil (cons h q)` is simply uninhabited. So given `h : p.Nil`, `cases h` leaves a single case, `nil`. In it, Lean has matched `p : G.Walk v w` against `nil : G.Walk u u`, so `p` becomes `nil` and `v`, `w` and `u` all become a single vertex. Lean keeps the name `v` for it, so in the example below `w` disappears and the goal becomes `v = v`.

```
example (p : G.Walk v w) (h : p.Nil) : v = w := by
  cases h with
  | nil => -- here the goal is v = v
    rfl
```

Notice that the proof of `Nil.eq` is exactly this same pattern matching.

```
#source Walk.Nil.eq -- lemma Nil.eq {p : G.Walk v w} : p.Nil → v = w | .nil => rfl
```

Moreover, the fact that a `cons` walk is not `Nil` can be proved by `nofun`. There are simply no cases to consider!

```
example {h : G.Adj u v} {p : G.Walk v w} : ¬(cons h p).Nil := by nofun
```

`Nil` is essentially a bespoke `Eq` for walks: both are single-constructor inductive types indexed by values, whose single constructor forces the indices to coincide.

```
#source Eq  -- inductive Eq : α → α → Prop where
            --   | refl (a : α) : Eq a a
```

Finally, we can prove that a non-nil walk has positive length as follows.

```
example (p : G.Walk v w) (hn : ¬p.Nil) : 0 < p.length := by
  cases p with
  | nil => exact hn.elim Nil.nil
  | cons _ _ => simp
```

The Mathlib API also relates `Nil` to `length` directly.

```
#check Walk.length_eq_zero_iff -- p.length = 0 ↔ p.Nil
```

Copy

Having solved the problem of type mismatches involving `nil`, we now turn to the general problem of type mismatches between arbitrary walks. For example if `p : G.Walk u v₁` and `q : G.Walk v₂ w` where `v₁` and `v₂` are provably, but not definitionally, equal then we can't directly form `p.append q` since `Walk.append` has a signature that expects compatible walks. If we try to write `p.append q` we get a type error.

```
/- Application type mismatch: The argument
  q
has type
```

```
  G.Walk v₂ w
but is expected to have type
  G.Walk v₁ w
-/
example {v₁ v₂ : V} (p : G.Walk u v₁) (q : G.Walk v₂ w) (h : v₂ = v₁) :
  G.Walk u w :=  p.append q
```

A quick fix for this is to change the type of `q` using `h : v₂ = v₁` and `▸`:

```
example {v₁ v₂ : V} (p : G.Walk u v₁) (q : G.Walk v₂ w) (h : v₂ = v₁) :
  G.Walk u w :=  p.append (h ▸ q)
```

This changes the type of `q` without changing the walk itself. Mathlib's `Walk.copy` packages this cast with a name, so we can state lemmas about it, and it allows us to change both endpoints simultaneously.

```
#source Walk.copy
example {v₁ v₂ : V} (p : G.Walk u v₁) (q : G.Walk v₂ w) (h : v₂ = v₁) :
  G.Walk u w :=  p.append (q.copy h rfl)

section snd
variable {u v w : V} {h : G.Adj u v} {p : G.Walk v w}
#check (p.cons h).tail -- G.Walk (cons h p).snd w

end snd
```

A simple example of where we need to introduce a `Walk.copy` is in the proof that `(p.cons h).tail = p`.

On paper this is completely obvious: `Walk.tail` is the walk obtained by dropping its first edge, so `(p.cons h).tail` first adds the edge `h` to the start of `p` and then drops it. However the type of `(p.cons h).tail` is `G.Walk (p.cons h).snd w` while `p` has type `G.Walk v w`. The vertex `(p.cons h).snd` reduces to `p.getVert 0`, which is provably but not definitionally equal to `v`. So the obvious equality does not type check. Instead we have to state this using `Walk.copy`. [1]

```
#check tail_cons -- (cons h p).tail = p.copy ⋯ ⋯
```

Having introduced `Walk.copy` it's instructive to look at its API. Given `p : G.Walk u v` together with proofs of equality `hu : u = u'` and `hv : v = v'`, `p.copy hu hv` is the walk `p` itself, retyped as a walk from `u'` to `v'`. So every statement about `p` has a counterpart for `p.copy hu hv`, saying the same thing. In each case the proof is simply `subst_vars` followed by `rfl`.

For example `length_copy`, `support_copy`, `edges_copy` and `isPath_copy`.

```
#check Walk.length_copy  -- (p.copy hu hv).length = p.length
#check Walk.support_copy -- (p.copy hu hv).support = p.support
#check Walk.edges_copy   -- (p.copy hu hv).edges = p.edges
#check Walk.isPath_copy  -- (p.copy hu hv).IsPath ↔ p.IsPath
```

The `subst_vars` is essential. As its source shows, `copy` is just the cast `hu ▸ hv ▸ p`, and like the cast `h ▸ v` in the disjoint union `Σg` (`SimpleGraph.sigma`), it is stuck while the endpoints are distinct variables. So not even the length of a copied walk reduces, and `rfl` fails.

[1] The definition of `getVert` matches on the walk before the index, so Lean cannot reduce `p.getVert 0` until it knows whether `p` is `nil` or a `cons`, although both cases give `v`.

```
/- error: Tactic rfl failed: The left-hand side
  (p.copy hu hv).length
is not definitionally equal to the right-hand side
  p.length
-/
example {u' v'} (p : G.Walk u v) (hu : u = u') (hv : v = v') :
    (p.copy hu hv).length = p.length := by rfl
```

Substituting identifies `u'` with *u* and `v'` with *v*, so both casts are now along definitional equations and disappear, after which `rfl` succeeds. (For the same reason `copy_rfl_rfl`, which copies along `rfl` proofs, is itself proved by `rfl`.)

```
example {u' v'} (p : G.Walk u v) (hu : u = u') (hv : v = v') :
    (p.copy hu hv).length = p.length := by
  subst_vars
  rfl
```

Notice that the four `copy` lemmas listed above are all `simp` lemmas with `p.copy` on the LHS so that `simp` will systematically remove `.copy` when possible by rewriting to the original walk.

Another class of results involving `Walk.copy` are those that allow us to manipulate a copied walk. For example if we add an edge to a `copy` using `cons` then this is the same as taking a copy of the equivalent `cons` walk.

```
#check Walk.cons_copy --  cons h (p.copy hv hw) = (cons ⋯ p).copy ⋯ hw
```

Some, but certainly not all, such results are `simp` lemmas. To understand why, recall that the left-hand side of a `simp` lemma should be in simp normal form (The Simplifier (p. 17)), and the doc-string for `Walk.copy` tells us what the normal form is in this case:

> The simp-normal form is for the `copy` to be pushed outward. That way calculations can occur within the "copy context".

What this means is that lemmas involving `Walk.copy` are only marked as `@[simp]` if they push the `copy` outwards. The `simp` normal form is `(some_walk_not_involving_copy).copy hv hw`. (With `copy` on the outside, `simp` can apply other walk `simp` lemmas freely to the inner walk without `copy` interfering.)

For example `cons_copy` and `append_copy_copy` are both good `simp` lemmas while `copy_cons` is not.

```
#source cons_copy --  cons h (p.copy hv hw) = (cons (hv ▸ h) p).copy rfl hw

#source append_copy_copy -- (p.copy hu hv) ++ (q.copy hv hw) = (p ++ q).copy hu hw

#source copy_cons --  (cons h p).copy hu hw = cons (hu ▸ h) (p.copy rfl hw)
```

To see what this means in practice, we consider the following concrete example. Suppose that `simp` encounters `cons h₁ (cons h₂ (p.copy hv hw))`. In this case `cons_copy` fires on the inner walk `cons h₂ (p.copy hv hw)`, rewriting to `cons h₁ ((cons (hv ▸ h₂) p).copy rfl hw)`. Now there is again a `copy` inside a `cons` so `cons_copy` fires again to give: `(cons h₁ (cons (hv ▸ h₂) p)).copy rfl hw`.

The `copy` is now on the outside and the result is in normal form: `(some_walk).copy rfl hw`.

If the equality proofs were both `rfl`, then `copy_rfl_rfl` would fire and eliminate the `copy` entirely!

```
#check Walk.copy_rfl_rfl -- (p.copy rfl rfl) = p
```

The following lemma (the `concat` analogue of `cons_copy` from above, not currently in Mathlib) is a good `simp` lemma.

```
@[simp]
lemma Walk.concat_copy {v w y v' w'} (h : G.Adj w y) (p : G.Walk v' w') (hv : v' = v)
    (hw : w' = w) : (p.copy hv hw).concat h = (p.concat (hw ▸ h)).copy hv rfl := by
  subst_vars
  rfl
```

As a worked example consider the following situation: we have `h₁ : G.Adj a b` and `h₂ : G.Adj c d`, as well as walks `p : G.Walk u v` and `q : G.Walk v y`; and proofs that `u = b`, `v = x` and `y = c`. Making heavy use of `Walk.copy` we can construct a walk `G.Walk a d` that is essentially the incorrectly typed walk `(cons h₁ (p ++ q)).concat h₂`. This walk has length `p.length + q.length + 2`.

```
example {a b c d x y} (hub : u = b) (hvx : v = x) (hyc : y = c)
    (p : G.Walk u v) (q : G.Walk v y) (h₁ : G.Adj a b) (h₂ : G.Adj c d) :
    ((cons h₁ ((p.copy hub hvx).append (q.copy hvx hyc))).concat h₂).length
      = p.length + q.length + 2 := by
  -- simp proves this, but it's nice to see the individual steps
  rw [append_copy_copy, cons_copy, concat_copy]
  -- the walk is now in simp normal form with copy on the outside
  -- ⊢ (((cons ⋯ (p.append q)).concat ⋯).copy ⋯ ⋯).length = p.length + q.length + 2
  rw  [length_copy, length_concat, length_cons, length_append]
```

Transfer

As well as encoding the endpoints of a walk, the type of a walk also encodes the graph in which it lives: `p : G.Walk u v`.

Given two graphs `G` and `H` on the same vertex set, and a walk `p` in `G` that only uses edges that are also in `H`, we would like to treat `p` as a walk in `H`. However if Lean is expecting a `H.Walk u v` we can't simply give it `p : G.Walk u v` since the type of `p` is wrong.

The solution is to use `Walk.transfer` which allows us to change the type of a walk from `G.Walk u v` to `H.Walk u v` provided we have a proof that every edge in `p` is also an edge of `H`.

```
#check Walk.transfer
```

For example, if we take the `sup` (i.e. union) of a graph `G` with a graph `H`, then any walk in `G` is also a walk in `G ⊔ H`.

```
example (p : G.Walk u v) (H : SimpleGraph V) : (G ⊔ H).Walk u v :=
  p.transfer _ fun _ he => edgeSet_mono le_sup_left (p.edges_subset_edgeSet he)
```

It is tempting to assume that `Walk.transfer` follows the same `simp` normal form as `Walk.copy`, but in fact the opposite is true. The difference is visible in the definition itself: `Walk.copy` is *not* marked `@[simp]` (it is just the cast `hu ▸ hv ▸ p`, which is stuck while the endpoints are variables), whereas the recursive definition of `Walk.transfer` *is* tagged `@[simp]`.

```
#source Walk.transfer --| nil => nil
                      --| cons' .. => cons _ (p.transfer ..)
```

So `simp` unfolds `Walk.transfer` through the structure of the walk: `(nil).transfer = nil` and `(cons h p).transfer = cons _ (p.transfer ..)`. So unlike `copy`, which can never be removed (except by `copy_rfl_rfl`) and is therefore collected on the *outside*, `transfer` is pushed *inwards* until it vanishes at the `nil` leaves. The `simp` normal form of `transfer` is therefore to *eliminate* it entirely; a `transfer` only survives when it is applied to an opaque walk whose constructor `simp` cannot see.

The `simp` lemmas all serve this purpose: `transfer_append` distributes `transfer` inwards over `append`, moving it towards the leaves:

```
#check transfer_append -- (p ++ q).transfer = (p.transfer ..) ++ (q.transfer ..)
```

Two nested transfers fuse into a single one via `transfer_transfer`.

```
#check transfer_transfer -- (p.transfer H hp).transfer K hp' = p.transfer K …
```

Finally, `length_transfer`, `support_transfer` and `edges_transfer` say that transferring a walk does not change its length, support or edges. So `simp` removes a `transfer` sitting inside any of these, even when the walk itself is opaque.

```
#check length_transfer  -- (p.transfer H hp).length = p.length
#check support_transfer -- (p.transfer H hp).support = p.support
#check edges_transfer   -- (p.transfer H hp).edges = p.edges
```

Elsewhere in Mathlib

`Walk.copy` and `Walk.transfer` have no informal counterparts, but the problem they solve is not specific to walks. It arises whenever a type encodes data and we need to reconcile two versions of that data that are equal but not definitionally so.

`Walk.copy` shares its name and purpose with the `copy` functions found on many structures in Mathlib (`Submonoid.copy`, `Subgraph.copy`, ...). These follow the *range copy pattern*: given a structure and a set that is propositionally equal to one of its fields (such as its carrier), `copy` returns an equal structure in which that field is *definitionally* the new set. Whether the data being replaced lives in a field or, as for walks, in the type, the effect is the same: a defeq problem becomes a one-off proof obligation. We will meet another `copy` of this kind in the next chapter (`ColoringOn.copy`).

8.6 Mapping Walks

Given a walk p : `G.Walk u v` and a graph homomorphism f : `G →g H` we can map p to a walk in H from $f(u)$ to $f(v)$ using `Walk.map`.

```
#source Walk.map
example {W : Type*} {H : SimpleGraph W} (f : G →g H) (p : G.Walk u v) :
    H.Walk (f u) (f v) := p.map f
```

When $G \le H$ (and so trivially the edges of any walk in G lie in H) `Walk.transfer` is equal to `Walk.map` along the subgraph inclusion homomorphism `Hom.ofLE`.

```
#check Hom.ofLE -- (h : G ≤ H) : G →g H
#check Walk.transfer_eq_mapLe --  (GH : G ≤ H) : p.transfer H hp = mapLe GH p
```

Recall `P[n]`, the `pathGraph` with `n` vertices on `Fin n`, and the embedding `pathGraph.embedding n k : P[n] ↪g P[n + k]`, which sends `i ↦ i + k`, so its image is the last `n` vertices of `P[n + k]`.

```
#check pathGraph
#check pathGraph.embedding
```

We can use this embedding and `Walk.map` to define a walk from `0` to n in `P[`n `+ 1]`. In the n `+ 1` case the recursive call gives a walk `pathGraph.path` n `: P[`n `+ 1].Walk 0 (Fin.last` n`)`. Writing `f` for `(pathGraph.embedding (`n `+ 1) 1).toHom`, mapping this walk along `f` gives a walk of type `P[`n `+ 2].Walk (f 0) (f (Fin.last` n`))`, whereas to prepend the edge from `0` to `1` we need one of type `P[`n `+ 2].Walk 1 (Fin.last (`n `+ 1))`. These two types are in fact definitionally equal, so Lean would accept the definition without `Walk.copy`. However, seeing this requires unfolding ordinary `def`s such as `pathGraph.embedding` and `Fin.addNat`. When `rw` and `simp` check that a term is well typed they unfold `@[implicit_reducible]` definitions such as `HypercubeVert` (see Hypercube (p. 35)) but not ordinary `def`s, so they would fail on the resulting walk. The `.copy` replaces this hidden definitional equality with two explicit equations, each proved by `simp`, and `simp` lemmas such as `length_copy` then see straight through the `copy`.

The walk from `0` to n in the `pathGraph` on `Fin (`n `+ 1)`.

```
def pathGraph.path (n : ℕ) : P[n + 1].Walk 0 (Fin.last n) :=
  match n with
  | 0 => nil
  | n + 1 =>
      (pathGraph.path n).map (pathGraph.embedding (n + 1) 1).toHom |>.copy
        (by simp) (by simp) |>.cons
          (by simp [pathGraph_adj] : P[n + 2].Adj 0 1)
```

We could have defined this walk directly; indeed we previously defined walks directly in `pathNat` so why not give a direct definition here? The key difference here is that the vertex type of each `P[`n`]` is different so the induction would be more painful, and we would have fewer results to rely on in subsequent proofs. From the `Walk.map` definition the proof that this is a Hamiltonian path will drop out very easily below.

First we show that this walk has length n.

```
@[simp]
lemma pathGraph.length_path {n} : (pathGraph.path n).length = n := by
  induction n with
  | zero => rfl
  | succ _ ih => simp [ih, pathGraph.path]
```

Before showing that this walk is a path we need one small lemma. Any copy of a path is a path: the iff statement `isPath_copy` is already in Mathlib, but this version, with dot notation, is missing and is exactly what we need next.

```
lemma Walk.IsPath.copy {u v x v'} {p : G.Walk u v} (hp : p.IsPath) {hu : u = x}
    {hv : v = v'} : (p.copy hu hv).IsPath :=
  (isPath_copy ..).2 hp
```

This walk is a path.

```
lemma pathGraph.isPath_path {n} : (pathGraph.path n).IsPath := by
  induction n with
  | zero => exact IsPath.nil
  | succ n ih =>
    -- IsPath.copy peels off the copy, so that the goal displays the map and
    -- Lean can read off the homomorphism before it checks injective
    apply (IsPath.map (pathGraph.embedding ..).injective ih).copy.cons
    simp
```

Hence this walk is a Hamiltonian path.

```
lemma pathGraph.isHamiltonian_path {n} : (pathGraph.path n).IsHamiltonian :=
  isHamiltonian_iff_isPath_and_length_eq.mpr ⟨pathGraph.isPath_path, by simp⟩

end basics
```

8.7 Walks in the Hypercube

We will consider the two most natural walks in the hypercube: canonical shortest paths and Gray code paths.

Hamming Paths

Between any two vertices in the hypercube there exist many different shortest paths, each obtained by successively flipping one of the coordinates in which the two vertices differ. More precisely if `u` and `v` differ in `k` coordinates then there are `k!` different shortest paths between `u` and `v` obtained by flipping these coordinates in different orders. The length of any such path is the number of coordinates which need to be flipped, which is the *Hamming distance* between the two vertices.

We will define a canonical choice of such a path, which we will then call *the Hamming path* between the two vertices, by flipping coordinates from left to right. We will use induction on `n` and `Walk.map` to lift a path in `Q[n]` to `Q[n + 1]` via the `embedBool` embedding we defined earlier. (Recall that `embedBool n b` sends `v ↦ (v, b)`.)

```
#check hypercube.embedBool
```

We can define Hamming paths using `if-then-else` syntax to split on the value of the last coordinate. The walk obtained from `Walk.map` starts at `(embedBool n b) u`, which is `(u, b)` by definition, but only after unfolding the ordinary `def embedBool`. As with `pathGraph.path`, Lean would accept this walk as it stands, but `rw` and `simp` would fail on it, so in both cases we use `Walk.copy` to convert it to the expected type `Q[n + 1].Walk (u, b) (v, c)`. In the `b = c` case the `copy` is needed anyway, since the end vertex `(v, b)` equals `(v, c)` only via the hypothesis `h : b = c`.

Since a Hamming path is a path between two `HypercubeVert n` vertices, we choose to put it, and related definitions and results, in the `HypercubeVert` namespace. [2]

[2] Although `(embedBool n b) u` and `(u, b)` are defeq, we discharge the first side condition of each `Walk.copy` with `simp` rather than `rfl`. Lean stores each such proof as an auxiliary lemma, and with `rfl` its statement would be `(embedBool n b).toHom u = (embedBool n b).toHom u`. The definition would still be valid, but `rw` and `simp` would then fail to match this against `(u, b)`, so the proofs below could not rewrite the `copy` away. Conversely, the final `rfl` in the `else` branch cannot be a tactic proof. A term such as `rfl` is elaborated immediately, and this is what fixes the end vertex of the `concat` to be `(v, c)`. A tactic block is postponed until the rest of the expression has been elaborated, so the adjacency proof `by simp [h]` would then face an unknown vertex.

```
section hypercube
open hypercube HypercubeVert
```

The Hamming path between two vertices in the hypercube.

```
def HypercubeVert.hammingPath : {n : ℕ} → (u v : HypercubeVert n) → Q[n].Walk u v
  | 0, _, _ => nil
  | n + 1, (u, b), (v, c) => if h : b = c then
      (u.hammingPath v |>.map (embedBool n b).toHom).copy (by simp) (by simp [h])
    else
      ((u.hammingPath v).map (embedBool n b).toHom |>.concat (by simp [h])).copy
        (by simp) rfl
```

We will also need a notion of the Hamming distance between two vertices in the hypercube. Mathlib has a `hammingDist` function for finite Pi-types, but we can't use this for `HypercubeVert` (it isn't a Pi-type!) so we define a version for `HypercubeVert` directly below.

```
#check hammingDist -- [Fintype ι] [(i : ι) → DecidableEq (β i)]
                   --   (x y : (i : ι) → β i) : ℕ
```

Hamming distance in `Q[n]` defined for `HypercubeVert n` directly.

```
protected def HypercubeVert.hammingDist : {n : ℕ} → (u v : HypercubeVert n) → ℕ
| 0, _, _ => 0
| _ + 1, (u, b), (v, c) => u.hammingDist v + if b = c then 0 else 1
```

There is only one `HypercubeVert 0` vertex, so the Hamming distance between any two such vertices is `0`.

```
@[simp]
lemma HypercubeVert.hammingDist_zero {u v : HypercubeVert 0} :
    u.hammingDist v = 0 := rfl
```

The Hamming distance between two vertices in `Q[n + 1]` is the Hamming distance between the first n coordinates, plus `1` iff the last coordinates differ.

```
@[simp]
lemma HypercubeVert.hammingDist_succ {n : ℕ} {u v : HypercubeVert (n + 1)} :
  u.hammingDist v =
    u.1.hammingDist v.1 + if u.2 = v.2 then 0 else 1 := rfl
```

The proofs of basic facts about `hammingPath` are by standard induction, using the `split_ifs` tactic to deal with the `if-then-else` when needed.

The length of the Hamming path is equal to the Hamming distance between the two vertices.

```
@[simp]
lemma HypercubeVert.length_hammingPath {n} (u v : HypercubeVert n) :
    (u.hammingPath v).length = u.hammingDist v := by
  induction n with
  | zero => rfl
  | succ n ih =>
    rcases u
    rcases v
    rw [hammingPath]
```

```
    split_ifs <;> simp_all
```

A Hamming path is a path.

```
@[simp]
lemma HypercubeVert.isPath_hammingPath {n} (u v : HypercubeVert n) :
    (u.hammingPath v).IsPath := by
  induction n with
  | zero => exact Walk.IsPath.nil
  | succ n ih =>
    rcases u with ⟨u, b⟩
    rcases v with ⟨v, c⟩
    rw [hammingPath]
    split_ifs with h <;> apply Walk.IsPath.copy
    · apply IsPath.map (embedBool ..).injective (ih _ _)
    · apply IsPath.map (embedBool ..).injective (ih _ _) |>.concat
      -- the mapped path lies in the slice with last coordinate b, so it cannot
      -- visit the new end vertex (v, c), as b ≠ c
      simp_rw [Walk.support_map, List.mem_map]
      rintro ⟨w, -, hw⟩
      -- hw : (w, b) = (v, c), so comparing last coordinates gives b = c
      exact h (congrArg Prod.snd hw)
```

We will use Hamming paths to show that the hypercube is connected in Connected Graphs (p. 279), and in Distance in the Hypercube (p. 335) we will see that the Hamming distance is the graph distance in `Q[n]`.

Gray Code

The hypercube contains a canonical Hamiltonian path called the *Gray code* [Gray 1953] which we will now define. Its endpoints are marked `@[implicit_reducible]` (as `HypercubeVert` is). Lean would accept the definition of `grayPath` below without this, but checking that its `Walk.copy`s are well typed requires unfolding `grayStart (n + 1)` to `(grayStart n, false)`, and similarly for `grayEnd`. Without the attribute `rw` and `simp` cannot do this, and the proof of `isPath_grayPath` fails.

The first vertex of the Gray code walk is a nested tuple of `false`.

```
@[implicit_reducible]
def HypercubeVert.grayStart : (n : ℕ) → HypercubeVert n
  | 0 => ()
  | n + 1 => (grayStart n, false)
```

The last vertex of the Gray code walk is the same as the first, but flipped to `true` in the new dimension.

```
@[implicit_reducible]
def HypercubeVert.grayEnd : (n : ℕ) → HypercubeVert n
  | 0 => ()
  | n + 1 => (grayStart n, true)
```

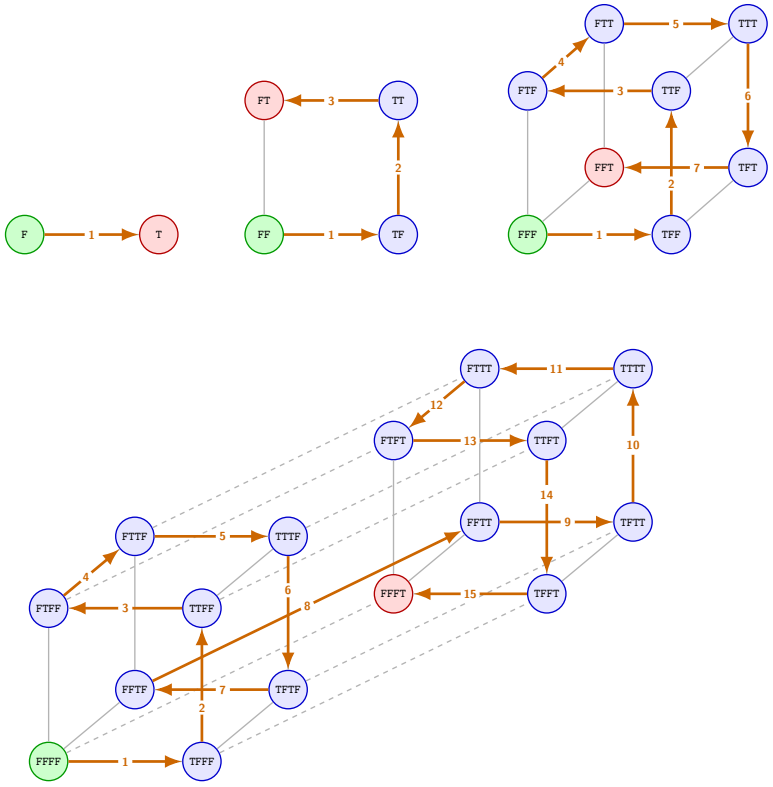
The following `simp` lemmas unfold both endpoints in dimension `n + 1`. We use them to discharge the side conditions of the `Walk.copy`s below.

```
@[simp]
lemma HypercubeVert.grayStart_succ (n : ℕ) :
    grayStart (n + 1) = (grayStart n, false) := rfl

@[simp]
lemma HypercubeVert.grayEnd_succ (n : ℕ) :
    grayEnd (n + 1) = (grayStart n, true) := rfl
```

The Gray code walk in `Q[n]`.

```
def hypercube.grayPath : (n : ℕ) → Q[n].Walk (grayStart n) (grayEnd n)
  | 0 => nil  -- The nil walk from () to ()
  | n + 1 =>
    -- First half: map the n-dim walk onto the false slice of Q[n + 1]
    -- This is a walk from (grayStart n, false) to (grayEnd n, false)
    -- in Q[n + 1]. We use Walk.copy to avoid mistyped endpoints in later proofs.
    let firstHalf := ((grayPath n).map (embedFalse n).toHom).copy (by simp) (by simp)
    -- There is an edge joining the two halves of the
    -- walk and the two slices of Q[n + 1]:
    -- (grayEnd n, false) is adjacent to (grayEnd n, true).
    let crossAdj : Q[n + 1].Adj (grayEnd n, false) (grayEnd n, true) := by simp
    -- Second half: reverse the n-dim walk, and map it onto the true slice. This
    -- is a walk from (grayEnd n, true) to (grayStart n, true) in Q[n + 1].
    let secondHalf := ((grayPath n).reverse.map (embedTrue n).toHom).copy
                          (by simp) (by simp)
    -- Now we append the second half to the first half using the cross edge
    firstHalf.append ◁ cons crossAdj secondHalf
```



The Gray code walk (`grayPath n`) in `Q[n]` for `n = 1, 2, 3, 4`, from `grayStart n` to `grayEnd n`

Textbooks usually list the Gray code as `000`, `001`, `011`, `010`, and so on. Our strings are these reversed (`000`, `100`, `110`, `010`, ...). The vertices of `Q[n + 1]` are `HypercubeVert n × Bool`, so we write the extra coordinate on the right rather than the left. This convention keeps the index of each coordinate fixed as the dimension grows.

We can now easily prove that this walk has length `2 ^ n - 1` and that it is a path.

```
@[simp]
lemma hypercube.length_grayPath {n} : (grayPath n).length = 2 ^ n - 1 := by
  induction n with
  | zero => rfl
  | succ n ih =>
    grind [grayPath, length_append, length_cons, length_copy, length_map,
           length_reverse]
```

The Gray code walk is a path.

```
@[simp]
lemma hypercube.isPath_grayPath (n) : (grayPath n).IsPath := by
  induction n with
  | zero => simp [grayPath]
  | succ n ih =>
    apply (IsPath.map (embedBool ..).injective ih).copy.append_cons
          (IsPath.map (embedBool ..).injective ih.reverse).copy
    -- dot notation with IsPath has reduced our goal to proving that the
    -- two parts of the walk have disjoint support
    intro y hy hz
    rw [Walk.support_copy, Walk.support_map, List.mem_map] at hy hz
```

```
    -- the first half lies in the false slice and the second in the true slice,
    -- so y = (a, false) = (b, true), which is impossible
    obtain ⟨a, -, rfl⟩ := hy
    obtain ⟨b, -, hb⟩ := hz
    have hlast : true = false := congrArg Prod.snd hb
    contradiction
```

Moreover it is clearly a Hamiltonian path, since `Q[n]` has `2 ^ n` vertices.

```
@[simp]
lemma hypercube.isHamiltonian_grayPath (n : ℕ) : (grayPath n).IsHamiltonian :=
  isHamiltonian_iff_isPath_and_length_eq.mpr
    ⟨isPath_grayPath _, by rw [length_grayPath, card_hypercubeVert]⟩

end hypercube
```

From here it is a short step to getting a Hamiltonian cycle in `Q[n]` (for `n > 1`); see the Exercises Walks (p. 239) for details.

8.8 Walks in the Mycielskian

Recall `mμ(G)`, the `mycielskian` of a graph `G`. Given any non-nil walk in `G`, there is a natural way to define a walk in `mμ(G)`. We explore this construction in detail below, using it to prove that every edge of `G` gives a 5-cycle in `mμ(G)`, and that `mμ(G)` contains a Hamiltonian path or cycle whenever `G` does (provided `G` has at least two vertices).

Rather than defining the walks that give the Hamiltonian path and cycle directly, we first define a zig-zag walk, `mycielskianZigZag`, that has a simple recursive definition and from which it is easy to build paths and cycles in `mμ(G)` from paths and cycles in `G`.

First, note that if `u – w` is a walk of length 1 in `G` (i.e. a single edge) then there is a corresponding walk of length 2 in `mμ(G)`, given by u^l – w^l – u^r. Further, if `u – v – w` is a walk of length 2 in `G`, then there is a corresponding walk of length 4 in `mμ(G)`: u^l – v^r – w^l – v^l – u^r. Notice that this second example can be obtained from the first by taking the walk corresponding to the single edge `v – w`, reversing it and adding the edges u^l – v^r and v^l – u^r to the start and end respectively.

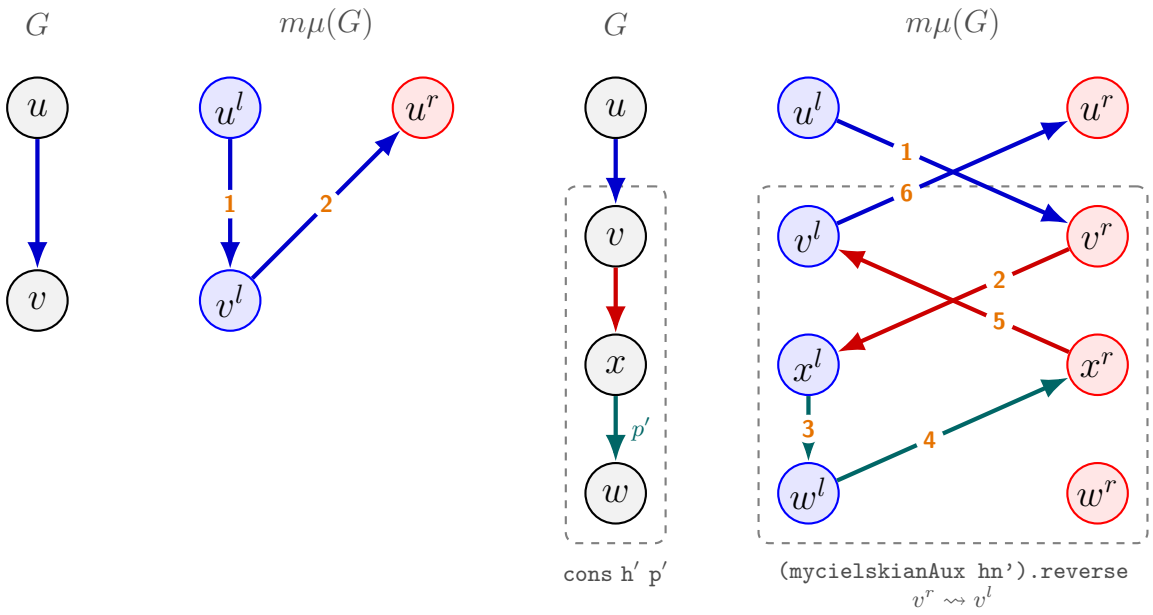
This generalizes, and allows us to define the walk `mycielskianZigZag` recursively. Suppose we have built the walk in `mμ(G)` from v^l to v^r corresponding to a non-nil walk `p : G.Walk v w`, and let `h : G.Adj u v`. The walk for `cons h p` is the edge u^l – v^r, then the reverse of the walk for `p`, then the edge v^l – u^r: `mycielskianZigZag(cons h p) = u^l – [mycielskianZigZag(p).reverse] – u^r`.

```
section mycielskian
variable {V : Type*} {G : SimpleGraph V}
```

Given a non-nil walk from `u` to `w` in `G`, this is the zig-zag walk from u^l to u^r in `mμ(G)`.

```
def Walk.mycielskianZigZag {u w : V} {p : G.Walk u w} (hn : ¬p.Nil) :
    mμ(G).Walk u^l u^r :=
  match p with
  | nil => (hn .nil).elim  -- This case is ruled out by the ¬p.Nil hypothesis
  | cons' u v w h p =>
    -- the last edge of the walk v^l – u^r
    have hl : mμ(G).Adj v^l u^r := h.symm
```

```
    match p with
    | nil => cons' _ vˡ _ h ◁ cons hl nil -- uˡ - vˡ - uʳ
    | cons h' p' =>
      have hn' : ¬(p'.cons h').Nil := nofun
      cons' _ vʳ _ h ((mycielskianZigZag hn').reverse.concat hl)
```



The two cases of the walk `mycielskianZigZag`: `cons h nil` (left) and `cons h (cons h' p')` (right)

If we had marked the definition of `mycielskianZigZag` with the `@[simp]` attribute then its automatically generated equational lemmas would be available for `simp` to use with names like `mycielskianZigZag.eq_1`. We instead state these as simp lemmas that have more meaningful names.

```
@[simp]
lemma Walk.mycielskianZigZag_cons_nil {u v} (h : G.Adj u v)
    (hn : ¬(cons h nil).Nil) : mycielskianZigZag hn = cons (by simpa using h)
      (cons (by simpa using h.symm : mμ(G).Adj vˡ uʳ) nil) := rfl

@[simp]
lemma Walk.mycielskianZigZag_cons_cons {u v x w} (h : G.Adj u v) (h' : G.Adj v x)
    (p' : G.Walk x w) (hn : ¬(cons h (cons h' p')).Nil) :
    mycielskianZigZag hn = cons (by simpa using h)
      ((mycielskianZigZag (nofun : ¬(cons h' p').Nil)).reverse.concat
        (by simpa using h.symm)) := rfl
```

From the definition of `mycielskianZigZag`, Lean automatically generates a *functional induction* principle `mycielskianZigZag.induct`. It has one case for each `match` case of the definition, named `case1` and `case2`, whose variables are the pattern variables of that case, together with an induction hypothesis wherever the definition calls itself. We use it with `induction hn using mycielskianZigZag.induct`: its cases follow the definition exactly, which plain induction on the walk does not.

```
#check mycielskianZigZag.induct -- slightly scary looking! (but useful)
```

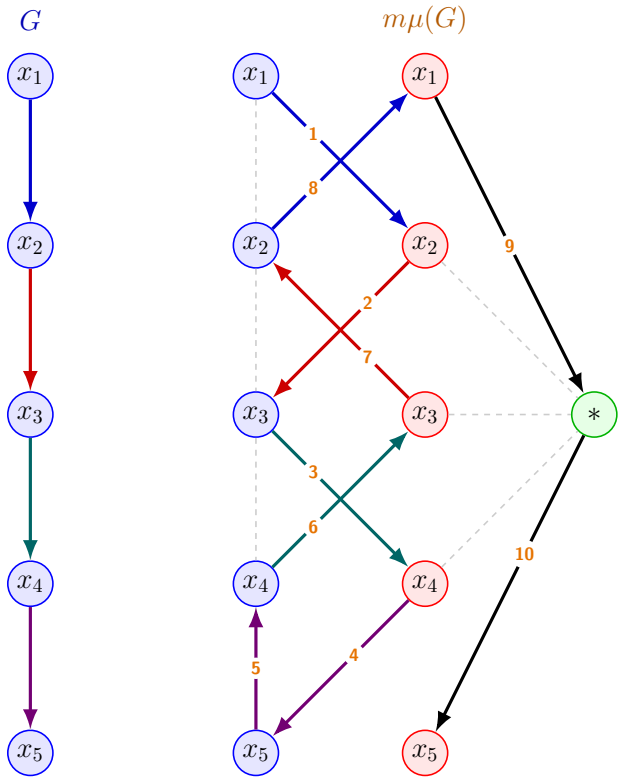
The zig-zag walk is twice as long as the original walk.

```
@[simp]
lemma Walk.length_mycielskianZigZag {u w} {p : G.Walk u w} (hn : ¬p.Nil) :
    (mycielskianZigZag hn).length = 2 * p.length := by
  induction hn using mycielskianZigZag.induct with
  | case1 => rfl
  | case2 =>
    grind [length_cons, mycielskianZigZag_cons_cons, length_concat, length_reverse]
```

We can use the zig-zag walk to map any non-nil walk of length `k` in `G` to a walk of length `2k + 2` in the Mycielskian. This preserves the property of being a path and in particular maps Hamiltonian paths to Hamiltonian paths. We simply add the two edges `uʳ — none — wʳ` at the end of the zig-zag walk.

Given a non-nil walk from `u` to `w` in `G`, this is a walk from `uˡ` to `wʳ` in `mμ(G)`. If the original walk was a (Hamiltonian) path then so is the new walk.

```
protected def Walk.mycielskian {u w} {p : Walk G u w} (hn : ¬p.Nil) :
    mμ(G).Walk uˡ wʳ :=
  mycielskianZigZag hn |>.concat (by simp) |>.concat (by simp : mμ(G).Adj none _)
```



Converting a path in `G` to one in its Mycielskian `mμ(G)`

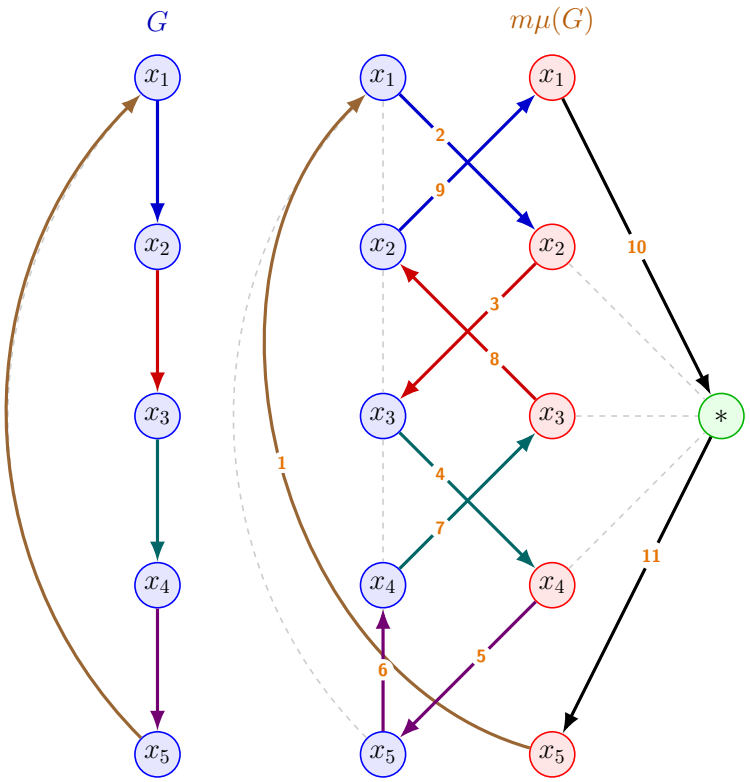
Using this walk we can also map closed walks in `G` to closed walks in the Mycielskian. This preserves the property of being a cycle and in particular maps Hamiltonian cycles to Hamiltonian cycles in the Mycielskian.

Given a closed walk from `u` to `u` in `G`, there is a walk from `uʳ` to `uʳ` in `mμ(G)`. If the original walk was a (Hamiltonian) cycle then so is this new walk.

```
def Walk.mycielskianClosed {u} (c : Walk G u u) : mμ(G).Walk uˣ uˣ :=
  match c with
  | nil => nil
  | cons h' nil => (h'.ne rfl).elim
  | cons' _ v _ h ◁ cons' _ w _ h' p =>
    Walk.mycielskian (not_nil_cons (h := h') (p := p)) |>.cons (by simpa using h)

@[simp]
lemma Walk.mycielskianClosed_nil {u} :
    mycielskianClosed (nil : Walk G u u) = nil := rfl

@[simp]
lemma Walk.mycielskianClosed_cons_cons {u v w} (h : G.Adj u v) (h' : G.Adj v w)
    (p : Walk G w u) : mycielskianClosed (cons h (cons h' p)) =
      (Walk.mycielskian (not_nil_cons (h := h') (p := p))).cons
        (by simpa using h) := rfl
```



Converting a cycle in `G` to one in its Mycielskian `mμ(G)`

The `Walk.mycielskian` of a non-nil walk `p` has length `2 * p.length + 2`.

```
@[simp]
lemma Walk.length_mycielskian {u w} (p : Walk G u w) (hn : ¬p.Nil) :
    (p.mycielskian hn).length = 2 * p.length + 2 := by
  simp [Walk.mycielskian]
```

The `Walk.mycielskianClosed` of a non-nil closed walk `c` has length `2 * c.length + 1`.

```
@[simp]
```

```
lemma Walk.length_mycielskianClosed {u} {c : Walk G u u} (hn : ¬c.Nil) :
    c.mycielskianClosed.length = 2 * c.length + 1 :=
  match c with
  | nil => (hn .nil).elim
  | cons h' nil => (h'.ne rfl).elim
  | cons h (cons h' p) => by
    grind [mycielskianClosed, length_cons, length_mycielskian]
```

Our next goal is to show that the `Walk.mycielskian` preserves the property of being a path. It will be useful to first establish some simple properties of the zig-zag walk `mycielskianZigZag`.

The zig-zag walk only visits left or right copies of vertices from the original walk.

```
lemma Walk.exists_of_mem_support_mycielskianZigZag {u w z} {p : G.Walk u w}
    (hn : ¬p.Nil) (hz : z ∈ (mycielskianZigZag hn).support) :
    ∃ y ∈ p.support, z = y.l ∨ z = y.r := by
  induction hn using mycielskianZigZag.induct <;> aesop
```

In particular, the zig-zag walk does not visit none.

```
@[simp]
lemma Walk.none_notMem_support_mycielskianZigZag {u w} {p : G.Walk u w}
    (hn : ¬p.Nil) :
    none ∉ (mycielskianZigZag hn).support := by
  intro h_in
  obtain ⟨y, _, hy⟩ := exists_of_mem_support_mycielskianZigZag hn h_in
  cases hy <;> contradiction
```

If the original walk `p : G.Walk u w` is a path, then the zig-zag walk does not visit w^r.

```
@[simp]
lemma Walk.end_right_notMem_support_mycielskianZigZag {u w} {p : G.Walk u w}
    (hn : ¬p.Nil) (hp : p.IsPath) : w.r ∉ (mycielskianZigZag hn).support := by
  induction hn using mycielskianZigZag.induct <;> aesop
```

The second vertex of the `Walk.mycielskian` is in the zig-zag walk.

```
@[simp]
lemma Walk.snd_mem_support_mycielskianZigZag {u w} {p : G.Walk u w} (hn : ¬p.Nil) :
  (p.mycielskian hn).snd ∈ (mycielskianZigZag hn).support := by
  induction hn using mycielskianZigZag.induct
    <;> simp_all [Walk.mycielskian]
```

If the original walk `p : G.Walk u w` is a path, then the zig-zag walk is also a path.

```
@[simp]
lemma Walk.IsPath.mycielskianZigZag {u w} {p : G.Walk u w} (hp : p.IsPath)
    (hn : ¬p.Nil) : (mycielskianZigZag hn).IsPath := by
  apply IsPath.mk'
  induction hn using mycielskianZigZag.induct with
  | case1 u v h hl hn => simpa using h.ne
  | case2 u v h hl w x h' p' hn hnp ih =>
    -- grind can solve this with a little help
    grind [mycielskianZigZag_cons_cons, support_concat, support_reverse, isPath_def,
      exists_of_mem_support_mycielskianZigZag hnp]
```

If the original walk `p : G.Walk u w` is a path, then the `Walk.mycielskian` is a path.

```
@[simp]
lemma Walk.IsPath.mycielskian {u w} {p : G.Walk u w} (hp : p.IsPath) (hn : ¬p.Nil) :
    (p.mycielskian hn).IsPath := by
  apply IsPath.mk'
  have := (hp.mycielskianZigZag hn).support_nodup
  grind [Walk.mycielskian, Walk.concat, end_right_notMem_support_mycielskianZigZag,
    support_append, support_nil, none_notMem_support_mycielskianZigZag]
```

If the original walk `p : G.Walk u u` is a cycle then the `Walk.mycielskianClosed` is a cycle. (In fact this holds under the weaker assumption that `p` is not nil and `p.support.tail` contains no repeated vertices.)

```
lemma Walk.isCycle_mycielskianClosed {u} {p : Walk G u u} (hn : ¬p.Nil)
    (hc : p.support.tail.Nodup) : (mycielskianClosed p).IsCycle :=
  match p with
  | nil => (hn .nil).elim
  | cons h' nil => (h'.ne rfl).elim
  -- we use the cons' pattern here to make the InfoView more readable.
  | cons' _ v _ h (cons' _ w _ h' p) => by
    -- we now use cons_isCycle_iff_length_ne_one that we proved earlier to reduce
    -- this to showing that the Walk.mycielskian is a path of length not equal to 1
    rw [mycielskianClosed, cons_isCycle_iff_length_ne_one]
    -- We know that the Walk.mycielskian of a path is a path and simp
    -- can take care of the length.
    exact ⟨(IsPath.mk' hc).mycielskian (by simp), by simp⟩
```

The weaker assumption is exactly what we need for one application of the result. If `h : G.Adj u v` then the closed walk `u - v - u` goes along an edge and straight back, so it is not a cycle. But it is not nil, and the tail `[v, u]` of its support has no repeated vertices, so its image u^{r} – v^{l} – u^{l} – v^{r} – none – u^{r} is a 5-cycle. Hence every edge of `G` gives a 5-cycle in `mμ(G)`. We will use this to find the bridges of `mμ(G)` in Bridges (p. 282) and to compute its girth in Girth of the Mycielskian (p. 345).

```
lemma isCycle_mycielskianClosed_of_adj {u v : V} (h : G.Adj u v) :
    (mycielskianClosed (cons h (cons h.symm nil))).IsCycle :=
  isCycle_mycielskianClosed (by simp) (by simp [h.ne.symm])
```

If the original walk `p : G.Walk u w` is a Hamiltonian path, then the `Walk.mycielskian` is a Hamiltonian path.

```
protected lemma Walk.IsHamiltonian.mycielskian [DecidableEq V] {u w} {p : G.Walk u w}
    (hp : p.IsHamiltonian) (hn : ¬p.Nil) : (p.mycielskian hn).IsHamiltonian := by
  have : Fintype V := hp.fintype
  refine isHamiltonian_iff_isPath_and_length_eq.2 ⟨hp.isPath.mycielskian hn, ?_⟩
  grind [length_mycielskian, Fintype.card_option, Fintype.card_sum,
    hp.length_add_one_eq]
```

If the original walk `p : G.Walk u u` is a Hamiltonian cycle, then the `Walk.mycielskianClosed` is a Hamiltonian cycle.

```
lemma Walk.IsHamiltonianCycle.mycielskianClosed [DecidableEq V] {u} {p : G.Walk u u}
    (hp : p.IsHamiltonianCycle) : p.mycielskianClosed.IsHamiltonianCycle := by
  have : Fintype V := hp.isHamiltonian_tail.fintype
```

```
  refine isHamiltonianCycle_iff_isCycle_and_length_eq.2
    ⟨isCycle_mycielskianClosed hp.isCycle.not_nil hp.isCycle.support_nodup, ?_⟩
  grind [length_mycielskianClosed hp.isCycle.not_nil, hp.length_eq,
        Fintype.card_option, Fintype.card_sum]
```

Finally we can show that if `G` is Hamiltonian then `mμ(G)` is also Hamiltonian (unless `G` has only one vertex, in which case `mμ(G)` is an edge plus an isolated vertex and so is clearly not Hamiltonian).

```
theorem IsHamiltonian.mycielskian [DecidableEq V] [Fintype V]
    (hH : G.IsHamiltonian) (hf : Fintype.card V ≠ 1) : mμ(G).IsHamiltonian := by
  intro
  have ⟨_, _, hc⟩ := hH hf
  exact ⟨_, _, hc.mycielskianClosed⟩

end mycielskian
```

8.9 Hamiltonian Graphs: Ore, Dirac and Bondy–Chvátal*

In this section we prove three classical sufficient conditions for a graph to be Hamiltonian. The oldest two are *Ore's theorem* [Ore 1960] and its immediate (but earlier) corollary *Dirac's theorem* [Dirac 1952]. Call a finite graph an *Ore graph* if the sum of the degrees of every pair of non-adjacent vertices is at least the number of vertices in the graph. Ore's theorem says that every Ore graph on at least 3 vertices is Hamiltonian. The special case when the minimum degree is at least half the number of vertices is Dirac's theorem.

Rather than prove Ore's theorem directly we prove the Bondy–Chvátal theorem [Bondy–Chvátal 1976]: a graph is Hamiltonian iff its *closure* is, where the closure is obtained by repeatedly joining non-adjacent vertices whose degree sum is at least the number of vertices. Ore's theorem falls out immediately, since the closure of an Ore graph is complete and complete graphs are Hamiltonian (`isHamiltonian_top` in Mathlib).

Recall that if `u ≠ v` then `G ⊔ edge u v` is the graph obtained from `G` by adding the edge `u — v`.

The proof has four steps.

1. A *maximal* path (one that cannot be extended at either end) whose endpoints satisfy Ore's degree condition can be closed up into a cycle visiting exactly the same vertices. This uses Pósa rotations and a counting argument.

2. If `u` and `v` are distinct non-adjacent vertices whose degree sum is at least `|V|`, then `G ⊔ edge u v` is Hamiltonian iff `G` is. Adding an edge can only help, so the hard part is the converse: a Hamiltonian cycle of `G ⊔ edge u v` gives one of `G`. If the cycle avoids the new edge `u — v`, it already lies in `G`. Otherwise, deleting `u — v` leaves a Hamiltonian path from `u` to `v` in `G`. A Hamiltonian path is certainly maximal, so step 1 closes it up into a Hamiltonian cycle of `G`.

3. Adding such edges repeatedly, in rounds, shows that a graph is Hamiltonian iff its closure is.

4. Ore's and Dirac's theorems follow, since the closure of an Ore graph is complete.

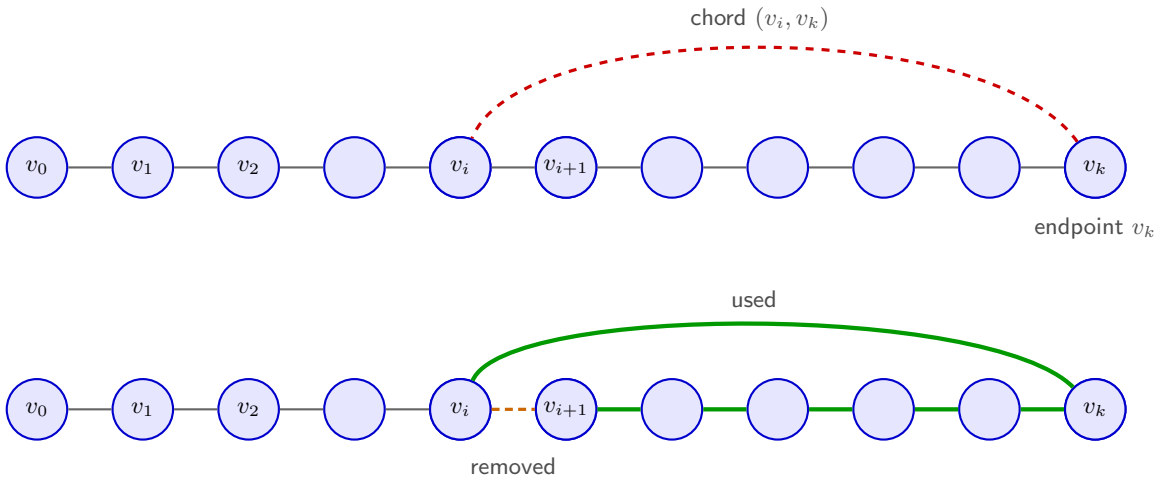
Pósa Rotations

Before starting the proof of Bondy–Chvátal we first introduce the notion of a *Pósa rotation* [Pósa 1976] of a walk. This is a key tool in the proof, but is also a more generally useful technique for working with paths and cycles that has found numerous applications in modern graph theory.

Given a walk `P = v₀, v₁, ..., vₖ` and an edge `vᵢ – vₖ` in the graph, we form a new walk: `v₀, v₁, ..., vᵢ, vₖ, vₖ₋₁, ..., vᵢ₊₁`. This new "rotated" walk has the same length as `P`, and still starts at `v₀`, but now ends at `vᵢ₊₁`.

Currently this isn't in Mathlib but we can easily define it for ourselves.

Given a walk `p : G.Walk u v` and an edge from the `i`-th vertex of `p` to its end vertex `v`, i.e. `h : G.Adj (p.getVert i) v`, we split `p` at index `i` using `Walk.take i` and `Walk.drop i`, which have types `G.Walk u (p.getVert i)` and `G.Walk (p.getVert i) v` respectively. The rotated walk is then the concatenation of `p.take i` with the edge from `p.getVert i` to `v` (i.e. `h`) followed by the reverse of the tail of `p.drop i`, which goes from `v` to `p.getVert (i + 1)`.



A Pósa rotation. Top: a path with a chord from `vᵢ` to its endpoint `vₖ`. Bottom: the rotated path, which ends at `vᵢ₊₁`

```
section bondy_chvatal
variable {V : Type*} {G : SimpleGraph V}
section posa
variable {u v : V} {p : G.Walk u v} {i : ℕ}
```

The existence of the edge `h : G.Adj (p.getVert i) v` helpfully implies that `i < p.length`.

```
lemma Walk.lt_length_of_adj_getVert_end (h : G.Adj (p.getVert i) v) :
    i < p.length := by
  by_contra!
  exact (p.getVert_of_length_le this ▸ h).ne rfl
```

One issue with giving the definition of the Pósa rotation is that although the reverse of the tail of `p.drop i` goes from `v` to `p.getVert (i + 1)`, this is not obvious from the type of this walk.

```
#check (p.drop i).tail.reverse -- G.Walk v (p.drop i).snd
```

So to give the definition of Pósa rotation we need to use `Walk.copy` and give a proof that `(p.drop i).snd = p.getVert (i + 1)` (which `simp` can provide).

The i-th Pósa rotation of a walk p.

```
def Walk.posaRotate (p : G.Walk u v) (h : G.Adj (p.getVert i) v) :
  G.Walk u (p.getVert (i + 1)) :=
  (p.take i).append ◁ cons h ((p.drop i).tail.copy (by simp) rfl).reverse
```

If p is a path then any Pósa rotation of p is also a path. The proof of this makes heavy use of the `IsPath` dot notation.

The *Pósa rotation* of a path is a path.

```
theorem Walk.IsPath.posaRotate (hp : p.IsPath) (h : G.Adj (p.getVert i) v) :
  (p.posaRotate h).IsPath := by
  -- there are six applications of .IsPath notation in the next line
  apply (hp.take _).append_cons (hp.drop _).tail.copy.reverse
  -- it remains to prove that the two parts of the walk are disjoint
  have hnn : ¬(p.drop i).Nil := by
    grind [nil_drop_iff, lt_length_of_adj_getVert_end h]
  rw [← p.append_take_drop_eq i, isPath_def, support_append, List.nodup_append',
      ← (p.drop i).support_tail_of_not_nil hnn] at hp
  simpa [hnn] using hp.2.2
```

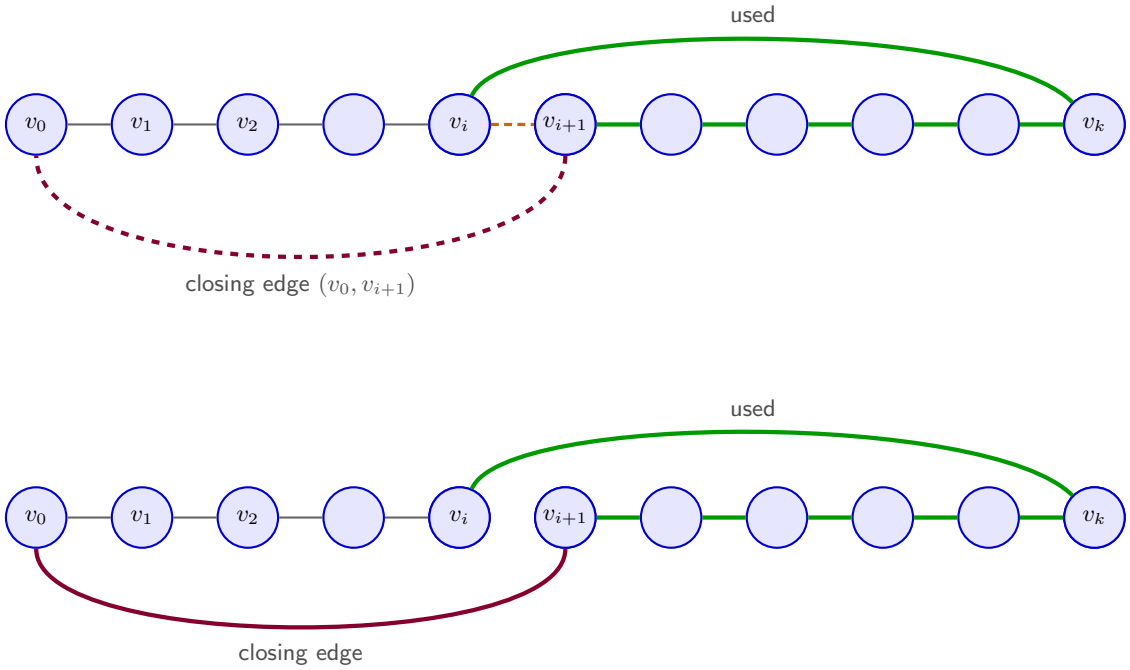
The length of a walk is preserved under Pósa rotations.

```
@[simp]
lemma Walk.length_posaRotate (p : G.Walk u v) (h : G.Adj (p.getVert i) v) :
    (p.posaRotate h).length = p.length := by
  grind [lt_length_of_adj_getVert_end h, posaRotate, length_append, length_cons,
    length_reverse, length_copy, take_length, Walk.tail, drop_length]
```

A Pósa rotation visits exactly the same vertices as the original walk.

Notice that in the last line of the proof below we use `conv_rhs`. At that point we want to express p on the RHS of an $\leftrightarrow$ in terms of `p.take i` and `p.drop i` etc, but if we perform standard rewrites then *every* occurrence of p would be replaced, resulting in a mess that we want to avoid. By using `conv_rhs` we focus on the RHS and only rewrite there.

```
@[simp]
lemma Walk.mem_support_posaRotate_iff {u v : V} {i : ℕ} (p : G.Walk u v)
    (h₁ : G.Adj (p.getVert i) v) (x : V) :
    x ∈ (p.posaRotate h₁).support ↔ x ∈ p.support := by
  have hi := lt_length_of_adj_getVert_end h₁
  have hdropnn : ¬(p.drop i).Nil := by grind [nil_drop_iff]
  rw [posaRotate, mem_support_append_iff, support_cons, List.mem_cons,
      support_reverse, List.mem_reverse, support_copy,
      (p.drop i).support_tail_of_not_nil hdropnn]
  -- see note above about conv_rhs and why we use it here
  conv_rhs => rw [← p.append_take_drop_eq i, mem_support_append_iff,
      ← cons_tail_support (p.drop i), List.mem_cons]
```



Closing a Pósa rotation. Top: the rotated path (`posaRotate`) with an edge from its endpoint `v i+1` back to `v 0`. Bottom: the resulting cycle (`posaClosed`)

If *p* is a walk from *u* to *v* and *h₁* : `G.Adj (p.getVert i) v` then we can form the Pósa rotation ending at `p.getVert (i + 1)`. If there is also an edge *h₂* : `G.Adj (p.getVert (i + 1)) u` then we can use it to close the walk at *u*, giving a `G.Walk u u`.

The closed walk obtained from a Pósa rotation by concatenating an edge from the end of the rotated walk to the start of the original walk.

```
def Walk.posaClosed (p : G.Walk u v) (h₁ : G.Adj (p.getVert i) v)
    (h₂ : G.Adj (p.getVert (i + 1)) u) : G.Walk u u := (p.posaRotate h₁).concat h₂
```

The walk `p.posaClosed h₁ h₂` contains the same vertices as *p*.

```
@[simp]
lemma Walk.mem_support_posaClosed_iff {u v : V} {i : ℕ} (p : G.Walk u v)
    (h₁ : G.Adj (p.getVert i) v) (h₂ : G.Adj (p.getVert (i + 1)) u) (x : V) :
    x ∈ (p.posaClosed h₁ h₂).support ↔ x ∈ p.support := by
  grind [posaClosed, support_concat, mem_support_posaRotate_iff, p.start_mem_support]
```

The length of the closed Pósa rotation is one more than the length of the original walk.

```
@[simp]
lemma Walk.length_posaClosed (p : G.Walk u v) (h₁ : G.Adj (p.getVert i) v)
    (h₂ : G.Adj (p.getVert (i + 1)) u) :
    (p.posaClosed h₁ h₂).length = p.length + 1 := by simp [posaClosed]
```

If *p* is a path of length at least 2 then the `posaClosed` walk is a cycle.

```
lemma Walk.IsPath.isCycle_posaClosed (hp : p.IsPath) (h2 : 2 ≤ p.length)
    (h₁ : G.Adj (p.getVert i) v) (h₂ : G.Adj (p.getVert (i + 1)) u) :
    (p.posaClosed h₁ h₂).IsCycle := (concat_isCycle_iff_length_ne_one ..).mpr
      ⟨hp.posaRotate h₁, by grind [length_posaRotate]⟩
```

```
end posa
```

The key idea in all of the Hamiltonicity theorems we prove below goes back to Dirac. We state it here with Ore's degree condition. Suppose `p = v₀, v₁, ..., vₖ` (with `k ≥ 2`) is a *maximal* path whose endpoints, if non-adjacent, satisfy the Ore degree condition: `deg v₀ + deg vₖ ≥ |V|`. Then `p` can be closed into a cycle containing exactly the vertices of `p`.

If `v₀` and `vₖ` are adjacent this is immediate. Otherwise the Ore condition gives `deg v₀ + deg vₖ ≥ |V|`. Every neighbour of `v₀` is `vᵢ₊₁` for some `i < k`, and every neighbour of `vₖ` is `vⱼ` for some `j < k`. Since a path has distinct vertices, these indices `i` and `j` form two subsets of `{0, ..., k - 1}` of sizes `deg v₀` and `deg vₖ`, and `k < |V| ≤ deg v₀ + deg vₖ`. So by the pigeonhole principle the two subsets share an index `i`, and both edges `vᵢ₊₁ — v₀` and `vᵢ — vₖ` are present: a *crossing pair*. The Pósa rotation `v₀, ..., vᵢ, vₖ, vₖ₋₁, ..., vᵢ₊₁` followed by the edge `vᵢ₊₁ — v₀` is then the required cycle (this is the `Walk.posaClosed` we just defined).

Counting Bijections and a Crossing Pair

We now prove a general counting lemma to help with the informal argument we gave above. If `p` is a path containing every neighbour of a vertex `x`, and $f : \mathbb{N} \to \mathbb{N}$ maps `{i | i < p.length}` into `{i | i ≤ p.length}`, is injective on `{i | i < p.length}` and surjective on `{j | j ≤ p.length ∧ p.getVert j ≠ x}`, then the number of `i < p.length` with `G.Adj (p.getVert (f i)) x` is `G.degree x`. As we can see in its proof, the bijection is `i ↦ p.getVert (f i)` onto the neighbours of `x`.

```
open Finset

lemma Walk.IsPath.card_filter_adj_getVert_eq_degree [DecidableRel G.Adj] {u v x : V}
    [Fintype (G.neighborSet x)]
    {p : G.Walk u v} (hp : p.IsPath) (hall : ∀ w, G.Adj x w → w ∈ p.support)
    (f : ℕ → ℕ)
    (hf_le : ∀ i ∈ range p.length, f i ≤ p.length)
    (hf_inj : Set.InjOn f (range p.length))
    (hf_surj : ∀ j ≤ p.length, p.getVert j ≠ x → ∃ i ∈ range p.length, f i = j) :
    #{i ∈ range p.length | G.Adj (p.getVert (f i)) x} = G.degree x := by
  rw [← card_neighborFinset_eq_degree]
  refine card_nbij (fun i => p.getVert (f i)) (fun i hi => ?_)
    (fun i₁ hi₁ i₂ hi₂ heq => ?_) ?_
  · -- i ↦ p.getVert (f i) maps the indices of neighbours of x to
    -- neighbours of x
    simpa [adj_comm] using (mem_filter.1 hi).2
  · -- it is injective on the indices of neighbours of x
    rw [coe_filter, Set.mem_ofPred_eq] at hi₁ hi₂
    exact hf_inj hi₁.1 hi₂.1 (hp.getVert_injOn (hf_le _ hi₁.1) (hf_le _ hi₂.1) heq)
  · -- it is surjective onto the neighbours of x
    intro w hw
    rw [mem_coe, mem_neighborFinset] at hw
    obtain ⟨n, rfl, hnle⟩ := mem_support_iff_exists_getVert.mp (hall _ hw)
    obtain ⟨i, hi, rfl⟩ := hf_surj n hnle hw.ne'
    exact ⟨i, mem_filter.2 ⟨hi, hw.symm⟩, rfl⟩
```

The indices `i < p.length` with `G.Adj (p.getVert (i + 1)) u` biject with the neighbours of `u`, hence number exactly `G.degree u`. (This is the previous lemma with `x = u` and `f i = i + 1`.)

```
lemma Walk.IsPath.card_succ_adj_start [DecidableRel G.Adj] {u v : V}
    [Fintype (G.neighborSet u)]
    {p : G.Walk u v} (hp : p.IsPath) (hall : ∀ w, G.Adj u w → w ∈ p.support) :
    #{i ∈ range p.length | G.Adj (p.getVert (i + 1)) u} = G.degree u := by
  refine hp.card_filter_adj_getVert_eq_degree hall (· + 1)
    (fun i hi => by grind)
    (fun a _ b _ h => by grind) ?_
  intro n hn hne
  have hn0 : n ≠ 0 := fun h0 => hne ((hp.getVert_eq_start_iff hn).mpr h0)
  exact ⟨n - 1, by grind⟩
```

The indices `i < p.length` with `G.Adj (p.getVert i) v` biject with the neighbours of v, hence number exactly `G.degree v`. (This is `IsPath.card_filter_adj_getVert_eq_degree` with `x = v` and `f i = i`.)

```
lemma Walk.IsPath.card_adj_end [DecidableRel G.Adj] {u v : V}
    [Fintype (G.neighborSet v)]
    {p : G.Walk u v} (hp : p.IsPath) (hall : ∀ w, G.Adj v w → w ∈ p.support) :
    #{i ∈ range p.length | G.Adj (p.getVert i) v} = G.degree v := by
  refine hp.card_filter_adj_getVert_eq_degree hall id
    (fun i hi => (mem_range.mp hi).le) (fun a _ b _ h => h) ?_
  intro n hn hne
  have hnl : n ≠ p.length := fun hL => hne ((hp.getVert_eq_end_iff hn).mpr hL)
  exact ⟨n, mem_range.mpr (by grind), rfl⟩
```

The next two results are about a *maximal path*. We define this as `Walk.IsMaximalPath`, a structure extending `Walk.IsPath`. (The `end` field is written «end» because `end` is a Lean keyword; French quotes turn it into an ordinary name.)

(Compare `Walk.IsMaximumPath` in the exercises: that is the stronger condition that no path anywhere in G is longer.)

A *maximal path*: a path that cannot be extended at either end, or equivalently one for which every neighbour of either endpoint already lies on it.

```
structure Walk.IsMaximalPath {u v : V} (p : G.Walk u v) : Prop extends
  isPath : p.IsPath where
  /-- Every neighbour of the start vertex lies on the path. -/
  start : ∀ w, G.Adj u w → w ∈ p.support
  /-- Every neighbour of the end vertex lies on the path. -/
  end : ∀ w, G.Adj v w → w ∈ p.support
```

For a maximal path whose endpoints satisfy Ore's degree sum condition, pigeonhole on the two counting lemmas gives a crossing pair: an index `i` with both `G.Adj (p.getVert i) v` and `G.Adj (p.getVert (i + 1)) u`.

```
lemma Walk.IsMaximalPath.exists_crossing_index [Fintype V] {u v : V}
    [Fintype (G.neighborSet u)] [Fintype (G.neighborSet v)] {p : G.Walk u v}
    (hmax : p.IsMaximalPath) (hcard : Fintype.card V ≤ G.degree u + G.degree v) :
    ∃ i, G.Adj (p.getVert i) v ∧ G.Adj (p.getVert (i + 1)) u := by
  classical
  let A := {i ∈ range p.length | G.Adj (p.getVert (i + 1)) u}
  let B := {i ∈ range p.length | G.Adj (p.getVert i) v}
  have hA : #A = G.degree u := hmax.isPath.card_succ_adj_start hmax.start
  have hB : #B = G.degree v := hmax.isPath.card_adj_end hmax.end
```

```
  -- A path has ≤ card V distinct vertices, so p.length < card V.
  have hlen : p.length + 1 ≤ Fintype.card V := hmax.isPath.length_lt
  -- A and B both sit inside range p.length, but |A| + |B| ≥ card V > length.
  have hsub : A ∪ B ⊆ range p.length :=
    union_subset (filter_subset _ _) (filter_subset _ _)
  have hcardU : #(A ∪ B) ≤ p.length :=
    (card_le_card hsub).trans_eq (card_range _)
  have hunion := card_union_add_card_inter A B
  obtain ⟨i, hi⟩ : (A ∩ B).Nonempty := by rw [← card_pos]; grind
  grind
```

Ore's Condition and Closing the Path

We can now put everything together. Given a maximal path of length ≥ 2, either the endpoints are adjacent (so it can be closed with `concat`) or the crossing pair gives a `Walk.posaClosed`. In both cases we obtain a cycle based at u containing exactly the vertices of `p`, of length `p.length + 1`.

Note that all we need for this is the degree-sum bound for the single pair of endpoints, and only when they are non-adjacent. This is *Ore's condition* at a pair of vertices, and since applying it one pair at a time will be a recurring theme, we make it a definition: `IsOrePair`.

Ore's condition at a pair of vertices: if u and v are distinct and non-adjacent then their degree sum is at least the number of vertices. (An equal or adjacent pair satisfies the condition vacuously.)

```
def IsOrePair (G : SimpleGraph V) [Fintype V] (u v : V)
    [Fintype (G.neighborSet u)] [Fintype (G.neighborSet v)] : Prop :=
  u ≠ v → ¬G.Adj u v → Fintype.card V ≤ G.degree u + G.degree v
```

The $u \neq v$ condition matters when we require Ore's condition of every pair of vertices to define an Ore graph below: without it the case $u = v$ would demand $|V| \leq 2 * \deg u$ for every vertex, which is Dirac's condition rather than Ore's.

```
lemma Walk.IsMaximalPath.exists_cycle_spanning [Fintype V] {u v : V}
    [Fintype (G.neighborSet u)] [Fintype (G.neighborSet v)] {p : G.Walk u v}
    (hmax : p.IsMaximalPath) (h2 : 2 ≤ p.length) (hcard : G.IsOrePair u v) :
    ∃ c : G.Walk u u, c.IsCycle ∧ c.length = p.length + 1 ∧
      (∀ x, x ∈ c.support ↔ x ∈ p.support) := by
  by_cases huv : G.Adj u v
  · -- Adjacent endpoints: close p with concat.
    refine ⟨p.concat huv.symm, (concat_isCycle_iff_length_ne_one p huv.symm).mpr
      ⟨hmax.isPath, by grind⟩, length_concat .., fun x => ?_⟩
    grind [support_concat, start_mem_support]
  · -- Non-adjacent endpoints: rotate via the crossing pair, close with posaClosed.
    have huv_ne : u ≠ v := by grind [isPath_iff_nil, hmax.isPath]
    obtain ⟨i, h₁, h₂⟩ := hmax.exists_crossing_index (hcard huv_ne huv)
    exact ⟨p.posaClosed h₁ h₂, hmax.isPath.isCycle_posaClosed h2 h₁ h₂,
      length_posaClosed .., fun x => by simp⟩
```

A Hamiltonian path is trivially maximal, since every vertex lies on it. So a Hamiltonian path of length at least 2 whose endpoints form an Ore pair closes up into a cycle visiting every vertex: a Hamiltonian cycle.

```
lemma Walk.IsHamiltonian.exists_isHamiltonianCycle_of_isOrePair [Fintype V]
    [DecidableEq V] {u v : V} [Fintype (G.neighborSet u)] [Fintype (G.neighborSet v)]
```

```
    {p : G.Walk u v} (hp : p.IsHamiltonian) (h2 : 2 ≤ p.length)
    (hcard : G.IsOrePair u v) : ∃ c : G.Walk u u, c.IsHamiltonianCycle := by
  have hmax : p.IsMaximalPath :=
    ⟨hp.isPath, fun w _ => hp.mem_support w, fun w _ => hp.mem_support w⟩
  obtain ⟨c, hcyc, hclen, -⟩ := hmax.exists_cycle_spanning h2 hcard
  exact ⟨c, isHamiltonianCycle_iff_isCycle_and_length_eq.2
    ⟨hcyc, hclen.trans hp.length_add_one_eq⟩⟩
```

The Bondy–Chvátal Theorem

The key observation of Bondy and Chvátal is that the degree condition above is only ever needed one pair at a time: if `u` and `v` are distinct non-adjacent vertices with `deg u + deg v ≥ |V|`, then `G` is Hamiltonian iff `G ⊔ edge u v` is Hamiltonian. Repeatedly adding such edges yields the *closure* of `G`.

The first step towards the Bondy–Chvátal theorem is a piece of cycle surgery. A Hamiltonian cycle of `G ⊔ edge u v` may use the new edge `u — v`, which is not available in `G`. Deleting `u — v` from such a cycle `c` should leave a Hamiltonian path from `u` to `v` that does lie in `G`, and the previous section then closes this path up into a Hamiltonian cycle of `G`. Hamiltonicity plays no part in the surgery itself, so we cut open an arbitrary cycle and record what the resulting path inherits from it.

We first need a simple fact about darts of a cycle `c : G.Walk u u`: if the first vertex of a dart `d` of `c` is `u`, then the second vertex of `d` is the second vertex of the cycle. Mathlib proves this for any walk whose `Walk.dropLast` is a path, and dropping the last dart of a cycle leaves a path.

```
#check Walk.IsPath.snd_eq_of_mem_darts
#check Walk.IsCycle.isPath_dropLast
```

Deleting an edge `u — v` from a cycle `c` leaves a path from `u` to `v` that is one edge shorter than `c`, visits exactly the vertices of `c`, and avoids `u — v`.

The edge `u — v` of the cycle is the image of a dart, which either points from `u` to `v` or from `v` to `u` (`dart_edge_eq_mk'_iff`). Rotating the cycle to be based at the dart's *first* vertex makes the dart the first dart of the cycle, and the reversed tail of the rotated cycle then runs from the dart's first vertex to its second. It is a path, since the tail of a cycle is a path; it visits every vertex of the cycle, since the base vertex reappears at the end of the tail; and it avoids the deleted first edge, since a cycle is a trail, so no edge is repeated. This is the required path in the first case, and its reverse is the required path in the second.

```
lemma Walk.IsCycle.exists_isPath_of_mem_edges {x u v : V} {c : G.Walk x x}
    (hc : c.IsCycle) (he : s(u, v) ∈ c.edges) :
    ∃ p : G.Walk u v, p.IsPath ∧ p.length + 1 = c.length ∧
      (∀ w, w ∈ p.support ↔ w ∈ c.support) ∧ s(u, v) ∉ p.edges := by
  classical
  -- cutting c open at a dart from a to b gives an a-b path
  have key : ∀ {a b : V}, ∀ d ∈ c.darts, d.fst = a → d.snd = b →
      ∃ p : G.Walk a b, p.IsPath ∧ p.length + 1 = c.length ∧
        (∀ w, w ∈ p.support ↔ w ∈ c.support) ∧ s(a, b) ∉ p.edges := by
    rintro a b d hd rfl rfl
    -- rotate c to be based at d.fst, so that d becomes its first dart
    have ha := c.dart_fst_mem_support_of_mem_darts hd
    have hr := hc.rotate ha
    rw [← hr.isPath_dropLast.snd_eq_of_mem_darts
```

```
      ((c.rotate_darts _ ha).perm.mem_iff.2 hd) rfl]
    -- the reversed tail of the rotated cycle is the required path
    refine ⟨_, hr.isPath_tail.reverse, ?_, fun w => ?_, ?_⟩
    · simpa using length_tail_add_one hr.not_nil
    · -- d.fst is dropped from the start of the support but is still the end
      have := cons_support_tail hr.not_nil
      grind [support_reverse, end_mem_support, mem_support_rotate_iff]
    · -- no edge is repeated, so the first edge does not occur in the tail
      have hnd := hr.edges_nodup
      rw [← (c.rotate _ ha).cons_tail_eq hr.not_nil, edges_cons] at hnd
      simpa using (List.nodup_cons.1 hnd).1
  -- the edge u — v is the image of a dart of c
  obtain ⟨d, hd, hde⟩ := List.mem_map.1 he
  rcases dart_edge_eq_mk'_iff'.1 hde with ⟨hfst, hsnd⟩ | ⟨hfst, hsnd⟩
  · -- the dart points from u to v: cut there
    exact key d hd hfst hsnd
  · -- the dart points from v to u: cut there and reverse the resulting path
    obtain ⟨p, h1, h2, h3, h4⟩ := key d hd hfst hsnd
    exact ⟨p.reverse, h1.reverse, by simpa using h2, by simpa using h3,
      by simpa [Sym2.eq_swap] using h4⟩
```

If the cycle is Hamiltonian then the path visits every vertex, so it is a Hamiltonian path (`IsPath.isHamiltonian_iff`).

```
lemma Walk.IsHamiltonianCycle.exists_isHamiltonian_walk_of_mem_edges [DecidableEq V]
    {x u v : V} {c : G.Walk x x} (hc : c.IsHamiltonianCycle)
    (he : s(u, v) ∈ c.edges) :
    ∃ p : G.Walk u v, p.IsHamiltonian ∧ s(u, v) ∉ p.edges := by
  obtain ⟨p, hp, -, hsupp, hpe⟩ := hc.isCycle.exists_isPath_of_mem_edges he
  exact ⟨p, hp.isHamiltonian_iff.2 fun w => (hsupp w).2 (hc.mem_support w), hpe⟩
```

Adding a Single Edge

We can now prove the heart of the Bondy–Chvátal theorem: if u and v satisfy Ore's condition `IsOrePair`, then $G \sqcup \mathrm{edge}\ u\ v$ is Hamiltonian iff G is.

The interesting direction takes a Hamiltonian cycle `c` of $G \sqcup \mathrm{edge}\ u\ v$. If `c` avoids the edge $u - v$ then it transfers directly to G. Otherwise our previous lemma cuts `c` open into a Hamiltonian path from u to v avoiding $u - v$, and this path transfers to G. Since the path is Hamiltonian, every neighbour of u and of v lies on it. So `IsHamiltonian.exists_isHamiltonianCycle_of_isOrePair` (via `IsMaximalPath.exists_cycle_spanning`) closes it back up into a cycle containing all of V, using a crossing pair if u and v are non-adjacent in G. This is where the degree condition enters. The reverse direction is simply monotonicity of Hamiltonicity (`IsHamiltonian.mono` in Mathlib).

In both cases we transfer a walk in $G \sqcup \mathrm{edge}\ u\ v$ that avoids the new edge $u - v$ to G, so we first record that every edge of such a walk is an edge of G.

```
lemma Walk.edges_mem_edgeSet_of_notMem {u v a b : V} (p : (G ⊔ edge u v).Walk a b)
    (hp : s(u, v) ∉ p.edges) : ∀ e ∈ p.edges, e ∈ G.edgeSet := fun e he => by
  have := p.edges_subset_edgeSet he
  grind [edgeSet_sup, edgeSet_edge_subset]

theorem isHamiltonian_sup_edge_iff [Fintype V] [DecidableEq V] {u v : V}
    [Fintype (G.neighborSet u)] [Fintype (G.neighborSet v)]
```

```
    (hcard : G.IsOrePair u v) :
    (G ⊔ edge u v).IsHamiltonian ↔ G.IsHamiltonian := by
  refine ⟨fun hG => ?_, fun hG => hG.mono le_sup_left⟩
  -- dispose of the degenerate cases, where no edge has been added
  by_cases hne : u = v
  · rwa [hne, sup_edge_self] at hG
  by_cases hadj : G.Adj u v
  · rwa [G.sup_edge_of_adj hadj] at hG
  intro h1
  obtain ⟨a, c, hc⟩ := hG h1
  -- the graph has at least three vertices, since it has a cycle
  have h3 : 3 ≤ Fintype.card V := hc.length_eq ▸ hc.isCycle.three_le_length
  by_cases hmem : s(u, v) ∈ c.edges
  · -- the cycle uses the new edge: cut it open into a Hamiltonian path
    obtain ⟨p, hham, hpe⟩ := hc.exists_isHamiltonian_walk_of_mem_edges hmem
    -- the path avoids the new edge, so it transfers to G, where we close it up
    have hq := hham.transfer (p.edges_mem_edgeSet_of_notMem hpe)
    exact ⟨u, hq.exists_isHamiltonianCycle_of_isOrePair
      (by grind [hq.length_add_one_eq]) hcard⟩
  · -- the cycle avoids the new edge: transfer it to G directly
    exact ⟨a, _, hc.transfer (c.edges_mem_edgeSet_of_notMem hmem)⟩
```

The Closure of a Graph

Call a pair of vertices *eligible* if they are distinct, non-adjacent, and have degree sum at least $|V|$. Any eligible pair is an `IsOrePair`, but not conversely: an adjacent pair satisfies `IsOrePair` vacuously but is not eligible.

By `isHamiltonian_sup_edge_iff`, adding an edge between an eligible pair does not change whether the graph is Hamiltonian, so we now simply keep adding such edges until no eligible pair remains. The resulting graph is the *Bondy–Chvátal closure* of G. A graph with no eligible pair, i.e. one in which any two distinct vertices with degree sum at least $|V|$ are already adjacent, is called *closed*. (As we will see below, Mathlib's theory of closure operators uses the same word for the same idea.)

A graph is *closed* (in the sense of Bondy and Chvátal) if it has no eligible pair: any two distinct vertices with degree sum at least $|V|$ are adjacent.

```
def IsClosed (G : SimpleGraph V) [Fintype V] [G.LocallyFinite] : Prop :=
  ∀ {x y}, x ≠ y → Fintype.card V ≤ G.degree x + G.degree y → G.Adj x y
```

How should we define such a repeated process in Lean? Every function must be total, so Lean has to be convinced that the recursion terminates. Our process does terminate: each step adds an edge, and there are only finitely many. A standard trick for such definitions is to add *fuel*: an extra natural number parameter that decreases by one at each recursive call. The recursion is then structural (on the fuel), so Lean accepts it with no termination proof at all.

Since this is our first encounter with fuel, we illustrate the idea with a simple example: computing the remainder of a division.

In `modAux` below, the remainder `a % b` is computed by repeated subtraction, using one unit of fuel per subtraction. (The test `0 < b` ensures we never waste time subtracting zero, and matches Lean's convention that `a % 0 = a`.)

Subtract b from a while `0 < b ≤ a`, at most `fuel` times.

```
private def modAux : ℕ → ℕ → ℕ → ℕ
  | 0, a, _ => a
  | fuel + 1, a, b => if 0 < b ∧ b ≤ a then modAux fuel (a - b) b else a
```

Lean accepts `modAux` with no termination proof because, in the second case, the recursive call `modAux fuel (a - b) b` is on `fuel`, which is one less than `fuel + 1`. Recursion in which every recursive call is on a smaller part of the matched argument, like `fuel` inside `fuel + 1`, is what we called *structural* above, and Lean checks this automatically.

If we start with enough fuel, `modAux` computes the remainder of a on division by b.

```
#eval modAux 6 17 5  -- 2 = 17 % 5
```

However if we start with too little fuel the computation may terminate too soon, giving the wrong answer.

```
#eval modAux 2 17 5   -- 7
```

How much fuel do we need? Since each subtraction reduces a by at least one, starting with `fuel` equal to a is always enough to reach the remainder. This can easily be proved by induction on the fuel, and in Lean such a proof can be written just like `modAux` itself: by pattern matching on the fuel, with a recursive call on `fuel` in the `fuel + 1` case. A lemma calling itself inside its own proof may look circular, but Lean accepts the call for the same reason as for `modAux`, and the recursive call is exactly the induction hypothesis. We use this pattern for the closure below.

Returning to the Bondy–Chvátal closure, the textbook definition adds eligible edges one at a time, in any order. For us it is more convenient to proceed in rounds: in each round we join all pairs that are currently eligible. A round adds at least one edge unless the graph is already closed, so starting with `fuel` equal to the number of missing edges of G (i.e. the number of edges in its complement G^c) is enough to reach a closed graph. Adding all eligible pairs at once also means that we never have to choose which pair to add next. That matters because, given only `[Fintype V]`, Lean has no computable way to pick out one eligible pair: as with `Equiv.ofBijective` in Functions, Embeddings and Equivalences (p. 98), it would have to *choose* one, making the definition `noncomputable` (unless we also assumed an ordering on V). Proceeding in rounds avoids this, so the closure is computable (at least in principle; see below).

A single round, `closureStep` below, is `G ⊔ fromRel r`, where `r u v` says that u and v have degree sum at least $|V|$. The graph `SimpleGraph.fromRel r` joins distinct u and v whenever `r u v` or `r v u`, so the new edges join exactly the eligible pairs. Since `closureStep` is an `abbrev`, Lean can see through it and find its `DecidableRel` instance automatically, from those for `⊔` and `fromRel`.

One round of the closure process: join all eligible pairs of G.

```
abbrev closureStep [Fintype V] (G : SimpleGraph V)
    [DecidableRel G.Adj] : SimpleGraph V :=
  G ⊔ fromRel fun u v => Fintype.card V ≤ G.degree u + G.degree v

lemma closureStep_adj [Fintype V] [DecidableRel G.Adj] {x y : V} :
    G.closureStep.Adj x y ↔
      G.Adj x y ∨ x ≠ y ∧ Fintype.card V ≤ G.degree x + G.degree y := by
  simp [add_comm]
```

Iterating `closureStep` needs a `DecidableRel` instance for each intermediate graph, so `closureAux` takes the instance for its graph as an argument, and each recursive call uses the instance for *G*. `closureStep` above.

Run `fuel` rounds of the closure process.

```
def closureAux [Fintype V] [DecidableEq V] :
    ℕ → (G : SimpleGraph V) → [DecidableRel G.Adj] → SimpleGraph V
  | 0, G, _ => G
  | fuel + 1, G, _ => closureAux fuel G.closureStep
```

The *Bondy–Chvátal closure* of *G*: repeatedly join eligible pairs (distinct non-adjacent vertices with degree sum at least |*V*|) until none remain.

```
variable (G) in
def closure [Fintype V] [DecidableEq V] [DecidableRel G.Adj] : SimpleGraph V :=
  closureAux #Gᶜ.edgeFinset G
```

The main result about the closure is the Bondy–Chvátal theorem: taking the closure does not change whether a graph is Hamiltonian. Before proving it we record two basic facts: the closure contains the original graph, and with enough fuel it really is closed. We start with the first of these.

```
lemma le_closureStep [Fintype V] [DecidableRel G.Adj] :
    G ≤ closureStep G := le_sup_left
```

The statement of `le_closureAux` puts the fuel, the graph and its `DecidableRel` instance after the colon, as a ∀, rather than as named arguments before it. This lets us prove it by pattern matching, with the same cases as the definition of `closureAux`. (The final _ in each pattern matches the instance, which we never need to name.) We follow the same approach when proving other results about `closureAux` below.

```
lemma le_closureAux [Fintype V] [DecidableEq V] :
    ∀ (fuel : ℕ) (G : SimpleGraph V) [DecidableRel G.Adj], G ≤ closureAux fuel G
  | 0, _, _ => le_rfl
  | fuel + 1, G, _ => le_closureStep.trans (le_closureAux fuel G.closureStep)


theorem le_closure [Fintype V] [DecidableEq V] [DecidableRel G.Adj] :
    G ≤ G.closure := le_closureAux ..
```

Now for the second fact. We first record that a round leaves a graph unchanged if the graph is closed, so that once the process reaches a closed graph it remains closed. Since `IsClosed` takes a `LocallyFinite` argument, we also need to transport closedness along an equality of graphs whose instances are unrelated; **`convert`** handles the instances.

```
lemma IsClosed.of_eq [Fintype V] {H : SimpleGraph V} [G.LocallyFinite]
    [H.LocallyFinite] (hGH : G = H) (hG : G.IsClosed) : H.IsClosed := by
  subst hGH
  convert hG


lemma IsClosed.closureStep_eq [Fintype V] [DecidableRel G.Adj]
    (hG : G.IsClosed) : closureStep G = G :=
  le_antisymm (fun _ _ h => (closureStep_adj.1 h).elim id fun ⟨hne, hd⟩ => hG hne hd)
    le_closureStep
```

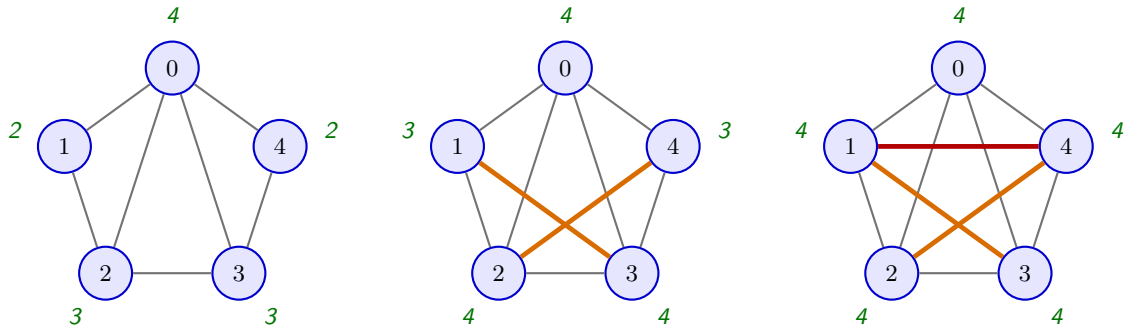
```
lemma closureAux_eq_self [Fintype V] [DecidableEq V] :
    ∀ (fuel : ℕ) (G : SimpleGraph V) [DecidableRel G.Adj], G.IsClosed →
      closureAux fuel G = G
  | 0, _, _, _ => rfl
  | fuel + 1, G, _, hG =>
    (closureAux_eq_self fuel G.closureStep
      (hG.of_eq hG.closureStep_eq.symm)).trans hG.closureStep_eq
```

To even state that `G.closure` is closed we need a `DecidableRel` instance for its adjacency relation. We define one by recursion on the fuel, from the instances for `closureStep` above. This makes adjacency in the closure decidable, but only in principle: deciding adjacency after `k` rounds needs the degrees after `k - 1` rounds, each of which needs `|V|` adjacency queries after `k - 1` rounds, and so on, so the cost grows exponentially with the number of rounds. (This could be made much more efficient but we do not need to do so here: just don't try to evaluate it on anything with more than a few vertices!)

```
instance decidableRelClosureAux [Fintype V] [DecidableEq V] :
    ∀ (fuel : ℕ) (G : SimpleGraph V) [DecidableRel G.Adj],
      DecidableRel (closureAux fuel G).Adj
  | 0, _, h => h
  | fuel + 1, G, _ => decidableRelClosureAux fuel G.closureStep

instance [Fintype V] [DecidableEq V] [DecidableRel G.Adj] :
    DecidableRel G.closure.Adj :=
  decidableRelClosureAux _ G
```

For example, `C[5]` with the two chords `0 — 2` and `0 — 3` added has 7 edges. It is not an Ore graph: the non-adjacent vertices `1` and `4` both have degree 2, so their degree sum is `4 < 5`. But the first round joins the eligible pairs `1, 3` and `2, 4`, after which `1` and `4` both have degree 3, so the second round joins them too, and the closure is the complete graph with `10` edges. So the closure can be complete even when Ore's condition fails.



Taking the `closure` of `C[5]` with the chords `0 — 2` and `0 — 3`

```
#eval (C[5] ⊔ edge 0 2 ⊔ edge 0 3).edgeFinset             -- 7 edges
#eval (C[5] ⊔ edge 0 2 ⊔ edge 0 3).closureStep.edgeFinset -- 9 edges
#eval (C[5] ⊔ edge 0 2 ⊔ edge 0 3).closure.edgeFinset     -- 10 edges (complete)
```

Conversely, a round leaves `G` unchanged only if `G` is closed, since it joins every eligible pair. So if `G` is not closed then a round adds at least one missing edge, and the number of missing edges strictly decreases.

```
lemma isClosed_iff_closureStep_eq [Fintype V] [DecidableRel G.Adj] :
    G.IsClosed ↔ G.closureStep = G := by
  refine ⟨IsClosed.closureStep_eq, fun h x y hne hd => ?_⟩
  have : G.closureStep.Adj x y := closureStep_adj.2 (Or.inr ⟨hne, hd⟩)
  rwa [h] at this

lemma card_compl_edgeFinset_closureStep_lt [Fintype V] [DecidableEq V]
    [DecidableRel G.Adj] (hG : ¬G.IsClosed) :
    #G.closureStepᶜ.edgeFinset < #Gᶜ.edgeFinset := by
  have hlt : G < G.closureStep :=
    lt_of_le_of_ne le_closureStep fun h => hG (isClosed_iff_closureStep_eq.2 h.symm)
  exact Finset.card_lt_card
    (edgeFinset_ssubset_edgeFinset.2 (compl_lt_compl_iff_lt.2 hlt))
```

We can now prove that the closure is closed, by induction on the fuel: each round that changes the graph decreases the number of missing edges, which can never go below zero, so if we start with enough fuel then the process must terminate with a closed graph.

```
lemma isClosed_closureAux [Fintype V] [DecidableEq V] :
    ∀ (fuel : ℕ) (G : SimpleGraph V) [DecidableRel G.Adj], #Gᶜ.edgeFinset ≤ fuel →
      (closureAux fuel G).IsClosed
  | 0, G, _, hf => by
    -- with no fuel left, G has no missing edges, so no eligible pair either
    intro x y hne _
    by_contra hadj
    have : s(x, y) ∈ Gᶜ.edgeFinset := by simpa [hne, closureAux] using hadj
    simp_all
  | fuel + 1, G, _, hf => by
    by_cases hG : G.IsClosed
    · -- G is already closed, and the closure does not change it
      exact hG.of_eq (closureAux_eq_self (fuel + 1) G hG).symm
    · -- some eligible pair is not yet adjacent, so a missing edge is added
      have := card_compl_edgeFinset_closureStep_lt hG
      exact isClosed_closureAux fuel G.closureStep (by grind)
```

The closure of a graph is closed.

```
theorem isClosed_closure [Fintype V] [DecidableEq V] [DecidableRel G.Adj] :
    G.closure.IsClosed :=
  isClosed_closureAux _ G le_rfl
```

We now prove the Bondy–Chvátal theorem. The lemma `isHamiltonian_sup_edge_iff` adds a single edge, whereas a round may add many, so we first add the edges of a round one at a time, by induction on a finset of pairs. Each pair was eligible in *G*, and adding edges only increases degrees, so it is still an `IsOrePair` in each of the intermediate graphs. (These intermediate graphs carry no `LocallyFinite` instances of their own, so the proof starts with **`classical`** to supply them.)

Adding any finite set of pairs that are eligible in *G* preserves Hamiltonicity.

```
lemma isHamiltonian_sup_sup_edge_iff [Fintype V] [DecidableEq V] [G.LocallyFinite]
    (s : Finset (V × V))
    (hs : ∀ p ∈ s, Fintype.card V ≤ G.degree p.1 + G.degree p.2) :
    (G ⊔ s.sup fun p => edge p.1 p.2).IsHamiltonian ↔ G.IsHamiltonian := by
  classical
```

```
  induction s using Finset.induction_on with
  | empty => simp
  | insert p s hp ih =>
    -- p still satisfies the degree condition in the bigger graph: degrees only
    -- grow when edges are added
    have hore : (G ⊔ s.sup fun q => edge q.1 q.2).IsOrePair p.1 p.2 := by
      intro _ _
      have hp' := hs p (Finset.mem_insert_self p s)
      exact hp'.trans (Nat.add_le_add (degree_le_of_le le_sup_left)
        (degree_le_of_le le_sup_left))
    rw [Finset.sup_insert, sup_comm (edge p.1 p.2), ← sup_assoc,
      isHamiltonian_sup_edge_iff hore,
      ih fun q hq => hs q (Finset.mem_insert_of_mem hq)]
```

A round adds exactly the edges `edge x y` for the pairs `(x, y)` with degree sum at least $|V|$, so a round does not change whether the graph is Hamiltonian, and nor does taking the closure. (For `x = y` the graph `edge x x` has no edges, so we don't need to exclude these pairs.)

```
lemma isHamiltonian_closureStep_iff [Fintype V] [DecidableEq V]
    [DecidableRel G.Adj] : G.closureStep.IsHamiltonian ↔ G.IsHamiltonian := by
  let s : Finset (V × V) := {p | Fintype.card V ≤ G.degree p.1 + G.degree p.2}
  have hs : closureStep G = G ⊔ s.sup fun p => edge p.1 p.2 := by
    ext x y
    simp_rw [s, Finset.sup_eq_iSup, sup_adj, iSup_adj, edge_adj]
    aesop
  rw [hs, isHamiltonian_sup_sup_edge_iff _ fun p hp => by simpa [s] using hp]

lemma isHamiltonian_closureAux_iff [Fintype V] [DecidableEq V] :
    ∀ (fuel : ℕ) (G : SimpleGraph V) [DecidableRel G.Adj],
      (closureAux fuel G).IsHamiltonian ↔ G.IsHamiltonian
  | 0, _, _ => Iff.rfl
  | fuel + 1, G, _ =>
    (isHamiltonian_closureAux_iff fuel G.closureStep).trans
      isHamiltonian_closureStep_iff
```

Bondy–Chvátal theorem: a graph is Hamiltonian iff its closure is.

```
theorem isHamiltonian_closure_iff [Fintype V] [DecidableEq V] [DecidableRel G.Adj] :
    G.closure.IsHamiltonian ↔ G.IsHamiltonian := isHamiltonian_closureAux_iff ..
```

Ore's Theorem and Dirac's Theorem

Combining the Bondy–Chvátal theorem `isHamiltonian_closure_iff` with `isHamiltonian_top`, which says that a complete graph on at least 3 vertices is Hamiltonian, gives the form we need: if G has at least 3 vertices and its closure is complete then G is Hamiltonian.

```
#check isHamiltonian_top

theorem isHamiltonian_of_closure_eq_top [Fintype V] [DecidableEq V]
    [DecidableRel G.Adj] (hn : 3 ≤ Fintype.card V) (h : G.closure = ⊤) :
    G.IsHamiltonian := by
  rw [← isHamiltonian_closure_iff, h]
```

```
  exact isHamiltonian_top hn
```

An *Ore graph*: every pair of vertices satisfies Ore's condition `IsOrePair`, i.e. any two distinct non-adjacent vertices have degree sum at least the number of vertices.

```
def IsOreGraph (G : SimpleGraph V) [Fintype V] [G.LocallyFinite] : Prop :=
  ∀ x y, G.IsOrePair x y
```

Ore's theorem is now a corollary: the closure of an Ore graph is complete. Two observations make this trivial. Firstly, Ore's condition is inherited by any supergraph: if `x ≠ y` are non-adjacent in `H ≥ G` then they are non-adjacent in `G`, so Ore's condition gives them degree sum at least `|V|` in `G`, hence also in `H`.

```
lemma IsOreGraph.mono [Fintype V] [G.LocallyFinite] {H : SimpleGraph V}
    [H.LocallyFinite] (hG : G.IsOreGraph)
    (hGH : G ≤ H) : H.IsOreGraph := fun x y hne hnadj =>
  (hG x y hne (fun h => hnadj (hGH h))).trans
    (Nat.add_le_add (degree_le_of_le hGH) (degree_le_of_le hGH))
```

Secondly, Ore's condition and closedness are complementary: the former says that non-adjacent pairs have degree sum at least `|V|`, the latter that such pairs are adjacent. So a closed Ore graph is complete.

```
lemma IsOreGraph.eq_top_of_isClosed [Fintype V] [G.LocallyFinite]
    (hO : G.IsOreGraph) (hC : G.IsClosed) : G = ⊤ :=
  eq_top_iff.2 fun x y hne => by_contra fun hn => hn (hC hne (hO x y hne hn))
```

The closure of `G` is a closed graph above `G`, so it inherits Ore's condition and is therefore complete.

```
theorem IsOreGraph.closure_eq_top [Fintype V] [DecidableEq V] [DecidableRel G.Adj]
    [G.LocallyFinite] (h : G.IsOreGraph) : G.closure = ⊤ :=
  (h.mono le_closure).eq_top_of_isClosed isClosed_closure
```

Ore's theorem: a finite graph on at least three vertices in which every pair of non-adjacent vertices has degree sum at least `|V|` is Hamiltonian.

```
theorem IsOreGraph.isHamiltonian [Fintype V] [DecidableEq V] [G.LocallyFinite]
    (h : G.IsOreGraph) (hn : 3 ≤ Fintype.card V) : G.IsHamiltonian := by
  classical
  exact isHamiltonian_of_closure_eq_top hn h.closure_eq_top
```

Dirac's theorem is an immediate corollary. We state this as `|V| ≤ 2 * δ(G)` rather than `|V| / 2 ≤ δ(G)`, because `Nat` division rounds down: for odd `|V|` the latter is a strictly weaker assumption and the statement would then be false. (Two triangles sharing a vertex have `|V| = 5` and `δ(G) = 2 = 5 / 2`, but the shared vertex is a cut vertex, so the graph has no Hamiltonian cycle.)

Dirac's theorem: a finite graph on at least three vertices satisfying `|V| ≤ 2 * δ(G)` is Hamiltonian.

```
theorem isHamiltonian_of_card_le_two_mul_minDegree [Fintype V] [DecidableEq V]
    [DecidableRel G.Adj]
    (hn : 3 ≤ Fintype.card V) (hdeg : Fintype.card V ≤ 2 * δ(G)) :
    G.IsHamiltonian := by
  refine IsOreGraph.isHamiltonian (fun x y _ _ => ?_) hn
  grind [G.minDegree_le_degree x, G.minDegree_le_degree y]
```

Since our aim is to give as many examples as possible of different graph-theoretic proof techniques in Lean, we also give the more direct classical proof of Ore's theorem, via maximum-length paths. This approach is developed in the exercises to this chapter.

The Closure is the Least Closed Supergraph

Our construction proceeds in rounds, whereas the textbook definition adds eligible pairs one at a time, in any order. A natural question is: could different orders produce a different "closure"? It is easy to see that this is not the case. The key observation is that a closed graph absorbs eligible edges: suppose $G \le K$ with K closed, and let u, v be an eligible pair in G. Adding edges only increases degrees, so u and v also have degree sum at least $|V|$ in K, and since K is closed they must be adjacent there; hence $G \sqcup \mathrm{edge}\ u\ v \le K$. Iterating, every graph obtained from G by adding eligible edges, in any order (or in rounds), lies below every closed graph above G. So if two runs of the process stop at closed graphs K_1 and K_2, then $K_1 \le K_2$ and $K_2 \le K_1$, hence they are equal.

For our rounds the absorption argument is again an induction on fuel.

```
lemma closureStep_le [Fintype V] [DecidableRel G.Adj]
    {K : SimpleGraph V} [K.LocallyFinite] (hGK : G ≤ K) (hK : K.IsClosed) :
    closureStep G ≤ K := by
  intro v w h
  rcases closureStep_adj.1 h with h1 | ⟨hne, hd⟩
  · exact hGK h1
  · exact hK hne ◁ hd.trans
      ◁ Nat.add_le_add (degree_le_of_le hGK) (degree_le_of_le hGK)

lemma closureAux_le [Fintype V] [DecidableEq V] :
    ∀ (fuel : ℕ) (G : SimpleGraph V) [DecidableRel G.Adj] (K : SimpleGraph V)
      [K.LocallyFinite], G ≤ K → K.IsClosed → closureAux fuel G ≤ K
  | 0, _, _, _, _, hGK, _ => hGK
  | fuel + 1, G, _, K, _, hGK, hK =>
    closureAux_le fuel G.closureStep K (closureStep_le hGK hK) hK

theorem closure_min [Fintype V] [DecidableEq V] [DecidableRel G.Adj]
    {K : SimpleGraph V} [K.LocallyFinite] (hGK : G ≤ K) (hK : K.IsClosed) :
    G.closure ≤ K :=
  closureAux_le _ _ _ hGK hK
```

So `G.closure` is the *least* closed graph above G: a characterization that mentions neither fuel nor rounds. Any other implementation of the closure process must therefore produce exactly the same graph; in particular the closure does not depend on the order in which eligible pairs are joined. Monotonicity and idempotence of the closure follow easily.

```
theorem closure_mono [Fintype V] [DecidableEq V] [DecidableRel G.Adj]
    {H : SimpleGraph V} [DecidableRel H.Adj] (h : G ≤ H) :
    G.closure ≤ H.closure :=
  closure_min (h.trans le_closure) isClosed_closure

@[simp]
theorem closure_closure [Fintype V] [DecidableEq V] [DecidableRel G.Adj] :
    G.closure.closure = G.closure :=
  le_antisymm (closure_min le_rfl isClosed_closure) le_closure
```

```
end bondy_chvatal
```

A monotone map on a partial order that is *inflationary* (`G ≤ G.closure`) and *idempotent* (`G.closure.closure = G.closure`) is what Mathlib calls a `ClosureOperator`, and Mathlib also calls its fixed points *closed*. (Packaging `closure` as a `ClosureOperator (SimpleGraph V)` takes a little more work, since `G.closure` needs a `DecidableRel G.Adj` instance, which an arbitrary graph does not come with.)

```
#check ClosureOperator
```

8.10 Exercises

1. Show that if `p : G.Walk u v` has length `0` then `u = v`. Hint: use `cases p`.

```
variable {V : Type*} {G : SimpleGraph V}
open HypercubeVert Walk

example {u v : V} (p : G.Walk u v) (h1 : p.length = 0) : u = v := by
  sorry
```

2. Prove by induction on `p` that a walk in `pathNat` from `i` to `j` has length at least `j - i`, so that `pathNat.path i j` is a shortest walk from `i` to `i + j`. Hint: in the `cons` case the first edge changes the vertex by one; `length_cons`.

```
example {i j : ℕ} (p : pathNat.Walk i j) : j ≤ i + p.length := by
  sorry
```

3. Let `c` be a `Bool`-colouring of `G`. Prove by induction on `p` that a walk has even length iff its endpoints have the same colour. Hint: in the `cons` case, `Bool.eq_not_of_ne` rewrites the colour of the first vertex as the negation of the colour of the second, and `Nat.even_add_one` handles the length. Finally, `cases` on a colour such as `c v` splits into the two cases `true` and `false`.

```
lemma Coloring.even_length_iff' (c : G.Coloring Bool) {u v : V} (p : G.Walk u v) :
    Even p.length ↔ c u = c v := by
  sorry
```

4. Show that splitting a walk at a vertex on its support preserves the total length. Hint: `Walk.take_spec`.

```
example [DecidableEq V] {u v w : V} (p : G.Walk u v) (h : w ∈ p.support) :
    (p.takeUntil w h).length + (p.dropUntil w h).length = p.length := by
  sorry
```

5. The walk `eulerC5` travels once around the cycle `C[5]`. Show that it is an Eulerian circuit. Hint: `decide` proves that it is Eulerian, using the `Decidable` instance from the chapter. For `IsCircuit`, rewrite with `isCircuit_def` and `isTrail_def` first.

The closed walk `0, 1, 2, 3, 4, 0` in the cycle `C[5]`.

```
private def eulerC5 : C[5].Walk 0 0 :=
  cons (show C[5].Adj 0 1 by decide) ◁ cons (show C[5].Adj 1 2 by decide) ◁
    cons (show C[5].Adj 2 3 by decide) ◁ cons (show C[5].Adj 3 4 by decide) ◁
      cons (by decide) nil

example : eulerC5.IsEulerian ∧ eulerC5.IsCircuit := by
  sorry
```

6. Use Exercise 3 and the closed walk `eulerC5` of length `5` to show that `C[5]` is not 2-colourable. Hint: `recolorOfEquiv` and `finTwoEquiv` turn a colouring by `Fin 2` into a `Bool`-colouring.

```
example : ¬C[5].Colorable 2 := by
  sorry
```

7. A `cons` walk is a Hamiltonian cycle iff its tail is Hamiltonian and has length at least two. Hint: `isHamiltonianCycle_isCycle_and_isHamiltonian_tail` and `cons_isCycle_iff_length_ne_one`.

```
lemma Walk.cons_isHamiltonianCycle_iff' [DecidableEq V] {u v : V} (p : G.Walk v u)
    (h : G.Adj u v) :
    (cons h p).IsHamiltonianCycle ↔ p.IsHamiltonian ∧ 2 ≤ p.length := by
  sorry
```

8. Show that the tail of a cycle is never nil. Hint: a cycle has length at least three.

```
lemma Walk.IsCycle.not_nil_tail' {u : V} {c : G.Walk u u} (hc : c.IsCycle) :
    ¬c.tail.Nil := by
  sorry
```

9. Show that reversing a copy of a walk is the same as copying its reverse. Hint: `subst_vars`.

```
example {u v u' v' : V} (p : G.Walk u v) (hu : u = u') (hv : v = v') :
    (p.copy hu hv).reverse = p.reverse.copy hv hu := by
  sorry
```

10. Translating the walk `pathNat.path 0 j` by `i`, with `Walk.map` along `pathNat.translate i`, gives a walk from `0 + i` to `(0 + j) + i`, so before we can compare it with `pathNat.path i j` we must `copy` it. Show that the two walks are then equal. Hint: compare supports with `Walk.ext_support`, then use `support_copy`, `Walk.support_map`, `pathNat.support_path` and `List.map_add_range'`.

```
example (i j : ℕ) :
    ((pathNat.path 0 j).map (pathNat.translate i).toHom).copy
      (by simp [pathNat.translate]) (by simp [pathNat.translate, add_comm]) =
      pathNat.path i j := by
  sorry
```

11. Show that the Hamming distance on the hypercube is symmetric.

```
lemma HypercubeVert.hammingDist_comm' {n : ℕ} (u v : HypercubeVert n) :
    u.hammingDist v = v.hammingDist u := by
  sorry
```

12. Prove that `Q[n]` is Hamiltonian unless `n = 1`. (Recall that `Q[0]` is a single vertex which is considered Hamiltonian.) Hint: `hypercube.isHamiltonian_grayPath` and `Walk.IsHamiltonian.isHamiltonian_of_adj`.

```
lemma isHamiltonian_hypercube' {n : ℕ} (h1 : n ≠ 1) : Q[n].IsHamiltonian :=
  sorry
```

13. Show that the Grötzsch graph `mμ(C[5])` is Hamiltonian. Hint: `IsHamiltonian.mycielskian` and `isHamiltonian_cycleGraph`.

```
example : grotzschGraph.IsHamiltonian := by
  sorry
```

14. If `p` is a Hamiltonian walk of length at least two and there is an edge `h₁` from its `i`-th vertex to its end and an edge `h₂` from its `(i + 1)`-th vertex to its start, then `p.posaClosed h₁ h₂` is a Hamiltonian cycle. Hint: `IsPath.isCycle_posaClosed` and `isHamiltonianCycle_iff_isCycle_and_length_eq`.

```
lemma Walk.IsHamiltonian.isHamiltonianCycle_posaClosed' {u v} {i} [DecidableEq V]
    {p : G.Walk u v} (hp : p.IsHamiltonian) (h2 : 2 ≤ p.length)
    (h₁ : G.Adj (p.getVert i) v) (h₂ : G.Adj (p.getVert (i + 1)) u) :
    (p.posaClosed h₁ h₂).IsHamiltonianCycle := by
  sorry
```

The next three exercises return to the example from the chapter's closure section: `C[5] ⊔ edge 0 2 ⊔ edge 0 3`, i.e. `C[5]` with the two chords `0 — 2` and `0 — 3`. Our instance `edgeDecidableRelAdj` from Decidability and Kernel Reduction (p. 57) takes priority over Mathlib's, so `decide` can check facts about this graph directly.

`C[5]` with the two chords `0 — 2` and `0 — 3`.

```
abbrev chordedC5 : SimpleGraph (Fin 5) := C[5] ⊔ edge 0 2 ⊔ edge 0 3
```

15. Check that the vertices `1` and `4` of `chordedC5` are not adjacent and that both have degree `2`.

```
example : ¬chordedC5.Adj 1 4 ∧ chordedC5.degree 1 = 2 ∧ chordedC5.degree 4 = 2 := by
  sorry
```

16. Show that `chordedC5` is not an Ore graph.

```
example : ¬chordedC5.IsOreGraph := by
  sorry
```

17. Show that the closure of `chordedC5` is complete, even though `chordedC5` is not an Ore graph. Hint: `ext`, `top_adj` and `decide +revert`.

```
example : chordedC5.closure = ⊤ := by
  sorry
```

The remaining exercises develop the classical direct proof of Ore's theorem via maximum-length paths. The chapter's crossing-pair machinery (`Walk.IsMaximalPath.exists_cycle_spanning`) will be useful here; the exercises supply the extremal argument that replaces the closure. We begin with some warm-up properties of Ore graphs.

18. In an Ore graph any two distinct vertices are either adjacent or have a common neighbour. Hint: if distinct vertices u and v are non-adjacent and have no common neighbour, then their neighbourhoods are disjoint subsets of $V \setminus \{u, v\}$, so their degree sum is at most $|V| - 2$, contradicting the Ore condition.

```
open Finset in
lemma IsOreGraph.adj_or_exists_adj_adj' [Fintype V] [G.LocallyFinite]
    (h : G.IsOreGraph) {u v : V} (hne : u ≠ v) :
    G.Adj u v ∨ ∃ w, G.Adj u w ∧ G.Adj v w := by
  sorry
```

19. Consequently any two vertices of an Ore graph are joined by a walk of length at most two (in the language of later chapters: an Ore graph is connected, with diameter at most two).

```
lemma IsOreGraph.exists_walk_length_le_two' [Fintype V] [G.LocallyFinite]
    (h : G.IsOreGraph) (u v : V) : ∃ p : G.Walk u v, p.length ≤ 2 := by
  sorry
```

20. An Ore graph on at least three vertices contains a vertex of degree at least two. Hint: among any three distinct vertices either some pair is non-adjacent, and so the Ore condition forces its degree sum to be at least three, or all three are pairwise adjacent.

```
open Finset in
lemma IsOreGraph.exists_two_le_degree' [Fintype V] [G.LocallyFinite]
    (h : G.IsOreGraph) (hn : 3 ≤ Fintype.card V) : ∃ w, 2 ≤ G.degree w := by
  sorry
```

21. Existence of a vertex of degree at least two implies existence of a path of length at least two.

```
lemma exists_isPath_of_two_le_degree' [G.LocallyFinite] {v : V}
    (hv : 2 ≤ G.degree v) :
    ∃ (x y : V) (q : G.Walk x y), q.IsPath ∧ 2 ≤ q.length := by
  sorry
```

Mathlib proves the existence of a maximum-length path in a graph with nonempty vertex set and finite edge set: `Walk.exists_isPath_forall_isPath_length_le_length`. Notice that the signature of this lemma doesn't actually mention the word "maximum" but spells it out explicitly. Since we will use this idea repeatedly we make a definition, bundling together the fact that p is a path

with the maximality of its length. (A note on terminology: such a p has *maximum* length: no path anywhere in G is longer. This is stronger than being *maximal*, which would only say that p itself cannot be extended to a longer path.)

A *maximum-length path*: a path such that no path in G is longer.

```
structure Walk.IsMaximumPath {u v : V} (p : G.Walk u v) : Prop extends
  isPath : p.IsPath where
  /-- No path in G is longer than p. -/
  length_le : ∀ ⦃x y : V⦄ ⦃q : G.Walk x y⦄, q.IsPath → q.length ≤ p.length
```

22. Restate Mathlib's existence result `exists_isPath_forall_isPath_length_le_length` compactly.

```
variable (G) in
lemma Walk.exists_isMaximumPath' [Nonempty V] [Finite G.edgeSet] :
    ∃ (u v : V) (p : G.Walk u v), p.IsMaximumPath := by
  sorry
```

23. The reverse of a maximum-length path is again a maximum-length path.

```
lemma Walk.IsMaximumPath.reverse' {u v : V} {p : G.Walk u v} (h : p.IsMaximumPath) :
    p.reverse.IsMaximumPath := by
  sorry
```

24. A maximum-length path in an Ore graph on at least three vertices has length at least two.

```
lemma IsOreGraph.two_le_length_of_isMaximumPath' [Fintype V] [G.LocallyFinite]
    {u v} (h : G.IsOreGraph)
    (hn : 3 ≤ Fintype.card V) {p : G.Walk u v} (hmax : p.IsMaximumPath) :
    2 ≤ p.length := by
  sorry
```

25. A maximum-length path is in particular a *maximal* path, in the sense of the chapter's `Walk.IsMaximalPath`: neither endpoint has a neighbour missing from it. Hint: if a neighbour of the start were missing then `cons` would give a strictly longer path; for the end vertex apply the same argument to `p.reverse`, using `IsMaximumPath.reverse'`.

```
lemma Walk.IsMaximumPath.isMaximalPath' {u v : V} {p : G.Walk u v}
    (hmax : p.IsMaximumPath) : p.IsMaximalPath := by
  sorry
```

26. If p is a walk from a vertex in a set `S` to a vertex outside of `S`, there are adjacent vertices `c` $\in$ `S` and `d` $\notin$ `S`. Hint: `Walk.exists_boundary_dart`.

```
lemma Walk.exists_cross_adj' {a b} {S : Set V} (p : G.Walk a b) (ha : a ∈ S)
    (hb : b ∉ S) : ∃ c ∈ S, ∃ d ∉ S, G.Adj c d := by
  sorry
```

27. In a non-nil closed walk the first vertex also occurs as the last, so a vertex lies in the tail of the support iff it lies in the support.

```
lemma Walk.mem_support_tail_iff' {a : V} {c : G.Walk a a} (hc : ¬c.Nil) {x : V} :
    x ∈ c.support.tail ↔ x ∈ c.support := by
  sorry
```

28. A cycle can be cut at any of its vertices to give a path from that vertex visiting exactly the cycle's vertices, of length one less than the cycle. Hint: deduce it from the chapter's `IsCycle.exists_isPath_of_mem_edges`, cutting the cycle at the first edge of `c.rotate y hy` (`Walk.mk_start_snd_mem_edges`, `Walk.rotate_edges`).

```
lemma Walk.IsCycle.exists_isPath_spanning' {a : V} {c : G.Walk a a}
    (hc : c.IsCycle) {y : V} (hy : y ∈ c.support) :
    ∃ (z : V) (q : G.Walk y z), q.IsPath ∧ q.length + 1 = c.length ∧
      (∀ x, x ∈ q.support ↔ x ∈ c.support) := by
  sorry
```

29. If p is a maximum-length path from u, every vertex is reachable from u, and some cycle through u of length one more than p visits exactly the vertices of p, then p is Hamiltonian. (Neither Ore's condition nor finiteness is needed.) Hint: if p missed a vertex, a walk from u to it and `exists_cross_adj'` would yield an edge leaving the cycle. Break the cycle with `IsCycle.exists_isPath_spanning'` and extend the resulting path with this edge to give a strictly longer path, contradicting maximality.

```
lemma Walk.IsMaximumPath.isHamiltonian_of_isCycle' [DecidableEq V] {u v : V}
    {p : G.Walk u v} (hmax : p.IsMaximumPath) (hreach : ∀ w, G.Reachable u w)
    {c : G.Walk u u} (hc : c.IsCycle) (hlen : c.length = p.length + 1)
    (hsupp : ∀ x, x ∈ c.support ↔ x ∈ p.support) : p.IsHamiltonian := by
  sorry
```

30. A maximum-length path in an Ore graph is Hamiltonian. Hint: apply `IsMaximumPath.isHamiltonian_of_isCycle'`. Ore's condition supplies the cycle, via `IsOreGraph.two_le_length_of_isMaximumPath'` and `IsMaximalPath.exists_cycle_spanning` (from the chapter; p is a maximal path by Exercise 25), and the reachability, via `IsOreGraph.exists_walk_length_le_two'`.

```
lemma Walk.isHamiltonian_of_isMaximumPath' [Fintype V] [DecidableEq V]
    [G.LocallyFinite] (h : G.IsOreGraph)
    (hn : 3 ≤ Fintype.card V) {u v : V} {p : G.Walk u v}
    (hmax : p.IsMaximumPath) : p.IsHamiltonian := by
  sorry
```

31. Give a direct proof of Ore's theorem, not using the closure: take a maximum-length path; it is Hamiltonian of length at least two, so `IsHamiltonian.exists_isHamiltonianCycle_of_isOrePair` closes it into a Hamiltonian cycle.

```
example [Fintype V] [DecidableEq V] [G.LocallyFinite] (h : G.IsOreGraph)
    (hn : 3 ≤ Fintype.card V) : G.IsHamiltonian := by
  sorry
```

9 Subgraphs

Informally, a *subgraph* of a graph `G` is a graph `H` with `V(H) ⊆ V(G)` and `E(H) ⊆ E(G)`. We have already seen the subgraph order `H ≤ G` on `SimpleGraph V`, which describes subgraphs with *the same vertex set* (or more precisely the same vertex type). For a general subgraph we need to allow the vertex set to vary. Mathlib formalizes this with the type `SimpleGraph.Subgraph`.

In this chapter we introduce `SimpleGraph.Subgraph` and the bounded lattice structure of subgraphs. We then look at ways of constructing subgraphs: from vertices, edges and walks, by taking induced subgraphs, and by deleting vertices and edges. Since a `Subgraph` is not a `SimpleGraph` we next consider the ways to convert between the two (`Subgraph.coe`, `Subgraph.spanningCoe` and `toSubgraph`), spanning subgraphs, and the parts of the `SimpleGraph` API that have to be duplicated for `Subgraph`. Since we now have four different ways of talking about subgraphs, we compare them and discuss when to use each. As an application we use the *De Bruijn–Erdős theorem* (`colorable_of_forall_finite_subgraph_colorable`) to extend our greedy colouring bound to infinite graphs (`colorable_of_degree_le`). We then introduce maps and comaps of subgraphs, with subgraphs of the Mycielskian as a worked example.

Matchings are subgraphs, so we next define the *matching number* `ν G` of a graph and prove weak duality `ν G ≤ τ G` (`matchingNum_le_vertexCoverNum`), where `τ G` is the *vertex cover number*. We then compute the matching number of the hypercube (`matchingNum_hypercube_succ`) and bound that of the Mycielskian (`two_mul_matchingNum_le_matchingNum_mycielskian`).

The final section is more advanced: we introduce *partial colourings* (`ColoringOn`), colourings that need only be valid on a given set of vertices. These are the main tool in our proof of *Brooks' theorem* in the next chapter Brooks' Theorem (p. 304).

9.1 Subgraphs

Given `G : SimpleGraph V`, `G.Subgraph` is the type of general subgraphs of `G`.

```
open Subgraph
section basics
variable {V : Type*} {G : SimpleGraph V} {H : G.Subgraph}

#source Subgraph
```

If `H : G.Subgraph` then we have the following data:

```
#check H.verts      -- Set V the vertex set of the subgraph H
#check H.Adj        -- V → V → Prop the adjacency relation of the subgraph H
#check H.adj_sub    -- H.Adj v w → G.Adj v w (i.e. E(H) ⊆ E(G))
#check H.edge_vert  -- H.Adj v w → v ∈ H.verts edges only join vertices of H
```

```
#check H.symm       -- Std.Symm H.Adj: H.Adj v w → H.Adj w v (unoriented edges)

example (H : G.Subgraph) (v w : V) (h : H.Adj v w) : H.Adj w v := h.symm
```

In order to prove that two subgraphs H_1 and H_2 of G : SimpleGraph V are equal we need to show that they have the same vertex set and the same edges. We can do this using `ext` as we did for SimpleGraph V (Subgraph also has the `@[ext]` attribute on its definition) but now this will give us two goals: one for equality of vertex sets and one for equality of adjacency relations (edges).

```
example (H₁ H₂ : G.Subgraph) (h₁ : H₁.verts = H₂.verts)
    (h₂ : ∀ v w, H₁.Adj v w ↔ H₂.Adj v w) : H₁ = H₂ := by
  ext v w
  · rw [h₁]
  · exact h₂ v w
```

The Subgraph Lattice

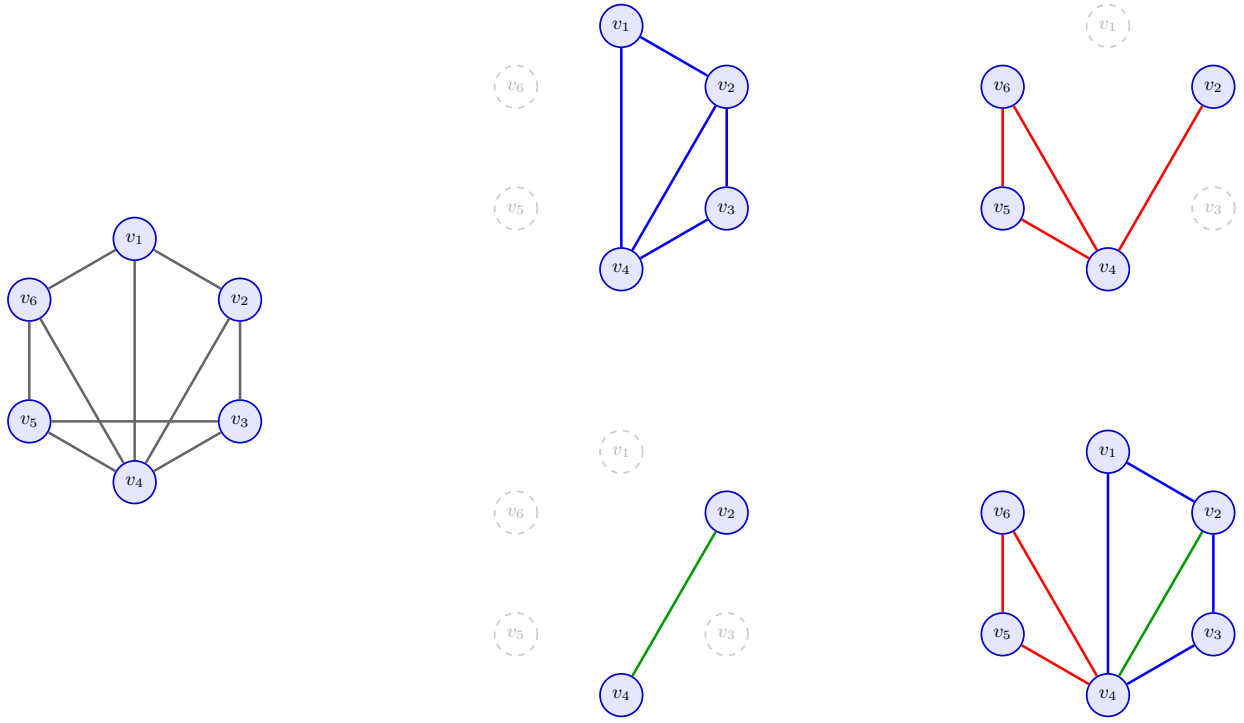
The subgraphs of a graph G form a complete bounded lattice under the partial order $H_1 \le H_2$ defined by H_1.verts $\subseteq$ H_2.verts and H_1.Adj v w → H_2.Adj v w. (So $H_1 \le H_2$ means that all vertices and edges of H_1 are also vertices and edges of H_2.)

```
example (H₁ H₂ : G.Subgraph) (h : H₁ ≤ H₂) (v w : V) (hadj : H₁.Adj v w) :
    H₂.Adj v w := h.2 hadj

example (H₁ H₂ : G.Subgraph) (h : H₁ ≤ H₂) (v : V) (hv : v ∈ H₁.verts) :
    v ∈ H₂.verts := h.1 hv
```

The `inf` of two subgraphs has vertex set given by the intersection of the vertex sets and edge set given by the intersection of the edge sets. While for `sup` it is the union of both.

```
#check Subgraph.verts_inf
#check Subgraph.inf_adj
#check Subgraph.verts_sup
#check Subgraph.sup_adj
```



Left: a graph `G`. Top: subgraphs `H₁` and `H₂` of `G`, with the vertices they omit dashed.
Bottom: their inf `H₁ ⊓ H₂` and sup `H₁ ⊔ H₂`, with the edges in both subgraphs in green

If `G : SimpleGraph V` then the top subgraph `⊤` of `G` has the same vertex set and edge set as `G`, so mathematically `⊤ : G.Subgraph` *is* `G` (but with type `G.Subgraph` rather than `SimpleGraph V`). The bottom subgraph `⊥` has empty vertex set and edge set.

```
#check Subgraph.verts_top
#check Subgraph.top_adj
#check Subgraph.verts_bot
#check Subgraph.not_bot_adj
#synth CompleteLattice (G.Subgraph)
```

Recall that `SimpleGraph V` is a Boolean algebra, so any `G : SimpleGraph V` has a complement `Gᶜ` with `G ⊓ Gᶜ = ⊥` and `G ⊔ Gᶜ = ⊤`.

The `G.Subgraph` lattice is *not* Boolean: in fact it is complemented if and only if `G` is the empty graph. To see why complements fail to exist, suppose that `G` is not the empty graph and has an edge `u – v`. Take `H₁` to be the subgraph with vertex set `{u}` and no edges, and suppose that `H₂` is a complement of `H₁`; i.e. `H₁ ⊓ H₂ = ⊥` and `H₁ ⊔ H₂ = ⊤`. Since `H₁.verts` and `H₂.verts` are disjoint, neither `H₁` nor `H₂` can contain the edge, so `H₁ ⊔ H₂` also cannot contain it. [1]

```
#synth BooleanAlgebra (SimpleGraph V)
#synth HeytingAlgebra G.Subgraph
```

[1] `H : G.Subgraph` does in fact have a different kind of complement `Hᶜ : G.Subgraph` defined in Mathlib, but this is the **Heyting complement** of `H`. This is the subgraph of `G` induced by `H.vertsᶜ`; i.e. `(⊤ : G.Subgraph).induce H.vertsᶜ`. (We leave the proof as an exercise.)

9.2 Constructing Subgraphs

There are several ways to construct a `G.Subgraph`. In this section we look at some common ones: from a single vertex, edge or walk of `G`; by taking the subgraph induced on a set of vertices; and by deleting vertices or edges from an existing subgraph.

Singletons, Edges and Walks

Any vertex or edge of a `SimpleGraph` gives rise to a subgraph consisting of that vertex or edge. Similarly any walk gives rise to a subgraph consisting of the vertices and edges of the walk.

```
section walksetc
variable {u v w : V}

#source SimpleGraph.singletonSubgraph -- verts := {v}, Adj := ⊥ ...

#source SimpleGraph.subgraphOfAdj -- verts := {v, w}, Adj a b := s(v, w) = s(a, b)
```

The `Subgraph` corresponding to a walk `p` is the subgraph with vertex set the vertices of `p` and edge set the edges of `p`.

```
#source Walk.toSubgraph -- p.toSubgraph : G.Subgraph
```

The vertex set of the subgraph of a walk is exactly the set of vertices in its support.

```
example (p : G.Walk u v) :
  p.toSubgraph.verts = {w | w ∈ p.support} := p.verts_toSubgraph
```

If we append two walks, the subgraph of the resulting walk is the `sup` of the subgraphs of the original walks.

```
example (p : G.Walk u v) (q : G.Walk v w) :
  (p.append q).toSubgraph = p.toSubgraph ⊔ q.toSubgraph := p.toSubgraph_append _
```

We can also talk about the `support` of any subgraph `H : G.Subgraph`, which is the `Subgraph` analogue of `SimpleGraph.support`; i.e. the set of vertices incident to edges of the subgraph.

```
#source Subgraph.support  -- def support (H : Subgraph G) : Set V
                          --        := SetRel.dom {(v, w) | H.Adj v w}
```

So we now have three different notions of support: `Walk.support`, `SimpleGraph.support` and `Subgraph.support`. The last two are mathematically the same, but `Walk.support` is different in two ways: it is a `List` rather than a `Set`, and if `p : G.Walk u v` is a `Nil` walk then `p.support = [u]` while `p.toSubgraph.support = ∅`.

```
#check Walk.support_nil

example (p : G.Walk u v) (h : p.Nil) : p.toSubgraph.support = ∅ := by
  ext x
  cases h
  simp [Subgraph.mem_support]
```

For non-nil walks the `Walk.toSubgraph.support` is the same as the set of vertices in the `List V` `Walk.support`.

```
example (p : G.Walk u v) (h : ¬p.Nil) :
  p.toSubgraph.support = {w | w ∈ p.support} := by
  ext x
  simp only [Subgraph.mem_support, Walk.adj_toSubgraph_iff_mem_edges,
    Sym2.mem_iff_exists, Walk.mem_support_iff_exists_mem_edges_of_not_nil h]
  grind
```

Recall that `Set.ncard` is essentially `Nat.card` for sets: both take the junk value `0` for infinite types/sets (see Cardinality (p. 66)).

If `c : G.Walk u u` is a cycle and `v` is a vertex in `c` then the set of neighbours of `v` in the subgraph corresponding to `c` has cardinality two. (Here `Subgraph.neighborSet` is the set of neighbours *within* the subgraph; we meet it properly in Duplication (p. 254).)

```
example {c : G.Walk u u} (hpc : c.IsCycle) (h : v ∈ c.support) :
  (c.toSubgraph.neighborSet v).ncard = 2 := hpc.ncard_neighborSet_toSubgraph_eq_two h

end walksetc
```

Induced Subgraphs

If `H : G.Subgraph` and `s : Set V` then `H.induce s` is the subgraph of `H` induced by `s`, and also has type `G.Subgraph`.

```
#source Subgraph.induce -- G'.induce s : G.Subgraph (for G' : G.Subgraph)

example {u v : V} (s : Set V) :
    (H.induce s).Adj u v ↔ u ∈ s ∧ v ∈ s ∧ H.Adj u v := by rfl
```

The vertex set of a subgraph induced by `s : Set V` is `s`. (Notice that this allows an induced subgraph of a subgraph to have vertices that were not present in the original subgraph.)

```
example (s : Set V) {H : G.Subgraph} : (H.induce s).verts = s := rfl
```

Recall that Mathlib also has `SimpleGraph.induce`. This gives the induced subgraph of `G : SimpleGraph V` on a set `s : Set V` as a `SimpleGraph ↑s` (here `↑s` denotes the subtype of `V` consisting of elements in `s`).

In particular if `G : SimpleGraph V` and `s : Set V` then `(⊤ : G.Subgraph).induce s` and `G.induce s` both correspond to what we would usually call the subgraph of `G` induced by `s`. Often `(⊤ : G.Subgraph).induce s` is more convenient to work with since it has the same vertex type as `G` and adjacency is nicely defined.

```
example {u v : V} (s : Set V) :
  ((⊤ : G.Subgraph).induce s).Adj u v ↔ u ∈ s ∧ v ∈ s ∧ G.Adj u v := Iff.rfl
```

For example expressing adjacency in an induced subgraph of an induced subgraph is straightforward with `Subgraph.induce`.

```
example {u v : V} (s t : Set V) : (((⊤ : G.Subgraph).induce s).induce t).Adj u v ↔
    u ∈ s ∩ t ∧ v ∈ s ∩ t ∧ G.Adj u v := Iff.intro (by simp_all) (by simp_all)
```

Compare the above with the corresponding statement using `SimpleGraph.induce` below.

Since `G.induce s` is a `SimpleGraph ↑s`, the second `induce` below cannot take a `Set V`; it needs a `Set ↑s`. To lift `t` to `↑s` we must assume `t ⊆ s` (and even then the lifted set is written as the preimage `(Subtype.val : s → V) ⁻¹' t`). Vertices of the graph are doubly nested subtype elements, so the `Adj` arguments have to be built explicitly out of `u v : V` together with proofs of their memberships. The resulting statement is almost unreadable (even if the proof is easy).

```
example {s t : Set V} (hts : t ⊆ s) {u v : V} (hu : u ∈ t) (hv : v ∈ t) :
    ((G.induce s).induce ((Subtype.val : s → V) ⁻¹' t)).Adj
        ⟨⟨u, hts hu⟩, hu⟩ ⟨⟨v, hts hv⟩, hv⟩ ↔ G.Adj u v := Iff.rfl
```

As well as constructing induced subgraphs there is a predicate for a subgraph to *be* induced. A subgraph is induced if whenever we take two vertices from its vertex set that are adjacent in the original graph then they are also adjacent in the subgraph.

```
#source Subgraph.IsInduced -- ∀ ⦃v⦄, v ∈ H.verts → ∀ ⦃w⦄, w ∈ H.verts →
                           --   G.Adj v w → H.Adj v w
```

So the induced subgraphs are exactly the subgraphs of the form `(⊤ : G.Subgraph).induce s`.

```
#check Subgraph.isInduced_iff_exists_eq_induce_top
```

Deleting Vertices and Edges

If `H : G.Subgraph` then we can form a new subgraph by deleting any set of vertices `s : Set V` from `H` (while also obviously removing any edges incident to those vertices).

Vertex deletion is defined by taking the subgraph of `H` induced by `H.verts \ s`.

```
example (s : Set V) {H : G.Subgraph} :
  H.deleteVerts s = H.induce (H.verts \ s) := rfl
```

A pair of vertices `v w : V` are adjacent in the subgraph obtained by deleting a set of vertices `s : Set V` from a subgraph `H : G.Subgraph` if and only if they are not in `s` and are adjacent in `H`. (Mathlib's `Subgraph.deleteVerts_adj` also includes the conditions `v ∈ H.verts` and `w ∈ H.verts`, which follow from `H.Adj v w`.)

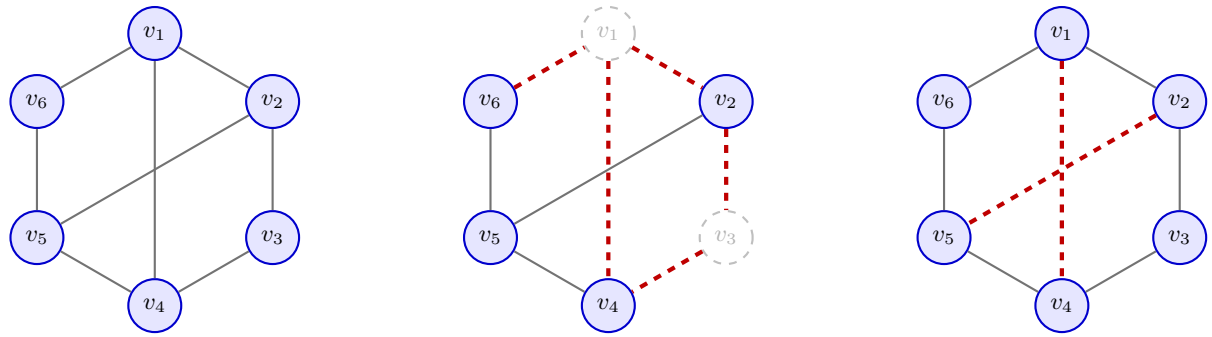
```
lemma Subgraph.deleteVerts_adj' {v w : V} (s : Set V) :
    (H.deleteVerts s).Adj v w ↔ v ∉ s ∧ w ∉ s ∧ H.Adj v w := by
  grind [Subgraph.induce_adj, H.edge_vert, Subgraph.Adj.symm]
```

We can also delete a set of edges `s : Set (Sym2 V)` from a subgraph (while leaving the vertex set unchanged).

```
#source Subgraph.deleteEdges -- G'.deleteEdges s : G.Subgraph

example {v w : V} (s : Set (Sym2 V)) :
    (H.deleteEdges s).Adj v w ↔ H.Adj v w ∧ s(v, w) ∉ s := by rfl

#check Subgraph.deleteEdges_verts -- (G'.deleteEdges s).verts = G'.verts
```



Left: a graph `G`. Centre: deleting the vertices `v₁` and `v₃`, which also removes their edges. Right: deleting the edges `v₁ — v₄` and `v₂ — v₅`

9.3 Subgraphs as Simple Graphs

A `G.Subgraph` is not a `SimpleGraph`, so results about the latter do not automatically apply to the former. In this section we look at how to pass between the two. We cover the coercions from a subgraph to a simple graph and back, and spanning subgraphs, for which the two points of view essentially agree. We also look at the parts of the `SimpleGraph` API that Mathlib duplicates for `Subgraph`.

Coercions

Mathematically `⊤ : G.Subgraph` is just `G`, but comparing the two is a type mismatch error:

```
/- error:
Type mismatch
  ⊤
has type
  G.Subgraph
but is expected to have type
  SimpleGraph V
-/
#check G = (⊤ : G.Subgraph)
```

In order to treat a `Subgraph` as a `SimpleGraph` we need to use a *coercion*.

There are two different ways to do this. If `H : G.Subgraph` then `H.coe` gives a graph on the vertex type `H.verts`, the subtype of `V` consisting of the vertices of `H`, i.e. `{v // v ∈ H.verts}`. The other coercion is `H.spanningCoe` which gives a graph on the vertex type `V` with the edges of `H` but with all the vertices of `G`.

```
#source Subgraph.coe -- H.coe : SimpleGraph ↑H.verts

#source Subgraph.spanningCoe -- H.spanningCoe : SimpleGraph V

example : (⊤ : G.Subgraph).spanningCoe = G := rfl
example : (⊥ : G.Subgraph).spanningCoe = ⊥ := rfl
```

The second example hides a change of meaning: `⊥ : G.Subgraph` is the *empty* subgraph, with no vertices, while `⊥ : SimpleGraph V` is the empty graph on `V`, which has every vertex of `V` and no edges. This is because `spanningCoe` forgets the vertex set and keeps only the edges. In particular

it is not injective: two subgraphs have the same `spanningCoe` iff they have the same edges, whatever their vertices. (For example the spanning coercion of a single-vertex subgraph `G.singletonSubgraph v` is also `⊥`; see the exercises.)

```
#check Subgraph.spanningCoe_inj -- H₁.spanningCoe = H₂.spanningCoe ↔ H₁.Adj = H₂.Adj
```

The two coercions are related through `SimpleGraph.spanningCoe` from SpanningCoe (p. 108): if we take the graph `H.coe` on `↑H.verts` and then add back the missing vertices of `V`, we get `H.spanningCoe`.

```
#check Subgraph.spanningCoe_coe -- H.coe.spanningCoe = H.spanningCoe
```

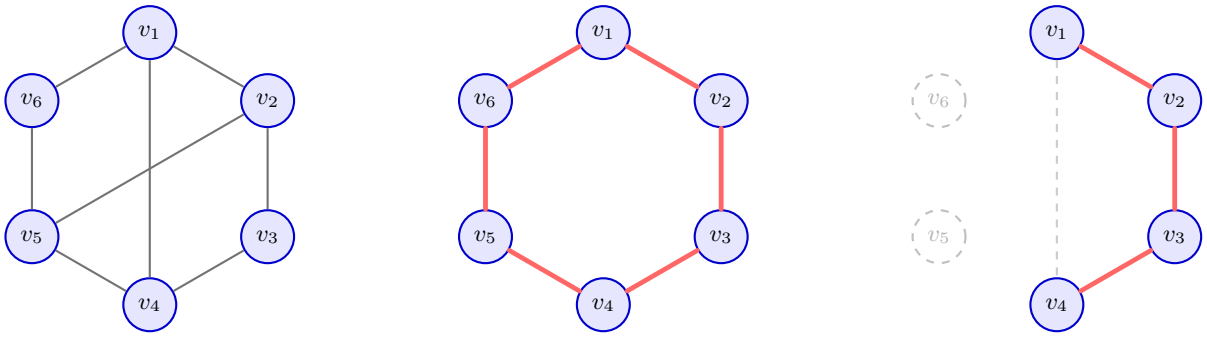
Note that the word *coercion* is used loosely here: `H.coe` and `H.spanningCoe` are ordinary functions, not coercions in Lean's technical sense. There is no `Coe` instance behind them, so Lean never inserts them for you: you must write `H.coe` or `H.spanningCoe` explicitly to get the corresponding `SimpleGraph`. [2]

In the opposite direction, if `G H : SimpleGraph V` and `H ≤ G` then we can convert `H` to a `G.Subgraph` using `toSubgraph H h` where `h : H ≤ G`.

```
#source SimpleGraph.toSubgraph
```

Spanning Subgraphs

A subgraph `H : G.Subgraph` is *spanning* if `H.verts = Set.univ`, i.e. it contains every vertex in `V`.



A graph `G`, a spanning subgraph, and a non-spanning subgraph

```
example (H : G.Subgraph) : H.IsSpanning ↔ H.verts = Set.univ := isSpanning_iff
```

If `G H : SimpleGraph V` and `h : H ≤ G` then `toSubgraph H h` is a spanning subgraph of `G`.

```
example (H : SimpleGraph V) (h : H ≤ G) : (toSubgraph H h).IsSpanning :=
  toSubgraph.isSpanning _ _
```

If we take the `spanningCoe` of a `toSubgraph` then we get back the original `SimpleGraph V`.

```
example (H : SimpleGraph V) (h : H ≤ G) : (toSubgraph H h).spanningCoe = H := rfl
```

The subgraph `⊤` is spanning.

[2] An example of a genuine coercion is the one from `ℕ` to `ℝ`: if `n : ℕ` and `x : ℝ` then `x + n` type-checks because Lean inserts `↑n : ℝ` automatically. We have in fact just seen an example of such a coercion: the `↑` in `SimpleGraph ↑H.verts`, which turns the set `H.verts` into a type.

```
example : (⊤ : G.Subgraph).IsSpanning := fun _ => trivial
```

The spanningCoe of the G.Subgraph consisting of a single edge is the SimpleGraph V constructed from the edge set containing this single edge.

```
example {v w : V} (hadj : G.Adj v w) :
    (G.subgraphOfAdj hadj).spanningCoe = fromEdgeSet {s(v, w)} :=
  Subgraph.spanningCoe_subgraphOfAdj hadj
```

If H : G.Subgraph then there is a graph embedding of H.coe into H.spanningCoe.

```
#check coeEmbeddingSpanningCoe
```

So a colouring of H.spanningCoe restricts to a colouring of H.coe. Conversely, a colouring of H.coe extends to H.spanningCoe by giving every vertex outside H colour 0, since these vertices are isolated in H.spanningCoe. This needs at least one colour: if H has no vertices then H.coe is 0-colourable, but H.spanningCoe is not unless V is empty.

```
lemma Subgraph.colorable_spanningCoe_iff {n : ℕ} (hn : 1 ≤ n) :
    H.spanningCoe.Colorable n ↔ H.coe.Colorable n := by
  classical
  refine ⟨fun h => h.of_hom H.coeEmbeddingSpanningCoe.toHom, fun ⟨C⟩ => ?_⟩
  refine ⟨Coloring.mk (fun v => if h : v ∈ H.verts then C ⟨v, h⟩ else ⟨0, hn⟩)
    fun {v w} hvw => ?_⟩
  simpa [H.edge_vert hvw, H.edge_vert hvw.symm] using
    C.valid (by simpa : H.coe.Adj ⟨v, _⟩ ⟨w, _⟩)
```

If H : G.Subgraph is a spanning subgraph then H.spanningCoe and H.coe are isomorphic. (They cannot be equal, since the former is a SimpleGraph V while the latter is a SimpleGraph ↑H.verts where H.verts = Set.univ.)

```
example (H : G.Subgraph) (h : H.IsSpanning) :
    H.spanningCoe ≃g H.coe := H.spanningCoeEquivCoeOfSpanning h
```

Duplication

For a G.Subgraph we don't have access to the many definitions we have already seen for SimpleGraph V such as neighborSet, degree, edgeSet, incidenceSet, support, map, comap (as well as some we will meet later such as Connected). These have to be defined separately for G.Subgraph and then related back to the original graph G when needed. This means there is a certain amount of unavoidable duplication between the theory of Subgraph and SimpleGraph. We have already met Subgraph.support; some of the others are listed below.

```
#source Subgraph.neighborSet  -- the neighbour set of a vertex in a subgraph
#source Subgraph.degree       -- the number of neighbours of a vertex in a subgraph
#source Subgraph.edgeSet      -- the set of edges of a subgraph
#source Subgraph.incidenceSet -- the set of edges incident to a vertex in a subgraph
```

The degree of a vertex of a subgraph is at most the degree of that vertex in the original graph.

```
#check Subgraph.degree_le
```

The duplicated definitions are related to each other by bridging lemmas. For example `Subgraph.coe_degree` says that the degree of `v` in the graph `H.coe` is its degree in the subgraph `H`. This lets us apply a theorem about degrees in a `SimpleGraph` to a subgraph, as we do in the proof of `colorable_of_degree_le` below.

```
#check Subgraph.coe_degree -- H.coe.degree v = H.degree ↑v
```

Which Notion of Subgraph?

We now have four different ways of saying that `H` is a subgraph of `G`. Which one we choose can make a big difference to how easy a proof is.

Representation	Vertex type	Use it when
`H ≤ G`	`V`	every vertex is kept, e.g. deleting edges
`G.induce s`, `H.coe`	`↑s`, `↑H.verts`	you need `SimpleGraph` theorems
`H : G.Subgraph`	`V`	the vertex set varies, e.g. matchings
`Copy A G`, `A ⊑ G`	any type	`A` is a fixed graph, e.g. `A`-free graphs

- `H ≤ G` with `H : SimpleGraph V` (Order (p. 11)) keeps the vertex type, so the whole `SimpleGraph` API applies and these subgraphs form a Boolean algebra.
- `G.induce s` (Induced Subgraphs via Comap (p. 100)) and `H.coe` are genuine `SimpleGraphs`, so every theorem about `SimpleGraph` applies to them, but their vertices live in a subtype. We pay for this whenever we pass between `v : V` and `⟨v, hv⟩ : ↑s`.
- `H : G.Subgraph` lets the vertex set vary inside `V`, and the subgraphs form a lattice. Use it when Mathlib defines the notion you need for subgraphs, such as `Walk.toSubgraph` or matchings (`Subgraph.IsMatching`). It is also the natural choice when we need several subgraphs with different vertex sets, since they all have the same type and can be compared with `≤` and combined with `⊔` and `⊓`.
- `Copy A G` is a labelled copy of a fixed graph `A` inside `G` whose vertices may have any type.

A single argument often needs more than one of these. The De Bruijn–Erdős theorem below is an example. Its proof treats the finite subgraphs of `G` as elements of `G.Subgraph`, where any two of them lie in their union `H₁ ⊔ H₂`. But colourings and homomorphisms are defined only for `SimpleGraphs`, so the theorem is stated for the graphs `H.coe`.

We have already met most of the conversions between these: `toSubgraph H h` turns `h : H ≤ G` into a `G.Subgraph`, while `H.spanningCoe` and `H.coe` turn `H : G.Subgraph` into a `SimpleGraph`. Copies and subgraphs are related by the following three definitions.

```
#source Copy.toSubgraph -- toSubgraph (f : Copy A B) : B.Subgraph := .map f.toHom ⊤
#source Copy.isoToSubgraph -- (f : Copy A B) : A ≃g f.toSubgraph.coe
#source Subgraph.coeCopy -- (H : G.Subgraph) : Copy H.coe G
```

Finally there are useful hybrids. When we define partial colourings in Partial Colourings (p. 268) we will use the graph `((⊤ : G.Subgraph).induce s).spanningCoe` whose edges are those of `G` induced by the set of vertices `s`. Since it is a `SimpleGraph V`, we can use the `SimpleGraph.Coloring` API.

De Bruijn–Erdős Theorem

The *De Bruijn–Erdős theorem* [de Bruijn–Erdős 1951] says that if every finite subgraph of a (possibly infinite) graph `G` is n-colourable then `G` itself is n-colourable.

Mathlib contains the following natural generalization of this theorem: if every finite subgraph of `G` has a homomorphism to a *finite* graph `F` then there is a homomorphism from `G` to `F`. [3]

```
#check nonempty_hom_of_forall_finite_subgraph_hom
```

Since an n-colouring is a homomorphism to `K[n]`, the De Bruijn–Erdős theorem is the special case in which `F` is `K[n]`.

De Bruijn–Erdős theorem: if every finite subgraph of `G` is n-colourable then `G` is n-colourable.

```
theorem colorable_of_forall_finite_subgraph_colorable {n : ℕ}
    (h : ∀ H : G.Subgraph, H.verts.Finite → H.coe.Colorable n) : G.Colorable n :=
  nonempty_hom_of_forall_finite_subgraph_hom fun H hH => (h H hH).some
```

This allows us to give the general greedy colouring bound using our earlier result for countable graphs, `colorable_of_degree_le_of_countable`, since a finite subgraph is countable.

```
theorem colorable_of_degree_le {Δ : ℕ} [G.LocallyFinite]
    (hdeg : ∀ v, G.degree v ≤ Δ) : G.Colorable (Δ + 1) := by
  refine colorable_of_forall_finite_subgraph_colorable fun H hH => ?_
  classical
  have : Finite H.verts := hH.to_subtype
  exact colorable_of_degree_le_of_countable fun _ =>
    H.coe_degree _ ▸ (H.degree_le _).trans (hdeg _)
```

It remains to explain why `nonempty_hom_of_forall_finite_subgraph_hom` is true.

If `G` is finite there is nothing to prove, since `G` is then a finite subgraph of itself, so suppose that `G` is infinite. The standard proof is a compactness argument. Let `W` be the vertex type of `F`, with the discrete topology. Since `W` is finite, the space of all functions `V → W` is compact by Tychonoff's theorem. A homomorphism from `G` to `F` is a function `φ : V → W` satisfying one condition for each edge `u – v` of `G`, namely that `φ u` and `φ v` are adjacent in `F`. Let `A ⊆ W × W` be the set of pairs `(a, b)` with `a` adjacent to `b` in `F`. For each edge `u – v` of `G`, let `πuv : (V → W) → W × W` be the projection `πuv(φ) = (φ u, φ v)` onto the coordinates `u` and `v`. The functions satisfying the condition for `u – v` then form the set `πuv⁻¹(A)`, which is closed: `πuv` is continuous, being a projection in the product topology, and every subset of the discrete space `W × W` is closed.

The homomorphisms from `G` to `F` are exactly the functions in the intersection of all the sets `πuv⁻¹(A)`, so we must show that this intersection is nonempty. In a compact space, a family of closed sets has nonempty intersection provided it has the *finite intersection property*: any finitely many of the sets have a common point. Given finitely many edges of `G`, let `H` be the finite subgraph consisting of these edges and their endpoints. By assumption there is a homomorphism from `H` to `F`, and extending it arbitrarily to the other vertices gives a function in the sets `πuv⁻¹(A)` for all of the given edges. (Such an extension exists because `W` is nonempty: `G` is infinite, so it has a vertex `v`, and by assumption the finite subgraph `G.singletonSubgraph v` has a homomorphism to `F`.)

Mathlib organizes the argument differently, so that most of it becomes an instance of a general compactness result. Instead of a single function on all of `V`, it looks for a homomorphism from each

[3] The finiteness of `F` cannot be dropped: every finite subgraph of `τ : SimpleGraph ℕ` has a homomorphism to the disjoint union of all the complete graphs `K[n]`, but `τ` itself does not; see the exercises for details.

finite subgraph, chosen so that they fit together. A *consistent family* consists of a homomorphism `ψ H` from `H` to `F` for each finite subgraph `H` of `G`, such that whenever `H₁ ≤ H₂`, the homomorphism `ψ H₁` is the restriction of `ψ H₂` to `H₁`.

Any homomorphism `φ` from `G` to `F` gives a consistent family, by taking `ψ H` to be the restriction of `φ` to `H`. Conversely, a consistent family can be glued together to give a homomorphism from `G` to `F`: send each vertex `v` to its image under `ψ (G.singletonSubgraph v)`. If `h : G.Adj u v`, then the singleton subgraphs of `u` and `v` are both contained in the subgraph `G.subgraphOfAdj h` consisting of the edge `u — v`. So by consistency `u` and `v` are sent to their images under `ψ (G.subgraphOfAdj h)`, which are adjacent in `F`. Restricting and gluing are inverse to each other. (In the language of category theory, the consistent families form the *inverse limit* of the sets of homomorphisms from the finite subgraphs, so restriction identifies the set of homomorphisms from `G` to `F` with this inverse limit.)

So it is enough to find a consistent family, and for this only three facts matter:

1. The finite subgraphs form a *directed* set: any two have a common upper bound, namely their union.

2. The set of homomorphisms from each finite subgraph to `F` is finite (as `F` is finite) and nonempty (by assumption).

3. The restriction maps are *compatible*. Restricting a homomorphism from `H` to itself leaves it unchanged. If `H₁ ≤ H₂ ≤ H₃`, then restricting a homomorphism from `H₃` to `H₂` and then to `H₁` gives the same result as restricting it from `H₃` to `H₁` directly.

To find a consistent family we again use Tychonoff's theorem. For each finite subgraph `H`, let `X H` be the set of homomorphisms from `H` to `F`, with the discrete topology. Since each `X H` is finite, the product `P` of all the sets `X H` is compact. A point `ψ` of `P` is a choice of a homomorphism `ψ H` in `X H` for every finite subgraph `H`. For each pair `H₁ ≤ H₂`, whether `ψ` satisfies the consistency condition for this pair depends only on `(ψ H₁, ψ H₂)`. So the points satisfying it form the preimage of a subset of `X H₁ × X H₂` under the projection `ψ ↦ (ψ H₁, ψ H₂)`, and this set is closed for the same reason as before: the projection is continuous and `X H₁ × X H₂` is discrete.

The consistent families are exactly the points in the intersection of all these closed sets, so again it is enough to check the finite intersection property. Given finitely many pairs `H₁ ≤ H₂`, let `K` be the union of all the subgraphs involved, a finite subgraph containing all of them; this is where directedness is used. By assumption there is a homomorphism `θ` from `K` to `F`. Take `ψ H` to be the restriction of `θ` to `H` for each subgraph `H` involved, and choose the other `ψ H` arbitrarily, which is possible since each `X H` is nonempty. For each of the given pairs, `ψ H₁` is the restriction of `θ` to `H₁`, which by compatibility is the restriction to `H₁` of `ψ H₂`. So `ψ` satisfies all of the given conditions, and by compactness a consistent family exists. The Mathlib proof is this argument, but in the language of category theory, with the compactness step proved as a general result:

```
open CategoryTheory in
#check nonempty_sections_of_finite_inverse_system
```

In our application, `J` is `G.Finsubgraph`, the directed set of finite subgraphs of `G`. The functor in the theorem packages the sets `X H` and their restriction maps. It is defined on `Jᵒᵖ` because restriction goes from larger to smaller subgraphs; its *sections* are the consistent families. Mathlib then glues a section into a homomorphism from `G` to `F` exactly as above. The same general result also gives Hall's

marriage theorem for infinitely many sets (Hall's Marriage Theorem (p. 406)) and the existence of ends (Ends (p. 444)).

9.4 Maps and Comaps

`Subgraph.map` is a *pushforward* operation: given a graph homomorphism `f : G →g K`, between `G : SimpleGraph V` and `K : SimpleGraph W`, and a subgraph `H : G.Subgraph`, `H.map f` is the subgraph of `K` whose vertex set is `f '' H.verts` and whose edges are the images `f u – f v` of edges `u – v` of `H`. Note the contrast with `SimpleGraph.map` (Maps (p. 107)), which takes a plain function: here `f` must be a homomorphism, since the image has to be a subgraph of the given graph `K`.

```
#source Subgraph.map
```

There is also `Subgraph.comap` which is a *pullback* operation: given a homomorphism `f : G →g K` and a subgraph `H` of `K`, `H.comap f` is the subgraph of `G` whose vertex set is the preimage `f⁻¹' H.verts` and whose edges are those edges `u – v` of `G` for which `f u` and `f v` are adjacent in `H`.

```
#source Subgraph.comap
```

As for simple graphs (`SimpleGraph.map_le_iff_le_comap`), `map f` and `comap f` form a Galois connection. Here this holds for every homomorphism `f`, not just for embeddings: a homomorphism never sends two adjacent vertices to the same vertex, so no edge is lost when we map forwards.

```
#check Subgraph.map_le_iff_le_comap -- H.map f ≤ H' ↔ H ≤ H'.comap f
```

We have `Subgraph.inclusion` which gives the graph homomorphism between two subgraphs `H ≤ K` (coerced to `SimpleGraph` on their respective vertex sets).

```
#check Subgraph.inclusion -- {H K : G.Subgraph} (h : H ≤ K) : H.coe →g K.coe
```

More generally, a homomorphism `f : G →g K` restricts to a homomorphism between subgraphs `A` of `G` and `B` of `K` whenever `B` contains the image `A.map f`. Mathlib only has the special case `Subgraph.inclusion` (where `f = Hom.id`), so we define it ourselves since we will need it later.

A homomorphism `f : G →g K` restricts to a homomorphism from a subgraph `A` of `G` to any subgraph `B` of `K` containing `A.map f`.

```
def Subgraph.homOfMapLE {W : Type*} {K : SimpleGraph W}
    (f : G →g K) {A : G.Subgraph} {B : K.Subgraph} (h : A.map f ≤ B) :
    A.coe →g B.coe where
  toFun v := ⟨f v, h.1 ⟨v, v.2, rfl⟩⟩
  map_rel' hvw := h.2 ⟨_, _, hvw, rfl, rfl⟩
```

We also have `Subgraph.hom` which gives the graph homomorphism from a subgraph `H` to the original graph `G`.

```
#check Subgraph.hom -- .. (H : G.Subgraph) : H.coe →g G
```

Combining `Subgraph.map` with `Subgraph.hom` gives `Subgraph.coeSubgraph` which allows us to take a subgraph of a subgraph coerced to `SimpleGraph` and get a subgraph of the original graph.

```
#source Subgraph.coeSubgraph --  G'.coe.Subgraph → G.Subgraph
```

```
example {A : G.Subgraph} (B : A.coe.Subgraph) (v w : V) : (A.coeSubgraph B).Adj v w
    ↔ ∃ (hv : v ∈ A.verts) (hw : w ∈ A.verts), B.Adj ⟨_, hv⟩ ⟨_, hw⟩ :=
  Subgraph.coeSubgraph_adj B v w
```

Combining `Subgraph.comap` with `Subgraph.hom` gives `Subgraph.restrict`: given subgraphs `A` and `B` of `G`, `A.restrict B` is the part of `B` that lies in `A`, as a subgraph of `A.coe` (i.e. with vertex type `A.verts`).

```
#source Subgraph.restrict
```

Vertices of `A` are adjacent in `A.restrict B` iff they are adjacent in both `A` and `B`.

```
example {A B : G.Subgraph} (v w : A.verts) :
    (A.restrict B).Adj v w ↔ A.Adj v w ∧ B.Adj v w := Iff.rfl
```

Subgraphs of Mycielskians

As worked examples of `Subgraph.map` and `Subgraph.comap` we consider subgraphs of the Mycielskian `mμ(G)`. If `H : G.Subgraph` then we can map `H` to a subgraph of `mμ(G)` using the embedding of `G` into `mμ(G)` given by `Embedding.mycielskianLeft G`.

The subgraph of the Mycielskian `mμ(G)` corresponding to a subgraph `H` of `G`.

```
@[simps!]
def Subgraph.mycielskianLeft (H : G.Subgraph) : mμ(G).Subgraph :=
  Subgraph.map (Embedding.mycielskianLeft G).toHom H
```

Pulling `mycielskianLeft H` back along the same embedding recovers `H`.

```
lemma Subgraph.comap_mycielskianLeft (H : G.Subgraph) :
    H.mycielskianLeft.comap (Embedding.mycielskianLeft G).toHom = H := by
  ext v w <;> simp_all [Relation.Map, H.adj_sub]
```

Combined with the Galois connection `Subgraph.map_le_iff_le_comap`, this shows that `mycielskianLeft` both preserves and reflects the subgraph order.

```
lemma Subgraph.mycielskianLeft_le_mycielskianLeft_iff {H₁ H₂ : G.Subgraph} :
    H₁.mycielskianLeft ≤ H₂.mycielskianLeft ↔ H₁ ≤ H₂ := by
  rw [Subgraph.mycielskianLeft, Subgraph.map_le_iff_le_comap,
    Subgraph.comap_mycielskianLeft]

end basics
```

9.5 Matching Number

Mathlib has the predicates `Subgraph.IsMatching` and `Subgraph.IsPerfectMatching` which say that a `Subgraph` is a matching or a perfect matching, as well as `SimpleGraph.IsMatchingFree` which says that a graph has no perfect matching.

```
section matchingnumber
variable {V : Type*} {G : SimpleGraph V}

#source Subgraph.IsMatching        -- ∀ ⦃v⦄, v ∈ M.verts → ∃! w, M.Adj v w
```

```
#source Subgraph.IsPerfectMatching -- M.IsMatching ∧ M.IsSpanning
#source SimpleGraph.IsMatchingFree --  ∀ M : Subgraph G, ¬M.IsPerfectMatching
```

A matching `M` is not spanning in general: its vertex set is the set of *matched* vertices, and each of these has exactly one neighbour in `M`. A perfect matching is then simply a spanning matching. (This makes it natural to describe a matching as a `G.Subgraph`.)



Left: a matching `M`, which leaves `c` and `f` unmatched. Right: a perfect matching

For example, a graph, or rather its `⊤` subgraph, is a perfect matching iff it is `1`-regular, i.e. every vertex has exactly one neighbour.

```
lemma Subgraph.isPerfectMatching_top_iff [G.LocallyFinite] :
    (⊤ : G.Subgraph).IsPerfectMatching ↔ G.IsRegularOfDegree 1 := by
  simp [isPerfectMatching_iff, IsRegularOfDegree,
    SimpleGraph.degree_eq_one_iff_existsUnique_adj]
```

Mathlib has the classical existence theorems (Hall's marriage theorem and Tutte's theorem) which we will discuss in Matchings (p. 406). But it does not (yet) define the *matching number* `ν G`: the maximum number of edges in a matching of `G`. Following Mathlib's `SimpleGraph.vertexCoverNum`, we define it as an indexed supremum in the extended naturals `ℕ∞`, counting the edges of each matching with `Set.encard`. So a graph with arbitrarily large matchings has matching number `⊤`.

```
#check Set.encard
#source vertexCoverNum
```

We introduce the standard `τ G` notation for the vertex cover number.

```
@[inherit_doc vertexCoverNum]
scoped notation "τ" => SimpleGraph.vertexCoverNum
```

The single `⨆` in the definition below is notation for the nested supremum `⨆ (M : G.Subgraph), ⨆ (_ : M.IsMatching), M.edgeSet.encard`, whose inner supremum ranges over proofs that `M` is a matching. Every `M : G.Subgraph` is an index of the outer supremum, but if `M` is not a matching the inner supremum is empty and contributes `0`, which does not change the result, so in effect only matchings count.

The *matching number* `ν G` of `G` is the maximum number of edges in a matching of `G`, as an element of `ℕ∞` (so `⊤` if `G` has arbitrarily large matchings).

```
noncomputable def matchingNum (G : SimpleGraph V) : ℕ∞ :=
  ⨆ (M : G.Subgraph) (_ : M.IsMatching), M.edgeSet.encard
```

We introduce the standard notation `ν G` for the matching number.

```
@[inherit_doc matchingNum]
scoped notation "ν" => SimpleGraph.matchingNum
```

Whenever we make a new definition we should immediately prove the basic lemmas that let us use it without unfolding it: its *API*. Because $\mathbb{N}\infty$ is a complete lattice the indexed supremum always exists, with no nonemptiness or boundedness side-condition to discharge (an `sSup`-in-$\mathbb{N}$ definition would need both). The two directions of the supremum are then immediate from the generic `iSup` API.

Any matching gives a lower bound for the matching number.

```
lemma Subgraph.IsMatching.le_matchingNum {M : G.Subgraph}
    (h : M.IsMatching) : M.edgeSet.encard ≤ ν G :=
  le_iSup₂ (f := fun (M : G.Subgraph) (_ : M.IsMatching) => M.edgeSet.encard) M h
```

To bound the matching number from above it suffices to bound the size of every matching.

```
lemma matchingNum_le {n : ℕ∞}
    (h : ∀ M : G.Subgraph, M.IsMatching → M.edgeSet.encard ≤ n) : ν G ≤ n :=
  iSup₂_le h
```

The empty subgraph is a matching (vacuously, since it has no vertices), so the supremum defining $\nu\ G$ is never over an empty family.

```
lemma Subgraph.isMatching_bot : (⊥ : G.Subgraph).IsMatching :=
  fun _ hv => hv.elim
```

It is sometimes convenient to take the supremum over the subtype of matchings, rather than over the doubly indexed family.

```
lemma matchingNum_eq_iSup_subtype : ν G =
    ⨆ M : {M : G.Subgraph // M.IsMatching}, (M : G.Subgraph).edgeSet.encard :=
  iSup_subtype'
```

For a finite type `Set.encard` is just the coercion of `Set.ncard`, and Mathlib's `Set.coe_ncard_eq_encard` records this. Rewriting with it right-to-left lets us move matching sizes from $\mathbb{N}\infty$ down to $\mathbb{N}$, where we can do arithmetic.

```
#check Set.coe_ncard_eq_encard
```

In a finite graph the supremum is attained by some matching. Completeness of $\mathbb{N}\infty$ does not give this for free, but with `[Finite V]` there are only finitely many subgraphs (`Subgraph.instFinite`), so the supremum, taken over the finite nonempty subtype of matchings, is a maximum (`exists_eq_ciSup_of_finite`). [4]

```
variable [Finite V] in
#synth Finite G.Subgraph

lemma exists_isMatching_matchingNum [Finite V] :
    ∃ M : G.Subgraph, M.IsMatching ∧ M.edgeSet.encard = ν G := by
  have : Nonempty {M : G.Subgraph // M.IsMatching} := ⟨⟨⊥, Subgraph.isMatching_bot⟩⟩
  obtain ⟨M, hM⟩ := exists_eq_ciSup_of_finite
    (f := fun x : {M : G.Subgraph // M.IsMatching} =>
      (x : G.Subgraph).edgeSet.encard)
  exact ⟨M, M.2, by rw [matchingNum_eq_iSup_subtype]; exact hM⟩
```

[4] Mathematically, finiteness is not necessary here, but the proof when $\nu\ G = \tau$ is more involved, requiring either a greedy chain argument or Zorn's lemma. See the exercises for details.

As a sanity check we show that the empty graph has matching number `0`: it has no edges, so neither does any of its matchings. (The API we have developed, in particular `matchingNum_le`, ensures our proof does not need to mention indexed suprema.)

```
lemma matchingNum_bot : ν (⊥ : SimpleGraph V) = 0 := by
  refine nonpos_iff_eq_zero.mp ◁ matchingNum_le fun M _ => ?_
  simp [Set.subset_empty_iff.mp (edgeSet_bot ▸ M.edgeSet_subset)]
```

The edges of `M.coe`, a graph on `M.verts`, correspond to the edges of `M` itself (`Subgraph.image_coe_edgeSet_coe`), so the two edge sets have the same size.

```
lemma Subgraph.ncard_edgeSet_coe (M : G.Subgraph) :
    M.coe.edgeSet.ncard = M.edgeSet.ncard := by
  rw [← M.image_coe_edgeSet_coe,
    Set.ncard_image_of_injective _ (Sym2.map.injective Subtype.val_injective)]
```

The number of vertices covered by a matching is twice its number of edges. The proof applies the degree-sum formula to the graph `M.coe`, in which every vertex has degree `1`.

```
lemma Subgraph.IsMatching.ncard_verts [Finite V] {M : G.Subgraph}
    (h : M.IsMatching) : M.verts.ncard = 2 * M.edgeSet.ncard := by
  classical
  have := Fintype.ofFinite V
  have hverts : M.verts.toFinset.card = 2 * M.coe.edgeFinset.card := by
    trans Fintype.card M.verts
    · exact Set.toFinset_card M.verts
    · rw [← M.coe.sum_degrees_eq_twice_card_edges, ← Finset.card_univ,
        Finset.card_eq_sum_ones]
      congr with x
      rw [coe_degree, eq_comm, degree_eq_one_iff_existsUnique_adj]
      obtain ⟨w, hw, huniq⟩ := h x.2
      exact ⟨w, hw, fun y hy => huniq _ hy⟩
  rw [Set.ncard_eq_toFinset_card', hverts, ← M.ncard_edgeSet_coe,
    Set.ncard_eq_toFinset_card']
  rfl
```

Mathlib's `Subgraph.IsMatching.toEdge` sends each vertex of a matching `M` to the edge of `M` containing it. Two distinct vertices sent to the same edge are its two endpoints, so they are adjacent in `M`. (This is the converse of Mathlib's `Subgraph.IsMatching.toEdge_eq_toEdge_of_adj`.)

```
lemma Subgraph.IsMatching.adj_of_toEdge_eq {M : G.Subgraph} (hM : M.IsMatching)
    {x y : M.verts} (hxy : hM.toEdge x = hM.toEdge y) (hne : (x : V) ≠ y) :
    M.Adj x y := by
  -- x and y both lie on the edge at y
  have hx : (x : V) ∈ (hM.toEdge y : Sym2 V) := hxy ▸ hM.mem_coe_toEdge x.2
  -- since x ≠ y, this edge must be s(x, y)
  have he : (hM.toEdge y : Sym2 V) = s((x : V), y) :=
    (Sym2.mem_and_mem_iff hne).1 ⟨hx, hM.mem_coe_toEdge y.2⟩
  rw [← Subgraph.mem_edgeSet, ← he]
  exact (hM.toEdge y).2
```

If p_1 is an independent set then every edge of a matching has at most one endpoint in p_1. So a matching with finitely many edges has at least as many edges as it has vertices in p_1: sending each such vertex to its edge in the matching is injective.

```
lemma Subgraph.IsMatching.ncard_inter_le_of_isIndepSet {p₁ : Set V}
    (h₁ : G.IsIndepSet p₁) {M : G.Subgraph} [Finite M.edgeSet]
    (hM : M.IsMatching) :
    (p₁ ∩ M.verts).ncard ≤ M.edgeSet.ncard := by
  rw [← Nat.card_coe_set_eq, ← Nat.card_coe_set_eq M.edgeSet]
  -- send each matched vertex of p₁ to its edge in M
  refine Nat.card_le_card_of_injective (fun v => hM.toEdge ⟨v, v.2.2⟩)
    fun x y hxy => ?_
  by_contra hne
  have hne' : (x : V) ≠ y := fun h => hne (Subtype.ext h)
  -- two distinct vertices on the same edge of M are adjacent, contradicting the
  -- independence of p₁
  exact h₁ x.2.1 y.2.1 hne' (hM.adj_of_toEdge_eq hxy hne').adj_sub
```

In particular, if G is bipartite with parts p_1 and p_2 (`IsBipartiteWith`) then p_1 is independent, so a matching with finitely many edges has at least as many edges as it has vertices in p_1. We use this later in the proof of Kőnig's Theorem (p. 409).

```
lemma Subgraph.IsMatching.ncard_inter_le {p₁ p₂ : Set V}
    (h₁ : G.IsBipartiteWith p₁ p₂) {M : G.Subgraph} [Finite M.edgeSet]
    (hM : M.IsMatching) :
    (p₁ ∩ M.verts).ncard ≤ M.edgeSet.ncard :=
  hM.ncard_inter_le_of_isIndepSet fun _ hx _ hy _ h =>
    Set.disjoint_left.1 h₁.disjoint hy (h₁.mem_of_mem_adj hx h)
```

A matching covers two vertices per edge, so the matching number of any finite graph is at most `|V| / 2`.

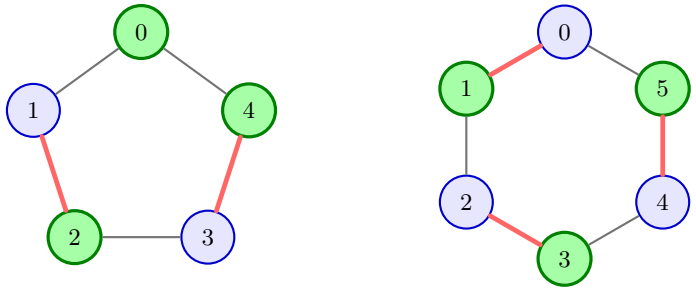
```
lemma matchingNum_le_card_div_two [Finite V] : ν G ≤ ↑(Nat.card V / 2) :=
  matchingNum_le fun N hN => by
    have := N.verts.ncard_le_card
    rw [← Set.coe_ncard_eq_encard, Nat.cast_le]
    grind [hN.ncard_verts]
```

If G has a perfect matching then this bound is attained: `ν G = |V| / 2`.

```
lemma Subgraph.IsPerfectMatching.matchingNum_eq [Finite V] {M : G.Subgraph}
    (h : M.IsPerfectMatching) : ν G = ↑(Nat.card V / 2) := by
  -- The perfect matching covers all of V, so it has |V| / 2 edges.
  have hcard : 2 * M.edgeSet.ncard = Nat.card V := by
    rw [← h.1.ncard_verts, h.2.verts_eq_univ, Set.ncard_univ]
  refine matchingNum_le_card_div_two.antisymm ◁ h.1.le_matchingNum.trans' ?_
  rw [← Set.coe_ncard_eq_encard, Nat.cast_le]
  grind
```

When `|V|` is even, the `|V| / 2` bound is tight exactly when G has a perfect matching.

A second upper bound comes from vertex covers. *Weak duality* says that `ν G ≤ τ G`, where `τ G` is the minimum size of a *vertex cover*, a set of vertices meeting every edge: the edges of a matching are pairwise disjoint, so a vertex cover must contain a distinct vertex from each of them. For *bipartite* graphs this is an equality (*Kőnig's theorem*, which is not currently in Mathlib; we prove it in Kőnig's Theorem (p. 409)), but it can be strict: the odd cycle `C[2 * k + 3]` has `ν = k + 1` and `τ = k + 2`. (The exercises check the smallest case, `K[3]`.)



Left: in `C[5]` a maximum matching (red) has 2 edges, but a minimum vertex cover (green) needs 3 vertices, so `ν = 2 < 3 = τ`. Right: in the bipartite graph `C[6]` the two are equal, `ν = τ = 3`

The proof of `ν G ≤ τ G` rests on two elementary facts, which we isolate as reusable lemmas.

The first says that a matching is *locally injective* on edges: since every vertex of a matching is incident to a unique edge, two matching edges that share a vertex must coincide.

```
lemma Subgraph.IsMatching.eq_of_mem_edge {M : G.Subgraph} (hM : M.IsMatching)
    {e₁ e₂ : Sym2 V} (he₁ : e₁ ∈ M.edgeSet) (he₂ : e₂ ∈ M.edgeSet)
    {v : V} (hv₁ : v ∈ e₁) (hv₂ : v ∈ e₂) : e₁ = e₂ := by
  obtain ⟨a, rfl⟩ := Sym2.mem_iff_exists.mp hv₁
  obtain ⟨b, rfl⟩ := Sym2.mem_iff_exists.mp hv₂
  exact congrArg (s(v, ·)) (hM.eq_of_adj_left he₁ he₂)
```

The second is the defining property of a vertex cover: every edge of `G` contains one of the cover's vertices.

```
lemma IsVertexCover.exists_mem_edge {C : Set V} (hC : G.IsVertexCover C)
    {e : Sym2 V} (he : e ∈ G.edgeSet) : ∃ v ∈ e, v ∈ C := by
  induction e using Sym2.ind with
  | _ a b => simpa using hC he
```

Weak duality: the matching number of a graph is at most its vertex cover number.

```
theorem matchingNum_le_vertexCoverNum (G : SimpleGraph V) :
    ν G ≤ τ G := by
  obtain ⟨C, hcard, hcover⟩ := G.vertexCoverNum_exists
  rw [← hcard]
  refine matchingNum_le fun M hM => ?_
  -- Each edge of the matching contains a vertex of the cover C.
  have hmem : ∀ e : M.edgeSet, ∃ v : C, (v : V) ∈ (e : Sym2 V) := fun e => by
    obtain ⟨v, hv, hvC⟩ := hcover.exists_mem_edge (M.edgeSet_subset e.2)
    exact ⟨⟨v, hvC⟩, hv⟩
  -- Sending each edge to a chosen cover vertex is an injection into C: distinct
  -- matching edges pick distinct vertices, by local injectivity of the matching.
  choose f hf using hmem
  exact ENat.card_le_card_of_injective (f := f) fun e₁ e₂ h =>
    Subtype.ext (hM.eq_of_mem_edge e₁.2 e₂.2 (hf e₁) (h ▸ hf e₂))
```

Matching Number of the Hypercube

The hypercube `Q[n + 1]` has a natural *perfect matching*: pair each vertex with the one obtained by flipping its first coordinate, so `v[0, …]` is matched with `v[1, …]`. (Flipping the first coordinate rather than the last lets us build the matching by recursion on `n`.)

Rather than reason about the hypercube directly, we isolate the underlying general fact about box products: if M is a perfect matching of G, then copying M into every G-fibre of $G \square H$ is a perfect matching of $G \square H$, for any graph H, whose edges play no role at all.

Copy a subgraph M of G into every G-fibre of $G \square H$: the vertex (v, b) is present iff $v \in M.\mathrm{verts}$, and (a, b) and (c, b) are adjacent iff $M.\mathrm{Adj}\ a\ c$. This is the box product of M with the edgeless (but spanning) subgraph on H.

```
@[simps]
def Subgraph.boxProdLeft {β : Type*} (M : G.Subgraph) (H : SimpleGraph β) :
    (G □ H).Subgraph where
  verts := {x | x.1 ∈ M.verts}
  Adj x y := M.Adj x.1 y.1 ∧ x.2 = y.2
  adj_sub h := boxProd_adj.2 (Or.inl ⟨M.adj_sub h.1, h.2⟩)
  edge_vert h := M.edge_vert h.1
  symm := ⟨fun _ _ ⟨h1, h2⟩ => ⟨h1.symm, h2.symm⟩⟩
```

If M is a perfect matching of G then $M.\mathrm{boxProdLeft}\ H$ is a perfect matching of $G \square H$: the unique neighbour of (v, b) is (w, b), where w is the M-partner of v.

```
lemma Subgraph.IsPerfectMatching.boxProdLeft {β : Type*} {M : G.Subgraph}
    (h : M.IsPerfectMatching) (H : SimpleGraph β) :
    (M.boxProdLeft H).IsPerfectMatching := by
  rw [Subgraph.isPerfectMatching_iff]
  rintro ⟨v, b⟩
  obtain ⟨w, hw, huniq⟩ := h.1 (h.2 v)
  refine ⟨(w, b), ⟨hw, rfl⟩, ?_⟩
  rintro ⟨c, d⟩ ⟨hadj, rfl⟩
  exact Prod.ext (huniq c hadj) rfl

end matchingnumber
```

The recursive perfect matching of the hypercube is now immediate.

A perfect matching of Q_{n+1} defined recursively: $\top$ on Q_1, and on Q_{n+2} the $\mathrm{Subgraph.boxProdLeft}$ of $\mathrm{hypercube.matching}\ n$ along K_2.

```
def hypercube.matching : (n : ℕ) → Q[n + 1].Subgraph
  | 0 => ⊤
  | n + 1 => (hypercube.matching n).boxProdLeft ⊤
```

By induction $\mathrm{hypercube.matching}\ n$ is a perfect matching of Q_{n+1}.

```
lemma hypercube.isPerfectMatching_matching :
    ∀ n : ℕ, (hypercube.matching n).IsPerfectMatching
  | 0 => Subgraph.isPerfectMatching_top_iff.mpr ◁ IsRegularOfDegree.hypercube _
  | n + 1 => (hypercube.isPerfectMatching_matching n).boxProdLeft ⊤
```

Since Q_{n+1} has 2^{n+1} vertices, this perfect matching gives $\nu\, Q_{n+1} = 2^n$.

```
lemma matchingNum_hypercube_succ (n : ℕ) : ν Q[n + 1] = 2 ^ n := by
  rw [(hypercube.isPerfectMatching_matching n).matchingNum_eq]
  have h : Nat.card (HypercubeVert (n + 1)) / 2 = 2 ^ n := by
    rw [Nat.card_eq_fintype_card, card_hypercubeVert, pow_succ]; grind
  exact_mod_cast h
```

Matching Number of the Mycielskian

If `M` is a matching in `G` then we can construct a matching in the Mycielskian `mμ(G)` by taking the edges $x^l - y^r$ and $x^r - y^l$ for each edge $x - y$ in `M`. This makes sense for any subgraph `M` of `G` (although it won't be a matching in general).

```
section mycielskian
variable {V : Type*} {G : SimpleGraph V}
```

Given a subgraph `M` of `G`, the `mycielskianDouble` of `M` is the subgraph of `mμ(G)` with edges given by $x^l - y^r$ and $x^r - y^l$ for each edge $x - y$ in `M`.

```
@[simps]
def Subgraph.mycielskianDouble (M : G.Subgraph) : Subgraph mμ(G) where
  verts := (·ˡ) '' M.verts ∪ (·ʳ) '' M.verts
  Adj
    | xˡ, yʳ => M.Adj x y
    | xʳ, yˡ => M.Adj x y
    | _, _ => False
  adj_sub := by
    rintro (_ | _ | _) (_ | _ | _) h <;> first | exact M.adj_sub h | exact h.elim
  edge_vert := by
    rintro (_ | _ | _) (_ | _ | _) h <;> grind [M.edge_vert]
  symm := ⟨by
    rintro (_ | _ | _) (_ | _ | _) h <;> grind [Adj.symm]⟩
```

The `mycielskianDouble` of a matching is a matching: each vertex in the left copy of the vertices of M is matched with the right copy of its match in M.

```
lemma Subgraph.IsMatching.mycielskianDouble {M : G.Subgraph}
    (h : M.IsMatching) : M.mycielskianDouble.IsMatching := by
  rintro a (⟨x, hx, rfl⟩ | ⟨x, hx, rfl⟩) <;> obtain ⟨y, hxy, huniq⟩ := h hx
  · -- a = xˡ: its unique neighbour is yʳ, where y is the M-partner of x.
    refine ⟨yʳ, hxy, ?_⟩
    rintro (_ | _ | w) hw
    exacts [hw.elim, hw.elim, huniq w hw ▸ rfl]
  · -- a = xʳ: its unique neighbour is yˡ.
    refine ⟨yˡ, hxy, ?_⟩
    rintro (_ | w | _) hw
    exacts [hw.elim, huniq w hw ▸ rfl, hw.elim]
```

The embedding of the darts of a subgraph into the edge set of its Mycielskian double.

```
def Subgraph.mycielskianDoubleDartEmbedding (M : G.Subgraph) :
    M.coe.Dart ↪ (M.mycielskianDouble).edgeSet where
  toFun d := ⟨s(d.1.1.valˡ, d.1.2.valʳ), d.adj⟩
  inj' d₁ d₂ h := by ext <;> grind [Subtype.mk.injEq]
```

Each dart of M gives a distinct edge of `M.mycielskianDouble`, and a finite graph has twice as many darts as edges, so the double has at least twice as many edges as M. Counting darts is a `Fintype` computation, so we do it in $\mathbb{N}$ with `Nat.card` and only cast up to $\mathbb{N}\infty$ at the end.

```
lemma Subgraph.two_mul_encard_edgeSet_le_encard_edgeSet_mycielskianDouble [Finite V]
    (M : G.Subgraph) :
    2 * M.edgeSet.encard ≤ (M.mycielskianDouble).edgeSet.encard := by
```

```
  classical
  have := Fintype.ofFinite V
  -- Each edge of M gives two darts, so M.coe has 2 * |E(M)| darts.
  have hdart : Nat.card M.coe.Dart = 2 * M.edgeSet.ncard := by
    rw [Nat.card_eq_fintype_card, dart_card_eq_twice_card_edges,
      ← M.ncard_edgeSet_coe, Set.ncard_eq_toFinset_card']
    rfl
  have hle : Nat.card M.coe.Dart ≤ Nat.card (M.mycielskianDouble).edgeSet :=
    Nat.card_le_card_of_injective _ M.mycielskianDoubleDartEmbedding.injective
  rw [hdart, Nat.card_coe_set_eq] at hle
  -- Move the ℕ inequality up to ℕ∞.
  rw [← Set.coe_ncard_eq_encard, ← Set.coe_ncard_eq_encard]
  exact_mod_cast hle
```

The matching number of the Mycielskian of G is at least twice the matching number of G.

```
lemma two_mul_matchingNum_le_matchingNum_mycielskian [Finite V] :
    2 * ν G ≤ ν mμ(G) := by
  obtain ⟨M, hM, hc⟩ := exists_isMatching_matchingNum (G := G)
  calc
    2 * ν G = 2 * M.edgeSet.encard := by rw [hc]
    _ ≤ (M.mycielskianDouble).edgeSet.encard :=
        two_mul_encard_edgeSet_le_encard_edgeSet_mycielskianDouble M
    _ ≤ ν mμ(G) := hM.mycielskianDouble.le_matchingNum
```

When G has a perfect matching the lower bound $2 * \nu\ G$ is exact: the Mycielskian has $2 * |V| + 1$ vertices, so by `matchingNum_le_card_div_two` no matching of $\mu(G)$ can have more than $|V| = 2 * \nu\ G$ edges.

(For graphs without a perfect matching the lower bound $2 * \nu\ G$ can be far from exact. If G is a disjoint union of two triangles it has matching number 2, but the cross edges $a^l - b^r, b^l - c^r, c^l - a^r$ of each triangle form a matching of size 6 in the Mycielskian.)

```
lemma Subgraph.IsPerfectMatching.matchingNum_mycielskian_eq [Finite V]
    {M : G.Subgraph} (h : M.IsPerfectMatching) : ν mμ(G) = 2 * ν G := by
  classical
  have := Fintype.ofFinite V
  -- A graph with a perfect matching has an even number of vertices.
  obtain ⟨k, hk⟩ : Even (Nat.card V) := by
    simpa [Nat.card_eq_fintype_card] using h.even_card
  -- The Mycielskian has 2 * |V| + 1 vertices.
  have hcard : Nat.card (Option (V ⊕ V)) = 2 * Nat.card V + 1 := by
    grind [card_mycielskianVert_one, ← Nat.card_eq_fintype_card]
  -- 2 * ν G = |V| in ℕ∞, since ν G = |V| / 2 and |V| is even.
  have h2ν : 2 * ν G = (Nat.card V : ℕ∞) := by
    rw [h.matchingNum_eq]; norm_cast; grind
  -- The upper bound ν mμ(G) ≤ (2|V|+1)/2 = |V| meets the lower bound 2 * ν G.
  refine ((matchingNum_le_card_div_two (G := mμ(G))).trans ?_).antisymm
    two_mul_matchingNum_le_matchingNum_mycielskian
  rw [h2ν, Nat.cast_le]
  grind

end mycielskian
```

9.6 Partial Colourings*

Informally, a *partial γ-colouring* of `G : SimpleGraph V` should be a map `C : V → γ` that is valid on a given subset of the vertices `s : Set V`, where valid means something like `∀ a b ∈ s, G.Adj a b → C a ≠ C b`.

There is currently no definition of partial colourings in Mathlib. However they arise in many situations, such as the proof of Brooks' theorem. (Indeed we will use them for exactly this; see Brooks' Theorem (p. 304).)

Below we discuss a possible implementation making use of the existing `SimpleGraph.Coloring` API where possible.

We want a partial colouring `C` to be a total function on `V` and we also want to reuse the `SimpleGraph.Coloring` API. The natural way to implement this is as a colouring of the subgraph of `G` induced by `s` but with all vertices of `V` retained: i.e. a colouring of `(G.induce s).spanningCoe`, where `spanningCoe` is `SimpleGraph.spanningCoe` from SpanningCoe (p. 108). Alternatively we could ask for `C` to be a colouring of `((⊤ : G.Subgraph).induce s).spanningCoe`, using `Subgraph.spanningCoe` instead.

These are the same graph (this is Mathlib's `spanningCoe_induce_top`), but they are not *definitionally* equal (Definitional and Propositional Equalities (p. 16)), and the latter is much easier to work with. Its adjacency relation is definitionally `a ∈ s ∧ b ∈ s ∧ G.Adj a b`. While in `(G.induce s).spanningCoe`, adjacency unfolds to an existential over vertices of the subtype `↑s`, coming from `SimpleGraph.map`: `a ≠ b ∧ ∃ u v : ↑s, (G.induce s).Adj u v ∧ ↑u = a ∧ ↑v = b`.

```
section partcol
variable {V : Type*} (G : SimpleGraph V)

example {s : Set V} :
    ((⊤ : G.Subgraph).induce s).spanningCoe = (G.induce s).spanningCoe :=
  spanningCoe_induce_top _

example {a b : V} {s : Set V} :
    ((⊤ : G.Subgraph).induce s).spanningCoe.Adj a b ↔ a ∈ s ∧ b ∈ s ∧ G.Adj a b :=
  Iff.rfl

example {a b : V} {s : Set V} :
    (G.induce s).spanningCoe.Adj a b ↔
      a ≠ b ∧ ∃ u v : ↑s, (G.induce s).Adj u v ∧ ↑u = a ∧ ↑v = b := Iff.rfl
```

`G.ColoringOn γ s` assigns a colour in `γ` to every vertex of `G`, but only requires adjacent vertices that both lie in `s` to get different colours: vertices outside `s` are coloured arbitrarily.

```
abbrev ColoringOn (γ : Type*) (s : Set V) :=
  ((⊤ : G.Subgraph).induce s).spanningCoe.Coloring γ
```

`G.ColorableOn n s` holds if there is a `G.ColoringOn (Fin n) s`: a colouring of every vertex of `G` with `n` colours in which adjacent vertices that both lie in `s` get different colours.

```
abbrev ColorableOn (n : ℕ) (s : Set V) := Nonempty (G.ColoringOn (Fin n) s)
```

Restricting and Copying Partial Colourings

A partial colouring of `s` can be *restricted* to any subset `t ⊆ s`: we use the same function and note that the validity condition holds for the subset. We call this `ColoringOn.restrict`.

```
variable {s t : Set V} {γ : Type*} {G}
```

Restrict `C : G.ColoringOn γ s` to any `t ⊆ s`.

```
@[simps]
protected def ColoringOn.restrict (C : G.ColoringOn γ s) (h : t ⊆ s) :
    G.ColoringOn γ t where
  toFun := C
  map_rel' := fun ⟨ha, hb, hab⟩ ↦ C.valid ⟨h ha, h hb, hab⟩
```

Like `Walk.copy`, `ColoringOn.restrict` moves a term to a type with a different index, here the set `t` in place of `s`. But `Walk.copy` is the cast `hu ▸ hv ▸ p`, which is stuck until we substitute the endpoints. Here, instead, we reuse the underlying function `C` unchanged and discharge the validity condition directly: an edge of the subgraph induced by `t` is an edge of the subgraph induced by `s`, so `C.valid` applies. Because the underlying function is untouched, a restriction agrees with the original at every vertex by `rfl`. The `@[simps]` attribute records this as the `simp` lemma `restrict_apply`.

```
#check ColoringOn.restrict_apply -- (C.restrict h) a = C a
```

In practice the most common use of `restrict` is the trivial one where the subset is equal to the original set. `ColoringOn` is a type that encodes data, and just as with `Walk.copy` we frequently need to change the set on which a colouring is valid to one that is equal but not definitionally equal to it, for example `↑(insert a s)` versus `insert a ↑s`. This is the *range copy pattern* from Elsewhere in Mathlib (p. 209), applied, as for `Walk.copy`, to data in the type rather than in a field. Here it is just restriction along `h.ge : t ⊆ s`, and we call it `ColoringOn.copy`. (Mathlib's `Subgraph.copy` is an instance of the pattern itself: it replaces the `verts` and `Adj` fields of a subgraph by propositionally equal ones.)

```
#check Subgraph.copy
```

Construct a `G.ColoringOn γ t` from `C : G.ColoringOn γ s` and a proof that `s = t`.

```
@[simps!]
protected def ColoringOn.copy (C : G.ColoringOn γ s) (h : s = t) :
    G.ColoringOn γ t :=
  C.restrict h.ge
```

The body of `ColoringOn.copy` is an application of `ColoringOn.restrict` rather than a structure, so we need `@[simps!]` to unfold it. The generated `simp` lemma `copy_apply` also serves as a named lemma for the explicit `rw` chains that step through a copy to expose the underlying `union`/`insert`.

```
#check ColoringOn.copy_apply -- (C.copy h) a = C a
```

By far the most common source of such mismatches for us will be due to the coercion from `Finset V` to `Set V`. Following the Mathlib convention that definitions are made for the more general type, our basic constructors are stated for `Set V`. In practice, however, we almost always colour finite sets. When we pass `ColoringOn` a `Finset V` it is automatically coerced to a `Set V` (since `Finset`

has a SetLike instance). Depending on how this coercion is applied we may end up with a type mismatch that we will need to fix with ColoringOn.copy.

For example, we might end up with a colouring of ↑s ∪ ↑t when the goal asks for a colouring of ↑(s ∪ t). For s t : Finset V, the first is the union of the two coerced sets in Set V, and the second is the coercion of their Finset union. These two sets are obviously equal but the proof is not rfl (it is the simp lemma Finset.coe_union).

```
/- error: Type mismatch -/
example (s t : Finset V) [DecidableEq V] (C : G.ColoringOn γ (↑s ∪ ↑t)) :
    G.ColoringOn γ ↑(s ∪ t) := C

example (s t : Finset V) [DecidableEq V] (C : G.ColoringOn γ (↑s ∪ ↑t)) :
    G.ColoringOn γ ↑(s ∪ t) := C.copy (by simp)
```

For this reason we will need to use ColoringOn.copy frequently in proofs. Fortunately Finset.coe_insert, Finset.coe_union, Finset.coe_empty etc. are all simp lemmas, so the proof obligation that ColoringOn.copy requires will almost always be discharged by simp.

```
#check Finset.coe_empty  -- ((∅ : Finset α) : Set α) = ∅
#check Finset.coe_insert -- ↑(insert a s) = insert a ↑s
#check Finset.coe_union  -- ↑(s₁ ∪ s₂) = ↑s₁ ∪ ↑s₂
```

Building Partial Colourings

One reason to care about partial colourings is that they can be used to build colourings of the whole graph, so we want to be able to turn a G.ColoringOn γ Set.univ into a G.Coloring γ.

A G.ColoringOn γ Set.univ is a G.Coloring γ.

```
@[simps]
def ColoringOn.toColoring (C : G.ColoringOn γ Set.univ) : G.Coloring γ :=
    ⟨C, fun h ↦ C.valid (by simpa using h)⟩
```

A colouring gives rise to a partial colouring on any set.

```
@[simps]
def Coloring.toColoringOn (C : G.Coloring γ) (s : Set V) : G.ColoringOn γ s :=
    ⟨C, fun h ↦ C.valid (by simpa using h.2.2)⟩
```

A graph is n-colourable if and only if G.ColorableOn n Set.univ.

```
lemma colorableOn_univ_iff_colorable (n : ℕ) :
    G.ColorableOn n (Set.univ : Set V) ↔ G.Colorable n := by
  constructor <;> intro ⟨c⟩
  · exact ⟨c.toColoring⟩
  · exact ⟨c.toColoringOn _⟩
```

There are a few basic partial colourings that we will need to construct: the partial colouring of the empty set, the partial colouring of a singleton, and the partial colouring of a pair of non-adjacent vertices.

The partial colouring of the empty set by a colour c.

```
@[simps!]
protected def ColoringOn.empty (c : γ) : G.ColoringOn γ ∅ :=
```

```
  Coloring.mk (fun _ => c) fun h => h.1.elim
```

The partial colouring of the singleton set $\{a\}$ by a colour c.

```
@[simps]
protected def ColoringOn.singleton (a : V) (c : γ) :
    G.ColoringOn γ {a} where
  toFun := fun _ ↦ c
  map_rel' := by simp
```

The partial colouring of a pair of non-adjacent vertices by a colour c.

```
@[simps]
def ColoringOn.ofNotAdj {a b : V} (h : ¬G.Adj a b) (c : γ) :
    G.ColoringOn γ {a, b} where
  toFun := fun _ ↦ c
  map_rel' := by
    rintro x y ⟨hx, hy, hxy⟩ -
    grind [adj_comm]
```

We can combine two partial colourings by extending the first using the second, provided they are compatible in the sense that there is no edge between a vertex coloured by C_1 and a vertex coloured only by C_2 using the same colour. This gives a partial colouring on the union of the two sets.

We can combine colourings C_1 of s and C_2 of t if they are compatible, i.e. `∀ v w, v ∈ s → w ∈ t \ s → G.Adj v w → C₁ v ≠ C₂ w`, to get a colouring of $s \cup t$. It agrees with C_1 on s and with C_2 on s^c.

```
@[simps]
protected def ColoringOn.union [DecidablePred (· ∈ s)] (C₁ : G.ColoringOn γ s)
    (C₂ : G.ColoringOn γ t)
    (h : ∀ {v w}, v ∈ s → w ∈ t \ s → G.Adj v w → C₁ v ≠ C₂ w) :
    G.ColoringOn γ (s ∪ t) where
  toFun := fun v ↦ ite (v ∈ s) (C₁ v) (C₂ v)
  map_rel' := by
    rintro v w ⟨hv', hw', hadj⟩ hf
    split_ifs at hf with hv hw hw
    · exact C₁.valid ⟨hv, hw, hadj⟩ hf
    · exact h hv ⟨hw'.resolve_left hw, hw⟩ hadj hf
    · exact h hw ⟨hv'.resolve_left hv, hv⟩ hadj.symm hf.symm
    · exact C₂.valid ⟨hv'.resolve_left hv, hw'.resolve_left hw, hadj⟩ hf

#check ColoringOn.union_apply -- (C₁.union C₂ h) v = if v ∈ s then C₁ v else C₂ v
```

Next we extend a colouring C of s to `insert x s` using a colour c that is not used by C to colour a neighbour of x in s. We could build this as `(ColoringOn.singleton x c).union C _`, followed by a `ColoringOn.copy` from `{x} ∪ s` to `insert x s`, but then `@[simps!]` would only unfold it as far as that `ColoringOn.union`. Instead we define it directly, with the same four cases in the validity proof as for `ColoringOn.union`, and `@[simps]` then generates exactly the lemma we want.

Extend `C : G.ColoringOn γ s` to `insert x s`, giving x a colour c that C does not use on any neighbour of x in s.

```
@[simps]
protected def ColoringOn.insert [DecidableEq V] (x : V) {c : γ}
```

```
    (C : G.ColoringOn γ s) (h : ∀ {v}, v ∈ s → G.Adj x v → C v ≠ c) :
    G.ColoringOn γ (insert x s) where
  toFun v := if v = x then c else C v
  map_rel' := by
    rintro v w ⟨hv, hw, hadj⟩ hf
    split_ifs at hf with hvx hwx hwx
    · exact hadj.ne (hvx.trans hwx.symm)
    · exact h (hw.resolve_left hwx) (hvx ▸ hadj) hf.symm
    · exact h (hv.resolve_left hvx) (hwx ▸ hadj.symm) hf
    · exact C.valid ⟨hv.resolve_left hvx, hw.resolve_left hwx, hadj⟩ hf
```

So `C.insert x h` colours x with c and agrees with C elsewhere.

```
#check ColoringOn.insert_apply -- (C.insert x h) v = if v = x then c else C v
```

Finally we can relabel the colours of a partial colouring along any injective map (in particular along any permutation of the colours): validity is preserved, since an injection cannot merge two distinct colours.

Relabel the colours of a partial colouring along an embedding.

```
@[simps!]
protected def ColoringOn.map {β : Type*} (f : γ ↪ β) (C : G.ColoringOn γ s) :
    G.ColoringOn β s :=
  (Embedding.completeGraph f).toHom.comp C

#check ColoringOn.map_apply --  (ColoringOn.map f C) x = f (C x)

end partcol
```

9.7 Exercises

1. If `H` and `K` are complementary subgraphs of G (i.e. `IsCompl H K`, which means `H ⊓ K = ⊥` and `H ⊔ K = ⊤`) then no edge of G can have only one endpoint in `H`.

```
variable {V : Type*} {G : SimpleGraph V}
lemma Subgraph.mem_verts_iff_of_isCompl' {H K : G.Subgraph} (h : IsCompl H K)
    {v w : V} (hadj : G.Adj v w) : v ∈ H.verts ↔ w ∈ H.verts := by
  sorry
```

2. Prove that the *Heyting complement* of a subgraph H (which is how H^c is defined) is `(⊤ : G.Subgraph).induce H.vertsᶜ`. Hint: `le_compl_iff_disjoint_right`.

```
example (H : G.Subgraph) : Hᶜ = (⊤ : G.Subgraph).induce H.vertsᶜ := by
  sorry
```

3. Strengthen `Walk.exists_isPath_or_odd_isCycle` so that the path or odd cycle it produces is a subgraph of the original walk. Hint: follow the proof of `Walk.exists_isPath_or_odd_isCycle`. Applying `Walk.toSubgraph` to `Walk.take_spec` and rewriting with `Walk.toSubgraph_append` shows that `takeUntil` and `dropUntil` give subgraphs of the walk they come from.

```
lemma Walk.exists_isPath_or_odd_isCycle_toSubgraph_le' {a b : V} (p : G.Walk a b) :
    (∃ q : G.Walk a b, q.IsPath ∧ (Even q.length ↔ Even p.length) ∧
        q.toSubgraph ≤ p.toSubgraph) ∨
      ∃ (v : V) (c : G.Walk v v), c.IsCycle ∧ Odd c.length ∧
        c.toSubgraph ≤ p.toSubgraph := by
  sorry
```

4. Deduce the corresponding strengthening of `Walk.exists_odd_isCycle`: a closed walk of odd length contains an odd cycle that is a subgraph of the walk. Hint: follow the proof of `Walk.exists_odd_isCycle`.

```
theorem Walk.exists_odd_isCycle_toSubgraph_le' {u : V} (w : G.Walk u u)
    (hw : Odd w.length) :
    ∃ (v : V) (c : G.Walk v v), c.IsCycle ∧ Odd c.length ∧
      c.toSubgraph ≤ w.toSubgraph := by
  sorry
```

5. Show that a trail that is a subgraph of a walk is no longer than the walk. Hint: every edge of the trail is an edge of the walk (`Walk.mem_edges_toSubgraph` and `Subgraph.edgeSet_mono`), and the edges of a trail are distinct (`Walk.IsTrail.edges_nodup`), so `List.Nodup.subperm` and `List.Subperm.length_le` apply.

```
theorem Walk.IsTrail.length_le_of_toSubgraph_le' {a b c d : V} {p : G.Walk a b}
    {w : G.Walk c d} (hp : p.IsTrail) (hle : p.toSubgraph ≤ w.toSubgraph) :
    p.length ≤ w.length := by
  sorry
```

6. Deduce that a closed walk of odd length `k` contains an odd cycle of length at most `k`.

```
theorem Walk.exists_odd_isCycle_length_le' {u : V} (w : G.Walk u u)
    (hw : Odd w.length) :
    ∃ (v : V) (c : G.Walk v v), c.IsCycle ∧ Odd c.length ∧ c.length ≤ w.length := by
  sorry
```

7. Show that any edge of an induced subgraph of a subgraph is an edge of `G`.

```
example (s : Set V) (v w : V) (H : G.Subgraph) (h : (H.induce s).Adj v w) :
    G.Adj v w :=
  sorry
```

8. Show that the induced subgraph on `Set.univ` is the top subgraph `⊤`.

```
example : (⊤ : G.Subgraph).induce Set.univ = ⊤ := by
  sorry
```

9. Show that an induced subgraph of `(⊤ : G.Subgraph)` is induced (in the sense of `Subgraph.IsInduced`).

```
example (s : Set V) : ((⊤ : G.Subgraph).induce s).IsInduced := by
  sorry
```

10. Show that the spanning coercion of a single-vertex subgraph is the empty graph `⊥ : SimpleGraph V`, which has *every* vertex of `V`. So `spanningCoe` forgets the vertex set of a subgraph.

```
example (v : V) : (G.singletonSubgraph v).spanningCoe = ⊥ := by
  sorry
```

11. Show that if every induced subgraph of `G` on a finite set of vertices is `n`-colourable then `G` is `n`-colourable. Hint: use the De Bruijn–Erdős theorem `colorable_of_forall_finite_subgraph_colorable`. For a finite subgraph `H`, `H.coe ≤ G.induce H.verts` (both graphs on `↑H.verts`), so `Colorable.mono_left` applies.

```
lemma colorable_of_forall_finite_induce_colorable' {n : ℕ}
    (h : ∀ s : Set V, s.Finite → (G.induce s).Colorable n) : G.Colorable n := by
  sorry
```

12. The finiteness of `F` in `nonempty_hom_of_forall_finite_subgraph_hom` cannot be dropped. Recall the sigma graph `sigmaCompleteNat`, the disjoint union `Σg (fun i => K[i])` of all the complete graphs `K[i]` (see `SimpleGraph.sigma`). Show that every finite subgraph of `⊤ : SimpleGraph ℕ` has a homomorphism to `sigmaCompleteNat`, but `⊤` itself does not. Hint: for the first part, number the vertices of `H` using `Finite.equivFin` and send them into `K[m]`, where `m` is the number of vertices. For the second, show that adjacent vertices of `sigmaCompleteNat` lie in the same `K[i]` and have distinct labels, as in the example following its definition. Every vertex of `⊤` is adjacent to `0` or equal to it, so a homomorphism `f` sends every vertex into the `K[i]` containing `f 0`, and reading off the labels gives an injection `ℕ → Fin i`.

```
example (H : (⊤ : SimpleGraph ℕ).Subgraph) (hH : H.verts.Finite) :
    Nonempty (H.coe →g sigmaCompleteNat) := by
  sorry

example : IsEmpty ((⊤ : SimpleGraph ℕ) →g sigmaCompleteNat) := by
  sorry
```

13. Show that the matching number is monotone: if `G ≤ H` then `ν G ≤ ν H`. Hint: turn a matching of `G` into a matching of `H` with the same edges by building the `H.Subgraph` directly, then use the `matchingNum` API (`matchingNum_le` and `Subgraph.IsMatching.le_matchingNum`). (Mathlib's `Subgraph.IsMatching.map_ofLE` says that `M.map (Hom.ofLE h)` is a matching of `H`, but its edges are then harder to count.)

```
lemma matchingNum_mono' {G H : SimpleGraph V} (h : G ≤ H) :
    ν G ≤ ν H := by
  sorry
```

14. Show that `ν K[3] = 1` but `τ K[3] = 2`, so weak duality `ν G ≤ τ G` can be strict. Hint: for the first, `matchingNum_le_card_div_two` gives the upper bound and a single edge is a matching (`Subgraph.IsMatching.subgraphOfAdj`). For the second, use Mathlib's `vertexCoverNum_top`.

```
example : ν K[3] = 1 := by
  sorry

example : τ K[3] = 2 := by
  sorry
```

15. Show that the infinite path has matching number `τ`. Hint: build the subgraph of `pathNat` that pairs `2 * k` with `2 * k + 1`. (An easy way to define this adjacency relation is as `pathNat.Adj m n ∧ m / 2 = n / 2`.) Show that it is a matching, and show that its edge set is infinite (`Set.infinite_of_injective_forall_mem`).

```
lemma matchingNum_pathNat' : ν pathNat = τ := by
  sorry
```

16. Show that in a finite graph `α G + ν G ≤ |V|`. Hint: let `I` be a maximum independent set (`exists_isNIndepSet_indepNum`) and `M` a maximum matching (`exists_isMatching_matchingNum`), so `#I = α G` and `|E(M)| = ν G`. Count the vertices of `I` inside and outside `M.verts` separately:

    - *Inside:* no edge of `M` has both ends in `I`, since `I` is independent. So sending each `v ∈ I ∩ M.verts` to the edge of `M` containing it is injective, and at most `|E(M)|` vertices of `I` lie in `M.verts` (`Subgraph.IsMatching.ncard_inter_le_of_isIndepSet`).
    - *Outside:* `M.verts` has `2 * |E(M)|` vertices (`Subgraph.IsMatching.ncard_verts`), so at most `|V| - 2 * |E(M)|` vertices lie outside it.

Adding these gives `α G ≤ |V| - ν G`. To do the arithmetic in $\mathbb{N}$, write `ν G` as the cast of `M.edgeSet.ncard` using `Set.coe_ncard_eq_encard`.

```
lemma indepNum_add_matchingNum_le' [Finite V] :
    (α G : ℕ∞) + ν G ≤ Nat.card V := by
  sorry
```

17. The supremum of an increasing sequence of matchings is a matching. Hint: a vertex of `⨆ n, M n` lies in some `M n` (`Subgraph.verts_iSup`) and an edge of `⨆ n, M n` lies in some `M m` (`Subgraph.iSup_adj`); compare the two at the later stage `max n m`.

```
lemma Subgraph.isMatching_iSup_of_monotone' {M : ℕ → G.Subgraph}
    (hmono : Monotone M) (hM : ∀ n, (M n).IsMatching) : (⨆ n, M n).IsMatching := by
  sorry
```

18. If `ν G = τ` then every finite set of vertices `S` misses some edge of `G`. Hint: otherwise `S` is a vertex cover, so weak duality (`matchingNum_le_vertexCoverNum`) and `IsVertexCover.vertexCoverNum_le` force `S.encard = τ`.

```
lemma exists_adj_notMem_of_matchingNum_eq_top' (h : ν G = ⊤) {S : Set V}
    (hS : S.Finite) : ∃ a b, G.Adj a b ∧ a ∉ S ∧ b ∉ S := by
  sorry
```

19. A matching M with finitely many vertices in a graph with $\nu\ G = \top$ can be extended by one edge. Hint: take an edge avoiding `M.verts` from the previous exercise and add it with `Subgraph.IsMatching.sup`, whose disjointness hypothesis is about supports (`Subgraph.IsMatching.support_eq_verts`), then count edges with `Set.encard_insert_of_notMem`.

```
lemma Subgraph.IsMatching.exists_le_encard_eq_add_one' (h : ν G = ⊤)
    {M : G.Subgraph} (hM : M.IsMatching) (hfin : M.verts.Finite) :
    ∃ N : G.Subgraph, M ≤ N ∧ N.IsMatching ∧ N.verts.Finite ∧
      N.edgeSet.encard = M.edgeSet.encard + 1 := by
  sorry
```

20. If $\nu\ G = \top$ then G has an infinite matching. Hint: the stub chooses a step function `next`, which extends each finite matching by one edge (the previous exercise), and iterates it from $\bot$; each $M\ n$ is then a finite matching by construction. Show by induction that it has exactly n edges. Each step extends the previous matching, so M is monotone (`monotone_nat_of_le_succ`) and $\bigsqcup n,\ M\ n$ is a matching by Exercise 17, with at least n edges for every n.

```
lemma exists_isMatching_encard_eq_top' (h : ν G = ⊤) :
    ∃ M : G.Subgraph, M.IsMatching ∧ M.edgeSet.encard = ⊤ := by
  -- Choose, for every finite matching, an extension by one edge. Packaging the
  -- finite matchings as a subtype FM makes this a total function next.
  let FM := {M : G.Subgraph // M.IsMatching ∧ M.verts.Finite}
  have : ∀ M : FM, ∃ N : FM,
      M.1 ≤ N.1 ∧ N.1.edgeSet.encard = M.1.edgeSet.encard + 1 := by
    rintro ⟨M, hM, hfin⟩
    obtain ⟨N, hle, hN, hNfin, hcard⟩ := hM.exists_le_encard_eq_add_one' h hfin
    exact ⟨⟨N, hN, hNfin⟩, hle, hcard⟩
  choose next hnext hcard using this
  -- Iterate from the empty matching.
  let M : ℕ → FM := fun n => next^[n] ⟨⊥, Subgraph.isMatching_bot, by simp⟩
  have hstep : ∀ n, M (n + 1) = next (M n) := fun n =>
    Function.iterate_succ_apply' _ _ _
  -- Each M n is a finite matching by construction; it has exactly n edges.
  sorry
```

21. Conclude that the matching number is attained in every graph, finite or not: this is the analogue of Mathlib's `vertexCoverNum_exists`. Hint: if $\nu\ G = \top$ use the previous exercise; otherwise a finite supremum in $\mathbb{N}\infty$ is attained, so conclude as in the proof of `exists_isMatching_matchingNum`, with `ENat.exists_eq_iSup_of_lt_top` in place of `exists_eq_ciSup_of_finite`.

```
lemma matchingNum_exists' (G : SimpleGraph V) :
    ∃ M : G.Subgraph, M.IsMatching ∧ M.edgeSet.encard = ν G := by
  sorry
```

22. Show that `α Q[n + 1] = 2 ^ n`. Hint: the upper bound follows from Exercise 16 and `matchingNum_hypercube_succ`. For the lower bound, `Q[n + 1]` is bipartite (`IsBipartite.hypercube`), so `Colorable.natCard_le_mul_indepNum` applies.

```
example (n : ℕ) : α Q[n + 1] = 2 ^ n := by
  sorry
```

23. Show that, provided there is at least one colour, `s` is `n`-colourable in the sense of `ColorableOn` iff the induced subgraph `G.induce s` is `n`-colourable. (So partial colourings really do capture colourings of induced subgraphs.) Why is the hypothesis `0 < n` needed?

```
lemma colorableOn_iff_colorable_induce' {n : ℕ} (hn : 0 < n) (s : Set V) :
    G.ColorableOn n s ↔ (G.induce s).Colorable n := by
  sorry
```

10 Connectivity

In this chapter we study graph connectivity. Building on the reachability relation `Reachable`, we start with Mathlib's notions `Preconnected` and `Connected`, connected components (`ConnectedComponent`) and bridges (`IsBridge`). We show that the Mycielskian `mμ(G)` is connected iff `G` has no isolated vertices (`connected_mycielskian_iff`), and that in a graph of maximum degree at most 2 every cycle is its own connected component (`Walk.IsCycle.toSubgraph_eq_of_degree_le_two`). We then look at trees, and show that the infinite path `pathNat` is one (`isTree_pathNat`). We prove *Euler's theorem*: a finite graph has an Eulerian trail iff all its edges lie in one component and it has either no vertex of odd degree or exactly two (`exists_isEulerian_iff`). Next we introduce Mathlib's edge connectivity (`IsEdgeConnected`), and give our own definition of vertex connectivity (`IsVertexConnected`). We show that `k`-vertex-connectivity implies `k`-edge-connectivity (`IsVertexConnected.isEdgeConnected`). We also define the edge and vertex connectivity of a graph as numbers in $\mathbb{N}\infty$ and prove Whitney's inequalities `κ(G) ≤ λ(G)` (`vertexConnectivity_le_edgeConnectivity`) and `λ(G) ≤ δ(G)` (`edgeConnectivity_le_degree`).

The final two sections are more advanced. We prove *Brooks' theorem* (`brooks`): a locally finite graph of maximum degree at most `Δ` is `Δ`-colourable unless it contains a `(Δ + 1)`-clique or (when `Δ = 2`) one of its components is an odd cycle. Then we define the *blocks* of a graph as the classes of an equivalence relation on edges, building our own quotient type just as Mathlib builds `ConnectedComponent`, and prove that, for `n ≥ 1`, a finite graph is `n`-colourable if each of its blocks is (`colorable_of_forall_block`).

10.1 Connectivity

In Mathlib connectedness is defined in two steps: a graph is *preconnected* (`Preconnected`) if there is a walk between every pair of vertices, and a graph is *connected* (`Connected`) if it is preconnected and nonempty. (Recall that existence of a walk between a pair of vertices is `Reachable` which we saw in Reachability (p. 196).)

```
open Walk
variable {V : Type*} {G : SimpleGraph V}

#source Reachable     -- def Reachable (u v : V) : Prop := Nonempty (G.Walk u v)
#source Preconnected  -- def Preconnected : Prop := ∀ u v : V, G.Reachable u v
#source Connected     -- structure Connected : Prop where
                      --   protected preconnected : G.Preconnected
                      --   protected [nonempty : Nonempty V]
```

Connected Graphs

The two notions differ only for the graph on an empty vertex type. This graph is vacuously preconnected, but it is not connected. With this convention a graph is connected exactly when it has precisely one connected component; the graph with no vertices has none.

```
example (h : G.Connected) {u v : V} : G.Reachable u v := h.preconnected u v
example (h : G.Connected) : Nonempty V := h.nonempty

example : (⊥ : SimpleGraph Empty).Preconnected := fun u => u.elim
example : ¬(⊥ : SimpleGraph Empty).Connected := fun h => h.nonempty.some.elim
```

Notice that rather than having to write `h.preconnected u v` for `G.Reachable u v` (where `h : G.Connected`) there is a `CoeFun` instance that lets us write the more succinct `h u v`:

```
example (h : G.Connected) {u v : V} : G.Reachable u v := h u v
```

The hypercube is connected: we can use the Hamming path to walk between any two vertices, and we already gave a `Nonempty` instance for its vertices which Lean will find automatically.

```
lemma connected_hypercube (n : ℕ) : Q[n].Connected where
  preconnected := fun u v => ⟨u.hammingPath v⟩
```

For a finite graph with decidable adjacency and whose vertex type has decidable equality, connectivity is decidable.

```
variable [DecidableEq V] [Fintype V] [DecidableRel G.Adj] in
#synth Decidable G.Connected

example : C[5].Connected := by decide
```

We have already met one source of connected graphs: a Hamiltonian graph is connected, since a Hamiltonian cycle passes through every vertex (Hamiltonian Paths and Cycles (p. 202)).

```
#check IsHamiltonian.connected
```

A connected graph has a path between any two vertices.

```
#check Connected.exists_isPath
```

A graph is connected iff there is a vertex `v` from which every other vertex is reachable.

```
#check connected_iff_exists_forall_reachable
```

In the Mycielskian of a graph `G`, any vertex `u` that is not isolated in `G` has both its left and right copies in `mμ(G)` connected to `none`. Given the edge `u - v` in `G` we have the walks `u.r - none` and `u.l - v.r - none` in `mμ(G)`. Conversely if `u` is an isolated vertex in `G` then `u.l` is an isolated vertex in `mμ(G)`.

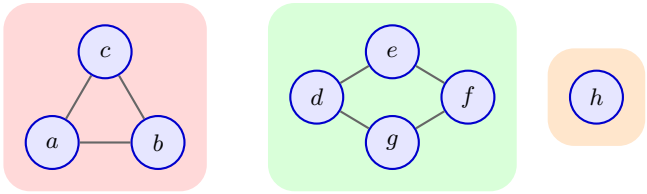
Hence `mμ(G)` is connected iff `G` has no isolated vertices.

```
theorem connected_mycielskian_iff : mμ(G).Connected ↔ ∀ v, ¬G.IsIsolated v := by
  simp_rw [IsIsolated, not_forall, not_not]
  constructor <;> intro h
  · intro v
    obtain ⟨p⟩ := h.preconnected v.l none
```

```
    cases p; grind [mycielskian_adj]
  · refine (connected_iff_exists_forall_reachable _).2 ⟨none, fun z => ?_⟩
    rcases z with (_ | ⟨v | v⟩)
    · rfl
    · obtain ⟨u, h⟩ := h v
      exact (by simp : mμ(G).Adj _ _).reachable.trans
            (by simpa using h.symm : mμ(G).Adj u✝ _).reachable
    · exact (by simp : mμ(G).Adj _ _).reachable
```

Connected Components

The *connected components* of `G` are the equivalence classes of `Reachable`.



A graph with three connected components

```
#source ConnectedComponent -- def ConnectedComponent := Quot G.Reachable
```

The component containing a given vertex.

```
#check G.connectedComponentMk  -- G.connectedComponentMk : V → G.ConnectedComponent
```

We introduce the notation `G⟦v⟧` for the connected component of `G` containing the vertex `v`.

```
@[inherit_doc connectedComponentMk]
scoped notation:max G "⟦" v "⟧" => SimpleGraph.connectedComponentMk G v
```

Two components are equal iff they are constructed from reachable vertices.

```
#check ConnectedComponent.eq
```

If we want to prove a fact about all components of a graph then we can use `ConnectedComponent.ind` which says that to prove `β c` holds, for all components `c`, it is sufficient to prove `β G⟦v⟧` for all vertices `v`.

```
#check ConnectedComponent.ind
```

If a pair of vertices are adjacent then they are in the same component.

```
#check ConnectedComponent.connectedComponentMk_eq_of_adj
```

For a finite graph with decidable adjacency, `G.ConnectedComponent` is a `Fintype`, so we can count the components. The empty graph `⊥` on `Fin 3` has three, while `C[5]` has one.

```
#eval Fintype.card (⊥ : SimpleGraph (Fin 3)).ConnectedComponent -- 3
#eval Fintype.card C[5].ConnectedComponent -- 1
```

A graph is connected iff it has exactly one component. Mathlib states this with `Nat.card`, so it applies to infinite graphs too.

```
#check card_connectedComponent_eq_one_iff
```

In informal graph theory we often think of a connected component as a graph, or subgraph, or subset of the vertex set of the original graph. But in Lean a connected component `C : G.ConnectedComponent` is none of these: it is an element of the *quotient type* `G.ConnectedComponent` of `V` by the equivalence relation `Reachable`. Such an element stands for an equivalence class of vertices, but it is not itself a set of vertices. However we can convert a connected component into all of these things.

The *support* of a component is the set of vertices in that component.

```
#check ConnectedComponent.supp --  (C : G.ConnectedComponent) : Set V
```

The `SimpleGraph` and `Subgraph` given by a component are `C.toSimpleGraph` and `C.toSubgraph`.

```
#source ConnectedComponent.toSimpleGraph -- .. SimpleGraph C := G.induce C.supp
#source ConnectedComponent.toSubgraph    -- .. G.Subgraph := Subgraph.induce ⊤ C.supp
```

Connectivity of a `G.Subgraph` is a separate notion, defined via the coercion of the subgraph to a `SimpleGraph` on its vertex set. This is one of the duplications discussed in Duplication (p. 254); we will use it in the proof of Brooks' theorem.

The subgraph of a connected component is connected.

```
#source Subgraph.Connected
#check ConnectedComponent.connected_toSubgraph
```

A set of vertices is maximal with respect to inducing a connected graph iff it is the support of a component.

```
#check ConnectedComponent.maximal_connected_induce_iff
```

If we want to give a colouring of a graph we can do this by giving colourings for each component. This is more generally true for homomorphisms.

```
#check homOfConnectedComponents
```

Recall our definition of a `SimpleGraph.sigma` graph: the disjoint union of an indexed family of graphs.

```
#check SimpleGraph.sigma
```

There is a natural isomorphism between a graph and the sigma graph of its connected components, sending a vertex `v` to the pair `⟨G⟦v⟧, v⟩`. Informally this is obvious, but in Lean we meet again the cast `h ▸ w` from the definition of `Σg` (`SimpleGraph.sigma`): the images of `v` and `w` are adjacent when there is a proof `h : G⟦v⟧ = G⟦w⟧` such that `v` is adjacent to `h ▸ w` in `G⟦v⟧.toSimpleGraph`. The cast is stuck until `h` is substituted away, and here we cannot do that: `subst` needs one side of the equation to be a variable, but `G⟦v⟧` and `G⟦w⟧` are both terms.

One solution is to prove the adjacency rule once, for indices that *are* variables. A component's graph `C.toSimpleGraph` is by definition the induced graph `G.induce C.supp`, so it is enough to describe adjacency in a sigma graph of induced graphs: `⟨i, u⟩` and `⟨j, v⟩` are adjacent iff `i = j` and `u` and `v` are adjacent in `G`. This statement involves no cast, and in its proof the pattern `rfl` substitutes `i = j` away, since `i` and `j` are variables.

```
lemma sigma_induce_adj {ι : Type*} {S : ι → Set V} {i j : ι} {u : S i} {v : S j} :
    (Σg fun i => G.induce (S i)).Adj ⟨i, u⟩ ⟨j, v⟩ ↔ i = j ∧ G.Adj u v := by
  constructor <;> rintro ⟨rfl, h⟩ <;> exact ⟨rfl, h⟩
```

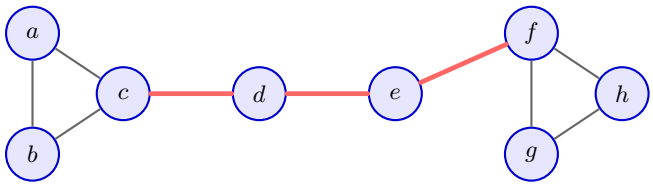
In the isomorphism the condition `G⟦v⟧ = G⟦w⟧` is then automatic, since adjacent vertices lie in the same component (`ConnectedComponent.connectedComponentMk_eq_of_adj`). The proof of `right_inv` uses the same substitution idea: a vertex of the sigma graph is a component `C` together with a vertex `v` and a proof of `v ∈ C`, which by definition says `G⟦v⟧ = C`, and since `C` is a variable the pattern `rfl` substitutes it.

The isomorphism between `G` and the sigma graph of its connected components.

```
def isoSigmaConnectedComponents :
    G ≃g Σg (fun C : G.ConnectedComponent => C.toSimpleGraph) where
  toFun v := ⟨G⟦v⟧, ⟨v, rfl⟩⟩
  invFun p := p.2
  left_inv v := rfl
  right_inv := by
    rintro ⟨_, ⟨v, rfl⟩⟩
    rfl
  map_rel_iff' := sigma_induce_adj.trans
    ⟨And.right, fun h => ⟨ConnectedComponent.connectedComponentMk_eq_of_adj h, h⟩⟩
```

Bridges

An edge e is a *bridge* if deleting it disconnects its two endpoints from each other.



A graph with its bridge edges highlighted

```
#source SimpleGraph.IsBridge
```

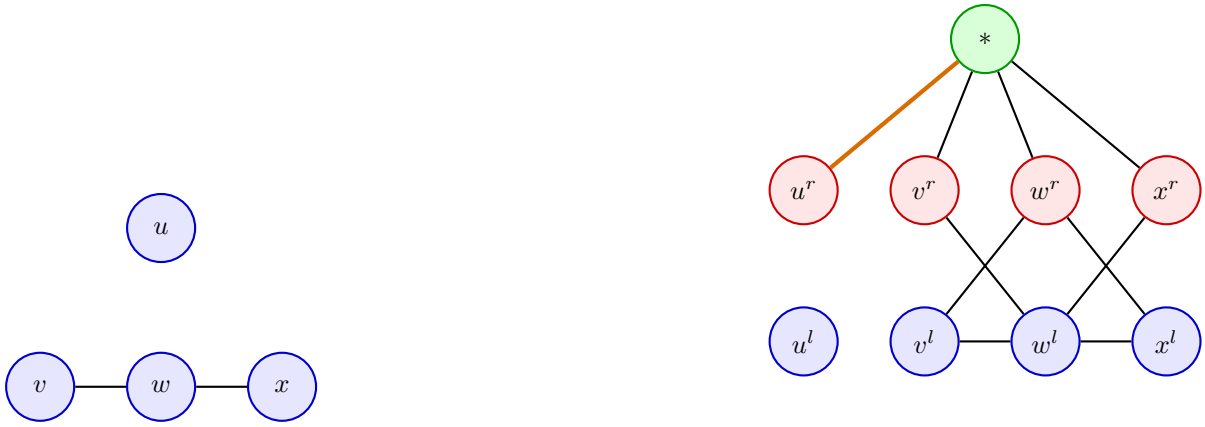
An edge of the graph `G` is a bridge iff it is not contained in any cycle in `G`.

```
#check isBridge_iff_forall_cycle_notMem
```

Recall from `isCycle_mycielskianClosed_of_adj` that any edge `u – v` in a graph gives rise to a 5-cycle in its Mycielskian.

```
#check isCycle_mycielskianClosed_of_adj
```

So every edge of `mμ(G)` that comes from an edge of `G` lies on a 5-cycle, and so does the edge `none – uᵣ` whenever `u` has a neighbour `v` in `G`. None of these edges is a bridge. On the other hand, if `u` is isolated in `G` then `none` is the only neighbour of `uᵣ`, so deleting the edge `none – uᵣ` cuts `uᵣ` off. Hence the bridges of `mμ(G)` are precisely the edges `none – uᵣ` where `u` is an isolated vertex of `G`.



Left: a graph `G` with an isolated vertex u. Right: its Mycielskian `mμ(G)`, with its unique bridge edge highlighted

```
lemma isBridge_mycielskian_iff {e} (he : e ∈ mμ(G).edgeSet) :
    mμ(G).IsBridge e ↔ ∃ x, G.IsIsolated x ∧ e = s(none, xʳ) := by
  refine ⟨fun h => ?_, ?_⟩
  · -- hncyc: the bridge e lies on no cycle of mμ(G)
    have hncyc : ∀ ⦃u⦄ ⦃p : mμ(G).Walk u u⦄, p.IsCycle → e ∉ p.edges :=
      fun _ _ hp => h.notMem_edges_of_isCycle hp
    induction e using Sym2.ind with
    | h x y =>
      have hxy : mμ(G).Adj x y := mμ(G).mem_edgeSet.mp he
      -- split on the types of the endpoints; simp only discharges the four pairs
      -- that are never edges: none – none, none – vˡ, uˡ – none, uʳ – vʳ
      rcases x with _ | (u | u) <;>
        rcases y with _ | (v | v) <;> simp only [mycielskian_adj] at hxy
      · -- none – vʳ: a neighbour w of v would put e on a 5-cycle, so v
        -- is isolated
        exact ⟨v, fun w hw => hncyc (isCycle_mycielskianClosed_of_adj hw.symm)
          (by simp [Sym2.eq_swap, Walk.mycielskian]), rfl⟩
      · -- uˡ – vˡ: impossible, it lies on the 5-cycle of the edge u – v
        exact (hncyc (isCycle_mycielskianClosed_of_adj hxy)
          (by simp [Walk.mycielskian])).elim
      · -- uˡ – vʳ: impossible, it lies on the 5-cycle of the edge u – v
        exact (hncyc (isCycle_mycielskianClosed_of_adj hxy)
          (by simp [Walk.mycielskian])).elim
      · -- uʳ – none: as in the first case, u is isolated
        exact ⟨u, fun w hw => hncyc (isCycle_mycielskianClosed_of_adj hw.symm)
          (by simp [Sym2.eq_swap, Walk.mycielskian]), Sym2.eq_swap⟩
      · -- uʳ – vˡ: impossible, it lies on the 5-cycle of the edge v – u
        exact (hncyc (isCycle_mycielskianClosed_of_adj hxy.symm)
          (by simp [Sym2.eq_swap, Walk.mycielskian])).elim
  · rintro ⟨u, hiso, rfl⟩
    rw [isBridge_iff]
    -- u is isolated in G, so none is the only neighbour of uʳ: once the
    -- edge none – uʳ is deleted, uʳ has no neighbours at all
    have hnb : ∀ w, ¬(mμ(G).deleteEdges {s(none, uʳ)}).Adj uʳ w := by
      rintro (_ | w | w) h <;> simp_all [Sym2.eq_swap, IsIsolated]
    -- so a walk from uʳ to none has nowhere to take its first step
    intro h
    obtain ⟨p⟩ := h.symm
```

```
    cases p with
    | cons hadj _ => exact hnb _ hadj
```

Cycles in Graphs of Maximum Degree Two

In a graph in which every vertex has degree at most 2, a cycle is an entire connected component. Indeed, at a vertex x of a cycle c, x has two neighbours on the cycle (`Walk.IsCycle.ncard_neighborSet_toSubgraph_eq_two`) and at most two neighbours in G. Hence its neighbours on the cycle are all its neighbours in G (`Set.eq_of_subset_of_ncard_le`), so no edge leaves the cycle.

```
lemma Walk.IsCycle.neighborSet_toSubgraph_eq_of_degree_le_two [G.LocallyFinite]
    (hdeg : ∀ v, G.degree v ≤ 2) {u x : V} {c : G.Walk u u} (hc : c.IsCycle)
    (hx : x ∈ c.support) : c.toSubgraph.neighborSet x = G.neighborSet x :=
  Set.eq_of_subset_of_ncard_le (fun _ h => c.toSubgraph.adj_sub h)
    (by grind [ncard_neighborSet, hc.ncard_neighborSet_toSubgraph_eq_two hx])
```

Hence a vertex lies on the cycle iff it is reachable from it, that is, a cycle passes through every vertex of its connected component. One direction holds for any walk (`Walk.takeUntil`). For the other, if some vertex reachable from the cycle c lay off it, then some edge `y – z` of a walk to it would leave the cycle, with `y ∈ c.support` and `z ∉ c.support` (`Walk.exists_boundary_dart`). But every neighbour of `y` in G is a neighbour of `y` on the cycle, so `z` would lie on the cycle after all.

```
lemma Walk.IsCycle.mem_support_iff_reachable_of_degree_le_two [G.LocallyFinite]
    (hdeg : ∀ v, G.degree v ≤ 2) {u v : V} {c : G.Walk u u} (hc : c.IsCycle) :
    v ∈ c.support ↔ G.Reachable u v := by
  refine ⟨fun hv => by classical exact ⟨c.takeUntil v hv⟩, fun huv => ?_⟩
  by_contra hv
  obtain ⟨d, _, hd₁, hd₂⟩ :=
    huv.some.exists_boundary_dart {x | x ∈ c.support} c.start_mem_support hv
  have : d.snd ∈ c.toSubgraph.neighborSet d.fst := by
    rw [hc.neighborSet_toSubgraph_eq_of_degree_le_two hdeg hd₁]
    exact d.adj
  exact hd₂ (c.mem_support_of_adj_toSubgraph this.symm)
```

Together, these two lemmas say that the cycle *is* its connected component, as a subgraph of G.

```
theorem Walk.IsCycle.toSubgraph_eq_of_degree_le_two [G.LocallyFinite]
    (hdeg : ∀ v, G.degree v ≤ 2) {u : V} {c : G.Walk u u} (hc : c.IsCycle) :
    c.toSubgraph = G⟦u⟧.toSubgraph := by
  have hmem (x : V) := hc.mem_support_iff_reachable_of_degree_le_two hdeg (v := x)
  ext x y
  · simp [hmem, ConnectedComponent.eq, reachable_comm]
  · simp only [Subgraph.induce_adj, ConnectedComponent.mem_supp_iff,
      ConnectedComponent.eq, Subgraph.top_adj, reachable_comm (v := u), ← hmem]
    refine ⟨fun h => ⟨c.mem_support_of_adj_toSubgraph h,
      c.mem_support_of_adj_toSubgraph h.symm, h.adj_sub⟩,
      fun ⟨hx, _, hxy⟩ => ?_⟩
    rw [← Subgraph.mem_neighborSet,
      hc.neighborSet_toSubgraph_eq_of_degree_le_two hdeg hx]
    exact hxy
```

In particular, every cycle in a connected graph of maximum degree at most 2 is a *Hamiltonian cycle* (`Walk.IsHamiltonianCycle`): the tail of a cycle is a path (`Walk.IsCycle.isPath_tail`), and a path

through every vertex is Hamiltonian (`Walk.IsPath.isHamiltonian_of_mem`).

```
lemma Walk.IsCycle.isHamiltonianCycle_of_degree_le_two [DecidableEq V]
    [G.LocallyFinite] (hconn : G.Connected) (hdeg : ∀ v, G.degree v ≤ 2)
    {u : V} {c : G.Walk u u} (hc : c.IsCycle) : c.IsHamiltonianCycle :=
  ⟨hc, hc.isPath_tail.isHamiltonian_of_mem fun v => by
    rw [support_tail_of_not_nil _ hc.not_nil, mem_support_tail_iff hc.not_nil]
    exact (hc.mem_support_iff_reachable_of_degree_le_two hdeg).mpr (hconn u v)⟩
```

10.2 Trees

A graph is *acyclic* (a *forest*) if it contains no cycles. A *tree* is a connected forest.

Acyclic Graphs, Trees and Forests

```
#source IsAcyclic
```

Acyclicity is downward monotone in the subgraph order (any subgraph of an acyclic graph is acyclic), and induced subgraphs of forests are forests.

```
#check SimpleGraph.IsAcyclic.anti
#check SimpleGraph.IsAcyclic.induce -- G.IsAcyclic → (G.induce s).IsAcyclic
```

`G.IsTree` says `G` is connected and acyclic, and each connected component of a forest is a tree.

```
#source SimpleGraph.IsTree
#check IsAcyclic.isTree_connectedComponent
```

A graph is acyclic iff every pair of vertices is connected by at most one path. (Recall that `Subsingleton` means that a type has at most one element.)

```
#check isAcyclic_iff_subsingleton_path
-- G.IsAcyclic ↔ ∀ (u v : V), Subsingleton (G.Path u v)
```

So a tree is a graph in which every pair of vertices is connected by a *unique* path. In Mathlib this characterization needs `Nonempty V`: the graph on an empty vertex type vacuously has unique paths, but is not connected, so it is not a tree.

```
#check isTree_iff_existsUnique_path
```

Trees are exactly the minimally connected graphs, and (again on a nonempty vertex type) exactly the maximally acyclic graphs.

```
#check isTree_iff_minimal_connected -- G.IsTree ↔ Minimal Connected G
#check isTree_iff_maximal_isAcyclic
```

Linking acyclicity and the cycle graphs: a graph is acyclic iff it does not contain a copy of any cycle graph of order at least 3.

```
#check isAcyclic_iff_free_cycleGraph
```

The Infinite Path is a Tree

Our running example `pathNat` is a tree. It is connected, since every vertex `n` can be reached from `0` along the walk `pathNat.path 0 n`. This walk ends at `0 + n`, which is not definitionally `n`, so we use `Walk.copy` (Copy (p. 205)) to change its endpoint.

```
lemma connected_pathNat : pathNat.Connected :=
  (connected_iff_exists_forall_reachable _).2
    ⟨0, fun n => ⟨(pathNat.path 0 n).copy rfl (by simp)⟩⟩
```

To see that `pathNat` is acyclic, consider the largest vertex `m` of a cycle. It has two neighbours on the cycle (`IsCycle.ncard_neighborSet_toSubgraph_eq_two`), but its only neighbour that is not larger than `m` is `m - 1`.

```
lemma isAcyclic_pathNat : pathNat.IsAcyclic := by
  intro v c hc
  -- let m be the largest vertex of the cycle c
  obtain ⟨m, hm, hmax⟩ := c.support.toFinset.exists_max_image id ⟨v, by simp⟩
  simp only [List.mem_toFinset] at hm hmax
  -- its cycle neighbours are adjacent to m and at most m, so equal m - 1
  have hsub : c.toSubgraph.neighborSet m ⊆ {m - 1} := fun w hw => by
    have := c.toSubgraph.adj_sub hw
    have := hmax w (c.mem_support_of_adj_toSubgraph hw.symm)
    grind
  -- but m has two neighbours on the cycle
  have := Set.ncard_le_ncard hsub
  rw [hc.ncard_neighborSet_toSubgraph_eq_two hm, Set.ncard_singleton] at this
  exact Nat.not_succ_le_self _ this

lemma isTree_pathNat : pathNat.IsTree := ⟨connected_pathNat, isAcyclic_pathNat⟩
```

The Edge Count Formula

A tree on `n` vertices has exactly `n - 1` edges. Mathlib states this as `#G.edgeFinset + 1 = Fintype.card V`, which avoids the truncated subtraction of $\mathbb{N}$: working with `n - 1` would need the side condition `1 ≤ n`.

```
#check IsTree.card_edgeFinset -- #G.edgeFinset + 1 = Fintype.card V
```

Conversely, a finite connected graph with this many edges is a tree.

```
#check isTree_iff_connected_and_card
```

More generally, a connected graph satisfies `|V| ≤ |E(G)| + 1`. This is stated with `Nat.card`, so it needs no finiteness assumption, but for an infinite graph it doesn't say anything useful: `Nat.card` of an infinite type is the junk value `0`, so for `pathNat` it says `0 ≤ 0 + 1`.

```
#check Connected.card_vert_le_card_edgeSet_add_one
```

Any nontrivial finite tree has at least two leaves (vertices of degree one), and so its minimum degree is `1`. (Note `Nontrivial V` is the type class saying that `V` contains at least two elements.)

```
#check IsTree.exists_ne_and_degree_eq_one
#check IsTree.minDegree_eq_one_of_nontrivial
```

Deleting a leaf from a connected graph leaves it connected. This is the basic tool for proving results about finite trees by induction on the number of vertices.

```
#check Connected.induce_compl_singleton_of_degree_eq_one
```

Trees and Bridges

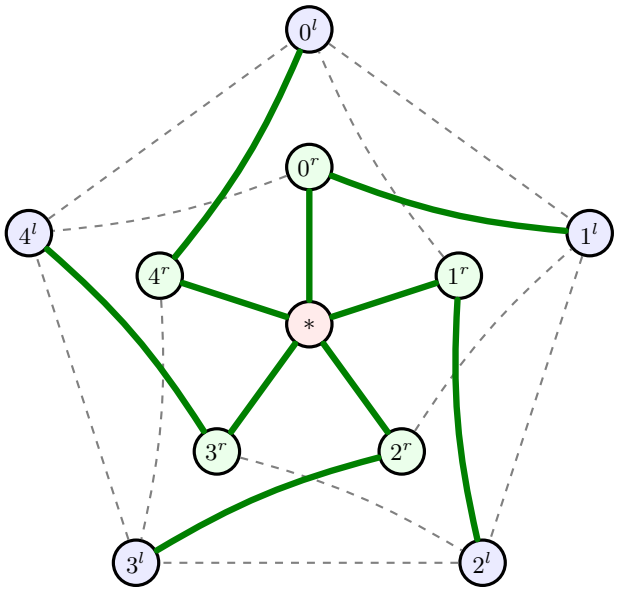
A graph is acyclic if and only if every edge is a bridge. For example, every edge of `pathNat` is a bridge.

```
#check isAcyclic_iff_forall_adj_isBridge
```

```
example {n : ℕ} : pathNat.IsBridge s(n, n + 1) :=
  isAcyclic_iff_forall_adj_isBridge.mp isAcyclic_pathNat (by simp)
```

Spanning Trees

A *spanning tree* of a connected graph `G` is a spanning subgraph that is a tree. Every connected graph has a spanning tree. Since `T` and `G` are both of type `SimpleGraph V`, the subgraph `T ≤ G` is automatically spanning (see Spanning Subgraphs (p. 253)).



A spanning tree in the `grotzschGraph`

```
#check Connected.exists_isTree_le
```

Every graph `G` has a spanning forest (an acyclic spanning subgraph `F` in which any pair of vertices is joined by a walk in `F` iff it is joined by a walk in `G`). Notice that there is no finiteness condition here; the proof uses a maximality argument via Zorn's lemma.

```
#check exists_isAcyclic_reachable_eq_le
```

10.3 Eulerian Circuits and Trails

Mathlib has the necessary conditions in Euler's theorem [Euler 1741] (`IsEulerian.even_degree_iff` and `IsEulerian.card_odd_degree`), but not the existence direction: a connected graph with all degrees even has an Eulerian circuit, and a connected graph with exactly two vertices of odd degree has an Eulerian trail. We prove these here with a splicing argument in the style of *Hierholzer's algorithm* [Hierholzer 1873]. Rather than running the algorithm we take a longest trail and show that, if it were not Eulerian, we could splice a circuit into it to make it longer.

A standard proof might typically prove this for the all-even circuit case first and then argue that the open trail case reduces to the closed circuit case by adding a new vertex and two new edges from the odd-degree vertices to the new vertex. This would definitely be possible in Lean, but somehow more complicated than necessary. Instead we prove the open case directly, reusing code from the circuit case.

Euler's theorem is often stated for connected graphs, but, as we saw in Eulerian Trails (p. 200), `IsEulerian` ignores isolated vertices (`IsEulerian.mem_support_iff`). The analogous necessary condition is that all the edges of `G` lie in a single connected component. We make this a definition below (`EdgesInOneComponent`). This is the weakest hypothesis under which the Eulerian existence theorems below hold, strictly weaker than `Connected`, which also forbids isolated vertices (when there are at least two vertices) and the graph with no vertices.

```
section EulerExistence
```

Every edge of `G` lies in a single connected component: any two non-isolated vertices are reachable from each other.

```
def EdgesInOneComponent (G : SimpleGraph V) : Prop :=
  ∀ u ∈ G.support, ∀ v ∈ G.support, G.Reachable u v
```

A connected graph has all its edges in one component (indeed all its vertices are mutually reachable). This allows the Eulerian existence results to be stated for the weaker `EdgesInOneComponent` and recovered for `Connected` as corollaries.

```
lemma Connected.edgesInOneComponent (h : G.Connected) : G.EdgesInOneComponent :=
  fun u _ v _ => h u v
```

Conversely, `EdgesInOneComponent` is necessary: an Eulerian walk uses every edge, so both endpoints of every edge lie on the walk, and all vertices of a walk are mutually reachable.

```
lemma Walk.IsEulerian.edgesInOneComponent [DecidableEq V] {u v} {p : G.Walk u v}
    (hp : p.IsEulerian) : G.EdgesInOneComponent := by
  have hmem : ∀ z ∈ G.support, z ∈ p.support := fun z hz =>
    hp.mem_support_of_not_isIsolated ((G.mem_support_iff_not_isIsolated).mp hz)
  exact fun x hx y hy =>
    ((p.takeUntil x (hmem x hx)).reachable).symm.trans
      (p.takeUntil y (hmem y hy)).reachable
```

Mathlib has the corresponding statement for `Connected`: if `G` has an Eulerian walk and no isolated vertices, then `G` is connected.

```
#check Walk.IsEulerian.connected_of_forall_not_isIsolated
```

As we saw in Eulerian Trails (p. 200), an Eulerian walk forces the edge set to be finite (`IsEulerian.fintypeEdgeSet`). So for any `G` with `[Nonempty V]`, `G` has an Eulerian walk iff

- `G.edgeSet.Finite`,
- `G.EdgesInOneComponent`, and
- the set of vertices of odd degree has exactly 0 or 2 elements.

We prove the existence direction in this generality, assuming only `[Finite G.edgeSet]`, so infinitely many isolated vertices are allowed. For the full characterization we assume `[Finite V]`, as Mathlib's half of it does.

Our statements assume `[G.LocallyFinite]` rather than `[DecidableRel G.Adj]`, since what they mention are degrees (see Instance Arguments in Statements (p. 63)), so they apply whichever instance a graph's neighbourhoods carry. The proofs also talk about degrees in the graphs `G.deleteEdges s` obtained by deleting edges, so we first give these graphs a `LocallyFinite` instance of their own.

Neighbourhoods in `G.deleteEdges s` are finite whenever those in `G` are.

```
@[instance_reducible]
def locallyFiniteDeleteEdges [G.LocallyFinite] (s : Set (Sym2 V))
    [DecidablePred (· ∈ s)] : (G.deleteEdges s).LocallyFinite := fun v =>
  Fintype.ofFinset ((G.neighborFinset v).filter fun w => s(v, w) ∉ s) fun w => by
    simp
```

As with `locallyFiniteOfFintypeEdgeSet` in Finite Neighbourhoods (p. 69), we make this a `def` rather than an `instance`. As a global instance it would create a diamond: when `V` is a `Fintype` and `G.Adj` is decidable, Mathlib already gives the neighbourhoods of `G.deleteEdges s` a `Fintype` instance, and ours would be a second, different one.

Instead we use the `attribute` command, which attaches an attribute to a declaration that already exists: `attribute [simp] foo` has the same effect as writing `@[simp]` in front of the declaration of `foo`. The keyword `local` limits the effect to the current `section` or `namespace`. So after the line below, type class inference uses `locallyFiniteDeleteEdges` until the end of this section and nowhere else.

```
attribute [local instance] locallyFiniteDeleteEdges
```

Extending Trails

In an all-even-degree graph, a trail that has not returned to its start can be extended: its current endpoint `w` is incident to an edge the trail has not used. (The number of edges in the trail that are incident to `w` is odd, while `degree w` is even, so some incident edge is still unused.)

We first prove a simple lemma about the number of edges of a trail `p` incident to a vertex `x`. Since a trail does not repeat edges, the number of edges of `G` at `x` used by a trail `p` is the number of entries of `p.edges` containing `x`. (This uses `List.countP`.)

```
namespace Walk
open Finset
variable [G.LocallyFinite]

lemma IsTrail.card_filter_incidenceFinset [DecidableEq V] {u v} {p : G.Walk u v}
    (hp : p.IsTrail) (x : V) :
    #((G.incidenceFinset x).filter (· ∈ p.edges))
      = p.edges.countP (fun e => x ∈ e) := by
  rw [List.countP_eq_length_filter,
    ← List.toFinset_card_of_nodup (hp.edges_nodup.filter _)]
  congr 1; ext e
  simp only [mem_filter, mem_incidenceFinset, incidenceSet, Set.mem_ofPred_eq,
    List.mem_toFinset, List.mem_filter, decide_eq_true_eq]
  grind [p.edges_subset_edgeSet]
```

So in an all-even-degree graph, a trail that has not returned to its start leaves some edge at its endpoint unused.

```
lemma IsTrail.exists_unused_edge_of_even {v w} (h2 : ∀ x, Even (G.degree x))
    {p : G.Walk v w} (hp : p.IsTrail) (hvw : v ≠ w) :
    ∃ x, G.Adj w x ∧ s(w, x) ∉ p.edges := by
  classical
  -- the number of edges in p that are incident to w is odd
  have hodd : ¬Even (p.edges.countP (fun e => w ∈ e)) := by
    rw [hp.even_countP_edges_iff w]; simp [hvw]
  -- if every edge at w were used, that number would be degree w, which is even
  by_contra! h
  apply hodd
  rw [← hp.card_filter_incidenceFinset, filter_true_of_mem,
    card_incidenceFinset_eq_degree]
  · exact h2 w
  · intro e he
    rw [mem_incidenceFinset] at he
    obtain ⟨x, rfl⟩ := Sym2.mem_iff_exists.mp he.2
    exact h x he.1
```

A trail repeats no edges, so its length is at most the number of edges in the graph.

```
#check IsTrail.length_le_ncard_edgeSet  -- p.IsTrail → p.length ≤ G.edgeSet.ncard
```

In an all-even-degree graph, any trail `p` starting at `v` can be extended to a closed trail from `v` of length at least `p.length`, by repeatedly applying `IsTrail.exists_unused_edge_of_even` until it returns to `v`. The proof applies `IsTrail.exists_closed_isTrail_length_le` recursively to longer trails. It isn't obvious to Lean that this is well-founded so we need to give a termination argument.

We know that no trail is longer than the number of edges in the graph (`IsTrail.length_le_ncard_edgeSet`), and the trail in the recursive call is strictly longer than the original trail `p`, so the measure `G.edgeSet.ncard - p.length` strictly decreases with each recursive call. This is what we are doing in the `termination_by` and `decreasing_by` section below. The lemma assumes `[Finite G.edgeSet]`: in a graph with infinitely many edges, such as the two-way infinite path, a trail can be extended forever.

```
lemma IsTrail.exists_closed_isTrail_length_le [Finite G.edgeSet]
    (h2 : ∀ x, Even (G.degree x)) {v w} {p : G.Walk v w} (hp : p.IsTrail) :
```

```
    ∃ q : G.Walk v v, q.IsTrail ∧ p.length ≤ q.length := by
  -- if v = w then p is already a closed trail so we are done
  by_cases hvw : v = w
  · subst hvw; exact ⟨p, hp, le_refl _⟩
  · -- we get an edge from the end of p that is not used by p
    obtain ⟨x, hadj, hnotin⟩ := hp.exists_unused_edge_of_even h2 hvw
    -- concatenating this edge to p gives a longer trail
    have hp' : (p.concat hadj).IsTrail := hp.concat hadj hnotin
    -- Call the theorem recursively on this longer trail.
    obtain ⟨q, hq, hlen⟩ := hp'.exists_closed_isTrail_length_le h2
    exact ⟨q, hq, by grind [length_concat]⟩
termination_by G.edgeSet.ncard - p.length
decreasing_by grind [hp'.length_le_ncard_edgeSet, length_concat]
```

Starting from any vertex of positive degree in an all-even-degree graph there is a circuit (a nonempty closed trail).

```
lemma _root_.SimpleGraph.exists_isCircuit_of_even [Finite G.edgeSet] {v}
    (hv : 0 < G.degree v) (h2 : ∀ x, Even (G.degree x)) :
    ∃ p : G.Walk v v, p.IsCircuit := by
  obtain ⟨u, hu⟩ := (G.degree_pos_iff_exists_adj v).mp hv
  obtain ⟨q, hq, hlen⟩ := hu.isPath_toWalk.isTrail.exists_closed_isTrail_length_le h2
  exact ⟨q, hq, by rintro rfl; simp at hlen⟩
```

Deleting the edges of a trail `p` changes the degree at each vertex `x` by the number of edges of `p` incident to `x`. Hence if this number has the same parity as `G.degree x` at every vertex, then every degree in the graph formed by deleting the edges of `p` is even.

```
variable [DecidableEq V]
lemma IsTrail.even_degree_deleteEdges {u v} {p : G.Walk u v} (hp : p.IsTrail)
    (hpar : ∀ x, Even (p.edges.countP (fun e => x ∈ e)) ↔ Even (G.degree x)) :
    ∀ x, Even ((G.deleteEdges {e | e ∈ p.edges}).degree x) := by
  intro x
  have hinc : (G.deleteEdges {e | e ∈ p.edges}).incidenceFinset x
                = (G.incidenceFinset x).filter (· ∉ p.edges) := by
    grind [mem_incidenceFinset, incidenceSet, edgeSet_deleteEdges]
  -- the deleted degree at x plus the number of edges of
  -- p at x is the original degree
  grind [card_filter_add_card_filter_not, card_incidenceFinset_eq_degree,
         card_filter_incidenceFinset]
```

So in an all-even-degree graph, deleting the edges of a closed trail preserves the property that every degree is even (a closed trail uses an even number of edges at every vertex).

```
lemma IsTrail.even_degree_deleteEdges_closed {v} {p : G.Walk v v} (hp : p.IsTrail)
    (h2 : ∀ x, Even (G.degree x)) :
    ∀ x, Even ((G.deleteEdges {e | e ∈ p.edges}).degree x) :=
  hp.even_degree_deleteEdges fun x => by rw [hp.even_countP_edges_iff x]; simp [h2 x]
```

Deleting the edges of an open trail from `a` to `b` flips the degree parity at exactly its two endpoints. So if `a` and `b` are the only vertices of odd degree, then every degree in the deleted graph is even.

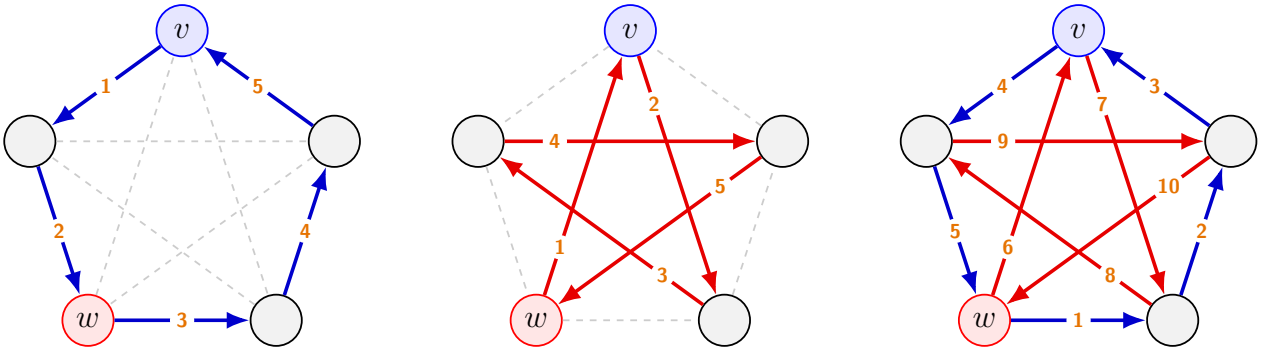
```
lemma IsTrail.even_degree_deleteEdges_open {a b} (hab : a ≠ b) {p : G.Walk a b}
    (hp : p.IsTrail) (hpar : ∀ x, Even (G.degree x) ↔ x ≠ a ∧ x ≠ b) :
```

```
    ∀ x, Even ((G.deleteEdges {e | e ∈ p.edges}).degree x) :=
  hp.even_degree_deleteEdges fun x =>
    by rw [hp.even_countP_edges_iff x, hpar x]; simp [hab]
```

Let `p : G.Walk u v` be a trail in a graph `G` whose edges all lie in one component. If `u` is not an isolated vertex and `p` is not Eulerian, then some vertex of `p` still has positive degree in the graph obtained by deleting the edges of `p`.

```
lemma IsTrail.exists_support_pos_deleted {u v} (hG : G.EdgesInOneComponent)
    (hu : u ∈ G.support) {p : G.Walk u v}
    (hp : p.IsTrail) (hne : ¬p.IsEulerian) :
    ∃ w ∈ p.support, 0 < (G.deleteEdges {e | e ∈ p.edges}).degree w := by
  let Gp := G.deleteEdges {e | e ∈ p.edges}
  -- some edge a — b of G is not used by p
  obtain ⟨e, he, hep⟩ : ∃ e ∈ G.edgeSet, e ∉ p.edges := by
    by_contra! h; exact hne (hp.isEulerian_of_forall_mem h)
  induction e using Sym2.ind with | _ a b => ?_
  have hab : G.Adj a b := he
  have hGab : Gp.Adj a b := by grind [deleteEdges_adj]
  have hpos : ∀ {x y}, Gp.Adj x y → 0 < Gp.degree x :=
    fun {x y} hxy => (Gp.degree_pos_iff_exists_adj x).mpr ⟨y, hxy⟩
  by_cases ha : a ∈ p.support
  · exact ⟨a, ha, hpos hGab⟩
  · obtain ⟨q⟩ := hG a hab.left_mem_support u hu
    obtain ⟨⟨⟨c, d⟩, hcd⟩, _, ⟨hc, hd⟩⟩ :=
      q.reverse.exists_boundary_dart {x | x ∈ p.support} p.start_mem_support ha
    have hcd_notin : s(c, d) ∉ p.edges :=
      fun hmem => hd (p.snd_mem_support_of_mem_edges hmem)
    have hcdGp : Gp.Adj c d := by grind [deleteEdges_adj]
    exact ⟨c, hc, hpos hcdGp⟩
```

A non-Eulerian trail `p` from `a` to `b` can be extended, provided every vertex has even degree in the graph obtained by deleting the edges of `p`. Find a vertex `w` on `p` incident to an unused edge, build a circuit `c` at `w` in the deleted graph, transfer it back into `G`, and splice it into `p` at `w` (`IsTrail.takeUntil_append_append_dropUntil`).



Splice a new circuit `c` (red) at `w` into the circuit `p` (blue) starting at `v`, to obtain an Eulerian circuit starting at `w` (right)

Notice that this lemma will allow us to handle both the closed case (`a = b`, the Eulerian *circuit*, where the parity hypothesis `hdel` comes from `IsTrail.even_degree_deleteEdges_closed`) and the open case (`a ≠ b`, the Eulerian *trail*, where it comes from `IsTrail.even_degree_deleteEdges_open`).

```
lemma IsTrail.exists_longer_of_not_isEulerian [Finite G.edgeSet] {a b}
    (hG : G.EdgesInOneComponent) (ha : a ∈ G.support) {p : G.Walk a b}
    (hp : p.IsTrail) (hdel : ∀ x, Even ((G.deleteEdges {e | e ∈ p.edges}).degree x))
    (hne : ¬p.IsEulerian) :
    ∃ q : G.Walk a b, q.IsTrail ∧ p.length < q.length := by
  -- some vertex w on p is incident to an edge of G unused by p
  obtain ⟨w, hw_supp, hw_pos⟩ := hp.exists_support_pos_deleted hG ha hne
  -- build a circuit c at w in the deleted graph, which has fewer edges than G
  have : Finite (G.deleteEdges {e | e ∈ p.edges}).edgeSet :=
    Finite.Set.subset _ (edgeSet_mono (deleteEdges_le _))
  obtain ⟨c, hc⟩ := exists_isCircuit_of_even hw_pos hdel
  -- every edge of c is an edge of G, so c transfers back into G
  have htr : ∀ e ∈ c.edges, e ∈ G.edgeSet := fun e he =>
    edgeSet_mono (deleteEdges_le _) (c.edges_subset_edgeSet he)
  -- the edges of the transferred c are unused by p
  have hc_notin : List.Disjoint (c.transfer G htr).edges p.edges := by
    intro e he hpe
    rw [edges_transfer] at he
    grind [edgeSet_deleteEdges, c.edges_subset_edgeSet he]
  -- splice c into p at w: walk a → w, then around c, then w → b. This
  -- is a strictly longer trail from a to b.
  refine ⟨_, hp.takeUntil_append_append_dropUntil hw_supp (hc.isTrail.transfer htr)
    hc_notin, ?_⟩
  grind [length_takeUntil_append_append_dropUntil, length_transfer,
         hc.three_le_length]

end Walk
```

Euler's Theorem

Both cases of Euler's theorem now follow. Let q be a longest trail from a to b (trails have length at most `G.edgeSet.ncard`, so one exists as long as there is any trail q_0 from a to b). If q is not Eulerian then `IsTrail.exists_longer_of_not_isEulerian` gives a longer trail, a contradiction. The two cases differ only in why deleting the edges of a trail leaves every degree even, so we take this as the hypothesis `hdel`.

```
variable [G.LocallyFinite] [DecidableEq V] [Finite G.edgeSet]

lemma exists_isEulerian_of_isTrail {a b} (hG : G.EdgesInOneComponent)
    (ha : a ∈ G.support) {q₀ : G.Walk a b} (hq₀ : q₀.IsTrail)
    (hdel : ∀ q : G.Walk a b, q.IsTrail →
      ∀ x, Even ((G.deleteEdges {e | e ∈ q.edges}).degree x)) :
    ∃ p : G.Walk a b, p.IsEulerian := by
  classical
  -- P k says there is a trail from a to b of length k
  let P : ℕ → Prop := fun k => ∃ q : G.Walk a b, q.IsTrail ∧ q.length = k
  -- let q be a longest trail from a to b
  obtain ⟨q, hq, hq_len⟩ := Nat.findGreatest_spec (P := P) (m := q₀.length)
    (n := G.edgeSet.ncard) hq₀.length_le_ncard_edgeSet ⟨q₀, hq₀, rfl⟩
  refine ⟨q, by_contra fun hne => ?_⟩
  -- the graph with q's edges deleted has even degrees, so q can be extended
  obtain ⟨q', hq', hlt⟩ := hq.exists_longer_of_not_isEulerian hG ha (hdel q hq) hne
  have : q'.length ≤ Nat.findGreatest P G.edgeSet.ncard :=
```

```
    Nat.le_findGreatest hq'.length_le_ncard_edgeSet ⟨q', hq', rfl⟩
  grind
```

Euler's theorem (existence direction, closed case): a graph with finitely many edges, all in one component (`EdgesInOneComponent`), in which every vertex has even degree has a closed Eulerian walk. `[Nonempty V]` is needed only so that we can name a start vertex in the case that the graph has no edges. In that case the walk is `nil`, which is not a circuit (`IsCircuit` requires `p ≠ nil`), so we don't call it an Eulerian circuit.

Euler's theorem, closed case: a graph with finitely many edges, all in one component, in which every vertex has even degree has a closed Eulerian walk.

```
theorem exists_closed_isEulerian [Nonempty V]
    (h : G.EdgesInOneComponent) (h2 : ∀ v, Even (G.degree v)) :
    ∃ u : V, ∃ (p : G.Walk u u), p.IsEulerian := by
  by_cases hsupp : G.support.Nonempty
  · -- pick a non-isolated start u; a longest closed trail from u is Eulerian
    obtain ⟨u, hu⟩ := hsupp
    exact ⟨u, exists_isEulerian_of_isTrail h hu .nil
      fun q hq => hq.even_degree_deleteEdges_closed h2⟩
  · -- no non-isolated vertex: G is edgeless, so nil is Eulerian
    obtain ⟨u⟩ := ‹Nonempty V›
    exact ⟨u, nil,
      (isEulerian_nil_iff u).mpr (Set.not_nonempty_iff_eq_empty.mp hsupp)⟩
```

If `G` has at least one edge then the closed Eulerian walk uses it, so it is not `nil` and is an Eulerian *circuit*: in Mathlib this is `IsEulerian` together with `IsCircuit`.

Euler's theorem, circuit form: a graph with at least one edge, finitely many edges, all in one component, and every degree even has an Eulerian circuit.

```
theorem exists_isEulerian_isCircuit (h : G.EdgesInOneComponent)
    (h2 : ∀ v, Even (G.degree v)) (hE : G.edgeSet.Nonempty) :
    ∃ u : V, ∃ p : G.Walk u u, p.IsEulerian ∧ p.IsCircuit := by
  obtain ⟨x, y, hxy⟩ := Sym2.exists.mp hE
  have : Nonempty V := ⟨x⟩
  obtain ⟨u, p, hp⟩ := exists_closed_isEulerian h h2
  -- p uses the edge s(x, y), so it is not nil
  refine ⟨u, p, hp, hp.isTrail, ?_⟩
  rintro rfl
  simpa using hp.mem_edges_iff.mpr hxy
```

Euler's theorem (existence direction, closed case) for a *connected* graph: follows from `exists_closed_isEulerian` via `Connected.edgesInOneComponent`.

Euler's theorem, closed case, for a connected graph: if every vertex has even degree then there is a closed Eulerian walk.

```
theorem Connected.exists_closed_isEulerian (h : G.Connected)
    (h2 : ∀ v, Even (G.degree v)) :
    ∃ u : V, ∃ (p : G.Walk u u), p.IsEulerian :=
  have : Nonempty V := h.nonempty
  SimpleGraph.exists_closed_isEulerian h.edgesInOneComponent h2
```

Euler's theorem (existence direction, open trail): if `G` has all edges in one component (`EdgesInOneComponent`) and `a ≠ b` are the only two odd-degree vertices, then `G` has an Eulerian trail from `a` to `b`.

Euler's theorem, open case: if the edges of `G` all lie in one component and `a ≠ b` are its only vertices of odd degree, then `G` has an Eulerian trail from `a` to `b`.

```
theorem exists_isEulerian_trail_of_two_odd {a b}
    (hG : G.EdgesInOneComponent) (hab : a ≠ b)
    (hpar : ∀ x, Even (G.degree x) ↔ x ≠ a ∧ x ≠ b) :
    ∃ p : G.Walk a b, p.IsEulerian := by
  -- the two odd-degree vertices a, b are non-isolated, hence in G.support
  have hsupp : a ∈ G.support ∧ b ∈ G.support := by
    simp only [← degree_pos_iff_mem_support]; grind [Odd.pos]
  -- since a and b lie in the same component there is a path, hence a trail
  obtain ⟨q₀, hq₀⟩ := (hG a hsupp.1 b hsupp.2).some.toPath
  exact exists_isEulerian_of_isTrail hG hsupp.1 hq₀.isTrail
    fun q hq => hq.even_degree_deleteEdges_open hab hpar
```

Euler's theorem (existence direction, open trail) for a *connected* graph: follows from `exists_isEulerian_trail_of_two_odd` via `Connected.edgesInOneComponent`.

Euler's theorem, open case, for a connected graph: if `a ≠ b` are its only vertices of odd degree, then there is an Eulerian trail from `a` to `b`.

```
theorem Connected.exists_isEulerian_trail {a b} (h : G.Connected)
    (hab : a ≠ b) (hpar : ∀ x, Even (G.degree x) ↔ x ≠ a ∧ x ≠ b) :
    ∃ p : G.Walk a b, p.IsEulerian :=
  exists_isEulerian_trail_of_two_odd h.edgesInOneComponent hab hpar
```

Euler's theorem, full characterization: a finite graph with at least one vertex has an Eulerian trail *iff* all its edges lie in one component and it has either no odd-degree vertex or exactly two. The forward direction is `IsEulerian.edgesInOneComponent` with Mathlib's `IsEulerian.card_odd_degree`; the reverse direction is `exists_closed_isEulerian` (no odd-degree vertex) or `exists_isEulerian_trail_of_two_odd` (exactly two). We only reason about the number of odd-degree vertices, so we count them with `Set.ncard` and `[Finite V]` suffices; it implies the section's `[Finite G.edgeSet]` assumption, which we `omit`. Mathlib's `IsEulerian.card_odd_degree` counts using `Fintype.card` instead, so the forward direction converts between the two.

```
omit [Finite G.edgeSet]
```

Euler's theorem: a finite graph with at least one vertex has an Eulerian trail iff its edges all lie in one component and it has `0` or `2` vertices of odd degree.

```
theorem exists_isEulerian_iff [Finite V] [Nonempty V] :
    (∃ u v, ∃ p : G.Walk u v, p.IsEulerian) ↔
      G.EdgesInOneComponent ∧ ({v | Odd (G.degree v)}.ncard = 0
        ∨ {v | Odd (G.degree v)}.ncard = 2) := by
  have := Fintype.ofFinite V
  refine ⟨fun ⟨_, _, _, hp⟩ => ⟨hp.edgesInOneComponent, by
    classical
    rw [← Nat.card_coe_set_eq, Nat.card_eq_fintype_card]
    convert hp.card_odd_degree⟩, ?_⟩
  rintro ⟨hG, h0 | h2⟩
  · -- no odd-degree vertex: every degree is even, so there is a closed Eulerian walk
    rw [Set.ncard_eq_zero, Set.eq_empty_iff_forall_notMem] at h0
```

```
    obtain ⟨u, p, hp⟩ := exists_closed_isEulerian hG fun x =>
      Nat.not_odd_iff_even.mp (h0 x)
    exact ⟨u, u, p, hp⟩
  · -- exactly two odd-degree vertices a, b: an Eulerian trail from a to b
    rw [Set.ncard_eq_two] at h2
    obtain ⟨a, b, hab, hS⟩ := h2
    have hpar (x) : Even (G.degree x) ↔ x ≠ a ∧ x ≠ b := by
      simpa [← Nat.not_odd_iff_even] using (Set.ext_iff.mp hS x).not
    exact ⟨a, b, exists_isEulerian_trail_of_two_odd hG hab hpar⟩

end EulerExistence
```

10.4 Edge and Vertex Connectivity

Informally, a graph is k-edge-connected if deleting less than k edges always leaves it connected, and k-vertex-connected if deleting less than k vertices always leaves it connected. Mathlib has the first notion, and we define the second.

These are the *deletion* forms of the definitions. *Menger's theorem* [Menger 1927] gives the equivalent *path* forms: a graph is k-edge-connected iff any two distinct vertices are joined by k edge-disjoint paths, and a graph with more than k vertices is k-vertex-connected iff any two distinct vertices are joined by k paths that share only their endpoints. Menger's theorem is not currently in Mathlib.

Edge Connectivity

In Mathlib edge connectivity is defined via `G.IsEdgeReachable`, which says that two vertices `u`, `v` are k-edge-reachable if after deleting less than k edges from `G` there is still a walk from `u` to `v`. Here the number of deleted edges is measured with `Set.encard`, which is `⊤` if the set is infinite and otherwise its cardinality as a natural number. The parameter k is also an element of $\mathbb{N}\infty$, so it can be `⊤`. Two vertices are `⊤`-edge-reachable iff they remain reachable after deleting any finite set of edges.

```
#source IsEdgeReachable
```

```
#check isEdgeReachable_top_iff_forall_finite
```

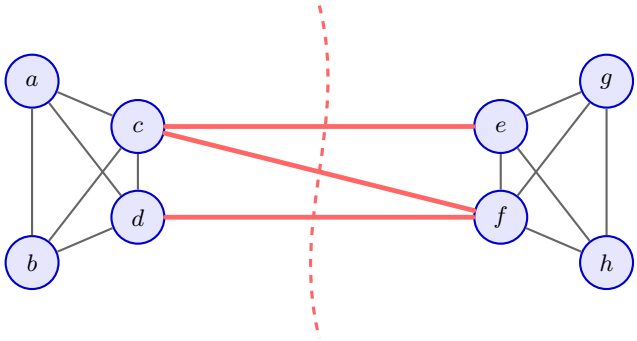
Notice that the set `s` of deleted edges is an arbitrary `Set (Sym2 V)`, so it may contain pairs that are not edges of `G`. Deleting these does nothing, so this is harmless, but it means that we sometimes have to filter them out (as in the proof of `IsVertexConnected.isEdgeConnected` below).

A graph is k-edge-connected if every pair of vertices is k-edge-reachable.

```
#source IsEdgeConnected
```

Beware that this only gives *pre*connectedness. If `G` is k-edge-connected for `k ≠ 0` then it is preconnected, but a graph with at most one vertex (including the graph with no vertices) is k-edge-connected for every k.

```
#check IsEdgeConnected.preconnected
```



A 3-edge-connected graph with a 3-edge cut

An edge is a bridge iff its endpoints are not 2-edge-reachable. So a graph is 2-edge-connected iff deleting any single `e : Sym2 V` leaves it preconnected. Here `e` need not be an edge of `G`, and deleting a non-edge doesn't change `G`, so this includes `G` itself being preconnected.

```
#check isBridge_iff_not_isEdgeReachable_two
#check isEdgeConnected_two_iff_forall_preconnected
```

A graph is acyclic (a forest) iff no two distinct vertices are 2-edge-reachable.

```
#check isAcyclic_iff_pairwise_not_isEdgeReachable_two
```

Vertex Connectivity

A graph is k-vertex-connected if deleting less than k vertices always leaves the graph connected. This isn't currently in Mathlib so we give a definition below. As is standard, when we say *k-connected* without qualification we mean k-vertex-connected.

Since graphs with no vertices are not connected we insist that the vertex type has more than k vertices. As for edge connectivity, k is an element of $\mathbb{N}\infty$, and we measure sets of vertices with `encard` and count the vertices with `ENat.card V`, so that infinite vertex types are handled correctly. In particular, a graph is $\top$-vertex-connected iff it has infinitely many vertices and deleting any finite set of vertices leaves it connected.

Rather than giving a definition in terms of the graph formed by deleting vertices, we ask that for any set `S` of less than k vertices, and any pair of vertices not in `S`, there is a walk in `G` between the pair which avoids `S`.

A graph is k-vertex-connected if deleting less than k vertices always leaves the graph connected.

```
def IsVertexConnected (G : SimpleGraph V) (k : ℕ∞) : Prop :=
  k + 1 ≤ ENat.card V ∧
    ∀ (S : Set V), S.encard < k → ∀ (u v : V), u ∉ S → v ∉ S →
      ∃ p : G.Walk u v, ∀ x ∈ p.support, x ∉ S
```

The graph obtained by deleting `S` is `G.induce Sᶜ`, but its vertex type is the subtype `↑Sᶜ`, so its walks run between vertices `⟨u, hu⟩` that carry proofs that `u ∉ S`. Walks in `G` that avoid `S` are much easier to construct and combine.

The two formulations agree. To see this we pass through Mathlib's `Subgraph` API (`Subgraph.induce`): `preconnected_induce_iff` turns preconnectedness of `G.induce Sᶜ` into preconnectedness of the subgraph `(⊤ : G.Subgraph).induce Sᶜ`, and `Subgraph.preconnected_iff_forall_`

`exists_walk_subgraph` characterizes the latter by walks p in G with `p.toSubgraph ≤ (⊤ : G.Subgraph).induce Sᶜ`. For an induced subgraph this just says that every vertex of p lies in `Sᶜ`.

```
lemma Walk.toSubgraph_le_induce_top_iff {W : Set V} {u v : V} (p : G.Walk u v) :
    p.toSubgraph ≤ (⊤ : G.Subgraph).induce W ↔ ∀ y ∈ p.support, y ∈ W := by
  refine ⟨fun h y hy => h.1 (p.mem_verts_toSubgraph.mpr hy), fun h => ?_⟩
  exact p.toSubgraph_le_induce_support.trans
    (Subgraph.induce_mono_right fun y hy => h y hy)

lemma isVertexConnected_iff_induce {k : ℕ∞} : G.IsVertexConnected k ↔
    k + 1 ≤ ENat.card V ∧
      ∀ S : Set V, S.encard < k → (G.induce Sᶜ).Preconnected := by
  refine and_congr_right fun _ => forall₂_congr fun S _ => ?_
  rw [preconnected_induce_iff, Subgraph.preconnected_iff_forall_exists_walk_subgraph]
  simp only [Subgraph.induce_verts, Set.mem_compl_iff,
    Walk.toSubgraph_le_induce_top_iff]
```

A set with less than k elements is finite, so it is enough to check the condition for finsets, and conversely we can apply it to finsets.

```
open Finset

lemma IsVertexConnected.of_finset {k : ℕ∞} (hcard : k + 1 ≤ ENat.card V)
    (h : ∀ (S : Finset V), #S < k → ∀ (u v : V), u ∉ S → v ∉ S →
      ∃ p : G.Walk u v, ∀ x ∈ p.support, x ∉ S) : G.IsVertexConnected k := by
  refine ⟨hcard, fun S hS u v hu hv => ?_⟩
  lift S to Finset V using Set.encard_lt_top_iff.mp (hS.trans_le le_top)
  exact h S (by simpa using hS) u v hu hv

lemma IsVertexConnected.exists_walk {k : ℕ∞} (hG : G.IsVertexConnected k)
    (S : Finset V) (hS : #S < k) (u v : V) (hu : u ∉ S) (hv : v ∉ S) :
    ∃ p : G.Walk u v, ∀ x ∈ p.support, x ∉ S :=
  hG.2 S (by simpa using hS) u v hu hv
```

A k-vertex-connected graph has more than k vertices, so any finite set of at most k vertices misses some vertex.

```
lemma IsVertexConnected.exists_notMem {k : ℕ∞} (hG : G.IsVertexConnected k)
    (A : Finset V) (hA : #A ≤ k) : ∃ x, x ∉ A :=
  A.exists_notMem_of_card_lt_enatCard
    ((ENat.natCast_lt_add_one_iff.mpr hA).trans_le hG.1)
```

Being k-vertex-connected is closed under decreasing k.

```
lemma IsVertexConnected.anti {k l : ℕ∞} (hkl : k ≤ l) (hG : G.IsVertexConnected l) :
    G.IsVertexConnected k :=
  ⟨le_trans (by gcongr) hG.1, fun S hS => hG.2 S (hS.trans_le hkl)⟩
```

If a graph is k-vertex-connected then any vertex has at least k neighbours: if there were less than k neighbours `N` of v, then a walk from v to a vertex outside `N ∪ {v}` avoiding `N` would have nowhere to take its first step. We state this first with `Set.encard`, so that no finiteness assumption on the neighbourhood is needed.

```
lemma IsVertexConnected.le_encard_neighborSet {k : ℕ∞} (hG : G.IsVertexConnected k)
    (v : V) : k ≤ (G.neighborSet v).encard := by
```

```
  by_contra! hlt
  obtain ⟨u, hu⟩ : ∃ u, u ∉ insert v (G.neighborSet v) := by
    by_contra! h
    -- h says that every vertex is v or a neighbour of v, so G would have
    -- less than k + 1 vertices
    have hlt' : k + 1 < k + 1 :=
      calc k + 1 ≤ ENat.card V := hG.1
        _ = (Set.univ : Set V).encard := (Set.encard_univ V).symm
        _ ≤ (insert v (G.neighborSet v)).encard :=
            Set.encard_le_encard fun x _ => h x
        _ ≤ (G.neighborSet v).encard + 1 := Set.encard_insert_le _ _
        _ < k + 1 := (ENat.add_lt_add_iff_right ENat.one_ne_top).mpr hlt
    exact lt_irrefl _ hlt'
  rw [Set.mem_insert_iff, not_or] at hu
  obtain ⟨p, hp⟩ := hG.2 _ hlt v u G.irrefl hu.2
  cases p with
  | nil => exact hu.1 rfl
  | cons hadj p' => exact hp _ (by simp) hadj

lemma IsVertexConnected.le_degree {k : ℕ∞} (hG : G.IsVertexConnected k) (v : V)
    [Fintype (G.neighborSet v)] : k ≤ G.degree v := by
  have h := hG.le_encard_neighborSet v
  rwa [← coe_neighborFinset, Set.encard_coe_eq_coe_finsetCard,
    card_neighborFinset_eq_degree] at h
```

Taking the box product with K_2 raises the vertex connectivity by at least one: if G is connected and k-vertex-connected then $G \square K_2$ is $(k+1)$-vertex-connected. Since Q_{n+1} is $Q_n \square K_2$, induction on n then shows that the hypercube Q_n is n-vertex-connected. Exercises 14–16 guide you through the proof, and Exercise 17 uses the next section to deduce that Q_n is also n-edge-connected.

Vertex Connectivity Implies Edge Connectivity

Any k-vertex-connected graph is also k-edge-connected.

Proof sketch: given a set s of less than k edges and vertices u and v, let $s_0 \subseteq s$ be those that are edges of G, since deleting the other elements of s changes nothing. We then pick from each edge of s_0 other than $s(u, v)$ an endpoint that is neither u nor v, and let t be the set of chosen endpoints (`Finset.exists_cover_avoiding`). A walk that avoids a set of vertices meeting every edge of s_0 survives the deletion of s (`Walk.reachable_deleteEdges_of_avoids`). If $s(u, v) \notin s_0$ then t meets every edge of s_0, and a walk from u to v avoiding t exists by k-vertex-connectivity. Otherwise t has at most $k - 2$ vertices, so there is a vertex w outside $t \cup \{u, v\}$, and we walk from u to w avoiding $t \cup \{v\}$ and then from w to v avoiding $t \cup \{u\}$; both of these sets meet every edge of s_0, including $u - v$.

Our first lemma concerns a finite set s of pairs $s(a, b)$ with $a \neq b$. It gives a set t of vertices, avoiding u and v, that meets every element of $s.\mathrm{erase}\ s(u, v)$ and has at most as many vertices as this set has elements.

```
lemma _root_.Finset.exists_cover_avoiding [DecidableEq V] (u v : V)
    (s : Finset (Sym2 V)) (hs : ∀ e ∈ s, ¬e.IsDiag) :
    ∃ t : Finset V, #t ≤ #(s.erase s(u, v)) ∧ u ∉ t ∧ v ∉ t ∧
      ∀ e ∈ s, e ≠ s(u, v) → ∃ x ∈ e, x ∈ t := by
```

```
  -- We use choose! below which needs V to be nonempty.
  have : Nonempty V := ⟨u⟩
  have hpick : ∀ e ∈ s.erase s(u, v), ∃ x ∈ e, x ≠ u ∧ x ≠ v := by
    intro e he
    obtain ⟨hne, hes⟩ := Finset.mem_erase.mp he
    have := hs e hes
    induction e using Sym2.ind with
    | h a b =>
      simp only [Sym2.mk_isDiag_iff] at this
      by_cases a = u <;> by_cases a = v <;>
        grind [Sym2.mem_mk_left, Sym2.mem_mk_right]
  -- Plain choose would give g : (e : Sym2 V) → e ∈ s.erase s(u, v) → V, whose
  -- second argument is a proof of membership, so (s.erase s(u, v)).image g would
  -- not type-check.
  -- Using choose! removes this proof argument, giving g : Sym2 V → V
  -- and hg : ∀ e ∈ s.erase s(u, v), g e ∈ e ∧ g e ≠ u ∧ g e ≠ v. The values of g
  -- outside s.erase s(u, v) are arbitrary, so choose! needs Nonempty V.
  choose! g hg using hpick
  refine ⟨(s.erase s(u, v)).image g, Finset.card_image_le, by grind, by grind, ?_⟩
  intro e hes hne
  have he := Finset.mem_erase.mpr ⟨hne, hes⟩
  exact ⟨g e, (hg e he).1, Finset.mem_image_of_mem _ he⟩
```

A walk avoiding a set `t` of vertices that meets every edge of `G` in `s` is still a walk after deleting the edges `s`. To move the walk into the graph `G.deleteEdges s` we use Mathlib's `Walk.toDeleteEdges`, which does this for any walk that uses no edge of `s`.

```
#check Walk.toDeleteEdges
```

```
lemma Walk.reachable_deleteEdges_of_avoids {s : Set (Sym2 V)} {t : Finset V}
    {x y : V} (p : G.Walk x y) (ht : ∀ e ∈ s, e ∈ G.edgeSet → ∃ z ∈ e, z ∈ t)
    (hp : ∀ z ∈ p.support, z ∉ t) : (G.deleteEdges s).Reachable x y :=
  ⟨p.toDeleteEdges s fun e he hes => by
    obtain ⟨z, hze, hzt⟩ := ht e hes (p.edges_subset_edgeSet he)
    exact hp z (p.mem_support_of_mem_edges he hze) hzt⟩
```

A k-vertex-connected graph is k-edge-connected. We first reduce to the case where k is a natural number: a $\top$-vertex-connected graph is n-vertex-connected for every $n : \mathbb{N}$, and by Mathlib's `isEdgeConnected_top_iff_forall_nat` a graph that is n-edge-connected for every $n : \mathbb{N}$ is $\top$-edge-connected.

```
theorem IsVertexConnected.isEdgeConnected {k : ℕ∞} (hG : G.IsVertexConnected k) :
    G.IsEdgeConnected k := by
  classical
  suffices h : ∀ n : ℕ, G.IsVertexConnected n → G.IsEdgeConnected n by
    cases k with
    | top =>
      exact isEdgeConnected_top_iff_forall_nat.mpr fun n => h n (hG.anti le_top)
    | coe k => exact h k hG
  intro k hG u v s hs
  -- s₀: the finset of edges of G that lie in s, so #s₀ < k
  have hsfin : s.Finite :=
    Set.encard_lt_top_iff.mp (hs.trans (ENat.natCast_lt_top k))
  set s₀ := hsfin.toFinset.filter (· ∈ G.edgeSet)
```

```
  have hmem {e : Sym2 V} : e ∈ s₀ ↔ e ∈ s ∧ e ∈ G.edgeSet := by simp [s₀]
  have hs₀ : #s₀ < k := by
    rw [hsfin.encard_eq_coe_toFinset_card] at hs
    exact (Finset.card_filter_le _ _).trans_lt (by exact_mod_cast hs)
  obtain ⟨t, htcard, htu, htv, ht⟩ := s₀.exists_cover_avoiding u v
    fun e he => G.not_isDiag_of_mem_edgeSet (hmem.mp he).2
  -- a set containing t meets every edge of s₀, except possibly s(u, v)
  have hcover (A : Finset V) (hA : t ⊆ A) (huv : s(u, v) ∈ s₀ → u ∈ A ∨ v ∈ A) :
      ∀ e ∈ s, e ∈ G.edgeSet → ∃ x ∈ e, x ∈ A := by
    intro e hes heG
    by_cases he : e = s(u, v)
    · subst he
      rcases huv (hmem.mpr ⟨hes, heG⟩) with h | h
      exacts [⟨u, Sym2.mem_mk_left _ _, h⟩, ⟨v, Sym2.mem_mk_right _ _, h⟩]
    · obtain ⟨x, hx, hxt⟩ := ht e (hmem.mpr ⟨hes, heG⟩) he
      exact ⟨x, hx, hA hxt⟩
  by_cases huv : s(u, v) ∈ s₀
  · -- the edge u — v is deleted, so #t ≤ k - 2
    -- we need to find another vertex w outside t ∪ {u, v}
    have hne : u ≠ v := (G.mem_edgeSet.mp (hmem.mp huv).2).ne
    have htk : #t + 2 ≤ k := by grind
    have hins (a : V) : #(insert a t) ≤ #t + 1 := Finset.card_insert_le _ _
    obtain ⟨w, hw⟩ := hG.exists_notMem (insert u (insert v t))
      (Nat.cast_le.mpr (by grind))
    simp only [Finset.mem_insert, not_or] at hw
    obtain ⟨hwu, hwv, hwt⟩ := hw
    -- we can walk from u to w avoiding t ∪ {v}
    obtain ⟨p₁, hp₁⟩ := hG.exists_walk (insert v t)
      (Nat.cast_lt.mpr (by grind)) u w (by grind) (by grind)
    -- we can walk from w to v avoiding t ∪ {u}
    obtain ⟨p₂, hp₂⟩ := hG.exists_walk (insert u t)
      (Nat.cast_lt.mpr (by grind)) w v (by grind) (by grind)
    exact (p₁.reachable_deleteEdges_of_avoids (hcover _ (Finset.subset_insert _ _)
        fun _ => .inr (Finset.mem_insert_self _ _)) hp₁).trans
      (p₂.reachable_deleteEdges_of_avoids (hcover _ (Finset.subset_insert _ _)
        fun _ => .inl (Finset.mem_insert_self _ _)) hp₂)
  · -- otherwise t meets every edge of s₀
    have := Finset.card_erase_le (s := s₀) (a := s(u, v))
    obtain ⟨p, hp⟩ := hG.exists_walk t (Nat.cast_lt.mpr (by grind)) u v htu htv
    exact p.reachable_deleteEdges_of_avoids (hcover t subset_rfl (absurd · huv)) hp
```

Connectivity Numbers*

The *edge connectivity* $\lambda(G)$ of a graph is the largest k for which G is k-edge-connected, and its *vertex connectivity* $\kappa(G)$ is the largest k for which G is k-vertex-connected. Neither is currently in Mathlib. We define both as suprema over $k : \mathbb{N}\infty$, which allows the value $\top$ when a graph is k-edge-connected (or k-vertex-connected) for every k.

The edge connectivity of G: the supremum of the k for which G is k-edge-connected.

```
noncomputable def edgeConnectivity (G : SimpleGraph V) : ℕ∞ :=
  ⨆ (k : ℕ∞) (_ : G.IsEdgeConnected k), k
```

The vertex connectivity of G: the supremum of the k for which G is k-vertex-connected.

```
noncomputable def vertexConnectivity (G : SimpleGraph V) : ℕ∞ :=
  ⨆ (k : ℕ∞) (_ : G.IsVertexConnected k), k
```

We introduce the standard notation $\lambda(G)$ for the edge connectivity and $\kappa(G)$ for the vertex connectivity.

```
@[inherit_doc edgeConnectivity] scoped notation "λ(" G ")" =>
  SimpleGraph.edgeConnectivity G
@[inherit_doc vertexConnectivity] scoped notation "κ(" G ")" =>
  SimpleGraph.vertexConnectivity G
```

Both properties are closed under decreasing k (`IsVertexConnected.anti`, and `IsEdgeConnected.anti` below), so the k for which G is k-edge-connected form an initial segment of $\mathbb{N}\infty$. A graph that is n-edge-connected for every $n : \mathbb{N}$ is $\top$-edge-connected, so this initial segment contains its supremum: $k \le \lambda(G)$ holds exactly when G is k-edge-connected. The same holds for $\kappa(G)$ if V is nonempty (the graph with no vertices is not even 0-vertex-connected, yet $0 \le \kappa(G)$).

We need a general fact about an `Antitone` property P of $\mathbb{N}\infty$ that is closed under decreasing k, which holds at 0, and holds at $\top$ whenever it holds at every natural number. This has nothing to do with graphs, so we put it in the `ENat` namespace. Mathlib does not currently seem to have it.

```
lemma _root_.ENat.le_biSup_iff {P : ℕ∞ → Prop} (hP : Antitone P) (h0 : P 0)
    (htop : (∀ n : ℕ, P n) → P ⊤) {k : ℕ∞} :
    k ≤ ⨆ (j : ℕ∞) (_ : P j), j ↔ P k := by
  refine ⟨fun hk => ?_, fun h => le_iSup₂ (f := fun j _ => j) k h⟩
  -- it is enough to show P n for every natural number n ≤ k
  suffices h : ∀ n : ℕ, (n : ℕ∞) ≤ ⨆ (j : ℕ∞) (_ : P j), j → P n by
    cases k with
    | top => exact htop fun n => h n (le_top.trans hk)
    | coe k => exact h k hk
  intro n hn
  cases n with
  | zero => exact h0
  | succ n =>
    -- some j with P j exceeds n, so n + 1 ≤ j and P (n + 1) holds
    rw [Nat.cast_add_one, ENat.add_one_le_iff (ENat.natCast_ne_top n)] at hn
    obtain ⟨j, hj, hnj⟩ : ∃ j, P j ∧ (n : ℕ∞) < j := by simpa [lt_iSup_iff] using hn
    rw [Nat.cast_add_one]
    exact hP (Order.add_one_le_of_lt hnj) hj


lemma IsEdgeConnected.anti {k l : ℕ∞} (hkl : k ≤ l) (hG : G.IsEdgeConnected l) :
    G.IsEdgeConnected k :=
  fun u v => (hG u v).anti hkl
```

Every nonempty graph is 0-vertex-connected, but the graph with no vertices is not. As for Mathlib's `isEdgeConnected_top_iff_forall_nat`, a graph is $\top$-vertex-connected iff it is n-vertex-connected for every $n : \mathbb{N}$: it then has more than n vertices for every n, and a finite set `S` has less than `S.ncard + 1` elements.

```
lemma isVertexConnected_zero [Nonempty V] : G.IsVertexConnected 0 :=
  ⟨by simp [Order.one_le_iff_pos], fun S hS => absurd hS (by simp)⟩
```

```
lemma isVertexConnected_top_iff_forall_nat :
    G.IsVertexConnected ⊤ ↔ ∀ n : ℕ, G.IsVertexConnected n := by
  refine ⟨fun h n => h.anti le_top, fun h => ⟨?_, fun S hS => ?_⟩⟩
  · rw [ENat.eq_top_iff_forall_ge.mpr fun n => le_self_add.trans (h n).1]
    exact le_top
  · exact (h (S.ncard + 1)).2 S
      (by simp [← (Set.encard_lt_top_iff.mp hS).cast_ncard_eq, -Nat.cast_add])

lemma le_edgeConnectivity_iff {k : ℕ∞} : k ≤ λ(G) ↔ G.IsEdgeConnected k :=
  ENat.le_biSup_iff (P := G.IsEdgeConnected) (fun _ _ hjl h => h.anti hjl) .zero
    isEdgeConnected_top_iff_forall_nat.mpr

lemma le_vertexConnectivity_iff [Nonempty V] {k : ℕ∞} :
    k ≤ κ(G) ↔ G.IsVertexConnected k :=
  ENat.le_biSup_iff (P := G.IsVertexConnected) (fun _ _ hjl h => h.anti hjl)
    isVertexConnected_zero isVertexConnected_top_iff_forall_nat.mpr
```

In particular both suprema are attained: G is $\lambda(G)$-edge-connected, and (if V is nonempty) $\kappa(G)$-vertex-connected.

```
lemma isEdgeConnected_edgeConnectivity : G.IsEdgeConnected λ(G) :=
  le_edgeConnectivity_iff.mp le_rfl

lemma isVertexConnected_vertexConnectivity [Nonempty V] :
    G.IsVertexConnected κ(G) :=
  le_vertexConnectivity_iff.mp le_rfl
```

Since k-vertex-connectivity implies k-edge-connectivity, $\kappa(G) \le \lambda(G)$.

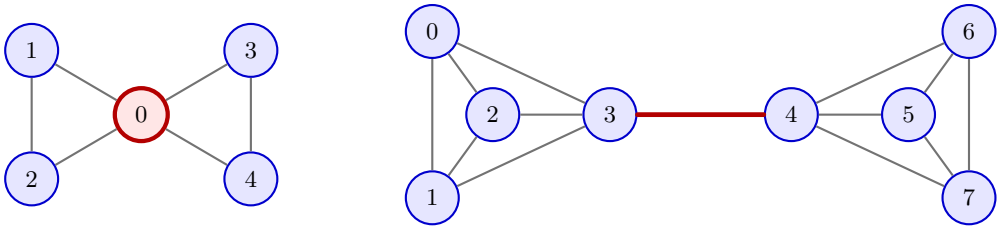
```
theorem vertexConnectivity_le_edgeConnectivity : κ(G) ≤ λ(G) :=
  iSup₂_le fun _ hk => le_edgeConnectivity_iff.mpr hk.isEdgeConnected
```

Both numbers are bounded by the degree of any vertex. For $\kappa(G)$ this is `IsVertexConnected.le_encard_neighborSet`; for $\lambda(G)$ we use Mathlib's `IsEdgeConnected.le_degree`, which needs G to have at least two vertices.

```
lemma vertexConnectivity_le_encard_neighborSet (v : V) :
    κ(G) ≤ (G.neighborSet v).encard :=
  iSup₂_le fun _ hk => hk.le_encard_neighborSet v

lemma edgeConnectivity_le_degree [Nontrivial V] (v : V) [Fintype (G.neighborSet v)] :
    λ(G) ≤ G.degree v :=
  iSup₂_le fun _ hk => hk.le_degree (u := v)
```

Together these give *Whitney's inequalities*: a finite graph G with at least two vertices satisfies $\kappa(G) \le \lambda(G) \le \delta(G)$. The second inequality is `edgeConnectivity_le_degree` at a vertex of minimum degree (Exercise 18). Both inequalities can be strict (Exercises 19 and 20). In the *bowtie*, two triangles sharing a vertex, deleting the shared vertex disconnects the graph, so $\kappa = 1$, while $\lambda = \delta = 2$. Two copies of `K[4]` joined by a single edge (the *barbell*) have $\lambda = 1$, since that edge is a bridge, while $\delta = 3$.



Left: the bowtie, with `κ = 1 < λ = 2`. Right: the barbell, with `λ = 1 < δ = 3`

The two definitions disagree on graphs with at most one vertex. Mathlib's `IsEdgeConnected k` has no condition on the number of vertices, so such a graph is `k`-edge-connected for every `k`, whereas our `IsVertexConnected k` asks for more than `k` vertices.

```
lemma edgeConnectivity_of_subsingleton [Subsingleton V] : λ(G) = ⊤ :=
  top_le_iff.mp (le_edgeConnectivity_iff.mpr .of_subsingleton)

lemma vertexConnectivity_of_subsingleton [Subsingleton V] : κ(G) = 0 := by
  refine (iSup₂_le fun k hk => ?_).antisymm (by simp)
  -- k + 1 ≤ ENat.card V ≤ 1, so k < 1
  have := (ENat.add_one_le_iff' ENat.one_ne_top).mp (hk.1.trans ENat.card_le_one)
  exact (Order.lt_one_iff.mp this).le
```

10.5 Brooks' Theorem*

The greedy colouring bound `colorable_of_degree_le` says that a graph of maximum degree at most `Δ` is `(Δ + 1)`-colourable. *Brooks' theorem* [Brooks 1941] tells us exactly when this can be improved to `Δ`. It is usually stated for connected graphs: a finite connected graph of maximum degree `Δ` is `Δ`-colourable unless it is complete or (when `Δ = 2`) an odd cycle. We prove a form with no connectivity hypothesis that also covers infinite graphs: a graph with all degrees at most `Δ` is `Δ`-colourable unless it contains a `(Δ + 1)`-clique or (when `Δ = 2`) one of its components is an odd cycle (`brooks`). On finite graphs the two forms agree: when all degrees are at most `Δ`, each vertex of a `(Δ + 1)`-clique already has all `Δ` of its neighbours in the clique, so the clique is a whole component. Odd cycles are only an obstruction when `Δ = 2`: for example the Petersen graph is 3-regular and contains 5-cycles, but it is 3-colourable.

The cases `Δ = 2` and `Δ ≥ 3` are very different. The case `Δ = 2` follows quickly from our characterization of bipartite graphs, `isBipartite_iff_forall_isCycle_even`, while the case `Δ ≥ 3` needs a real argument, for which we use the partial colourings `ColoringOn`. At the end we combine the cases into a single statement for every `Δ`.

Maximum Degree Two

The case `Δ = 2` combines two earlier results. A graph is 2-colourable iff it has no odd cycle (`isBipartite_iff_forall_isCycle_even`), and in a graph of maximum degree at most 2 every cycle is a whole connected component (`Walk.IsCycle.toSubgraph_eq_of_degree_le_two`). So a graph of maximum degree at most 2 is 2-colourable unless one of its connected components is an odd cycle. Indeed, if it is not 2-colourable then it has an odd cycle `c`, based at `u` say, and by `Walk.IsCycle.toSubgraph_eq_of_degree_le_two` that cycle *is* its component: `c.toSubgraph = G⟦u⟧.toSubgraph`.

Brooks' theorem (Δ = 2): a graph with all degrees at most 2 is 2-colourable unless it has a component that is an odd cycle.

```
theorem brooks_two [G.LocallyFinite] (hdeg : ∀ v, G.degree v ≤ 2)
    (hcyc : ¬∃ (u : V) (c : G.Walk u u),
      c.IsCycle ∧ Odd c.length ∧ c.toSubgraph = G⟦u⟧.toSubgraph) :
    G.Colorable 2 := by
  apply isBipartite_iff_forall_isCycle_even.mpr
  intro u c hc
  by_contra hodd
  exact hcyc ⟨u, c, hc, Nat.not_even_iff_odd.mp hodd,
    hc.toSubgraph_eq_of_degree_le_two hdeg⟩
```

Maximum Degree at Least Three

From now on we assume that `Δ ≥ 3`. The classical greedy colouring proof due to Lovász [Lovász 1975] uses the *blocks* of a graph, its maximal connected subgraphs without a cut vertex, which we study in the next section, Cut Vertices and Blocks (p. 317). Instead, the proof we give is a variation of Lovász's approach that avoids blocks: it uses partial colourings and the idea behind greedy colouring, but replaces the need for 2-connected blocks with strong induction on vertex subsets and a gluing step across a pair of non-adjacent vertices. It is inspired by a short proof of Zając [Zając 2018], which also avoids any appeal to connectivity.

The basic idea is to colour vertices greedily but in an order chosen so that we never reach an uncoloured vertex with `Δ` distinctly coloured neighbours, hence the greedy colouring will never use more than `Δ` colours.

Proof sketch: we prove the stronger statement that *every* finset of vertices `s` satisfies `G.ColorableOn Δ ↑s` (`colorableOn_of_cliqueFree_of_degree_le`), by strong induction on `s` under `⊂`, so our induction hypothesis is: `ih : ∀ t ⊂ s, G.ColorableOn Δ ↑t`.

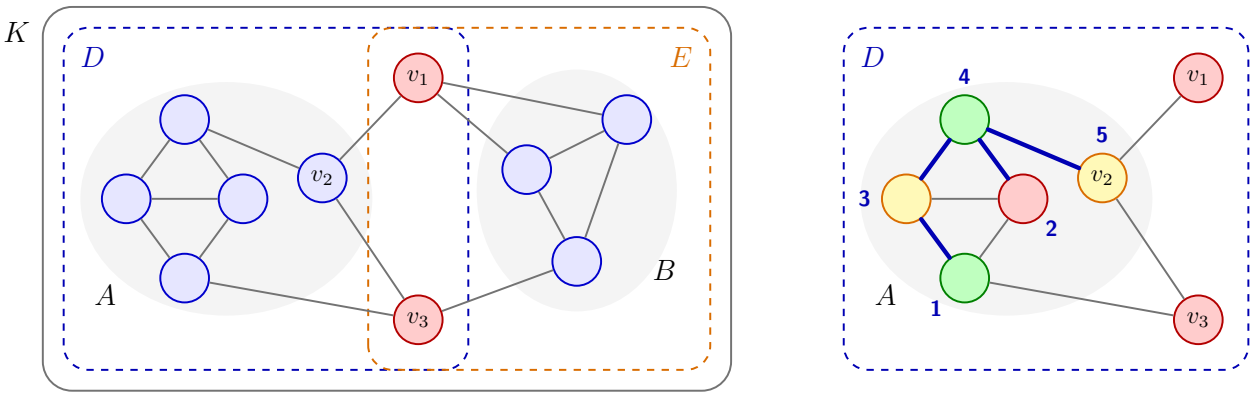
Degeneracy reduction: if some `a ∈ s` has less than `Δ` neighbours in `s`, then apply `ih` to `s.erase a` and extend the resulting colouring to `a` using a free colour (the neighbours of `a` use less than `Δ` colours; `ColoringOn.exists_free_color_of_degIn`).

So wlog every vertex of `s` has at least `Δ` neighbours in `s`. Since all degrees are at most `Δ`, every vertex of `s` must have *exactly* `Δ` neighbours, all of which lie in `s` (so `s` is *closed under adjacency*: every neighbour of a vertex of `s` lies in `s`).

Regular case (`regular_case`, `closed_regular_case`): write `K := s`, which is closed under adjacency and `Δ`-regular, and pick any $v_2 \in K$.

Since `G` has no `(Δ + 1)`-clique, the neighbourhood of v_2 is not a clique, so v_2 has two neighbours v_1 and v_3 that are not adjacent to each other (`CliqueFree.exists_ne_adj_not_adj_of_le_degree`). Write `W := K \ {v₁, v₃}` for the remaining vertices of `K`. Let `A` be the vertex set of the connected component of v_2 in the subgraph induced by `W` (`walkComponent`): the vertices of `W` that can be reached from v_2 by a walk inside `W`. Let `B := W \ A` be the rest, so there are no edges between `A` and `B` (`adj_mem_walkComponent`).

Now `D := A ∪ {v₁, v₃}` and `E := B ∪ {v₁, v₃}` together cover `K` and meet only in the non-adjacent pair `{v₁, v₃}`; moreover `A`, `B` and `E` are proper subsets of `K` (the first misses v_1, the others v_2), so `ih` applies to them. The plan is to colour `D` and `E` separately and then glue them together using our `ColoringOn.union` construction.



Left: the sets A, B, D and E when Δ = 3. Right: Step 1 on D, with the colouring order numbered and the search from v_2 in blue

1. *D has a colouring in which v_1 and v_3 have the same colour.* (exists_coloring_eq)Colour v_1 and v_3 first, both with colour 0 (legal, since they are not adjacent). Then colour the vertices of A other than v_2 greedily, in *reverse search order from v_2*. That is, start with S = {v_2} and repeatedly add to S a vertex of A outside S that has a neighbour in S; we say such a vertex has been *settled*. Since A is connected (connected_induce_walkComponent), every vertex of A is eventually settled (exists_adj_sdiff_of_connected). Now colour the vertices in the reverse of the order in which they were settled (exists_coloringOn_sdiff). Each vertex was settled because of an edge to an already-settled neighbour, which is either v_2 or a vertex coloured later; so when a vertex is coloured it still has an uncoloured neighbour, hence at most Δ - 1 coloured neighbours, and a free colour exists.Finally colour v_2: among its Δ neighbours are v_1 and v_3, which received the *same* colour, so its neighbours use at most Δ - 1 colours and a free colour remains (exists_coloringOn_insert_of_eq).

2. *Gluing.* Since the only edges from D to E \ D ⊆ B leave from {v_1, v_3} (there are no A–B edges), two colourings of D and E can be glued together as long as they agree on {v_1, v_3}. Moreover they can be made to agree by permuting the colours of one of them (Equiv.exists_perm_of_eq_iff_eq), provided that they agree on *whether* v_1 and v_3 receive equal colours. So it suffices (ColoringOn.nonempty_union_of_eq_iff_eq) to find colourings of D and E with the same *pattern* (equal or distinct) on {v_1, v_3} (exists_colorings_pattern_iff).By Step 1, D has a colouring with v_1 and v_3 *equal*. Also, for v ∈ {v_1, v_3} the Δ neighbours of v split between A and B, with at least one (namely v_2) in A.*Case (a).* If v_1 and v_3 have a total of at most Δ - 1 neighbours in B (as happens when B is empty), colour B by ih and extend to E giving v_1, v_3 a common free colour (exists_two_eq); pair this with the equal-pattern colouring of D.*Case (b).* Otherwise v_1 and v_3 have a total of at least Δ neighbours in B, and hence a total of at most Δ in A. Swapping them if necessary, we may assume that v_1 has at most Δ - 2 neighbours in A and v_3 at most Δ - 1. Indeed, both have at least one neighbour in A (namely v_2), so each has at most Δ - 1 there; and they cannot both have Δ - 1, as 2 * (Δ - 1) > Δ when Δ ≥ 3, so say v_1 has at most Δ - 2. Now a colouring of A by ih extends to D giving v_1, v_3 *distinct* colours (exists_two_ne): first v_3 takes a colour not used on its neighbours in A, then v_1 avoids that colour as well as the at most Δ - 2 colours on its own neighbours in A. So D has colourings of both patterns, and we pair the ih-colouring of E with whichever one matches it.

Free Colours and Neighbours in a Set

We start with a simple observation (an instance of Mathlib's `Finset.exists_mem_notMem_of_card_lt_card`): less than Δ colours cannot exhaust the palette `Fin Δ`.

```
section brooks
open Finset ColoringOn
variable {Δ : ℕ}

private lemma exists_color_avoiding (s : Finset (Fin Δ)) (h : #s < Δ) :
    ∃ c, c ∉ s := by
  simpa using exists_mem_notMem_of_card_lt_card (t := univ) (by simpa using h)
```

If the colours that C assigns to the neighbours of a lying in s number less than Δ, then some colour `c : Fin Δ` is avoided by every neighbour of a in s.

```
lemma ColoringOn.exists_free_color [DecidableRel G.Adj] {s : Finset V}
    (C : G.ColoringOn (Fin Δ) ↑s) (a : V)
    (h : #({v ∈ s | G.Adj a v}.image fun v => C v) < Δ) :
    ∃ c : Fin Δ, ∀ v ∈ s, G.Adj a v → C v ≠ c := by
  obtain ⟨c, hc⟩ := exists_color_avoiding _ h
  exact ⟨c, fun v hv hadj heq =>
    hc (mem_image.mpr ⟨v, mem_filter.mpr ⟨hv, hadj⟩, heq⟩)⟩
```

We count the neighbours of a lying in a finite set s with `G.degIn s a` (see `degIn`). Each of them gets one colour, so C uses at most `G.degIn s a` colours on them (`card_image_le`).

```
lemma ColoringOn.card_image_le_degIn {γ : Type*} [DecidableRel G.Adj] [DecidableEq γ]
    {s : Finset V} (C : G.ColoringOn γ ↑s) (a : V) :
    #({v ∈ s | G.Adj a v}.image fun v => C v) ≤ G.degIn s a :=
  card_image_le
```

Combining this with `exists_free_color`: if a has less than Δ neighbours in s, then the number of colours used on those neighbours is less than Δ, so some colour is free for a.

```
lemma ColoringOn.exists_free_color_of_degIn [DecidableRel G.Adj]
    {s : Finset V} (C : G.ColoringOn (Fin Δ) ↑s) (a : V) (h : G.degIn s a < Δ) :
    ∃ c : Fin Δ, ∀ v ∈ s, G.Adj a v → C v ≠ c :=
  C.exists_free_color a ◁ (C.card_image_le_degIn a).trans_lt h
```

In a $(\Delta + 1)$-clique-free graph, a vertex a of degree Δ has two distinct non-adjacent neighbours (its neighbourhood is not a clique). (Notice that the hypotheses preclude the cases $\Delta = 0$ and $\Delta = 1$ so we don't need to assume $\Delta \geq 2$ explicitly.)

```
lemma CliqueFree.exists_ne_adj_not_adj_of_le_degree [G.LocallyFinite]
    (hcf : G.CliqueFree (Δ + 1)) {a : V} (hd : Δ ≤ G.degree a) :
    ∃ v₁ v₃, v₁ ≠ v₃ ∧ G.Adj a v₁ ∧ G.Adj a v₃ ∧ ¬G.Adj v₁ v₃ := by
  classical
  by_contra! h
  refine hcf.mono (by grind : Δ + 1 ≤ G.degree a + 1)
    (insert a (G.neighborFinset a)) ⟨fun x hx y hy hxy => ?_, ?_⟩
  · grind [adj_symm, mem_neighborFinset]
  · rw [card_insert_of_notMem (by simp), card_neighborFinset_eq_degree]
```

Mathlib lacks the converse of the following lemma, so we prove it.

```
#check exists_ne_map_eq_of_card_image_lt
```

If f identifies two distinct elements of s, its image is strictly smaller than s.

```
lemma _root_.Finset.card_image_lt_card_of_ne_map_eq {W : Type*} [DecidableEq W]
    {s : Finset V} (f : V → W) {a b : V} (ha : a ∈ s) (hb : b ∈ s) (hab : a ≠ b)
    (hfab : f a = f b) : #(s.image f) < #s :=
  (card_image_le).lt_of_ne fun h =>
    hab (card_image_iff.mp h (mem_coe.mpr ha) (mem_coe.mpr hb) hfab)
```

Step 1: Colouring in Reverse Search Order

We express "A is connected" as connectedness of the subgraph `(⊤ : G.Subgraph).induce A` (`Subgraph.induce`). Mathlib characterizes this by walks in G itself (`Subgraph.connected_iff_forall_exists_walk_subgraph`): any two vertices are joined by a walk p with `p.toSubgraph ≤ (⊤ : G.Subgraph).induce A`. As we saw in Vertex Connectivity (p. 297), for an induced subgraph this just says that every vertex of p lies in A (`Walk.toSubgraph_le_induce_top_iff`).

If the subgraph induced by W is connected, and $S \subset W$ is a set of settled vertices containing v_2, then some unsettled vertex has a settled neighbour: a walk inside W from v_2 to an unsettled vertex must at some point leave S.

```
private lemma exists_adj_sdiff_of_connected [DecidableEq V] {W S : Finset V}
    {v₂ : V} (hconn : ((⊤ : G.Subgraph).induce (W : Set V)).Connected)
    (hv : v₂ ∈ S) (hSW : S ⊆ W) (hne : (W \ S).Nonempty) :
    ∃ z ∈ W \ S, ∃ y ∈ S, G.Adj z y := by
  obtain ⟨r, hr⟩ := hne
  obtain ⟨p, hp⟩ :=
    ((Subgraph.connected_iff_forall_exists_walk_subgraph _).mp hconn).2
      (mem_coe.mpr (hSW hv)) (mem_coe.mpr (mem_sdiff.mp hr).1)
  have hp := p.toSubgraph_le_induce_top_iff.mp hp
  obtain ⟨d, hd, hd1, hd2⟩ := p.exists_boundary_dart ↑S (mem_coe.mpr hv) (by grind)
  exact ⟨d.toProd.2, mem_sdiff.mpr
    ⟨hp _ (p.dart_snd_mem_support_of_mem_darts hd), by grind⟩, d.toProd.1,
      by simpa using hd1, d.adj.symm⟩
```

The reverse-search colouring of Step 1, apart from the final step of colouring v_2. For any settled set S with $v_2 \in S \subseteq W$ we colour $\{v_1, v_3\} \cup (W \setminus S)$, giving v_1 and v_3 the same colour, by induction on the number of unsettled vertices.

```
private lemma exists_coloringOn_sdiff [G.LocallyFinite] [DecidableEq V]
    (W : Finset V) (v₂ : V) {v₁ v₃ : V} (hΔ : 0 < Δ)
    (hnadj : ¬G.Adj v₁ v₃) (hdeg : ∀ x ∈ W, G.degree x = Δ)
    (hW : Disjoint W {v₁, v₃})
    (hconn : ((⊤ : G.Subgraph).induce W).Connected) :
    ∀ (S : Finset V), v₂ ∈ S → S ⊆ W →
      ∃ C : G.ColoringOn (Fin Δ) ↑({v₁, v₃} ∪ (W \ S)), C v₁ = C v₃ := by
  classical
  suffices H : ∀ n (S : Finset V), v₂ ∈ S → S ⊆ W → #(W \ S) = n →
      ∃ C : G.ColoringOn (Fin Δ) ↑({v₁, v₃} ∪ (W \ S)), C v₁ = C v₃ by
    intro S hv hS; exact H _ S hv hS rfl
  intro n
  induction n with
  | zero =>
```

```
    -- every vertex of W is settled: colour just v₁ and v₃, both with
    -- colour 0
    intro S _ _ hcard
    have hWS : W \ S = ∅ := card_eq_zero.mp hcard
    exact ⟨(ColoringOn.ofNotAdj hnadj ⟨0, hΔ⟩).copy (by simp [hWS]), rfl⟩
  | succ n ih =>
    intro S hv hS hcard
    have hWS : (W \ S).Nonempty := card_pos.mp (by grind)
    -- settle an unsettled vertex z with a settled neighbour y; colour the rest
    -- by induction, then give z a free colour
    obtain ⟨z, hzWS, y, hyS, hadj⟩ := exists_adj_sdiff_of_connected hconn hv hS hWS
    obtain ⟨hzW, hzS⟩ := mem_sdiff.mp hzWS
    obtain ⟨C, hC⟩ := ih (insert z S) (mem_insert_of_mem hv) (insert_subset hzW hS)
      (by rw [sdiff_insert, card_erase_of_mem hzWS, hcard]; rfl)
    -- the settled neighbour y of z is still uncoloured
    have hy : y ∉ {v₁, v₃} ∪ (W \ insert z S) := by
      have := Finset.disjoint_left.mp hW (hS hyS)
      grind
    have hlt : G.degIn ({v₁, v₃} ∪ (W \ insert z S)) z < Δ := by
      have h := degIn_lt_degree_of_adj_notMem _ hadj hy
      rwa [hdeg z hzW] at h
    obtain ⟨c, hc⟩ := C.exists_free_color_of_degIn z hlt
    have hz : z ∉ ({v₁, v₃} : Finset V) := Finset.disjoint_left.mp hW hzW
    refine ⟨(C.insert z fun v hv hadj => hc v hv hadj).copy (by grind), ?_⟩
    grind [ColoringOn.copy_apply, ColoringOn.insert_apply]
```

The end of Step 1 of the sketch: all Δ neighbours of v_2 lie in `K.erase v₂` and are already coloured, but two of them, v_1 and v_3, have the *same* colour. So the neighbours of v_2 use at most `Δ - 1` colours and v_2 can be coloured, giving a colouring of K in which v_1 and v_3 still share a colour.

```
private lemma exists_coloringOn_insert_of_eq [DecidableEq V] [G.LocallyFinite]
    {K : Finset V} {v₁ v₂ v₃ : V} (hv₂K : v₂ ∈ K)
    (C : G.ColoringOn (Fin Δ) ↑(K.erase v₂))
    (hne13 : v₁ ≠ v₃) (hdv₂ : G.degree v₂ = Δ)
    (ha1 : G.Adj v₂ v₁) (ha3 : G.Adj v₂ v₃)
    (hnbhd : G.neighborFinset v₂ ⊆ K.erase v₂) (hcol1 : C v₁ = C v₃) :
    ∃ C' : G.ColoringOn (Fin Δ) ↑K, C' v₁ = C' v₃ := by
  classical
  have hnb1 : v₁ ∈ G.neighborFinset v₂ := (mem_neighborFinset _ _ _).mpr ha1
  have hnb3 : v₃ ∈ G.neighborFinset v₂ := (mem_neighborFinset _ _ _).mpr ha3
  have hfilter : {v ∈ K.erase v₂ | G.Adj v₂ v} = G.neighborFinset v₂ := by
    rw [filter_adj_eq_inter]; exact inter_eq_left.mpr hnbhd
  have hcard : #({v ∈ K.erase v₂ | G.Adj v₂ v}.image fun v => C v) < Δ := by
    rw [hfilter]
    refine (card_image_lt_card_of_ne_map_eq _ hnb1 hnb3 hne13 hcol1).trans_le ?_
    rw [card_neighborFinset_eq_degree, hdv₂]
  obtain ⟨c, hc⟩ := C.exists_free_color v₂ hcard
  refine ⟨(C.insert v₂ (fun v hv hadj => hc v hv hadj)).copy (by simp_all), ?_⟩
  simp_all [ha1.ne.symm, ha3.ne.symm]
```

Step 1 of the sketch: K has a Δ-colouring giving v_1 and v_3 the *same* colour.

```
private lemma exists_coloring_eq [DecidableEq V] [G.LocallyFinite] (K : Finset V)
    {v₁ v₂ v₃ : V} (hΔ : 0 < Δ)
```

```
    (hv₂K : v₂ ∈ K) (hv₁K : v₁ ∈ K) (hv₃K : v₃ ∈ K)
    (hne13 : v₁ ≠ v₃) (hnadj : ¬G.Adj v₁ v₃) (hdv₂ : G.degree v₂ = Δ)
    (ha1 : G.Adj v₂ v₁) (ha3 : G.Adj v₂ v₃)
    (hdeg : ∀ x ∈ K \ {v₁, v₃}, G.degree x = Δ)
    (hnbhd : G.neighborFinset v₂ ⊆ K.erase v₂)
    (hconn : ((⊤ : G.Subgraph).induce (↑(K \ {v₁, v₃}) : Set V)).Connected) :
    ∃ C : G.ColoringOn (Fin Δ) ↑K, C v₁ = C v₃ := by
  classical
  have hv₂W : v₂ ∈ K \ {v₁, v₃} := mem_sdiff.mpr ⟨hv₂K, by grind⟩
  obtain ⟨C, hC⟩ := exists_coloringOn_sdiff (K \ {v₁, v₃}) v₂ hΔ hnadj hdeg
    sdiff_disjoint hconn {v₂} (mem_singleton_self v₂) (singleton_subset_iff.mpr hv₂W)
  exact exists_coloringOn_insert_of_eq hv₂K (C.copy (by rw [coe_inj]; grind))
    hne13 hdv₂ ha1 ha3 hnbhd hC
```

Step 2: Gluing Along the Pair

Case (a) of Step 2. Extend a colouring of B by two new non-adjacent vertices x, y giving them the *same* colour, when their numbers of neighbours in B sum to less than Δ, so that together they see less than Δ colours.

```
private lemma exists_two_eq [DecidableRel G.Adj] [DecidableEq V] (B : Finset V)
    (C : G.ColoringOn (Fin Δ) ↑B) {x y : V} (hnxy : ¬G.Adj x y)
    (h : G.degIn B x + G.degIn B y < Δ) :
    ∃ C' : G.ColoringOn (Fin Δ) ↑(insert x (insert y B)), C' x = C' y := by
  classical
  have hcard : #({v ∈ B | G.Adj x v}.image (fun v => C v) ∪
      {v ∈ B | G.Adj y v}.image (fun v => C v)) < Δ := by
    have hx := C.card_image_le_degIn x
    have hy := C.card_image_le_degIn y
    grind
  obtain ⟨c, hc⟩ := exists_color_avoiding _ hcard
  have hcy : ∀ v ∈ B, G.Adj y v → C v ≠ c := fun v hv hadj heq => by grind
  let Cy : G.ColoringOn (Fin Δ) ↑(insert y B) :=
    (C.insert y fun v hv hadj => hcy v hv hadj).copy (by simp)
  have hCy_y : Cy y = c := by simp [Cy]
  have hCyB : ∀ v ∈ B, v ≠ y → Cy v = C v := fun v _ hv => by simp [Cy, hv]
  have hcx : ∀ v ∈ insert y B, G.Adj x v → Cy v ≠ c := by grind
  exact ⟨(Cy.insert x fun v hv hadj => hcx v hv hadj).copy (by simp), by simp [Cy]⟩
```

Case (b) of Step 2. Extend a colouring of A by two new non-adjacent vertices x, y so that they receive *different* colours: y has less than Δ neighbours in A, so it gets a free colour, and x has less than $\Delta - 1$, so a free colour distinct from that of y remains.

```
private lemma exists_two_ne [DecidableRel G.Adj] [DecidableEq V] (A : Finset V)
    (C : G.ColoringOn (Fin Δ) ↑A) {x y : V} (hxy : x ≠ y) (hnxy : ¬G.Adj x y)
    (hy : G.degIn A y < Δ) (hx : G.degIn A x + 1 < Δ) :
    ∃ C' : G.ColoringOn (Fin Δ) ↑(insert x (insert y A)), C' x ≠ C' y := by
  classical
  obtain ⟨cy, hcy⟩ := C.exists_free_color_of_degIn y hy
  let Cy : G.ColoringOn (Fin Δ) ↑(insert y A) :=
    (C.insert y fun v hv hadj => hcy v hv hadj).copy (by simp)
  have hCy_y : Cy y = cy := by simp [Cy]
  have hCyA : ∀ v ∈ A, v ≠ y → Cy v = C v := fun v _ hv => by simp [Cy, hv]
```

```
  -- x is not adjacent to y, so it sees the same colours on insert y A as
  -- on A
  have hximg : {v ∈ insert y A | G.Adj x v}.image (fun v => Cy v) =
      {v ∈ A | G.Adj x v}.image (fun v => C v) := by grind
  let F := ({v ∈ insert y A | G.Adj x v}.image fun v => Cy v) ∪ {cy}
  have hFcard : #F < Δ := by grind [C.card_image_le_degIn x]
  obtain ⟨cx, hcx⟩ := exists_color_avoiding F hFcard
  have hcx_nb : ∀ v ∈ insert y A, G.Adj x v → Cy v ≠ cx := fun v hv hadj heq =>
    hcx (mem_union_left _
      (mem_image.mpr ⟨v, mem_filter.mpr ⟨hv, hadj⟩, heq⟩))
  have hcx_cy : cx ≠ cy := fun heq =>
    hcx (mem_union_right _ (heq ▸ mem_singleton_self _))
  exact ⟨(Cy.insert x fun v hv hadj => hcx_nb v hv hadj).copy (by simp),
    by simp_all [Ne.symm]⟩
```

The sketch uses the set `A`: the vertices of `W := K \ {v₁, v₃}` reachable from `v₂` by a walk inside `W`, i.e. the vertex set of the component of `v₂` in the subgraph induced by `W`. We define this, as a `Finset`, for any finite set `W` and record the facts we need: it contains `v₂` (when `v₂ ∈ W`; `self_mem_walkComponent`), it is closed under adjacency into `W` (`adj_mem_walkComponent`), and the subgraph it induces is connected (`connected_induce_walkComponent`).

The set of vertices of `W` reachable from `v₂` by a walk lying inside `W`.

```
noncomputable def walkComponent (G : SimpleGraph V) (v₂ : V) (W : Finset V) :
    Finset V :=
  by classical exact {a ∈ W | ∃ p : G.Walk v₂ a, ∀ y ∈ p.support, y ∈ W}

lemma mem_walkComponent {W : Finset V} {v₂ a : V} :
    a ∈ G.walkComponent v₂ W ↔
      a ∈ W ∧ ∃ p : G.Walk v₂ a, ∀ y ∈ p.support, y ∈ W := by
  simp [walkComponent]

lemma walkComponent_subset {W : Finset V} {v₂ : V} : G.walkComponent v₂ W ⊆ W := by
  simp [walkComponent]

lemma self_mem_walkComponent {W : Finset V} {v₂ : V} (h : v₂ ∈ W) :
    v₂ ∈ G.walkComponent v₂ W :=
  mem_walkComponent.mpr ⟨h, Walk.nil, by simpa using h⟩
```

Extending a walk inside `W` from `v₂` to `a` by an edge `a – b` with `b ∈ W` shows that `walkComponent` is closed under adjacency into `W`.

```
lemma adj_mem_walkComponent {W : Finset V} {v₂ a b : V}
    (ha : a ∈ G.walkComponent v₂ W) (hb : b ∈ W) (hab : G.Adj a b) :
    b ∈ G.walkComponent v₂ W := by
  obtain ⟨_, p, hp⟩ := mem_walkComponent.mp ha
  refine mem_walkComponent.mpr ⟨hb, p.concat hab, fun y hy => ?_⟩
  grind [Walk.support_concat, List.mem_singleton]
```

Truncating a walk inside `W` from `v₂` to `a` at its first visit to a vertex `y` on it shows that `y` lies in `walkComponent` too, so the whole walk does.

```
lemma exists_walk_mem_walkComponent {W : Finset V} {v₂ a : V}
    (ha : a ∈ G.walkComponent v₂ W) :
```

```
    ∃ p : G.Walk v₂ a, ∀ y ∈ p.support, y ∈ G.walkComponent v₂ W := by
  classical
  obtain ⟨_, p, hp⟩ := mem_walkComponent.mp ha
  exact ⟨p, fun y hy => mem_walkComponent.mpr ⟨hp y hy, p.takeUntil y hy,
    fun z hz => hp z (p.support_takeUntil_subset_support hy hz)⟩⟩
```

Joining two of these walks via v_2 shows that the subgraph induced by `walkComponent v₂ W` is connected.

```
lemma connected_induce_walkComponent {W : Finset V} {v₂ : V} (h : v₂ ∈ W) :
    ((⊤ : G.Subgraph).induce (G.walkComponent v₂ W : Set V)).Connected := by
  refine (Subgraph.connected_iff_forall_exists_walk_subgraph _).mpr
    ⟨⟨v₂, self_mem_walkComponent h⟩, fun {u v} hu hv => ?_⟩
  obtain ⟨p, hp⟩ := exists_walk_mem_walkComponent (mem_coe.mp hu)
  obtain ⟨q, hq⟩ := exists_walk_mem_walkComponent (mem_coe.mp hv)
  refine ⟨p.reverse.append q,
    (Walk.toSubgraph_le_induce_top_iff _).mpr fun y hy => ?_⟩
  rw [Walk.mem_support_append_iff, Walk.support_reverse, List.mem_reverse] at hy
  exact mem_coe.mpr (hy.elim (hp y) (hq y))
```

In Step 2 of the sketch two colourings are made to agree on $\{v_1, v_3\}$ by permuting the colours of one of them. This is possible because whenever $a = b \leftrightarrow c = d$, some permutation (a composition of at most two transpositions) maps a to c and b to d.

```
lemma _root_.Equiv.exists_perm_of_eq_iff_eq {α : Type*}
    {a b c d : α} (h : c = d ↔ a = b) :
    ∃ σ : Equiv.Perm α, σ a = c ∧ σ b = d := by
  classical
  exact ⟨(Equiv.swap a c).trans (Equiv.swap ((Equiv.swap a c) b) d),
         by grind, by grind⟩
```

Combined with `ColoringOn.union`, this gives the gluing step. Suppose that A and B meet only in $\{x, y\}$, and that there are no edges between $A \setminus B$ and $B \setminus A$. Then colourings C of A and D of B with the same pattern on $\{x, y\}$ can be glued into a colouring of $A \cup B$: permute the colours of D so that it agrees with C on $\{x, y\}$, and use C on A and the permuted D on $B \setminus A$.

```
lemma ColoringOn.nonempty_union_of_eq_iff_eq {γ : Type*} {A B : Set V} {x y : V}
    (C : G.ColoringOn γ A) (D : G.ColoringOn γ B) (hmeet : A ∩ B ⊆ {x, y})
    (hcross : ∀ u ∈ A \ B, ∀ v ∈ B \ A, ¬G.Adj u v)
    (hpat : C x = C y ↔ D x = D y) : Nonempty (G.ColoringOn γ (A ∪ B)) := by
  classical
  obtain ⟨σ, hσx, hσy⟩ := Equiv.exists_perm_of_eq_iff_eq hpat
  refine ⟨C.union (D.map σ.toEmbedding) fun u v hu hv huv => ?_⟩
  by_cases hu' : u ∈ B
  · -- u is x or y, where C agrees with the permuted D
    have hσu : σ (D u) = C u := by
      obtain rfl | rfl : u = x ∨ u = y := by simpa using hmeet ⟨hu, hu'⟩
      exacts [hσx, hσy]
    rw [ColoringOn.map_apply, ← hσu]
    exact fun h => D.valid ⟨hu', hv.1, huv⟩ (σ.injective h)
  · -- otherwise u ∈ A \ B and v ∈ B \ A, which are not adjacent
    exact (hcross u ⟨hu, hu'⟩ v hv).elim huv
```

Step 2 of the sketch: colourings of `D := A ∪ {v₁, v₃}` and `E := B ∪ {v₁, v₃}` with the same pattern (equal or distinct) on $\{v_1, v_3\}$. Here A is given abstractly, with the properties of the walk-component that we need as hypotheses, and `B := K \ (A ∪ {v₁, v₃})`. Cases (a) and (b) of Step 2 are the two branches of the `by_cases` on the number of neighbours of v_1 and v_3 in B.

```
private lemma exists_colorings_pattern_iff [DecidableEq V] [G.LocallyFinite]
    (hΔ : 3 ≤ Δ) (K A : Finset V) {v₁ v₂ v₃ : V}
    (hclosed : ∀ u ∈ K, G.neighborFinset u ⊆ K)
    (hregK : ∀ u ∈ K, G.degree u = Δ) (ih : ∀ t ⊂ K, G.ColorableOn Δ ↑t)
    (hne13 : v₁ ≠ v₃) (hnadj : ¬G.Adj v₁ v₃)
    (ha1 : G.Adj v₂ v₁) (ha3 : G.Adj v₂ v₃)
    (hAsub : A ⊆ K \ {v₁, v₃}) (hv₂A : v₂ ∈ A)
    (hAconn : ((⊤ : G.Subgraph).induce (A : Set V)).Connected)
    (hAclosed : ∀ ⦃a⦄, a ∈ A → ∀ ⦃b⦄, b ∈ K \ {v₁, v₃} → G.Adj a b → b ∈ A) :
    ∃ (C_D : G.ColoringOn (Fin Δ) ↑(A ∪ {v₁, v₃}))
      (C_E : G.ColoringOn (Fin Δ) ↑((K \ (A ∪ {v₁, v₃})) ∪ {v₁, v₃})),
      (C_D v₁ = C_D v₃) ↔ (C_E v₁ = C_E v₃) := by
  classical
  let B := K \ (A ∪ {v₁, v₃})
  have hv₂K : v₂ ∈ K := by grind
  have hd2 : G.degree v₂ = Δ := by grind
  have hv1K : v₁ ∈ K := hclosed v₂ hv₂K ((mem_neighborFinset _ _ _).mpr ha1)
  have hv3K : v₃ ∈ K := hclosed v₂ hv₂K ((mem_neighborFinset _ _ _).mpr ha3)
  have h13K : {v₁, v₃} ⊆ K := by grind
  have hAK : A ⊆ K := by grind
  have hv1A : v₁ ∉ A := by grind
  -- A and B partition K \ {v₁, v₃}
  have hABdisj : Disjoint A B := Finset.disjoint_left.mpr
    (fun z hzA hzB => (mem_sdiff.mp hzB).2 (mem_union_left _ hzA))
  have hABcov : A ∪ B = K \ {v₁, v₃} := by grind
  -- the Δ neighbours of v₁ (resp. v₃) split between A and B,
  -- with v₂ in A
  have hNsub : ∀ v ∈ ({v₁, v₃} : Finset V), G.neighborFinset v ⊆ K \ {v₁, v₃} := by
    grind [mem_neighborFinset, adj_symm]
  have hNK : ∀ v ∈ ({v₁, v₃} : Finset V), G.degIn A v + G.degIn B v = Δ := by
    intro v hv
    rw [← degIn_union A B v hABdisj, hABcov, degIn_eq_degree (hNsub v hv),
      hregK v (h13K hv)]
  have hAv1 := hNK v₁ (by grind)
  have hAv3 := hNK v₃ (by grind)
  have hA1pos : 1 ≤ G.degIn A v₁ :=
    card_pos.mpr ⟨v₂, mem_filter.mpr ⟨hv₂A, ha1.symm⟩⟩
  have hA3pos : 1 ≤ G.degIn A v₃ :=
    card_pos.mpr ⟨v₂, mem_filter.mpr ⟨hv₂A, ha3.symm⟩⟩
  -- the neighbours of v₂ are in D, since A is closed
  -- under adjacency into K \ {v₁, v₃}
  have hnbhdD : G.neighborFinset v₂ ⊆ (A ∪ {v₁, v₃}).erase v₂ := by
    intro w hw
    grind [(mem_neighborFinset _ _ _).mp hw]
  have hMA : (A ∪ {v₁, v₃}) \ {v₁, v₃} = A := by grind
  -- D always has a colouring with v₁, v₃ the same colour (Step 1 applied to D)
  obtain ⟨C_D_eq, hC_D_eq⟩ := exists_coloring_eq (A ∪ {v₁, v₃}) (by grind)
    (mem_union_left _ hv₂A) (mem_union_right _ (by simp))
    (mem_union_right _ (by simp)) hne13 hnadj hd2 ha1 ha3
```

```
    (by grind) hnbhdD (by grind)
  have hAss : A ⊂ K := (ssubset_iff_of_subset hAK).mpr ⟨v₁, hv1K, hv1A⟩
  -- E = B ∪ {v₁, v₃} is a proper subset of K (it misses v₂ ∈ A)
  have hv₂E : v₂ ∉ B ∪ {v₁, v₃} := by grind
  have hEss : B ∪ {v₁, v₃} ⊂ K := by grind
  -- by_cases! runs by_cases and then push Not in the second (negated) subgoal
  by_cases! hsum : G.degIn B v₁ + G.degIn B v₃ ≤ Δ - 1
  · -- Case (a): colour B by ih and give v₁, v₃ a common free colour
    have hBss : B ⊂ K := (ssubset_iff_of_subset sdiff_subset).mpr
      ⟨v₂, hv₂K, fun h => hv₂E (mem_union_left _ h)⟩
    obtain ⟨C_B0⟩ := ih B hBss
    obtain ⟨C_E, hC_E⟩ := exists_two_eq B C_B0 hnadj (by grind)
    exact ⟨C_D_eq, C_E.copy (by grind), iff_of_true hC_D_eq hC_E⟩
  · -- Case (b): colour A by ih and give v₁, v₃ distinct colours
    obtain ⟨C_A⟩ := ih A hAss
    have hDdist : ∃ C : G.ColoringOn (Fin Δ) ↑(A ∪ {v₁, v₃}), C v₁ ≠ C v₃ := by
      -- one of v₁, v₃ has at most Δ - 2 neighbours in A
      -- ("say v₁" in the sketch)
      by_cases h2 : G.degIn A v₁ + 1 < Δ
      · obtain ⟨C', hC'⟩ := exists_two_ne A C_A hne13 hnadj (by grind) h2
        exact ⟨C'.copy (by grind), hC'⟩
      · obtain ⟨C', hC'⟩ :=
          exists_two_ne A C_A (Ne.symm hne13) (fun h => hnadj h.symm)
            (by grind) (by grind)
        exact ⟨C'.copy (by grind), fun h => hC' h.symm⟩
    obtain ⟨C_D_dist, hC_D_dist⟩ := hDdist
    obtain ⟨C_E⟩ := ih _ hEss
    -- pair the ih-colouring of E with the colouring of D of the same pattern
    by_cases hEpat : C_E v₁ = C_E v₃
    · exact ⟨C_D_eq, C_E, iff_of_true hC_D_eq hEpat⟩
    · exact ⟨C_D_dist, C_E, iff_of_false hC_D_dist hEpat⟩
```

Completing the Proof

The colouring of K in the regular case of the sketch: K is a Δ-regular set containing v_2, closed under adjacency, with $\Delta \geq 3$ and no $(\Delta + 1)$-clique, and every proper subset of K is Δ-colourable.

```
private lemma closed_regular_case [G.LocallyFinite] (hΔ : 3 ≤ Δ)
    (hcf : G.CliqueFree (Δ + 1))
    (K : Finset V) (v₂ : V) (hv₂ : v₂ ∈ K)
    (hclosed : ∀ u ∈ K, G.neighborFinset u ⊆ K)
    (hregK : ∀ u ∈ K, G.degree u = Δ) (ih : ∀ t ⊂ K, G.ColorableOn Δ ↑t) :
    G.ColorableOn Δ ↑K := by
  classical
  have : NeZero Δ := ⟨by grind⟩
  have hd2 : G.degree v₂ = Δ := hregK v₂ hv₂
  obtain ⟨v₁, v₃, hne13, ha1, ha3, hnadj⟩ :=
    hcf.exists_ne_adj_not_adj_of_le_degree hd2.symm.le
  have hn1 : v₁ ∈ G.neighborFinset v₂ := (mem_neighborFinset _ _ _).mpr ha1
  have hn3 : v₃ ∈ G.neighborFinset v₂ := (mem_neighborFinset _ _ _).mpr ha3
  have hv1K : v₁ ∈ K := hclosed v₂ hv₂ hn1
  have hv3K : v₃ ∈ K := hclosed v₂ hv₂ hn3
  have hv₂W : v₂ ∈ K \ {v₁, v₃} :=
```

```
    mem_sdiff.mpr ⟨hv₂, by simpa using ⟨ha1.ne, ha3.ne⟩⟩
  -- A is the component of v₂ in W = K \ {v₁, v₃} and B is the rest
  let A := G.walkComponent v₂ (K \ {v₁, v₃})
  let B := K \ (A ∪ {v₁, v₃})
  have hAsub : A ⊆ K \ {v₁, v₃} := walkComponent_subset
  have hv₂A : v₂ ∈ A := self_mem_walkComponent hv₂W
  have hAclosed : ∀ ⦃a⦄, a ∈ A → ∀ ⦃b⦄, b ∈ K \ {v₁, v₃} → G.Adj a b → b ∈ A :=
    fun a ha b hb hab => adj_mem_walkComponent ha hb hab
  -- colourings of D and E with the same pattern on {v₁, v₃}
  obtain ⟨C_D, C_E₀, hpat⟩ :
      ∃ (C_D : G.ColoringOn (Fin Δ) ↑(A ∪ {v₁, v₃}))
        (C_E₀ : G.ColoringOn (Fin Δ) ↑(B ∪ {v₁, v₃})),
        (C_D v₁ = C_D v₃) ↔ (C_E₀ v₁ = C_E₀ v₃) :=
    exists_colorings_pattern_iff hΔ K A hclosed hregK ih hne13 hnadj ha1 ha3
      hAsub hv₂A (connected_induce_walkComponent hv₂W) hAclosed
  -- D ∪ E = K
  have hDEK : (A ∪ {v₁, v₃}) ∪ (B ∪ {v₁, v₃}) = K := by grind
  -- D and E meet in {v₁, v₃}, and there are no edges between A and B, so
  -- the two colourings can be glued
  obtain ⟨C⟩ := C_D.nonempty_union_of_eq_iff_eq C_E₀ (by grind)
    (fun u hu w hw huw => by grind) hpat
  exact ⟨C.copy (by rw [← coe_union, hDEK])⟩
```

The regular case of the sketch: every vertex of s has degree Δ and all its neighbours in s, so `closed_regular_case` colours s (taking `K = s`).

```
private lemma regular_case [G.LocallyFinite] [DecidableRel G.Adj] (hΔ : 3 ≤ Δ)
    (hcf : G.CliqueFree (Δ + 1))
    (hdeg : ∀ v, G.degree v ≤ Δ) (s : Finset V) (hne : s.Nonempty)
    (hreg : ∀ a ∈ s, Δ ≤ G.degIn s a) (ih : ∀ t ⊂ s, G.ColorableOn Δ ↑t) :
    G.ColorableOn Δ ↑s := by
  classical
  obtain ⟨v₂, hv₂⟩ := hne
  have hdegs : ∀ a ∈ s, G.degree a = Δ := fun a ha =>
    (hdeg a).antisymm ((hreg a ha).trans (degIn_le_degree s a))
  have hsubs : ∀ a ∈ s, G.neighborFinset a ⊆ s := fun a ha =>
    neighborFinset_subset_of_degree_le_degIn a s ((hdeg a).trans (hreg a ha))
  exact closed_regular_case hΔ hcf s v₂ hv₂ hsubs hdegs ih
```

Any locally finite graph that is $(\Delta + 1)$-clique-free with maximum degree at most $\Delta \geq 3$ is Δ-colourable on any finite vertex set s.

```
lemma colorableOn_of_cliqueFree_of_degree_le [G.LocallyFinite] (hΔ : 3 ≤ Δ)
    (hcf : G.CliqueFree (Δ + 1)) (hdeg : ∀ v, G.degree v ≤ Δ) (s : Finset V) :
    G.ColorableOn Δ ↑s := by
  classical
  induction s using strongInduction with
  | _ s ih =>
  rcases s.eq_empty_or_nonempty with rfl | hne
  · -- s = ∅: ColoringOn.empty needs a colour; Fin Δ is nonempty since 0 < Δ
    exact ⟨(ColoringOn.empty ⟨0, by grind⟩).copy (by simp)⟩
  by_cases! hex : ∃ a ∈ s, G.degIn s a < Δ
  · -- Degeneracy reduction: erase a low-degree vertex and extend greedily
    obtain ⟨a, ha, hlt⟩ := hex
```

```
    obtain ⟨C⟩ := ih _ (erase_ssubset ha)
    have hlt' : G.degIn (s.erase a) a < Δ :=
      (degIn_mono (erase_subset a s) a).trans_lt hlt
    obtain ⟨c, hc⟩ := C.exists_free_color_of_degIn a hlt'
    exact ⟨(C.insert a (fun v hv hadj => hc v hv hadj)).copy (by simp_all)⟩
  · -- Regular case: every vertex of s has all Δ of its neighbours inside s
    exact regular_case hΔ hcf hdeg s hne hex ih
```

Brooks' theorem (for finite graphs with $\Delta \geq 3$): a finite graph that is $(\Delta + 1)$-clique-free with maximum degree at most $\Delta \geq 3$ is Δ-colourable.

```
private theorem brooks_of_finite [G.LocallyFinite] [Finite V] (hΔ : 3 ≤ Δ)
    (hcf : G.CliqueFree (Δ + 1))
    (hdeg : ∀ v, G.degree v ≤ Δ) : G.Colorable Δ := by
  have := Fintype.ofFinite V
  exact (G.colorableOn_univ_iff_colorable Δ).1
    (by simpa using colorableOn_of_cliqueFree_of_degree_le hΔ hcf hdeg univ)

end brooks
```

By De Bruijn–Erdős, Brooks' theorem extends to infinite graphs.

Brooks' theorem ($\Delta \geq 3$): a locally finite graph that is $(\Delta + 1)$-clique-free with maximum degree at most $\Delta \geq 3$ is Δ-colourable.

```
theorem colorable_of_cliqueFree_of_degree_le {Δ : ℕ} [G.LocallyFinite]
    (hΔ : 3 ≤ Δ) (hcf : G.CliqueFree (Δ + 1)) (hdeg : ∀ v, G.degree v ≤ Δ) :
    G.Colorable Δ :=
  colorable_of_forall_finite_subgraph_colorable fun H hH => by
    classical
    have := hH.to_subtype
    refine brooks_of_finite hΔ (hcf.comap ⟨H.coeCopy⟩) fun w => ?_
    rw [Subgraph.coe_degree]
    exact (H.degree_le w).trans (hdeg w.val)
```

Brooks' Theorem Full Version

We can now put the cases together. When $\Delta \leq 1$ there is nothing to prove: a graph with no 1-clique has no vertices, and a graph with no 2-clique has no edges (`cliqueFree_one` and `cliqueFree_two`).

Brooks' theorem: a graph with all degrees at most Δ and no $(\Delta + 1)$-clique is Δ-colourable, unless $\Delta = 2$ and one of its components is an odd cycle.

```
theorem brooks [G.LocallyFinite] {Δ : ℕ}
    (hdeg : ∀ v, G.degree v ≤ Δ) (hcf : G.CliqueFree (Δ + 1))
    (hcyc : Δ = 2 → ¬∃ (u : V) (c : G.Walk u u),
      c.IsCycle ∧ Odd c.length ∧ c.toSubgraph = G⟦u⟧.toSubgraph) :
    G.Colorable Δ := by
  rcases (by grind : Δ = 0 ∨ Δ = 1 ∨ Δ = 2 ∨ 3 ≤ Δ) with rfl | rfl | rfl | hΔ
  · exact G.colorable_zero_iff.mpr (cliqueFree_one.mp hcf)
  · obtain rfl := cliqueFree_two.mp hcf
    exact ⟨Coloring.mk (fun _ => 0) fun h => h.elim⟩
  · exact brooks_two hdeg (hcyc rfl)
  · exact colorable_of_cliqueFree_of_degree_le hΔ hcf hdeg
```

10.6 Cut Vertices and Blocks*

A *cut vertex* is a vertex x whose removal splits the connected component containing x into two or more components. (For a finite graph this is the same as saying that removing x increases the number of components, but that formulation fails for graphs with infinitely many components.)

A *block* of a graph G is a subgraph B that is connected and has no cut vertex of its own, and that is maximal among such subgraphs: no larger subgraph of G is both connected and free of cut vertices. So every block is of one of three kinds: a maximal 2-connected subgraph, a bridge (the subgraph consisting of just that edge and its two endpoints), or an isolated vertex. One reason for caring about blocks is that they can help with colouring a graph: for $n \geq 1$, a graph is n-colourable iff each of its blocks is. In this section we prove the harder direction for finite graphs (`colorable_of_forall_block`); the exercises give the converse and extend the result to all graphs (`colorable_iff_forall_block`).

A separate pedagogical motivation for introducing blocks is that it gives us a good excuse to describe a graph-theoretic construction built from first principles using an equivalence relation and Lean's `Quotient` type.

Mathlib does not define cut vertices. In Lean the natural definition talks about walks rather than components: x is a cut vertex if some two vertices other than x are joined by a walk in G, but every such walk passes through x.

x is a *cut vertex* of G: some two vertices other than x are joined by a walk, but every walk between them passes through x.

```
def IsCutVertex (G : SimpleGraph V) (x : V) : Prop :=
  ∃ u v, u ≠ x ∧ v ≠ x ∧ G.Reachable u v ∧ ∀ p : G.Walk u v, x ∈ p.support
```

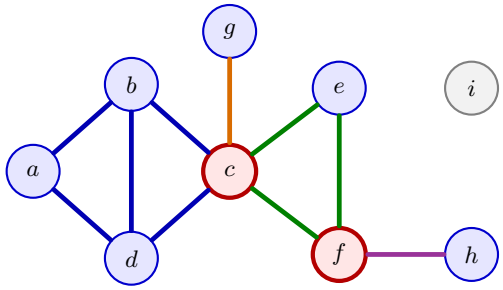
Cut vertices are closely related to vertex connectivity (`IsVertexConnected`). In the definition of `IsVertexConnected 2` the deleted set `S` has at most one vertex. When `S` is empty the condition says that G is connected, and when `S = {x}` it says (for connected G) that x is not a cut vertex. So a graph is 2-vertex-connected iff it has at least three vertices, is connected and has no cut vertex.

```
lemma isVertexConnected_two_iff : G.IsVertexConnected 2 ↔
    3 ≤ ENat.card V ∧ G.Connected ∧ ∀ x, ¬G.IsCutVertex x := by
  have h3 : (2 : ℕ∞) + 1 = 3 := by norm_num
  constructor
  · rintro ⟨hcard, h⟩
    rw [h3] at hcard
    have : Nonempty V :=
      (ENat.card_ne_zero_iff_nonempty V).mp fun h0 => by
        rw [h0] at hcard; norm_num at hcard
    refine ⟨hcard, ⟨fun u v => ?_⟩, fun x ⟨u, v, hux, hvx, _, hp⟩ => ?_⟩
    · -- deleting no vertices: G is connected
      obtain ⟨p, -⟩ := h ∅ (by simp) u v (by simp) (by simp)
      exact ⟨p⟩
    · -- deleting x: some walk from u to v avoids x
      obtain ⟨p, hp'⟩ :=
        h {x} (by simp) u v (by simpa using hux) (by simpa using hvx)
      exact hp' x (hp p) rfl
  · rintro ⟨hcard, hconn, hcut⟩
    refine ⟨h3 ▸ hcard, fun S hS u v hu hv => ?_⟩
    -- a set of less than two vertices is empty or a single vertex x
    rcases Set.encard_le_one_iff_eq.mp (by simpa using hS) with rfl | ⟨x, rfl⟩
```

```
  · obtain ⟨p⟩ := hconn u v
    exact ⟨p, by simp⟩
  · by_contra! h
    exact hcut x ⟨u, v, by grind, by grind, hconn u v, fun p => by grind [h p]⟩
```

A cut vertex can lie in several blocks, so the blocks do not partition the vertices of a graph, but they do partition its *edges*. So we define blocks in the same way as connected components, via an equivalence relation, but on edges rather than vertices. The relation we use is *cycle-equivalence*: two edges are equivalent if they are the same edge or lie in a common cycle. The equivalence classes of this relation will be the blocks of the graph. A bridge lies on no cycles, so it forms a block on its own.

Apart from isolated vertices, the blocks in the sense defined at the start of this section are exactly the subgraphs spanned by these equivalence classes, though we will not prove this. An isolated vertex lies in no edge, so it belongs to no block in our sense; in particular a graph with no edges has no blocks at all. (This is why our colouring results assume `n ≥ 1`: the graph `K[1]` has no blocks, so all its blocks are (vacuously) `0`-colourable, but `K[1]` is not `0`-colourable.)



A graph with four blocks, its edges coloured by block. The cut vertices `c` and `f` lie in three and two blocks, and the isolated vertex `i` lies in no block

Like the deleted sets in `IsEdgeReachable`, cycle-equivalence is most conveniently defined on all of `Sym2 V`, not just on `G.edgeSet`. We call an element `s(u, v)` of `Sym2 V` a *potential edge*, and reserve *edge* for the elements of `G.edgeSet`. A potential edge that is not an edge lies on no cycles, so it is cycle-equivalent only to itself.

Cycle-equivalence, a relation on `Sym2 V`: two potential edges are related if they are equal or lie on a common cycle.

```
def CycleEquivalent (G : SimpleGraph V) : Sym2 V → Sym2 V → Prop :=
  fun e₁ e₂ => e₁ = e₂ ∨
    ∃ (u : V) (c : G.Walk u u), c.IsCycle ∧ e₁ ∈ c.edges ∧ e₂ ∈ c.edges
```

Cycle-Equivalence

`CycleEquivalent` is reflexive by definition. It is also symmetric, and preserves membership in the edge set: if `e ≠ f` are cycle-equivalent, both lie on a cycle of `G`, so both are edges of `G`.

```
lemma CycleEquivalent.symm {e f : Sym2 V} (h : G.CycleEquivalent e f) :
    G.CycleEquivalent f e := by
  rcases h with (rfl | ⟨u, c, hc, h1, h2⟩)
  · left; rfl
  · right; exact ⟨u, c, hc, h2, h1⟩
```

```
lemma CycleEquivalent.mem_edgeSet_iff {e f : Sym2 V} (h : G.CycleEquivalent e f) :
    e ∈ G.edgeSet ↔ f ∈ G.edgeSet := by
  rcases h with rfl | ⟨_, p, _, he, hf⟩
  · rfl
  · exact iff_of_true (p.edges_subset_edgeSet he) (p.edges_subset_edgeSet hf)
```

`CycleEquivalent` is in fact also transitive: two cycles sharing an edge `f`, one containing the edge `e` and the other containing the edge g, can be spliced into a single cycle through `e` and g. Proving this in Lean would be painful (but not impossible); however, we can avoid it. Mathlib allows us to form the equivalence relation *generated* by any relation.

```
#source Relation.EqvGen
```

Using this we can define *block-equivalence* of potential edges to be the equivalence relation generated by `CycleEquivalent`.

Two potential edges are *block-equivalent* if they are related by the equivalence relation generated by `CycleEquivalent`.

```
def BlockEquivalent (G : SimpleGraph V) : Sym2 V → Sym2 V → Prop :=
  Relation.EqvGen G.CycleEquivalent
```

`BlockEquivalent` is an equivalence relation (by construction).

```
lemma blockEquivalent_equivalence : Equivalence G.BlockEquivalent :=
  Relation.EqvGen.is_equivalence _

lemma BlockEquivalent.refl (e : Sym2 V) : G.BlockEquivalent e e :=
  G.blockEquivalent_equivalence.refl e

lemma BlockEquivalent.symm {e f : Sym2 V} (h : G.BlockEquivalent e f) :
    G.BlockEquivalent f e :=
  G.blockEquivalent_equivalence.symm h

lemma BlockEquivalent.trans {e f g : Sym2 V} (h₁ : G.BlockEquivalent e f)
    (h₂ : G.BlockEquivalent f g) : G.BlockEquivalent e g :=
  G.blockEquivalent_equivalence.trans h₁ h₂
```

Cycle-equivalent potential edges are block-equivalent.

```
lemma CycleEquivalent.blockEquivalent {e f : Sym2 V} (h : G.CycleEquivalent e f) :
    G.BlockEquivalent e f := Relation.EqvGen.rel _ _ h
```

So block-equivalence also preserves membership in the edge set. Notice that we use induction on `h : G.BlockEquivalent e f` in the proof.

```
lemma BlockEquivalent.mem_edgeSet_iff {e f : Sym2 V} (h : G.BlockEquivalent e f) :
    e ∈ G.edgeSet ↔ f ∈ G.edgeSet := by
  induction h with
  | rel x y hxy => exact hxy.mem_edgeSet_iff
  | refl x => exact Iff.rfl
  | symm x y _ ih => exact ih.symm
  | trans x y z _ _ ih₁ ih₂ => exact ih₁.trans ih₂
```

Blocks as a Quotient of Edges

Just as `ConnectedComponent` is the quotient of `V` by `Reachable`, a *block* is an equivalence class of edges under `BlockEquivalent`, packaged as a quotient of `G.edgeSet`. (Mathlib defines `ConnectedComponent` as `Quot G.Reachable`: a setoid on `V` cannot be an instance, since `Setoid V` does not mention `G`. Our setoid lives on `G.edgeSet`, so it can be an instance, and we use `Quotient`. The two are interchangeable, since `Quotient s` is by definition `Quot s.r`.)

The setoid is built in two steps. First, `Relation.EqvGen.setoid G.CycleEquivalent` bundles block-equivalence (the equivalence relation generated by `CycleEquivalent`) as a setoid on all of `Sym2 V`. But we only want classes of edges, not of all potential edges. So we pull the setoid back along the inclusion `Subtype.val : G.edgeSet → Sym2 V` using `Setoid.comap`, which relates two edges exactly when their underlying potential edges are related. We make the result an instance, so that we can write `e ≈ f` for block-equivalent edges.

The setoid on the edges of `G` given by block-equivalence (analogue of Mathlib's `G.reachableSetoid`).

```
instance blockSetoid (G : SimpleGraph V) : Setoid G.edgeSet :=
  Setoid.comap Subtype.val (Relation.EqvGen.setoid G.CycleEquivalent)
```

Unfolding the definitions, two edges are related by `blockSetoid` exactly when they are block-equivalent.

```
example (e f : G.edgeSet) : e ≈ f ↔ G.BlockEquivalent e f := Iff.rfl
```

A *block* of `G` is a block-equivalence class of edges of `G` (analogue of `ConnectedComponent`).

```
def Block (G : SimpleGraph V) := Quotient G.blockSetoid
```

The block containing a given edge of `G`.

```
def blockMk (G : SimpleGraph V) (e : G.edgeSet) : G.Block := Quotient.mk _ e
```

We write `G⦃e⦄` for the block containing the edge `e`, with round double brackets in place of the square ones of the component notation `G⟦v⟧`.

`G⦃e⦄` is the block of `G` containing the edge `e`.

```
scoped notation:max G "⦃" e "⦄" => SimpleGraph.blockMk G e
```

For an edge given by an adjacency proof `h : G.Adj u v` we write `h.block` for its block.

The block containing the edge `u – v`.

```
abbrev Adj.block {u v : V} (h : G.Adj u v) : G.Block := G⦃⟨s(u, v), h⟩⦄
```

Reversing an edge does not change its block.

```
lemma Adj.block_symm {u v : V} (h : G.Adj u v) : h.symm.block = h.block := by
  simp only [Adj.block, Sym2.eq_swap]
```

Two edges lie in the same block exactly when they are block-equivalent.

```
lemma blockMk_eq_blockMk_iff {e f : G.edgeSet} :
    G⦃e⦄ = G⦃f⦄ ↔ e ≈ f :=
  Quotient.eq
```

Mathlib makes a connected component `C` into a subgraph by taking its vertices `C.supp` (the vertices whose class is `C`) and forming the induced subgraph `(⊤ : G.Subgraph).induce C.supp`. We could do the same here, inducing on the endpoints of the edges in a block `B`. For components this is fine, since any edge between two vertices of a component clearly lies in that component. For blocks the corresponding fact, that an edge of `G` joining two vertices of a block belongs to the block, is also true (adding such an edge to a 2-connected subgraph keeps it 2-connected), but proving it needs a cycle-splicing argument like the one we avoided above. Without it, the edges of the induced subgraph would not provably be the edges of `B`. So instead we define the subgraph underlying a block `B` via the edges of `G` whose class is `B`, together with their endpoints.

The subgraph of `G` underlying a block `B`: the edges of `G` whose block is `B`, and their endpoints (analogue of `ConnectedComponent.toSubgraph`).

```
def Block.toSubgraph (B : G.Block) : G.Subgraph where
  verts := {v | ∃ e : G.edgeSet, v ∈ (e : Sym2 V) ∧ G⟦e⟧ = B}
  Adj v w := ∃ h : G.Adj v w, h.block = B
  adj_sub := fun ⟨h, _⟩ => h
  edge_vert := fun ⟨h, hB⟩ => ⟨⟨_, h⟩, Sym2.mem_mk_left _ _, hB⟩
  symm := ⟨fun _ _ ⟨h, hB⟩ => ⟨h.symm, h.block_symm.trans hB⟩⟩


@[simp] lemma Block.toSubgraph_adj {B : G.Block} {v w : V} :
    B.toSubgraph.Adj v w ↔ ∃ h : G.Adj v w, h.block = B :=
  Iff.rfl


@[simp] lemma Block.mem_toSubgraph_verts {B : G.Block} {v : V} :
    v ∈ B.toSubgraph.verts ↔ ∃ e : G.edgeSet, v ∈ (e : Sym2 V) ∧ G⟦e⟧ = B :=
  Iff.rfl
```

In the block of an edge `e`, two vertices are adjacent exactly when they are joined by an edge of `G` that is block-equivalent to `e`.

```
lemma blockMk_toSubgraph_adj {e : G.edgeSet} {v w : V} :
    G⟦e⟧.toSubgraph.Adj v w ↔ G.Adj v w ∧ G.BlockEquivalent e s(v, w) :=
  ⟨fun ⟨h, hB⟩ => ⟨h, (blockMk_eq_blockMk_iff.mp hB).symm⟩,
    fun ⟨h, he⟩ => ⟨h, blockMk_eq_blockMk_iff.mpr he.symm⟩⟩
```

Blocks behave well under taking supergraphs and, more generally, under copies `f : Copy G H` (injective homomorphisms): these map cycles to cycles, and hence preserve cycle-equivalence.

```
lemma CycleEquivalent.map {W : Type*} {H : SimpleGraph W} (f : Copy G H)
    {a b : Sym2 V} (h : G.CycleEquivalent a b) :
    H.CycleEquivalent (a.map f) (b.map f) := by
  rcases h with rfl | ⟨u, c, hc, ha, hb⟩
  · exact Or.inl rfl
  · exact Or.inr ⟨f u, c.map f.toHom,
      (Walk.isCycle_map_iff_of_injective f.injective).mpr hc,
      by simpa using ⟨a, ha, rfl⟩, by simpa using ⟨b, hb, rfl⟩⟩
```

Block-equivalence is preserved by copies.

```
lemma BlockEquivalent.map {W : Type*} {H : SimpleGraph W} (f : Copy G H)
    {a b : Sym2 V} (h : G.BlockEquivalent a b) :
    H.BlockEquivalent (a.map f) (b.map f) := by
  induction h with
```

```
  | rel x y hxy => exact (hxy.map f).blockEquivalent
  | refl x => exact .refl _
  | symm x y _ ih => exact ih.symm
  | trans x y z _ _ ih₁ ih₂ => exact ih₁.trans ih₂
```

So a copy `f : Copy G H` sends the block of an edge `e` to the block of `e.map f`, and by `BlockEquivalent.map` this does not depend on the choice of `e`. So `f` induces a map `Block.map f : G.Block → H.Block`, just as a homomorphism induces a map of connected components (`ConnectedComponent.map`).

The image of a block under a copy `f`: the block containing the images of its edges.

```
def Block.map {W : Type*} {H : SimpleGraph W} (f : Copy G H) (B : G.Block) :
    H.Block :=
  Quotient.map (fun e => ⟨e.1.map f, f.toHom.map_mem_edgeSet e.2⟩)
    (fun _ _ => BlockEquivalent.map f) B
```

Moreover, `f` maps each block `B` into the block `B.map f`: an edge `v — w` of `G` in `B` goes to the edge `f v — f w` of `H`, which lies in `B.map f` by the definition of `Block.map`.

```
lemma Block.map_toSubgraph_le {W : Type*} {H : SimpleGraph W} (f : Copy G H)
    (B : G.Block) : B.toSubgraph.map f.toHom ≤ (B.map f).toSubgraph := by
  constructor
  · rintro _ ⟨v, ⟨e, hv, rfl⟩, rfl⟩
    exact ⟨⟨_, f.toHom.map_mem_edgeSet e.2⟩, Sym2.mem_map.mpr ⟨v, hv, rfl⟩, rfl⟩
  · rintro _ _ ⟨v, w, ⟨h, rfl⟩, rfl, rfl⟩
    exact ⟨f.toHom.map_adj h, rfl⟩
```

Colouring a Graph from its Blocks

We now prove `colorable_of_forall_block`: for `n ≥ 1`, a finite graph is `n`-colourable if each of its blocks is. The idea is to cut `G` into two pieces that share at most one vertex, colour each piece, and glue the colourings using `Colorable.sup_of_subsingleton_inter`.

```
#check Colorable.sup_of_subsingleton_inter
```

To pull back colourings of blocks of `G` to blocks of a subgraph `G₁ ≤ G` with `Colorable.of_hom`, we turn the inclusion given by `Block.map_toSubgraph_le` (for the copy `Copy.ofLE`) into a homomorphism between the coerced graphs using `Subgraph.homOfMapLE`.

```
#check Subgraph.homOfMapLE -- (f : G →g K) (h : A.map f ≤ B) : A.coe →g B.coe
```

We also need to extend colourings in the other direction. If a subgraph `H` contains every edge of `G`, then `G ≤ H.spanningCoe`, so for `n ≥ 1` a colouring of `H.coe` gives one of `G` by `Subgraph.colorable_spanningCoe_iff` and `Colorable.mono_left`.

```
#check Subgraph.colorable_spanningCoe_iff
#check Colorable.mono_left
```

We say that `G` *splits at* `x` if it is the union of two proper subgraphs whose supports meet in at most `x`.

```
def SplitsAt (G : SimpleGraph V) (x : V) : Prop :=
  ∃ G₁ G₂ : SimpleGraph V, G₁ < G ∧ G₂ < G ∧ G₁ ⊔ G₂ = G ∧
```

```
    G₁.support ∩ G₂.support ⊆ {x}
```

Splitting at x is not the same as x being a cut vertex. A graph with a cut vertex x splits at x, but a graph whose edges lie in two different components splits at every vertex, whether or not it is a cut vertex. Isolated vertices also play no role in `SplitsAt`. (For a connected graph the two notions do agree.) `SplitsAt` is what our colouring argument needs. There are two cases: if G splits at some vertex, it is glued together from two proper pieces sharing at most one vertex, which we can colour separately; if G splits nowhere, all its edges lie in one block, so a colouring of that block colours G. The second case would fail for cut vertices, since a graph with no cut vertex can have one block in each of its components.

We show that a graph with a cut vertex x splits at x (`IsCutVertex.splitsAt`), and so does a graph with edges in two different connected components, at any vertex (`splitsAt_of_not_reachable`). Both follow from a single lemma about walks. Fix a non-isolated vertex $u \neq x$, and call a vertex *near* if there is a walk from u to it avoiding x. Let `G₁` consist of the edges of G with a near endpoint, and let `G₂ = G \ G₁` (both as `SimpleGraph V`). A vertex `z ≠ x` adjacent to a near vertex is itself near, so the supports of `G₁` and `G₂` meet in at most x. The edges of G incident to u lie in `G₁`, so `G₂` is proper, and `G₁` is proper as long as some edge `a – b` of G has no near endpoint, that is, every walk from u to a or to b goes via x.

```
lemma splitsAt_of_forall_walk_mem_support {u x a b : V} (hux : u ≠ x)
    (hu : ¬G.IsIsolated u) (hab : G.Adj a b) (ha : ∀ p : G.Walk u a, x ∈ p.support)
    (hb : ∀ p : G.Walk u b, x ∈ p.support) : G.SplitsAt x := by
  -- R z: the vertex z is near.
  let R : V → Prop := fun z => ∃ p : G.Walk u z, x ∉ p.support
  have hR : ∀ {y z}, R y → G.Adj y z → z ≠ x → R z := by
    rintro y z ⟨p, hp⟩ hyz hzx
    exact ⟨p.concat hyz, by simp [Walk.support_concat, hp, hzx.symm]⟩
  let G₁ := G ⊓ fromRel fun y z => R y ∨ R z
  let G₂ := G \ G₁
  obtain ⟨u', hu'⟩ := exists_adj_iff_not_isIsolated.2 hu
  have hRu : R u := ⟨.nil, by simpa using hux.symm⟩
  have hRa : ¬R a := fun ⟨p, hp⟩ => hp (ha p)
  have hRb : ¬R b := fun ⟨p, hp⟩ => hp (hb p)
  refine ⟨G₁, G₂, lt_of_le_not_ge inf_le_left fun h => ?_,
    lt_of_le_not_ge sdiff_le fun h => ?_, sup_sdiff_cancel_right inf_le_left, ?_⟩
  · simpa [G₁, hRa, hRb] using h hab
  · simpa [G₂, G₁, hRu, hu'.ne] using h hu'
  · rintro z ⟨⟨y, hy⟩, ⟨y', hy'⟩⟩
    by_contra hzx
    simp only [sdiff_adj, inf_adj, fromRel_adj, Set.mem_ofPred_eq, G₁, G₂] at hy hy'
    have hz : ¬R z := by grind
    have : ¬R y := fun hRy => hz (hR hRy hy.1.symm hzx)
    grind
```

If x is a cut vertex then G splits at x. Suppose x separates u from `v`; we apply `splitsAt_of_forall_walk_mem_support` with near vertices taken relative to u. Then u and `v` are distinct, so both are non-isolated. Let `y` be a neighbour of `v`. Neither `v` nor `y` is near: every walk from u to `v` passes through x, and so does every walk from u to `y`, since otherwise it would extend to a walk from u to `v` avoiding x. So the edge `v – y` has no near endpoint, and `splitsAt_of_forall_walk_mem_support` applies.

```
lemma IsCutVertex.splitsAt {x : V} (h : G.IsCutVertex x) : G.SplitsAt x := by
```

```
  obtain ⟨u, v, hux, hvx, ⟨p⟩, hp⟩ := h
  -- u ≠ v, since the trivial walk at u avoids x
  have huv : u ≠ v := by
    rintro rfl
    have := hp .nil
    simp only [support_nil, List.mem_singleton] at this
    exact hux this.symm
  -- so u and v both have neighbours; let y be a neighbour of v
  have hu := exists_adj_iff_not_isIsolated.1 ⟨_, p.adj_snd (not_nil_of_ne huv)⟩
  have hvy := p.reverse.adj_snd (not_nil_of_ne huv.symm)
  refine splitsAt_of_forall_walk_mem_support hux hu hvy hp fun q => ?_
  -- a walk from u to y avoiding x would extend to one from u to v
  by_cases hyx : p.reverse.snd = x
  · exact hyx ▸ q.end_mem_support
  · simpa [support_concat, hvx.symm] using hp (q.concat hvy.symm)
```

If G has edges in two different components then it splits at every vertex x. Suppose the edges $a - b$ and $c - d$ lie in different components. If $a \neq x$, then neither endpoint of $c - d$ is near relative to a, so `splitsAt_of_forall_walk_mem_support` applies. If $a = x$ we swap the roles of the two edges.

```
lemma splitsAt_of_not_reachable {a b c d : V} (hab : G.Adj a b) (hcd : G.Adj c d)
    (hac : ¬G.Reachable a c) (x : V) : G.SplitsAt x := by
  rcases eq_or_ne a x with rfl | hax
  · -- take u := c; no walk from c reaches the edge a — b
    exact splitsAt_of_forall_walk_mem_support (fun h => hac (by rw [h]))
      (exists_adj_iff_not_isIsolated.1 ⟨_, hcd⟩) hab
      (fun p => (hac p.reachable.symm).elim)
      (fun p => (hac (p.concat hab.symm).reachable.symm).elim)
  · -- take u := a; no walk from a reaches the edge c — d
    exact splitsAt_of_forall_walk_mem_support hax
      (exists_adj_iff_not_isIsolated.1 ⟨_, hab⟩) hcd
      (fun p => (hac ⟨p⟩).elim) (fun p => (hac ⟨p.concat hcd.symm⟩).elim)
```

For the second case of the colouring argument, we show that if G does not split at any vertex then all its edges lie in a single block. We first show this for two edges $x - u$ and $x - w$ at a common vertex x. If some walk from u to w avoids x, then shortening it to a path and closing it up with the two edges gives a cycle through both. Otherwise every walk from u to w passes through x, while $u - x - w$ joins them, so x is a cut vertex and G splits at x.

```
lemma blockEquivalent_of_adj_of_adj (hG : ∀ x, ¬G.SplitsAt x) {x u w : V}
    (hu : G.Adj x u) (hw : G.Adj x w) : G.BlockEquivalent s(x, u) s(x, w) := by
  classical
  rcases eq_or_ne u w with rfl | huw
  · exact .refl _
  by_cases! h : ∃ p : G.Walk u w, x ∉ p.support
  · obtain ⟨p, hxp⟩ := h
    have hxP : x ∉ p.bypass.support :=
      fun h => hxp (p.support_bypass_subset_support h)
    -- closing up the path p.bypass with the edges w — x — u gives a cycle
    have hcyc := (cons_isCycle_iff_length_ne_one _ hu).mpr
      ⟨p.bypass_isPath.concat hxP hw.symm,
        by simpa using fun h => huw (eq_of_length_eq_zero h)⟩
    exact CycleEquivalent.blockEquivalent (.inr ⟨x, _, hcyc, by simp, by simp⟩)
  · exact (hG x (IsCutVertex.splitsAt
```

```
      ⟨u, w, hu.ne.symm, hw.ne.symm, hu.symm.reachable.trans hw.reachable, h⟩)).elim
```

For two arbitrary edges $a - b$ and $c - d$ we chain the previous result along a walk from a to c. Such a walk exists: otherwise the edges $a - b$ and $c - d$ lie in different components, so G would split at every vertex.

```
lemma blockEquivalent_of_not_splitsAt (hG : ∀ x, ¬G.SplitsAt x) {a b c d : V}
    (hab : G.Adj a b) (hcd : G.Adj c d) : G.BlockEquivalent s(a, b) s(c, d) := by
  have key : ∀ {y} (q : G.Walk y c) {z}, G.Adj y z →
      G.BlockEquivalent s(y, z) s(c, d) := by
    intro y q
    induction q with
    | nil => exact fun hz => blockEquivalent_of_adj_of_adj hG hz hcd
    | @cons u v _ h q ih =>
      intro z hz
      refine (blockEquivalent_of_adj_of_adj hG hz h).trans ?_
      rw [Sym2.eq_swap]
      exact ih hcd h.symm
  by_cases hac : G.Reachable a c
  · exact key hac.some hab
  · exact (hG a (splitsAt_of_not_reachable hab hcd hac a)).elim
```

For finite graphs we can now argue by well-founded induction on G, with respect to `<`. This is allowed because there are only finitely many graphs on a finite vertex type (`SimpleGraph.instFinite`), and a strict order on a finite type is well-founded. If G splits at some vertex, then both pieces are proper subgraphs, whose blocks map into blocks of G along the inclusion `Copy.ofLE` (`Block.map_toSubgraph_le`), so we can colour them by induction and glue the colourings together. Otherwise G is either edgeless or all its edges lie in the block of any one of them, and a colouring of that block extends to G (`Subgraph.colorable_spanningCoe_iff`).

```
theorem colorable_of_forall_block [Finite V] {n : ℕ} (hn : 1 ≤ n)
    (G : SimpleGraph V) (hcol : ∀ B : G.Block, B.toSubgraph.coe.Colorable n) :
    G.Colorable n := by
  induction G using WellFoundedLT.induction with
  | _ G ih =>
    have hlt : ∀ H < G, H.Colorable n := fun H hH => ih H hH fun B =>
      (hcol _).of_hom (Subgraph.homOfMapLE _
        (B.map_toSubgraph_le (Copy.ofLE _ _ hH.le)))
    by_cases! hs : ∃ x, G.SplitsAt x
    · obtain ⟨x, G₁, G₂, h₁, h₂, hG, hmeet⟩ := hs
      rw [← hG]
      exact Colorable.sup_of_subsingleton_inter hmeet (hlt _ h₁) (hlt _ h₂)
    · by_cases! he : ∃ a b, G.Adj a b
      · obtain ⟨a, b, hab⟩ := he
        have hB := (Subgraph.colorable_spanningCoe_iff hn).mpr (hcol hab.block)
        exact hB.mono_left fun c d hcd => blockMk_toSubgraph_adj.mpr
          ⟨hcd, blockEquivalent_of_not_splitsAt hs hab hcd⟩
      · exact ⟨Coloring.mk (fun _ => ⟨0, hn⟩) fun {a b} hab => (he a b hab).elim⟩
```

In the exercises we use the *De Bruijn–Erdős theorem* (`colorable_of_forall_finite_subgraph_colorable`) to remove the finiteness assumption: for $n \geq 1$, any graph is n-colourable iff each of its blocks is.

10.7 Exercises

1. Show that for any graph `G` on a nonempty vertex type, either `G` or its complement `Gᶜ` is connected. Hint: if `u` and `v` are not reachable from each other in `G`, show that any two vertices are joined in `Gᶜ`, either directly or via `u` or `v`. (Mathlib has this as `connected_or_connected_compl`, but try to prove it directly.)

```
variable {V : Type*} {G : SimpleGraph V}
open Walk

example [Nonempty V] : G.Connected ∨ Gᶜ.Connected := by
  sorry
```

2. Show that the complete bipartite graph `K[s + 1, t + 1]` is connected.

```
example (s t : ℕ) : K[s + 1, t + 1].Connected := by
  sorry
```

3. Construct the equivalence between the connected components of the empty graph and its vertex type. Hint: `ConnectedComponent.lift` and `ConnectedComponent.ind`; what does a walk in `⊥` look like?

```
def botComponentEquiv' : (⊥ : SimpleGraph V).ConnectedComponent ≃ V :=
  sorry
```

4. Prove the universal property of the disjoint union: a family of homomorphisms `G i →g H` into a common graph `H` gives a homomorphism `Σg G →g H`. Hint: adjacent vertices of the disjoint union have the same index.

```
def Hom.sigmaElim' {ι : Type*} {V : ι → Type*} {W : Type*}
    (G : (i : ι) → SimpleGraph (V i))
    {H : SimpleGraph W} (f : (i : ι) → G i →g H) : Σg G →g H :=
  sorry
```

5. Use the previous exercise and `isoSigmaConnectedComponents` to build a homomorphism `G →g H` from a homomorphism out of each connected component of `G`. (Mathlib provides this map as `SimpleGraph.homOfConnectedComponents`.)

```
example {W : Type*} {H : SimpleGraph W}
    (f : (C : G.ConnectedComponent) → C.toSimpleGraph →g H) : G →g H :=
  sorry
```

6. Show that if `G` has no isolated vertices then the Mycielskian `mμ(G)` has no bridges. Hint: `isBridge_mycielskian_iff`.

```
example (hG : ∀ v, ¬G.IsIsolated v) {e : Sym2 (Option (V ⊕ V))}
    (he : e ∈ mμ(G).edgeSet) : ¬mμ(G).IsBridge e := by
  sorry
```

7. Show that a tree has at most one connected component.

```
example (h : G.IsTree) : Subsingleton G.ConnectedComponent := by
  sorry
```

8. Show that the path graph `P[n + 1]` is a tree. (Mathlib has this as `isTree_pathGraph_add_one`, but prove it directly.) Hint: deduce acyclicity from `isAcyclic_pathNat` using `IsAcyclic.comap`.

```
example (n : ℕ) : P[n + 1].IsTree := by
  sorry
```

9. Show that a finite tree has at least as many leaves (vertices of degree one) as the degree of any of its vertices.

Hint: every vertex `u ≠ v` has degree at least one (`Preconnected.degree_pos_of_nontrivial`), so `G.degree u + (if G.degree u = 1 then 1 else 0)` is at least 2. Sum this over all vertices and compare with `IsTree.card_edgeFinset` and the handshake lemma in the form `sum_degrees_eq_twice_card_edges'` from The Handshake Lemma (p. 73).

```
open Finset in
lemma IsTree.degree_le_card_leaves' [Fintype V] [G.LocallyFinite] (h : G.IsTree)
    (v : V) : G.degree v ≤ #{u | G.degree u = 1} := by
  sorry
```

10. Show that deleting a leaf (a vertex of degree one) from a tree leaves a tree. Hint: `Connected.induce_compl_singleton_of_degree_eq_one` and `IsAcyclic.induce`.

```
example [G.LocallyFinite] (hT : G.IsTree) {v : V}
    (hv : G.degree v = 1) : (G.induce {v}ᶜ).IsTree := by
  sorry
```

11. Show that every finite connected graph with at least two vertices has a vertex whose deletion leaves it connected (so not every vertex is a cut vertex). Hint: take a leaf v of a spanning tree `T` (`IsTree.exists_vert_degree_one_of_nontrivial`). Deleting it leaves `T` connected (`Connected.induce_compl_singleton_of_degree_eq_one`), and `Connected.mono` transfers this to G.

```
example [Finite V] [Nontrivial V] (h : G.Connected) :
    ∃ v, (G.induce {v}ᶜ).Connected := by
  sorry
```

12. Use Euler's theorem (`Connected.exists_closed_isEulerian`) to show that the complete graph `K[2 * n + 3]` has a closed Eulerian walk, and so does the hypercube `Q[2 * n + 2]`.

```
example (n : ℕ) : ∃ u, ∃ p : K[2 * n + 3].Walk u u, p.IsEulerian := by
  sorry
```

```
example (n : ℕ) : ∃ u, ∃ p : Q[2 * n + 2].Walk u u, p.IsEulerian := by
  sorry
```

13. Show that a k-vertex-connected graph has minimum degree at least k. Hint: `exists_minimal_degree_vertex` and `IsVertexConnected.le_degree`.

```
lemma IsVertexConnected.le_minDegree' {k : ℕ∞} [Fintype V] [Nonempty V]
    [DecidableRel G.Adj] (hG : G.IsVertexConnected k) : k ≤ δ(G) := by
  sorry
```

14. Exercises 14–16 show that if G is connected and k-vertex-connected then $G \square K_2$ is $(k+1)$-vertex-connected, and deduce that the hypercube `Q[n]` is n-vertex-connected.

For a finset S of vertices of $G \square K_2$ and `b : Bool`, let `S.fiberProj b` be the finset of vertices `z` of G whose copy `(z, b)` in the G-fibre at `b` lies in S.

The vertices `z` of G whose copy `(z, b)` in the G-fibre at `b` of $G \square K_2$ lies in S.

```
noncomputable def _root_.Finset.fiberProj (S : Finset (V × Bool)) (b : Bool) :
    Finset V :=
  S.preimage (·, b) (Prod.mk_left_injective b).injOn

@[simp] lemma _root_.Finset.mem_fiberProj {S : Finset (V × Bool)} {b : Bool}
    {z : V} :
    z ∈ S.fiberProj b ↔ (z, b) ∈ S := Finset.mem_preimage
```

Show that if `0 < k` and S has at most k elements then some G-fibre contains less than k vertices of S.

Hint: the sizes of `S.fiberProj false` and `S.fiberProj true` add up to the size of S; use `Finset.card_preimage` and `Finset.card_filter_add_card_filter_not`.

```
lemma _root_.Finset.exists_fiberProj_card_lt' {k : ℕ} {S : Finset (V × Bool)}
    (hS : S.card ≤ k) (hk : 0 < k) : ∃ b, (S.fiberProj b).card < k := by
  sorry
```

15. Let G be k-vertex-connected and let S be a finset of at most k vertices of $G \square K_2$. Show that from any vertex `(z, c)` not in S there is a walk into the G-fibre at b that avoids S.

Hint: if `c ≠ b` and `(z, b) ∈ S`, find a neighbour `w` of z such that neither `(w, false)` nor `(w, true)` lies in S: z has at least k neighbours by `IsVertexConnected.le_encard_neighborSet`, but less than k elements of S have a first coordinate other than z. Then step to `(w, c)` and cross to `(w, b)`.

```
private lemma exists_good_neighbor' {k : ℕ} (hG : G.IsVertexConnected k)
    {S : Finset (V × Bool)} (hS : S.card ≤ k) {z : V} {b : Bool} (hzb : (z, b) ∈ S) :
    ∃ w, G.Adj z w ∧ ∀ c, (w, c) ∉ S := by
  sorry

lemma IsVertexConnected.exists_walk_to_fiber' {k : ℕ} (hG : G.IsVertexConnected k)
    {S : Finset (V × Bool)} (hS : S.card ≤ k) {z : V} {c : Bool} (hzc : (z, c) ∉ S)
    (b : Bool) :
```

```
    ∃ (z' : V) (p : (G □ (⊤ : SimpleGraph Bool)).Walk (z, c) (z', b)),
      ∀ x ∈ p.support, x ∉ S := by
  sorry
```

16. Mathlib's `Walk.boxProdLeft` copies a walk of G into the G-fibre at b of $G \square H$. It is defined as a `Walk.map`, but its type records the endpoints as (u, b) and (v, b), whereas the endpoints of the `Walk.map` walk only reduce to these after unfolding `boxProdLeft`. We record the support of the copy as a `simp` lemma.

```
@[simp] lemma Walk.support_boxProdLeft {β : Type*} (H : SimpleGraph β) (b : β)
    {u v : V} (p : G.Walk u v) :
    (p.boxProdLeft H b).support = p.support.map (·, b) :=
  Walk.support_map _ _
```

Show that if G is connected and k-vertex-connected then $G \square K_2$ is $(k+1)$-vertex-connected, and deduce that the hypercube `Q[n]` is n-vertex-connected. (For $k \geq 1$ connectedness follows from k-vertex-connectivity, but two isolated vertices form a 0-vertex-connected graph G for which $G \square K_2$ is disconnected.)

Hint: $G \square K_2$ has twice as many vertices as G, and by `IsVertexConnected.of_finset` it is enough to avoid finsets `S`. If $k = 0$ then `S` is empty and $G \square K_2$ is connected by `Connected.boxProd`. Otherwise, take vertices (u, c) and (v, d) not in `S`. Pick b such that the G-fibre at b contains less than k vertices of `S` (Exercise 14), and walk from both vertices into the G-fibre at b avoiding `S` (Exercise 15). Then join the ends of these walks by a walk in G avoiding `S.fiberProj b`, copied into the G-fibre at b with `Walk.boxProdLeft`.

```
lemma IsVertexConnected.boxProd_top_bool' {k : ℕ} (hG : G.IsVertexConnected k)
    (hG_conn : G.Connected) :
    (G □ (⊤ : SimpleGraph Bool)).IsVertexConnected (k + 1) := by
  sorry

lemma isVertexConnected_hypercube' (n : ℕ) : Q[n].IsVertexConnected n := by
  sorry
```

17. Deduce that the hypercube `Q[n]` is n-edge-connected, and that `Q[n + 2]` has no bridges. Hint: Exercise 16 and `IsVertexConnected.isEdgeConnected`. Then `Q[n + 2]` is 2-edge-connected by `IsEdgeConnected.anti`; now use `isBridge_iff_not_isEdgeReachable_two`.

```
lemma isEdgeConnected_hypercube' (n : ℕ) : Q[n].IsEdgeConnected n := by
  sorry

example (n : ℕ) {u v : HypercubeVert (n + 2)} (h : Q[n + 2].Adj u v) :
    ¬Q[n + 2].IsBridge s(u, v) := by
  sorry
```

18. Show that the edge connectivity of a finite graph with at least two vertices is at most its minimum degree. Together with `vertexConnectivity_le_edgeConnectivity` this gives Whitney's inequalities [Whitney 1932] $\kappa(G) \leq \lambda(G) \leq \delta(G)$. Hint: `exists_minimal_degree_vertex` and `edgeConnectivity_le_degree`.

```
lemma edgeConnectivity_le_minDegree' [Fintype V] [Nontrivial V]
    [DecidableRel G.Adj] : λ(G) ≤ δ(G) := by
  sorry
```

19. The *bowtie* consists of two triangles `0, 1, 2` and `0, 3, 4` sharing the vertex `0`. Instance search does not unfold the definition of `bowtie`, so we supply its `DecidableRel` instance by hand; this lets **`decide`** check properties of the bowtie.

The *bowtie*: two triangles `0, 1, 2` and `0, 3, 4` sharing the vertex `0`.

```
def bowtie : SimpleGraph (Fin 5) :=
  fromRel fun i j => (i, j) ∈ [(0, 1), (1, 2), (2, 0), (0, 3), (3, 4), (4, 0)]
deriving instance DecidableRel for (bowtie.Adj)
```

Show that `κ(bowtie) < λ(bowtie)`, so the first of Whitney's inequalities can be strict. Hint: the bowtie is 2-edge-connected by `isEdgeConnected_two_iff_forall_preconnected` and **`decide`**. It is not 2-vertex-connected, by `isVertexConnected_iff_induce` with `S = {0}` and **`decide`**. Finally use `le_edgeConnectivity_iff` and `le_vertexConnectivity_iff`.

```
example : κ(bowtie) < λ(bowtie) := by
  sorry
```

20. The *barbell* consists of two copies of `K[4]`, on `0, 1, 2, 3` and on `4, 5, 6, 7`, joined by the edge `3 — 4`. As for the bowtie, we supply its `DecidableRel` instance by hand.

The *barbell*: two copies of `K[4]`, on `0, 1, 2, 3` and `4, 5, 6, 7`, joined by the edge `3 — 4`.

```
def barbell : SimpleGraph (Fin 8) :=
  fromRel fun i j => (i < 4 ∧ j < 4) ∨ (4 ≤ i ∧ 4 ≤ j) ∨ (i = 3 ∧ j = 4)
deriving instance DecidableRel for (barbell.Adj)
```

Show that `λ(barbell) < δ(barbell)`, so the second of Whitney's inequalities can also be strict. Hint: the barbell is not 2-edge-connected, by `isEdgeConnected_two_iff_forall_preconnected` with `e = s(3, 4)` and **`decide`**. Every vertex has degree at least 3, by `le_minDegree_of_forall_le_degree` and **`decide`**.

```
example : λ(barbell) < δ(barbell) := by
  sorry
```

21. Show that the vertex connectivity of the hypercube `Q[n]` is exactly *n*. Hint: the lower bound is Exercise 16 (via `le_vertexConnectivity_iff`); for the upper bound use `vertexConnectivity_le_encard_neighborSet` at any vertex, together with `IsRegularOfDegree.hypercube`.

```
lemma vertexConnectivity_hypercube' (n : ℕ) : κ(Q[n]) = n := by
  sorry
```

22. Show that the edge connectivity of `Q[n + 1]` is `n + 1`, but that the edge connectivity of `Q[0]` is `⊤`. Hint: use Exercise 21 and `vertexConnectivity_le_edgeConnectivity`, then

edgeConnectivity_le_degree, which needs a Nontrivial (HypercubeVert (n + 1)) instance, and edgeConnectivity_of_subsingleton. Instance search deliberately does not see through HypercubeVert, so you will have to supply these instances yourself, for example with inferInstanceAs (Nontrivial (HypercubeVert n × Bool)).

```
lemma edgeConnectivity_hypercube_succ' (n : ℕ) : λ(Q[n + 1]) = n + 1 := by
  sorry

lemma edgeConnectivity_hypercube_zero' : λ(Q[0]) = ⊤ := by
  sorry
```

23. Show that in a forest the block of an edge u — v is G.subgraphOfAdj h, the subgraph consisting of that edge and its two endpoints. Hint: show that block-equivalent potential edges of a forest are equal, by induction on BlockEquivalent, and use blockMk_eq_blockMk_iff.

```
lemma IsAcyclic.toSubgraph_blockMk' (hG : G.IsAcyclic) {u v : V} (h : G.Adj u v) :
    h.block.toSubgraph = G.subgraphOfAdj h := by
  sorry
```

24. Show that for n ≥ 1 a graph is n-colourable iff each of its blocks is. Hint: the forward direction follows from Colorable.of_hom and Subgraph.hom. For the reverse direction use the De Bruijn–Erdős theorem (colorable_of_forall_finite_subgraph_colorable) to reduce to a finite subgraph H, which is colourable by colorable_of_forall_block provided its blocks are. Each block of H maps into a block of G along the copy H.coeCopy (Block.map_toSubgraph_le), and Subgraph.homOfMapLE turns this into a homomorphism.

```
theorem colorable_iff_forall_block' {n : ℕ} (hn : 1 ≤ n) :
    G.Colorable n ↔ ∀ B : G.Block, B.toSubgraph.coe.Colorable n := by
  sorry
```

11 Distance

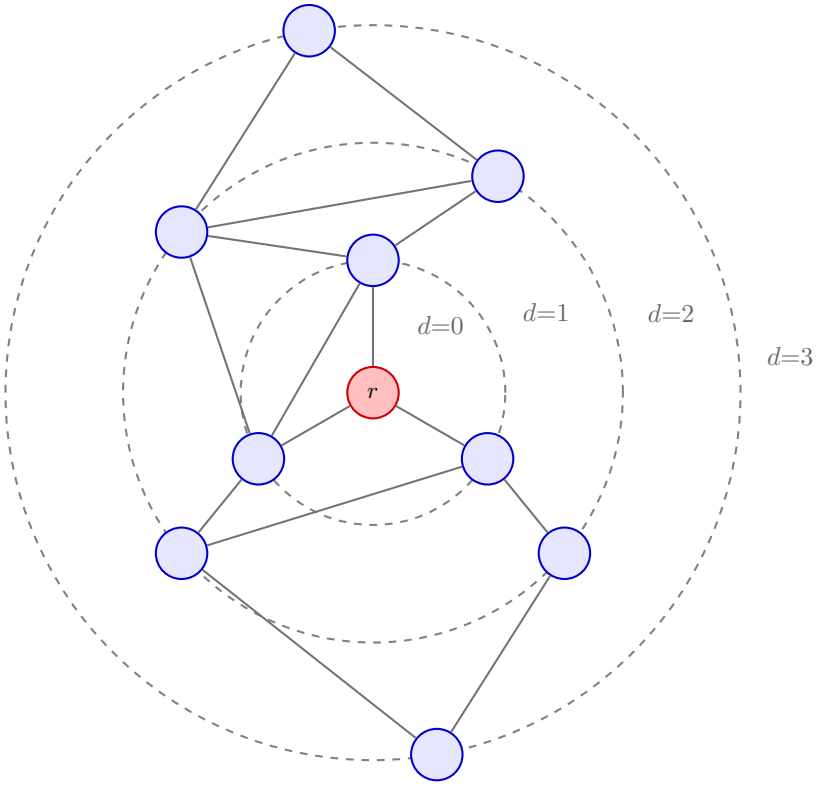
The *distance* between two vertices of a graph is the length of a shortest walk between them. In a connected graph it makes the vertex set into a metric space, and many important graph parameters are defined in terms of it: the *eccentricity* (`eccent`) of a vertex as well as the *diameter* (`diam`, `ediam`) and *radius* (`radius`) of a graph. Distance is also closely related to the *girth* (`girth`, `egirth`).

In this chapter we introduce Mathlib's distance `edist` and its basic properties, and show that the distance in the hypercube `Q[n]` is the Hamming distance (`edist_eq_hammingDist`). We study eccentricity, diameter and radius, with vertex-transitive graphs, the hypercube and the Mycielskian as examples. As an application of distance to colouring we prove *Albertson's theorem*: a precolouring of vertices that are pairwise far apart extends to a colouring with one extra colour (`Colorable.extend_precoloring`). We then focus on girth, which we compute for complete bipartite graphs, cycle graphs, hypercubes and Mycielskians.

The final section looks at counting vertices in the distance layers from a fixed vertex. This gives the *Moore bound* (`moore_bound`) and the *cage bound* (`cage_bound_odd`, `cage_bound_even`), which relate the number of vertices of a graph to its degrees, diameter and girth. Complete graphs and odd cycles show that both bounds can be attained. Rarer examples, the Petersen graph and the Hoffman–Singleton graph, appear in Algebraic Graph Theory (p. 360).

11.1 Graph Distance

The *distance* between two vertices `u` and `v` in a graph `G` is the length of a shortest walk from `u` to `v` (or infinity if no such walk exists). Mathlib provides two versions: `edist` takes values in $\mathbb{N}\infty$ (the extended naturals, where `⊤ = ∞`), and `dist` takes values in $\mathbb{N}$, with the convention that unreachable pairs are at distance `0`. In this chapter we will work almost entirely with `edist`, and pass to `dist` only when a natural number is needed; we explain why in edist vs dist (p. 334).



Breadth-first search layers from a root vertex partition `V` by graph distance

```
variable {V : Type*} {G : SimpleGraph V}
open Walk

#source SimpleGraph.edist -- def edist (u v : V) : ℕ∞ := ⨅ w : G.Walk u v, w.length
#source SimpleGraph.dist  -- def dist (u v : V) : ℕ := (G.edist u v).toNat
```

The basic properties of `edist` are all in Mathlib. It is the infimum of the lengths of the walks from `u` to `v`, so any walk gives an upper bound; it is zero iff the endpoints coincide and one iff they are adjacent; and it is symmetric and satisfies the triangle inequality. In other words `edist` makes the vertex set into an extended metric space.

```
#check SimpleGraph.edist_eq_sInf         -- G.edist u v = sInf {p.length | p}
#check SimpleGraph.edist_le              -- G.edist u v ≤ ↑p.length
#check SimpleGraph.edist_eq_zero_iff     -- G.edist u v = 0 ↔ u = v
#check SimpleGraph.edist_eq_one_iff_adj  -- G.edist u v = 1 ↔ G.Adj u v
#check SimpleGraph.edist_comm            -- G.edist u v = G.edist v u
#check SimpleGraph.edist_triangle        -- G.edist u w ≤ G.edist u v + G.edist v w
```

The distance is finite exactly when the two vertices are reachable from each other, in which case some walk realizes it. Adding edges can only decrease distances.

```
#check SimpleGraph.edist_ne_top_iff_reachable
#check SimpleGraph.Reachable.exists_walk_length_eq_edist
#check SimpleGraph.edist_anti  -- G ≤ G' → G'.edist u v ≤ G.edist u v
```

The $\mathbb{N}$-valued `dist` agrees with `edist` on reachable pairs, and in a connected graph every distance is realized by a walk.

```
#check SimpleGraph.Reachable.coe_dist_eq_edist
#check SimpleGraph.Connected.exists_walk_length_eq_dist
```

`edist` vs `dist`

Why do we prefer `edist`? The triangle inequality for `edist` holds for all vertices, with no hypotheses. For `dist` it can fail: if `u` and `v` are adjacent and `w` is an isolated vertex, then `G.dist u v = 1`, while `G.dist u w + G.dist w v = 0 + 0`, because `0` is also the value that `dist` takes on unreachable pairs. So Mathlib's triangle inequalities for `dist` need a connectivity or reachability hypothesis.

```
#check SimpleGraph.dist_eq_zero_iff_eq_or_not_reachable
#check SimpleGraph.Connected.dist_triangle
#check SimpleGraph.Reachable.dist_triangle_left
```

This is the convention for cardinalities from Counting in $\mathbb{N}\infty$: ENat.card (p. 68) again: `Nat.card α` is `(ENat.card α).toNat`, so an infinite type has `Nat.card` equal to `0`, just as `G.dist u v` is `(G.edist u v).toNat`, which sends `⊤` to `0`. Each $\mathbb{N}\infty$-valued parameter in this chapter is related to its $\mathbb{N}$-valued version in the same way.

So when should we use `dist`? Statements and hypotheses are best phrased with `edist`, but inside a proof `dist` is often more convenient. Once we know that `u` and `v` are reachable, `G.dist u v` is a natural number equal to their distance (`Reachable.coe_dist_eq_edist`), so **`grind`** works directly, as do subtraction, induction on the distance and indexing such as `getVert`. With `edist` we must first **`lift`** the distance to $\mathbb{N}$ or move between $\mathbb{N}$ and $\mathbb{N}\infty$ with **`norm_cast`**, as many of the proofs below do. Some Mathlib lemmas are also stated only for `dist`: to show that a shortest walk is a path we use `Walk.isPath_of_length_eq_dist`, which has no `edist` counterpart; the walk itself shows that its endpoints are reachable, so the junk value cannot arise. Finally, for a finite connected graph the $\mathbb{N}$-valued version is often the more natural statement, as in `diam_hypercube` below.

Distance under Homomorphisms and Isomorphisms

A graph homomorphism maps walks to walks of the same length, and so does not increase distances. Consequently a graph isomorphism preserves distances. Mathlib has both facts, as `Hom.edist_le` and `Iso.edist_eq`, and we will use them repeatedly: to show that automorphisms preserve eccentricities, and in Algebraic Graph Theory (p. 360) to transport the distance between two constructions of the hypercube.

```
example {W : Type*} {H : SimpleGraph W} (e : G →g H) (u v : V) :
    H.edist (e u) (e v) ≤ G.edist u v := e.edist_le ..
```

A graph isomorphism preserves the graph distance.

```
example {W : Type*} {H : SimpleGraph W} (e : G ≃g H) (u v : V) :
    H.edist (e u) (e v) = G.edist u v := e.edist_eq ..
```

Distance in Box Products

The distance in a box product is the sum of the distances in its two factors (`edist_boxProd`): a walk in `G □ H` moves in one coordinate at a time, so it projects to a walk in each factor, and conversely

walks in the two factors lift to walks in `G □ H` that can be appended. In order to use this in our hypercube construction `Q[n]` we also need distances in a complete graph (`edist_top`).

```
#check edist_boxProd  -- (G □ H).edist x y = G.edist x.1 y.1 + H.edist x.2 y.2
#check edist_top      -- (⊤ : SimpleGraph V).edist u v = if u = v then 0 else 1
```

Distance in the Hypercube

By definition `Q[n + 1]` is the box product `Q[n] □ ⊤` of `Q[n]` with the complete graph on `Bool` (`hypercube_succ`). So the box product formula `edist_boxProd` splits the distance in `Q[n + 1]` into the distance in `Q[n]` plus the distance in `⊤`. The latter is `0` or `1` according to whether the last coordinates agree (`edist_top`). This is exactly the recursion defining the Hamming distance `HypercubeVert.hammingDist`, so by induction the graph distance *is* the Hamming distance.

```
lemma edist_eq_hammingDist {n : ℕ} {u v : HypercubeVert n} :
    Q[n].edist u v = u.hammingDist v := by
  induction n with
  | zero => simp [HypercubeVert.hammingDist_zero, Subsingleton.elim u v]
  | succ n ih =>
    rw [hypercube_succ, edist_boxProd, ih, edist_top, HypercubeVert.hammingDist_succ]
    split_ifs <;> simp
```

In particular the Hamming distance inherits the properties of the graph distance, such as the triangle inequality. And since adjacent vertices are those at distance one (`edist_eq_one_iff_adj`), adjacency in `Q[n]` means Hamming distance one.

```
lemma hypercube_adj_iff_hammingDist_eq_one {n : ℕ} {u v : HypercubeVert n} :
    Q[n].Adj u v ↔ u.hammingDist v = 1 := by
  rw [← edist_eq_one_iff_adj, edist_eq_hammingDist, Nat.cast_eq_one]
```

Distance in Acyclic Graphs

In an acyclic graph (forest) there is at most one path between any two vertices (`IsAcyclic.subsingleton_path`), while between reachable vertices there is always a path whose length is their distance (`Reachable.exists_path_of_dist`). So in a forest the distance between two vertices joined by a path is simply the length of that path.

```
lemma IsAcyclic.edist_eq_length (hG : G.IsAcyclic) {u v : V} {p : G.Walk u v}
    (hp : p.IsPath) : G.edist u v = p.length := by
  obtain ⟨q, hq, hql⟩ := p.reachable.exists_path_of_dist
  obtain rfl : p = q :=
    congrArg Subtype.val ((hG.subsingleton_path u v).elim ⟨p, hp⟩ ⟨q, hq⟩)
  rw [← p.reachable.coe_dist_eq_edist, hql]
```

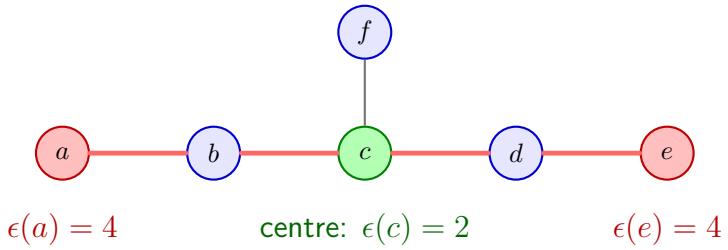
For example, the infinite path `pathNat` is a tree (`isTree_pathNat`). In Constructing Walks (p. 188) we built the walk `pathNat.path i j` of length `j`, and `pathNat.isPath_path` shows that it is a path. Hence the distance from `i` to `i + j` is `j`.

```
#check pathNat.length_path -- (pathNat.path i j).length = j
#check pathNat.isPath_path -- (pathNat.path i j).IsPath
```

```
lemma edist_pathNat (i j : ℕ) : pathNat.edist i (i + j) = j := by
  rw [isAcyclic_pathNat.edist_eq_length (pathNat.isPath_path i j),
    pathNat.length_path]
```

11.2 Eccentricity, Diameter and Radius

For a vertex `v`, its *eccentricity* is the supremum of the distances from `v` to the other vertices. The *diameter* is the supremum of the eccentricities, and the *radius* is their infimum. The vertices whose eccentricity equals the radius form the *centre* of the graph. As with the distance, Mathlib provides an $\mathbb{N}\infty$-valued diameter `ediam` and an $\mathbb{N}$-valued `diam`. The eccentricity and the radius come only in $\mathbb{N}\infty$-valued versions.



A graph of radius 2 and diameter 4 with a diametric path highlighted

```
#source SimpleGraph.eccent  -- .. : ℕ∞ := ⨆ v, G.edist u v
#source SimpleGraph.ediam   -- .. : ℕ∞ := ⨆ u, G.eccent u
#source SimpleGraph.diam    -- .. : ℕ := G.ediam.toNat
#source SimpleGraph.radius  -- .. : ℕ∞ := ⨅ u, G.eccent u
#source SimpleGraph.center  -- .. : Set α := {u | G.eccent u = G.radius}
```

The eccentricity and the diameter bound pairwise distances, and conversely a bound on all distances bounds the eccentricity and the diameter.

```
#check SimpleGraph.edist_le_eccent  -- G.edist u v ≤ G.eccent u
#check SimpleGraph.edist_le_ediam   -- G.edist u v ≤ G.ediam
#check SimpleGraph.eccent_le_iff    -- G.eccent u ≤ k ↔ ∀ v, G.edist u v ≤ k
#check SimpleGraph.ediam_le_iff     -- G.ediam ≤ k ↔ ∀ u v, G.edist u v ≤ k
```

A finite graph on a nonempty vertex type is connected iff its diameter is finite, in which case the diameter is attained by some pair of vertices.

```
#check SimpleGraph.connected_iff_ediam_ne_top
#check SimpleGraph.exists_edist_eq_ediam_of_ne_top
```

The finiteness hypothesis cannot be dropped. The infinite path `pathNat` is connected (`connected_pathNat`), but the distance from `0` to `n` is `n` (`edist_pathNat`), so its diameter is `⊤`. Its $\mathbb{N}$-valued diameter is therefore the junk value `0`.

In the proof, `ediam_eq_top` gives us some `n : ℕ∞` with `n < ⊤`, which we `lift` to $\mathbb{N}$.

```
lemma ediam_pathNat : pathNat.ediam = ⊤ := by
  refine ediam_eq_top.mpr fun n hn => ?_
  lift n to ℕ using hn.ne
  exact ⟨0, n + 1, by rw [← zero_add (n + 1), edist_pathNat]; norm_cast; grind⟩
```

```
lemma diam_pathNat : pathNat.diam = 0 :=
  diam_eq_zero_of_ediam_eq_top ediam_pathNat
```

The radius is at most the diameter, provided that there is at least one vertex: a graph with no vertices has radius `⊤`, an infimum over no vertices, but diameter `0`, a supremum over no vertices. And the diameter is at most twice the radius (by the triangle inequality via a central vertex).

```
#check SimpleGraph.radius_le_eccent         -- G.radius ≤ G.eccent u
#check SimpleGraph.radius_le_ediam          -- [Nonempty α] : G.radius ≤ G.ediam
#check SimpleGraph.radius_eq_top_of_isEmpty -- [IsEmpty α] : G.radius = ⊤
#check SimpleGraph.ediam_le_two_mul_radius  -- G.ediam ≤ 2 * G.radius
```

The complete graph on at least two vertices has diameter and radius `1`, and it is the only graph with diameter `1`.

```
#check SimpleGraph.diam_top      -- [Nontrivial α] : (⊤ : SimpleGraph α).diam = 1
#check SimpleGraph.radius_top    -- [Nontrivial α] : (⊤ : SimpleGraph α).radius = 1
#check SimpleGraph.ediam_eq_one  -- [Nontrivial α] : G.ediam = 1 ↔ G = ⊤
```

Distance Invariants under Isomorphism

Since isomorphisms preserve distances they preserve eccentricities, and hence the diameter and the radius are isomorphism invariants.

```
lemma Iso.eccent_eq {W : Type*} {H : SimpleGraph W} (e : G ≃g H) (u : V) :
    H.eccent (e u) = G.eccent u :=
  (e.toEquiv.iSup_comp (g := H.edist (e u))).symm.trans (iSup_congr (e.edist_eq u))

lemma Iso.ediam_eq {W : Type*} {H : SimpleGraph W} (e : G ≃g H) :
    G.ediam = H.ediam :=
  (iSup_congr fun u => (e.eccent_eq u).symm).trans
    (e.toEquiv.iSup_comp (g := H.eccent))

lemma Iso.radius_eq {W : Type*} {H : SimpleGraph W} (e : G ≃g H) :
    G.radius = H.radius :=
  (iInf_congr fun u => (e.eccent_eq u).symm).trans
    (e.toEquiv.iInf_comp (g := H.eccent))
```

In a vertex-transitive graph (`IsVertexTransitive`) all eccentricities coincide, so the radius equals the diameter and every vertex is central. We use this below to compute the radius of the hypercube from its diameter.

```
lemma IsVertexTransitive.eccent_eq (h : G.IsVertexTransitive) (u v : V) :
    G.eccent u = G.eccent v := by
  obtain ⟨f, hf⟩ := h u v
  rw [← hf, f.eccent_eq]

lemma IsVertexTransitive.radius_eq_ediam [Nonempty V] (h : G.IsVertexTransitive) :
    G.radius = G.ediam :=
  radius_eq_ediam_iff.mpr
    ⟨G.eccent (Classical.arbitrary V), fun u => h.eccent_eq u _⟩
```

```
lemma IsVertexTransitive.center_eq_univ [Nonempty V] (h : G.IsVertexTransitive) :
    G.center = Set.univ :=
  center_eq_univ_iff_radius_eq_ediam.mpr h.radius_eq_ediam
```

Diameter of Box Products

Since distances add in a box product, so do diameters, provided both factors have a vertex (if one factor has no vertices then neither does the product, and its diameter is `0`). The upper bound is immediate from `edist_boxProd`. For the lower bound we write each diameter as a single supremum over pairs of vertices (`ediam_def`) and use `ENat.iSup_add_iSup_le`, which bounds a sum of two suprema term by term.

```
lemma ediam_boxProd {α β : Type*} {G : SimpleGraph α} {H : SimpleGraph β}
    [Nonempty α] [Nonempty β] : (G □ H).ediam = G.ediam + H.ediam := by
  refine (ediam_le_of_edist_le fun x y => ?_).antisymm ?_
  · rw [edist_boxProd]
    exact add_le_add edist_le_ediam edist_le_ediam
  · rw [ediam_def, ediam_def]
    refine ENat.iSup_add_iSup_le fun p q => ?_
    simpa using edist_le_ediam (G := G □ H) (u := (p.1, q.1)) (v := (p.2, q.2))
```

Diameter and Radius of the Hypercube

Diameters add under box products (`ediam_boxProd`) and the complete graph on `Bool` has diameter `1` (`ediam_top`), so by induction the diameter of `Q[n]` is n. The base case is the one-vertex graph `Q[0]`, whose diameter is `0` (`ediam_eq_zero_of_subsingleton`), conveniently we already gave `HypercubeVert 0` a `Subsingleton` instance, `hypercubeZeroSubsingleton`.

```
lemma ediam_hypercube {n : ℕ} : Q[n].ediam = n := by
  induction n with
  | zero => exact ediam_eq_zero_of_subsingleton
  | succ n ih => rw [hypercube_succ, ediam_boxProd, ih, ediam_top, Nat.cast_succ]

lemma diam_hypercube {n : ℕ} : Q[n].diam = n := by
  rw [SimpleGraph.diam, ediam_hypercube, ENat.toNat_natCast]
```

Since the hypercube is vertex-transitive (`isVertexTransitive_hypercube`) its radius equals its diameter, and every vertex is central.

```
lemma radius_hypercube {n : ℕ} : Q[n].radius = n := by
  rw [(isVertexTransitive_hypercube n).radius_eq_ediam, ediam_hypercube]
```

Distances in the Mycielskian

If G is a graph without isolated vertices then, whatever the diameter of G, its Mycielskian has radius at most 2 and diameter at most 4. The apex `none` is adjacent to every right copy v^{r}, and each left copy v^{l} is at distance at most 2 from the apex, via the right copy of any neighbour of v.

```
@[simp]
lemma edist_mycielskian_none_right {v : V} : mμ(G).edist none v^r = 1 := by
```

```
  simp [edist_eq_one_iff_adj]

lemma edist_mycielskian_none_left_le_two {u v : V} (h : G.Adj u v) :
    mμ(G).edist none v[l] ≤ 2 := by
  have h1 : mμ(G).Adj none u[r] := by simp
  have h2 : mμ(G).Adj u[r] v[l] := by simpa
  exact edist_le (cons h1 ◁ cons h2 nil)

theorem radius_mycielskian_le_two (h : ∀ v : V, ¬G.IsIsolated v) :
    mμ(G).radius ≤ 2 := by
  refine radius_le_eccent.trans ((eccent_le_iff none _).mpr ?_)
  rintro (_ | ⟨v | v⟩)
  · simp
  · obtain ⟨u, hu⟩ := exists_adj_iff_not_isIsolated.2 (h v)
    exact edist_mycielskian_none_left_le_two hu.symm
  · simp
```

The diameter bound follows from `ediam_le_two_mul_radius`, which leaves the goal `2 * mμ(G).radius ≤ 4`. We close it with **`grw`** ("generalized rewrite"), which rewrites with an inequality instead of an equation, using the same monotonicity facts as **`gcongr`**: since multiplication by `2` is monotone, the bound `mμ(G).radius ≤ 2` turns the goal into `2 * 2 ≤ 4`, which holds by `rfl`.

```
theorem ediam_mycielskian_le_four (h : ∀ v : V, ¬G.IsIsolated v) :
    mμ(G).ediam ≤ 4 :=
  ediam_le_two_mul_radius.trans ◁ by grw [radius_mycielskian_le_two h]; rfl
```

11.3 Albertson's Precolouring Extension Theorem

Albertson's precolouring extension theorem [Albertson 1998] says that if G is k-colourable and s is a set of vertices that are pairwise at distance at least 4, then any assignment `f : s → Fin (k + 1)` of colours to the vertices of s extends to a proper `(k + 1)`-colouring of G. (In general a *precolouring* must be a proper colouring of the subgraph induced by s, but here no two vertices of s are adjacent, so every assignment of colours is allowed.)

The hypothesis is stated with `edist`. Had we written `4 ≤ G.dist u w` instead, the junk value `0` for unreachable pairs would forbid precoloured vertices in different components, giving a strictly weaker theorem.

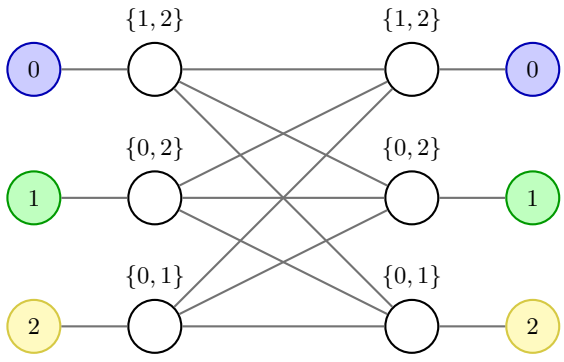
Take any k-colouring `c` of G, so the extra colour `Fin.last k` is unused. Build the extension c_1 by recolouring only near s: every `w ∈ s` gets its target colour `f w`, and whenever `f w` is one of the original k colours, each neighbour of `w` that already used that colour is swapped to the spare colour `Fin.last k`. The distance hypothesis guarantees that the recoloured neighbourhoods of distinct vertices of s are non-adjacent (any edge between them would give a walk of length ≤ 3 between two vertices of s), so no new monochromatic edge is created.

Albertson's precolouring extension theorem: if G is k-colourable, then any assignment of colours from `Fin (k + 1)` to a set of vertices that are pairwise at distance at least 4 extends to a `(k + 1)`-colouring of G.

```
theorem Colorable.extend_precoloring {k : ℕ} (hk : G.Colorable k) {s : Set V}
    (hs : s.Pairwise fun u w ↦ 4 ≤ G.edist u w) (f : s → Fin (k + 1)) :
    ∃ C : G.Coloring (Fin (k + 1)), ∀ w : s, C w = f w := by
```

```
  classical
  obtain ⟨c⟩ := hk
  -- Two vertices of s joined by a walk of length at most 3 are equal; we use
  -- this for walks of length 1, 2 and 3.
  have close {a b : V} (ha : a ∈ s) (hb : b ∈ s) (p : G.Walk a b)
      (hp : p.length ≤ 3) : a = b := by
    by_contra hne
    have := (hs ha hb hne).trans (edist_le p)
    grind [Nat.cast_le.mp this]
  have h1 {a b : V} (ha : a ∈ s) (hb : b ∈ s) (h : G.Adj a b) : False :=
    h.ne (close ha hb (cons h nil) (by simp))
  have h2 {a b x : V} (ha : a ∈ s) (hb : b ∈ s) (h₁ : G.Adj a x) (h₂ : G.Adj x b) :
      a = b := close ha hb (cons h₁ (cons h₂ nil)) (by simp)
  have h3 {a b x y : V} (ha : a ∈ s) (hb : b ∈ s) (h₁ : G.Adj a x)
      (h₂ : G.Adj x y) (h₃ : G.Adj y b) : a = b :=
    close ha hb (cons h₁ (cons h₂ (cons h₃ nil))) (by simp)
  -- P v: a neighbour w ∈ s of v has target colour f w equal to c v.
  let P (v : V) : Prop := ∃ w : s, G.Adj w v ∧ f w = (c v).castSucc
  -- A vertex v ∈ s gets its target colour f v. Any other v gets the spare
  -- colour Fin.last k if P v holds, and otherwise keeps its colour c v (cast
  -- into Fin (k + 1)).
  let c₁ : V → Fin (k + 1) := fun v ↦
    if hv : v ∈ s then f ⟨v, hv⟩ else if P v then Fin.last k else (c v).castSucc
  refine ⟨Coloring.mk c₁ fun {x y} hxy => ?_, fun w => by simp [c₁, Coloring.mk]⟩
  simp only [c₁, P]
  split_ifs <;> grind [c.valid hxy, Adj.symm]
```

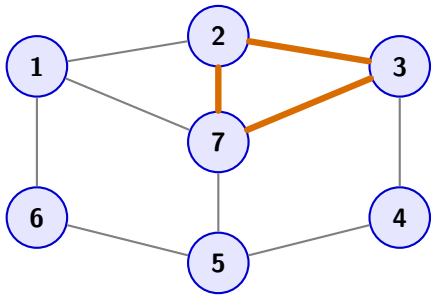
Distance at least 4 cannot be replaced by 3. Take `K[3, 3]` and attach a pendant vertex to each of its six vertices; the pendants are pairwise at distance at least 3. On each side, colour the three pendants 0, 1 and 2, so the vertices on that side can only use the colours {1, 2}, {0, 2} and {0, 1} respectively. No colour is available to all three, so each side needs to use at least two colours; but the two sides of `K[3, 3]` need disjoint sets of colours, which is impossible with three colours. The graph is bipartite, so this precolouring of a 2-colourable graph does not extend to a 3-colouring.



`K[3, 3]` with a precoloured pendant at each vertex, and the colours still available to each vertex of `K[3, 3]`

11.4 Girth

The *girth* of a graph is the length of its shortest cycle. Mathlib again provides two versions: `G.egirth : ℕ∞`, which is `⊤` for an acyclic graph, and `G.girth : ℕ`, which is `0` for an acyclic graph. Girth is the natural counterpart of the diameter: if the maximum degree is bounded, small diameter forces *few* vertices, while if the minimum degree is bounded below, large girth forces *many* vertices. These are the Moore and cage bounds of Moore and Cage Bounds (p. 346). Girth also enters the Mycielski construction: `mμ(G)` raises the chromatic number (`chromaticNumber_mycielskian`) while staying triangle-free (`CliqueFree.mycielskian`), and below we compute its girth exactly.



A graph of girth 3, with a shortest cycle highlighted

```
#check egirth  -- G.egirth : ℕ∞
#check girth   -- G.girth : ℕ
```

A graph is acyclic exactly when its `egirth` is `⊤`, or equivalently when its `girth` is `0`. The length of any cycle is an upper bound for `egirth`. Every cycle has length at least 3, so `egirth` is always at least 3, but `girth` is at least 3 only for a graph with a cycle, because of the junk value `0`.

```
#check egirth_eq_top                  -- G.egirth = ⊤ ↔ G.IsAcyclic
#check girth_eq_zero                  -- G.girth = 0 ↔ G.IsAcyclic
#check Walk.IsCycle.egirth_le_length  -- w.IsCycle → G.egirth ≤ ↑w.length
#check three_le_egirth                -- 3 ≤ G.egirth
#check three_le_girth                 -- ¬G.IsAcyclic → 3 ≤ G.girth
```

Girth and diameter are related: a shortest cycle has length at most `2 * diam + 1`, since otherwise two antipodal vertices of the cycle would be joined by a shorter path, producing a shorter cycle. Mathlib states this in three ways, and their hypotheses show the junk values at work.

- For `egirth` we must exclude acyclic graphs, whose `egirth` is `⊤`.
- For `girth` no hypothesis is needed: the junk value `0` of an acyclic graph satisfies the bound anyway.
- To use `diam` instead of `ediam` we must assume that the diameter is finite, since otherwise `diam` is `0` and the right-hand side is just `1`.

So a junk value sometimes removes a hypothesis and sometimes adds one. The same happens for containment: a graph containing a copy of `H` has girth at most that of `H`, and the `girth` version needs `H` to have a cycle. Finally, isomorphisms preserve the girth.

```
#check egirth_le_two_mul_ediam_add_one -- ¬G.IsAcyclic → G.egirth ≤ 2 * G.ediam + 1
#check girth_le_two_mul_ediam_add_one  -- G.girth ≤ 2 * G.ediam + 1
#check girth_le_two_mul_diam_add_one   -- G.ediam ≠ ⊤ → G.girth ≤ 2 * G.diam + 1
```

```
#check IsContained.egirth_le          -- H ⊑ G → G.egirth ≤ H.egirth
#check IsContained.girth_le           -- H ⊑ G → ¬H.IsAcyclic → G.girth ≤ H.girth
#check Iso.egirth_eq                  -- G ≃g H → G.egirth = H.egirth
```

Excluding Short Cycles

A graph has no cycle of length at most k exactly when it contains no copy of `C[n]` for `3 ≤ n ≤ k`, i.e. when it is `C[n]`-free (in the sense of `Free`). Mathlib's `cycleGraph_isContained_iff` says that G contains a copy of `C[n]` (for `n ≥ 3`) iff it has a cycle of length `n`. (Every cycle has length at least `3`, so only `n ≥ 3` matters.) Excluding these copies therefore bounds the girth from below.

```
lemma le_egirth_of_free {k : ℕ} (h : ∀ n, 3 ≤ n → n ≤ k → C[n].Free G) :
    k + 1 ≤ G.egirth := by
  rw [le_egirth]
  intro a w hw
  by_contra! hlt
  have hk : w.length ≤ k := by exact_mod_cast Order.le_of_lt_add_one hlt
  have h3 := hw.three_le_length
  exact h _ h3 hk ((cycleGraph_isContained_iff (by grind)).mpr ⟨_, _, hw, rfl⟩)
```

Being triangle-free is the case `n = 3`: `C[3]` is the complete graph on three vertices (`cycleGraph_three_eq_top`), and Mathlib's `cliqueFree_iff_free_top_fin` identifies `G.CliqueFree n` with being `K[n]`-free.

```
lemma cliqueFree_three_iff_free_cycleGraph : G.CliqueFree 3 ↔ C[3].Free G := by
  rw [cycleGraph_three_eq_top, cliqueFree_iff_free_top_fin]
```

So a triangle-free graph has girth at least `4`, and if it is also `C[4]`-free then its girth is at least `5`.

```
lemma four_le_egirth_of_cliqueFree_three (h : G.CliqueFree 3) : 4 ≤ G.egirth :=
  le_egirth_of_free fun n h3 hn => by
    obtain rfl : n = 3 := by grind
    exact cliqueFree_three_iff_free_cycleGraph.mp h

lemma five_le_egirth_of_cliqueFree_three_of_free (h3 : G.CliqueFree 3)
    (h4 : C[4].Free G) : 5 ≤ G.egirth :=
  le_egirth_of_free fun n h3n hn => by
    obtain rfl | rfl : n = 3 ∨ n = 4 := by grind
    exacts [cliqueFree_three_iff_free_cycleGraph.mp h3, h4]
```

A graph G is `C[4].Free` precisely when any two distinct vertices have at most one common neighbour. If distinct vertices `x` and `y` had two distinct common neighbours `u` and `v`, then `x`, `u`, `y` and `v` would be four distinct vertices (adjacent vertices are distinct), so `![x, u, y, v]` would be a copy of `C[4]`. Conversely, a copy `f` of `C[4]` gives distinct vertices `f 0` and `f 2` with two distinct common neighbours `f 1` and `f 3`.

```
lemma cycleGraph_four_free_iff_common_neighbor_unique :
    C[4].Free G ↔
      ∀ x y u v, G.Adj x u → G.Adj u y → G.Adj x v → G.Adj v y → x ≠ y → u = v := by
  refine ⟨fun hf x y u v hxu huy hxv hvy hxy => ?_, fun h => ?_⟩
  · by_contra huv
    have hinj : Function.Injective ![x, u, y, v] := by
      intro i j hij
```

```
    fin_cases i <;> fin_cases j <;> simp_all
  refine hf ⟨⟨![x, u, y, v], fun {i j} hij => ?_⟩, hinj⟩⟩
  fin_cases i <;> fin_cases j <;>
    first | exact absurd hij (by decide) | simp [*, Adj.symm]
· rintro ⟨f⟩
  have hadj {i j : Fin 4} (ha : C[4].Adj i j) : G.Adj (f i) (f j) :=
    f.toHom.map_adj ha
  have heq := h (f 0) (f 2) (f 1) (f 3) (hadj (by decide))
    (hadj (by decide)) (hadj (by decide)) (hadj (by decide))
    (fun h => absurd (f.injective h) (by decide))
  exact absurd (f.injective heq) (by decide)
```

Girth of Complete Bipartite Graphs

A bipartite graph is triangle-free, so its girth is at least 4. When both sides have at least two vertices, the complete bipartite graph contains a copy of C[4]: the vertices Sum.inl 0 and Sum.inl 1 have the two common neighbours Sum.inr 0 and Sum.inr 1, so cycleGraph_four_free_iff_common_neighbor_unique applies. A graph containing a copy of H has girth at most that of H (Mathlib's IsContained.egirth_le), and C[4] has girth at most 4, since it contains the cycle cycleGraph.cycle 1 of length 4 (cycleGraph.isCycle_cycle). So the girth is exactly 4.

```
lemma egirth_completeBipartiteGraph (s t : ℕ) : K[s + 2, t + 2].egirth = 4 := by
  -- bipartite, so triangle-free
  have h2 : K[s + 2, t + 2].Colorable 2 :=
    (completeBipartiteGraph.bicoloring _ _).colorable
  have h3 : K[s + 2, t + 2].CliqueFree 3 := h2.cliqueFree (by norm_num)
  -- .inl 0 and .inl 1 have the two common neighbours .inr 0 and .inr 1
  -- (the four adjacencies and .inl 0 ≠ .inl 1 are checked by simp)
  have h4 : C[4] ⊑ K[s + 2, t + 2] := not_free.mp fun h => by
    simpa using cycleGraph_four_free_iff_common_neighbor_unique.mp h
      (.inl 0) (.inl 1) (.inr 0) (.inr 1) (by simp) (by simp) (by simp) (by simp)
      (by simp)
  refine (four_le_egirth_of_cliqueFree_three h3).antisymm' ◁ h4.egirth_le.trans ?_
  simpa using (cycleGraph.isCycle_cycle (n := 1)).egirth_le_length
```

Girth and Diameter of Cycle Graphs

The cycle graph C[n + 3] contains the cycle cycleGraph.cycle n of length n + 3, so its girth is at most n + 3. For the reverse inequality we use the fact from Cycles in Graphs of Maximum Degree Two (p. 284) that in a connected graph in which every vertex has degree at most 2, every cycle is a Hamiltonian cycle (Walk.IsCycle.isHamiltonianCycle_of_degree_le_two). The cycle graph is connected (connected_cycleGraph_add_one) and 2-regular (Mathlib's cycleGraph_degree_three_le). A Hamiltonian cycle has length equal to the number of vertices (Walk.IsHamiltonianCycle.length_eq), so every cycle in C[n + 3] has length exactly n + 3.

```
lemma egirth_cycleGraph (n : ℕ) : (cycleGraph (n + 3)).egirth = n + 3 := by
  refine le_antisymm ?_ ?_
  · simpa using cycleGraph.isCycle_cycle.egirth_le_length
  · rw [le_egirth]
    intro a w hw
    -- C[n + 3] is connected and 2-regular, so the cycle w is Hamiltonian
```

```
    have hham := hw.isHamiltonianCycle_of_degree_le_two connected_cycleGraph_add_one
      fun _ => cycleGraph_degree_three_le.le
    simp [hham.length_eq, add_assoc]
```

The girth also determines the diameter of `C[n + 3]`. In any graph with a cycle the girth is at most `2 * ediam + 1` (`egirth_le_two_mul_ediam_add_one`), which here gives the lower bound `(n + 3) / 2 ≤ ediam`. For the upper bound we use the vertex-transitivity of cycle graphs (`isVertexTransitive_cycleGraph`): the diameter is the radius, so it is at most the eccentricity of the vertex `0`, and going one way or the other around `cycleGraph.cycle n` reaches any vertex `a` from `0` in `a` or `n + 3 - a` steps.

```
lemma ediam_cycleGraph (n : ℕ) : (cycleGraph (n + 3)).ediam = ((n + 3) / 2 : ℕ) := by
  refine le_antisymm ?_ ?_
  · rw [← (isVertexTransitive_cycleGraph _).radius_eq_ediam]
    refine (radius_le_eccent (u := 0)).trans ((eccent_le_iff _ _).mpr fun v => ?_)
    obtain ⟨a, ha⟩ := v
    -- cycleGraph.cycle n runs backwards: it reaches a after n + 3 - a steps
    have hv : (cycleGraph.cycle n).getVert (n + 3 - a) = ⟨a, ha⟩ := by
      simp [cycleGraph.getVert_cycle (by grind : n + 3 - a ≤ n + 3),
        Nat.mod_eq_of_lt ha, Nat.sub_sub_self ha.le]
    have h₁ := edist_le (((cycleGraph.cycle n).take (n + 3 - a)).copy rfl hv)
    have h₂ := edist_le (((cycleGraph.cycle n).drop (n + 3 - a)).copy hv rfl).reverse
    simp only [length_copy, length_reverse, take_length, drop_length,
      cycleGraph.length_cycle] at h₁ h₂
    rcases le_total a (n + 3 - a) with h | h
    · exact h₂.trans (Nat.cast_le.mpr (by grind))
    · exact h₁.trans (Nat.cast_le.mpr (by grind))
  · have h := egirth_le_two_mul_ediam_add_one (G := cycleGraph (n + 3))
      fun hG => hG _ cycleGraph.isCycle_cycle
    rw [egirth_cycleGraph] at h
    lift (cycleGraph (n + 3)).ediam to ℕ
      using connected_iff_ediam_ne_top.mp connected_cycleGraph_add_one with m
    norm_cast at h ⊢
    grind
```

Girth of the Hypercube

The hypercube is triangle-free (`cliqueFree_hypercube_three`), so its girth is at least 4, and for `n ≥ 2` it contains a 4-cycle. In the exercises on walks (Exercises Walks (p. 239)) we showed that `Q[n]` is Hamiltonian for `n ≠ 1` (`isHamiltonian_hypercube`), and a Hamiltonian cycle in `Q[2]`, which has 4 vertices, is a 4-cycle. Each larger cube contains a copy of the previous one, via the embedding `hypercube.embedFalse`, so by `IsContained.egirth_le` again its girth is at most that of the smaller cube.

```
lemma egirth_hypercube {n : ℕ} : egirth Q[n + 2] = 4 := by
  refine
    (four_le_egirth_of_cliqueFree_three (cliqueFree_hypercube_three _)).antisymm' ?_
  induction n with
  | zero =>
    have ⟨_, _, hp⟩ := isHamiltonian_hypercube (n := 2) (by simp) (by simp)
    exact hp.egirth_le_length.trans (by simp [hp.length_eq])
  | succ n ih =>
```

```
    exact (IsContained.egirth_le ⟨(hypercube.embedFalse _).toCopy⟩).trans ih
```

Girth of the Mycielskian

The Mycielskian of a triangle-free graph is triangle-free, so its girth is at least 4. Whether it is exactly 4 depends on whether G has two edges sharing an endpoint: if it does, the two edges give a 4-cycle through the apex; if not, the edges of G form a matching, and then the girth of $\mu(G)$ is 5 provided G has an edge. (If G has no edges then $\mu(G)$ is a star centred at the apex plus isolated vertices, so it is acyclic.)

The edges of G form a matching: no two edges share an endpoint, that is, every vertex has at most one neighbour.

```
abbrev EdgesFormMatching (G : SimpleGraph V) : Prop :=
  ∀ x y z : V, G.Adj x y → G.Adj x z → y = z
```

A pair of edges with a common endpoint, $u - v$ and $u - w$, gives rise to a walk in the Mycielskian, $\mathrm{none} - v^{\mathrm{r}} - u^{\mathrm{l}} - w^{\mathrm{r}} - \mathrm{none}$. (If $v \neq w$ this is a 4-cycle.)

```
def mycielskianFourCycle {u v w : V} (h1 : G.Adj u v) (h2 : G.Adj u w) :
    μ(G).Walk none none :=
  cons (by simp : μ(G).Adj none v.r) ◁ cons (by simp [h1.symm])
    ◁ cons (by simpa : μ(G).Adj u.l w.r) ◁ cons (by simp) nil

lemma isCycle_mycielskianFourCycle {u v w : V} (h1 : G.Adj u v) (h2 : G.Adj u w)
    (h : v ≠ w) : (mycielskianFourCycle h1 h2).IsCycle := by
  simp [mycielskianFourCycle, cons_isCycle_iff, h]

@[simp]
lemma length_mycielskianFourCycle {u v w : V} (h1 : G.Adj u v) (h2 : G.Adj u w) :
    (mycielskianFourCycle h1 h2).length = 4 := rfl
```

If G is triangle-free and its edges do not form a matching, then $\mu(G)$ has girth 4.

```
lemma egirth_mycielskian_eq_four (h3 : G.CliqueFree 3)
    (hnm : ¬G.EdgesFormMatching) : egirth μ(G) = 4 := by
  obtain ⟨x, y, z, h1, h2, hvw⟩ : ∃ x y z, G.Adj x y ∧ G.Adj x z ∧ y ≠ z := by grind
  refine (four_le_egirth_of_cliqueFree_three h3.mycielskian).antisymm' ?_
  simpa using (isCycle_mycielskianFourCycle h1 h2 hvw).egirth_le_length
```

If the edges of G form a matching then $\mu(G)$ is `C[4].Free`. We use `cycleGraph_four_free_iff_common_neighbor_unique`: given distinct vertices `x` and `y` with common neighbours `u` and `v`, we split into cases according to whether each of `x`, `y`, `u` and `v` is the apex, a left copy or a right copy. The adjacency rules of the Mycielskian dispose of most of the 81 cases, and `grind` finishes the rest.

```
lemma cycleGraph_four_free_mycielskian (hmatch : G.EdgesFormMatching) :
    C[4].Free μ(G) := by
  apply cycleGraph_four_free_iff_common_neighbor_unique.mpr
  -- each of x, y, u and v is the apex, a left copy or a right copy: 81 cases
  rintro (_ | (x | x)) (_ | (y | y)) (_ | (u | u)) (_ | (v | v)) h1 h2 h3 h4 hxy <;>
    simp_all only [mycielskian_adj] <;> -- 12 goals remain
    grind [Adj.symm]
```

A graph in which no two edges share a common endpoint is triangle-free.

```
lemma EdgesFormMatching.cliqueFree_three (hmatch : G.EdgesFormMatching) :
    G.CliqueFree 3 := fun t ht => by
  classical
  obtain ⟨a, b, c, hab, hac, hbc, -⟩ := is3Clique_iff.1 ht
  exact hbc.ne (hmatch a b c hab hac)
```

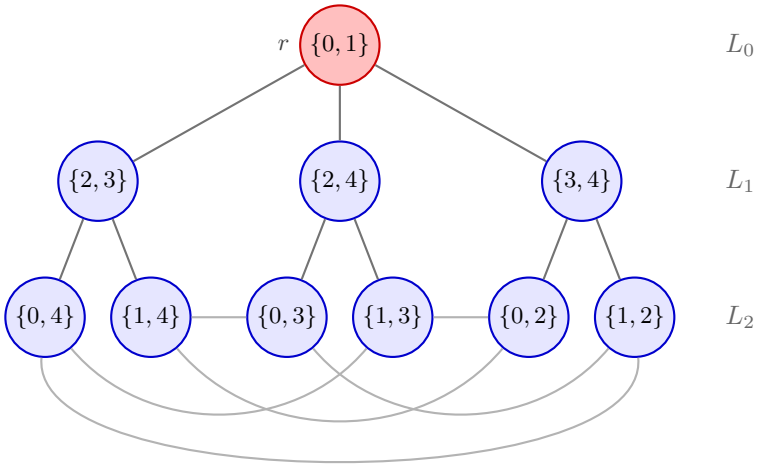
If G has an edge and its edges form a matching, then $\mu(G)$ has girth 5.

```
lemma egirth_mycielskian_eq_five {u v : V} (hadj : G.Adj u v)
    (hmatch : G.EdgesFormMatching) : egirth mμ(G) = 5 := by
  refine le_antisymm ?_ (five_le_egirth_of_cliqueFree_three_of_free
    hmatch.cliqueFree_three.mycielskian (cycleGraph_four_free_mycielskian hmatch))
  simpa [mycielskianClosed] using
    (isCycle_mycielskianClosed_of_adj hadj).egirth_le_length
```

The 5-cycle of `isCycle_mycielskianClosed_of_adj` exists for every edge of G, so the Mycielskian of any graph with an edge has girth at most 5, however large the girth of G. So Mycielski's construction gives triangle-free graphs of large chromatic number, but not graphs of large girth and large chromatic number. That such graphs exist is a famous theorem of Erdős [Erdős 1959], proved by the probabilistic method.

11.5 Moore and Cage Bounds*

Fix a vertex `r` (for *root*). The *distance layers* around `r` are the sets of vertices at distance exactly `0, 1, 2, …` from `r`: they are the levels of a breadth-first search tree rooted at `r`, as in the figure below of the Petersen graph (`petersenGraph`). Comparing the sizes of consecutive layers gives two classical bounds relating the number of vertices of a graph to its degrees, its diameter and its girth.



The `petersenGraph` by distance from the root `r = {0, 1}`

- The *Moore bound* [Hoffman–Singleton 1960]: a graph with maximum degree at most `d + 1` and diameter at most `k` has at most `1 + (d + 1) * (1 + d + ⋯ + d ^ (k - 1))` vertices.
- The *cage bound*: a graph with minimum degree at least `d + 1` and girth at least `2 * k + 1` has at least `1 + (d + 1) * (1 + d + ⋯ + d ^ (k - 1))` vertices.

Note that we write the degree bounds as `d + 1` so that the formulas do not use truncated subtraction in $\mathbb{N}$.

Proof sketch: write `L i` for the `i`-th layer around `r`, so `L 0 = {r}` and `L 1` is the neighbourhood of `r`.

- Moore: each `u ∈ L i` with `i ≥ 1` has a neighbour in `L (i-1)`, so it has at most `d` neighbours in `L (i+1)`. Hence `#L (i+1) ≤ d * #L i`, and if the diameter is at most `k` then the layers `L 0, …, L k` cover `V`.
- Cage: if the girth is at least `2 * k + 1` then the graph looks like a tree near `r`. For `1 ≤ i < k`, all but one of the neighbours of `u ∈ L i` lie in `L (i+1)`, and two vertices of `L i` have no common neighbour there, since otherwise there would be a cycle of length at most `2 * i + 2 ≤ 2 * k`. Hence `#L (i+1) ≥ d * #L i`, and the disjoint layers `L 0, …, L k` contain the claimed number of vertices.

Distance Layers

We need two simple facts about distances from a fixed vertex `r`. Adjacent vertices have distances from `r` that differ by at most one, and a vertex at distance `i + 1` from `r` has a neighbour at distance `i`, namely the second-to-last vertex of a shortest walk from `r`.

```
lemma Adj.edist_le_add_one {u v : V} (r : V) (h : G.Adj u v) :
    G.edist r v ≤ G.edist r u + 1 := by
  simpa [edist_eq_one_iff_adj.mpr h] using
    G.edist_triangle (u := r) (v := u) (w := v)

lemma exists_adj_edist_eq {r v : V} (i : ℕ) (h : G.edist r v = i + 1) :
    ∃ u, G.Adj u v ∧ G.edist r u = i := by
  -- a shortest walk from v back to r starts with an edge v — u
  obtain ⟨p, hp⟩ := exists_walk_of_edist_eq_coe (k := i + 1)
    (by rw [edist_comm]; exact_mod_cast h)
  cases p with
  | nil => simp at hp
  | @cons _ u _ hadj q =>
    refine ⟨u, hadj.symm, le_antisymm ?_ ?_⟩
    · rw [edist_comm]
      exact (edist_le q).trans_eq (by simpa using hp)
    · have := hadj.symm.edist_le_add_one r
      rwa [h, ENat.add_le_add_iff_right ENat.one_ne_top] at this
```

To count vertices in each layer we package them as `Finset`s, so we assume that the vertex type is finite whenever we use layers. For our purposes this isn't a real restriction: the cage bounds hold trivially for infinite graphs, and we show in Finite Eccentricity (p. 348) that the hypotheses of the Moore bound force the vertex type to be finite. The predicate `G.edist r v = i` is decidable, since $\mathbb{N}\infty$ has decidable equality, so the set-builder notation produces a `Finset`. But `edist` is defined as an infimum over all walks, which Lean cannot compute, so the definition must be marked `noncomputable`.

Since `layer` is noncomputable anyway, a `Fintype V` argument would be pointless, so we assume only `[Finite V]` and pick a `Fintype` instance inside the definition with `Fintype.ofFinite`. As a bonus, no lemma about layers depends on a choice of `Fintype` instance (see Stating Results Across a Diamond (p. 80)).

The i-th *distance layer* around r: the vertices at distance exactly i from r.

```
noncomputable def layer [Finite V] (G : SimpleGraph V) (r : V) (i : ℕ) :
    Finset V :=
  have := Fintype.ofFinite V
  {v | G.edist r v = i}
```

We first prove some very basic but useful facts about layers.

```
open Finset

@[simp]
lemma mem_layer [Finite V] {r v : V} {i : ℕ} :
    v ∈ G.layer r i ↔ G.edist r v = i := by
  simp [layer]

lemma layer_zero [Finite V] (r : V) : G.layer r 0 = {r} := by
  ext v
  simp only [mem_layer, Nat.cast_zero, edist_eq_zero_iff, Finset.mem_singleton]
  exact eq_comm

lemma layer_one [Finite V] [G.LocallyFinite] (r : V) :
    G.layer r 1 = G.neighborFinset r := by
  ext v
  simp [edist_eq_one_iff_adj]

lemma disjoint_layer [Finite V] {r : V} {i j : ℕ} (h : i ≠ j) :
    Disjoint (G.layer r i) (G.layer r j) :=
  Finset.disjoint_left.2 fun v hi hj => h (by simp_all)
```

Finite Eccentricity

The degree hypothesis of the Moore bound only makes sense when every vertex has finitely many neighbours, that is, when G is *locally finite*, and this is the only finiteness assumption the Moore bound needs. Every vertex at distance at most $i + 1$ from r is either at distance at most i or adjacent to such a vertex, so by induction each ball around r in a locally finite graph is finite.

```
lemma finite_setOf_edist_le [G.LocallyFinite] (r : V) (i : ℕ) :
    {v | G.edist r v ≤ i}.Finite := by
  induction i with
  | zero => simp
  | succ i ih =>
    refine (ih.union (ih.biUnion fun u _ => (G.neighborSet u).toFinite)).subset ?_
    intro v (hv : G.edist r v ≤ ↑(i + 1))
    rcases hv.lt_or_eq with hlt | heq
    · push_cast at hlt
      exact Or.inl (Order.le_of_lt_add_one hlt)
    · obtain ⟨u, huv, hu⟩ := exists_adj_edist_eq i (by exact_mod_cast heq)
      exact Or.inr (Set.mem_biUnion (x := u) (by simp [hu]) huv)
```

If r has eccentricity at most k then every vertex lies in the ball of radius k around r, so a locally finite graph with a vertex of finite eccentricity is finite.

```
theorem finite_of_eccent_le [G.LocallyFinite] {r : V} {k : ℕ} (h : G.eccent r ≤ k) :
    Finite V :=
  Set.finite_univ_iff.mp
    ((finite_setOf_edist_le r k).subset fun _ _ => edist_le_eccent.trans h)
```

Counting Between Layers

Both Moore and cage bounds compare consecutive layers by counting the edges between them, using two double-counting lemmas about finsets `A` and `B` of vertices. We count the adjacent pairs `(u, v)` with `u ∈ A` and `v ∈ B`.

- If every vertex of `B` has a neighbour in `A`, and every vertex of `A` has at most `d` neighbours in `B`, then `#B ≤ d * #A`, since each `v ∈ B` lies in at least one such pair and each `u ∈ A` in at most `d`.
- If every vertex of `A` has at least `d` neighbours in `B`, and no two distinct vertices of `A` have a common neighbour in `B`, then `d * #A ≤ #B`, since now each `u ∈ A` lies in at least `d` pairs and each `v ∈ B` in at most one.

Both are instances of Mathlib's double-counting lemmas `Finset.card_mul_le_card_mul'` and `Finset.card_mul_le_card_mul`. We state them using `G.degIn B u = #{v ∈ B | G.Adj u v}`, the number of neighbours of `u` in `B` (see `degIn`). In Mathlib's language these neighbours form `B.bipartiteAbove G.Adj u`, whose size is `G.degIn B u` by definition, and the neighbours of `v` in `A` form `A.bipartiteBelow G.Adj v`.

```
lemma card_le_mul_card_of_adj [DecidableRel G.Adj] {A B : Finset V} {d : ℕ}
    (hB : ∀ v ∈ B, ∃ u ∈ A, G.Adj u v) (hA : ∀ u ∈ A, G.degIn B u ≤ d) :
    #B ≤ d * #A := by
  have := card_mul_le_card_mul' G.Adj (n := 1) (fun v hv => by
    obtain ⟨u, hu, huv⟩ := hB v hv
    exact card_pos.2 ⟨u, (mem_bipartiteBelow _).2 ⟨hu, huv⟩⟩) hA
  grind

lemma mul_card_le_card_of_adj [DecidableRel G.Adj] {A B : Finset V} {d : ℕ}
    (hA : ∀ u ∈ A, d ≤ G.degIn B u)
    (hAB : ∀ u ∈ A, ∀ x ∈ A, ∀ v ∈ B, G.Adj u v → G.Adj x v → u = x) :
    d * #A ≤ #B := by
  have := card_mul_le_card_mul G.Adj hA fun v hv =>
    card_le_one.2 fun u hu x hx => by
      rw [mem_bipartiteBelow] at hu hx
      exact hAB u hu.1 x hx.1 v hv hu.2 hx.2
  grind
```

To bound `G.degIn B u` in terms of the degree of `u`, we use two lemmas about `degIn`. The Moore bound uses `degIn_lt_degree_of_adj_notMem`: if some neighbour of `u` lies outside `B` then `G.degIn B u < G.degree u`. The cage bounds need `degree_le_degIn_add_one`: if every neighbour of `u`, except possibly `w`, lies in `B`, then `G.degree u ≤ G.degIn B u + 1`.

```
#check degIn_lt_degree_of_adj_notMem
#check degree_le_degIn_add_one
```

The Moore Bound

In a graph of maximum degree at most $d + 1$, a vertex u in the layer L_{i+1} has a neighbour $w \in L_i$, which is not in L_{i+2}, so u has at most d neighbours in L_{i+2}. By the first counting lemma, $\#L_{i+2} \le d * \#L_{i+1}$.

```
lemma card_layer_add_two_le [Finite V] [G.LocallyFinite] {d : ℕ}
    (h_deg : ∀ v, G.degree v ≤ d + 1) (r : V) (i : ℕ) :
    #(G.layer r (i + 2)) ≤ d * #(G.layer r (i + 1)) := by
  classical
  refine card_le_mul_card_of_adj (G := G) (fun v hv => ?_) (fun u hu => ?_)
  · rw [mem_layer] at hv
    -- push_cast pushes the coercion ℕ → ℕ∞ inwards so that ring can finish
    obtain ⟨u, huv, hu⟩ :=
      exists_adj_edist_eq (i + 1) (by rw [hv]; push_cast; ring)
    exact ⟨u, mem_layer.mpr hu, huv⟩
  · -- u has a neighbour w ∈ L_i, which is not in L_{i+2}
    obtain ⟨w, hwu, hw⟩ := exists_adj_edist_eq i (by simpa using hu)
    have := degIn_lt_degree_of_adj_notMem (G.layer r (i + 2)) hwu.symm
      (by grind [mem_layer, Nat.cast_inj])
    grind
```

Since L_1 is the neighbourhood of r, induction gives $\#L_{i+1} \le (d + 1) * d ^ i$.

```
lemma card_layer_le [Finite V] [G.LocallyFinite] {d : ℕ}
    (h_deg : ∀ v, G.degree v ≤ d + 1) (r : V) (i : ℕ) :
    #(G.layer r (i + 1)) ≤ (d + 1) * d ^ i := by
  induction i with
  | zero => simpa [layer_one] using h_deg r
  | succ i ih =>
    calc #(G.layer r (i + 2)) ≤ d * #(G.layer r (i + 1)) :=
          card_layer_add_two_le h_deg r i
      _ ≤ d * ((d + 1) * d ^ i) := by gcongr
      _ = (d + 1) * d ^ (i + 1) := by ring
```

If every vertex is within distance k of r, i.e. `G.eccent r ≤ k`, then V is finite by `finite_of_eccent_le`, the layers $L_0, \ldots, L_k$ cover V, and we can sum these bounds. Since V need not be finite in advance, we state the bound with `ENat.card V`, not `Nat.card V`. The latter is the junk value `0` for an infinite type, so a bound on it would hold trivially for infinite graphs, while a bound on `ENat.card V` also says that V is finite (`ENat.card_lt_top`).

```
lemma card_le_of_eccent_le [G.LocallyFinite] {d k : ℕ}
    (h_deg : ∀ v, G.degree v ≤ d + 1) {r : V} (h : G.eccent r ≤ k) :
    ENat.card V ≤ (1 + (d + 1) * ∑ i ∈ range k, d ^ i : ℕ) := by
  classical
  have := finite_of_eccent_le h
  have := Fintype.ofFinite V
  rw [ENat.card_eq_coe_natCard, Nat.cast_le]
  have hcover : (univ : Finset V) ⊆ (range (k + 1)).biUnion (G.layer r) := by
    intro v _
    have hv : G.edist r v ≤ k := edist_le_eccent.trans h
    lift G.edist r v to ℕ using ne_top_of_le_ne_top (by simp) hv with i hi
    simpa [mem_range] using ⟨i, by exact_mod_cast hv, hi.symm⟩
  calc Nat.card V = #(univ : Finset V) := Nat.card_eq_fintype_card
    _ ≤ #((range (k + 1)).biUnion (G.layer r)) := card_le_card hcover
```

```
    _ ≤ ∑ i ∈ range (k + 1), #(G.layer r i) := card_biUnion_le
    _ = ∑ i ∈ range k, #(G.layer r (i + 1)) + 1 := by
        rw [sum_range_succ', layer_zero, card_singleton]
    _ ≤ ∑ i ∈ range k, (d + 1) * d ^ i + 1 := by
        gcongr with i
        exact card_layer_le h_deg r i
    _ = 1 + (d + 1) * ∑ i ∈ range k, d ^ i := by rw [mul_sum, add_comm]
```

The Moore bound is usually stated in terms of the diameter, but the proof only uses a single vertex r of eccentricity at most k, so a bound on the radius suffices (the minimum eccentricity is attained, by `exists_eccent_eq_radius`). Since the radius is at most the diameter this gives the usual form. Note that no connectivity hypothesis is needed: a finite radius already forces the graph to be connected.

The **Moore bound**, radius form: a graph with maximum degree at most $d + 1$ and radius at most k has at most $1 + (d + 1) * \sum i < k, d ^ i$ vertices.

```
theorem moore_bound_of_radius_le (G : SimpleGraph V) [G.LocallyFinite]
    (d k : ℕ) (h_deg : ∀ v, G.degree v ≤ d + 1) (h : G.radius ≤ k) :
    ENat.card V ≤ (1 + (d + 1) * ∑ i ∈ Finset.range k, d ^ i : ℕ) := by
  rcases isEmpty_or_nonempty V with hV | hV
  · simp [(ENat.card_eq_zero_iff_empty V).mpr hV]
  obtain ⟨r, hr⟩ := exists_eccent_eq_radius (G := G)
  exact card_le_of_eccent_le h_deg (hr ▸ h)
```

Finally we give the Moore bound in its usual form. Here it matters that the hypothesis is stated with `ediam` rather than `diam`. The empty graph `E[2]` has maximum degree `0`, and its `diam` is the junk value `0`, since it is disconnected; but it has `2` vertices, while the bound with $d = k = 0$ is `1`.

The **Moore bound**: a graph with maximum degree at most $d + 1$ and diameter at most k has at most $1 + (d + 1) * \sum i < k, d ^ i$ vertices.

```
theorem moore_bound (G : SimpleGraph V) [G.LocallyFinite] (d k : ℕ)
    (h_deg : ∀ v, G.degree v ≤ d + 1) (h : G.ediam ≤ k) :
    ENat.card V ≤ (1 + (d + 1) * ∑ i ∈ Finset.range k, d ^ i : ℕ) := by
  rcases isEmpty_or_nonempty V with hV | hV
  · simp [(ENat.card_eq_zero_iff_empty V).mpr hV]
  exact moore_bound_of_radius_le G d k h_deg (radius_le_ediam.trans h)
```

Short Cycles Between Layers

For the cage bound we need to know that a graph of large girth looks like a tree near `r`. The key lemma says that if a vertex `v` has two distinct neighbours `a` and `b`, neither further from `r` than `v`, then G has a cycle of length at most `G.edist r a + G.edist r b + 2`.

To see this, extend shortest walks from `r` to `a` and to `b` by the edges to `v`. Both resulting walks are *paths*. A shortest walk is a path, and it cannot pass through `v`, since every vertex on a shortest walk to `a` other than `a` itself is strictly closer to `r` than `a` is, while `v` is not. The two paths to `v` are different, since their second-to-last vertices $a \neq b$ differ; so we can apply `IsPath.exists_isCycle_length_le_add_of_ne` to obtain the desired short cycle.

```
#check IsPath.exists_isCycle_length_le_add_of_ne
```

```
lemma Walk.notMem_support_of_length_eq_edist {r a v : V} {p : G.Walk r a}
    (hp : p.length = G.edist r a) (hav : G.edist r a ≤ G.edist r v) (hva : v ≠ a) :
    v ∉ p.support := by
  classical
  intro hv
  have h1 : ((p.takeUntil v hv).length : ℕ∞) < p.length := by
    exact_mod_cast p.length_takeUntil_lt_length hv hva
  exact ((edist_le _).trans_lt h1).not_ge (hp ▸ hav)

theorem egirth_le_edist_add_edist_add_two {r a b v : V} (hab : a ≠ b)
    (ha : G.Adj a v) (hb : G.Adj b v)
    (hav : G.edist r a ≤ G.edist r v) (hbv : G.edist r b ≤ G.edist r v) :
    G.egirth ≤ G.edist r a + G.edist r b + 2 := by
  rcases eq_or_ne (G.edist r a) ⊤ with hta | hta
  · simp [hta]
  rcases eq_or_ne (G.edist r b) ⊤ with htb | htb
  · simp [htb]
  obtain ⟨p, hp⟩ := exists_walk_of_edist_ne_top hta
  obtain ⟨q, hq⟩ := exists_walk_of_edist_ne_top htb
  have hpP : (p.concat ha).IsPath :=
    (p.isPath_of_length_eq_dist (by rw [dist, ← hp, ENat.toNat_natCast])).concat
      (notMem_support_of_length_eq_edist hp hav ha.ne.symm) ha
  have hqP : (q.concat hb).IsPath :=
    (q.isPath_of_length_eq_dist (by rw [dist, ← hq, ENat.toNat_natCast])).concat
      (notMem_support_of_length_eq_edist hq hbv hb.ne.symm) hb
  have hne : p.concat ha ≠ q.concat hb := fun h => hab (concat_inj h).1
  obtain ⟨_, -, -, c, hc, hlen⟩ := hpP.exists_isCycle_length_le_add_of_ne hqP hne
  calc G.egirth ≤ c.length := hc.egirth_le_length
    _ ≤ ((p.concat ha).length + (q.concat hb).length : ℕ) := by exact_mod_cast hlen
    _ = G.edist r a + G.edist r b + 2 := by
      simp only [length_concat, Nat.cast_add, Nat.cast_one, hp, hq]
      ring
```

We use this in two ways. Firstly, if $u \in L_{i+1}$ has a neighbour $w \in L_i$, then every other neighbour x of u lies in L_{i+2}, as long as $2 * i + 3$ is less than the girth (otherwise x and w would be two neighbours of u no further from r than u). Secondly, two distinct vertices of L_j have no common neighbour in L_{j+1}, as long as $2 * j + 2$ is less than the girth. (In each proof we pass from $\mathbb{N}\infty$ to $\mathbb{N}$ as soon as we know that the distances involved are finite.)

```
lemma edist_eq_add_two_of_adj {r u w x : V} {i : ℕ} (hu : G.edist r u = i + 1)
    (hw : G.edist r w = i) (hwu : G.Adj w u) (hux : G.Adj u x) (hxw : x ≠ w)
    (hg : 2 * i + 3 < G.egirth) : G.edist r x = i + 2 := by
  -- x is adjacent to u, so it is at distance at most i + 2 from r
  have hx : G.edist r x ≤ i + 2 := by
    simpa [hu, add_assoc, one_add_one_eq_two] using hux.edist_le_add_one r
  -- if x were at distance at most i + 1 from r, then shortest paths from r
  -- to x and to w, together with x – u – w, would give a cycle of length at
  -- most 2 * i + 3, which is less than the girth
  have hcyc (hx : G.edist r x ≤ i + 1) : G.egirth ≤ G.edist r x + i + 2 := by
    have hw_le : G.edist r w ≤ G.edist r u := by simp [hu, hw]
    rw [← hw]
    exact egirth_le_edist_add_edist_add_two hxw hux.symm hwu (hu ▸ hx) hw_le
  -- the distance from r to x is finite, so we can work in ℕ
```

```
  lift G.edist r x to ℕ using ne_top_of_le_ne_top (by simp) hx with m
  norm_cast at hx hcyc ⊢
  by_contra hne
  have hlt : 2 * i + 3 < m + i + 2 := by exact_mod_cast hg.trans_le (hcyc (by grind))
  grind

lemma eq_of_adj_of_adj_of_lt_egirth {r u v x : V} {j : ℕ} (hu : G.edist r u = j)
    (hv : G.edist r v = j) (hx : G.edist r x = j + 1) (hux : G.Adj u x)
    (hvx : G.Adj v x) (hg : 2 * j + 2 < G.egirth) : u = v := by
  by_contra hne
  have := egirth_le_edist_add_edist_add_two hne hux hvx (by rw [hu, hx]; simp)
    (by rw [hv, hx]; simp)
  rw [hu, hv] at this
  have := hg.trans_le this
  norm_cast at this
  grind
```

The Cage Bound for Odd Girth

If the minimum degree is at least $d + 1$ and $2 * i + 4$ is less than the girth, then a vertex of L_{i+1} has at least d neighbours in L_{i+2} (all its neighbours except one in L_i), and distinct vertices of L_{i+1} have no common neighbours there. By the second counting lemma (`mul_card_le_card_of_adj`), we have $d * \#L_{i+1} \le \#L_{i+2}$.

```
lemma mul_card_layer_le [Finite V] [G.LocallyFinite] {d : ℕ}
    (h_deg : ∀ v, d + 1 ≤ G.degree v) (r : V) {i : ℕ} (hg : 2 * i + 4 < G.egirth) :
    d * #(G.layer r (i + 1)) ≤ #(G.layer r (i + 2)) := by
  classical
  refine mul_card_le_card_of_adj (G := G) (fun u hu => ?_)
    (fun u hu x hx v hv huv hxv => ?_)
  · obtain ⟨w, hwu, hw⟩ := exists_adj_edist_eq i (by simpa using hu)
    -- every neighbour of u other than w lies in L_{i+2}
    have hnbr : ∀ x, G.Adj u x → x ≠ w → x ∈ G.layer r (i + 2) := by
      intro x hux hxw
      have hx := edist_eq_add_two_of_adj (by simpa using hu) hw hwu hux hxw
        (hg.trans_le' (by norm_cast; grind))
      rw [mem_layer]
      exact_mod_cast hx
    have hdeg : G.degree u ≤ G.degIn (G.layer r (i + 2)) u + 1 :=
      degree_le_degIn_add_one _ w hnbr
    grind
  · simp only [mem_layer] at hu hx hv
    exact eq_of_adj_of_adj_of_lt_egirth hu hx (by rw [hv]; rfl) huv hxv
      (by push_cast; convert hg using 1; ring)
```

By induction $\#L_{i+1} \ge (d + 1) * d \hat{} i$, provided that $2 * i + 2$ is less than the girth.

```
lemma le_card_layer [Finite V] [G.LocallyFinite] {d : ℕ}
    (h_deg : ∀ v, d + 1 ≤ G.degree v) (r : V) {i : ℕ} (hg : 2 * i + 2 < G.egirth) :
    (d + 1) * d ^ i ≤ #(G.layer r (i + 1)) := by
  induction i with
  | zero => simpa [layer_one] using h_deg r
  | succ i ih =>
```

```
    calc (d + 1) * d ^ (i + 1) = d * ((d + 1) * d ^ i) := by ring
      _ ≤ d * #(G.layer r (i + 1)) := by
          gcongr
          exact ih (hg.trans_le' (by norm_cast; grind))
      _ ≤ #(G.layer r (i + 2)) :=
          mul_card_layer_le h_deg r (by push_cast at hg; convert hg using 1; ring)
```

If the girth is at least `2 * k + 1` this holds for every `i < k`, and since the layers are disjoint, L_0, …, L_k contain at least `1 + (d + 1) * ∑ i < k, d ^ i` vertices. The layers around any vertex `r` already contain this many vertices, so `G` need not be connected; we only need `V` to be nonempty. We state the girth hypothesis with `egirth`, so it also covers acyclic graphs (whose `egirth` is `⊤`). As for the Moore bound, we state it with `ENat.card V`, so that no `[Finite V]` hypothesis is needed, but here for the opposite reason: `ENat.card V` is `⊤` for an infinite type, so a lower bound on it holds trivially for infinite graphs.

The **cage bound**, odd girth: a graph with at least one vertex, minimum degree at least `d + 1` and girth at least `2 * k + 1` has at least `1 + (d + 1) * ∑ i < k, d ^ i` vertices.

```
theorem cage_bound_odd (G : SimpleGraph V) [G.LocallyFinite]
    [Nonempty V] (d k : ℕ) (h_deg : ∀ v, d + 1 ≤ G.degree v)
    (h_girth : 2 * k + 1 ≤ G.egirth) :
    (1 + (d + 1) * ∑ i ∈ range k, d ^ i : ℕ) ≤ ENat.card V := by
  classical
  rcases finite_or_infinite V with hV | hV
  · rw [ENat.card_eq_coe_natCard, Nat.cast_le]
    have := Fintype.ofFinite V
    obtain ⟨r⟩ := ‹Nonempty V›
    calc 1 + (d + 1) * ∑ i ∈ range k, d ^ i
        = ∑ i ∈ range k, (d + 1) * d ^ i + 1 := by rw [mul_sum, add_comm]
      _ ≤ ∑ i ∈ range k, #(G.layer r (i + 1)) + #(G.layer r 0) := by
          gcongr with i hi
          · exact le_card_layer h_deg r (h_girth.trans_lt' (by norm_cast; grind))
          · simp [layer_zero]
      _ = ∑ i ∈ range (k + 1), #(G.layer r i) :=
          (sum_range_succ' (fun i => #(G.layer r i)) k).symm
      _ = #((range (k + 1)).biUnion (G.layer r)) :=
          (card_biUnion fun i _ j _ hij => disjoint_layer hij).symm
      _ ≤ Nat.card V := Nat.card_eq_fintype_card (α := V) ▸ card_le_univ _
  · simp
```

Moore Graphs

A graph attaining the Moore bound is called a *Moore graph*. Equality forces every layer to have the largest possible size, which (one can show) makes a Moore graph of diameter `k` with `d ≥ 1` a `(d + 1)`-regular graph of girth `2 * k + 1`, so that it attains the cage bound as well. (For `d = 0` the single edge `K[2]` attains the Moore bound but has no cycles.)

The simplest examples are the complete graphs: for `d ≥ 1`, `K[d + 2]` is `(d + 1)`-regular, has diameter `1` and girth `3`, and both bounds with `k = 1` evaluate to `d + 2`. Next come the odd cycles: for `k ≥ 1`, `C[2 * k + 1]` is 2-regular (so `d = 1`), has diameter `k` (`ediam_cycleGraph`) and girth `2 * k + 1` (`egirth_cycleGraph`), and both bounds evaluate to `2 * k + 1`, as the following computation checks.

```
example (k : ℕ) : 1 + (1 + 1) * ∑ i ∈ Finset.range k, 1 ^ i = 2 * k + 1 := by
  simp [add_comm]
```

Another Moore graph is the Petersen graph, shown in the figure at the start of this section. It has `10 = 1 + 3 * (1 + 2)` vertices, is 3-regular, and has diameter 2 and girth 5, so it attains both bounds with `d = 2` and `k = 2`. We prove this in Moore Graphs of Diameter Two (p. 375), where the theory of strongly regular graphs allows us to give short proofs.

Moore graphs are rare. Besides `C[5]` and the Petersen graph, the only known Moore graph of diameter 2 is the Hoffman–Singleton graph, which is 7-regular with 50 vertices. The Hoffman–Singleton theorem [Hoffman–Singleton 1960] (Strongly Regular Graphs (p. 373)) shows that any other example would be 57-regular, and whether one exists is a famous open problem. For diameter at least 3, it is known that the only Moore graphs are the odd cycles.

The Cage Bound for Even Girth

The cage bound for even girth `2 * k` follows essentially the same argument as for odd girth but working from an edge `r - s` instead of a vertex (and the details are more involved). Let `Aᵢ` be the set of vertices at distance `i` from `r` and `i + 1` from `s` (so `A₀ = {r}`), and let `Bᵢ` be the same with the roles of `r` and `s` swapped.

A vertex `u ∈ Aᵢ` has a neighbour `w` at distance `i` from `s`. If `i + 1 < k` then all the other neighbours of `u` are at distance `i + 2` from `s` (by `edist_eq_add_two_of_adj` with root `s`), and hence at distance `i + 1` from `r` (since `r` and `s` are adjacent), so they lie in `Aᵢ₊₁`. Two vertices of `Aᵢ` have no common neighbour in `Aᵢ₊₁` (by `eq_of_adj_of_adj_of_lt_egirth` with root `r`). So `d * #Aᵢ ≤ #Aᵢ₊₁`, and by induction `#Aᵢ ≥ d ^ i` for `i < k`.

```
lemma mul_card_inter_layer_le [Finite V] [G.LocallyFinite] [DecidableEq V]
    {d : ℕ} (h_deg : ∀ v, d + 1 ≤ G.degree v) {r s : V} (hrs : G.Adj r s) {i : ℕ}
    (hg : 2 * i + 3 < G.egirth) :
    d * #(G.layer r i ∩ G.layer s (i + 1)) ≤
      #(G.layer r (i + 1) ∩ G.layer s (i + 2)) := by
  classical
  refine mul_card_le_card_of_adj (G := G) (fun u hu => ?_)
    (fun u hu x hx v hv huv hxv => ?_)
  · simp only [mem_inter, mem_layer, Nat.cast_add, Nat.cast_one] at hu
    obtain ⟨w, hwu, hw⟩ := exists_adj_edist_eq i hu.2
    -- every neighbour of u other than w lies in Aᵢ₊₁
    have hnbr : ∀ x, G.Adj u x → x ≠ w →
        x ∈ G.layer r (i + 1) ∩ G.layer s (i + 2) := by
      intro x hux hxw
      have h2 : G.edist s x = i + 2 := edist_eq_add_two_of_adj hu.2 hw hwu hux hxw hg
      have h1 : G.edist r x = i + 1 := by
        refine le_antisymm (by simpa [hu.1] using hux.edist_le_add_one r) ?_
        -- cancel the 1 (which is finite) from i + 2 ≤ G.edist r x + 1
        have hsx := hrs.edist_le_add_one x
        rw [edist_comm (u := x) (v := s), edist_comm (u := x) (v := r), h2] at hsx
        exact (ENat.add_le_add_iff_right ENat.one_ne_top).mp
          (by simpa [add_assoc, one_add_one_eq_two] using hsx)
      simp_rw [mem_inter, mem_layer]
      exact ⟨by exact_mod_cast h1, by exact_mod_cast h2⟩
    have hdeg :
        G.degree u ≤ G.degIn (G.layer r (i + 1) ∩ G.layer s (i + 2)) u + 1 :=
```

```
      degree_le_degIn_add_one _ w hnbr
    grind
  · simp only [mem_inter, mem_layer] at hu hx hv
    exact eq_of_adj_of_adj_of_lt_egirth hu.1 hx.1 (by rw [hv.1]; push_cast; rfl) huv
      hxv (hg.trans_le' (by norm_cast; grind))

lemma pow_le_card_inter_layer [Finite V] [G.LocallyFinite] [DecidableEq V]
    {d : ℕ} (h_deg : ∀ v, d + 1 ≤ G.degree v) {r s : V} (hrs : G.Adj r s) {i : ℕ}
    (hg : 2 * i + 1 < G.egirth) : d ^ i ≤ #(G.layer r i ∩ G.layer s (i + 1)) := by
  induction i with
  | zero =>
    refine card_pos.mpr ⟨r, ?_⟩
    simp [edist_eq_one_iff_adj.mpr hrs.symm]
  | succ i ih =>
    calc d ^ (i + 1) = d * d ^ i := by ring
      _ ≤ d * #(G.layer r i ∩ G.layer s (i + 1)) := by
          gcongr
          exact ih (hg.trans_le' (by norm_cast; grind))
      _ ≤ #(G.layer r (i + 1) ∩ G.layer s (i + 2)) :=
          mul_card_inter_layer_le h_deg hrs
            (by push_cast at hg; convert hg using 1; ring)
```

The sets A_i and B_j are pairwise disjoint: a vertex of A_i is closer to r than to s, a vertex of B_j is closer to s than to r, and each family is contained in the distance layers around one endpoint. So together they contain at least $2 * \sum i < k, d ^ i$ vertices.

The **cage bound**, even girth: a graph with at least one vertex, minimum degree at least $d + 1$ and girth at least $2 * k$ has at least $2 * \sum i < k, d ^ i$ vertices.

```
theorem cage_bound_even (G : SimpleGraph V) [G.LocallyFinite]
    [Nonempty V] (d k : ℕ) (h_deg : ∀ v, d + 1 ≤ G.degree v)
    (h_girth : 2 * k ≤ G.egirth) :
    (2 * ∑ i ∈ range k, d ^ i : ℕ) ≤ ENat.card V := by
  classical
  rcases finite_or_infinite V with hV | hV
  · rw [ENat.card_eq_coe_natCard, Nat.cast_le]
    have := Fintype.ofFinite V
    obtain ⟨r⟩ := ‹Nonempty V›
    obtain ⟨s, hrs⟩ := (G.degree_pos_iff_exists_adj r).mp (by have := h_deg r; grind)
    let A : ℕ → Finset V := fun i => G.layer r i ∩ G.layer s (i + 1)
    let B : ℕ → Finset V := fun i => G.layer s i ∩ G.layer r (i + 1)
    have hdisj : ∀ i j (v : V), v ∈ A i ∪ B i → v ∈ A j ∪ B j → i = j := by
      intro i j v hi hj
      simp only [A, B, mem_union, mem_inter, mem_layer] at hi hj
      rcases hi with ⟨h1, h2⟩ | ⟨h1, h2⟩ <;> rcases hj with ⟨h3, h4⟩ | ⟨h3, h4⟩ <;>
        simp_all only [Nat.cast_inj] <;> grind
    have hAB : ∀ i, Disjoint (A i) (B i) := fun i =>
      Finset.disjoint_left.mpr fun v hA hB => by
        simp only [A, B, mem_inter, mem_layer] at hA hB
        rw [hA.1] at hB
        norm_cast at hB
        grind
    calc 2 * ∑ i ∈ range k, d ^ i = ∑ i ∈ range k, (d ^ i + d ^ i) := by
          rw [sum_add_distrib, two_mul]
```

```
    _ ≤ ∑ i ∈ range k, (#(A i) + #(B i)) := by
        gcongr with i hi <;> have := mem_range.mp hi
        · exact pow_le_card_inter_layer h_deg hrs
            (h_girth.trans_lt' (by norm_cast; grind))
        · exact pow_le_card_inter_layer h_deg hrs.symm
            (h_girth.trans_lt' (by norm_cast; grind))
    _ = ∑ i ∈ range k, #(A i ∪ B i) :=
        sum_congr rfl fun i _ => (card_union_of_disjoint (hAB i)).symm
    _ = #((range k).biUnion fun i => A i ∪ B i) := by
        refine (card_biUnion fun i _ j _ hij => ?_).symm
        rw [Function.onFun, Finset.disjoint_left]
        exact fun v hi hj => hij (hdisj i j v hi hj)
    _ ≤ Nat.card V := Nat.card_eq_fintype_card (α := V) ▸ card_le_univ _
  · simp
```

The complete bipartite graphs attain the even cage bound with $k = 2$. For $d \geq 1$, $K[d + 1, d + 1]$ is $(d + 1)$-regular, has girth 4 (`egirth_completeBipartiteGraph`), and has $2 * (d + 1)$ vertices.

```
example (d : ℕ) :
    ENat.card (Fin (d + 1) ⊕ Fin (d + 1)) = 2 * ∑ i ∈ Finset.range 2, d ^ i := by
  simp [two_mul]
```

11.6 Exercises

1. Show that two vertices with a common neighbour are at distance at most 2.

```
variable {V : Type*} {G : SimpleGraph V}
example {u v w : V} (huw : G.Adj u w) (hwv : G.Adj w v) : G.edist u v ≤ 2 := by
  sorry
```

2. Show that in a triangle-free graph two distinct vertices with a common neighbour are at distance exactly 2. Hint: `edist_eq_two_iff`.

```
example (h3 : G.CliqueFree 3) {u v w : V} (huw : G.Adj u w) (hwv : G.Adj w v)
    (huv : u ≠ v) : G.edist u v = 2 := by
  sorry
```

3. Deduce from `Hom.edist_le` that adding edges cannot increase distances (this is Mathlib's `edist_anti`), and that the distance between two vertices of an induced subgraph is at least their distance in G.

Hint: `Hom.ofLE` and `Embedding.induce`.

```
example {H : SimpleGraph V} (h : G ≤ H) (u v : V) : H.edist u v ≤ G.edist u v := by
  sorry

example (s : Set V) (u v : s) : G.edist u v ≤ (G.induce s).edist u v := by
  sorry
```

4. Use `edist_eq_hammingDist` to show that the Hamming distance (between vertices of the hypercube) satisfies the triangle inequality.

Hint: rewrite the triangle inequality for `edist` in `Q[n]` using `edist_eq_hammingDist`, and cast the result back to $\mathbb{N}$.

```
example {n : ℕ} (u v w : HypercubeVert n) :
    u.hammingDist w ≤ u.hammingDist v + v.hammingDist w := by
  sorry
```

5. Show that the complete bipartite graph `K[s + 2, t + 1]` has diameter 2.

```
example (s t : ℕ) : K[s + 2, t + 1].ediam = 2 := by
  sorry
```

6. Show that if G has diameter at least 3 then its complement has diameter at most 3.

Hint: take two vertices u and v at distance more than 2 in G (`two_lt_edist_iff`), and show that every vertex is adjacent to u or to v in G^c, or equal to one of them.

```
example (h : 3 ≤ G.ediam) : Gᶜ.ediam ≤ 3 := by
  sorry
```

7. Show that if v has a neighbour in G then in the Mycielskian `mμ(G)` the two copies v^l and v^r of v are at distance exactly 2. Hint: `edist_eq_two_iff`.

```
example {u v : V} (h : G.Adj u v) : mμ(G).edist v.l v.r = 2 := by
  sorry
```

8. Show that every vertex of the cycle graph `C[n + 3]` has eccentricity `(n + 3) / 2`.

Hint: `IsVertexTransitive.eccent_eq`, `ediam` and `ediam_cycleGraph`.

```
example (n : ℕ) (v : Fin (n + 3)) : C[n + 3].eccent v = ((n + 3) / 2 : ℕ) := by
  sorry
```

9. Show that the Grötzsch graph `mμ(C[5])` has girth 4. Hint: use `egirth_mycielskian_eq_four`.

```
example : grotzschGraph.egirth = 4 := by
  sorry
```

10. Use the Moore bound to show that a 3-regular graph of diameter at most 2 has at most 10 vertices. (The Petersen graph shows that this is best possible.)

```
example [G.LocallyFinite] (h : G.IsRegularOfDegree 3) (hd : G.ediam ≤ 2) :
    ENat.card V ≤ 10 := by
  sorry
```

11. Use the cage bound to show that a graph with minimum degree at least 3 and girth at least 5 has at least 10 vertices. (The Petersen graph shows that this is best possible.)

```
example [G.LocallyFinite] [Nonempty V] (h : ∀ v, 3 ≤ G.degree v)
    (hg : 5 ≤ G.egirth) : 10 ≤ ENat.card V := by
  sorry
```

12 Algebraic Graph Theory

Algebraic graph theory studies a graph via algebraic objects attached to it, above all its *automorphism group* (`Group (G ≃g G)`), which measures how symmetric the graph is, and its *adjacency matrix* (`adjMatrix`), whose powers count walks in the graph. The two are related in an obvious way: a permutation of the vertices is an automorphism iff it preserves the adjacency matrix.

In this chapter we start from Mathlib's group structure on the automorphisms of a graph, and show that isomorphic graphs have isomorphic automorphism groups (`Iso.autCongr`). We show that the automorphism group of the infinite path `pathNat` is trivial (`aut_pathNat_eq_one`) and that of the cycle graph `C[n]`, for `n ≥ 3`, is the dihedral group (`autCycleGraphMulEquivDihedral`).

Next we introduce the adjacency matrix. We show that every vertex of a vertex-transitive graph lies on the same number of closed walks of each length (`IsVertexTransitive.card_walk_self_eq_trace_div`), and that isomorphic graphs have the same characteristic polynomial (`Iso.charpoly_adjMatrix`), although non-isomorphic graphs can share one. We then show that eigenvalues (`spectrum`) of `G` give eigenvalues of its Mycielskian, scaled by the golden ratio (`mem_spectrum_adjMatrix_mycielskian`). We also survey Mathlib's results on the Laplacian (`lapMatrix`), whose kernel has dimension equal to the number of connected components (`card_connectedComponent_eq_finrank_ker_toLin'_lapMatrix`), and on strongly regular graphs (`IsSRGWith`) such as the Petersen graph (`petersenGraph`) and the Hoffman–Singleton graph (`hoffmanSingletonGraph`). To prepare for the later sections, we show that the Hamming cube model `hypercubeHamming` of the hypercube is isomorphic to our recursive box product construction `Q[n]` (`hypercubeIsoHamming`).

The final three sections are more advanced. Using the Hamming cube model of `Q[n]`, we prove that its automorphism group is the hyperoctahedral group `ℤ/2 ≀ Sₙ`, of order `2 ^ n * n!` (`autHypercubeMulEquivHyperoctahedral`). We then count copies of one graph in another. The unlabelled copies of `K[s]` are just the `s`-cliques, and in general the number of labelled copies is the number of unlabelled copies times the number of automorphisms (`labelledCopyCount_eq_copyCount_mul_labelledCopyCount`). Finally we show that every copy of `Q[k]` in `Q[n]` is an induced subcube, and count these copies (`labelledCopyCount_hypercube`).

12.1 Automorphism Groups

Mathlib provides a `Group` instance on the automorphisms `G ≃g G` of any graph, with multiplication given by composition: `(e * f) v = e (f v)`. (Recall that `SimpleGraph.Iso` is an abbrev for `RelIso`, which has a `Group` instance.)

```
variable {V : Type*} {G : SimpleGraph V}
#synth Group (G ≃g G) -- RelIso.instGroup
```

An isomorphism `e : G ≃g H` transports automorphisms of `G` to automorphisms of `H` by conjugation, `f ↦ e ∘ f ∘ e.symm`, and this is an isomorphism of automorphism groups. So isomorphic graphs have isomorphic automorphism groups, and we may compute an automorphism group using whichever model of a graph is most convenient.

The underlying bijection is Mathlib's `RelIso.relIsoCongr e e`, which transports isomorphisms along isomorphisms by pre- and post-composition, so we only need to check that it is multiplicative. Mathlib writes `≃*` (`MulEquiv`) for an isomorphism of groups, and `→*` (`MonoidHom`) for a homomorphism.

```
#check RelIso.relIsoCongr -- (r₁ ≃r r₂) → (s₁ ≃r s₂) → ((r₁ ≃r s₁) ≃ (r₂ ≃r s₂))
```

Transport of the automorphism group of a graph along an isomorphism `e : G ≃g H`, by conjugation: `f ↦ e ∘ f ∘ e.symm`.

```
@[simps!]
def Iso.autCongr {V W : Type*} {G : SimpleGraph V} {H : SimpleGraph W} (e : G ≃g H) :
    (G ≃g G) ≃* (H ≃g H) :=
  { RelIso.relIsoCongr e e with
    map_mul' f g := by ext v; simp [RelIso.relIsoCongr_apply] }
```

Automorphisms of the Infinite Path

A vertex-transitive graph has lots of symmetry: any vertex can be sent to any other by an automorphism. At the opposite extreme are graphs with no symmetry at all: a graph is *asymmetric* if its only automorphism is the identity, i.e. its automorphism group is trivial. A simple example is the infinite path `pathNat`. This may seem surprising since, as we saw in Maps (p. 98), `pathNat` has many embeddings into itself: for each `j : ℕ` the embedding `pathNat.translate j` sends `i` to `i + j`. However, none of these is surjective, except for the identity `pathNat.translate 0`.

The key observation is that `0` is the only vertex of `pathNat` of degree one (all the other vertices have degree two), and isomorphisms preserve degrees. So every automorphism of `pathNat` fixes `0`.

```
#check degree_pathNat -- pathNat.degree n = if n = 0 then 1 else 2
#check Iso.degree_eq  -- (f : G ≃g G') (x : V) : G'.degree (f x) = G.degree x

lemma aut_pathNat_apply_zero (e : pathNat ≃g pathNat) : e 0 = 0 := by
  have h := e.degree_eq 0
  rw [degree_pathNat, degree_pathNat] at h
  grind
```

We can now walk along the path, showing that an automorphism `e` fixes every vertex by induction using `Nat.twoStepInduction` (a similar argument will identify the automorphisms of cycle graphs in the next section). Since `1` is the only neighbour of `e 0 = 0`, `e` fixes `1`. If `e` fixes `k` and `k + 1` then `e (k + 2)` is a neighbour of `e (k + 1) = k + 1`, so it is either `k` or `k + 2`. It cannot be `k = e k` since `e` is injective, so `e` fixes `k + 2`.

```
lemma aut_pathNat_apply (e : pathNat ≃g pathNat) (k : ℕ) : e k = k := by
  induction k using Nat.twoStepInduction with
  | zero => exact aut_pathNat_apply_zero e
  | one =>
    have h : pathNat.Adj (e 0) (e 1) := e.map_adj_iff.2 (by simp)
    grind [aut_pathNat_apply_zero]
```

```
  | more k ih₀ ih₁ =>
    have h : pathNat.Adj (e (k + 1)) (e (k + 2)) := e.map_adj_iff.2 (by simp)
    have hne : e (k + 2) ≠ e k := fun h => by simpa using e.injective h
    grind
```

So the automorphism group of `pathNat` is trivial: every automorphism is the identity `1`, and there is exactly one of them.

```
lemma aut_pathNat_eq_one (e : pathNat ≃g pathNat) : e = 1 := by
  ext k
  exact aut_pathNat_apply e k


lemma card_aut_pathNat : Nat.card (pathNat ≃g pathNat) = 1 :=
  Nat.card_eq_one_iff_exists.2 ⟨1, aut_pathNat_eq_one⟩
```

In particular `pathNat` is not vertex-transitive.

```
lemma not_isVertexTransitive_pathNat : ¬pathNat.IsVertexTransitive := by
  intro h
  obtain ⟨e, he⟩ := h 0 1
  simp [aut_pathNat_apply_zero] at he
```

What makes `pathNat` asymmetric is that it has a single endpoint (a vertex of degree one), `0`. A finite path with at least two vertices has two endpoints, and reversing it is a nontrivial automorphism; likewise the two-way infinite path on `ℤ` has the reflection `i ↦ -i` and the translations `i ↦ i + j` as automorphisms.

Automorphisms of Cycle Graphs

In `isVertexTransitive_cycleGraph` we showed that the cycle graph `C[n]` is vertex-transitive, using the rotations `cycleGraph.rotate d`, which send the vertex `d` to `0`. We now determine its whole automorphism group. Besides the rotations there is an automorphism given by reflecting the vertices, i.e. sending `k` to `-k` modulo `n`.

The automorphism of the cycle graph `C[n]` given by reflecting the vertices.

```
@[simps]
def cycleGraph.reflect {n : ℕ} : C[n] ≃g C[n] where
  toFun k := -k
  invFun k := -k
  left_inv k := by simp
  right_inv k := by simp
  map_rel_iff' := by
    match n with
    | 0 | 1 => simp
    | n + 2 =>
      intro a b
      grind [cycleGraph_adj, Equiv.coe_fn_mk]
```

If `e : C[n + 2] ≃g C[n + 2]` is an automorphism of the cycle graph and `e 1 - e 0 = 1` then `e` is a rotation, namely the one sending the vertex `e.symm 0` to `0`. [1]

[1] We work with `C[n + 2]` throughout since `cycleGraph_adj` only describes adjacency in `C[n + 2]`. The statements would remain (trivially) true for `C[n + 1]`, but the classification of automorphisms below is **false** for `C[0]`.

```
lemma aut_cycleGraph_eq_rotate {n : ℕ} (e : C[n + 2] ≃g C[n + 2])
    (h : e 1 - e 0 = 1) : e = cycleGraph.rotate (e.symm 0) := by
  let i := e 0
  have hstep : e 1 = 1 + i := by grind
  have hall : ∀ (j : ℕ) (hj : j < n + 2),
      e ⟨j, hj⟩ = ⟨j, hj⟩ + i := by
    intro j
    induction j using Nat.twoStepInduction with
    | zero => simp [i]
    | one  =>  simpa [add_comm] using hstep
    | more j ihj ihj1 =>
      intro hj2
      have hadj : C[n + 2].Adj (e ⟨j + 2, hj2⟩) (e ⟨j + 1, by grind⟩) :=
        e.toHom.map_adj (by
          rw [cycleGraph_adj]
          exact Or.inl (Fin.ext (by rw [Fin.sub_val_of_le (by simp)]; simp)))
      have hs : (⟨j + 1, by grind⟩ : Fin (n + 2)) = 1 + ⟨j, by grind⟩ := by
        apply Fin.ext
        change j + 1 = (1 + j) % (n + 2)
        rw [Nat.mod_eq_of_lt (by grind), add_comm]
      have hnext : (⟨j + 2, hj2⟩ : Fin (n + 2)) = 1 + ⟨j + 1, by grind⟩ := by
        apply Fin.ext
        change j + 2 = (1 + (j + 1)) % (n + 2)
        rw [Nat.mod_eq_of_lt (by grind), add_comm 1]
      rcases hadj with hadj | hadj
      · grind
      · have heq : e ⟨j + 2, hj2⟩ = e ⟨j, by grind⟩ := by grind
        have : j + 2 = j := Fin.mk.inj (e.toEquiv.injective heq)
        grind
  have h0 : e (-i) = 0 := by
    simpa [i] using hall (-i).val (-i).isLt
  have hsymm : e.symm 0 = -i := e.symm_apply_eq.2 h0.symm
  rw [hsymm]
  ext k
  simpa [i, sub_neg_eq_add] using congrArg Fin.val (hall k.val k.isLt)
```

If instead `e 0 - e 1 = 1` then `e` is a rotation followed by the reflection. To see this note that `f = cycleGraph.reflect ∘ e` satisfies `f 1 - f 0 = 1`, so `f` is a rotation by the previous lemma, and hence `e = cycleGraph.reflect ∘ f` is as claimed (the reflection is an involution).

```
lemma aut_cycleGraph_eq_rotate_trans_reflect {n : ℕ} (e : C[n + 2] ≃g C[n + 2])
    (h : e 0 - e 1 = 1) :
    e = (cycleGraph.rotate (e.symm 0)).trans cycleGraph.reflect := by
  let f := e.trans cycleGraph.reflect
  have hf : f 1 - f 0 = 1 := by
    simpa [f, sub_eq_add_neg, add_comm] using h
  have hsymm : e.symm 0 = f.symm 0 := by simp [f]
  rw [hsymm, ← aut_cycleGraph_eq_rotate f hf]
  ext k
  simp [f]
```

Hence any automorphism of the cycle graph `C[n + 2]` is either a rotation or a rotation followed by reflection.

```
lemma aut_cycleGraph {n : ℕ} (e : C[n + 2] ≃g C[n + 2]) : ∃ i : Fin (n + 2),
    e = cycleGraph.rotate i ∨
      e = (cycleGraph.rotate i).trans cycleGraph.reflect := by
  have h0 : C[n + 2].Adj (e 0) (e 1) :=
    e.toHom.map_adj (by grind [cycleGraph_adj])
  rcases h0 with h | h
  · exact ⟨_, Or.inr (aut_cycleGraph_eq_rotate_trans_reflect e h)⟩
  · exact ⟨_, Or.inl (aut_cycleGraph_eq_rotate e h)⟩
```

For `n ≥ 3` the automorphism group of `C[n]` is the dihedral group `DihedralGroup n`, which is why we state the result for `C[n + 3]`. For `n < 3` it fails for trivial reasons:

- `C[0]` has vertex type `Fin 0`, hence exactly one automorphism, while `DihedralGroup 0` is infinite;
- `C[1]` has one vertex, hence one automorphism, while `DihedralGroup 1` has two elements;
- `C[2]` is a single edge, with two automorphisms, while `DihedralGroup 2` has four elements.

Our rotations match Mathlib's conventions for `DihedralGroup` exactly: `r i` corresponds to `cycleGraph.rotate i` and, since `sr i = sr 0 * r i`, the reflection `sr i` corresponds to `cycleGraph.rotate i` followed by the reflection. In both cases the index `i` is the vertex sent to `0`, i.e. `e.symm 0`.

```
#source DihedralGroup
```

Multiplicativity is easiest to check in the direction from `DihedralGroup (n + 3)`, where it amounts to four short computations with rotations and reflections, so we build the isomorphism in that direction and then take its inverse (`MulEquiv.symm`).

The automorphism group of the cycle graph `C[n + 3]` is isomorphic to the dihedral group of order `2 * (n + 3)`.

```
def autCycleGraphMulEquivDihedral (n : ℕ) :
    (C[n + 3] ≃g C[n + 3]) ≃* DihedralGroup (n + 3) :=
  MulEquiv.symm
    { toFun := fun d => match d with
        | .r i => cycleGraph.rotate ((ZMod.finEquiv (n + 3)).symm i)
        | .sr i => (cycleGraph.rotate ((ZMod.finEquiv (n + 3)).symm i)).trans
            cycleGraph.reflect
      invFun e := if e 1 - e 0 = 1 then .r (ZMod.finEquiv (n + 3) (e.symm 0))
        else .sr (ZMod.finEquiv (n + 3) (e.symm 0))
      left_inv d := by
        have hneg : (-1 : Fin (n + 3)) ≠ 1 := by simp [Fin.ext_iff]
        rcases d with i | i <;> simp [hneg]
      right_inv e := by
        by_cases h : e 1 - e 0 = 1
        · simpa [h] using (aut_cycleGraph_eq_rotate e h).symm
        · have h0 : C[n + 3].Adj (e 0) (e 1) :=
            e.toHom.map_adj (by grind [cycleGraph_adj])
          rw [cycleGraph_adj] at h0
          simpa [h] using
            (aut_cycleGraph_eq_rotate_trans_reflect e (h0.resolve_right h)).symm
      map_mul' := by
```

```
    rintro (i | i) (j | j) <;> ext k <;>
      simp [sub_sub, sub_sub_eq_add_sub, add_comm] }
```

Combining this with the isomorphism *hypercubeTwoIsoCycleFour* : Q[2] ≃g C[4] the automorphism group of the square Q[2] is the dihedral group of order 8. (We identify the automorphism group of every hypercube in The Automorphism Group of the Hypercube (p. 379).)

The automorphism group of the square Q[2] is the dihedral group of order 8.

```
def autHypercubeTwoMulEquivDihedral : (Q[2] ≃g Q[2]) ≃* DihedralGroup 4 :=
  hypercubeTwoIsoCycleFour.autCongr.trans (autCycleGraphMulEquivDihedral 1)
```

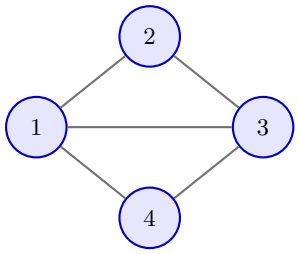
12.2 The Adjacency Matrix

A graph can also be studied via matrices indexed by its vertices. Mathlib's Matrix *V V* R is the type of functions *V* → *V* → R, so no ordering of the vertices is needed.

```
section adjmatrix
open Finset Polynomial Matrix
variable {V : Type*} {G : SimpleGraph V}

#source adjMatrix -- Matrix.of fun i j => if G.Adj i j then (1 : α) else 0
```

The *adjacency matrix* *G*.adjMatrix R of *G* over a type R with a zero and a one ([Zero R], [One R]) has (i, j) entry 1 if i and j are adjacent in *G* and 0 otherwise. (Informally we will denote it by A.)



$$\begin{pmatrix} 0 & 1 & 1 & 1 \\ 1 & 0 & 1 & 0 \\ 1 & 1 & 0 & 1 \\ 1 & 0 & 1 & 0 \end{pmatrix}$$

A graph *G* and its adjacency matrix *G*.adjMatrix

Mathlib writes a matrix row by row with the !![…] notation, separating the entries of a row by commas and the rows by semicolons.

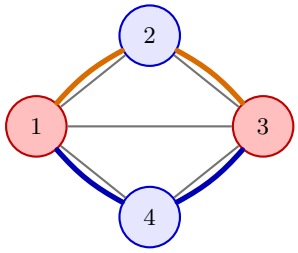
```
#check !![0, 1; 2, 3; 0, 0]  -- !![0, 1; 2, 3; 0, 0] : Matrix (Fin 3) (Fin 2) ℕ
#eval C[4].adjMatrix ℕ       -- !![0, 1, 0, 1; 1, 0, 1, 0; 0, 1, 0, 1; 1, 0, 1, 0]
#eval C[4].adjMatrix (Fin 2) -- !![0, 1, 0, 1; 1, 0, 1, 0; 0, 1, 0, 1; 1, 0, 1, 0]
#eval C[4].adjMatrix (Fin 1) -- !![0, 0, 0, 0; 0, 0, 0, 0; 0, 0, 0, 0; 0, 0, 0, 0]
```

Since a SimpleGraph is symmetric and loopless, its adjacency matrix is symmetric with zero diagonal.

```
#check isSymm_adjMatrix
#check diag_adjMatrix
```

The (u, v) entry of A^n counts the walks of length n from u to v (adjMatrix_pow_apply_eq_card_walk). In particular the diagonal entries of A^2 are the degrees (adjMatrix_mul_self_apply_self).

The walks of a given length between two vertices form a `Finset`, `finsetWalkLength`, so both sides can be computed. [2]



$$\begin{pmatrix} 3 & 1 & 2 & 1 \\ 1 & 2 & 1 & 2 \\ 2 & 1 & 3 & 1 \\ 1 & 2 & 1 & 2 \end{pmatrix}$$

The two walks of length 2 from vertex 1 to vertex 3, and the $(1, 3)$ entry of A^2, which is 2

```
#source finsetWalkLength
#check adjMatrix_pow_apply_eq_card_walk
#check adjMatrix_mul_self_apply_self

#eval (C[5].adjMatrix ℕ ^ 5) 0 0                    -- 2
#eval (C[5].finsetWalkLength 5 0 0).card            -- 2
#eval (K[2, 4].adjMatrix ℕ ^ 2) (.inl 0) (.inl 0)   -- 4
```

Summing the diagonal entries, the trace of A^k is the total number of closed walks of length k. We count the walks with `finsetWalkLength` rather than the set used by `adjMatrix_pow_apply_eq_card_walk`: `card_set_walk_length_eq` relates the two, and `simp` rewrites the set form into the `Finset` form using `Fintype.card_ofFinset`.

```
theorem trace_adjMatrix_pow_eq_sum_card_walk [Fintype V] [DecidableEq V]
    [DecidableRel G.Adj] {R : Type*} [Semiring R] (k : ℕ) :
    (G.adjMatrix R ^ k).trace = ∑ v, (#(G.finsetWalkLength k v v) : R) := by
  simp [Matrix.trace, adjMatrix_pow_apply_eq_card_walk]
```

The adjacency matrix determines the graph, and a bijection of the vertices $\sigma : V \simeq V$ is an automorphism exactly when it preserves the adjacency matrix: relabelling the rows and columns of A by σ leaves A unchanged. In matrix language, the permutation matrix P of σ satisfies $P\ A\ P^{\mathsf{T}} = A$, i.e. P commutes with A. In Lean we relabel with `Matrix.submatrix`, which composes the row and column indices with σ.

A permutation σ of the vertices is an automorphism, i.e. preserves adjacency, iff it preserves the adjacency matrix.

```
theorem adjMatrix_submatrix_eq_iff [DecidableRel G.Adj] {R : Type*} [Semiring R]
    [Nontrivial R] (σ : V ≃ V) :
    (G.adjMatrix R).submatrix σ σ = G.adjMatrix R ↔
      ∀ u v, G.Adj (σ u) (σ v) ↔ G.Adj u v := by
  constructor
  · intro h u v
    have := congrFun₂ h u v
    by_cases h₁ : G.Adj (σ u) (σ v) <;> by_cases h₂ : G.Adj u v <;>
```

[2] Defining `G.adjMatrix R` only requires `DecidableRel G.Adj` of `G`, but a matrix product sums over all of `V`, so results that multiply adjacency matrices assume `[Fintype V]`, and powers also need `[DecidableEq V]`, since the identity matrix `1` is defined by comparing indices. `[G.LocallyFinite]` is not enough: it makes the number of walks of each length between two vertices finite, which is all `finsetWalkLength` needs, but Mathlib's matrix product needs a finite index type.

```
      simp_all [adjMatrix_apply]
  · intro h
    ext u v
    simp [adjMatrix_apply, h]
```

For an automorphism `f`, Mathlib's `Iso.reindex_adjMatrix` already gives `(G.adjMatrix R).reindex f f = G.adjMatrix R`. Relabelling rows and columns is a ring isomorphism (`Matrix.reindexRingEquiv`), so it commutes with taking powers, and hence every power `G.adjMatrix R ^ k` is fixed by `f` as well. Combined with the walk-counting formula, this shows that automorphisms preserve the number of walks of each length between two vertices. In a vertex-transitive graph it follows that every vertex lies on the same number of closed walks of length `k`.

In a vertex-transitive graph the diagonal entries of each power of the adjacency matrix are all equal.

```
theorem IsVertexTransitive.adjMatrix_pow_apply_self [Fintype V] [DecidableEq V]
    [DecidableRel G.Adj] {R : Type*} [Semiring R] (h : G.IsVertexTransitive)
    (k : ℕ) (u v : V) :
    (G.adjMatrix R ^ k) u u = (G.adjMatrix R ^ k) v v := by
  obtain ⟨f, rfl⟩ := h u v
  have hA : (G.adjMatrix R).reindex f f = G.adjMatrix R := f.reindex_adjMatrix R
  have hk : (G.adjMatrix R ^ k).reindex f f = G.adjMatrix R ^ k := by
    rw [← coe_reindexRingEquiv, map_pow, coe_reindexRingEquiv, hA]
  simpa using congrFun₂ hk (f u) (f u)
```

So, in a vertex-transitive graph, the $\mathrm{tr}(A^k)$ closed walks of length `k` are shared equally among its vertices.

```
theorem IsVertexTransitive.card_walk_self_eq_trace_div [Fintype V] [DecidableEq V]
    [DecidableRel G.Adj] (h : G.IsVertexTransitive) (k : ℕ) (v : V) :
    #(G.finsetWalkLength k v v) = (G.adjMatrix ℕ ^ k).trace / Fintype.card V := by
  -- every vertex lies in as many closed walks of length k as v
  have hu (u : V) : #(G.finsetWalkLength k u u) = #(G.finsetWalkLength k v v) := by
    simpa [adjMatrix_pow_apply_eq_card_walk] using
      h.adjMatrix_pow_apply_self (R := ℕ) k u v
  simp_rw [trace_adjMatrix_pow_eq_sum_card_walk, Nat.cast_id,
    Finset.sum_eq_card_nsmul fun u _ => hu u, Finset.card_univ, smul_eq_mul,
    Nat.mul_div_cancel_left _ (Fintype.card_pos_iff.2 ⟨v⟩)]
```

For example, the cycle `C[5]` is vertex-transitive (`isVertexTransitive_cycleGraph`) and has `10` closed walks of length `5`, so each of its `5` vertices lies on two of them: the two directions round the cycle.

```
#eval (C[5].adjMatrix ℕ ^ 5).trace -- 10

example (v : Fin 5) : #(C[5].finsetWalkLength 5 v v) = 2 := by
  rw [(isVertexTransitive_cycleGraph 5).card_walk_self_eq_trace_div]
  decide
```

Eigenvalues

The eigenvalues of the adjacency matrix make up the *spectrum* of `G`, a central object of algebraic graph theory. (Mathlib's `spectrum R M` is the set of `ξ : R` for which `ξ • 1 - M` is not invertible; for a matrix over a field these are its eigenvalues.)

A d-regular graph has eigenvalue d, with the all-ones eigenvector. (In The Laplacian (p. 372) we show that its multiplicity is the number of connected components.)

```
#check IsRegularOfDegree.mem_spectrum_adjMatrix
```

Isomorphic graphs have the same adjacency matrix up to relabelling, so they have the same characteristic polynomial.

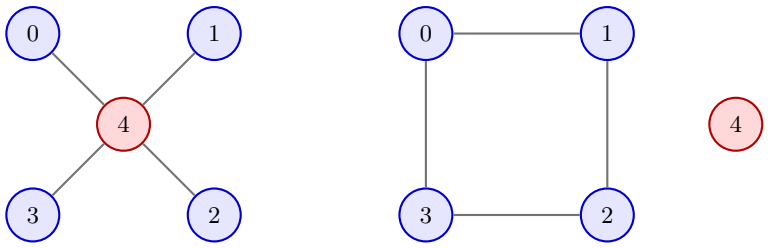
```
theorem Iso.charpoly_adjMatrix [Fintype V] [DecidableEq V] [DecidableRel G.Adj]
    {W : Type*} [Fintype W] [DecidableEq W] {H : SimpleGraph W} [DecidableRel H.Adj]
    {R : Type*} [CommRing R] (f : G ≃g H) :
    (G.adjMatrix R).charpoly = (H.adjMatrix R).charpoly := by
  rw [← f.reindex_adjMatrix R, charpoly_reindex]
```

In particular, isomorphic graphs have the same spectrum. Mathlib's `spectrum` is a set, so it forgets multiplicities, and the spectrum of a matrix is determined by its characteristic polynomial (`mem_spectrum_iff_not_isUnit_eval_charpoly`).

```
theorem Iso.spectrum_adjMatrix [Fintype V] [DecidableEq V] [DecidableRel G.Adj]
    {W : Type*} [Fintype W] [DecidableEq W] {H : SimpleGraph W} [DecidableRel H.Adj]
    {R : Type*} [CommRing R] (f : G ≃g H) :
    spectrum R (G.adjMatrix R) = spectrum R (H.adjMatrix R) := by
  ext ξ
  simp_rw [mem_spectrum_iff_not_isUnit_eval_charpoly, f.charpoly_adjMatrix]
```

The converse fails: the star `K[4, 1]` and the disjoint union `C[4] ⊕g K[1]` of a 4-cycle and a single vertex are two graphs on the same vertex type `Fin 4 ⊕ Fin 1`, with the same characteristic polynomial `X ^ 5 - 4 * X ^ 3`, but are not isomorphic. Mathlib's polynomials are not computable, and neither `#eval` nor `decide` can help directly with proving this.

We relabel the vertices as `Fin 5` (using `finSumFinEquiv`) and use `decide` only to check the resulting integer matrix, a finite computation. We then expand the determinant along its rows with `det_succ_row_zero`, and finish the proof with `ring`.



Left: the star `K[4, 1]`. Right: the disjoint union of `C[4]` and `K[1]`. Vertices are numbered by `finSumFinEquiv` as in the matrices below. Both graphs have characteristic polynomial `X ^ 5 - 4 * X ^ 3`

```
example : (K[4, 1].adjMatrix ℤ).charpoly = X ^ 5 - 4 * X ^ 3 := by
  rw [← charpoly_reindex finSumFinEquiv]
  have hM : (K[4, 1].adjMatrix ℤ).reindex finSumFinEquiv finSumFinEquiv =
      !![0, 0, 0, 0, 1; 0, 0, 0, 0, 1; 0, 0, 0, 0, 1;
        0, 0, 0, 0, 1; 1, 1, 1, 1, 0] := by
    decide
  rw [hM, charpoly, det_succ_row_zero]
  -- expanding the determinant along its rows leaves a polynomial identity
  suffices X * (X * (X * (X * X + -1) + -X) + (-1) ^ 3 * (X * X)) +
      -((-1) ^ 4 * (X * (X * X))) = X ^ 5 - 4 * X ^ 3 by
```

```
    simpa [Fin.sum_univ_succ, det_succ_row_zero, charmatrix_apply, submatrix_apply,
      Fin.succAbove]
  ring

example : ((C[4] ⊕g K[1]).adjMatrix ℤ).charpoly = X ^ 5 - 4 * X ^ 3 := by
  rw [← charpoly_reindex finSumFinEquiv]
  have hM : ((C[4] ⊕g K[1]).adjMatrix ℤ).reindex finSumFinEquiv finSumFinEquiv =
      !![0, 1, 0, 1, 0; 1, 0, 1, 0, 0; 0, 1, 0, 1, 0;
        1, 0, 1, 0, 0; 0, 0, 0, 0, 0] := by
    decide
  rw [hM, charpoly, det_succ_row_zero]
  -- expanding the determinant along its rows leaves a polynomial identity
  suffices X * (X * (X * (X * X) + -X) + -(X * X)) +
      (-(X * (X * X)) + (-1) ^ 3 * (X * (X * X))) = X ^ 5 - 4 * X ^ 3 by
    simpa [Fin.sum_univ_succ, det_succ_row_zero, charmatrix_apply, submatrix_apply,
      Fin.succAbove]
  ring
```

Graphs with the same eigenvalues, counted with multiplicity, cannot be told apart by counting edges or triangles. The eigenvalues determine the trace of every power A^k, and hence the number of closed walks of each length (`trace_adjMatrix_pow_eq_sum_card_walk`), and $\mathrm{tr}(A^2)$ is twice the number of edges while $\mathrm{tr}(A^3)$ is six times the number of triangles. (Both of our graphs have 4 edges and no triangles.)

But `K[4, 1]` has no isolated vertices, while `C[4] ⊕g K[1]` obviously does.

```
example : IsEmpty (K[4, 1] ≃g C[4] ⊕g K[1]) := ⟨fun e => by
  have h₁ : ∀ v, ∃ w, K[4, 1].Adj v w := by decide
  have h₂ : ∀ w, ¬(C[4] ⊕g K[1]).Adj (.inr 0) w := by decide
  obtain ⟨w, hw⟩ := h₁ (e.symm (.inr 0))
  exact h₂ (e w) (by simpa using e.map_adj_iff.2 hw)⟩
```

Eigenvalues of Mycielskians

Apart from the eigenvalue d of a d-regular graph, Mathlib has no eigenvalues of particular graphs. But we can often find them by exhibiting eigenvectors, as Mathlib does for d. As an example we show that eigenvalues of G give eigenvalues of its Mycielskian $\mu(G)$, scaled by the golden ratio.

Mathlib writes `M *ᵥ x` (`Matrix.mulVec`) for the product of a matrix M and a column vector x, whose entry at i is $\sum_j M_{ij} x_j$. So a nonzero vector x is an eigenvector of M with eigenvalue ξ iff $Mx = \xi x$.

We list the vertices of $\mu(G)$ in the order given by their type `Option (V ⊕ V)`: first the apex `none`, then the left copies v^{l}, then the right copies v^{r}. Let A be the adjacency matrix of G, and write $\mathbf{1}$ for the all-ones vector. A left copy v^{l} is adjacent to both copies of each neighbour of v, and a right copy v^{r} is adjacent to the left copies of the neighbours of v and to the apex. So the adjacency matrix of $\mu(G)$ has the block form shown in the figure.

$$\begin{array}{c} \\ \text{apex} \\ \text{left} \\ \text{right} \end{array}\!\!\begin{array}{c} \begin{array}{ccc} \text{apex} & \text{left} & \text{right} \end{array} \\ \begin{pmatrix} 0 & 0 & \mathbf{1}^\mathsf{T} \\ 0 & A & A \\ \mathbf{1} & A & 0 \end{pmatrix} \end{array}\!\!\begin{pmatrix} 0 \\ tx \\ x \end{pmatrix} = \begin{pmatrix} \mathbf{1}^\mathsf{T} x \\ (t+1)Ax \\ tAx \end{pmatrix}$$

The adjacency matrix of the Mycielskian, applied to a vector with blocks `0`, `t • x` and `x`

Now let `x` be an eigenvector of `A` with eigenvalue `ξ` whose entries sum to zero, i.e. `x` is orthogonal to `1`: so `A x = ξ x` and `1ᵀ x = 0`. The vector `y` with blocks `0`, `t • x` and `x` is sent to the vector with blocks `0`, `(t + 1) • ξ • x` and `t • ξ • x`. So `y` is an eigenvector with eigenvalue `t * ξ` provided `t + 1 = t ^ 2`. The roots of this quadratic are the golden ratio `φ = (1 + √5) / 2` and its conjugate `ψ = (1 - √5) / 2`. In Mathlib these are `Real.goldenRatio` and `Real.goldenConj`, with notation `φ` and `ψ` in the `goldenRatio` scope, and `Real.goldenRatio_sq` and `Real.goldenConj_sq` say that they are these roots. [3]

In Lean, if we define `y` by pattern matching on the three kinds of vertex and mark the definition `@[simp]`, then `simp` evaluates `y` at `none`, `v ˡ` and `v ʳ` but leaves `y` alone when it isn't applied to a vertex.

The vector on the vertices of `mμ(G)` with value `0` at `none`, `t * x v` at `v ˡ` and `x v` at `v ʳ`.

```
@[simp]
def mycielskianLift {R : Type*} [Zero R] [Mul R] (t : R) (x : V → R) :
    Option (V ⊕ V) → R
  | none => 0
  | vˡ => t * x v
  | vʳ => x v
```

The proof is the computation from the figure, with one case for each kind of vertex. Sums over `Option (V ⊕ V)` split in the same order: `Fintype.sum_option` separates the term at `none`, and `Fintype.sum_sum_type` splits the rest into the left and right copies. We work over any commutative ring, and with any root `t` of `t ^ 2 = t + 1`.

If `A *ᵥ x = ξ • x`, the entries of `x` sum to zero and `t ^ 2 = t + 1`, then `mycielskianLift t x` satisfies the corresponding equation for `mμ(G)`, with eigenvalue `t * ξ`.

```
theorem adjMatrix_mycielskian_mulVec_mycielskianLift [Fintype V]
    [DecidableRel G.Adj] {R : Type*} [CommRing R] {t ξ : R} (ht : t ^ 2 = t + 1)
    {x : V → R} (hx : G.adjMatrix R *ᵥ x = ξ • x) (hsum : ∑ v, x v = 0) :
    mμ(G).adjMatrix R *ᵥ mycielskianLift t x = (t * ξ) • mycielskianLift t x := by
  -- the entries of A *ᵥ x and of A *ᵥ (t • x)
  have hv (v : V) : ∑ u, (if G.Adj v u then x u else 0) = ξ * x v := by
    simpa [mulVec, dotProduct, adjMatrix_apply] using congrFun hx v
  have htv (v : V) :
      ∑ u, (if G.Adj v u then t * x u else 0) = t * (ξ * x v) := by
    simp_rw [← hv, Finset.mul_sum, mul_ite, mul_zero]
  -- at a left copy we need t ^ 2 = t + 1
```

[3] An eigenvalue `ξ` of `G` need not have an eigenvector whose entries sum to zero, and then `φ * ξ` and `ψ * ξ` need not be eigenvalues of `mμ(G)`. For example, the only eigenvectors of `K[2]` with eigenvalue `1` are the constant vectors, and neither `φ` nor `ψ` is an eigenvalue of `mμ(K[2])`, which is the `5`-cycle (see below). In a `d`-regular graph, however, every eigenvalue other than `d` has such an eigenvector: eigenvectors of a symmetric matrix with distinct eigenvalues are orthogonal, and `1` is an eigenvector with eigenvalue `d`.

```
  have hleft (v : V) : t * (ξ * x v) + ξ * x v = t * ξ * (t * x v) := by
    -- linear_combination e closes the goal if ring shows it is e rearranged
    linear_combination -(ξ * x v) * ht
  ext (_ | v | v) <;> simp_all [mulVec, dotProduct, mul_assoc]
```

Since `mycielskianLift t x` takes the value `x v` at v^{r}, it is nonzero whenever `x` is.

```
lemma mycielskianLift_ne_zero {R : Type*} [Zero R] [Mul R] (t : R) {x : V → R}
    (hx : x ≠ 0) : mycielskianLift t x ≠ 0 := by
  contrapose hx
  ext v
  simpa using congrFun hx v.r
```

An eigenvector gives an element of the spectrum. Mathlib's eigenvalue API (`Module.End.HasEigenvalue`) is stated for linear maps rather than matrices, so, as in Mathlib's proof of `IsRegularOfDegree.mem_spectrum_adjMatrix`, we pass from the matrix to the linear map `Matrix.toLin'` it defines. This map has the same spectrum (`Matrix.spectrum_toLin'`), and eigenvalues of a linear map lie in its spectrum (`Module.End.HasEigenvalue.mem_spectrum`). The proof first shows that `t * ξ` lies in the spectrum for every real root `t` of `t ^ 2 = t + 1`, and then applies this to φ and ψ, using `Real.goldenRatio_sq` and `Real.goldenConj_sq`.

If `x ≠ 0` is an eigenvector of `A` with eigenvalue `ξ` whose entries sum to zero, then `φ * ξ` and `ψ * ξ` are eigenvalues of `mμ(G)`.

```
open scoped goldenRatio in
theorem mem_spectrum_adjMatrix_mycielskian [Fintype V] [DecidableEq V]
    [DecidableRel G.Adj] {ξ : ℝ} {x : V → ℝ} (hx0 : x ≠ 0)
    (hx : G.adjMatrix ℝ *ᵥ x = ξ • x) (hsum : ∑ v, x v = 0) :
    φ * ξ ∈ spectrum ℝ (mμ(G).adjMatrix ℝ) ∧
      ψ * ξ ∈ spectrum ℝ (mμ(G).adjMatrix ℝ) := by
  have key {t : ℝ} (ht : t ^ 2 = t + 1) :
      t * ξ ∈ spectrum ℝ (mμ(G).adjMatrix ℝ) := by
    rw [← spectrum_toLin']
    apply Module.End.HasEigenvalue.mem_spectrum
    exact Module.End.hasEigenvalue_of_hasEigenvector
      ⟨Module.End.mem_genEigenspace_one.2 (by
        simpa using adjMatrix_mycielskian_mulVec_mycielskianLift ht hx hsum),
       mycielskianLift_ne_zero t hx0⟩
  exact ⟨key Real.goldenRatio_sq, key Real.goldenConj_sq⟩
```

For example, `K[2]` has the eigenvector `![1, -1]` with eigenvalue `-1`, and its entries sum to zero. Since `mμ(K[2])` is isomorphic to `C[5]` (`mycielskianCompleteTwoIsoCycleFive`), the 5-cycle has eigenvalues `-φ` and `-ψ`.

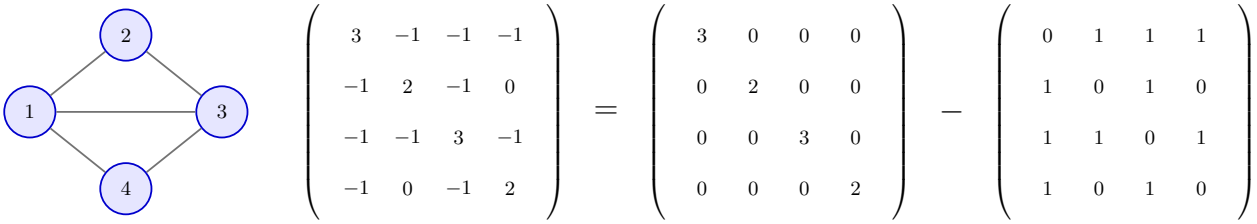
```
open scoped goldenRatio in
example :
    -φ ∈ spectrum ℝ (C[5].adjMatrix ℝ) ∧ -ψ ∈ spectrum ℝ (C[5].adjMatrix ℝ) := by
  have hK : K[2].adjMatrix ℝ *ᵥ ![1, -1] = (-1 : ℝ) • ![1, -1] := by
    ext i
    fin_cases i <;> simp [mulVec, dotProduct, Fin.sum_univ_two, adjMatrix_apply]
  rw [← mycielskianCompleteTwoIsoCycleFive.spectrum_adjMatrix]
  simpa using mem_spectrum_adjMatrix_mycielskian (by simp) hK (by simp)
```

For a d-regular graph `G` this construction accounts for all but three eigenvalues of `mμ(G)`, counted with multiplicity; the other three come from eigenvectors that are constant on the left and on the right copies. For `C[5]` these are `2`, `-φ` and `-ψ`, so `-φ` and `-ψ` in fact have multiplicity two (Exercise 9).

12.3 The Laplacian

The *degree matrix* (`degMatrix`) is the diagonal matrix of degrees, so its diagonal entries form the `degreeMultiset`, and listing them in non-increasing order gives the `degreeSequence`.

The *Laplacian* (`lapMatrix`) is given by `L = D - A`.



A graph `G` and its Laplacian `L = D - A`

```
#source degMatrix -- Matrix V V R := Matrix.diagonal (G.degree ·)
#source lapMatrix -- Matrix V V R := G.degMatrix R - G.adjMatrix R

#eval C[4].lapMatrix ℤ -- !![2, -1, 0, -1; -1, 2, -1, 0; 0, -1, 2, -1; -1, 0, -1, 2]
```

The key property of the Laplacian is its quadratic form: for `x : V → R` we have `xᵀ L x = ∑ (x u - x v)²`, summed over the edges `u – v`. (Mathlib sums over ordered pairs of adjacent vertices and divides by 2.)

```
#check lapMatrix_toLinearMap₂'
-- xᵀ L x = (∑ i, ∑ j, if G.Adj i j then (x i - x j) ^ 2 else 0) / 2
```

Over `ℝ`, this form is a sum of squares, so the Laplacian is positive semidefinite. It vanishes iff `x` is constant along every edge, and hence on every connected component. For a positive semidefinite matrix the form vanishes exactly on the *kernel*, the set of vectors `x` with `L *ᵥ x = 0`, so the kernel of `L` consists of the functions that are constant on components. In particular the all-ones vector `1` lies in the kernel, so `0` is always an eigenvalue of the Laplacian.

```
#check posSemidef_lapMatrix -- (G.lapMatrix R).PosSemidef

#check lapMatrix_mulVec_eq_zero_iff_forall_reachable
-- G.lapMatrix ℝ *ᵥ x = 0 ↔ ∀ i j, G.Reachable i j → x i = x j

#check zero_mem_spectrum_lapMatrix -- 0 ∈ spectrum R (G.lapMatrix R)
```

Counting dimensions gives the first theorem of spectral graph theory: the number of connected components of `G` is the dimension of the kernel of its Laplacian, i.e. the multiplicity of the eigenvalue `0`. In Lean, `M.toLin'` is the matrix `M` viewed as a linear map `(V → ℝ) → (V → ℝ)`, `LinearMap.ker` is its kernel, and `Module.finrank ℝ` gives the dimension of a subspace.

```
#check card_connectedComponent_eq_finrank_ker_toLin'_lapMatrix
```

```
example : Module.finrank ℝ ((C[3] ⊕g C[4]).lapMatrix ℝ).toLin'.ker = 2 := by
  rw [← card_connectedComponent_eq_finrank_ker_toLin'_lapMatrix]
  decide
```

For a d-regular graph the degree matrix is $d \cdot 1$, so $L = d \cdot 1 - A$, and the kernel of L is the eigenspace of A for the eigenvalue d. So the multiplicity of d as an eigenvalue of A is the number of connected components. This sharpens `IsRegularOfDegree.mem_spectrum_adjMatrix` above. In Lean, `Module.End.eigenspace f ξ` is the kernel of `f - ξ • 1`.

```
theorem IsRegularOfDegree.finrank_eigenspace_adjMatrix [Fintype V] [DecidableEq V]
    [DecidableRel G.Adj] {d : ℕ} (hd : G.IsRegularOfDegree d) :
    Module.finrank ℝ (Module.End.eigenspace (G.adjMatrix ℝ).toLin' d) =
      Fintype.card G.ConnectedComponent := by
  rw [card_connectedComponent_eq_finrank_ker_toLin'_lapMatrix,
    Module.End.eigenspace_def, lapMatrix, hd.degMatrix_eq ℝ]
  have : (G.adjMatrix ℝ).toLin' - (d : ℝ) • 1 =
      -((d : Matrix V V ℝ) - G.adjMatrix ℝ).toLin' := by
    ext i j
    simp [toLin'_apply, Pi.single_apply, Matrix.natCast_apply, eq_comm]
  rw [this, LinearMap.ker_neg]

end adjmatrix
```

For example, the hypercube `Q[n]` is n-regular and connected, so n is a simple eigenvalue of its adjacency matrix.

```
example (n : ℕ) :
    Module.finrank ℝ (Module.End.eigenspace (Q[n].adjMatrix ℝ).toLin' n) = 1 := by
  rw [(IsRegularOfDegree.hypercube n).finrank_eigenspace_adjMatrix]
  -- Q[n] is connected, so it has exactly one connected component
  rw [Fintype.card_eq_nat_card]
  exact card_connectedComponent_eq_one_iff.2 (connected_hypercube n)
```

The *incidence matrix* (`incMatrix`) has rows indexed by vertices and columns by `Sym2 V`, with `(v, e)` entry `1` iff `v` is an endpoint of the edge `e`. The product of the incidence matrix and its transpose is `D + A`, the *signless* Laplacian.

```
#check incMatrix_mul_transpose
```

12.4 Strongly Regular Graphs

A graph is *strongly regular* (`IsSRGWith`) with parameters `(n, k, ℓ, μ)` if it has `n` vertices, is `k`-regular, any two adjacent vertices have exactly ℓ common neighbours, and any two distinct non-adjacent vertices have exactly `μ` common neighbours.

```
#source IsSRGWith
```

The Petersen graph `petersenGraph = Kn[5, 2]` is strongly regular with parameters `(10, 3, 0, 1)`. Two adjacent vertices are disjoint 2-subsets of `Fin 5`, and no third 2-subset is disjoint from both, so they have no common neighbours. Two distinct non-adjacent vertices share exactly one element, so their union has three elements, and the remaining two elements of `Fin 5` form their only common neighbour. We can check each field with `decide`, but two need an initial `intro` since

`decide` needs a `Decidable` instance for the exact statement and neither `IsRegularOfDegree` nor `Pairwise` has one.

```
lemma IsSRGWith.petersenGraph : petersenGraph.IsSRGWith 10 3 0 1 where
  card := by decide
  regular := by intro v; decide +revert
  of_adj := by decide
  of_not_adj := by intro u v; decide +revert
```

Counting the paths of length `2` from a fixed vertex `v` to a vertex that is not adjacent to `v` in two different ways shows that the parameters satisfy `k * (k - ℓ - 1) = (n - k - 1) * μ`. For the Petersen graph this is `3 * 2 = 6 * 1`.

```
#check IsSRGWith.param_eq
-- G.IsSRGWith n k ℓ μ → 0 < n → k * (k - ℓ - 1) = (n - k - 1) * μ
```

The diameter of a strongly regular graph with `μ ≠ 0` that is not complete is `2`.

```
#check IsSRGWith.ediam_eq_two -- G.IsSRGWith n k ℓ μ → G ≠ ⊤ → μ ≠ 0 → G.ediam = 2
```

This gives us the diameter of the Petersen graph. We cannot ask `decide` for it directly, since `edist` is not computable (Distance Layers (p. 347)), but `decide` can check the facts we need to apply `IsSRGWith.ediam_eq_two`.

```
lemma ediam_petersenGraph : petersenGraph.ediam = 2 := by
  have : Nontrivial {s : Finset (Fin 5) // s.card = 2} :=
    Fintype.one_lt_card_iff_nontrivial.mp (by decide)
  have hne : ¬petersenGraph.Adj ⟨{0, 1}, rfl⟩ ⟨{0, 2}, rfl⟩ := by decide
  refine IsSRGWith.petersenGraph.ediam_eq_two (fun h => hne ?_) one_ne_zero
  rw [h, top_adj]
  decide
```

The complement of a strongly regular graph is strongly regular. The complement of the Petersen graph is the line graph of `K[5]`, with parameters `(10, 6, 3, 4)`.

```
example : petersenGraphᶜ.IsSRGWith 10 6 3 4 :=
  IsSRGWith.petersenGraph.compl
```

More generally, the line graph `L[K[n]]` is strongly regular with parameters `(n * (n - 1) / 2, 2 * (n - 2), n - 2, 4)`. Its vertices are the edges of `K[n]`. Two edges `{a, b}` and `{a, c}` that share an endpoint have the `n - 2` common neighbours `{b, c}` and `{a, x}` for `x ∉ {a, b, c}`, while two disjoint edges have the `4` common neighbours that join an endpoint of one to an endpoint of the other. Mathlib has no API yet for counting neighbours in line graphs, so a general proof would be tedious, but `decide` can check the case `n = 7`.

```
example : L[K[7]].IsSRGWith 21 10 5 4 where
  card := by decide
  regular := by intro v; decide +revert
  of_adj := by decide
  of_not_adj := by intro u v; decide +revert
```

The Paley graph of a field with `q ≡ 1 mod 4` elements (`paleyGraph`) is strongly regular with parameters `(q, (q - 1) / 2, (q - 5) / 4, (q - 1) / 4)`. The general proof counts the elements `x`

for which x and x - d are both nonzero squares, using character sums over the field, but for q = 17 we can again use decide.

```
example : (paleyGraph (ZMod 17)).IsSRGWith 17 8 3 4 where
  card := by decide
  regular := by intro v; decide +revert
  of_adj := by decide
  of_not_adj := by intro u v; decide +revert
```

In terms of the adjacency matrix A, the definition of a strongly regular graph says exactly that A ^ 2 = k • 1 + ℓ • A + μ • (J - 1 - A), where J is the all-ones matrix and J - 1 - A is the adjacency matrix of the complement.

```
#check IsSRGWith.matrix_eq
```

This quadratic equation forces A to have at most three distinct eigenvalues. It is the starting point of the Hoffman–Singleton theorem: a Moore graph of diameter 2 (attaining moore_bound) must have girth 5 and be strongly regular with ℓ = 0 and μ = 1. For such a graph the equation reads A ^ 2 + A - (k - 1) • 1 = J, and a linear-algebra argument then shows that its degree k must be 2, 3, 7 or 57. (Existence of such a graph with k = 57 is currently an open problem.)

Moore Graphs of Diameter Two

For each of $k = 2$, 3 and 7 there is exactly one Moore graph of diameter 2 (see Moore Graphs (p. 354)). For $k = 2$ and $k = 3$ these are C[5] and the Petersen graph, and for $k = 7$ it is the *Hoffman–Singleton graph* [Hoffman–Singleton 1960].

Strongly regular graphs with ℓ = 0 and μ = 1 have girth at least 5. Since ℓ = 0, adjacent vertices have no common neighbours, so there are no triangles. Since ℓ = 0 and μ = 1, two distinct vertices have at most one common neighbour, so by cycleGraph_four_free_iff_common_neighbor_unique there is no copy of C[4].

```
lemma IsSRGWith.five_le_egirth {V : Type*} [Fintype V] {G : SimpleGraph V}
    [DecidableRel G.Adj] {n k : ℕ} (hG : G.IsSRGWith n k 0 1) : 5 ≤ G.egirth := by
  refine five_le_egirth_of_cliqueFree_three_of_free (fun t ht => ?_) ?_
  · classical
    obtain ⟨a, b, c, hab, hac, hbc, rfl⟩ := is3Clique_iff.mp ht
    exact (Fintype.card_eq_zero_iff.mp (hG.of_adj a b hab)).false ⟨c, hac, hbc⟩
  · refine cycleGraph_four_free_iff_common_neighbor_unique.mpr
      fun x y u v hxu huy hxv hvy hxy => ?_
    have h1 : Fintype.card (G.commonNeighbors x y) ≤ 1 := by
      by_cases hadj : G.Adj x y
      · simp [hG.of_adj x y hadj]
      · simp [hG.of_not_adj hxy hadj]
    exact congrArg Subtype.val ◁
      Fintype.card_le_one_iff.mp h1 ⟨u, hxu, huy.symm⟩ ⟨v, hxv, hvy.symm⟩
```

The Petersen graph has 10 = 1 + 3 * (1 + 2) vertices, is 3-regular and has diameter 2 (ediam_petersenGraph), so it attains the Moore bound with d = 2 and $k = 2$. Its girth is 5, so it attains the cage bound as well. The lower bound comes from IsSRGWith.five_le_egirth. For the upper bound we exhibit a copy of C[5]: the pentagon {0, 1} – {2, 3} – {0, 4} – {1, 2} – {3, 4} – {0, 1}, and decide checks that the map is an injective homomorphism.

```
lemma egirth_petersenGraph : petersenGraph.egirth = 5 := by
  refine le_antisymm ?_ IsSRGWith.petersenGraph.five_le_egirth
  have h5 : C[5] ⊑ petersenGraph :=
    ⟨⟨![⟨{0, 1}, rfl⟩, ⟨{2, 3}, rfl⟩, ⟨{0, 4}, rfl⟩, ⟨{1, 2}, rfl⟩,
      ⟨{3, 4}, rfl⟩], by decide⟩, by decide⟩⟩
  exact h5.egirth_le.trans ((egirth_cycleGraph 2).trans_le (by norm_num))
```

The Hoffman–Singleton graph has `50 = 1 + 7 * (1 + 6)` vertices. We build it from five pentagons `P₀, …, P₄` and five pentagrams `Q₀, …, Q₄`, each with vertices `0, …, 4`. Taking all arithmetic mod 5, vertex `j` of the pentagon `Pₕ` is adjacent to `j - 1` and `j + 1` in `Pₕ`, vertex `j` of the pentagram `Qᵢ` is adjacent to `j - 2` and `j + 2` in `Qᵢ`, and vertex `j` of `Pₕ` is adjacent to vertex `h * i + j` of `Qᵢ` for every `i`. So each vertex has 2 neighbours in its own pentagon or pentagram and one in each of the five of the other kind, and the graph is 7-regular.

In Lean the vertex type is a sum of two copies of `ZMod 5 × ZMod 5`, one for the pentagons and one for the pentagrams, and we define adjacency by cases, as for the Mycielskian. We use `ZMod 5` rather than `Fin 5` since the construction uses ring arithmetic mod 5, and Mathlib does not make `Fin 5` a ring.

The vertex type of the Hoffman–Singleton graph: `Sum.inl (h, j)` is vertex `j` of the pentagon `Pₕ` and `Sum.inr (i, j)` is vertex `j` of the pentagram `Qᵢ`.

```
abbrev HoffmanSingletonVert := (ZMod 5 × ZMod 5) ⊕ (ZMod 5 × ZMod 5)
```

The Hoffman–Singleton graph: vertex `j` of the pentagon `Pₕ` is adjacent to `j ± 1` in `Pₕ` and to `h * i + j` in the pentagram `Qᵢ`, and vertex `j` of `Qᵢ` is adjacent to `j ± 2` in `Qᵢ`.

```
def hoffmanSingletonGraph : SimpleGraph HoffmanSingletonVert where
  Adj x y := match x, y with
    | .inl (h, j), .inl (h', j') => h = h' ∧ (j' = j + 1 ∨ j = j' + 1)
    | .inr (i, j), .inr (i', j') => i = i' ∧ (j' = j + 2 ∨ j = j' + 2)
    | .inl (h, j), .inr (i, k) => k = h * i + j
    | .inr (i, k), .inl (h, j) => k = h * i + j
  symm := ⟨by rintro (⟨_, _⟩ | ⟨_, _⟩) (⟨_, _⟩ | ⟨_, _⟩) <;> decide +revert⟩
  loopless := ⟨by rintro (⟨_, _⟩ | ⟨_, _⟩) <;> decide +revert⟩
```

As we did for the Mycielskian, we can get decidable adjacency by splitting into cases.

The Hoffman–Singleton graph has decidable adjacency.

```
instance hoffmanSingletonGraphDecidableRelAdj :
    DecidableRel hoffmanSingletonGraph.Adj := fun x y => by
  rcases x with ⟨_, _⟩ | ⟨_, _⟩ <;> rcases y with ⟨_, _⟩ | ⟨_, _⟩ <;>
    unfold hoffmanSingletonGraph <;> infer_instance
```

The Hoffman–Singleton graph is strongly regular with parameters `(50, 7, 0, 1)`, and as for the Petersen graph `decide` can check each field. The two fields about common neighbours range over all 2500 pairs of vertices, so for these we use `decide +kernel`. This leaves the check to the kernel alone, instead of first evaluating the statement in the elaborator, and saves several seconds.

```
lemma IsSRGWith.hoffmanSingletonGraph :
    hoffmanSingletonGraph.IsSRGWith 50 7 0 1 where
  card := by decide
  regular := by intro v; decide +revert
  of_adj := by decide +kernel
```

```
  of_not_adj := by intro u v; decide +kernel +revert
```

So, as for the Petersen graph, it has diameter `2` by `IsSRGWith.ediam_eq_two` and girth `5` by `IsSRGWith.five_le_egirth`, with the pentagon P_0 giving a copy of `C[5]`. Hence it attains both the Moore bound and the cage bound with `d = 6` and `k = 2`.

```
lemma ediam_hoffmanSingletonGraph : hoffmanSingletonGraph.ediam = 2 := by
  have hne : ¬hoffmanSingletonGraph.Adj (.inl (0, 0)) (.inl (0, 2)) := by decide
  refine IsSRGWith.hoffmanSingletonGraph.ediam_eq_two (fun h => hne ?_) one_ne_zero
  rw [h, top_adj]
  decide


lemma egirth_hoffmanSingletonGraph : hoffmanSingletonGraph.egirth = 5 := by
  refine IsSRGWith.hoffmanSingletonGraph.five_le_egirth.antisymm' ?_
  have h5 : C[5] ⊑ hoffmanSingletonGraph :=
    ⟨⟨fun j => .inl (0, j), by decide⟩, by decide⟩
  exact h5.egirth_le.trans ((egirth_cycleGraph 2).trans_le (by norm_num))
```

12.5 The Hypercube and the Hamming Cube

When we first introduced the hypercube we gave two other constructions. One of these was the Hamming cube `hypercubeHamming`: the graph on `Fin n → Bool` in which two vertices are adjacent iff their Hamming distance, computed by Mathlib's `hammingDist` for Pi-types, is one. Having identified the graph distance in `Q[n]` with the Hamming distance (`edist_eq_hammingDist`), we are now in a position to prove that this construction agrees with the one we adopted, i.e. `Q[n] ≃g hypercubeHamming n`.

First we need to identify the two vertex types, i.e. give an equiv `HypercubeVert n ≃ (Fin n → Bool)`. We build this by recursion on `n`. The base case holds since both types have exactly one element. In the inductive step we send `(v, b)` to the function whose first `n` coordinates are those of `v` and whose last coordinate is `b`. Mathlib's `Fin.snoc`, which appends an element at the end of a tuple, does exactly this (and `Fin.snocEquiv` is `Fin.snoc` as an equiv).

```
#check Fin.snoc       -- append an element at the end of an n-tuple
#check Fin.snocEquiv  -- Fin.snoc as an equiv
```

The equiv between the two vertex types of the hypercube, `HypercubeVert n` and `Fin n → Bool`, sending `(v, b)` to `Fin.snoc (equivPi n v) b`, so `b` is the last coordinate.

```
def HypercubeVert.equivPi : (n : ℕ) → HypercubeVert n ≃ (Fin n → Bool)
  | 0 => Equiv.ofUnique Unit _
  | n + 1 => (equivPi n).prodCongr (Equiv.refl _) |>.trans ◁
      (Equiv.prodComm _ _).trans ◁ Fin.snocEquiv fun _ => _
```

Unfolding the equiv one step is definitionally `Fin.snoc`, and marking this as `@[simp]` lets `simp` compute with `HypercubeVert.equivPi` recursively.

```
@[simp]
lemma HypercubeVert.equivPi_succ_apply {n : ℕ} (v : HypercubeVert n) (b : Bool) :
    equivPi (n + 1) (v, b) = Fin.snoc (v.equivPi n) b := rfl
```

To transfer distances along the equiv we need to know how Mathlib's `hammingDist` interacts with `Fin.snoc`: appending a coordinate to each of two vertices increases their Hamming distance by one exactly when the appended coordinates differ. (Note how this mirrors the recursion in `HypercubeVert.hammingDist`.) To prove this we write the Hamming distance as a sum of indicators, using `Finset.card_filter`, and split off the last coordinate with `Fin.sum_univ_castSsucc`.

```
lemma _root_.hammingDist_snoc {n : ℕ} {β : Type*} [DecidableEq β] (x y : Fin n → β)
    (a b : β) :
    hammingDist (Fin.snoc x a : Fin (n + 1) → β) (Fin.snoc y b) =
      hammingDist x y + if a = b then 0 else 1 := by
  simp [hammingDist, Finset.card_filter, Fin.sum_univ_castSucc]
```

The Hamming distance of `HypercubeVert n` vertices agrees with the Hamming distance of the corresponding functions `Fin n → Bool`, by induction on `n`.

```
@[simp]
lemma HypercubeVert.hammingDist_equivPi {n : ℕ} (u v : HypercubeVert n) :
    _root_.hammingDist (u.equivPi n) (v.equivPi n) = u.hammingDist v := by
  induction n with
  | zero => rfl
  | succ n ih =>
    obtain ⟨u, b⟩ := u
    obtain ⟨v, c⟩ := v
    simp [hammingDist_snoc, ih]
```

Adjacency in the Hamming cube is Hamming distance one, by definition.

```
@[simp]
lemma hypercubeHamming_adj {n : ℕ} {x y : Fin n → Bool} :
    (hypercubeHamming n).Adj x y ↔ hammingDist x y = 1 := Iff.rfl
```

We can now give the graph isomorphism: the underlying equiv is `HypercubeVert.equivPi`, and it respects adjacency since both adjacency relations say exactly "Hamming distance one".

The hypercube `Q[n]` (the box product construction) is isomorphic to the Hamming cube construction `hypercubeHamming n` on `Fin n → Bool`.

```
def hypercubeIsoHamming (n : ℕ) : Q[n] ≃g hypercubeHamming n where
  toEquiv := HypercubeVert.equivPi n
  map_rel_iff' := by
    simp [hypercube_adj_iff_hammingDist_eq_one]
```

This isomorphism is a bridge between the two constructions: results proved for `Q[n]` by induction transport to `hypercubeHamming`, while statements that need the coordinates of a vertex, such as identifying the automorphism group of the hypercube, are more naturally proved for `hypercubeHamming` and transported back.

Transporting `edist_eq_hammingDist` along `hypercubeIsoHamming` shows that the graph distance in the Hamming cube is exactly the Hamming distance.

```
lemma edist_hypercubeHamming {n : ℕ} (x y : Fin n → Bool) :
    (hypercubeHamming n).edist x y = hammingDist x y := by
  obtain ⟨u, rfl⟩ := (HypercubeVert.equivPi n).surjective x
  obtain ⟨v, rfl⟩ := (HypercubeVert.equivPi n).surjective y
  exact ((hypercubeIsoHamming n).edist_eq u v).trans (by simp [edist_eq_hammingDist])
```

12.6 The Automorphism Group of the Hypercube*

As a substantial application of the bridge we identify the automorphism group of the hypercube: it is the *hyperoctahedral group*, the wreath product $\mathbb{Z}/2 \wr S_n$ of order $2^n \cdot n!$. Concretely every automorphism of the hypercube is uniquely a composition of a *translation* (`xor` by a fixed vertex) and a *coordinate permutation*, and the wreath product is the semidirect product $(\mathbb{Z}/2)^n \rtimes S_n$ that records how coordinate permutations act on translations.

We work in the Hamming model and transport the result to Q_n at the end. The proof has four steps.

1. Translations and coordinate permutations are automorphisms of the Hamming cube.
2. Together they give a group homomorphism from $(\mathbb{Z}/2)^n \rtimes S_n$ to the Hamming cube's automorphism group.
3. This homomorphism is injective: evaluate at 0 and at the standard basis vectors e_i. (The standard basis vectors are given by `Pi.single` in Lean so our informal e_i refers to `Pi.single i true`. We introduce the notation `e i` below.)
4. It is also surjective. After composing with a translation, an automorphism fixes 0. It then preserves Hamming weight, so it permutes the basis vectors, which gives a permutation σ. Undoing σ leaves an automorphism fixing 0 and every e_i, which is the identity by a rigidity argument: the coordinates of a vertex can be read off from its distances to 0 and to the e_i.

The key to making this precise in Lean is that Mathlib makes `Bool` a `BooleanRing`, with `0 = false`, `1 = true` and addition given by `xor`. Consequently the vertex type `Fin n → Bool` of the Hamming cube is an additive group, namely $(\mathbb{Z}/2)^n$, with pointwise `xor` (`^^`) as addition: translations are simply `(t + ·)`.

```
example {n : ℕ} : AddCommGroup (Fin n → Bool) := inferInstance
example (a b : Bool) : a + b = (a ^^ b) := rfl
```

Addition on `Bool` vectors has characteristic two: negation is the identity and adding the same vector twice gives 0. Mathlib proves both facts in any Boolean ring, as `BooleanRing.neg_eq` and `BooleanRing.add_self`, but it has no instance making a product of Boolean rings a Boolean ring, so they do not apply to `Fin n → Bool` directly. The missing instance is easy: Mathlib already provides the pointwise ring structure on functions, and multiplication is idempotent because it is idempotent in each coordinate.

A product of Boolean rings is a Boolean ring under the pointwise operations.

```
instance _root_.Pi.instBooleanRing {ι : Type*} {R : ι → Type*}
    [∀ i, BooleanRing (R i)] : BooleanRing (∀ i, R i) where
  isIdempotentElem a := funext fun i => BooleanRing.mul_self (a i)
```

The two lemmas are `simp` lemmas only once the `BooleanRing` namespace is open (they are *scoped*), so we open it for the rest of the chapter with `open scoped BooleanRing`.

```
open scoped BooleanRing

example {n : ℕ} (x : Fin n → Bool) : -x = x := by simp
example {n : ℕ} (x : Fin n → Bool) : x + x = 0 := by simp
```

`simp` cannot prove `x + (x + y) = y` from these two lemmas, so we add this cancellation as a `simp` lemma of our own.

```
@[simp]
lemma _root_.boolVec_add_add_cancel {n : ℕ} (x y : Fin n → Bool) :
    x + (x + y) = y := by
  rw [← add_assoc, BooleanRing.add_self, zero_add]
```

The vectors of Hamming weight (`hammingNorm`) one are exactly the standard basis vectors `Pi.single i true` (all coordinates `false` except the `i`-th). For the rest of the chapter we write `e i` for `Pi.single i true`, our informal e_i. The notation is scoped to `hypercubeHamming`, so later files can use it with `open scoped hypercubeHamming`.

`e i` is the standard basis vector `Pi.single i true` of `Fin n → Bool`, which is `true` in coordinate `i` and `false` elsewhere.

```
scoped[SimpleGraph.hypercubeHamming] notation:lead "e " i:max => Pi.single i true

open scoped hypercubeHamming

@[simp]
lemma _root_.hammingNorm_single {n : ℕ} (i : Fin n) :
    hammingNorm (e i : Fin n → Bool) = 1 := by
  simp [hammingNorm, Pi.single_apply, Finset.filter_eq']

lemma _root_.hammingNorm_eq_one_iff {n : ℕ} {x : Fin n → Bool} :
    hammingNorm x = 1 ↔ ∃ i, x = e i := by
  constructor
  · -- the set of true coordinates of x is a singleton {i}
    intro hx
    rw [hammingNorm, Finset.card_eq_one] at hx
    obtain ⟨i, hi⟩ := hx
    refine ⟨i, funext fun j => ?_⟩
    -- so x j = true exactly when j = i
    have hj : x j = true ↔ j = i := by
      simpa [Bool.zero_eq_false] using Finset.ext_iff.mp hi j
    by_cases hji : j = i <;> simpa [hji, Bool.zero_eq_false] using hj
  · -- conversely, a basis vector has weight one
    rintro ⟨i, rfl⟩
    exact hammingNorm_single i
```

Adjacency in the Hamming cube has a simple algebraic description: x and y are adjacent iff they differ by a basis vector.

```
lemma hypercubeHamming_adj_iff_exists {n : ℕ} {x y : Fin n → Bool} :
    (hypercubeHamming n).Adj x y ↔ ∃ i, y = x + e i := by
  rw [hypercubeHamming_adj, hammingDist_eq_hammingNorm,
    BooleanRing.neg_eq, hammingNorm_eq_one_iff]
  constructor
  · rintro ⟨i, hi⟩
    refine ⟨i, ?_⟩
    rw [← hi, boolVec_add_add_cancel]
  · rintro ⟨i, rfl⟩
    exact ⟨i, boolVec_add_add_cancel _ _⟩
```

Translations and Coordinate Permutations

Translation by a fixed vector t is an automorphism of the Hamming cube: it preserves the Hamming distance since $(t + x) - (t + y) = x - y$. Under the isomorphism `hypercubeIsoHamming`, this corresponds to the automorphism `hypercube.translate` of $\mathbb{Q}[n]$. There we had to define `xor` on `HypercubeVert n` and prove its basic properties (such as `HypercubeVert.xor_xor`) by hand. On `Fin n → Bool`, addition is pointwise `xor`, and the group structure (associativity, `0`, negation) comes for free from Mathlib, as do the characteristic-two facts once we have the `BooleanRing` instance above.

Translation by t (pointwise `xor`) as an automorphism of the Hamming cube.

```
def hypercubeHamming.translate {n : ℕ} (t : Fin n → Bool) :
    hypercubeHamming n ≃g hypercubeHamming n where
  toEquiv := Equiv.addLeft t
  map_rel_iff' := by
    intro x y
    grind [Equiv.coe_addLeft, hypercubeHamming_adj, hammingDist_eq_hammingNorm]

@[simp]
lemma hypercubeHamming.translate_apply {n : ℕ} (t x : Fin n → Bool) :
    hypercubeHamming.translate t x = t + x := rfl
```

Permuting the coordinates by σ : `Equiv.Perm (Fin n)` also preserves the Hamming distance, and hence is an automorphism of the Hamming cube.

```
lemma _root_.hammingDist_comp_perm {ι : Type*} [Fintype ι] {β : Type*}
    [DecidableEq β] (σ : Equiv.Perm ι) (x y : ι → β) :
    hammingDist (x ∘ σ) (y ∘ σ) = hammingDist x y := Finset.card_equiv σ (by simp)
```

Permutation of coordinates by σ as an automorphism of the Hamming cube; x is sent to $x \circ \sigma$. `symm` so that the `i`-th coordinate of the image is the σ^{-1} `i`-th coordinate of x.

```
def hypercubeHamming.permute {n : ℕ} (σ : Equiv.Perm (Fin n)) :
    hypercubeHamming n ≃g hypercubeHamming n where
  toEquiv := σ.arrowCongr (Equiv.refl Bool)
  map_rel_iff' {x y} := by
    rw [hypercubeHamming_adj, hypercubeHamming_adj,
      ← hammingDist_comp_perm σ.symm x y]
    rfl

@[simp]
lemma hypercubeHamming.permute_apply {n : ℕ} (σ : Equiv.Perm (Fin n))
    (x : Fin n → Bool) : hypercubeHamming.permute σ x = x ∘ σ.symm := rfl
```

In the automorphism group the inverse f^{-1} is by definition `f.symm`, so the formulae for the inverses of translations and coordinate permutations hold by `rfl`. Since $-t = t$, the inverse of translation by t is translation by t itself, so translations are involutions (characteristic two again).

```
@[simp]
lemma hypercubeHamming.translate_inv_apply {n : ℕ} (t x : Fin n → Bool) :
    (hypercubeHamming.translate t)⁻¹ x = t + x := rfl
```

```
@[simp]
lemma hypercubeHamming.permute_inv_apply {n : ℕ} (σ : Equiv.Perm (Fin n))
    (x : Fin n → Bool) : (hypercubeHamming.permute σ)⁻¹ x = x ∘ σ := rfl
```

The Hyperoctahedral Group

We can now describe the translations and coordinate permutations as a single group: the semidirect product of $(\mathbb{Z}/2)^n$ (as `Fin n → Bool` under pointwise `xor`) by the symmetric group S_n, where a permutation acts on a translation vector by permuting its coordinates. This is exactly the wreath product $\mathbb{Z}/2 \wr S_n$, known as the *hyperoctahedral group*.

Mathlib's `SemidirectProduct N G φ` (notation `N ⋊[φ] G`) is stated for multiplicative groups, so we need to wrap our additive group `Fin n → Bool` in `Multiplicative`. [4] The action φ sends σ to the group automorphism `t ↦ t ∘ σ.symm` of `Fin n → Bool`, which Mathlib gives as `AddEquiv.arrowCongr σ (AddEquiv.refl Bool)`.

The action of `Equiv.Perm (Fin n)` on translation vectors by permuting coordinates, as a `MonoidHom` into the automorphisms of `Multiplicative (Fin n → Bool)`.

```
def hyperoctahedralAction (n : ℕ) :
    Equiv.Perm (Fin n) →* MulAut (Multiplicative (Fin n → Bool)) where
  toFun σ := AddEquiv.toMultiplicative (AddEquiv.arrowCongr σ (AddEquiv.refl Bool))
  map_one' := by ext t; rfl
  map_mul' σ ρ := by ext t; rfl
```

The hyperoctahedral group $(\mathbb{Z}/2)^n \rtimes S_n$, i.e. the wreath product $\mathbb{Z}/2 \wr S_n$, realized as a semidirect product with $(\mathbb{Z}/2)^n$ as `Fin n → Bool` under pointwise `xor`.

```
abbrev _root_.HyperoctahedralGroup (n : ℕ) : Type :=
  Multiplicative (Fin n → Bool) ⋊[hyperoctahedralAction n] Equiv.Perm (Fin n)
```

Translations and coordinate permutations give monoid homomorphisms into the automorphism group of the Hamming cube. (Recall that multiplication of automorphisms is composition, `(f * g) x = f (g x)`.)

Translations as a monoid homomorphism into the automorphism group of the Hamming cube.

```
@[simps]
def hypercubeHamming.translateHom (n : ℕ) :
    Multiplicative (Fin n → Bool) →* (hypercubeHamming n ≃g hypercubeHamming n) where
  toFun t := hypercubeHamming.translate t.toAdd
  map_one' := by ext x; simp
  map_mul' t s := by ext x; simp [add_assoc]
```

Coordinate permutations as a monoid homomorphism into the automorphism group of the Hamming cube.

```
@[simps]
def hypercubeHamming.permuteHom (n : ℕ) :
    Equiv.Perm (Fin n) →* (hypercubeHamming n ≃g hypercubeHamming n) where
  toFun := hypercubeHamming.permute
```

[4] This is a type synonym: `Multiplicative α` has the same elements as α, but its group operation $*$ is the $+$ of α. We pass between the two with the identity maps `Multiplicative.ofAdd` and `Multiplicative.toAdd`, and turn an additive automorphism into a multiplicative one with `AddEquiv.toMultiplicative`.

```
  map_one' := by ext x; rfl
  map_mul' σ ρ := by ext x; rfl
```

These two homomorphisms are compatible with the action `hyperoctahedralAction`: conjugating a translation by a coordinate permutation is the translation by the permuted vector. `SemidirectProduct.lift` combines them into a monoid homomorphism from the hyperoctahedral group to the automorphism group.

The homomorphism from the hyperoctahedral group to the automorphisms of the Hamming cube, sending `(t, σ)` to the automorphism `x ↦ t + x ∘ σ.symm`, i.e. coordinate permutation by σ followed by translation by `t`.

```
def hyperoctahedralToAut (n : ℕ) :
    HyperoctahedralGroup n →* (hypercubeHamming n ≃g hypercubeHamming n) :=
  SemidirectProduct.lift (hypercubeHamming.translateHom n)
    (hypercubeHamming.permuteHom n) (by
    intro σ
    ext t x i
    -- Look at the goal: it is buried under Additive.toMul,
    -- MonoidHom.toAdditiveRight and MulAut.conj
    -- change gives us the simple equation it unfolds to
    change t (σ.symm i) + x i = t (σ.symm i) + x (σ (σ.symm i))
    rw [Equiv.apply_symm_apply])
```

Injectivity

The homomorphism into the automorphism group is injective: if permuting coordinates by σ and then translating by `t` fixes every vertex, then evaluating at `0` shows `t = 0`, and evaluating at the basis vectors shows `σ = 1`. For the latter we use Mathlib's `Pi.single_left_injective`: the map `i ↦ e i` is injective, since `true` is nonzero in `Bool`.

```
#check Pi.single_left_injective
```

```
lemma hyperoctahedralToAut_injective (n : ℕ) :
    Function.Injective (hyperoctahedralToAut n) := by
  rw [injective_iff_map_eq_one]
  rintro ⟨t, σ⟩ h
  have happ (x : Fin n → Bool) : t.toAdd + x ∘ σ.symm = x := DFunLike.congr_fun h x
  obtain rfl : t = 1 := by simpa using happ 0
  obtain rfl : σ = 1 := Equiv.ext fun j => Eq.symm ◁ by
    simpa [Pi.single_apply, Equiv.symm_apply_eq] using
      congrFun (happ (e j)) j
  rfl
```

Rigidity

For surjectivity we need a rigidity lemma: an automorphism g fixing `0` and all the basis vectors is the identity. The point is that the coordinates of a vertex can be read off from distances, which automorphisms preserve. Since the graph distance in the Hamming cube is the Hamming distance (`edist_hypercubeHamming`), every automorphism preserves the Hamming distance. An automor-

phism fixing `0` therefore also preserves the Hamming weight (`hammingNorm`), the number of `true` coordinates, since this is the Hamming distance to `0`.

```
lemma hammingDist_aut_hypercubeHamming {n : ℕ}
    (g : hypercubeHamming n ≃g hypercubeHamming n) (x y : Fin n → Bool) :
    hammingDist (g x) (g y) = hammingDist x y := by
  have h := g.edist_eq x y
  rw [edist_hypercubeHamming, edist_hypercubeHamming] at h
  exact_mod_cast h


lemma hammingNorm_aut_hypercubeHamming {n : ℕ}
    {g : hypercubeHamming n ≃g hypercubeHamming n} (hg : g 0 = 0)
    (x : Fin n → Bool) : hammingNorm (g x) = hammingNorm x := by
  rw [← hammingDist_zero_right, ← hg, hammingDist_aut_hypercubeHamming,
    hammingDist_zero_right]
```

Adding a basis vector e_i flips the i-th coordinate; if that coordinate was `true` this decreases the weight by one.

```
lemma _root_.hammingNorm_add_single {n : ℕ} {x : Fin n → Bool} {i : Fin n}
    (h : x i = true) : hammingNorm (x + e i) + 1 = hammingNorm x := by
  have hset : ({k | (x + e i : Fin n → Bool) k ≠ 0} : Finset _) =
      ({k | x k ≠ 0} : Finset _).erase i := by
    ext j
    by_cases hj : j = i
    · simp [hj, h, Bool.zero_eq_false, Bool.add_eq_xor]
    · simp [hj]
  rw [hammingNorm, hset, Finset.card_erase_add_one (by simp [h, Bool.zero_eq_false])]
  rfl
```

The vertex $x + e_k$ differs from x only in coordinate k, so its weight is one less than that of x if $x\ k = \mathtt{true}$, and one more otherwise. Since the distance from x to e_k is the weight of $x + e_k$, we see that $x\ k = \mathtt{true}$ exactly when x is closer to e_k than to 0.

```
lemma _root_.apply_eq_true_iff_hammingDist_lt {n : ℕ} (x : Fin n → Bool)
    (k : Fin n) : x k = true ↔ hammingDist x (e k) < hammingDist x 0 := by
  rw [hammingDist_zero_right, hammingDist_eq_hammingNorm, BooleanRing.neg_eq]
  cases hk : x k
  · have h : (x + e k : Fin n → Bool) k = true := by
      simp [hk, Bool.add_eq_xor]
    have := hammingNorm_add_single h
    grind [add_assoc, BooleanRing.add_self, add_zero]
  · have := hammingNorm_add_single hk
    grind
```

An automorphism g fixing `0` and every e_k preserves the distances from any x to `0` and to each e_k, so by the previous lemma $g\ x$ and x agree in every coordinate k, i.e. $g\ x = x$.

```
lemma aut_hypercubeHamming_eq_id_of_fix_zero_and_singles {n : ℕ}
    {g : hypercubeHamming n ≃g hypercubeHamming n} (h0 : g 0 = 0)
    (h1 : ∀ i, g (e i) = e i) (x : Fin n → Bool) :
    g x = x := by
  funext k
  -- g preserves Hamming distance and fixes 0 and e k
  have h₀ : hammingDist (g x) 0 = hammingDist x 0 := by
```

```
    simpa [h0] using hammingDist_aut_hypercubeHamming g x 0
  have hₖ : hammingDist (g x) (e k) = hammingDist x (e k) := by
    simpa [h1 k] using hammingDist_aut_hypercubeHamming g x (e k)
  rw [Bool.eq_iff_iff, apply_eq_true_iff_hammingDist_lt,
    apply_eq_true_iff_hammingDist_lt, h₀, hₖ]
```

Surjectivity

Given an arbitrary automorphism `f` we first compose with a translation to obtain an automorphism `g` fixing `0`. Since `g` preserves weights it permutes the basis vectors, which yields a permutation `σ` of the coordinates; composing with `σ⁻¹` gives an automorphism fixing `0` and all basis vectors, which by rigidity is the identity. Hence `f` is the image of `(f 0, σ)`: permutation of the coordinates by `σ` followed by translation by `f 0`.

```
lemma hyperoctahedralToAut_surjective (n : ℕ) :
    Function.Surjective (hyperoctahedralToAut n) := by
  intro f
  let g := (hypercubeHamming.translateHom n (Multiplicative.ofAdd (f 0)))⁻¹ * f
  have hg0 : g 0 = 0 := by
    simp [g]
  have hsingle : ∀ i : Fin n, ∃ j, g (e i) = e j := by
    intro i
    exact hammingNorm_eq_one_iff.mp
      (by rw [hammingNorm_aut_hypercubeHamming hg0, hammingNorm_single])
  choose p hp using hsingle
  have hpinj : Function.Injective p := by
    intro i j hij
    have h' : g (e i) = g (e j) := by
      rw [hp i, hp j, hij]
    exact Pi.single_left_injective (by decide) (g.injective h')
  let σ : Equiv.Perm (Fin n) :=
    Equiv.ofBijective p (Finite.injective_iff_bijective.mp hpinj)
  have hh0 : ((hypercubeHamming.permuteHom n σ)⁻¹ * g) 0 = 0 := by
    simp [hg0]
  have hhsingle : ∀ i,
      ((hypercubeHamming.permuteHom n σ)⁻¹ * g) (e i) = e i := by
    intro i
    ext k; simp [hp i, σ, Pi.single_apply, hpinj.eq_iff]
  have hone : (hypercubeHamming.permuteHom n σ)⁻¹ * g = 1 := by
    ext x : 1
    simpa using aut_hypercubeHamming_eq_id_of_fix_zero_and_singles hh0 hhsingle x
  -- so g is the permutation by σ
  have hg : g = hypercubeHamming.permuteHom n σ := (inv_mul_eq_one.mp hone).symm
  -- and g is f followed by undoing the translation by f 0, so f is the
  -- permutation by σ followed by that translation
  have hf : f = hypercubeHamming.translateHom n (Multiplicative.ofAdd (f 0)) *
      hypercubeHamming.permuteHom n σ := by
    rw [← hg, mul_inv_cancel_left]
  exact ⟨⟨Multiplicative.ofAdd (f 0), σ⟩, hf.symm⟩
```

Characterization

Converting the bijective homomorphism into a group isomorphism identifies the automorphism group of the Hamming cube.

The automorphism group of the Hamming cube is the hyperoctahedral group $\mathbb{Z}/2 \wr S_n$.

```
noncomputable def autHypercubeHammingMulEquivHyperoctahedral (n : ℕ) :
    (hypercubeHamming n ≃g hypercubeHamming n) ≃* HyperoctahedralGroup n :=
  (MulEquiv.ofBijective (hyperoctahedralToAut n)
    ⟨hyperoctahedralToAut_injective n, hyperoctahedralToAut_surjective n⟩).symm
```

Transporting this along `hypercubeIsoHamming` (using `Iso.autCongr`) gives the promised characterization for our hypercube `Q[n]`: its automorphism group is the hyperoctahedral group $\mathbb{Z}/2 \wr S_n$.

The automorphism group of the hypercube `Q[n]` is the hyperoctahedral group $\mathbb{Z}/2 \wr S_n$.

```
noncomputable def autHypercubeMulEquivHyperoctahedral (n : ℕ) :
    (Q[n] ≃g Q[n]) ≃* HyperoctahedralGroup n :=
  (hypercubeIsoHamming n).autCongr.trans
    (autHypercubeHammingMulEquivHyperoctahedral n)
```

In particular the hypercube `Q[n]` has exactly `2 ^ n * n!` automorphisms.

```
lemma card_aut_hypercube (n : ℕ) :
    Nat.card (Q[n] ≃g Q[n]) = 2 ^ n * n.factorial := by
  rw [Nat.card_congr (autHypercubeMulEquivHyperoctahedral n).toEquiv,
    Nat.card_congr SemidirectProduct.equivProd, Nat.card_prod,
    Nat.card_congr Multiplicative.toAdd]
  simp [Nat.card_eq_fintype_card, Fintype.card_perm]
```

12.7 Counting Copies*

In this section we count copies of a graph `A` in a graph `B`, in two senses: the *labelled* copies are the elements of `Copy A B` from Copies (p. 104), and the *unlabelled* copies are the subgraphs of `B` isomorphic to `A`. We first check that isomorphic graphs have the same numbers of copies, and then show that the unlabelled copies of a complete graph `K[s]` are just the `s`-cliques. Automorphisms then explain the difference between labelled and unlabelled copies: we derive the basic fact that (for finite graphs) the number of labelled copies of `A` in `B` is equal to the number of unlabelled copies of `A` in `B`, multiplied by the number of automorphisms of `A`. (Mathlib has the inequality `copyCount_le_labelledCopyCount`, but not this identity.) We then apply it to cycles.

The following maps (Which Notion of Subgraph? (p. 255)) pass between the two kinds of copy.

```
section copies
open Subgraph Finset
variable {V W : Type*} {G : SimpleGraph V}

#check Copy.toSubgraph     -- (f : Copy A B) : B.Subgraph
#check Copy.isoToSubgraph  -- (f : Copy A B) : A ≃g f.toSubgraph.coe
#check Subgraph.coeCopy    -- (C : B.Subgraph) : Copy C.coe B
```

Copy Counts and Isomorphisms

Mathlib's `copyCount` counts the unlabelled copies of `H` in `G` as the subgraphs of `G` that are isomorphic to `H`, and `labelledCopyCount` counts the labelled copies. [5]

```
#source copyCount         -- #{G' : G.Subgraph | Nonempty (H ≃g G'.coe)}
#source labelledCopyCount -- Fintype.card (Copy H G)
```

Isomorphisms `A ≃g A'` and `B ≃g B'` identify the copies of `A` in `B` with the copies of `A'` in `B'`, by composing on both sides. So isomorphic graphs have the same copy counts. This lets us count copies in whichever model of a graph is most convenient.

Isomorphisms `A ≃g A'` and `B ≃g B'` identify the copies of `A` in `B` with the copies of `A'` in `B'`.

```
def Iso.copyCongr {V' W' : Type*} {A : SimpleGraph V} {A' : SimpleGraph V'}
    {B : SimpleGraph W} {B' : SimpleGraph W'} (eA : A ≃g A') (eB : B ≃g B') :
    Copy A B ≃ Copy A' B' where
  toFun f := (eB.toCopy.comp f).comp eA.symm.toCopy
  invFun f := (eB.symm.toCopy.comp f).comp eA.toCopy
  left_inv f := by ext; simp
  right_inv f := by ext; simp


lemma Iso.labelledCopyCount_eq {V' W' : Type*} [Fintype V] [Fintype V'] [Fintype W]
    [Fintype W'] {A : SimpleGraph V} {A' : SimpleGraph V'} {B : SimpleGraph W}
    {B' : SimpleGraph W'} (eA : A ≃g A') (eB : B ≃g B') :
    B.labelledCopyCount A = B'.labelledCopyCount A' := by
  simp only [labelledCopyCount, Fintype.card_eq_nat_card]
  exact Nat.card_congr (eA.copyCongr eB)
```

For unlabelled copies only the host graphs need to be finite, so the next lemma assumes nothing about `A` and `A'`. Mathlib's `labelledCopyCount` also asks for the copied graph to be finite, because it is defined as `Fintype.card (Copy A B)`, and the `Fintype` instance on `Copy A B` is built from the one on homomorphisms `A →g B`. Mathematically we lose nothing: if `B` is finite and `A` is infinite then there are no copies at all.

The proof sends each subgraph of `B` to its image in `B'`, and `Copy.isoSubgraphMap` carries a labelling of a subgraph over to its image.

```
lemma Iso.copyCount_eq {V' W' : Type*} [Fintype W] [Fintype W'] {A : SimpleGraph V}
    {A' : SimpleGraph V'} {B : SimpleGraph W} {B' : SimpleGraph W'} (eA : A ≃g A')
    (eB : B ≃g B') : B.copyCount A = B'.copyCount A' := by
  classical
  unfold copyCount
  refine Finset.card_nbij' (Subgraph.map eB.toHom) (Subgraph.map eB.symm.toHom)
    (fun C hC => ?_) (fun C hC => ?_) (fun C _ => ?_) (fun C _ => ?_)
  · obtain ⟨f⟩ := (Finset.mem_filter.1 hC).2
    exact Finset.mem_filter.2
      ⟨Finset.mem_univ _, ⟨((eA.symm.trans f).trans (eB.toCopy.isoSubgraphMap C))⟩⟩
  · obtain ⟨f⟩ := (Finset.mem_filter.1 hC).2
    exact Finset.mem_filter.2
      ⟨Finset.mem_univ _, ⟨((eA.trans f).trans (eB.symm.toCopy.isoSubgraphMap C))⟩⟩
  -- mapping along eB and then back along eB.symm gives the original subgraph
```

[5] Note the argument order: `Copy A B` lists the copied graph first, like `A →g B`, while `B.labelledCopyCount A` and `B.copyCount A` are invariants of the host graph `B`, which comes first.

```
  · rw [← Subgraph.map_comp]; ext <;> simp
  · rw [← Subgraph.map_comp]; ext <;> simp
```

Copies of Complete Graphs

`copyCount` is noncomputable, so we cannot simply evaluate it. But the unlabelled copies of `K[s]` are easy to count: they are the `s`-cliques. Any two vertices in the image of a copy of a complete graph are adjacent, so the copy is an embedding (`Copy.topEmbedding`). An embedding maps induced subgraphs to induced subgraphs (`Subgraph.IsInduced.map`), so the subgraph picked out by the copy is the subgraph induced on its image (`Subgraph.IsInduced.induce_top_verts`).

```
lemma Copy.toSubgraph_top (f : Copy (⊤ : SimpleGraph W) G) :
    f.toSubgraph = (⊤ : G.Subgraph).induce (Set.range f) := by
  -- the image of the induced subgraph ⊤ under f.topEmbedding is induced
  have h := (Subgraph.IsInduced.top.map f.topEmbedding).induce_top_verts
  exact h.symm.trans (by simp)
```

So a copy of the complete graph on `W` is determined by its vertex set, which is a clique with `Fintype.card W` vertices (`isNClique_map_copy_top`). Conversely, matching the vertices of such a clique `t` with `W` (`Fintype.equivOfCardEq`) gives a copy with vertex set `t`. To compare with `copyCount`, we use `copyCount_eq_card_image_copyToSubgraph`, which counts the subgraphs `f.toSubgraph` instead.

```
#check copyCount_eq_card_image_copyToSubgraph
```

In a finite graph counting the number of copies of a finite complete graph on `W` is the same as counting the cliques of `G` with `Fintype.card W` vertices.

```
theorem copyCount_top [Fintype W] [Fintype V] [DecidableEq V] [DecidableRel G.Adj] :
    G.copyCount (⊤ : SimpleGraph W) = #(G.cliqueFinset (Fintype.card W)) := by
  classical
  rw [copyCount_eq_card_image_copyToSubgraph]
  -- send a copy to its vertex set, and a clique to the subgraph it induces
  refine card_nbij' (fun G' => G'.verts.toFinset)
    (fun t => (⊤ : G.Subgraph).induce t) (fun G' hG' => ?_) (fun t ht => ?_)
    (fun G' hG' => ?_) (fun t _ => by simp)
  · -- the vertex set of a copy of ⊤ is a clique of the right size
    obtain ⟨f, -, rfl⟩ := mem_image.1 hG'
    simpa [Set.toFinset_range, map_eq_image, Copy.toEmbedding] using
      isNClique_map_copy_top f
  · -- the subgraph induced on a clique t is a copy of ⊤
    replace ht := mem_cliqueFinset_iff.1 (mem_coe.1 ht)
    let φ : W ≃ t := Fintype.equivOfCardEq (by rw [Fintype.card_coe, ht.2])
    -- distinct vertices of the clique t are adjacent
    have hadj : ∀ i j, i ≠ j → G.Adj (φ i) (φ j) := by
      intro i j hij
      exact ht.1 (φ i).2 (φ j).2 (by simpa using hij)
    let f : Copy ⊤ G := ⟨⟨fun i => φ i, fun hij => hadj _ _ hij.ne⟩,
      Subtype.val_injective.comp φ.injective⟩
    -- the vertex set of f is t, since φ is a bijection onto t
    have hf : Set.range f = t := by
      ext x
      simp [f, φ.exists_congr_left]
    exact mem_coe.2 (mem_image.2 ⟨f, mem_univ _, by rw [Copy.toSubgraph_top, hf]⟩)
```

```
  · -- a copy of τ is the subgraph induced on its vertex set
    obtain ⟨f, -, rfl⟩ := mem_image.1 hG'
    rw [Copy.toSubgraph_top]
    simp
```

In particular `G.copyCount K[s]` is the number of s-cliques of `G`. Since `cliqueFinsets` are computable, we can now compute `copyCount` for complete graphs. For example, `K[3][3]` the 3-uniform blow-up of `K[3]` contains 27 triangles.

```
example : K[3][3].copyCount K[3] = 27 := by
  rw [copyCount_top]
  decide +kernel
```

Labelled and Unlabelled Copies

The identity relating labelled and unlabelled copies follows from a general equivalence: `A.Copy B` is equivalent to the product of the subtype of subgraphs of `B` that are isomorphic to `A` with the automorphisms of `A`.

This is obvious informally: labelled copies of `A` in `B` are the same as unlabelled copies of `A` in `B` together with a choice of labelling, and the labellings of a fixed unlabelled copy of `A` are equivalent to automorphisms of `A`.

We build this equivalence in two steps. First, `copyEquivSigmaIso` shows that a copy of `A` in `B` is the same as a subgraph `C` together with a labelling `A ≃g C.coe`, packaged as a `Sigma` type. No choice is needed for this step, since the copy itself provides the labelling.

Rather than prove `right_inv` directly, we show that `invFun` is injective and deduce `right_inv` using `Function.LeftInverse.rightInverse_of_injective`. This avoids comparing labellings of two subgraphs that are equal but not definitionally equal (Definitional and Propositional Equalities (p. 16)), and so have different types. [6]

```
#check Function.LeftInverse.rightInverse_of_injective
```

`A.Copy B` is equivalent to the Sigma type of subgraphs of `B` that are isomorphic to `A` with their isomorphisms.

```
@[simps]
noncomputable def copyEquivSigmaIso (A : SimpleGraph V) (B : SimpleGraph W) :
    A.Copy B ≃
      Σ C : {C : B.Subgraph | Nonempty (A ≃g C.coe)}, A ≃g (C : B.Subgraph).coe where
  toFun c := ⟨⟨c.toSubgraph, ⟨c.isoToSubgraph⟩⟩, c.isoToSubgraph⟩
  invFun y := (y.1 : B.Subgraph).coeCopy.comp y.2.toCopy
  -- left_inv is filled in automatically (by ext; rfl)
  -- although we still have to prove it again below inside right_inv
  right_inv := by
    refine Function.LeftInverse.rightInverse_of_injective (fun c => by ext; rfl) ?_
    rintro ⟨⟨C, hC⟩, f⟩ ⟨⟨D, hD⟩, g⟩ h
    -- The images agree, and C and D are local variables, so we
    -- can substitute C = D.
    obtain rfl : C = D := by
```

[6] This is essentially the difficulty we met for walks, which `Walk.copy` handles: the type `G.Walk u v` records the endpoints of a walk, just as the type of a labelling records the subgraph it labels.

```
      simpa [Copy.toSubgraph, Copy.comp, coeCopy, map_comp, map_iso_top,
        map_hom_top] using congrArg Copy.toSubgraph h
    -- Now f and g are labellings of the same subgraph, so ext applies.
    obtain rfl : f = g := by ext a; exact DFunLike.congr_fun h a
    rfl
```

Next we identify the labellings of a fixed subgraph `C` with the automorphisms of `A`. Given a *reference* labelling `e : A ≃g C.coe`, a labelling `f` corresponds to the automorphism `f.trans e.symm`, which records how `f` differs from `e`. Composing with fixed isomorphisms like this is `RelIso.relIsoCongr` in Mathlib, since a graph isomorphism is a `RelIso` between adjacency relations.

For `C` in the subtype `{C : B.Subgraph | Nonempty (A ≃g C.coe)}`, however, the membership proof `Nonempty (A ≃g C.coe)` is a proposition: it says that a labelling of `C` exists, but does not provide any particular one. So we use choice to pick a reference labelling `C.property.some` (`Nonempty.some`) for each `C`. (Since the membership proof is a proposition, an element of the subtype is determined by its subgraph alone, so this choice depends only on `C`.)

Once every fibre of the `Sigma` type is identified with `A ≃g A`, the `Sigma` type collapses to a product (`Equiv.sigmaEquivProdOfEquiv`).

```
#check RelIso.relIsoCongr
#check Equiv.sigmaEquivProdOfEquiv
```

`A.Copy B` is equivalent to the product of the subtype of subgraphs of `B` that are isomorphic to `A` with the automorphisms of `A`.

```
noncomputable
def copyEquivSubgraphIsoProdAut (A : SimpleGraph V) (B : SimpleGraph W) :
    A.Copy B ≃ {C : B.Subgraph | Nonempty (A ≃g C.coe)} × A ≃g A :=
  (copyEquivSigmaIso A B).trans ◁ Equiv.sigmaEquivProdOfEquiv fun C =>
    RelIso.relIsoCongr (.refl _) C.property.some.symm
```

This is an instance of a general fact about group actions. The automorphisms of `A` act freely (only the identity fixes a copy) on `A.Copy B` by precomposition, and the orbits are the unlabelled copies. For any free action Mathlib's `MulAction.selfEquivOrbitsQuotientProd` gives the same decomposition, using `Quotient.out` to choose a representative of each orbit just as we chose a reference labelling of each subgraph.

We can now deduce the promised counting identity. The number of labelled copies of `A` in `B` (`B.labelledCopyCount A`) equals the number of unlabelled copies (`B.copyCount A`) multiplied by the number of automorphisms of `A`.

Rather than state this in terms of `Fintype.card (A ≃g A)`, which would require a `Fintype` instance on `A ≃g A`, we use the fact that for finite graphs `A ≃g A` is equivalent to `Copy A A`, for which Mathlib already provides a `Fintype` instance. So we already have a name for the number of automorphisms of `A`: it is `A.labelledCopyCount A`. In fact given any isomorphism `A ≃g B`, with `A` a finite graph, we can identify `Copy A B` with `A ≃g B`.

For finite graphs, an isomorphism `A ≃g B` witnesses that copies of `A` in `B` are the same as isomorphisms `A ≃g B`.

```
noncomputable def Iso.copyEquivIso {A : SimpleGraph V} {B : SimpleGraph W}
    (g : A ≃g B) [Finite V] : Copy A B ≃ (A ≃g B) where
  toFun f :=
    { Equiv.ofBijective f.toHom ⟨f.injective,
```

```
      (Finite.injective_iff_surjective_of_equiv g.toEquiv).1 f.injective⟩ with
    map_rel_iff' {a b} := by
      refine ⟨fun h => ?_, f.toHom.map_rel⟩
      -- f.mapEdgeSet is an injection between finite sets of the same size,
      -- hence a surjection, so the edge s(f a, f b) of B is the image of an
      -- edge of A.
      obtain ⟨⟨e', he'⟩, heq⟩ :=
        (Finite.injective_iff_surjective_of_equiv g.mapEdgeSet).1
          f.mapEdgeSet.injective ⟨s(f a, f b), h⟩
      obtain rfl : e' = s(a, b) :=
        Sym2.map.injective f.injective (congrArg Subtype.val heq)
      exact he' }
  invFun φ := φ.toCopy
```

Copies of a finite graph in itself are equivalent to automorphisms of the graph.

```
noncomputable def copyEquivAut (A : SimpleGraph V) [Finite V] :
    Copy A A ≃ (A ≃g A) := Iso.copyEquivIso .refl
```

For `A` a finite graph, `A.Copy B` is equivalent to the product of the subtype of subgraphs of `B` that are isomorphic to `A` with copies of `A` in itself.

```
noncomputable
def copyEquivSubgraphIsoProdCopy (A : SimpleGraph V) (B : SimpleGraph W) [Finite V] :
    A.Copy B ≃ {C : B.Subgraph | Nonempty (A ≃g C.coe)} × A.Copy A :=
  (copyEquivSubgraphIsoProdAut A B).trans
    (.prodCongr (Equiv.refl _) (copyEquivAut A).symm)
```

Hence, for `A` and `B` finite graphs, the number of labelled copies of `A` in `B` is equal to the number of unlabelled copies of `A` in `B` multiplied by the number of labelled copies of `A` in itself (i.e. the number of automorphisms of `A`).

```
theorem labelledCopyCount_eq_copyCount_mul_labelledCopyCount [Fintype V] [Fintype W]
    (A : SimpleGraph V) (B : SimpleGraph W) :
    B.labelledCopyCount A = B.copyCount A * A.labelledCopyCount A := by
  classical
  rw [labelledCopyCount, Fintype.card_congr (A.copyEquivSubgraphIsoProdCopy B),
    Fintype.card_prod, copyCount, Fintype.card_subtype]
  rfl
```

Counting Copies of Cycles

By `labelledCopyCount_eq_copyCount_mul_labelledCopyCount`, the number of labelled copies of a finite graph `A` in a finite graph `B` is the number of unlabelled copies times the number of automorphisms of `A`. Since the automorphism group of `C[n + 3]` is the dihedral group of order `2 * (n + 3)` (`autCycleGraphMulEquivDihedral`), this expresses the number of labelled copies of a cycle in a graph in terms of the number of unlabelled copies.

```
theorem labelledCopyCount_cycleGraph [Fintype V] (G : SimpleGraph V) (n : ℕ) :
    G.labelledCopyCount C[n + 3] = G.copyCount C[n + 3] * (2 * (n + 3)) := by
  rw [labelledCopyCount_eq_copyCount_mul_labelledCopyCount, labelledCopyCount,
    ← DihedralGroup.card, ← Fintype.card_congr ((copyEquivAut C[n + 3]).trans
    (autCycleGraphMulEquivDihedral n).toEquiv)]
```

```
  -- Both sides now have the form G.copyCount C[n + 3] * Fintype.card (Copy …),
  -- but the Fintype instances differ: the left one comes from
  -- labelledCopyCount, where it is built classically, and the right one from type
  -- class search in Fintype.card_congr. rfl would have to check that they are
  -- definitionally equal, which times out. convert closes the gap using
  -- Subsingleton (any two Fintype instances are equal).
  convert rfl

end copies
```

12.8 Subcubes of the Hypercube*

As a second application of the Hamming cube model we determine all copies of a smaller cube `Q[k]` inside `Q[n]`. The main result is another rigidity theorem: every copy of `Q[k]` in `Q[n]` is *canonical*. That is, for some choice of `k` distinct coordinates of `Q[n]` and a vertex `b` of `Q[n]`, it sends each vertex `x` of `Q[k]` to `b + y`, where `y` is obtained by placing the coordinates of `x` in the chosen positions, with `false` elsewhere. (In particular `b` is the image of `0`.) So its image is a *subcube*, a translate of a coordinate subspace. We prove this by induction on the Hamming weight, using a "parallelogram lemma" about the 4-cycles of the Hamming cube.

A canonical copy is determined by `b` and the chosen coordinates (in order), so counting these gives

- `Q[n].labelledCopyCount Q[k] = 2 ^ n * n.descFactorial k`, and
- `Q[n].copyCount Q[k] = n.choose k * 2 ^ (n - k)` (when `k ≤ n`).

The second is the familiar answer "choose the `k` free coordinates and fix the others"; the first refines it by the `2 ^ k * k!` automorphisms of `Q[k]`. Finally, canonical copies preserve distances, so every copy of `Q[k]` in `Q[n]` is induced and the labelled count is also the number of embeddings `Q[k] ↪g Q[n]`. As for the automorphism group, we work with the Hamming cube and transport the results to `Q[n]` at the end.

The Parallelogram Lemma

If $x\ u\ y\ v$ form a 4-cycle in the Hamming cube (i.e. $u \neq v$ are common neighbours of $x \neq y$), then $u + v = x + y$; equivalently the cycle is a two-dimensional "face" $x,\ x + e_i,\ x + e_i + e_j,\ x + e_j$.

```
open hypercubeHamming
```

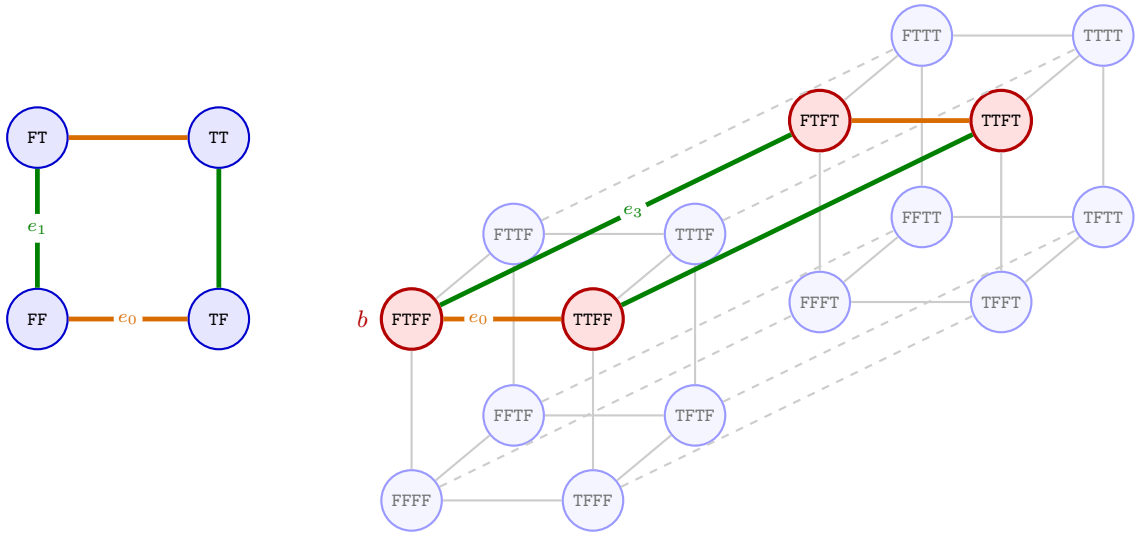
The **parallelogram lemma**: if $u \neq v$ are common neighbours of $x \neq y$ in the Hamming cube, then $u + v = x + y$.

```
lemma hypercubeHamming_parallelogram {n : ℕ} {x y u v : Fin n → Bool}
    (hxu : (hypercubeHamming n).Adj x u) (huy : (hypercubeHamming n).Adj u y)
    (hxv : (hypercubeHamming n).Adj x v) (hvy : (hypercubeHamming n).Adj v y)
    (hxy : x ≠ y) (huv : u ≠ v) : u + v = x + y := by
  obtain ⟨i, rfl⟩ := hypercubeHamming_adj_iff_exists.1 hxu
  obtain ⟨j, rfl⟩ := hypercubeHamming_adj_iff_exists.1 huy
  obtain ⟨k, rfl⟩ := hypercubeHamming_adj_iff_exists.1 hxv
  obtain ⟨l, hq⟩ := hypercubeHamming_adj_iff_exists.1 hvy
```

```
  -- cancel the common x: eᵢ + eⱼ = eₖ + eₗ
  rw [add_assoc, add_assoc] at hq
  have hkey := add_left_cancel hq
  -- Mathlib classifies the solutions of eᵢ + eⱼ = eₖ + eₗ: either i = k and
  -- j = l, or i = l and j = k, or i = j and k = l (as true + true = 0)
  rcases (Pi.single_add_single_eq_single_add_single (by decide) (by decide)).1
    hkey with ⟨rfl, rfl⟩ | ⟨-, rfl, rfl⟩ | ⟨-, rfl, -⟩
  · exact absurd rfl huv
  · abel -- abel proves equations in commutative additive groups
  · exact absurd (by simp [add_assoc]) hxy
```

Canonical Subcube Copies

A canonical copy of `Q[k]` inside `Q[n]` is determined by a base point `b : Fin n → Bool` and an injection `d : Fin k ↪ Fin n` selecting the `k` free directions: the vertex `x` of `Q[k]` is sent to `b + extendEmb d x`, where `extendEmb d x` places the coordinates of `x` in the positions selected by `d` and is `false` elsewhere.



Left: `Q[2]`. Right: the canonical copy `subcubeCopy b d` of `Q[2]` in `Q[4]`, with base point `b = ![false, true, false, false]` (FTFF) and free directions `d 0 = 0` and `d 1 = 3`

```
namespace hypercubeHamming
```

Extend a `k`-vector to an `n`-vector along an embedding `d : Fin k ↪ Fin n`, placing `x i` in position `d i` and `false` elsewhere.

```
noncomputable def extendEmb {n k : ℕ} (d : Fin k ↪ Fin n) (x : Fin k → Bool) :
    Fin n → Bool :=
  Function.extend d x 0

@[simp]
lemma extendEmb_apply {n k : ℕ} (d : Fin k ↪ Fin n) (x : Fin k → Bool) (i : Fin k) :
    extendEmb d x (d i) = x i :=
  d.injective.extend_apply x 0 i

lemma extendEmb_apply_of_not_exists {n k : ℕ} {d : Fin k ↪ Fin n} (x : Fin k → Bool)
    {j : Fin n} (hj : ¬∃ i, d i = j) : extendEmb d x j = 0 :=
```

```
  Function.extend_apply' x (0 : Fin n → Bool) j hj

@[simp]
lemma extendEmb_zero {n k : ℕ} (d : Fin k ↪ Fin n) : extendEmb d 0 = 0 :=
  Function.extend_zero d

lemma extendEmb_add {n k : ℕ} (d : Fin k ↪ Fin n) (x y : Fin k → Bool) :
    extendEmb d (x + y) = extendEmb d x + extendEmb d y := by
  simpa [extendEmb] using Function.extend_add d x y 0 0

lemma extendEmb_single {n k : ℕ} (d : Fin k ↪ Fin n) (i : Fin k) (b : Bool) :
    extendEmb d (Pi.single i b) = Pi.single (d i) b := by
  funext j
  by_cases hj : ∃ l, d l = j
  · obtain ⟨l, rfl⟩ := hj
    simp [Pi.single_apply, d.injective.eq_iff]
  · have hji : j ≠ d i := fun h => hj ⟨i, h.symm⟩
    simp [extendEmb_apply_of_not_exists _ hj, Bool.zero_eq_false, hji]

lemma extendEmb_injective {n k : ℕ} (d : Fin k ↪ Fin n) :
    Function.Injective (extendEmb d) :=
  Function.extend_injective d.injective 0
```

The homomorphism `hypercubeHamming k →g hypercubeHamming n` induced by an embedding of directions.

```
@[simps]
noncomputable def extendHom {n k : ℕ} (d : Fin k ↪ Fin n) :
    hypercubeHamming k →g hypercubeHamming n where
  toFun := extendEmb d
  map_rel' := by
    intro x y hxy
    obtain ⟨i, rfl⟩ := hypercubeHamming_adj_iff_exists.1 hxy
    exact hypercubeHamming_adj_iff_exists.2
      ⟨d i, by rw [extendEmb_add, extendEmb_single]⟩
```

The canonical copy of `Q[k]` in `Q[n]` with base point `b` and free directions selected by `d`: the subcube `{ b + extendEmb d x | x : Fin k → Bool }`.

```
noncomputable def subcubeCopy {n k : ℕ} (b : Fin n → Bool) (d : Fin k ↪ Fin n) :
    Copy (hypercubeHamming k) (hypercubeHamming n) where
  toHom := (hypercubeHamming.translate b).toHom.comp
    (hypercubeHamming.extendHom d)
  injective' := fun _ _ h => extendEmb_injective d (add_left_cancel h)

@[simp]
lemma subcubeCopy_apply {n k : ℕ} (b : Fin n → Bool) (d : Fin k ↪ Fin n)
    (x : Fin k → Bool) : subcubeCopy b d x = b + extendEmb d x := rfl
```

The base point and direction embedding are recoverable from the copy: `subcubeCopy` is injective in `(b, d)`.

```
lemma subcubeCopy_injective {n k : ℕ} :
    Function.Injective fun bd : (Fin n → Bool) × (Fin k ↪ Fin n) =>
      subcubeCopy bd.1 bd.2 := by
  rintro ⟨b, d⟩ ⟨b', d'⟩ h
  have happ : ∀ x, b + extendEmb d x = b' + extendEmb d' x := by
    intro x
    simpa using DFunLike.congr_fun h x
  have hb : b = b' := by simpa using happ 0
  subst hb
  have hd : d = d' := by
    apply DFunLike.ext
    intro i
    have hx := add_left_cancel (happ (e i))
    rw [extendEmb_single, extendEmb_single] at hx
    exact Pi.single_left_injective (by decide) hx
  rw [hd]
```

Rigidity: Every Copy is Canonical

Every copy of `Q[k]` in `Q[n]` is of the form `subcubeCopy b d`. Note that a copy is an *injective homomorphism*: it preserves adjacency in the forward direction only, so, unlike for automorphisms, no distance or weight preservation is available. We use the parallelogram lemma instead (`hypercubeHamming_parallelogram`). The proof has two steps. First we read off the direction embedding `d` from the images of the basis vectors. Then we show that `f x = f 0 + extendEmb d x` by strong induction on the Hamming weight of `x`. Vertices of weight `0` and `1` are `0` and the basis vectors, which are covered by the first step. A vertex y of weight at least `2` has two `true` coordinates `i ≠ j`. So it is the top corner of a 4-cycle whose other three vertices, $y + e_i$, $y + e_j$ and $y + e_i + e_j$, have smaller weight, and the parallelogram lemma determines `f y` from their images.

First we prove a small helper lemma: a vector of Hamming weight at least two has two distinct `true` coordinates.

```
private lemma exists_ne_true_true_of_one_lt_hammingNorm {n : ℕ} {y : Fin n → Bool}
    (hy : 1 < hammingNorm y) : ∃ i j, i ≠ j ∧ y i = true ∧ y j = true := by
  obtain ⟨i, hi, j, hj, hij⟩ := Finset.one_lt_card.mp hy
  use i, j, hij
  simp_all [Bool.zero_eq_false]
```

A copy maps each square y, $y + e_i$, $y + e_i + e_j$, $y + e_j$ of `Q[k]` to a 4-cycle in `Q[n]`, so by the parallelogram lemma the image of any corner is the sum of the images of the other three.

```
private lemma copy_apply_eq_add_add {n k : ℕ}
    (f : Copy (hypercubeHamming k) (hypercubeHamming n)) (y : Fin k → Bool)
    {i j : Fin k} (hij : i ≠ j) :
    f y = f (y + e i + e j) + f (y + e i) + f (y + e j) := by
  have hadj : ∀ (x : Fin k → Bool) (a : Fin k),
      (hypercubeHamming n).Adj (f x) (f (x + e a)) :=
    fun x a => f.toHom.map_adj (hypercubeHamming_adj_iff_exists.2 ⟨a, rfl⟩)
  -- three sides of the square y, y + e i, y + e i + e j, y + e j are mapped to
  -- edges by hadj; for the fourth we reorder the sum
  have h₄ : (hypercubeHamming n).Adj (f (y + e j)) (f (y + e i + e j)) := by
    rw [add_right_comm]
```

```
    exact hadj _ i
  -- opposite corners of the square are distinct, and so are their images
  have hne₁ : f y ≠ f (y + e i + e j) := fun h => by
    simpa [Pi.single_apply, hij] using congrFun (f.injective h) i
  have hne₂ : f (y + e i) ≠ f (y + e j) := fun h =>
    hij (Pi.single_left_injective (by decide) (add_left_cancel (f.injective h)))
  have hpar : f (y + e i) + f (y + e j) = f y + f (y + e i + e j) :=
    hypercubeHamming_parallelogram (hadj y i) (hadj _ j) (hadj y j) h₄ hne₁ hne₂
  -- substitute hpar and cancel the two copies of f (y + e i + e j)
  rw [add_assoc, hpar, add_comm (f y), boolVec_add_add_cancel]
```

For the first step, each basis vector `e i` is a neighbour of `0`, so its image is a neighbour of `f 0` and differs from it in a single coordinate, which we call `d i`. Two basis vectors with the same direction would have the same image, so `d` is injective because `f` is.

```
private lemma exists_apply_single_eq_add_single {n k : ℕ}
    (f : Copy (hypercubeHamming k) (hypercubeHamming n)) :
    ∃ d : Fin k ↪ Fin n, ∀ i, f (e i) = f 0 + e (d i) := by
  -- each basis vector is mapped to a neighbour of f 0, defining the
  -- direction map d
  have hD : ∀ i : Fin k, ∃ p : Fin n,
      f (e i) = f 0 + e p := by
    intro i
    have hadj : (hypercubeHamming n).Adj (f 0) (f (e i)) :=
      f.toHom.map_adj (hypercubeHamming_adj_iff_exists.2 ⟨i, (zero_add _).symm⟩)
    exact hypercubeHamming_adj_iff_exists.1 hadj
  choose d hd using hD
  have hdinj : Function.Injective d := by
    intro i j hdij
    have h : f (e i) = f (e j) := by
      rw [hd i, hd j, hdij]
    exact Pi.single_left_injective (by decide) (f.injective h)
  exact ⟨⟨d, hdinj⟩, hd⟩
```

For the second step we show that the images of `0` and of the basis vectors determine the whole copy. Here the direction embedding `d` is an input rather than an output, so the statement applies to any `d` satisfying `hd`.

```
private lemma copy_apply_eq_add_extendEmb_of_single {n k : ℕ}
    {f : Copy (hypercubeHamming k) (hypercubeHamming n)} {d : Fin k ↪ Fin n}
    (hd : ∀ i, f (e i) = f 0 + e (d i))
    (x : Fin k → Bool) : f x = f 0 + extendEmb d x := by
  suffices H : ∀ (m : ℕ) (y : Fin k → Bool), hammingNorm y = m →
      f y = f 0 + extendEmb d y from H _ x rfl
  intro m
  induction m using Nat.strong_induction_on with
  | _ m ih =>
    rcases m with _ | m
    · -- weight zero: y = 0
      intro y hy
      rw [hammingNorm_eq_zero] at hy
      simp [hy]
    rcases m with _ | m
    · -- weight one: y is a basis vector, handled by hd
```

```
      intro y hy
      obtain ⟨i, rfl⟩ := hammingNorm_eq_one_iff.mp hy
      rw [hd i, extendEmb_single]
    -- weight m + 2: y has two distinct true coordinates i ≠ j
    intro y hy
    obtain ⟨i, j, hij, hyi, hyj⟩ :=
      exists_ne_true_true_of_one_lt_hammingNorm (by grind : 1 < hammingNorm y)
    -- the three vertices below y in the four-cycle have smaller weight
    have hn₁ : hammingNorm (y + e i) = m + 1 := by
      grind [hammingNorm_add_single hyi]
    have hn₂ : hammingNorm (y + e j) = m + 1 := by
      grind [hammingNorm_add_single hyj]
    have h2j : (y + e i : Fin k → Bool) j = true := by
      simp [hij.symm, hyj]
    have hn₃ : hammingNorm (y + e i + e j) = m := by
      grind [hammingNorm_add_single h2j]
    have hf₁ := ih (m + 1) (by grind) _ hn₁
    have hf₂ := ih (m + 1) (by grind) _ hn₂
    have hf₃ := ih m (by grind) _ hn₃
    -- f y is determined by the three smaller corners of the square
    rw [copy_apply_eq_add_add f y hij, hf₁, hf₂, hf₃]
    simp [extendEmb_add, add_assoc, add_comm, add_left_comm]
```

Combining the two steps:

```
private lemma exists_apply_eq_add_extendEmb {n k : ℕ}
    (f : Copy (hypercubeHamming k) (hypercubeHamming n)) :
    ∃ d : Fin k ↪ Fin n, ∀ x, f x = f 0 + extendEmb d x := by
  obtain ⟨d, hd⟩ := exists_apply_single_eq_add_single f
  exact ⟨d, copy_apply_eq_add_extendEmb_of_single hd⟩
```

Every copy of `Q[k]` in `Q[n]` *is* a canonical subcube copy.

```
lemma exists_eq_subcubeCopy {n k : ℕ}
    (f : Copy (hypercubeHamming k) (hypercubeHamming n)) :
    ∃ (b : Fin n → Bool) (d : Fin k ↪ Fin n), f = subcubeCopy b d := by
  obtain ⟨d, hd⟩ := exists_apply_eq_add_extendEmb f
  exact ⟨f 0, d, Copy.ext fun x => by simpa using hd x⟩

end hypercubeHamming
```

The Labelled Count

Rigidity and `subcubeCopy_injective` combine into a classification of the copies of `Q[k]` in `Q[n]`: they correspond exactly to pairs of a base point and a direction embedding. The labelled copy count follows by counting the pairs.

Sanity checks: for `k = n` the count is `2 ^ n * n!`, the order of the automorphism group by `card_aut_hypercube` (every copy of `Q[n]` in itself is an automorphism); for `k = 0` it is `2 ^ n`, the number of vertices; for `k = 1` it is `2 ^ n * n`, the number of edges counted with direction.

Copies of `Q[k]` in `Q[n]` (in the Hamming model) are equivalent to pairs `(b, d)` of a base point and a direction embedding. The inverse map sends `(b, d)` to `subcubeCopy b d`; it is injective by `subcubeCopy_injective` and surjective by the rigidity theorem `exists_eq_subcubeCopy`.

```
noncomputable def hypercubeHamming.copyEquivProd (n k : ℕ) :
    Copy (hypercubeHamming k) (hypercubeHamming n) ≃
      (Fin n → Bool) × (Fin k ↪ Fin n) :=
  (Equiv.ofBijective (fun bd => subcubeCopy bd.1 bd.2)
    ⟨subcubeCopy_injective, fun f => by
      obtain ⟨b, d, hf⟩ := exists_eq_subcubeCopy f
      exact ⟨(b, d), hf.symm⟩⟩).symm

lemma labelledCopyCount_hypercubeHamming (n k : ℕ) :
    (hypercubeHamming n).labelledCopyCount (hypercubeHamming k) =
      2 ^ n * n.descFactorial k := by
  simp_rw [labelledCopyCount, Fintype.card_eq_nat_card,
    Nat.card_congr (hypercubeHamming.copyEquivProd n k), Nat.card_prod,
    ← Fintype.card_eq_nat_card, Fintype.card_embedding_eq, Fintype.card_fun]
  simp
```

To transport this to `Q[n]` we use `Iso.labelledCopyCount_eq`: isomorphic graphs have the same copy counts.

```
theorem labelledCopyCount_hypercube (n k : ℕ) :
    Q[n].labelledCopyCount Q[k] = 2 ^ n * n.descFactorial k :=
  ((hypercubeIsoHamming k).labelledCopyCount_eq (hypercubeIsoHamming n)).trans
    (labelledCopyCount_hypercubeHamming n k)
```

The Unlabelled Count

Taking $n = k$, the cube `Q[k]` has `2 ^ k * k!` automorphisms (as we already knew from `card_aut_hypercube`), and dividing the labelled count by this number gives the unlabelled count.

The number of subgraphs of `Q[n]` isomorphic to `Q[k]`: choose the k free coordinates, then fix the remaining $n - k$ coordinates arbitrarily.

```
theorem copyCount_hypercube (n k : ℕ) (h : k ≤ n) :
    Q[n].copyCount Q[k] = n.choose k * 2 ^ (n - k) := by
  have hcopy : Q[k].labelledCopyCount Q[k] = 2 ^ k * k.factorial := by
    rw [labelledCopyCount_hypercube, Nat.descFactorial_self]
  have hlabel := labelledCopyCount_hypercube n k
  rw [labelledCopyCount_eq_copyCount_mul_labelledCopyCount,
    Nat.descFactorial_eq_factorial_mul_choose, hcopy] at hlabel
  -- positivity proves goals 0 < e or 0 ≤ e from the form of e
  exact Nat.eq_of_mul_eq_mul_right (by positivity : 0 < 2 ^ k * k.factorial)
    (by grind [← pow_add, Nat.add_sub_cancel' h])
```

Every Copy is Induced

A canonical subcube copy preserves Hamming distances: translation is an automorphism, and `extendEmb d` preserves the Hamming weight since the coordinates outside the image of d are all `false`. So a canonical copy preserves graph distances, i.e. it is an *isometric* embedding.

```
lemma hypercubeHamming.hammingNorm_extendEmb {n k : ℕ} (d : Fin k ↪ Fin n)
    (z : Fin k → Bool) : hammingNorm (extendEmb d z) = hammingNorm z := by
  simp only [hammingNorm]
  rw [← Finset.card_map d]
```

```
  congr 1
  ext j
  by_cases hj : ∃ i, d i = j
  · obtain ⟨i, rfl⟩ := hj
    simp
  · simp_all [extendEmb_apply_of_not_exists _ hj, Bool.zero_eq_false]

lemma hypercubeHamming.hammingDist_extendEmb {n k : ℕ} (d : Fin k ↪ Fin n)
    (x y : Fin k → Bool) :
    hammingDist (extendEmb d x) (extendEmb d y) = hammingDist x y := by
  rw [hammingDist_eq_hammingNorm, hammingDist_eq_hammingNorm,
    BooleanRing.neg_eq, BooleanRing.neg_eq, ← extendEmb_add, hammingNorm_extendEmb]

lemma hypercubeHamming.edist_subcubeCopy {n k : ℕ} (b : Fin n → Bool)
    (d : Fin k ↪ Fin n) (x y : Fin k → Bool) :
    (hypercubeHamming n).edist (subcubeCopy b d x) (subcubeCopy b d y) =
      (hypercubeHamming k).edist x y := by
  have h := hammingDist_aut_hypercubeHamming (hypercubeHamming.translate b)
    (extendEmb d x) (extendEmb d y)
  simp only [hypercubeHamming.translate_apply] at h
  rw [edist_hypercubeHamming, edist_hypercubeHamming, subcubeCopy_apply,
    subcubeCopy_apply, h, hammingDist_extendEmb]
```

Since adjacency is distance one, a canonical copy reflects adjacency as well as preserving it, and by rigidity every copy is canonical. So every copy of `Q[k]` in `Q[n]` is *induced*. We prove this first in the Hamming model beore transporting it to `Q[n]`.

```
lemma copy_hypercubeHamming_adj_iff {n k : ℕ}
    (f : Copy (hypercubeHamming k) (hypercubeHamming n)) {x y : Fin k → Bool} :
    (hypercubeHamming n).Adj (f x) (f y) ↔ (hypercubeHamming k).Adj x y := by
  obtain ⟨b, d, rfl⟩ := exists_eq_subcubeCopy f
  rw [← edist_eq_one_iff_adj, ← edist_eq_one_iff_adj, edist_subcubeCopy]
```

Every copy of `Q[k]` in `Q[n]` is induced.

```
lemma copy_hypercube_adj_iff {n k : ℕ} (f : Copy Q[k] Q[n])
    {x y : HypercubeVert k} : Q[n].Adj (f x) (f y) ↔ Q[k].Adj x y := by
  have h := copy_hypercubeHamming_adj_iff
    ((hypercubeIsoHamming k).copyCongr (hypercubeIsoHamming n) f)
    (x := hypercubeIsoHamming k x) (y := hypercubeIsoHamming k y)
  simpa [Iso.copyCongr, Iso.map_adj_iff, -hypercubeHamming_adj] using h
```

A copy that reflects adjacency is a graph embedding `A ↪g B` (`Embedding`), and Mathlib writes `A ⊴ B` (`IsIndContained`) for the existence of one. So if every copy of `A` in `B` reflects adjacency, then the copies of `A` in `B` and the embeddings `A ↪g B` are the same thing. [7]

If every copy of `A` in `B` reflects adjacency, then copies of `A` in `B` are the same as graph embeddings `A ↪g B`.

```
def copyEquivEmbedding {W X : Type*} {A : SimpleGraph W} {B : SimpleGraph X}
    (h : ∀ (f : Copy A B) {a b : W}, B.Adj (f a) (f b) → A.Adj a b) :
    Copy A B ≃ (A ↪g B) where
```

[7] Mathlib currently has no induced analogue of `labelledCopyCount`.

```
  toFun f := ⟨f.toEmbedding, ⟨h f, f.toHom.map_adj⟩⟩
  invFun := Embedding.toCopy
```

So the labelled count of copies of `Q[k]` in `Q[n]` also counts the embeddings.

```
theorem card_embedding_hypercube (n k : ℕ) :
    Nat.card (Q[k] ↪g Q[n]) = 2 ^ n * n.descFactorial k := by
  have h := labelledCopyCount_hypercube n k
  rwa [labelledCopyCount, Fintype.card_eq_nat_card, Nat.card_congr
    (copyEquivEmbedding fun f _ _ => (copy_hypercube_adj_iff f).1)] at h
```

12.9 Exercises

1. Let `M` be a group. Prove that for any `s : Set M` the Cayley graph `mulCayley s` is vertex-transitive. (Start by showing that left multiplication by any `g : M` is an automorphism of `mulCayley s`.)

```
open Finset
variable {V : Type*} {G : SimpleGraph V}
def autMulLeftCayley' {M : Type*} [Group M] (s : Set M) (g : M) :
    mulCayley s ≃g mulCayley s :=
  sorry

lemma isVertexTransitive_mulCayley' {M : Type*} [Group M] (s : Set M) :
    IsVertexTransitive (mulCayley s) := by
  sorry
```

2. Show that the automorphisms of a complete graph are exactly the permutations of its vertices, i.e. that its automorphism group is the symmetric group `Equiv.Perm V`, and deduce that `K[n]` has `n!` automorphisms. Hint: Mathlib's `Iso.completeGraph` turns a bijection into an isomorphism of complete graphs, and `Nat.card_perm` counts permutations.

```
def autTopMulEquivPerm' (V : Type*) :
    ((⊤ : SimpleGraph V) ≃g (⊤ : SimpleGraph V)) ≃* Equiv.Perm V :=
  sorry

lemma card_aut_completeGraph' (n : ℕ) : Nat.card (K[n] ≃g K[n]) = n.factorial := by
  sorry
```

3. Show that a graph and its complement have the same automorphisms. Hint: for the inverse map, treat the pairs of vertices `a = b` and `a ≠ b` separately.

```
def autComplMulEquiv' : (G ≃g G) ≃* (Gᶜ ≃g Gᶜ) :=
  sorry
```

4. Show that every vertex of a vertex-transitive graph with an odd number of vertices has even degree. Hint: such a graph is regular by `IsVertexTransitive.exists_isRegularOfDegree`; now use the handshake lemma in the form `sum_degrees_eq_twice_card_edges'` of The Handshake Lemma (p. 73).

```
lemma even_degree_of_isVertexTransitive' [Fintype V] [G.LocallyFinite]
    (h : G.IsVertexTransitive) (hodd : Odd (Fintype.card V)) (v : V) :
    Even (G.degree v) := by
  sorry
```

5. Show that the Kneser graph `Kn[n, k]` is vertex-transitive, and deduce that so is the Petersen graph. Hint: a permutation σ of `Fin n` gives an automorphism `x ↦ x.map σ`. Given two k-subsets `s` and `t`, glue a bijection between `s` and `t` to one between their complements, using `Fintype.equivOfCardEq` and `Equiv.subtypeCongr`, to get a permutation sending `s` onto `t`.

```
lemma isVertexTransitive_kneserGraph' (n k : ℕ) : Kn[n, k].IsVertexTransitive := by
  sorry

example : petersenGraph.IsVertexTransitive := by
  sorry
```

6. Show that the cycle `C[n + 3]` has `2 * (n + 3)` automorphisms. Hint: use `autCycleGraphMulEquivDihedral` and `DihedralGroup.nat_card`.

```
lemma card_aut_cycleGraph' (n : ℕ) :
    Nat.card (C[n + 3] ≃g C[n + 3]) = 2 * (n + 3) := by
  sorry
```

7. Show that the trace of the square of the adjacency matrix of a finite graph is twice the number of edges.

```
example [Fintype V] [DecidableEq V] [DecidableRel G.Adj] :
    (G.adjMatrix ℕ ^ 2).trace = 2 * #G.edgeFinset := by
  sorry
```

8. In the chapter, `mycielskianLift t` turns an eigenvector x of G whose entries sum to zero into an eigenvector `mycielskianLift t x` of `mμ(G)`, for any root t of `t ^ 2 = t + 1` (`adjMatrix_mycielskian_mulVec_mycielskianLift`). Show that the entries of `mycielskianLift t x` again sum to zero, so that the construction can be repeated, and deduce that the Grötzsch graph `grotzschGraph`, i.e. `mμ(C[5])`, has eigenvalues `1`, `-φ ^ 2` and `-ψ ^ 2`. Hint: `grotzschGraph` is isomorphic to `mμ(mμ(K[2]))`, by `Iso.mapMycielskian` applied to `mycielskianCompleteTwoIsoCycleFive`.

```
lemma sum_mycielskianLift' [Fintype V] {R : Type*} [CommRing R] (t : R)
    {x : V → R} (hsum : ∑ v, x v = 0) : ∑ w, mycielskianLift t x w = 0 := by
  sorry

open scoped goldenRatio in
example : 1 ∈ spectrum ℝ (grotzschGraph.adjMatrix ℝ) ∧
    -φ ^ 2 ∈ spectrum ℝ (grotzschGraph.adjMatrix ℝ) ∧
    -ψ ^ 2 ∈ spectrum ℝ (grotzschGraph.adjMatrix ℝ) := by
  sorry
```

9. Show that the characteristic polynomial of `C[5]` is `(X - 2) * (X ^ 2 + X - 1) ^ 2`, so `-φ` and `-ψ` are eigenvalues of `C[5]` of multiplicity two. Hint: proceed as for the cospectral graphs in Eigenvalues (p. 367). The vertex type of `C[5]` is already `Fin 5`, so no relabelling is needed.

```
open Polynomial in
example : (C[5].adjMatrix ℤ).charpoly = (X - 2) * (X ^ 2 + X - 1) ^ 2 := by
  sorry
```

10. Show that the entries in each row of the Laplacian sum to `0`.

```
example [Fintype V] [DecidableEq V] [DecidableRel G.Adj] (i : V) :
    ∑ j, G.lapMatrix ℤ i j = 0 := by
  sorry
```

11. Show that if `G` has no isolated vertices, then the kernel of the Laplacian of the Mycielskian `mμ(G)` is 1-dimensional.

```
example [Fintype V] [DecidableEq V] [DecidableRel G.Adj]
    (h : ∀ v, ¬G.IsIsolated v) :
    Module.finrank ℝ (mμ(G).lapMatrix ℝ).toLin'.ker = 1 := by
  sorry
```

12. Show that `C[5]` is strongly regular with parameters `(5, 2, 0, 1)`, and deduce that its diameter is 2.

```
example : C[5].IsSRGWith 5 2 0 1 ∧ C[5].ediam = 2 := by
  sorry
```

13. Show that there is no strongly regular graph with parameters `(10, 4, 0, 1)`.

```
example [Fintype V] [DecidableRel G.Adj] :
    ¬G.IsSRGWith 10 4 0 1 := by
  sorry
```

14. Show that the Hamming cube is isomorphic to the third construction of the hypercube, the Hasse diagram `hypercubeHasse` of the subsets of `Fin n`, by sending a vertex `x` to the set of coordinates `i` with `x i = true`. Hint: adding the basis vector `e i` to `x` removes `i` from this set if it is there, and inserts it otherwise. Use `hypercubeHamming_adj_iff_exists`, `hasse_adj`, `Finset.covBy_iff_exists_insert`, `Finset.covBy_insert` and `Finset.erase_covBy`.

```
def boolVecEquivFinset' (n : ℕ) : (Fin n → Bool) ≃ Finset (Fin n) where
  toFun x := {i | x i = true}
  invFun s i := decide (i ∈ s)
  left_inv x := sorry
  right_inv s := sorry
```

```
open scoped hypercubeHamming in
lemma boolVecEquivFinset_add_single' {n : ℕ} (x : Fin n → Bool) (i : Fin n) :
    boolVecEquivFinset' n (x + e i) =
      if x i = true then (boolVecEquivFinset' n x).erase i
      else insert i (boolVecEquivFinset' n x) := by
  sorry

lemma hypercubeHamming_adj_of_covBy' {n : ℕ} {x y : Fin n → Bool}
    (h : boolVecEquivFinset' n x ⋖ boolVecEquivFinset' n y) :
    (hypercubeHamming n).Adj x y := by
  sorry

def hypercubeHammingIsoHasse' (n : ℕ) : hypercubeHamming n ≃g hypercubeHasse n :=
  sorry
```

15. Show that the Hamming cube is *arc-transitive*: for any two edges `x - y` and `u - v` there is an automorphism sending `x` to `u` and `y` to `v`. Hint: if `y = x + e i` and `v = u + e j`, compose a translation, the coordinate permutation `Equiv.swap i j` and another translation.

```
open scoped hypercubeHamming in
lemma exists_aut_hypercubeHamming_of_adj' {n : ℕ} {x y u v : Fin n → Bool}
    (h : (hypercubeHamming n).Adj x y) (h' : (hypercubeHamming n).Adj u v) :
    ∃ g : hypercubeHamming n ≃g hypercubeHamming n, g x = u ∧ g y = v := by
  sorry
```

16. Show that the copies of any graph `A` in a complete graph are just the injective maps between the vertex types. Deduce that `K[n]` contains `n.descFactorial (k + 3) = n * (n - 1) * ⋯ * (n - k - 2)` labelled copies of `C[k + 3]`. Hint: `Fintype.card_embedding_eq`.

```
def copyTopEquiv' {W : Type*} (A : SimpleGraph V) :
    Copy A (⊤ : SimpleGraph W) ≃ (V ↪ W) :=
  sorry

lemma labelledCopyCount_completeGraph_cycleGraph' (n k : ℕ) :
    K[n].labelledCopyCount C[k + 3] = n.descFactorial (k + 3) := by
  sorry
```

17. Deduce that `K[n + 3]` contains `(n + 3)! / (2 * (n + 3))` Hamiltonian cycles, i.e. unlabelled copies of `C[n + 3]`. Hint: `labelledCopyCount_cycleGraph`.

```
lemma copyCount_completeGraph_cycleGraph' (n : ℕ) :
    2 * (n + 3) * K[n + 3].copyCount C[n + 3] = (n + 3).factorial := by
  sorry
```

18. Show that `C[5]` has `10` labelled copies in itself, using Exercise 6.

```
lemma labelledCopyCount_cycleGraph_five_self' :
    C[5].labelledCopyCount C[5] = 10 := by
  sorry
```

19. Show that the number of labelled copies of a finite complete graph `T` in a finite graph `G` is the number of cliques of `G` of the same size, multiplied by the number of permutations of the vertices of `T`. Hint: combine `copyCount_top` and `labelledCopyCount_eq_copyCount_mul_labelledCopyCount` with Exercise 2.

```
lemma labelledCopyCount_top' {W : Type*} [Fintype V] [Fintype W] [DecidableEq V]
    [DecidableRel G.Adj] :
    G.labelledCopyCount (T : SimpleGraph W) =
      #(G.cliqueFinset (Fintype.card W)) * (Fintype.card W).factorial := by
  sorry
```

20. Count the 4-cycles in the hypercube: show that `Q[n]` contains `2 ^ n * (n * (n - 1))` labelled copies of `C[4]` and, for `2 ≤ n`, `n.choose 2 * 2 ^ (n - 2)` unlabelled ones. Hint: transport `labelledCopyCount_hypercube` and `copyCount_hypercube` along the isomorphism `hypercubeTwoIsoCycleFour : Q[2] ≃g C[4]`, using `Iso.labelledCopyCount_eq` and `Iso.copyCount_eq`.

```
lemma labelledCopyCount_hypercube_cycleGraph_four' (n : ℕ) :
    Q[n].labelledCopyCount C[4] = 2 ^ n * (n * (n - 1)) := by
  sorry

lemma copyCount_hypercube_cycleGraph_four' (n : ℕ) (hn : 2 ≤ n) :
    Q[n].copyCount C[4] = n.choose 2 * 2 ^ (n - 2) := by
  sorry
```

13 Theorems

Mathlib formalizes quite a few classical theorems of graph theory. In this final chapter we survey some of them, building on earlier chapters: we state each result, explain how its statement is phrased in Lean, and (when applicable) apply it to our running examples. Where a natural strengthening or application is missing from Mathlib we prove it ourselves.

We begin with matchings: Hall's marriage theorem (`exists_isMatching_of_forall_ncard_le`) and its deficiency version (`exists_isMatching_of_forall_ncard_le_add`), from which we deduce Kőnig's theorem (`IsBipartite.matchingNum_eq_vertexCoverNum`), and finally Tutte's theorem (`tutte`). We then turn to extremal graph theory: Turán's theorem (`isTuranMaximal_iff_nonempty_iso_turanGraph`, with Mantel's theorem as a special case), Füredi's stability theorem (`CliqueFree.exists_colorable_deleteEdges`), the Andrásfai–Erdős–Sós theorem (`colorable_of_cliqueFree_lt_minDegree`) and extremal numbers (`extremalNumber`), together with their asymptotic behaviour: the Turán density (`turanDensity`) and the minimum degree Erdős–Stone theorem (`eventually_completeEquipartiteGraph_isContained_of_minDegree`).

The last three sections are more advanced, and they are independent of each other: each builds only on the first two sections, so they can be read in any order. The first proves Zykov's theorem on the number of cliques (`CliqueFree.card_cliqueFinset_le`) together with its case of equality (`CliqueFree.card_cliqueFinset_eq_iff_nonempty_iso_turanGraph`). It is a complete proof rather than a survey, and by far the longest section of the chapter. The second covers Szemerédi's regularity lemma (`szemeredi_regularity`) and the triangle removal lemma (`triangle_removal`), applied to Roth's theorem on arithmetic progressions (`roth_3ap_theorem_nat`) and the Ruzsa–Szemerédi theorem (`ruzsaSzemerediNumberNat_isLittleO`). The third looks at two results about infinite graphs, Kőnig's lemma (`exists_seq_covby_of_forall_covby_finite`), from which we deduce its graph form (`Connected.pathNat_isContained`), and the existence of ends (`nonempty_ends_of_infinite`).

Many famous theorems are not (yet) in Mathlib. These include Ramsey's theorem for graphs, the Kővári–Sós–Turán theorem, Kirchhoff's matrix-tree theorem [Kirchhoff 1847], Cayley's formula [Cayley 1889], Euler's circuit theorem, Euler's formula [Euler 1758] and planarity, Menger's theorem, Brooks' theorem, Vizing's theorem, Berge's theorem [Berge 1957], Kőnig's theorem on bipartite matchings and the theorems of Dirac and Ore. Some of these are proved in this guide:

- the simplest case `R(3, 3) = 6` of Ramsey's theorem (`not_cliqueFree_three_and_indepSetFree_three`)
- Euler's theorem on Eulerian trails (`exists_isEulerian_iff`)
- the theorems of Dirac (`isHamiltonian_of_card_le_two_mul_minDegree`) and Ore (`IsOreGraph.isHamiltonian`)

- Brooks' theorem (`brooks`)
- Kőnig's theorem (p. 409) (`IsBipartite.matchingNum_eq_vertexCoverNum`)

Whole areas of graph theory are also missing. The biggest gap is probabilistic graph theory: Mathlib defines the binomial random graph `G(V, p)` (`binomialRandom`), in which each pair of vertices is joined independently with probability `p`, but proves almost nothing about it, not even that a given pair is an edge with probability `p`. So the classical applications of the probabilistic method to graphs are missing too, including Erdős's lower bound for Ramsey numbers and his theorem on graphs of large girth and large chromatic number [Erdős 1959].

13.1 Matchings

Recall that a matching in `G` (`Subgraph.IsMatching`) is a subgraph `M` in which every vertex of `M` has exactly one neighbour in `M`. It need not be spanning: its vertices are the matched vertices. A perfect matching is a spanning matching, so every vertex of `G` has exactly one `M`-neighbour.

```
variable {V : Type*} {G : SimpleGraph V}
open Finset

example (M : G.Subgraph) : M.IsPerfectMatching ↔ ∀ v, ∃! w, M.Adj v w :=
  Subgraph.isPerfectMatching_iff
```

The vertices of a perfect matching are paired up, so a finite graph with a perfect matching has an even number of vertices.

```
#check Subgraph.IsPerfectMatching.even_card
```

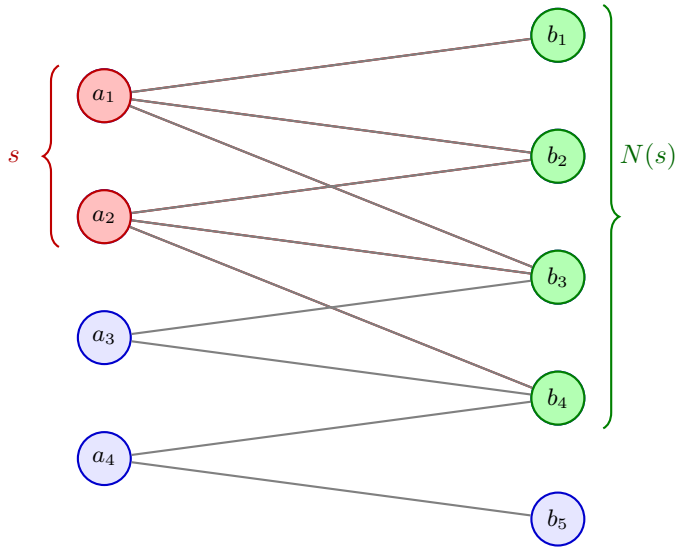
For example, the Mycielskian `mμ(G)` of a finite graph has `2 * |V| + 1` vertices, an odd number, so it never has a perfect matching.

```
example [Finite V] (M : mμ(G).Subgraph) : ¬M.IsPerfectMatching := fun hM => by
  have := Fintype.ofFinite V
  simpa [Nat.even_add_one] using hM.even_card
```

The classical existence results for perfect matchings are Hall's marriage theorem (for bipartite graphs) and Tutte's theorem (for general graphs).

Hall's Marriage Theorem

Let `G` be bipartite with parts p_1 and p_2 (`IsBipartiteWith`). *Hall's condition* says that every finite set $s \subseteq p_1$ has at least $|s|$ neighbours. *Hall's theorem* [Hall 1935] says that this condition implies that `G` has a matching whose vertices include all of p_1. (The converse is easy: the partners of the vertices in `s` are distinct neighbours of `s`.)



A bipartite graph with a subset `s` and its open neighbourhood `N(s)`

```
#check exists_isMatching_of_forall_ncard_le
```

Notice that *V* need not be finite: Mathlib only assumes that *G* is locally finite, and we will see that even this is more than the proof needs. Hall's condition is stated with `Set.ncard`, which is `0` on infinite sets, so it holds trivially for infinite `s` and only finite sets need to be checked.

```
#check Set.Infinite.ncard -- s.Infinite → s.ncard = 0

example {s : Set V} (hs : s.Infinite) :
    s.ncard ≤ (⋃ x ∈ s, G.neighborSet x).ncard := by
  simp [hs.ncard]
```

For a perfect matching, Mathlib asks for Hall's condition for every set *s* of vertices, which may contain vertices from both sides.

```
#check exists_isPerfectMatching_of_forall_ncard_le
```

Both theorems come from the statement about *systems of distinct representatives*: a family of finite sets `t : ι → Finset α` has an injective choice function iff every union of `k` of the sets has at least `k` elements. Mathlib states it for any index type `ι`. For infinite `ι` it follows from the finite case by a compactness argument: each finite set of indices has at least one injective choice and only finitely many, and `nonempty_sections_of_finite_inverse_system` makes the choices agree.

```
#check Finset.all_card_le_biUnion_card_iff_exists_injective
```

Mathlib applies it to the finsets `G.neighborFinset x` with `x ∈ p₁`, and it is to form these that it assumes `[G.LocallyFinite]`. For a perfect matching it applies it to all of *V*, which is why the condition is stated for all sets of vertices. This gives injections `p₁ → p₂` and `p₂ → p₁` along edges. If *V* is finite then `|p₁| = |p₂|`, and either one is already a perfect matching. In general Mathlib combines them using a relational form of the Schröder–Bernstein theorem (`Function.Embedding.schroeder_bernstein_of_rel`), which yields a bijection `p₁ → p₂` along edges.

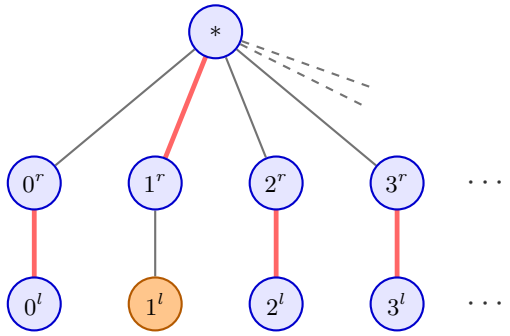
```
#check exists_bijective_of_forall_ncard_le
```

Local finiteness cannot be dropped from the classical theorem. An example is a *subdivided star*: a star with each edge subdivided by a new vertex. Its vertices are a centre and two copies of an index type `ι`, so we reuse the vertex type `Option (ι ⊕ ι)` of the Mycielskian and its notation: the centre none is adjacent to every $\mathtt{i}^{\mathtt{r}}$, and each $\mathtt{i}^{\mathtt{l}}$ is adjacent only to $\mathtt{i}^{\mathtt{r}}$.

The *subdivided star* on `ι`: the centre none is adjacent to every $\mathtt{i}^{\mathtt{r}}$, and each $\mathtt{i}^{\mathtt{l}}$ is adjacent only to $\mathtt{i}^{\mathtt{r}}$.

```
@[simps]
def subdividedStar (ι : Type*) : SimpleGraph (Option (ι ⊕ ι)) where
  Adj x y := match x, y with
    | none, _ʳ | _ʳ, none => True
    | iˡ, jʳ | jʳ, iˡ => i = j
    | _, _ => False
  symm := ⟨by rintro (_ | _ | _) (_ | _ | _) h <;> simp_all [eq_comm]⟩
  loopless := ⟨by rintro (_ | _ | _) <;> simp⟩
```

In `subdividedStar ℕ`, every finite set of the vertices none, $0^{\mathtt{l}}$, $1^{\mathtt{l}}$, … has at least as many neighbours as elements, but no matching covers them all: whichever $\mathtt{j}^{\mathtt{r}}$ is matched to none, the vertex $\mathtt{j}^{\mathtt{l}}$ is left unmatched, since $\mathtt{j}^{\mathtt{r}}$ is its only neighbour. In particular, a subdivided star has no perfect matching.



The `subdividedStar ℕ`. A matching (red) covering none (drawn as ⊠) leaves a vertex (orange) unmatched

A subdivided star has no perfect matching.

```
lemma not_exists_isPerfectMatching_subdividedStar {ι : Type*} :
    ¬∃ M : (subdividedStar ι).Subgraph, M.IsPerfectMatching := by
  rintro ⟨M, hM, hspan⟩
  -- none is matched to a right vertex jʳ
  obtain ⟨w, hw, -⟩ := hM (hspan none)
  obtain _ | j | j := w
  · simpa using M.adj_sub hw
  · simpa using M.adj_sub hw
  -- jˡ is matched to jʳ as well
  obtain ⟨u, hu, -⟩ := hM (hspan jˡ)
  obtain _ | k | k := u
  · simpa using M.adj_sub hu
  · simpa using M.adj_sub hu
  obtain rfl : j = k := by simpa using M.adj_sub hu
  simpa using hM.eq_of_adj_right hw hu
```

In Lean, though, this graph fails Hall's condition for the set {none}: none has infinitely many neighbours, so the `Set.ncard` of its neighbourhood is `0`.

```
example : ¬({none} : Set (Option (ℕ ⊕ ℕ))).ncard ≤
    (⋃ x ∈ ({none} : Set _), (subdividedStar ℕ).neighborSet x).ncard := by
  -- the neighbours of none are the right vertices iʳ
  have h : ((subdividedStar ℕ).neighborSet none).Infinite :=
    Set.infinite_of_injective_forall_mem mycielskian.right_injective (by simp)
  simp [h.ncard]
```

So Lean's statement of Hall's condition already forces every vertex of `p₁` to have finite degree. `exists_isMatching_of_forall_ncard_le` assumes `[G.LocallyFinite]`, which asks the same of `p₂`, but its proof does not need this: it picks partners for the vertices of `p₁` from their neighbourhoods and never looks at the neighbourhood of a vertex in `p₂`. Mathlib's relational form of the theorem on systems of distinct representatives asks for exactly this one-sided finiteness: each `a : α` must be related to finitely many elements of `β`, and nothing is assumed about `β`.

```
#check Fintype.all_card_le_rel_image_card_iff_exists_injective
```

Kőnig's Theorem

In `matchingNum_le_vertexCoverNum` we proved weak duality `ν G ≤ τ G`: a vertex cover needs to hit every edge of a matching. *Kőnig's theorem* [Kőnig 1931] says that for bipartite graphs this is an equality. It is not in Mathlib, but we can deduce it from a *deficiency version* of Hall's theorem.

Suppose that every finite `s ⊆ p₁` has at least `|s| - d` neighbours. We show that we can then match all but at most `d` vertices of `p₁`. Add `d` new vertices adjacent to every vertex of `p₁`. Hall's condition now holds, so the systems of distinct representatives theorem gives an injective choice of neighbours `g`, with values in `V ⊕ Fin d`. At most `d` vertices of `p₁` are sent to new vertices, and the others are sent injectively to neighbours in `G`, which gives a matching by `exists_isMatching_of_injective`. We state this last step separately: it is the only one that uses the bipartition.

Mathlib's proofs of Hall's theorem build the matching from the chosen neighbours by hand, but the same construction is available as a lemma: a bijection `f : s ≃ t` between disjoint sets, with each `v` adjacent to `f v`, gives a matching with vertex set `s ∪ t`.

```
#check Subgraph.IsMatching.exists_of_disjoint_sets_of_equiv
```

In a bipartite graph, an injective choice of neighbours `f` for the vertices of `s ⊆ p₁` is such a bijection onto its range, since the range lies in `p₂`, which is disjoint from `s`. So the edges `x -ᵥ f x` form a matching covering `s`.

```
lemma exists_isMatching_of_injective {p₁ p₂ s : Set V} (h₁ : G.IsBipartiteWith p₁ p₂)
    (hs : s ⊆ p₁) (f : s → V) (hf : Function.Injective f)
    (hadj : ∀ x : s, G.Adj x (f x)) :
    ∃ M : Subgraph G, s ⊆ M.verts ∧ M.IsMatching := by
  have hd : Disjoint s (Set.range f) := h₁.disjoint.mono hs
    (Set.range_subset_iff.mpr fun x =>
      h₁.mem_of_mem_adj (hs x.2) (hadj x))
  obtain ⟨M, hverts, hM⟩ :=
    Subgraph.IsMatching.exists_of_disjoint_sets_of_equiv
      hd (Equiv.ofInjective f hf) hadj
  exact ⟨M, hverts ▸ Set.subset_union_left, hM⟩
```

We count the unmatched vertices with `Set.encard`, not `Set.ncard`. Since the latter is `0` on infinite sets, `(p₁ \ M.verts).ncard ≤ d` would also hold if infinitely many vertices of `p₁` were unmatched.

An injective choice, for each vertex of p_1, of either a neighbour or one of d new vertices gives a matching that covers all but at most d vertices of p_1.

```
lemma exists_isMatching_of_injective_sum {p₁ p₂ : Set V} {d : ℕ}
    (h₁ : G.IsBipartiteWith p₁ p₂) (g : p₁ → V ⊕ Fin d) (hg : Function.Injective g)
    (hadj : ∀ x y, g x = .inl y → G.Adj x y) :
    ∃ M : Subgraph G, M.IsMatching ∧ (p₁ \ M.verts).encard ≤ d := by
  -- the vertices X of p₁ that chose an old vertex are matched in G
  let X : Set V := {x | ∃ hx : x ∈ p₁, (g ⟨x, hx⟩).isLeft}
  have hX : X ⊆ p₁ := fun x hx => hx.1
  let f : X → V := fun x => (g ⟨x, x.2.1⟩).getLeft x.2.2
  have hgf (x : X) : g ⟨x, x.2.1⟩ = .inl (f x) := (Sum.inl_getLeft _ _).symm
  have hf : Function.Injective f := fun x y hxy => by
    have hgxy : g ⟨x, x.2.1⟩ = g ⟨y, y.2.1⟩ := by rw [hgf, hgf, hxy]
    simpa [Subtype.ext_iff] using hg hgxy
  obtain ⟨M, hXM, hM⟩ :=
    exists_isMatching_of_injective h₁ hX f hf fun x => hadj _ _ (hgf x)
  refine ⟨M, hM, (Set.encard_le_encard (Set.sdiff_subset_sdiff_right hXM)).trans ?_⟩
  -- the other vertices of p₁ chose distinct new vertices
  have hp (x : ↑(p₁ \ X)) : (x : V) ∈ p₁ := x.2.1
  have hr (x : ↑(p₁ \ X)) : (g ⟨x, hp x⟩).isRight := by
    simpa [X, hp x] using x.2.2
  let φ : ↑(p₁ \ X) → Fin d := fun x => (g ⟨x, hp x⟩).getRight (hr x)
  have hgφ (x : ↑(p₁ \ X)) : g ⟨x, hp x⟩ = .inr (φ x) := (Sum.inr_getRight _ _).symm
  have hφ : Function.Injective φ := fun x y hxy => by
    have hgxy : g ⟨x, hp x⟩ = g ⟨y, hp y⟩ := by rw [hgφ, hgφ, hxy]
    simpa [Subtype.ext_iff] using hg hgxy
  simpa using ENat.card_le_card_of_injective hφ
```

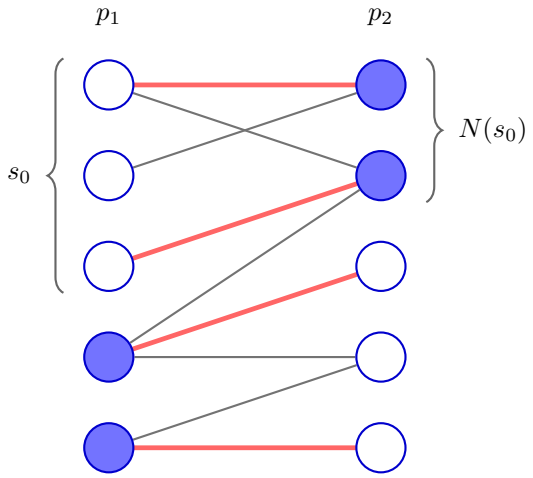
What remains is to check Hall's condition after adding the new vertices. For `d > 0` the deficiency condition (h_2 below) holds for `{x}` even if `x` has infinite degree, so we need to assume that the vertices of p_1 have finite degree. (The vertices of p_2 may have infinite degree.) With this assumption, the junk value of `Set.ncard` does no harm in the deficiency condition: it only makes the condition hold automatically for infinite `s`.

Hall's theorem with deficiency d: if the vertices of p_1 have finite degree and every `s ⊆ p₁` has at least `|s| - d` neighbours, then a matching covers all but at most d vertices of p_1.

```
theorem exists_isMatching_of_forall_ncard_le_add {p₁ p₂ : Set V} {d : ℕ}
    (h₁ : G.IsBipartiteWith p₁ p₂) (hfin : ∀ x ∈ p₁, (G.neighborSet x).Finite)
    (h₂ : ∀ s ⊆ p₁, s.ncard ≤ (⋃ x ∈ s, G.neighborSet x).ncard + d) :
    ∃ M : Subgraph G, M.IsMatching ∧ (p₁ \ M.verts).encard ≤ d := by
  classical
  -- each vertex of p₁ may choose a neighbour, or one of d new vertices
  let N (x : p₁) : Finset V := (hfin x x.2).toFinset
  let t : p₁ → Finset (V ⊕ Fin d) := fun x => (N x).map .inl ∪ univ.map .inr
  -- Hall's condition for t: a nonempty s sees its neighbours and all d new
  -- vertices
  have hHall : ∀ s : Finset p₁, #s ≤ #(s.biUnion t) := by
    intro s
    rcases s.eq_empty_or_nonempty with rfl | ⟨x₀, hx₀⟩
    · simp
    have hs := h₂ (s.image Subtype.val) (by simp)
    rw [Set.ncard_coe_finset,
      card_image_of_injective _ Subtype.val_injective] at hs
```

```
    have hN : (⋃ x ∈ (s.image Subtype.val : Set V), G.neighborSet x) =
        (s.biUnion N : Set V) := by
      ext; simp [N]
    rw [hN, Set.ncard_coe_finset] at hs
    have : s.biUnion t = (s.biUnion N).map .inl ∪ univ.map .inr := by
      ext (v | i)
      · simp [t]
      · simpa [t] using ⟨x₀.1, x₀.2, hx₀⟩
    rw [this, card_union_of_disjoint (disjoint_map_inl_map_inr _ _), card_map,
      card_map, card_univ, Fintype.card_fin]
    exact hs
  obtain ⟨g, hg₁, hg₂⟩ :=
    (Finset.all_card_le_biUnion_card_iff_exists_injective t).1 hHall
  exact exists_isMatching_of_injective_sum h₁ g hg₁ fun x y hxy => by
    simpa [t, N, hxy] using hg₂ x
```

Now let `s₀ ⊆ p₁` have the largest deficiency `d = |s₀| - |N(s₀)|`, where `N(s)` is the open neighbourhood of `s`, i.e. `⋃ x ∈ s, G.neighborSet x`. Every `s ⊆ p₁` has at least `|s| - d` neighbours, so there is a matching with at least `|p₁| - d` edges (by `Subgraph.IsMatching.ncard_inter_le`). On the other hand the vertices of `p₁ \ s₀`, together with `N(s₀)`, form a vertex cover: an edge with an endpoint in `p₁` either starts in `p₁ \ s₀` or ends in `N(s₀)`. This vertex cover has `|p₁| - |s₀| + |N(s₀)| = |p₁| - d` vertices, so `τ G ≤ ν G`.



A bipartite graph with parts `p₁` and `p₂`, in which `s₀` has the largest deficiency `d = |s₀| - |N(s₀)| = 1`. The vertex cover `(p₁ \ s₀) ∪ N(s₀)` and a maximum matching both have `|p₁| - d = 4` elements.

Kőnig's theorem: in a finite bipartite graph the matching number equals the vertex cover number.

```
theorem IsBipartite.matchingNum_eq_vertexCoverNum [Finite V] (h : G.IsBipartite) :
    ν G = τ G := by
  classical
  have := Fintype.ofFinite V
  obtain ⟨p₁, p₂, h₁⟩ := h.exists_isBipartiteWith
  refine G.matchingNum_le_vertexCoverNum.antisymm ?_
  -- choose s₀ ⊆ p₁ of maximum deficiency |s₀| - |N(s₀)|
  obtain ⟨s₀, hs₀, hmax⟩ :=
    (Set.toFinite {s : Set V | s ⊆ p₁}).toFinset.exists_max_image
      (fun s => (s.ncard : ℤ) - (⋃ x ∈ s, G.neighborSet x).ncard) ⟨∅, by simp⟩
```

```
  simp_rw [Set.Finite.mem_toFinset] at hs₀ hmax
  let N₀ := ⋃ x ∈ s₀, G.neighborSet x
  let d := s₀.ncard - N₀.ncard
  -- the deficiency of s₀ is at least that of ∅, which is 0
  have h0 : N₀.ncard ≤ s₀.ncard := by simpa using hmax ∅ (Set.empty_subset _)
  obtain ⟨M, hM, hMd⟩ := exists_isMatching_of_forall_ncard_le_add (d := d) h₁
    (fun _ _ => Set.toFinite _) fun s hs => by grind
  -- (p₁ \ s₀) ∪ N₀ is a vertex cover
  have hC : G.IsVertexCover ((p₁ \ s₀) ∪ N₀) := fun u v huv => by
    rcases h₁.mem_of_adj huv with ⟨hu, -⟩ | ⟨-, hv⟩
    · by_cases hus : u ∈ s₀
      exacts [.inr (.inr (Set.mem_biUnion hus huv)), .inl (.inl ⟨hu, hus⟩)]
    · by_cases hvs : v ∈ s₀
      exacts [.inl (.inr (Set.mem_biUnion hvs huv.symm)), .inr (.inl ⟨hv, hvs⟩)]
  -- it has at most |p₁| - d vertices, and M has at least |p₁| - d edges
  replace hMd : (p₁ \ M.verts).ncard ≤ d := by
    rwa [← Set.Finite.cast_ncard_eq (Set.toFinite _), Nat.cast_le] at hMd
  have := Set.ncard_union_le (p₁ \ s₀) N₀
  have := Set.ncard_sdiff_add_ncard_of_subset hs₀
  have := Set.ncard_inter_add_ncard_sdiff_eq_ncard p₁ M.verts
  have := hM.ncard_inter_le h₁
  calc τ G ≤ ((p₁ \ s₀) ∪ N₀).encard := hC.vertexCoverNum_le
    _ = ((p₁ \ s₀) ∪ N₀).ncard := (Set.Finite.cast_ncard_eq (Set.toFinite _)).symm
    _ ≤ M.edgeSet.ncard := by grind [Nat.cast_le]
    _ = M.edgeSet.encard := Set.Finite.cast_ncard_eq (Set.toFinite _)
    _ ≤ ν G := hM.le_matchingNum
```

The hypercube `Q[n + 1]` is bipartite (`IsBipartite.hypercube`) and has matching number `2 ^ n` (`matchingNum_hypercube_succ`). So it also has vertex cover number `2 ^ n`.

```
lemma vertexCoverNum_hypercube_succ (n : ℕ) : τ Q[n + 1] = 2 ^ n := by
  rw [← (IsBipartite.hypercube _).matchingNum_eq_vertexCoverNum,
    matchingNum_hypercube_succ]
```

Symmetric Differences of Matchings

The classical approach to matchings studies the symmetric difference of two matchings. Since each vertex belongs to exactly one edge in each of two perfect matchings `M` and `N`, every vertex of `M ∆ N` has degree `0` or `2`, so `M ∆ N` is a union of disjoint cycles (finite or infinite) whose edges alternate between `M` and `N`. Mathlib states this for `SimpleGraph`s rather than subgraphs, so the matchings are first converted with `spanningCoe`.

```
#source IsCycles -- ∀ {v}, (G.neighborSet v).Nonempty → (G.neighborSet v).ncard = 2
```

Notice how the definition carefully avoids mentioning `Set.ncard = 0`, which would allow vertices of infinite degree.

```
#check Subgraph.IsPerfectMatching.symmDiff_isCycles
#check Subgraph.IsPerfectMatching.isAlternating_symmDiff_left
```

This is the machinery behind Mathlib's proof of Tutte's theorem below. Each vertex of `M ∆ N` has no neighbours or exactly two, so by `Walk.IsCycle.toSubgraph_eq_of_degree_le_two` every cycle in `M ∆ N` is a whole component of `M ∆ N`. That is the easy part, since we are given the cycle. The

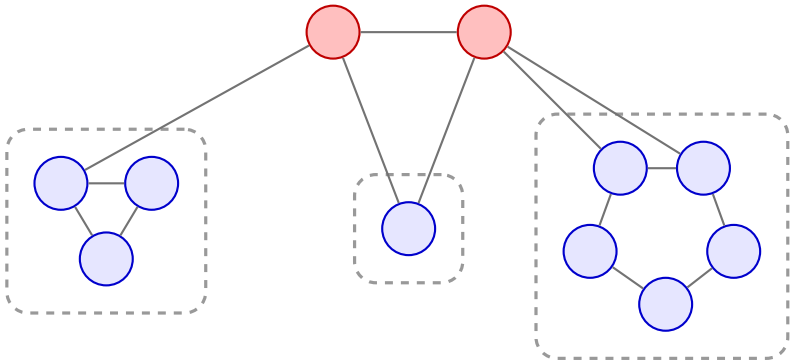
hard part is finding one in the first place. If `V` is finite, every non-isolated vertex lies on a cycle whose vertices form its whole component.

```
#check IsCycles.exists_cycle_toSubgraph_verts_eq_connectedComponentSupp
```

The assumption `[Finite V]` cannot be dropped. On the vertices `ℤ`, let `M` match each even `i` with `i + 1`, and let `N` match it with `i - 1`. Then `M ∆ N` is the two-way infinite path through all of `ℤ`, which has no cycles.

Tutte's Theorem

Tutte's theorem [Tutte 1947] characterizes the graphs with a perfect matching, bipartite or not. If `G` has a perfect matching and we delete a set `U` of vertices, then each odd component of what remains (i.e. of `G - U`) must have a vertex matched into `U`, so there are at most `|U|` odd components. A set `U` whose deletion leaves more than `|U|` odd components is called a *Tutte violator*. Tutte's theorem says that `G` has a perfect matching iff there is no Tutte violator.



A Tutte violator `U` with three odd components of `G - U`

```
#source SimpleGraph.IsTutteViolator
```

Here `G - U` is the subgraph `(⊤ : G.Subgraph).deleteVerts U`, viewed as a simple graph via `Subgraph.coe`.

A graph with a perfect matching has no Tutte violator, and Tutte's theorem is the converse.

```
#check SimpleGraph.not_isTutteViolator_of_isPerfectMatching
#check SimpleGraph.tutte
```

Mathlib proves Tutte's theorem for `[Finite V]`. As with Hall's condition, the definition of a Tutte violator uses `Set.ncard`, which is `0` on infinite sets. So in an infinite graph, deleting every vertex but one, `x`, gives a Tutte violator: the infinite set `{x}ᶜ` has `Set.ncard` equal to `0`, while `x` alone forms one odd component. This is an accident of the junk value, but the classical theorem fails for infinite graphs too. In `subdividedStar ℕ` from the Hall section, deleting any finite set `U` leaves at most `|U|` finite odd components, yet there is no perfect matching (`not_exists_isPerfectMatching_subdividedStar`).

13.2 Turán's Theorem, Stability and Asymptotics

Extremal graph theory asks how many edges a graph on `n` vertices can have while avoiding a given subgraph. The foundational result is *Turán's theorem*, which answers this question when the for-

bidden subgraph is a complete graph. Recall from Cliques (p. 127) that `G.CliqueFree (r + 1)` says that `G` has no clique of size `r + 1`.

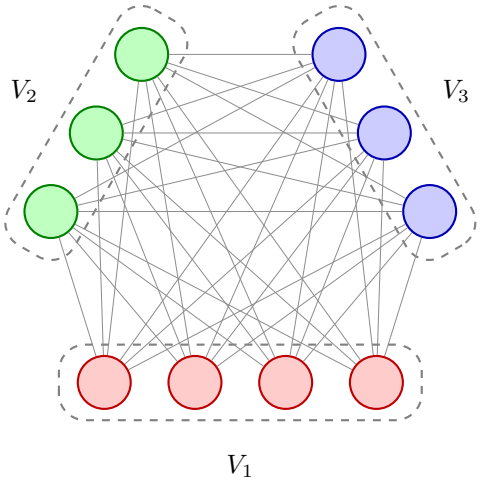
Turán Graphs

The extremal graphs are the *Turán graphs*. Mathlib defines `turanGraph n r` on `Fin n`, and we introduce the notation `T[n, r]`.

```
#source turanGraph
```

`T[n, r]` is the Turán graph `turanGraph n r`: the complete `r`-partite graph on `Fin n` whose parts are the residue classes modulo `r`.

```
scoped notation "T[" n ", " r "]" => turanGraph n r
```



The Turán graph `T[10, 3]`

Two vertices of `T[n, r]` are adjacent iff they have different residues modulo `r`, so it is the complete `r`-partite graph whose parts are the residue classes. These parts are *balanced*: any two differ in size by at most one, since the residue classes of `0, …, n - 1` modulo `r` have sizes `⌊n / r⌋` or `⌈n / r⌉`. In Mathlib's terms they form an *equipartition* (`Finpartition.IsEquipartition`, defined using `Set.EquitableOn`). For example, when `r` divides `n` the Turán graph `T[n, r]` is isomorphic to Mathlib's `completeEquipartiteGraph r (n / r)`, which has `r` parts of size `n / r`.

```
#check completeEquipartiteGraph.turanGraph
```

The case `r = 0` is a junk value. Since `a % 0 = a` in Lean, any two distinct vertices are adjacent in `T[n, 0]`, so `T[n, 0] = K[n]`. Most results about Turán graphs assume `0 < r`.

```
example (n : ℕ) : T[n, 0] = K[n] := by
  simp
```

For instance, a graph is 1-clique-free iff it has no vertices, so `T[n, 0]` is not 1-clique-free when `0 < n`.

```
#check cliqueFree_one
#check turanGraph_cliqueFree -- 0 < r → T[n, r].CliqueFree (r + 1)
```

The number of edges of `T[n, r]` is called `turanNumber n r`. Turán graphs have decidable adjacency, so we can compute it.

```
#eval turanNumber 10 3 -- 33
#check turanNumber_eq
```

In particular `turanNumber n r` is at most `(1 - 1 / r) * n ^ 2 / 2`.

```
#check mul_turanNumber_le -- 2 * r * turanNumber n r ≤ (r - 1) * n ^ 2
```

Turán's Theorem

Mathlib calls a graph `G` *Turán-maximal* if it is `(r + 1)`-clique-free and has at least as many edges as any other `(r + 1)`-clique-free graph on the same vertices. This is a special case of `IsExtremal`, which says that a graph has the most edges among the graphs on its vertex type with a given property.

```
#source IsExtremal
#source IsTuranMaximal
```

Turán's theorem [Turán 1941] says that for `0 < r` the Turán-maximal graphs are exactly the graphs isomorphic to the Turán graph.

```
#check isTuranMaximal_iff_nonempty_iso_turanGraph

example (n : ℕ) : T[n, 4].IsTuranMaximal 4 :=
  isTuranMaximal_turanGraph (by decide)
```

The version of Turán's theorem that is usually quoted is the resulting upper bound on the number of edges.

```
#check CliqueFree.card_edgeFinset_le
```

In terms of the clique number `ω` (`cliqueNum`): a finite graph with `ω G ≤ r` has no `(r + 1)`-clique, so it has at most `turanNumber n r` edges.

```
example [Fintype V] [DecidableRel G.Adj] {r : ℕ} (h : ω G ≤ r) :
    #G.edgeFinset ≤ turanNumber (Fintype.card V) r :=
  CliqueFree.card_edgeFinset_le fun s hs => by
    have := hs.le_cliqueNum
    grind
```

Mantel's Theorem

The case `r = 2` of Turán's theorem is *Mantel's theorem* [Mantel 1907]: a triangle-free graph on `n` vertices has at most `⌊n² / 4⌋` edges. Equality holds for the balanced complete bipartite graph `T[n, 2]`.

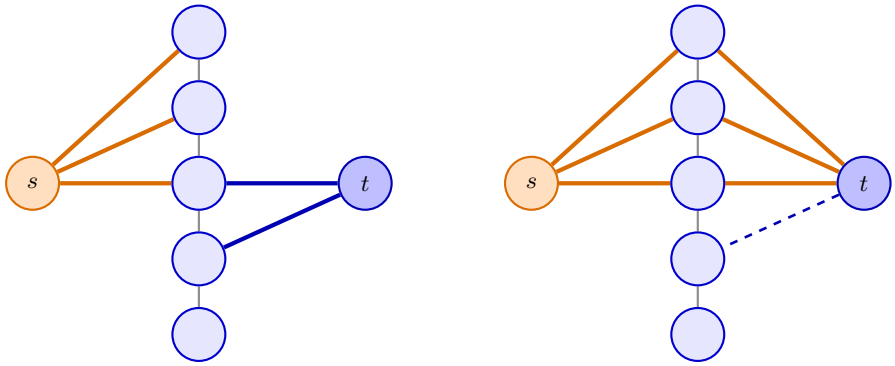
```
#check turanNumber_two -- turanNumber n 2 = n ^ 2 / 4
```

By `CliqueFree.mycielskian`, the Mycielskian of a triangle-free graph on `n` vertices is a triangle-free graph on `2 * n + 1` vertices, so Mantel's theorem bounds its number of edges by `(2 * n + 1) ^ 2 / 4`.

```
example [Fintype V] [DecidableRel G.Adj] (h3 : G.CliqueFree 3) :
    #mμ(G).edgeFinset ≤ (2 * Fintype.card V + 1) ^ 2 / 4 := by
  convert CliqueFree.card_edgeFinset_le h3.mycielskian
  simp [two_mul, turanNumber_two]
```

Proof of Turán's Theorem

Mathlib's proof of Turán's theorem uses *Zykov symmetrization*: replacing a vertex `t` by a copy of a non-adjacent vertex `s` (giving `t` exactly the neighbours of `s`) cannot create an `(r + 1)`-clique, and it removes the edges at `t` while adding as many edges as `s` has. This operation is `replaceVertex`. In a Turán-maximal graph no such replacement can increase the number of edges, so non-adjacent vertices have the same degree (`IsTuranMaximal.degree_eq_of_not_adj`). It follows that non-adjacency is transitive (`IsTuranMaximal.not_adj_trans`), so the graph is complete multipartite (`IsCompleteMultipartite`).



Left: a graph with non-adjacent vertices `s` and `t`. Right: `G.replaceVertex s t`.

```
#source replaceVertex
```

Mathlib finishes its proof of Turán's theorem by showing that a Turán-maximal graph, which is now known to be complete multipartite, is isomorphic to the Turán graph.

```
#check IsTuranMaximal.nonempty_iso_turanGraph
```

It does this through a series of lemmas about the parts of the graph, such as `IsTuranMaximal.card_parts`. Besides being `(r + 1)`-clique-free, the argument only needs three properties of a Turán-maximal graph: it is complete multipartite, its parts are balanced, and it has an `r`-clique unless it is complete. We state this last one as `∀ x y, x ≠ y → ¬G.Adj x y → ¬G.CliqueFree r` (the hypothesis `hsat` below), which is how the argument uses it. For a Turán-maximal graph, adding the edge `x — y` creates an `(r + 1)`-clique, and removing `x` from it leaves an `r`-clique in `G`. Below we reprove Mathlib's lemmas for any complete multipartite graph with these properties, following Mathlib's proofs very closely (often verbatim). We will need these results for Zykov's theorem (p. 426).

The parts of a finite complete multipartite graph form a `Finpartition` of its vertices, whose parts are the equivalence classes of non-adjacency (`IsCompleteMultipartite.setoid`).

```
namespace IsCompleteMultipartite
variable [Fintype V] [DecidableEq V] [DecidableRel G.Adj]

instance (hc : G.IsCompleteMultipartite) : DecidableRel hc.setoid.r :=
```

```
  inferInstanceAs (DecidableRel (¬G.Adj · ·))
```

The parts of a finite complete multipartite graph, as a `Finpartition` of its vertices.

```
def finpartition (hc : G.IsCompleteMultipartite) : Finpartition (univ : Finset V) :=
  Finpartition.ofSetoid hc.setoid
```

Two vertices are non-adjacent iff they lie in the same part.

```
lemma not_adj_iff_part_eq (hc : G.IsCompleteMultipartite) {s t : V} :
    ¬G.Adj s t ↔ hc.finpartition.part s = hc.finpartition.part t := by
  change hc.setoid.r s t ↔ _
  rw [← Finpartition.mem_part_ofSetoid_iff_rel]
  change t ∈ hc.finpartition.part s ↔ _
  rw [hc.finpartition.mem_part_iff_part_eq_part (mem_univ t) (mem_univ s), eq_comm]
```

The part of v consists of the non-neighbours of v.

```
lemma part_finpartition (hc : G.IsCompleteMultipartite) (v : V) :
    hc.finpartition.part v = ({x | ¬G.Adj v x} : Finset V) := by
  ext x
  simp [finpartition, Finpartition.mem_part_ofSetoid_iff_rel, setoid]
```

If the parts are balanced, they form an equipartition.

```
lemma isEquipartition (hc : G.IsCompleteMultipartite)
    (hbal : ∀ v w, #{x | ¬G.Adj w x} ≤ #{x | ¬G.Adj v x} + 1) :
    hc.finpartition.IsEquipartition := by
  intro large small hl hs
  obtain ⟨w, hw⟩ := hc.finpartition.nonempty_of_mem_parts hl
  obtain ⟨v, hv⟩ := hc.finpartition.nonempty_of_mem_parts hs
  rw [← hc.finpartition.part_eq_of_mem hl hw, ← hc.finpartition.part_eq_of_mem hs hv,
    part_finpartition, part_finpartition]
  exact hbal v w
```

An $(r + 1)$-clique-free complete multipartite graph has at most r parts, since one vertex from each part gives a clique.

```
lemma card_parts_le (hc : G.IsCompleteMultipartite) {r : ℕ}
    (hcf : G.CliqueFree (r + 1)) :
    #hc.finpartition.parts ≤ r := by
  by_contra! l
  obtain ⟨z, -, hz⟩ := hc.finpartition.exists_subset_part_bijOn
  have ncf : ¬G.CliqueFree #z := by
    refine IsNClique.not_cliqueFree ⟨fun v hv w hw hn ↦ ?_, rfl⟩
    contrapose hn
    exact hz.injOn hv hw (by rwa [← hc.not_adj_iff_part_eq])
  rw [Finset.card_eq_of_equiv hz.equiv] at ncf
  exact absurd (hcf.mono (Nat.succ_le_of_lt l)) ncf
```

If G has an r-clique unless it is complete, then it has exactly `min n r` parts, where `n` is the number of vertices.

```
lemma card_parts (hc : G.IsCompleteMultipartite) {r : ℕ} (hcf : G.CliqueFree (r + 1))
    (hsat : ∀ x y, x ≠ y → ¬G.Adj x y → ¬G.CliqueFree r) :
```

```
    #hc.finpartition.parts = min (Fintype.card V) r := by
  refine le_antisymm (le_min hc.finpartition.card_parts_le_card
    (hc.card_parts_le hcf)) ?_
  by_contra! l
  rw [lt_min_iff] at l
  obtain ⟨x, -, y, -, hn, he⟩ := exists_ne_map_eq_of_card_lt_of_maps_to l.1
    fun a _ ↦ hc.finpartition.part_mem.2 (mem_univ a)
  apply hsat x y hn (hc.not_adj_iff_part_eq.2 he)
  intro z hz
  have hinj : Set.InjOn hc.finpartition.part (z : Set V) := fun a ha b hb hab =>
    by_contra fun hne => hc.not_adj_iff_part_eq.2 hab (hz.1 ha hb hne)
  exact l.2.not_ge (hz.card_eq ▸ card_le_card_of_injOn _
    (fun a _ => hc.finpartition.part_mem.2 (mem_univ a)) hinj)
```

Vertices whose parts have different sizes lie in different parts and so are adjacent.

```
omit [DecidableEq V]
lemma adj_of_card_ne (hc : G.IsCompleteMultipartite) {v w : V}
    (hne : #{x | ¬G.Adj v x} ≠ #{x | ¬G.Adj w x}) : G.Adj v w := by
  by_contra hn
  refine hne (congrArg card ?_)
  ext x
  simp_rw [mem_filter, mem_univ, true_and]
  exact ⟨hc.trans _ _ _ fun h => hn h.symm, hc.trans _ _ _ hn⟩
```

A complete multipartite $(r + 1)$-clique-free graph with balanced parts, which has an r-clique unless it is complete, is isomorphic to the Turán graph.

```
theorem nonempty_iso_turanGraph {r : ℕ} (hc : G.IsCompleteMultipartite)
    (hcf : G.CliqueFree (r + 1))
    (hbal : ∀ v w, #{x | ¬G.Adj w x} ≤ #{x | ¬G.Adj v x} + 1)
    (hsat : ∀ x y, x ≠ y → ¬G.Adj x y → ¬G.CliqueFree r) :
    Nonempty (G ≃g T[Fintype.card V, r]) := by
  classical
  -- number the vertices so that the parts are the residue classes
  obtain ⟨zm, zp⟩ := (hc.isEquipartition hbal).exists_partPreservingEquiv
  use (Equiv.subtypeUnivEquiv mem_univ).symm.trans (zm.trans (finCongr card_univ))
  intro a b
  simp_rw [turanGraph_adj, Equiv.trans_apply, Equiv.subtypeUnivEquiv_symm_apply,
    finCongr_apply, Fin.val_cast]
  have := zp ⟨a, mem_univ a⟩ ⟨b, mem_univ b⟩
  rw [← hc.not_adj_iff_part_eq] at this
  rw [← not_iff_not, not_ne_iff, this, hc.card_parts hcf hsat]
  rcases le_or_gt r (Fintype.card V) with c | c
  · rw [min_eq_right c]
  · have lc : ∀ x, zm ⟨x, _⟩ < Fintype.card V := fun x ↦ (zm ⟨x, mem_univ x⟩).2
    rw [min_eq_left c.le, Nat.mod_eq_of_lt (lc a), Nat.mod_eq_of_lt (lc b),
      Nat.mod_eq_of_lt ((lc a).trans c), Nat.mod_eq_of_lt ((lc b).trans c)]

end IsCompleteMultipartite
```

Füredi's Stability Theorem

Turán's theorem says that, up to isomorphism, the Turán graph is the only `(r + 1)`-clique-free graph on `n` vertices with `turanNumber n r` edges. A *stability theorem* says that any `(r + 1)`-clique-free graph with "almost" this many edges is "almost" the Turán graph, where the exact meaning of "almost" can vary.

Füredi [Füredi 2015] gave a short proof of a stability theorem in which being "almost" the Turán graph means becoming `r`-colourable after deleting a few edges. If *G* is `(r + 1)`-clique-free and has `turanNumber n r - t` edges, then deleting at most *t* edges of *G* leaves an `r`-colourable graph. We prove this below in an additive form: there is a set *s* of edges of *G* such that *G*`.deleteEdges s` is `r`-colourable and `e(G) + #s ≤ turanNumber n r`, where `e(G)` is the number of edges of *G*.

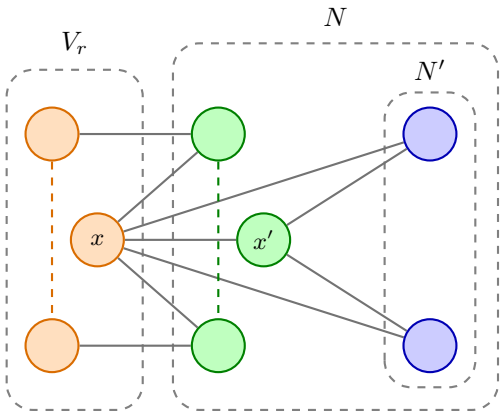
The proof idea goes back to Erdős [Erdős 1970]. Pick a vertex `x` of maximum degree and put all of its non-neighbours (including `x` itself) into one part. Each of them has at most `deg x` neighbours, which is the number of vertices outside their part. Then continue inside the neighbourhood of `x`, which is `r`-clique-free since `x` is adjacent to all of it. Erdős used this to show that *G* has at most as many edges as the complete `r`-partite graph with these parts. Füredi's observation is that the same argument leaves enough room to also count the edges of *G* with both endpoints in the same part.

To make the induction on `r` work we prove a statement about a set `S` of vertices that contains no `(r + 1)`-clique, and record the parts as the colour classes of a function *c* `: V → ℕ` taking values less than `r` on `S`. Degrees are now taken within `S`: for `v ∈ S` the number of neighbours of `v` in `S` is *G*`.degIn S v` (see `degIn`), and `x` is chosen to maximize this. Then the key inequality is:

`∑ v ∈ S, (G.degIn S v + #{w ∈ S | G.Adj v w ∧ c w = c v})` is at most `∑ v ∈ S, #{w ∈ S | c w ≠ c v}`.

The left-hand side is twice the number of edges in `S` plus twice the number of *monochromatic* edges in `S`, those whose endpoints have the same colour. The right-hand side is twice the number of edges of the complete multipartite graph on `S` whose parts are the colour classes. For `S = V`, let *s* be the set of monochromatic edges of *G*, so that *G*`.deleteEdges s` is `r`-colourable. Then the left-hand side is `2 * e(G) + 2 * #s`, and Turán's theorem bounds the right-hand side by `2 * turanNumber n r`. Dividing by 2 gives Füredi's theorem as stated above.

In the induction step we split `S` into `N = S ∩ N(x)`, the neighbours of `x` in `S`, and the set `Vᵣ` of non-neighbours of `x` in `S`. We colour `Vᵣ` with the new colour `r`. Each of the three sums over `S` then splits into four blocks, according to whether `v` and `w` lie in `N` or in `Vᵣ`.



`Vᵣ` (`x` and its non-neighbours) gets colour `r` (orange). In `N`, `x'` and its non-neighbours get `r - 1` (green) and `N'` is coloured recursively (blue). Dashed edges are monochromatic

```
lemma _root_.Finset.sum_card_filter_union [DecidableEq V] {s t : Finset V}
    (h : Disjoint s t) (p : V → V → Prop) [DecidableRel p] :
    ∑ v ∈ s ∪ t, #{w ∈ s ∪ t | p v w} =
      ∑ v ∈ s, #{w ∈ s | p v w} + ∑ v ∈ s, #{w ∈ t | p v w} +
        ∑ v ∈ t, #{w ∈ s | p v w} + ∑ v ∈ t, #{w ∈ t | p v w} := by
  simp [sum_union h, filter_union, card_union_of_disjoint (disjoint_filter_filter h),
        sum_add_distrib, add_assoc]
```

The edges between N and V_r are counted once from each side, and the double-counting lemma sum_card_bipartiteAbove_eq_sum_card_bipartiteBelow says that the two counts agree.

```
lemma sum_degIn_comm [DecidableRel G.Adj] (s t : Finset V) :
    ∑ v ∈ s, G.degIn t v = ∑ v ∈ t, G.degIn s v := by
  simpa [degIn, bipartiteAbove, bipartiteBelow, adj_comm] using
    sum_card_bipartiteAbove_eq_sum_card_bipartiteBelow (s := s) (t := t) G.Adj
```

The induction step only uses that V_r is a new colour class whose vertices have at most #N neighbours in S, which holds because x has maximum degree. We state it for any disjoint N and V_r and a colouring c that is less than r on N and equal to r on V_r.

```
private lemma furedi_step [DecidableRel G.Adj] [DecidableEq V] {N V_r : Finset V}
    {c : V → ℕ} {r : ℕ} (hNV : Disjoint N V_r) (hN : ∀ v ∈ N, c v < r)
    (hV : ∀ v ∈ V_r, c v = r)
    (hdeg : ∀ v ∈ V_r, G.degIn (N ∪ V_r) v ≤ #N)
    (hc : ∑ v ∈ N, (G.degIn N v + #{w ∈ N | G.Adj v w ∧ c w = c v}) ≤
      ∑ v ∈ N, #{w ∈ N | c w ≠ c v}) :
    ∑ v ∈ N ∪ V_r, (G.degIn (N ∪ V_r) v + #{w ∈ N ∪ V_r | G.Adj v w ∧ c w = c v}) ≤
      ∑ v ∈ N ∪ V_r, #{w ∈ N ∪ V_r | c w ≠ c v} := by
  -- colours on N differ from those on V_r, which all equal r
  have hne : ∀ v ∈ N, ∀ w ∈ V_r, c v ≠ c w := fun v hv w hw => by
    rw [hV w hw]; exact (hN v hv).ne
  -- monochromatic edges: none between N and V_r, and every edge inside V_r
  have m1 : ∑ v ∈ N, #{w ∈ V_r | G.Adj v w ∧ c w = c v} = 0 :=
    sum_eq_zero fun v hv => card_eq_zero.2 ◁ filter_false_of_mem fun w hw h =>
      hne v hv w hw h.2.symm
  have m2 : ∑ v ∈ V_r, #{w ∈ N | G.Adj v w ∧ c w = c v} = 0 :=
    sum_eq_zero fun v hv => card_eq_zero.2 ◁ filter_false_of_mem fun w hw h =>
      hne w hw v hv h.2
  have m3 : ∑ v ∈ V_r, #{w ∈ V_r | G.Adj v w ∧ c w = c v} = ∑ v ∈ V_r, G.degIn V_r v :=
    sum_congr rfl fun v hv => congrArg card ◁ filter_congr fun w hw => by
      simp [hV v hv, hV w hw]
  -- pairs of different colours: every pair between N and V_r, none inside V_r
  have d1 : ∑ v ∈ N, #{w ∈ V_r | c w ≠ c v} = #N * #V_r := by
    rw [sum_const_nat fun v hv => congrArg card ◁ filter_true_of_mem fun w hw =>
      (hne v hv w hw).symm]
  have d2 : ∑ v ∈ V_r, #{w ∈ N | c w ≠ c v} = #V_r * #N := by
    rw [sum_const_nat fun v hv => congrArg card ◁ filter_true_of_mem fun w hw =>
      hne w hw v hv]
  have d3 : ∑ v ∈ V_r, #{w ∈ V_r | c w ≠ c v} = 0 :=
    sum_eq_zero fun v hv => card_eq_zero.2 ◁ filter_false_of_mem fun w hw h =>
      h (by rw [hV v hv, hV w hw])
  -- the vertices of V_r have at most #N neighbours in N ∪ V_r
  have hD : ∑ v ∈ V_r, G.degIn N v + ∑ v ∈ V_r, G.degIn V_r v ≤ #V_r * #N := by
    rw [← sum_add_distrib, ← smul_eq_mul, ← sum_const]
```

```
    exact sum_le_sum fun v hv => (degIn_union _ _ _ hNV).symm.le.trans (hdeg v hv)
  have e1 := sum_card_filter_union hNV G.Adj
  have e2 := sum_card_filter_union hNV (fun v w => G.Adj v w ∧ c w = c v)
  have e3 := sum_card_filter_union hNV (fun v w => c w ≠ c v)
  have hdc := sum_degIn_comm (G := G) N Vᵣ
  simp_rw [degIn_eq_card_filter, sum_add_distrib] at *
  grind
```

The local form of Füredi's inequality, proved by induction on r. A set S with no $(r+1)$-clique has a colouring c, taking values less than r on S, with the following property. Count the edges of G in S, counting the monochromatic ones twice: the total is at most the number of edges of the complete multipartite graph whose parts are the colour classes.

```
private lemma furedi_aux [DecidableRel G.Adj] (r : ℕ) :
    ∀ S : Finset V, (∀ t ⊆ S, ¬G.IsNClique (r + 1) t) →
      ∃ c : V → ℕ, (∀ v ∈ S, c v < r) ∧
        ∑ v ∈ S, (G.degIn S v + #{w ∈ S | G.Adj v w ∧ c w = c v}) ≤
          ∑ v ∈ S, #{w ∈ S | c w ≠ c v} := by
  classical
  induction r with
  | zero =>
    -- a set with no 1-clique is empty
    intro S hS
    obtain rfl : S = ∅ :=
      eq_empty_of_forall_notMem fun x hx => hS {x} (by simpa) (by simp)
    exact ⟨fun _ => 0, by simp, by simp⟩
  | succ r ih =>
    intro S hS
    rcases S.eq_empty_or_nonempty with rfl | hne
    · exact ⟨fun _ => 0, by simp, by simp⟩
    -- x has maximum degree into S; split S into N = S ∩ N(x) and the rest
    obtain ⟨x, hxS, hx⟩ := exists_max_image S (G.degIn S) hne
    set N := {w ∈ S | G.Adj x w}
    set Vᵣ := {w ∈ S | ¬G.Adj x w}
    have hdisj : Disjoint N Vᵣ := disjoint_filter_filter_not _ _ _
    have huni : N ∪ Vᵣ = S := filter_union_filter_not_eq _ _
    -- N has no (r + 1)-clique, since x would extend it to an (r + 2)-clique
    have hN : ∀ t ⊆ N, ¬G.IsNClique (r + 1) t := fun t ht hc =>
      hS (insert x t) (insert_subset hxS (ht.trans (filter_subset _ _)))
        (hc.insert fun w hw => (mem_filter.1 (ht hw)).2)
    obtain ⟨c', hc'r, hc'⟩ := ih N hN
    -- colour N by c' and Vᵣ by the new colour r
    let c : V → ℕ := N.piecewise c' fun _ => r
    have hcN : ∀ v ∈ N, c v = c' v := fun v hv => piecewise_eq_of_mem _ _ _ hv
    have hcr : ∀ v ∈ N, c v < r := fun v hv => hcN v hv ▸ hc'r v hv
    have hcV : ∀ v ∈ Vᵣ, c v = r := fun v hv =>
      piecewise_eq_of_notMem _ _ _ (disjoint_right.1 hdisj hv)
    refine ⟨c, fun v hv => ?_, ?_⟩
    · rcases mem_union.1 (huni ▸ hv) with h | h
      · exact (hcr v h).trans r.lt_succ_self
      · exact (hcV v h).trans_lt r.lt_succ_self
    rw [← huni]
    refine furedi_step hdisj hcr hcV (fun v hv => huni ▸ hx v (mem_filter.1 hv).1) ?_
    -- on N the colouring c agrees with c'
```

```
    have h1 : ∀ v ∈ N, #{w ∈ N | G.Adj v w ∧ c w = c v} =
        #{w ∈ N | G.Adj v w ∧ c' w = c' v} := fun v hv =>
      congrArg card (filter_congr fun w hw => by rw [hcN v hv, hcN w hw])
    have h2 : ∀ v ∈ N, #{w ∈ N | c w ≠ c v} = #{w ∈ N | c' w ≠ c' v} := fun v hv =>
      congrArg card (filter_congr fun w hw => by rw [hcN v hv, hcN w hw])
    calc _ = _ := sum_congr rfl fun v hv => by rw [h1 v hv]
      _ ≤ _ := hc'
      _ = _ := (sum_congr rfl h2).symm
```

In the proof below the complete r-partite graph is `K`, in which two vertices are adjacent iff they have different colours. Deleting the monochromatic edges of G leaves $G \sqcap$ `K`.

Füredi's stability theorem: an $(r + 1)$-clique-free graph G on `n` vertices can be made r-colourable by deleting at most `turanNumber n r - e`(G) edges.

```
theorem CliqueFree.exists_colorable_deleteEdges [Fintype V] [DecidableRel G.Adj]
    {r : ℕ} (h : G.CliqueFree (r + 1)) :
    ∃ s ⊆ G.edgeFinset, (G.deleteEdges s).Colorable r ∧
      #G.edgeFinset + #s ≤ turanNumber (Fintype.card V) r := by
  classical
  obtain ⟨c, hcr, hc⟩ := furedi_aux (G := G) r univ fun t _ ht => h t ht
  -- the complete multipartite graph whose parts are the colour classes of c
  let K : SimpleGraph V := (⊤ : SimpleGraph ℕ).comap c
  have hK : ∀ u v, K.Adj u v ↔ c u ≠ c v := fun u v => by simp [K]
  have hKcol : K.Colorable r :=
    ⟨Coloring.mk (fun v => (⟨c v, hcr v (mem_univ v)⟩ : Fin r)) fun {u v} huv => by
      simpa [Fin.ext_iff] using (hK u v).1 huv⟩
  have hle : G ⊓ K ≤ G := inf_le_left
  -- delete the edges of G that are not edges of G ⊓ K
  refine ⟨G.edgeFinset \ (G ⊓ K).edgeFinset, sdiff_subset, ?_, ?_⟩
  · rw [coe_sdiff, coe_edgeFinset, coe_edgeFinset, deleteEdges_sdiff_eq_of_le hle]
    exact hKcol.mono_left inf_le_right
  have hdeg (H : SimpleGraph V) [DecidableRel H.Adj] (v : V) :
      H.degree v = #{w | H.Adj v w} := by
    rw [← card_neighborFinset_eq_degree, neighborFinset_eq_filter]
  have dK : ∀ v, K.degree v = #{w | c w ≠ c v} := fun v => by
    simp [hdeg, hK, eq_comm]
  -- deg_G v splits into the edges of G ⊓ K at v and the monochromatic ones
  have dG : ∀ v, G.degree v = (G ⊓ K).degree v + #{w | G.Adj v w ∧ c w = c v} :=
    fun v => by
      rw [hdeg, hdeg]
      simpa [inf_adj, hK, filter_filter, eq_comm] using
        (card_filter_add_card_filter_not (s := {w | G.Adj v w})
          (fun w => c w ≠ c v)).symm
  have e1 :
      ∑ v, G.degree v + ∑ v, #{w | G.Adj v w ∧ c w = c v} ≤ ∑ v, K.degree v := by
    simp_rw [← sum_add_distrib, dK, hdeg G]
    exact hc
  have e2 : ∑ v, G.degree v =
      ∑ v, (G ⊓ K).degree v + ∑ v, #{w | G.Adj v w ∧ c w = c v} := by
    rw [← sum_add_distrib]
    exact sum_congr rfl fun v _ => dG v
  have := (hKcol.cliqueFree (Nat.lt_succ_self r)).card_edgeFinset_le
  have := sum_degrees_eq_twice_card_edges K
```

```
  have := sum_degrees_eq_twice_card_edges G
  -- our primed version accepts the Fintype (G ⊓ K).edgeSet instance found here
  have := sum_degrees_eq_twice_card_edges' (G ⊓ K)
  have := card_sdiff_of_subset (edgeFinset_mono hle)
  grind
```

Dropping `#s`, we recover the edge bound of Turán's theorem. This is not an independent proof, since Füredi's theorem used that bound for the graph `K`.

```
example [Fintype V] [DecidableRel G.Adj] {r : ℕ} (h : G.CliqueFree (r + 1)) :
    #G.edgeFinset ≤ turanNumber (Fintype.card V) r := by
  obtain ⟨s, -, -, hs⟩ := h.exists_colorable_deleteEdges
  grind
```

For `r = 2` this is a stability version of Mantel's theorem: a triangle-free graph on `n` vertices with at least `n ^ 2 / 4 - t` edges can be made bipartite by deleting at most `t` edges.

```
example [Fintype V] [DecidableRel G.Adj] {t : ℕ} (h : G.CliqueFree 3)
    (he : Fintype.card V ^ 2 / 4 - t ≤ #G.edgeFinset) :
    ∃ s ⊆ G.edgeFinset, (G.deleteEdges s).IsBipartite ∧ #s ≤ t := by
  obtain ⟨s, hs, hcol, hle⟩ := h.exists_colorable_deleteEdges (r := 2)
  rw [turanNumber_two] at hle
  exact ⟨s, hs, hcol, by grind⟩
```

Mathlib has a name for the opposite situation: `G.DeleteFar p t` says that at least `t` edges *must* be deleted from `G` to get a graph with property `p`. The threshold `t` lives in an ordered ring rather than `ℕ`, so that it can be `ε * n ^ 2` with `ε` real, as in `FarFromTriangleFree` below.

```
#source DeleteFar
```

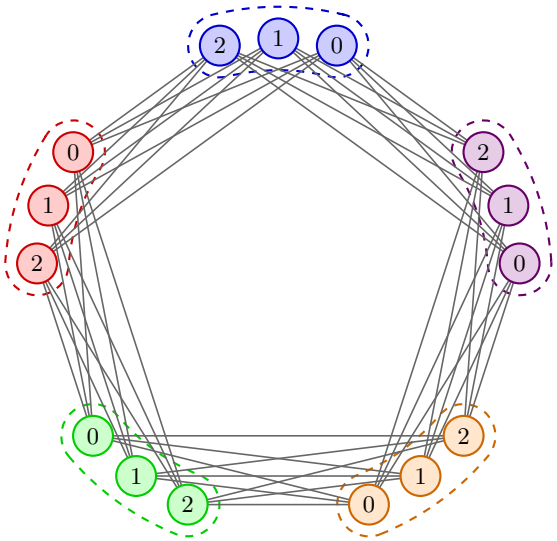
In these terms, Füredi's theorem says that if at least `t` edges must be deleted to make an `(r + 1)`-clique-free graph `r`-colourable, then it has at most `turanNumber n r - t` edges.

```
example [Fintype V] [DecidableRel G.Adj] {r t : ℕ} (h : G.CliqueFree (r + 1))
    (hfar : G.DeleteFar (fun H => H.Colorable r) (t : ℤ)) :
    #G.edgeFinset ≤ turanNumber (Fintype.card V) r - t := by
  obtain ⟨s, hs, hcol, hle⟩ := h.exists_colorable_deleteEdges
  have := hfar hs hcol
  grind
```

The Andrásfai–Erdős–Sós Theorem

Turán's theorem says that the densest `(r + 1)`-clique-free graphs are Turán graphs, while Füredi's stability theorem tells us that graphs that are almost extremal, in terms of number of edges, are almost `r`-colourable.

The Andrásfai–Erdős–Sós theorem [Andrásfai–Erdős–Sós 1974] gives a different type of stability result related to the minimum degree: any `(r + 1)`-clique-free graph with minimum degree greater than `(3 * r - 4) * n / (3 * r - 1)` must be `r`-colourable. For `r = 2` this says that a triangle-free graph with minimum degree greater than `2 * n / 5` is bipartite. The uniform blow-up `C[5][t]` shows that `2 * n / 5` cannot be lowered (Exercise 12).



The 3-uniform blow-up `C[5][3]`: a non-bipartite triangle-free graph with minimum degree `2 * n / 5`

```
#check colorable_of_cliqueFree_lt_minDegree
```

Put the other way round: a triangle-free graph that contains an odd closed walk, and so is not bipartite (`isBipartite_iff_forall_walk_even_length`), has minimum degree at most `2 * n / 5`.

```
example [Fintype V] [DecidableRel G.Adj] (h3 : G.CliqueFree 3) {u : V}
    (w : G.Walk u u) (hw : Odd w.length) : δ(G) ≤ 2 * Fintype.card V / 5 := by
  by_contra! h
  have hb : G.IsBipartite := colorable_of_cliqueFree_lt_minDegree h3 h
  exact Nat.not_even_iff_odd.2 hw (isBipartite_iff_forall_walk_even_length.1 hb u w)
```

Extremal Numbers

Turán's theorem answers the extremal question for complete graphs. For a general forbidden graph `H`, the *extremal number* `extremalNumber n H` is the maximum number of edges in an `H`-free graph on `n` vertices. Recall that `H.Free G` means that `G` contains no copy of `H`. Mathlib defines `extremalNumber` as a supremum over all graphs on `Fin n`.

```
#source extremalNumber -- sup { G : SimpleGraph (Fin n) | H.Free G } (#·.edgeFinset)
```

Although this is a supremum over a finite set, `extremalNumber` is **`noncomputable`**. The definition opens `Classical` and uses it twice: to count the edges of each graph `G : SimpleGraph (Fin n)` in the supremum, and to decide whether `G` is `H`-free. Instance arguments cannot help with the first: `G` is bound by the supremum rather than being an argument of `extremalNumber`, so there is nowhere to supply `DecidableRel G.Adj`. A computable version could instead maximize over edge sets `s : Finset (Sym2 (Fin n))`, since Lean can decide adjacency in the graph `fromEdgeSet s` with these edges. For the second, Mathlib has no `Decidable (H ⊑ G)` instance, but one can be written by searching all maps `W → Fin n` for an injective homomorphism, given `Fintype W`, `DecidableEq W` and `DecidableRel H.Adj`. Such a version would be extremely slow, checking up to `2 ^ n.choose 2 * n ^ |W|` pairs of an edge set and a map.

The key facts about it are the following results.

```
#check card_edgeFinset_le_extremalNumber
#check IsContained.of_extremalNumber_lt_card_edgeFinset
#check extremalNumber_le_iff
```

Turán's theorem, restated in this language, computes the extremal numbers of complete graphs.

```
#check extremalNumber_top
#check card_edgeFinset_eq_extremalNumber_top_iff_nonempty_iso_turanGraph
```

For bipartite forbidden graphs the extremal numbers are much smaller. The *Zarankiewicz number* [Zarankiewicz 1951] (`zarankiewicz`) is the maximum number of edges in a `K[s, t]`-free bipartite graph with parts of sizes `m` and `n`.

```
#source zarankiewicz
#check zarankiewicz_le_extremalNumber
#check two_mul_extremalNumber_le_zarankiewicz_symm
```

The main upper bound for these numbers, the *Kővári–Sós–Turán theorem* [Kővári–Sós–Turán 1954], says that `extremalNumber n K[s, t] = O(n ^ (2 - 1 / s))`. This is not yet in Mathlib.

Turán Density and the Erdős–Stone Theorem

How does `extremalNumber n H` grow with `n`? Dividing by the number of possible edges `n.choose 2` gives a sequence that is decreasing and so converges. Its limit is the *Turán density* (`turanDensity`).

Statements about limits in Mathlib use filters: `Tendsto f atTop (𝓝 a)` says that `f n → a` as `n → ∞`.

```
section asymptotics
open Filter Asymptotics

#source turanDensity
#check tendsto_turanDensity
```

When the Turán density is nonzero, `extremalNumber n H` is asymptotically `turanDensity H * n.choose 2`, which Mathlib writes with `~[atTop]` (asymptotic equivalence).

```
#check isEquivalent_extremalNumber -- ex (n, H) ~ H.turanDensity * n.choose 2
```

Phrases like "for all sufficiently large `n`" are written with `∀ᶠ (n : ℕ) in atTop`. For example, for any `ε > 0`, every large enough graph with at least `(H.turanDensity + ε) * n.choose 2` edges contains a copy of `H`.

```
#check eventually_isContained_of_card_edgeFinset
```

The *Erdős–Stone theorem* [Erdős–Stone 1946] says that a large graph whose density is a little above `1 - 1 / r`, the Turán density of `K[r + 1]` (by Turán's theorem), contains not just `K[r + 1]` but a complete `(r + 1)`-partite graph with large parts: `completeEquipartiteGraph (r + 1) t`.

Mathlib has the minimum degree version of this theorem.

```
#check eventually_completeEquipartiteGraph_isContained_of_minDegree
```

```
end asymptotics
```

The Erdős–Stone–Simonovits formula [Erdős–Simonovits 1966], which follows from the Erdős–Stone theorem, is not yet in Mathlib: if `χ H = r + 2` then `turanDensity H = 1 - 1 / (r + 1)`. (Writing `r + 2` excludes an `H` with no edges: it is contained in every large graph, so its Turán density is `0`, while `1 - 1 / 0 = 1` in Lean.)

13.3 Zykov's Theorem*

Turán's theorem bounds the number of edges of an `(r + 1)`-clique-free graph. An edge is just a `2`-clique, and Zykov [Zykov 1949] proved the following generalization: any `(r + 1)`-clique-free graph on `n` vertices has at most as many `s`-cliques as the Turán graph `T[n, r]`. Moreover, except for the degenerate cases of `s < 2`, `s > r` or `s > n`, the Turán graph is (up to isomorphism) the unique maximizer.

For example, `T[7, 3]` is `4`-clique-free and its parts have sizes `3`, `2` and `2`, so it has `3 * 2 * 2 = 12` triangles. Zykov's theorem says that no `4`-clique-free graph on `7` vertices has more triangles than this, and any `4`-clique-free graph on `7` vertices with this many triangles is isomorphic to `T[7, 3]`.

```
#eval #(T[7, 3].cliqueFinset 3) -- 12
```

Nothing later in the chapter depends on the proof below, so a reader who only wants the statement can go straight to Regularity and Removal (p. 438) or Infinite Graphs (p. 443).

The proof follows the same basic strategy as Mathlib's proof of Turán's theorem (by Zykov symmetrization), but we have to work a bit harder.

Fix `s ≥ 2` and let `G` have the most `s`-cliques among the `(r + 1)`-clique-free graphs on `V`. We show in turn that `G` is complete multipartite, that its parts are balanced, and that it has an `r`-clique unless it is complete. Then, as we proved above, such a graph is isomorphic to `T[n, r]` (`IsCompleteMultipartite.nonempty_iso_turanGraph`). For edges, Mathlib settles each of the first two steps by comparing degrees; for cliques each needs a new idea.

We first make "the most `s`-cliques" precise. Mathlib defines `IsTuranMaximal r` as `IsExtremal (CliqueFree · (r + 1))`, where `IsExtremal p` says that a graph has property `p` and the most edges among the graphs on its vertex type with property `p`. To count `s`-cliques instead of edges, we generalize `IsExtremal` from edges to copies of any graph `H`, counted by `copyCount`. By `copyCount_top` the copies of `K[s]` are the `s`-cliques, and an edge is a `2`-clique.

`G` is *extremal for copies of* `H` among the graphs with property `p` if it has property `p` and no graph on the same vertex set with property `p` has more copies of `H`.

```
def IsCopyExtremal [Fintype V] {W : Type*} (G : SimpleGraph V) (H : SimpleGraph W)
    (p : SimpleGraph V → Prop) : Prop :=
  p G ∧ ∀ ⦃G' : SimpleGraph V⦄, p G' → G'.copyCount H ≤ G.copyCount H
```

Call `G` *`s`-clique-maximal* for `r` if it is `(r + 1)`-clique-free and no `(r + 1)`-clique-free graph on the same vertex set has more `s`-cliques. So `IsNCliqueMaximal 2 r` is equivalent to `IsTuranMaximal r`.

```
def IsNCliqueMaximal [Fintype V] (G : SimpleGraph V) (s r : ℕ) : Prop :=
  G.IsCopyExtremal K[s] (CliqueFree · (r + 1))
```

In the proofs we count s-cliques directly, via `cliqueFinset`, using the following reformulation.

```
lemma isNCliqueMaximal_iff [Fintype V] [DecidableEq V] [DecidableRel G.Adj]
    {s r : ℕ} : G.IsNCliqueMaximal s r ↔ G.CliqueFree (r + 1) ∧
      ∀ {H : SimpleGraph V} [DecidableRel H.Adj], H.CliqueFree (r + 1) →
        #(H.cliqueFinset s) ≤ #(G.cliqueFinset s) := by
  refine and_congr_right fun _ => ⟨fun h H _ hH => ?_, fun h H hH => ?_⟩
  · simpa only [copyCount_top, Fintype.card_fin] using h hH
  · classical
    simpa only [copyCount_top, Fintype.card_fin] using h hH
```

An s-clique-maximal graph has at least as many s-cliques as any `(r + 1)`-clique-free graph on the same vertex set.

```
lemma IsNCliqueMaximal.card_cliqueFinset_le [Fintype V] [DecidableEq V]
  [DecidableRel G.Adj] {s r : ℕ} (h : G.IsNCliqueMaximal s r) {H : SimpleGraph V}
  [DecidableRel H.Adj] (hH : H.CliqueFree (r + 1)) :
    #(H.cliqueFinset s) ≤ #(G.cliqueFinset s) :=
  (isNCliqueMaximal_iff.1 h).2 hH
```

Symmetrizing Cliques

Write $\mathtt{d_s(v)}$ for the number of s-cliques containing `v`. First we need the analogue of `card_edgeFinset_replaceVertex_of_not_adj` for cliques.

The s-cliques of `G.replaceVertex a b` that do not contain b are the s-cliques of G that do not contain b.

```
lemma isNClique_replaceVertex_iff_of_notMem [DecidableEq V] {a b : V} {s : ℕ}
    {t : Finset V} (hb : b ∉ t) :
    (G.replaceVertex a b).IsNClique s t ↔ G.IsNClique s t := by
  have hadj {x y} (hx : x ∈ t) (hy : y ∈ t) :
      (G.replaceVertex a b).Adj x y ↔ G.Adj x y :=
    adj_replaceVertex_iff_of_ne _ _ (ne_of_mem_of_not_mem hx hb)
      (ne_of_mem_of_not_mem hy hb)
  simp_all [isNClique_iff, isClique_iff, Set.Pairwise]
```

The s-cliques of `G.replaceVertex a b` containing b correspond to the s-cliques of G that contain a but not b.

```
lemma card_filter_mem_cliqueFinset_replaceVertex [Fintype V] [DecidableEq V]
    [DecidableRel G.Adj] {a b : V} (hab : a ≠ b) (s : ℕ) :
    #{t ∈ (G.replaceVertex a b).cliqueFinset s | b ∈ t} =
      #{t ∈ G.cliqueFinset s | a ∈ t ∧ b ∉ t} := by
  refine card_nbij' (fun t => insert a (t.erase b)) (fun t => insert b (t.erase a))
    (fun t ht => ?_) (fun t ht => ?_) (fun t ht => ?_) (fun t ht => ?_) <;>
    simp_rw [coe_filter, Set.mem_ofPred_eq, mem_cliqueFinset_iff] at ht ⊢
  · -- swapping b for a maps into the target
    have hb : b ∉ insert a (t.erase b) := by simp [hab.symm]
    refine ⟨(isNClique_replaceVertex_iff_of_notMem hb).1
      (ht.1.insert_erase (fun w hw => ?_) ht.2), mem_insert_self _ _, hb⟩
    rw [mem_sdiff, mem_singleton] at hw
    -- in G.replaceVertex a b, a and b have the same neighbours other than b
    grind [adj_replaceVertex_iff_of_ne_left, adj_replaceVertex_iff_of_ne_right,
      ht.1.1 ht.2 hw.1 (Ne.symm hw.2)]
```

```
  · -- swapping a for b maps back
    refine ⟨((isNClique_replaceVertex_iff_of_notMem ht.2.2).2 ht.1).insert_erase
      (fun w hw => ?_) ht.2.1, mem_insert_self _ _⟩
    grind [adj_replaceVertex_iff_of_ne_right, ht.1.1 ht.2.1]
  · -- left inverse
    have hat : a ∉ t := fun hat =>
      not_adj_replaceVertex_same G a b (ht.1.1 hat ht.2 hab)
    grind
  · -- right inverse
    grind
```

The number of s-cliques in `G.replaceVertex a b` expressed in terms of the number of s-cliques in G (avoiding $\mathbb{N}$-subtraction).

```
lemma card_cliqueFinset_replaceVertex [Fintype V] [DecidableEq V]
    [DecidableRel G.Adj] {a b : V} (hab : a ≠ b) (s : ℕ) :
    #((G.replaceVertex a b).cliqueFinset s) + #{t ∈ G.cliqueFinset s | b ∈ t} =
      #(G.cliqueFinset s) + #{t ∈ G.cliqueFinset s | a ∈ t ∧ b ∉ t} := by
  have h1 : {t ∈ (G.replaceVertex a b).cliqueFinset s | b ∉ t} =
      {t ∈ G.cliqueFinset s | b ∉ t} := by
    ext t; simpa using and_congr_left isNClique_replaceVertex_iff_of_notMem
  have e1 := card_filter_add_card_filter_not
    (s := (G.replaceVertex a b).cliqueFinset s) (b ∈ ·)
  have e2 := card_filter_add_card_filter_not (s := G.cliqueFinset s) (b ∈ ·)
  grind [card_filter_mem_cliqueFinset_replaceVertex hab]
```

If a and b are not adjacent then no clique contains both, so `G.replaceVertex a b` loses the s-cliques containing b and gains one s-clique for each s-clique containing a.

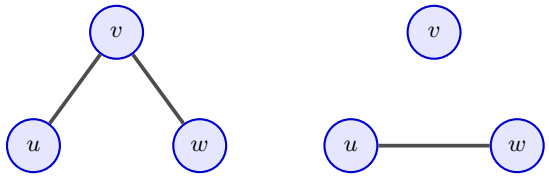
```
lemma card_cliqueFinset_replaceVertex_of_not_adj [Fintype V] [DecidableEq V]
    [DecidableRel G.Adj] {a b : V} (hn : ¬G.Adj a b) (s : ℕ) :
    #((G.replaceVertex a b).cliqueFinset s) + #{t ∈ G.cliqueFinset s | b ∈ t} =
      #(G.cliqueFinset s) + #{t ∈ G.cliqueFinset s | a ∈ t} := by
  rcases eq_or_ne a b with rfl | hab
  · simp
  rw [card_cliqueFinset_replaceVertex hab]
  congr 2
  refine filter_congr fun t ht => ⟨fun h => h.1, fun hat => ⟨hat, fun hbt => ?_⟩⟩
  exact hn ((mem_cliqueFinset_iff.1 ht).1 hat hbt hab)
```

As in Mathlib's proof of Turán's theorem, performing Zykov symmetrization on a pair of non-adjacent vertices in an s-clique-maximal graph shows that they lie in the same number of s-cliques.

```
lemma IsNCliqueMaximal.card_filter_mem_eq_of_not_adj [Fintype V] [DecidableEq V]
    [DecidableRel G.Adj] {s r : ℕ} (h : G.IsNCliqueMaximal s r)
    {a b : V} (hn : ¬G.Adj a b) :
    #{t ∈ G.cliqueFinset s | a ∈ t} = #{t ∈ G.cliqueFinset s | b ∈ t} := by
  have kab := card_cliqueFinset_replaceVertex_of_not_adj hn s
  have kba := card_cliqueFinset_replaceVertex_of_not_adj (fun h' => hn h'.symm) s
  have := h.card_cliqueFinset_le (h.1.replaceVertex a b)
  have := h.card_cliqueFinset_le (h.1.replaceVertex b a)
  grind
```

Complete Multipartite Maximizers

Now comes the first real difference from the proof for edges (Turán's theorem). Suppose that u and w are adjacent, while v is adjacent to neither. Mathlib calls this `IsPathGraph3Compl`, since u, v and w induce the complement of a path on three vertices, and a graph is complete multipartite exactly when it contains no such triple (`not_isCompleteMultipartite_iff_exists_isPathGraph3Compl`).



Left: `P[3]`. Right: its complement `P[3]ᶜ`.

Symmetrizing twice, replacing u and w by copies of v, changes the number of s-cliques by `dₛ(v) - dₛ(u) + dₛ(v) - dₛ(w) + dₛ(u, w)`, where `dₛ(u, w)` counts the s-cliques containing both u and w (`card_cliqueFinset_replaceVertex_replaceVertex` below). Maximality then forces `dₛ(u) = dₛ(v) = dₛ(w)` (`IsNCliqueMaximal.card_filter_mem_eq_of_not_adj`), but `dₛ(u, w)` can be `0`, so there is no contradiction. Indeed an s-clique-maximal graph need not be complete multipartite: for `s > r` every `(r + 1)`-clique-free graph has no s-cliques, so every one of them is s-clique-maximal. (For edges, `s = 2` and `dₛ(u, w) = 1`, so any 2-clique-maximal graph is complete multipartite, by `IsTuranMaximal.not_adj_trans`.)

The number of s-cliques in `(G.replaceVertex v u).replaceVertex v w` in terms of the number of s-cliques in G, when the only edge induced by $\{u, v, w\}$ is $u - w$.

```
lemma card_cliqueFinset_replaceVertex_replaceVertex [Fintype V] [DecidableEq V]
    [DecidableRel G.Adj] {u v w : V} (h : G.IsPathGraph3Compl v u w) (s : ℕ) :
    #(((G.replaceVertex v u).replaceVertex v w).cliqueFinset s) +
        #{t ∈ G.cliqueFinset s | u ∈ t} + #{t ∈ G.cliqueFinset s | w ∈ t} =
      #(G.cliqueFinset s) + 2 * #{t ∈ G.cliqueFinset s | v ∈ t} +
        #{t ∈ G.cliqueFinset s | u ∈ t ∧ w ∈ t} := by
  have nvu : v ≠ u := h.ne_fst
  have nuw : u ≠ w := h.fst_ne_snd
  obtain ⟨huw, hvu, hvw⟩ := h
  have hn₁ : ¬(G.replaceVertex v u).Adj v w :=
    (G.adj_replaceVertex_iff_of_ne v nvu nuw.symm).not.mpr hvw
  have hv : {t ∈ (G.replaceVertex v u).cliqueFinset s | v ∈ t} =
      {t ∈ G.cliqueFinset s | v ∈ t} := by
    ext t
    simp only [mem_filter, mem_cliqueFinset_iff, and_congr_left_iff]
    intro hvt
    by_cases hut : u ∈ t
    · exact iff_of_false
        (fun ht => G.not_adj_replaceVertex_same v u (ht.1 hvt hut nvu))
        fun ht => hvu (ht.1 hvt hut nvu)
    · exact isNClique_replaceVertex_iff_of_notMem hut
  have hw : {t ∈ (G.replaceVertex v u).cliqueFinset s | w ∈ t} =
      {t ∈ G.cliqueFinset s | w ∈ t ∧ u ∉ t} := by
    ext t
    simp only [mem_filter, mem_cliqueFinset_iff]
    constructor
    · rintro ⟨ht, hwt⟩
      have hut : u ∉ t := fun hut => hvw ◁
```

```
      (adj_replaceVertex_iff_of_ne_right G v nuw.symm).1 (ht.1 hut hwt nuw)
    exact ⟨(isNClique_replaceVertex_iff_of_notMem hut).1 ht, hwt, hut⟩
  · rintro ⟨ht, hwt, hut⟩
    exact ⟨(isNClique_replaceVertex_iff_of_notMem hut).2 ht, hwt⟩
have huw' : #{t ∈ G.cliqueFinset s | u ∈ t ∧ w ∈ t} +
    #{t ∈ G.cliqueFinset s | w ∈ t ∧ u ∉ t} = #{t ∈ G.cliqueFinset s | w ∈ t} := by
  rw [← card_filter_add_card_filter_not (s := {t ∈ G.cliqueFinset s | w ∈ t})
    (u ∈ ·), filter_filter, filter_filter]
  simp_rw [and_comm]
have k₁ := card_cliqueFinset_replaceVertex_of_not_adj hvu s
have k₂ := card_cliqueFinset_replaceVertex_of_not_adj hn₁ s
grind
```

The problem is caused by edges that lie in no s-clique. Deleting all such edges does not change the number of s-cliques, and will yield a subgraph that we *can* prove is complete multipartite.

We do this by forming the graph `G.cliqueUnion s`, which is the union of all the s-cliques of G.

The union of the s-cliques of G: distinct vertices are adjacent iff they lie in a common s-clique of G.

```
@[simps]
def cliqueUnion (G : SimpleGraph V) (s : ℕ) : SimpleGraph V where
  Adj x y := x ≠ y ∧ ∃ t, G.IsNClique s t ∧ x ∈ t ∧ y ∈ t

instance [Fintype V] [DecidableEq V] [DecidableRel G.Adj] {s : ℕ} :
    DecidableRel (G.cliqueUnion s).Adj := fun x y =>
  decidable_of_iff (x ≠ y ∧ ∃ t ∈ G.cliqueFinset s, x ∈ t ∧ y ∈ t) (by simp)
```

The union of the s-cliques of G is a subgraph of G.

```
lemma cliqueUnion_le (s : ℕ) : G.cliqueUnion s ≤ G :=
  fun _ _ ⟨hxy, _, ht, hx, hy⟩ => ht.1 hx hy hxy
```

The s-cliques of `G.cliqueUnion s` are the same as those of G.

```
@[simp]
lemma isNClique_cliqueUnion_iff {s : ℕ} {t : Finset V} :
    (G.cliqueUnion s).IsNClique s t ↔ G.IsNClique s t :=
  ⟨fun ht => ht.mono (G.cliqueUnion_le s),
    fun ht => ⟨fun _ hx _ hy hxy => ⟨hxy, t, ht, hx, hy⟩, ht.2⟩⟩
```

The finite set of s-cliques of `G.cliqueUnion s` is the same as the finite set of s-cliques of G.

```
@[simp]
lemma cliqueFinset_cliqueUnion [Fintype V] [DecidableEq V] [DecidableRel G.Adj]
    (s : ℕ) : (G.cliqueUnion s).cliqueFinset s = G.cliqueFinset s := by
  ext t; simp
```

Every edge of `G.cliqueUnion s` lies in an s-clique of `G.cliqueUnion s`.

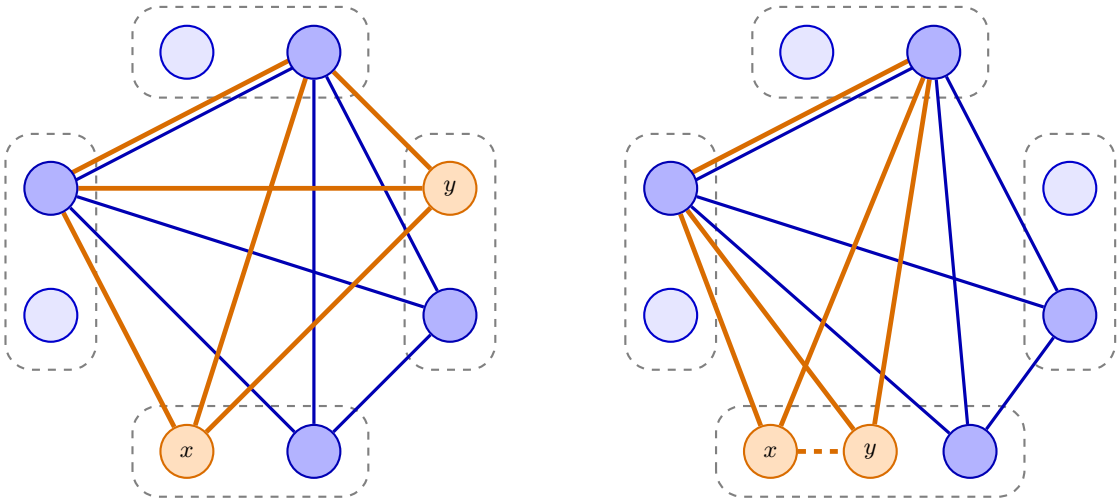
```
lemma cliqueUnion_cliqueUnion (s : ℕ) :
    (G.cliqueUnion s).cliqueUnion s = G.cliqueUnion s := by
  ext; simp
```

Our next lemma looks a little odd mathematically. The hypothesis `G ≤ G.cliqueUnion s` should be read as "every edge of `G` lies in an `s`-clique". (Obviously we also have `G.cliqueUnion s ≤ G` so in fact `G = G.cliqueUnion s`.)

If `G` is `s`-clique-maximal and every edge of `G` lies in an `s`-clique, then `G` is complete multipartite.

```
lemma IsNCliqueMaximal.isCompleteMultipartite_of_le_cliqueUnion [Fintype V]
    {s r : ℕ} (h : G.IsNCliqueMaximal s r)
    (hG : G ≤ G.cliqueUnion s) : G.IsCompleteMultipartite := by
  classical
  by_contra hcm
  obtain ⟨v, u, w, h3⟩ := exists_isPathGraph3Compl_of_not_isCompleteMultipartite hcm
  have hk := card_cliqueFinset_replaceVertex_replaceVertex h3 s
  have := h.card_filter_mem_eq_of_not_adj h3.not_adj_fst
  have := h.card_filter_mem_eq_of_not_adj h3.not_adj_snd
  have := h.card_cliqueFinset_le ((h.1.replaceVertex v u).replaceVertex v w)
  -- the edge u – w lies in an s-clique
  obtain ⟨-, t, ht, hut, hwt⟩ := hG h3.adj
  have : 0 < #{t ∈ G.cliqueFinset s | u ∈ t ∧ w ∈ t} :=
    card_pos.2 ⟨t, by simp [ht, hut, hwt]⟩
  grind
```

The next key observation is that a complete multipartite graph with one `s`-clique has many. Let `t` be an `s`-clique and let `x` and `y` be adjacent. At most one vertex of `t` is not adjacent to `x`, since two such vertices would both lie in the part of `x`, and similarly for `y`. So at least `s - 2` vertices of `t` are adjacent to both `x` and `y`, and together with `x` and `y` they form an `s`-clique. We allow the edge `x – y` to come from a larger graph `H`, so that `x` and `y` may lie in the same part of `G`, as on the right of the figure below. We need this twice: in `IsNCliqueMaximal.isCompleteMultipartite`, where `G.cliqueUnion s` plays the role of `G` and `G` plays the role of `H`, and in `IsNCliqueMaximal.not_cliqueFree_of_not_adj`, where `H` is `G` with an edge added inside a part.



A complete multipartite graph with four parts and a 4-clique `t` (blue). Left: if `x` and `y` lie in different parts, the vertices of `t` outside the parts of `x` and `y` form a 4-clique with `x` and `y` (orange). Right: if `x` and `y` lie in the same part and the edge between them (dashed) is added, any two vertices of `t` outside that part form a 4-clique with `x` and `y`.

If `G` is complete multipartite and has an `s`-clique, then every edge of a graph `H` containing `G` lies in an `s`-clique of `H`.

```
lemma IsCompleteMultipartite.exists_isNClique_of_le (hc : G.IsCompleteMultipartite)
    {H : SimpleGraph V} (hGH : G ≤ H) {s : ℕ} (hs : 2 ≤ s) {t : Finset V}
    (ht : G.IsNClique s t) {x y : V} (hxy : H.Adj x y) :
    ∃ t', H.IsNClique s t' ∧ x ∈ t' ∧ y ∈ t' := by
  classical
  -- at most one vertex of t is not adjacent to a given vertex z
  have hle (z : V) : #{a ∈ t | ¬G.Adj z a} ≤ 1 := by
    refine card_le_one.2 fun a ha b hb => ?_
    rw [mem_filter] at ha hb
    by_contra hab
    exact hc.trans _ _ _ (fun h => ha.2 h.symm) hb.2 (ht.1 ha.1 hb.1 hab)
  have hcard : s - 2 ≤ #{a ∈ t | G.Adj x a ∧ G.Adj y a} := by
    have := card_filter_add_card_filter_not (s := t) fun a => G.Adj x a ∧ G.Adj y a
    simp only [not_and_or, filter_or] at this
    have := card_union_le {a ∈ t | ¬G.Adj x a} {a ∈ t | ¬G.Adj y a}
    grind [hle x, hle y, ht.2]
  obtain ⟨t₀, ht₀, ht₀2⟩ := exists_subset_card_eq hcard
  have hxyt₀ (a) (ha : a ∈ t₀) : G.Adj x a ∧ G.Adj y a := (mem_filter.1 (ht₀ ha)).2
  have ht₀' : H.IsNClique (s - 2) t₀ :=
    ⟨(ht.1.subset (coe_subset.2 (ht₀.trans (filter_subset _ _)))).mono hGH, ht₀2⟩
  have hyt₀ := ht₀'.insert fun b hb => hGH (hxyt₀ b hb).2
  have hxt₀ := hyt₀.insert ((forall_mem_insert _ _ _).2
    ⟨hxy, fun b hb => hGH (hxyt₀ b hb).1⟩)
  exact ⟨_, Nat.sub_add_cancel hs ▸ hxt₀, mem_insert_self _ _,
    mem_insert_of_mem (mem_insert_self _ _)⟩
```

Now let `G` be `s`-clique-maximal with at least one `s`-clique. Then `G.cliqueUnion s` is `s`-clique-maximal too, and every edge of it lies in an `s`-clique, so it is complete multipartite. Applying the previous lemma to it, with `H = G`, shows that every edge of `G` lies in an `s`-clique, so `G` is complete multipartite as well.

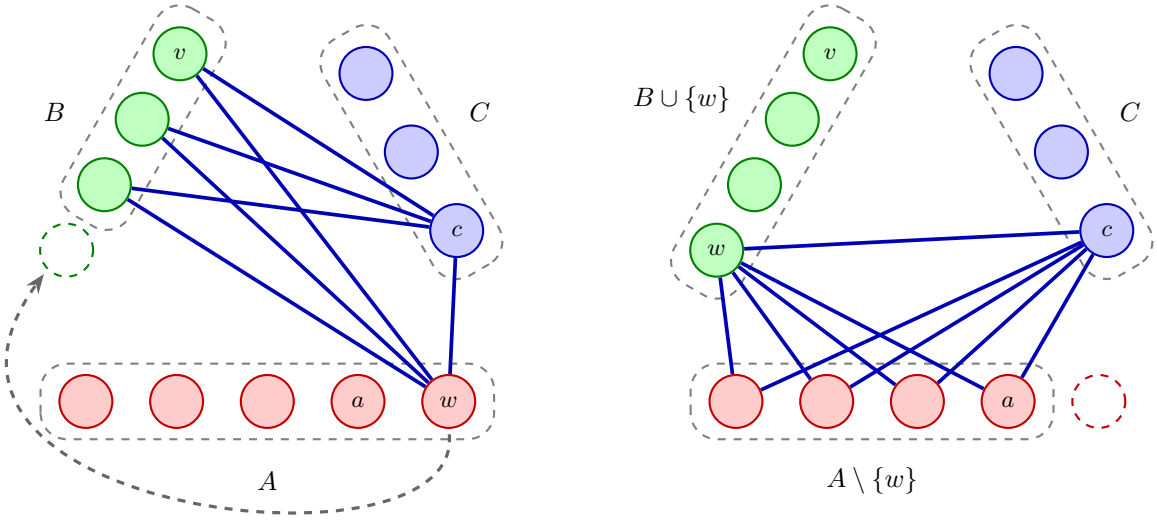
```
lemma IsNCliqueMaximal.isCompleteMultipartite [Fintype V] {s r : ℕ}
    (h : G.IsNCliqueMaximal s r) (hs : 2 ≤ s)
    {t : Finset V} (ht : G.IsNClique s t) : G.IsCompleteMultipartite := by
  classical
  have h₀ : (G.cliqueUnion s).IsNCliqueMaximal s r :=
    ⟨h.1.anti (G.cliqueUnion_le s), fun H hH => by
      simpa [copyCount_top] using h.2 hH⟩
  have hc₀ :=
    h₀.isCompleteMultipartite_of_le_cliqueUnion (cliqueUnion_cliqueUnion s).ge
  refine h.isCompleteMultipartite_of_le_cliqueUnion fun x y hxy => ?_
  obtain ⟨t', ht', hx, hy⟩ := hc₀.exists_isNClique_of_le (G.cliqueUnion_le s) hs
    (isNClique_cliqueUnion_iff.2 ht) hxy
  exact ⟨hxy.ne, t', ht', hx, hy⟩
```

Balancing the Parts

The Turán graph `T[n, r]` is not just complete multipartite, it is also balanced. For general `s`-cliques this "balancing of parts" argument is again more involved.

Suppose that `G` is an unbalanced complete multipartite graph. Let `A` and `B` be parts with `#B + 1 < #A`, and move a vertex `w` from `A` to `B` while keeping the graph complete multipartite: this is `G.replaceVertex v w` for any `v` in `B`. The number of `s`-cliques changes by `#A - #B - 1` times the

number of (`s - 2`)-cliques in the other parts. To contradict maximality we need this change to be positive, and it is as long as `G` has an `s`-clique.

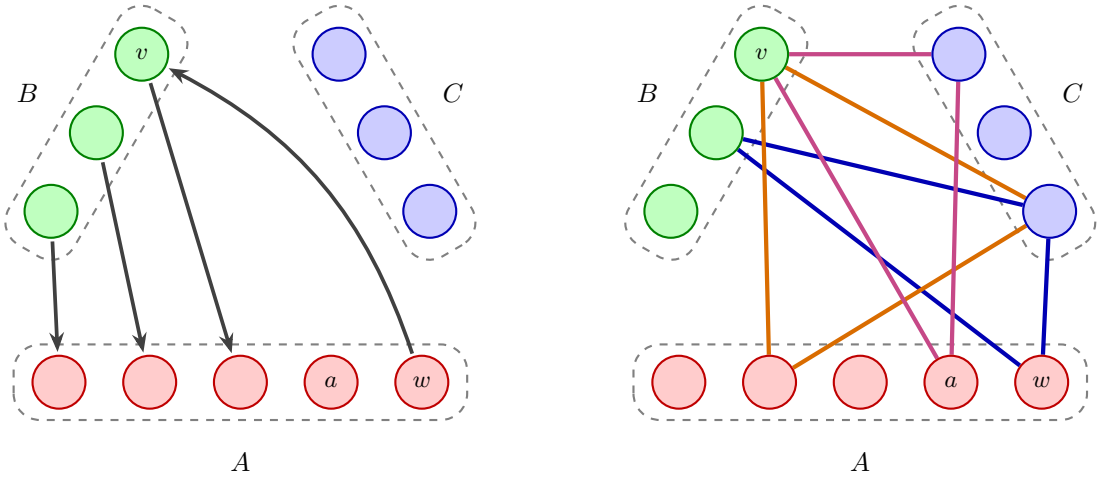


Moving `w` from the part `A` to the part `B` of `v` turns `G` (left) into `G.replaceVertex v w` (right). The triangles containing `w` and a vertex `c` of another part `C` change from one for each vertex of `B` to one for each vertex of `A` other than `w`, in this case from 3 to 4.

The figure above shows why moving `w` works, but the proof does not compute these numbers directly. Instead it compares the cliques that the move destroys with those that it creates. By `card_cliqueFinset_replaceVertex`, the move destroys the `s`-cliques containing `w` and creates one for each `s`-clique of `G` that contains `v` but not `w`, since `w` takes over the neighbours of `v`. So it is enough to map the first set injectively into the second, missing at least one clique.

Since the parts are unbalanced, `A` has at least `#B + 2` vertices, so we can choose a vertex `a` in `A \ {w}`. We first build the map that will transfer the cliques. Map `B` injectively into `A \ {w, a}` using some φ, and extend φ to a map `σ : V → V` that sends `w` to `v`, and fixes every other vertex.

Every `s`-clique containing `w` lies in its closed neighbourhood `insert w (G.neighborSet w)`, which consists of `w` and the vertices outside `A`. On these vertices σ sends `w` into `B`, sends `B` into `A`, and fixes the rest. As `w` is the only vertex of `A` in this neighbourhood, σ sends vertices in different parts to vertices in different parts, so it preserves adjacency there. It is also injective there, and never takes the value `w` or `a`.



Left: the map σ sends `w` to `v`, maps `B` injectively into `A \ {w, a}`, and fixes every other vertex.
Right: σ pairs each triangle that the move destroys (blue) with a triangle containing `v` but not `w` (orange); the move creates a copy of it with `w` in place of `v`. No triangle is paired with the magenta one containing `a`, so the move creates more triangles than it destroys.

Let `G` be complete multipartite, let the part of `w` have at least two more vertices than the part of `v`, and let `a ≠ w` lie in the part of `w`. Then some map sends `w` to `v`, and is injective and preserves adjacency on the closed neighbourhood of `w`, where it avoids `w` and `a`.

```
lemma IsCompleteMultipartite.exists_transfer [Fintype V] [DecidableRel G.Adj]
    (hc : G.IsCompleteMultipartite) {v w a : V} (hvw : G.Adj v w)
    (hwa : ¬G.Adj w a) (haw : a ≠ w)
    (hlt : #{x | ¬G.Adj v x} + 1 < #{x | ¬G.Adj w x}) :
    ∃ σ : V → V, Set.InjOn σ (insert w (G.neighborSet w)) ∧
      (∀ ⦃x⦄, x ∈ insert w (G.neighborSet w) → ∀ ⦃y⦄,
        y ∈ insert w (G.neighborSet w) → G.Adj x y → G.Adj (σ x) (σ y)) ∧
      σ w = v ∧ ∀ x ∈ insert w (G.neighborSet w), σ x ≠ w ∧ σ x ≠ a := by
  classical
  set A : Finset V := {x | ¬G.Adj w x}
  set B : Finset V := {x | ¬G.Adj v x}
  have nt {x y z : V} (h₁ : ¬G.Adj x y) (h₂ : ¬G.Adj y z) : ¬G.Adj x z :=
    hc.trans _ _ _ h₁ h₂
  have hwA : w ∈ A := by simp [A]
  have haA : a ∈ A := by simp [A, hwa]
  -- every vertex of A is adjacent to every vertex of B
  have hAB {x y : V} (hx : x ∈ A) (hy : y ∈ B) : G.Adj x y := by grind [Adj.symm]
  -- an injection φ from B to A \ {w, a}
  have hcard : Fintype.card B ≤ Fintype.card ((A.erase w).erase a) := by
    simp only [Fintype.card_coe, card_erase_of_mem (mem_erase.2 ⟨haw, haA⟩),
      card_erase_of_mem hwA]
    grind
  obtain ⟨φ⟩ := Function.Embedding.nonempty_of_card_le hcard
  let σ : V → V := fun x => if x = w then v else if hx : x ∈ B then φ ⟨x, hx⟩ else x
  refine ⟨σ, fun x hx y hy hxy => ?_, fun x hx y hy hxy => ?_, by simp [σ],
    fun x hx => ?_⟩ <;>
      grind [mem_neighborSet, φ.injective.eq_iff, Subtype.ext_iff, Adj.symm]
```

Now let `t` be an `s`-clique containing `w`. It lies in the closed neighbourhood of `w`, so `t.image σ` is again

an s-clique, by `IsNClique.image_of_adj` from Cliques (p. 127). It contains v but neither w nor a. Different cliques have different images, since σ is injective on the whole closed neighbourhood and not just on each clique. So `t ↦ t.image σ` is an injection from the s-cliques containing w to those containing v that avoid w and a: this is the inequality below. Avoiding a is exactly what we need in `IsNCliqueMaximal.card_le_card_add_one`, where we find an s-clique containing v and a but not w. It contains a, so the injection misses it and the change is strictly positive.

```
lemma IsCompleteMultipartite.card_filter_mem_cliqueFinset_le [Fintype V]
    [DecidableEq V] [DecidableRel G.Adj] (hc : G.IsCompleteMultipartite) {v w a : V}
    (hvw : G.Adj v w) (hwa : ¬G.Adj w a) (haw : a ≠ w)
    (hlt : #{x | ¬G.Adj v x} + 1 < #{x | ¬G.Adj w x}) (s : ℕ) :
    #{t ∈ G.cliqueFinset s | w ∈ t} ≤
      #{t ∈ G.cliqueFinset s | v ∈ t ∧ w ∉ t ∧ a ∉ t} := by
  obtain ⟨σ, hinj, hadj, hσw, hσ⟩ := hc.exists_transfer hvw hwa haw hlt
  -- the cliques containing w lie in its closed neighbourhood
  have hsub {t : Finset V} (ht : G.IsNClique s t) (hwt : w ∈ t) :
      (t : Set V) ⊆ insert w (G.neighborSet w) := fun x hx =>
    (eq_or_ne x w).imp_right fun hxw => ht.1 hwt hx hxw.symm
  refine card_le_card_of_injOn (fun t => t.image σ) (fun t ht => ?_)
    (fun t ht t' ht' htt' => ?_)
  · simp only [coe_filter, Set.mem_ofPred_eq, mem_cliqueFinset_iff] at ht ⊢
    obtain ⟨ht, hwt⟩ := ht
    refine ⟨ht.image_of_adj fun x hx y hy => hadj (hsub ht hwt hx) (hsub ht hwt hy),
      mem_image.2 ⟨w, hwt, hσw⟩, fun hw => ?_, fun ha => ?_⟩
    · obtain ⟨x, hx, hxw⟩ := mem_image.1 hw
      exact (hσ x (hsub ht hwt hx)).1 hxw
    · obtain ⟨x, hx, hxa⟩ := mem_image.1 ha
      exact (hσ x (hsub ht hwt hx)).2 hxa
  · simp only [coe_filter, Set.mem_ofPred_eq, mem_cliqueFinset_iff] at ht ht' htt'
    apply Finset.coe_injective
    exact (hinj.image_eq_image_iff
      (hsub ht.1 ht.2) (hsub ht'.1 ht'.2)).1
        (by exact_mod_cast htt')
```

An s-clique-maximal graph with at least one s-clique has balanced parts: any two vertices have numbers of non-neighbours that differ by at most one, and the non-neighbours of a vertex form its part (`IsCompleteMultipartite.part_finpartition`).

```
lemma IsNCliqueMaximal.card_le_card_add_one [Fintype V] [DecidableRel G.Adj]
    {s r : ℕ} (h : G.IsNCliqueMaximal s r)
    (hs : 2 ≤ s) {t : Finset V} (ht : G.IsNClique s t) (v w : V) :
    #{x | ¬G.Adj w x} ≤ #{x | ¬G.Adj v x} + 1 := by
  classical
  by_contra! hlt
  have hc := h.isCompleteMultipartite hs ht
  have hvw := hc.adj_of_card_ne (v := v) (w := w) (by grind)
  obtain ⟨a, ha, haw⟩ := exists_mem_ne (s := {x | ¬G.Adj w x}) (by grind) w
  have hwa : ¬G.Adj w a := (mem_filter.1 ha).2
  -- some s-clique contains v and a, and hence not w
  have hva : G.Adj v a := by grind [hc.trans, Adj.symm]
  obtain ⟨t', ht', hvt', hat'⟩ := hc.exists_isNClique_of_le le_rfl hs ht hva
  have hwt' : w ∉ t' := fun hwt' => hwa (ht'.1 hwt' hat' haw.symm)
  have hd := hc.card_filter_mem_cliqueFinset_le hvw hwa haw hlt s
  have hd' : #{t ∈ G.cliqueFinset s | v ∈ t ∧ w ∉ t ∧ a ∉ t} <
```

```
      #{t ∈ G.cliqueFinset s | v ∈ t ∧ w ∉ t} :=
    card_lt_card ◁ (ssubset_iff_of_subset
      (monotone_filter_right _ fun _ _ ht => ⟨ht.1, ht.2.1⟩)).2
        ⟨t', by simp [ht', hvt', hwt'], by simp [hat']⟩
  have hk := card_cliqueFinset_replaceVertex (G := G) hvw.ne s
  have := h.card_cliqueFinset_le (h.1.replaceVertex v w)
  grind
```

Any s-clique-maximal graph with at least one s-clique is either complete or contains an r-clique.

```
lemma IsNCliqueMaximal.not_cliqueFree_of_not_adj [Fintype V] {s r : ℕ}
    (h : G.IsNCliqueMaximal s r) (hs : 2 ≤ s) {t : Finset V}
    (ht : G.IsNClique s t) {x y : V} (hxy : x ≠ y) (hn : ¬G.Adj x y) :
    ¬G.CliqueFree r := by
  intro hcf
  classical
  have hxy' : (G ⊔ edge x y).Adj x y := by simp [edge_adj, hxy]
  obtain ⟨t', ht', hxt', hyt'⟩ :=
    (h.isCompleteMultipartite hs ht).exists_isNClique_of_le le_sup_left hs ht hxy'
  have hlt : G.cliqueFinset s ⊂ (G ⊔ edge x y).cliqueFinset s :=
    (ssubset_iff_of_subset (cliqueFinset_mono _ le_sup_left)).2
      ⟨t', mem_cliqueFinset_iff.2 ht',
        fun hG => hn ((mem_cliqueFinset_iff.1 hG).1 hxt' hyt' hxy)⟩
  exact (card_lt_card hlt).not_ge (h.card_cliqueFinset_le (hcf.sup_edge x y))
```

The Bound and the Extremal Graphs

We have shown that, for $2 \le s$, an s-clique-maximal graph with an s-clique is complete multipartite, that its parts are balanced, and that it has an r-clique unless it is complete. So it is a Turán graph, by `IsCompleteMultipartite.nonempty_iso_turanGraph` (Proof of Turán's Theorem (p. 416)).

```
theorem IsNCliqueMaximal.nonempty_iso_turanGraph [Fintype V] {s r : ℕ}
    (h : G.IsNCliqueMaximal s r) (hs : 2 ≤ s)
    {t : Finset V} (ht : G.IsNClique s t) :
    Nonempty (G ≃g T[Fintype.card V, r]) := by
  classical
  exact (h.isCompleteMultipartite hs ht).nonempty_iso_turanGraph h.1
    (h.card_le_card_add_one hs ht) fun _ _ => h.not_cliqueFree_of_not_adj hs ht
```

Two small facts remain to prove.

For $s \le 1$ every set of s vertices is a clique, so the s-cliques do not depend on the graph.

```
lemma cliqueFinset_eq_of_le_one [Fintype V] [DecidableEq V] [DecidableRel G.Adj]
    {H : SimpleGraph V} [DecidableRel H.Adj] {s : ℕ} (hs : s ≤ 1) :
    G.cliqueFinset s = H.cliqueFinset s := by
  -- interval_cases s splits into one case for each value of s allowed by the
  -- bounds (here 0 and 1)
  interval_cases s <;> ext <;> simp [isNClique_zero, isNClique_one]
```

Isomorphic graphs have the same number of s-cliques.

```
lemma Iso.card_cliqueFinset_eq [Fintype V] [DecidableEq V] [DecidableRel G.Adj]
    {W : Type*} [Fintype W] [DecidableEq W] {H : SimpleGraph W} [DecidableRel H.Adj]
```

```
    (e : G ≃g H) (s : ℕ) : #(G.cliqueFinset s) = #(H.cliqueFinset s) := by
  -- the s-cliques are the copies of K[s] (copyCount_top)
  have h := Iso.copyCount_eq (.refl : K[s] ≃g K[s]) e
  rwa [copyCount_top, copyCount_top, Fintype.card_fin] at h
```

Zykov's theorem follows. Let `G` be `(r + 1)`-clique-free. The `(r + 1)`-clique-free graphs on `V` form a finite set, which contains `G` and so is nonempty. A graph `H` in it with the most `s`-cliques is `s`-clique-maximal, and `G` has at most as many `s`-cliques as `H`. If `H` has no `s`-clique then neither does `G`, so the bound holds trivially. If `H` has an `s`-clique and `2 ≤ s`, then `H` is a Turán graph, which gives the bound. For `s ≤ 1` we instead compare both graphs with an empty graph, using `cliqueFinset_eq_of_le_one`.

Zykov's theorem: an `(r + 1)`-clique-free graph on `n` vertices has at most as many `s`-cliques as the Turán graph `T[n, r]`.

```
theorem CliqueFree.card_cliqueFinset_le [Fintype V] [DecidableEq V]
    [DecidableRel G.Adj] {r : ℕ} (h : G.CliqueFree (r + 1)) (s : ℕ) :
    #(G.cliqueFinset s) ≤ #(T[Fintype.card V, r].cliqueFinset s) := by
  rcases lt_or_ge s 2 with hs | hs
  · -- for s ≤ 1 compare the empty graphs on V and Fin (card V)
    let e := Iso.emptyGraph (Fintype.equivFin V)
    rw [cliqueFinset_eq_of_le_one (H := ⊥) (by grind),
      cliqueFinset_eq_of_le_one (G := T[_, r]) (H := ⊥) (by grind)]
    exact (e.card_cliqueFinset_eq s).le
  classical
  -- G is (r + 1)-clique-free, so a graph H with the most s-cliques exists
  obtain ⟨H, hH, hmax⟩ :=
    ({H | H.CliqueFree (r + 1)} : Finset (SimpleGraph V)).exists_max_image
      (·.copyCount K[s]) ⟨G, by simpa using h⟩
  have hH' : H.IsNCliqueMaximal s r :=
    ⟨by simpa using hH, fun H' hH' => hmax H' (by simpa using hH')⟩
  have hGH := hH'.card_cliqueFinset_le h
  rcases (H.cliqueFinset s).eq_empty_or_nonempty with he | ⟨t, ht⟩
  · grind
  · obtain ⟨e⟩ := hH'.nonempty_iso_turanGraph hs (mem_cliqueFinset_iff.1 ht)
    exact hGH.trans (e.card_cliqueFinset_eq s).le
```

So a 4-clique-free graph on 7 vertices has at most 12 triangles.

```
example [Fintype V] [DecidableEq V] [DecidableRel G.Adj] (hV : Fintype.card V = 7)
    (h : G.CliqueFree 4) : #(G.cliqueFinset 3) ≤ 12 :=
  (h.card_cliqueFinset_le (r := 3) 3).trans_eq (by rw [hV]; decide)
```

Turán's theorem also says that the Turán graph is the *only* extremal graph (`isTuranMaximal_iff_nonempty_iso_turanGraph`). The same is true for Zykov's theorem when `2 ≤ s ≤ r` and `s ≤ n`. The other cases are degenerate, since all `(r + 1)`-clique-free graphs on `n` vertices then have the same number of `s`-cliques: `1` if `s = 0`, `n` if `s = 1`, and `0` if `s > r` or `s > n`.

When `s ≤ r` and `s ≤ n`, the vertices `0, …, s - 1` of the Turán graph lie in different parts, so it has an `s`-clique. A graph with as many `s`-cliques as the Turán graph is `s`-clique-maximal, since by the bound just proved (`CliqueFree.card_cliqueFinset_le`) no `(r + 1)`-clique-free graph has more. So it has an `s`-clique too, and it is a Turán graph by `IsNCliqueMaximal.nonempty_iso_turanGraph`.

Zykov's theorem, equality case: for `2 ≤ s ≤ r` and `s ≤ n`, an `(r + 1)`-clique-free graph on `n` ver-

tices has as many s-cliques as `T[n, r]` iff it is isomorphic to `T[n, r]`.

```
theorem CliqueFree.card_cliqueFinset_eq_iff_nonempty_iso_turanGraph [Fintype V]
    [DecidableEq V] [DecidableRel G.Adj] {r s : ℕ} (h : G.CliqueFree (r + 1))
    (hs : 2 ≤ s) (hsr : s ≤ r) (hsn : s ≤ Fintype.card V) :
    #(G.cliqueFinset s) = #(T[Fintype.card V, r].cliqueFinset s) ↔
      Nonempty (G ≃g T[Fintype.card V, r]) := by
  refine ⟨fun hk => ?_, fun ⟨e⟩ => e.card_cliqueFinset_eq s⟩
  have hT : T[Fintype.card V, r].IsNClique s
      (univ.map (Fin.castLEEmb hsn)) := by
    refine ⟨?_, by simp⟩
    simp only [coe_map, coe_univ, Set.image_univ]
    rintro _ ⟨i, rfl⟩ _ ⟨j, rfl⟩ hij
    simpa [turanGraph_adj, Nat.mod_eq_of_lt (i.isLt.trans_le hsr),
      Nat.mod_eq_of_lt (j.isLt.trans_le hsr), Fin.val_inj] using hij
  obtain ⟨t, ht⟩ : (G.cliqueFinset s).Nonempty := by
    rw [← card_pos, hk]
    exact card_pos.2 ⟨_, mem_cliqueFinset_iff.2 hT⟩
  exact IsNCliqueMaximal.nonempty_iso_turanGraph (isNCliqueMaximal_iff.2
    ⟨h, fun H _ hH => (hH.card_cliqueFinset_le s).trans hk.ge⟩) hs
    (mem_cliqueFinset_iff.1 ht)
```

So among `(r + 1)`-clique-free graphs on `n` vertices, one with the most s-cliques, for any `2 ≤ s ≤ min r n`, also has the most edges. For example, a 4-clique-free graph with as many triangles as `T[n, 3]` has `turanNumber n 3` edges.

```
example [Fintype V] [DecidableEq V] [DecidableRel G.Adj] {r s : ℕ}
    (h : G.CliqueFree (r + 1)) (hs : 2 ≤ s) (hsr : s ≤ r) (hsn : s ≤ Fintype.card V)
    (hk : #(G.cliqueFinset s) = #(T[Fintype.card V, r].cliqueFinset s)) :
    #G.edgeFinset = turanNumber (Fintype.card V) r := by
  obtain ⟨e⟩ := (h.card_cliqueFinset_eq_iff_nonempty_iso_turanGraph hs hsr hsn).1 hk
  exact e.card_edgeFinset_eq
```

For `s = 2` the s-cliques are just the edges, so Zykov's theorem contains Turán's theorem, including the uniqueness of the extremal graph (Exercises 16–18). Our proof of Zykov's theorem does not use Turán's theorem, so this gives a second proof of it.

13.4 Regularity and Removal*

The Szemerédi regularity lemma (`szemeredi_regularity`) is one of the most important tools in modern extremal combinatorics. Mathlib has it together with two classic applications: the triangle removal lemma (`triangle_removal`) and a graph-theoretic proof of Roth's theorem on three-term arithmetic progressions (`roth_3ap_theorem_nat`). We end with a third application, which is not quite in Mathlib but follows very easily: the Ruzsa–Szemerédi theorem (`ruzsaSzemerediNumberNat_isLittleO`).

Edge Density and Uniform Pairs

The *edge density* between two finite sets `s` and `t` of vertices is the fraction of pairs in `s ×ˢ t` that are edges. It is a rational number, so we can compute it.

```
#check edgeDensity_def -- G.edgeDensity s t = #(G.interedges s t) / (#s * #t)
#eval C[6].edgeDensity {0, 2, 4} {1, 3, 5} -- 2 / 3
```

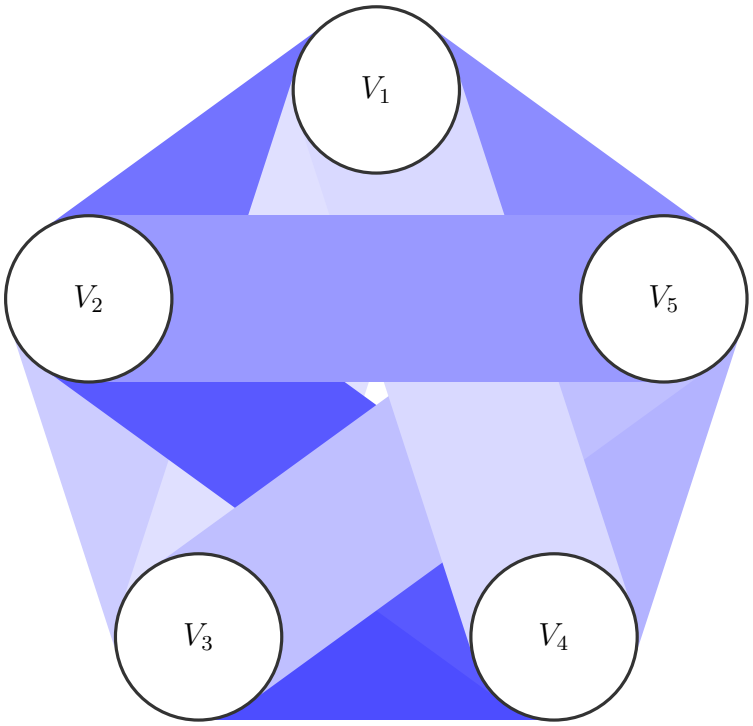
```
#eval K[6].edgeDensity {0, 2, 4} {1, 3, 5} -- 1
```

A pair `(s, t)` is ε-*uniform* if the edges between them are spread evenly: any `s' ⊆ s` and `t' ⊆ t` containing at least an ε fraction of `s` and `t` have edge density within ε of the density between `s` and `t`. This mimics a random bipartite subgraph.

```
#source IsUniform
```

The Szemerédi Regularity Lemma

The *regularity lemma* [Szemerédi 1978] says that for every `ε > 0`, every finite graph can be approximated by a structured one: its vertices can be partitioned into a bounded number of almost equal parts so that most pairs of parts are ε-uniform.



An ε-uniform partition; the band between two parts is shaded by their edge density

In Mathlib's statement, `Finpartition.IsEquipartition` says the parts of a `Finpartition` differ in size by at most `1`, and `Finpartition.IsUniform` says that at most an `ε` fraction of the pairs of parts are not ε-uniform. Given `l ≤ Fintype.card V`, the number of parts is between `l` and `bound ε l`. Crucially, this bound depends only on `ε` and `l`, not on the graph.

```
#source Finpartition
#check szemeredi_regularity
```

The Triangle Removal Lemma

If `H` has an edge (`H ≠ ⊥`), any graph can be made `H`-free by deleting one edge from each copy of `H`. Mathlib calls the resulting graph `killCopies`, and it has lost no more than `G.copyCount H` edges, one for each copy of `H` in `G` (see `copyCount`).

```
#check free_killCopies -- H ≠ ⊥ → H.Free (G.killCopies H)
#check le_card_edgeFinset_killCopies
```

The triangle removal lemma [Ruzsa–Szemerédi 1978] is a far stronger statement: for every `ε > 0`, there is `δ > 0`, independent of `n`, such that any graph on `n` vertices with fewer than `δ * n³` triangles can be made triangle-free by deleting fewer than `ε * n²` edges. Thus `o(n³)` triangles can be eliminated by deleting `o(n²)` edges. In Mathlib a graph is *ε-far from triangle-free* if at least `ε * n²` edges must be deleted to make it triangle-free.

```
#source FarFromTriangleFree
```

The lemma is proved by applying the regularity lemma, and then a *counting lemma*: three parts that are pairwise uniform and dense span many triangles.

```
#check triangle_counting
#check FarFromTriangleFree.le_card_cliqueFinset
#check triangle_removal
```

Roth's Theorem

The triangle removal lemma has a striking application in additive number theory.

Roth's theorem [Roth 1953] says that a set of integers with positive density contains a three-term arithmetic progression `a, a + d, a + 2 * d` with `d ≠ 0`. More precisely, for every `ε > 0` and all large enough `n`, every subset of `{0, …, n - 1}` with at least `ε * n` elements contains one. Mathlib phrases this as `¬ThreeAPFree`.

```
#check roth_3ap_theorem_nat
```

The proof goes via the *corners theorem*: a dense set of points in the grid contains a corner `(x, y), (x + d, y), (x, y + d)` with `d ≠ 0`.

```
#check corners_theorem_nat
```

The Ruzsa–Szemerédi Theorem

A graph is *locally linear* (`LocallyLinear`) if every edge lies in exactly one triangle. Mathlib splits this into two conditions: the triangles are edge-disjoint (`EdgeDisjointTriangles`, any two triangles share at most one vertex), and every edge lies in some triangle.

```
#source LocallyLinear
```

For example, take the Paley graph on `9` vertices. Mathlib's field with `9` elements, `GaloisField 3 2`, is not computable, so we build one that is: `𝔽₃[i]`, where `i² = -1`. Mathlib's `QuadraticAlgebra R a b` adjoins to `R` an element `ω` with `ω² = a + b ω`, and stores `x + y ω` as the pair `⟨x, y⟩`. It is a field if `R` is a field and `ω² = a + b ω` has no solution in `R`. Here `-1` is not a square in `ZMod 3`, and as with `Fact (Nat.Prime 17)` in Paley Graphs (p. 29), we supply this as an instance. Mathlib has no `Fintype` instance for `QuadraticAlgebra`, so we transfer one from `R × R`.

```
instance : Fact (¬IsSquare (-1 : ZMod 3)) := ⟨by decide⟩
```

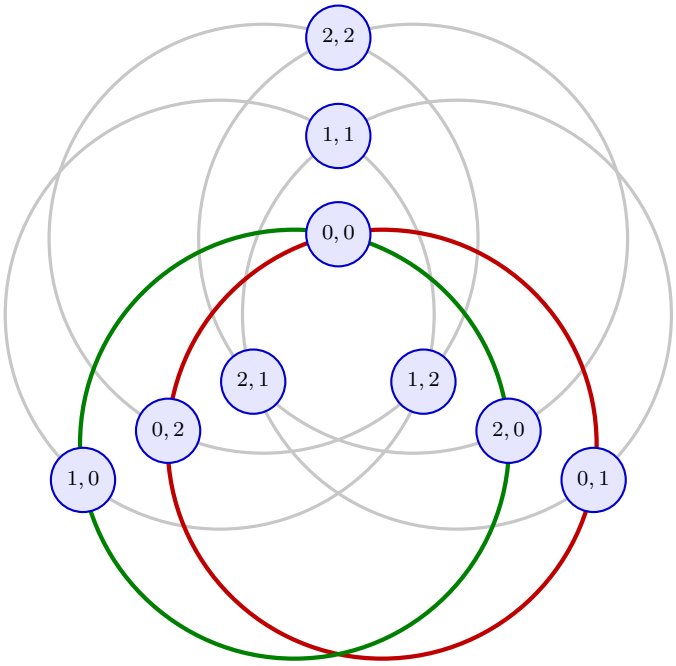
```
instance {R : Type*} [Fintype R] {a b : R} : Fintype (QuadraticAlgebra R a b) :=
  Fintype.ofEquiv _ (QuadraticAlgebra.equivProd a b).symm
```

The Paley graph on 9 vertices, of the field $\mathbb{F}_3[i]$, where $i^2 = -1$.

```
abbrev paleyGraph₉ : SimpleGraph (QuadraticAlgebra (ZMod 3) (-1) 0) :=
  paleyGraph _
```

The nonzero squares in $\mathbb{F}_3[i]$ are ± 1 and $\pm i$, the nonzero elements with a zero coordinate. So two vertices are adjacent when they agree in one coordinate: they lie in the same row or column of the 3×3 grid. By the parameters in Strongly Regular Graphs (p. 373), it is strongly regular with $\ell = (9 - 5) / 4 = 1$: two adjacent vertices have exactly one common neighbour, the third vertex of their row or column. So each edge lies in exactly one triangle, and `paleyGraph₉` is locally linear. It has 18 edges and 6 triangles. (We use `decide +kernel` since it can handle the recursion depth here.)

```
example : paleyGraph₉.LocallyLinear := by decide +kernel
#eval #paleyGraph₉.edgeFinset -- 18
#eval #(paleyGraph₉.cliqueFinset 3) -- 6
```



The Paley graph `paleyGraph₉` on 9 vertices. Its 6 triangles are drawn as circles, whose three arcs are edges.

Locally linear graphs are behind the proof of the corners theorem: a corner-free set `A` gives a locally linear graph with `#A` triangles, and the removal lemma shows that this graph has few triangles, so `A` is small.

The *Ruzsa–Szemerédi problem* asks for the largest number of edges in a locally linear graph on `n` vertices. A locally linear graph has three times as many edges as triangles (`LocallyLinear.card_edgeFinset`), so we can count triangles instead: Mathlib's `ruzsaSzemerediNumberNat n` is the largest number of triangles in such a graph.

In the other direction, sets without three-term arithmetic progressions give locally linear graphs

with many triangles: a `ThreeAPFree` subset of `{0, …, n - 1}` with `m` elements gives a locally linear graph on `6 * n + 3` vertices with `(2 * n + 1) * m` triangles. Behrend's construction [Behrend 1946] of large `ThreeAPFree` sets then shows that `ruzsaSzemerediNumberNat n` is at least a constant times `n² * exp (-4 * √(log n))`, which is larger than `n ^ (2 - c)` for every `c > 0` once `n` is large.

```
#check rothNumberNat_le_ruzsaSzemerediNumberNat
#check ruzsaSzemerediNumberNat_asymptotic_lower_bound
```

Mathlib does not have the upper bound `o(n²)`, but the argument for corners gives it for every graph `G` whose triangles are edge-disjoint. Suppose that `G` has at least `ε * n²` triangles. Deleting an edge destroys at most one triangle, so `G` is `ε`-far from triangle-free, and the removal lemma gives at least `δ * n³` triangles, where `δ = triangleRemovalBound ε`. But the triangles have disjoint sets of edges, so there are at most as many triangles as edges, and hence at most `n²`. This is less than `δ * n³` when `n > 1 / δ`.

```
theorem EdgeDisjointTriangles.card_cliqueFinset_lt [Fintype V] [DecidableEq V]
    [DecidableRel G.Adj] (hG : G.EdgeDisjointTriangles) {ε : ℝ} (hε : 0 < ε)
    (hn : (triangleRemovalBound ε)⁻¹ < Fintype.card V) :
    (#(G.cliqueFinset 3) : ℝ) < ε * Fintype.card V ^ 2 := by
  by_contra! h
  have hδ := triangleRemovalBound_pos hε
  have hV : (0 : ℝ) < Fintype.card V := (inv_pos.2 hδ).trans hn
  -- the removal lemma gives δ * n³ triangles
  have h₁ := (hG.farFromTriangleFree (by exact_mod_cast h)).le_card_cliqueFinset
  -- but there are at most n² / 3 of them
  have h₂ : 3 * #(G.cliqueFinset 3) ≤ Fintype.card V ^ 2 :=
    hG.card_edgeFinset_le.trans ◁
      G.card_edgeFinset_le_card_choose_two.trans (Nat.choose_le_pow ..)
  have h₃ : (3 * #(G.cliqueFinset 3) : ℝ) ≤ Fintype.card V ^ 2 := by
    exact_mod_cast h₂
  rw [inv_lt_iff_one_lt_mul₀' hδ] at hn
  linarith [mul_lt_mul_of_pos_right hn (pow_pos hV 2)]
```

The **Ruzsa–Szemerédi theorem**: `ruzsaSzemerediNumberNat n` is `o(n²)`, so a locally linear graph on `n` vertices has `o(n²)` edges. This answers the Ruzsa–Szemerédi problem up to the gap between `o(n²)` and Behrend's lower bound.

```
open Filter Asymptotics in
theorem ruzsaSzemerediNumberNat_isLittleO :
    (fun n => (ruzsaSzemerediNumberNat n : ℝ)) =o[atTop] fun n => (n : ℝ) ^ 2 := by
  simp only [isLittleO_iff, eventually_atTop, norm_pow, RCLike.norm_natCast]
  -- the filter and Landau notation have gone, leaving the obvious goal
  -- ⊢ ∀ c, 0 < c → ∃ a, ∀ b, a ≤ b → ↑(ruzsaSzemerediNumberNat b) ≤ c * ↑b ^ 2
  refine fun ε hε => ⟨⌊(triangleRemovalBound ε)⁻¹⌋₊ + 1, fun n hn => ?_⟩
  obtain ⟨G, _, hG, hlin⟩ := ruzsaSzemerediNumber_spec (α := Fin n)
  have := hlin.edgeDisjointTriangles.card_cliqueFinset_lt hε
    (by simpa using Nat.lt_of_floor_lt hn)
  rw [ruzsaSzemerediNumberNat, ← hG]
  simpa using this.le
```

Ruzsa and Szemerédi [Ruzsa–Szemerédi 1978] stated their result as the *(6, 3) theorem*: a family of 3-element subsets of an `n`-element set in which no 6 points contain 3 of the sets has `o(n²)` sets. This

follows from the graph form above. If two sets share two points, then no other set meets them, so these pairs occupy disjoint sets of 4 points. Discard their at most `n / 2` triples. Joining two points whenever one of the remaining triples contains both gives a graph whose triangles are exactly those triples: an extra triangle would have its edges supplied by three distinct triples whose union has at most 6 points, a contradiction. Every edge lies in one of these triangles, and the triangles are edge-disjoint, so the graph is locally linear. By the Ruzsa–Szemerédi theorem there are `o(n²)` remaining sets. In total the family has at most `n / 2 + o(n²)` sets, which is still `o(n²)`.

13.5 Infinite Graphs*

Most of this guide deals with finite graphs or with infinite graphs through their finite parts. We end with two results about the infinite structure of a graph.

Kőnig's Lemma

Kőnig's infinity lemma [Kőnig 1927] (not to be confused with Kőnig's theorem on matchings) says that an infinite, connected, locally finite graph contains a *ray* (a one-way infinite path). Mathlib has the order-theoretic form of this result: in a (strongly atomic) partial order in which each element is covered by finitely many elements, an element with infinitely many elements above it starts an infinite chain `b = f 0 ⋖ f 1 ⋖ f 2 ⋖ ⋯` of covering steps.

```
#check exists_seq_covby_of_forall_covby_finite
```

Mathlib has no notion of a ray in a graph, so it does not yet state the graph form. A ray in `G` is a copy of the infinite path `pathNat`, so the graph form says that `pathNat ⊑ G` for every infinite, connected, locally finite graph `G`. We deduce it from the order-theoretic form.

Fix a vertex `v` and order the vertices so that `u ≤ w` iff `u` lies on a shortest path from `v` to `w`, that is, iff `G.dist v u + G.dist u w = G.dist v w`. We use `dist` rather than `edist`: as `G` is connected, `dist` has no junk values, and the proof cancels terms from sums, which does not work in $\mathbb{N}\infty$. Inside the proof, `let _ : PartialOrder V` makes this order a local instance, so the instance search finds it for the rest of the proof and nowhere else.

We then check the hypotheses of Kőnig's lemma. The function `G.dist v` is strictly monotone, so the order is well founded, and Mathlib has an instance saying that a well-founded order is strongly atomic: if `u < w` then some `x ≤ w` covers `u`. If `w` covers `u` then `u` and `w` are adjacent: the second vertex `x` of a shortest walk from `u` to `w` satisfies `u ≤ x ≤ w` and `x ≠ u`, so `x = w`. So the vertices covering `u` are neighbours of `u`, and there are finitely many of them. Finally, every vertex lies above `v`, so infinitely many vertices do.

Kőnig's lemma now gives a chain `v = f 0 ⋖ f 1 ⋖ f 2 ⋖ ⋯` whose consecutive vertices are adjacent. We use the version `exists_orderEmbedding_covby_of_forall_covby_finite`, which returns the chain as an order embedding `f : ℕ ↪o V`, so the injectivity needed for a copy of `pathNat` comes for free.

Kőnig's lemma: an infinite, connected, locally finite graph has a ray.

```
theorem Connected.pathNat_isContained [Infinite V] [G.LocallyFinite]
    (hG : G.Connected) : pathNat ⊑ G := by
  obtain ⟨v⟩ := hG.nonempty
  -- u ≤ w iff u lies on a shortest path from v to w
  let _ : PartialOrder V :=
    { le u w := G.dist v u + G.dist u w = G.dist v w
```

```
    le_refl u := by simp
    le_trans u w x h₁ h₂ := by
      have := hG.dist_triangle (u := v) (v := u) (w := x)
      have := hG.dist_triangle (u := u) (v := w) (w := x)
      grind
    le_antisymm u w h₁ h₂ := by
      have := G.dist_comm (u := u) (v := w)
      exact hG.dist_eq_zero_iff.1 (by grind) }
-- G.dist v is strictly monotone, so the order is well founded
have hdist : StrictMono (G.dist v) := fun u w h => by
  have := hG.pos_dist_of_ne h.ne
  have : G.dist v u + G.dist u w = G.dist v w := h.le
  grind
have : WellFoundedLT V := hdist.wellFoundedLT
-- if w covers u then they are adjacent
have hadj {u w : V} (h : u ⋖ w) : G.Adj u w := by
  obtain ⟨_ | ⟨_, x, _, hux, p⟩, hp⟩ := hG.exists_walk_length_eq_dist u w
  · exact absurd rfl h.ne
  rw [Walk.length_cons] at hp
  have := dist_le p
  have := dist_eq_one_iff_adj.2 hux
  have := hG.dist_triangle (u := v) (v := u) (w := x)
  have := hG.dist_triangle (u := v) (v := x) (w := w)
  have : G.dist v u + G.dist u w = G.dist v w := h.le
  obtain rfl | rfl := h.eq_or_eq (c := x)
    (show G.dist v u + G.dist u x = G.dist v x by grind)
    (show G.dist v x + G.dist x w = G.dist v w by grind)
  · exact (hux.ne rfl).elim
  · exact hux
-- so each u has finitely many covers, and every vertex lies above v
obtain ⟨f, -, hf⟩ := exists_orderEmbedding_covby_of_forall_covby_finite
  (fun u => (G.neighborSet u).toFinite.subset fun _ h => hadj h)
  (Set.infinite_univ.mono fun w _ => (by simp : G.dist v v + G.dist v w = _))
refine ⟨⟨f, ?_⟩, f.injective⟩
rintro i j (rfl | rfl)
· exact hadj (hf i)
· exact (hadj (hf j)).symm
```

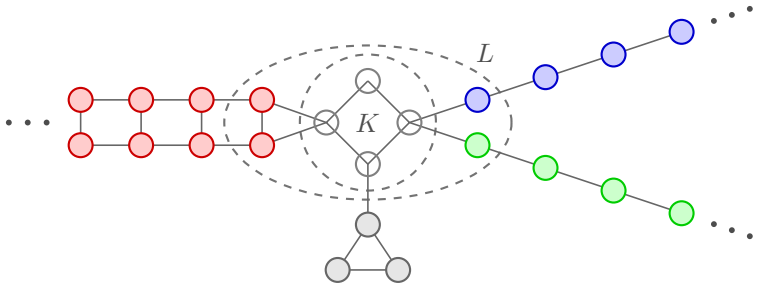
Like the De Bruijn–Erdős theorem (`colorable_of_forall_finite_subgraph_colorable`) and Hall's theorem for infinite graphs (`exists_isMatching_of_forall_ncard_le`), this is a *compactness* argument, deducing an infinite object from finite information.

Ends

The *ends* of an infinite graph describe the different ways of going to infinity. The usual definition is in terms of rays. Two rays are *equivalent* if no finite set of vertices separates them: for every finite set `K`, both rays have tails in the same component of the graph with `K` deleted. An end is an equivalence class of rays.

Mathlib describes ends without rays by recording where each ray ends up. Write `G.ComponentCompl K` for the set of components of `G - K`. Given a ray and a finite set `K`, only finitely many vertices of the ray lie in `K`, since a copy of `pathNat` is injective, so some tail of the ray avoids `K`. A tail is connected, so it lies in a single component of `G - K`, and any two tails of the ray share a tail, so this

component does not depend on the choice of tail. So a ray chooses, for each finite `K`, a component of `G - K`, and these choices are consistent: if `K ⊆ L` then the component chosen for `L` lies inside the one chosen for `K`. Any consistent choice of components is called a *direction*.



A graph with three ends: removing the finite set `K` leaves three infinite components (red, blue, green) and a finite one (grey); removing a larger finite set `L ⊇ K` leaves a tail of each infinite component.

```
section ends
open CategoryTheory Opposite

#source ComponentCompl -- abbrev ComponentCompl := (G.induce Kᶜ).ConnectedComponent
```

A *direction* of `G` chooses, for every finite set `K` of vertices, a component `f K` of `G - K`, consistently: if `K ⊆ L` then `f L ⊆ f K`.

```
def IsDirection (f : (K : Finset V) → G.ComponentCompl K) : Prop :=
  ∀ {K L : Finset V}, K ⊆ L → (f L : Set V) ⊆ f K
```

Two rays give the same direction exactly when they are equivalent, and Diestel and Kühn [Diestel–Kühn 2003] showed that every direction comes from a ray, so ends and directions are the same thing. Mathlib defines the ends of `G` as its directions, but phrased in the language of category theory. `G.componentComplFunctor` is the functor sending each finite set `K` to `G.ComponentCompl K`, and each inclusion `K ⊆ L` to the map `ComponentCompl.hom` sending a component of `G - L` to the component of `G - K` that contains it. These maps run from `L` to `K` (the opposite direction to the inclusion `K ⊆ L`), so the functor is defined on the opposite category `(Finset V)ᵒᵖ`. A direction is then a *section* of this functor: an element of each set, compatible with all of the maps.

```
#check componentComplFunctor -- (G : SimpleGraph V) : (Finset V)ᵒᵖ ⥤ Type u
#check SimpleGraph.end -- Set ((j : (Finset V)ᵒᵖ) → G.componentComplFunctor.obj j)
```

Unwinding the definitions, the elements of `G.end` are exactly the directions. (Here `op K` is `K` regarded as an object of `(Finset V)ᵒᵖ`.)

The ends of `G`, as defined in Mathlib, are exactly its directions.

```
theorem mem_end_iff {f : (K : (Finset V)ᵒᵖ) → G.componentComplFunctor.obj K} :
    f ∈ G.end ↔ G.IsDirection fun K => f (op K) := by
  constructor <;> intro hf K L h
  · exact (ComponentCompl.hom_eq_iff_le _ (Finset.coe_subset.2 h) _).1
      (hf (opHomOfLE h))
  · exact (ComponentCompl.hom_eq_iff_le _ (Finset.coe_subset.2 (leOfHom h.unop)) _).2
      (hf (leOfHom h.unop))
```

A finite graph has no ends, since `G - V` is empty; Mathlib has this as an instance, so `inferInstance` proves it.

```
example {V : Type} [Finite V] (G : SimpleGraph V) : IsEmpty G.end := inferInstance
```

Kőnig's lemma reappears in the language of ends: every infinite, connected, locally finite graph has at least one end. This follows from `Connected.pathNat_isContained`, since the ray it gives chooses a direction as described at the start of this section (Exercise 19), but Mathlib proves it directly, without rays. Getting a ray back from an end takes another compactness argument (the locally finite case of the Diestel–Kühn result above), so Mathlib's theorem does not give `Connected.pathNat_isContained` directly.

```
#check nonempty_ends_of_infinite
```

Connectivity and local finiteness ensure that `G - K` has only finitely many components (Mathlib's instance `componentCompl_finite`). For nonempty `K`, every component contains a neighbour of `K`, since `G` is connected, and `K` has only finitely many neighbours, since `G` is locally finite.

```
#check componentCompl_finite
```

Mathlib's proof then applies `nonempty_sections_of_finite_inverse_system`, the compactness theorem used earlier for finite subgraphs and Hall's theorem. It generalizes Kőnig's lemma: a functor on `(Finset V)ᵒᵖ` whose sets are all finite and nonempty has a section. Here they are nonempty because deleting finitely many vertices from an infinite graph leaves a vertex. Being able to reuse general results like this is why Mathlib defines ends using category theory.

To compute the ends of a graph we need to understand the components of `G - K`. Our main tool is the following observation: if `H` is connected and `φ : H →g G` is a homomorphism whose image avoids `K`, then the image lies in a single component of `G - K`.

```
theorem ComponentCompl.mem_of_hom {W : Type*} {H : SimpleGraph W}
    (hH : H.Preconnected) (φ : H →g G) {K : Set V} (hK : ∀ w, φ w ∉ K)
    {C : G.ComponentCompl K} {u : W} (hu : φ u ∈ C) (w : W) : φ w ∈ C := by
  obtain ⟨p⟩ := hH u w
  induction p with
  | nil => exact hu
  | cons h _ ih => exact ih (mem_of_adj _ _ hu (hK _) (φ.map_adj h))
```

This makes precise the step at the start of this section where a tail of a ray lies in a single component: a tail of a ray `f` is the image of `pathNat` under `f` composed with `pathNat.translate N`, and `pathNat` is connected.

Each end chooses infinite components of `G - K` (`end_componentCompl_infinite`): if a chosen component `C` were finite, the choice outside `K ∪ C` would have to lie in `C` while avoiding it. So to show that a graph has at most one end it is enough to find, for each finite set `K`, a vertex lying in every infinite component of `G - K`: any two ends then choose components sharing this vertex, which must be equal.

```
lemma subsingleton_end_of_common_vertex {V : Type} {G : SimpleGraph V}
    (h : ∀ K : Finset V, ∃ v : V,
      ∀ C : G.ComponentCompl (K : Set V), C.supp.Infinite → v ∈ C) :
    Subsingleton G.end := by
  constructor
  rintro ⟨e, he⟩ ⟨f, hf⟩
```

```
  apply Subtype.ext
  funext K
  obtain ⟨v, hv⟩ := h K.unop
  exact ComponentCompl.pairwise_disjoint.eq (Set.not_disjoint_iff.2 ⟨v,
    hv _ (end_componentCompl_infinite _ ⟨e, he⟩ K),
    hv _ (end_componentCompl_infinite _ ⟨f, hf⟩ K)⟩)
```

For example, `pathNat` has exactly one end. Given a finite set `K`, choose `N` with `K ⊆ [0, N]`. The translate `N + 1, N + 2, …` of `pathNat` avoids `K`, so it lies in a single component of `G - K`. An infinite component contains some `m > N`, and hence contains `N + 1`. So `N + 1` lies in every infinite component of `G - K`. Since `pathNat` is infinite, connected and locally finite, it also has at least one end by Kőnig's lemma (`nonempty_ends_of_infinite`).

```
example : Nat.card pathNat.end = 1 := by
  rw [Nat.card_eq_one_iff_unique]
  refine ⟨subsingleton_end_of_common_vertex fun K => ?_, ?_⟩
  · obtain ⟨N, hN⟩ := K.finite_toSet.bddAbove
    let φ := (pathNat.translate (N + 1)).toHom
    have hφ (n : ℕ) : φ n = n + (N + 1) := rfl
    have hK (n : ℕ) : φ n ∉ K := fun h => by
      have := hN h; rw [hφ] at this; grind
    -- every infinite component of G - K contains N + 1
    refine ⟨N + 1, fun C hC => ?_⟩
    obtain ⟨m, hmC, hm⟩ := hC.exists_gt N
    have h₁ : φ (m - (N + 1)) = m := by rw [hφ]; grind
    have h₂ : φ 0 = N + 1 := by rw [hφ]; grind
    exact h₂ ▸ ComponentCompl.mem_of_hom connected_pathNat.preconnected φ hK
      (h₁ ▸ hmC) 0
  · have : Fact pathNat.Preconnected := ⟨connected_pathNat.preconnected⟩
    exact (nonempty_ends_of_infinite _).to_subtype

end ends
```

A graph can have more than one end. The two-way infinite path `hasse ℤ` has two, one in each direction, and the infinite binary tree has uncountably many, one for each infinite sequence of left and right turns starting at the root. In Exercise 20 we show that the grid `ℤ × ℤ`, by contrast, has exactly one end. Note that in Lean this grid is the box product `hasse ℤ □ hasse ℤ`.

13.6 Exercises

1. Show that a graph with an isolated vertex has no perfect matching.

```
variable {V : Type*} {G : SimpleGraph V}
open Finset

example (v : V) (hv : G.IsIsolated v) :
    ¬∃ M : G.Subgraph, M.IsPerfectMatching := by
  sorry
```

2. Show that the complete graph on a finite type `V` has a perfect matching iff `Fintype.card V` is even. Hint: `IsClique.even_iff_exists_isMatching`.

```
example [Fintype V] :
    (∃ M : (⊤ : SimpleGraph V).Subgraph, M.IsPerfectMatching) ↔
      Even (Fintype.card V) := by
  sorry
```

3. Use Hall's theorem to show that the complete bipartite graph `K[n, n]` has a perfect matching. Hint: `Sum.swap` maps every set of vertices injectively into its neighbourhood.

```
example (n : ℕ) : ∃ M : K[n, n].Subgraph, M.IsPerfectMatching := by
  sorry
```

4. Use Hall's theorem to show that the infinite path `pathNat` has a perfect matching. Hint: `n ↦ n + 1` maps every finite set of vertices injectively into its neighbourhood. What happens for infinite sets?

```
example : ∃ M : pathNat.Subgraph, M.IsPerfectMatching := by
  sorry
```

5. Deduce the two-sided version of Hall's theorem: if every vertex lies in p_1 or p_2, and Hall's condition holds for the subsets of each part, then G has a perfect matching. Why is the hypothesis hV needed? Hint: reduce to `exists_isPerfectMatching_of_forall_ncard_le` by splitting `s` into `s ∩ p₁` and `s ∩ p₂`, whose neighbourhoods are disjoint.

```
example [G.LocallyFinite] {p₁ p₂ : Set V} (h₁ : G.IsBipartiteWith p₁ p₂)
    (hV : p₁ ∪ p₂ = Set.univ)
    (h₂ : ∀ s ⊆ p₁, s.ncard ≤ (⋃ x ∈ s, G.neighborSet x).ncard)
    (h₃ : ∀ s ⊆ p₂, s.ncard ≤ (⋃ x ∈ s, G.neighborSet x).ncard) :
    ∃ M : G.Subgraph, M.IsPerfectMatching := by
  sorry
```

6. Show that a k-regular bipartite graph with `k > 0` has a perfect matching. The graph need not be finite. Hint: check Hall's condition for a finite set `s` by counting the edges between `s` and its neighbourhood in two ways, using `Finset.card_mul_le_card_mul`.

```
theorem IsRegularOfDegree.exists_isPerfectMatching' [G.LocallyFinite] {k : ℕ}
    (hb : G.IsBipartite) (hG : G.IsRegularOfDegree k) (hk : 0 < k) :
    ∃ M : G.Subgraph, M.IsPerfectMatching := by
  sorry
```

7. *Kőnig's line colouring theorem* says that a finite bipartite graph G has chromatic index `χ' G = Δ(G)` (see Edge Colourings (p. 169)). Prove it for regular graphs: the edges of a k-regular bipartite graph can be properly coloured with k colours. (The general case reduces to this one, since every finite bipartite graph is a subgraph of a `Δ(G)`-regular bipartite graph.) Deduce that the hypercube `Q[n]` has chromatic index `n`. Hint: by Exercise 6, if `k > 0` then G has a perfect matching; removing it leaves a `(k - 1)`-regular bipartite graph, so induct on k.

```
theorem IsRegularOfDegree.colorable_lineGraph' [G.LocallyFinite] {k : ℕ}
    (hb : G.IsBipartite) (hG : G.IsRegularOfDegree k) : L[G].Colorable k := by
  sorry

example (n : ℕ) : χ' Q[n] = n := by
  sorry
```

8. Show that the Turán graph `T[n, r]` is r-colourable for `r > 0`. (So by Turán's theorem every Turán-maximal `(r + 1)`-clique-free graph is r-colourable.) Mathlib has this, but prove it directly.

```
example (n r : ℕ) (hr : 0 < r) : T[n, r].Colorable r := by
  sorry
```

9. Deduce from Füredi's theorem that an `(r + 1)`-clique-free graph on n vertices with `turanNumber n r` edges is r-colourable. Hint: then no edges need to be deleted (`card_eq_zero`, `deleteEdges_empty`).

```
example [Fintype V] [DecidableRel G.Adj] {r : ℕ} (h : G.CliqueFree (r + 1))
    (he : #G.edgeFinset = turanNumber (Fintype.card V) r) : G.Colorable r := by
  sorry
```

10. The necessity of the Erdős–Gallai conditions (`erdos_gallai`) is a double-counting argument like those in this chapter. Show that the degrees of any set S of `k` vertices satisfy `∑ v ∈ S, G.degree v ≤ k * (k - 1) + ∑ v ∈ Sᶜ, min k (G.degree v)`. Hint: split each degree into the neighbours in S and in S^c with `degIn_union`, bound the first part by `k - 1`, and count the edges between S and S^c from the other side with `sum_degIn_comm`, as in the proof of Füredi's theorem (p. 419).

```
theorem sum_degree_le_erdosGallai' [Fintype V] [DecidableEq V] [DecidableRel G.Adj]
    (S : Finset V) :
    ∑ v ∈ S, G.degree v ≤ #S * (#S - 1) + ∑ v ∈ Sᶜ, min #S (G.degree v) := by
  sorry
```

11. Deduce that every graphic multiset satisfies the Erdős–Gallai inequalities. (Together with `Graphic.even_sum` this proves that the conditions in `erdos_gallai` are necessary.) Hint: first show that a sub-multiset of `s.map f` is the image of a sub-multiset of s, using `Multiset.coe_le` and `List.sublist_map_iff`. Then, with `L` the sorted list of degrees, realize `L.take k` as the degrees of a set S of `k` vertices, so that `L.drop k` is the list of degrees of S^c.

```
lemma _root_.Multiset.exists_le_map_eq' {α β : Type*} {f : α → β}
    {s : Multiset α} {t : Multiset β} (h : t ≤ s.map f) : ∃ u ≤ s, u.map f = t := by
  sorry

theorem Graphic.erdosGallaiLE' {m : Multiset ℕ} (h : Graphic m) {k : ℕ}
    (hk : k ≤ m.card) : ErdosGallaiLE m k := by
  sorry
```

12. Show that the bound in the Andrásfai–Erdős–Sós theorem cannot be improved for `r = 2`: the blow-up `C[5][t]` with `t ≥ 1` has `n = 5 * t` vertices and is triangle-free with minimum degree `2 * t = 2 * n / 5`, but it is not bipartite.

```
example (t : ℕ) :
    C[5][t + 1].CliqueFree 3 ∧ ¬C[5][t + 1].Colorable 2 ∧
      ∀ v, C[5][t + 1].degree v = 2 * (t + 1) := by
  sorry
```

13. Restate Mantel's theorem as a formula for an extremal number.

```
lemma extremalNumber_completeGraph_three' (n : ℕ) :
    extremalNumber n K[3] = n ^ 2 / 4 := by
  sorry
```

14. Deduce that a graph on `n` vertices with more than `n ^ 2 / 4` edges contains a triangle.

```
example [Fintype V] [DecidableRel G.Adj]
    (h : Fintype.card V ^ 2 / 4 < #G.edgeFinset) : K[3] ⊑ G := by
  sorry
```

15. Use Zykov's theorem to show that a 4-clique-free graph on 6 vertices has at most as many triangles as the octahedron `completeEquipartiteGraph 3 2`. Hint: Mathlib's `completeEquipartiteGraph.turanGraph` is an isomorphism from `completeEquipartiteGraph r t` to `turanGraph (r * t) r`.

```
example [Fintype V] [DecidableEq V] [DecidableRel G.Adj] (hV : Fintype.card V = 6)
    (h : G.CliqueFree 4) :
    #(G.cliqueFinset 3) ≤ #((completeEquipartiteGraph 3 2).cliqueFinset 3) := by
  sorry
```

16. Show that a graph has as many 2-cliques as edges. Hint: `Sym2.toFinset` sends the edge `s(x, y)` to `{x, y}`.

```
lemma card_cliqueFinset_two' [Fintype V] [DecidableEq V] [DecidableRel G.Adj] :
    #(G.cliqueFinset 2) = #G.edgeFinset := by
  sorry
```

17. Deduce the edge bound of Turán's theorem, `CliqueFree.card_edgeFinset_le`, from Zykov's theorem with `s = 2`. Hint: Exercise 16.

```
example [Fintype V] [DecidableRel G.Adj] {r : ℕ}
    (h : G.CliqueFree (r + 1)) : #G.edgeFinset ≤ turanNumber (Fintype.card V) r := by
  sorry
```

18. Deduce the uniqueness part of Turán's theorem from the equality case of Zykov's theorem with `s = 2`: for `2 ≤ r` and `2 ≤ n`, an `(r + 1)`-clique-free graph on `n` vertices with `turanNumber n r` edges is isomorphic to `T[n, r]`. Hint: Exercise 16.

```
example [Fintype V] [DecidableRel G.Adj] {r : ℕ} (h : G.CliqueFree (r + 1))
    (hr : 2 ≤ r) (hn : 2 ≤ Fintype.card V)
    (he : #G.edgeFinset = turanNumber (Fintype.card V) r) :
    Nonempty (G ≃g T[Fintype.card V, r]) := by
  sorry
```

19. Show that a graph containing a ray has an end, by formalizing the argument at the start of Ends (p. 444). Together with `Connected.pathNat_isContained`, this gives another proof of `nonempty_ends_of_infinite`. Hint: a copy `f` of `pathNat` is injective, so for each finite set `K` some tail `n ↦ f (n + N)` avoids `K`, and by `ComponentCompl.mem_of_hom` it lies in a single component of `G - K`. Two such tails share a vertex, so `ComponentCompl.hom_eq_iff_not_disjoint` shows that these components form a direction. Finish with `mem_end_iff`.

```
theorem nonempty_ends_of_pathNat_isContained' (h : pathNat ⊑ G) :
    G.end.Nonempty := by
  sorry
```

20. Show that the grid `hasse ℤ □ hasse ℤ` has exactly one end, as in Ends (p. 444). Hint: if `K` lies in the square `[-N, N] × [-N, N]` then the rows and columns outside the square avoid `K`. Use `ComponentCompl.mem_of_hom` with `boxProdLeft` and `boxProdRight` to show that every infinite component of `G - K` contains `(N + 1, N + 1)`, and apply `subsingleton_end_of_common_vertex`.

```
example : Nat.card (hasse ℤ □ hasse ℤ).end = 1 := by
  sorry
```

Solutions

This chapter collects the solutions to the exercises from each chapter.

Chapter 2: SimpleGraph

1. Show that the edge set of `pathNat` is `{e | ∃ n, e = s(n, n + 1)}`. Hint: use `induction e using Sym2.ind`.

```
example : pathNat.edgeSet = {e | ∃ n, e = s(n, n + 1)} := by
  ext e
  induction e using Sym2.ind with
  | _ x y => simp only [mem_edgeSet, Set.mem_ofPred_eq, Sym2.eq_iff]; grind
```

2. Show that `pathNat` has support `ℕ`. (Remember `ℕ` is a type not a set so we need to use `Set.univ` to talk about `ℕ` as a `Set ℕ`.)

```
example : pathNat.support = Set.univ := by
  apply Set.eq_univ_of_forall
  intro n
  use n + 1
  simp
```

3. Show that the neighbours of `n + 1` in `pathNat` are exactly `n` and `n + 2`.

```
example (n : ℕ) : pathNat.neighborSet (n + 1) = {n, n + 2} := by
  ext a
  simp [or_comm, Eq.comm]
```

4. Show that the darts whose first vertex is `v` are exactly the darts obtained from the neighbours of `v` by `dartOfNeighborSet`.

```
example {G : SimpleGraph V} {v : V} :
    {d : G.Dart | d.fst = v} = Set.range (G.dartOfNeighborSet v) := by
  -- we use ext to prove set equality by proving mutual inclusion
  ext d
  constructor
  · rintro rfl
    -- the rintro rfl uses the hypothesis d.toProd.1 = v to replace
    -- v by d.toProd.1 everywhere, so v has vanished:
```

```
    -- the goal is now d ∈ Set.range (G.dartOfNeighborSet d.toProd.1)
    exact ⟨⟨d.snd, by simp⟩, rfl⟩
  · rintro ⟨a, rfl⟩
    -- this time the hypothesis is G.dartOfNeighborSet v a = d, so rfl replaces
    -- d and it is d that vanishes:
    -- the goal is now G.dartOfNeighborSet v a ∈ {d | d.toProd.1 = v}
    rfl
```

5. Show that $G \le H$ if and only if $H^c \le G^c$.

```
example (G H : SimpleGraph V) : G ≤ H ↔ Hᶜ ≤ Gᶜ := by
  simp
```

6. Show that the complement of the complete graph is the empty graph.

```
example : (⊤ : SimpleGraph V)ᶜ = ⊥ := by
  simp
```

7. For `i : ℕ`, define the graph `shiftedPathNat i` to be a `SimpleGraph ℕ` with edges from `n` to `n + 1` iff `i ≤ n`. (If `aesop_graph` can prove the `symm` and `loopless` fields for your definition then you can simply delete those two lines.)

Notice we don't need to give proofs of `symm` and `loopless`, since `aesop_graph` can prove these. The graph on $\mathbb{N}$ with an edge from `n` to `n + 1` whenever `i ≤ n`.

```
@[simps]
def shiftedPathNat (i : ℕ) : SimpleGraph ℕ where
  Adj n m := n + 1 = m ∧ i ≤ n ∨ m + 1 = n ∧ i ≤ m
```

8. Prove that `shiftedPathNat 0 = pathNat`.

```
example : shiftedPathNat 0 = pathNat := by
  ext u v
  simp
```

9. For `i : ℕ`, define the graph `shortPathNat i` to be a `SimpleGraph ℕ` with edges from `n` to `n + 1` iff `n < i`.

The graph on $\mathbb{N}$ with an edge from `n` to `n + 1` whenever `n < i`.

```
@[simps]
def shortPathNat (i : ℕ) : SimpleGraph ℕ where
  Adj n m := n + 1 = m ∧ n < i ∨ m + 1 = n ∧ m < i
```

10. Prove that `shortPathNat i ⊔ shiftedPathNat i = pathNat`.

```
example (i : ℕ) : shortPathNat i ⊔ shiftedPathNat i = pathNat := by
  ext u v
  grind [shortPathNat_adj, shiftedPathNat_adj]
```

11. Show that the graph `shiftedPathNat j` is a subgraph of `shiftedPathNat i` if and only if $i \le j$.

```
example (i j : ℕ) : shiftedPathNat j ≤ shiftedPathNat i ↔ i ≤ j := by
  refine ⟨fun h => ?_, fun h a b => by grind [shiftedPathNat_adj]⟩
  have := h (by simp : (shiftedPathNat j).Adj j (j + 1))
  grind [shiftedPathNat_adj]
```

12. Prove that `shortPathNat i ⊓ shiftedPathNat i` is the empty graph (on $\mathbb{N}$).

```
example (i : ℕ) : shortPathNat i ⊓ shiftedPathNat i = ⊥ := by
  ext u v
  grind [inf_adj, shortPathNat_adj, shiftedPathNat_adj]
```

13. Prove that `shortPathNat i ⊔ shortPathNat j = shortPathNat (max i j)`.

```
example (i j : ℕ) : shortPathNat i ⊔ shortPathNat j = shortPathNat (max i j) := by
  ext u v
  grind [shortPathNat_adj]
```

14. Show that `pathNat \ shortPathNat i` is `shiftedPathNat i`.

```
example (i : ℕ) : pathNat \ shortPathNat i = shiftedPathNat i := by
  ext u v
  grind [sdiff_adj, shortPathNat_adj, shiftedPathNat_adj]
```

15. `SimpleGraph` has an *indexed supremum* operation `⨆ i, G i` which is the graph with edges between `u` and `v` if and only if there exists an index i such that `u` and `v` are adjacent in `G i`. Prove that `pathNat = ⨆ i : ℕ, shortPathNat i`.

```
example : pathNat = ⨆ i : ℕ, shortPathNat i := by
  ext u v
  simp only [iSup_adj, shortPathNat_adj]
  constructor
  · rintro (h | h)
    · exact ⟨u + 1, by grind⟩
    · exact ⟨v + 1, by grind⟩
  · grind
```

Chapter 3: Constructions

1. Prove that the cycle graph `C[3]` is the complete graph on `Fin 3`. (Mathlib has this as `cycleGraph_three_eq_top`, but try to prove it directly.)

```
example : C[3] = K[3] := by
  ext
  decide +revert
```

2. Prove that the cycle graph `C[n + 2]` is `circulantGraph {1}`. (Mathlib has this, for `C[n + 1]`, as `cycleGraph_eq_circulantGraph`, but prove it directly.)

```
example (n : ℕ) : C[n + 2] = circulantGraph {1} := by
  ext u v
  grind [cycleGraph_adj, circulantGraph_adj]
```

3. Show that the Paley graph on `17` vertices is a circulant graph with the following differences. Hint: first show that the set of squares in `ZMod 17` is `{0, 1, 2, 4, 8, 9, 13, 15, 16}`. After `ext` the goal mentions a new variable of type `ZMod 17`, but `decide` only works on statements with no free variables. Instead use `decide +revert`, which first reverts the variable and then checks the resulting statement `∀ y : ZMod 17, …` case by case. Finally, `circulantGraph_eq_erase_zero` lets you drop `0` from the set of squares.

```
lemma paleyGraph₁₇_eq_circulant :
    paleyGraph (ZMod 17) = circulantGraph {1, 2, 4, 8, 9, 13, 15, 16} := by
  have : {y : ZMod 17 | IsSquare y} = {0, 1, 2, 4, 8, 9, 13, 15, 16} := by
    ext x
    constructor <;> decide +revert
  simp [this, circulantGraph_eq_erase_zero]
```

4. Show that a Cayley graph defined on a commutative additive group is simply a circulant graph. Hint: `addCayley_adj'`, `circulantGraph_adj` and `add_sub_cancel_left` could be useful.

```
example (G : Type*) [AddCommGroup G] (s : Set G) :
    addCayley s = circulantGraph s := by
  ext u v
  rw [addCayley_adj', circulantGraph_adj]
  grind [add_sub_cancel_left]
```

5. Prove that the union (i.e. sup `⊔`) of the complete bipartite graph between `V` and `W` with the disjoint union `⊕g` (i.e. `sum`) of complete graphs on `V` and `W` is the complete graph on `V ⊕ W`.

```
example {V W : Type*} : completeBipartiteGraph V W ⊔ (⊤ ⊕g ⊤) = ⊤ := by
  ext u v
  cases u <;> cases v <;> simp
```

6. Prove that the disjoint union of empty graphs is the empty graph.

```
example {ι : Type*} {V : ι → Type*} : Σg (fun i => (⊥ : SimpleGraph (V i))) = ⊥ := by
  ext ⟨i, u⟩ ⟨j, v⟩
  simp
```

7. Prove that every left copy of a vertex is isolated in the Mycielskian of the empty graph. (So `mμ(⊥)` is a star with centre none whose leaves are the right copies, together with the isolated left copies.)

```
example {V : Type*} (u : V) : mμ(ι).IsIsolated u^l := by
  rintro (_ | (v | v)) <;> simp
```

8. Prove that the complete 1-partite graph with a single class $\mathbb{N}$ is the empty graph. Hint: if `i : Fin 1` then `fin_cases i` splits the goal into one case for each possible value of `i`.

```
example : completeMultipartiteFinNat 1 = ⊥ := by
  ext ⟨i, _⟩ ⟨j, _⟩
  fin_cases i; fin_cases j; simp
```

9. Prove that `sigmaCompleteNat` is the complement of the complete multipartite graph `completeMultipartiteNatFin`. Hint: after `ext ⟨i, a⟩ ⟨j, b⟩`, consider the cases `i = j` and `i ≠ j` separately. In the first case `subst` the equation, so that `a` and `b` have the same type.

```
example : sigmaCompleteNat = completeMultipartiteNatFin^c := by
  ext ⟨i, a⟩ ⟨j, b⟩
  by_cases h : i = j
  · subst h
    simp
  · simp [h]
```

Chapter 4: Decidability, Finiteness and Degrees

1. Give a decidable adjacency instance for the line graph `L[G]` of a graph on a vertex type with decidable equality. Recall that the vertices of `L[G]` are the edges of `G`, and that two of them are adjacent iff they are distinct and share a vertex (`lineGraph_adj_iff_exists`).

Hint: `∃ v, P v` is not decidable in general, but the bounded `∃ v ∈ s, P v` is when `s` is a `Finset` and `P` is decidable. Turn `e₁ : Sym2 V` into a `Finset V` with `Sym2.toFinset`, note that `v ∈ e₂` is decidable (`Sym2.Mem.decidable`), and finish with `decidable_of_iff`. Decidability of the adjacency relation for the line graph.

```
instance [DecidableEq V] : DecidableRel L[G].Adj := fun e₁ e₂ =>
  -- Bound the shared vertex by the Finset of endpoints of e₁
  decidable_of_iff (e₁ ≠ e₂ ∧ ∃ v ∈ (e₁ : Sym2 V).toFinset, v ∈ (e₂ : Sym2 V))
    (by simp [lineGraph_adj_iff_exists])

example : L[C[5]].Adj ⟨s(0, 1), by decide⟩ ⟨s(1, 2), by decide⟩ := by decide
```

2. In the chapter we saw that `bad_card_edgeFinset` demanded more instances than its statement used. Prove that a graph has at most `Fintype.card (Sym2 V)` edges, assuming only the two instances that the statement itself mentions.

```
example [Fintype G.edgeSet] [Fintype (Sym2 V)] :
    G.edgeFinset.card ≤ Fintype.card (Sym2 V) :=
  Finset.card_le_univ _
```

Assuming `[Fintype V] [DecidableRel G.Adj]` would restrict the lemma to callers who obtain `Fintype G.edgeSet` by that particular route, exactly as for `bad_card_edgeFinset`.

3. Show that the edge set of a graph on a finite vertex type is finite, using `Finite` rather than `Fintype`. Note that no `DecidableRel G.Adj` is needed, unlike for `Fintype G.edgeSet`: why not?

```
example [Finite V] : Finite G.edgeSet :=
  -- Sym2 V is finite since V is (it is a quotient of V × V: Quot.finite),
  -- and a subtype of a finite type is finite (Subtype.finite). Being a
  -- proposition, Finite carries no data and so has nothing to compute; the
  -- decidability was only ever needed to *filter* Finset.univ down to the edges.
  inferInstance
```

4. The chapter showed `G.edgeSet.ncard = G.edgeFinset.card`. Prove the $\mathbb{N}\infty$ version.

```
example [Fintype G.edgeSet] : G.edgeSet.encard = G.edgeFinset.card := by
  rw [← coe_edgeFinset, Set.encard_coe_eq_coe_finsetCard]
```

5. The infinite path `pathNat` has infinitely many edges. Show that the $\mathbb{N}\infty$-valued `encard` of its edge set is therefore `⊤`, while the $\mathbb{N}$-valued `ncard` returns the junk value `0`. Hint: the edges `s(n, n + 1)` are distinct for distinct `n`, so `Set.infinite_of_injective_forall_mem` shows that the edge set is infinite.

```
example : pathNat.edgeSet.encard = ⊤ ∧ pathNat.edgeSet.ncard = 0 := by
  have h : pathNat.edgeSet.Infinite :=
    Set.infinite_of_injective_forall_mem (f := fun n : ℕ => s(n, n + 1))
      (fun a b hab => by grind [Sym2.eq_iff]) (fun n => by simp)
  exact ⟨h.encard_eq, h.ncard⟩
```

6. The degree of a vertex is the `Nat.card` of its neighbourhood.

```
example (v : V) [Fintype (G.neighborSet v)] :
    Nat.card (G.neighborSet v) = G.degree v := by
  rw [Nat.card_eq_fintype_card, card_neighborSet_eq_degree]
```

7. In the complete graph every vertex is adjacent to every other vertex: its `neighborFinset` is `Finset.univ` with the vertex itself removed.

```
example (n : ℕ) (v : Fin n) : K[n].neighborFinset v = Finset.univ.erase v := by
  ext w; simp [ne_comm]
```

8. If `G` is locally finite there is a vertex of degree at least 3 iff `∃ a b c d, G.Adj a b ∧ G.Adj a c ∧ G.Adj a d ∧ b ≠ c ∧ b ≠ d ∧ c ≠ d`.

```
lemma exists_three_le_degree_iff [G.LocallyFinite] : (∃ v, 3 ≤ G.degree v) ↔
    ∃ a b c d, G.Adj a b ∧ G.Adj a c ∧ G.Adj a d ∧ b ≠ c ∧ b ≠ d ∧ c ≠ d := by
  simp only [Nat.succ_le_iff, ← card_neighborFinset_eq_degree,
             Finset.two_lt_card_iff, mem_neighborFinset]
```

9. We saw that a vertex `w` of the n-uniform blow-up `G[n]` has a finite neighbourhood whenever `w.fst` has a finite neighbourhood in `G`. In this exercise we generalize this to blow-ups whose classes may have different sizes and need not all be finite: it is enough that the classes of the neighbours of `w.fst` are finite. As with `blowUpNFintypeNeighborSet`, we make this a `def` rather than an `instance`.

A vertex `w` of a blow-up of `H` has a finite neighbourhood if `w.fst` has a finite neighbourhood in `H` and the classes of its neighbours are finite.

```
@[instance_reducible]
def blowUpGraphFintypeNeighborSet {ι : Type*} (H : SimpleGraph ι) (V : ι → Type*)
    (w : Σ i, V i) [Fintype (H.neighborSet w.fst)]
    [∀ k : H.neighborSet w.fst, Fintype (V k)] :
    Fintype ((H.blowUpGraph V).neighborSet w) :=
  Fintype.ofEquiv (Σ k : H.neighborSet w.fst, V k)
    { toFun := fun ⟨⟨k, hk⟩, y⟩ => ⟨⟨k, y⟩, hk⟩
      invFun := fun ⟨⟨k, y⟩, hadj⟩ => ⟨⟨k, hadj⟩, y⟩}
```

10. The infinite path `pathNat` has no minimum or maximum degree in the sense of `minDegree` and `maxDegree`, since its vertex type is infinite. Show instead that `1` is the least, and `2` the greatest, degree of a vertex of `pathNat`.

```
example : IsLeast {d | ∃ v, pathNat.degree v = d} 1 ∧
    IsGreatest {d | ∃ v, pathNat.degree v = d} 2 := by
  -- the degrees 1 and 2 are attained at the vertices 0 and 1
  refine ⟨⟨0, by simp⟩, ?_⟩, ⟨1, by simp⟩, ?_⟩⟩
  all_goals
    rintro _ ⟨v, rfl⟩
    simp only [degree_pathNat]
    split_ifs <;> simp
```

11. Show that `k` times the number of vertices of degree at least `k` is at most twice the number of edges. (The set of such vertices is a `Finset` written with the set-builder notation of Set-Builder Notation (p. 61). Compare it with the handshake lemma `sum_degrees_eq_twice_card_edges'`.)

```
example [Fintype V] [Fintype G.edgeSet] [G.LocallyFinite] (k : ℕ) :
    k * ({v | k ≤ G.degree v} : Finset V).card ≤ 2 * G.edgeFinset.card := by
  rw [← sum_degrees_eq_twice_card_edges']
  calc k * ({v | k ≤ G.degree v} : Finset V).card
      = ∑ v ∈ ({v | k ≤ G.degree v} : Finset V), k := by simp [mul_comm]
    _ ≤ ∑ v ∈ ({v | k ≤ G.degree v} : Finset V), G.degree v :=
        Finset.sum_le_sum fun v hv => by simpa using hv
    _ ≤ ∑ v, G.degree v := Finset.sum_le_sum_of_subset (Finset.subset_univ _)
```

12. A graph on an odd number of vertices that is regular of degree `d` must have `d` even. Hint: use `IsRegularOfDegree.two_mul_card_edgeFinset`. The statement does not mention the edge set, so get the `Fintype G.edgeSet` instance that the proof needs from `classical`.

```
example [Fintype V] [G.LocallyFinite] {d : ℕ} (h : G.IsRegularOfDegree d)
    (hV : Odd (Fintype.card V)) : Even d := by
  classical
  have h2 : Even (Fintype.card V * d) := h.two_mul_card_edgeFinset ▸ even_two_mul _
  exact (Nat.even_mul.1 h2).resolve_left (Nat.not_even_iff_odd.2 hV)
```

13. Prove that the complete graph on `Fin (n + 1)` has `n * (n + 1) / 2` edges.

```
example (n : ℕ) : K[n + 1].edgeFinset.card = n * (n + 1) / 2 := by
  rw [card_edgeFinset_top_eq_card_choose_two, Fintype.card_fin, Nat.choose_two_right,
      Nat.add_sub_cancel_right, mul_comm]
```

14. Show that in any finite graph `G.edgeFinset.card ≤ Fintype.card V * (Fintype.card V - 1) / 2`.

```
example [Fintype V] [Fintype G.edgeSet] :
    G.edgeFinset.card ≤ Fintype.card V * (Fintype.card V - 1) / 2 :=
  G.card_edgeFinset_le_card_choose_two.trans (Nat.choose_two_right _).le
```

15. The complement of the hypercube `Q[n]` is regular of degree `2 ^ n - 1 - n`. Hint: use `IsRegularOfDegree.compl`, `IsRegularOfDegree.hypercube` and `card_hypercubeVert`.

```
example (n : ℕ) : Q[n]ᶜ.IsRegularOfDegree (2 ^ n - 1 - n) := by
  have := (IsRegularOfDegree.hypercube n).compl
  rwa [card_hypercubeVert] at this
```

16. Prove that if `V` is finite then the `n`-th iterated Mycielskian has `2 ^ n * Fintype.card V + 2 ^ n - 1` vertices.

```
lemma card_mycielskianVert (n : ℕ) [Fintype V] :
    Fintype.card mμᵥ^n(V) = 2 ^ n * Fintype.card V + 2 ^ n - 1 := by
  induction n with
  | zero => unfold MycielskianVert; grind
  | succ n ih =>
    unfold MycielskianVert
    grind [card_mycielskianVert_one, Nat.mul_sub_left_distrib, mul_add,
           ← mul_assoc, ← pow_succ']
```

17. The degree of the new vertex `none` in the Mycielskian is the number of vertices in the original graph.

```
@[simp]
lemma degree_mycielskian_none [Fintype V] [Fintype (mμ(G).neighborSet none)] :
    mμ(G).degree none = Fintype.card V := by
  classical
  have hnb : mμ(G).neighborFinset none = (univ : Finset V).image (·ᴿ) := by
    ext x; simp [← mem_neighborSet]
  rw [← card_neighborFinset_eq_degree, hnb,
    card_image_of_injective _ mycielskian.right_injective, card_univ]
```

18. The degree of the right copy of a vertex in the Mycielskian is one more than the degree of the vertex in the original graph.

```
@[simp]
lemma degree_mycielskian_right (v : V) [Fintype (G.neighborSet v)]
    [Fintype (mμ(G).neighborSet v^r)] :
    mμ(G).degree v^r = G.degree v + 1 := by
  classical
  have hnb :
      mμ(G).neighborFinset v^r = (G.neighborFinset v).image (·^l) ∪ {none} := by
    ext x; simp [← mem_neighborSet]
  rw [← card_neighborFinset_eq_degree, ← card_neighborFinset_eq_degree, hnb,
    card_union_of_disjoint (by simp [← Finset.disjoint_coe]),
    card_image_of_injective _ mycielskian.left_injective, card_singleton]
```

19. The number of edges in `mμ(G)` is `3 * G.edgeFinset.card + Fintype.card V`. Hint: use `sum_degrees_eq_twice_card_edges'`.

```
@[simp]
lemma card_edgeFinset_mycielskian [Fintype V] [Fintype G.edgeSet]
    [Fintype mμ(G).edgeSet] :
    #mμ(G).edgeFinset = 3 * #G.edgeFinset + Fintype.card V := by
  classical
  apply (Nat.mul_right_inj two_ne_zero).1
  simp_rw [← sum_degrees_eq_twice_card_edges' mμ(G), Fintype.sum_option,
    degree_mycielskian_none, Fintype.sum_sum_type, degree_mycielskian_left,
    ← mul_sum, degree_mycielskian_right, sum_add_distrib,
    sum_degrees_eq_twice_card_edges' G, sum_const, card_univ, smul_eq_mul, mul_one]
  ring
```

20. The maximum degree of `mμ(G)` is the maximum of `2 * Δ(G)` and the number of vertices in `G`. Hint: `exists_maximal_degree_vertex`, after treating `IsEmpty V` separately.

```
lemma maxDegree_mycielskian [Fintype V] [DecidableRel G.Adj] :
    Δ(mμ(G)) = max (2 * Δ(G)) (Fintype.card V) := by
  apply le_antisymm
  · apply maxDegree_le_of_forall_degree_le
    intro v
    rcases v with none | (v | v)
    · simp
    · rw [degree_mycielskian_left]
      exact (Nat.mul_le_mul_left 2 (G.degree_le_maxDegree v)).trans (le_max_left _ _)
    · rw [degree_mycielskian_right]
      exact (Nat.succ_le_of_lt (G.degree_lt_card_verts v)).trans (le_max_right _ _)
  · cases isEmpty_or_nonempty V
    · simp
    · apply max_le
      · obtain ⟨v, hv⟩ := G.exists_maximal_degree_vertex
        rw [hv, ← degree_mycielskian_left]
        exact mμ(G).degree_le_maxDegree v^l
      · rw [← G.degree_mycielskian_none]
        exact mμ(G).degree_le_maxDegree none
```

21. The minimum degree of `mμ(G)` is the minimum of `2 * δ(G)` and `δ(G) + 1`. Deduce that `δ(mμ(G)) = δ(G) + 1` whenever `δ(G) ≥ 1`. Hint: `exists_minimal_degree_vertex`, after treating `IsEmpty V` separately.

```
lemma minDegree_mycielskian [Fintype V] [DecidableRel G.Adj] :
    δ(mμ(G)) = min (2 * δ(G)) (δ(G) + 1) := by
  obtain hV | hV := isEmpty_or_nonempty V
  · simp [minDegree]
  apply le_antisymm
  · obtain ⟨v, hv⟩ := G.exists_minimal_degree_vertex
    apply le_min
    · rw [hv, ← degree_mycielskian_left]
      exact mμ(G).minDegree_le_degree v.l
    · rw [hv, ← degree_mycielskian_right]
      exact mμ(G).minDegree_le_degree v.r
  · apply le_minDegree_of_forall_le_degree
    intro v
    rcases v with none | (v | v)
    · rw [degree_mycielskian_none]
      exact (min_le_right _ _).trans (Nat.succ_le_of_lt G.minDegree_lt_card)
    · rw [degree_mycielskian_left]
      exact (min_le_left _ _).trans (Nat.mul_le_mul_left 2 (G.minDegree_le_degree v))
    · rw [degree_mycielskian_right]
      -- min_le_right .. is min_le_right _ _: .. fills in all the remaining
      -- explicit arguments with _
      exact (min_le_right ..).trans
        (Nat.add_le_add_right (G.minDegree_le_degree v) 1)

lemma minDegree_mycielskian_of_one_le [Fintype V] [DecidableRel G.Adj]
    (h : 1 ≤ δ(G)) : δ(mμ(G)) = δ(G) + 1 := by
  grind [minDegree_mycielskian]
```

22. Every graphic multiset has an even sum. (This shows that the parity condition in the Erdős–Gallai theorem is necessary.)

```
theorem Graphic.even_sum {m : Multiset ℕ} (h : Graphic m) : Even m.sum := by
  obtain ⟨V, _, G, _, rfl⟩ := h
  -- the sum of the degree multiset is, by definition, the sum of the degrees
  change Even (∑ v, G.degree v)
  rw [sum_degrees_eq_twice_card_edges]
  exact even_two_mul _
```

23. In the chapter we computed the degree of the new vertex of `coneGraph H s`. Show that an old vertex's degree goes up by one exactly when it lies in `s`.

```
@[simp]
theorem degree_coneGraph_some [DecidableEq V] (H : SimpleGraph V) (s : Finset V)
    (w : V) [Fintype (H.neighborSet w)]
    [Fintype ((coneGraph H s).neighborSet (some w))] :
    (coneGraph H s).degree (some w) = H.degree w + (if w ∈ s then 1 else 0) := by
  rw [← card_neighborFinset_eq_degree]
  have hset : (coneGraph H s).neighborFinset (some w) =
```

```
    (H.neighborFinset w).map Function.Embedding.some ∪
      (if w ∈ s then {none} else ∅) := by
    ext x
    cases x <;> by_cases hw : w ∈ s <;> simp [hw]
  rw [hset]
  split_ifs with h <;> simp
```

24. The `degreeMultiset` of the cone: the new vertex contributes `#s`, and every old vertex has its old degree plus one when it lies in `s`. Hint: the stub below records that `Finset.univ` on `Option V` is none followed by the image of `Finset.univ` under `some`. This holds by `rfl`.

```
theorem degreeMultiset_coneGraph [Fintype V] [DecidableEq V] (H : SimpleGraph V)
    [DecidableRel H.Adj] (s : Finset V) :
    (coneGraph H s).degreeMultiset = #s ::ₘ (Finset.univ : Finset V).val.map
      (fun w => H.degree w + if w ∈ s then 1 else 0) := by
  -- Finset.univ on Option V is by definition none followed by the image of
  -- Finset.univ under some
  have h : (Finset.univ : Finset (Option V)).val =
      none ::ₘ (Finset.univ : Finset V).val.map some := rfl
  unfold degreeMultiset
  rw [h]
  simp
```

25. The `degreeMultiset` of `mμ(G)` is the multiset of degrees of `G` with each degree doubled, together with the degrees of the new vertices. Hint: the stub below records how `Finset.univ` unfolds on `Option (V ⊕ V)` and on `V ⊕ V`. Both facts hold by `rfl`, as in Exercise 24.

```
lemma degreeMultiset_mycielskian [Fintype V] [DecidableRel G.Adj] :
    mμ(G).degreeMultiset = {Fintype.card V} + G.degreeMultiset.map (2 * ·)
                         + G.degreeMultiset.map (· + 1) := by
  simp only [degreeMultiset]
  -- Split the universe of Option (V ⊕ V) into the new vertex none, the left
  -- copies and the right copies.
  have h1 : (Finset.univ : Finset (Option (V ⊕ V))).val =
        none ::ₘ Multiset.map Option.some (Finset.univ : Finset (V ⊕ V)).val := rfl
  have h2 : (Finset.univ : Finset (V ⊕ V)).val = Multiset.map Sum.inl
              Finset.univ.val + Multiset.map Sum.inr Finset.univ.val := rfl
  rw [h1, h2]
  simp_rw [Multiset.map_cons, Multiset.map_add, Multiset.map_map, Function.comp,
    degree_mycielskian_none, degree_mycielskian_left, degree_mycielskian_right]
  grind [← Multiset.singleton_add]
```

Chapter 5: Maps

1. Show that there is no graph homomorphism from `K[n + 1]` to `K[n]` (i.e. `K[n + 1]` is not `n`-colourable).

```
example (n : ℕ) : IsEmpty (K[n + 1] →g K[n]) :=
  ⟨fun f => by simpa using Fintype.card_le_of_injective f f.injective_of_top_hom⟩
```

2. Define an embedding of `P[n]` into `P[n + k]` so that the image is exactly the last n vertices.

The embedding of `P[n]` into `P[n + k]` as the last n vertices.

```
@[simps]
def pathGraph.embedding (n k : ℕ) : P[n] ↪g P[n + k] where
  toFun i := i.addNat k
  inj' := (Fin.addNatEmb k).injective
  map_rel_iff' := by
    grind [Function.Embedding.coeFn_mk, pathGraph_adj]

#check pathGraph.embedding_apply
```

3. Show that any $G \hookrightarrow g\ H$ gives rise to an embedding of G into any blow-up of H with nonempty parts.

An embedding $e : G \hookrightarrow g\ H$ gives an embedding of G into any blow-up of H with nonempty parts: compose e with `Embedding.blowUp`.

```
def Embedding.toBlowUp {G : SimpleGraph V} {ι : Type*} {H : SimpleGraph ι}
    (e : G ↪g H) (U : ι → Type*) (f : ∀ (i : ι), U i) :
    G ↪g H.blowUpGraph U :=
  (Embedding.blowUp H U f).comp e
```

4. Show that if $G \le H$ are graphs on the same vertex type then $G \sqsubseteq H$. (The homomorphism `Hom.ofLE h` is the identity on vertices, so it is injective. Mathlib calls this result `IsContained.of_le`.)

```
example {G H : SimpleGraph V} (h : G ≤ H) : G ⊑ H :=
  ⟨(Hom.ofLE h).toCopy Function.injective_id⟩
```

5. Show that if $G \simeq g\ H$ then $G^c \simeq g\ H^c$.

```
example (G : SimpleGraph V) (H : SimpleGraph W) (f : G ≃g H) : Gᶜ ≃g Hᶜ :=
  ⟨f, by simp [f.map_adj_iff, f.injective.eq_iff]⟩
```

6. Show that a graph isomorphic to $\bot$ is itself $\bot$.

```
example (G : SimpleGraph V) (f : G ≃g (⊥ : SimpleGraph W)) : G = ⊥ := by
  ext
  simp [← f.map_adj_iff]
```

7. Show that the path graph `P[4]` is *self-complementary*, i.e. isomorphic to its own complement. Hint: find a suitable permutation of the vertices; you can define it and its inverse using `Matrix` vector notation `![_, _, _, _]`.

```
example : P[4] ≃g P[4]ᶜ where
  toFun := ![1, 3, 0, 2]
  invFun := ![2, 0, 3, 1]
  left_inv := by decide
  right_inv := by decide
  map_rel_iff' := by decide
```

8. Show that the cycle `C[5]` is also self-complementary. Hint: try `i ↦ 2 * i`. (Multiplication by 2 sends the edges `{i, i + 1}` of `C[5]` to the pairs `{2 * i, 2 * i + 2}`, which are exactly the edges of the complement; its inverse is multiplication by 3.)

```
example : C[5] ≃g C[5]ᶜ where
  toFun := ![0, 2, 4, 1, 3]
  invFun := ![0, 3, 1, 4, 2]
  left_inv := by decide
  right_inv := by decide
  map_rel_iff' := by decide
```

9. We saw that for an embedding `f : V ↪ W` we have `(G.map f).comap f = G`, but only `(H.comap f).map f ≤ H`. Show that this inequality can be strict. (It is not an equality because any edge of `H` touching a vertex outside the range of `f` is lost by the comap. Here we embed `Fin 1` into `Fin 2` and lose the only edge of `⊤`.)

```
example : ∃ (f : Fin 1 ↪ Fin 2) (H : SimpleGraph (Fin 2)),
    (H.comap f).map f ≠ H := by
  refine ⟨Fin.castSuccEmb, ⊤, fun h => ?_⟩
  have h01 : (⊤ : SimpleGraph (Fin 2)).Adj 0 1 := by decide
  rw [← h, map_adj] at h01
  -- an edge of the comap is an edge of a graph on Fin 1, which has no edges
  obtain ⟨u, v, huv, -⟩ := h01
  exact huv.ne (Subsingleton.elim u v)
```

10. Define an embedding of `Q[n]` into `Q[n + k]` by iterating `hypercube.embedFalse`.

The embedding `Q[n] ↪g Q[n + k]` obtained by iterating `hypercube.embedFalse`.

```
def hypercube.embedding (n : ℕ) : (k : ℕ) → Q[n] ↪g Q[n + k]
  | 0 => Embedding.refl
  | k + 1 => (hypercube.embedFalse (n + k)).comp (hypercube.embedding n k)
```

The next question asks for an isomorphism `Q[2] ≃g C[4]`. In Algebraic Graph Theory (p. 360) we use it to identify the automorphism group of the 2-dimensional hypercube.

11. Show that `Q[2]` is isomorphic to the cycle `C[4]`. Both graphs are concrete finite graphs with decidable adjacency (`DecidableRel Q[n].Adj` is `hypercubeDecidableRelAdj`, and Mathlib provides `instDecidableRelFinAdjCycleGraph` for `C[n]`), so every proof obligation can be checked by `decide`. Recall also that `v[i, j]` where `i, j ∈ {0, 1}` is scoped notation for the vertices of `Q[2]`.

The Gray-code isomorphism `Q[2] ≃g C[4]`, sending the vertices `00, 10, 11, 01` to `0, 1, 2, 3`.

```
def hypercubeTwoIsoCycleFour : Q[2] ≃g C[4] where
  toFun v := match v with
    | v[0, 0] => 0
    | v[1, 0] => 1
    | v[1, 1] => 2
    | v[0, 1] => 3
  invFun := ![v[0, 0], v[1, 0], v[1, 1], v[0, 1]]
  left_inv := by decide
  right_inv := by decide
  map_rel_iff' := by decide
```

The next two exercises are about vertex-transitive graphs.

12. Show that if G is vertex-transitive then so is its complement G^c, and deduce that the empty graph $\bot$ is vertex-transitive. Hint: as in Exercise 5, an automorphism of G is also an automorphism of G^c; and `compl_top` says $\top^c = \bot$.

```
lemma IsVertexTransitive.compl {G : SimpleGraph V} (h : G.IsVertexTransitive) :
    Gᶜ.IsVertexTransitive := by
  intro u v
  obtain ⟨f, hf⟩ := h u v
  -- f preserves adjacency and distinctness, so it also preserves non-adjacency
  exact ⟨⟨f, by simp [f.map_adj_iff, f.injective.eq_iff]⟩, hf⟩

lemma isVertexTransitive_bot : (⊥ : SimpleGraph V).IsVertexTransitive := by
  simpa using (isVertexTransitive_top (V := V)).compl
```

13. Our characterization `isVertexTransitive_mycielskian_iff` of the vertex-transitive Mycielskians assumes that V is nonempty. Show that this assumption is needed: if V is empty then the Mycielskian of any graph on V is vertex-transitive, but the graph is not isomorphic to `K[2]`.

Hint: if V is empty then the Mycielskian has the single vertex `none`.

```
lemma isVertexTransitive_mycielskian_of_isEmpty [IsEmpty V] (G : SimpleGraph V) :
    mμ(G).IsVertexTransitive :=
  fun _ _ => ⟨.refl, Subsingleton.elim _ _⟩

lemma isEmpty_iso_completeGraph_two_of_isEmpty [IsEmpty V] (G : SimpleGraph V) :
    IsEmpty (G ≃g K[2]) :=
  ⟨fun e => isEmptyElim (e.symm 0)⟩
```

Exercises 14–18 are about the indexed tensor product `⊗g i, G i` (`piTensorProd`) from Indexed Tensor Products (p. 117), in the starred section on tensor products.

14. Show that for each index j the projection out of the indexed tensor product `⊗g i, G i` is a graph homomorphism to $G\ j$.

```
example {ι : Type*} {U : ι → Type*} (G : ∀ i, SimpleGraph (U i)) (j : ι) :
    ⊗g i, G i →g G j where
  toFun f := f j
  map_rel' h := (piTensorProd_adj.1 h).1 j
```

15. The universal property of the indexed tensor product. Given graph homomorphisms from K to each G i, there is a homomorphism from K to the indexed tensor product ⊗g i, G i (provided ι is nonempty, which here follows from the existence of i_0; the next exercise looks at the case when ι is empty).

```
example {ι : Type*} {U : ι → Type*} (i₀ : ι) {K : SimpleGraph X}
    (G : ∀ i, SimpleGraph (U i)) (f : ∀ i, K →g G i) : K →g ⊗g i, G i where
  toFun x i := f i x
  map_rel' h := by
    have : Nonempty ι := ⟨i₀⟩
    rw [piTensorProd_adj_of_nonempty]
    exact fun i => (f i).map_adj h
```

16. Prove that if ι is empty then there is a graph homomorphism from a graph K to the indexed tensor product ⊗g i, G i iff K is the empty graph ⊥.

```
lemma nonempty_hom_piTensorProd_iff_of_isEmpty {ι : Type*} [IsEmpty ι]
    {U : ι → Type*} {K : SimpleGraph X} (G : ∀ i, SimpleGraph (U i)) :
    Nonempty (K →g ⊗g i, G i) ↔ K = ⊥ := by
  constructor
  · rintro ⟨f⟩
    ext a b
    simp only [bot_adj, iff_false]
    intro h
    apply (f.map_adj h).ne
    -- The goal is now ⊢ f a = f b
    -- Note that f.toFun is a map from X to the vertex type
    -- of the indexed tensor product
    have _ : X → (i : ι) → U i := f.toFun
    -- So f a and f b are both elements of (i : ι) → U i.
    have _ : (i : ι) → U i := f a
    have _ : (i : ι) → U i := f b
    -- Since ι is empty, (i : ι) → U i contains a unique element
    -- (you can define it by fun i => isEmptyElim i)
    have : Unique ((i : ι) → U i) := inferInstance
    -- Lean has the Subsingleton type class for types with at most one element.
    have : Subsingleton ((i : ι) → U i) := inferInstance
    -- So f a = f b since they are both elements of a Subsingleton type.
    exact Subsingleton.elim _ _
    -- Note: none of the have statements above are needed; they are for exposition.
  · rintro rfl
    -- We need to show that there is a graph homomorphism from ⊥ to the indexed
    -- tensor product. Since ι is empty there is only one choice for the
    -- toFun. Since ⊥ has no edges, the proof of map_rel' is trivial.
    exact ⟨{ toFun := fun _ i => isEmptyElim i, map_rel' := fun h => h.elim }⟩
```

17. The degree of f in the indexed tensor product is the product of the degrees of its components. Hint: the neighbour set is the `Fintype.piFinset` of the componentwise neighbour sets, and (using [`Nonempty ι`]) the ≠ condition in the adjacency is automatic.

```
example {ι : Type*} {U : ι → Type*} {G : (i : ι) → SimpleGraph (U i)}
    {f : (i : ι) → U i} [Fintype ι] [Nonempty ι]
```

```
    [∀ i, Fintype ((G i).neighborSet (f i))] [Fintype ((⊗g i, G i).neighborSet f)] :
    (⊗g i, G i).degree f = ∏ i, (G i).degree (f i) := by
  classical
  simp_rw [← card_neighborFinset_eq_degree, ← Fintype.card_piFinset]
  congr 1
  ext g
  simp
```

18. Given graph homomorphisms from K to each G `i`, one of which, $f\ i_0$, is a graph embedding, we have a graph embedding of K into the indexed tensor product. (We say that $f\ i_0$ is a graph embedding by giving both additional properties that allow the original homomorphism to be upgraded: injectivity *inj* and adjacency reflection *reflect*.)

```
example {ι : Type*} {U : ι → Type*} (i₀ : ι) {K : SimpleGraph X}
    (G : ∀ i, SimpleGraph (U i)) (f : ∀ i, K →g G i)
    (inj : Function.Injective (f i₀))
    (reflect : ∀ {x y}, (G i₀).Adj (f i₀ x) (f i₀ y) → K.Adj x y) :
    K ↪g ⊗g i, G i where
  toFun x i := f i x
  inj' _ _ h := inj (congrFun h i₀)
  map_rel_iff' := by
    have : Nonempty ι := ⟨i₀⟩
    intro x y
    rw [piTensorProd_adj_of_nonempty]
    exact ⟨fun h => reflect (h i₀), fun hk i => (f i).map_adj hk⟩
```

The last exercise is about `ClawFree`.

19. Show that `K[4]` is claw-free, but contains a non-induced copy of the claw `K[1, 3]`. (An induced claw would need two of its leaves to be non-adjacent, but any two distinct vertices of `K[4]` are adjacent. On the other hand vertex `0` and its three neighbours `1, 2, 3` give a copy of the claw.)

```
example : K[4].ClawFree ∧ K[1, 3] ⊑ K[4] := by
  refine ⟨fun ⟨f⟩ => ?_, ?_⟩
  · -- the images of two leaves are distinct, hence adjacent in K[4]
    have h : K[4].Adj (f (.inr 0)) (f (.inr 1)) := by simp
    -- but f reflects adjacency, and the two leaves are not adjacent in the claw
    simp [f.map_adj_iff] at h
  · apply claw_isContained (a := 0) (b := 1) (c := 2) (d := 3) <;> decide
```

Chapter 6: Cliques

1. Show that any finite subset t of a clique s is a $\#t$-clique.

```
example {s : Set V} {t : Finset V} (hc : G.IsClique s) (h : ↑t ⊆ s) :
    G.IsNClique #t t :=
  ⟨hc.subset h, rfl⟩
```

2. Show that `{0, 1, 2}` is a triangle in the Paley graph on 17 vertices.

```
example : (paleyGraph (ZMod 17)).IsNClique 3 {0, 1, 2} := by decide
```

3. Show that the Paley graph on 17 vertices is 4-clique-free. Hint: `decide` will probably time out, so use `native_decide` instead.

```
lemma cliqueFree_paleyGraph_seventeen_four :
    (paleyGraph (ZMod 17)).CliqueFree 4 := by
  native_decide
```

4. Show that the clique number of finite graphs is monotone: if $G \le H$ then $\omega(G) \le \omega(H)$. (There is a one-line proof using a lemma from the chapter, and a direct proof using `exists_isNClique_cliqueNum`.)

```
lemma cliqueNum_mono [Finite V] {H : SimpleGraph V} (h : G ≤ H) : ω G ≤ ω H :=
  -- G ≤ H gives a homomorphism Hom.ofLE h : G →g H, so this is a special case of
  -- Hom.cliqueNum_le. Alternatively, directly: a clique s in G achieving the
  -- clique number of G is also a clique in H, so its size is at most ω H:
  -- obtain ⟨s, hs⟩ := G.exists_isNClique_cliqueNum; exact (hs.mono h).le_cliqueNum
  (Hom.ofLE h).cliqueNum_le
```

5. Show that the clique number of the Paley graph on 17 vertices is 3. (Recall that we have `IsNClique.cliqueNum_eq_of_cliqueFree`, so we just need to give an example of a 3-clique and show that the graph is 4-clique-free.)

```
example : ω (paleyGraph (ZMod 17)) = 3 := by
  -- the Paley graph on 17 vertices is 4-clique-free (Exercise 3)
  have h4 : (paleyGraph (ZMod 17)).CliqueFree 4 :=
    cliqueFree_paleyGraph_seventeen_four
  -- and it contains a triangle (Exercise 2)
  have h3 : (paleyGraph (ZMod 17)).IsNClique 3 {0, 1, 2} := by decide
  exact h3.cliqueNum_eq_of_cliqueFree h4
```

In the next few exercises we look at the clique number of graph products starting with the tensor product $G \otimes g\ H$. Recall that two vertices $(v_1, w_1), (v_2, w_2)$ of the tensor product are adjacent if and only if `G.Adj v₁ v₂` and `H.Adj w₁ w₂`.

6. The first projection of a clique in the tensor product $G \otimes g\ H$ is a clique in G.

```
example {W : Type*} {H : SimpleGraph W} {s : Set (V × W)}
    (h : (G ⊗g H).IsClique s) : G.IsClique (Prod.fst '' s) := by
  -- take two distinct vertices in the image, lifted to (v₁, w₁), (v₂, w₂) ∈ s
  rintro _ ⟨⟨v₁, w₁⟩, hp, rfl⟩ _ ⟨⟨v₂, w₂⟩, hq, rfl⟩ hne
  -- the lifts are distinct, so adjacent; adjacency in G ⊗g H gives G.Adj v₁ v₂
  exact (h hp hq (fun he => hne (congrArg Prod.fst he))).1
```

7. If G is n-clique-free then so is its tensor product $G \otimes g\ H$ with any graph H.

```
lemma CliqueFree.tensorProd_left {W : Type*} {H : SimpleGraph W} {n : ℕ}
    (h : G.CliqueFree n) : (G ⊗g H).CliqueFree n :=
  -- unlike the box product, the first projection is a homomorphism G ⊗g H →g G
  -- (tensorProd.fstHom), so it maps any n-clique to an n-clique of G
  h.of_hom tensorProd.fstHom
```

8. The j-th projection of a clique in the indexed tensor product `⊗g i, G i` is a clique in `G j`.

```
example {ι : Type*} {U : ι → Type*} {G : (i : ι) → SimpleGraph (U i)} (j : ι)
    {s : Set ((i : ι) → U i)} (h : (⊗g i, G i).IsClique s) :
    (G j).IsClique ((· j) '' s) := by
  rintro _ ⟨f, hf, rfl⟩ _ ⟨g, hg, rfl⟩ hne
  -- f and g differ at j, so are distinct, hence adjacent in every coordinate
  have hadj := h hf hg (fun he => hne (congrFun he j))
  rw [piTensorProd_adj] at hadj
  exact hadj.1 j
```

9. The clique number of `G ⊗g H` is `min (ω G) (ω H)` (for finite graphs). Hint: for the lower bound, let `m = min (ω G) (ω H)` and use `not_cliqueFree_iff_top_isContained` to find copies of `K[m]` in both G and H. Pair them with `tensorProd.prod` to get a homomorphism `K[m] →g G ⊗g H`.

```
lemma cliqueNum_tensorProd {W : Type*} [Finite V] [Finite W] {H : SimpleGraph W} :
    ω (G ⊗g H) = min (ω G) (ω H) := by
  classical
  apply le_antisymm
  · -- upper bound: both projections tensorProd.fstHom, tensorProd.sndHom are
    -- homomorphisms, so the clique number cannot exceed either factor's
    exact le_min tensorProd.fstHom.cliqueNum_le tensorProd.sndHom.cliqueNum_le
  · -- lower bound: with m := min (ω G) (ω H), neither G nor H is
    -- m-clique-free (each has a clique of size at least m)
    set m := min (ω G) (ω H)
    obtain ⟨s, hs⟩ := G.exists_isNClique_cliqueNum
    obtain ⟨t, ht⟩ := H.exists_isNClique_cliqueNum
    have hG : ¬G.CliqueFree m := fun hf => hf.mono (min_le_left ..) s hs
    have hH : ¬H.CliqueFree m := fun hf => hf.mono (min_le_right ..) t ht
    -- so both contain a copy of the complete graph on Fin m
    rw [not_cliqueFree_iff_top_isContained] at hG hH
    obtain ⟨f⟩ := hG
    obtain ⟨g⟩ := hH
    -- pairing the two copies gives a homomorphism from ⊤ into G ⊗g H
    simpa using (tensorProd.prod f.toHom g.toHom).cliqueNum_le
```

We now turn to the box product `G □ H`. Recall that two vertices `(v₁, w₁), (v₂, w₂)` of the box product are adjacent if and only if either `G.Adj v₁ v₂` and `w₁ = w₂`, or `v₁ = v₂` and `H.Adj w₁ w₂`. This implies that cliques are always constant in one coordinate.

10. A clique s in G gives a "row" clique in the box product `G □ H` (so long as there is a vertex w_0 in H to fix the second coordinate).

```
example {W : Type*} {H : SimpleGraph W} {s : Set V} (w₀ : W) (h : G.IsClique s) :
    (G □ H).IsClique ((·, w₀) '' s) := by
  rintro _ ⟨a, ha, rfl⟩ _ ⟨b, hb, rfl⟩ hne
  -- both vertices share the second coordinate w₀, so they are adjacent in G □ H
  -- precisely because a and b are adjacent in G.
  rw [boxProd_adj]
  exact Or.inl ⟨h ha hb (fun he => hne (by rw [he])), rfl⟩
```

11. In any finite box product $G \square H$ with nonempty second factor we have $\omega(G) \le \omega(G \square H)$: the embedding `boxProdLeft` can transport a maximum clique of G to a clique of the same size in $G \square H$.

```
lemma cliqueNum_le_cliqueNum_boxProd_left {W : Type*} [Finite V] [Finite W]
    [Nonempty W] {H : SimpleGraph W} : ω G ≤ ω (G □ H) :=
  (boxProdLeft G H (Classical.arbitrary W)).toHom.cliqueNum_le
```

12. A clique of $G \square H$ containing a "horizontal" edge (two vertices sharing their second coordinate) is constant in the second coordinate.

```
lemma IsClique.boxProd_snd_eq {W : Type*} {H : SimpleGraph W} {s : Set (V × W)}
    {x y : V × W} (hs : (G □ H).IsClique s) (hx : x ∈ s) (hy : y ∈ s)
    (hG : G.Adj x.1 y.1) (hsnd : x.2 = y.2) : ∀ z ∈ s, z.2 = x.2 := by
  intro z hz
  -- z is equal or adjacent to both x and y; if z differed from x in the
  -- second coordinate it would share its first coordinate with both, forcing
  -- x.1 = y.1, which contradicts G.Adj x.1 y.1
  have h1 := hs hz hx
  have h2 := hs hz hy
  grind [boxProd_adj]
```

13. `Prod.fst` is injective on a clique of $G \square H$ whose vertices all share the same second coordinate, so such a clique projects to a clique of G of the same size.

```
lemma IsNClique.boxProd_image_fst {W : Type*} [DecidableEq V] {H : SimpleGraph W}
    {m : ℕ} {t : Finset (V × W)} {w : W} (ht : (G □ H).IsNClique m t)
    (hconst : ∀ z ∈ t, z.2 = w) : G.IsNClique m (t.image Prod.fst) := by
  -- it suffices to show that Prod.fst preserves adjacency on t (injectivity on
  -- the clique t is then automatic); an edge of t cannot be vertical, since the
  -- vertices of t agree in the second coordinate
  refine ht.image_of_adj fun p hp q hq h => ?_
  grind [boxProd_adj]
```

14. If G is finite then a clique of $G \square H$ containing a "horizontal" edge has size at most $\omega(G)$.

```
lemma IsNClique.boxProd_le_cliqueNum_left {W : Type*} [Finite V]
    {H : SimpleGraph W} {m : ℕ} {t : Finset (V × W)} {x y : V × W}
    (ht : (G □ H).IsNClique m t) (hx : x ∈ t) (hy : y ∈ t) (hG : G.Adj x.1 y.1)
    (hsnd : x.2 = y.2) : m ≤ ω G := by
  classical
  -- every vertex of t shares the second coordinate x.2
```

```
  have hconst : ∀ z ∈ t, z.2 = x.2 :=
    fun z hz => ht.isClique.boxProd_snd_eq hx hy hG hsnd z hz
  exact (ht.boxProd_image_fst hconst).le_cliqueNum
```

15. The clique number of $G \square H$ is $\max\ (\omega\ G)\ (\omega\ H)$ (for finite nonempty graphs).

```
lemma cliqueNum_boxProd {W : Type*} [Finite V] [Finite W] [Nonempty V]
    [Nonempty W] {H : SimpleGraph W} :
    ω (G □ H) = max (ω G) (ω H) := by
  classical
  apply le_antisymm
  · -- upper bound: a maximum clique has an edge, horizontal or vertical
    obtain ⟨t, ht⟩ := (G □ H).exists_isNClique_cliqueNum
    rcases Nat.lt_or_ge #t 2 with hsmall | hbig
    · -- a clique of size at most one is bounded since ω G ≥ 1
      grind [ht.card_eq, one_le_cliqueNum]
    · -- pick two distinct adjacent vertices x, y
      obtain ⟨x, hx, y, hy, hxy⟩ := Finset.one_lt_card.mp hbig
      have hadj := ht.isClique hx hy hxy
      rw [boxProd_adj] at hadj
      rcases hadj with ⟨hG, hsnd⟩ | ⟨hH, hfst⟩
      · -- horizontal edge: bound the clique directly by ω G
        exact (ht.boxProd_le_cliqueNum_left hx hy hG hsnd).trans (le_max_left _ _)
      · -- vertical edge: swap the factors with boxProdComm to turn it into a
        -- horizontal edge of H □ G, then reuse the same bound, now giving ω H
        have htH := ht.image (boxProdComm ..).toHom
        exact (htH.boxProd_le_cliqueNum_left (Finset.mem_image_of_mem _ hx)
          (Finset.mem_image_of_mem _ hy) hH hfst).trans (le_max_right _ _)
  · -- lower bound: a row realizes ω G (boxProdLeft) and a column realizes ω H
    -- (boxProdRight); both are graph embeddings into G □ H
    exact max_le cliqueNum_le_cliqueNum_boxProd_left
      (boxProdRight G H (Classical.arbitrary V)).toHom.cliqueNum_le
```

16. The clique number of the hypercube $Q[n]$ is 1 for $n = 0$ and 2 otherwise. (There are easier ways to prove this: for instance, once we have colourings we could use a two-colouring to prove $\omega\ Q[n] \le 2$, but we can also use the result of the previous exercise.)

```
lemma cliqueNum_hypercube {n : ℕ} : ω Q[n] = if n = 0 then 1 else 2 := by
  induction n with
  | zero =>
    exact one_le_cliqueNum.antisymm' (by simp)
  | succ n ih =>
    simp only [Nat.add_eq_zero_iff, one_ne_zero, and_false, ↓reduceIte]
    have := cliqueNum_boxProd (G := Q[n]) (H := (⊤ : SimpleGraph Bool))
    rw [ih, cliqueNum_top] at this
    cases n <;> simpa
```

17. Show that a set of vertices is a clique in G if and only if it is an independent set in the complement of G. (Mathlib has this as `isIndepSet_compl`, but prove it directly from the definitions, using `compl_adj`.)

```
example {s : Set V} : G.IsClique s ↔ Gᶜ.IsIndepSet s := by
  constructor
  · -- distinct vertices of a clique are adjacent in G, so not adjacent in Gᶜ
    intro h x hx y hy hne hadj
    exact ((compl_adj G x y).mp hadj).2 (h hx hy hne)
  · -- if distinct x, y were not adjacent in G they would be adjacent in Gᶜ
    intro h x hx y hy hne
    by_contra hnadj
    exact h hx hy hne ((compl_adj G x y).mpr ⟨hne, hnadj⟩)
```

18. Show that the Paley graph on 17 vertices has independence number 3. Together with Exercise 3 this shows that there is a graph on 17 vertices with no 4-clique and no independent set of size 4, so the Ramsey number `R(4, 4)` is at least 18. (In fact `R(4, 4) = 18`.) Hint: as in the chapter, rewrite with `cliqueFree_compl`, then use **`native_decide`** as in Exercise 3.

```
example : α (paleyGraph (ZMod 17)) = 3 := by
  -- {0, 3, 6} is independent since 3 and 6 are not squares mod 17
  have h3 : (paleyGraph (ZMod 17)).IsNIndepSet 3 {0, 3, 6} := by decide
  exact h3.indepNum_eq_of_indepSetFree (by rw [← cliqueFree_compl]; native_decide)
```

19. Show that a clique and an independent set share at most one vertex, and deduce that `ω G + α G ≤ Nat.card V + 1` in any finite graph.

```
lemma IsClique.card_inter_le_one_of_isIndepSet [DecidableEq V] {s t : Finset V}
    (hs : G.IsClique ↑s) (ht : G.IsIndepSet ↑t) : #(s ∩ t) ≤ 1 := by
  -- two distinct common vertices would be both adjacent and non-adjacent
  rw [Finset.card_le_one]
  intro x hx y hy
  by_contra hne
  simp only [Finset.mem_inter] at hx hy
  exact ht hx.2 hy.2 hne (hs hx.1 hy.1 hne)

lemma cliqueNum_add_indepNum_le [Finite V] : ω G + α G ≤ Nat.card V + 1 := by
  classical
  have := Fintype.ofFinite V
  obtain ⟨s, hs⟩ := G.exists_isNClique_cliqueNum
  obtain ⟨t, ht⟩ := G.exists_isNIndepSet_indepNum
  -- a maximum clique and a maximum independent set share at most one vertex
  have hst : #(s ∩ t) ≤ 1 :=
    hs.isClique.card_inter_le_one_of_isIndepSet ht.isIndepSet
  -- inclusion-exclusion, and s ∪ t has at most Nat.card V elements
  have := Finset.card_union_add_card_inter s t
  have := (s ∪ t).card_le_univ
  rw [← hs.card_eq, ← ht.card_eq, Nat.card_eq_fintype_card]
  grind
```

20. Show that any graph on at least 6 vertices contains a triangle or an independent set of size 3, so that `R(3, 3) = 6`. Hint: suppose not, and apply `degree_le_indepNum_of_triangleFree` to both `G` and `Gᶜ`, bounding both independence numbers with `CliqueFree.cliqueNum_le`. Mathlib's `degree_compl` relates the degrees of a vertex in `G` and `Gᶜ`.

```
theorem not_cliqueFree_three_and_indepSetFree_three [Fintype V]
    (h : 6 ≤ Fintype.card V) : ¬(G.CliqueFree 3 ∧ G.IndepSetFree 3) := by
  classical
  rintro ⟨h3, hi3⟩
  -- Gᶜ is also triangle-free, since G has no independent set of size 3
  have hc3 : Gᶜ.CliqueFree 3 := (G.cliqueFree_compl).mpr hi3
  obtain ⟨v⟩ : Nonempty V := Fintype.card_pos_iff.mp (by grind)
  -- so v has degree at most α G in G and at most α Gᶜ in Gᶜ
  have h1 := degree_le_indepNum_of_triangleFree h3 v
  have h2 := degree_le_indepNum_of_triangleFree hc3 v
  -- and both independence numbers are at most 2
  have hi : α G ≤ 2 := by simpa using hc3.cliqueNum_le
  have hic : α Gᶜ ≤ 2 := by simpa using h3.cliqueNum_le
  -- but the two degrees add up to Fintype.card V - 1 ≥ 5
  have := G.degree_compl v
  grind
```

21. Unlike the junk value `α pathNat = 0` of the independence number, the clique number of the infinite path on $\mathbb{N}$ is what you expect. Show that `ω pathNat = 2`. Hint: `pathNat` is triangle-free, since adjacent vertices have different parity. The lemma `IsNClique.le_cliqueNum` needs `[Finite V]`, but `IsNClique.cliqueNum_eq_of_cliqueFree` needs no finiteness.

```
example : ω pathNat = 2 := by
  -- pathNat is triangle-free: adjacent vertices have different parity
  have h3 : pathNat.CliqueFree 3 := by
    intro t ht
    obtain ⟨a, b, c, hab, hac, hbc, rfl⟩ := card_eq_three.mp ht.card_eq
    have h1 : pathNat.Adj a b := ht.isClique (by grind) (by grind) hab
    have h2 : pathNat.Adj a c := ht.isClique (by grind) (by grind) hac
    have h3 : pathNat.Adj b c := ht.isClique (by grind) (by grind) hbc
    grind
  -- the edge {0, 1} is a 2-clique, and IsNClique.cliqueNum_eq_of_cliqueFree
  -- needs no finiteness: h3 bounds the size of every clique
  have h2 : pathNat.IsNClique 2 {0, 1} := is2Clique_pair_iff.mpr (by decide)
  exact h2.cliqueNum_eq_of_cliqueFree h3
```

22. Show that the Kneser graph `Kn[n, 1]` is isomorphic to the complete graph `K[n]`. (Its vertices are the singletons `{a}`, and distinct singletons are disjoint.)

```
example (n : ℕ) : Nonempty (Kn[n, 1] ≃g K[n]) := by
  -- the map a ↦ {a} is a bijection from Fin n to the vertices of Kn[n, 1]
  let f : Fin n → {s : Finset (Fin n) // #s = 1} := fun a => ⟨{a}, card_singleton a⟩
  have hf : Function.Bijective f := by
    refine ⟨fun a b h => by simpa [f] using congrArg Subtype.val h, fun s => ?_⟩
    obtain ⟨a, ha⟩ := card_eq_one.mp s.2
    exact ⟨a, Subtype.ext ha.symm⟩
  refine ⟨((Equiv.ofBijective f hf).symm, ?_⟩⟩
  -- it remains to check that adjacency is preserved
  intro a b
  obtain ⟨a, rfl⟩ := hf.2 a
  obtain ⟨b, rfl⟩ := hf.2 b
```

```
  simp [f]
```

23. Compute the clique number of the Mycielskian of the edgeless graph $\bot$. The answer depends on whether V is empty. If V is empty then $\mu(\bot)$ has only the vertex none, so its clique number is 1. If V is nonempty then none and v^x are adjacent and $\mu(\bot)$ is triangle-free, so its clique number is 2.

```
lemma cliqueNum_mycielskian_bot_of_isEmpty [IsEmpty V] :
    ω mμ((⊥ : SimpleGraph V)) = 1 :=
  -- mμ(⊥) has exactly one vertex, none
  one_le_cliqueNum.antisymm'
    (by simpa using mμ((⊥ : SimpleGraph V)).cliqueNum_le_natCard)

lemma cliqueNum_mycielskian_bot [Nonempty V] :
    ω mμ((⊥ : SimpleGraph V)) = 2 := by
  classical
  -- none and v^x are adjacent, giving a 2-clique
  obtain ⟨v⟩ := ‹Nonempty V›
  have h2 : mμ(⊥).IsNClique 2 {none, v^x} :=
    is2Clique_pair_iff.mpr (by simp)
  -- moreover mμ(⊥) is 3-clique-free since ⊥ is trivially triangle-free
  exact h2.cliqueNum_eq_of_cliqueFree (cliqueFree_bot (by norm_num)).mycielskian
```

The last few exercises are on independent sets in complete bipartite graphs (and their Mycielskians).

24. In a complete bipartite graph the non-neighbours of a left vertex are precisely the left part, and similarly for right vertices.

```
lemma completeBipartiteGraph_not_adj_inl_iff {W : Type*} {v : V} {x : V ⊕ W} :
    ¬(completeBipartiteGraph V W).Adj (.inl v) x ↔
      x ∈ Set.range .inl := by
  cases x <;> simp

lemma completeBipartiteGraph_not_adj_inr_iff {W : Type*} {w : W} {x : V ⊕ W} :
    ¬(completeBipartiteGraph V W).Adj (.inr w) x ↔
      x ∈ Set.range .inr := by
  cases x <;> simp
```

25. An independent set of a complete bipartite graph is contained entirely in the left part or entirely in the right part: if it contains a left vertex then all its vertices are non-neighbours of that vertex, so all lie in the left part.

```
lemma completeBipartiteGraph_isIndepSet_subset_left_or_right {W : Type*}
    {s : Set (V ⊕ W)} (hs : (completeBipartiteGraph V W).IsIndepSet s) :
    s ⊆ Set.range .inl ∨ s ⊆ Set.range .inr := by
  by_cases h : ∃ v, .inl v ∈ s
  · obtain ⟨v, hv⟩ := h
    exact Or.inl fun x hx =>
      completeBipartiteGraph_not_adj_inl_iff.mp fun hadj => hs hv hx hadj.ne hadj
```

```
  · refine Or.inr fun x hx => ?_
    rcases x with x | x
    · exact absurd ⟨x, hx⟩ h
    · exact Set.mem_range_self x
```

26. Any `Finset` contained in the range of a function `f : α → β` with finite domain has at most `Nat.card α` elements. In particular a `Finset (V ⊕ W)` contained in the left part has at most `Nat.card V` elements (and similarly on the right), since the left part is the range of `Sum.inl`.

```
lemma _root_.Finset.card_le_of_subset_range {α β : Type*} [Finite α] {f : α → β}
    {s : Finset β} (h : ↑s ⊆ Set.range f) : #s ≤ Nat.card α := by
  classical
  have := Fintype.ofFinite α
  -- s is contained in the image of univ under f, which has at most #univ
  -- elements
  have hsub : s ⊆ univ.image f := fun x hx => by simpa using h hx
  simpa using (card_le_card hsub).trans card_image_le
```

27. Any set of vertices contained in one part of a complete bipartite graph is independent, since a part contains no edges (we only need this for the right part).

```
lemma isIndepSet_completeBipartiteGraph_of_subset_range_inr {W : Type*}
    {s : Set (V ⊕ W)} (h : s ⊆ Set.range .inr) :
    (completeBipartiteGraph V W).IsIndepSet s := by
  intro u hu v hv hne hadj
  obtain ⟨x, rfl⟩ := h hu
  obtain ⟨y, rfl⟩ := h hv
  simp at hadj
```

28. The independence number of a complete bipartite graph is the size of the larger part; we state this as `α K[a, b] = b` when `a ≤ b`. The right part is a `b`-element independent set, and by Exercises 25 and 26 no independent set is larger.

```
lemma indepNum_completeBipartiteGraph {a b : ℕ} (h : a ≤ b) :
    α K[a, b] = b := by
  apply IsNIndepSet.indepNum_eq_of_indepSetFree (s := univ.map .inr)
  · exact ⟨isIndepSet_completeBipartiteGraph_of_subset_range_inr (by simp), by simp⟩
  · intro s hs
    rcases completeBipartiteGraph_isIndepSet_subset_left_or_right hs.isIndepSet
      with hL | hR
    · grind [hs.card_eq, card_le_of_subset_range hL, Nat.card_fin]
    · grind [hs.card_eq, card_le_of_subset_range hR, Nat.card_fin]
```

If we want to bound the independence number of the Mycielskian of a complete bipartite graph we need a bound on the size of any independent set `s` of `K[a, b]` plus the size of the complement of its open neighbourhood `N(s)ᶜ`. We give this bound in the next exercise. The last exercise uses `le_indepNum_mycielskian` and `mycielskian_indepSet_card_le` from the starred section Independence Number of the Mycielskian (p. 145).

29. Any independent set s of $K[a, b]$ (with $a \le b$) satisfies $\#s + \#N(s)^c \le 2 * b$. We state this with an abstract finset t in place of $N(s)^c$, characterized by its membership property, so that the lemma applies whatever decidability instances $N(s)^c$ is written with. If s is empty then t is all $a + b$ vertices; otherwise s lies in a single part (Exercise 25) and t lies in the same part (every vertex of the other part is a neighbour of any vertex of s), so each has size at most that of the part.

```
lemma completeBipartiteGraph_isIndepSet_card_add_le {a b : ℕ} (hab : a ≤ b)
    {s t : Finset (Fin a ⊕ Fin b)} (hs : K[a, b].IsIndepSet ↑s)
    (ht : ∀ x, x ∈ t ↔ ∀ y ∈ s, ¬K[a, b].Adj y x) :
    #s + #t ≤ 2 * b := by
  rcases s.eq_empty_or_nonempty with rfl | ⟨x, hx⟩
  · -- s = ∅ : t has at most a + b ≤ 2 * b vertices
    grind [Fintype.card_sum, Fintype.card_fin, t.card_le_univ]
  rcases completeBipartiteGraph_isIndepSet_subset_left_or_right hs with hL | hR
  · -- s lies in the left part, and so does t
    obtain ⟨y, rfl⟩ := hL (mem_coe.mpr hx)
    have h1 : #s ≤ a := by simpa using card_le_of_subset_range hL
    have h2 : #t ≤ a := by
      simpa using card_le_of_subset_range fun z hz =>
        completeBipartiteGraph_not_adj_inl_iff.mp ((ht z).mp (mem_coe.mp hz) _ hx)
    grind
  · -- s lies in the right part, and so does t
    obtain ⟨y, rfl⟩ := hR (mem_coe.mpr hx)
    have h1 : #s ≤ b := by simpa using card_le_of_subset_range hR
    have h2 : #t ≤ b := by
      simpa using card_le_of_subset_range fun z hz =>
        completeBipartiteGraph_not_adj_inr_iff.mp ((ht z).mp (mem_coe.mp hz) _ hx)
    grind
```

30. The independence number of the Mycielskian of $K[a, b]$ is $2 * b$ whenever $a \le b$ and $b \neq 0$: the right part B is an independent set with $B \subseteq N(B)^c$, so it contributes at least $2 * b$, and no independent set does better by Exercise 29. (Note that $b \neq 0$ is needed: $\mu(K[0, 0])$ still has the vertex none, so its independence number is 1.)

```
lemma indepNum_mycielskian_completeBipartiteGraph {a b : ℕ} [NeZero b]
    (h : a ≤ b) : α μ(K[a, b]) = 2 * b := by
  refine le_antisymm ?_ ?_
  · -- ≤ : a maximum independent set T of μ(K[a, b]) satisfies
    -- #T ≤ #A + #N(A)ᶜ ≤ 2 * b, where A is the left part of T
    obtain ⟨T, hT⟩ := μ(K[a, b]).exists_isNIndepSet_indepNum
    rw [← hT.card_eq]
    exact le_trans (mycielskian_indepSet_card_le hT.isIndepSet)
      (completeBipartiteGraph_isIndepSet_card_add_le h
        (isIndepSet_preimage_left_finset hT.isIndepSet)
        fun _ => by simp)
  · -- ≥ : the right part is an independent set s with #s = b and s ⊆ N(s)ᶜ
    have hind : K[a, b].IsIndepSet ↑(univ.map .inr : Finset (Fin a ⊕ Fin b)) :=
      isIndepSet_completeBipartiteGraph_of_subset_range_inr (by simp)
    refine le_trans ?_ (le_indepNum_mycielskian hind)
    have h1 : #(univ.map .inr : Finset (Fin a ⊕ Fin b)) = b := by simp
    have h2 := card_le_card hind.subset_compl_openNeighborhoodFinset
```

```
    grind
```

Chapter 7: Colorings

1. Show that the cycle graph `C[5]` is not 2-colourable. Hint: `chromaticNumber_cycleGraph_of_odd`.

```
example : ¬C[5].Colorable 2 := by
  rw [← chromaticNumber_le_iff_colorable,
    chromaticNumber_cycleGraph_of_odd 5 (by decide) (by decide)]
  decide
```

2. Show that a graph `G` on `n` vertices satisfies `n ≤ χ G * χ Gᶜ`. Hint: combine `natCard_le_chromaticNumber_mul_indepNum` with `cliqueNum_compl` and `cliqueNum_le_chromaticNumber`.

```
example : Nat.card V ≤ χ G * χ Gᶜ :=
  -- an independent set of G is a clique of Gᶜ
  calc (Nat.card V : ℕ∞) ≤ χ G * α G := natCard_le_chromaticNumber_mul_indepNum
    _ = χ G * ω Gᶜ := by rw [cliqueNum_compl]
    -- gcongr ("generalized congruence") proves an inequality between two terms of
    -- the same shape by comparing the parts that differ: since multiplication by
    -- χ G is monotone, it reduces the goal to ω Gᶜ ≤ χ Gᶜ
    _ ≤ χ G * χ Gᶜ := by gcongr; exact cliqueNum_le_chromaticNumber
```

3. Recall the Kneser graph `Kn[n, k]`: the graph whose vertices are the `k`-element subsets of `Fin n = {0, ..., n - 1}` with edges between disjoint subsets. Show that it has an `(n - 2 * k + 2)`-colouring for `2 * k ≤ n` and `0 < k`. The colouring is given below: colour a vertex by its least element, merging all the large colours into one. This is the hardest exercise in this chapter: the case where both vertices receive the merged colour needs a pigeonhole argument, which is `isIndepSet_kneserGraph_of_card_lt` from Cliques (p. 127). Here `s.min' h` is the least element of a nonempty finset `s`, and its basic properties are `Finset.min'_mem` and `Finset.min'_le`.

The `(n - 2 * k + 2)`-colouring of the Kneser graph `Kn[n, k]`.

```
def kneserGraph.coloring (n k : ℕ) (hn : 2 * k ≤ n) (hk : 0 < k) :
    Kn[n, k].Coloring (Fin (n - 2 * k + 2)) where
  toFun s :=
    -- A vertex is s : Finset (Fin n) of size k. Since 0 < k, s is
    -- nonempty. We colour s by its minimum element m.
    let m : Fin n := s.val.min' (card_pos.mp (by rw [s.property]; exact hk))
    -- Minima below n - 2 * k + 1 keep their own colour; every larger minimum
    -- shares the last colour n - 2 * k + 1.
    if h : m.val < n - 2 * k + 1 then
      ⟨m.val, by grind⟩
    else
      ⟨n - 2 * k + 1, by grind⟩
  map_rel' := by
    intro u v hadj hcolor
```

```
    simp only [kneserGraph_adj, ne_eq, Order.lt_add_one_iff] at hadj hcolor
    split_ifs at hcolor with h1 h2 h3
    · -- the colour is the common minimum of u and v, which are disjoint
      have hune : u.val.Nonempty := card_pos.mp (by rw [u.2]; exact hk)
      have hvne : v.val.Nonempty := card_pos.mp (by rw [v.2]; exact hk)
      have h : u.val.min' hune = v.val.min' hvne :=
        Fin.val_injective (by simpa using congrArg Fin.val hcolor)
      exact Finset.disjoint_left.mp hadj.1 (min'_mem _ hune) (h ▸ min'_mem _ hvne)
    · grind -- impossible: one colour is ≤ n - 2 * k, the other is n - 2 * k + 1
    · grind -- impossible: one colour is ≤ n - 2 * k, the other is n - 2 * k + 1
    · -- push Not pushes the negations inwards (¬a ≤ b becomes b < a)
      push Not at h1 h3
      -- both u and v lie in {n - 2 * k + 1, ..., n - 1}, which has only
      -- 2 * k - 1 elements, so they lie in an independent set of Kn[n, k]
      -- (isIndepSet_kneserGraph_of_card_lt from the previous chapter)
      let a : Fin n := ⟨n - 2 * k, by grind⟩
      have hsub (s : Finset (Fin n)) (hne : s.Nonempty)
          (hs : n - 2 * k < ↑(s.min' hne)) : s ⊆ Ioi a :=
        fun x hx => mem_Ioi.2 (hs.trans_le (min'_le _ x hx))
      have hind := isIndepSet_kneserGraph_of_card_lt (k := k) (s := Ioi a)
        (by grind [Fin.card_Ioi])
      exact hind (hsub _ _ h1) (hsub _ _ h3) hadj.2 hadj
```

4. Show that if `2 * k ≤ n`, `0 < k` and `Kn[n, k]` is m-colourable then $n \le k * m$, so that $n \le k * \chi$ `Kn[n, k]`. Hint: use `Colorable.natCard_le_mul_indepNum` and the independence number of the Kneser graph, `indepNum_kneserGraph`, from the previous chapter; `Nat.add_one_mul_choose_eq` will help with the binomial coefficients.

```
theorem le_mul_of_colorable_kneserGraph {n k m : ℕ} (hn : 2 * k ≤ n) (hk : 0 < k)
    (h : Kn[n, k].Colorable m) : n ≤ k * m := by
  have : NeZero k := ⟨hk.ne'⟩
  -- Kn[n, k] has n.choose k vertices and independence number
  -- (n - 1).choose (k - 1)
  have hcard := h.natCard_le_mul_indepNum
  have hkn : k ≤ n / 2 := by grind
  simp only [indepNum_kneserGraph, hkn, ↓reduceIte, Nat.card_eq_fintype_card,
    Fintype.card_finset_len, Fintype.card_fin] at hcard
  -- n * (n - 1).choose (k - 1) = n.choose k * k
  have key := Nat.add_one_mul_choose_eq (n - 1) (k - 1)
  rw [Nat.sub_add_cancel (by grind), Nat.sub_add_cancel hk] at key
  have hpos : 0 < (n - 1).choose (k - 1) := Nat.choose_pos (by grind)
  refine Nat.le_of_mul_le_mul_right ?_ hpos
  calc n * (n - 1).choose (k - 1) = k * n.choose k := by rw [key, mul_comm]
    _ ≤ k * (m * (n - 1).choose (k - 1)) := Nat.mul_le_mul_left k hcard
    _ = k * m * (n - 1).choose (k - 1) := by ring
```

5. Use the previous two exercises to show that the Petersen graph `Kn[5, 2]` has chromatic number 3.

```
example : χ Kn[5, 2] = 3 :=
  -- kneserGraph.coloring uses 5 - 2 * 2 + 2 = 3 colours, and 2 colours would
  -- need 5 ≤ 2 * 2
```

```
  chromaticNumber_eq_iff_colorable_not_colorable.2
    ⟨by simpa using (kneserGraph.coloring 5 2 (by norm_num) (by norm_num)).colorable,
     fun h => by
       simpa using le_mul_of_colorable_kneserGraph (by norm_num) (by norm_num) h⟩
```

6. Show that the chromatic number of the hypercube `Q[n + 1]` is 2. Hint: `chromaticNumber_eq_two_iff` reduces this to facts proved in this chapter.

```
example (n : ℕ) : χ Q[n + 1] = 2 := by
  -- bipartiteness was proved in this chapter, so it only remains to show
  -- Q[n + 1] ≠ ⊥ for which we need a vertex of Q[n] (each gives an edge in
  -- the next dimension)
  obtain ⟨v⟩ : Nonempty (HypercubeVert n) := inferInstance
  refine chromaticNumber_eq_two_iff.2 ⟨IsBipartite.hypercube _, ?_⟩
  exact ne_bot_iff_exists_adj.2 ⟨(v, false), (v, true), by simp⟩
```

7. Prove the claim from the chapter that for `G : SimpleGraph ℕ` a vertex `v` has at most `v` earlier neighbours.

```
lemma degreeLT_le_self (G : SimpleGraph ℕ) (v : ℕ) [Fintype (G.neighborLTSet v)] :
    G.degreeLT v ≤ v := by
  -- We want to change the 2nd occurrence of v to #(range v) in the goal,
  -- but rw will change all occurrences, so we use nth_rw 2 below
  nth_rw 2 [← Finset.card_range v]
  apply Finset.card_le_card
  intro x hx; simp_all
```

8. Show that the greedy colouring really is greedy: if `k` is less than the greedy colour of `v`, then some earlier neighbour of `v` has greedy colour `k`. Hint: rewrite with `greedy_eq_greedyChoice` and use `Nat.find_min`.

```
lemma exists_greedy_eq_of_lt_greedy (G : SimpleGraph ℕ) [DecidableRel G.Adj]
    {v k : ℕ} (hk : k < G.greedy v) :
    ∃ u ∈ G.neighborLTFinset v, G.greedy u = k := by
  rw [greedy_eq_greedyChoice] at hk
  -- k is below the least colour that no earlier neighbour uses, so one uses it
  simpa using not_not.mp ◁ Nat.find_min (Finset.exists_notMem _) hk
```

9. An ordering of the vertices of `G` is a *perfect elimination ordering* if the earlier neighbours of every vertex form a clique. (For example, ordering the vertices of an interval graph by the left endpoints of their intervals gives such an ordering.) Show that if the natural ordering of `G` is a perfect elimination ordering and `G` has no `(k + 1)`-clique then `G` is `k`-colourable, so that the greedy colouring uses no more colours than the largest clique needs. Hint: `v` together with its earlier neighbours forms a `(degreeLT v + 1)`-clique, so `CliqueFree.mono` bounds `degreeLT v`; now use `colorable_of_degreeLT_le`.

```
example (G : SimpleGraph ℕ) {k : ℕ}
    (hG : G.CliqueFree (k + 1)) (h : ∀ v, G.IsClique (G.neighborLTSet v)) :
    G.Colorable k := by
```

```
  classical
  have hdeg : ∀ v, G.degreeLT v + 1 ≤ k := by
    intro v
    by_contra! hlt
    -- v and its earlier neighbours form a (degreeLT v + 1)-clique
    have hcl : G.IsNClique (G.degreeLT v + 1) (insert v (G.neighborLTFinset v)) := by
      refine ⟨?_, ?_⟩
      · rw [coe_insert, neighborLTFinset, Set.coe_toFinset]
        refine (h v).insert fun u hu _ => ?_
        exact ((G.mem_neighborLTSet v u).1 hu).2
      · rw [card_insert_of_notMem (by simp), degreeLT]
    exact hG.mono hlt _ hcl
  -- so k = Δ + 1, where every vertex has at most Δ earlier neighbours
  obtain ⟨Δ, rfl⟩ : ∃ Δ, k = Δ + 1 := ⟨k - 1, by grind [hdeg 0]⟩
  exact G.colorable_of_degreeLT_le fun v => by grind [hdeg v]
```

10. Show that if the greedy colour of v is k then G contains an increasing path $f\ 0 < f\ 1 < \cdots < f\ k = v$, in which consecutive vertices are adjacent. (We describe the path by a function $f : \mathbb{N} \to \mathbb{N}$, since walks only appear in the next chapter.) Hint: prove the statement for all v with $G.\mathrm{greedy}\ v = k$ by induction on k. Exercise 8 gives the vertex u before v, and `Function.update f (k + 1) v` extends the path f ending at u.

```
lemma exists_increasing_path_greedy (G : SimpleGraph ℕ) [DecidableRel G.Adj]
    (v : ℕ) : ∃ f : ℕ → ℕ, f (G.greedy v) = v ∧
      ∀ i < G.greedy v, f i < f (i + 1) ∧ G.Adj (f i) (f (i + 1)) := by
  suffices ∀ k v, G.greedy v = k → ∃ f : ℕ → ℕ, f k = v ∧
      ∀ i < k, f i < f (i + 1) ∧ G.Adj (f i) (f (i + 1)) from this _ v rfl
  intro k
  induction k with
  | zero => exact fun v _ => ⟨fun _ => v, rfl, by simp⟩
  | succ k ih =>
    intro v hv
    -- an earlier neighbour u of v has colour k, so a path ends at u
    have hk : k < G.greedy v := by grind
    obtain ⟨u, hu, hgu⟩ := G.exists_greedy_eq_of_lt_greedy hk
    rw [mem_neighborLTFinset] at hu
    obtain ⟨f, hfk, hf⟩ := ih u hgu
    -- and we extend it by v
    refine ⟨Function.update f (k + 1) v, by simp, fun i hi => ?_⟩
    rcases Nat.lt_succ_iff_lt_or_eq.mp hi with hi | rfl
    · simpa [Function.update_of_ne (show i ≠ k + 1 by grind),
        Function.update_of_ne (show i + 1 ≠ k + 1 by grind)] using hf i hi
    · simpa [Function.update_of_ne (show i ≠ i + 1 by grind), hfk] using
        ⟨hu.1, hu.2.symm⟩
```

11. Deduce that if G has no increasing path on $k + 1$ vertices then G is k-colourable. In particular, a graph in which no path has $k + 1$ vertices is k-colourable. Hint: as in `colorable_of_degreeLT_le`, use `colorable_iff_exists_bdd_nat_coloring` with `greedyColoringNat`.

```
example (G : SimpleGraph ℕ) {k : ℕ}
    (h : ¬∃ f : ℕ → ℕ, ∀ i < k, f i < f (i + 1) ∧ G.Adj (f i) (f (i + 1))) :
```

```
    G.Colorable k := by
  classical
  refine (colorable_iff_exists_bdd_nat_coloring _).2
    ⟨G.greedyColoringNat, fun v => ?_⟩
  -- if k ≤ greedy v then the first k + 1 vertices of the path ending at v
  -- contradict h
  by_contra! hk
  obtain ⟨f, -, hf⟩ := G.exists_increasing_path_greedy v
  exact h ⟨f, fun i hi => hf i (by grind [greedyColoringNat_apply])⟩
```

12. Show that the greedy colouring of the complete graph on $\mathbb{N}$ colours vertex n with colour n. (See `greedy_crownGraph_eq` for a similar proof using `Nat.strong_induction_on`.)

```
example (n : ℕ) : (⊤ : SimpleGraph ℕ).greedy n = n := by
  -- as in greedy_crownGraph_eq we use strong induction on n
  induction n using Nat.strong_induction_on with
  | _ n ih =>
  -- the earlier neighbours of n are {0, ..., n - 1}, and by induction they use
  -- exactly the colours {0, ..., n - 1}
  refine (⊤ : SimpleGraph ℕ).greedy_eq_of_image_eq_range ?_
  ext c
  grind [mem_neighborLTFinset, top_adj]
```

13. The *infinite star* `starNat` joins `0` to every other natural number. Show that `0` has infinitely many neighbours, so that `starNat` is not locally finite and the bound `colorable_of_degree_le_of_countable` does not apply, but that `starNat` is nevertheless 2-colourable, using `colorable_of_degreeLT_le`: every vertex has at most one earlier neighbour.

```
example : (starNat.neighborSet 0).Infinite ∧ starNat.Colorable 2 := by
  refine ⟨Set.infinite_of_injective_forall_mem (f := Nat.succ) Nat.succ_injective
    (by simp), starNat.colorable_of_degreeLT_le (Δ := 1) fun v => ?_⟩
  -- the only earlier neighbour that a vertex can have is 0
  rw [degreeLT, Finset.card_le_one]
  intro a ha b hb
  rw [mem_neighborLTFinset] at ha hb
  grind
```

14. The crown graph shows that a bad ordering of the vertices can make the greedy colouring use arbitrarily many colours. Conversely, show that a good ordering makes it optimal: if C is a colouring of G in which every vertex comes before all neighbours of larger colour, then the greedy colour of each vertex v is at most $C\ v$. Hint: use strong induction on v and `greedy_le_card_image`, showing that the greedy colours of the earlier neighbours of v all lie in `range (C v)`.

```
example (G : SimpleGraph ℕ) [DecidableRel G.Adj] (C : G.Coloring ℕ)
    (hC : ∀ u v, G.Adj u v → C u < C v → u < v) (v : ℕ) : G.greedy v ≤ C v := by
  induction v using Nat.strong_induction_on with
  | _ v ih =>
  refine (G.greedy_le_card_image v).trans ?_
  -- every earlier neighbour u of v has greedy u ≤ C u < C v
```

```
  have hsub : (G.neighborLTFinset v).image G.greedy ⊆ range (C v) := by
    intro c hc
    obtain ⟨u, hu, rfl⟩ := mem_image.mp hc
    rw [mem_neighborLTFinset] at hu
    have hne : C u ≠ C v := C.valid hu.2.symm
    have hle : ¬C v < C u := fun h => (hC v u hu.2 h).not_gt hu.1
    exact mem_range.2 ((ih u hu.1).trans_lt (by grind))
  simpa using card_le_card hsub
```

Chapter 8: Walks

1. Show that if `p : G.Walk u v` has length `0` then `u = v`. Hint: use `cases p`.

```
example {u v : V} (p : G.Walk u v) (hl : p.length = 0) : u = v := by
  cases p with
  | nil => rfl
  | cons h p => simp at hl
```

2. Prove by induction on `p` that a walk in `pathNat` from `i` to `j` has length at least `j - i`, so that `pathNat.path i j` is a shortest walk from `i` to `i + j`. Hint: in the `cons` case the first edge changes the vertex by one; `length_cons`.

```
example {i j : ℕ} (p : pathNat.Walk i j) : j ≤ i + p.length := by
  induction p with
  | nil => simp
  -- the first edge changes the vertex by one, and ih bounds the rest of the walk
  | cons h p ih => grind [length_cons]
```

3. Let `c` be a `Bool`-colouring of `G`. Prove by induction on `p` that a walk has even length iff its endpoints have the same colour. Hint: in the `cons` case, `Bool.eq_not_of_ne` rewrites the colour of the first vertex as the negation of the colour of the second, and `Nat.even_add_one` handles the length. Finally, `cases` on a colour such as `c v` splits into the two cases `true` and `false`.

```
lemma Coloring.even_length_iff (c : G.Coloring Bool) {u v : V} (p : G.Walk u v) :
    Even p.length ↔ c u = c v := by
  induction p with
  | nil => simp
  | @cons a b w h p ih =>
    -- one more edge flips the parity of the length and the colour of the start
    rw [length_cons, Nat.even_add_one, ih, Bool.eq_not_of_ne (c.valid h)]
    cases c b <;> cases c w <;> simp
```

4. Show that splitting a walk at a vertex on its support preserves the total length. Hint: `Walk.take_spec`.

```
example [DecidableEq V] {u v w : V} (p : G.Walk u v) (h : w ∈ p.support) :
    (p.takeUntil w h).length + (p.dropUntil w h).length = p.length := by
  rw [← length_append, take_spec]
```

5. The walk `eulerC5` travels once around the cycle `C[5]`. Show that it is an Eulerian circuit. Hint: `decide` proves that it is Eulerian, using the `Decidable` instance from the chapter. For `IsCircuit`, rewrite with `isCircuit_def` and `isTrail_def` first.

```
example : eulerC5.IsEulerian ∧ eulerC5.IsCircuit := by
  rw [isCircuit_def, isTrail_def]
  decide
```

6. Use Exercise 3 and the closed walk `eulerC5` of length `5` to show that `C[5]` is not `2`-colourable. Hint: `recolorOfEquiv` and `finTwoEquiv` turn a colouring by `Fin 2` into a `Bool`-colouring.

```
example : ¬C[5].Colorable 2 := by
  rintro ⟨c⟩
  -- as a Bool-colouring, c would give the closed walk eulerC5 even length
  have := ((C[5]).recolorOfEquiv finTwoEquiv c).even_length_iff eulerC5
  exact absurd (this.2 rfl) (by decide)
```

7. A `cons` walk is a Hamiltonian cycle iff its tail is Hamiltonian and has length at least two. Hint: `isHamiltonianCycle_isCycle_and_isHamiltonian_tail` and `cons_isCycle_iff_length_ne_one`.

```
lemma Walk.cons_isHamiltonianCycle_iff [DecidableEq V] {u v : V} (p : G.Walk v u)
    (h : G.Adj u v) :
    (cons h p).IsHamiltonianCycle ↔ p.IsHamiltonian ∧ 2 ≤ p.length := by
  rw [isHamiltonianCycle_isCycle_and_isHamiltonian_tail,
    cons_isCycle_iff_length_ne_one, and_assoc]
  constructor <;> rintro ⟨h1, h2⟩
  · exact ⟨by simpa using h2.2, by grind⟩
  · exact ⟨h1.isPath, by grind, by simpa using h1⟩
```

8. Show that the tail of a cycle is never nil. Hint: a cycle has length at least three.

```
lemma Walk.IsCycle.not_nil_tail {u : V} {c : G.Walk u u} (hc : c.IsCycle) :
    ¬c.tail.Nil := by
  grind [length_tail_add_one, hc.three_le_length]
```

9. Show that reversing a copy of a walk is the same as copying its reverse. Hint: `subst_vars`.

```
example {u v u' v' : V} (p : G.Walk u v) (hu : u = u') (hv : v = v') :
    (p.copy hu hv).reverse = p.reverse.copy hv hu := by
  subst_vars
  rfl
```

10. Translating the walk `pathNat.path 0 j` by `i`, with `Walk.map` along `pathNat.translate i`, gives a walk from `0 + i` to `(0 + j) + i`, so before we can compare it with `pathNat.path i j` we must `copy` it. Show that the two walks are then equal. Hint: compare supports with `Walk.ext_support`, then use `support_copy`, `Walk.support_map`, `pathNat.support_path` and `List.map_add_range'`.

```
example (i j : ℕ) :
    ((pathNat.path 0 j).map (pathNat.translate i).toHom).copy
      (by simp [pathNat.translate]) (by simp [pathNat.translate, add_comm]) =
      pathNat.path i j := by
  apply Walk.ext_support
  rw [support_copy, Walk.support_map, pathNat.support_path, pathNat.support_path]
  have h : ⇑(pathNat.translate i).toHom = (i + ·) := funext fun k => add_comm k i
  rw [h, List.map_add_range', add_zero]
```

11. Show that the Hamming distance on the hypercube is symmetric.

```
lemma HypercubeVert.hammingDist_comm {n : ℕ} (u v : HypercubeVert n) :
    u.hammingDist v = v.hammingDist u := by
  induction n with
  | zero => rfl
  | succ n ih =>
    rcases u with ⟨u, b⟩
    rcases v with ⟨v, c⟩
    grind [HypercubeVert.hammingDist]
```

12. Prove that `Q[n]` is Hamiltonian unless `n = 1`. (Recall that `Q[0]` is a single vertex which is considered Hamiltonian.) Hint: `hypercube.isHamiltonian_grayPath` and `Walk.IsHamiltonian.isHamiltonian_of_adj`.

```
lemma isHamiltonian_hypercube {n : ℕ} (h1 : n ≠ 1) : Q[n].IsHamiltonian :=
  match n with
  | 0 => fun h => by simp at h
  | 1 => (h1 rfl).elim
  -- the Gray code walk is a Hamiltonian path whose endpoints differ only in the last
  -- coordinate, so they are adjacent and the path closes up into a Hamiltonian cycle
  | n + 2 => (hypercube.isHamiltonian_grayPath _).isHamiltonian_of_adj (by simp)
      (by simp [Nat.pow_succ])
```

13. Show that the Grötzsch graph `mμ(C[5])` is Hamiltonian. Hint: `IsHamiltonian.mycielskian` and `isHamiltonian_cycleGraph`.

```
example : grotzschGraph.IsHamiltonian :=
  IsHamiltonian.mycielskian (isHamiltonian_cycleGraph (by grind)) (by simp)
```

14. If `p` is a Hamiltonian walk of length at least two and there is an edge `h₁` from its `i`-th vertex to its end and an edge `h₂` from its `(i + 1)`-th vertex to its start, then `p.posaClosed h₁ h₂` is a Hamiltonian cycle. Hint: `IsPath.isCycle_posaClosed` and `isHamiltonianCycle_iff_isCycle_and_length_eq`.

```
lemma Walk.IsHamiltonian.isHamiltonianCycle_posaClosed {u v} {i} [DecidableEq V]
    {p : G.Walk u v} (hp : p.IsHamiltonian) (h2 : 2 ≤ p.length)
    (h₁ : G.Adj (p.getVert i) v) (h₂ : G.Adj (p.getVert (i + 1)) u) :
    (p.posaClosed h₁ h₂).IsHamiltonianCycle := by
  have : Fintype V := hp.fintype
```

```
  refine isHamiltonianCycle_iff_isCycle_and_length_eq.2
    ⟨hp.isPath.isCycle_posaClosed h2 h₁ h₂, ?_⟩
  simp [hp.length_add_one_eq]
```

15. Check that the vertices `1` and `4` of `chordedC5` are not adjacent and that both have degree `2`.

```
example : ¬chordedC5.Adj 1 4 ∧ chordedC5.degree 1 = 2 ∧ chordedC5.degree 4 = 2 := by
  decide
```

16. Show that `chordedC5` is not an Ore graph.

```
example : ¬chordedC5.IsOreGraph := by
  -- the non-adjacent vertices 1 and 4 have degree sum 4 < 5
  intro h
  have h14 := h 1 4
  -- contrapose! h14 swaps the hypothesis h14 and the goal False, negating both
  -- (plain contrapose does the same without simplifying the negations)
  contrapose! h14
  unfold IsOrePair
  decide
```

17. Show that the closure of `chordedC5` is complete, even though `chordedC5` is not an Ore graph. Hint: `ext`, `top_adj` and `decide +revert`.

```
example : chordedC5.closure = ⊤ := by
  ext x y
  simp only [top_adj]
  decide +revert
```

18. In an Ore graph any two distinct vertices are either adjacent or have a common neighbour. Hint: if distinct vertices u and v are non-adjacent and have no common neighbour, then their neighbourhoods are disjoint subsets of $V \setminus \{u, v\}$, so their degree sum is at most $|V| - 2$, contradicting the Ore condition.

```
open Finset in
lemma IsOreGraph.adj_or_exists_adj_adj [Fintype V] [G.LocallyFinite]
    (h : G.IsOreGraph) {u v : V} (hne : u ≠ v) :
    G.Adj u v ∨ ∃ w, G.Adj u w ∧ G.Adj v w := by
  classical
  by_cases huv : G.Adj u v
  · exact Or.inl huv
  refine Or.inr ?_
  -- The union of the neighbourhoods of u and v lives in V \ {u, v}
  have hsub : G.neighborFinset u ∪ G.neighborFinset v ⊆ univ \ {u, v} := by
    intro x hx
    simp_rw [mem_union, mem_neighborFinset, mem_sdiff, mem_univ, true_and,
      mem_insert, mem_singleton] at hx ⊢
    grind [adj_symm]
  -- So the cardinality of the union of the neighbourhoods is at most |V| - 2
  have hcard := card_le_card hsub
```

```
  -- The cardinality of V \ {u, v} is exactly |V| - 2
  have huniv : (univ \ {u, v} : Finset V).card = Fintype.card V - 2 := by
    grind
  -- there is w in the intersection of the neighbourhoods of u and v
  obtain ⟨w, hw⟩ : (G.neighborFinset u ∩ G.neighborFinset v).Nonempty := by
    by_contra! hempty
    rw [← disjoint_iff_inter_eq_empty] at hempty
    -- the neighbourhoods are disjoint, so hcard gives deg u + deg v ≤ |V| - 2
    rw [card_union_of_disjoint hempty] at hcard
    -- Since G.degree x is the cardinality of G.neighborFinset x,
    -- grind can close the arithmetic contradiction directly using Ore's
    -- condition at the pair u, v
    grind [card_neighborFinset_eq_degree, h u v hne huv]
  exact ⟨w, by simpa using hw⟩
```

19. Consequently any two vertices of an Ore graph are joined by a walk of length at most two (in the language of later chapters: an Ore graph is connected, with diameter at most two).

```
lemma IsOreGraph.exists_walk_length_le_two [Fintype V] [G.LocallyFinite]
    (h : G.IsOreGraph) (u v : V) : ∃ p : G.Walk u v, p.length ≤ 2 := by
  by_cases heq : u = v
  · exact heq ▸ ⟨nil, by simp⟩
  · rcases h.adj_or_exists_adj_adj heq with huv | ⟨w, h1, h2⟩
    · exact ⟨cons huv nil, by simp⟩
    · exact ⟨cons h1 (cons h2.symm nil), by simp⟩
```

20. An Ore graph on at least three vertices contains a vertex of degree at least two. Hint: among any three distinct vertices either some pair is non-adjacent, and so the Ore condition forces its degree sum to be at least three, or all three are pairwise adjacent.

```
open Finset in
lemma IsOreGraph.exists_two_le_degree [Fintype V] [G.LocallyFinite]
    (h : G.IsOreGraph) (hn : 3 ≤ Fintype.card V) : ∃ w, 2 ≤ G.degree w := by
  obtain ⟨a, b, c, hab, hac, hbc⟩ := Fintype.two_lt_card_iff.mp hn
  by_cases hadj : G.Adj a b
  · by_cases hadj' : G.Adj b c
    · refine ⟨b, ?_⟩
      rw [← card_neighborFinset_eq_degree]
      exact one_lt_card_iff.2 ⟨a, c, by simp [hadj.symm, hadj', hac]⟩
    · have : 3 ≤ G.degree b + G.degree c := hn.trans (h b c hbc hadj')
      grind
  · have : 3 ≤ G.degree a + G.degree b := hn.trans (h a b hab hadj)
    grind
```

21. Existence of a vertex of degree at least two implies existence of a path of length at least two.

```
lemma exists_isPath_of_two_le_degree [G.LocallyFinite] {v : V}
    (hv : 2 ≤ G.degree v) :
    ∃ (x y : V) (q : G.Walk x y), q.IsPath ∧ 2 ≤ q.length := by
  rw [← card_neighborFinset_eq_degree] at hv
  obtain ⟨x, y, hx, hy, hxy⟩ := Finset.one_lt_card_iff.1 hv
```

```
  rw [mem_neighborFinset] at hx hy
  exact ⟨x, y, cons hx.symm (cons hy nil), by simp_all [hx.symm.ne, hy.ne]⟩
```

22. Restate Mathlib's existence result `exists_isPath_forall_isPath_length_le_length` compactly.

```
variable (G) in
lemma Walk.exists_isMaximumPath [Nonempty V] [Finite G.edgeSet] :
    ∃ (u v : V) (p : G.Walk u v), p.IsMaximumPath :=
  let ⟨u, v, p, hp, hmax⟩ := exists_isPath_forall_isPath_length_le_length G
  ⟨u, v, p, hp, hmax⟩
```

23. The reverse of a maximum-length path is again a maximum-length path.

```
lemma Walk.IsMaximumPath.reverse {u v : V} {p : G.Walk u v} (h : p.IsMaximumPath) :
    p.reverse.IsMaximumPath :=
  ⟨h.isPath.reverse, fun _ _ _ hq => p.length_reverse ▸ h.length_le hq⟩
```

24. A maximum-length path in an Ore graph on at least three vertices has length at least two.

```
lemma IsOreGraph.two_le_length_of_isMaximumPath [Fintype V] [G.LocallyFinite]
    {u v} (h : G.IsOreGraph)
    (hn : 3 ≤ Fintype.card V) {p : G.Walk u v} (hmax : p.IsMaximumPath) :
    2 ≤ p.length := by
  obtain ⟨w, hw⟩ := h.exists_two_le_degree hn
  obtain ⟨x, y, q, hq, hlen⟩ := exists_isPath_of_two_le_degree hw
  exact hlen.trans (hmax.length_le hq)
```

25. A maximum-length path is in particular a *maximal* path, in the sense of the chapter's `Walk.IsMaximalPath`: neither endpoint has a neighbour missing from it. Hint: if a neighbour of the start were missing then `cons` would give a strictly longer path; for the end vertex apply the same argument to `p.reverse`, using `IsMaximumPath.reverse`.

```
lemma Walk.IsMaximumPath.isMaximalPath {u v : V} {p : G.Walk u v}
    (hmax : p.IsMaximumPath) : p.IsMaximalPath := by
  -- the start case, stated for an arbitrary maximum-length path so that we can
  -- reuse it for the reverse of p
  have hstart : ∀ {x y : V} {q : G.Walk x y}, q.IsMaximumPath →
      ∀ w, G.Adj x w → w ∈ q.support := by
    intro x y q hq w h
    by_contra hw
    simpa using hq.length_le ◁ (q.cons_isPath_iff h.symm).mpr ⟨hq.isPath, hw⟩
  exact ⟨hmax.isPath, hstart hmax,
    fun w h => by grind [hstart hmax.reverse w h, support_reverse]⟩
```

26. If `p` is a walk from a vertex in a set `S` to a vertex outside of `S`, there are adjacent vertices `c` $\in$ `S` and `d` $\notin$ `S`. Hint: `Walk.exists_boundary_dart`.

```
lemma Walk.exists_cross_adj {a b} {S : Set V} (p : G.Walk a b) (ha : a ∈ S)
    (hb : b ∉ S) : ∃ c ∈ S, ∃ d ∉ S, G.Adj c d := by
  obtain ⟨⟨c, d⟩, hcd⟩, -, ⟨hc, hd⟩⟩ := p.exists_boundary_dart S ha hb
  use c, hc, d, hd, hcd
```

27. In a non-nil closed walk the first vertex also occurs as the last, so a vertex lies in the tail of the support iff it lies in the support.

```
lemma Walk.mem_support_tail_iff {a : V} {c : G.Walk a a} (hc : ¬c.Nil) {x : V} :
    x ∈ c.support.tail ↔ x ∈ c.support := by
  refine ⟨List.mem_of_mem_tail, fun hx => ?_⟩
  rw [← c.cons_tail_support] at hx
  obtain rfl | h := List.mem_cons.mp hx
  · exact c.end_mem_tail_support hc
  · exact h
```

28. A cycle can be cut at any of its vertices to give a path from that vertex visiting exactly the cycle's vertices, of length one less than the cycle. Hint: deduce it from the chapter's `IsCycle.exists_isPath_of_mem_edges`, cutting the cycle at the first edge of `c.rotate y hy` (`Walk.mk_start_snd_mem_edges`, `Walk.rotate_edges`).

```
lemma Walk.IsCycle.exists_isPath_spanning {a : V} {c : G.Walk a a}
    (hc : c.IsCycle) {y : V} (hy : y ∈ c.support) :
    ∃ (z : V) (q : G.Walk y z), q.IsPath ∧ q.length + 1 = c.length ∧
      (∀ x, x ∈ q.support ↔ x ∈ c.support) := by
  classical
  -- Rotating the cycle to start at y gives an edge of c at y.
  have he := mk_start_snd_mem_edges (hc.rotate hy).not_nil
  rw [(c.rotate_edges y hy).perm.mem_iff] at he
  -- Cutting c open at this edge gives the required path.
  obtain ⟨q, hq, h1en, hsupp, _⟩ := hc.exists_isPath_of_mem_edges he
  exact ⟨_, q, hq, h1en, hsupp⟩
```

29. If p is a maximum-length path from u, every vertex is reachable from u, and some cycle through u of length one more than p visits exactly the vertices of p, then p is Hamiltonian. (Neither Ore's condition nor finiteness is needed.) Hint: if p missed a vertex, a walk from u to it and `exists_cross_adj` would yield an edge leaving the cycle. Break the cycle with `IsCycle.exists_isPath_spanning` and extend the resulting path with this edge to give a strictly longer path, contradicting maximality.

```
lemma Walk.IsMaximumPath.isHamiltonian_of_isCycle [DecidableEq V] {u v : V}
    {p : G.Walk u v} (hmax : p.IsMaximumPath) (hreach : ∀ w, G.Reachable u w)
    {c : G.Walk u u} (hc : c.IsCycle) (h1en : c.length = p.length + 1)
    (hsupp : ∀ x, x ∈ c.support ↔ x ∈ p.support) : p.IsHamiltonian := by
  rw [hmax.isPath.isHamiltonian_iff]
  by_contra! hcon
  -- Some vertex w is missing from the path.
  obtain ⟨w, hw⟩ := hcon
  -- The support of c (as a Set V) does not contain w
  have hw' : w ∉ {x | x ∈ c.support} := fun hwc => hw ((hsupp w).mp hwc)
```

```
  -- A boundary edge leaves c; break the cycle there and prepend it.
  obtain ⟨q⟩ := hreach w
  obtain ⟨x, hxc, z, hzc, hxz⟩ := q.exists_cross_adj c.start_mem_support hw'
  obtain ⟨z', q, hqpath, hqlen, hqsupp⟩ := hc.exists_isPath_spanning hxc
  have hznotin : z ∉ q.support := fun hzq => hzc ((hqsupp z).mp hzq)
  have hlong : (q.cons hxz.symm).IsPath :=
    (cons_isPath_iff hxz.symm q).mpr ⟨hqpath, hznotin⟩
  grind [length_cons, hmax.length_le hlong]
```

30. A maximum-length path in an Ore graph is Hamiltonian. Hint: apply `IsMaximumPath.isHamiltonian_of_isCycle`. Ore's condition supplies the cycle, via `IsOreGraph.two_le_length_of_isMaximumPath` and `IsMaximalPath.exists_cycle_spanning` (from the chapter; *p* is a maximal path by Exercise 25), and the reachability, via `IsOreGraph.exists_walk_length_le_two`.

```
lemma Walk.isHamiltonian_of_isMaximumPath [Fintype V] [DecidableEq V]
    [G.LocallyFinite] (h : G.IsOreGraph)
    (hn : 3 ≤ Fintype.card V) {u v : V} {p : G.Walk u v}
    (hmax : p.IsMaximumPath) : p.IsHamiltonian := by
  have h2 : 2 ≤ p.length := h.two_le_length_of_isMaximumPath hn hmax
  -- Build a cycle through exactly the vertices of p.
  obtain ⟨c, hcyc, hclen, hcsupp⟩ :=
    hmax.isMaximalPath.exists_cycle_spanning h2 (h u v)
  refine hmax.isHamiltonian_of_isCycle (fun w => ?_) hcyc hclen hcsupp
  obtain ⟨q, -⟩ := h.exists_walk_length_le_two u w
  exact q.reachable
```

31. Give a direct proof of Ore's theorem, not using the closure: take a maximum-length path; it is Hamiltonian of length at least two, so `IsHamiltonian.exists_isHamiltonianCycle_of_isOrePair` closes it into a Hamiltonian cycle.

```
example [Fintype V] [DecidableEq V] [G.LocallyFinite] (h : G.IsOreGraph)
    (hn : 3 ≤ Fintype.card V) : G.IsHamiltonian := by
  intro
  have : Nonempty V := Fintype.card_pos_iff.mp (hn.trans_lt' (by decide))
  -- A maximum-length path p is Hamiltonian and has length at least two, so it
  -- closes up into a Hamiltonian cycle.
  obtain ⟨u, v, p, hmax⟩ := exists_isMaximumPath G
  have hham := p.isHamiltonian_of_isMaximumPath h hn hmax
  exact ⟨u, hham.exists_isHamiltonianCycle_of_isOrePair
    (h.two_le_length_of_isMaximumPath hn hmax) (h u v)⟩
```

Chapter 9: Subgraphs

1. If *H* and *K* are complementary subgraphs of *G* (i.e. `IsCompl H K`, which means $H \sqcap K = \bot$ and $H \sqcup K = \top$) then no edge of *G* can have only one endpoint in *H*.

```
lemma Subgraph.mem_verts_iff_of_isCompl {H K : G.Subgraph} (h : IsCompl H K)
    {v w : V} (hadj : G.Adj v w) : v ∈ H.verts ↔ w ∈ H.verts := by
```

```
  -- The vertex sets of H and K are disjoint.
  have hdisj : Disjoint H.verts K.verts := by
    simpa [Set.disjoint_iff_inter_eq_empty] using
      congrArg Subgraph.verts h.inf_eq_bot
  -- The G-edge v — w lies in the sup, hence in H or K.
  have hsup : H.Adj v w ∨ K.Adj v w := Subgraph.sup_adj.1 (h.sup_eq_top ▸ hadj)
  -- In either case both endpoints lie on the same side of the partition.
  rcases hsup with hH | hK
  · exact ⟨fun _ => hH.snd_mem, fun _ => hH.fst_mem⟩
  · exact iff_of_false (hdisj.notMem_of_mem_right hK.fst_mem)
      (hdisj.notMem_of_mem_right hK.snd_mem)
```

2. Prove that the *Heyting complement* of a subgraph `H` (which is how `Hᶜ` is defined) is `(⊤ : G.Subgraph).induce H.vertsᶜ`. Hint: `le_compl_iff_disjoint_right`.

```
example (H : G.Subgraph) : Hᶜ = (⊤ : G.Subgraph).induce H.vertsᶜ := by
  -- We show Hᶜ ≤ ⊤.induce H.vertsᶜ and ⊤.induce H.vertsᶜ ≤ Hᶜ.
  -- We use disjoint_compl_left : Disjoint Hᶜ H, which forces every vertex and edge
  -- of Hᶜ to avoid H.
  have hdisj : Disjoint Hᶜ H := disjoint_compl_left
  have hverts : ∀ {v}, v ∈ Hᶜ.verts → v ∉ H.verts := fun {v} hvc hvH => by
    have : v ∈ (Hᶜ ⊓ H).verts := ⟨hvc, hvH⟩
    rw [disjoint_iff.mp hdisj] at this
    exact this
  apply le_antisymm
  · -- Hᶜ ≤ ⊤.induce H.vertsᶜ
    exact ⟨fun v hv => hverts hv, fun u v huv =>
      ⟨hverts huv.fst_mem, hverts huv.snd_mem, huv.adj_sub⟩⟩
  · -- ⊤.induce H.vertsᶜ ≤ Hᶜ: it is disjoint from H,
    -- so lies below the Heyting complement.
    rw [le_compl_iff_disjoint_right]
    refine disjoint_iff_inf_le.mpr ⟨?_, ?_⟩
    · -- verts of the inf are ⊆ ⊥.verts = ∅
      rintro v ⟨hv, hvH⟩
      exact (hv hvH).elim
    · -- adjacencies of the inf imply ⊥.Adj
      rintro u v ⟨⟨hu, _, _⟩, hH⟩
      exact hu hH.fst_mem
```

3. Strengthen `Walk.exists_isPath_or_odd_isCycle` so that the path or odd cycle it produces is a subgraph of the original walk. Hint: follow the proof of `Walk.exists_isPath_or_odd_isCycle`. Applying `Walk.toSubgraph` to `Walk.take_spec` and rewriting with `Walk.toSubgraph_append` shows that `takeUntil` and `dropUntil` give subgraphs of the walk they come from.

The proof of `Walk.exists_isPath_or_odd_isCycle` goes through almost unchanged: at each step the new path or cycle consists of part of the old path together with at most the one new edge of the walk. Since `(cons h p).toSubgraph` is by definition `G.subgraphOfAdj h ⊔ p.toSubgraph`, each step is a simple inequality in the lattice `G.Subgraph` (`sup_le_sup_left` or `le_sup_of_le_right`).

```
lemma Walk.exists_isPath_or_odd_isCycle_toSubgraph_le {a b : V} (p : G.Walk a b) :
    (∃ q : G.Walk a b, q.IsPath ∧ (Even q.length ↔ Even p.length) ∧
```

```
      q.toSubgraph ≤ p.toSubgraph) ∨
    ∃ (v : V) (c : G.Walk v v), c.IsCycle ∧ Odd c.length ∧
      c.toSubgraph ≤ p.toSubgraph := by
  classical
  induction p with
  | nil => exact .inl ⟨nil, .nil, Iff.rfl, le_rfl⟩
  | @cons a _ _ h p ih =>
    rcases ih with ⟨q, hq, hpar, hle⟩ | ⟨v, c, hc, hodd, hle⟩
    · by_cases ha : a ∈ q.support
      · have hlen := length_append _ _ ▸ congrArg length (q.take_spec ha)
        -- takeUntil and dropUntil are subgraphs of q
        have hsp := congrArg Walk.toSubgraph (q.take_spec ha)
        rw [toSubgraph_append] at hsp
        rcases Nat.even_or_odd (q.takeUntil a ha).length with he | ho
        · exact .inr ⟨a, cons h (q.takeUntil a ha),
            (cons_isCycle_iff_length_ne_one _ h).mpr ⟨hq.takeUntil ha, by grind⟩,
            by grind [length_cons],
            sup_le_sup_left ((hsp ▸ le_sup_left).trans hle) _⟩
        · exact .inl ⟨q.dropUntil a ha, hq.dropUntil ha, by grind [length_cons],
            le_sup_of_le_right ((hsp ▸ le_sup_right).trans hle)⟩
      · exact .inl ⟨cons h q, hq.cons ha, by grind [length_cons],
          sup_le_sup_left hle _⟩
    · exact .inr ⟨v, c, hc, hodd, le_sup_of_le_right hle⟩
```

4. Deduce the corresponding strengthening of `Walk.exists_odd_isCycle`: a closed walk of odd length contains an odd cycle that is a subgraph of the walk. Hint: follow the proof of `Walk.exists_odd_isCycle`.

```
theorem Walk.exists_odd_isCycle_toSubgraph_le {u : V} (w : G.Walk u u)
    (hw : Odd w.length) :
    ∃ (v : V) (c : G.Walk v v), c.IsCycle ∧ Odd c.length ∧
      c.toSubgraph ≤ w.toSubgraph := by
  cases w with
  | nil => simp at hw
  | cons h p =>
    rcases p.exists_isPath_or_odd_isCycle_toSubgraph_le with
      ⟨q, hq, hpar, hle⟩ | ⟨v, c, hc, hodd, hle⟩
    · have hqe : Even q.length := by grind [length_cons]
      exact ⟨_, cons h q, (cons_isCycle_iff_length_ne_one _ h).mpr ⟨hq, by grind⟩,
        by grind [length_cons], sup_le_sup_left hle _⟩
    · exact ⟨v, c, hc, hodd, le_sup_of_le_right hle⟩
```

5. Show that a trail that is a subgraph of a walk is no longer than the walk. Hint: every edge of the trail is an edge of the walk (`Walk.mem_edges_toSubgraph` and `Subgraph.edgeSet_mono`), and the edges of a trail are distinct (`Walk.IsTrail.edges_nodup`), so `List.Nodup.subperm` and `List.Subperm.length_le` apply.

```
theorem Walk.IsTrail.length_le_of_toSubgraph_le {a b c d : V} {p : G.Walk a b}
    {w : G.Walk c d} (hp : p.IsTrail) (hle : p.toSubgraph ≤ w.toSubgraph) :
    p.length ≤ w.length := by
  have hsub : p.edges ⊆ w.edges := fun e he => by
    simpa using Subgraph.edgeSet_mono hle (p.mem_edges_toSubgraph.mpr he)
```

```
  simpa using (hp.edges_nodup.subperm hsub).length_le
```

6. Deduce that a closed walk of odd length `k` contains an odd cycle of length at most `k`.

```
theorem Walk.exists_odd_isCycle_length_le {u : V} (w : G.Walk u u)
    (hw : Odd w.length) :
    ∃ (v : V) (c : G.Walk v v), c.IsCycle ∧ Odd c.length ∧ c.length ≤ w.length := by
  obtain ⟨v, c, hc, hodd, hle⟩ := w.exists_odd_isCycle_toSubgraph_le hw
  exact ⟨v, c, hc, hodd, hc.isTrail.length_le_of_toSubgraph_le hle⟩
```

7. Show that any edge of an induced subgraph of a subgraph is an edge of G.

```
example (s : Set V) (v w : V) (H : G.Subgraph) (h : (H.induce s).Adj v w) :
    G.Adj v w :=
  h.2.2.adj_sub
```

8. Show that the induced subgraph on `Set.univ` is the top subgraph `⊤`.

```
example : (⊤ : G.Subgraph).induce Set.univ = ⊤ := by
  ext <;> simp
```

9. Show that an induced subgraph of `(⊤ : G.Subgraph)` is induced (in the sense of `Subgraph.IsInduced`).

```
example (s : Set V) : ((⊤ : G.Subgraph).induce s).IsInduced :=
  -- Any two vertices of s adjacent in G are adjacent in the induced subgraph.
  fun _ hu _ hv hadj => ⟨hu, hv, hadj⟩
```

10. Show that the spanning coercion of a single-vertex subgraph is the empty graph `⊥ : SimpleGraph V`, which has *every* vertex of V. So `spanningCoe` forgets the vertex set of a subgraph.

```
example (v : V) : (G.singletonSubgraph v).spanningCoe = ⊥ := by
  ext a b
  simp
```

11. Show that if every induced subgraph of G on a finite set of vertices is n-colourable then G is n-colourable. Hint: use the De Bruijn–Erdős theorem `colorable_of_forall_finite_subgraph_colorable`. For a finite subgraph `H`, `H.coe ≤ G.induce H.verts` (both graphs on `↑H.verts`), so `Colorable.mono_left` applies.

```
lemma colorable_of_forall_finite_induce_colorable {n : ℕ}
    (h : ∀ s : Set V, s.Finite → (G.induce s).Colorable n) : G.Colorable n :=
  colorable_of_forall_finite_subgraph_colorable fun H hH =>
    -- H.coe ≤ G.induce H.verts: every edge of H is an edge of G.
    (h H.verts hH).mono_left (G := H.coe) fun _ _ hab => H.adj_sub hab
```

12. The finiteness of `F` in `nonempty_hom_of_forall_finite_subgraph_hom` cannot be dropped. Recall the sigma graph `sigmaCompleteNat`, the disjoint union `Σg (fun i => K[i])` of all the complete graphs `K[i]` (see `SimpleGraph.sigma`). Show that every finite subgraph of `T : SimpleGraph ℕ` has a homomorphism to `sigmaCompleteNat`, but `T` itself does not. Hint: for the first part, number the vertices of `H` using `Finite.equivFin` and send them into `K[m]`, where `m` is the number of vertices. For the second, show that adjacent vertices of `sigmaCompleteNat` lie in the same `K[i]` and have distinct labels, as in the example following its definition. Every vertex of `T` is adjacent to `0` or equal to it, so a homomorphism `f` sends every vertex into the `K[i]` containing `f 0`, and reading off the labels gives an injection `ℕ → Fin i`.

```
example (H : (T : SimpleGraph ℕ).Subgraph) (hH : H.verts.Finite) :
    Nonempty (H.coe →g sigmaCompleteNat) := by
  have := hH.to_subtype
  -- Number the m vertices of H as 0, 1, …, m - 1 and send them into K[m].
  let e := Finite.equivFin H.verts
  refine ⟨⟨fun v => ⟨Nat.card H.verts, e v⟩, fun {u v} huv => ⟨rfl, ?_⟩⟩⟩
  -- Adjacent vertices are distinct, so their labels are distinct.
  exact e.injective.ne huv.ne

example : IsEmpty ((T : SimpleGraph ℕ) →g sigmaCompleteNat) := by
  refine ⟨fun f => ?_⟩
  -- Adjacent vertices of sigmaCompleteNat lie in the same K[i] and have
  -- distinct labels.
  have key : ∀ {p q : Σ i, Fin i}, sigmaCompleteNat.Adj p q →
      p.1 = q.1 ∧ (p.2 : ℕ) ≠ q.2 := by
    rintro ⟨i, a⟩ ⟨j, b⟩ ⟨rfl, h⟩
    exact ⟨rfl, Fin.val_ne_of_ne h⟩
  -- Every vertex is adjacent to 0 (or equal to it), so f maps every vertex into
  -- the same complete graph K[i] as f 0.
  have hfst : ∀ k, (f k).1 = (f 0).1 := by
    intro k
    rcases eq_or_ne k 0 with rfl | hk
    · rfl
    · exact (key (f.map_adj hk)).1
  -- Reading off the label in K[i] gives an injection ℕ → Fin i.
  let g : ℕ → Fin (f 0).1 := fun k => ⟨(f k).2, hfst k ▸ (f k).2.isLt⟩
  have hg : g.Injective := by
    intro a b hab
    by_contra hne
    exact (key (f.map_adj hne)).2 (Fin.ext_iff.mp hab)
  have := Finite.of_injective g hg
  exact not_finite ℕ
```

13. Show that the matching number is monotone: if `G ≤ H` then `ν G ≤ ν H`. Hint: turn a matching of `G` into a matching of `H` with the same edges by building the `H.Subgraph` directly, then use the `matchingNum` API (`matchingNum_le` and `Subgraph.IsMatching.le_matchingNum`). (Mathlib's `Subgraph.IsMatching.map_ofLE` says that `M.map (Hom.ofLE h)` is a matching of `H`, but its edges are then harder to count.)

```
lemma matchingNum_mono {G H : SimpleGraph V} (h : G ≤ H) :
    ν G ≤ ν H :=
```

```
  -- every matching of G is a matching of H with the same vertices and edges:
  -- only the adj_sub field needs updating, using G ≤ H
  matchingNum_le fun M hM => Subgraph.IsMatching.le_matchingNum
    (M := ⟨M.verts, M.Adj, fun hadj => h (M.adj_sub hadj), M.edge_vert, M.symm⟩) hM
```

14. Show that `ν K[3] = 1` but `τ K[3] = 2`, so weak duality `ν G ≤ τ G` can be strict. Hint: for the first, `matchingNum_le_card_div_two` gives the upper bound and a single edge is a matching (`Subgraph.IsMatching.subgraphOfAdj`). For the second, use Mathlib's `vertexCoverNum_top`.

```
example : ν K[3] = 1 := by
  refine le_antisymm ?_ ?_
  · -- K[3] has 3 vertices, so a matching has at most 3 / 2 = 1 edges.
    simpa using matchingNum_le_card_div_two (G := K[3])
  · -- The single edge 0 — 1 is a matching with one edge.
    have h : K[3].Adj 0 1 := by decide
    simpa using (Subgraph.IsMatching.subgraphOfAdj h).le_matchingNum

example : τ K[3] = 2 := by
  rw [vertexCoverNum_top, ENat.card_eq_coe_fintype_card, Fintype.card_fin]
  rfl
```

15. Show that the infinite path has matching number `τ`. Hint: build the subgraph of `pathNat` that pairs `2 * k` with `2 * k + 1`. (An easy way to define this adjacency relation is as `pathNat.Adj m n ∧ m / 2 = n / 2`.) Show that it is a matching, and show that its edge set is infinite (`Set.infinite_of_injective_forall_mem`).

```
lemma matchingNum_pathNat : ν pathNat = τ := by
  -- Pair 2 * k with 2 * k + 1.
  let M : pathNat.Subgraph :=
    { verts := Set.univ
      Adj m n := pathNat.Adj m n ∧ m / 2 = n / 2
      adj_sub h := h.1
      edge_vert _ := trivial
      symm := ⟨fun _ _ h => ⟨h.1.symm, h.2.symm⟩⟩ }
  have hM : M.IsMatching := by
    intro v _
    refine ⟨if v % 2 = 0 then v + 1 else v - 1, ?_, ?_⟩
    · simp only [M]
      split_ifs <;> grind
    · rintro w ⟨hw, hw2⟩
      simp only at hw
      split_ifs <;> grind
  -- The edges s(2 * k, 2 * k + 1) are distinct edges of M.
  have hinf : M.edgeSet.Infinite := by
    refine Set.infinite_of_injective_forall_mem
      (f := fun k : ℕ => s(2 * k, 2 * k + 1)) (fun a b h => ?_) (fun k => ?_)
    · grind [Sym2.eq_iff]
    · grind [Subgraph.mem_edgeSet]
  exact top_le_iff.mp (hinf.encard_eq ▸ hM.le_matchingNum)
```

16. Show that in a finite graph `α G + ν G ≤ |V|`. Hint: let `I` be a maximum independent set (`exists_isNIndepSet_indepNum`) and `M` a maximum matching (`exists_isMatching_matchingNum`), so `#I = α G` and `|E(M)| = ν G`. Count the vertices of `I` inside and outside `M.verts` separately:

    - *Inside:* no edge of `M` has both ends in `I`, since `I` is independent. So sending each `v ∈ I ∩ M.verts` to the edge of `M` containing it is injective, and at most `|E(M)|` vertices of `I` lie in `M.verts` (`Subgraph.IsMatching.ncard_inter_le_of_isIndepSet`).
    - *Outside:* `M.verts` has `2 * |E(M)|` vertices (`Subgraph.IsMatching.ncard_verts`), so at most `|V| - 2 * |E(M)|` vertices lie outside it.

Adding these gives `α G ≤ |V| - ν G`. To do the arithmetic in $\mathbb{N}$, write `ν G` as the cast of `M.edgeSet.ncard` using `Set.coe_ncard_eq_encard`.

```
lemma indepNum_add_matchingNum_le [Finite V] :
    (α G : ℕ∞) + ν G ≤ Nat.card V := by
  classical
  have := Fintype.ofFinite V
  obtain ⟨I, hI⟩ := G.exists_isNIndepSet_indepNum
  obtain ⟨M, hM, hcard⟩ := exists_isMatching_matchingNum (G := G)
  -- Sending each vertex of I covered by M to its edge of M is injective
  -- (IsMatching.ncard_inter_le_of_isIndepSet).
  have h1 : (↑I ∩ M.verts).ncard ≤ M.edgeSet.ncard :=
    hM.ncard_inter_le_of_isIndepSet hI.isIndepSet
  -- The other vertices of I lie outside M.verts.
  have h2 : (↑I \ M.verts).ncard ≤ Nat.card V - 2 * M.edgeSet.ncard := by
    rw [← hM.ncard_verts, ← Set.ncard_univ, ← Set.ncard_sdiff (Set.subset_univ _)]
    exact Set.ncard_le_ncard (fun v hv => ⟨trivial, hv.2⟩)
  have h3 : I.card = (↑I ∩ M.verts).ncard + (↑I \ M.verts).ncard := by
    rw [Set.ncard_inter_add_ncard_sdiff_eq_ncard, Set.ncard_coe_finset]
  have h4 : 2 * M.edgeSet.ncard ≤ Nat.card V := by
    rw [← hM.ncard_verts]
    exact Set.ncard_le_card _
  rw [← hcard, ← Set.coe_ncard_eq_encard, ← hI.card_eq]
  norm_cast
  grind
```

17. The supremum of an increasing sequence of matchings is a matching. Hint: a vertex of `⨆ n, M n` lies in some `M n` (`Subgraph.verts_iSup`) and an edge of `⨆ n, M n` lies in some `M m` (`Subgraph.iSup_adj`); compare the two at the later stage `max n m`.

```
lemma Subgraph.isMatching_iSup_of_monotone {M : ℕ → G.Subgraph}
    (hmono : Monotone M) (hM : ∀ n, (M n).IsMatching) : (⨆ n, M n).IsMatching := by
  intro v hv
  rw [Subgraph.verts_iSup, Set.mem_iUnion] at hv
  obtain ⟨n, hn⟩ := hv
  obtain ⟨w, hw, -⟩ := hM n hn
  refine ⟨w, Subgraph.iSup_adj.mpr ⟨n, hw⟩, fun w' hw' => ?_⟩
  obtain ⟨m, hm⟩ := Subgraph.iSup_adj.mp hw'
  -- Both edges are present at the later stage max n m, which is a matching.
  obtain ⟨u, -, hu⟩ := hM (max n m) ((hmono (le_max_left n m)).1 hn)
```

```
  rw [hu w' ((hmono (le_max_right n m)).2 hm),
    hu w ((hmono (le_max_left n m)).2 hw)]
```

18. If ν G = τ then every finite set of vertices S misses some edge of G. Hint: otherwise S is a vertex cover, so weak duality (matchingNum_le_vertexCoverNum) and IsVertexCover.vertexCoverNum_le force S.encard = τ.

```
lemma exists_adj_notMem_of_matchingNum_eq_top (h : ν G = τ) {S : Set V}
    (hS : S.Finite) : ∃ a b, G.Adj a b ∧ a ∉ S ∧ b ∉ S := by
  by_contra! hcon
  -- Otherwise S is a vertex cover, contradicting weak duality ν G ≤ τ G.
  have hcover : G.IsVertexCover S := by
    intro a b hab
    by_cases ha : a ∈ S
    · exact Or.inl ha
    · exact Or.inr (hcon a b hab ha)
  have := (G.matchingNum_le_vertexCoverNum).trans hcover.vertexCoverNum_le
  rw [h, top_le_iff, Set.encard_eq_top_iff] at this
  exact this hS
```

19. A matching M with finitely many vertices in a graph with ν G = τ can be extended by one edge. Hint: take an edge avoiding M.verts from the previous exercise and add it with Subgraph.IsMatching.sup, whose disjointness hypothesis is about supports (Subgraph.IsMatching.support_eq_verts), then count edges with Set.encard_insert_of_notMem.

```
lemma Subgraph.IsMatching.exists_le_encard_eq_add_one (h : ν G = τ)
    {M : G.Subgraph} (hM : M.IsMatching) (hfin : M.verts.Finite) :
    ∃ N : G.Subgraph, M ≤ N ∧ N.IsMatching ∧ N.verts.Finite ∧
      N.edgeSet.encard = M.edgeSet.encard + 1 := by
  obtain ⟨a, b, hab, ha, hb⟩ := exists_adj_notMem_of_matchingNum_eq_top h hfin
  refine ⟨M ⊔ G.subgraphOfAdj hab, le_sup_left, ?_, ?_, ?_⟩
  · -- The new edge avoids M, so the union is a matching.
    refine hM.sup (Subgraph.IsMatching.subgraphOfAdj hab) ?_
    rw [hM.support_eq_verts,
        (Subgraph.IsMatching.subgraphOfAdj hab).support_eq_verts,
        subgraphOfAdj_verts, Set.disjoint_left]
    rintro x hx (rfl | rfl)
    · exact ha hx
    · exact hb hx
  · rw [Subgraph.verts_sup, subgraphOfAdj_verts]
    exact hfin.union (Set.toFinite _)
  · rw [Subgraph.edgeSet_sup, edgeSet_subgraphOfAdj, Set.union_singleton,
        Set.encard_insert_of_notMem]
    rw [Subgraph.mem_edgeSet]
    exact fun hadj => ha (M.edge_vert hadj)
```

20. If ν G = τ then G has an infinite matching. Hint: the stub chooses a step function next, which extends each finite matching by one edge (the previous exercise), and iterates it from ⊥; each M n is then a finite matching by construction. Show by induction that it has exactly n edges. Each step extends the previous matching, so M is monotone (monotone_nat_of_le_succ) and ⊔ n, M n is a matching by Exercise 17, with at least n edges for every n.

```
lemma exists_isMatching_encard_eq_top (h : ν G = ⊤) :
    ∃ M : G.Subgraph, M.IsMatching ∧ M.edgeSet.encard = ⊤ := by
  -- Choose, for every finite matching, an extension by one edge. Packaging the
  -- finite matchings as a subtype FM makes this a total function next.
  let FM := {M : G.Subgraph // M.IsMatching ∧ M.verts.Finite}
  have : ∀ M : FM, ∃ N : FM,
      M.1 ≤ N.1 ∧ N.1.edgeSet.encard = M.1.edgeSet.encard + 1 := by
    rintro ⟨M, hM, hfin⟩
    obtain ⟨N, hle, hN, hNfin, hcard⟩ := hM.exists_le_encard_eq_add_one h hfin
    exact ⟨⟨N, hN, hNfin⟩, hle, hcard⟩
  choose next hnext hcard using this
  -- Iterate from the empty matching.
  let M : ℕ → FM := fun n => next^[n] ⟨⊥, Subgraph.isMatching_bot, by simp⟩
  have hstep : ∀ n, M (n + 1) = next (M n) := fun n =>
    Function.iterate_succ_apply' _ _ _
  -- Each M n is a finite matching by construction; it has exactly n edges.
  have hcount : ∀ n : ℕ, (M n).1.edgeSet.encard = n := by
    intro n
    induction n with
    | zero => simp [M]
    | succ n ih => rw [hstep, hcard, ih, Nat.cast_succ]
  have hmono : Monotone fun n => (M n).1 := monotone_nat_of_le_succ fun n => by
    rw [hstep]
    exact hnext (M n)
  refine ⟨⨆ n, (M n).1,
    Subgraph.isMatching_iSup_of_monotone hmono fun n => (M n).2.1, ?_⟩
  -- The union contains a matching with n edges for every n, so it is infinite.
  have hle : ∀ n : ℕ, (n : ℕ∞) ≤ (⨆ n, (M n).1).edgeSet.encard := fun n => by
    rw [← hcount n]
    exact Set.encard_le_encard ◁ Subgraph.edgeSet_mono ◁ le_iSup_of_le n le_rfl
  exact ENat.eq_top_iff_forall_ge.2 hle
```

21. Conclude that the matching number is attained in every graph, finite or not: this is the analogue of Mathlib's `vertexCoverNum_exists`. Hint: if `ν G = ⊤` use the previous exercise; otherwise a finite supremum in `ℕ∞` is attained, so conclude as in the proof of `exists_isMatching_matchingNum`, with `ENat.exists_eq_iSup_of_lt_top` in place of `exists_eq_ciSup_of_finite`.

```
lemma matchingNum_exists (G : SimpleGraph V) :
    ∃ M : G.Subgraph, M.IsMatching ∧ M.edgeSet.encard = ν G := by
  by_cases htop : ν G = ⊤
  · obtain ⟨M, hM, hcard⟩ := exists_isMatching_encard_eq_top htop
    exact ⟨M, hM, hcard.trans htop.symm⟩
  · -- A finite supremum in ℕ∞ is attained.
    have : Nonempty {M : G.Subgraph // M.IsMatching} :=
      ⟨⟨⊥, Subgraph.isMatching_bot⟩⟩
    obtain ⟨M, hM⟩ := ENat.exists_eq_iSup_of_lt_top
      (f := fun x : {M : G.Subgraph // M.IsMatching} =>
        (x : G.Subgraph).edgeSet.encard)
      (by grind [← matchingNum_eq_iSup_subtype])
    exact ⟨M, M.2, by rw [matchingNum_eq_iSup_subtype]; exact hM⟩
```

22. Show that `α Q[n + 1] = 2 ^ n`. Hint: the upper bound follows from Exercise 16 and `matchingNum_hypercube_succ`. For the lower bound, `Q[n + 1]` is bipartite (`IsBipartite.hypercube`), so `Colorable.natCard_le_mul_indepNum` applies.

```
example (n : ℕ) : α Q[n + 1] = 2 ^ n := by
  refine le_antisymm ?_ ?_
  · -- The upper bound α G + ν G ≤ |V|, with ν Q[n + 1] = 2 ^ n.
    have h := indepNum_add_matchingNum_le (G := Q[n + 1])
    rw [matchingNum_hypercube_succ, Nat.card_eq_fintype_card, card_hypercubeVert,
      pow_succ, mul_two] at h
    have h' : α Q[n + 1] + 2 ^ n ≤ 2 ^ n + 2 ^ n := by exact_mod_cast h
    grind
  · -- Q[n + 1] is 2-colourable, so it has at most 2 * α Q[n + 1] vertices.
    have h := (IsBipartite.hypercube (n + 1)).natCard_le_mul_indepNum
    rw [Nat.card_eq_fintype_card, card_hypercubeVert, pow_succ] at h
    grind
```

23. Show that, provided there is at least one colour, `s` is `n`-colourable in the sense of `ColorableOn` iff the induced subgraph `G.induce s` is `n`-colourable. (So partial colourings really do capture colourings of induced subgraphs.) Why is the hypothesis `0 < n` needed?

A `G.ColoringOn (Fin n) s` colours every vertex of `G`, including those outside `s`. If `n = 0` and `V` is nonempty there is no colour for them, so `G.ColorableOn 0 ∅` fails, although `G.induce ∅` has no vertices and so is `0`-colourable.

```
lemma colorableOn_iff_colorable_induce {n : ℕ} (hn : 0 < n) (s : Set V) :
    G.ColorableOn n s ↔ (G.induce s).Colorable n := by
  classical
  constructor
  · -- Restrict the colouring to s.
    rintro ⟨C⟩
    exact ⟨Coloring.mk (fun v => C v) fun {v w} h => C.valid ⟨v.2, w.2, h⟩⟩
  · -- Colour the vertices outside s with colour 0.
    rintro ⟨c⟩
    refine ⟨Coloring.mk (fun v => if hv : v ∈ s then c ⟨v, hv⟩ else ⟨0, hn⟩) ?_⟩
    rintro v w ⟨hv, hw, hadj⟩
    simp only [hv, hw, ↓reduceDIte]
    exact c.valid (by simpa using hadj)
```

Chapter 10: Connectivity

1. Show that for any graph `G` on a nonempty vertex type, either `G` or its complement `Gᶜ` is connected. Hint: if `u` and `v` are not reachable from each other in `G`, show that any two vertices are joined in `Gᶜ`, either directly or via `u` or `v`. (Mathlib has this as `connected_or_connected_compl`, but try to prove it directly.)

If `G` is not preconnected, pick vertices `u` and `v` that are not reachable from each other. Vertices `x` and `y` that are not reachable from each other in `G` are distinct and not adjacent, so they are adjacent in `Gᶜ`. If `x` and `y` are reachable from each other in `G`, then they lie in a component of `G` that misses `u` or `v`, and both are adjacent in `Gᶜ` to whichever of these it misses.

```
example [Nonempty V] : G.Connected ∨ Gᶜ.Connected := by
  by_cases hG : G.Preconnected
  · exact .inl ⟨hG⟩
  -- otherwise some pair u, v is not reachable in G
  simp only [Preconnected, not_forall] at hG
  obtain ⟨u, v, huv⟩ := hG
  -- vertices that are not reachable from each other in G are adjacent in Gᶜ
  have hadj (x y : V) (h : ¬G.Reachable x y) : Gᶜ.Adj x y :=
    ⟨fun hxy => h (hxy ▸ .refl _), fun hxy => h hxy.reachable⟩
  refine .inr ⟨fun x y => ?_⟩
  by_cases hxy : G.Reachable x y
  · -- x and y lie in one component of G, which misses u or v
    by_cases hxu : G.Reachable x u
    · have hxv : ¬G.Reachable x v := fun h => huv (hxu.symm.trans h)
      have hyv : ¬G.Reachable y v := fun h => hxv (hxy.trans h)
      exact (hadj x v hxv).reachable.trans (hadj y v hyv).reachable.symm
    · have hyu : ¬G.Reachable y u := fun h => hxu (hxy.trans h)
      exact (hadj x u hxu).reachable.trans (hadj y u hyu).reachable.symm
  · exact (hadj x y hxy).reachable
```

2. Show that the complete bipartite graph `K[s + 1, t + 1]` is connected.

```
example (s t : ℕ) : K[s + 1, t + 1].Connected := by
  refine (connected_iff_exists_forall_reachable _).2 ⟨.inl 0, fun v => ?_⟩
  match v with
  | .inl b =>
    have h : K[s + 1, t + 1].Adj (.inl 0) (.inr 0) := by simp
    use .cons h ◁ .cons (by simp) .nil
  | .inr b =>
    exact (by simp : K[s + 1, t + 1].Adj (.inl 0) (.inr b)).reachable
```

3. Construct the equivalence between the connected components of the empty graph and its vertex type. Hint: `ConnectedComponent.lift` and `ConnectedComponent.ind`; what does a walk in `⊥` look like?

Any walk in `⊥` is `nil`, so reachability in `⊥` is equality and `connectedComponentMk` is a bijection. The connected components of `⊥` correspond to its vertices.

```
def botComponentEquiv : (⊥ : SimpleGraph V).ConnectedComponent ≃ V where
  toFun := ConnectedComponent.lift id fun v w p _ => by
    cases p with
    | nil => rfl
    | cons h _ => exact h.elim
  invFun := (⊥ : SimpleGraph V).connectedComponentMk
  left_inv := ConnectedComponent.ind fun v => rfl
```

4. Prove the universal property of the disjoint union: a family of homomorphisms `G i →g H` into a common graph `H` gives a homomorphism `Σg G →g H`. Hint: adjacent vertices of the disjoint union have the same index.

An edge `⟨i, u⟩ — ⟨i, v⟩` of the disjoint union is an edge of `G i`, which `f i` preserves. The universal property of the disjoint union: a family of homomorphisms `G i →g H` into a common graph combines into a single homomorphism `Σg G →g H` out of the disjoint union.

```
def Hom.sigmaElim {ι : Type*} {V : ι → Type*} {W : Type*}
    (G : (i : ι) → SimpleGraph (V i))
    {H : SimpleGraph W} (f : (i : ι) → G i →g H) : Σg G →g H where
  toFun p := f p.1 p.2
  map_rel' := by
    rintro ⟨i, u⟩ ⟨j, v⟩ ⟨rfl, hadj⟩
    exact (f i).map_rel' hadj
```

5. Use the previous exercise and `isoSigmaConnectedComponents` to build a homomorphism `G →g H` from a homomorphism out of each connected component of `G`. (Mathlib provides this map as `SimpleGraph.homOfConnectedComponents`.)

We transport the homomorphism out of the disjoint union of the components along the isomorphism `isoSigmaConnectedComponents`: each vertex is sent via the homomorphism of its own component.

```
example {W : Type*} {H : SimpleGraph W}
    (f : (C : G.ConnectedComponent) → C.toSimpleGraph →g H) : G →g H :=
  (Hom.sigmaElim _ f).comp isoSigmaConnectedComponents.toHom
```

6. Show that if `G` has no isolated vertices then the Mycielskian `mμ(G)` has no bridges. Hint: `isBridge_mycielskian_iff`.

```
example (hG : ∀ v, ¬G.IsIsolated v) {e : Sym2 (Option (V ⊕ V))}
    (he : e ∈ mμ(G).edgeSet) : ¬mμ(G).IsBridge e := fun hb => by
  obtain ⟨u, hu, -⟩ := (isBridge_mycielskian_iff he).mp hb
  exact hG u hu
```

7. Show that a tree has at most one connected component.

```
example (h : G.IsTree) : Subsingleton G.ConnectedComponent :=
  h.connected.preconnected.subsingleton_connectedComponent
```

8. Show that the path graph `P[n + 1]` is a tree. (Mathlib has this as `isTree_pathGraph_add_one`, but prove it directly.) Hint: deduce acyclicity from `isAcyclic_pathNat` using `IsAcyclic.comap`.

The path graph is connected, and `Fin.val` is an injective homomorphism from `P[n + 1]` into the infinite path `pathNat`, which is acyclic. So any cycle in `P[n + 1]` would map to a cycle in `pathNat`.

```
example (n : ℕ) : P[n + 1].IsTree :=
  ⟨connected_pathGraph_add_one n,
    isAcyclic_pathNat.comap ⟨Fin.val, by grind [pathGraph_adj]⟩ Fin.val_injective⟩
```

We can also prove that the path graph is a tree by showing that it has the correct number of edges (`isTree_iff_connected_and_card`).

```
example (n : ℕ) : P[n + 1].IsTree := by
  refine isTree_iff_connected_and_card.2 ⟨connected_pathGraph_add_one _, ?_⟩
  simp only [Nat.card_eq_fintype_card, card_edgeSet, Fintype.card_fin,
    Nat.add_right_cancel_iff]
  -- show P[n + 1].edgeFinset.card = n by exhibiting a bijection from Fin n
  have he : P[n + 1].edgeFinset =
      Finset.univ.image fun i : Fin n => s(i.castSucc, i.succ) := by
    ext e
    induction e using Sym2.ind with | _ a b => ?_
    simp only [mem_edgeFinset, mem_edgeSet, pathGraph_adj, Finset.mem_image,
      Finset.mem_univ, true_and]
    constructor
    · rintro (h | h)
      · exact ⟨⟨a, by grind⟩, by grind⟩
      · exact ⟨⟨b, by grind⟩, by grind⟩
    · grind
  rw [he, Finset.card_image_of_injective _ fun a b h => by grind, Finset.card_univ,
    Fintype.card_fin]
```

9. Show that a finite tree has at least as many leaves (vertices of degree one) as the degree of any of its vertices.

Hint: every vertex `u ≠ v` has degree at least one (`Preconnected.degree_pos_of_nontrivial`), so `G.degree u + (if G.degree u = 1 then 1 else 0)` is at least 2. Sum this over all vertices and compare with `IsTree.card_edgeFinset` and the handshake lemma in the form `sum_degrees_eq_twice_card_edges'` from The Handshake Lemma (p. 73).

```
open Finset in
lemma IsTree.degree_le_card_leaves [Fintype V] [G.LocallyFinite] (h : G.IsTree)
    (v : V) : G.degree v ≤ #{u | G.degree u = 1} := by
  classical
  -- each vertex u ≠ v has degree ≥ 1, so its degree plus leaf indicator is ≥ 2
  have hge : ∀ u ∈ univ.erase v,
      2 ≤ G.degree u + if G.degree u = 1 then 1 else 0 := by
    intro u hu
    have : Nontrivial V := ⟨u, v, ne_of_mem_erase hu⟩
    have := h.connected.preconnected.degree_pos_of_nontrivial u
    split_ifs <;> grind
  have hsum := sum_le_sum hge
  rw [sum_const, card_erase_of_mem (mem_univ v), card_univ, smul_eq_mul] at hsum
  -- split off v from the full sum, which is 2 * #edges + #leaves
  have hsplit := add_sum_erase univ
    (fun u => G.degree u + if G.degree u = 1 then 1 else 0) (mem_univ v)
  have htot : ∑ u, (G.degree u + if G.degree u = 1 then 1 else 0)
      = 2 * #G.edgeFinset + #{u | G.degree u = 1} := by
    rw [sum_add_distrib, G.sum_degrees_eq_twice_card_edges', card_filter]
  grind [h.card_edgeFinset]
```

10. Show that deleting a leaf (a vertex of degree one) from a tree leaves a tree. Hint: `Connected.induce_compl_singleton_of_degree_eq_one` and `IsAcyclic.induce`.

```
example [G.LocallyFinite] (hT : G.IsTree) {v : V}
    (hv : G.degree v = 1) : (G.induce {v}ᶜ).IsTree :=
  ⟨hT.connected.induce_compl_singleton_of_degree_eq_one hv, hT.isAcyclic.induce _⟩
```

11. Show that every finite connected graph with at least two vertices has a vertex whose deletion leaves it connected (so not every vertex is a cut vertex). Hint: take a leaf `v` of a spanning tree `T` (`IsTree.exists_vert_degree_one_of_nontrivial`). Deleting it leaves `T` connected (`Connected.induce_compl_singleton_of_degree_eq_one`), and `Connected.mono` transfers this to `G`.

Mathlib has this result as `Connected.exists_connected_induce_compl_singleton_of_finite_nontrivial`.

```
example [Finite V] [Nontrivial V] (h : G.Connected) :
    ∃ v, (G.induce {v}ᶜ).Connected := by
  classical
  have := Fintype.ofFinite V
  -- a spanning tree T of G has a leaf v
  obtain ⟨T, hTG, hT⟩ := h.exists_isTree_le
  obtain ⟨v, hv⟩ := hT.exists_vert_degree_one_of_nontrivial
  -- deleting a leaf keeps T connected, and T.induce {v}ᶜ ≤ G.induce {v}ᶜ
  exact ⟨v, (hT.connected.induce_compl_singleton_of_degree_eq_one hv).mono
    fun _ _ h => hTG h⟩
```

12. Use Euler's theorem (`Connected.exists_closed_isEulerian`) to show that the complete graph `K[2 * n + 3]` has a closed Eulerian walk, and so does the hypercube `Q[2 * n + 2]`.

Both graphs are connected with all degrees even, so Euler's theorem applies directly.

```
example (n : ℕ) : ∃ u, ∃ p : K[2 * n + 3].Walk u u, p.IsEulerian := by
  apply Connected.exists_closed_isEulerian
  · exact connected_top_iff.mpr inferInstance
  · intro v
    have : (⊤ : SimpleGraph (Fin (2 * n + 3))).degree v = 2 * n + 2 := by
      simp [complete_graph_degree]
    rw [this]
    exact ⟨n + 1, by grind⟩

example (n : ℕ) : ∃ u, ∃ p : Q[2 * n + 2].Walk u u, p.IsEulerian :=
  (connected_hypercube _).exists_closed_isEulerian fun v => by
    rw [IsRegularOfDegree.hypercube]
    exact ⟨n + 1, by grind⟩
```

13. Show that a k-vertex-connected graph has minimum degree at least k. Hint: `exists_minimal_degree_vertex` and `IsVertexConnected.le_degree`.

```
lemma IsVertexConnected.le_minDegree {k : ℕ∞} [Fintype V] [Nonempty V]
    [DecidableRel G.Adj] (hG : G.IsVertexConnected k) : k ≤ δ(G) := by
  obtain ⟨v, hv⟩ := G.exists_minimal_degree_vertex
  rw [hv]
  exact hG.le_degree v
```

14. Exercises 14–16 show that if G is connected and k-vertex-connected then $G \square K_2$ is $(k+1)$-vertex-connected, and deduce that the hypercube `Q[n]` is n-vertex-connected.

For a finset S of vertices of $G \square K_2$ and `b : Bool`, let `S.fiberProj b` be the finset of vertices `z` of G whose copy `(z, b)` in the G-fibre at `b` lies in S. Show that if `0 < k` and S has at most k elements then some G-fibre contains less than k vertices of S. Hint: the sizes of `S.fiberProj false` and `S.fiberProj true` add up to the size of S; use `Finset.card_preimage` and `Finset.card_filter_add_card_filter_not`. The two sizes add up to at most k, so they cannot both be at least `k > 0`.

```
lemma _root_.Finset.exists_fiberProj_card_lt {k : ℕ} {S : Finset (V × Bool)}
    (hS : S.card ≤ k) (hk : 0 < k) : ∃ b, (S.fiberProj b).card < k := by
  classical
  have hc (b : Bool) : (S.fiberProj b).card = (S.filter (·.2 = b)).card := by
    rw [Finset.fiberProj, Finset.card_preimage]; congr 1; ext ⟨x, c⟩; simp [eq_comm]
  have h := Finset.card_filter_add_card_filter_not (s := S) (fun x => x.2 = false)
  simp only [Bool.not_eq_false, ← hc] at h
  grind
```

15. Let G be k-vertex-connected and let S be a finset of at most k vertices of $G \square K_2$. Show that from any vertex `(z, c)` not in S there is a walk into the G-fibre at `b` that avoids S.

Hint: if `c ≠ b` and `(z, b) ∈ S`, find a neighbour `w` of `z` such that neither `(w, false)` nor `(w, true)` lies in S: `z` has at least k neighbours by `IsVertexConnected.le_encard_neighborSet`, but less than k elements of S have a first coordinate other than `z`. Then step to `(w, c)` and cross to `(w, b)`. If `c = b` the empty walk works, and if `(z, b) ∉ S` we cross directly. Otherwise the counting argument of the hint gives a neighbour `w` of `z` with both copies outside S, and we step to `(w, c)` and cross to `(w, b)`.

```
private lemma exists_good_neighbor {k : ℕ} (hG : G.IsVertexConnected k)
    {S : Finset (V × Bool)} (hS : S.card ≤ k) {z : V} {b : Bool} (hzb : (z, b) ∈ S) :
    ∃ w, G.Adj z w ∧ ∀ c, (w, c) ∉ S := by
  classical
  by_contra! h
  have hz : z ∈ S.image Prod.fst := Finset.mem_image_of_mem _ hzb
  -- every neighbour of z is a first coordinate of S other than z
  have hsub : G.neighborSet z ⊆ ↑((S.image Prod.fst).erase z) := fun w hw => by
    obtain ⟨c, hc⟩ := h w hw
    exact Finset.mem_erase.mpr ⟨hw.ne', Finset.mem_image_of_mem Prod.fst hc⟩
  have hk := (hG.le_encard_neighborSet z).trans (Set.encard_le_encard hsub)
  grind [Set.encard_coe_eq_coe_finsetCard, Nat.cast_le, Finset.card_erase_of_mem hz]

lemma IsVertexConnected.exists_walk_to_fiber {k : ℕ} (hG : G.IsVertexConnected k)
    {S : Finset (V × Bool)} (hS : S.card ≤ k) {z : V} {c : Bool} (hzc : (z, c) ∉ S)
    (b : Bool) :
    ∃ (z' : V) (p : (G □ (⊤ : SimpleGraph Bool)).Walk (z, c) (z', b)),
      ∀ x ∈ p.support, x ∉ S := by
  by_cases hcb : c = b
  · subst hcb
    exact ⟨z, .nil, by simp [hzc]⟩
  have hcross (w : V) : (G □ ⊤).Adj (w, c) (w, b) := boxProd_adj_right.mpr hcb
  by_cases hzb : (z, b) ∈ S
  · obtain ⟨w, hzw, hw⟩ := exists_good_neighbor hG hS hzb
```

```
    exact ⟨w, .cons (boxProd_adj_left.mpr hzw) (hcross w).toWalk, by simp [hzc, hw]⟩
  · exact ⟨z, (hcross z).toWalk, by simp [hzc, hzb]⟩
```

16. Show that if G is connected and k-vertex-connected then $G \square K_2$ is $(k + 1)$-vertex-connected, and deduce that the hypercube `Q[n]` is n-vertex-connected. (For $k \geq 1$ connectedness follows from k-vertex-connectivity, but two isolated vertices form a 0-vertex-connected graph G for which $G \square K_2$ is disconnected.)

Hint: $G \square K_2$ has twice as many vertices as G, and by `IsVertexConnected.of_finset` it is enough to avoid finsets S. If $k = 0$ then S is empty and $G \square K_2$ is connected by `Connected.boxProd`. Otherwise, take vertices `(u, c)` and `(v, d)` not in S. Pick b such that the G-fibre at b contains less than k vertices of S (Exercise 14), and walk from both vertices into the G-fibre at b avoiding S (Exercise 15). Then join the ends of these walks by a walk in G avoiding S`.fiberProj b`, copied into the G-fibre at b with `Walk.boxProdLeft`. The walk from `(u, c)` into the G-fibre at b, the copy of the walk in G, and the reverse of the walk from `(v, d)` into the G-fibre at b together form a walk from `(u, c)` to `(v, d)`. It avoids S, since the copy only visits vertices (z, b) with $z \notin S$`.fiberProj b`. Since `Q[n + 1]` is `Q[n]` $\square$ K_2, the statement about the hypercube follows by induction on n.

```
lemma IsVertexConnected.boxProd_top_bool {k : ℕ} (hG : G.IsVertexConnected k)
    (hG_conn : G.Connected) :
    (G □ (⊤ : SimpleGraph Bool)).IsVertexConnected (k + 1) := by
  refine .of_finset ?_ ?_
  · -- |V × Bool| = 2 |V| and k + 1 ≤ |V|
    have h := hG.1
    rw [ENat.card_prod, ENat.card_eq_coe_fintype_card (α := Bool), Fintype.card_bool,
      Nat.cast_ofNat]
    calc (k : ℕ∞) + 1 + 1 ≤ ((k : ℕ∞) + 1) * 2 := by norm_cast; grind
      _ ≤ _ := by gcongr
  intro S hS ⟨u, c⟩ ⟨v, d⟩ hu hv
  replace hS : S.card < k + 1 := by exact_mod_cast hS
  rcases Nat.eq_zero_or_pos k with rfl | hk
  · -- for k = 0 the set S is empty and G □ K₂ is connected
    obtain ⟨p⟩ := (hG_conn.boxProd connected_top).preconnected (u, c) (v, d)
    exact ⟨p, by simp [Finset.card_eq_zero.mp (Nat.lt_one_iff.mp hS)]⟩
  have hS' : S.card ≤ k := Nat.le_of_lt_succ hS
  obtain ⟨b, hb⟩ := S.exists_fiberProj_card_lt hS' hk
  obtain ⟨u', pu, hpu⟩ := hG.exists_walk_to_fiber hS' hu b
  obtain ⟨v', pv, hpv⟩ := hG.exists_walk_to_fiber hS' hv b
  obtain ⟨q, hq⟩ := hG.exists_walk (S.fiberProj b) (Nat.cast_lt.mpr hb) u' v'
    (by simpa using hpu _ pu.end_mem_support)
    (by simpa using hpv _ pv.end_mem_support)
  refine ⟨pu.append ((q.boxProdLeft ⊤ b).append pv.reverse), fun x hx => ?_⟩
  simp only [Walk.mem_support_append_iff, Walk.support_reverse, List.mem_reverse,
    Walk.support_boxProdLeft, List.mem_map] at hx
  grind [Finset.mem_fiberProj]

lemma isVertexConnected_hypercube (n : ℕ) : Q[n].IsVertexConnected n := by
  induction n with
  | zero =>
    refine ⟨?_, fun S hS => absurd hS (by simp)⟩
    simp
  | succ n ih =>
```

```
    exact ih.boxProd_top_bool (connected_hypercube n)
```

17. Deduce that the hypercube `Q[n]` is n-edge-connected, and that `Q[n + 2]` has no bridges. Hint: Exercise 16 and `IsVertexConnected.isEdgeConnected`. Then `Q[n + 2]` is 2-edge-connected by `IsEdgeConnected.anti`; now use `isBridge_iff_not_isEdgeReachable_two`.

```
lemma isEdgeConnected_hypercube (n : ℕ) : Q[n].IsEdgeConnected n :=
  (isVertexConnected_hypercube n).isEdgeConnected

example (n : ℕ) {u v : HypercubeVert (n + 2)} (h : Q[n + 2].Adj u v) :
    ¬Q[n + 2].IsBridge s(u, v) := fun hb =>
  (isBridge_iff_not_isEdgeReachable_two h).mp hb
    ((isEdgeConnected_hypercube (n + 2)).anti (by simp) u v)
```

18. Show that the edge connectivity of a finite graph with at least two vertices is at most its minimum degree. Together with `vertexConnectivity_le_edgeConnectivity` this gives Whitney's inequalities [Whitney 1932] $\kappa(G) \le \lambda(G) \le \delta(G)$. Hint: `exists_minimal_degree_vertex` and `edgeConnectivity_le_degree`.

```
lemma edgeConnectivity_le_minDegree [Fintype V] [Nontrivial V]
    [DecidableRel G.Adj] : λ(G) ≤ δ(G) := by
  obtain ⟨v, hv⟩ := G.exists_minimal_degree_vertex
  rw [hv]
  exact edgeConnectivity_le_degree v
```

19. Show that `κ(bowtie) < λ(bowtie)`, so the first of Whitney's inequalities can be strict. Hint: the bowtie is 2-edge-connected by `isEdgeConnected_two_iff_forall_preconnected` and `decide`. It is not 2-vertex-connected, by `isVertexConnected_iff_induce` with `S = {0}` and `decide`. Finally use `le_edgeConnectivity_iff` and `le_vertexConnectivity_iff`.

In fact `κ(bowtie) = 1` and `λ(bowtie) = 2`, the minimum degree.

```
example : κ(bowtie) < λ(bowtie) := by
  -- deleting any single edge leaves the bowtie preconnected
  have h2e : bowtie.IsEdgeConnected 2 :=
    isEdgeConnected_two_iff_forall_preconnected.mpr (by decide)
  -- deleting the shared vertex 0 disconnects it
  have h2v : ¬bowtie.IsVertexConnected 2 := fun h =>
    absurd ((isVertexConnected_iff_induce.mp h).2 {0} (by simp)) (by decide)
  have hκ : κ(bowtie) < 2 := not_le.mp (mt le_vertexConnectivity_iff.mp h2v)
  exact hκ.trans_le (le_edgeConnectivity_iff.mpr h2e)
```

20. Show that `λ(barbell) < δ(barbell)`, so the second of Whitney's inequalities can also be strict. Hint: the barbell is not 2-edge-connected, by `isEdgeConnected_two_iff_forall_preconnected` with `e = s(3, 4)` and `decide`. Every vertex has degree at least 3, by `le_minDegree_of_forall_le_degree` and `decide`.

In fact `λ(barbell) = 1`, since `3 — 4` is a bridge, and `δ(barbell) = 3`.

```
example : λ(barbell) < δ(barbell) := by
  -- deleting the bridge 3 — 4 disconnects the barbell
  have h2e : ¬barbell.IsEdgeConnected 2 := fun h =>
    absurd (isEdgeConnected_two_iff_forall_preconnected.mp h s(3, 4)) (by decide)
  have hl : λ(barbell) < 2 := not_le.mp (mt le_edgeConnectivity_iff.mp h2e)
  -- every vertex has degree at least 3
  have hδ : 3 ≤ δ(barbell) := barbell.le_minDegree_of_forall_le_degree 3 (by decide)
  exact hl.trans_le (by exact_mod_cast (by norm_num : 2 ≤ 3).trans hδ)
```

21. Show that the vertex connectivity of the hypercube `Q[n]` is exactly `n`. Hint: the lower bound is Exercise 16 (via `le_vertexConnectivity_iff`); for the upper bound use `vertexConnectivity_le_encard_neighborSet` at any vertex, together with `IsRegularOfDegree.hypercube`.

The lower bound is n-vertex-connectivity of `Q[n]`, and the upper bound is the degree n of any vertex.

```
lemma vertexConnectivity_hypercube (n : ℕ) : κ(Q[n]) = n := by
  obtain ⟨v⟩ : Nonempty (HypercubeVert n) := inferInstance
  refine le_antisymm ((vertexConnectivity_le_encard_neighborSet v).trans_eq ?_)
    (le_vertexConnectivity_iff.mpr (isVertexConnected_hypercube n))
  rw [← coe_neighborFinset, Set.encard_coe_eq_coe_finsetCard,
    card_neighborFinset_eq_degree, IsRegularOfDegree.hypercube n v]
```

22. Show that the edge connectivity of `Q[n + 1]` is `n + 1`, but that the edge connectivity of `Q[0]` is `⊤`. Hint: use Exercise 21 and `vertexConnectivity_le_edgeConnectivity`, then `edgeConnectivity_le_degree`, which needs a `Nontrivial (HypercubeVert (n + 1))` instance, and `edgeConnectivity_of_subsingleton`. Instance search deliberately does not see through `HypercubeVert`, so you will have to supply these instances yourself, for example with `inferInstanceAs (Nontrivial (HypercubeVert n × Bool))`.

For `Q[n + 1]` we have `n + 1 = κ(Q[n + 1]) ≤ λ(Q[n + 1])`, and `λ(Q[n + 1])` is at most the degree `n + 1` of any vertex, since `Q[n + 1]` has at least two vertices. The graph `Q[0]` has a single vertex, and Mathlib's `IsEdgeConnected k` has no condition on the number of vertices, so `Q[0]` is `k`-edge-connected for every `k`.

```
lemma edgeConnectivity_hypercube_succ (n : ℕ) : λ(Q[n + 1]) = n + 1 := by
  have : Nontrivial (HypercubeVert (n + 1)) :=
    inferInstanceAs (Nontrivial (HypercubeVert n × Bool))
  obtain ⟨v⟩ : Nonempty (HypercubeVert (n + 1)) := inferInstance
  refine le_antisymm ((edgeConnectivity_le_degree v).trans_eq ?_) ?_
  · rw [IsRegularOfDegree.hypercube (n + 1) v]; rfl
  · exact_mod_cast (vertexConnectivity_hypercube (n + 1)).symm.le.trans
      vertexConnectivity_le_edgeConnectivity

lemma edgeConnectivity_hypercube_zero : λ(Q[0]) = ⊤ :=
  have : Subsingleton (HypercubeVert 0) := inferInstanceAs (Subsingleton Unit)
  edgeConnectivity_of_subsingleton
```

23. Show that in a forest the block of an edge `u — v` is `G.subgraphOfAdj h`, the subgraph consisting of that edge and its two endpoints. Hint: show that block-equivalent potential edges of a forest are equal, by induction on `BlockEquivalent`, and use `blockMk_eq_blockMk_iff`.

Two distinct edges of a forest never lie on a common cycle, so cycle-equivalence, and hence block-equivalence, is just equality.

```
lemma IsAcyclic.toSubgraph_blockMk (hG : G.IsAcyclic) {u v : V} (h : G.Adj u v) :
    h.block.toSubgraph = G.subgraphOfAdj h := by
  -- block-equivalent potential edges of a forest are equal
  have key {f g : Sym2 V} (h : G.BlockEquivalent f g) : f = g := by
    induction h with
    | rel x y hxy => exact hxy.resolve_right fun ⟨_, c, hc, _⟩ => hG c hc
    | refl x => rfl
    | symm x y _ ih => exact ih.symm
    | trans x y z _ _ ih₁ ih₂ => exact ih₁.trans ih₂
  ext x y
  · simp only [Block.mem_toSubgraph_verts, subgraphOfAdj_verts, Set.mem_insert_iff,
      Set.mem_singleton_iff]
    constructor
    · -- the edge through x is s(u, v) itself
      rintro ⟨e, hx, he⟩
      rw [key (blockMk_eq_blockMk_iff.mp he)] at hx
      simpa using hx
    · rintro (rfl | rfl)
      exacts [⟨⟨_, h⟩, Sym2.mem_mk_left _ _, rfl⟩,
        ⟨⟨_, h⟩, Sym2.mem_mk_right _ _, rfl⟩]
  · simp only [blockMk_toSubgraph_adj, subgraphOfAdj_adj]
    refine ⟨fun ⟨_, hy⟩ => key hy, fun hy => ⟨?_, hy ▸ .refl _⟩⟩
    rcases Sym2.eq_iff.mp hy with ⟨rfl, rfl⟩ | ⟨rfl, rfl⟩
    exacts [h, h.symm]
```

24. Show that for `n ≥ 1` a graph is `n`-colourable iff each of its blocks is. Hint: the forward direction follows from `Colorable.of_hom` and `Subgraph.hom`. For the reverse direction use the De Bruijn–Erdős theorem (`colorable_of_forall_finite_subgraph_colorable`) to reduce to a finite subgraph `H`, which is colourable by `colorable_of_forall_block` provided its blocks are. Each block of `H` maps into a block of `G` along the copy `H.coeCopy` (`Block.map_toSubgraph_le`), and `Subgraph.homOfMapLE` turns this into a homomorphism.

A colouring of `G` restricts to each block, since blocks are subgraphs of `G`. For the converse, by De Bruijn–Erdős it suffices to colour every finite subgraph `H` of `G`. Each block of `H` maps, via the copy `H.coeCopy`, into a block of `G` (`Block.map_toSubgraph_le`), so a colouring of the blocks of `G` pulls back to a colouring of the blocks of `H` (`Subgraph.homOfMapLE`), and `colorable_of_forall_block` colours `H`.

```
theorem colorable_iff_forall_block {n : ℕ} (hn : 1 ≤ n) :
    G.Colorable n ↔ ∀ B : G.Block, B.toSubgraph.coe.Colorable n := by
  constructor
  · -- a colouring of G restricts to every block, along its inclusion into G
    exact fun hG B => hG.of_hom B.toSubgraph.hom
  · intro hcol
    -- by De Bruijn–Erdős it suffices to colour every finite subgraph H of G
```

```
    refine colorable_of_forall_finite_subgraph_colorable fun H hf => ?_
    have := hf.to_subtype
    -- each block B of H maps into the block B.map H.coeCopy of G
    exact colorable_of_forall_block hn H.coe fun B =>
      (hcol _).of_hom (Subgraph.homOfMapLE _
        (B.map_toSubgraph_le H.coeCopy))
```

Chapter 11: Distance

1. Show that two vertices with a common neighbour are at distance at most 2.

```
example {u v w : V} (huw : G.Adj u w) (hwv : G.Adj w v) : G.edist u v ≤ 2 :=
  (edist_le (Walk.cons huw (Walk.cons hwv Walk.nil))).trans (by simp)
```

2. Show that in a triangle-free graph two distinct vertices with a common neighbour are at distance exactly 2. Hint: `edist_eq_two_iff`.

The vertices u and v are not adjacent, since otherwise u, v and w would form a triangle, and w is a common neighbour.

```
example (h3 : G.CliqueFree 3) {u v w : V} (huw : G.Adj u w) (hwv : G.Adj w v)
    (huv : u ≠ v) : G.edist u v = 2 := by
  classical
  exact edist_eq_two_iff.mpr ⟨huv, fun h => h3 _ (is3Clique_triple_iff.2
    ⟨h, huw, hwv.symm⟩), w, G.mem_commonNeighbors.2 ⟨huw, hwv.symm⟩⟩
```

3. Deduce from `Hom.edist_le` that adding edges cannot increase distances (this is Mathlib's `edist_anti`), and that the distance between two vertices of an induced subgraph is at least their distance in G.

Hint: `Hom.ofLE` and `Embedding.induce`. Both inequalities apply `Hom.edist_le` to a homomorphism that is the identity on vertices: the inclusion `Hom.ofLE h : G →g H`, and the embedding `Embedding.induce s : G.induce s ↪g G` of the induced subgraph.

```
example {H : SimpleGraph V} (h : G ≤ H) (u v : V) : H.edist u v ≤ G.edist u v :=
  (Hom.ofLE h).edist_le u v

example (s : Set V) (u v : s) : G.edist u v ≤ (G.induce s).edist u v :=
  (Embedding.induce s).toHom.edist_le u v
```

4. Use `edist_eq_hammingDist` to show that the Hamming distance (between vertices of the hypercube) satisfies the triangle inequality.

Hint: rewrite the triangle inequality for `edist` in `Q[n]` using `edist_eq_hammingDist`, and cast the result back to $\mathbb{N}$.

```
example {n : ℕ} (u v w : HypercubeVert n) :
    u.hammingDist w ≤ u.hammingDist v + v.hammingDist w := by
  have := Q[n].edist_triangle (u := u) (v := v) (w := w)
  rw [edist_eq_hammingDist, edist_eq_hammingDist, edist_eq_hammingDist] at this
  exact_mod_cast this
```

5. Show that the complete bipartite graph `K[s + 2, t + 1]` has diameter 2.

In the complete bipartite graph any two vertices on the same side have a common neighbour on the other side, so the diameter is at most 2, while two distinct vertices on the same side are not adjacent, so it is at least 2.

```
example (s t : ℕ) : K[s + 2, t + 1].ediam = 2 := by
  refine le_antisymm (ediam_le_of_edist_le fun u v => not_lt.1 fun h => ?_) ?_
  · obtain ⟨-, hadj, hcn⟩ := two_lt_edist_iff.mp h
    rw [Set.eq_empty_iff_forall_notMem] at hcn
    rcases u with a | a <;> rcases v with b | b
    · exact hcn (.inr 0) (by simp [mem_commonNeighbors])
    · simp at hadj
    · simp at hadj
    · exact hcn (.inl 0) (by simp [mem_commonNeighbors])
  · exact (edist_eq_two_iff.mpr ⟨by simp, by simp, .inr 0,
      by simp [mem_commonNeighbors]⟩).ge.trans
      (edist_le_ediam (u := .inl 0) (v := .inl 1))
```

6. Show that if G has diameter at least 3 then its complement has diameter at most 3.

Hint: take two vertices u and v at distance more than 2 in G (`two_lt_edist_iff`), and show that every vertex is adjacent to u or to v in G^c, or equal to one of them.

```
example (h : 3 ≤ G.ediam) : Gᶜ.ediam ≤ 3 := by
  obtain ⟨u, v, huv⟩ : ∃ u v, 2 < G.edist u v := by
    by_contra! hc
    exact absurd (h.trans (ediam_le_of_edist_le hc)) (by decide)
  obtain ⟨hne, hnadj, hcn⟩ := two_lt_edist_iff.mp huv
  -- in Gᶜ, every vertex is within distance 1 of u or v, and u and v are
  -- within distance 1 of each other
  have hnear : ∀ x, ∃ w ∈ ({u, v} : Set V), Gᶜ.edist x w ≤ 1 := by
    intro x
    simp only [Set.mem_insert_iff, Set.mem_singleton_iff, exists_eq_or_imp,
      exists_eq_left, edist_le_one_iff_adj_or_eq, compl_adj]
    by_contra! hx
    have : x ∈ G.commonNeighbors u v := by
      simp only [mem_commonNeighbors]
      exact ⟨((hx.1.1 hx.1.2).symm, (hx.2.1 hx.2.2).symm⟩
    simp [hcn] at this
  have huv1 : Gᶜ.edist u v ≤ 1 := edist_le_one_iff_adj_or_eq.mpr (.inl ⟨hne, hnadj⟩)
  have hpair : ∀ a ∈ ({u, v} : Set V), ∀ b ∈ ({u, v} : Set V), Gᶜ.edist a b ≤ 1 := by
    rintro a (rfl | rfl) b (rfl | rfl) <;>
      first | simp | exact huv1 | exact edist_comm.trans_le huv1
  refine ediam_le_of_edist_le fun x y => ?_
  obtain ⟨a, ha, hxa⟩ := hnear x
```

```
  obtain ⟨b, hb, hyb⟩ := hnear y
  calc Gᶜ.edist x y ≤ Gᶜ.edist x a + Gᶜ.edist a b + Gᶜ.edist b y :=
        Gᶜ.edist_triangle.trans (add_le_add_left Gᶜ.edist_triangle _)
    _ ≤ 1 + 1 + 1 := by gcongr; exacts [hpair a ha b hb, edist_comm.trans_le hyb]
    _ = 3 := by norm_num
```

7. Show that if v has a neighbour in G then in the Mycielskian $\mu(G)$ the two copies v^{l} and v^{r} of v are at distance exactly 2. Hint: `edist_eq_two_iff`.

The two copies of v are distinct and not adjacent (v is not adjacent to itself in G), and the left copy u^{l} of any neighbour u of v is a common neighbour.

```
example {u v : V} (h : G.Adj u v) : mμ(G).edist v^l v^r = 2 := by
  rw [edist_eq_two_iff]
  refine ⟨by simp, by simp, ⟨u^l, ?_⟩⟩
  simp [mem_commonNeighbors, h.symm]
```

8. Show that every vertex of the cycle graph `C[n + 3]` has eccentricity `(n + 3) / 2`.

Hint: `IsVertexTransitive.eccent_eq`, `ediam` and `ediam_cycleGraph`.

```
example (n : ℕ) (v : Fin (n + 3)) : C[n + 3].eccent v = ((n + 3) / 2 : ℕ) := by
  rw [← ediam_cycleGraph n, ediam,
    iSup_congr fun u => (isVertexTransitive_cycleGraph _).eccent_eq u v, iSup_const]
```

9. Show that the Grötzsch graph `mμ(C[5])` has girth 4. Hint: use `egirth_mycielskian_eq_four`.

`C[5]` is triangle-free, and its vertex `0` has the two neighbours `1` and `4`, so `egirth_mycielskian_eq_four` applies. Both facts are finite checks.

```
example : grotzschGraph.egirth = 4 := by
  refine egirth_mycielskian_eq_four (by decide) fun h => ?_
  exact absurd (h 0 1 4 (by decide) (by decide)) (by decide)
```

10. Use the Moore bound to show that a 3-regular graph of diameter at most 2 has at most 10 vertices. (The Petersen graph shows that this is best possible.)

```
example [G.LocallyFinite] (h : G.IsRegularOfDegree 3) (hd : G.ediam ≤ 2) :
    ENat.card V ≤ 10 := by
  simpa [Finset.sum_range_succ] using moore_bound G 2 2 (fun v => (h v).le) hd
```

11. Use the cage bound to show that a graph with minimum degree at least 3 and girth at least 5 has at least 10 vertices. (The Petersen graph shows that this is best possible.)

```
example [G.LocallyFinite] [Nonempty V] (h : ∀ v, 3 ≤ G.degree v)
    (hg : 5 ≤ G.egirth) : 10 ≤ ENat.card V := by
  simpa [Finset.sum_range_succ] using cage_bound_odd G 2 2 h hg
```

Chapter 12: Algebraic Graph Theory

1. Let `M` be a group. Prove that for any `s : Set M` the Cayley graph `mulCayley s` is vertex-transitive. (Start by showing that left multiplication by any `g : M` is an automorphism of `mulCayley s`.)

Left multiplication by `g` as an automorphism of the Cayley graph `mulCayley s`.

```
def autMulLeftCayley {M : Type*} [Group M] (s : Set M) (g : M) :
    mulCayley s ≃g mulCayley s :=
  ⟨Equiv.mulLeft g, by simp⟩

lemma isVertexTransitive_mulCayley {M : Type*} [Group M] (s : Set M) :
    IsVertexTransitive (mulCayley s) :=
  fun u v => ⟨autMulLeftCayley s (v * u⁻¹), by simp [autMulLeftCayley]⟩
```

2. Show that the automorphisms of a complete graph are exactly the permutations of its vertices, i.e. that its automorphism group is the symmetric group `Equiv.Perm V`, and deduce that `K[n]` has `n!` automorphisms. Hint: Mathlib's `Iso.completeGraph` turns a bijection into an isomorphism of complete graphs, and `Nat.card_perm` counts permutations.

An automorphism of `τ` is determined by its underlying permutation, and conversely every permutation `σ` preserves adjacency (`σ a ≠ σ b ↔ a ≠ b`), which is `Iso.completeGraph σ`. The inverse laws (filled in automatically) and multiplicativity hold by `rfl`. The automorphism group of a complete graph is the symmetric group.

```
def autTopMulEquivPerm (V : Type*) :
    ((τ : SimpleGraph V) ≃g (τ : SimpleGraph V)) ≃* Equiv.Perm V where
  toFun e := e.toEquiv
  invFun σ := Iso.completeGraph σ
  map_mul' _ _ := rfl

lemma card_aut_completeGraph (n : ℕ) : Nat.card (K[n] ≃g K[n]) = n.factorial := by
  rw [Nat.card_congr (autTopMulEquivPerm (Fin n)).toEquiv, Nat.card_perm,
    Nat.card_fin]
```

3. Show that a graph and its complement have the same automorphisms. Hint: for the inverse map, treat the pairs of vertices `a = b` and `a ≠ b` separately.

An automorphism `e` of `G` preserves non-adjacency of distinct vertices, so `simp` shows that it is an automorphism of `Gᶜ`. Conversely, if `e` is an automorphism of `Gᶜ` then for `a ≠ b` (so `e a ≠ e b`) the two complement adjacencies agree, and negating gives the adjacencies in `G`; for `a = b` both sides are false. A graph and its complement have the same automorphisms.

```
def autComplMulEquiv : (G ≃g G) ≃* (Gᶜ ≃g Gᶜ) where
  toFun e := ⟨e.toEquiv, by simp [e.map_adj_iff]⟩
  invFun e := ⟨e.toEquiv, fun {a b} => by
    have h := e.map_adj_iff (v := a) (w := b)
    by_cases hab : a = b
    · subst hab; simp
    · have hab' : e a ≠ e b := e.injective.ne hab
```

```
      rw [← not_iff_not]
      simpa [compl_adj, hab, hab'] using h⟩
  map_mul' _ _ := rfl
```

4. Show that every vertex of a vertex-transitive graph with an odd number of vertices has even degree. Hint: such a graph is regular by `IsVertexTransitive.exists_isRegularOfDegree`; now use the handshake lemma in the form `sum_degrees_eq_twice_card_edges'` of The Handshake Lemma (p. 73).

If G is d-regular then the handshake lemma says that $|V| * d$ is twice the number of edges, hence even, and since $|V|$ is odd d must be even.

```
lemma even_degree_of_isVertexTransitive [Fintype V] [G.LocallyFinite]
    (h : G.IsVertexTransitive) (hodd : Odd (Fintype.card V)) (v : V) :
    Even (G.degree v) := by
  classical
  obtain ⟨d, hd⟩ := IsVertexTransitive.exists_isRegularOfDegree h
  have hsum := G.sum_degrees_eq_twice_card_edges'
  rw [Finset.sum_congr rfl fun v _ => hd v, Finset.sum_const, Finset.card_univ,
    smul_eq_mul] at hsum
  rw [hd v]
  exact (Nat.even_mul.1 ⟨_, hsum.trans (two_mul _)⟩).resolve_left
    (Nat.not_even_iff_odd.2 hodd)
```

5. Show that the Kneser graph `Kn[n, k]` is vertex-transitive, and deduce that so is the Petersen graph. Hint: a permutation σ of `Fin n` gives an automorphism `x ↦ x.map σ`. Given two k-subsets `s` and `t`, glue a bijection between `s` and `t` to one between their complements, using `Fintype.equivOfCardEq` and `Equiv.subtypeCongr`, to get a permutation sending `s` onto `t`.

```
lemma isVertexTransitive_kneserGraph (n k : ℕ) : Kn[n, k].IsVertexTransitive := by
  classical
  -- a permutation σ of Fin n gives the automorphism x ↦ x.map σ
  let aut (σ : Equiv.Perm (Fin n)) : Kn[n, k] ≃g Kn[n, k] :=
    { toFun x := ⟨x.val.map σ.toEmbedding, by simpa using x.prop⟩
      invFun x := ⟨x.val.map σ.symm.toEmbedding, by simpa using x.prop⟩
      left_inv x := by ext; simp
      right_inv x := by ext; simp
      map_rel_iff' := by intro x y; simp [Finset.disjoint_map, Subtype.ext_iff] }
  intro s t
  -- glue bijections s ≃ t and sᶜ ≃ tᶜ into a permutation sending s onto t
  have hs : Fintype.card {x // x ∈ s.val} = Fintype.card {x // x ∈ t.val} := by
    simp [s.prop, t.prop]
  have hc : Fintype.card {x // x ∉ s.val} = Fintype.card {x // x ∉ t.val} := by
    simp [Fintype.card_subtype_compl, s.prop, t.prop]
  let σ : Equiv.Perm (Fin n) :=
    Equiv.subtypeCongr (Fintype.equivOfCardEq hs) (Fintype.equivOfCardEq hc)
  refine ⟨aut σ, Subtype.ext ?_⟩
  apply Finset.eq_of_subset_of_card_le
  · intro y hy
    obtain ⟨x, hx, rfl⟩ := Finset.mem_map.mp hy
    have h : σ x = ((Fintype.equivOfCardEq hs) ⟨x, hx⟩ : Fin n) := by
```

```
      simp [σ, Equiv.subtypeCongr, Equiv.sumCompl_symm_apply_of_pos hx]
    simp [h]
  · simp [aut, s.prop, t.prop]

example : petersenGraph.IsVertexTransitive := isVertexTransitive_kneserGraph 5 2
```

6. Show that the cycle `C[n + 3]` has `2 * (n + 3)` automorphisms. Hint: use `autCycleGraphMulEquivDihedral` and `DihedralGroup.nat_card`.

The cycle `C[n + 3]` has `2 * (n + 3)` automorphisms.

```
lemma card_aut_cycleGraph (n : ℕ) :
    Nat.card (C[n + 3] ≃g C[n + 3]) = 2 * (n + 3) := by
  rw [Nat.card_congr (autCycleGraphMulEquivDihedral n).toEquiv,
    DihedralGroup.nat_card]
```

7. Show that the trace of the square of the adjacency matrix of a finite graph is twice the number of edges.

```
example [Fintype V] [DecidableEq V] [DecidableRel G.Adj] :
    (G.adjMatrix ℕ ^ 2).trace = 2 * #G.edgeFinset := by
  simp_rw [Matrix.trace, Matrix.diag, sq, adjMatrix_mul_self_apply_self,
    Nat.cast_id]
  exact sum_degrees_eq_twice_card_edges G
```

8. In the chapter, `mycielskianLift t` turns an eigenvector `x` of `G` whose entries sum to zero into an eigenvector `mycielskianLift t x` of `mμ(G)`, for any root `t` of `t ^ 2 = t + 1` (`adjMatrix_mycielskian_mulVec_mycielskianLift`). Show that the entries of `mycielskianLift t x` again sum to zero, so that the construction can be repeated, and deduce that the Grötzsch graph `grotzschGraph`, i.e. `mμ(C[5])`, has eigenvalues `1`, `-φ ^ 2` and `-ψ ^ 2`. Hint: `grotzschGraph` is isomorphic to `mμ(mμ(K[2]))`, by `Iso.mapMycielskian` applied to `mycielskianCompleteTwoIsoCycleFive`.

```
lemma sum_mycielskianLift [Fintype V] {R : Type*} [CommRing R] (t : R)
    {x : V → R} (hsum : ∑ v, x v = 0) : ∑ w, mycielskianLift t x w = 0 := by
  simp [Fintype.sum_option, Fintype.sum_sum_type, ← Finset.mul_sum, hsum]
```

Lifting the eigenvector `![1, -1]` of `K[2]` twice, first with `s` and then with `φ` or `ψ`, gives the eigenvalues `φ * (s * -1)` and `ψ * (s * -1)` of `mμ(mμ(K[2]))`, for `s = φ` and `s = ψ`. Since `φ * ψ = -1`, these are `-φ ^ 2`, `1` and `-ψ ^ 2`.

```
open Matrix in
open scoped goldenRatio in
example : 1 ∈ spectrum ℝ (grotzschGraph.adjMatrix ℝ) ∧
    -φ ^ 2 ∈ spectrum ℝ (grotzschGraph.adjMatrix ℝ) ∧
    -ψ ^ 2 ∈ spectrum ℝ (grotzschGraph.adjMatrix ℝ) := by
  have hK : K[2].adjMatrix ℝ *ᵥ ![1, -1] = (-1 : ℝ) • ![1, -1] := by
    ext i
    fin_cases i <;> simp [mulVec, dotProduct, Fin.sum_univ_two, adjMatrix_apply]
  have hx0 : (![1, -1] : Fin 2 → ℝ) ≠ 0 := by simp
```

```
  have hsum : ∑ i, (![1, -1] : Fin 2 → ℝ) i = 0 := by simp
  have h (s : ℝ) (hs : s ^ 2 = s + 1) :
      φ * (s * -1) ∈ spectrum ℝ (mμ(mμ(K[2])).adjMatrix ℝ) ∧
        ψ * (s * -1) ∈ spectrum ℝ (mμ(mμ(K[2])).adjMatrix ℝ) :=
    mem_spectrum_adjMatrix_mycielskian (mycielskianLift_ne_zero s hx0)
      (adjMatrix_mycielskian_mulVec_mycielskianLift hs hK hsum)
      (sum_mycielskianLift s hsum)
  rw [mycielskianCompleteTwoIsoCycleFive.symm.mapMycielskian.spectrum_adjMatrix]
  obtain ⟨h₁, h₂⟩ := h φ Real.goldenRatio_sq
  obtain ⟨h₃, h₄⟩ := h ψ Real.goldenConj_sq
  refine ⟨?_, ?_, ?_⟩
  · simpa using h₃
  · convert h₁
    ring
  · convert h₄
    ring
```

9. Show that the characteristic polynomial of `C[5]` is `(X - 2) * (X ^ 2 + X - 1) ^ 2`, so `-φ` and `-ψ` are eigenvalues of `C[5]` of multiplicity two. Hint: proceed as for the cospectral graphs in Eigenvalues (p. 367). The vertex type of `C[5]` is already `Fin 5`, so no relabelling is needed.

```
open Polynomial in
example : (C[5].adjMatrix ℤ).charpoly = (X - 2) * (X ^ 2 + X - 1) ^ 2 := by
  have hM : C[5].adjMatrix ℤ = !![0, 1, 0, 0, 1; 1, 0, 1, 0, 0; 0, 1, 0, 1, 0;
      0, 0, 1, 0, 1; 1, 0, 0, 1, 0] := by
    decide
  rw [hM, Matrix.charpoly, Matrix.det_succ_row_zero]
  -- expanding the determinant along its rows leaves a polynomial identity
  suffices (X : ℤ[X]) * (X * (X * (X * X + -1) + -X) + (1 + -(X * X))) +
      (X + -(X * (X * X + -1)) + -1 +
        -((-1) ^ 4 * (1 + (-(X * (-(X * X) + 1)) + -X)))) =
      (X - 2) * (X ^ 2 + X - 1) ^ 2 by
    simpa [Fin.sum_univ_succ, Matrix.det_succ_row_zero, Matrix.charmatrix_apply,
      Matrix.submatrix_apply, Fin.succAbove]
  ring
```

10. Show that the entries in each row of the Laplacian sum to `0`.

```
example [Fintype V] [DecidableEq V] [DecidableRel G.Adj] (i : V) :
    ∑ j, G.lapMatrix ℤ i j = 0 := by
  simpa [Matrix.mulVec, dotProduct] using
    congr_fun (G.lapMatrix_mulVec_one_eq_zero ℤ) i
```

11. Show that if `G` has no isolated vertices, then the kernel of the Laplacian of the Mycielskian `mμ(G)` is 1-dimensional.

```
example [Fintype V] [DecidableEq V] [DecidableRel G.Adj]
    (h : ∀ v, ¬G.IsIsolated v) :
    Module.finrank ℝ (mμ(G).lapMatrix ℝ).toLin'.ker = 1 := by
  rw [← card_connectedComponent_eq_finrank_ker_toLin'_lapMatrix,
    Fintype.card_eq_nat_card]
```

```
  exact card_connectedComponent_eq_one_iff.2 (connected_mycielskian_iff.2 h)
```

12. Show that `C[5]` is strongly regular with parameters `(5, 2, 0, 1)`, and deduce that its diameter is 2.

```
example : C[5].IsSRGWith 5 2 0 1 ∧ C[5].ediam = 2 := by
  have h : C[5].IsSRGWith 5 2 0 1 :=
    { card := by decide
      regular := by intro v; decide +revert
      of_adj := by decide
      of_not_adj := by intro u v; decide +revert}
  refine ⟨h, h.ediam_eq_two (fun h' => ?_) one_ne_zero⟩
  have hne : ¬C[5].Adj 0 2 := by decide
  exact hne (by rw [h', top_adj]; decide)
```

13. Show that there is no strongly regular graph with parameters `(10, 4, 0, 1)`.

```
example [Fintype V] [DecidableRel G.Adj] :
    ¬G.IsSRGWith 10 4 0 1 := fun h => by
  have := h.param_eq G (by norm_num)
  norm_num at this
```

14. Show that the Hamming cube is isomorphic to the third construction of the hypercube, the Hasse diagram `hypercubeHasse` of the subsets of `Fin n`, by sending a vertex `x` to the set of coordinates `i` with `x i = true`. Hint: adding the basis vector `e i` to `x` removes `i` from this set if it is there, and inserts it otherwise. Use `hypercubeHamming_adj_iff_exists`, `hasse_adj`, `Finset.covBy_iff_exists_insert`, `Finset.covBy_insert` and `Finset.erase_covBy`.

We first give the bijection and compute the effect of adding a basis vector. Bool vectors of length n correspond to subsets of `Fin n`.

```
def boolVecEquivFinset (n : ℕ) : (Fin n → Bool) ≃ Finset (Fin n) where
  toFun x := {i | x i = true}
  invFun s i := decide (i ∈ s)
  left_inv x := by ext i; simp
  right_inv s := by ext i; simp

open scoped hypercubeHamming in
lemma boolVecEquivFinset_add_single {n : ℕ} (x : Fin n → Bool) (i : Fin n) :
    boolVecEquivFinset n (x + e i) =
      if x i = true then (boolVecEquivFinset n x).erase i
      else insert i (boolVecEquivFinset n x) := by
  ext j
  by_cases hj : j = i <;> split_ifs <;>
    simp_all [boolVecEquivFinset, Bool.add_eq_xor, Bool.zero_eq_false]
```

If the set of x is covered by the set of y, then y is x with one more coordinate set to `true`, so x and y are adjacent. Conversely, adding a basis vector either inserts or erases an element, giving a covering relation in one direction or the other.

```
lemma hypercubeHamming_adj_of_covBy {n : ℕ} {x y : Fin n → Bool}
    (h : boolVecEquivFinset n x ⋖ boolVecEquivFinset n y) :
    (hypercubeHamming n).Adj x y := by
  obtain ⟨a, ha, hS⟩ := Finset.covBy_iff_exists_insert.1 h
  have hxa : x a = false := by simpa [boolVecEquivFinset] using ha
  refine hypercubeHamming_adj_iff_exists.2 ⟨a, (boolVecEquivFinset n).injective ?_⟩
  rw [boolVecEquivFinset_add_single, hxa]
  simpa using hS.symm
```

The Hamming cube is isomorphic to the Hasse diagram of the subsets of `Fin n`.

```
def hypercubeHammingIsoHasse (n : ℕ) : hypercubeHamming n ≃g hypercubeHasse n where
  toEquiv := boolVecEquivFinset n
  map_rel_iff' {x y} := by
    rw [hasse_adj]
    refine ⟨fun h => h.elim hypercubeHamming_adj_of_covBy
      fun h => (hypercubeHamming_adj_of_covBy h).symm, fun h => ?_⟩
    obtain ⟨i, rfl⟩ := hypercubeHamming_adj_iff_exists.1 h
    rw [boolVecEquivFinset_add_single]
    split_ifs with hi
    · exact .inr (Finset.erase_covBy (by simpa [boolVecEquivFinset] using hi))
    · exact .inl (Finset.covBy_insert (by simpa [boolVecEquivFinset] using hi))
```

15. Show that the Hamming cube is *arc-transitive*: for any two edges `x - y` and `u - v` there is an automorphism sending `x` to `u` and `y` to `v`. Hint: if `y = x + e i` and `v = u + e j`, compose a translation, the coordinate permutation `Equiv.swap i j` and another translation.

Translating by `x` sends `x` to `0` and `y = x + eᵢ` to `eᵢ`; swapping the coordinates `i` and `j` fixes `0` and sends `eᵢ` to `eⱼ`; and translating by `u` sends `0` to `u` and `eⱼ` to `v`.

```
open scoped hypercubeHamming in
lemma exists_aut_hypercubeHamming_of_adj {n : ℕ} {x y u v : Fin n → Bool}
    (h : (hypercubeHamming n).Adj x y) (h' : (hypercubeHamming n).Adj u v) :
    ∃ g : hypercubeHamming n ≃g hypercubeHamming n, g x = u ∧ g y = v := by
  obtain ⟨i, rfl⟩ := hypercubeHamming_adj_iff_exists.1 h
  obtain ⟨j, rfl⟩ := hypercubeHamming_adj_iff_exists.1 h'
  refine ⟨(hypercubeHamming.translate x).trans
    ((hypercubeHamming.permute (Equiv.swap i j)).trans
      (hypercubeHamming.translate u)), ?_, ?_⟩
  · simp [BooleanRing.add_self]
  · have : (e i : Fin n → Bool) ∘ Equiv.swap i j = e j := by
      simpa using Pi.single_comp_equiv (Equiv.swap i j) i true
    simp [this]
```

16. Show that the copies of any graph `A` in a complete graph are just the injective maps between the vertex types. Deduce that `K[n]` contains `n.descFactorial (k + 3) = n * (n - 1) * ⋯ * (n - k - 2)` labelled copies of `C[k + 3]`. Hint: `Fintype.card_embedding_eq`.

Adjacency in `⊤` is `≠`, so an injective map sends adjacent vertices (which are distinct) to adjacent vertices: every injective map is a copy. Copies of `A` in a complete graph are the injective maps of vertices.

```
def copyTopEquiv {W : Type*} (A : SimpleGraph V) :
    Copy A (⊤ : SimpleGraph W) ≃ (V ↪ W) where
  toFun f := f.toEmbedding
  invFun f := ⟨⟨f, fun hab => f.injective.ne hab.ne⟩, f.injective⟩

lemma labelledCopyCount_completeGraph_cycleGraph (n k : ℕ) :
    K[n].labelledCopyCount C[k + 3] = n.descFactorial (k + 3) := by
  rw [labelledCopyCount, Fintype.card_eq_nat_card, Nat.card_congr (copyTopEquiv _),
    Nat.card_eq_fintype_card, Fintype.card_embedding_eq]
  simp
```

17. Deduce that `K[n + 3]` contains `(n + 3)! / (2 * (n + 3))` Hamiltonian cycles, i.e. unlabelled copies of `C[n + 3]`. Hint: `labelledCopyCount_cycleGraph`.

```
lemma copyCount_completeGraph_cycleGraph (n : ℕ) :
    2 * (n + 3) * K[n + 3].copyCount C[n + 3] = (n + 3).factorial := by
  have h := labelledCopyCount_cycleGraph K[n + 3] n
  rw [labelledCopyCount_completeGraph_cycleGraph, Nat.descFactorial_self] at h
  rw [h, mul_comm]
```

18. Show that `C[5]` has `10` labelled copies in itself, using Exercise 6.

```
lemma labelledCopyCount_cycleGraph_five_self :
    C[5].labelledCopyCount C[5] = 10 := by
  rw [labelledCopyCount, Fintype.card_eq_nat_card, Nat.card_congr (copyEquivAut _),
    card_aut_cycleGraph 2]
```

19. Show that the number of labelled copies of a finite complete graph `⊤` in a finite graph `G` is the number of cliques of `G` of the same size, multiplied by the number of permutations of the vertices of `⊤`. Hint: combine `copyCount_top` and `labelledCopyCount_eq_copyCount_mul_labelledCopyCount` with Exercise 2.

The unlabelled copies are the cliques (`copyCount_top`), and by Exercise 2 the automorphisms of `⊤` are the permutations of its vertices.

```
lemma labelledCopyCount_top {W : Type*} [Fintype V] [Fintype W] [DecidableEq V]
    [DecidableRel G.Adj] :
    G.labelledCopyCount (⊤ : SimpleGraph W) =
      #(G.cliqueFinset (Fintype.card W)) * (Fintype.card W).factorial := by
  rw [labelledCopyCount_eq_copyCount_mul_labelledCopyCount, copyCount_top,
    labelledCopyCount, Fintype.card_eq_nat_card (α := Copy (⊤ : SimpleGraph W) ⊤),
    Nat.card_congr ((copyEquivAut _).trans (autTopMulEquivPerm W).toEquiv),
    Nat.card_perm, Nat.card_eq_fintype_card]
```

20. Count the 4-cycles in the hypercube: show that `Q[n]` contains `2 ^ n * (n * (n - 1))` labelled copies of `C[4]` and, for `2 ≤ n`, `n.choose 2 * 2 ^ (n - 2)` unlabelled ones. Hint: transport `labelledCopyCount_hypercube` and `copyCount_hypercube` along the isomorphism `hypercubeTwoIsoCycleFour : Q[2] ≃g C[4]`, using `Iso.labelledCopyCount_eq` and `Iso.copyCount_eq`.

The labelled count is `labelledCopyCount_hypercube n 2`, since `n.descFactorial 2 = n * (n - 1)`, and the unlabelled count is `copyCount_hypercube n 2`.

```
lemma labelledCopyCount_hypercube_cycleGraph_four (n : ℕ) :
    Q[n].labelledCopyCount C[4] = 2 ^ n * (n * (n - 1)) := by
  rw [← hypercubeTwoIsoCycleFour.labelledCopyCount_eq .refl,
    labelledCopyCount_hypercube]
  simp [Nat.descFactorial, mul_comm]


lemma copyCount_hypercube_cycleGraph_four (n : ℕ) (hn : 2 ≤ n) :
    Q[n].copyCount C[4] = n.choose 2 * 2 ^ (n - 2) := by
  rw [← hypercubeTwoIsoCycleFour.copyCount_eq .refl, copyCount_hypercube n 2 hn]
```

Chapter 13: Theorems

1. Show that a graph with an isolated vertex has no perfect matching.

```
example (v : V) (hv : G.IsIsolated v) :
    ¬∃ M : G.Subgraph, M.IsPerfectMatching := by
  rintro ⟨M, hM⟩
  obtain ⟨w, hw, -⟩ := Subgraph.isPerfectMatching_iff.1 hM v
  exact hv w hw.adj_sub
```

2. Show that the complete graph on a finite type `V` has a perfect matching iff `Fintype.card V` is even. Hint: `IsClique.even_iff_exists_isMatching`.

```
example [Fintype V] :
    (∃ M : (⊤ : SimpleGraph V).Subgraph, M.IsPerfectMatching) ↔
      Even (Fintype.card V) := by
  refine ⟨fun ⟨M, hM⟩ => hM.even_card, fun h => ?_⟩
  have hK : (⊤ : SimpleGraph V).IsClique Set.univ := isClique_univ.2 rfl
  obtain ⟨M, hverts, hM⟩ := (hK.even_iff_exists_isMatching Set.finite_univ).1
    (by rwa [Set.ncard_univ, Nat.card_eq_fintype_card])
  exact ⟨M, hM, Subgraph.isSpanning_iff.2 hverts⟩
```

3. Use Hall's theorem to show that the complete bipartite graph `K[n, n]` has a perfect matching. Hint: `Sum.swap` maps every set of vertices injectively into its neighbourhood.

```
example (n : ℕ) : ∃ M : K[n, n].Subgraph, M.IsPerfectMatching := by
  refine exists_isPerfectMatching_of_forall_ncard_le
    (IsBipartiteWith.completeBipartiteGraph _ _) fun s => ?_
  refine Set.ncard_le_ncard_of_injOn Sum.swap (fun x hx => ?_)
    Sum.swap_leftInverse.injective.injOn (Set.toFinite _)
  exact Set.mem_biUnion hx (by cases x <;> simp)
```

4. Use Hall's theorem to show that the infinite path `pathNat` has a perfect matching. Hint: `n ↦ n + 1` maps every finite set of vertices injectively into its neighbourhood. What happens for infinite sets?

```
example : ∃ M : pathNat.Subgraph, M.IsPerfectMatching := by
  refine exists_isPerfectMatching_of_forall_ncard_le
    pathNat.bicoloring.isBipartiteWith fun s => ?_
  by_cases hs : s.Finite
  · refine Set.ncard_le_ncard_of_injOn (· + 1) (fun x hx => ?_)
      (add_left_injective 1).injOn
      (hs.biUnion fun x _ => (pathNat.neighborSet x).toFinite)
    exact Set.mem_biUnion hx (by simp)
  · rw [Set.Infinite.ncard hs]
    exact Nat.zero_le _
```

5. Deduce the two-sided version of Hall's theorem: if every vertex lies in p_1 or p_2, and Hall's condition holds for the subsets of each part, then G has a perfect matching. Why is the hypothesis hV needed? Hint: reduce to `exists_isPerfectMatching_of_forall_ncard_le` by splitting `s` into `s ∩ p₁` and `s ∩ p₂`, whose neighbourhoods are disjoint.

```
example [G.LocallyFinite] {p₁ p₂ : Set V} (h₁ : G.IsBipartiteWith p₁ p₂)
    (hV : p₁ ∪ p₂ = Set.univ)
    (h₂ : ∀ s ⊆ p₁, s.ncard ≤ (⋃ x ∈ s, G.neighborSet x).ncard)
    (h₃ : ∀ s ⊆ p₂, s.ncard ≤ (⋃ x ∈ s, G.neighborSet x).ncard) :
    ∃ M : G.Subgraph, M.IsPerfectMatching := by
  -- hV is needed: a vertex in neither part has no neighbours, so no perfect
  -- matching covers it, although the one-sided conditions can still hold.
  refine exists_isPerfectMatching_of_forall_ncard_le h₁ fun s => ?_
  rcases s.finite_or_infinite with hs | hs
  · -- split s into its parts in p₁ and p₂, whose neighbourhoods are disjoint
    let N : Set V → Set V := fun t => ⋃ x ∈ t, G.neighborSet x
    have hfin {t : Set V} (ht : t ⊆ s) : (N t).Finite :=
      (hs.subset ht).biUnion fun x _ => (G.neighborSet x).toFinite
    have hsplit : s = (s ∩ p₁) ∪ (s ∩ p₂) := by
      rw [← Set.inter_union_distrib_left, hV, Set.inter_univ]
    have hN₁ : N (s ∩ p₁) ⊆ p₂ :=
      Set.iUnion₂_subset fun x hx => isBipartiteWith_neighborSet_subset h₁ hx.2
    have hN₂ : N (s ∩ p₂) ⊆ p₁ :=
      Set.iUnion₂_subset fun x hx => isBipartiteWith_neighborSet_subset h₁.symm hx.2
    calc s.ncard = (s ∩ p₁).ncard + (s ∩ p₂).ncard := by
          nth_rw 1 [hsplit]
          exact Set.ncard_union_eq
            (h₁.disjoint.mono Set.inter_subset_right Set.inter_subset_right)
            (hs.subset Set.inter_subset_left) (hs.subset Set.inter_subset_left)
      _ ≤ (N (s ∩ p₁)).ncard + (N (s ∩ p₂)).ncard :=
          add_le_add (h₂ _ Set.inter_subset_right) (h₃ _ Set.inter_subset_right)
      _ = (N s).ncard := by
          rw [← Set.ncard_union_eq (h₁.disjoint.symm.mono hN₁ hN₂)
            (hfin Set.inter_subset_left) (hfin Set.inter_subset_left)]
          nth_rw 3 [hsplit]
          simp only [N, Set.biUnion_union]
  · simp [hs.ncard]
```

6. Show that a k-regular bipartite graph with $k > 0$ has a perfect matching. The graph need not be finite. Hint: check Hall's condition for a finite set `s` by counting the edges between `s` and its neighbourhood in two ways, using `Finset.card_mul_le_card_mul`.

A k-regular bipartite graph with $k > 0$ has a perfect matching.

```
theorem IsRegularOfDegree.exists_isPerfectMatching [G.LocallyFinite] {k : ℕ}
    (hb : G.IsBipartite) (hG : G.IsRegularOfDegree k) (hk : 0 < k) :
    ∃ M : G.Subgraph, M.IsPerfectMatching := by
  obtain ⟨p₁, p₂, h₁⟩ := hb.exists_isBipartiteWith
  refine exists_isPerfectMatching_of_forall_ncard_le h₁ fun s => ?_
  classical
  rcases s.finite_or_infinite with hs | hs
  · -- s sends k * |s| edges to N, and each vertex of N receives at most k
    let t := hs.toFinset
    let N := t.biUnion fun x => G.neighborFinset x
    have hN : ⋃ x ∈ s, G.neighborSet x = N := by ext; simp [N, t]
    have hcount : #t * k ≤ #N * k := by
      refine card_mul_le_card_mul G.Adj (fun a ha => ?_) (fun b _ => ?_)
      · -- each a ∈ t has all k of its neighbours in N
        have hsub : G.neighborFinset a ⊆ N :=
          subset_biUnion_of_mem (fun x => G.neighborFinset x) ha
        exact (hG a).ge.trans (degIn_eq_degree hsub).ge
      · -- each b ∈ N has at most k neighbours in t
        rw [← G.degIn_eq_card_bipartiteBelow, ← hG b]
        exact degIn_le_degree t b
    rw [hN, Set.ncard_coe_finset, Set.ncard_eq_toFinset_card s hs]
    exact Nat.le_of_mul_le_mul_right hcount hk
  · simp [hs.ncard]
```

7. *Kőnig's line colouring theorem* says that a finite bipartite graph G has chromatic index $\chi'(G) = \Delta(G)$ (see Edge Colourings (p. 169)). Prove it for regular graphs: the edges of a k-regular bipartite graph can be properly coloured with k colours. (The general case reduces to this one, since every finite bipartite graph is a subgraph of a $\Delta(G)$-regular bipartite graph.) Deduce that the hypercube Q_n has chromatic index n. Hint: by Exercise 6, if $k > 0$ then G has a perfect matching; removing it leaves a $(k-1)$-regular bipartite graph, so induct on k.

Kőnig's line colouring theorem for regular graphs: the edges of a k-regular bipartite graph can be properly coloured with k colours.

```
theorem IsRegularOfDegree.colorable_lineGraph [G.LocallyFinite] {k : ℕ}
    (hb : G.IsBipartite) (hG : G.IsRegularOfDegree k) : L[G].Colorable k := by
  classical
  induction k generalizing G with
  | zero =>
    -- a 0-regular graph has no edges
    refine colorable_zero_iff.2 ⟨fun ⟨e, he⟩ => ?_⟩
    induction e using Sym2.ind with | h a b => ?_
    have := hG a
    rw [← ncard_neighborSet, Set.ncard_eq_zero (Set.toFinite _)] at this
    exact this.subset (mem_neighborSet _ _ _ |>.2 he)
  | succ k ih =>
    -- remove a perfect matching M
    obtain ⟨M, hM⟩ := hG.exists_isPerfectMatching hb k.succ_pos
    let G' := G \ M.spanningCoe
    have hle : G' ≤ G := sdiff_le
    let : G'.LocallyFinite := fun v =>
```

```
      ((G.neighborSet v).toFinite.subset fun w hw => hle hw).fintype
    -- what is left is k-regular
    have hG' : G'.IsRegularOfDegree k := fun v => by
      obtain ⟨w, hw, hw'⟩ := Subgraph.isPerfectMatching_iff.1 hM v
      have : G'.neighborSet v = G.neighborSet v \ {w} := by
        ext x
        have hMx : M.Adj v x ↔ x = w := ⟨hw' x, fun h => h ▸ hw⟩
        simp only [G', mem_neighborSet, sdiff_adj, Subgraph.spanningCoe_adj,
          Set.mem_sdiff, Set.mem_singleton_iff, hMx]
      rw [← ncard_neighborSet, this,
        Set.ncard_sdiff_singleton_of_mem ((G.mem_neighborSet v w).2 hw.adj_sub),
        ncard_neighborSet, hG v, Nat.add_sub_cancel]
    obtain ⟨C⟩ := ih (hb.mono_left hle) hG'
    -- colour the edges of M with the new colour k
    have hmem {e : Sym2 V} (he : e ∈ G.edgeSet) (heM : e ∉ M.edgeSet) :
        e ∈ G'.edgeSet := by
      simpa [G', edgeSet_sdiff, he] using heM
    refine ⟨Coloring.mk (fun e => if h : (e : Sym2 V) ∈ M.edgeSet then Fin.last k
      else (C ⟨e, hmem e.2 h⟩).castSucc) ?_⟩
    rintro ⟨e, he⟩ ⟨f, hf⟩ ⟨hne, x, hxe, hxf⟩
    by_cases heM : e ∈ M.edgeSet <;> by_cases hfM : f ∈ M.edgeSet <;>
      simp only [heM, hfM, dite_true, dite_false, ne_eq]
    · -- two edges of a matching sharing the vertex x are equal
      obtain ⟨y, rfl⟩ := Sym2.mem_iff_exists.1 hxe
      obtain ⟨z, rfl⟩ := Sym2.mem_iff_exists.1 hxf
      obtain rfl := hM.1.eq_of_adj_left heM hfM
      exact (hne rfl).elim
    · -- e has the new colour k, f an old colour
      exact (Fin.castSucc_ne_last _).symm
    · -- f has the new colour k, e an old colour
      exact Fin.castSucc_ne_last _
    · -- both edges keep their colours from C, and they are adjacent in L[G']
      have hadj : L[G'].Adj ⟨e, hmem he heM⟩ ⟨f, hmem hf hfM⟩ :=
        ⟨by simpa using hne, x, hxe, hxf⟩
      exact Fin.castSucc_injective _ |>.ne (C.valid hadj)

example (n : ℕ) : χ' Q[n] = n := by
  have hG := IsRegularOfDegree.hypercube n
  refine le_antisymm (hG.colorable_lineGraph (IsBipartite.hypercube n)
    |>.chromaticNumber_le) ?_
  have := Q[n].degree_le_chromaticIndex (Classical.arbitrary _)
  rwa [hG] at this
```

8. Show that the Turán graph `T[n, r]` is r-colourable for r `> 0`. (So by Turán's theorem every Turán-maximal `(r + 1)`-clique-free graph is r-colourable.) Mathlib has this, but prove it directly.

```
example (n r : ℕ) (hr : 0 < r) : T[n, r].Colorable r :=
  ⟨Coloring.mk (fun v => ⟨v % r, Nat.mod_lt _ hr⟩) fun h => by
    simpa [Fin.ext_iff] using turanGraph_adj.1 h⟩
```

9. Deduce from Füredi's theorem that an `(r + 1)`-clique-free graph on `n` vertices with `turanNumber n r` edges is `r`-colourable. Hint: then no edges need to be deleted (`card_eq_zero`, `deleteEdges_empty`).

```
example [Fintype V] [DecidableRel G.Adj] {r : ℕ} (h : G.CliqueFree (r + 1))
    (he : #G.edgeFinset = turanNumber (Fintype.card V) r) : G.Colorable r := by
  obtain ⟨s, -, hs, hle⟩ := h.exists_colorable_deleteEdges
  obtain rfl : s = ∅ := card_eq_zero.1 (by grind)
  rwa [coe_empty, deleteEdges_empty] at hs
```

10. The necessity of the Erdős–Gallai conditions (`erdos_gallai`) is a double-counting argument like those in this chapter. Show that the degrees of any set `S` of `k` vertices satisfy `∑ v ∈ S, G.degree v ≤ k * (k - 1) + ∑ v ∈ Sᶜ, min k (G.degree v)`. Hint: split each degree into the neighbours in `S` and in `Sᶜ` with `degIn_union`, bound the first part by `k - 1`, and count the edges between `S` and `Sᶜ` from the other side with `sum_degIn_comm`, as in the proof of Füredi's theorem (p. 419).

The degrees of a set `S` of vertices sum to at most `#S * (#S - 1) + ∑ v ∈ Sᶜ, min #S (G.degree v)`.

```
theorem sum_degree_le_erdosGallai [Fintype V] [DecidableEq V] [DecidableRel G.Adj]
    (S : Finset V) :
    ∑ v ∈ S, G.degree v ≤ #S * (#S - 1) + ∑ v ∈ Sᶜ, min #S (G.degree v) := by
  -- each degree splits into the neighbours in S and those in Sᶜ
  have hsplit (v : V) : G.degree v = G.degIn S v + G.degIn Sᶜ v := by
    rw [← degIn_union _ _ _ disjoint_compl_right, union_compl, degIn_eq_card_filter,
      ← card_neighborFinset_eq_degree]
    congr 1; ext; simp
  -- a vertex of S has at most #S - 1 neighbours in S
  have h1 : ∑ v ∈ S, G.degIn S v ≤ #S * (#S - 1) := by
    rw [← smul_eq_mul]
    refine sum_le_card_nsmul _ _ _ fun v hv => ?_
    calc G.degIn S v ≤ #(S.erase v) := card_le_card fun w hw => by
            rw [mem_bipartiteAbove] at hw
            exact mem_erase.2 ⟨(G.ne_of_adj hw.2).symm, hw.1⟩
      _ = #S - 1 := card_erase_of_mem hv
  -- count the edges between S and Sᶜ from the Sᶜ side
  have h2 : ∑ v ∈ S, G.degIn Sᶜ v ≤ ∑ v ∈ Sᶜ, min #S (G.degree v) := by
    rw [sum_degIn_comm]
    exact sum_le_sum fun v _ => le_min (card_filter_le _ _) (degIn_le_degree _ _)
  rw [sum_congr rfl fun v _ => hsplit v, sum_add_distrib]
  grind
```

11. Deduce that every graphic multiset satisfies the Erdős–Gallai inequalities. (Together with `Graphic.even_sum` this proves that the conditions in `erdos_gallai` are necessary.) Hint: first show that a sub-multiset of `s.map f` is the image of a sub-multiset of `s`, using `Multiset.coe_le` and `List.sublist_map_iff`. Then, with `L` the sorted list of degrees, realize `L.take k` as the degrees of a set `S` of `k` vertices, so that `L.drop k` is the list of degrees of `Sᶜ`.

A sub-multiset of `s.map f` is the image under `f` of a sub-multiset of `s`.

```
lemma _root_.Multiset.exists_le_map_eq {α β : Type*} {f : α → β}
    {s : Multiset α} {t : Multiset β} (h : t ≤ s.map f) : ∃ u ≤ s, u.map f = t := by
  induction s using Quotient.inductionOn
  induction t using Quotient.inductionOn
  obtain ⟨l', hp, hs⟩ := Multiset.coe_le.1 h
  obtain ⟨l'', hl'', rfl⟩ := List.sublist_map_iff.1 hs
  exact ⟨l'', Multiset.coe_le.2 hl''.subperm, Multiset.coe_eq_coe.2 hp⟩
```

A graphic multiset satisfies the Erdős–Gallai inequality `ErdosGallaiLE m k` for every `k ≤ m.card`.

```
theorem Graphic.erdosGallaiLE {m : Multiset ℕ} (h : Graphic m) {k : ℕ}
    (hk : k ≤ m.card) : ErdosGallaiLE m k := by
  obtain ⟨V, _, G, _, rfl⟩ := h
  classical
  set L := G.degreeMultiset.sort (· ≥ ·)
  have hL : (L.take k : Multiset ℕ) + L.drop k = G.degreeMultiset := by
    rw [Multiset.coe_add, List.take_append_drop, Multiset.sort_eq]
  -- L.take k is the multiset of degrees of a set S of vertices
  obtain ⟨u, hu, hmap⟩ := Multiset.exists_le_map_eq
    (hL ▸ Multiset.le_add_right _ _ : (L.take k : Multiset ℕ) ≤ G.degreeMultiset)
  let S : Finset V := ⟨u, Multiset.nodup_of_le hu univ.nodup⟩
  have hS : #S = k := by
    rw [card_mk, ← Multiset.card_map, hmap, Multiset.coe_card,
      List.length_take, Multiset.length_sort]
    exact min_eq_left hk
  -- and L.drop k is the multiset of degrees of Sᶜ
  have hc : Sᶜ.val.map (G.degree ·) = (L.drop k : Multiset ℕ) := by
    have := hL
    rw [degreeMultiset, ← union_compl S,
      ← disjUnion_eq_union _ _ disjoint_compl_right, disjUnion_val,
      Multiset.map_add] at this
    exact (add_left_cancel (this.trans (by rw [← hmap]))).symm
  calc (L.take k).sum = ∑ v ∈ S, G.degree v := by
        rw [← Multiset.sum_coe, ← hmap]; rfl
    _ ≤ k * (k - 1) + ∑ v ∈ Sᶜ, min k (G.degree v) :=
        hS ▸ sum_degree_le_erdosGallai S
    _ = _ := by
        rw [← Multiset.sum_coe, ← Multiset.map_coe, ← hc, Multiset.map_map]; rfl
```

12. Show that the bound in the Andrásfai–Erdős–Sós theorem cannot be improved for `r = 2`: the blow-up `C[5][t]` with `t ≥ 1` has `n = 5 * t` vertices and is triangle-free with minimum degree `2 * t = 2 * n / 5`, but it is not bipartite.

```
example (t : ℕ) :
    C[5][t + 1].CliqueFree 3 ∧ ¬C[5][t + 1].Colorable 2 ∧
      ∀ v, C[5][t + 1].degree v = 2 * (t + 1) := by
  refine ⟨(cliqueFree_blowUpGraph_iff _ _ fun _ => 0).2 (by decide), fun h => ?_, ?_⟩
  · have h5 := ((colorable_blowUpGraph_iff _ fun _ => 0).1 h).chromaticNumber_le
    rw [chromaticNumber_cycleGraph_of_odd 5 (by norm_num) (by decide)] at h5
    exact absurd h5 (by decide)
  · rintro ⟨v, i⟩
    rw [degree_blowUpN, cycleGraph_degree_three_le, mul_comm]
```

13. Restate Mantel's theorem as a formula for an extremal number.

```
lemma extremalNumber_completeGraph_three (n : ℕ) :
    extremalNumber n K[3] = n ^ 2 / 4 := by
  rw [extremalNumber_top, Fintype.card_fin]
  exact turanNumber_two
```

14. Deduce that a graph on *n* vertices with more than `n ^ 2 / 4` edges contains a triangle.

```
example [Fintype V] [DecidableRel G.Adj]
    (h : Fintype.card V ^ 2 / 4 < #G.edgeFinset) : K[3] ⊑ G :=
  IsContained.of_extremalNumber_lt_card_edgeFinset
    (extremalNumber_completeGraph_three (Fintype.card V) ▸ h)
```

15. Use Zykov's theorem to show that a 4-clique-free graph on 6 vertices has at most as many triangles as the octahedron `completeEquipartiteGraph 3 2`. Hint: Mathlib's `completeEquipartiteGraph.turanGraph` is an isomorphism from `completeEquipartiteGraph r t` to `turanGraph (r * t) r`.

```
example [Fintype V] [DecidableEq V] [DecidableRel G.Adj] (hV : Fintype.card V = 6)
    (h : G.CliqueFree 4) :
    #(G.cliqueFinset 3) ≤ #((completeEquipartiteGraph 3 2).cliqueFinset 3) := by
  have e := completeEquipartiteGraph.turanGraph (r := 3) (t := 2)
  have := h.card_cliqueFinset_le (r := 3) 3
  rwa [hV, ← e.card_cliqueFinset_eq] at this
```

16. Show that a graph has as many 2-cliques as edges. Hint: `Sym2.toFinset` sends the edge `s(x, y)` to `{x, y}`.

A graph has as many 2-cliques as edges.

```
lemma card_cliqueFinset_two [Fintype V] [DecidableEq V] [DecidableRel G.Adj] :
    #(G.cliqueFinset 2) = #G.edgeFinset := by
  symm
  refine card_bij (fun e _ => e.toFinset) (fun e he => ?_) (fun e _ e' _ h => ?_)
    (fun t ht => ?_)
  · induction e using Sym2.ind with
    | _ x y =>
      rw [mem_edgeFinset, mem_edgeSet] at he
      rw [Sym2.toFinset_mk_eq, mem_cliqueFinset_iff]
      exact ⟨by rw [coe_pair]; exact isClique_pair.2 fun _ => he, card_pair he.ne⟩
  · exact Sym2.ext fun x => by rw [← Sym2.mem_toFinset, h, Sym2.mem_toFinset]
  · rw [mem_cliqueFinset_iff] at ht
    obtain ⟨x, y, hxy, rfl⟩ := card_eq_two.1 ht.card_eq
    refine ⟨s(x, y), ?_, Sym2.toFinset_mk_eq⟩
    rw [mem_edgeFinset, mem_edgeSet]
    exact ht.isClique (by simp) (by simp) hxy
```

17. Deduce the edge bound of Turán's theorem, `CliqueFree.card_edgeFinset_le`, from Zykov's theorem with `s = 2`. Hint: Exercise 16.

```
example [Fintype V] [DecidableRel G.Adj] {r : ℕ}
    (h : G.CliqueFree (r + 1)) : #G.edgeFinset ≤ turanNumber (Fintype.card V) r := by
  classical
  have := h.card_cliqueFinset_le 2
  rwa [card_cliqueFinset_two, card_cliqueFinset_two] at this
```

18. Deduce the uniqueness part of Turán's theorem from the equality case of Zykov's theorem with `s = 2`: for `2 ≤ r` and `2 ≤ n`, an `(r + 1)`-clique-free graph on `n` vertices with `turanNumber n r` edges is isomorphic to `T[n, r]`. Hint: Exercise 16.

```
example [Fintype V] [DecidableRel G.Adj] {r : ℕ} (h : G.CliqueFree (r + 1))
    (hr : 2 ≤ r) (hn : 2 ≤ Fintype.card V)
    (he : #G.edgeFinset = turanNumber (Fintype.card V) r) :
    Nonempty (G ≃g T[Fintype.card V, r]) := by
  classical
  refine (h.card_cliqueFinset_eq_iff_nonempty_iso_turanGraph le_rfl hr hn).1 ?_
  rwa [card_cliqueFinset_two, card_cliqueFinset_two]
```

19. Show that a graph containing a ray has an end, by formalizing the argument at the start of Ends (p. 444). Together with `Connected.pathNat_isContained`, this gives another proof of `nonempty_ends_of_infinite`. Hint: a copy `f` of `pathNat` is injective, so for each finite set `K` some tail `n ↦ f (n + N)` avoids `K`, and by `ComponentCompl.mem_of_hom` it lies in a single component of `G - K`. Two such tails share a vertex, so `ComponentCompl.hom_eq_iff_not_disjoint` shows that these components form a direction. Finish with `mem_end_iff`.

A graph containing a ray has an end.

```
theorem nonempty_ends_of_pathNat_isContained (h : pathNat ⊑ G) : G.end.Nonempty := by
  obtain ⟨f⟩ := h
  -- from N K onwards the ray avoids K
  let N (K : Finset V) : ℕ := (K.preimage f f.injective.injOn).sup id + 1
  have hN (K : Finset V) (n : ℕ) : f (n + N K) ∉ K := fun hK => by
    have := Finset.le_sup (s := K.preimage f f.injective.injOn) (f := id)
      (Finset.mem_preimage.2 hK)
    grind
  -- so the tail from N K lies in a single component d K of G - K
  let d (K : Finset V) : G.ComponentCompl K := G.componentComplMk (hN K 0)
  have hd (K : Finset V) (n : ℕ) : f (n + N K) ∈ d K :=
    ComponentCompl.mem_of_hom connected_pathNat.preconnected
      (f.toHom.comp (pathNat.translate (N K)).toHom) (hN K)
      (G.componentComplMk_mem (hN K 0)) n
  -- two tails share the vertex f (N K + N L), so d is a direction
  have hdir : G.IsDirection d := fun K L hKL => by
    have hK := hd K (N L)
    rw [add_comm] at hK
    have : (d L).hom (Finset.coe_subset.2 hKL) = d K :=
      ((d L).hom_eq_iff_not_disjoint _ _).2
        (Set.not_disjoint_iff.2 ⟨_, hd L (N K), hK⟩)
    exact this ▸ (d L).subset_hom _
  exact ⟨_, (mem_end_iff (f := fun K => d K.unop)).2 hdir⟩
```

20. Show that the grid `hasse ℤ □ hasse ℤ` has exactly one end, as in Ends (p. 444). Hint: if `K` lies in the square `[-N, N] × [-N, N]` then the rows and columns outside the square avoid `K`. Use `ComponentCompl.mem_of_hom` with `boxProdLeft` and `boxProdRight` to show that every infinite component of `G - K` contains `(N + 1, N + 1)`, and apply `subsingleton_end_of_common_vertex`.

```
example : Nat.card (hasse ℤ □ hasse ℤ).end = 1 := by
  have hadj (n : ℤ) : (hasse ℤ).Adj n (n + 1) :=
    hasse_adj.2 (.inl (Order.covBy_add_one n))
  have hP : (hasse ℤ).Preconnected := preconnected_hasse_of_succOrder ℤ
  rw [Nat.card_eq_one_iff_unique]
  refine ⟨subsingleton_end_of_common_vertex fun K => ?_, ?_⟩
  · obtain ⟨N, hN⟩ :=
      (K.finite_toSet.image fun v : ℤ × ℤ => max |v.1| |v.2|).bddAbove
    -- the column {a} × ℤ and the row ℤ × {b} avoid K when N < |a|, N < |b|
    let col (a : ℤ) := (boxProdRight (hasse ℤ) (hasse ℤ) a).toHom
    let row (b : ℤ) := (boxProdLeft (hasse ℤ) (hasse ℤ) b).toHom
    have hcol (a : ℤ) (ha : N < |a|) (n : ℤ) : col a n ∉ K := fun hK =>
      ha.not_ge ((le_max_left _ _).trans (hN ⟨_, hK, rfl⟩))
    have hrow (b : ℤ) (hb : N < |b|) (n : ℤ) : row b n ∉ K := fun hK =>
      hb.not_ge ((le_max_right _ _).trans (hN ⟨_, hK, rfl⟩))
    have hN1 : N < |N + 1| := (lt_add_one N).trans_le (le_abs_self _)
    -- every infinite component of G - K contains (N + 1, N + 1)
    refine ⟨(N + 1, N + 1), fun C hC => ?_⟩
    obtain ⟨⟨a, b⟩, habC, hbox⟩ :=
      (hC.sdiff ((Set.finite_Icc (-N) N).prod (Set.finite_Icc (-N) N))).nonempty
    simp only [Set.mem_prod, Set.mem_Icc, ← abs_le, not_and_or, not_le] at hbox
    rcases hbox with ha | hb
    · have := ComponentCompl.mem_of_hom hP (col a) (hcol a ha) (u := b) habC
      exact ComponentCompl.mem_of_hom hP (row _) (hrow _ hN1) (u := a) (this _) _
    · have := ComponentCompl.mem_of_hom hP (row b) (hrow b hb) (u := a) habC
      exact ComponentCompl.mem_of_hom hP (col _) (hcol _ hN1) (u := b) (this _) _
  · have (n : ℤ) : Fintype ((hasse ℤ).neighborSet n) :=
      Fintype.ofFinset {n - 1, n + 1} fun m => by
        grind [hasse_adj, Order.covBy_iff_add_one_eq, mem_neighborSet]
    have : Fact (hasse ℤ □ hasse ℤ).Preconnected := ⟨hP.boxProd hP⟩
    exact (nonempty_ends_of_infinite _).to_subtype
```

Appendix

Notation

This guide introduces quite a bit of ”standard” graph theory notation not found in Mathlib.

Notation	Description	Definition / Target
`K[n]`	Complete graph on `n` vertices	`completeGraph (Fin n)`
`E[n]`	Empty graph on `n` vertices	`emptyGraph (Fin n)`
`C[n]`	Cycle graph on `n` vertices	`cycleGraph n`
`K[s, t]`	Complete bipartite graph	`completeBipartiteGraph (Fi`
`L[G]`	Line graph of a graph `G`	`lineGraph G`
`P[n]`	Path graph on `n` vertices	`pathGraph n`
`Q[n]`	`n`-dimensional hypercube	`hypercube n`
`v[a, b, …]`	Hypercube vertex (`open HypercubeVert`)	`((((), a), b), …)`
`G[n]`	`n`-uniform blow-up of a graph `G`	`blowUpN G n`
`mμ(G)`	Mycielskian of a graph `G`	`mycielskian G`
`v^l`	Left copy of `v` in `Option (V ⊕ V)`	`mycielskian.left v`
`v^r`	Right copy of `v` in `Option (V ⊕ V)`	`mycielskian.right v`
`mμ_v^n(V)`	Vertices of the `n`-th iterated Mycielskian	`MycielskianVert V n`
`mμ^n(G)`	`n`-th iterated Mycielskian of a graph `G`	`iterMycielskian G n`
`Kn[n, k]`	Kneser graph on `k`-subsets of `Fin n`	`kneserGraph (Fin n) k`
`T[n, r]`	Turán graph on `n` vertices with `r` parts	`turanGraph n r`
`Σg H`	Disjoint union of a family of graphs `H i`	`SimpleGraph.sigma H`
`x ⊕ₕ y`	Componentwise `xor` of hypercube vertices	`HypercubeVert.xor x y`
`G ⊗g H`	Tensor product of `G` and `H`	`SimpleGraph.tensorProd G H`
`⊗g i, G i`	Tensor product of a family of graphs	`SimpleGraph.piTensorProd G`
`δ(G)`	Minimum degree of a graph `G`	`SimpleGraph.minDegree G`
`Δ(G)`	Maximum degree of a graph `G`	`SimpleGraph.maxDegree G`
`τ G`	Vertex cover number of a graph `G`	`SimpleGraph.vertexCoverNum`
`ν G`	Matching number of a graph `G`	`SimpleGraph.matchingNum G`
`ω G`	Clique number of a graph `G`	`SimpleGraph.cliqueNum G`
`α G`	Independence number of a graph `G`	`SimpleGraph.indepNum G`
`β_k`	`k`-subsets of `β` (Chapter 6)	`{s : Finset β // #s = k}`
`N(s)`	Open neighbourhood of `s` (Chapter 6)	`openNeighborhoodFinset G s`
`χ G`	Chromatic number of a graph `G`	`SimpleGraph.chromaticNumbe`
`χ' G`	Chromatic index of a graph `G`	`SimpleGraph.chromaticIndex`
`G⟦v⟧`	Connected component of `G` containing `v`	`connectedComponentMk G v`
`λ(G)`	Edge connectivity of a graph `G`	`SimpleGraph.edgeConnectivi`
`κ(G)`	Vertex connectivity of a graph `G`	`SimpleGraph.vertexConnecti`
`G⟪e⟫`	Block of `G` containing the edge `e`	`SimpleGraph.blockMk G e`
`e i`	Basis vector of `Fin n → Bool` (`open scoped hypercubeHamming`)	`Pi.single i true`

Definitions

The following custom definitions and abbreviations are introduced in this guide:

Chapter 2: SimpleGraph

- `pathNat`: path graph on $\mathbb{N}$ defined using the default `SimpleGraph` constructor.
- `pathNatFromRel`: path graph on $\mathbb{N}$ defined using `SimpleGraph.fromRel`.
- `pathNatFromEdgeSet`: path graph on $\mathbb{N}$ defined using `SimpleGraph.fromEdgeSet`.

From the exercises:

- `shiftedPathNat`: graph on $\mathbb{N}$ with an edge from `n` to `n + 1` for every `n ≥ i`.
- `shortPathNat`: graph on $\mathbb{N}$ with an edge from `n` to `n + 1` for every `n < i`.

Chapter 3: Constructions

- `paleyGraph`: Paley graph of a field `F`.
- `cayleyS4`: Cayley graph of the symmetric group on `Fin 4` generated by a 3-cycle and a 4-cycle.
- `SimpleGraph.sigma` / `Σg`: disjoint union of an indexed family of graphs.
- `sigmaCompleteNat`: disjoint union of complete graphs `K[n]`.
- `hypercubeHasse`: Hasse diagram construction of the `n`-dimensional hypercube.
- `hypercubeHamming`: Hamming cube definition of the `n`-dimensional hypercube.
- `HypercubeVert`: recursive definition of the vertex set of the hypercube.
- `hypercube` / `Q[n]`: recursive box product definition of the `n`-dimensional hypercube.
- `mycielskian` / `mμ(G)`: the Mycielskian of a graph `G`.
- `grotzschGraph`: Grötzsch graph (i.e. `mμ(C[5])`).
- `mycielskian.left` / $\mathtt{v}^{\mathtt{l}}$: left copy of a vertex in `Option (V ⊕ V)`, the vertex type of the Mycielskian.
- `mycielskian.right` / $\mathtt{v}^{\mathtt{r}}$: right copy of a vertex in `Option (V ⊕ V)`, the vertex type of the Mycielskian.
- `MycielskianVert` / $\mathtt{mμ}_{\mathtt{v}}\mathtt{\hat{}n(V)}$: vertex type of the `n`-th iterated Mycielskian.
- `iterMycielskian` / `mμ^n(G)`: `n`-th iterated Mycielskian of `G`.
- `completeMultipartiteFinNat`: complete multipartite graph with copies of $\mathbb{N}$ as classes.
- `completeMultipartiteNatFin`: complete multipartite graph with classes `Fin n`.
- `blowUpGraph`: blow-up of a graph `G` by replacing each vertex with a class of vertices.
- `blowUpN` / `G[n]`: `n`-uniform blow-up of a graph `G`.
- `kneserGraph` / `Kn[n, k]`: the Kneser graph — vertices are the `k`-element subsets of `β` (or of `Fin n`), with edges joining distinct disjoint subsets.
- `petersenGraph`: the Petersen graph, i.e. `Kn[5, 2]`.

Chapter 4: Decidability, Finiteness and Degrees

- `myIfThenElse`: branches with **`bif`** on the Boolean `decide p` from a `Decidable p` instance to return a string, mirroring how `ite` is defined.
- `myClassicalIfThenElse`: the same function built with **`classical`**, hence noncomputable.
- `pathNatFromRelAbbrev`: `pathNatFromRel` restated as an **`abbrev`**, so that Lean can synthesize `DecidableRel` for its adjacency.
- `locallyFiniteOfFintypeEdgeSet`: a graph with finitely many edges is locally finite.
- `blowUpNFintypeNeighborSet`: a vertex `⟨v, i⟩` of the blow-up `G[n]` has a finite neighbourhood when `v` has one in `G` (a **`def`**, not an instance).
- `degIn`: the number of neighbours of `v` lying in a finite set `s`, defined as the size of Mathlib's `s.bipartiteAbove G.Adj v`.
- `hypercubeVertToBin`: conversion of a hypercube vertex to its binary string representation.
- `hypercubeFintypeNeighborSet`: an efficient `Fintype` instance for the neighbourhoods of `Q[n]`, built through the box product (a **`def`**, not an instance).
- `degreeMultiset`: the multiset of degrees of all vertices in a finite graph.
- `degreeSequence`: the list of degrees of all vertices in a finite graph, sorted in non-increasing order.
- `Graphic`: predicate for a multiset being the degree multiset of some finite simple graph.
- `ErdosGallaiLE`: the `k`-th Erdős–Gallai inequality for a multiset of degrees.
- `havelHakimiAux`: the Havel–Hakimi test on a non-increasing list, merging rather than re-sorting between rounds.
- `havelHakimi`: decides whether a multiset of degrees is graphic via Havel–Hakimi.
- `coneGraph`: adjoins a new vertex to a graph, adjacent to a chosen set of vertices.
- `havelHakimiEdges`: constructive Havel–Hakimi, returning the edge list of a graph realizing a labelled graphic degree sequence.
- `graphOfEdgeList`: the graph on `Fin n` with a given list of edges, used to check the output of `havelHakimiEdges`.

From the exercises:

- `blowUpGraphFintypeNeighborSet`: a vertex `w` of a blow-up has a finite neighbourhood if `w.fst` does and the classes over its neighbours are finite.

Chapter 5: Maps

- `pathNat.translate`: embedding of the infinite path `pathNat` into itself.
- `Embedding.blowUp`: canonical embedding of a graph `G` into its blow-up.
- `Iso.emptyGraph`: the isomorphism `emptyGraph α ≃g emptyGraph β` induced by an equivalence `α ≃ β`.
- `hypercube.embedBool` / `hypercube.embedFalse` / `hypercube.embedTrue`: embeddings of `Q[n]` into `Q[n + 1]`.
- `Embedding.mycielskianLeft`: embedding of a graph `G` into its Mycielskian `mμ(G)`.
- `Hom.mapMycielskian`: graph homomorphism `mμ(G) →g mμ(H)` induced by a homomorphism `G →g H`.
- `Embedding.mapMycielskian`: graph embedding `mμ(G) ↪g mμ(H)` induced by an embedding `G ↪g H`.
- `Iso.mapMycielskian`: graph isomorphism `mμ(G) ≃g mμ(H)` induced by an isomorphism `G ≃g H`.
- `IsVertexTransitive`: predicate for a graph being vertex-transitive.
- `cycleGraph.rotate`: automorphism of `C[n]` rotating the vertices modulo `n`, sending `d` to `0`.
- `HypercubeVert.xor` / `⊕ₕ`: bitwise `xor` addition on hypercube vertices.
- `hypercube.translate`: automorphism of `Q[n]` by translating by a vertex.
- `mycielskianCompleteTwoIsoCycleFive`: the isomorphism `mμ(K[2]) ≃g C[5]`.
- `tensorProd` / `G ⊗g H`: tensor (categorical) product of two graphs, on the vertex type `V × W`.
- `tensorProdFintypeNeighborSet`: finite neighbourhoods in `G ⊗g H` from finite neighbourhoods in `G` and `H` (a `def`, not an instance).
- `tensorProd.fstHom` / `tensorProd.sndHom`: the two projections out of the tensor product as graph homomorphisms.
- `tensorProd.prod`: the homomorphism into `tensorProd G H` induced by a pair of homomorphisms (universal property of the product).
- `piTensorProd` / `⊗g i, G i`: tensor product of an arbitrary family of graphs.
- `piTensorProdFintypeNeighborSet`: finite neighbourhoods in `⊗g i, G i` from finite neighbourhoods in each `G i`, for a finite nonempty index type (a `def`, not an instance).
- `IndFree`: `A.IndFree B` means that `B` contains no induced copy of `A`.
- `ClawFree`: a graph is *claw-free* if it contains no induced copy of the claw `K[1, 3]`.
- `lineGraphCompleteThreeIsoCompleteThree`: the isomorphism `L[K[3]] ≃g K[3]`.
- `lineGraphClawIsoCompleteThree`: the isomorphism `L[K[1, 3]] ≃g K[3]`.

From the exercises:

- `pathGraph.embedding`: embedding of a path graph `P[n]` into a longer path `P[n + k]`.
- `Embedding.toBlowUp`: embedding of *G* into the blow-up `H.blowUpGraph U` given *G* `↪g H` and nonempty parts.
- `hypercube.embedding`: embedding of `Q[n]` into `Q[n + k]` by iterating `hypercube.embedFalse`.
- `hypercubeTwoIsoCycleFour`: the Gray-code isomorphism `Q[2] ≃g C[4]`.

Chapter 6: Cliques

- `mycielskianProj`: projection from the vertex set of `mμ(G)` to *G* mapping both left and right copies to the original vertex.
- `openNeighborhoodSet`: the open neighbourhood of a set `s`, i.e. the union of the neighbourhoods of its vertices.
- `openNeighborhoodFinset`: the `Finset` version, for a locally finite graph: the vertices adjacent to at least one vertex of `s`.

Chapter 7: Colorings

- `completeBipartiteGraph.bicoloring`: the natural `Bool`-colouring of `completeBipartiteGraph V W` (`true` on the right part); a correctly named alias of Mathlib's `CompleteBipartiteGraph.bicoloring`.
- `pathNat.bicoloring`: the natural `Bool`-colouring of the infinite path `pathNat` by parity.
- `Coloring.ofBoxProdLeft` / `Coloring.ofBoxProdRight`: the colourings of *G* and `H` obtained by restricting a colouring of *G* `□ H` along a fixed vertex of the other factor.
- `Coloring.boxProd`: the colouring `(v, w) ↦ C₁ v + C₂ w` of *G* `□ H` built from colourings of *G* and `H`, for colours with a cancellative addition (`IsCancelAdd`), such as `Fin k` (addition modulo `k`).
- `hypercube.bicoloring`: the natural `Bool`-colouring of the hypercube `Q[n]`, built by induction on `n` with `Coloring.boxProd`.
- `HypercubeVert.parity`: the parity (`Bool.xor` of the coordinates) of a hypercube vertex, which is its colour under `hypercube.bicoloring`.
- `Coloring.mycielskianLast`: the equivalent `(k + 1)`-colouring of `mμ(G)` in which the vertex `none` receives colour `k`.
- `Coloring.ofMycielskian`: the `k`-colouring of *G* induced by a `(k + 1)`-colouring of `mμ(G)`.
- `Coloring.mycielskian`: the `(k + 1)`-colouring of `mμ(G)` induced by a `k`-colouring of *G*.
- `chromaticIndex`: the chromatic index `χ' G` of a graph, the chromatic number `χ L[G]` of its line graph.

- `neighborLTSet`: set of neighbours of a vertex `v` that are smaller than `v`.
- `neighborLTFinset`: finset of neighbours of a vertex `v` that are smaller than `v`.
- `degreeLT`: the number of earlier neighbours of a vertex `v`.
- `LocallyLTFinite`: every vertex has finitely many earlier neighbours (the `LT` analogue of `LocallyFinite`).
- `greedyChoice`: the least colour not used by any earlier neighbour of a vertex, via `Nat.find`.
- `greedy`: the greedy $\mathbb{N}$-colouring of a graph on $\mathbb{N}$, defined by well-founded recursion.
- `greedyColoringNat`: `greedy` bundled as a `G.Coloring ℕ` for `G : SimpleGraph ℕ`.
- `crownGraph`: the complete bipartite graph on the even and odd naturals minus the perfect matching `2 * n ↦ 2 * n + 1`.
- `crownGraph.bicoloring`: the natural `Bool`-colouring of `crownGraph` by parity.
- `SimpleGraph.encode`: a graph on an encodable vertex type pushed forward to $\mathbb{N}$ along `Encodable.encode` via `SimpleGraph.map`; its adjacency is decided using `Encodable.decode₂`.
- `Embedding.encode`: the graph embedding of `G` into `G.encode`, via Mathlib's `Embedding.map`.
- `greedyColoringFin`: the greedy `Fin (Δ + 1)`-colouring of a locally finite graph with all degrees at most `Δ` on an encodable vertex type.
- `greedyTraced`: a copy of `greedy` that `dbg_trace`s a `*` on every call, used to count recursive calls.
- `withTraceCount`: runs a computation with its output captured, returning the number of `*`s it prints with `dbg_trace`.
- `greedyList`: the list of the first `n` greedy colours, built by structural recursion so that each colour is computed once.
- `greedyMemo`: the memoized greedy colouring, reading the last entry of `greedyList`; `greedy` is compiled to it via `@[csimp]`.
- `greedyListTraced`: a copy of `greedyList` that `dbg_trace`s a `*` on every call to `greedyChoice`, showing that memoization makes the number of calls linear.
- `greedyColoring`: the greedy $\mathbb{N}$-colouring of a graph on an encodable vertex type, colouring each vertex by the `greedy` colour of its encoding.

From the exercises:

- `kneserGraph.coloring`: the explicit `(n - 2 * k + 2)`-colouring of the Kneser graph `Kn[n, k]` by least element.
- `starNat`: the infinite star on $\mathbb{N}$, in which `0` is adjacent to every other vertex.

Chapter 8: Walks

- `pathNat.path`: walk on the infinite path from `i` to `i + j`.
- `pathGraph.path`: walk along the finite path graph `P[n + 1]` from `0` to `Fin.last n`.
- `HypercubeVert.hammingPath`: path between two hypercube vertices.
- `HypercubeVert.hammingDist`: Hamming distance between two hypercube vertices (number of differing coordinates).
- `HypercubeVert.grayStart` / `HypercubeVert.grayEnd`: start and end vertices of the Gray code walk.
- `hypercube.grayPath`: Gray code walk on `Q[n]`.
- `Walk.mycielskianZigZag`: zig-zag walk in `mμ(G)`.
- `Walk.mycielskian`: walk in `mμ(G)` constructed from a walk in `G` (`protected`, to coexist with the graph constructor `mycielskian`).
- `Walk.mycielskianClosed`: closed walk in `mμ(G)` constructed from a closed walk in `G` (a cycle if the original is one).
- `Walk.posaRotate`: Pósa rotation of a path walk.
- `Walk.posaClosed`: closing a Pósa path to obtain a cycle.
- `Walk.IsMaximalPath`: a *maximal* path — a path that cannot be extended at either end, i.e. every neighbour of either endpoint already lies on it.
- `IsOrePair`: Ore's condition at a pair of vertices — if distinct and non-adjacent, their degree sum is at least `|V|`.
- `IsClosed`: a *closed* graph — one with no eligible pair, i.e. any two distinct vertices with degree sum at least `|V|` are adjacent.
- `closureStep`: one round of the closure process — join every eligible pair at once.
- `closureAux`: fuel-limited iteration of `closureStep`.
- `closure`: the Bondy–Chvátal closure of a graph (computable).
- `IsOreGraph`: an *Ore graph* — every pair of vertices satisfies Ore's condition `IsOrePair`.

From the exercises:

- `chordedC5`: `C[5]` with the two chords `0 – 2` and `0 – 3`, a graph that is not an Ore graph but whose closure is complete.
- `Walk.IsMaximumPath`: a *maximum-length* path — a path such that no path in `G` is longer.

Chapter 9: Subgraphs

- `Subgraph.homOfMapLE`: a homomorphism `f` restricts to a homomorphism `A.coe →g B.coe` whenever `A.map f ≤ B`.
- `Subgraph.mycielskianLeft`: lift of a subgraph in *G* to a subgraph in `mμ(G)`.
- `matchingNum` / `ν G`: matching number of *G* (in $\mathbb{N}\infty$, mirroring `SimpleGraph.vertexCoverNum`), the maximum number of edges in a matching of *G*.
- `Subgraph.boxProdLeft`: a subgraph `M` of *G* copied into every *G*-fibre of `G □ H`.
- `hypercube.matching`: perfect matching of `Q[n + 1]`, defined recursively as `τ` on `Q[1]` and the fibre-copy of the previous matching thereafter.
- `Subgraph.mycielskianDouble`: the subgraph of `mμ(G)` with edges $\texttt{x}^{\texttt{l}}$ – $\texttt{y}^{\texttt{r}}$ and $\texttt{x}^{\texttt{r}}$ – $\texttt{y}^{\texttt{l}}$ for each edge `x – y` of a subgraph of *G*.
- `Subgraph.mycielskianDoubleDartEmbedding`: embedding of the darts of a subgraph into the edge set of its Mycielskian double.
- `ColoringOn`: colouring of every vertex of *G* that need only be proper on the edges of *G* with both ends in `s`.
- `ColorableOn`: predicate for the existence of a `G.ColoringOn (Fin n) s`.
- `ColoringOn.restrict`: restriction of a `G.ColoringOn γ s` to any `t ⊆ s`.
- `ColoringOn.copy`: transport a `G.ColoringOn γ s` along a proof that `s = t`.
- `ColoringOn.toColoring`: a `G.ColoringOn γ Set.univ` is a `G.Coloring γ`.
- `Coloring.toColoringOn`: the `ColoringOn` of a set `s` induced by a colouring of the whole graph.
- `ColoringOn.empty`: the colouring of the empty set of vertices by a given colour `c`.
- `ColoringOn.singleton`: colouring of `{a}` by any colour `c`.
- `ColoringOn.ofNotAdj`: colouring of `{a, b}` by a single colour `c` when `¬G.Adj a b`.
- `ColoringOn.union`: combination of compatible colourings of `s` and `t` into a colouring of `s ∪ t`.
- `ColoringOn.insert`: extension of a colouring of `s` to `insert a s` using a colour not used on neighbours of `a` in `s`.
- `ColoringOn.map`: relabelling of the colours of a partial colouring along an injective map.

Chapter 10: Connectivity

- `isoSigmaConnectedComponents`: the isomorphism between *G* and the sigma graph of its connected components.
- `EdgesInOneComponent`: every edge of *G* lies in a single connected component (any two non-isolated vertices are reachable from each other).
- `locallyFiniteDeleteEdges`: the graph *G*`.deleteEdges s` obtained by deleting edges from a locally finite graph is locally finite.
- `IsVertexConnected`: predicate for a graph being *k*-vertex-connected (*k* `:` $\mathbb{N}\infty$, as for Mathlib's `IsEdgeConnected`).
- `edgeConnectivity` / $\lambda(G)$: the edge connectivity $\lambda(G)$ `:` $\mathbb{N}\infty$, the supremum of the *k* for which *G* is *k*-edge-connected.
- `vertexConnectivity` / $\kappa(G)$: the vertex connectivity $\kappa(G)$ `:` $\mathbb{N}\infty$, the supremum of the *k* for which *G* is *k*-vertex-connected.
- `walkComponent`: the set of vertices of a finite set `W` reachable from v_2 by a walk lying inside `W`.
- `IsCutVertex`: `x` is a cut vertex of *G* (some two vertices other than `x` are joined by a walk, but every walk between them passes through `x`).
- `CycleEquivalent`: two potential edges (elements of `Sym2 V`) are cycle-equivalent if they are equal or lie on a common cycle.
- `BlockEquivalent`: the equivalence relation on potential edges generated by `CycleEquivalent`.
- `blockSetoid`: the setoid instance on *G*`.edgeSet` given by block-equivalence.
- `Block`: a block of *G*, i.e. a block-equivalence class of edges.
- `blockMk`: the block containing a given edge.
- `Adj.block`: the block containing the edge `u – v`, written `h.block` for `h : G.Adj u v`.
- `Block.toSubgraph`: the subgraph of *G* underlying a block, formed by the edges of *G* in that block and their endpoints.
- `Block.map`: the image of a block under a copy `f : Copy G G'`, the block containing the images of its edges.
- `SplitsAt`: *G* is the union of two proper subgraphs whose supports meet in at most a given vertex.

From the exercises:

- `botComponentEquiv`: equivalence between the connected components of the empty graph and its vertex type.
- `Hom.sigmaElim`: the universal property of the disjoint union: a family of homomorphisms *G* `i →g H` gives a homomorphism `Σg G →g H`.

- `Finset.fiberProj`: the vertices `z` of G whose copy `(z, b)` in the G-fibre at `b` of $G \square K_2$ lies in a given vertex set.
- `bowtie`: two triangles sharing a vertex, a graph with $\kappa(G) < \lambda(G)$.
- `barbell`: two copies of `K[4]` joined by an edge, a graph with $\lambda(G) < \delta(G)$.

Chapter 11: Distance

- `EdgesFormMatching`: the edges form a matching, i.e. every vertex has at most one neighbour.
- `mycielskianFourCycle`: 4-cycle in $\mu(G)$ using `none` and left/right copies.
- `layer`: the `i`-th distance layer around a vertex `r`, i.e. the `Finset` of vertices at distance exactly `i` from `r`.

Chapter 12: Algebraic Graph Theory

- `Iso.autCongr`: transport of the automorphism group of a graph along an isomorphism.
- `cycleGraph.reflect`: automorphism of `C[n]` reflecting the vertices (`k ↦ -k`).
- `autCycleGraphMulEquivDihedral`: the automorphism group of `C[n + 3]` is isomorphic to `DihedralGroup (n + 3)`.
- `autHypercubeTwoMulEquivDihedral`: the automorphism group of `Q[2]` is isomorphic to `DihedralGroup 4`.
- `mycielskianLift`: the vector on the vertices of $\mu(G)$ with value `0` at the apex, `t * x v` at v^{l} and `x v` at v^{r}.
- `HoffmanSingletonVert`: the vertex type of the Hoffman–Singleton graph, two copies of `ZMod 5 × ZMod 5` (five pentagons and five pentagrams).
- `hoffmanSingletonGraph`: the Hoffman–Singleton graph, built from five pentagons and five pentagrams.
- `HypercubeVert.equivPi`: equivalence between the hypercube vertex types `HypercubeVert n` and `Fin n → Bool`, via `Fin.snoc`.
- `hypercubeIsoHamming`: the hypercube `Q[n]` is isomorphic to the Hamming cube `hypercubeHamming n`.
- `hypercubeHamming.translate` / `hypercubeHamming.permute`: translation by a fixed vector (pointwise `xor`) and coordinate permutation as automorphisms of the Hamming cube.
- `hyperoctahedralAction`: the action of `Equiv.Perm (Fin n)` on translation vectors by permuting coordinates.
- `HyperoctahedralGroup`: the hyperoctahedral group $(\mathbb{Z}/2)^n \rtimes S_n$, i.e. the wreath product $\mathbb{Z}/2 \wr S_n$, as a semidirect product.
- `hypercubeHamming.translateHom` / `hypercubeHamming.permuteHom`: translations and coordinate permutations as monoid homs into the automorphism group of the Hamming cube.

- `hyperoctahedralToAut`: the homomorphism from the hyperoctahedral group to the automorphisms of the Hamming cube.
- `autHypercubeHammingMulEquivHyperoctahedral`: the automorphism group of the Hamming cube is the hyperoctahedral group.
- `autHypercubeMulEquivHyperoctahedral`: the automorphism group of `Q[n]` is the hyperoctahedral group $\mathbb{Z}/2 \wr S_n$.
- `copyEquivSigmaIso`: equivalence between `A.Copy B` and the Sigma type of subgraphs of `B` isomorphic to `A` with their isomorphisms.
- `copyEquivSubgraphIsoProdAut`: equivalence `A.Copy B ≃ {C : B.Subgraph | Nonempty (A ≃g C.coe)} × (A ≃g A)`.
- `Iso.copyEquivIso`: for finite `A`, an isomorphism `A ≃g B` identifies copies of `A` in `B` with isomorphisms `A ≃g B`.
- `copyEquivAut`: copies of a finite graph in itself are equivalent to its automorphisms.
- `copyEquivSubgraphIsoProdCopy`: `A.Copy B` is equivalent to the subgraphs of `B` isomorphic to `A` paired with copies of `A` in itself.
- `Iso.copyCongr`: isomorphisms `A ≃g A'` and `B ≃g B'` identify the copies of `A` in `B` with the copies of `A'` in `B'`.
- `hypercubeHamming.extendEmb` / `hypercubeHamming.extendHom`: extension of a `k`-vector to an `n`-vector along an embedding of directions `Fin k ↪ Fin n`, and the induced homomorphism of Hamming cubes.
- `hypercubeHamming.subcubeCopy`: the canonical copy of `Q[k]` in `Q[n]` with a given base point and set of free directions.
- `hypercubeHamming.copyEquivProd`: the copies of `Q[k]` in `Q[n]` correspond to pairs of a base point and a direction embedding.
- `copyEquivEmbedding`: if every copy of `A` in `B` reflects adjacency, then copies of `A` in `B` are the same as embeddings `A ↪g B`.

From the exercises:

- `autMulLeftCayley`: left multiplication by `g` as an automorphism of the Cayley graph `mulCayley s`.
- `autTopMulEquivPerm`: the automorphism group of a complete graph is the symmetric group.
- `autComplMulEquiv`: a graph and its complement have the same automorphism group.
- `boolVecEquivFinset`: Bool vectors of length `n` correspond to subsets of `Fin n`.
- `hypercubeHammingIsoHasse`: the Hamming cube is isomorphic to the Hasse diagram `hypercubeHasse n`.
- `copyTopEquiv`: copies of a graph in a complete graph are the injective maps of vertices.

Chapter 13: Theorems

- `subdividedStar`: the subdivided star on `ι`, with centre `none` adjacent to every `i ʳ` and each `i ˡ` adjacent only to `i ʳ`.
- `IsCompleteMultipartite.finpartition`: the parts of a finite complete multipartite graph, as a `Finpartition` of its vertices.
- `IsCopyExtremal`: *`G`* has property `p` and the most copies of `H` among the graphs on its vertices with property `p`, generalizing Mathlib's `IsExtremal` from edges.
- `IsNCliqueMaximal`: *`G`* is `s`-clique-maximal, that is, `(r + 1)`-clique-free and with the most `s`-cliques among such graphs on its vertices.
- `cliqueUnion`: the union of the `s`-cliques of *`G`*; distinct vertices are adjacent when they lie in a common `s`-clique.
- `paleyGraph₉`: the Paley graph on `9` vertices, of the field `𝔽₃[i]` built as `QuadraticAlgebra (ZMod 3) (-1) 0`; two vertices are adjacent when they agree in one coordinate.
- `IsDirection`: a direction of *`G`*, a consistent choice of a component of *`G`* `- K` for every finite set `K` of vertices (the component for `L` lies inside the one for `K` whenever `K ⊆ L`).

Results

The following results are proved in this guide and its exercises:

Chapter 3: Constructions (Results)

From the exercises:

- `paleyGraph₁₇_eq_circulant`: the Paley graph on 17 vertices is equal to the circulant graph with differences `{1, 2, 4, 8, 9, 13, 15, 16}`.

Chapter 4: Decidability, Finiteness and Degrees (Results)

- `good_card_edgeFinset`: the number of edges of *`G`* depends only on `[Fintype G.edgeSet]`; contrast `bad_card_edgeFinset`, which needlessly assumes `[Fintype V]` and `[DecidableRel G.Adj]`.
- `degree_pathNat`: degrees in the infinite path graph on $\mathbb{N}$, which is locally finite without having a finite vertex type.
- `degree_blowUpN` / `IsRegularOfDegree.blowUpN`: each vertex of the `n`-uniform blow-up *`G`*`[n]` has degree `n` times its degree in *`G`*, so the blow-up of a `d`-regular graph is `d * n`-regular.
- `isRegularOfDegree_card_sub_one_iff_eq_top`: a graph on a finite vertex type is regular of degree `|V| - 1` iff it is complete.
- `sum_degrees_eq_twice_card_edges'`: the handshake lemma with minimal instance arguments, dropping the `[DecidableRel G.Adj]` of the Mathlib version; the `Fintype` diamond this exposes is closed by `Subsingleton.elim`.

- `IsRegularOfDegree.two_mul_card_edgeFinset` / `IsRegularOfDegree.card_edgeFinset`: a d-regular graph on n vertices has `d * n / 2` edges.
- `card_hypercubeVert`: the hypercube `Q[n]` has `2 ^ n` vertices.
- `IsRegularOfDegree.hypercube`: the hypercube `Q[n]` is n-regular.
- `IsRegularOfDegree.boxProd`: a box product of regular graphs is regular, with the degrees adding.
- `card_edgeFinset_hypercube`: the hypercube `Q[n]` has `n * 2 ^ (n - 1)` edges.
- `degree_mycielskian_left`: the degree of a left vertex v^l of the Mycielskian is twice the degree of `v`.
- `Graphic.exists_card_eq`: a graphic multiset `m` is the degree multiset of a graph with exactly `m.card` vertices.
- `graphic_degreeMultiset` / `graphic_degreeSequence`: the degree multiset, and degree sequence, of a finite simple graph are graphic.
- `degree_coneGraph_none`: the new vertex of `coneGraph H s` has degree `s.card`, a key ingredient of the Havel–Hakimi theorem.
- `erdos_gallai` / `havel_hakimi`: the two characterizations of graphic multisets. Both are *stated but not proved* (each is a `sorry`); `havel_hakimi` is used only to give a `DecidablePred Graphic` instance that `#eval!` can run.

From the exercises:

- `degree_mycielskian_right` / `degree_mycielskian_none`: a right vertex v^r of the Mycielskian has degree one more than `v`, and the apex has degree `|V|`.
- `card_edgeFinset_mycielskian`: the Mycielskian `mμ(G)` has `3 * |E| + |V|` edges.
- `maxDegree_mycielskian` / `minDegree_mycielskian`: the maximum and minimum degrees of `mμ(G)` are `max (2 * Δ(G)) (Fintype.card V)` and `min (2 * δ(G)) (δ(G) + 1)`.
- `Graphic.even_sum`: a graphic multiset has an even sum.
- `degree_coneGraph_some` / `degreeMultiset_coneGraph`: the remaining degrees of `coneGraph H s`, and its degree multiset.

Chapter 5: Maps (Results)

- `IsVertexTransitive.exists_isRegularOfDegree`: a locally finite vertex-transitive graph is regular.
- `isVertexTransitive_top`: the complete graph is vertex-transitive.
- `isVertexTransitive_cycleGraph`: the cycle graph `C[n]` is vertex-transitive.
- `isVertexTransitive_hypercube`: the hypercube `Q[n]` is vertex-transitive.

- `isVertexTransitive_mycielskian_iff`: for finite nonempty *V*, the Mycielskian `mμ(G)` is vertex-transitive iff *G* `≃g K[2]`.
- `degree_tensorProd`: the degree of `(v, w)` in the tensor product is `G.degree v * H.degree w`.
- `clawFree_lineGraph`: line graphs are claw-free.
- `top_three_isContained_lineGraph_iff`: the line graph of *G* contains a triangle iff *G* contains a triangle or a (not necessarily induced) copy of the claw.

From the exercises:

- `IsVertexTransitive.compl` / `isVertexTransitive_bot`: the complement of a vertex-transitive graph is vertex-transitive, so `⊥` is vertex-transitive.

Chapter 6: Cliques (Results)

- `Hom.cliqueNum_le`: a homomorphism into a finite graph cannot decrease the clique number.
- `Iso.cliqueNum_eq`: isomorphic graphs have the same clique number (no finiteness needed).
- `CliqueFree.cliqueNum_le`: an `(n + 1)`-clique-free graph has clique number at most `n`.
- `IsNClique.cliqueNum_eq_of_cliqueFree` / `IsNIndepSet.indepNum_eq_of_indepSetFree`: a graph with an `n`-clique but no `(n + 1)`-clique has `ω G = n`, and likewise for independent sets and `α G`.
- `degree_le_indepNum_of_triangleFree`: in a finite triangle-free graph every vertex has degree at most `α G`.
- `indepNum_kneserGraph`: the independence number of `Kn[n, k]` (for `k ≠ 0`) is `(n - 1).choose (k - 1)` if `2 * k ≤ n` and `n.choose k` otherwise, via the Erdős–Ko–Rado theorem.
- `CliqueFree.mycielskian`: the Mycielskian of a triangle-free graph is triangle-free.
- `CliqueFree.iterMycielskian` / `cliqueFree_three_iterMycielskian_completeGraph_two`: the iterated Mycielskians of a triangle-free graph are triangle-free; in particular so is every `mμ^n(K[2])`.
- `IsNClique.mycielskian`: the Mycielskian embedding maps `k`-cliques in *G* to `k`-cliques in `mμ(G)`.
- `mycielskian_none_notMem_clique`: the apex `none` does not belong to any `k`-clique in `mμ(G)` for `k ≥ 3`.
- `IsNClique.of_mycielskian`: any `k`-clique in `mμ(G)` (with `k ≥ 3`) projects to a `k`-clique in *G* under the Mycielskian projection.
- `CliqueFree.mycielskian_of_three_le`: for `r ≥ 3`, the Mycielskian of an `r`-clique-free graph is `r`-clique-free.
- `cliqueNum_mycielskian_of_ne_bot`: the clique number of `mμ(G)` equals that of *G* whenever *G* is finite and has an edge.

- `cliqueNum_iterMycielskian_completeGraph_two`: every iterated Mycielskian of `K[2]` has clique number 2.
- `IsIndepSet.mycielskian`: if `s` is an independent set of *G* then the left copies of `s` together with the right copies of every vertex having no neighbour in `s` form an independent set of `mμ(G)`.
- `indepNum_mycielskian`: the independence number of `mμ(G)` is the maximum, over the independent sets `s` of *G*, of `#s` plus the number of vertices with no neighbour in `s`.

From the exercises:

- `cliqueNum_mono`: the clique number of a finite graph is monotone in the subgraph order.
- `cliqueNum_add_indepNum_le`: `ω G + α G ≤ |V| + 1` for a finite graph.
- `not_cliqueFree_three_and_indepSetFree_three`: every graph on at least 6 vertices contains a triangle or an independent set of size 3, i.e. `R(3, 3) ≤ 6`.
- `cliqueNum_mycielskian_bot_of_isEmpty` / `cliqueNum_mycielskian_bot`: the clique number of `mμ(⊥)` is 1 if *V* is empty and 2 otherwise.

Chapter 7: Colorings (Results)

- `chromaticNumber_ne_top_of_finite`: a graph on a finite vertex type has finite chromatic number.
- `Colorable.natCard_le_mul_indepNum` / `natCard_le_chromaticNumber_mul_indepNum`: a graph has at most `χ G * α G` vertices, since colour classes are independent (trivially if *V* is infinite, when `Nat.card V = 0`).
- `Colorable.sup_of_isClique_inter` / `Colorable.sup_of_subsingleton_inter`: colourings of G_1 and G_2 from a common palette amalgamate to a colouring of $G_1 \sqcup G_2$ when the supports meet in a clique (in particular in at most one vertex).
- `colorable_blowUpGraph_iff`: a blow-up of *G* with nonempty parts is k-colourable iff *G* is.
- `Colorable.boxProd` / `colorable_boxProd_iff` / `chromaticNumber_boxProd`: `G □ H` is k-colourable when both factors are (iff, for nonempty *G* and H), and `χ (G □ H) = max (χ G) (χ H)` for nonempty *G* and H.
- `IsBipartite.hypercube` / `cliqueFree_hypercube_three`: the hypercube `Q[n]` is bipartite, hence triangle-free.
- `colorable_mycielskian_succ_iff`: *G* is k-colourable iff `mμ(G)` is `(k + 1)`-colourable.
- `chromaticNumber_mycielskian`: `χ mμ(G) = χ G + 1`.
- `chromaticNumber_iterMycielskian` / `chromaticNumber_iterMycielskian_completeGraph_two`: the n-th iterated Mycielskian of *G* has chromatic number `χ G + n`, so that of `K[2]` has chromatic number `n + 2`.

- `mycielski`: Mycielski's theorem, `mμ^n(K[2])` has clique number `2` and chromatic number `n + 2`.
- `degree_le_chromaticIndex` / `maxDegree_le_chromaticIndex`: the chromatic index `χ' G` is at least the degree of every vertex, and so at least `Δ(G)` for a finite graph.
- `greedy_lt` / `Adj.greedy_ne`: the greedy colour of `v` is at most `degreeLT v`, and adjacent vertices receive different greedy colours.
- `colorable_of_degreeLT_le`: *the greedy bound* for graphs on `ℕ`: if every vertex has at most `Δ` earlier neighbours then the graph is `(Δ + 1)`-colourable.
- `neighborLTFinset_crownGraph` / `greedy_crownGraph_eq`: in `crownGraph` the earlier neighbours of `n` are the `n / 2` vertices of the other parity below `n`, and the greedy colouring colours `n` with `n / 2`, the worst possible.
- `degreeLT_encode_le`: `degreeLT` of an encoded vertex is at most the degree of the original vertex.
- `colorable_of_degree_le_of_countable`: a locally finite graph on a countable vertex type with all degrees at most `Δ` is `(Δ + 1)`-colourable.
- `greedyMemo_eq_greedy` / `greedy_eq_greedyMemo`: the memoized greedy colouring agrees with `greedy`; the second is the `@[csimp]` lemma that makes `#eval greedy` run the memoized version.

From the exercises:

- `le_mul_of_colorable_kneserGraph`: if the Kneser graph `Kn[n, k]` is `m`-colourable then `n ≤ k * m`, so `n ≤ k * χ Kn[n, k]`.

Chapter 8: Walks (Results)

- `Walk.exists_odd_isCycle` / `isBipartite_iff_forall_isCycle_even`: a closed walk of odd length contains a cycle of odd length, so a graph is bipartite iff it has no odd cycle.
- `Walk.IsEulerian.even_degree_iff'`: Mathlib's `even_degree_iff` (in a graph with an Eulerian trail every vertex has even degree, except the endpoints of a non-closed trail) without `Fintype V`, needing only a finite neighbour set.
- `HypercubeVert.isPath_hammingPath` / `HypercubeVert.length_hammingPath`: the Hamming walk between any two vertices in `Q[n]` is a path of length equal to their Hamming distance.
- `hypercube.isHamiltonian_grayPath`: the Gray code walk on `Q[n]` is Hamiltonian.
- `isCycle_mycielskianClosed_of_adj`: every edge of `G` gives a `5`-cycle in `mμ(G)`.
- `IsHamiltonian.mycielskian`: if `G` is Hamiltonian and does not have exactly one vertex, then its Mycielskian `mμ(G)` is Hamiltonian.
- `Walk.IsPath.posaRotate` / `Walk.IsPath.isCycle_posaClosed`: a Pósa rotation of a path is again a path; closing a path of length at least two at a crossing pair yields a cycle.

- `Walk.IsMaximalPath.exists_crossing_index`: pigeonhole counting — a maximal path whose endpoints have degree sum at least `|V|` has a *crossing pair*.
- `Walk.IsMaximalPath.exists_cycle_spanning`: a maximal path of length at least two whose endpoints form an `IsOrePair` can be closed (by `concat` or `posaClosed`) into a cycle through exactly its vertices.
- `Walk.IsCycle.exists_isPath_of_mem_edges`: a cycle through an edge `u – v` can be cut open into a path from `u` to `v` through the same vertices, avoiding `u – v`.
- `Walk.IsHamiltonianCycle.exists_isHamiltonian_walk_of_mem_edges`: a Hamiltonian cycle through an edge `u – v` can be cut open into a Hamiltonian walk from `u` to `v` avoiding `u – v`.
- `isHamiltonian_sup_edge_iff`: joining a non-adjacent pair with degree sum at least `|V|` does not change whether the graph is Hamiltonian.
- `isHamiltonian_closureStep_iff`: a round of the closure process does not change whether the graph is Hamiltonian.
- `isHamiltonian_closure_iff`: the Bondy–Chvátal theorem — a graph is Hamiltonian iff its closure is.
- `le_closure`: `G ≤ G.closure`, i.e. forming the closure only adds edges.
- `isClosed_closure`: the closure is closed — no eligible pair remains in it.
- `closure_min`: the closure is below every closed graph above `G`, hence is the *least* closed supergraph — independent of the order in which eligible pairs are joined.
- `closure_mono` / `closure_closure`: monotonicity and idempotence of the closure, each a one-line consequence of `closure_min`.
- `isHamiltonian_of_closure_eq_top`: a graph on at least three vertices whose closure is complete is Hamiltonian.
- `IsOreGraph.mono` / `IsOreGraph.eq_top_of_isClosed`: Ore's condition is inherited by supergraphs, and a closed Ore graph is complete.
- `IsOreGraph.closure_eq_top`: the closure of an Ore graph is the complete graph.
- `IsOreGraph.isHamiltonian`: Ore's theorem — a graph on at least three vertices satisfying Ore's degree condition is Hamiltonian.
- `isHamiltonian_of_card_le_two_mul_minDegree`: Dirac's theorem — a graph on at least three vertices with minimum degree at least `|V| / 2` is Hamiltonian.

From the exercises:

- `HypercubeVert.hammingDist_comm`: the Hamming distance on the hypercube is symmetric.
- `isHamiltonian_hypercube`: the hypercube `Q[n]` is Hamiltonian for `n ≠ 1`.

- `Walk.IsHamiltonian.isHamiltonianCycle_posaClosed`: closing a Hamiltonian path via a Pósa rotation yields a Hamiltonian cycle.
- `Walk.IsMaximumPath.isMaximalPath`: a maximum-length path is in particular a maximal path, so the chapter's closing construction applies to it.
- `Walk.isHamiltonian_of_isMaximumPath`: a maximum-length path in an Ore graph is Hamiltonian — the classical longest-path proof of Ore's theorem.

Chapter 9: Subgraphs (Results)

- `colorable_of_forall_finite_subgraph_colorable`: the De Bruijn–Erdős theorem, a graph is `n`-colourable if all its finite subgraphs are.
- `colorable_of_degree_le`: a locally finite graph of maximum degree at most `Δ` is `(Δ + 1)`-colourable.
- `Subgraph.IsMatching.le_matchingNum` / `matchingNum_le` / `exists_isMatching_matchingNum`: the basic API for the matching number `ν G` (lower bounds from matchings, upper bounds from uniform bounds, attainment).
- `Subgraph.IsMatching.ncard_inter_le`: in a graph that is bipartite with parts p_1 and p_2, a finite matching has at least as many edges as it covers vertices of p_1.
- `matchingNum_le_card_div_two` / `Subgraph.IsPerfectMatching.matchingNum_eq`: a finite graph has `ν G ≤ |V| / 2`, with equality if it has a perfect matching.
- `matchingNum_le_vertexCoverNum`: weak duality `ν G ≤ τ G`.
- `matchingNum_hypercube_succ`: `ν Q[n + 1] = 2 ^ n`.
- `two_mul_matchingNum_le_matchingNum_mycielskian` / `Subgraph.IsPerfectMatching.matchingNum_mycielskian_eq`: `2 * ν G ≤ ν mμ(G)`, with equality if `G` has a perfect matching.

From the exercises:

- `Subgraph.mem_verts_iff_of_isCompl`: if subgraphs `H` and `K` are complementary (`IsCompl H K`) then no edge of `G` has exactly one endpoint in `H`.
- `Walk.exists_odd_isCycle_toSubgraph_le` / `Walk.exists_odd_isCycle_length_le`: a closed walk of odd length `k` contains an odd cycle that is a subgraph of the walk, and so has length at most `k`.
- `Walk.IsTrail.length_le_of_toSubgraph_le`: a trail that is a subgraph of a walk is no longer than the walk.
- `colorable_of_forall_finite_induce_colorable`: a graph is `n`-colourable if all its finite induced subgraphs `G.induce s` are.
- `matchingNum_mono`: the matching number is monotone in the subgraph order.
- `matchingNum_pathNat`: the infinite path has matching number `ℵ`.

- `indepNum_add_matchingNum_le`: in a finite graph `α G + ν G ≤ |V|`.
- `matchingNum_exists`: the matching number is attained in every graph, finite or not; when `ν G = ⊤` a greedy chain of matchings produces an infinite one.
- `colorableOn_iff_colorable_induce`: for `n > 0`, `G.ColorableOn n s` iff `G.induce s` is n-colourable.

Chapter 10: Connectivity (Results)

- `connected_hypercube`: the n-dimensional hypercube is connected.
- `connected_mycielskian_iff`: the Mycielskian `mμ(G)` is connected iff `G` has no isolated vertices.
- `isBridge_mycielskian_iff`: the bridges of `mμ(G)` are the edges `none — u^r` with u isolated in `G`.
- `Walk.IsCycle.mem_support_iff_reachable_of_degree_le_two` / `Walk.IsCycle.toSubgraph_eq_of_degree_le_two` / `Walk.IsCycle.isHamiltonianCycle_of_degree_le_two`: in a graph of maximum degree at most 2, every cycle passes through every vertex of its component, and as a subgraph is equal to that component, so in a connected such graph every cycle is a Hamiltonian cycle.
- `connected_pathNat` / `isAcyclic_pathNat` / `isTree_pathNat`: the infinite path on $\mathbb{N}$ is a tree (so every edge is a bridge).
- `exists_closed_isEulerian`: Euler's theorem, existence direction (in a graph with finitely many edges, if all edges lie in one component and every vertex has even degree, there is a closed Eulerian walk, which is `nil` if there are no edges).
- `exists_isEulerian_isCircuit`: the same, with at least one edge, gives an Eulerian circuit (`IsEulerian` and `IsCircuit`).
- `exists_isEulerian_trail_of_two_odd`: in a graph with finitely many edges, if all edges lie in one component and exactly two vertices `a ≠ b` have odd degree, there is an Eulerian trail from a to b.
- `Connected.exists_closed_isEulerian` / `Connected.exists_isEulerian_trail`: the same two results for a connected graph, the form in which Euler's theorem is usually stated.
- `exists_isEulerian_iff`: Euler's theorem in full — a finite graph with at least one vertex has an Eulerian trail iff all its edges lie in one component and it has either no vertex of odd degree or exactly two.
- `isVertexConnected_iff_induce`: `G` is k-vertex-connected iff it has more than k vertices and `G.induce S^c` is preconnected whenever `S.encard < k`.
- `IsVertexConnected.isEdgeConnected`: a k-vertex-connected graph is k-edge-connected.
- `vertexConnectivity_le_edgeConnectivity`: `κ(G) ≤ λ(G)` for every graph.
- `brooks_two`: Brooks' theorem for `Δ = 2`, a graph of maximum degree at most 2 is 2-colourable unless one of its components is an odd cycle.

- `colorable_of_cliqueFree_of_degree_le`: Brooks' theorem for `Δ ≥ 3`, a locally finite graph with maximum degree at most `Δ` and no `(Δ + 1)`-clique is `Δ`-colourable.
- `brooks`: Brooks' theorem for every `Δ`.
- `isVertexConnected_two_iff`: a graph is 2-vertex-connected iff it has at least three vertices, is connected and has no cut vertex.
- `IsCutVertex.splitsAt` / `splitsAt_of_not_reachable`: a graph splits at each of its cut vertices, and at every vertex if its edges lie in two different components.
- `colorable_of_forall_block`: for `n ≥ 1`, a finite graph is `n`-colourable if each of its blocks is.

From the exercises:

- `IsTree.degree_le_card_leaves`: a finite tree has at least as many leaves as the degree of any of its vertices.
- `IsVertexConnected.le_minDegree`: a `k`-vertex-connected graph has minimum degree at least `k`.
- `IsVertexConnected.boxProd_top_bool`: if *G* is connected and `k`-vertex-connected then *G* `□` K_2 is `(k + 1)`-vertex-connected.
- `isVertexConnected_hypercube` / `isEdgeConnected_hypercube`: the `n`-dimensional hypercube is `n`-vertex-connected and `n`-edge-connected.
- `edgeConnectivity_le_minDegree`: `λ(G) ≤ δ(G)` for a finite graph with at least two vertices (completing Whitney's inequalities).
- `vertexConnectivity_hypercube` / `edgeConnectivity_hypercube_succ`: `κ(Q[n]) = n`, and `λ(Q[n + 1]) = n + 1` (while `λ(Q[0]) = ⊤`).
- `IsAcyclic.toSubgraph_blockMk`: in a forest the block of an edge is the subgraph consisting of that edge and its endpoints.
- `colorable_iff_forall_block`: for `n ≥ 1`, a graph is `n`-colourable iff each of its blocks is.

Chapter 11: Distance (Results)

- `edist_eq_hammingDist`: the graph distance between any two vertices in `Q[n]` is exactly equal to their Hamming distance.
- `hypercube_adj_iff_hammingDist_eq_one`: adjacency in `Q[n]` is equivalent to Hamming distance one.
- `IsAcyclic.edist_eq_length`: in an acyclic graph the distance between the endpoints of a path is its length.
- `edist_pathNat` / `ediam_pathNat` / `diam_pathNat`: in the infinite path the distance from `m` to `m + k` is `k`, so its diameter is `⊤` and its $\mathbb{N}$-valued diameter is the junk value `0`.
- `Iso.eccent_eq` / `Iso.ediam_eq` / `Iso.radius_eq`: eccentricity, diameter and radius are isomorphism invariants.

- `IsVertexTransitive.eccent_eq` / `IsVertexTransitive.radius_eq_ediam` / `IsVertexTransitive.center_eq_univ`: in a vertex-transitive graph all eccentricities are equal, the radius equals the diameter and every vertex is central.
- `ediam_boxProd`: the diameter of a box product of graphs on nonempty vertex types is the sum of the diameters.
- `ediam_hypercube` / `radius_hypercube`: the hypercube `Q[n]` has diameter and radius `n`.
- `radius_mycielskian_le_two` / `ediam_mycielskian_le_four`: the Mycielskian of a graph without isolated vertices has radius at most `2` and diameter at most `4`.
- `Colorable.extend_precoloring`: Albertson's theorem, a precolouring of vertices pairwise at distance at least `4` in a `k`-colourable graph extends to a `(k + 1)`-colouring.
- `le_egirth_of_free`: a graph containing no copy of `C[n]` for `3 ≤ n ≤ k` has girth at least `k + 1`.
- `cliqueFree_three_iff_free_cycleGraph`: a graph is triangle-free iff it contains no copy of `C[3]`.
- `four_le_egirth_of_cliqueFree_three` / `five_le_egirth_of_cliqueFree_three_of_free`: a triangle-free graph has girth at least `4`, and at least `5` if it also has no `4`-cycle.
- `egirth_completeBipartiteGraph`: the complete bipartite graph `K[s + 2, t + 2]` has girth `4`.
- `egirth_cycleGraph` / `ediam_cycleGraph`: the cycle graph `C[n + 3]` has girth `n + 3` and diameter `(n + 3) / 2`.
- `egirth_hypercube`: the girth of the hypercube `Q[n + 2]` is `4`.
- `egirth_mycielskian_eq_four` / `egirth_mycielskian_eq_five`: for triangle-free *G* the girth of mμ(*G*) is `4` if two edges of *G* share an endpoint, and `5` if *G* has at least one edge and its edges form a matching.
- `finite_of_eccent_le`: a locally finite graph with a vertex of finite eccentricity is finite.
- `moore_bound` / `moore_bound_of_radius_le`: the Moore bound on the order of a graph of maximum degree at most `d + 1` and diameter (or radius) at most `k`.
- `egirth_le_edist_add_edist_add_two`: two distinct neighbours `a` and `b` of `v`, neither further from `r` than `v`, give a cycle of length at most `G.edist r a + G.edist r b + 2`.
- `cage_bound_odd` / `cage_bound_even`: the cage bounds on the order of a graph of minimum degree at least `d + 1` and girth at least `2 * k + 1` (resp. `2 * k`).

Chapter 12: Algebraic Graph Theory (Results)

- `aut_pathNat_eq_one` / `card_aut_pathNat` / `not_isVertexTransitive_pathNat`: the infinite path `pathNat` is asymmetric, i.e. its only automorphism is the identity, so it is not vertex-transitive.

- `aut_cycleGraph`: every automorphism of `C[n + 2]` is a rotation, or a rotation followed by the reflection.
- `trace_adjMatrix_pow_eq_sum_card_walk`: the trace of A^k is the total number of closed walks of length `k`.
- `adjMatrix_submatrix_eq_iff`: a permutation of the vertices is an automorphism iff it preserves the adjacency matrix.
- `IsVertexTransitive.adjMatrix_pow_apply_self`: in a vertex-transitive graph the diagonal entries of each power of the adjacency matrix are all equal, so every vertex lies on the same number of closed walks of each length.
- `IsVertexTransitive.card_walk_self_eq_trace_div`: in a vertex-transitive graph on `n` vertices each vertex lies on $\mathrm{tr}(A^k)$ / `n` closed walks of length `k`.
- `Iso.charpoly_adjMatrix` / `Iso.spectrum_adjMatrix`: isomorphic graphs have adjacency matrices with the same characteristic polynomial and the same spectrum.
- `adjMatrix_mycielskian_mulVec_mycielskianLift` / `mem_spectrum_adjMatrix_mycielskian`: if `x ≠ 0` is an eigenvector of `G` with eigenvalue `ξ` whose entries sum to zero, then `mycielskianLift φ x` and `mycielskianLift ψ x` are eigenvectors of `mμ(G)` with eigenvalues `φ * ξ` and `ψ * ξ`, where `φ` and `ψ` are the golden ratio and its conjugate.
- `IsRegularOfDegree.finrank_eigenspace_adjMatrix`: in a `d`-regular graph the multiplicity of the eigenvalue `d` is the number of connected components.
- `IsSRGWith.petersenGraph`: the Petersen graph is strongly regular with parameters `(10, 3, 0, 1)`.
- `IsSRGWith.five_le_egirth`: a strongly regular graph with ℓ `= 0` and `μ = 1` has girth at least `5`.
- `ediam_petersenGraph` / `egirth_petersenGraph`: the Petersen graph has diameter `2` and girth `5`, so it is a Moore graph.
- `IsSRGWith.hoffmanSingletonGraph`: the Hoffman–Singleton graph is strongly regular with parameters `(50, 7, 0, 1)`.
- `ediam_hoffmanSingletonGraph` / `egirth_hoffmanSingletonGraph`: the Hoffman–Singleton graph has diameter `2` and girth `5`, so it is a Moore graph.
- `HypercubeVert.hammingDist_equivPi`: `HypercubeVert.equivPi` identifies `HypercubeVert.hammingDist` with Mathlib's `hammingDist`.
- `edist_hypercubeHamming`: the graph distance in the Hamming cube is the Hamming distance.
- `hammingDist_aut_hypercubeHamming` / `hammingNorm_aut_hypercubeHamming`: automorphisms of the Hamming cube preserve Hamming distance (and weight, if they fix `0`).
- `aut_hypercubeHamming_eq_id_of_fix_zero_and_singles`: an automorphism of the Hamming cube fixing `0` and all basis vectors is the identity.

- `card_aut_hypercube`: the hypercube `Q[n]` has exactly `2 ^ n * n.factorial` automorphisms.
- `copyCount_top`: the unlabelled copies of the complete graph on `W` in `G` are the cliques of `G` with `Fintype.card W` vertices; so `G.copyCount K[s]` counts `s`-cliques.
- `labelledCopyCount_eq_copyCount_mul_labelledCopyCount`: the number of labelled copies of a finite graph `A` in `B` is the number of unlabelled copies times the number of automorphisms of `A`.
- `labelledCopyCount_cycleGraph`: the number of labelled copies of `C[n + 3]` in a graph is `2 * (n + 3)` times the number of unlabelled copies.
- `labelledCopyCount_hypercube` / `copyCount_hypercube`: the number of labelled (resp. unlabelled) copies of `Q[k]` in `Q[n]` is `2 ^ n * n.descFactorial k` (resp. `n.choose k * 2 ^ (n - k)`).
- `hypercubeHamming.edist_subcubeCopy`: canonical subcube copies are isometric embeddings.
- `copy_hypercube_adj_iff` / `card_embedding_hypercube`: every copy of `Q[k]` in `Q[n]` is induced, so there are `2 ^ n * n.descFactorial k` embeddings `Q[k] ↪g Q[n]`.

Mathlib theorems surveyed in this chapter:

- `adjMatrix_pow_apply_eq_card_walk`: the `(u, v)` entry of the `n`-th power of the adjacency matrix is the number of walks of length `n` from `u` to `v`.
- `IsRegularOfDegree.mem_spectrum_adjMatrix`: a `d`-regular graph has eigenvalue `d`.
- `card_connectedComponent_eq_finrank_ker_toLin'_lapMatrix`: the number of connected components of a finite graph is the dimension of the kernel of its Laplacian.
- `IsSRGWith.param_eq`: the parameters of a strongly regular graph satisfy `k * (k - ℓ - 1) = (n - k - 1) * μ`.
- `IsSRGWith.ediam_eq_two`: a strongly regular graph with `μ ≠ 0` that is not complete has diameter 2.

From the exercises:

- `isVertexTransitive_mulCayley`: the Cayley graph of a group is vertex-transitive.
- `card_aut_cycleGraph` / `card_aut_completeGraph`: `C[n + 3]` has `2 * (n + 3)` automorphisms and `K[n]` has `n.factorial`.
- `even_degree_of_isVertexTransitive`: in a vertex-transitive graph with an odd number of vertices every degree is even.
- `isVertexTransitive_kneserGraph`: Kneser graphs, and so the Petersen graph, are vertex-transitive.
- `exists_aut_hypercubeHamming_of_adj`: the Hamming cube is arc-transitive.

- `copyCount_completeGraph_cycleGraph`: `K[n + 3]` has `(n + 3).factorial / (2 * (n + 3))` Hamiltonian cycles.
- `copyCount_hypercube_cycleGraph_four`: `Q[n]` contains `n.choose 2 * 2 ^ (n - 2)` 4-cycles.

Chapter 13: Theorems (Results)

- `exists_isMatching_of_forall_ncard_le_add`: Hall's theorem with deficiency `d` — if every `s ⊆ p₁` has at least `|s| - d` neighbours, a matching covers all but at most `d` vertices of `p₁`.
- `IsBipartite.matchingNum_eq_vertexCoverNum`: Kőnig's theorem — `ν G = τ G` for a finite bipartite graph.
- `vertexCoverNum_hypercube_succ`: `τ Q[n + 1] = 2 ^ n`, by Kőnig's theorem.
- `IsCompleteMultipartite.nonempty_iso_turanGraph`: a complete multipartite `(r + 1)`-clique-free graph with balanced parts, to which no edge can be added without creating an `(r + 1)`-clique, is isomorphic to the Turán graph.
- `CliqueFree.exists_colorable_deleteEdges`: Füredi's stability theorem — an `(r + 1)`-clique-free graph can be made `r`-colourable by deleting at most `turanNumber n r - e(G)` edges.
- `card_cliqueFinset_replaceVertex`: giving `b` the neighbours of `a` (the graph `G.replaceVertex a b`) removes the `s`-cliques through `b` and adds as many as there are `s`-cliques through `a` but not `b`.
- `IsCompleteMultipartite.exists_isNClique_of_le`: in a complete multipartite graph with an `s`-clique (`2 ≤ s`), any edge `x - y` of a larger graph lies in an `s`-clique of that graph.
- `IsNCliqueMaximal.nonempty_iso_turanGraph`: an `(r + 1)`-clique-free graph with the most `s`-cliques, where `2 ≤ s`, is a Turán graph if it has an `s`-clique at all.
- `Iso.card_cliqueFinset_eq`: isomorphic graphs have the same number of `s`-cliques.
- `CliqueFree.card_cliqueFinset_le`: Zykov's theorem — an `(r + 1)`-clique-free graph on `n` vertices has at most as many `s`-cliques as `T[n, r]`.
- `CliqueFree.card_cliqueFinset_eq_iff_nonempty_iso_turanGraph`: uniqueness in Zykov's theorem — for `2 ≤ s ≤ r` and `s ≤ n`, an `(r + 1)`-clique-free graph has as many `s`-cliques as `T[n, r]` iff it is isomorphic to it.
- `EdgeDisjointTriangles.card_cliqueFinset_lt`: a graph on `n` vertices whose triangles are edge-disjoint has less than `ε * n²` triangles once `n` is large.
- `ruzsaSzemerediNumberNat_isLittleO`: the Ruzsa–Szemerédi number of `n` is `o(n²)`.
- `mem_end_iff`: the ends of `G`, as defined in Mathlib, are exactly its directions.
- `ComponentCompl.mem_of_hom`: the image of a connected graph under a homomorphism avoiding `K` lies in a single component of `G - K`.

Mathlib theorems surveyed in this chapter:

- `exists_isMatching_of_forall_ncard_le` / `exists_isPerfectMatching_of_forall_ncard_le`: Hall's marriage theorem for locally finite bipartite graphs, giving a matching that covers one side, or a perfect matching.
- `Subgraph.IsPerfectMatching.symmDiff_isCycles`: the symmetric difference of two perfect matchings is a disjoint union of cycles.
- `tutte`: Tutte's theorem — a finite graph has a perfect matching iff it has no Tutte violator.
- `isTuranMaximal_iff_nonempty_iso_turanGraph`: Turán's theorem — for `0 < r` the `K[r + 1]`-free graphs with the most edges are exactly the Turán graphs.
- `CliqueFree.card_edgeFinset_le`: a `K[r + 1]`-free graph on `n` vertices has at most `turanNumber n r` edges; with `turanNumber_two` this is Mantel's theorem.
- `IsBipartite.four_mul_encard_edgeSet_le`: a bipartite graph on `n` vertices has at most `n ^ 2 / 4` edges, with $\mathbb{N}\infty$-valued cardinalities so that no finiteness assumption is needed.
- `colorable_of_cliqueFree_lt_minDegree`: the Andrásfai–Erdős–Sós theorem — a `K[r + 1]`-free graph of large minimum degree is `r`-colourable.
- `extremalNumber_top`: Turán's theorem for extremal numbers — the extremal number of a complete graph on `r + 1` vertices is the number of edges of `T[n, r]`.
- `tendsto_turanDensity` / `isEquivalent_extremalNumber`: the extremal number of `H` is asymptotically its Turán density times `n.choose 2`.
- `eventually_completeEquipartiteGraph_isContained_of_minDegree`: the Erdős–Stone theorem, in its minimum degree form.
- `szemeredi_regularity`: Szemerédi's regularity lemma.
- `triangle_removal`: the triangle removal lemma.
- `roth_3ap_theorem_nat`: Roth's theorem — a set of natural numbers of positive density contains a three-term arithmetic progression.
- `corners_theorem_nat`: the corners theorem — a dense set of points in the grid contains a corner `(x, y), (x + d, y), (x, y + d)` with `d ≠ 0`.
- `exists_seq_covby_of_forall_covby_finite`: Kőnig's lemma, in an order-theoretic form.
- `Connected.pathNat_isContained`: Kőnig's lemma — an infinite, connected, locally finite graph contains a ray, a copy of `pathNat`.
- `nonempty_ends_of_infinite`: an infinite, connected, locally finite graph has at least one end.

From the exercises:

- `IsRegularOfDegree.exists_isPerfectMatching`: a `k`-regular bipartite graph with `k > 0`, not necessarily finite, has a perfect matching.

- `IsRegularOfDegree.colorable_lineGraph`: Kőnig's line colouring theorem for regular graphs — the edges of a `k`-regular bipartite graph can be properly coloured with `k` colours.
- `sum_degree_le_erdosGallai`: the degrees of a set `S` of `k` vertices sum to at most `k * (k - 1) + ∑ v ∈ Sᶜ, min k (G.degree v)`.
- `Graphic.erdosGallaiLE`: a graphic multiset satisfies the Erdős–Gallai inequalities, so with `Graphic.even_sum` the conditions in `erdos_gallai` are necessary.
- `card_cliqueFinset_two`: a graph has as many 2-cliques as edges, so Zykov's theorem for `s = 2` is the edge bound of Turán's theorem.
- `nonempty_ends_of_pathNat_isContained`: a graph containing a ray (a copy of `pathNat`) has an end.

Bibliography

- **[Albertson 1998]** M. O. Albertson, *You can't paint yourself into a corner*[1], J. Combin. Theory Ser. B **73** (1998), no. 2, 189–194.
- **[Andrásfai–Erdős–Sós 1974]** B. Andrásfai, P. Erdős and V. T. Sós, *On the connection between chromatic number, maximal clique and minimal degree of a graph*[2], Discrete Math. **8** (1974), no. 3, 205–218.
- **[Behrend 1946]** F. A. Behrend, *On sets of integers which contain no three terms in arithmetical progression*[3], Proc. Nat. Acad. Sci. U.S.A. **32** (1946), no. 12, 331–332.
- **[Berge 1957]** C. Berge, *Two theorems in graph theory*[4], Proc. Nat. Acad. Sci. U.S.A. **43** (1957), no. 9, 842–844.
- **[Bondy–Chvátal 1976]** J. A. Bondy and V. Chvátal, *A method in graph theory*[5], Discrete Math. **15** (1976), no. 2, 111–135.
- **[Brooks 1941]** R. L. Brooks, *On colouring the nodes of a network*[6], Proc. Cambridge Philos. Soc. **37** (1941), no. 2, 194–197.
- **[Cayley 1878]** A. Cayley, *Desiderata and suggestions: No. 2. The theory of groups: graphical representation*[7], Amer. J. Math. **1** (1878), no. 2, 174–176.
- **[Cayley 1889]** A. Cayley, *A theorem on trees*[8], Quart. J. Pure Appl. Math. **23** (1889), 376–378. Reprinted in The Collected Mathematical Papers XIII, 26–28.
- **[de Bruijn–Erdős 1951]** N. G. de Bruijn and P. Erdős, *A colour problem for infinite graphs and a problem in the theory of relations*[9], Indag. Math. **13** (1951), no. 5, 371–373.
- **[Diestel–Kühn 2003]** R. Diestel and D. Kühn, *Graph-theoretical versus topological ends of graphs*[10], J. Combin. Theory Ser. B **87** (2003), no. 1, 197–206.

[1] https://doi.org/10.1006/jctb.1998.1827
[2] https://doi.org/10.1016/0012-365X%2874%2990133-2
[3] https://doi.org/10.1073/pnas.32.12.331
[4] https://doi.org/10.1073/pnas.43.9.842
[5] https://doi.org/10.1016/0012-365X%2876%2990078-9
[6] https://doi.org/10.1017/S030500410002168X
[7] https://doi.org/10.2307/2369306
[8] https://doi.org/10.1017/CBO9780511703799.010
[9] https://doi.org/10.1016/S1385-7258%2851%2950053-7
[10] https://doi.org/10.1016/S0095-8956%2802%2900034-5

- **[Dirac 1952]** G. A. Dirac, *Some theorems on abstract graphs*[11], Proc. London Math. Soc. (3) **2** (1952), no. 1, 69–81.
- **[Erdős 1959]** P. Erdős, *Graph theory and probability*[12], Canad. J. Math. **11** (1959), 34–38.
- **[Erdős 1970]** P. Erdős, *Turán Pál gráf tételéről*[13] [On the graph theorem of Turán], Mat. Lapok **21** (1970), 249–251.
- **[Erdős–Gallai 1960]** P. Erdős and T. Gallai, *Gráfok előírt fokú pontokkal*[14] [Graphs with prescribed degrees of vertices], Mat. Lapok **11** (1960), 264–274.
- **[Erdős–Ko–Rado 1961]** P. Erdős, C. Ko and R. Rado, *Intersection theorems for systems of finite sets*[15], Quart. J. Math. Oxford Ser. (2) **12** (1961), 313–320.
- **[Erdős–Simonovits 1966]** P. Erdős and M. Simonovits, *A limit theorem in graph theory*[16], Studia Sci. Math. Hungar. **1** (1966), 51–57.
- **[Erdős–Stone 1946]** P. Erdős and A. H. Stone, *On the structure of linear graphs*[17], Bull. Amer. Math. Soc. **52** (1946), no. 12, 1087–1091.
- **[Euler 1741]** L. Euler, *Solutio problematis ad geometriam situs pertinentis*[18] [The solution of a problem relating to the geometry of position], Comment. Acad. Sci. Petropol. **8** (1741), 128–140. Eneström E53.
- **[Euler 1758]** L. Euler, *Elementa doctrinae solidorum*[19] [Elements of the doctrine of solids], Novi Comment. Acad. Sci. Petropol. **4** (1758), 109–140. Eneström E230.
- **[Füredi 2015]** Z. Füredi, *A proof of the stability of extremal graphs, Simonovits' stability from Szemerédi's regularity*[20], J. Combin. Theory Ser. B **115** (2015), 66–71.
- **[Gray 1953]** F. Gray, *Pulse code communication*[21], U.S. Patent 2,632,058, 1953. Filed 13 November 1947.
- **[Grötzsch 1959]** H. Grötzsch, *Zur Theorie der diskreten Gebilde, VII: Ein Dreifarbensatz für dreikreisfreie Netze auf der Kugel*[22] [On the theory of discrete structures, VII: A three-colour theorem for triangle-free networks on the sphere], Wiss. Z. Martin-Luther-Univ. Halle-Wittenberg Math.-Natur. Reihe **8** (1959), 109–120.
- **[Hakimi 1962]** S. L. Hakimi, *On realizability of a set of integers as degrees of the vertices of a linear graph. I*[23], J. Soc. Indust. Appl. Math. **10** (1962), no. 3, 496–506.

[11] https://doi.org/10.1112/plms/s3-2.1.69
[12] https://doi.org/10.4153/CJM-1959-003-9
[13] https://www.renyi.hu/~p_erdos/1971-27.pdf
[14] https://www.renyi.hu/~p_erdos/1961-05.pdf
[15] https://doi.org/10.1093/qmath/12.1.313
[16] https://mathscinet.ams.org/mathscinet/relay-station?mr=0205876
[17] https://doi.org/10.1090/S0002-9904-1946-08715-7
[18] https://scholarlycommons.pacific.edu/euler-works/53/
[19] https://scholarlycommons.pacific.edu/euler-works/230/
[20] https://doi.org/10.1016/j.jctb.2015.05.001
[21] https://patents.google.com/patent/US2632058A/en
[22] https://mathscinet.ams.org/mathscinet/relay-station?mr=0116320
[23] https://doi.org/10.1137/0110037

- **[Hall 1935]** P. Hall, *On representatives of subsets*[24], J. London Math. Soc. **10** (1935), no. 1, 26–30.
- **[Hamming 1950]** R. W. Hamming, *Error detecting and error correcting codes*[25], Bell System Tech. J. **29** (1950), no. 2, 147–160.
- **[Havel 1955]** V. Havel, *Poznámka o existenci konečných grafů*[26] [A remark on the existence of finite graphs], Časopis Pěst. Mat. **80** (1955), no. 4, 477–480.
- **[Hierholzer 1873]** C. Hierholzer, *Ueber die Möglichkeit, einen Linienzug ohne Wiederholung und ohne Unterbrechung zu umfahren*[27] [On the possibility of traversing a line system without repetition and without interruption], Math. Ann. **6** (1873), no. 1, 30–32. Published after his death by C. Wiener.
- **[Hoffman–Singleton 1960]** A. J. Hoffman and R. R. Singleton, *On Moore graphs with diameters 2 and 3*[28], IBM J. Res. Develop. **4** (1960), no. 5, 497–504.
- **[Kirchhoff 1847]** G. R. Kirchhoff, *Ueber die Auflösung der Gleichungen, auf welche man bei der Untersuchung der linearen Vertheilung galvanischer Ströme geführt wird*[29] [On the solution of the equations obtained from the investigation of the linear distribution of galvanic currents], Ann. Phys. Chem. **72** (1847), no. 12, 497–508.
- **[Kneser 1955]** M. Kneser, *Aufgabe 360*[30] [Problem 360], Jahresber. Deutsch. Math.-Verein. **58** (1955), no. 2, 27. See also Martin Kneser, Collected Works, Springer, 2021, 729–742.
- **[Kőnig 1916]** D. Kőnig, *Über Graphen und ihre Anwendung auf Determinantentheorie und Mengenlehre*[31], Math. Ann. **77** (1916), no. 4, 453–465.
- **[Kőnig 1927]** D. Kőnig, *Über eine Schlußweise aus dem Endlichen ins Unendliche*[32] [On a mode of inference from the finite to the infinite], Acta Litt. Sci. Szeged **3** (1927), 121–130.
- **[Kőnig 1931]** D. Kőnig, *Graphok és matrixok*[33] [Graphs and matrices], Mat. Fiz. Lapok **38** (1931), 116–119.
- **[Kővári–Sós–Turán 1954]** T. Kővári, V. T. Sós and P. Turán, *On a problem of K. Zarankiewicz*[34], Colloq. Math. **3** (1954), no. 1, 50–57.
- **[Lovász 1975]** L. Lovász, *Three short proofs in graph theory*[35], J. Combin. Theory Ser. B **19** (1975), no. 3, 269–271.
- **[Mantel 1907]** W. Mantel, *Vraagstuk XXVIII*[36] [Problem 28], Wiskundige Opgaven **10** (1907), 60–61. Solution by H. Gouwentak, W. Mantel, J. Teixeira de Mattes, F. Schuh and W. A. Wythoff.

[24] https://doi.org/10.1112/jlms/s1-10.37.26
[25] https://doi.org/10.1002/j.1538-7305.1950.tb00463.x
[26] https://doi.org/10.21136/CPM.1955.108220
[27] https://doi.org/10.1007/BF01442866
[28] https://doi.org/10.1147/rd.45.0497
[29] https://doi.org/10.1002/andp.18471481202
[30] https://doi.org/10.1007/978-3-030-81625-4_6
[31] https://doi.org/10.1007/BF01456961
[32] https://acta.bibl.u-szeged.hu/13338/
[33] https://real-j.mtak.hu/7307/
[34] https://doi.org/10.4064/cm-3-1-50-57
[35] https://doi.org/10.1016/0095-8956%2875%2990089-1
[36] https://zbmath.org/2643703

- **[Menger 1927]** K. Menger, *Zur allgemeinen Kurventheorie*[37] [On the general theory of curves], Fund. Math. **10** (1927), 96–115.
- **[Mycielski 1955]** J. Mycielski, *Sur le coloriage des graphs*[38] [On the colouring of graphs], Colloq. Math. **3** (1955), no. 2, 161–162.
- **[Ore 1960]** Ø. Ore, *Note on Hamilton circuits*[39], Amer. Math. Monthly **67** (1960), no. 1, 55.
- **[Paley 1933]** R. E. A. C. Paley, *On orthogonal matrices*[40], J. Math. Phys. **12** (1933), 311–320.
- **[Petersen 1898]** J. Petersen, *Sur le théorème de Tait* [On Tait's theorem], L'Intermédiaire des Mathématiciens **5** (1898), 225–227.
- **[Pósa 1976]** L. Pósa, *Hamiltonian circuits in random graphs*[41], Discrete Math. **14** (1976), no. 4, 359–364.
- **[Ramsey 1930]** F. P. Ramsey, *On a problem of formal logic*[42], Proc. London Math. Soc. (2) **30** (1930), 264–286.
- **[Roth 1953]** K. F. Roth, *On certain sets of integers*[43], J. London Math. Soc. **28** (1953), no. 1, 104–109.
- **[Ruzsa–Szemerédi 1978]** I. Z. Ruzsa and E. Szemerédi, *Triple systems with no six points carrying three triangles*[44], in Combinatorics (Proc. Fifth Hungarian Colloq., Keszthely, 1976), Vol. II (A. Hajnal and V. T. Sós, eds.), Colloq. Math. Soc. János Bolyai 18, North-Holland, Amsterdam, 1978, pp. 939–945.
- **[Szemerédi 1978]** E. Szemerédi, *Regular partitions of graphs*[45], in Problèmes combinatoires et théorie des graphes (Orsay, 1976), Colloq. Internat. CNRS 260, CNRS, Paris, 1978, pp. 399–401.
- **[Turán 1941]** P. Turán, *Egy gráfelméleti szélsőértékfeladatról*[46] [On an extremal problem in graph theory], Mat. Fiz. Lapok **48** (1941), 436–452.
- **[Tutte 1947]** W. T. Tutte, *The factorization of linear graphs*[47], J. London Math. Soc. **22** (1947), no. 2, 107–111.
- **[Vizing 1964]** V. G. Vizing, *On an estimate of the chromatic class of a p-graph*[48] (in Russian), Diskret. Analiz (1964), no. 3, 25–30.
- **[Whitney 1932]** H. Whitney, *Congruent graphs and the connectivity of graphs*[49], Amer. J. Math. **54** (1932), no. 1, 150–168.

[37] https://doi.org/10.4064/fm-10-1-96-115
[38] https://doi.org/10.4064/cm-3-2-161-162
[39] https://doi.org/10.2307/2308928
[40] https://doi.org/10.1002/sapm1933121311
[41] https://doi.org/10.1016/0012-365X%2876%2990068-6
[42] https://doi.org/10.1112/plms/s2-30.1.264
[43] https://doi.org/10.1112/jlms/s1-28.1.104
[44] https://mathscinet.ams.org/mathscinet/relay-station?mr=0519318
[45] https://mathscinet.ams.org/mathscinet/relay-station?mr=0540024
[46] https://mathscinet.ams.org/mathscinet/relay-station?mr=0018405
[47] https://doi.org/10.1112/jlms/s1-22.2.107
[48] https://mathscinet.ams.org/mathscinet/relay-station?mr=0180505
[49] https://doi.org/10.2307/2371086

- **[Zając 2018]** M. Zając, *A short proof of Brooks' theorem*[50], 2018, arXiv:1805.11176.
- **[Zarankiewicz 1951]** K. Zarankiewicz, *Problem P 101*, Colloq. Math. **2** (1951), 301.
- **[Zykov 1949]** A. A. Zykov, *On some properties of linear complexes*[51] (in Russian), Mat. Sb. (N.S.) **24(66)** (1949), no. 2, 163–188.

[50] https://arxiv.org/abs/1805.11176
[51] https://eudml.org/doc/65433